Administration SDKs, APIs, and CLI

Use the VCF SDK and VCF APIs, and VCF PowerCLI as a developer to automate infrastructure deployment and management programmatically for improving productivity and minimize manual errors.

In This Chapter

- VMware Cloud Foundation APIs and SDKs
- VMware Cloud Foundation SDKs Developer's Setup Guide
- VMware Cloud Foundation Programming Guide
- VMware Cloud Foundation PowerCLI
- Understanding the VMware Cloud Foundation Operations API
- APIs for VM Apps Organization Management
- VCF Automation and APIs

Getting to Know VCF

To apply this documentation, you must be acquainted with the VCF Product Overview, VCF Design, and VCF Release Notes.

VMware Cloud Foundation APIs and SDKs

VMware Cloud Foundation (VCF) is a comprehensive solution for building software-defined data centers with compute, storage, networking, devops, and hybrid cloud management. VMware offers various APIs to help manage your virtualization infrastructure, VCF, or vSphere Foundation.

VMware developed vCenter in the early 2000s to help customers manage large data centers with many ESX hosts. After several products that are intended to simplify multi-site management, VMware Cloud Foundation appeared about 20 years later, with SDDC Manager to organize multiple vCenters. Built on top of vCenter, SDDC Manager offers an intuitive interface to convert, import, and synchronize existing vSphere data centers into an easy-to-manage software-defined data center (SDDC). In addition, a VCF installer tool is available to help unify and build multi-function vSphere data centers.

VMware Cloud Foundation (VCF) provides a unified platform for traditional enterprise and modern applications. Based on a comprehensive software stack, VCF provides a set of services for compute, storage, network, container, and cloud management. VCF is a proven infrastructure package that offers efficient operations for private and hybrid clouds.

VMware vSphere Foundation (vSphere Foundation) is an alternative for customers with smaller data centers. It provides vCenter to manage ESX hosts, but no cloud management features. vSphere Foundation can create an aggregated infrastructure including compute, storage, and networking, and can manage the infrastructure as a unified operating environment.

VMware offers various APIs and SDKs to help manage your virtualization infrastructure, some for VCF only, others for vSphere Foundation as well. APIs help manage components of a data center, such as vCenter, ESX hosts, virtual machines, aggregated storage, NSX networking, Kubernetes containers, fleet management operations, and VCF Automation. This publication introduces APIs in VCF release 9.0 and later.

Revision History, Audience

Parts of this document were previously published as *An Introduction: Getting Started with vSphere APIs and SDK*. Most of the material was extended for VCF 9.0, and again for VCF 9.1 to include vSphere Kubernetes Service (VKS).

Audience

For vSphere automation and management, programmers employ Java or Python. OpenAPI definitions may be used to create bindings for other languages. For VMware Kubernetes Service, developers use mostly REST calls and `kubectl` commands.

Getting Started with the VMware Cloud Foundation APIs and SDKs

VMware Cloud Foundation offers a range of APIs and SDKs designed to help manage and automate various aspects your cloud infrastructure.

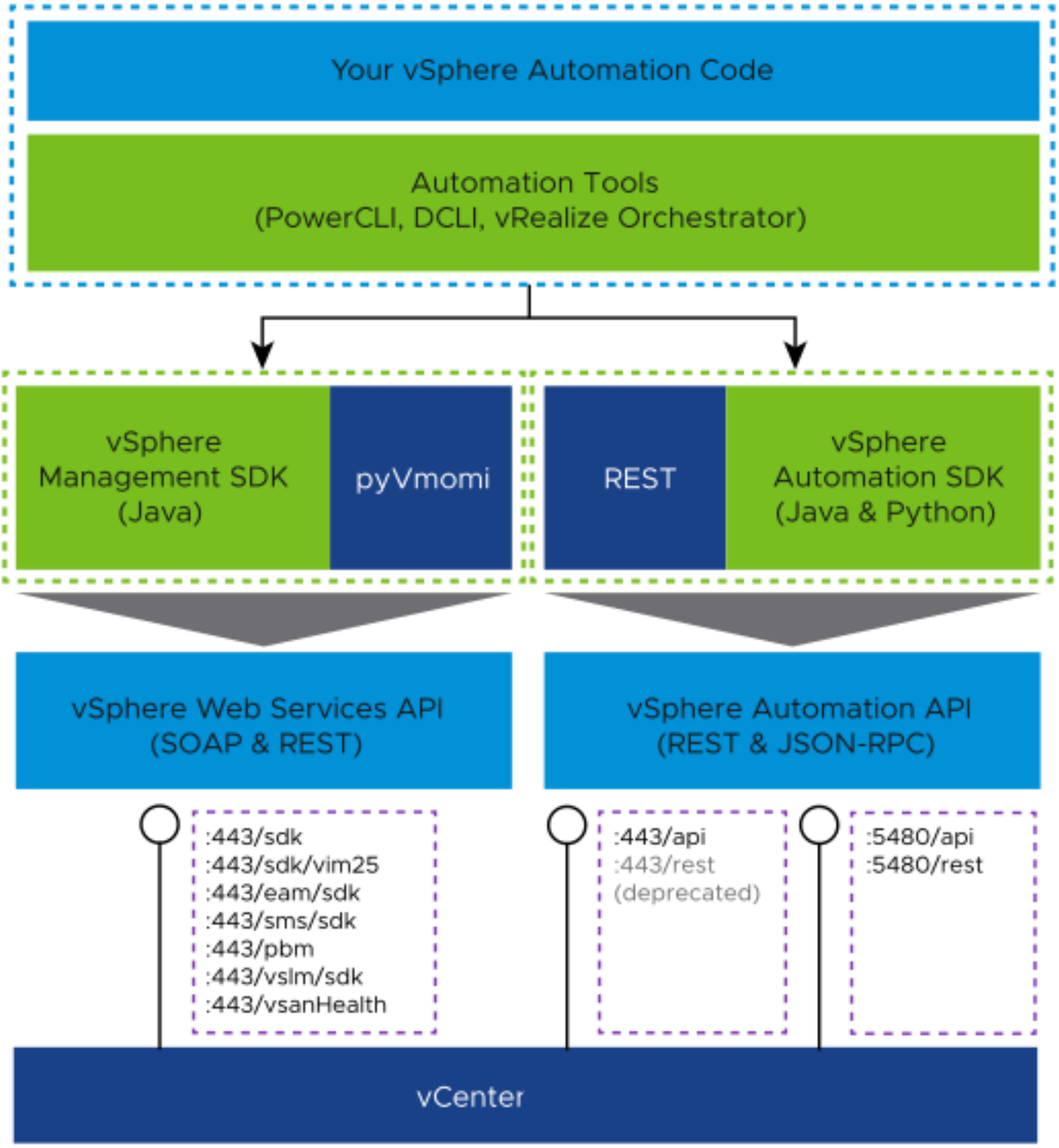
The following table compares VMware Cloud Foundation (VCF) APIs based on their functional capabilities and supported programming languages. Each row shows the protocol and language support available for an API—this can include full SDKs, add-on libraries, or, sometimes, partial library support. JSON (JavaScript object notation) is more succinct than XML used for SOAP.

Table 1848: Characteristics of VCF and vSphere APIs

API	Function of API or SDK	Transport	Java	Python
VMware Cloud Foundation API	manage and automate cloud infrastructure with vSphere, vSAN, NSX, and VCF	JSON, REST	SDK	SDK
Cloud Consumption Interface (CCI)	Kubernetes interface for Supervisor Service and VCF Automation	YAML	no	Kubernetes
vSphere Automation API	data center management features	JSON, REST	SDK or REST	SDK or REST
VIM API	multifaceted data center infrastructure	SOAP	SDK	SDK (pyvmomi)
VI/JSON	same as above	JSON	SDK	pyvmomi
vCenter Management API	vCenter administration	JSON	SDK	SDK
Storage Policy Based Management (SPBM)	storage affinity profiles	SOAP	SDK	pyvmomi
Storage Monitoring Service (SMS)	manage storage arrays	SOAP	SDK	pyvmomi
Virtual Storage Lifecycle Management (VSLM)	manage standalone virtual disks	SOAP	SDK	pyvmomi
ESX Agent Manager (EAM)	vSphere extensions and services	SOAP	SDK	pyvmomi
vSAN API	pooling ESX attached storage	SOAP	snapshot service	snapshot service
VCF Automation API (formerly Aria)	automate clusters to enforce cost, security, and performance best practices	REST		
VCF Operations API	interface to establish, monitor, manage, and troubleshoot virtualized datacenters	REST		
VCF Operations for networks, VCF Operations for logs	discover and analyze networks and logs to improve application performance	REST		

The following diagram shows key service endpoints used by different vSphere APIs.

Figure 543: vCenter endpoints



OAuth Token Support for API and CLI Access

As of VCF 9.1, VMware provides unified API and CLI access across most VCF components with OAuth standards-based token authentication, based on VCF Identity Broker (VIDB). Sample code for vSphere often demonstrates user-password authentication, however token authentication is more suitable for customers developing automation applications with an emphasis on security.

Administrators start in the VCF Operations console with Fleet Management Identity & Access, where they create API clients and generate API refresh tokens. Then a bearer access token, which is derived from the API refresh token and transmitted as JSON, can authenticate across all components in a VCF single sign-on (SSO) domain. The previous user-password authentication method remains available locally.

For VCF Operations, Orchestrator, HCX, and NSX, the flow is as follows. Other VCF components take slightly different approaches.

```
1. POST https://{vidb.host}/acs/t/{role}/token
   Body: grant_type=...&api_token={vidb_token}
   Response: {"access_token": "{bearer_token}"}
2. In API calls, include
   Header: Authorization: Bearer {bearer_token}
```

Some advantages of OAuth:

- Enhanced security by using tokens with limited lifespan instead of passwords.
- Password-less authentication helps customers integrate their tools with VCF.
- Automation scripts can obtain access without being tied to a user.
- Access is delegated and role-limited to prevent accidental or malicious use.
- Token distribution is scalable and efficient.
- Multiple sign-ons are possible with unlimited authentication across datacenters.

OAuth establishes a durable API refresh token which usually lasts for months, though it can be revoked. API clients then generate a bearer access token, which usually lasts for only minutes. An API client is associated with VCF role(s), but not with a user, so password sign-in is avoided. Roles can be limited in scope to promote security. VCF components include vCenter, NSX, VCF Operations, (for logs, for networks, Orchestrator), VCF Operations HCX, and VCF Automation.

VCF Identity Broker (VIDB) can be federated with external identity providers such as Okta and Entra ID. These identity providers can issue an OAuth bearer access token for federated SSO users. External identity providers exchange their own ID provider tokens for a VIDB issued bearer access token, and in specific cases, can generate durable API refresh tokens through a component user interface. For details, see topic "Deployment Modes of the Identity Broker" under VCF *Fleet Management*.

Eventually in VCF, every API and CLI should accept OAuth standard VIDB bearer access tokens to authenticate against VCF components. API clients can be created and audited with specific roles for API and CLI access. As stated before, traditional login-password methods continue to work, but security conscious customers need not deploy them.

Prerequisites:

- VCF SSO must be enabled in the Fleet for one or more VIDB instances.
- VCF role assignment to VIDB principals: federated users/groups, and VIDB owned API clients.

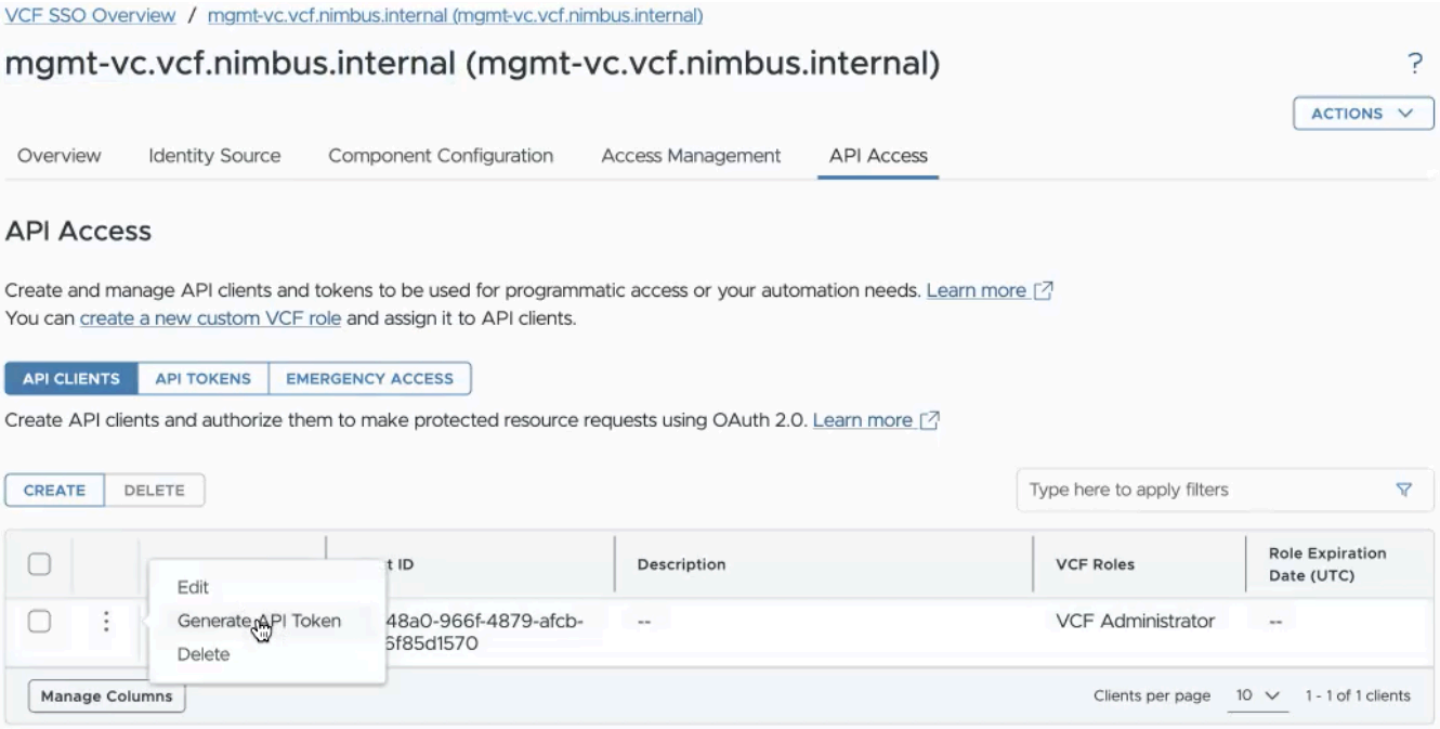
Implementation of OAuth in VCF

For programmatic access, VCF administrators create and manage API clients, separate from regular users and groups. Clients acquire an API refresh token and exchange it for a bearer access token, which authorizes actions of client applications and scripts.

In the VCF Operations user interface under Fleet Management, visit Identity & Access > Manage > VCF SSO Overview > API Clients. After an API Client is created, click # (three vertical dots) and select **Generate API Token**. This produces an API refresh token, which you should retain (copy and paste it into a file) so that the API client can continue exchanging it for a bearer access token.

For steps to create an API client and generate API refresh tokens using VCF Operations, see "Managing API Clients and Tokens" in VCF documentation under Fleet Management > Managing Identity. A screenshot of the generate step follows.

Figure 544: API access panel in VCF Operations



Note:

The API refresh token is virtually impossible to memorize. Once issued, you cannot retrieve it, though it can be revoked. It's best to store it in a file.

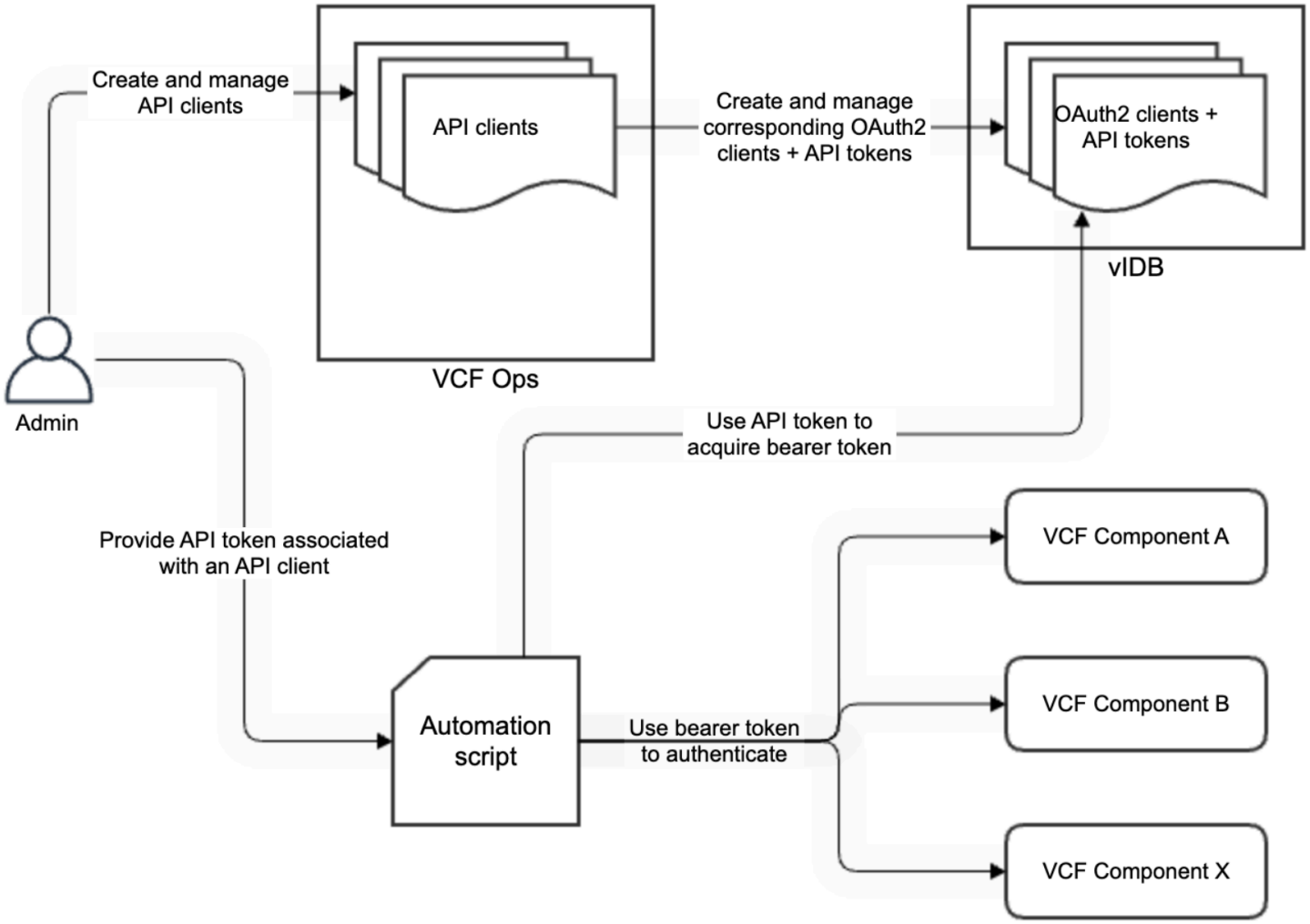
VCF 9.1 offers token types `tenant`, `vidb_refresh`, `vidb_access`, plus `external_idp` for vSphere, and `provider` for VCF Automation.

Token Exchange Architecture

After creating an API client, the administrator hands off the API refresh token to a client. The API client exchanges it for a short-duration bearer access token, which then authorizes the actions of client applications and scripts.

Not all VCF components employ identical token authentication methods, but many of them start with VCF Operations and VCF Identity Broker (VIDB). The following figure shows the architecture of OAuth implementation in VCF.

Figure 545: Steps for clients to acquire bearer access token from API refresh token



Steps to acquire a bearer access token for authentication:

1. Administrator creates API clients with credentials that are recorded in VIDB.
2. Administrator requests a long-lived API refresh token from the VCF Operations UI.
3. Automation script passes the API refresh token to VIDB and gets a bearer access token in return.

4. Automation script uses the bearer access token to authenticate with VCF components.

Authentication Flow for VCF Components

VCF components exchange VIDB's API refresh tokens for bearer access tokens and authenticate differently, as shown in the following table. Many VCF components dispense API refresh tokens from the client name dropdown menu. The exchange process ranges from two to four steps. The access token is then passed with the `Bearer` keyword for authorization.

Component	How to Obtain	Token Exchange Steps	Token Location	Authorization Header
VCF Operations	Name dropdown	VIDB → Bearer	Response body: `access_token`	`Authorization: Bearer {token}`
NSX	Name dropdown	VIDB → Bearer	Response body: `access_token`	`Authorization: Bearer {token}`
VCF Operations Orchestrator	Name dropdown	VIDB → Bearer	Response body: `access_token`	`Authorization: Bearer {token}`
VCF Operations HCX	Name dropdown	VIDB → Bearer	Response body: `access_token`	`Authorization: Bearer {token}`
vCenter	Name dropdown	VIDB → Bearer → SAML → gZip	Response body: `access_token`(multi-step response)	`vmware-api-session-id: {session_id}`
VCF Automation	**MyAccount** page	VIDB → Bearer → Session	Response header: `X-VMWARE-VCLOUD-ACCESS-TOKEN`	`Authorization: Bearer {token}`
VCF Operations for networks	Like VCF Operations	VIDB → Bearer → JWT (through VCF Operations)	Response body: `jwtToken`	`Authorization: Bearer {token}`
VCF Operations for logs	Like VCF Operations	VIDB → Bearer → JWT (through VCF Operations)	Response body: `jwtToken`	`Authorization: X-JWT-token {token}`

All components use JSON parsing except VCF Automation, which uses HTTP header parsing, and places the token in the header instead of the body.

All components follow OAuth 2.0 standards, except vCenter only partially with custom header and compression using `gzip`.

Tokens for API Clients and SSO Users

In addition to API clients, programmatic access can be arranged for SSO users, as needed for on-demand automation tasks. For SSO users, tokens fail after a personal account is discontinued. Whereas API clients persist until removed.

The least privilege principle recommends creating separate clients for distinct functions, such as one for Backup and another for Monitoring. Token permissions are inherited from the access levels assigned to the associated principal (API client or SSO user) at the VCF level. Administrator privilege for vCenter does not grant administrator privilege at the fleet level.

Various built-in roles are available for vCenter, VCF Operations, NSX, and VCF Automation. You can also create custom roles such as vcf-fleet-admin, which enable fleet management operations for API tokens assigned to that role. Built-in and custom roles may be assigned to an API client. Tokens subsequently generated for an API client inherit the privileges of those roles. Built-in roles include:

- Identity Management
  - identity.management.manage
  - identity. management.view
- Certificate Management
  - vcf_certificates.manage
  - vcf_certificates.view
- Password Management
  - vcf_password.manage

  - vcf_password.view
- Fleet Settings (password, NTP, DNS)
  - administration.fleetSettings.manage
  - administration.fleetSettings.view
- Config Management
  - configuration_drifts.manage
  - configuration_drifts.view
- Tag Management
  - tag_management.assignments
  - tag_management.manage
  - tag_management.view
- Task Tracking
  - ops.administration.management_tasks.manage
  - ops.administration.management_tasks.view
  - administration.api.read_access

After defining a client, you can generate one or more API refresh tokens for it. When the client makes a call, its API refresh token goes to VMware Identity Broker (VIDB), which authenticates the client and issues a bearer access token for making API calls. Here is a token exchange request to the VIDB access control service (ACS):

```
curl --request POST \
  --url https://{vidb-fqdn}/acs/t/{client}/token \
  --header 'content-type: application/x-www-form-urlencoded' \
  --data grant_type=urn:custom:vcf:params:oauth:grant-type:api-token \
  --data api_token=vidb_xxxxxxxxxxxxxxxxxx
```

Obtaining an API Token as an SSO User

Procedure: SSO users log into a VCF Components UI (such as VCF Operations, vCenter, NSX, or VCF Automation) to generate an API token. The initial API refresh token is exchanged via Identity Broker for a short-lived bearer access token used for fleet-wide operations. API refresh tokens inherit permissions assigned only at the VCF Fleet level. For example, if a "vCenter Admin" is only a "VCF Viewer" at the fleet level, the exchanged token grants only Viewer permissions.

VCF SSO API tokens can be generated by an SSO user or under an API client. These tokens have a lifespan ranging from 30 through 180 days.

SSO User Tokens: Tied directly to personal human accounts and restricted to manually triggered, on-demand activities. Because these tokens are not used for automated system integrations, their expiration has no impact on background infrastructure operations. SSO users can simply log into the VCF user interface and manually generate a new one.

API Client Tokens: Highly recommended for use in system-to-system integrations (such as automation pipelines, backups, or component monitoring). To maintain continuous, uninterrupted system integrations and prevent production connection failures, VCF SSO provides a dedicated token regeneration API to programmatically cycle these secrets.

By determining the SSO realm ID, creating an API client, assigning to it a role, generating an initial API refresh tokan for the client, determining its expiration date, and regenerating the token when near expiration, organizations can achieve password-less continuous system integrations across all VCF components.

Different Patterns for Token Exchange

This topic shows examples of token exchanges with various VCF components.

Most components authenticate directly with VIDB, using the standard `Authorization` header, and return ready-to-use tokens in the response body. However vCenter uses a custom header name, session ID value, and requires client-side token compression. VCF Automation returns tokens in the response header, and uses HTTP header instead of JSON parsing.

VCF Operations for networks and VCF Operations for logs cannot exchange tokens with VIDB. They use VCF Operations as a proxy.

It is the responsibility of the VCF administrator to create API clients, assign permissions and roles to clients, and generate the initial API refresh token for each API client. For procedures to create an API client and generate API refresh tokens using VCF Operations, see section "Managing API Clients and Tokens" under Fleet Management in VCF documentation.

To generate the API refresh token, VCF Operations has a drop-down menu under the API client name. Other user interfaces may have a Generate button to click. VCF Automation has a link to generate the API refresh token on the user's MyAccount page.

For specific instructions, go to one of these topics:

- Token Exchange for VCF Operations
- Token Exchange for vCenter
- Token Exchange for VCF Automation
- Token Exchange for NSX
- Token Exchange VCF Orchestrator for networks
- Token Exchange VCF Orchestrator for logs

Token Exchange for VCF Operations

Many VCF components perform token exchange using VCF Operations communicating with VIDB, or follow a similar workflow.

VCF Operations, VCF Operations Orchestrator, VCF Operations HCX, and NSX networking use a flow that exchanges an API refresh token directly for a bearer access token:

```
1. POST /acs/t/CUSTOMER/token
   Body: grant_type=...&api_token={vidb_token}
   Response: {"access_token": "{bearer_token}"}
2. in API Calls
   Header: Authorization: Bearer {bearer_token}
```

Here is an example showing the API to exchange the VIDB Token for an API access token.

```
curl --request POST \
  --url https://{vidb-fqdn}/acs/t/{client-name}/token \
  --header 'content-type: application/x-www-form-urlencoded' \
  --data grant_type=urn:custom:vcf:params:oauth:grant-type:api-token \
  --data 'api_token={apiToken}'
```

For detailed procedures to create API clients and generate API refresh tokens, among other tasks, see Managing API Clients and Tokens in the Fleet Management section of the *VMware Cloud Foundation 9.1* guide for VCF Operations.

Token Exchange for vCenter

With vCenter, the client must perform base64 decode, `gzip` compression, and base64 encode.

```
# Input: SAML token (base64 string)
saml_token="PD94bWwgdmVyc2lvbj0iMS4wIiBlbmNvZGluZz0iVVRGLTgiPz4..."
# Required processing:
compressed_token=$(echo "$saml_token" | base64 -d | gzip | base64 -w0)
# Output: Compressed token ready for session creation
```

Flow for vCenter with multi-step exchange and compression:

```
1. POST /acs/t/CUSTOMER/token → Bearer Access Token (response body)
2. POST /api/vcenter/authentication/token → SAML Token (response body)
3. Client processing: base64 decode → gzip compress → base64 encode
4. POST /api/session → Session ID (response body)
5. API Calls with header: vmware-api-session-id: {session_id}
```

Token Exchange for VCF Automation

With VCF Automation token exchange, the token must be extracted from the response header, not the response body. Request and response:

```
POST /cloudapi/1.0.0/sessions/provider
Authorization: Bearer {bearer_token};org=system
Accept: application/json;version=9.1.0

HTTP/1.1 200 OK
X-VMWARE-VCLOUD-ACCESS-TOKEN: eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
{
  "user": "administrator@system",
  "org": "System"
}
```

Flow for VCF Automation with multi-step exchange and token in response header:

```
1. POST /acs/t/CUSTOMER/token → Bearer Access Token (response body)
2. POST /cloudapi/1.0.0/sessions/provider → Session Token (response header)
   Header: Authorization: Bearer {bearer_token};org=system
   Response Header: X-VMWARE-VCLOUD-ACCESS-TOKEN: {session_token}
3. API Calls with header: Authorization: Bearer {session_token}
```

Token Exchange for NSX

If you are a federated SSO user and granted NSX access, you can generate an API refresh token and exchange it for a bearer access token to authenticate public APIs.

1. Log in to NSX Manager as a VCF user, providing a username and password.
2. On the NSX Home page, click **Generate API Tokens** from the drop-down menu under the profile name.
3. On the VCF Identity Broker page, click Generate to create an API refresh token.
4. After obtaining an API refresh token, exchange it with a bearer access token for API calls.

```
1. POST /acs/t/CUSTOMER/token
   Body: grant_type=...&api_token={vidb_token}
   Response: {"access_token": "{bearer_token}"}
2. in API Calls
   Header: Authorization: Bearer {bearer_token}
```

Token Exchange for HCX

If you are a federated SSO user, you can generate an API refresh token and exchange it for a bearer access token to authenticate public APIs in VCF Operations HCX.

1. Log in to VCF Operations as an administrator, providing a username and password.
2. On the API Access page, click **Generate API Tokens** from the drop-down menu under the profile name.
3. On the VCF Identity Broker page, click Generate to create an API refresh token.
4. After obtaining an API refresh token, exchange it with a bearer access token for API calls.

```
1. POST /acs/t/CUSTOMER/token
   Body: grant_type=...&api_token={vidb_token}
   Response: {"access_token": "{bearer_token}"}
2. in API Calls
   Header: Authorization: Bearer {bearer_token}
```

Token Exchange VCF Operations for networks

VCF Operations for networks and VCF Operations for logs follow the same steps as VCF Operations, except they also exchange a JWT token from VCF Operations.

1. Get a bearer access token by generating it from VIDB.

```
curl --request POST \
 --url https://acs/t/CUSTOMER/token \
 --header 'Content-Type: application/x-www-form-urlencoded' \
 --data grant_type=urn:custom:vcf:params:oauth:grant-type:api-token \
 --data api_token={API_REFRESH_TOKEN
```

   The bearer access token appears in the response body, along with `serviceKeys`.

```
Body: {"serviceKeys": ["ops-ni"]}
```

2. Exchange the bearer access token for a JWT token (for logs, use `ops-li`).

```
curl --request POST \
  '{vcfops_url}/suite-api/api/auth/token/exchange?includeServiceDetails=false' \
  -H 'accept: application/json' \
  -H 'Authorization: VIDBToken {ACCESS_TOKEN_FROM_STEP_1}' \
  -H 'Content-Type: application/json' \
  -d '{ "serviceKeys": [ "ops-ni" ] }'
```

3. Use that JWT token to call the public API (for logs, the bearer label changes).

```
curl --request GET \
  --url '{ops4netwk_url}/api/v2/fields' \
  --header 'Bearer: {JWT_FROM_STEP_2}'
```

Token Exchange VCF Operations for logs

Token exchange in VCF Operations for logs follows the same steps as VCF Operations for networks, except for minor differences summarized here.

1. Get a bearer access token by generating a token from VIDB, as above.
2. In step 2, substitute "li" (log insight) for "ni" (network insight)

```
-d '{ "serviceKeys": [ "ops-li" ] }'
```

3. Exchange the bearer access token for a JWT token, as above. But in step 3, advertise JWT instead of Bearer token.

```
curl --request GET \
  --url '{ops2logs_url}/api/v2/fields' \
  --header 'X-JWT-TOKEN: {JWT_FROM_STEP_2}'
```

VMware Cloud Foundation API Sample Programs

VMware Cloud Foundation offers an extensive set of REST APIs. Some control SDDC Manager, while others control VCF Installer.

As of VCF 9.0, the VCF Installer appliance replaces the Cloud Builder appliance in VCF 5.x, and new functionality in VCF Installer and VCF Operations replaces VCF Import Tool (the `vcf_brownfield.py`Python script).

The VCF SDK for Python on Github under vcf-sdk-python contains many code examples demonstrating use cases for SDDC Manager and VCF Import.

github.com/vmware/vcf-sdk-python

Likewise, the VCF SDK for Java on Github under vcf-sdk-java contains similar code examples demonstrating use cases for SDDC Manager and VCF Import.

github.com/vmware/vcf-sdk-java

Furthermore, the Broadcom developer site offers an excellent VMware Cloud Foundation API reference under APIs > Hyperconverged Infrastructure. Current link is here:

developer.broadcom.com/xapis/vmware-cloud-foundation-api/latest/

The VCF 9.0 API is similar to the VCF 5.2.1 API, with some new interfaces and some discontinued. However the API for VCF Installer is new, and documented here:

broadcom.net/apis/vcf-installer/latest/

vSphere Foundation and VCF Import

The VCF SDK for Python and for Java each contain a `vcf-installer-samples` directory providing sample scripts for interacting with the VCF Installer APIs, in addition to extensive vSphere, NSX, SDDC Manager, and VCF Operations samples.

In the VCF Python SDK on Github under vcf-sdk-python, `utils/client.py` defines VcfInstallerClient to handle authentication, create stubs for API invocations, and perform automatic token refresh. It requests a Session and then returns the VcfInstallerClient that was created.

Sample code `system/get_appliance_info.py` demonstrates how to obtain information about a virtual appliance by calling `get_appliance_info()` on the host address of the VCF Installer appliance.

A "bundle" contains software for various VCF components, including vCenter, VCF Operations, NSX, SDDC Manager, and others. An "online depot" is a repository of components and add-ons for running vSphere and VCF.

In the `bundle/` subfolder, three samples for vSphere Foundation demonstrate how to configure an online depot and download bundles necessary for deploying vSphere Foundation greenfield, vSphere Foundation with existing vCenter, and vSphere Foundation with existing vCenter and VCF Operations.

One sample for VCF demonstrates how to configure an online depot and download bundles necessary for deploying a new VCF Fleet with its first VCF instance, including VCF Operations, Fleet Management, VCF Operations Collector, VCF Automation, vCenter, NSX, and SDDC Manager. Two other samples demonstrate how to do this with vCenter and NSX already deployed, and how to extend a VCF Fleet with a VCF Instance. Multiple VCF instances are useful for geographic dispersal.

In the `deploy/` subfolder, two vSphere Foundation samples demonstrate how to deploy a new vSphere Foundation instance, and how to deploy a vSphere Foundation instance from existing components. Four VCF samples demonstrate (1) how to deploy a new vSphere Foundation instance, (2) how to deploy a new vSphere Foundation instance with existing vCenter, (3) how to deploy a new VCF Fleet with existing vCenter and NSX, and (4) how to extend an existing VCF fleet with new VCF Instance, reusing existing components.

All of the deploy sample programs accomplish their purpose by assembling an SDDC specification and then calling `deploy_sddc()` function as follows:

```
sddc_task = client.v1.Sddcs.deploy_sddc(sddc_spec)
```

SDDC Manager Interfaces

The VCF SDK for Python and for Java each contain a subfolder for `sddc-manager-samples`, in addition to extensive vSphere samples.

In the VCF Python SDK on Github under vcf-sdk-python, `utils/client.py` defines VcfInstallerClient to handle authentication, create stubs for API invocations, and perform automatic token refresh.

The `hosts_commission.py` script adds ESX hosts into the VCF data center. It reads a JSON file with details about the ESX hosts to add, including network pool information and FQDN. After passwords and network pool information are established, the hosts are added by calling `commission_hosts()`.

The `hosts_decommission.py` script goes the other way, reading a JSON file listing the FQDN of hosts to remove, then calling `decommission_hosts()` on the ESX hosts in the list. Both scripts poll the task status and exit upon completion.

The `get_cluster.py` script, after calling `create_sddc_manager_client()`, retrieves information about clusters that are already set up in the data center.

To expand an existing cluster, the `add_host_to_cluster.py` script reads a JSON file to obtain parameters to create a network pool, commission the host, ensure the cluster and domain are present in SDDC, check that at least one host is available in the SDDC inventory, and verify license keys for the ESX product and others if required.

The `create_and_delete_cluster.py` script demonstrates the complete procedure to add a cluster based on JSON specification, populate the cluster with available ESX hosts, validate the cluster for compliance, and finally remove the cluster by calling `delete_cluster()`.

The `cluster_compaction.py` script removes an ESX host from its cluster, after checking that a host with the given name exists inside the chosen cluster, and ensuring that enough hosts remain to facilitate continued vSAN availability, thereby avoiding data loss or reversion to read-only datastore.

The typical VCF installation has one management domain and at least one workload domain, which run virtual machines and/or Kubernetes containers. Given the name and credentials of an SDDC Manager, the `get_domains_info.py` script retrieves information about management domains and workload domains that it knows about.

The `create_and_delete_domain.py` script demonstrates the complete procedure to create a workload domain based on JSON specification, monitor creation of the domain, and upon successful creation, proceed with deleting the workload domain. First it creates the network pool, commissions the hosts (if at least three are available), then it checks license keys for the product types ESX, vSAN, and NSX. The JSON specification for workload domain is complex, and worth perusing.

Core vSphere APIs

The user interface enables administrators to manage vSphere by calling various vSphere APIs to perform backend operations. Two main sets of APIs provide core vSphere management capabilities: VIM and vAPI.

The vAPI (vSphere Automation API) uses REST principles with JSON-based protocols, providing a simpler interface for automation. The VIM API, on the other hand, follows RPC principles and uses a SOAP protocol, offering broader functionality and compatibility with previous vSphere versions.

More recently, VI/JSON enables VIM-like operations over a JSON transport.

The core functional areas can be viewed as configuration and management of hosts, virtual machines, and networks, as well as storage at an abstract level. The two core APIs that address these functional areas complement each other both in function and in design principles. As the code samples under Tagging Examples Using vSphere APIs demonstrate, you can use the two APIs together to achieve complete functionality.

vSphere Automation API

The VMware vSphere Automation SDK is optimized for developing applications that use major vSphere features such as virtual machine management, content libraries, resource deployment, tagging, and managing security certificates.

The vSphere Automation API follows a resource-based REST architecture with native JSON requests and responses. Clients can use the following technologies to send requests to vSphere Automation endpoints:

- Java SDK
- Python SDK
- REST
- PowerCLI™

Additional language bindings are available through open-source projects on GitHub.

The vSphere Automation API currently provides core features including:

- Federated authentication with support for AD FS and Okta.
- VMware vSphere Lifecycle Manager supports desired-state management of hosts in a cluster.
- The content library feature manages templating, storage, and deployment of virtual machines and vApps.
- Metadata tagging of vSphere objects facilitates filtering and sorting of objects when managing large data centers.
- Management of security certificates issued by the VMware Certificate Authority (VMCA) or third-party and custom-made certificates in your environment.

Most major new vSphere features are introduced through the Automation API.

The Automation API supports two protocols: JSON-RPC, used by the Automation SDKs, and a simplified JSON protocol for REST clients. The SDKs are available for download from Broadcom.

The Automation API service endpoint is `https://{domain}/api/{service}`

where `{domain}` is the server's IP address or fully qualified domain name, and `{service}` is the name of a specific interface.

For example, you can reach the Session service through the following URL:

```
https://<vCenter_IPaddr_or_FQDN>/api/session
```

For more information about using the vSphere Automation API, see the following publications:

- *vSphere Automation API REST Programming Guide*
- *vSphere Automation API Java Programming Guide*
- *vSphere Automation API Python Programming Guide*
- *VCF PowerCLI User's Guide*
- *vSphere Automation REST API Reference*

vSphere Web Services API

The vSphere Web Services API (VIM API) provides programmatic access to essential resources for building robust, fault-tolerant virtualized applications across compute, networking, and storage. It supports resource types such as virtual machines, ESX hosts, clusters, datastores, networks, and system abstractions including events, alarms, authorization, and plug-in extensions.

The vSphere Web Services API is a mature, comprehensive management interface that operates with both ESX and vCenter. It provides access to a hierarchy of managed objects used to configure, monitor, and manage data center resources. Formerly called VIM API, a name that started with `vim25` for version 2.5 and still exists in the SDK source tree.

Developers have several options for accessing the VIM API:

- vSphere Management SDK (Java): Provides Java bindings and samples that use SOAP.
- pyvmomi SDK (Python): An SDK that provides bindings and utilities for Python developers interacting with the VIM API.
- The open source govmomi SDK is a comprehensive set of bindings, utilities, examples, and a CLI for Go developers.
- WSDL Code Generation (Language-neutral): The vSphere Management SDK supplies language-neutral WSDL files for the SOAP protocol, enabling you to generate client bindings in your preferred language.
- OpenAPI Specification (Language-neutral): The vSphere Management SDK includes a language-neutral OpenAPI specification, allowing you to generate code for a REST-like JSON protocol in your chosen language.

The vSphere Web Services API includes the following core features:

- Data center inventory management, using a hierarchy of folders and resource objects
- A powerful server-side data retrieval feature, the Property Collector
- Host system configuration and management
- Datastore configuration and management
- Network configuration and management
- Virtual machine configuration and management, including guest OS operations
- Virtual machine encryption
- Cluster and resource pool configuration and management
- Management of events and alarms

The vSphere Web Services API is included in the vSphere Management SDK. Both support either a SOAP-XML RPC protocol or a JSON-based protocol. The Management SDK provides RPC stubs for clients with Java language bindings.

The vSphere Web Services API endpoint is

```
https://{domain}/sdk
```

The VI/JSON protocol service endpoint is

```
http://{domain}/sdk/vim25/8.0.1.0
```

For more information, see the following documentation.

- *vSphere Web Services SDK Developer Setup Guide*
- *vSphere Web Services SDK Programming Guide*

Python with pyvmomi

For Python programmers, `pyvmomi` offers an open-source Python binding for vSphere Management and Automation APIs.

The `pyvmomi` package provides a convenience layer on top of the core VIM API and the following related extension APIs that use the SOAP/XML protocol:

- ESX Agent Manager (EAM)
- Storage Based Policy Management (SPBM)
- Storage Monitoring Service (SMS)
- Virtual Storage Lifecycle Management (VSLM)

The `pyvmomi` package is notable for:

- Simplified approach to connection, authentication, and retrieval of the ServiceContent object
- Module to handle client-server synchronization and asynchronous task tracking
- Convenient retrieval of managed object properties
- Facility to interface with vCenter Single Sign-On (SSO)

For more information about using the `pyvmomi`, see documentation in the VCF SDK.

Supplementary vSphere APIs

Supplementary vSphere APIs enhance the core APIs with several extensions that provide specialized capabilities for managing specific areas of your cloud infrastructure.

The vSAN API

The vSAN management API is a Web service (like VIM) that runs on both vCenter and ESX systems. The API provides access to methods that you can use to configure and manage vSAN storage. VMware vSAN is a distributed software layer that aggregates local storage devices to create a shared storage pool across hosts in a cluster.

The vSAN API centers around a small set of managed objects, around 30. Methods associated with these managed objects allow developers to set up and configure all aspects of vSAN, query runtime state, and check storage health.

The vSAN API can be used for solution integration, allowing you to configure and monitor vSAN disk clusters and related services on ESX hosts and vCenter instances. It also manages virtual disk functions such as mounting, partitioning, secure erasing, and snapshots.

Functional features of the vSAN API include:

- Cluster configuration and query methods.
- Stretched cluster configuration and query methods.
- Performance, health, and diagnostics.
- Virtual disk formatting and space usage queries.
- Software upgrade handling.
- Cloud Native Storage (CNS): a Tanzu feature that helps Kubernetes provision storage on-demand, and provides visibility into container volumes through the vSphere Client UI.
- VMware Live Site (and Cyber) Recovery are optional data backup products offering self-service data protection and fast recovery of business-critical data by saving virtual machines to a remote cloud location.

The vSAN API uses a SOAP-XML RPC protocol, modeled after the vSphere Web Services (VIM) API, and is included as part of the vSphere Management SDK.

The vSAN API service endpoints for ESX and vCenter are:

```
https://{domain}/vsan
https://{domain}/vsanHealth
```

For more information, see the following documentation.

- *vSAN SDKs Programming Guide*: mostly Python code examples
- *vSAN Data Protection SDK for Python*
- *vSAN Data Protection SDK for Java*
- *REST API for vSAN Snapshot Service*

Virtual Storage Lifecycle Management API

The Virtual Storage Lifecycle Management (VSLM) API is a Web service integrated into vCenter, operating as part of the Storage Policy Services (SPS). This API allows you to connect to the VSLM endpoint to access information about First Class Disks (FCDs). In addition to VSLM, SPS also encompasses Storage Policy Based Management (SPBM) and the Storage Management Service (SMS).

First class disks, also called improved virtual disks (IVD), are disks not associated with a virtual machine. FCD simplifies management of stand-alone virtual disks, avoiding creation of a dummy VM with one or a few attached disks. Snapshots worked on a per-VM basis, and to snapshot a single virtual disk formerly required configuration overhead. VSLM allows snapshot of just an FCD.

FCD offers full lifecycle management of virtual storage objects, independent of any VM. VSLM APIs are able to create, delete, snapshot, backup, restore, and do other lifecycle tasks on VMDK objects without requiring VM attachment.

An FCD is identified by UUID and human readable name. The UUID is globally unique and the primary identifier. The UUID remains valid even if a snapshot is taken or its FCD relocated by XvMotion.

The VSLM API uses a SOAP-XML RPC protocol, modeled after the vSphere Web Services (VIM) API, and is included as part of the vSphere Management SDK.

The VSLM API endpoint is

```
https://{domain}/vslm
```

For more information, see the following documentation.

- *Virtual Storage Lifecycle Management API*

Storage Policy Based Management API

The Storage Policy Based Management (SPBM) API is an extension of the Web Services (VIM) API. Storage policies simplify the task of matching available storage to virtual machines.

A vSphere storage profile defines storage policy information that describes storage requirements for virtual machines and storage capabilities of storage providers. You use the VMware Storage Policy API to manage the association between virtual machines and datastores.

With appropriate policies, SPBM governs virtual machine placement into appropriate datastore, accounting for RAID levels, pre-zeroed allocated disk, and space efficient thin provisioned disk. Storage policies may also request specific data services for virtual disks, such as caching, replication, and deduplication.

The Storage Policy API supports the following capabilities:

- Access to vSphere storage profiles and to entities associated with profiles
- Compliance checking of profiles and entities
- Placement compatibility checking

The Storage Policy API uses a SOAP-XML RPC protocol, similar to the vSphere Web Services (VIM) API. It is bundled in the vSphere Management SDK. The Management SDK provides RPC stubs for clients with Java language bindings.

The Storage Policy service endpoint is

```
https://{domain}/pbm
```

For more information, see the following documentation:

- *VMware Storage Policy API*

Storage Monitoring Service API

The Storage Monitoring Service (SMS) API provides methods to retrieve information about available storage topology, capabilities, and state.

The vSphere API for Storage Awareness (VASA) permits storage arrays to integrate with vCenter for management functionality. VASA providers expose features of the physical storage devices, such as storage health status, configuration info, and storage capacity.

SMS establishes and maintains connections with VASA providers. SMS retrieves information about storage availability from the providers, and clients can use the SMS API to perform the following operations.

- Identify VASA providers.
- Retrieve information about storage arrays.
- Identify vSphere inventory entities (hosts and datastores) which are associated with external storage entities on the storage arrays.

The SMS API uses a SOAP-XML RPC protocol, modeled after the vSphere Web services (VIM) API, and is included as part of the vSphere Management SDK.

The SMS API endpoint is

```
https://{domain}/sms
```

For more information, see the following documentation.

- *vCenter Storage Monitoring Service API*

ESX Agent Manager

You can add functions to vSphere by developing software applications that you register as vCenter extensions. A vSphere Solution is an extension that registers with vCenter and implements some or all of the extension features of the vSphere API.

You can deploy a solution as an Open Virtualization Format (OVF) package, with optional VMware vSphere Installation Bundles (VIB). You can also install solutions by using an installer, such as Windows Installer (MSI) or RPM Package Manager. Most of the extension functions in this API are independent of the technology that you use to deploy a solution. If you deploy a solution by using OVF, you can use the vCenter Extension vService to simplify the registration of the solution with vCenter.

You can create two types of vSphere solutions:

- A vService is a service that a solution provides to specific applications that run inside virtual machines and vApps. A solution can provide several types of vServices. Virtual machines or vApps can have dependencies on several types of vServices.
- A vSphere ESX agent is a virtual machine and an optional vSphere Installation Bundle (VIB) that extends the functions of an ESX host to provide additional services that a vSphere solution requires.

The vService Manager and ESX Agent Manager are pre-built solutions that are part of the vCenter Extensions functionality within the vCenter.

vSphere ESX Agent Manager is an intermediary between vCenter and a solution with two key aspects:

- provisioning agent virtual machines and VIB modules for the solution onto a scope in vCenter
- monitoring changes to the agent virtual machines and scope in vCenter, and reporting them back to the solution

vSphere ESX Agent Manager exposes a SOAP-based API similar to the VIM API, that solutions use to register and monitor agents. The SDK includes a Model-View-Controller framework and library functions for developing vSphere extensions that interface with the vSphere Web Services (VIM) API. It is bundled in the vSphere Management SDK.

The EAM service endpoint is

```
https://{domain}/eam/sdk
```

For more information about using the ESX Agent Manager, see the following publication:

- *Developing and Deploying vSphere Solutions, vServices, and ESX Agents*

vCenter Management API

You can use the vCenter Management API to automate management tasks for vCenter. Instead of managing data center infrastructure, these interfaces manage the vCenter instance itself.

Features of the vCenter Management API include:

- Installation and Setup
- Configuration
- Patching and Upgrade
- Backup and Restore

The vCenter Management API uses a REST JSON RPC protocol, associated with the Automation API. It is available for download in the vSphere Automation SDKs.

For most operations, the vCenter Management API uses the following service endpoint, where `{service}` is the name of a specific service interface.

```
https://{domain}/api/{service}
```

During setup, until networking is configured and the service is available on port 443, the vCenter Management API uses the 5480 service endpoint:

```
https://{domain}:5480/api/{service}
```

For more information about using the vCenter Management API, see the following publications:

- vCenter Management topics in the *vSphere Automation REST API Programming Guide*
- vCenter vCenterManagement topics in the *VMware Cloud Foundation SDKs Programming Guide*
- Related interfaces in the *vSphere Automation SDKs Programming Guide*

vCenter Single Sign-On API

The vCenter Single Sign-On API is a programming interface for secure communications with vCenter Single Sign-On service.

The vCenter Single Sign-On (SSserver provides a Security Token Service (STS). A token uses the Security Assertion Markup Language (SAML), which is an XML encoding of authentication data.

vCenter Single Sign-On authentication can use the following identity store technologies:

- Windows Active Directory
- OpenLDAP (Lightweight Directory Access Protocol)
- Local user accounts (vCenter Single Sign-On server resident on the vCenter machine)
- vCenter Single Sign-On user accounts

This API defines a set of request operations that correspond to the WS-Trust 1.4 bindings:

- Issue – Obtains a token from a vCenter Single Sign-On server.
- Renew – Renews an existing token.
- Validate – Validates an existing token.
- Challenge – Part of a negotiation with a vCenter Single Sign-On server to obtain a token.

The vCenter Single Sign-On SDK includes Java bindings for the vCenter Single Sign-On WSDL. The SDK also contains sample code that demonstrates client-side support for the WSSecurityPolicy standard. Security policies specify the elements that provide SOAP message security. To secure SOAP messages, a client inserts digital signatures, certificates, and SAML tokens into the SOAP headers for vCenter Single Sign-On requests. The Java sample includes a JAX-WS implementation of SOAP header methods that support the vCenter Single Sign-On security policies.

The Single Sign-On service endpoint is

```
https://{domain}/STS/STSService
```

For more information about using the vCenter Single Sign-On API, see the following documentation:

- *vCenter Single Sign-On Programming Guide*

Virtual Disk API

The Virtual Disk API provides methods for backup and recovery, cloning, and other operations on virtual disks.

The Virtual Disk API can connect to local or network virtual disks, either for ESX or hosted applications.

The Virtual Disk API supports the following functions:

- Create, open, close, and delete virtual disks
- Read and write both data and metadata
- Grow, shrink, and defragment virtual disks
- Manage virtual disk snapshot chains
- Quiesce virtual machine access and flush buffers
- Clone virtual disks

The Virtual Disk API is provided as a C language library, running in-process as part of user-defined utility software. It is often used in conjunction with Java-based vStorage APIs for Data Protection (VADP) for developing backup applications.

The Virtual Disk API is available as part of the Virtual Disk Development Kit (VDDK).

For more information about the Virtual Disk API, see the *Virtual Disk Development Kit Programming Guide*.

VKS (vSphere Kubernetes Service) Programming

VKS (vSphere Kubernetes Service) is included with VCF, providing self-service operations for developers working with Kubernetes containers.

VKS provides self-service lifecycle management of Kubernetes workload clusters. VKS with Supervisor runs with vSphere and integrates with underlying infrastructure including ESX, vCenter, cloud native storage, and virtual networking. Using VKS, developers can provision standard Kubernetes clusters and can receive upstream package refreshes.

VCF Automation provides an additional layer for service administrators to make life easier for tenant organization, especially DevOps programmers setting up Kubernetes clusters with minimal assistance. Although not documented here, service administrators can deploy Terraform for further automation.

VKS management allows vSphere administrators to control VKS clusters, possibly of different versions, running on various platforms. Admins download the VKS Management Proxy YAML manifest, upload it as a new service in vCenter, enable it on a `host:port`, and finally verify service health. DevOps programmers can request Kubernetes containers by creating a YAML (example below) and running `kubectl apply`.

```
kind: Cluster
metadata:
  name: test-cluster
  namespace: test-namespace
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080
    hostPort: 30070
```

For developer-oriented reference documentation about VKS, see **VCF 9.1 APIs > VCF Automation APIs > VMware vSphere Kubernetes Service (VKS)** on the developer.broadcom.com website. The API reference documentation is organized by packages and resource types, which describe the mandatory and optional fields that constitute the declarative YAML descriptions for cluster management.

For administrator-oriented documentation about VKS, see *vSphere Supervisor Services and Standalone Components* on the techdocs.broadcom.com website. You sometimes see the acronym VKSM, an abbreviation for VKS cluster Management, which is the same thing as VKS.

API Design Standards

- Write APIs are usually declarative, supporting the `kubectl apply` command.
- For read-only objects, write methods are rejected by role-based access control (RBAC).
- All APIs support pagination, and return generic read-only metadata fields, including UID, Generation, and resourceVersion if applicable.
- Status follows the Kubernetes pattern of phase, conditions, and other fields as needed.
- All APIs support labels and annotations. System labels are in format: `mgmt.k8s.vmware.com/<key>: <value>`
- APIs support standard Kubernetes label selectors (for 9.1) and field selectors (for 9.2). System labels should be added for any fields that are commonly used for filtering.
  - Fields in `spec.selector.matchNames` must be added as system labels to any filter resources that use label selectors. Example:
    ```
    kubectl get clusterresources -l mgmt.k8s.vmware.com/cluster=cl-01
    ```
  - Other fields in `spec` and `status` that can be used as field selectors should be mentioned in documentation. Example:
    ```
    kubectl get clusters --field-selector status.health!=HEALTHY
    ```
  - If set-based filtering is required on a field, a system label should be added for ease of filtering using a label selector. Example:
    ```
    kubectl get clusterresources -l mgmt.k8s.vmware.com/object-kind (Deployment, DaemonSet)
    ```

Cluster and Cluster Resource APIs

Cluster observability APIs are for viewing clusters and resources that VKS manages.

These are declarative APIs. After you create a workable description in YAML, run `kubectl get` to request information about the cluster. YAML examples follow below.

Methods	Kubernetes URN	Description
GET	/apis/core.management.kubernetes.com/v1alpha1/clusters	View all clusters available for managing VKS, along with status.
GET	/apis/core.management.kubernetes.com/v1alpha1/clusterresources	View various custom resources (CRs) in VKS clusters

Cluster

```
apiVersion: core.management.kubernetes.vmware.com/v1alpha1
kind: Cluster
metadata:
  creationTimestamp: "2025-09-17T05:58:28Z"
  labels:
    mgmt.k8s.vmware.com/name: cl-01
    mgmt.k8s.vmware.com/supervisor-namespace: ns-01
  name: kubernetes-cluster-vksm:supervisor-namespace:ns-01
  namespace: project-a
  resourceVersion: "45"
  uid: c:01K7E2C6J3VY3814BCBKSS3KPD
spec:
  selector:
```

```
    matchNames:
      supervisorNamespace: ns-01
status:
  conditions:
  - lastTransitionTime: "1970-01-01T00:00:00Z"
    message: cluster is connected to VKSM and healthy
    reason: 'phase: COMPLETE, health: HEALTHY'
    status: "True"
    type: READY
  - lastTransitionTime: "1970-01-01T00:00:00Z"
    message: all extensions are up-to-date
    reason: ExtensionsUpToDate
    status: "False"
    type: ExtensionsOutOfSync
  health:
    controllerManagerHealth:
      message: Healthy
      name: controller-manager
      state: HEALTHY
    etcdHealth:
    - message: Healthy
      name: etcd-0
      state: HEALTHY
    lastUpdatedTime: "2025-09-17T05:58:28Z"
    message: Cluster is healthy
    schedulerHealth:
      message: Healthy
      name: scheduler
      state: HEALTHY
  phase: READY
  state: HEALTHY
```

ClusterResource

```
apiVersion: core.management.kubernetes.vmware.com/v1alpha1
kind: ClusterResource
metadata:
  creationTimestamp: "2025-09-17T05:60:47Z"
    labels:
      key1: val1     # pod labels
      key2: val2     # pod labels
      mgmt.k8s.vmware.com/cluster: cl-01
      mgmt.k8s.vmware.com/kind: Deployment
      mgmt.k8s.vmware.com/name: test-app-xxx
      mgmt.k8s.vmware.com/supervisorNamespace: ns-01
    name: test-app-xxx:cluster:cl-01:supervisor-namespace:ns-01
    namespace: project-a
    resourceVersion: "4789557"
    uid: db8d7ce8-c2fd-49e5-a246-8726f285ff64
  spec:
    selector:
      matchNames:
        cluster: cl-01
        supervisorNamespace: ns-01
    clusterNamespace: my-test-ns
    gvk:
      group: apps
      kind: Deployment
```

```
      version: v1
    object:
      # Actual Deployment object
  status:
    aggregatedData:
      cpuLimits: 500m
      cpuRequests: 100m
      memoryLimits: 1Gi
      memoryRequests: 256Mi
      totalPods: 1
    state: HEALTHY`
```

Cluster Policy Management APIs

Cluster policy management APIS are for viewing policy templates, schemas, or insights, and for managing templates and policies themselves.

These are declarative APIs. After you create a workable description in YAML, run `kubectl apply` to view policies or update the cluster to its desired state. YAML examples follow below.

Methods	Kubernetes URN	Description
GET CREATE UPDATE DELETE APPLY	/apis/policy.management.kubernetes.vmware.com/v1alpha1/policytemplates	Manage Policy templates with VKSM (e.g. custom Gatekeeper ConstraintTemplates)
GET	/apis/policy.management.kubernetes.vmware.com/v1alpha1/policyschemas	View all the schemas available for creating policies
GET CREATE UPDATE DELETE APPLY	/apis/policy.management.kubernetes.vmware.com/v1alpha1/policies	Control Kubernetes policies with VKS management
GET	/apis/policy.management.kubernetes.vmware.com/v1alpha1/policyinsights	View various insights about your policies

PolicyTemplate

```
apiVersion: core.management.kubernetes.vmware.com/v1alpha1
kind: ClusterPolicyTemplate
metadata:
  name: test-template
  namespace: @org
  labels:
    mgmt.k8s.vmware.com/policy-type: custom-policy
    # below label for built-in read-only templates
    # mgmt.k8s.vmware.com/built-in: true
spec:
  templateType: "OPAGatekeeper"
  objectType: "ConstraintTemplate"
  # ConstraintTemplate yaml object below
  object: |
    ...
  # optional cached resources needed by Gatekeeper
  dataInventory:
  - group: ""
    version: "v1"
    kind: "Pod"
  policyUpdateStrategy:
```

```
    type: INPLACE_UPDATE
```

PolicySchema

```
apiVersion: policy.management.kubernetes.vmware.com/v1alpha1
kind: ClusterPolicySchema
metadata:
  creationTimestamp: "1970-01-01T00:00:00Z"
  name: baseline:security-policy
  namespace: '@org'
  uid: pr:01EKZHJ3AZBT9QZCKP0VA24B50
spec:
  type: security-policy
  clusterPolicyTemplateRefs:
  - name: vksm-system-block-privileged-container
    namespace: '@org'
  - name: vksm-system-block-privilege-escalation
    namespace: '@org'
  - name: vksm-system-block-host-namespace
    namespace: '@org'
  - name: vksm-system-enforce-host-networking
    namespace: '@org'
  - name: vksm-system-allowed-volumes
    namespace: '@org'
  - name: vksm-system-allowed-users
    namespace: '@org'
  - name: vksm-system-linux-capabilities
    namespace: '@org'
  - name: vksm-system-allowed-host-paths
    namespace: '@org'
  - name: vksm-system-allowed-procmount-types
    namespace: '@org'
  - name: vksm-system-forbidden-sysctls
    namespace: '@org'
  openAPIV3Schema:
    # schema of the inputs required for creating a policy using this policy schema
```

ClusterResource

```
apiVersion: core.management.kubernetes.vmware.com/v1alpha1
kind: ClusterResource
metadata:
  creationTimestamp: "2025-09-17T05:60:47Z"
    labels:
      key1: val1     # pod labels
      key2: val2     # pod labels
      mgmt.k8s.vmware.com/cluster: cl-01
      mgmt.k8s.vmware.com/kind: Deployment
      mgmt.k8s.vmware.com/name: test-app-xxx
      mgmt.k8s.vmware.com/supervisorNamespace: ns-01
    name: test-app-xxx:cluster:cl-01:supervisor-namespace:ns-01
    namespace: project-a
    resourceVersion: "4789557"
    uid: db8d7ce8-c2fd-49e5-a246-8726f285ff64
  spec:
    selector:
      matchNames:
        cluster: cl-01
        supervisorNamespace: ns-01
```

```
    clusterNamespace: my-test-ns
    gvk:
      group: apps
      kind: Deployment
      version: v1
    object:
      # Actual Deployment object
  status:
    aggregatedData:
      cpuLimits: 500m
      cpuRequests: 100m
      memoryLimits: 1Gi
      memoryRequests: 256Mi
      totalPods: 1
    state: HEALTHY
```

ProjectPolicy

```
apiVersion: core.kubernetes.management.vmware.com/v1alpha1
kind: ClusterPolicy
metadata:
  name: my-test-policy1
  namespace: project-a
  labels:
    mgmt.k8s.vmware.com/name: my-test-policy1
spec:
  # Any schema from PolicySchema API
  schema: custom:image-registry-policy
  # Input following the inputSchema in the corresponding PolicySchema
  input:
    enforcementAction: warn
    rules:
    - hostname: "dev.com"
  # optional workload namespace selector
  clusterNamespaceSelector:
  - matchExpressions:
      key: "env"
      operator: "In"
      values: ["dev", "staging"]
```

ClusterPolicy

```
apiVersion: core.kubernetes.management.vmware.com/v1alpha1
kind: ClusterPolicy
metadata:
  name: my-test-policy2:cluster:cl-01:supervisor-namespace:ns-01
  namespace: project-a
  labels:
    mgmt.k8s.vmware.com/name: my-test-policy2
    mgmt.k8s.vmware.com/cluster: cl-01
    mgmt.k8s.vmware.com/supervisor-namespace: ns-01
spec:
  selector:
    matchNames:
      cluster: cl-01
      supervisorNamespace: ns-01
  # Any schema from PolicySchema API
  schema: restricted:security-policy
  # Input following the inputSchema in the corresponding PolicySchema
```

```
  input:
    enforcementAction: deny
  # optional workload namespace selector
  clusterNamespaceSelector:
  - matchExpressions:
      key: "env"
      operator: "In"
      values: ["production"]
```

PolicyInsight

```
apiVersion: core.kubernetes.management.vmware.com/v1alpha1
kind: ClusterPolicyInsight
metadata:
  name: <uid>
  namespace: project-a
spec:
  selector:
    matchNames:
      cluster: cl-01
      supervisorNamespace: ns-01
  clusterNamespace: test-ns
  # Source policy identifier
  policy:
    name: my-test-policy
    namespace: project-a
    selector:
      matchNames:
        cluster: cl-01
        supervisorNamespace: ns-01
status:
  policy-type: image-registry-policy
  insight-type: Violation
  details: "... violation message details ..."
  lastUpdate: "2025-09-02T07:04:12Z"
```

VKS Data Protection APIs

Service administrators can schedule backup operations and can restore portions of VKS clusters using these data protection APIs.

For more information about VKS backup and restore, see documentation about VMware storage APIs for Data Protection (VADP) in the *Virtual Disk Development (VDDK) Programming Guide*.

Methods	Kubernetes URN	Description
GET LIST CREATE DELETE	`clusterdataprotection`	Returns VKS clusters with data protection enabled.
GET LIST CREATE UPDATE DELETE	`clusterbackuplocation`	Sets the backup location for data protection.
GET LIST CREATE DELETE	`clusterbackup`	Shows time of previous backups, given a namespace.
GET LIST CREATE DELETE	`clusterrestore`	Choose cluster and initiate restore, given a namespace.
GET LIST CREATE UPDATE DELETE	`clusterschedule`	Set a schedule for VKS cluster backup.

When deleting a backup file that refuses to disappear, the Kubernetes data protection API does not support the `force=true` option; `false` is the option default. You can use the VCF Automation UI to **Delete backup**files, selecting the option "Destroy all backup files associated with this cluster."

When disabling data protection on a cluster, the Kubernetes data protection API does not support the `deleteBackups=true` option; `false` is the option default. The system will eventually delete backup files from remote storage.

In both cases, another solution is to manually delete backup files from remote storage, for example using S3 filesystem tools.

vSphere Add-On SDKs

Add-on API layers are available for some popular vSphere APIs. Open-source communities can contribute new language bindings or usability improvements.

Additionally, Broadcom offers many stand-alone SDKs. Two prominent ones are the vSphere Client SDK, used to develop plug-ins and user interfaces for the vSphere Client, and `pyvmomi`, the Python virtual machine object managment interface, Guest SDK.

Go Language with govmomi

For Go language programmers, `govmomi` is a library for interacting with the VMware Web Services API for ESX and vCenter.

The code in the `govmomi` package is a wrapper for the code that is generated from the vSphere API description. It primarily provides convenience functions for working with the vSphere API:

- Generated types, managed objects, and methods for the complete VIM API
- Client structure that abstracts a session and its immutable state
- Session Manager abstraction that allows a user to login and logout
- Wrappers for a selection of managed objects
- Generated coverage for some other SOAP-based APIs, such as SMS, SPBM, and vSAN
- Partial coverage for the vSphere Automation API

For more information about using `govmomi`, see the following documentation:

- https://pkg.go.dev/github.com/vmware/govmomi

The vSphere Client SDK

The vSphere Client is an HTML5 web application that allows plug-in extensions to control various subsystems, especially modules created by partners.

The vSphere Client SDK provides tools and examples that help you create custom plug-ins to extend its functionality. See Client SDK on the Broadcom developer site for details.

VMware Guest SDK

The Guest SDK offers sample code and documentation about public interfaces to VMware guest operating systems.

- The virtual machine guest library VMGuestLib
- Toolbox for extended guest statistics
- DataSets for guest management
- High availability (HA) application monitoring.

An example use case for manipulating Datasets appears below in Guest SDK and Datasets.

Tagging Examples Using vSphere APIs

This chapter contains a preliminary section about required resources, and then sections providing a summary of authentication options, followed by examples in different languages to create a tagged folder.

Subsequent sections describe how to set up a test environment for various SDKs so you can run sample programs that:

1. Open a connection to the server and create an authenticated session.
2. Create a 'Hello' folder in a data center.
3. Create a tag category.
4. Create a tag within the category.
5. Attach the tag to the folder.

These operations are implemented in several languages, including a shell script that uses `curl` requests to accomplish these steps.

Although these code examples create several objects, they do not delete anything. You can verify the results by using the VMware vSphere Client™, then delete the sample objects manually if you want to run another example.

> **Note:**
> The code examples in this book use basic authentication and omit server certificate verification for simplicity. In a production environment you should use stricter security practices.

Preliminary Resources for vSphere Programming

This topic explains the vSphere resources you need to access when you run the examples provided in subsequent chapters.

To run the API examples shown in this book, you need access to a test installation of vSphere and a development workstation or virtual machine, including:

- vCenter
- Data center storage space to create virtual machines
- A user role with permissions on vCenter that allow you to create folders and tag objects in the vSphere inventory.
- A developer workstation, with space to download & install SDKs

The example code creates an inventory object folder and a tag for the folder. You can run these examples in your on-premises data center.

If you do not have a vSphere test environment readily available, you can easily create a test environment with VMware Cloud™ on AWS. Refer to the following documentation:

Getting Started With VMware Cloud on AWS

Alternatively, you may be able to install vCenter as a virtual machine running on VMware Workstation™ or VMware Fusion®. This is not a supported feature, but some users have used the Open Virtualization Format Tool to set up test installations using the vCenter Appliance™ running on Workstation or Fusion. If you choose this method, the following links are helpful:

https://williamlam.com/2022/10/how-to-deploy-the-vcenter-server-appliance-vcsa-8-0-to-vmware-fusion-or-workstation.html

https://developer.broadcom.com/tools/open-virtualization-format-ovf-tool/latest/

Authentication with vSphere APIs

This chapter provides an overview of authentication options for vSphere users. You can use basic authentication or SAML token-based authentication to access vSphere APIs.

Because it is the simplest method, this guide uses basic authentication to access vSphere APIs in the following chapters.

You can use both the vSphere Management APIs and the vSphere Automation API to manage your vSphere environment. With the VMware Cloud Foundation Java and Python SDKs, you gain access to a wider range of supported automation features. Use the:

- vSphere Management APIs for host management, virtual machine provisioning, data center monitoring, virtual storage management, and more.
- vSphere Automation APIs for configuration, authentication, certificate management, tagging, lifecycle management, content libraries, Kubernetes cluster management, and more.

Choosing Your Identity Source

To administer vSphere users and groups, you must first choose your identity source. You can either:

Use the vCenter built-in identity source	vSphere comes with a built-in identity provider, called vCenter Single Sign-On. By default, vCenter uses the `vsphere.local` domain as the identity source (but you can change this domain during installation). A vCenter Single Sign-On administrator can manage users and groups in the `vsphere.local` domain from the vSphere Client. You can configure the vCenter built-in identity provider to use Active Directory (AD) as its identity source using LDAPS, OpenLDAP, and Integrated Windows Authentication (IWA). Such configurations allow customers to log into vCenter using their Active Directory or LDAP accounts.
Use an external identity provider as the identity source	In vSphere 7.0 and later, you can outsource identity management to an external identity provider through federated authentication. You replace vCenter as the identity provider and the administration of users and groups occurs on the external identity source. When you configure vSphere to use an external identity provider, the external identity provider interacts with the identity sources on behalf of vCenter . You can federate vCenter to the following external identity providers: • Active Directory Federated Service (ADFS), starting in vSphere 7.0 • Okta, starting in vSphere 8.0 Update 1 • Microsoft Entra AD, starting in vSphere 8.0 Update 2

For more information on vSphere identity sources and identity management, see the topic "Authentication Mechanisms" in the *vSphere Automation REST API* chapter.

Obtaining an Identifier and Unified Session

Before using the API services, principals must first obtain session identifiers for the vSphere Management API and the vSphere Automation API. Valid session identifiers guarantee the identity of the principals and establish trust with the API endpoints.

To obtain a session identifier for the vSphere APIs, you can use different authentication mechanisms, as covered below. Once obtained, the session identifier is sent with every HTTPS request, or added to the security context for authentication.

Before vSphere 8.0 Update 3, separate sessions were required for vSphere Management API and for vSphere Automation API. As of vSphere 8.0 U3, unified sessions are supported. Now, developers can re-use a vSphere Management (VIM) session for vSphere Automation (vAPI) calls, and vice versa. See Unified Management and Automation Session for details and code examples.

Starting with VCF 9.1, client applications can access vSphere APIs without first obtaining a session identifier. Instead, pass a VIDB Bearer Access Token (VAT) directly in the `Authorization: Bearer` HTTP header. Your existing vCenter RBAC permissions apply when using OAuth bearer access token authentication. This approach eliminates the token-to-session exchange step that is required in previous releases and is recommended for new automation workflows.

For APIs that maintain server-side state across multiple calls (for example, Property Collectors and Event Collectors), vCenter returns a Server-side State Cookie (`vmware-vsphere-state`) on the first request. Include this cookie alongside the bearer access token in subsequent API calls to preserve the stateful session context.

Choosing Your Authentication Method

To obtain a session identifier, you can choose among several authentication mechanisms depending on your needs and requirements. Simply stated, these mechanisms can be divided in two major groups: basic authentication and token-based authentication.

Basic Authentication

With basic authentication, you supply your vCenter or external provider user name and password when connecting to the vSphere Management or vSphere Automation endpoints.

- If you use the built-in vCenter identity source, you connect with your local domain credentials.
- With an external identity source, use your identity provider credentials to connect to the vSphere API endpoints. This workflow is possible thanks to the OAuth 2.0 Password grant type.

For more information, see these topics:

- *Authenticate to vCenter with vCenter Single Sign-On Credentials* in the *VMware vSphere Automation REST API* chapter.
- *Authenticate by Using the Password Grant Type* in the *VMware vSphere Automation REST API* chapter.
- *Create a vSphere Automation Session with User Credentials* in the *VMware vSphere Automation SDKs* chapter.

> **Note:**
>
> Use token-based authentication. It provides stronger security and greater flexibility for client applications.

SAML Token-Based Authentication

With token-based authentication, principals issue a token request to the vCenter Single Sign-On built-in service. In response, vCenter Single Sign-On issues a Security Assertion Markup Language (SAML) token that principals use to obtain session identifiers for the vSphere Management API and the vSphere Automation API.

Obtaining a SAML token

You can obtain a SAML token by using either the `Issue` service of the vSphere Management API or the `AuthenticationToken` service of the vSphere Automation API. For more information, see these topics:

- "Authenticate to an AD FS-Federated vCenter by Using the Authorization Code Grant Type" in the *VMware vSphere Automation REST API* chapter.
- "Acquiring a SAML Token from a vCenter Single Sign-On Server" in the *vCenter Single Sign-On* chapter.
- "Retrieve a SAML Token" in the *VMware vSphere Automation SDKs* chapter.

> **Important:**
>
> If you are using an external identity provider as the identity source, you must first obtain a JSON Web Token (JWT) from the identity provider and then exchange it for a SAML token. For more information, see the topic "Authenticate to an AD FS-Federated vCenter by Using the Authorization Code Grant Type" in the *VMware vSphere Automation REST API* chapter.

Creating a Session with the SAML Token

After you receive the SAML token, you use it to obtain a session identifier from the vSphere API endpoints.

- For the vSphere Management API, use the `LoginByToken` service (in `SessionManager`).
- For the vSphere Automation API, use the `CISSession` service.

Once you have obtained a session identifier, you can use it for authentication in your subsequent calls or in your security context.

> **Note:**
> Before vSphere 8.0 U3, you had to create separate authentication sessions for the vSphere Management API and the vSphere Automation API. That is, you needed a separate session identifier for each of the APIs. With a unified session, this is no longer necessary.

For more information, see chapter "Authentication Mechanisms" in the *VMware vSphere Automation REST API* chapter and "LoginByToken Example" in the *vCenter Single Sign-On* chapter.

Unified Management and Automation Session

As of vSphere 8.0 Update 3, developers can re-use a vSphere Management (VIM) session for making vSphere Automation (vAPI) calls, or re-use a vAPI session for making VIM calls. Examples follow.

Here is a Python code example to establish a vSphere Automation (vAPI) session to then invoke a vSphere Management (VIM) call:

```
import requests
from pyVim.connect import SmartConnect
from vmware.vapi.vsphere.client import create_vsphere_client

vc_host = '10.168.203.112'
user = 'Administrator@vsphere.local'
password = 'NKCF_n4PagP0jmz_'

# vAPI setup
session = requests.session()
session.verify = False

client = create_vsphere_client(vc_host, user, password, session=session)

# Acquire session_id
session_id = client.get_session_id()

# Reuse session_id in SmartConnect
si = SmartConnect(host=vc_host,
                  port=443,
                  disableSslCertValidation=True,
                  sessionId=session_id)
content = si.RetrieveContent()
dc_moref = content.rootFolder.CreateDatacenter("TEST")
print(dc_moref)
```

Here is a Python code example to establish a vSphere Management (VIM) session to then invoke a vSphere Automation (vAPI) call:

```
import requests
from pyVim.connect import SmartConnect
from vmware.vapi.vsphere.client import create_vsphere_client

vc_host = '10.168.203.112'
user = 'Administrator@vsphere.local'
password = 'NKCF_n4PagP0jmz_'

# Login with pyvmomi
si = SmartConnect(host=vc_host,
                  port=443,
                  user=user,
                  pwd=password,
                  disableSslCertValidation=True)

# TODO GetSessionId documentation
session_id = si._GetStub().GetSessionId()

# vAPI setup
session = requests.session()
session.verify = False

client = create_vsphere_client(vc_host, session=session, session_id=session_id)
print(client.vcenter.compute.Policies.list())
```

Here are example Curl commands to establish a VIM session via JSON, make a vAPI call, and then call the REST tag API. The second and third are get requests and do not apply anything. The second command returns a list of folders. The third returns a list of tags for folder `group-d3`.

```
$ curl "https://$VC/sdk/vim25/8.0.2.0/SessionManager/SessionManager/Login"
  -H 'Content-Type: application/json'
  -d '{"userName": "Administrator@vsphere.local", "password": "$PSWD"}' -ki
$ curl https://$VC/api/vcenter/folder -H "vmware-api-session-id: $VIM_SESSION" -k
$ curl https://$VC/sdk/vim25/8.0.2.0/Folder/group-d3/tag
  -H "vmware-api-session-id: $VIM_SESSION" -k
```

Here are example Curl commands to create a REST session and use it to call the VIM tag API:

```
$ curl -X POST https://$VC/api/session -u "administrator@vsphere.local:$PSWD" -k
$ curl "https://$VC/sdk/vim25/8.0.2.0/Folder/group-d1001/tag"
  -H "vmware-api-session-id: $VAPI_SESSION" -k
```

The following caveats and limitations apply to unified VIM/vAPI sessions:

- Session timeout can be configured with property `config.vmacore.soap.sessionTimeout` in file `/etc/vmware-vpx/vpx.conf` on vCenter, or using the vSphere Client UI.
- Default session lifespan can be configured with property `vpx.sso.sts.tokenLifetime`, also in `vpx.conf`, or using the vSphere Client UI.
- The maximum number of inflight sessions can be configured with property `config.vmacore.soap.maxSessionCount`, also in `vpx.conf`, or using the vSphere Client UI.
- Session IDs from either endpoint work in the other endpoint. Clients that want the benefit of a single session should adjust to accept variable-size Session ID tokens.

Python Access to vSphere APIs

For the Python developer, this chapter shows how to install the Python SDKs and run example programs against vSphere APIs.

VMware vSphere offers two important programming interfaces: (1) Web Services API in the vSphere Management SDK, which is SOAP-based and the integration point for `pyvmomi`. (2) vSphere Automation API, which is primarily REST-based but has additional bindings for Python and Java. SOAP interfaces were first used decades ago to implement the vSphere Client UI. The Automation API is newer with additional features and some duplication. Code samples in this chapter use them together.

Prerequisite: Python 3.8 or higher. Recent Linux distributions and MacOS include higher Python versions. The command name is probably `python3` but can be aliased.

Installing the Python SDKs for vSphere

Python access to vSphere APIs is most convenient with the `pyvmomi` library. To install `pyvmomi`:

1. If not already installed, get the Python installer `pip`. On Ubuntu systems, the package is named `python-pip`. On MacOS there are several alternatives; one is to download and run `get-pip.py`. In any case, add `pip`'s location to your PATH.
2. Update `pip` and setup tools, then install (or upgrade) `pyvmomi` with the `pip` command.

   ```
   pip install --upgrade pip
   pip install --upgrade setuptools
   pip install --upgrade pyvmomi
   ```

3. Install the vSphere Automation SDK for Python from its Github archive. The expression `git+https` means to use secure HTTP for downloading the Github install.

   ```
   pip install --upgrade git+https://github.com/vmware/vsphere-automation-sdk-python.git
   ```

   You could also download the vSphere Automation SDK for Python (as a ZIP file) from its https://developer.broadcom.com/sdks landing page. Install instructions, including for a secure air-gapped environment, are

available in the Quick Start Guide section of README.md on the http://github.com/vmware/vsphere-automation-sdk-python web page.

4. Now you are ready to run the code samples below. Copy and paste the code into a file and run the `python` or `python3` command on the file. You can run one at a time, or all three copy-and-pastes together at once.

Basic Authentication Using pyvmomi

The `pyvmomi` SDK simplifies code for connecting to the VIM endpoint by using a `SmartConnect` function that accepts basic user credentials, opens a session with the server, and returns the `ServiceInstance` object. `ServiceInstance` contains references to specific managed objects needed to use VIM API features.

The `vsphere_client` object contains interface stubs for all Automation API endpoints. This code excerpt shows a function that connects to both the VIM endpoint and the Automation API endpoint and returns both the `vsphere_client` object and the `ServiceInstance` object.

```
hostname = 'vc1.example.com'
username = 'administrator@vsphere.local'
password = 'TooSecret2C!'

from pyVim.connect import SmartConnect
from vmware.vapi.vsphere.client import create_vsphere_client, VsphereClient
import requests, urllib3
import sys
from pyVmomi import vim

def connect(host: str, user: str, pwd: str, insecure: bool) -> tuple[VsphereClient, vim.ServiceInstance]:
    session = requests.session()
    if insecure:
        session.verify = False
        urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
    print('Creating an Automation API session.')
    vsphere_client = create_vsphere_client(host, user, pwd, session=session)
    print('Creating a VIM/SOAP session.')
    si = SmartConnect(host=host,
                      user=user,
                      pwd=pwd,
                      disableSslCertValidation=insecure)
    return vsphere_client, si

(vsphere_client, service_instance) = connect(hostname,
                                             username,
                                             password,
                                             insecure=True)
```

At this point, your session is established and the session ID is stored in a cookie that will be passed along with future requests.

Creating a Hello Folder with the VIM API Using pyvmomi

The `ServiceContent` object contains a reference to the root folder of the managed object hierarchy. You can invoke the `CreateFolder()` method on the `rootFolder` object to create a subfolder named `Hello Folder`.

```
...
print('Using VIM to create Hello Folder.')
try:
  hello_folder = service_instance.content.rootFolder.CreateFolder('Hello Folder')
except vim.fault.DuplicateName as oops:
  print(oops)
  print('Please delete folder from Inventory and try again.\n')
  sys.exit(1)
```

```
print('Hello Folder MOref: ', hello_folder, type(hello_folder), '\n')

# Convert object IDs from VIM to VAPI.
def vim_moref_to_moid(moref):
  return moref._GetMoId()
...
```

Tagging the Folder with Automation API for Python

To tag a folder, start by creating a tag category, then create a tag in that category, and finally attach the tag to the folder.

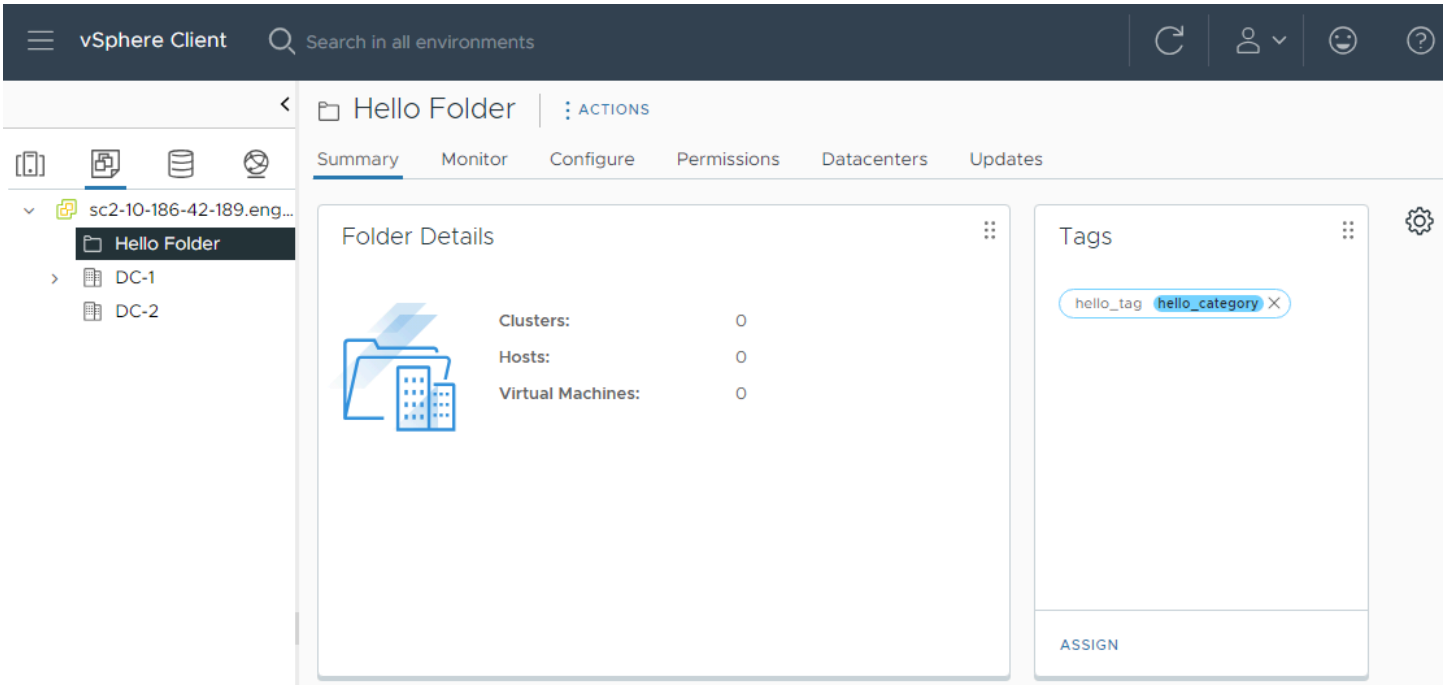
```
...
# Interface factory object:
stub_config = vsphere_client._stub_config

print('Automation API creating a tag category.')
from com.vmware.vapi.std_client import DynamicID
from com.vmware.cis.tagging_client import Category, CategoryModel, Tag, TagAssociation
import com.vmware.vapi.std.errors_client
category_id = None
tag_id = None
category_stub = Category(stub_config)
tc_create_spec = category_stub.CreateSpec(name='hello_category',
   description = 'example of tagging category from pyvmomi',
   cardinality = CategoryModel.Cardinality.MULTIPLE,
   associable_types = set(['Folder']))
try:
  category_id = category_stub.create(tc_create_spec)
except com.vmware.vapi.std.errors_client.AlreadyExists as oops:
  print(oops)
  print('Please delete category from Tags & Custom Attributes and try again.\n')
  sys.exit(1)
print('New category created: ', category_id)

print('VAPI creating a tag in new category.')
tag_stub = Tag(stub_config)
tag_create_spec = tag_stub.CreateSpec(name='hello_tag',
description= 'example of a tag',
category_id=category_id)
try:
  tag_id = tag_stub.create(tag_create_spec)
except errors_client.NotFound as oops:
  print('Exception: \n', oops, '\n')
  sys.exit(1)
print('New tag created: ', tag_id)

print('VAPI creating association of tag with Hello Folder.')
dynamic_id = DynamicID(type='Folder',
 id=vim_moref_to_moid(hello_folder))
tag_association_stub = TagAssociation(stub_config)
tag_association_stub.attach(tag_id=tag_id,
 object_id=dynamic_id)
print('Tag attached to folder.')
```

The following image shows the Hello Folder and Category as it looks in the vSphere Client UI.



What To Do Next

You can download community-sourced `pyvmomi` examples three different ways:

- `git clone https://github.com/vmware/pyvmomi-community-samples`
- the Zip file: https://github.com/vmware/pyvmomi-community-samples/zipball/master
- the tarball: https://github.com/vmware/pyvmomi-community-samples/tarball/master

Java Access to vSphere APIs

For the Java developer, this chapter shows how to install the Java SDKs and run example programs against vSphere APIs.

Installing the vSphere Automation SDK and Web Services SDK for Java

The vSphere Web Services SDK, along with several other VMware SDKs, is contained in the vSphere Management SDK, available for download at https://developer.broadcom.com/sdks/vsphere-management-sdk/latest/.

- Use Java SE with JDK 11 for production environments. vCenter supports HTTPS connections with Java 1.8, and later, but not with Java 1.7.
- Download and install the vSphere Management SDK from the developer site. Click the **SDKs** link to reach a landing page that has links to all vSphere SDKs.
  After you download and expand the vSphere Management SDK package, the Web Services SDK is in the subdirectory `SDK/vsphere-ws`
- Download and install the Automation SDK from the developer site.
- Set the `JAVAHOME` environment variable to the root path of the Java Runtime Environment (JRE), such as `C:\Program Files\Java\jdk11.0.16_221`
- If needed, build the samples for the vSphere Web Services SDK.
  This Hello Folder example uses `wssamples.jar` and `vim25.jar` to communicate with the vSphere Web Services API.
- Install these external dependencies:
  - `slf4j-api-2.0.9.jar`
  - `jackson-databind-2.15.2.jar` and its dependencies
  - `httpcomponents-httpcore-4.4.13.jar`

  - org-apache-commons-logging.jar
- Set your runtime classpath to include the following:
  - org-apache-commons-logging.jar
  - target/classes
  - vapi-runtime-2.40.0.jar
  - vim25.jar
  - wssamples.jar
  - vapi-authentication-2.40.0.jar
  - slf4j-api-2.0.9.jar
  - jackson-databind-2.15.2.jar
  - jackson-core-2.15.2.jar
  - jackson-annotations-2.15.2.jar
  - httpclient/httpclient-4.5.6.jar
  - httpcomponents-httpcore-4.4.13.jar
  - org-apache-commons-logging.jar
  - vcenter-bindings-4.1.0.jar

Creating and Tagging a Folder Using Java

The vSphere Automation (VAPI) SDK and the vSphere Web Services (VIM) SDK can be used together to supplement each API with features present in the other API. This example shows:

1. Authentication with both VAPI and VIM.
2. Using VIM to create a folder as a child of the root folder in the vSphere inventory.
3. Using VAPI to create a tag category.
4. Using VAPI to create a tag in the tag category.
5. Using VAPI to associate the tag with the folder.
6. Logging out of both sessions.

> **Note:**
> This code is for example purposes only. The example skips SSL verification and uses only basic authentication. In a production environment, both these practices are unsafe.

```
package example;

import java.security.KeyStore;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.HashSet;
import java.util.Set;

import com.vmware.vapi.bindings.StubConfiguration;
import com.vmware.vapi.protocol.HttpConfiguration;
import com.vmware.vapi.protocol.HttpConfiguration.KeyStoreConfig;
import com.vmware.vapi.protocol.HttpConfiguration.SslConfiguration;

import vmware.samples.common.authentication.VapiAuthenticationHelper;
import vmware.samples.common.authentication.VimAuthenticationHelper;
import vmware.samples.common.SslUtil;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.vapi.bindings.StubConfiguration;
```

```
import com.vmware.vapi.protocol.HttpConfiguration;
import com.vmware.vapi.protocol.HttpConfiguration.KeyStoreConfig;
import com.vmware.vapi.protocol.HttpConfiguration.SslConfiguration;

import vmware.samples.common.authentication.VapiAuthenticationHelper;
import vmware.samples.common.authentication.VimAuthenticationHelper;
import vmware.samples.common.SslUtil;
import com.vmware.connection.ConnectedVimServiceBase;
import com.vmware.vim25.ManagedObjectReference;
import com.vmware.vim25.ServiceContent;
import com.vmware.vim25.VimPortType;

import com.vmware.cis.tagging.Category;
import com.vmware.cis.tagging.CategoryModel.Cardinality;
import com.vmware.cis.tagging.CategoryTypes;
import com.vmware.cis.tagging.Tag;
import com.vmware.cis.tagging.TagAssociation;
import com.vmware.cis.tagging.TagTypes;
import com.vmware.vapi.std.DynamicID;


public class Example() extends ConnectedVimServiceBase {
  public Example() {
    this.serverName = "vc1.example.com";
    this.userName   = "administrator@vsphere.local";
    this.password   = "TooSecret2C!";
  }

  // Server URL and credentials for authentication:
  protected String serverName;
  protected String userName;
  protected String password;

  /**
   * Builds the SSL configuration to be applied for the connection to the
   * server
   *
   * Note: Below code circumvents SSL trust if "skip-server-verification" is
   * specified. Circumventing SSL trust is unsafe and should not be used
   * in production software. It is ONLY FOR THE PURPOSE OF DEVELOPMENT
   * ENVIRONMENTS.
   */
  protected SslConfiguration buildSslConfiguration() throws Exception {
      SslConfiguration sslConfig;
      /*
       * Below code enables all VIM & vAPI connections to the server
       * without validating the server certificates..
       *
       * Note: Below code is to be used ONLY IN DEVELOPMENT ENVIRONMENTS.
       * Circumventing SSL trust is unsafe and should not be used in
       * production software.
       */
      SslUtil.trustAllHttpsCertificates();
      sslConfig = new SslConfiguration.Builder()
                      .disableCertificateValidation()
                      .disableHostnameVerification()
                      .getConfig();
```

```
    return sslConfig;
}

protected HttpConfiguration buildHttpConfiguration() throws Exception {
  HttpConfiguration httpConfig =
          new HttpConfiguration.Builder()
          .setSslConfiguration(buildSslConfiguration())
          .getConfig();
  return httpConfig;
}

/**
 * Creates dual sessions with server for VIM & VAPI, using username/password.
 * @throws Exception
 */
protected VapiAuthenticationHelper vapiAuthHelper;
protected VimAuthenticationHelper vimAuthHelper;
protected StubConfiguration sessionStubConfig;

protected void login() throws Exception {
  this.vapiAuthHelper = new VapiAuthenticationHelper();
  this.vimAuthHelper = new VimAuthenticationHelper();
  HttpConfiguration httpConfig = buildHttpConfiguration();
  this.sessionStubConfig =
          vapiAuthHelper.loginByUsernameAndPassword(
              this.server, this.username, this.password, httpConfig);
  this.vimAuthHelper.loginByUsernameAndPassword(
              this.server, this.username, this.password);
}

protected void logout() throws Exception {
  this.vapiAuthHelper.logout();
  this.vimAuthHelper.logout();
}

protected ManagedObjectReference createHelloFolder() throws Exception {
  System.out.println("Use VIM API to create a Hello Folder.");
  ServiceContent content = this.vimAuthHelper.getServiceContent();
  ManagedObjectReference folderMoRef = content.getRootFolder();
  VimPortType vimPort = this.vimAuthHelper.getVimPort();
  ManagedObjectReference hello =
     vimPort.createFolder(folderMoRef, "Hello Folder");
  System.out.println("Sucessfully created Hello Folder.");
  return hello;
}

protected String createTagCategory() throws Exception {
  System.out.println("VAPI creating a tag category.");
  Category categoryStub =
     this.vapiAuthHelper.getStubFactory().createStub(Category.class,
                                              sessionStubConfig);
  CategoryTypes.CreateSpec createSpec = new CategoryTypes.CreateSpec();
  createSpec.setName("hello_category");
  createSpec.setDescription("example of tagging category from Java");
  createSpec.setCardinality(Cardinality.SINGLE);
  Set<String> set = new HashSet<>();
  set.add("Folder");
```

```
    createSpec.setAssociableTypes(set);
    String newCategoryId = categoryStub.create(createSpec);
    System.out.println("Created \"hello_category\".");
    return newCategoryId;
  }

  protected String createTag(String cat) throws Exception {
    System.out.println("VAPI creating a tag.");
    Tag tagStub =
       this.vapiAuthHelper.getStubFactory().createStub(Tag.class,
                                                sessionStubConfig);
    TagTypes.CreateSpec createSpec = new TagTypes.CreateSpec();
    createSpec.setName("hello_tag");
    createSpec.setDescription("example of a tag from Automation API");
    createSpec.setCategoryId(cat);
    String newTagId = tagStub.create(createSpec);
    System.out.println("Created \"hello_tag\".");
    return newTagId;
  }

  protected void associateTag(String tag, DynamicID folder) throws Exception {
    System.out.println("Using Java to associate a tag to a folder.");
    TagAssociation assocStub =
       this.vapiAuthHelper.getStubFactory().createStub(TagAssociation.class,
                                                sessionStubConfig);
    assocStub.attach(tag, folder);
    System.out.println("Associated \"hello_tag\" with \"Hello Folder\".");
  }

  public static void main(String[] args) throws Exception {
    Example example = new Example();
    example.login();
    System.out.println("Logged in to VIM and to VAPI.");
    ManagedObjectReference helloFolder = example.createHelloFolder();
    String cat = example.createTagCategory();
    String tag = example.createTag(cat);
    DynamicID folder =
                new DynamicID(helloFolder.getType(),
                              helloFolder.getValue());
    example.associateTag(tag, folder);
    example.logout();
  }
}
```

Golang Access to vSphere APIs

For the Golang developer, this chapter shows how to install the govmomi SDK and run example programs against vSphere APIs.

Installing the govmomi SDK for vSphere

To access vSphere APIs using the Go language, the `govmomi` library is most convenient. To install `govmomi`, open a terminal window and use these commands:

- mkdir govmomi

- cd govmomi
- go mod init example/govmomi
- go get -u github.com/vmware/govmomi

Basic Authentication with the VIM API Using govmomi

The `govmomi` SDK simplifies the connection code with the `govmomi.NewClient` function. The function accepts a service URL containing basic user credentials, opens a session with the server, and returns a `client` object containing the `ServiceContent` object. `ServiceContent` contains references to specific managed objects needed to use VIM API features.

```
package main

import (
        "context"
        "fmt"
        "net/url"

        "github.com/vmware/govmomi"
        "github.com/vmware/govmomi/vim25/methods"
        "github.com/vmware/govmomi/vim25/types"
        "github.com/vmware/govmomi/vapi/rest"
        "github.com/vmware/govmomi/vapi/tags"
)

// Connection parameters.
var hostname = "vc1.example.com"
var username = "administrator@vsphere.local"
var password = "TooSecret2C!"

func main() {
        fmt.Println("Creating a VIM/SOAP session.")
        vcURL := "https://" + username + ":" + password + "@" + hostname + "/sdk"
        u, err := url.Parse(vcURL)
        if err != nil {
                fmt.Printf("Error parsing url %s\n", vcURL)
                return
        }
        ctx, cancel := context.WithCancel(context.Background())
        defer cancel()
        vimClient, err := govmomi.NewClient(ctx, u, true)
        if err != nil {
                fmt.Printf("Error logging in: %s\n", err.Error())
                return
        }
        fmt.Println("Login successful")
        ...
```

At this point, your session is established and the session ID is stored in the `client` object. The session ID will be passed to the server with future requests.

Creating a Hello Folder with the VIM API Using govmomi

The `ServiceContent` object contains a reference to the root folder of the managed object hierarchy. You can invoke the `CreateFolder()` method on the remote `rootFolder` object to create a subfolder named `Hello Folder`.

```
        ...
        fmt.Println("Using VIM to create Hello Folder.")
        req := types.CreateFolder{
```

```
        This: vimClient.Client.ServiceContent.RootFolder,
        Name: "Hello Folder",
    }
    res, err := methods.CreateFolder(ctx, vimClient.Client, &req)
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println(res.Returnval.Type, "created")
    }
    helloFolder := res.Returnval
    fmt.Println(helloFolder)
    ...
```

Basic Authentication with the Automation API Using govmomi

To create an authenticated session with the Automation API, you can use code like the following.

```
    ...
    fmt.Println("Creating an Automation API session.")
    vapiClient := rest.NewClient(vimClient.Client)
    err = vapiClient.Login(ctx, url.UserPassword(username, password))
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println("VAPI session created.")
    ...
```

Tagging the Hello World Folder with the Automation API Using govmomi

To tag a folder, start by creating a tag category, then create a tag in that category, and finally associate the tag with the folder.

```
...
    fmt.Println("VAPI creating a tag category.")
    m := tags.NewManager(vapiClient)
    catId, err := m.CreateCategory(ctx, &tags.Category{
        AssociableTypes: []string{"Folder"},
        Cardinality:     "SINGLE",
        Description:     "example of tagging category from govmomi",
        Name:            "hello_category",
    })
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println("New category created: ", catId)

    fmt.Println("VAPI creating a tag in new category.")
    tagId, err := m.CreateTag(ctx, &tags.Tag{
         CategoryID:  catId,
         Description: "example of a tag from govmomi",
         Name:        "hello_tag",
    })
    if err != nil {
        fmt.Println(err)
        return
```

```
        }
        fmt.Println("New tag created: ", tagId)

        fmt.Println("VAPI creating association of tag with Hello Folder.")
        err = m.AttachTag(ctx, tagId, helloFolder)
        if err != nil {
            fmt.Println(err)
            return
        }
        fmt.Println("Tag attached to folder: ", tagId, helloFolder)
}
```

REST Access to vSphere APIs

This chapter shows how to run example programs by using the `vSphere Automation REST API` and the `Virtual Infrastructure JSON API`.

The example code used here is in the form of a `bash` shell script that invokes `curl` commands and captures the results in environment variables that can be used by subsequent `curl` commands. The example code creates a 'hello' folder and applies a 'hello' tag. The example is divided into sections for clarity, but you can copy and paste the sections into a single plain text file named `example.sh` and execute it with the command `bash example.sh` in the directory where you saved the file.

If you have access to a Unix or Linux development machine, you might need to use your Linux distribution's package manager to install the `jq` utility, which is used to parse JSON response bodies. If you have access to a Windows development machine, you can run the bash example by first installing the Cygwin utility, which provides a Unix-like shell, then installing the `jq` package that is available for Cygwin.

Installing Cygwin for Windows

To use this bash shell script example on Windows, install Cygwin and the jq packages:

1. Download and run the Cygwin setup executable.
2. Answer the installation configuration questions.
3. Select **Up to Date** from the **View** pull-down menu and click through to install the Cygwin base packages.
4. Run the Cygwin setup executable again.
5. Select **Full** from the **View** pull-down menu and search for `jq`.
6. For the `jq` and `libjq1` packages, change the value in the **New** column from `Skip` to the latest release, then click through to install the `jq` dependencies.

Basic Authentication to the VIM API

Starting with vSphere 8.0 Update 1, you can use the `Virtual Infrastructure JSON API` as a substitute for the SOAP-based `vSphere Management API`. The `Virtual Infrastructure JSON API` is an HTTP-based wire protocol that maps directly to the `vSphere Management API`. The `Virtual Infrastructure JSON API` follows a resource-based REST architecture with JavaScript Object Notation (JSON) requests and responses. You use the main CRUD (Create, Retrieve, Update, Delete) functions over HTTP to manage object representations.

1. To authenticate with the `Virtual Infrastructure JSON API`, you must first retrieve the `SessionManager` managed object ID by using the `Service Instance Get Content` operation.
   ```
   SESSION_MANAGER_MOID=$(curl -k https://$VC/sdk/vim25/8.0.1.0/ServiceInstance/ServiceInstance/content \
                            | jq .sessionManager.value \
                            | xargs -t)
   ```
2. Then, you use the `SessionManager Login` service to retrieve a session identifier for the `Virtual Infrastructure JSON API`.
   ```
   VC='vc1.example.com'
   UNAME='administrator@vsphere.local'
   PWORD='TooSecret2C!'
   ```

```
BODY='{"userName": "'"$UNAME"'", "password": "'"$PWORD"'"}'
VIM_SESSION_ID=$(curl -k -i "https://$VC/sdk/vim25/8.0.1.0/SessionManager/$SESSION_MANAGER_MOID/Login" \
                          -H 'Content-Type: application/json' \
                          -d "$BODY" \
                          | tr -d '\r' \
                          | awk '/vmware-api-session-id/ {print $2}' )
```

In the response headers, you receive the `vmware-api-session-id:<mySessionIdentifier>` key-value pair. Use it in the header of subsequent API requests for authentication.

Creating a Hello Folder with the VIM API Through REST

The `ServiceContent` object contains a reference to the root folder of the managed object hierarchy. You can retrieve that reference by using the `ServiceInstance Get Content` operation. Then you invoke the `CreateFolder()` method on the `rootFolder` object to create a subfolder named `Hello Folder`.

```
echo 'Using REST to create Hello Folder.'
BODY='{"name": "Hello Folder"}'
AUTH_H="vmware-api-session-id: $SESSION_ID"
HELLO_FOLDER_MOID=$(curl -kv \
    "https://$VC/sdk/vim25/8.0.1.0/Folder/$ROOT_FOLDER_MOID/CreateFolder" \
                        -H "$AUTH_H" -H "Content-Type: application/json" \
                        -d "$BODY" \
                        | jq .object.value \
                        | xargs -t)
echo HELLO_FOLDER_MOID=${HELLO_FOLDER_MOID}
```

Basic Authentication to the `vSphere Automation` API

To obtain a session identifier, you call the `Create Session` service with the base-64 encoded value of your vCenter Single Sign-On user name and password separated by a colon (`username:password`) in the authorization header.

```
auth=$(printf '%s:%s' "$UNAME" "$PWORD" | base64)
VAPI_SESSION_ID=$(curl -k -i -X POST -H "Authorization: Basic $auth" "https://$VC/api/session" \
                          | tr -d '\r' \
                          | awk '/vmware-api-session-id/ {print $2}' )
```

In the response, you receive the `vmware-api-session-id:<mySessionIdentifier>` key-value pair. Use it in the header of subsequent API calls for authentication.

Tagging the Hello World Folder with the Automation API Through REST

To tag a folder, start by creating a tag category, then create a tag in that category, and finally associate the tag with the folder.

1. Create a tag category.

   ```
   echo 'Using REST to create tag category.'
   BODY='{"name": "hello_category",
          "description": "A tag category for custom folders",
          "cardinality": "SINGLE",
          "associable_types": ["Folder"]}'
   CAT_ID=$(curl -k \
                 -X POST \
                 -H "vmware-api-session-id: $VAPI_SESSION_ID" \
                 -H "Content-Type: application/json" \
                 -d "$BODY" \
                 "https://$VC/api/cis/tagging/category" \
                 | xargs -t)
   ```

   The response contains the details of the newly created tag category, including its UUID.
2. Create a tag in that category. Use the tag category ID you created in the previous step.

```
echo 'Using REST to create tag.'
BODY='{"name": "hello_tag",
       "description": "example of a tag from Automation API",
       "category_id": "'${CAT_ID}'"}'
TAG_ID=$(curl -k \
              -X POST \
              -H "vmware-api-session-id: $VAPI_SESSION_ID" \
              -H "Content-Type: application/json" \
              -d "$BODY" \
              "https://$VC/api/cis/tagging/tag" \
              | xargs -t)
```

The response contains the details of the newly created tag, including its UUID.

3. Associate the tag with the *Hello World* folder. You must have the *Hello World* folder ID and the tag ID you created in the previous step.

```
echo 'Using REST to tag Hello Folder.'
BODY='{ "object_id": {"id": "'${HELLO_FOLDER_MOID}'", "type": "Folder"} }'
curl -k \
     -X POST \
     -H "vmware-api-session-id: $VAPI_SESSION_ID" \
     -H "Content-Type: application/json" \
     -d "$BODY" \
      "https://$VC/api/cis/tagging/tag-association/${TAG_ID}?action=attach"
```

On success, the operation returns HTTP Code 204 - No Content.

PowerCLI Access to vSphere APIs

This chapter shows how to run example scripts by using the PowerCLI SDK cmdlets that bind directly to the vSphere APIs.

You can use the following mechanisms to access the vSphere APIs with PowerCLI:

- For the `vSphere Management API`: use the `Get-View` cmdlet that allows you to manipulate VIM objects.
- For the `vSphere Automation API`: use the `vSphere Automation SDK for PowerShell` with its `Initialize-` and `Invoke-` cmdlets.

Using Get-View

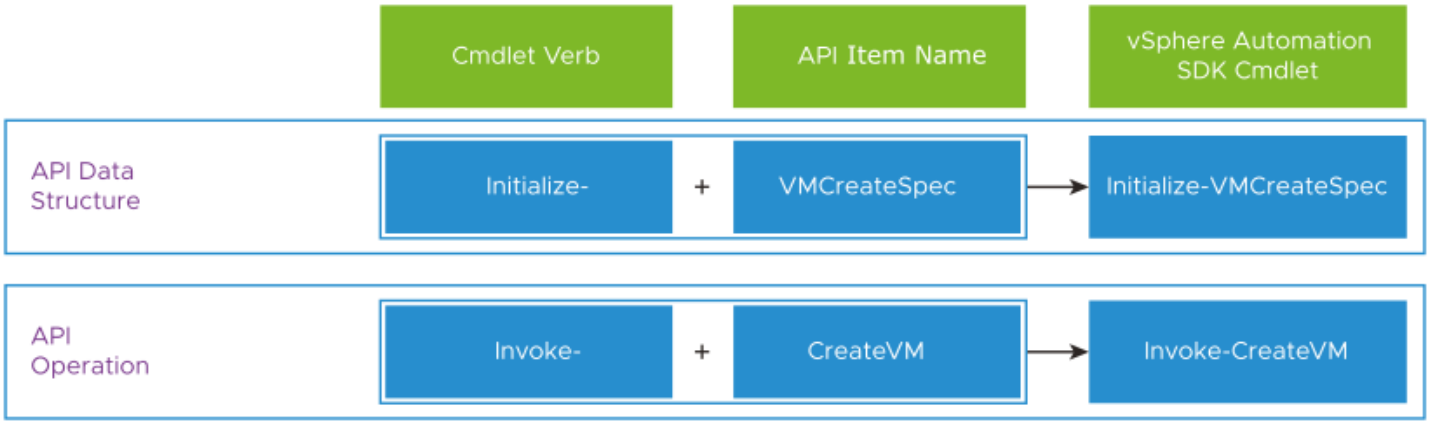
Use the Get-View cmdlet to view objects and call methods on the `vSphere Management API`. The Get-View cmdlet offers high performance by binding directly to the vSphere Management API.

Using the vSphere Automation SDK for PowerShell

Starting from VMware PowerCLI 12.4.0, you can use the `vSphere Automation SDK for PowerShell` to communicate with the `vSphere Automation API`.

You can construct `vSphere Automation SDK` cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the vSphere Automation API data structures or operations.

Figure 546: Constructing vSphere Automation SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the `vSphere Automation API` server.
- Use `Invoke-` cmdlets to execute the operations on the server side.

Installing Microsoft PowerShell and VMware PowerCLI

To access vSphere APIs through cmdlets, you must have Microsoft PowerShell and VMware PowerCLI installed on your machine.

1. Verify that PowerShell is available on your system. Windows users have PowerShell pre-installed, while Linux and macOS users must install it manually. For more information, see the PowerCLI Installation Guide.
2. To install PowerCLI, open PowerShell and run the command:
   ```
   Install-Module -NameVMware.PowerCLI
   ```

Basic Authentication to the VIM API and the vSphere Automation API

With PowerCLI, you connect with a single command to the `vSphere Management API` and the `vSphere Automation API`.

1. Run the `Connect-VIServer` cmdlet with your credentials and you create authentication sessions to both vSphere APIs automatically.
   ```
   Connect-VIServer -Server vc1.example.com -User 'MyUser' -Password 'MyPassword'
   ```

Creating a Hello Folder with the VIM API Using PowerCLI

Because the `CreateFolder` method is a part of the `vSphere Management API`, you must use the `Get-View` cmdlet.

1. Create a variable with the `ServiceInstance` object view.
   ```
   $serviceInstance = Get-View ServiceInstance
   ```
2. Create a variable with the `RootFolder` object view.
   ```
   $rootFolderView = Get-View -Id $serviceInstance.Content.RootFolder
   ```
3. To create your folder, invoke the `CreateFolder` method on the `RootFolder` view.
   ```
   $FolderMOID = $rootFolderView.CreateFolder('Hello Folder')
   ```

Tagging the Hello World Folder with the Automation API Through PowerCLI

The Tagging APIs are included in the `vSphere Automation API` as a part of the `CIS REST APIs` package (CIS stands for common infrastructure services). Therefore, you can use the `vSphere Automation SDK for PowerShell` to access the Tagging APIs with PowerCLI.

To tag a folder, start by creating a tag category, then create a tag in that category, and finally associate the tag with the folder. Each step in the tag creation and application process requires two PowerCLI verbs, both the `Initialize` and the `Invoke`.

1. Create a tag category.
```
$TaggingCategoryCreateSpec = Initialize-TaggingCategoryCreateSpec -Name "helloCategory" -Description "A
 tag category for custom folders" -Cardinality "SINGLE" -AssociableTypes "Folder"

$tagCategoryID = Invoke-CreateCategory -TaggingCategoryCreateSpec $TaggingCategoryCreateSpec
```
   The system returns the ID of the newly created tag category.
2. Create a tag in that category. You use the tag category ID from the previous step.
```
$TaggingTagCreateSpec = Initialize-TaggingTagCreateSpec -Name "helloTag" -Description "example of a tag
 from PowerCLI" -CategoryId $tagCategoryID

$tagID = Invoke-CreateTag -TaggingTagCreateSpec $TaggingTagCreateSpec
```
   The system returns the ID of the newly created tag.
3. Associate the tag with the *Hello World* folder. You use the *Hello World* folder ID and the tag ID that you created in the previous step.
```
$FolderID = Initialize-StdDynamicID -Type "Folder" -Id $FolderMOID.value

$TaggingTagAssociationAttachRequestBody = Initialize-TaggingTagAssociationAttachRequestBody -ObjectId
 $FolderID

Invoke-AttachTagIdTagAssociation -TagId $tagID -TaggingTagAssociationAttachRequestBody
 $TaggingTagAssociationAttachRequestBody
```

You can now see the *Hello World* folder with the tag and category in the vSphere Client.

Some Use Cases for VMware SDKs

VMware provides several stand-alone SDKs for specific use cases, including disk development for backup and restore, interfacing with guest virtual machines, and developing feature plug-ins for data center management with the vSphere Client.

Partners and customers can develop backup applications using the virtual disk development kit (VDDK). A code walk-though of the Safekeeping application appears below.

The Guest SDK enables operations and communication with guest virtual machines. An example use of Datasets appears below, to notify users of data synchronization status.

The vSphere Client SDK is for partners and customers creating a (remote) plug-in extension for the vSphere Client. This is often recommended or required for third party extensions to vSphere or VCF. See *Developing Plug-ins with vSphere Client SDK* for details.

Virtual Disk Development Kit

This SDK is most frequently used to implement backup and restore applications. This section is a walk-though of code to restore a virtual machine.

The Virtual Disk Development Kit (VDDK) is a set of libraries and utilities that enable software developers to create applications that can access and manipulate virtual disks used for storage by products such as vSphere, Workstation, and Fusion. VDDK is a valuable tool for developers of storage and backup applications.

VDDK provides APIs for backup and recovery, cloning, and other operations on virtual disks. With VDDK, developers can create applications that perform backup and restore operations for virtual machines, as well as applications that create clones or snapshots of virtual disks. VDDK also includes utilities for converting virtual disks between different formats and for creating virtual disks from scratch.

Safekeeping – a VDDK Use Case

Based on VDDK, Safekeeping is a "proof of concept" application to demonstrate backup and restore of vSphere VMs, vApps, and first-class disk. You can find its source code on Github, as linked from the developer.broadcom.com SDK

landing page for VDDK. This section focuses on `RestoreVm.java`, a source code module that recovers a VM previously saved by the `BackupVm.java` module.

```
safekeeping backup vm:myTestVM
safekeeping restore vm:myTestVM
```

After package and interface imports, class `RestoreVm` extends method `AbstractRestoreFcoWithDisk`, which in turn extends method `AbstractRestoreFco` for restoring a first class object (FCO) but with disk, as in a vSphere VM. (A vApp is an FCO that includes VMs but not disk.) `RestoreVm` must be passed a VIM connection, user-specified command options, and a log file.

The `checkOptions` method parses passed-in options, including (if specified) VM name, parent vApp, data center, resource pool, datastore, and storage profile. If the destination VM folder cannot be restored to, `RestoreVM` uses a different location.

Snapshot is a typical operation for both backup and restore. The `createSnapshot` method takes a vSphere snapshot so the VM can continue running while being restored, and also to quiesce the virtual disk while it is being replaced. If a snapshot is unnecessary because the VM does not exist, snapshot is skipped.

If the VM is being restored because it no longer runs, the `destroy` method deletes the old VM so it does not conflict with the VM to be restored.

The `disksMapping` method is one of the most complicated in `RestoreVM`. It needs to know the number of disks to restore, the storage profile of disks, and the location of disk data in the backup. This information comes from the `managedInfo` object, combined with the generated `profile` object.

Once virtual disks are correctly mapped, they are restored by the `disksRestore` method. This could take a lot of time, so the `final` loop blocks until disks are restored.

If a failed VM was not deleted, the `manageExistingFco` method deals with naming conflicts between the pre-existing VM and the VM to be restored. The VM might be part of a vApp, which allows duplicate names. If the pre-existing VM is powered on, the backed-up VM cannot be restored. With duplicate names, the user can be given a chance to rename VM to be restored.

The `networksMapping` method determines what networks the backed-up VM used for each virtual NIC, and attempts to reformulate them using existing networks.

Finally the `powerOn` method starts up the restored VM so it can be configured. A VM can be powered on before the Guest OS is ready to run.

The `reconfig` method configures the VM based on profiles of its virtual disks and networks determined above. Configuration may be interrupted by a fault. Similar `reconfigDisk` and `reconfigNetwork` methods take care of disk backing and network backing for the VM to be restored. The `removeSnapshot` methods cleans up the snapshot previously created.

Finally the `restore` method, the main restore function, puts this all together. It retrieves the restore profile and calls snapshot, start-access, `disksRestore`, and end-access in an if statement. This is where the C++ functions in VixDiskLib get called (start and end access prevent vMotion during restore).

```
if (createSnapshot(rar) && startVddkAccess(rar, jvddk) && disksRestore(rar, jvddk)
                        && endVddkAccess(rar, jvddk)) {
   final String st = rar.getFcoToString() + " Restore Success - start cleaning";
}
```

The restored VM could either be newly created with the `restoreByCreate` method, or recovered with import of virtual disk data by the `restoreByImport` method. It is the `restoreMetadata` method that decides which. The `restoreManagedInfo` method takes care of items managed by vCenter, such as folder groupings and resource pools.

Guest SDK and Datasets

The Datasets facility in the Guest SDK makes it possible to communicate to and from a guest virtual machine.

Suppose that a data preservation service offers synchronization so that user folders on a guest virtual machine are copied to backup media. The service can create a `syncStatus` Dataset on each participating virtual machine, to provide the date and time of last synchronization.

```
POST https://{VC.IP}/api/vcenter/vm/{vm-ID}/data-sets
# request body specifies Dataset name = syncStatus
```

After running `rsync` or similar application, synchronization service writes the date and time (syncDataTime) of the successful synchronization into the Dataset on virtual machine *`vm-ID`*.

```
PUT https://{VC.IP}/api/vcenter/vm/{vm-ID}/data-sets/syncStatus/entries/{key}
# request body specifies actual data and time "value" for key = syncDateTime
```

If users are interested in knowing synchronization status, they can run the following command on their local machine.

```
vmtoolsd --cmd 'datasets-get-entry {"keys":["syncDateTime}"], "dataset":"syncStatus"}'
```

VMware Cloud Foundation SDKs Developer's Setup Guide

The VMware Cloud Foundation (VCF) SDKs help you develop management and automation applications for VMware Cloud Foundation .

The VCF SDK includes components necessary to work with VMware Cloud Foundation APIs, including libraries, sample code, and documentation. This guide covers SDK installation and setup.

Intended Audience

This guide is intended for anyone who wants to develop applications using the VMware Cloud Foundation SDKs. Such developers typically include software programmers creating client applications using Java or Python and targeting vSphere and data center services.

Revision History

Date	Description
February 2026	For VMware Cloud Foundation 9.1. Minor updates and corrections.
17 June 2025	For VMware Cloud Foundation 9.0. New revision, covering setup for vSphere Management, vSphere Automation, and software defined data center management.
September 2023	For vSphere 8.0 Update 3. The last of previous versions for vSphere Web Services.

What to Know Before Using the VMware Cloud Foundation SDKs

Application development with the VMware Cloud Foundation SDKs requires programming knowledge, typically using Java or Python. You should understand the following:

- Virtual Infrastructure Management (VIM) is a protocol for VMware's enterprise-targeted server products, ESX and vCenter. Originated in the early 2000s, and intended for languages such as Java with Web Services methods, it is also called the Virtualization Management Object Definition Language (VMODL).
- The vSphere Automation API (vAPI) is a suite of tools to build and test micro services in vSphere data centers. Originated in 2015, it was designed to provide interfaces in multiple programming languages. Python and REST interfaces proved to be most popular.
- VIM protocol communicates over the network with XML data encapsulated as SOAP, whereas vAPI communicates over the network with JSON data. Both VIM and vAPI employ HTTP or preferably HTTPS connections.
- In vSphere 8.x, VIM was extended to allow JSON data communications, in an effort to unify the two APIs and simplify development of automation and management applications.
- In VMware Cloud Foundation 9.0, additional APIs are included for software defined data center (SDDC) management.

Programming Language Support in the VMware Cloud Foundation SDKs

Python and Java are supported with libraries and sample code. REST interfaces for data center automation are provided as well.

The VCF SDKs support:

- Java developers
- Python developers
- OpenAPI specifications
- SOAP for legacy uses

Language and Toolkit	Java	Python	OpenAPI
Development environment	OpenJDK or JDK 11, JDK 17, and JDK 21	Python 3.9 or later	REST
Web Services and/or open source samples	Maven central repository	PyPI (Python Package Index)	OpenAPI 3.0.1

Applications You Can Build with the VMware Cloud Foundation SDKs

With the VMware Cloud Foundation SDKs, you can create client applications to manage and automate your VMware Cloud Foundation installation, SDDC Manager, and vCenter instance workflows.

The following are some tasks that you could automate using the VMware Cloud Foundation SDK.

- Install VMware Cloud Foundation and manage it with the SDDC Manager API, SDDC Manager UI, or Cloud Foundation UI.
- Create, configure, power cycle, or suspend virtual machines.
- Create, configure, and manage virtual devices, such as virtual disks and network devices.
- Connect, power cycle, and disconnect ESX hosts.
- Snapshot the state of a virtual machine, and later restore virtual machine state.
- Gather statistics about ESX hosts and virtual machine performance.
- Manage events generated by vSphere, such as alarms set for specific thresholds.
- Migrate virtual machines between hosts automatically.
- Manage load balancing and failover with vCenter using distributed resource scheduler (DRS) and high availability (HA).

The following are additional tasks that you can automate using the vAPI.

- Manage the life cycle of hosts and clusters.
- Access the Content Library to access OVF packages and VM templates.
- Create a vSphere tag to add object metadata in the vSphere inventory.
- Configure the workload control plane to run Kubernetes microservices.

The vSphere Client is an example of a client application that was developed using VIM and vAPI. Many controls in the vSphere Client UI call underlying vSphere API methods.

Scope of Changes from vSphere to VMware Cloud Foundation SDK

The VMware Cloud Foundation SDK supersedes both the vSphere Management SDK and the vSphere Automation SDKs. A summary of the components follows below.

The vSphere Management SDK contained the following interfaces. All are Web Services, in the sense that they communicate through a network endpoint in XML as defined by their Web Services Description Language (WSDL) definitions. As of vSphere 8.x, applications can also communicate in JSON, for interoperability with vSphere Automation APIs.

- ESX Agent Manager (EAM) SDK

- Storage Management Service (SMS) SDK
- Storage Policy Based Management (SPBM) SDK
- Single Sign On (SSO) Client SDK
- vSphere Web Services SDK, also called VIM
- Virtual Storage Lifecycle Management (VSLM) SDK

The vSAN Management SDK, similar to the vSphere Web Services SDK but adding storage, is included in the VMware Cloud Foundation SDK. The vSphere Automation SDK, also called vAPI, is also included in the VMware Cloud Foundation SDK. Most developers use REST or Python bindings for vSphere Automation, although Java and other language bindings are offered. The VMware Cloud Foundation SDKs includes libraries from the following:

- vSphere Automation SDK for Python and Java
- pyvmomi (Python VM object management infrastructure) for vCenter and vSAN
- Web Services SDK for Java
- vSAN Management SDK for Java

Note:

The VCF SDKs also include new interfaces for SDDC Manager and VCF Installer. SDDC Manager is a user interface to manage a software defined data center. VCF Installer helps automate cluster setup.

Consolidated Contents of the VCF SDK

Various interfaces were bundled inside the vSphere Management SDK, including the Web Services SDK, which remains available under folder `sdk/vim25` (instead of vsphere-ws). Formerly available mostly on Github, vSphere Automation SDKs are now bundled into the VCF SDK.

The vSphere Automation SDKs were originally provided on Github, but are now downloadable from the Broadcom developer site. Zip files for Java and Python are available for download, however code repositories and release notes still reside on Github.

The vSphere Management SDK contained the following packaged interfaces. All are Web Services, in the sense that they communicate through a network endpoint in XML as defined by the Web Services Description Language (WSDL) standard. In vSphere 8 update releases, applications can communicate in JSON also, which increases interoperability with vSphere Automation APIs.

- The vSphere Web Services SDK is the most popular. It supports development of applications that call the vSphere API to manage virtual machines and virtual infrastructure components such as data centers, datastores, resource pools, and networks. See the *vSphere Web Services API Reference* for managed objects, methods, and data structures in each release.
- ESX Agent Manager (EAM) offers support for vSphere solutions such as High Availability, NSX virtual networks, and IO Filters.
- Storage Management Service (SMS) provides access to storage capabilities, associations, and space usage. SMS has its own in-memory database manager (the SMS cache) that periodically synchronizes data with the storage information provider database
- Storage Policy Based Management (SPBM) helps administrators use and define storage profiles that automate storage provisioning for virtual machines. Client applications can manipulate storage policies with the API, and with the vSphere Client.
- Single Sign On (SSO) provides a security service for vSphere authentication. Client applications call the API to obtain Security Assertion Markup Language (SAML) tokens for logging into vCenter. For multifactor authentication, programs must use vSphere Automation.
- Virtual Storage Lifecycle Management (VSLM) are APIs to manage First Class Disk (FCD), also called Improved Virtual Disk. FCD is storage that exists independently of a virtual machine.

The vSAN Management API is a superset of the Web Services API, with additional methods and objects for handling vSphere storage area network (vSAN). Java and Python language bindings are available and recommended.

How to Access or Download the VCF SDK

The VCF SDK is provided as open source on the web. If you are a developer working inside a firewall on a limited access network, or prefer Zip format, you can download equivalent Zip files.

Java developers can obtain the VCF SDK on the Maven Central repository. POM files are provided for build environments Maven, Gradle, Kotlin, and others. The role of Maven Central is to ensure that dependencies are downloaded in a standard way for the build process. The SDK itself is located in Github, or can be downloaded from the Broadcom developer website.

```
https://central.sonatype.com/artifact/com.vmware.sdk/vcf-sdk-bom
https://github.com/vmware/vcf-sdk-java
```

Python developers can obtain the VCF SDK from the Python package index (PyPI). The role of PyPI is to ensure that dependencies are downloaded in a standard way for the build process. The SDK itself is located in Github, or can be downloaded from the Broadcom developer website.

```
https://pypi.org/project/vcf-sdk/
https://github.com/vmware/vcf-sdk-python
```

OpenAPI for REST operations and WSDL definitions for Web Services are available on Github.

```
https://github.com/vmware/vcf-api-specs
```

Alternatively, you can download the VCF SDKs from the Broadcom developer site. If you don't see VCF, click the SDKs link for Cloud Management.

```
https://developer.broadcom.com/sdks
```

Sample Authentication with vCenter Single Sign-On Configurations

You can choose to authenticate your client with vCenter Single Sign-On (SS0). This topic explains how to do vCenter SSO authentication with the SDK samples.

Currently, VCF install vCenter with an embedded Platform Services Controller, which includes vCenter Single Sign-On Server, VMware License Service, and other features. This configuration combines all platform services into one vCenter installation.

With this configuration, there is no need to provide the vCenter Single Sign-On URL (because vCenter SSO service is embedded in the management node) and the SDK will continue to work as before.

The SDK samples have always had the option to explicitly specify the vCenter Single Sign-On URL. This was useful in cases where the vCenter SSO service was deployed outside the vSphere management node.

When you log in to a vCenter SSO service, you must be in a domain that has been added as a vCenter SSO identity source. If that domain is not the default domain, you must include the domain name as part of your user name, for example *`administrator@vsphere.local`*. To learn how to configure vCenter SSO service, see the *vSphere Security* documentation.

SDK Version Support and Limitations of the ESX API

VMware supports older versions of vSphere, although some features of VMC are not backward compatible, such as SDDC Manager. ESX hosts have some limitations that vCenter does not.

The VCF SDK and vSphere Web Services support various versions of vSphere, including:

- ESX versions 9.x, 8.0 and update releases
- vCenter 9.x, 8.0 and update releases

For older releases of ESX and vCenter, VMware recommends using an older matching SDK.

The vSphere API is a Web Service that runs on vCenter and ESX hosts. The API exposed is the same in both products. However, vCenter provides the following capabilities that are not available through ESX Web Service:

- Collecting historical performance data

- Optimizing resources, including managing distributed resources
- Enabling migration from one host system to another by using VMware vMotion
- Providing distributed resource management, including recovery, across all host systems under its control

If you attempt to invoke an operation on an ESX system that is supported only on vCenter, the server returns a fault saying “not implemented” or “not supported.” For example, the ExtensionManager API is available only on vCenter. Attempting to register an extension to an ESX host returns a fault, “not supported.”

Setup for Java Development

This chapter explains how to set up an environment to develop VCF applications using Java.

Java Development Requirements for the VMware Cloud Foundation SDK

To develop Java client applications using the VMware Cloud Foundation Java SDK, you need Java SE and a Java Web services development tool set.

The minimum Java requirement is to use Java platform Standard Edition (SE) version 11. For best results use one of the Long-Term-Support (LTS) Java SE releases 11, 17 or 21.

Setup For Python Development

This chapter explains how to set up an environment to develop VCF applications using Python.

The VCF SDK for Python supports both VMware Cloud Foundation (VCF) and its subset VMware vSphere Foundation (vSphere Foundation). The VCF SDK supersedes vSphere Management SDK with pyvmomi, vSphere Automation SDK for Python, vSAN SDK for Python, and vSphere Data Protection SDK for Python. It also contains interfaces for VCF Installer and SDDC Manager.

Previously, Python libraries were shipped with the SDKs. Starting in VCF 9.0, these libraries are published to PyPI (https://pypi.org) and must be installed locally for the SDK samples to work.

> **Note:**
>
> If you operate in an air-gapped environment off the Internet, you can download equivalent Python libraries from the Broadcom developer site and install them locally.

Hierarchy of SDK components:

```
vcf-sdk
   vcf-installer
   vmware-sddc-manager
   vvf-sdk
      pyvmomi
      vmware-vsan-data-protection
      vmware-vcenter
         vmware-vapi-runtime
         vmware-vapi-common-client
```

The last two components, vAPI runtime and common-client, get installed transitively by installing any bindings other than `pyvmomi`.

Prerequisites for Python SDK

Before downloading the VCF SDK for Python, check that your Linux, Mac, or Windows system has the following items.

- `python` (version 3.9 through 3.13)
- `pip` (version 24.2 or later)
- `openssl` (3.0 or later)
- Recommended: editor such as Visual Studio Code or PyCharm

You must create an SSH key for secure connections to Github.

```
https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account
```

A Python virtual environment is highly recommended, although it is possible to work with the samples without one.

Installing the Python SDK

The easiest way to install the Python SDK is with the Python package index (PyPI) over the network, but you can also install it in air-gapped environments.

The typical and recommended way to install the VCF SDK for Python is using `pip`. By default, `pip` install *`package`* fetches the latest version of a package from the Python Package Index (PyPI). The installation also gets its dependencies from the PyPI artifactory. The vSphere Foundation SDK is a subset of the VCF SDK for vSphere-only development. Using one of these commands, you can install the VCF or vSphere Foundation SDK from PyPI:

```
pip install vcf-sdk
pip install vvf-sdk
```

Fastpath:

Unless you have an air-gapped development environment, stop here.

Installation in an air-gapped environment is more involved, but helped by a requirements file. The Zip file includes only the Python SDK deliverables. Third-party dependencies must be installed before installing the SDK from the Zip. In this example, third-party dependencies are in a third party requirements.txt file. On a system that is connected to the Internet, download third-party libraries from PyPI, and Zip them up.

```
pip download -r requirements_third_party.txt -d third_party_libs_pypi
zip -r third_party_libs_pypi.zip third_party_libs_pypi/
```

After moving that system into the air-gapped environment, unzip the downloaded third-party library dependencies, and install from Wheel files:

```
unzip third_party_libs_pypi.zip
pip install -U third_party_libs_pypi/**/*.whl
```

Because the downloaded VCF SDK contains compressed folders, you must unzip again:

```
unzip v9.0.0.0.zip -d vmware-cloud-foundation-sdk-python
cd vmware-cloud-foundation-sdk-python
```

Now you can install the VCF or vSphere Foundation SDK, as above from PiPI over the network.

```
pip install -U lib/**/*.whl
pip install -U lib/vvf/**/*.whl
```

Running the SDK Samples

After you install the VCF SDK for Python, and have a VMware Cloud or vSphere installation, you can run the sample programs and watch them work.

The VCF SDK for Python samples are distributed across three different Github repositories. The first one, VCF SDK for Python, must be cloned. The remaining repositories can be fetched as submodules.

- vcf-sdk-python
- vsphere-automation-sdk-python
- pyvmomi-community-samples

After you clone the SDK samples, you can set up a Python environment to run them.

1. Clone the SDK to get all samples in one place.
   ```
   git clone --recurse-submodules https://github.com/vmware/vcf-sdk-python.git
   ```

```
git submodule init
git submodule update
```

2. Update PYTHONPATH to use SDK samples and helper methods. On MacOS or Linux:

```
cd vcf-sdk-python
export PYTHONPATH=${PWD}:$PYTHONPATH
```

   On Windows:

```
cd vcf-sdk-python
set PYTHONPATH=%cd%\sdk;%PYTHONPATH%
```

3. Using the VCF SDK repository, connect to SDDC Manager for samples execution. (If you have vSphere Foundation instead of VCF, skip this step.) For demo purposes, this code uses password authentication and disables certificate verification, which is not recommended in a production environment.

```
import requests
import urllib3
from vmware.vapi.sddc_manager.client import create_sddc_manager_client
session = requests.session()
session.verify = False
sddc_client = create_sddc_manager_client(server='<sddc_manager_ip>',
   username='<username>', password='<password>', session=session)
sddc_client.v1.Domains.get_domains()
```

4. Using the VCF SDK repository, connect to vCenter for samples execution. See remark about "demo purposes" above.

```
import requests
import urllib3
from vmware.vapi.vsphere.client import create_vsphere_client
session = requests.session()
session.verify = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
vsphere_client = create_vsphere_client(server='<vc_ip>',
  username='<vc_username>', password='<vc_password>', session=session)
vsphere_client.vcenter.VM.list()
```

5. Run the samples against SDDC Manager.

```
$ python create_domains.py --server <SDDC_Manager_IP> --user <username> --password <password>
```

6. Run the samples against vCenter.

```
$ python create_domains.py --server <vCenterServer_IP> --user <username> --password <password>
```

Overview of Python Sample Programs

The Python sample programs include previous content from vSphere, including Web Services SDK sample, vSphere Automation (vAPI) samples and `pyvmomi` community samples. New sample programs were added for VCF installer and SDDC Manager.

Sample program folders in the SDK all contain README.md files, which document requirements, compatibility, how to run the code, and what the samples are all about. Also, the DocString in each Python sample program explains what the sample accomplishes, and pre-requisites to run it.

Folder with Python Sample Programs	Description of Contents
vcf-installer-samples	Installing VCF or upgrading vSphere to VCF
sddc-manager-samples	Programs to automate SDDC Manager functions
vsphere-samples/pyvmomi-community-samples	Demonstrate combined use of vAPI and VIM, and previously, managervSphere with Python
vsphere-samples/vsphere-automation-sdk	Appliances, Compute Policy, Content library, Tagging, vCenter, SSO
vsphere-samples/*/vmware/sdk/vim25	VIM = VMware infrastructure manager for JSON
vsphere-samples/*/vmware/sdk/pbm	(S)PBM = storage policy based management

Folder with Python Sample Programs	Description of Contents
vsphere-samples/*/vmware/sdk/sms	SMS = storage management service
vsphere-samples/*/vmware/sdk/ssoclient	vCenter SSO (single sign on)
vsphere-samples/*/vmware/sdk/vslm	VSLM = virtual storage lifecycle management
vsphere-samples/*/vsan/snapservice	Control vSAN ESA snapshot service with Python

Setup for Development with OpenAPI

This chapter explains where to locate OpenAPI specifications for VMware Cloud Foundation interfaces, and how to set up an environment to develop VCF applications based on OpenAPI.

VCF 9.0 provides officially supported ready-to-use component modules for Java and Python developers, along with guidelines and code samples, available at Maven Central and PyPI respectively. For further development especially in alternate languages, documentation and specifications might be needed.

Developers who use other programming languages can use API definitions under the specifications folder to manually generate client code and server stubs in many languages. For example, Go language programmers could run `openapi-generator` (or an alternative also available on Github) to produce a Go client library and skeleton Go server, to be filled in with back-end Go language code. Actually Java and Python programmers could do this, but don't need to.

You can find specifications to browse at github.com/vmware/vcf-api-specs, or to download as a Zip archive on the Broadcom developer website. The Github repository holds API specifications and documentation for VCF products, replicated in the Zip file for developers working off-network.

The VCF 9.0 release contains JSON files conforming to the OpenAPI 3.0.1 standard. The following table summarizes content of the repository.

documentation/vsphere/wsdl	Zip files containing Web Service style API references for EAM, PBM, SMS, VIM, vSAN, and VSLM.
scripts/java	Folders containing unsupported Gradle 8.7-based projects that you can use to generate client-side bindings. Intended for backward compatibility with Java 8 now that vSAN and WS-SDK moved to Java 11.
specifications/sddc-manager	JSON file with OpenAPI specification for the SDDC Manager, for fleet management.
specifications/vcf-installer	JSON file with OpenAPI specification for the VCF Installer, formerly Cloud Builder.
specifications/vsan-data-protection	YAML file with OpenAPI specification for the Snap Service, a vSAN feature new in 8.0 U3.
specifications/vsphere/openapi/automation	YAML file with OpenAPI specification for vSphere Automation, also called vAPI.
specifications/vsphere/openapi/vi-json	YAML file with OpenAPI specifications for all the Web Service APIs enumerated above for WSDL, including the popular VIM (vim25).

These specifications are available:

- vSphere
  - VI/JSON
  - Automation - file dependencies, e.g. vCenter all depend on vAPI
  - vSAN data protection (snapservice)
  - Legacy use cases, before WSDL was superseded by VI/JSON
    - JAX-WS / CXF generator / wsimport

  - CXF 4.0.5
  - Old SDKs used JAX-WS RI 3.x
- SDDC Manager
- VCF Installer

OpenAPI Specifications for vSphere

Developers can use the OpenAPI specifications for vSphere to manually generate client code and server stubs in alternate programming languages.

Under the `specifications/vsphere` folder reside two YAML files containing OpenAPI definitions:

First, the VI/JSON file describes VIM (virtual infrastructure management, also called vim25) interfaces with JSON instead of XML data transfer. This is a large file with all the former Web Services API categories, including EAM, PBM, SMS, SSO, VIM, and VSLM.

Second, the vCenter YAML file describes vSphere Automation, also called vAPI. This is a more limited API focused on the vCenter appliance, content library, tagging, and ESX host management.

Closely related, the vSAN data protection YAML file describes Snap Service, which provides support for vSAN native snapshots.

Also for backward compatibility with legacy use cases, before WSDL was superseded by VI/JSON, the `wsdl` folder contains per-service definition files, XSD and WSDL. The previous VIM interfaces used JAX-WS RI 3.x and JAX-B libraries. For code creation based on these definition files, you can use the CXF 4.0.5 generator, and/or the `wsimport` command.

OpenAPI for VCF Installer

New in VCF 9.0, the VCF Installer transforms an existing vSphere data center into a full VMware Cloud Foundation installation. VCF Installer is similar to VMware Cloud Builder in previous releases.

Supported VCF features will eventually include usage meter, operations, fleet management, operations for logs, operations for networks, diagnostics, HCX, identity broker, VCF Automation, provider management, orchestrator, Kubernetes service, and data services manager.

Here are the operations, listed as REST endpoints, in the initial release.

```
/v1/system/settings/depot
/v1/tokens
/v1/sddcs
/v1/sddcs/vcfops-discovery
/v1/sddcs/vcenter-discovery
/v1/sddcs/validations
/v1/sddcs/network-config-profiles
/v1/sddcs/installer-mode
/v1/sddc-manager/trusted-certificates
/v1/bundles
/v1/tokens/access-token/refresh
/v1/tasks/{id}
/v1/system
/v1/system/settings/depot/depot-sync-info
/v1/system/proxy-configuration
/v1/system/ceip
/v1/sddcs/{id}
/v1/bundles/{id}
/v1/vcf-services
/v1/vcf-services/{id}
/v1/tasks
/v1/system/vcf-management-components
```

```
/v1/system/appliance-info
/v1/sddcs/{id}/spec
/v1/sddcs/validations/{id}
/v1/sddcs/validations/latest
/v1/sddcs/latest
/v1/releases
/v1/releases/{sku}/release-components
/v1/releases/system
/v1/releases/domains/{domainId}/future-releases
/v1/releases/domains/{domainId}/custom-patches
/v1/releases/custom-patches
/v1/bundles/download-status
/v1/bundles/domains/{id}
/v1/tokens/refresh-token
/v1/sddc-manager/trusted-certificates/{alias}
```

OpenAPI for SDDC Manager

SDDC Manager, also called fleet management, is a virtual appliance that helps datacenter administrators deploy, manage, and operate their private cloud. It simplifies the process of provisioning new infrastructure, managing passwords, installing certificates, and patching or upgrading infrastructure.

Currently there are about 280 interfaces in the SDDC Manager API, also available as REST commands.

Sample Programs for VCF

Java and Python sample programs are available for use cases involving VCF Installer and SDDC Manager. Topics below compare various use cases implemented for these two languages.

VCF Installer is similar to an older program called Cloud Builder. SDDC Manager provides fleet management capabilities for the underlying virtual infrastructure.

The Broadcom developer site offers extensive reference documentation for the VCF APIs at this address:

developer.broadcom.com/xapis/vmware-cloud-foundation-api/latest

Samples for VCF Installer

The VCF Github projects for Java and Python contain comparable sample programs under `vcf-installer-samples`. These are available in Zip files for download, or at the following locations:

```
https://github-vcf.devops.broadcom.net/vcf/vcf-sdk-java/
https://github-vcf.devops.broadcom.net/vcf/vcf-sdk-python/
```

Table 1849: Comparison of sample VCF Installer programs

Name of Java sample	Name of Python sample
GetApplianceInfo	get_appliance_info
DownloadBundlesVcfFleetFirstVcfInstance	download_bundles_vcf_fleet_first_vcf_instance
DownloadBundlesVcfFleetFirstVcfInstanceFromExistingComponents	download_bundles_vcf_fleet_first_vcf_instance_from_existing_components
DownloadBundlesVvfInstance	download_bundles_vvf_instance
DownloadBundlesVvfInstanceExistingVcenter	download_bundles_vvf_instance_existing_vcenter
DownloadBundlesVvfInstanceExistingVcfOps	download_bundles_vvf_instance_existing_vcf_ops
DownloadBundlesExtendVcfFleetWithVcfInstance	download_bundles_extend_vcf_fleet_with_vcf_instance
DeployVcfFleetFirstVcfInstance	deploy_vcf_fleet_first_vcf_instance

Name of Java sample	Name of Python sample
DeployVcfFleetFirstVcfInstanceFromExistingComponents	deploy_vcf_fleet_first_vcf_instance_from_existing_components
DeployVvfInstance	deploy_vvf_instance
DeployVvfInstanceFromExistingComponents	deploy_vvf_instance_from_existing_components
ExtendVcfFleetWithVcfInstance	extend_vcf_fleet_with_vcf_instance
ExtendVcfFleetWithVcfInstanceFromExistingComponents	extend_vcf_fleet_with_vcf_instance_with_existing_components

Samples for SDDC Manager

The VCF Github projects for Java and Python contain comparable sample programs under `sddc-manager-samples`. These are available in Zip files for download, or in the following locations:

```
https://github-vcf.devops.broadcom.net/vcf/vcf-sdk-java/
https://github-vcf.devops.broadcom.net/vcf/vcf-sdk-python/
```

Table 1850: Comparison of sample SDDC Manager programs

Name of Java sample	Name of Python Sample
ApiClientUtil	client
AddHostExample	add_host_to_cluster
RemoveHostExample	cluster_compaction
AddClusterExample	get_cluster
DeleteClusterExample	create_and_delete_cluster
CreateDomainExample	get_domains_info
DeleteDomainExample	create_and_delete_domain
SddcManagerHelper	helper ssl_helper
HostCommissionExample	hosts_commission
HostDecommissionExample	hosts_decommission
CreateNetworkPoolExample	
TaskHelper (track task list)	sddc_manager_filter
TaskStatus (of specific task)	
	get_licensing_info
	get_aria_suite_lifecycle
SddcUtil (factory class)	
StringUtil (object to JSON)	

VMware Cloud Foundation Programming Guide

VMware Cloud Foundation APIs and SDKs are designed to help manage and automate various aspects of your cloud infrastructure.

Introduction to the VMware Cloud Foundation REST API

Use the VMware Cloud Foundation REST API to manage your virtual infrastructure by using HTTP.

With the VMware Cloud Foundation REST API, you can manage vSphere components connected to authentication, tagging, content libraries, clusters, internal and external certificates, namespaces, vSphere Trust Authority, vSphere Supervisor, and more.

This programming guide introduces the most important features and use cases of the VMware Cloud Foundation REST API. You can use this guide in combination with the *VMware Cloud Foundation API Reference* that features all available API services, as well as many code samples.

Components of the Virtualization Layer

At the core of VMware Cloud Foundation is vSphere, which provides the virtualization layer of the software-defined data center. You can use vSphere deployment options for vCenter and ESX hosts to build virtual environments of different scales.

Components and Services of the vSphere Environment

vCenter is a preconfigured virtual machine optimized for running the vCenter service and the vCenter components. The vCenter service acts as a central administrator for ESX hosts.

Components Installed with vCenter

vCenter is a central administration point for ESX hosts. The group of components installed when you install vCenter include the vCenter service, vSphere Client, VMware vSphere®Auto Deploy™, VMware vSphere®ESX™ Dump Collector, VMware vSphere® Syslog Collector, and vSphere Lifecycle Manager service.

You can use the VMware Cloud Foundation API endpoint to access the following services running on vCenter.

Content Library	You can use content libraries to share virtual machines, vApps, and other files, such as ISO, OVA, and text files, across the software-defined data center. You can create, share, and subscribe to content libraries on the same vCenter instance or on a remote instance. Sharing content libraries promotes consistency, compliance, efficiency, and automation in deploying workloads at scale. You can also create OVF and VM templates from virtual machines and vApps in hosts, resource pools, and clusters. You can then use the OVF and VM templates to deploy new virtual machines and vApps. Starting with vSphere 7.0, you can edit the contents of a VM template. You can check out the library item that contains the VM template. After editing the VM template, check in the library item to save the changes to the virtual machine.
Virtual Machine	You can use the VMware Cloud Foundation APIs to create, configure, and manage the life cycle of virtual machines in your environment. Starting with vSphere 7.0, you can also clone, create an instant clone, migrate, register, and unregister a virtual machine.
vSphere Lifecycle Manager	The life cycle of ESX hosts and clusters can be managed through the VMware vSphere®Lifecycle Manager™ feature. Based on the current state of the hosts in a cluster, you can easily create a desired software specification by using the contents of a software depot. Then you validate the desired software specification and you apply the specification on all hosts in the cluster.
vSphere Tags	With vSphere tags you can attach metadata to vSphere objects, and as a result, make it easier to filter and sort these objects. You can use the VMware Cloud Foundation APIs to automate the management of vSphere tags.
vSphere Supervisor	You can enable vSphere Supervisor on an existing vSphere cluster in your environment. Create and configure namespaces on the Supervisors to run Kubernetes workloads in dedicated resource pools.

About vSphere

vSphere is the VMware software stack that implements private-cloud data center management and the on-premises component of hybrid-cloud deployments.

A vSphere installation includes one or more instances of vCenter configured to manage one or more virtual data centers. Each virtual data center includes one or more instances of VMware ESX.

About ESX

Each instance of ESX includes management agents and the VMware hypervisor layer, which runs several virtual machines. Each virtual machine contains a guest operating system, such as Windows or Linux, capable of running IT or user applications.

vCenter runs as a virtual machine on an ESX host. vCenter provides an independent endpoint capable of handling API requests both for vCenter and for the vCenter Management API.

vCenter Management Overview

vCenter runs on a Photon OS™ guest operating system.

vCenter is a collection of services designed for managing and monitoring vSphere installations. vCenter responds to CLI commands, requests from the vSphere Client, and API requests from custom clients. API clients can be written in a choice of several software languages.

vCenter is managed by CLI, Web interfaces, or API requests. These requests help you manage vCenter configuration, monitor resource usage, or back up and restore the vCenter instance. You can also use API requests to check the health of vCenter. This programming guide explains how to use the VMware Cloud Foundation APIs that are available to manage and monitor vCenter.

For more information about the capabilities of vCenter, see *vCenter Configuration*.

Understanding the VMware Cloud Foundation API

You can use the VMware Cloud Foundation REST services to automate many aspects of your vSphere environment such as configuration, authentication, certificate management, tagging, managing the life cycle of clusters, content libraries, Kubernetes cluster management, and many more.

- How Does the VMware Cloud Foundation API Work
- The VMware Cloud Foundation API in Context
- Which REST Clients to Use with the VMware Cloud Foundation API
- Why Use the VMware Cloud Foundation API Through REST

How Does the VMware Cloud Foundation API Work

The VMware Cloud Foundation API follows a resource-based REST architecture with JavaScript Object Notation (JSON) requests and responses. REST, an acronym for Representational State Transfer, is a software architectural style in which programs use the Hypertext Transfer Protocol (HTTP) to exchange serialized representations of objects between a client and a server. The client sends an HTTP request and receives an HTTP response from the server. Request and response bodies are encoded using JSON, and must conform to the JSON schema associated with each operation.

The VMware Cloud Foundation API uses the main CRUD (Create, Retrieve, Update, Delete) functions over HTTP to manage representations of the objects that the API defines. In the VMware Cloud Foundation API, these objects are represented by a collection of JSON schemas.

You use:

- HTTP GET to retrieve a representation of an object
- HTTP POST to create an object
- HTTP PATCH or PUT to modify an object
- HTTP DELETE to delete an object

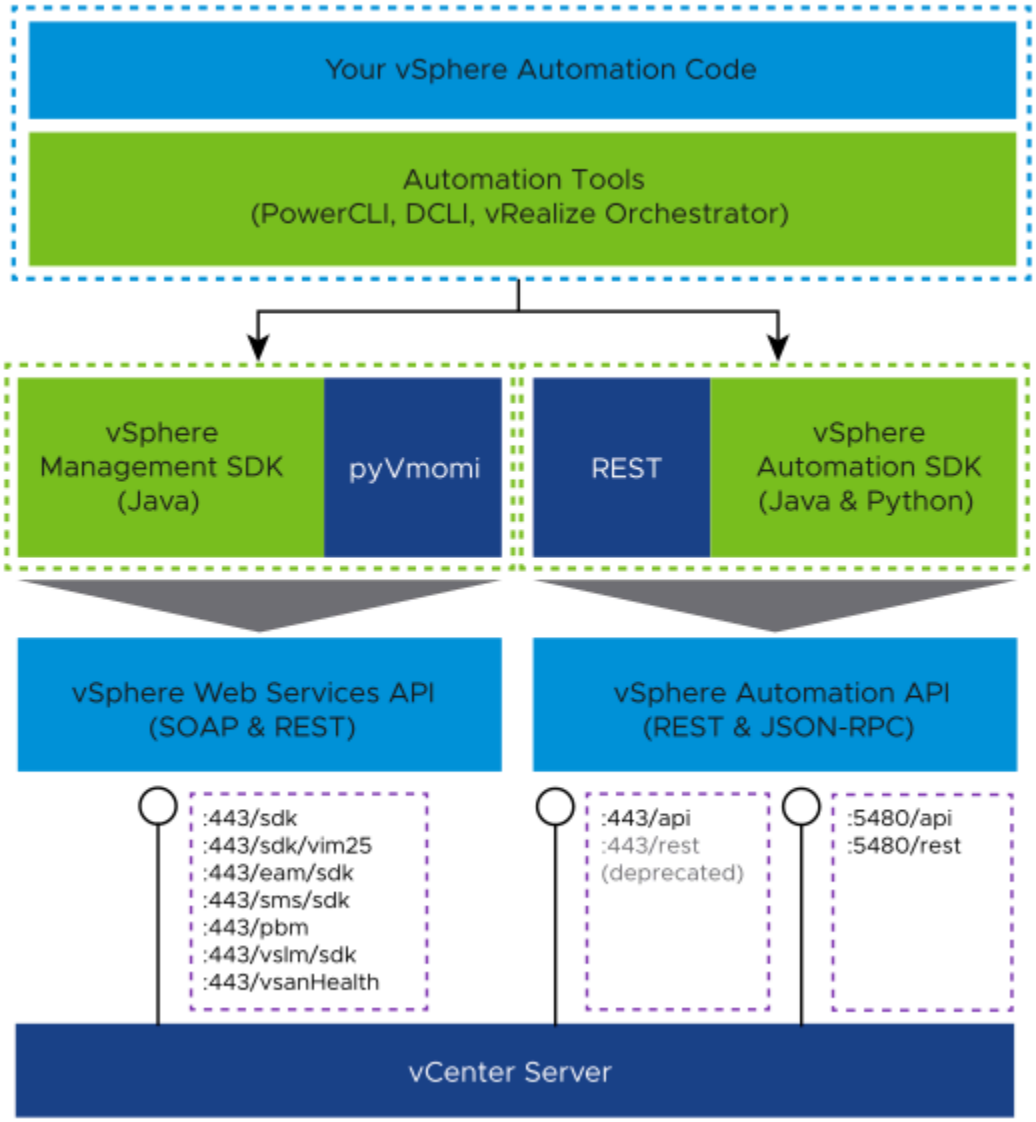
In addition, the HTTP POST operation is used for some custom actions outside of the CRUD framework such as registering or relocating virtual machines.

The HTTP verbs are also called methods. Each API operation is identified by a method and a URL. Operations can also contain a body that is composed of one or more data structures. The operation body contains parameters that are structured as key-value pairs. For more information, see VMware Cloud Foundation API Requests.

The VMware Cloud Foundation API in Context

To take full advantage of the VMware Cloud Foundation API, you must understand the broader context in which it functions.

Figure 547: vCenter Automation Architecture



VMware Cloud Foundation API: the REST API for vSphere Management

The underlying platform of the VMware Cloud Foundation API is vSphere, which consists of vCenter and ESX. The VMware Cloud Foundation API is available to all licensed vSphere users.

Starting with vSphere 6.0, VMware introduced the VMware Cloud Foundation API to supplement the SOAP-based vSphere Web Services API. Since then, VMware engineers have been designing new vSphere features with a RESTful approach and these new services have been integrated into the VMware Cloud Foundation API. As a result, you can now use the VMware Cloud Foundation API to manage and automate many aspects of your vSphere environment such as vCenter configuration, authentication, tagging, content libraries, managing the life cycle of clusters, managing internal and external certificates, vSphere Trust Authority, vSphere Supervisor, and so on.

However, you cannot manage all aspects of your vSphere environment with the VMware Cloud Foundation API as it does not cover all vSphere features. To expose the full set of vSphere functionalities you must use both the VMware Cloud Foundation API and the vSphere Web Services API. For both APIs, VMware has compiled SDKs that help programmers automate tasks.

vSphere Web Services API: the SOAP API for vSphere Management

The vSphere Web Services API, which is based on the SOAP protocol, provides basic vSphere functionalities such as host management, virtual machine provisioning, monitoring, vSAN and many more.

	It uses port 443 and the `/sdk` and `/sdk/vim25` base paths. The vSphere Web Services API can be consumed through: • the vSphere Management SDK for Java • the Python SDK for managing vSphere (pyVmomi) • the Virtual Infrastructure JSON API Starting in vSphere version 8.0.1, you can communicate with the vSphere Web Services API by using HTTP and JSON. For more information, see the *vSphere Web Services SDK Programming Guide*. • customer generated bindings from the WSDL files included in the vSphere Management SDK
VMware Cloud Foundation API Ports	The VMware Cloud Foundation API uses the standard HTTPS network port 443. A subset of the VMware Cloud Foundation API related to appliance configuration and life cycle is available also on port 5480. You can use the services on port 5480 immediately after installing vCenter and during life cycle and configuration operations when the services on port 443 are unavailable.
VMware Cloud Foundation API Base Paths	All existing and non-deprecated HTTP operations of the VMware Cloud Foundation API are available on port 443 and the `/api` base path. The APIs released up to vSphere version 7.0.2 are also available on the deprecated `/rest` base path. The `/api` base path will remain the only active base path when the `/rest` base path is removed in a future vSphere release.
VMware Cloud Foundation API Security	The VMware Cloud Foundation REST APIs are designed with security best practices in mind. The HTTP link between the client application and the VMware Cloud Foundation API is established through the Transport Layer Security (TLS) cryptographic protocol. User identity security is guaranteed by token-based authentication and by the OAuth 2.0 workflows for federated authentication.
SDKs for the VMware Cloud Foundation API	VMware supports open-source SDKs that empower developers to consume the VMware Cloud Foundation API through a programming language. The vSphere Automation SDK for Java and the vSphere Automation SDK for Python contain tools, libraries, documentation, and sample code for vSphere automation. Both SDKs are based on the JSON-RPC protocol. You can download or contribute to the vSphere Automation SDKs on GitHub.
vSphere Automation Tools	Automation tools such as VCF PowerCLI, Data Center CLI (DCLI), and VMware Cloud Foundation Operations orchestrator provide another option for consuming the vSphere APIs. For example, you can use VCF PowerCLI and the vSphere Automation SDK for PowerShell to communicate directly with the VMware Cloud Foundation API by using PowerShell scripts. For more information, see the *PowerCLI User's Guide*.
User Interface Tools	User interface tools such as the vSphere Client and the vCenter Management Interface consume the VMware Cloud Foundation API and the vSphere Web Services API in the back end.

Which REST Clients to Use with the VMware Cloud Foundation API

Client Applications for REST	You can use any REST client to communicate with the VMware Cloud Foundation API. Some popular applications are: • cURL - http://curl.haxx.se • Postman - https://www.getpostman.com/ • jq parser - https://stedolan.github.io/jq/
API Explorer in the vSphere Client	Use the API Explorer, a built-in tool in the vSphere Client, to explore and try out vSphere APIs.

Why Use the VMware Cloud Foundation API Through REST

You can choose whether to access the VMware Cloud Foundation API directly through REST or through the various SDKs and automation tools binded to the API.

Possible use cases for using the VMware Cloud Foundation API through REST:

- There is no SDK available for your programming language. For example, you use PHP, JavaScript, Swift, Rust, and so on.
- Your integration footprint is small and you can't justify the overhead of installing and maintaining the VMware Cloud Foundation SDKs for Java or Python. For example, if you are designing a client application that fetches a few parameters from your vSphere environment, you might decide that installing and maintaining the Java or Python SDKs is unnecessary.

VMware Cloud Foundation API Base Path

You connect to the VMware Cloud Foundation REST API services through the `/api` base path.

The VMware Cloud Foundation REST API services append to the `/api` base path, following the host and port segments of the URL.

For example, you can reach the `Session` service through the following URL.

```
https://<vcenter_server_ip_address_or_fqdn>/api/session
```

For the full list of VMware Cloud Foundation API services, see the *vSphere Automation API Reference*.

> **CAUTION:**
>
> Starting with vSphere 7.0 Update 2, you should use the `/api` base path to connect to the VMware Cloud Foundation API. The old `/rest` base path has been deprecated and will be removed in a future release. All existing and non-deprecated HTTP operations that use the `/rest` base path are available on the `/api` base path, which will remain the only active base path after the `/rest` base path is removed.

VMware Cloud Foundation API Requests

To retrieve or manage resources, clients make HTTP requests to a VMware Cloud Foundation API server.

An API request through HTTP must include a method (or verb), a URL, and headers. Depending on the type of operation, the request might or might not include a body.

HTTP Method

The methods of the VMware Cloud Foundation API follow the logic of the CRUD (Create, Retrieve, Update, Delete) functions used to interact with database applications.

Table 1851: VMware Cloud Foundation API Methods

HTTP Method/Verb	Operation Type	Operation Summary
`POST`	Create	Creates a resource. Also used for some custom actions such as registering or relocating virtual machines.
`GET`	Retrieve	Retrieves the representation of an existing resource in its current state.
`PUT`	Update	Updates or replaces an entire resource.
`PATCH`	Update	Updates or replaces a part of a resource.
`DELETE`	Delete	Deletes a resource.

URL

You can reach each VMware Cloud Foundation API operation through a specific URL.

The URL is typically composed of:

- the vCenter IP address or FQDN.
- the `/api` base path
- the package name
- the service (also called interface) name
- the resource identifier (if applicable)

```
https://<vcenter_server_ip_address_or_fqdn>/api/<package_name>/<service_name>/<resource_identifier>
```

To determine what you want to do with the resource, you associate an HTTP verb (POST, GET, PUT, PATCH, or DELETE) to the URL.

For example, to retrieve information about a virtual machine, you contact the `VM` service of the `vcenter` package with the GET method. You pass the virtual machine identifier as a path parameter.

```
GET https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/vm/<vm_identifier>
```

HTTP Headers

The HTTP headers transfer metadata associated with the API requests and responses. They are an integral part of HTTP requests and responses as they convey information about authorization, request and response content type, response caching, and so on.

You must typically set the following HTTP headers in your requests.

`Authorization`	All requests to create a VMware Cloud Foundation API session must include an `Authorization` header in the form prescribed by your identity provider. For vCenter Single Sign-On basic authentication, you must use the base-64 encoded value of your user name and password separated by a colon (`username:password`). For more information, see Authentication Mechanisms.
`vmware-api-session-id`	This is a custom header that you use to authenticate your subsequent calls after you obtain a session identifier. Paste the session identifier as the header value.
`Content-Type`	This header is used to indicate the data format of the request body. Use *`application/x-www-form-urlencoded`* for requests to the `Authentication Token` service. Use *`application/json`* for all other requests.

Request Body

The HTTP body must be in the JSON format.

Some VMware Cloud Foundation API requests contain a body and some do not. For example, retrieve (GET) operations do not contain a body in the request.

To explore the request and response schemas, visit the *vSphere Automation API Reference*.

VMware Cloud Foundation API Responses

The VMware Cloud Foundation API responds to every HTTP request with either a success or an error status code.

The VMware Cloud Foundation API uses a specific set of HTTP success and error codes for the responses. The response structure depends on the request type: some responses include a document body, some include a string, and some are empty.

Table 1852: HTTP Success Codes

HTTP Code	Message Type	Message Description
200	OK	The request is valid and was completed. The response includes a document body.
201	`Created`	The request is valid. The requested object was created and its identifier is returned with the response.
202	`Accepted`	The request is valid and a task was created to handle it. The response usually contains a task identifier.
204	`No Content`	The request is valid and was completed. The response does not include a body.

Table 1853: HTTP Error Codes

HTTP Code	Error Type	Error Description
400	`ALREADY_EXISTS`	An attempt was made to create an entity but an entity with the same name or identifier already exists in that context.
400	`ALREADY_IN_DESIRED_STATE`	Indicates that an attempt to change the state of a resource or service had no effect because the resource or service is already in the desired state.

HTTP Code	Error Type	Error Description
400	CANCELED	Indicates that an operation was canceled due to an explicit request to do so.
400	FEATURE_IN_USE	Indicates that an action cannot be completed because a feature is in use. For example, trying to deactivate snapshots on a virtual machine that has a snapshot.
400	INVALID_ARGUMENT	Indicates that the values accepted for one or more parameters are not acceptable. Examples: • A parameter value that is not of the expected type. • A parameter value that is not in the required range. • A parameter value that is different from the specifically allowed strings.
400	INVALID_ELEMENT_CONFIGURATION	Indicates that an attempt to modify the configuration of an element or a group containing the element failed due to the configuraton of the element. Examples: • An attempt to move a host with a fault-tolerant virtual machine out of a cluster. • An attempt to remove a host from a DRS cluster without putting the host into maintenance mode.
400	INVALID_ELEMENT_TYPE	Indicates that the server was unable to fulfill the request because an element of a specific type does not fit into a particular container type. Examples: • An attempt to put a virtual machine into a folder that can only contain hosts. • An attempt to attach an SCSI virtual disk to an IDE port.
400	INVALID_REQUEST	Indicates that the request is formed in such a way that the server is unable to process it. For example, sending an invalid JSON structure.
400	NOT_ALLOWED_IN_CURRENT_STATE	Indicates that the requested operation is not allowed with a resource or service in its current state. Examples: • Trying to upgrade the virtual hardware version of a suspended virtual machine. • Trying to power off, reset, or suspend a virtual machine that is not powered on.
400	RESOURCE_IN_USE	Indicates that the operation could not be completed because a resource is in use. Examples: • Trying to remove a datastore when there is a virtual machine registered on a host attached to the datastore. • Trying to add a virtual switch if the physical network adapter being bridged is already in use.
400	UNEXPECTED_INPUT	Indicates that the request body contains a parameter or a field whose name is unknown to the server.
400	UNSUPPORTED	Indicates that the operation is not supported by the underlying platform. Examples: • Trying to hot-plug a CPU when the current virtual machine configuration does not support CPU hot-plugging . • Trying to change the memory size to a value beyond the acceptable guest memory limits supported by the virtual machine's host.
400	UNVERIFIED_PEER	Indicates that an attempt to connect to an unknown or untrusted endpoint failed because the system was unable to verify the identity of the endpoint. Typically, the error data field of this error contains information about the endpoint. If you decide to trust the endpoint, the request can be resubmitted with an indication that the endpoint should be trusted. Examples: • The client provides an IP address or an URL of an endpoint the system should communicate with using a TLS connection, but the endpoint's TLS certificate is self-signed, expired, or otherwise untrustworthy. • The client provides an IP address of a host, with which the system must communicate through SSH, but SSH does not recognize the host's public key.
401	UNAUTHENTICATED	Indicates that the operation requires authentication and the user is not authenticated. For example, if the session identifier in the request header is missing or if it identifies a session that has expired.
403	UNAUTHORIZED	Indicates that the user is not authorized to perform the operation. For example, some operations might require that the user has one or more privileges over the operation or over a resource identifier but the user identified by the operation does not have the required privileges.

HTTP Code	Error Type	Error Description
404	NOT_FOUND	Indicates that a specified resource could not be found. Examples: • Trying to retrieve information about a virtual machine by passing an ID that does not match an existing virtual machine. • Trying to remove a vSwitch by passing an ID that does not match an existing vSwitch.
404	OPERATION_NOT_FOUND	Indicates that the API infrastructure is unable to find the requested service or operation. For example, providing invalid REST API service name or method.
409	CONCURRENT_CHANGE	Indicates that a data structure, entity, or resource has been modified in comparison to some earlier point in time. For example, when the client is doing the write portion of a read-modify-write sequence and indicates that it wants the server to notify it if the data in the server has changed after the read was done, so that overwriting a change can be avoided.
500	ERROR	Indicates that the operation resulted in some error that does not fit into the standard error types.
500	INTERNAL_SERVER_ERROR	Indicates that the server encountered an unexpected condition that prevented it from fulfilling the request. Examples: • The operation returns a value type that does not match the operation schema. • The operation returns an error that is not included in the list of VMware Cloud Foundation API errors.
500	RESOURCE_BUSY	Indicates that the operation could not be completed because a resource is busy. For example, trying to power off a virtual machine that is in the process of being powered on.
500	RESOURCE_INACCESSIBLE	Indicates that the operation could not be completed because an entity is not accessible. For example, attempting to invoke some operation on a virtual machine when the virtual machine's configuration file is not accessible.
500	TIMED_OUT	Indicates that the operation did not complete within the allowed amount of time. The operation might or might not complete after the TIMED_OUT error is reported.
500	UNABLE_TO_ALLOCATE_RESOURCE	Indicates that the operation failed because it was unable to allocate or acquire a required resource. Examples: • Trying to power on a virtual machine when there are not enough licenses to do so. • Trying to power on a virtual machine that would violate a resource usage policy.
503	SERVICE_UNAVAILABLE	Indicates that the service you are trying to reach is unavailable. Examples: • Trying to reach the server when it is too busy. • Trying to reach the server while it's undergoing maintenance.

Supplementing the VMware Cloud Foundation APIs

Some features are not accessible by API. These features require direct shell access or specific user interfaces.

vSphere Client

The vSphere Client is a user interface for general management tasks. All administrative functions are available through the vSphere Client.

The vSphere Client is a cross-platform application that can connect only to vCenter. It has a full range of administrative functionality and an extensible plug-in-based architecture. Typical users are virtual infrastructure administrators, help desk, network operations center operators, and virtual machine owners.

vCenter Server Management Interface

The vCenter Server Management Interface is an interface for configuring, monitoring, and patching vCenter.

The vCenter Server Management Interface runs in a browser that connects to port 5480 of vCenter. The vCenter Server Management Interface provides access to all the service APIs of vCenter.

Direct Console User Interface

The Direct Console User Interface (DCUI) provides access to basic operations for vCenter management and set up.

The DCUI provides access to a subset of management functions. It provides direct access to vCenter if the vSphere Client and the vCenter Management Interface become unavailable.

After the vCenter startup is complete, the DCUI displays basic CPU, memory, and network information on the operator console. The root user can use the DCUI screen to configure network interfaces, DNS, and super administrator password.

vCenter Appliance Bash Shell

You can use the vCenter appliance Bash shell to access all vCenter commands and plug-ins that you use for monitoring, troubleshooting, and configuring the vCenter instance through the API. For more information about the appliance Bash shell, see *vCenter Configuration*.

Data Center CLI

The Data Center CLI (DCLI) is a CLI client of the VMware Cloud Foundation APIs. Almost all methods that are available in the VMware Cloud Foundation APIs are available as DCLI commands. For more information about DCLI, see *DCLI User's Guide*.

VCF PowerCLI

VCF PowerCLI contains modules of cmdlets based on Microsoft PowerShell. The main VCF PowerCLI module provides cmdlets for automated administration of the vSphere environment.

VCF PowerCLI, the most popular vSphere automation tool, provides alternative, and often more user-friendly, methods to consume the vSphere APIs. The vSphere Automation SDK for PowerShell that was introduced with VMware VCF PowerCLI 12.4, enables you to prepare data structures and call the vSphere REST APIs directly from PowerShell. It uses the `Initialize-` and `Invoke-` cmdlet prefixes to prepare data structures and call API endpoints. This approach keeps the REST workflow intact but transfers it to a PowerShell interface. For more information, see the *Managing the vSphere Automation API with VMware PowerCLI* chapter in the *PowerCLI User's Guide*.

vSphere Web Services API

The vSphere API is exposed as a Web service, running on VMware vSphere server systems. The API provides access to the vSphere management components - the managed objects that you can use to manage, monitor, and control life-cycle operations of virtual machines and other virtual infrastructure components, such as data centers, datastores, networks, and so on. For more information about the vSphere Web Services API, see the *vSphere Web Services API Reference*.

Authentication Mechanisms

To perform operations in the vSphere environment, principals must authenticate to the VMware Cloud Foundation services. You can use the default vCenter Single Sign-On identity source or federate your vCenter to an external identity provider such as Microsoft Active Directory Federation Services, Okta, or Azure AD.

After successful authentication, client applications and users receive a session identifier or access token that grants access to the vSphere Automation services.

Choosing Your Identity Source

First, you must decide whether to authenticate to vCenter by using the default identity provider, vCenter Single Sign-On, or leverage your enterprise configuration to an external identity provider.

Default or External Identity Provider

vCenter acts as the default identity provider, by using its built-in vCenter Single Sign-On service to authenticate users and service accounts.

In vSphere 7.0 and later, you can delegate vSphere identity management to an external identity provider and take advantage of increased security and multifactor authentication. In this scenario, vCenter Single Sign-On is replaced by an external identity provider as the entity that performs principals' authentication.

Identity provider federation enhances the security and compliance of your applications and provides access to flexible benefits such as multifactor authentication (MFA) and automatic account synchronization.

You can federate vCenter authentication to:

- AD FS (starting in vSphere 7.0)
- Okta (starting in vSphere 8.0 Update 1)
- Azure AD (starting in vSphere 8.0 Update 2)

For more information, see Identity Provider Federation for vCenter.

Basic or Token-Based Authentication

You can use basic and token-based authentication to authenticate principals to the VMware Cloud Foundation services. VMware encourages you to use token-based authentication as it provides more security and flexibility for your applications and users.

Basic Authentication

Basic authentication passes a user name and password to vCenter.

By default, principals can use the vCenter Single Sign-On user name and password to connect to the VMware Cloud Foundation endpoint. The VMware Cloud Foundation endpoint checks whether the user name and password are present in the VMware Directory Service. On success, the system returns a session identifier for the VMware Cloud Foundation endpoint.

If your vCenter is federated to an external identity provider, you can use basic authentication through the OAuth 2.0 Password grant type.

> **Important:**
>
> VMware encourages you to refrain from using basic authentication and adopt token-based authentication, as it is more secure and provides more flexibility.

SAML Token-Based Authentication

Token-based authentication involves the use of an encrypted token that provides authentication and authorization data to the VMware Cloud Foundation endpoint. The vCenter token complies with the Security Assertion Markup Language (SAML) specification, an XML-based schema for communicating authentication data.

To acquire a SAML token, client applications issue a token request to vCenter Single Sign-On. Client applications then send the SAML token to the VMware Cloud Foundation API endpoint in exchange for a session identifier.

Starting with vSphere 7.0, you can use token-based authentication for your federated vCenter through a 3-step process:

1. Principals use the various OAuth 2.0 grant types with the external identity provider to obtain identity and access tokens in JSON Web Token (JWT) format.
2. Principals exchange the JWT tokens for a vCenter Single Sign-On SAML token.
3. Principals use the SAML token to obtain a session identifier from the VMware Cloud Foundation endpoint.

Authentication Terminology

To use the vSphere programming features effectively, you must understand a set of specific authentication terms and concepts.

Table 1854: vCenter Single Sign-On Glossary

Term	Definition
Principal	An entity that can be authenticated, such as a user.
Identity Provider	A service that manages identity sources and authenticates principals. Examples: Microsoft Active Directory Federation Services (AD FS) and vCenter Single Sign-On.
Identity Source (Directory Service)	Stores and manages principals. Principals consist of a collection of attributes about a user or a service account such as name, address, email, and group membership. Examples: Microsoft Active Directory and VMware Directory Service (vmdir).
Authentication	The means of determining whether someone or something is, in fact, who or what it declares itself to be. For example, users are authenticated when they provide their credentials, such as smart cards, user name and correct password, and so on.
Authorization	The process of verifying what objects principals have access to.
Token	A signed collection of data comprising the identity information for a given principal. A token might include not only basic information about the principal such as email address and full name, but also, depending on the token type, the principal's groups and roles.
vmdir	VMware Directory Service. The internal (local) LDAP repository in vCenter that contains user identities, groups, and configuration data.
OAuth 2.0	An open authorization standard that enables the exchange of information among principals and web services without exposing principals' credentials.
OpenID Connect (OIDC)	Authentication protocol based on OAuth 2.0 that augments OAuth with user-identifying information. It is represented by the ID token that the authorization server returns together with the access token during OAuth authentication. vCenter uses OIDC capabilities when interacting with Active Directory Federation Services (AD FS), Okta, Microsoft Entra ID, and PingFederate.
System for Cross-domain Identity Management (SCIM)	The standard for automating the exchange of user identity information between identity domains or IT systems.
VMware Identity Services	Starting in version 8.0 Update 1, VMware Identity Services is a built-in container within vCenter that you can use for identity federation to external identity providers. It serves as an independent identity broker within vCenter and comes with its own set of APIs. Currently, VMware Identity Services supports Okta, Microsoft Entra ID, and PingFederate as external identity providers.
Tenant	A VMware Identity Services concept. A tenant provides a logical separation of data from other tenants' data in the same virtual environment.
JSON Web Token (JWT)	A token format defined by the OAuth 2.0 specification. A JWT token carries authentication and authorization information about a principal.
Relying party	A relying party "relies" on the authorization server, VMware Identity Services, or AD FS, for identity management. For example, through federation, vCenter establishes relying party trust to VMware Identity Services or AD FS.
Security Assertion Markup Language (SAML)	An XML-based open standard for exchanging authentication and authorization data between parties that is used by vCenter. Principals obtain a SAML token from vCenter Single Sign-On and then send it to the vSphere Automation API endpoint for a session identifier.

Authenticate to vCenter with vCenter Single Sign-On Credentials

You can use your vCenter Single Sign-On credentials with basic authentication to establish a session with a vCenter system.

To perform this task, you must have:

- The IP address or FQDN of your vCenter system
- Valid vCenter Single Sign-On user name and password

The single-factor basic authentication is the simplest method to authenticate to vCenter. You connect to the VMware Cloud Foundation API endpoint with your vCenter Single Sign-On user name and password. The VMware Cloud Foundation endpoint checks whether the user credentials are present in vmdir, a vCenter Single Sign-On component. On success, the system returns a session identifier that you can use for subsequent API calls.

1. Call the `Session` interface with the base-64 encoded value of your vCenter Single Sign-On user name and password separated by a colon (`username:password`) in the authorization header.

   ```
   POST https://<vcenter_server_ip_address_or_fqdn>/api/session
   Authorization: Basic <username:password>
   ```

   On success (status code 201), the system returns a session identifier. For example, `"a5adf880fc8e1e2944a541a4c7d74465"`.

2. To authenticate subsequent API calls, pass the session identifier as a request header.

   ```
   vmware-api-session-id: <session_ID>
   ```

   Important:

   VMware encourages you to refrain from using basic authentication and adopt token-based authentication, as it is more secure and provides more flexibility.

Identity Provider Federation for vCenter

Starting with vSphere 7.0, you can replace through federation your built-in vCenter identity source with an external identity source. Depending on your vCenter version, you can use external identity providers such as AD FS, Azure AD, or Okta.

With identity federation, you can use the same corporate identity source for your vCenter that you use for your other federated desktop and cloud applications.

vCenter Identity Provider Federation Basics

In vSphere 7.0 and later, vCenter supports federated authentication. In this scenario, when a user logs in to vCenter, vCenter redirects the user login to the external identity provider. The user credentials are no longer provided to vCenter directly. Instead, the user provides credentials to the external identity provider. vCenter trusts the external identity provider to perform the authentication. In the federation model, users never provide credentials directly to any service or application but only to the identity provider. As a result, you "federate" your applications and services, such as vCenter, with your identity provider.

Why Is Identity Provider Federation Useful

Federating vCenter to an enterprise identity provider alleviates the burden of identity management and provides flexible options such as multifactor authentication (MFA), automatic registration and termination of users across services, and many more. Identity provider federation uses token-based authentication and minimizes the risk of bad actors acquiring protected credentials such as user names and passwords. Identity provider federation also helps your organization with compliance as various international standards require MFA to guarantee data security. In addition, with identity provider federation, you can automate vCenter user management because you utilize the users and groups from your enterprise identity source.

vCenter Server External Identity Provider Support

vCenter supports the following external identity providers:

- AD FS (vSphere 7.0 and later)
- Okta (vSphere 8.0 Update 1 and later)
- Microsoft Entra ID, formerly called Azure AD (vSphere 8.0 Update 2 and later)
- PingFederate (starting in vSphere 8.0 Update 3)

Federation to Microsoft Active Directory Federation Services (AD FS)

In vSphere 7.0 and later, you can activate identity federation to Microsoft Active Directory Federation Services (AD FS). In this scenario, vCenter federates directly to the enterprise identity provider, AD FS, without the use of an authentication intermediary service.

You can configure federation to AD FS with the help of the VMware Cloud Foundation API. For more information, see Federate vCenter to Microsoft Active Directory Federation Services (AD FS).

Federation to Okta and Azure AD Through VMware Identity Broker - vCenter

Starting from vSphere 8.0 Update 1, you can activate federation to Okta as the identity provider. Starting from vSphere 8.0 Update 2, you can activate federation to Azure AD as the identity provider. Both configurations use VMware Identity Broker - vCenter, an authentication intermediary that functions as a built-in container within vCenter. With VMware Identity Broker - vCenter, you can configure principals to authenticate to vCenter by using an external identity provider.

vCenter can support the default `vsphere.local` identity source and only one configured external identity provider. You cannot use multiple external identity providers.

Once you set up your federated authentication, you can use OAuth for login to vCenter.

> **Note:**
>
> You can federate vCenter to Okta or Azure AD by using the vSphere Client. For more information, see `Configure vCenter Identity Provider Federation for Okta` and `Configure vCenter Identity Provider Federation for Microsoft Entra ID` in the *vSphere Authentication Guide*.

Authentication to a Federated vCenter

Once you have configured vCenter to point to AD FS, Okta, or Azure AD, you can use the VMware Cloud Foundation and the VMware Identity Broker - vCenter APIs to authenticate principals to your vCenter. Depending on the type of user or application you want to authenticate, you can choose among different token-based authentication options. vSphere supports the various OAuth 2.0 grant types.

Once you have configured vCenter to point to AD FS, you can use the VMware Cloud Foundation API to authenticate principals to your vCenter. Depending on the type of user or application you want to authenticate, you can choose among different token-based authentication options. vSphere supports the various OAuth 2.0 grant types.

What is OAuth 2.0

OAuth 2.0 is an open authorization framework that allows information exchanges among web services without exposing sensitive data such as user names and passwords. OAuth 2.0 boosts security and prevents malicious actors from stealing users' credentials over the Internet. OAuth 2.0 is an open framework and can be customized according to the organizational requirements.

You can use different OAuth 2.0 grant types to authenticate your applications to vCenter. The OAuth grant types represent different methods by which a principal can obtain an access token from an authorization server. In the vSphere implementation, the access token is used to obtain a SAML token and authenticate principals to vCenter.

OAuth 2.0 Grant Types

Password	In the Password grant type, the client application exchanges the user's credentials, user name and password, for an access token, and optionally an ID token (JWT tokens), from the authorization server. OAuth 2.0 has been designed to prevent the exchange of user credentials, so the Password grant type must be avoided whenever possible, especially for third-party application authentication. The latest *OAuth 2.0 Security Best Current Practice* recommends avoiding the Password grant type altogether. As a single-factor method, the Password grant type is incompatible with the requirements of multifactor authentication

	(MFA). For more information, see Authenticate to an AD FS-Federated vCenter by Using the Password Grant Type. **Important:** The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.
Authorization Code	In the Authorization Code grant type, the client application uses an authorization code to obtain JWT tokens from the authorization server. This grant type requires a user to verify an authentication request through a web browser. The Authorization Code grant type is compatible with multi-factor authentication and guarantees a substantial degree of security. This grant type is appropriate for regular web applications running on a server. For more information, see Authenticate to an AD FS - Federated vCenter by Using the Authorization Code Grant Type.
Client Credentials	The Client Credentials grant type permits web services to authenticate to other web services without the involvement of a user. This grant type is useful for applications that are resource owners and can take advantage of headless, or backend automation, when a physical user is not present. For example, a client application might use this grant type to authenticate and fetch details about a virtual environment.
Refresh Token	The Refresh Token grant type is used by clients to exchange an expiring access token for a refresh token that can be used to receive a new access token. With this grant type, a client application can fetch a valid access token without asking the user to log in again. It allows client applications to provide a seamless user experience without further interaction with the user after timeouts.

What Is OpenID Connect (OIDC)

The vSphere implementation of OAuth 2.0 uses the OpenID Connect (OIDC) authentication protocol. With OIDC, the authorization server returns an ID token containing data about who the users are and what permissions they have on the resource server. As a result, vCenter can identify users and grant them the respective permissions that are defined on the authorization server.

AD FS Federation Workflows

You can federate your vCenter to Microsoft Active Directory Federation Services (AD FS) and use the OAuth 2.0 grant types to authenticate.

Federate vCenter to Microsoft Active Directory Federation Services (AD FS)
You can federate vCenter to Microsoft Active Directory Federation Services (AD FS) as an external identity provider by using the vCenter `Identity Providers` service.

> **Note:**
> This process of configuring an AD FS identity provider requires that you have administrative access to both your vCenter and your AD FS server. During the configuration process, you enter information first in your vCenter, then in your AD FS server, then in your vCenter.

Active Directory Federation Services requirements:

- AD FS for Windows Server 2016 or later must already be deployed.
- AD FS must be connected to Active Directory.
- An Application Group for vCenter must be created in AD FS as part of the configuration process. See the VMware knowledge base article at https://kb.vmware.com/s/article/78029.
- An AD FS server certificate (or a CA or intermediate certificate that signed the AD FS server certificate) that you add to the Trusted Root Certificates Store.
- You have created a vCenter administrators group in AD FS that contains the users you want to grant vCenter administrator privileges to.

For more information about configuring AD FS, see the Microsoft documentation.

vCenter and other requirements:

- vSphere 7.0 or later

- vCenter must be able to connect to the AD FS discovery endpoint, and the authorization, token, logout, JWKS, and any other endpoints advertised in the discovery endpoint metadata.
- You need the **VcIdentityProviders** > **Manage** privilege to create, update, or delete a vCenter Identity Provider that is required for federated authentication. To limit a user to view the Identity Provider configuration information only, assign the **VcIdentityProviders** > **Read** privilege.

1. Authenticate to the VMware Cloud Foundation API endpoint and establish a session.
2. Add your AD FS root CA certificate to the Trusted Root Certificates Store.

   See Managing Certificates.
3. Create a `ProvidersTypes.OidcCreateSpec` object by using the Application Group configuration from AD FS.

Method	Description
`setDiscoveryEndpoint`	The OpenID address of the AD FS server.
`setClientId`	The client identifier of the AD FS Application Group.
`setClientSecret`	The secret shared between the client and the provider.
`setClaimMap`	This parameter is required but not applicable to AD FS. Use an empty array `[]`.

4. Create an object of type `ProvidersTypes.ActiveDirectoryOverLdap`.

Method	Description
`setUserName`	The user name of a user in the domain who has a minimum of read-only access to base Distinguished Name (DN) for users and groups.
`setPassword`	The password of a user in the domain who has a minimum of read-only access to base DN for users and groups.
`setUsersBaseDn`	The base DN for users in the Active Directory environment connected to AD FS that you want to be able to federate with vCenter.
`setGroupsBaseDn`	The base DN for groups in the Active Directory environment connected to AD FS that you want to be able to federate with vCenter.
`setServerEndpoints`	Active directory server endpoints. At least one Active Directory server endpoint must be set. Use the format `ldap://<hostname>:<port>` or `ldaps://<hostname>:<port>`. The port is typically 389 for LDAP connections and 636 for LDAPS connections. For Active Directory multi-domain controller deployments, the port is typically 3268 for LDAP and 3269 for LDAPS.
`setCertChain`	The SSL certificate chain in base64 encoding. You can skip this parameter only if all the Active Directory server endpoints use the LDAP (and not the LDAPS) protocol.

5. Add the identity provider by using the `Identity ProvidersIdentity Providers` service.
   a) Create an object of type `ProvidersTypes.CreateSpec`.

Method	Description
`setConfigTag`	The configuration type of the identity provider. The possible values are *`Oauth2`* and *`Oidc`*. For AD FS federation, use *`Oidc`*.
`setName`	The user-friendly name for the identity provider. You must use the exact string *`Microsoft ADFS`* for proper configuration.

Method	Description
`setUpnClaim`	The name of the claim in the AD FS JWT token that contains the user principal name of the user that is logging in. You must use the same value that you used when you set up the AD FS Application Group. The procedure from the article in the prerequisites uses *`upn`*. If unset, the default value is *`acct`*.
`setGroupsClaim`	The name of the claim in the AD FS JWT token that contains the group membership of the user that is logging in. You must use the same value that you used when you set up the AD FS Application Group. The procedure from the article in the prerequisites uses *`group`*. If unset, the groups for the subject consist of the groups in `group_names` and `group_ids` claims.
`setIsDefault`	Set to *`true`*. Specifies whether the provider is the default provider. Setting to *`true`* makes all other providers non-default. If unset: • In case it is the first created provider, it is set as the default provider. • In case it is not the first created provider, it is not set as the default provider.
`setOidc`	Use the `ProvidersTypes.OidcCreateSpec` object.
`setIdmProtocol`	The communication protocol used to connect to AD FS to search for users and groups when assigning permissions in vCenter. You must use *`LDAP`*. If unset, no communication protocol is configured for the users and groups search.
`setActiveDirectoryOverLdap`	Use the `ProvidersTypes.ActiveDirectoryOverLdap` object.

b) To add the provider, call the `create(ProvidersTypes.CreateSpec)` method.

The operation returns the ID of the provider you added.

6. Configure vCenter permissions for Active Directory users or groups in your AD FS environment.

   You can do this in two ways:

   - Add a user from your AD FS environment to a group in vCenter.
   - Configure Global Permissions for an AD FS user.

   Note:

   In vSphere 8.0 and later, you cannot configure permissions through the VMware Cloud Foundation API. Instead, you use either the vSphere Client or the vSphere Web Services API. For more information, see the *vSphere Authentication Guide* or the *vSphere Web Services SDK Programming Guide*.

7. Optional: Copy the two redirect URIs from the Identity Provider Configuration page in the vSphere Client and add them to your AD FS Application Group.

   Note:

   You must do this step to enable logging in to vCenter through AD FS by using the vSphere Client.

You configured vCenter to use AD FS as the identity provider.

Authenticate to an AD FS-Federated vCenter by Using the Password Grant Type

If your vCenter is federated to AD FS, you can authenticate with the OAuth 2.0 Password grant type.

- Verify that your vCenter is federated to AD FS. For more information, see *Federate vCenter to Active Directory Federation Services (AD FS).*

- You must have an AD FS account and user credentials with the necessary permissions to view and manage vCenter.
- You must register an OAuth client for your application on the authorization server (AD FS).

You can use the Password grant type to exchange user credentials for an access token and an ID token from the authorization server. From the user's perspective, the password grant type functions exactly as basic authentication with the only difference that you use your AD FS, and not your local vCenter Single Sign-On credentials.

The Password grant type is possible among native apps but is not recommended for authentication to third-party apps. Modern security best practices require that primary user credentials such as passwords do not leave the native API environment. Therefore, it is more secure to use other OAuth 2.0 grant types for authentication to third-party apps.

To use the Password grant type with AD FS, you provide your AD FS user name and password to the `Common Infrastructure Services (CIS) Session` endpoint.

In the background, the client application calls the authorization server (AD FS) and obtains an access token and an ID token in JWT format. vCenter Single Sign-On converts the JWT tokens into a SAML token which is used to obtain a session identifier and authenticate your application to vCenter.

> **Important:**
> The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.

1. Call the `Session` interface with the base-64 encoded value of your AD FS user name and password separated by a colon (`username:password`) in the authorization header.

   ```
   POST https://<vcenter_server_ip_address_or_fqdn>/api/session
   Authorization: Basic <username:password>
   ```

   On success (status code 201), the system returns an authentication session identifier.

2. To authenticate subsequent API calls, pass the session identifier as a request header.

   ```
   vmware-api-session-id: <session_ID>
   ```

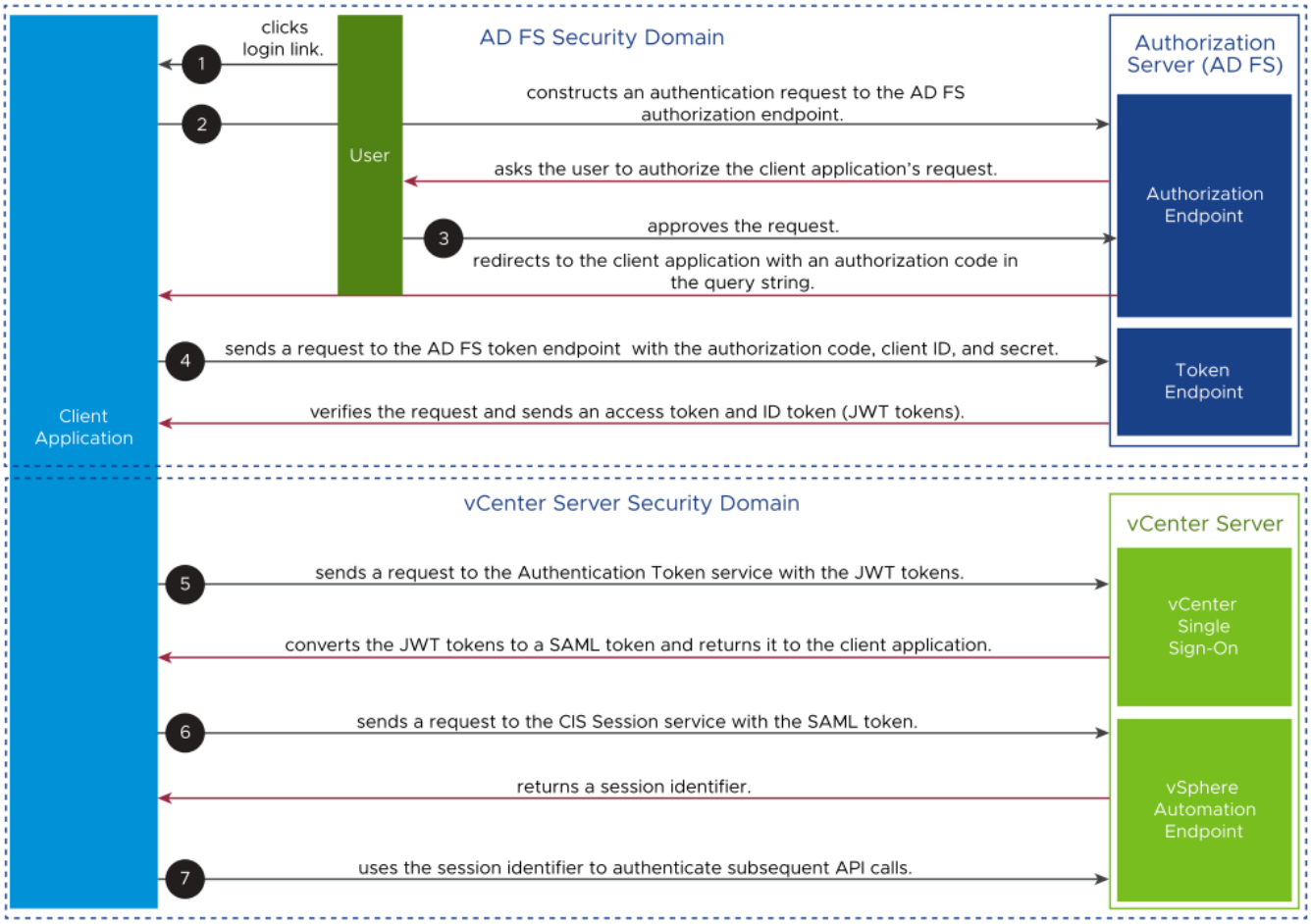
Authenticate to an AD FS-Federated vCenter by Using the Authorization Code Grant Type

If your vCenter is federated to an enterprise identity provider, such as Active Directory Federation Services (AD FS), you can authenticate your client applications by using the OAuth 2.0 Authorization Code grant type.

- Verify that your vCenter is federated to AD FS. For more information, see *Federate vCenter to Active Directory Federation Services (AD FS).*
- You must have an AD FS account and user credentials with the necessary permissions to view and manage vCenter.
- You must register an OAuth client for your application on the authorization server (AD FS).

You can use the Authorization Code grant type if you want to authenticate client apps to vCenter through multifactor authentication (MFA). This grant type is applicable for regular web applications that involve a human user interacting with a server.

Figure 548: Authentication to an AD FS-Federated vCenter Through the Authorization Code Grant Type



A principal authenticates to vCenter through the Authorization Code grant type as follows.

1. The user clicks **Login** within the client application.
2. The client application constructs a request to the AD FS authorization endpoint.

   To retrieve the authorization endpoint, you must query the AD FS discovery endpoint at, for example, `https://<your_adfs_server.com>/adfs/.well-known/openid-configuration`.

   The client application's request must include the application's client ID and a redirect URI that tells the authorization server where to redirect the user after approving the request.

   As a result, the authorization server opens a prompt page asking the user to authorize the client application's request.
3. The user confirms the request.

   The browser is redirected back to the client application with an authorization code in the query string.
4. The client application sends a request to the AD FS token endpoint with the authorization code, client ID, and secret.

   The authorization server verifies the request and sends an access token and an ID token (JWT tokens).
5. The client application sends the JWT tokens to vCenter Single Sign-On by using the `Token` interface of the `com.vmware.vcenter.authentication` packageAuthentication Token service.
   a) Create an object of type `TokenTypes.IssueSpec` with the following modifiers.

Method	Value
`setSubjectTokenType`	*`urn:ietf:params:oauth:token-type:access_token`*
`setSubjectToken`	*<ACCESS_TOKEN>*

Method	Value
setActorTokenType	*urn:ietf:params:oauth:token-type:id_token*
setActorToken	<*ID_TOKEN*>
setGrantType	*urn:ietf:params:oauth:grant-type:token-exchange*
setRequestedTokenType	*urn:ietf:params:oauth:token-type:saml2*

b) Call the `issue(TokenTypes.IssueSpec)` method

For example,

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/authentication/
token' \
--header 'Authorization: Bearer <ACCESS_TOKEN>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'subject_token_type=urn:ietf:params:oauth:token-type:access_token' \
--data-urlencode "subject_token=<ACCESS_TOKEN>" \
--data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
--data-urlencode "actor_token=<ID_TOKEN>" \
--data-urlencode 'grant_type=urn:ietf:params:oauth:grant-type:token-exchange' \
--data-urlencode 'requested_token_type=urn:ietf:params:oauth:token-type:saml2'
```

vCenter Single Sign-On returns a SAML token.

6. The client application calls the `Common Infrastructure Services (CIS) Session` service.

   By using the `CIS Session` service, principals can manage session tokens - including creation and deletion of session identifiers, and obtaining information about sessions.

   a) Base-64 decode the SAML token, compress it, and then Base-64 encode it again.

   b) Call the `session.create()` methodSession service with the SAML token.

   For example,

   ```
   curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/session' \
   --header 'Authorization: SIGN token=<COMP_TOKEN>'
   ```

   On success (status code 201), the VMware Cloud Foundation endpoint returns a session identifier in the response.

7. To authenticate, the client application uses the session identifier in subsequent API calls to the VMware Cloud Foundation API endpoints.

   For example,

   ```
   curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/cis/tagging/category' \
   --header 'vmware-api-session-id: <session_ID>'
   ```

VMware Identity Broker - vCenter Workflows

You can federate your vCenter to an external identity provider through VMware Identity Broker - vCenter. It is a built-in container within vCenter that enables federation to an external identity provider such as Okta or Azure AD.

What Is VMware Identity Broker - vCenter

VMware Identity Broker - vCenter is VMware's specialized authentication solution that allows you to federate your apps to external identity providers such as Okta or Azure AD. VMware Identity Broker - vCenter functions as a container built-in within your vCenter. VMware Identity Broker - vCenter comes with its own `VMware Identity Broker - vCenter API` that is separate from the VMware Cloud Foundation API and the vSphere Web Services API.

You can use VMware Identity Broker - vCenter to federate to:

- Okta (starting in vSphere 8.0 Update 1)
- Azure AD (starting in vSphere 8.0 Update 2)

Configure vCenter Identity Provider Federation to Okta or Azure AD

To configure your vCenter to point to Okta or Azure AD as the identity provider, you must use the Okta or Azure AD interface and the vSphere Client. This integration uses VMware Identity Broker - vCenter. For more information, see `Configure vCenter Identity Provider Federation for Okta` and `Configure vCenter Identity Provider Federation for Microsoft Entra ID` from the *vSphere Authentication Guide*.

OAuth Authentication to Your Federated vCenter, App, or Script

Once configured, you can use the VMware Cloud Foundation and the VMware Identity Broker - vCenter APIs to authenticate to your federated vCenter, application, or script.

You can use the following OAuth 2.0 grant types:

- Password (not recommended)
- Authorization Code
- Client Credentials
- Refresh Token

Federate Apps or Scripts to an External Identity Provider by Using Your VMware Identity Broker - vCenter Configuration

Configure a client application or script for authentication by using an already existing vCenter identity provider configuration on VMware Identity Broker - vCenter .

Okta and Azure AD requirements:

- You are a customer of Okta or Microsoft and have a dedicated Okta domain space or an Azure AD account.
- You have identified the Okta or Azure AD users and groups that you want to share with your app.

vCenter and other requirements:

- vSphere 8.0 Update 1 or later, with the VMware Identity Broker - vCenter activated (they are activated by default).
- A vCenter that is federated to Okta or Azure AD. (You must know the IP address or the FQDN of the vCenter.)
- The users and groups from the identity provider are provisioned in your vCenter.
- You must know the tenant that you want to use for your client app or script. The default tenant that is created during vCenter federation is called *`CUSTOMER`*.
- You must have the **VcIdentityProviders** > **Manage** privilege.

You can leverage the identity provider configuration you created for vCenter for other apps or scripts that you want to federate to Okta or Azure AD.

For this automation scenario you must use services from the VMware Cloud Foundation API and the VMware Identity Broker - vCenter API. The VMware Identity Broker - vCenter API endpoints are located also on your vCenter but are taxonomically separated from the VMware Cloud Foundation API endpoints.

1. Get a session identifier for the VMware Cloud Foundation API.

   For more information see Authentication Mechanisms.

2. Get a tenant admin client token.

   To authenticate to the VMware Identity Broker - vCenter endpoints, you must obtain a tenant admin client token by using the `Tenant Admin Client` service of the VMware Cloud Foundation API.

   To authenticate to VMware Identity Broker - vCenter, you must obtain a tenant admin client token by calling the `get(tenant)` method of the `AdminClient` interface from the `com.vmware.vcenter.identity.broker.tenants` package.

   The default `tenant` that is created during the vCenter federation is called *`CUSTOMER`*. You can use this tenant for your app or script.

   For authentication, use the session identifier that you obtained in the previous step.

   Example request:

   ```
   curl --location --request GET 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/identity/broker/
   tenants/<tenant>/admin-client' \
   ```

```
--header 'vmware-api-session-id: <session_id>' \
--header 'Content-Type: application/json'
```

In the response, you get an access token in JWT format that you use to authenticate to VMware Identity Broker - vCenter.

3. Create an OAuth 2.0 client for your app.

   To authenticate to the external identity provider, you must create an OAuth 2.0 client for your app on VMware Identity Broker - vCenter. You can do this by using the `Create Broker OAuth 2 Client` service of the VMware Identity Broker - vCenter API.

   For authentication, you must use the tenant admin client token obtained in the previous step.

   Example request:

```
curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>
/broker/oauth2-clients' \
--header 'Content-Type: application/
vnd.vmware.horizon.manager.accesscontrol.broker.oauth2client.with.rule.sets+json' \
--header 'Accept: application/vnd.vmware.horizon.manager.accesscontrol.broker.oauth2client.with.rule.sets
+json' \
--header 'Authorization: HZN <my_admin_client_token>' \
--data-raw '{
    "client_id": "<my_app_client_id>",
    "grant_types": [
        "refresh_token",
        "client_credentials",
        "password",
        "authorization_code"
    ],
    "scope": [
        "email",
        "profile",
        "user",
        "openid",
        "group"
    ],
    "secret": "<my_app_client_secret>",
    "redirect_uris": [
        "https://<my_app_redirect_uri1>",
        "https://<my_app_redirect_uri2>"
    ]
}'
```

   On success (status code 201), the operation returns the data about the new OAuth 2.0 client.

Authenticate your app or script to the external identity provider by using the OAuth 2.0 grant types.

Obtain JWT Tokens from VMware Identity Broker - vCenter by Using the Password Grant Type

If your vCenter or other application is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access and ID token in JWT format by using the OAuth Password grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.
- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id, client secret, and the redirect URIs for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *CUSTOMER*.

- You must have an Okta or Azure AD account and user credentials with the necessary permissions to view and manage vCenter.

You can use the Password grant type to exchange user credentials for an access token and an ID token from the authorization server. From the user's perspective, the password grant type functions similarly to local domain authentication with the difference that principals use their external identity provider, and not their local vCenter Single Sign-On credentials, to authenticate.

The Password grant type involves the client collecting the principal's user name and password directly and exchanging them for an access token from the authorization server. From a security perspective, handling user credentials directly increases the risk of exposure and compromise. From a user experience perspective, users might be reluctant to expose their credentials to third-party apps, as it goes against the principle of separating authentication from authorization. Therefore, using the OAuth 2.0 Password grant type is not recommended.

> **Important:**
> The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.

Obtain an access and ID token.

Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint with the required grant type, your app's client ID and secret, and the user name and password of a registered user.

In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (`client_id:secret`).

The content type must be *`x-www-form-urlencoded`*.

Example request:

```
curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'username=<my_user_name>' \
--data-urlencode 'password=<my_password>'
```

The authorization server returns the JWT tokens corresponding to the grant type: access token , ID token, and refresh token.

Exchange the JWT tokens for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Obtain JWT Tokens from VMware Identity Broker - vCenter by Using the Authorization Code Grant Type
If your vCenter or other application is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access and ID token in JWT format by using the OAuth 2.0 Authorization Code grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.
- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id, client secret, and the redirect URIs for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *`CUSTOMER`*.
- You must have an Okta or Azure AD account and user credentials with the necessary permissions to view and manage vCenter.

The Authorization Code grant type is appropriate for regular web applications that involve a human user interacting with a server, especially if you want to activate multifactor authentication (MFA). The Authorization Code grant type protects user credentials as they do not leave the API environment.

1. Obtain an authorization code.
   a) Construct a query request in the browser with the following parameters.

client_id	The client identifier you obtained when you registered the OAuth 2.0 client on VMware Identity Services.
redirect_uri	The callback endpoint in your application that must receive the authorization code. It must be one of the `redirect_uris` defined in your OAuth 2.0 client registration. When sending the `redirect_uri` as a URL parameter it must be URL-encoded.
state	A random string that your application generates and that is sent back as a parameter during the URI redirect. Serves as a security mechanism as per the OAuth 2.0 specification.
response_type	Specifies what must be included in the authorization response. For the authorization code grant type, use *code*.

   b) Send the query request to the VMware Identity Broker - vCenter`Authorize` endpoint.

      For example:

```
https://<vcenter_server_fqdn>/acs/t/<tenant>/authorize?
client_id=<my_app_client_id>&redirect_uri=https://
<my_app_redirect_uri1>&state=<generated_random_string>&response_type=code
```

      VMware Identity Broker - vCenter opens a dialog box asking the user to authenticate.
   c) The user authenticates to the identity provider.

      The authentication mechanism depends on the specific user configuration on the extern?l identity provider website. It could be basic authentication, biometrics, multifactor authentication, and so on.

      The authorization server directs the user to the client app's redirect URI with the authorization code in the query string.

2. Obtain an access and ID token.

   Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint containing the required grant type, the authorization code, the redirect URI, and your app's client ID and secret.

   In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (*my_app_client_id*:*my_app_client_secret*).

   The content type must be *x-www-form-urlencoded*.

   Example request:

```
curl -- location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=authorization_code' \
--data-urlencode 'code=<my_app_authorization_code>' \
--data-urlencode 'redirect_uri=https://<my_app_redirect_uri1>'
```

   The authorization server returns the JWT tokens corresponding to the grant type: access token , ID token, and refresh token.

Exchange the JWT tokens for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Obtain JWT Token from VMware Identity Broker - vCenter by Using the Client Credentials Grant Type
If your vCenter, app, or script is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access token in JWT format by using the OAuth Client Credentials grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.

- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id and secret for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *CUSTOMER*.

You can use the Client Credentials grant type if you want to authenticate vCenter or another application to your external identity provider without the presence of a human user. The Client Credentials grant type is appropriate for headless automation when you want a web service to authenticate automatically against an identity provider.

Obtain an access token for your app.

Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint with the required grant type.

In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (`client_id:secret`).

The content type must be *x-www-form-urlencoded*.

Example request:

```
curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=client_credentials'
```

The authorization server returns the JWT token corresponding to the grant type: access token.

Exchange the JWT token for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier

You can exchange the JWT tokens that you obtained from the authorization server for a vCenter SAML token and create an authenticated session.

- You must have a valid access and ID token (in JWT format) from the authorization server (AD FS or VMware Identity Broker - vCenter) for the user or application you want to authenticate.

1. Call the `Authentication Token` service with the JWT token(s) you obtained.

   For example,

   ```
   curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/authentication/
   token' \
   --header 'Authorization: Bearer <ACCESS_TOKEN>' \
   --header 'Content-Type: application/x-www-form-urlencoded' \
   --data-urlencode 'subject_token_type=urn:ietf:params:oauth:token-type:access_token' \
   --data-urlencode "subject_token=<ACCESS_TOKEN>" \
   --data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
   --data-urlencode "actor_token=<ID_TOKEN>" \
   --data-urlencode 'grant_type=urn:ietf:params:oauth:grant-type:token-exchange' \
   --data-urlencode 'requested_token_type=urn:ietf:params:oauth:token-type:saml2'
   ```

   On success (code 200), vCenter Single Sign-On returns an `Oauth2TokenInfo` data structure containing the SAML token.

2. Send the JWT tokens to vCenter Single Sign-On by using the `Token` interface of the `com.vmware.vcenter.authentication` package.

   a) Create an object of type `TokenTypes.IssueSpec` with the following modifiers.

Method	Value
`setSubjectTokenType`	*urn:ietf:params:oauth:token-type:access_token*
`setSubjectToken`	*<ACCESS_TOKEN>*
`setActorTokenType`	*urn:ietf:params:oauth:token-type:id_token*
`setActorToken`	*<ID_TOKEN>*

Method	Value
`setGrantType`	*`urn:ietf:params:oauth:grant-type:token-exchange`*
`setRequestedTokenType`	*`urn:ietf:params:oauth:token-type:saml2`*

b) Call the `issue(TokenTypes.IssueSpec)` method

On success, vCenter Single Sign-On returns a `TokenInfo` object containing the SAML token.

3. Prepare the SAML token for the API call.

   Base-64 decode the SAML token, compress it, and then Base-64 encode it again.

4. To obtain a session identifier, call the `session.create()` methodSession service with the SAML token.

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/session' \
--header 'Authorization: SIGN token=<COMP_TOKEN>'
```

   On success (code 201), the VMware Cloud Foundation endpoint returns a session identifier in the response. On success, the VMware Cloud Foundation API returns a session identifier.

5. To authenticate, use the session identifier in subsequent API calls to the VMware Cloud Foundation API endpoints. To authenticate, add the session identifier to your security context instance.

   For example:

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/cis/tagging/category' \
--header 'vmware-api-session-id: <session_ID>'
```

You created an authenticated session and now you can access and use the VMware Cloud Foundation services.

Bash Shell

This Bash script illustrates the use case where you get JWT tokens from the authorization server (AD FS or VMware Identity Broker - vCenter), exchange them for a vCenter SAML token, and obtain an authentication session identifier from the VMware Cloud Foundation endpoint.

This script consists of three parts:

1. Obtain JWT tokens from the authorization server (AD FS or VMware Identity Broker - vCenter) by using the OAuth 2.0 Password grant type.
2. Exchange the JWT tokens for a vCenter SAML token.
3. Use the SAML token to obtain a session identifier for the VMware Cloud Foundation API.

```
#!/bin/bash

: '
Variable definitions:
$vcip = The IP address or FQDN of your vCenter
$ACCESS_TOKEN = The access token in JWT format that you received from the
 authentication server.
$ID_TOKEN = The ID token in JWT format that you received from the authentication
 server.
'
if [ -z "$1" ] || [ -z "$2" ] || [ -z "$3" ] || [ -z "$4" ] || [ -z "$5" ] || [ -z
 "$6" ]; then
  echo "Usage: <vc-ip> <pwgrant-userid> <pwgrant-password> <client-id> <client-secret>
 <token-endpoint>"
  exit 0
fi

vcip="$1"
userid="$2"
password="$3"
clientid="$4"
clientsecret="$5"
tokenendpoint="$6"
```

```
echo "Obtaining JWT access and ID tokens for user $userid ..."

PWGRANT_OUTPUT=$(curl -k --silent --location -u "$clientid:$clientsecret" --request
 POST "$tokenendpoint" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=password" \
  --data-urlencode "username=$userid" \
  --data-urlencode "password=$password")
ACCESS_TOKEN=$(echo $PWGRANT_OUTPUT | jq -r '.access_token')
ID_TOKEN=$(echo $PWGRANT_OUTPUT | jq -r '.id_token')

echo
echo "Access token: $ACCESS_TOKEN"
echo
echo "ID token: $ID_TOKEN"
echo

echo "Exchanging JWT tokens for vCenter SAML token ..."

TOKEN_EXCHANGE_OUTPUT=$(curl -k --silent --location --request POST "https://$vcip/api/
vcenter/authentication/token" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --header "Authorization: Bearer $ACCESS_TOKEN" \
  --data-urlencode "subject_token_type=urn:ietf:params:oauth:token-type:access_token"
 \
  --data-urlencode "subject_token=$ACCESS_TOKEN" \
  --data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
  --data-urlencode "actor_token=$ID_TOKEN" \
  --data-urlencode "grant_type=urn:ietf:params:oauth:grant-type:token-exchange" \
  --data-urlencode "requested_token_type=urn:ietf:params:oauth:token-type:saml2")

echo
echo "$TOKEN_EXCHANGE_OUTPUT"
echo

SAML_TOKEN=$(echo $TOKEN_EXCHANGE_OUTPUT | jq -r '.access_token')

echo "vCenter SAML token: $SAML_TOKEN"
echo

echo "Establishing vCenter session with SAML token ${SAML_TOKEN::7}..."
echo
COMP_TOKEN=$(echo $SAML_TOKEN | base64 -d | gzip | base64 -w0)

SESSION_OUTPUT=$(curl -k --silent --location --request POST "https://$vcip/api/
session" \
  --header "Authorization: SIGN token=\"$COMP_TOKEN\"")

echo "Create Session Response: $SESSION_OUTPUT"

SESSION_ID=$(echo "$SESSION_OUTPUT" | tr -d '"')

echo "Tagging categories:"
curl -k "https://$vcip/api/cis/tagging/category" --header "vmware-api-session-id:
 $SESSION_ID"

echo
```

```
echo "Done!"
```

ESX Hosts

Use the VMware Cloud Foundation APIs to run general operations on the ESX hosts in your vSphere environment.

You can retrieve information about the hosts, create a standalone host, disconnect, and reconnect an ESX host to a vCenter system.

How to Retrieve Information About ESX Hosts

You retrieve information about the ESX hosts running in a vCenter instance by listing only the ESX hosts that you are interested in.

To filter the ESX hosts on a vCenter instance and get only the ones you want, call the `list` function and pass your criteria with a `HostTypes.FilterSpec` instance.use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request and specify your filter criteria through the query parameters. Combine several filter criteria by including one or more of the following parameters:

- The name or unique identifier of the host.
- Clusters, data centers, or folders that contain the host.
- Connection state of the host which can be one of the following: `CONNECTED`, `DISCONNECTED`, or `NOT_RESPONDING`.
- Power state of the host which can be one of the following: `POWERED_OFF`, `POWERED_ON`, or `STANDBY`.

The function returns a list of `com.vmware.vcenter.HostTypes.Summary` objects that contain information about up to 2500 hosts that match all specified criteria. You can use the list to retrieve information about the returned ESX hosts.

The request returns a `HostSummary` JSON object for each host that matches your filter. You can view information for up to 2500 hosts.

How to Add a Standalone ESX Host to vCenter

You can use the VMware Cloud FoundationREST APIs to add a standalone host to a vCenter instance.

Add a single ESX host to a vCenter instance by calling the `create` function and passing a `com.vmware.vcenter.HostTypes.CreateSpec` instance as parameter. When you create the host specification, make sure that you set the IP address or the DNS resolvable host name and the administrator credentials.

Add a single ESX host to a vCenter instance by using the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` request and submitting a `HostCreateSpec` JSON object in the request body. When you construct the host object, make sure that you set values for the IP address or the DNS resolvable host name and the administrator credentials.

How to Disconnect and Reconnect ESX Hosts

You can use the VMware Cloud Foundation APIs to connect ESX hosts to a vCenter instance and make the hosts managed. You can temporarily disconnect a managed host from a vCenter instance and reconnect the host, for example, to refresh the ESX agents on the host.

When you add a host to a vCenter instance, the host is connected to vCenter and becomes a managed host. To disconnect a managed host from a vCenter instance, call the `disconnect` function and pass the host identifier as a parameteruse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>?action=disconnect` request and submit the host identifier as a path parameter. The managed host and its associated virtual machines remain in the inventory but vCenter temporarily stops managing and monitoring them.

To reconnect a managed host to a vCenter instance, call the `connect` function and pass the host identifier as a parameteruse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>?`

`action=connect` request and submit the host identifier as a path parameter. As a result, the connection status of the host changes, and vCenter resumes managing the host and its associated virtual machines.

If you want to delete a host and all its associated virtual machines from the inventory, you can remove the host from the vCenter instance. To delete a disconnected host from a vCenter instance, call the `delete` function and pass the host identifier as a parameteruse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>` and submit the host identifier as a path parameter.

How to Configure ESX Entropy

You can use the VMware Cloud Foundation API to feed external entropy data to an ESX host in your inventory. You can also query the current entropy levels on the host and add external entropy data when needed.

Entropy is a measure of the randomness or diversity of a data-generating function. In releases prior to vSphere 8.0 Update 1, ESX supported only CPU-based entropy data generated from interrupts or manufacturer provided interfaces, such as RDSEED and RDRAND. High-quality entropy is important for the proper functioning of security-related operations such as generating encryption keys for secure communication over the network. Starting with vSphere 8.0 Update 1, you can add external entropy sources to an ESX host and in this way ensure the high quality of the entropy data on that host. You provide external entropy data by using devices such as hardware security modules (HSMs) which are FIPS 140-3 and EAL4 certified.

You can configure the ESX entropy sources by using the VMkernel boot options. To use external entropy sources, set the `entropySources` value to more than or equal to `8`. For more information about how to set the desired entropy sources by using the VMkernel boot options, see Controlling ESX Entropy in the *vSphere Security* documentation.

You can also configure external entropy sources in the kickstart file for the ESX scripted installation. See Configuring External Entropy Sources During Scripted Installation.

> **Note:**
>
> If a host is configured to use only external entropy sources, that is, `entropySources` is set to `8`, you must keep supplying the external entropy data through the VMware Cloud Foundation API. In case the entropy in the host gets exhausted, the host becomes unresponsive and might require a hard reboot or re-installation to recover the host from this situation.

Querying Entropy Data on a Host

To retrieve details about the external entropy available on an ESX host, use the `ExternalPool` service. You must have the **Host.Entropy.Read** privilege.

You can check whether an external entropy source is added to a host by calling the `get(host_id)` method of the `com.vmware.vcenter.host.entropy.ExternalPool` interface and passing the host ID as parameter. The method returns an `ExternalPool.Info` instance that contains detailed information about the external entropy data on the hostusing the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>/entropy/external-pool` HTTP request and passing the host ID as a path parameter. The operation returns an `ExternalPool.Info` JSON object in the response body that contains detailed information about the external entropy data on the host.

Table 1855: Details for the External Entropy Data on a Host

External Entropy Data Detail Property	Description
`getStatus() status`	Indicates whether an external entropy source is added for a host.
`getCapacity() capacity`	Shows the maximum capacity of external entropy data in bytes that a host can store in the VMkernel entropy pool. When you feed the host with additional external entropy data, make sure that you do not exceed this maximum capacity. Otherwise, all extra entropy data is discarded.
`getCurrentlyAvailable() currently_available`	Indicates the current amount of entropy data in bytes available in the VMkernel entropy pool on the host.

External Entropy Data Detail Property	Description
`getInactiveSourceTimeout() inactive_source_timeout`	Indicates the amount of time in seconds that can elapse without any activity between the host and the external entropy source. Your application must check periodically the levels of entropy data on the host and send entropy data from the external source when required. When this timeout exceeds an `esx.audit.entropy.external.source.disconnected` VMkernel Observation (VOB) is logged. This system event suggests possible loss of connection to the external entropy source. For the full list of available entropy VOBs in vSphere 8.0, see the VMware knowledge base article at https://kb.vmware.com/s/article/89074.
`getLowWatermark() low_watermark`	Indicates the threshold in bits sufficient for the in-memory cache to operate successfully. When the in-memory entropy cache is running low and the threshold is reached, an `esx.audit.entropy.available.low` VOB is logged. For more information about the entropy VOBs, see the VMware knowledge base article at https://kb.vmware.com/s/article/89074.

Adding External Entropy Data to a Host

To add entropy data from an external entropy source to a host, you must have the **Host.Entropy.Write** privilege. Use the `ExternalPool` service and call the `add(host_id, add_spec)` method. Pass the host ID and an `ExternalPool.AddSpec` instance as method parameters. The `ExternalPool.AddSpec` instance contains the Base64 encoded external entropy data. You must convert the binary data coming from the external entropy source to Base64 format so that the host can consume it. As a result, the method returns an instance of the `ExternalPool.AddResult` class.

To add entropy data from an external entropy source to a host, you must have the **Host.Entropy.Write** privilege. Use the `ExternalPool` service and send the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>/entropy/external-pool?action=add` request. Pass the host ID as a path parameter and an `ExternalPool.AddSpec` data structure in the request body. The `ExternalPool.AddSpec` JSON object contains the Base64 encoded external entropy data. You must convert the binary data coming from the external entropy source to Base64 format so that the host can consume it. The request returns an `ExternalPool.AddResult` JSON object in the response body, if the request succeeds.

When the external entropy data reaches the host, the entropy daemon stores it first in the in-memory cache and the storage cache. Then the external entropy data is pushed to the VMkernel entropy pool from which it is fed to the applications in need.

The entropy data in the storage cache persists on the host disk and is only consumed during the ESX booting. The default storage file size is 4MiB and can be configured through the ESXCLI commands. See the *ESXCLI Command Reference* documentation.

Managing the Life Cycle of Hosts and Clusters

You can manage the life cycle of ESX hosts collectively by using the vSphere Lifecycle Manager feature through the VMware Cloud Foundation API. Starting with vSphere 8.0, you can manage the lifecycle of a standalone ESX host by using an image only through the vSphere Lifecycle Manager automation API.

You can automate the life cycle management of a cluster or standalone host by performing the following operations:

- Retrieve information about the current state of the cluster or the standalone host.
- Create a desired state that includes a specific version of the ESX host. You can also add some compatible partner software and firmware components and add-ons.
- Validate the desired state to detect any discrepancies between the desired state and the host hardware.
- Check the compliance of a cluster or host against the desired state and determine whether some additional steps must be taken to ensure the success of the cluster or host remediation.
- Apply the desired state on a cluster or a standalone host.

You can use the vSphere Lifecycle Manager to manage the life cycle of hosts in a cluster that meet the following requirements:

- Hosts must be of version 7.0 and later.
- Hosts must be stateful.
- All hosts in the cluster must be from the same vendor and with identical hardware.
- The hosts must include only integrated solutions, such as VMware vSAN™, vSphere Supervisor, NSX and VMware vSphere® High Availability.

A standalone host is a host that is added to a vCenter instance but is not part of any cluster. For more information about how to add, connect, and disconnect standalone host, see ESX Hosts. You can manage the life cycle of a standalone host by performing almost all vSphere Lifecycle Manager operations that you can perform on a cluster level. The only limitation for managing the life cycle of a standalone host through the VMware Cloud Foundation API, is that you can't update the firmware of the host.

vSphere Lifecycle Manager Terms

You must understand the basic terminology that is used within this chapter to be able to use the vSphere Lifecycle Manager functionality efficiently.

vSphere Lifecycle Manager Terminology

Term	Definition
Upgrade, update, and patch	You can upgrade to another major version of the software running on an ESX host, and install patches and updates that include smaller changes, bug fixes, or other small improvements.
Depot	A depot is a well-defined folder structure that is used for distributing payloads and their metadata. Depots are consumed by different products and features such as the vSphere Lifecycle Manager and ESXCLI. The vSphere Lifecycle Manager works with three types of depots: online, offline, and UMDS. See Software Depots.
Component	A component is the smallest unit that the vSphere Lifecycle Manager uses during the installation and update processes. Software vendors use components to encapsulate a group of payloads that can be managed together.
Base image	A base image is a collection of components that shape the bootable ESX used for the installation or upgrade process. Base images are currently distributed only by VMware and support x86 servers. VMware provides new versions of the base image for each upgrade, update, and patch release of the ESX. Base images are hosted at the VMware online depot that is available by default to the vSphere Lifecycle Manager. Furthermore, you can download a different base image version, in the form of an offline ZIP bundle, from https://my.vmware.com/web/vmware/downloads.
Vendor add-on	An add-on is a collection of components that different OEMs provide on top of a base image. Vendors use an add-on to group some customizations for a family of servers. Unlike base images, add-ons are not complete and are not sufficient to boot an ESX. Each add-on must have a unique name and version. An add-on allows vendors to add, remove, or update components that are part of the VMware base image, if there are no unresolved dependencies and conflicts between the components.

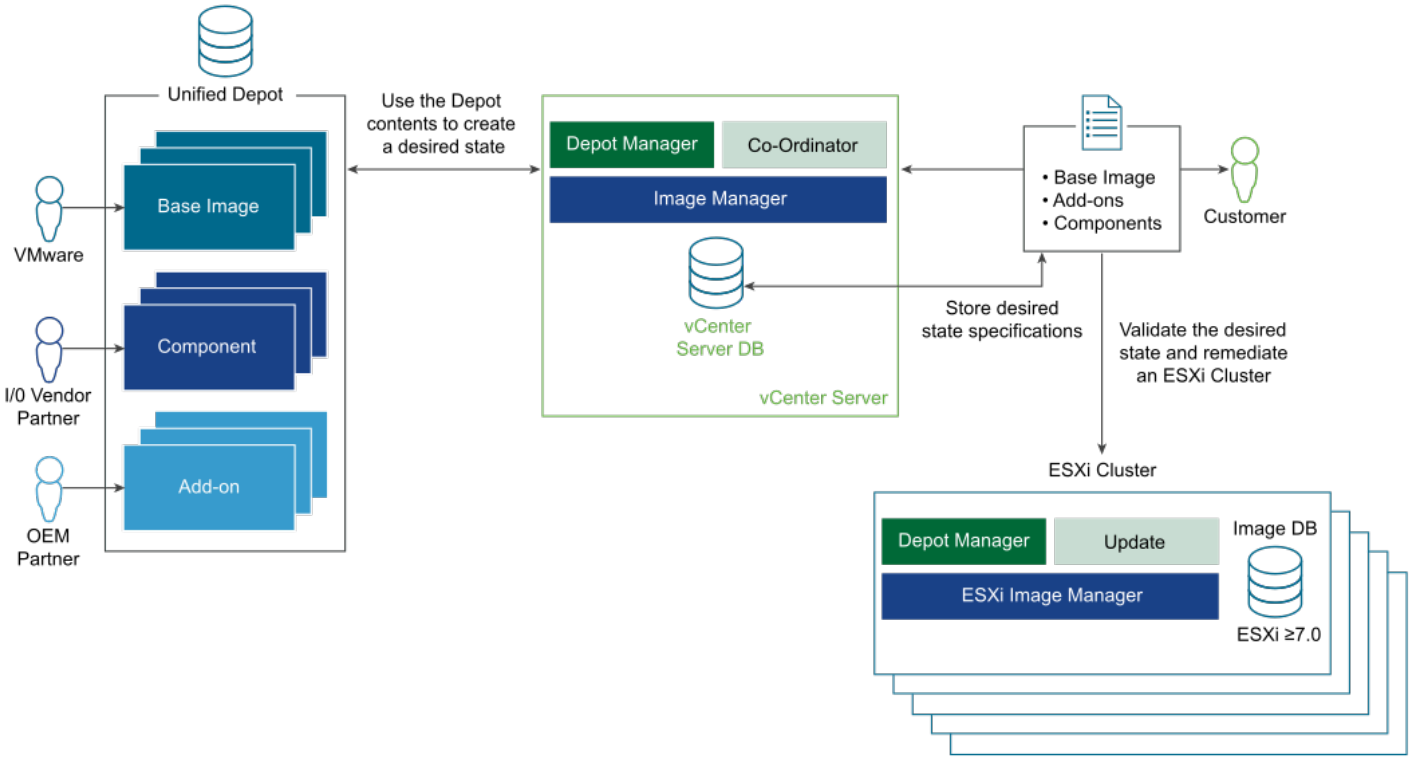
Term	Definition
Solution	A solution contains one or more components, and provides information about its constraints and compatibility with the different ESX versions. For example, from the perspective of the vSphere Lifecycle Manager solutions are VMware NSX, VMware vSphere® High Availability, vSAN.
Desired state	A desired state of a cluster is represented with a software specification. The desired state defines a set of components that a user wants to install on a single ESX host or on a cluster of hosts.
OEMs	Original Equipment Manufacturers. VMware partners enrolled in the VMware Partner Connect application, such as Dell, Inc., HP Inc., Lenovo Group Ltd., and so on.
IOVP	I/O Vendor Partner. Qualified VMware partners providing certified I/O device drivers for network and storage host bus adapters.
Third-party software providers	Providers of I/O filters, device drivers, CIM modules, and so on, that are not part of VMware partner programs.
vSphere Configuration Profiles	With vSphere Configuration Profiles, you can manage the configuration of all hosts in a cluster collectively. This ensures consistency in host configuration at a cluster level. You can enable vSphere Configuration Profiles only on clusters that have the vSphere Lifecycle Manager enabled.

vSphere Lifecycle Manager Overview

The vSphere Lifecycle Manager feature provides means for managing the life cycle of hosts on a cluster level. The functionality of this feature is achieved through several major services that are running on both vCenter and ESX.

After you install vSphere 8.0 Update 1, you can access the feature through several major services available on vCenter and on each ESX host.

Figure 549: vSphere Lifecycle Manager System Architecture



Depot Manager

For each upgrade and patch release, VMware, OEMs, and other third-party companies make the software updates of their products available to the customers. Software updates are distributed to different locations and in different formats depending on the way they are accessed, downloaded, and used. Depot Manager allows these different software resources to be presented to the vSphere customers in a unified format and as a result, makes them easier to use.

Depending on your environment and specific use case, Depot Manager gives you access to the software updates within three different types of depots: online, offline, and UMDS. See Software Depots.

Depot Manager that runs on the vCenter instance achieves the following goals:

- Represents the contents of all depots in a unified way.
- Caches the payloads and their metadata locally prior to their use.
- Enables depot overrides on a cluster or host level for Remote Office/Branch Office (ROBO) environments, or Edge computing environments.

Depot Manager that runs on the hosts serves as a proxy to the vCenter Depot Manager. If ROBO cluster or ROBO host, or Edge computing environments are enabled, Depot Manager that runs on the ESX hosts serves as a proxy to the nearby vSphere Lifecycle Manager compatible depot.

Image Manager

Image Manager allows you to create a desired state that you can apply on a cluster or a standalone host. The specification describes all components, add-ons, and the base image that you can use to update or upgrade the hosts in your environment. Image Manager supports validation of the desired state and the detecting drifts from the desired state.

Coordinator and ESX Updater Managers

Coordinator Manager runs on the vCenter instance and makes sure that the desired state is applied to all hosts in the cluster. This module also runs pre-checks to evaluate how each host in the cluster is affected by the remediation and

whether you must take some additional steps to ensure the success of the procedure. Coordinator Manager also allows you to query the status of the remediation operation for the cluster and for each host part of the cluster.

The ESX Updater Manager takes care of the actual remediation happening on each host.

Options for Managing the ESX Life Cycle

Based on your needs and environment setup, you can choose from several methods for managing the life cycle of the ESX hosts. The vSphere Lifecycle Manager provides means for updating all hosts in a cluster or a standalone host with a desired software state.

To manage the life cycle of the hosts in your environment, you can use the vSphere Lifecycle Manager through the vSphere Client. See Managing Host and Cluster Lifecycle.

This chapter of the VMware Cloud Foundation SDKsREST API Programming Guide discusses how you can access and use the functionality provided by the vSphere Lifecycle Manager Automation API.

vSphere Lifecycle Manager Features

You can use the VMware Cloud Foundation API to manage the life cycle of all hosts in a cluster or of standalone hosts by using a vSphere Lifecycle Manager image. You can access and use the following vSphere Lifecycle Manager functionality:

- Depot management. You can add, remove, explore the contents of different types of depots. See Software Depots. The content of the depots is provided by VMware and VMware partners. Partners can use the ESX Packaging Kit (EPK) to assemble a custom bootable ESX image. The custom image can then be shared to other third-party customers and used through the Depot Manager service. For more information about how to create custom ESX images, see *ESX Packaging Kit (EPK) Development Guide*.
- Desired software state. You can create, edit, and delete a desired software state for a cluster or a standalone host on which the vSphere Lifecycle Manager is enabled. A desired software state must contain at least a single ESX image provided by VMware. You can also set an add-on provided by OEMs, and one or more components by different software vendors. Furthermore, during the process of creating the desired software state, you can check the validity of the specification and compare the current state of the hosts in the cluster or the standalone host with the desired software state.
- Cluster remediation. You can apply the desired state on each of the hosts in a cluster which current state is different from the desired specification. Applying a desired state on a cluster level has the following prerequisites:
  - The cluster must have the vSphere Lifecycle Manager enabled.
  - All hosts in the cluster must store their data on a local or remote disk, or on a USB drive.
  - All hosts in the cluster must be of version 7.0 or higher.
  - All hosts must contain only components that the vSphere Lifecycle Manager can recognize and maintain. If a host contains some old content that the vSphere Lifecycle Manager does not recognize, the content is removed from the host during remediation.
- Standalone host remediation. You can apply the desired software state on a standalone host that is managed with baselines or on a standalone host that is managed with images but its current state differs from the desired state.

Required Privileges for Using vSphere Lifecycle Manager and vSphere Configuration Profiles

To configure vSphere Lifecycle Manager settings and to use successfully vSphere Lifecycle Manager baselines and images, you must have the proper privileges. Similarly, you need the proper privileges to use vSphere Configuration Profiles.

You can assign vSphere Lifecycle Manager and vSphere Configuration Profiles privileges to different roles in the vSphere Client.

vSphere Lifecycle Manager Privileges For Using Images

When you use vSphere Lifecycle Manager images, you need a different set of privileges for each task.

Table 1856: VMware vSphere vSphere Lifecycle Manager Privileges For Using Images

Task	Privilege in the vSphere Client	Privilege in the API
Set Up Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Import Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Upload File > Upload upgrade images and offline bundles**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read` • `VcIntegrity.FileUpload.com.vmware.vcIntegrity.ImportFile`
Export Image	**VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read**	`VcIntegrity.lifecycleSoftwareSpecification.Read`
Edit Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Work with Recommendations	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Work with Depot	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Write**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write`
Manage Depot Overrides	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Write**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write`

Task	Privilege in the vSphere Client	Privilege in the API
Check Compliance	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Read**	`VcIntegrity.lifecycleSoftwareSpecification.Read`
Run Remediation Pre-Check	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: General Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **ESXi Health Perspectives** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Read**	• `VcIntegrity.lifecycleGeneral.Read` • `VcIntegrity.lifecycleHealth.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Read`
Remediate Against an Image	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: General Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **ESXi Health Perspectives** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Write** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read**	• `VcIntegrity.lifecycleGeneral.Read` • `VcIntegrity.lifecycleHealth.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Write` • `VcIntegrity.lifecycleSettings.Read`
Edit Remediation Settings	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Write**	• `VcIntegrity.lifecycleSettings.Read` • `VcIntegrity.lifecycleSettings.Write`
Update Firmware	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Write** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Write`
View Hardware Compatibility	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Hardware Compatibility Privileges** > **Access Hardware Compatibility**	`VcIntegrity.HardwareCompatibility.Read`
Stage an Image	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Write**	`VcIntegrity.lifecycleSoftwareRemediation.Write`

For more information about managing users, groups, roles, and permissions, see the *vSphere Security* documentation.

Required Privileges for Using vSphere Configuration Profiles

When you use vSphere Configuration Profiles, you need a different set of privileges for each task.

Table 1857: Required Privileges For Using vSphere Configuration Profiles

Task	Privilege in the vSphere Client	Privilege in the API	Description
Create Cluster with vSphere Configuration Profiles enabled	**Host** > **Inventory** > **CreateCluster**	`Host.Inventory.CreateCluster`	Create a cluster along with its initial configuration.
View Host Settings in the Cluster Configuration	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Read-only access to desired configuration management platform**	`VcIntegrity.ClusterConfiguration.View`	View the draft configuration document, current configuration settings, compliance reports, and pre-check reports.
Check Cluster Compliance			
View Compliance Results			
Run Remediation Pre-Check			
View Remediation Pre-Check Results			
View Draft Pre-Check Results			
Remediate a Cluster Against the Desired Cluster Configuration	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Remediate cluster to the desired configuration.**	`VcIntegrity.ClusterConfiguration.Remediate`	Remediate a cluster against the draft configuration or transition a cluster to the desired configuration.
Transition to vSphere Configuration Profiles			
Export a Configuration or Configuration Schema	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Export desired cluster configuration**	`VcIntegrity.ClusterConfiguration.Export`	Export the desired cluster configuration document.
Extract a Configuration Document from a Reference Host	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Modify desired cluster configuration.**	`VcIntegrity.ClusterConfiguration.Modify`	Modify the desired cluster configuration by crating a draft configuration, extracting a configuration document from a reference host, or importing a configuration.
Create a Draft Configuration for a Cluster			
Import a Configuration			

Software Depots

A software depot represents a well-defined file structure used for storing and hosting the ESX software updates, patches, and upgrades that VMware, partners, and third-party vendors provide. You can use the VMware Cloud Foundation APIs to manage the life cycle of the hosts in your environment by applying software updates hosted on different depots.

Software depots are managed by the Depot Manager which is part of the vSphere Lifecycle Manager. Software depots contain the actual payloads and the metadata of the software updates. Depending on the way you access the software updates, the Depot Manager recognizes three types of software depots: online, offline, and UMDS.

The following section of the documentation explains the concept of a software depot in terms of the vSphere Lifecycle Manager feature. You can also find common use cases available through the APIs for working with the different types of depots and their content.

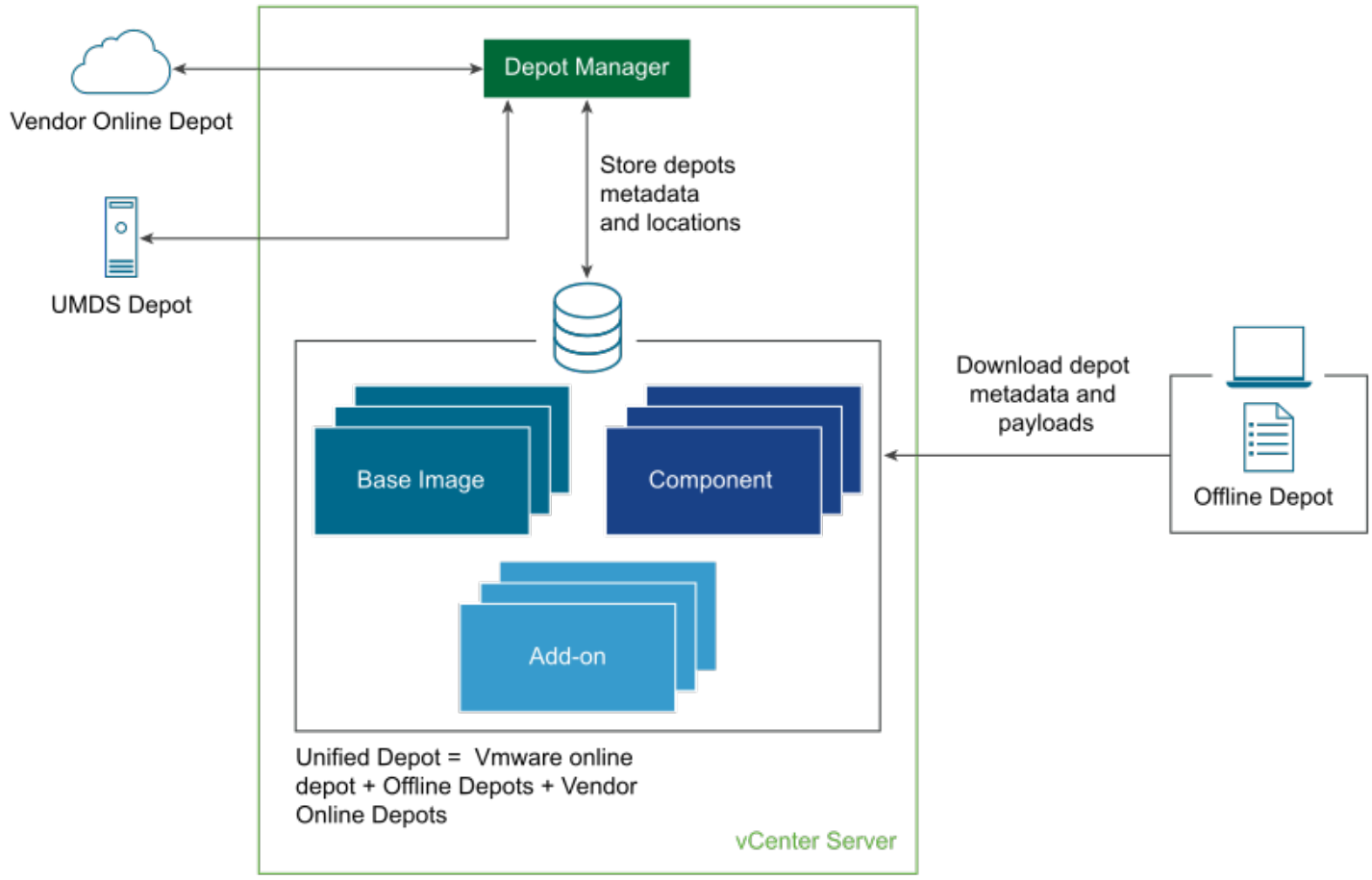
Types of Software Depots

Depot Manager works with the software updates provided by three different types of software depots: online, offline, and UMDS.

Regardless of the different way in which you access each type of software depot, all depots have the same structure. The same depot structure allows content from different vendors to be uploaded to one depot. Partners and third-party customers can use the ESX Packaging Kit to build and distribute software updates in the form of offline or online depots.

You can access their content by adding the online vendor depot to the vSphere Lifecycle Manager or by downloading the content of the offline depot to the vCenter instance.

Figure 550: Types of Software Depots



Online Depot

VMware and partners upload software updates to the VMware online depot at `https://hostupdate.vmware.com/software/VUM/PRODUCTION/...` or to a custom online depot. Software updates can be patches of the ESX base image, different versions of the partner add-ons, the IOVP drivers certified by VMware, and the VMware Tools™ updates. Online depots are accessible through a URL. By default, you can see the base images, add-ons, and components provided within the VMware online depot at the following locations:

- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/addon-main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/iovp-main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/vmtools-main/vmw-depot-index.xml`

When you deploy the vCenter, the vSphere Lifecycle Manager is configured to access the VMware online depot, by default. You can use the VMware Cloud Foundation APIs to add a custom online depot to be managed by Depot Manager. The metadata of the newly added online depot is not synchronized immediately. To synchronize the metadata, you can run a synchronization operation or wait for the scheduled synchronization to take place.

The Depot Manager stores in the vCenter database only the metadata of the software updates and the location of the added online depots. You can create a schedule to synchronize the software updates metadata stored in the vCenter with the metadata available in the accessible depots. The payloads of the software updates are downloaded only during the cluster remediation process.

To add, remove, list, and retrieve information about the online depots, you can use the `com.vmware.esx.settings.depots.Online` interface service. See Working with Online Depots.

Offline Depot

The offline depot is also called an offline bundle and is distributed as a downloadable ZIP file. Offline depots contain both the metadata and the payloads of the software update. Partners and third-party customers can use the ESX Packaging Kit to build and distribute offline bundles. You can download offline bundles from the VMware website or from the websites of third-party vendors. When you add an offline depot to the vSphere Lifecycle Manager depot, the software updates are downloaded to the vCenter database.

To manage offline depots, you can use the `com.vmware.esx.settings.depots.Offline` interface service. See Working with Offline Depots.

UMDS Depot

In case, the vCenter instance is in an air-gapped environment and has no access to any wire or wireless network, you can use a UMDS depot. The Update Manager Download Service (UMDS) is available as a `VMware-UMDS-8.0.1.-build_number.tar.gz` file within the ISO image of the vCenter appliance 8.0 UMDS is a 64-bit application and requires a 64-bit Linux-based system. Install UMDS on a machine that has Internet access and is different from the machine on which the vSphere Lifecycle Manager is running. For further information about how to install and configure the UMDS module, see the *Managing Host and Cluster Lifecycle*.

You can set up a synchronization schedule for downloading specific software updates from online vendor depots to the UMDS depot. Then use these updates to create desired software state for the clusters in your environment.

To manage UMDS depots through the VMware Cloud Foundation API, you can use the `com.vmware.esx.settings.depots.Umds` interface `UMDS` service.

Working with Online Depots

You can use the VMware Cloud Foundation APIs to add online depots to the list of currently configured online software depots.

Use online depots to add new content over time to the management scope of Depot Manager. Depot Manager periodically updates the software depots metadata stored on the vCenter instance. In case, new software updates are uploaded to the online depots, the Depot Manager makes sure that the metadata stored on the vCenter database is updated accordingly.

To add an online depot to the Depot Manager, you must first create the online depot specification by using the `com.vmware.esx.settings.depots.OnlineTypes.CreateSpec` class`Online.CreateSpec` data structure. To specify the URI to the `vendor-index.xml` file of the online depot, use the `setLocation(location)` method of the `OnlineTypes.CreateSpec` classuse the `location` property of the `Online.CreateSpec` data structure. Optionally, you can add a description and enable the depot. By default, when you add an online depot to the Depot Manager, the depot is enabled and its metadata is synchronized following the defined schedule. If you want to synchronize the added online depot immediately, call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request. When you complete the depot specification, call the `create(spec)` method of the `com.vmware.esx.settings.depots.Online` interface to add the depotuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online` request and submit the `Online.CreateSpec` JSON object to add the depot.

You can edit the depot description and deactivate the depot by creating an `com.vmware.esx.settings.depots.OnlineTypes.UpdateSpec` object and pass it to the `update( depot, update_spec)` method of the `com.vmware.esx.settings.depots.Online` interfaceusing the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>` request and submitting the depot ID as path parameter and an `Online.UpdateSpec` JSON object in the request body.

You can remove an online depot from the list of currently configured depots by using the `delete(depotID)` method of the `com.vmware.esx.settings.depots.Online` interface`DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>?vmw-task=true` request and submitting the depot ID as a pat parameter. This operation does not remove the already downloaded metadata and payloads from the deleted depot. You cannot delete the default VMware online depot, you can only deactivate it.

To retrieve a list of currently configured online depots, call the `list()` method of the `Online` interfaceuse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online` request. You can also retrieve

information about a currently configured online depot by using the `get(depotID)` method of the `Online` interface`GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>` request and submitting the depot ID as path parameter.

Working with UMDS Depots

In an air-gapped vCenter environment, you can use the VMware Cloud FoundationREST API to add a UMDS depot to the depots managed by Depot Manager.

After you install and configure the Update Manager Download Service (UMDS) on a physical machine with Internet access, you can add the UMDS depot to Depot Manager. Only one UMDS depot can be added at a time to Depot Manager. When you add a UMDS depot, its content is not immediately synchronized. To synchronize the content of the UMDS depots, you must call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request or wait for the scheduled synchronization to take place.

To add a UMDS depot, call the `set(set_spec)` method of the `com.vmware.esx.settings.depots.Umds` interface and pass a `com.vmware.esx.settings.depots.UmdsTypes.SetSpec` object as an argumentuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request and submit a `Umds.SetSpec` JSON object in the request body.. The UMDS specification must contain the URI location to the `index.xml` file of the depot. Optionally, you can set a description and indicate whether the depot must be enabled. By enabling the UMDS depot, you instruct Depot Manager to synchronize only the content that is available on that depot.

You can always edit the initial UMDS depot settings, by calling the `update(update_spec)` method of the `Umds` interface and passing an `com.vmware.esx.settings.depots.UmdsTypes.UpdateSpec` object.using the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request and submit a `Umds.UpdateSpec` JSON object in the request body.

To retrieve information about the currently configured UMDS depot, use the `get()` method of the `Umds` interface`GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request. You can remove a currently configured UMDS depot and all its downloaded content from Depot Manager by calling the `delete_Task()` method of the `com.vmware.esx.settings.depots.Umds` interface.using the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds?vmw-task=true` request.

Synchronizing Software Depots

The VMware online depot, the vendor online depots, and the UMDS depot must be synchronized regularly if you want to have the most recent software updates delivered by VMware, partners, and other third-party vendors. Use the VMware Cloud Foundation APIs to create a synchronization schedule or to synchronize the added depot immediately.

The Depot Manager does not synchronize immediately the metadata of the newly added online and UMDS depots. If you want to force the synchronization and not wait for the scheduled synchronization to take place, call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request. . You can also define a custom schedule to sync the metadata from the currently configured online or UMDS depots.

To create a custom schedule for checking for new software updates, you must first define the schedule parameters by using the `com.vmware.esx.settings.depots.SyncScheduleTypes.Schedule` class`SyncSchedule.Schedule` data structure.. Then you can add the schedule to the schedule specification by using the `setSchedule(schedule)` method of the `com.vmware.esx.settings.depots.SyncScheduleTypes.Spec` class`SyncSchedule.Spec` data structure.. Optionally, you can use the schedule specification to add an email to which notifications will be sent and define whether updates will be downloaded automatically. To apply the custom schedule, call the `set(spec)` method of the `com.vmware.esx.settings.depots.SyncSchedule` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/sync-schedule` request and submit the `SyncSchedule.Spec` JSON object in the request body.

The default schedule is set to update the metadata daily at a random time. To reset the schedule to the default settings, call the `set(spec)` of the `SyncSchedule` interface and pass `null` as an argumentuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/sync-schedule` and leave the `SyncSchedule.Spec` data structure unset..

Working with Offline Depots

An offline depot is a ZIP file that contains the metadata and payloads of software updates and follows the same structure as the online depot. Use the VMware Cloud Foundation APIs to import the content of an offline depot to the vCenter database.

To add an offline depot to the depots managed by the Depot Manager, you must create an offline depot specification by using the `com.vmware.esx.settings.depots.OfflineTypes.CreateSpec` class`Offline.CreateSpec` data structure. When you define the offline depot parameters, call the `create_Task(create_spec)` method of the `com.vmware.esx.settings.depots.Offline` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/offline?vmw-task=true` request and submit the `Offline.CreateSpec` JSON object. Depending on the location of the offline depot, when you create the offline depot specification, you must provide either the URI location or the file ID returned by the Jetty Web server embedded in the vSphere Lifecycle Manager. You set the type of the source from which the offline depot is downloaded by using the `setSourceType(sourceType)` method of the `OfflineTypes.CreateSpec` class`source_type` property of the `Offline.CreateSpec` data structure.

Pull Depot Content from a URI

To indicate that the offline depot resides on a URI location, call the `setLocation(java.net.URI location)` method of the `OfflineTypes.CreateSpec` class. You can pass as an argument the depot location in one of the following URI schemes: `http`, `https`, or `file`set the depot location as a value to the `location` property of the `Offline.CreateSpec` data structure. You can use one of the following URI schemes: `http`, `https`, or `file`. If you provide an HTTPS location to the offline depot, make sure you also provide a certificate trusted by the VMware Certificate Authority (VMCA) or a custom certificate from the VMware Endpoint Certificate Store (VECS). For detailed information about how to manage certificates, see the *vSphere Authentication* documentation.

Managing Depot Overrides

In case, you have a smaller Remote Office/ Branch Office (ROBO) cluster environment, or Edge computing environment, your clusters have no, or limited access to the Internet and limited connection to a vCenter instance. In such an environment, you can use Depot Manager to fetch the metadata and payloads of a desired software state from a local to the cluster depot.

To remediate a ROBO cluster, you must have access to a local software depot that hosts the components of the desired state. You can use Depot Manager to either export the whole vSphere Lifecycle Manager depot to an offline bundle, or export only the content of the desired state required for remediating the ROBO cluster.

To export the desired state image from the vSphere Lifecycle Manager depot, call the `export(cluster,export_spec)` method of the `com.vmware.esx.settings.clusters.Software` interface. The method returns the URI of the offline bundle that is hosted on the vSphere Lifecycle Manager Jetty Web serveruse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_ID>/software?action=export` request and submit the cluster ID as path parameter and an `Software.ExportSpec` JSON object. You receive the URI of the offline bundle that is hosted on the vSphere Lifecycle Manager Jetty Web server. To move the content of the offline bundle to the ROBO location, you must physically copy the ZIP file, unarchive, and mount its content to an HTTP server inside the ROBO environment.

To redirect a ROBO cluster to download software updates from a local repository and not from the vSphere Lifecycle Manager depot on the vCenter instance, the VMware Cloud FoundationREST API offer the cluster `DepotOverrides` service. To add a depot override location to a ROBO cluster, call the `add(cluster,override_depot)` method of the `DepotOverrides` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_ID>/depot-overrides?action=add` request. Specify the URI location of the local depot with the `setLocation(location)` method of the `com.vmware.esx.settings.clusters.DepotOverridesTypes.Depot` class`location` property of the `DepotOverrides.Depot` data structure which you submit to the request body. During the ROBO cluster remediation, Depot Manager instructs the hosts in the ROBO cluster to download the software updates from the configured local depots within the ROBO cluster.

Inspecting Depot Contents

You can use the VMware Cloud FoundationREST API to inspect the contents of the already synchronized and imported depots. You can list the available base images, add-ons, and components, or retrieve some detailed information about a specific software update.

To retrieve a list of the base images available on a vCenter instance, call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.BaseImages` interface. To narrow the list of returned base images and retrieve only items matching to specific criteria, you must pass a `com.vmware.esx.settings.depot_content.BaseImagesTypes.FilterSpec` instance as an argument to the list method. Use the retrieved list to get some information about each base image, including their display name and version, release date, and their category. You can also get some detailed information about a single base image by using the `get(version)` method of the `com.vmware.esx.settings.depot_content.base_images.Versions` interface. The information includes a list of the components present in this base image.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/base-images` request. You receive in the response body, a list of `BaseImages.Summary` JSON objects for each available base image. To get more information about a single base image, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/base-images/versions/<version>` request. You also get information about the components present in this base image.

To retrieve a list of all currently available add-ons in the vSphere Lifecycle Manager depot, call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.AddOns` interface and pass `null` as an argument. You can filter the available add-ons by using some specific criteria such as the add-on vendor, name, versions, or minimum version. You can retrieve some detailed information about a single add-on version, by calling the `get(name,version)` method of the `com.vmware.esx.settings.depot_content.add_ons.Versions` interface. The information includes the list of components part of the add-on, and the list of components that were removed by this add-on version.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/add-ons` request and submit some of the query parameters to filter the list of retrieved add-ons. To retrieve some detailed information about an add-on, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/add-ons/<name>/versions/<version>` request and submit the name and version of the add-on.

To retrieve a list of all components currently available in the vSphere Lifecycle Manager depot, you can call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.Components` interface and pass `null` as an argument. To retrieve a list of components that matches some specific criteria, define your preferences with a `com.vmware.esx.settings.depot_content.ComponentsTypes.FilterSpec` instance and pass it as an argument to the list method. Use the retrieved list to get some information regarding each component.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/components` request and submit some of the query parameters to filter the list of retrieved components. To retrieve detailed information about a component, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/components/<name>/versions/<version>` request and submit the name and version of the component.

Enabling a Cluster to Use a Software Specification

If you want to use the vSphere Lifecycle Manager to manage the life cycle of clusters in your environment, you have two options for enabling this feature. You can enable the vSphere Lifecycle Manager when you create the cluster. You can also turn an already created cluster into one managed by the vSphere Lifecycle Manager.

Creating a Cluster with Enabled vSphere Lifecycle Manager

You can use only the vSphere Web Services API or the Virtual Infrastructure Management API to create a cluster in your virtual environment and specify the initial desired state of the cluster. Use the VMware Cloud FoundationREST API to specify the detailed desired state after the cluster creation.

Starting with vSphere 8.0 Update 1, you can use the Virtual Infrastructure Management API which introduced a new way of accessing and using the vSphere Web Services. That is, you can now use the JSON data format over HTTP and create REST-like requests to the vSphere Web Services. Use the Virtual Infrastructure API or the vSphere Web Services API to create a cluster. For more information about how to authenticate and use the Virtual Infrastructure Management API, see the *vSphere Web Services SDK Programming Guide*, *An Introduction to vSphere APIs*, and *Getting Started with vSphere APIs* documentation. You can also check the *Virtual Infrastructure JSON API* reference documentation at https://developer.vmware.com/apis/vi-json/latest/.

You can use the `POST https://<vcenter_ip_address_or_fqdn> /sdk/vim25/<release_number>/Folder/<moID>/CreateClusterEx` request and submit the name of the cluster and a `ClusterConfigSpecEx` data object in the request body. In the data object, among other properties, you can specify the desired state for the cluster. The `desiredSoftwareSpec` property in the `ComputeResourceConfigSpec` data object contains the desired software specification for the cluster. This property is available for applications using the vSphere Web Services API of version 7.0 and later. You can create a `DesiredSoftwareSpec` data object and specify the base image that must be applied on the cluster with the `baseImageSpec` property. Optionally, you can specify a vendor add-on to be added to the software specification with the `vendorAddOnSpec` property.

Starting with vSphere 8.0, you can also enable the cluster to use vSphere Configuration Profiles by setting the `enableConfigManager` property of the `ComputeResourceConfigSpec` data object to `true`.

Enabling an Existing Cluster to Use vSphere Lifecycle Manager

If you want to manage the life cycle of a cluster by using a single software specification, you must first enable vSphere Lifecycle Manager on that cluster. You can use the VMware Cloud FoundationREST API to enable a cluster to use the vSphere Lifecycle Manager feature.

Before you enable vSphere Lifecycle Manager on a cluster, you can check whether the cluster meets all prerequisites. vSphere Lifecycle Manager can be enabled for a cluster only if the following requirements are met:

- All hosts in the cluster are of version 7.0 or later.
- All hosts in the cluster are stateful.
- All hosts in the cluster include only components that belong to integrated solutions, such as VMware vSAN™ and VMware vSphere® High Availability.
- None of the hosts in the cluster are in the process of active remediation through the VMware vSphere®Update Manager™.
- The cluster has a desired state already created for it.

If you want to run a preliminary check about whether all hosts in the cluster meet these requirements, call the `check_Task(cluster_ID,check_spec)` method of the `com.vmware.esx.settings.clusters.enablement.Software` interface. Pass as arguments the cluster ID, and optionally, a `com.vmware.esx.settings.clusters.enablement.SoftwareTypes.CheckSpec` instanceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software?action=check&vmw-task=true` request and submit the cluster ID and optionally a `Software.CheckSpec` JSON object.

The cluster ID represents the unique identifier for a cluster resource. To retrieve commonly used information about clusters including their IDs, call the `list(filter_spec)` method of the `com.vmware.vcenter.Cluster` interface and pass as argument a `com.vmware.vcenter.ClusterTypes.FilterSpec` instance to list the clusters that match specific criteria. You receive a list of `com.vmware.vcenter.ClusterTypes.Summary` objects which you can use to get the cluster ID.To retrieve a list of clusters available on your vCenter instance, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/cluster` request and filter the list of clusters by using the query parameters. You receive an array of `Cluster.Summary` JSON objects that contain the cluster ID among the other commonly used information.

You pass a `CkeckSpec` instance You submit a `Software.CheckSpec` structure in the request body to specify which checks can be skipped during the cluster preliminary check. Though you can skip some checks with this operation, the Image Manager runs all checks during the enablement operation. If you leave the check specification empty, all checks are run for each host in the cluster. You can select among the following checks to be skipped when running a pre-check operation:

- `SOFTWARE`. Checks whether there are any orphaned vSphere Installation Bundles (VIBs) and any software that cannot co-exist with vSphere Lifecycle Manager.
- `VERSION`. Checks whether all hosts in the cluster are of version greater than a predefined one.
- `STATELESSNESS`. Checks whether there are any stateless hosts in the cluster. vSphere Lifecycle Manager can be enabled only if the cluster does not contain stateless hosts.
- `VUM_REMEDIATION`. Checks whether any of the hosts in the cluster are currently remediated through the VMware vSphere®Update Manager™.

- SOFTWARE_SPECIFICATION_EXISTENCE. Checks whether there is a software specification already associated with this cluster. In case, this check reports that the cluster does not have a software specification, you must first create a draft software specification for this cluster and then commit the draft.
- VSAN_WITNESS_ELIGIBILITY. Checks whether the software specification can be used on any vSAN witness hosts in the cluster. For information about how you can manage a vSAN cluster by using vSphere Lifecycle Manager, see *vSAN Clusters and vSphere Lifecycle Manager* chapter in the *Managing Host and Cluster Lifecycle* documentation.

To enable a cluster to be managed with vSphere Lifecycle Manager, call the `enable_Task(cluster_ID,enable_spec)` method of the `com.vmware.esx.settings.clusters.enablement.Software` interface. Pass as arguments the cluster ID and optionally, a `com.vmware.esx.settings.clusters.enablement.SoftwareTypes.EnableSpec` instance. To specify checks that you want to be skipped during the enablement process, pass the `EnableSpec` instance. Currently, you can only skip the `SoftwareTypes.CheckType.SOFTWARE` check.

To enable a cluster to be managed with vSphere Lifecycle Manager, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software?vmw-task=true` request and submit the cluster ID as path parameter. To specify which check can be skipped during the enablement process, submit a `Software.EnableSpec` JSON object. Currently, you can only skip the software checks.

You can also get information about which clusters in your environment are managed with a single software specification. Call the `get(cluster_ID)` method and pass the cluster ID as an argument. Use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software` request and submit the cluster ID as a path parameter.

Enabling a Standalone Host to Use a Software Specification

Starting with vSphere 8.0, you can use the vSphere Lifecycle Manager to manage the life cycle of standalone hosts in your vCenter system.

You can add a standalone host to a vCenter instance under a data center object or into a folder. See ESX Hosts. If you remove a host from a cluster but leave it in the data center, the host is also considered a standalone host.

To manage a standalone host with a software specification, you must first enable the vSphere Lifecycle Manager on that host. You can run a preliminary check to establish whether the standalone host meets all requirements for enabling the vSphere Lifecycle Manager. Call the `check_Task(host_ID, check_spec)` method of the `com.vmware.esx.settings.hosts.enablement.Software` interface and pass as arguments the host identifier and a `CheckSpec` instanceUse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/enablement/software?action=check&vmw-task=true` request and submit the host ID as path parameter. The pre-checks that you can skip before enabling the vSphere Lifecycle Manager on a standalone host are the same as for a cluster.

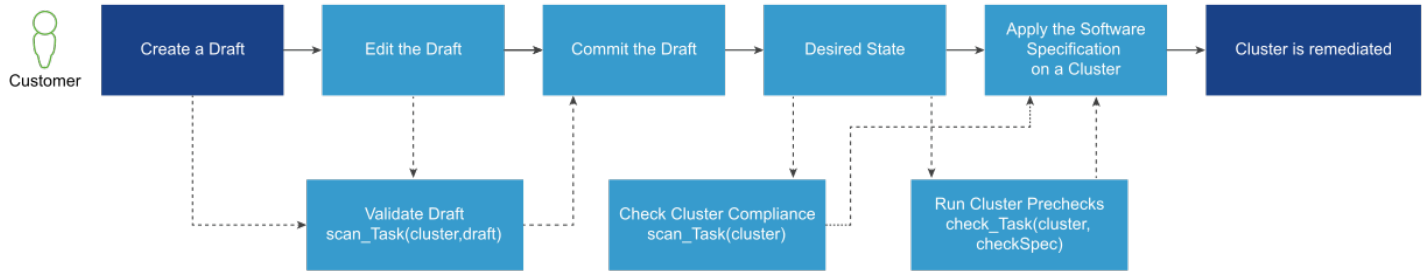
To enable vSphere Lifecycle Manager on a standalone host, call the `enable_Task(host_ID, check_spec)` method of the `com.vmware.esx.settings.hosts.enablement.Software` interface. Pass as arguments the host unique identifier and optionally, a `com.vmware.esx.settings.hosts.enablement.SoftwareTypes.EnableSpec` instance to define whether any checks can be skipped during the feature enablement operationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/enablement/software?vmw-task=true` request and submit the host ID as path parameter. Optionally, to define whether you want to skip any checks during the feature enablement, you can add a `Software.EnableSpec` data structure to the request body. Currently, you can skip only the check for orphaned vSphere Installation Bundles (VIBs).

Working with Draft Software Specifications

You create a draft software specification to describe the components of the desired state that you want to apply on a cluster or a standalone host.

The draft software specification is the working copy of the desired software state. Only one user at a time is allowed to edit a single draft for a cluster or a standalone host. Before saving the changes to the edited draft version, you can validate the content of the draft.

Figure 551: Drafts Workflow for a Cluster



A typical workflow for working with draft software specifications starts with creating a draft software specification for a specific cluster or a standalone host. If the cluster or the standalone host already have a software specification defined, this action takes the latest committed draft and you can edit its contents according to your needs. If the cluster or the standalone host have no software specification created yet, this method creates an empty draft. The only mandatory item for a draft is a base image of a specific version.

After you complete adding components, an add-on, and a base image, you can save your changes by committing the draft. This operation results in setting the committed draft as the current desired state of the cluster or the standalone host. Before committing the draft software specification, you can validate the contents of the draft or check whether all hosts in the cluster or the single standalone host are compliant with the draft.

If the commit operation is successful, the draft becomes the desired state for the cluster or the standalone host. You can now export the software specification and use it, for example, in a ROBO cluster scenario or another standalone host. You can also validate the compliance of the hosts in the cluster or the standalone host against the desired state and then apply the software specification, if feasible.

Creating a Draft Software Specification

To describe the components of a desired state for a cluster or a standalone host, create a draft software specification and save the desired state when ready.

To edit an existing desired state or to create an empty draft software specification:

- For a cluster, call the `create(cluster_ID)` method of the `com.vmware.esx.settings.clusters.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts` request.
- For a standalone host, call the `create(host_ID)` method of the `com.vmware.esx.settings.hosts.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts` request.

Pass the cluster ID or the host ID as an argument to the respective method. Submit the cluster ID or the host ID as path parameters to the respective request. As a result, you receive a draft ID which you can use to add a base image, an add-on, or some components to the draft.

Editing a Draft Software Specification

After you have created a draft software specification, use the VMware Cloud FoundationREST API to edit its items.

To set a base image to a draft software specification:

- For a cluster, call the `set(cluster_ID, draft_ID, base_image_spec)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.BaseImage` interface and pass as arguments the cluster and the draft IDs, and a base image specificationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/base-image` request and submit the cluster and draft IDs as path parameters and a base image specification in the request body.
- For a standalone host, call the `set(host_ID, draft_ID, base_image_spec)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.BaseImage` interface and

pass as arguments the standalone host and draft ID, and a base image specificationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/base-image` request and submit the host and draft IDs as path parameters and a base image specification in the request body.

If the draft contains a base image, this method overwrites the existing image. The base image specification contains the version of the bootable ESX that must be included in the desired state. To retrieve details about the base image that is currently present in a draft, call the `get(cluster_ID,draft_ID)` or `get(host_ID, draft_ID)` methods of the `com.vmware.esx.settings.clusters.software.drafts.software.BaseImage` interface. Pass as arguments the cluster or the standalone host ID and the draft ID. Use the returned `com.vmware.esx.settings.BaseImageInfo` object to query the version, display name and version, and the release date of the ESX hostuse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/base-image` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/base-image` requests and submit as path parameters the cluster or the standalone host ID, and the draft ID. You receive a `BaseImageInfo` JSON object in the request body that holds the details about the version, display name and version, and the release date of the ESX host.

To add an OEM add-on to a draft software specification:

- For a cluster, call the `set(cluster_ID,draft_ID,addon_spec)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.AddOn` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/add-on` request.
- For a standalone host, call the `set(host_ID,draft_ID,addon_spec)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.AddOn` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/add-on` request.

Pass as arguments to these methods the cluster or the standalone host ID, the draft ID, and the add-on specification. If you want to remove an add-on from a draft, call the `delete(cluster_ID,draft_ID)` or `delete(host_ID,draft_ID)` method of the respective `AddOn` interface and pass as arguments the cluster or the standalone host ID, and the draft IDuse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/add-on` and `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/add-on` and submit as path parameters the cluster or the standalone host ID, and the draft ID.

You can add a component, change the version, or delete an existing component from a draft software specification. To change the version of a component included in a draft:

- For a cluster, call the `set(cluster_ID,draft_ID,component_ID,version)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.Components` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components/<component_id>` request.
- For a standalone host, call the `set(host_ID, draft_ID, component_ID, version)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.Components` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components/<component_id>` request.

As a result, you add the component specified with the `component_ID` and `version` arguments to the draft, if it is missing. To remove a component from a draft, call the `delete(cluster_ID,draft_ID,component_ID)` or `delete(host_ID,draft_ID,component_ID)` method of the respective `Components` interfaceuse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components/<component_id>` and `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components/<component_id>` requests. You can change multiple components in a draft by calling the `update(cluster_ID,draft_ID,update_spec)` or `update(host_ID,draft_ID,update_spec)` method of the respective `Components` interface. To specify the components you want to remove, add, or update for a draft, pass a cluster or a host `ComponentsTypes.UpdateSpec` instance as an argument to the update methodusing the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components`

and `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components` requests. To specify the components you want to remove, add, or update for a given draft, submit a cluster or a host `Components.UpdateSpec` JSON object to the request body of the respective request.

To retrieve information about all components present in a draft, or a single component, you can use the list methods of the respective `Components` interfaceuse the list requests of the respective `Components` service.

Validating the Draft Software Specification

Before saving a draft and turning it into a desired state for a cluster or a standalone host, you can check whether the specification is complete and valid. You can also check whether the draft specification drifts in any way from the current state of the cluster or the host.

The `Drafts` service offers two methods for validating the draft software specification. To check whether a draft is complete and there are no conflicts between the draft components, or unresolved dependencies, call the `validate_Task(cluster_ID,draft_ID)` or `validate_Task(host_ID,draft_ID)` method of the respective `Drafts` interface. Pass as arguments to the method the cluster or the standalone host ID, and the draft IDuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=validate&vmw-task=true` or `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=validate&vmw-task=true` request depending on the targeted vSphere inventory object. Submit as path parameters the cluster ID or the standalone host ID, and the draft ID. You validate whether there were any other drafts committed for this cluster or standalone host, which can make the current commit operation invalid. The method also validates whether all components defined in the software specification are available in the depot metadata. This method does not run compliance checks against the cluster or the standalone host.

To check whether all hosts in the cluster or the standalone host are compliant with the draft software specification, call the `scan_Task(cluster_ID,draft_ID)` or `scan_Task(host_ID,draft_ID)` method of the respective `Drafts` interface. Pass as arguments to this method the cluster or the standalone host ID, and the draft IDuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=scan&vmw-task=truePOST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=scan&vmw-task=true` request depending on the targeted vSphere inventory object. Submit as path parameters the cluster ID or the standalone host ID, and the draft ID. This method results in running a comparison between the draft specification and the current state of each host in the cluster or the current state of the standalone host.

Committing the Draft Software Specification

When you commit a draft software specification, it becomes the desired state for the cluster or the standalone host.

To save the draft that you created for a cluster or a standalone host, call the `commit_Task(cluster_ID,draft_ID,draft_commit_spec)` or `commit_Task(host_ID, draft_ID, draft_commit_spec)` method of the respective `Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=commit&vmw-task=true` and `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=commit&vmw-task=true` request depending on the targeted vSphere inventory object. The Image Manager component runs a validation check before the draft gets saved to the database. This method returns an identifier of the commit operation. You can use the ID to retrieve information about a specific commit such as the author of the commit operation, the time when the draft was committed, and so on.

Working with Desired Software States

When you commit a draft software specification, you make the committed draft the desired state for that cluster or standalone host. If all hosts in the cluster are compliant with the desired state, you can remediate the cluster. If the single standalone host is compliant with the desired state, you can update the ESX host to that state.

You can use the methods provided with the `com.vmware.esx.settings.clusters.Software` or `com.vmware.esx.settings.hosts.Software` interfacethe cluster and host `Software` services to manage a

desired state for a cluster or a standalone host. Before you apply a desired state on a cluster or a standalone host, you can run pre-checks to ensure that all hosts in the cluster or the single host are in a good state to be remediated. The pre-checks verify whether any of the hosts in the cluster or the single standalone host must be rebooted or are in maintenance mode. You can also check the compliance of the cluster or the standalone host against the desired state. See Checking the Compliance Against the Desired State.

You can export a software specification created for a cluster or a standalone host by using one of the following formats:

- An offline bundle in a ZIP file format.
- An ISO image.
- A JSON file.

Use the vSphere Lifecycle Manager API to import a software specification as a draft and then edit it. You have several options for running the import operation depending on the location and format of the desired software state.

Exporting and Importing a Desired State
Use the VMware Cloud FoundationREST API to export the desired software state of a cluster or a standalone host. Then you can import the desired state to a different cluster or host in the same or a different vCenter instance.

Exporting a Desired State

To export a desired state, you can use one of the following methods:

- For a cluster, call the `export(cluster_ID, export_spec)` method of the `com.vmware.esx.settings.clusters.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=export` request.
- For a standalone host, call the `export(host_ID, export_spec)` method of the `com.vmware.esx.settings.hosts.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=export` request.

This method does not export any information about the solutions available on the cluster or the standalone host since the constraints set by these solutions might not be applicable for another cluster or host. Pass as parameters the cluster or the standalone host ID, and an instance of the respective `SoftwareTypes.ExportSpec` classSubmit the cluster or the standalone host ID, and a `Software.ExportSpec` JSON object in the request body.

You can choose how to export the desired software specification.

- Export as an ISO image. Call the `setExportIsoImage(exportIsoImage)` method of the `ExportSpec` instance and pass `true` as an argument.Set `true` to the `export_iso_image` property of the `Software.ExportSpec` JSON object that you submit in the request body. Use the exported ISO image for performing clean installs and for bootstrapping purposes. You can upload the ISO file into the Jetty Web server on the target vCenter instance but you cannot use ISO files to manage the life cycle of clusters or a standalone host through the vSphere Lifecycle Manager feature.
- Export as an offline bundle in a ZIP file format. Call the `getExportOfflineBundle()` method of the `ExportSpec` instance. Set `true` to the `export_offline_bundle` property of the `Software.ExportSpec` JSON object that you submit in the request body. You can use the exported offline bundle to create a depot and add its components to the resources managed by the Depot Manager module.
- Export as a JSON file holding the desired state specification. Call the `setExportSoftwareSpec(exportSoftwareSpec)` method of the `ExportSpec` instance and pass `true` as argument.Set `true` to the `export_software_spec` property of the `Software.ExportSpec` JSON object that you submit to the request body. You can then reuse the JSON file to apply the desired state that it contains to another cluster or standalone host in the same or in a different vCenter instance. Note that the JSON file holds only the description of the desired state. You must check whether all components described in the JSON file are available in the depot for the target cluster. See Importing a Desired State Specification for information about how you can use a desired state specification for another cluster or standalone host.

Importing a Desired State Specification

To import a desired state of a cluster or a standalone host and assign it to another cluster or standalone host in the same or different vCenter instance:

- For a cluster, call the `importSoftwareSpec(cluster_ID, drafts_import_spec)` method of the `com.vmware.esx.settings.clusters.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts?action=import-software-spec` request.
- For a standalone host, call the `importSoftwareSpec(host_ID, drafts_import_spec)` method of the `com.vmware.esx.settings.hosts.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts?action=import-software-spec` request.

Pass as parameters the cluster or the standalone host ID, and an instance of the respective `DraftsTypes.ImportSpec` class. Submit as pat parameters the cluster or the standalone host ID, and a `Drafts.ImportSpec` JSON object in the request body. Use the instance of the import specification to describe the download source and the source type of the imported software specification. Depending on the location and the source type of the exported desired state, you can choose from the following import options:

- Import a file from the vCenter or your local file system. Call the `setFileId(fileId)` method of the `ImportSpec` instance. Pass as argument the file ID of the software specification which was previously uploaded on the Jetty Web server running on the vCenter at `https://<vcenter_FQDN>:9087/vum-fileupload` URL.Use the `file_id` property of the `Drafts.ImportSpec` data structure to set the file ID of the software specification which was previously uploaded on the Jetty Web server running on the vCenter at `https://<vcenter_FQDN>:9087/vum-fileupload` URL. You can also use this option to import a specification file that resides on your local file system. Make sure you set the source type of the import specification to `PUSH` through the `setSourceType(sourceType)` method of the `ImportSpec` instance`source_type` property of the `Drafts.ImportSpec` data structure.
- Import a file that resides on a URI location. Call the `setLocation(location)` method of the `ImportSpec` instance. Pass as argument the URI location of the software specification file. The software specification can be pulled from a URI location with one of the following schemes: `file`, `http`, or `https`. You can use this import mechanism only if you set the source type to `SourceType.PULL`Use the `location` property of the `Drafts.ImportSpec` data structure to submit the URI location of the software specification file. The software specification can be pulled from a URI location with one of the following schemes: `file`, `http`, or `https`. You can use this import mechanism only if you set the `source_type` property to `PULL`.
- Import a desired state as a JSON string. Call the `setSoftwareSpec(softwareSpec)` method of the `ImportSpec` instance. Pass as argument the JSON string representing the software specification you want to import. Use this mechanism only if you set the source type to `SourceType.JSON_STRING`.Use the `software_spec` property of the `Drafts.ImportSpec` data structure to set the JSON string representing the software specification you want to import. Use this method only if you set the `source_type` property to `JSON_STRING`.

Checking the Compliance Against the Desired State

Before applying a desired state on a cluster, you can scan all hosts in the cluster against the desired state and check the cluster compliance against the desired state. Before applying a desired state on a standalone host, you can scan the host and check its compliance against the desired state.

To check the compliance of all hosts in a cluster or of the single standalone host:

- For a cluster, call the `scan_Task(cluster_ID)` method of the `com.vmware.esx.settings.clusters.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=scan&vmw-task=true` request.
- For a standalone host, call `scan_Task(host_ID)` method of the `com.vmware.esx.settings.hosts.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=scan&vmw-task=true`request.

Pass as an parameter the cluster or the host identifier. The method compares the desired state against the current state of each host in the cluster or of the single host and as a result calculates the compliance.

You can retrieve the cluster or the standalone host compliance status by calling the `get(cluster_ID)` or `get(host_ID)` method of the respective `Compliance` interface and passing as argument the cluster or the standalone host ID. As a result you receive a `com.vmware.esx.settings.ClusterCompliance` or a `com.vmware.esx.settings.HostCompliance` object. You can use the `ClusterCompliance` or the `HostCompliance` instance to retrieve the following informationusing the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/compliance` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/compliance` requests and passing the cluster or standalone host ID as path parameter. As a result, you receive a `ClusterComliance` or `HostCompliance` JSON object that contains the following information:

- The overall cluster compliance status regarding the target version of the components described within the cluster desired state.
- The impact of applying the desired state on the cluster in case the cluster is non-compliant.
- A list of all compliant hosts in the cluster.
- A list of the incompatible hosts in the cluster.
- A list of the non-compliant hosts in the cluster.
- A list of the unavailable hosts which cannot be checked for compliance against the desired state.
- The compliance status of each host in the cluster.
- The notifications returned by the compliance check operation.
- The time that the compliance check takes.
- The ID of the committed draft for which the compliance check is performed. If the `getCommit()` method of the `ClusterCompliance` or the `HostCompliance` instance returns `null`If the `commit` property of the cluster or host compliance data structure is unset, the compliance check is run against a draft software specification.
- The compliance overall stage status of the cluster or the standalone host and is relevant when the compliance status of the cluster or the standalone host is `NON_COMPLIANT`.
- The compliance of the OEM add-on on the standalone host with respect to the add-on in the desired state.
- The compliance of the base image on the standalone host with respect to the base image in the desired state.
- The compliance of all effective components and all components present on the standalone host in respect to the components in the desired state.
- The compliance of all DPU devices on the standalone host with respect to the desired state.
- The compliance of the hardware support on the standalone host with respect to the hardware support defined in the target state.

A cluster or a standalone host can have one of the following compliance statuses regarding the target versions:

- `COMPLIANT`. The target versions of the components described in the desired state of the cluster or the standalone host are the same as the versions of the components currently present on the hosts in the cluster or on the standalone host.
- `NON_COMPLIANT`. The desired state of the cluster or the standalone host describes components with higher versions than the versions of the components currently present on the hosts in the cluster or on the standalone host. Non-compliant clusters or hosts are those clusters or hosts which have orphaned VIBs, or components on the hosts that are not present in the desired state specification.
- `INCOMPATIBLE`. One or more hosts in the cluster or in the standalone host have components with higher versions than the components described in the desired state specification.
- `UNAVAILABLE`. The current state of one or more hosts in the cluster or of the standalone host cannot be retrieved and as a result the compliance check cannot be performed.

You can check the compliance impact of applying the desired state on a non-compliant cluster or a standalone host by calling the `getImpact()` method of the `ClusterCompliance` or the `HostCompliance` instance. A `com.vmware.esx.settings.ComplianceImpact` object is returnedusing the `impact` property of the respective compliance data structure which contains the `ComplianceImpact` data structure. Use it to retrieve information about the steps you must take to remediate the cluster or the standalone host successfully. You might need to reboot a host or put a host into maintenance mode to remediate the cluster or the standalone host successfully.

Hardware Compatibility Data

The hardware compatibility data contains information about the compatibility between ESX hosts and ESX versions.

You can use the `com.vmware.esx.hcl.CompatibilityDataCompatibilityData` service to retrieve information about the compatibility data or to update the local compatibility data on a vCenter instance. To retrieve information about the compatibility data stored on your vCenter instance, call the `get()` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/compatibility-data/status` request. To update the local compatibility data with the latest version available from the official VMware source, call the `update_Task()` methoduse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/compatibility-data?action=download&vmw-task=true` request.

Checking the Hardware Compatibility of an ESX Host

You can query the hardware compatibility for a host before upgrading to a new ESX version. You can also download the information generated by the hardware compatibility report.

To use services form the `com.vmware.esx.hcl.hostsesx hcl` package, you must verify that you have accepted to participate in the CEIP and there is available compatibility data.

You can use the `com.vmware.esx.hcl.hosts.CompatibilityReleases` interface`CompatibilityReleases` service to list available releases for generating a compatibility report for a specific ESX host. To list the locally available ESX releases for the host that can be used to generate a compatibility report, call the `list(host_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-releases` request. The list includes only major and update releases. Patch releases are not listed.

You can use the `com.vmware.esx.hcl.hosts.CompatibilityReport` interface`CompatibilityReport` service to generate a hardware compatibility report for an ESX host against a specific ESX release. To return the last generated hardware compatibility report for a specific host, call the `get(host_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-report` request. To generate a hardware compatibility report for a specific host against specific ESX release, call the `create_Task(host_ID,spec)` methoduse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-report?vmw-task=true` request.

You can use the `com.vmware.esx.hcl.Reports` interface`Reports` service to download information generated by the hardware compatibility report. To retrieve the URI location for downloading a compatibility report, call the `get(report_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/reports/<report_id>` request.

Configuring Remediation Settings

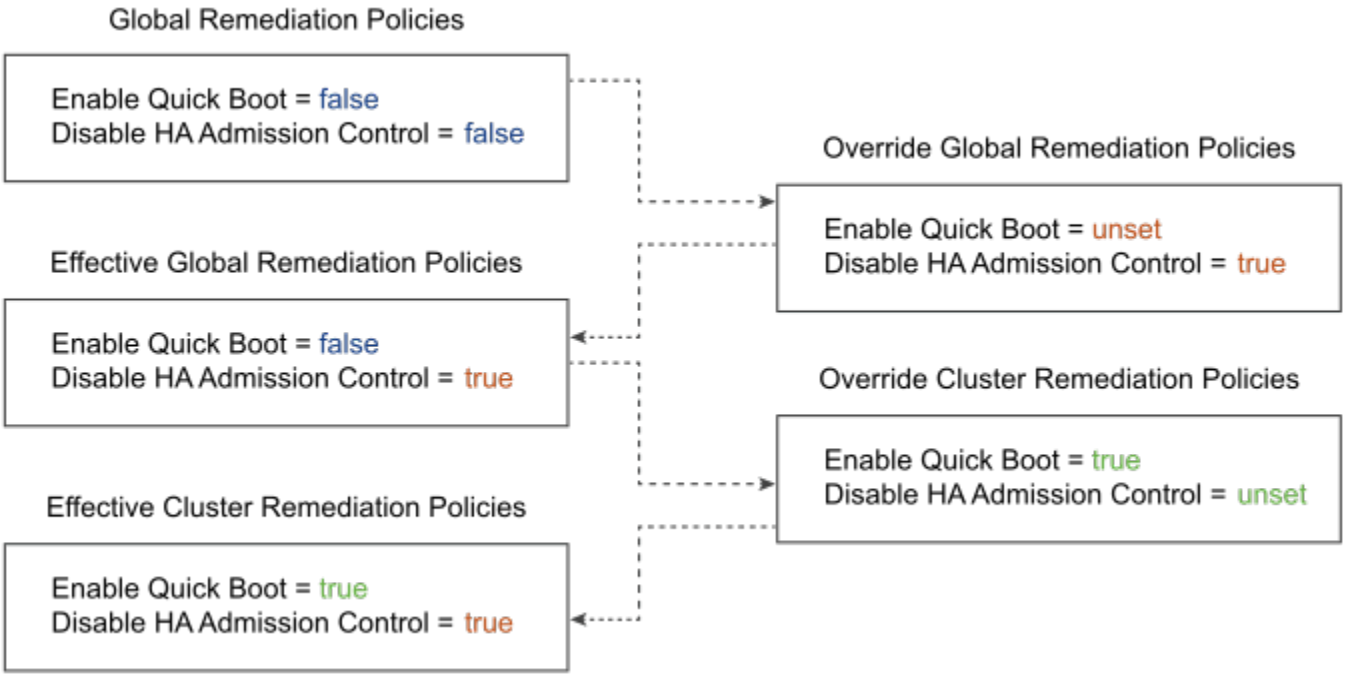
You can control the behavior of the ESX hosts and virtual machines during the remediation process. You can create a global remediation policy that applies to all clusters and standalone hosts in a vCenter instance. You can also set a remediation policy to a specific cluster and a standalone host.

When you run cluster and standalone host compliance checks, the Coordinator module runs a series of checks on each host to determine their state and whether some additional actions must be taken to ensure the success of the remediation operation. In case one or more hosts in the cluster and any of the standalone hosts are evaluated as non-compliant, additional checks are run on those hosts to evaluate whether they must be rebooted or put into maintenance mode. Currently, VMware provides a set of behavior controls (remediation policies) regarding the virtual machines and the hosts in a cluster or a standalone host. This set of remediation policies might change with the next vSphere release.

How Remediation Policies Overrides Work

The vSphere Lifecycle Manager provides a default global policy configuration that must be applied on each cluster and standalone host during remediation. Through the VMware Cloud FoundationREST API, you can change the global policies and create some cluster-and host-specific policies. Before remediating a cluster and standalone host, you can use the API to determine the effective global and cluster-and host-specific remediation policies. The following graphic describes how the mechanism of the policy overrides works.

Figure 552: How Remediation Policies Work



All clusters and standalone hosts in a vCenter instance inherit the default or the overridden global policy settings unless the global policy is explicitly overridden on a cluster and host level.

Editing Global or Cluster- and Host-Specific Remediation Policies

To view the currently set global remediation policy, call the `get(cluster_ID)` or `get(host_id)` method of the `com.vmware.esx.settings.defaults.clusters.policies.Apply` or `com.vmware.esx.settings.defaults.hosts.policies.Apply` interface. You receive a default `ApplyTypes.ConfiguredPolicySpec` instance that contains the configuration settings of the global remediation policy. To edit a global remediation policy, call the `set(policy_spec)` method of the respective `com.vmware.esx.settings.defaults.clusters.policies.Apply` or `com.vmware.esx.settings.defaults.hosts.policies.Apply`interface. Pass as an argument an `ApplyTypes.ConfiguredPolicySpec` instance and define new values to the global policy settings. To view the effective global remediation policy settings for a cluster and host, call the `get()` method of the respective `com.vmware.esx.settings.defaults.clusters.policies.apply.Effective` or `com.vmware.esx.settings.defaults.hosts.policies.apply.Effective` interface. The method returns an `EffectivePolicySpec` instance that contains the effective global policies applicable for all clusters and hosts in your vCenter environment.

To view the currently set global remediation policy, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply` request. You receive a `Apply.ConfiguredPolicySpec` data structure that contains the settings of the global remediation policy. To edit a global remediation policy, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply` or `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply` request and submit the respective `Apply.ConfiguredPolicySpec` data structure to define new values to the global policy settings. To view the effective global remediation policy settings for a cluster and host, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply/effective` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply/effective` request. You receive the respective `Effective.EffectivePolicySpec` JSON object that contains the effective global policies applicable for all clusters and hosts in your vCenter environment.

To view the cluster- and host-specific remediation policies, call the `get(cluster_ID)` or `get(host_id)` method of the respective `com.vmware.esx.settings.clusters.policies.Apply` or `com.vmware.esx.settings.hosts.policies.Apply` interface. The method returns an instance of

the com.vmware.esx.settings.clusters.policies.ApplyTypes.ConfiguredPolicySpec or com.vmware.esx.settings.hosts.policies.ApplyTypes.ConfiguredPolicySpec class which contain the cluster- and host-specific policies to be applied during remediation. To change the cluster- and host-specific policy, call the set(cluster_ID,policy_spec) or set(host_ID,policy_spec) method of the respective Apply interface. Pass as argument an ApplyTypes.ConfiguredPolicySpec instance and describe the cluster- and host-specific remediation policies. To view the effective cluster- and host-specific policies, call the get(cluster_ID) or get(host_id) method of the com.vmware.esx.settings.clusters.policies.apply.Effective or com.vmware.esx.settings.hosts.policies.apply.Effective interface. The method returns an EffectivePolicySpec instance that describes the effective cluster- and host-specific policies.

To view the cluster- and host-specific remediation policies, use the GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply or GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply request. You receive an Apply.ConfiguredPolicySpec JSON object that contain the cluster- and host-specific policies to be applied during remediation. To change the cluster- and host-specific policy, use the PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply or PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply request and submit anApply.ConfiguredPolicySpec data structure to describe the cluster- and host-specific remediation policies. To view the effective cluster- and host-specific policies, use the GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply/effective or GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply/effective request. You receive the respective Effective.EffectivePolicySpec data structure that describes the effective cluster- and host-specific policies.

Remediation Policy Options for Clusters

To describe a global or cluster-specific remediation policy, use the com.vmware.esx.settings.defaults.clusters.policies.ApplyTypes.ConfiguredPolicySpec and com.vmware.esx.settings.clusters.policies.ApplyTypes.ConfiguredPolicySpec classesthe respective

cluster `Apply.ConfiguredPolicySpec` data structure. For the vSphere 8.0 release, VMware provides the following methods to configure a global or cluster-specific policy.

MethodProperty	Description
`setDisableDpm(disableDpm)disable_dpm`	Deactivate the VMware Distributed Power Management (DPM) feature for all clusters or for a specific cluster. DPM monitors the resource consumption of the virtual machines in a cluster. If the total available resource capacity of the hosts in a cluster is exceeded, DPM powers off (or recommends powering off) one or more hosts after migrating their virtual machines. When resources are considered underutilized and capacity is needed, DPM powers on (or recommends powering on) hosts. Virtual machines are migrated back to these hosts. During the cluster remediation, the vSphere Lifecycle Manager cannot wake up and remediate hosts that are automatically put into a stand-by mode by DPM. These hosts stay non-compliant when DPM turns them on. The vSphere Distributed Resource Scheduler (DRS) is unable to migrate virtual machines to the hosts which are not remediated with the desired state for the cluster. To deactivate DPM during the cluster remediation, call the `setDisableDpm(disableDpm)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `disable_dpm` property. By default, the vSphere Lifecycle Manager temporarily deactivates DPM and turns on the hosts to complete the remediation. DPM is enabled again when the cluster remediation finishes.
`setDisableHac(disableHac)disable_hac`	Deactivate the vSphere HA admission control. vSphere HA uses admission control to ensure that a cluster has sufficient resources to guarantee the virtual machines recovery when a host fails. If vSphere HA admission control is enabled during remediation, putting a cluster into maintenance mode fails because vMotion cannot migrate virtual machines within the cluster for capacity reasons. To allow the vSphere Lifecycle Manager to temporary deactivate vSphere HA admission control, call the `setDisableHac(disableHac)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `disable_hac` property. By default, the vSphere HA admission control is enabled because DRS should be able to detect issues with the admission control and deactivate it to allow the remediation to complete.
`setEvacuateOfflineVms(evacuateOfflineVms)evacuate_offline_vms`	Migrate the suspended and powered off virtual machines from the hosts that must enter maintenance mode to other hosts in the cluster. To enable this remediation policy, call the `setEvacuateOfflineVms(evacuateOfflineVms)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `evacuate_offline_vms` property. By default, this setting is deactivated in the global remediation policy.

MethodProperty	Description
`setFailureAction(failureAction)failure_action`	Specify what actions vSphere Lifecycle Manager must take if a host fails to enter maintenance mode during the remediation. To configure this policy on a global or cluster-specific level, call the `setFailureAction(failureAction)` method of the `ConfiguredPolicySpec` instance. Pass as argument an `ApplyTypes.FailureAction` instanceset the properties of the `Apply.FailureAction` data structure. You can set the number of times that vSphere Lifecycle Manager tries to put a host into maintenance mode and the delay between the tries. When the threshold is reached and the host failed to enter maintenance mode, the cluster remediation fails. By default, vSphere Lifecycle Manager tries to put a host into maintenance mode three times with a five minute delay between each try before the cluster remediation fails.
`setEnforceHclValidation(enforceHclValidation)enforce_hcl_validation`	Prevents the remediation of vSAN clusters if vSphere Lifecycle Manager reports hardware compatibility issues during the hardware compatibility check performed as part of the remediation pre-check or the remediation tasks of the cluster. If you pass `null` as a parameterIf you leave the `enforce_hcl_validation` property unset, detected hardware issues are reported as warnings and do not prevent the remediation of the vSAN cluster.
`setParallelRemediationAction(parallelRemediationAction)parallel_remediation_action`	Enable simultaneous remediation of all hosts that are in maintenance mode with in the cluster. Pass an instance of the `com.vmware.esx.settings.defaults.clusters.policies.ApplyTypes.ParallelRemediationAction` classSubmit the `Apply.ParallelRemediationAction` data structure to indicate the maximum number of hosts that can be remediated in parallel. **Note:** If the hosts have NSX virtual distributed switches that are ready to be migrated to vSphere Distributed Switches, you must manually set the maximum number of parallel remediations to no more than 4. In cases when host switch migration is needed, if more than 4 hosts are remediated in parallel, the remediation might fail, because the host switch migration takes more time than the time vSphere Lifecycle Manager needs to complete the parallel remediation.

MethodProperty	Description
`setPreRemediationPowerAction (preRemediationPowerAction)pre_remediation_power_action`	Specify how the power state of the virtual machines must change before the host enters maintenance mode. If DRS is not enabled on a cluster or the automation level of a DRS cluster is not set to fully automated, the Coordinator module fails to remediate the cluster if the remediation requires a reboot or maintenance mode. You can set a policy that powers off or suspends the virtual machines on hosts that must be rebooted or must enter maintenance mode during remediation. The DRS takes care of changing the power state of the virtual machines when the host enters and exits maintenance mode. To set a policy for the power state of the virtual machines during the remediation, call the `setPreRemediationPowerAction(preRemediationPowerAction)` method of the `ConfiguredPolicySpec` instance. Pass as a parameter an instance of the `PreRemediationPowerAction` enumeration class and specifysubmit the `Apply.ConfiguredPolicySpec.PreRemediationPowerAction` enumerated type and set one of the following values: • `DO_NOT_CHANGE_VMS_POWER_STATE`. Indicates that the power state of the virtual machines must not be changes. • `POWER_OFF_VMS`. Indicates that the virtual machines must be powered off before the hosts enter maintenance mode. • `SUSPEND_VMS`. Indicates that the virtual machines must be suspended before the hosts enter maintenance mode. • `SUSPEND_VMS_TO_MEMORY`. Indicates that the virtual machines must be suspended before the hosts enter maintenance mode. By default, the Coordinator must leave the power state of the virtual machines unchanged.
`setEnableQuickBoot(enableQuickBoot)enable_quick_boot`	Reduce the reboot time of an ESX host by skipping all the hardware initialization processes and restarting only the hypervisor. This policy is applicable only if the host platform supports the Quick Boot feature. To enable the Quick Boot feature on the hosts during the remediation, call the `setEnableQuickBoot(enableQuickBoot)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `enable_quick_boot` property. By default, this policy is deactivated.

Remediation Policy Options for Standalone Hosts

To describe a global or cluster-specific remediation policy, use the `com.vmware.esx.settings.defaults.hosts.policies.ApplyTypes.ConfiguredPolicySpec` and `com.vmware.esx.settings.hosts.policies.ApplyTypes.ConfiguredPolicySpec` classesrespective host

`Apply.ConfiguredPolicySpec` data structure. For the vSphere 8.0 release, VMware provides the following methods to configure a global or host-specific policy.

MethodProperty	Description
`setEnableQuickBoot(enableQuickBoot)enable_quick_boot`	Optimize the host patching and upgrade operations by reducing the reboot time of an ESX host. Since the patching and upgrading operations do not affect the hardware of the host, the hardware initialization processes can be skipped. This policy is applicable only if the host platform supports the Quick Boot feature. For more information about which hosts are Quick Boot compatible, see the following KB article https://kb.vmware.com/s/article/52477.
`setPreRemediationPowerAction (preRemediationPowerAction)pre_remediation_power_action`	Specify how the power state of the virtual machines must change before the standalone host enters maintenance mode. You can choose between the following power actions: • `DO_NOT_CHANGE_VMS_POWER_STATE`. Indicates that the power state of the virtual machines must not be changed. • `POWER_OFF_VMS`. Indicates that the virtual machines must be powered off before the standalone host enters maintenance mode. • `SUSPEND_VMS`. Indicates that the virtual machines must be suspended before the standalone host enters maintenance mode. • `SUSPEND_VMS_TO_MEMORY`. Indicates that the virtual machines must be suspended before the standalone host enters maintenance mode.
`setFailureAction(failureAction)failure_action`	Specify what actions vSphere Lifecycle Manager must take if a standalone host fails to enter maintenance mode during the remediation. You can set the number of times that vSphere Lifecycle Manager tries to put a standalone host into maintenance mode and the delay between the tries. When the threshold is reached and the standalone host failed to enter maintenance mode, the host remediation fails.

Remediating an ESX Cluster and a Standalone Host

You can use the VMware Cloud Foundation API to initiate the remediation of a cluster or a standalone host through the vSphere Lifecycle Manager.

To remediate a cluster with a desired state, call the `apply_Task(cluster_ID, apply_spec)` method of the `com.vmware.esx.settings.clusters.Software` interface. Pass as argument a `SoftwareTypes.ApplySpec` instance and specify whether the VMware End User License Agreement (EULA) must be acceptedeuse the `POST https://<vcenter_ip_address_or_fqdn>api/esx/settings/clusters/<cluster_id>/software?action=apply&vmw-task=true` request and submit a `Software.ApplySpec` JSON object in the request body. Specify whether the VMware End User License Agreement (EULA) must be accepted. You can also set the minimum commit ID of the draft software specification that must be used for remediating the cluster. Upon successful completion of the remediation task, all hosts in the cluster have the same software state.

To remediate a standalone host with a desired state, call `apply_Task(host_ID, apply_spec)` method of the `com.vmware.esx.settings.hosts.Software` interface and pass as arguments the standalone host ID and an instance of the `SoftwareTypes.ApplySpec` classuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=apply&vmw-task=true` request and submit a `Software.ApplySpec` JSON object in the request body. Use the software apply specification to set the minimum commit identifier of the draft software specification. See Committing the Draft Software Specification. You can also specify whether the VMware End User License Agreement (EULA) must be accepted.

Integrate Third-Party Solutions with vSphere Lifecycle Manager

A solution is an ESX software package that extends the functionality and capabilities of a host and integrates with the vCenter system. To be able to manage the life cycle of your third-party solutions on a cluster managed with images, you must integrate the solutions with vSphere Lifecycle Manager.

Examples of VMware integrated solutions are VMware NSX, vSAN, and vSphere Supervisor. For more information about the VMware integrated solutions, see *vSphere Lifecycle Manager Images and Other VMware Products and Solutions* section in the *Managing Host and Cluster Lifecycle* documentation.

Third-party software providers can use the vSphere APIs, VMware vSphere APIs for I/O Filtering (VAIO), VMware Daemon Software Development Kit, and others to develop third-party solutions for their vSphere platforms.

You can use the vSphere Lifecycle Manager automation APIs to manage the life cycle of third-party solutions on a cluster managed with a single image. First you need to package and upload your solution components to the vSphere Lifecycle Manager depot. Use the depot to store and manage the software updates for your third-party solutions. To make a solution available on a cluster, create a software specification that contains the solution and remediate all hosts in the cluster with that image. vSphere Lifecycle Manager manages the life cycle of the solution components by consuming the software updates from the vSphere Lifecycle Manager depot.

> **Note:**
> If you export an image from the cluster where your third-party solution is running, the solution components are not part of the exported image.

Prerequisites

To enable vSphere Lifecycle Manager to manage your third-party solutions, you must use the ESX Packaging Kit (EPK) to create installable packages. As of the vSphere 7.0 release, partner development kits generate components as installation packages. For more information about how to use the EPK to create components, assemble components into add-ons, then merge the add-ons with the base image to author a depot, see the *ES Packaging Kit (EPK) Development Guide* documentation.

Integrate a Third-Party Solution to Work with vSphere Lifecycle Manager

1. Create an online or offline depot to host your third-party solutions. See Working with Online Depots and Working with Offline Depots.
   - The following example creates an online depot that can be accessed through the `http://my_online_depot.com` URL.

     ```
     POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/depots/online
      {
         "description" : "My online depot adds the My Solution component to the ESX 7.0U3d base image",
         "ownerdata" : "ACME Company",
         "location" : "http://my_online_depot.com",
         "enabled" : true
     }
     ```

   - The following example creates an offline depot, also called an offline bundle, that can be imported to the vSphere Lifecycle Manager depot.

     ```
     POST https://{server}/api/esx/settings/depots/offline?vmw-task=true

     {
         "file_id" : "string",
         "description" : "string",
         "ownerdata" : "string",
         "source_type" : "PULL",
         "location" : "http://myurl.com"
     }
     ```

2. Synchronize the online depot to download the depot metadata and make the vSphere Lifecycle Manager aware of your solutions. See Synchronizing Software Depots.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/depots/online?vmw-task=true&action=sync
```

3. Create a draft software specification. See Creating a Draft Software Specification.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts
```

4. Add your solution to the created draft software specification. See Editing a Draft Software Specification.

```
PUT https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/
software/components/<component_id>
```

5. Save the created draft software specification to make it the desired state for the cluster. See Committing the Draft Software Specification.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/
<draft_id>?action=commit&vmw-task=true
```

6. Remediate the ESX cluster with the desired state that contains your solution to apply the desired state on all hosts in that cluster. See Remediating an ESX Cluster and a Standalone Host.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=apply&vmw-
task=true
{
    "hosts" : [
        "obj-103",
        "obj-103"
    ],
    "commit" : "obj-103",
    "accept_eula" : true
}
```

Results

You set up an image for the cluster which includes your third-party solution. You now manage all hosts in the cluster collectively with a single image. Upon remediation, the image is installed on all hosts in the cluster.

What to do next

You can update, delete or add new solutions to the draft software specification and then commit the changes to make the draft the desired state for the cluster.

Enable vSphere Lifecycle Manager on a Cluster Managed with Baselines

For more information about how to convert a cluster to use vSphere Lifecycle Manager images instead of baselines, see Enabling an Existing Cluster to Use vSphere Lifecycle Manager.

vSphere Lifecycle Manager APIs Equivalent to the vSphere Host Patch Manager APIs

To manage the life cycle of a single host with baselines, you use the vSphere Host Patch Manager APIs which are part of the vSphere Web Services API. To manage the life cycle of all hosts in a cluster collectively with an image, you use the vSphere Lifecycle Manager APIs which are part of the VMware Cloud Foundation APIs. The following table compares the APIs for life cycle management of the hosts and clusters in your environment.

Operation	vSphere Lifecycle ManagerAPI	vSphere Host Patch Manager API
Check whether hosts can be remediated	`com.vmware.esx.settings.clusters.Software.check`	`vim.host.PatchManager.CheckHostPatch_Task`
Check the compliance of the cluster or host with the desired state	`com.vmware.esx.settings.clusters.Software.scan`	`vim.host.PatchManager.ScanHostPatchV2_Task`
Remediate the cluster or host with the desired state	`com.vmware.esx.settings.clusters.Software.apply`	`vim.host.PatchManager.InstallHostPatchV2_Task`

Operation	vSphere Lifecycle ManagerAPI	vSphere Host Patch Manager API
Retrieve information about the cluster or host that you want to remediate	• `com.vmware.esx.settings.clusters.Software.get` - View information about the current desired state. • `com.vmware.esx.settings.clusters.software.Components.list` - View all components in the current desired state. • `com.vmware.esx.settings.clusters.software.Components.get` - View detailed information about a specific component.	`vim.host.PatchManager.QueryHostPatch_Task` - View information about the bulletins installed on an ESX host.
Stage a desired state on a cluster or host	`com.vmware.esx.settings.clusters.Software.stage`	`vim.host.PatchManager.StageHostPatch_Task`
Uninstall a component	`com.vmware.esx.settings.clusters.Software.apply` - When a software component is removed from the desired software specification and then the cluster is remediated, the component will be uninstalled from all hosts in that cluster.	`vim.host.PatchManager.UninstallHostPatch_Task`

Virtual Machine Configuration and Management

A virtual machine is a software computer that, like a physical computer, runs an operating system and applications.

The virtual machine consists of a set of specification and configuration files and is backed by the physical resources of a host. Each virtual machine encapsulates a complete computing environment and runs independently of the underlying hardware.

Starting with vSphere 6.5, you can create virtual machines, configure virtual machine settings, and perform power operations on the virtual machines through the VMware Cloud Foundation APIs.

Starting with vSphere 7.0, you can use the VMware Cloud Foundation APIs to perform various virtual machine management operations. For example, you can deploy virtual machines by using several approaches, clone an existing virtual machine, create an instant clone of a running virtual machine. You can also install VMware Tools which enables you to manage the life cycle and customize the networking and identity settings of the guest operating system installed on the virtual machine.

Creating Virtual Machines

You can use the VMware Cloud Foundation APIs to create virtual machines depending on your needs and infrastructure setup.

You can create a basic virtual machine and then configure it according to your needs. You can also create a more comprehensive virtual machine and then edit its settings. To create a virtual machine, you must specify the datastore, resource pool, folder, or host where the virtual machine is placed. Later, you can customize the virtual machine by specifying the boot options, number of CPUs, the guest OS, and virtual NIC. See Creating a Virtual Machine Without a Clone or Template and Configuring Virtual Machines.

You can use a turned off virtual machine to create a VM template from which to deploy other virtual machines. See Create a VM Template in a Content Library from a Virtual Machine. You can also mark a virtual machine as template by calling the `VirtualMachine.MarkAsTemplate` method from the vSphere Web Services APIs. See *vSphere Web Services SDK Programming Guide*.

You can capture a virtual machine or a vApp in an OVF template and store the template in a content library. Then you can use the OVF template to deploy a virtual machine or a vApp in your environment. See Creating Virtual Machines and vApps from Templates in a Content Library.

Creating a Virtual Machine Without a Clone or Template

You can create a virtual machine by using the REST API.

When you create a virtual machine without a template or clone, you can configure the virtual hardware, including processors, hard disc, memory. To create a virtual machine, you must specify the virtual machine attributes by using the `VMCreateSpec` data structure. For example, you can specify a name, boot options, networking, and memory for the new virtual machine.

All attributes are optional except the virtual machine placement information that you must provide by using the `VMPlacementSpec` data structure. Use the virtual machine placement specification to set the datastore, cluster, folder, host, or resource pool of the created virtual machine. You must make sure that all these vSphere objects are located in the same data center in a vCenter instance.

For more information, refer to the *API Reference* documentation.

Configuring Virtual Machines

You configure a virtual machine in the process of creation.

You can later view and edit virtual machine settings by adding or changing the type of the storage controllers, configure the virtual disks, boot options, CPU and memory information, or networks. Virtual machine settings can be configured when cloning, registering and relocating an existing virtual machine.

Name and Location

You specify the display name and the location of the virtual machine by using the `VMCreateSpec` and `VMPlacementSpec` data structures.

When you create your virtual machine, use the `name` property of the `VMCreateSpec` data structure to set the display name of the virtual machine.

You must create also a `VMPlacementSpec` instance that describes the location of the virtual machine in regards to the resources of a given vCenter instance. Use the `placement` property of the `VMCreateSpec` data structure to set the placement information for the virtual machine. You can set one or all of the following vSphere resources: datastore, cluster, folder, host, and resource pool.

Hardware Version

The hardware version of a virtual machine reflects the virtual hardware features supported by a virtual machine. These features depend on the physical hardware available on the ESX host on which the virtual machine is running.

Virtual hardware features include the BIOS and Extensible Firmware Interface (EFI), the maximum number of CPUs, the maximum memory configuration, and other hardware characteristics.

When you create a virtual machine, the default hardware version of the virtual machine is the most recent version available on the host where the virtual machine is created. For information about the latest VMware products and virtual hardware versions, see Virtual machine hardware versions (1003746).

To set a hardware version different than the default, use the `hardware_version` enumerated type of the `VMCreateSpec` data structure. For information about the hardware features available for the virtual hardware versions, see Hardware features available with virtual machine compatibility settings (2051652).

You can set a lower virtual hardware version of a virtual machine than the highest supported by the ESX host on which the virtual machine is running. Setting a lower hardware version can provide flexibility and is useful in the following cases.

- To help you standardize testing and deployment in your environment.
- In case you do not need the hardware features of the latest hardware version of the host.
- To maintain compatibility with hosts with a lower hardware version.

Boot Options

You can configure the boot options of a virtual machine by using the `VmHardwareBootCreateSpec` data structure.

By specifying the properties of the `VmHardwareBootCreateSpec` data structure, you can select one of the following settings when booting the virtual machine.

- Delay - Indicates a delay in milliseconds before starting the firmware boot process when the virtual machine is powered on.
- Retry - Indicates whether the virtual machine automatically retries to boot after a failure.
- Retry delay - Indicates a delay in milliseconds before retrying the boot process after a failure.
- Enter setup mode - If set to `true`, indicates that the firmware boot process automatically enters BIOS setup mode the next time the virtual machine boots. The virtual machine resets this flag to `false` once it enters setup mode.
- EFI legacy boot - If set to `true`, indicates that the EFI legacy boot mode is used.

Guest Operating System

The guest operating system that you specify affects the supported devices and available number of virtual CPUs.

When you create a virtual machine, you specify the guest operating system by using the `VmGuestOS` enumerated type of the `VMCreateSpec` data structure. The `VmGuestOS` enumerated type defines the valid guest OS types that you can choose from for configuring a virtual machine.

After the create operation finishes successfully, you can install the guest operating system on the mew virtual machine in the same way as you install it on a physical machine. For further information on installing a guest operating system, refer to the *Guest Operating System Installation Guide* at http://partnerweb.vmware.com/GOSIG/home.html and the *vSphere Virtual Machine Administration* guide.

Starting with vSphere 7.0, you can use the VMware Cloud Foundation APIs to install the VMware Tool on the quest operating system and perform some guest OS customizations. See Installing VMware Tools.

CPU and Memory

The `VMCreateSpec` data structure allows you to specify the CPU and memory configuration of a virtual machine.

To change the CPU and memory configuration settings, use the `VmHardwareCpuUpdateSpec` and `VmHardwareMemoryUpdateSpec` data structures.

You can set the number of CPU cores in the virtual machine by using the `count` property of the `VmHardwareCpuUpdateSpec` data structure. The supported range of CPU cores depends on the guest operating system and virtual hardware version of the virtual machine. If you set the `hot_add_enabled` and `hot_remove_enabled` properties to `true`, you allow virtual processors to be added or removed from the virtual machine at runtime.

You can set the memory size of a virtual machine by using the `size_MiB` property of the `VmHardwareMemoryUpdateSpec` data structure. The supported range of memory sizes depends on the configured guest operating system and virtual hardware version of the virtual machine. If you set the `hot_add_enabled` property to `true` while the virtual machine is not powered on, you enable adding memory while the virtual machine is running.

Networks

You configure network settings so that a virtual machine can communicate with the host and with other virtual machines. When you configure a virtual machine, you can add network adapters (NICs) and specify the adapter type.

You can add virtual Ethernet adapters to a virtual machine by using the `VmHardwareEthernetCreateSpec` data structure. The `VmHardwareEthernetEmulationType` enumerated type defines the valid emulation types for a virtual Ethernet adapter. The `VmHardwareEthernetMacAddressType` enumerated type defines the valid MAC address origins for a virtual Ethernet adapter. You can set the MAC address type to `MANUAL`, `GENERATED`, or `ASSIGNED`. To specify the MAC address explicitly, select `MANUAL`.

You can specify also the physical resources that back a virtual Ethernet adapter by using the `VmHardwareEthernetBackingSpec` data structure. The `VmHardwareEthernetBackingType` enumerated type defines the valid backing types for a virtual Ethernet adapter. You can set the backing type to `STANDARD_PORTGROUP`, `HOST_DEVICE`, `DISTRIBUTED_PORTGROUP`, or `OPAQUE_NETWORK`.

Managing Virtual Machines

Virtual machines can be configured like physical computers.

You can change the guest operating system settings after installing VMware Tools. You can add and remove virtual machines from the vCenter inventory. You can also move virtual machines from one host or storage location to another.

Filtering Virtual Machines

You can retrieve commonly used information about virtual machines that match specific criteria. You can retrieve information for up to 4000 virtual machines in a single vCenter instance.

You can retrieve a list of virtual machines in a single vCenter instance by filtering the results based on a specific requirement. For example, you can use as filter criteria the power state of the virtual machines, or the host, cluster, data center, folder, or resource pool that must contain the virtual machines. In case you specify multiple filter criteria, only virtual machines that match all filter criteria are returned.

Installing VMware Tools

VMware Tools is a set of drivers and utilities that you install on the guest operating system of a virtual machine to enhance the performance of the guest OS. VMware Tools also improves the management of the virtual machine. For each guest OS, VMware provides a specific binary-compatible version of VMware Tools.

Before you install VMware tools, you must install and boot the guest operating system.

To mount and unmount the VMware Tools installer CD as a CD-ROM for the guest operating system, use the connect and disconnect operations of the `tools/installer` service.

To mount the VMware Tools, use the `POST https://<api_host>/api/vcenter/vm/<vm>/tools/installer?action=connect` HTTP request. On Windows guest operating systems with activated Autorun feature, this request automatically initiates the installation of the VMware Tools and requires a user input to complete. On other guest operating systems, this method only mounts the VMware Tools disk image on the virtual CD/DVD drive and the user is required to do some guest OS-specific actions. For example, for some Linux distributions, the user is required to extract the contents of the VMware Tools installation archive and run the installer.

To unmount the VMware Tools installer CD image from the virtual CD/DVD drive, use the `POST https://<api_host>/api/vcenter/vm/<vm>/tools/installer?action=disconnect` HTTP request.

To monitor the status of the VMware Tools installation, you can check the `Tools.Info.version-status` and `Tools.Info.run-state` from `Tools.get`.

To upgrade the VMware Tools, use the `POST https://<api_host>/api/vcenter/vm/<vm>/tools?action=upgrade` HTTP request. The request takes as arguments the ID of the virtual machine on which the VMware Tools is installed and running. Use the `command_line_options` string to specify the command-line options that you want to pass to the installer to modify the tools installation procedure. You can monitor the upgrade operation in the same way you monitor the installation operation.

To update the properties of the VMware Tools, use the `PATCH https://<api_host>/api/vcenter/vm/<vm>/tools` HTTP request. Pass as argument a `VmToolsUpdateSpec` data structure and define the tools upgrade policy settings for the virtual machine by using the `VmToolsUpgradePolicy` enumerated type.

Performing Virtual Machine Power Operations

You can start, stop, reboot, and suspend virtual machines by using the operations of the `Power` service.

A virtual machine can have one of the following power states:

- `POWERED_ON` - Indicates that the virtual machine is running. If a guest operating system is not currently installed, you can perform the guest OS installation in the same way as for a physical machine.
- `POWERED_OFF` - Indicates that the virtual machine is not running. You can still update the software on the physical disk of the virtual machine, which is impossible for physical machines.
- `SUSPENDED` - Indicates that the virtual machine is paused and can be resumed. This state is the same as when a physical machine is in standby or hibernate state.

To perform a power operation on a virtual machine, you can use one of the operations of the `Power` service. Before you use one of the operations to change the power state of a virtual machine, you must first check the current state of the virtual machine by using the `GET https://<api_host>/api/vcenter/vm/<vm>/power` HTTP request. Pass as argument the virtual machine identifier.

Following is a list if the power operations, which you can use with a `POST` HTTP request:

- `power?action=start` - Powers on a powered off or suspended virtual machine. The method takes as argument the virtual machine identifier.
- `power?action=stop` - Powers off a powered on or suspended virtual machine. The method takes as argument the virtual machine identifier.
- `power?action=suspend` - Pauses all virtual machine activity for a powered on virtual machine. The method takes as argument the virtual machine identifier.
- `power?action=reset` - Shuts down and restarts the guest operating system without powering off the virtual machine. Although this method functions as a `stop` operation that is followed by a `start` operation, the two operations are atomic with respect to other clients, meaning that other power operations cannot be performed until the `reset` operation completes.

Registering and Unregistering Virtual Machines

When you create a virtual machine, it becomes part of the vCenter inventory and is registered to the host and vCenter. If you remove a virtual machine from the vCenter inventory, it becomes unusable. Virtual machine files remain in the same datastore but you cannot power on the virtual machine when it is not registered in the inventory.

You can temporarily remove a virtual machine from vCenter by unregistering it. Virtual machine files are not deleted from the datastore. Though all high-level information about the virtual machine such as statistics, resource pool association, permissions, and alarms, is removed from the host and the vCenter instance. To remove a virtual machine from the inventory, use the `POST https://<api_host>/api/vcenter/vm/<vm>?action=unregister` HTTP request and specify the ID of the virtual machine.

To restore a virtual machine to the vCenter inventory, and make it usable again, use the `POST https://<api_host>/api/vcenter/vm/<vm>?action=register` HTTP request. You pass as argument a `VMRegisterSpec` data structure that contains information about the current location of the virtual machine files on the datastore. You can also define the location within the vCenter inventory, for example, the cluster, folder, or the host, where you want to register the virtual machine. After registration, the virtual machine takes its resources (CPU, memory, and so on) from the resource pool or host to which it is registered.

If you no longer need a virtual machine and you want to free up datastore space, you can permanently delete a virtual machine from the inventory. Use the `DELETE https://<api_host>/api/vcenter/vm/<vm>` HTTP request and specify the ID of the virtual machine. Upon a successful completion of the operation, the virtual machine files are removed from the datastore, including the configuration file and the virtual disk files.

Managing Data Sets

You can use data sets to share information between a virtual machine and its guest operating system.

Data sets provide a communication infrastructure that you can use to share information between the host and guest. This information can then be used by guest agents or applications to configure the guest itself or guest applications. The information is grouped into data sets, each of which contains key-value entries comprising the data. Each application that uses the service should have at least one unique data set in which to store its data to avoid conflict with other applications. Each data set has attributes that define its access control and interoperability configuration.

> **Note:**
> You should not store sensitive data, such as passwords or private keys, in plain text.

To perform data set operations, the virtual machine must be running virtual hardware version 20 or later.

You should modify each data set by using the application that originally created that data set. If your application modifies a data set owned by another application, the other application might stop working.

For more information about data sets, see *vSphere Virtual Machine Administration* and the *VMware Guest SDK Programming Guide*.

Data Set Operations

You can use the VMware Cloud Foundation API to manipulate entries in data sets.

You can use data set operations to share information between a virtual machine and its guest operating system. The available operations include listing data sets, retrieving data set information, creating, modifying, and deleting a data set. You can also manipulate individual entries in a data set. The available operations include listing entry keys in a data set, retrieving the value of an entry in a data set, modifying and deleting an entry in a data set.

Table 1858: User Operations

Operation	Description
List data sets	You can list all available data sets of a virtual machine.
Get data set	You can retrieve information about a specific data set.
Create data set	You can create a new data set.
Update data set	You can modify the attributes of a data set.
Delete data set	You can delete a data set.
List data set entry keys	You can list all entry keys in a data set.
Get data set entry key value	You can retrieve the value of a specific entry in a data set.
Set data set entry key value	You can create or update an entry in a data set. If the new entry does not contain a key that matches an existing key, the operation creates a new entry. If the new entry contains a key that matches an existing key, the new key overwrites the value of the existing key.
Delete data set entry key value	You can delete an entry in a data set.

HTTP Requests for Data Set Operations

You can use HTTP requests to retrieve, create, update, or delete data sets and date set entries.

HTTP Requests

The following HTTP requests show the syntax that you can use to perform the available user operations.

- List data sets

  ```
  GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets
  ```

- Get data set

  ```
  GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>
  ```

- Create data set

  ```
  POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets
  ```

- Update data set

  ```
  PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>
  ```

- Delete data set

  ```
  DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>?force={true}
  ```

- List data set entry keys

  ```
  GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>/entries
  ```

- Get data set entry key value

  ```
  GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>/entries/<key>
  ```

- Set data set entry key value

  ```
  PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>/entries/<key>
  ```

- Delete data set entry key value

  ```
  DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm/<vm>/data-sets/<dataSet>/entries/<key>
  ```

For information about the content and syntax of the HTTP query parameters, path parameters, and request body parameters, see the *API Reference* documentation.

Using Content Libraries

Use the VMware Cloud Foundation API to access the Content Library service and use it to store and share different types of content in your vSphere environment. Content libraries are vSphere container objects for storing and sharing OVF and OVA packages, VM templates, vApp templates, and other types of files.

You can use content libraries to share VM and vApp templates, and other types of files, such as ISO images, text files, and so on, across the vCenter instances in the same or different locations. Sharing templates across your virtual environment promotes consistency, compliance, efficiency, and automation in deploying workloads at scale.

You use library items to store and manage content in a content library. A single library item can contain one or more files. For example, an OVF package consists of a `.vmdk` file, manifest file, and others, but it is represented by a single library item.

Starting with vSphere 7.0 Update 3, you can add a security policy to a local or subscribed content library and thus protect your system when you synchronize or download library content from third party providers.

What Is a Content Library

A content library instance represents a container for a set of library items. A content library item instance represents the logical object stored in the content library, which might be one or more usable files.

You create and manage the content of a content library on a single vCenter instance, but you can distribute the content to other vCenter instances. Depending on your needs, you can maintain two types of content libraries: local and subscribed. You can shape the contents of a library item and then combine several library items in a local content library. Furthermore, you can publish the library to make its content available to other users.

Content Library Types

You can create two types of libraries, local and subscribed.

Local library.	You can create a local library as the source for content you want to save or share. Create the local library on a single vCenter instance. You can add items to a local library or remove them. You can publish a local library and as a result this content library service endpoint can be accessed by other vCenter instances in your virtual environment. When you publish a library, you can configure a password for authentication.
Subscribed library.	You can create a subscribed library and populate its content by synchronizing to a local library. A subscribed library contains copies of the local library files or just the metadata of the library items. The local library can be located on the same vCenter instance as the subscribed library, or the subscribed library can reference a local library on a different vCenter instance. You cannot add library items to a subscribed library. You can only add items to the source library. After synchronization, both libraries will contain the same items.

Content Library Items

Library items are VM templates, vApp templates, or other VMware objects that can be contained in a content library.

VMs and vApps have several files, such as log files, disk files, memory files, and snapshot files that are part of a single library item. You can create library items in a specific local library or remove items from a local library. You can also upload files to an item in a local library so that the libraries subscribed to it can download the files to their NFS or SMB server, or datastore.

For information about the tasks that you can perform by using the content library service, see How to Create and Publish Content LibrariesWorking with Content Libraries.

Content Library Storage

When you create a local library, you can store its contents on a datastore managed by the vCenter instance or on a remote file system.

Depending on the type of storage that you have, you can use Virtual Machine File System (VMFS) or Network File System (NFS) for storing content on a datastore.

For storing content on a remote file system, you can enter the path to the NFS storage that is mounted on the Linux file system of the vCenter instance. For example, you can use the following URI formats: `nfs://<server>/<path>?version=4` and `nfs://<server>/<path>`. If you have a vCenter instance that runs on a Windows machine, you can specify the Server Massage Block (SMB) URI to the Windows shared folders that store the library content. For example, you can use the following URI format: `smb://<unc-server>/<path>`.

Java

This example is based on the code in the `LibraryCrud.java` sample file and shows how to store library content on a datastore.

For more information about storing the contents of a local library, see Content Library Storage.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a StorageBacking instance for storing the library content on a datastore.

     StorageBacking libraryBacking = new StorageBacking();
     libraryBacking.setType(Type.DATASTORE);

/*
 * Pass the value of the datastore ManagedObjectReference.
 * See the vSphere Web Services SDK Programming Guide
 * and the vSphere Web Services SDK samples. In addition, the  vSphere Automation SDK
 for Java provides
 * the VimUtil utility class in the vmware.samples.vim.helpers package.
 * You can use the utility to retrieve the ManagedObjectReference
 * of the datastore entity.
 */
     libraryBacking.setDatastoreId("datastore-123");

// Create a LibraryModel that represents a local library backed on a datastore.

      LibraryModel libraryModel = new LibraryModel();
      libraryModel.setName("AcmeLibrary");
      libraryModel.setDescription("Local library backed by VC datastore");
      libraryModel.setType(LibraryType.LOCAL);
      libraryModel.setStorageBackings(Collections.singletonList(libraryBacking));
```

Python

This example is based on the code in the `library_crud.py` sample file and shows how to store library content on a datastore.

For more information about storing the contents of a local library, see Content Library Storage.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a StorageBacking instance of datastore type.
library_backing = library_client.StorageBacking()
library_backing.type = library_client.StorageBacking.Type.DATASTORE

# Pass the value of the datastore managed object reference.
```

```
# The  vSphere Automation SDK for Python contains
# the GetDatastoreByName class, which sample resource is located in the
# /client/samples/src/com/vmware/vcloud/suite/sample/vim/helpers/
 directory.
# You can use the utility to retrieve the managed object reference of the datastore
 entity.
library_backing.datastore_id = 'datastore-123'

# Create a LibraryModel that represents a local library backed on a datastore.
library_model = content_client.LibraryModel()
library_model.name = 'AcmeLibrary'
library_model.storage_backings = [library_backing]
```

Querying Content Libraries

You can create queries to find libraries that match your criteria. You can also retrieve a list of all libraries or only the libraries of a specific type.

Listing All Content Libraries

You can retrieve a list of all content library IDs in your virtual environment, regardless of their type, by using the `Library` service. You can use the `List Library` operation to retrieve all local and subscribed libraries in your system.

Listing Content Libraries of a Specific Type

You can use the VMware Cloud Foundation API to retrieve content libraries of a specific type. For example, you can list only the local libraries in your virtual environment.

If you want to retrieve only a list of the local libraries, you must retrieve the `LocalLibrary` service and use the `List Local Library` operation on the `LocalLibrary` service. To list only subscribed libraries, you must retrieve the `SubscribedLibrary` service and call the `List Subscribed Library` operation on the `SubscribedLibrary` service.

Listing Content Libraries by Using Specific Search Criteria

You can filter the list of content libraries and retrieve only the libraries that match your specific criteria. For example, you might want to publish all local libraries with a specific name.

To filter with specific search criteria, use the `Find Library` operation of the `Library` service. Specify the properties of the `LibraryFindSpec` data structure that contains your search criteria. Upon a successful completion of the operation, you receive a list of all content libraries that match your search criteria.

Working with Content Libraries

The Content Library API provides services that you can use to create and manage content libraries.

You can create a local library and can publish it for the entire virtual environment. You can also subscribe to use the contents of a local library and enable automatic synchronization to ensure that you have the latest content.

You can perform content library operations by using HTTP requests. Some operations require you to specify parameters in the body of the HTTP request according to your environment. For details about the syntax of each HTTP request body, see the *API Reference* documentation.

Creating a Local Content Library

You can create a local content library by using the `POST https://<vcenter_ip_address_or_fqdn>/api/content/local-library` HTTP request.

Publishing an Existing Content Library

To make the library content available for other vCenter instances across the VMware Cloud Foundation environment, you must publish the library. Depending on your workflow, select a method for publishing the local library. You can publish a local library that exists in your VMware Cloud Foundation environment by using the `POST https://<vcenter_ip_address_or_fqdn>/api/content/local-library/<library_id>?action=publish` HTTP request.

Subscribing to a Content Library

You can subscribe to local content libraries. When you subscribe to a library, you must specify the backing storage for the library content. If the library requires basic authentication, you must also provide the correct user name and password. You can create a subscribed library by using the `POST https://<vcenter_ip_address_or_fqdn>/api/content/subscribed-library` HTTP request.

Synchronizing a Subscribed Content Library

When you subscribe to a published library, you can configure the settings for downloading and updating the library content. You can force the synchronization of a subscribed library by using the `POST https://<vcenter_ip_address_or_fqdn>/api/content/subscribed-library/<library_id>?action=sync` HTTP request.

Editing the Settings of a Content Library

You can update the settings of content library types in your virtual environment. You can update the settings of a local library by using the `PATCH https://<vcenter_ip_address_or_fqdn>/api/content/local-library/<library_id>` HTTP request. You can update the settings of a subscribed library by using the `PATCH https://<vcenter_ip_address_or_fqdn>/api/content/subscribed-library/<library_id>` HTTP request.

Removing the Content of a Subscribed Library

You can free storage space in your virtual environment by removing the subscribed library content that you no longer need.

You can create a subscribed library with the option to download the library content on demand. As a result, only the metadata for the library items is stored in the associated with the subscribed library storage. When you want to deploy a virtual machine from a VM temple in the subscribed library, you must synchronize the subscribed library to download the entire published library content. When you no longer need the VM template, you can use the `POST https://<vcenter_ip_address_or_fqdn>/api/content/subscribed-library/<library_id>?action=evict` HTTP request. You must provide the subscribed library ID to this function. As a result, the subscribed library content that is cached on the backing storage is deleted.

If the subscribed library is not configured to synchronize on demand, an exception is thrown. In this case the subscribed library always attempts to have the most recent published library content.

Deleting a Content Library

When you no longer need a content library, you can use the `Delete` operation on either the `Local Library` or the `Subscribed Library` service depending on the library type. You can delete a local library by using the `DELETE https://<vcenter_ip_address_or_fqdn>/api/content/local-library/<library_id>` HTTP request. You can delete a subscribed library by using the `DELETE https://<vcenter_ip_address_or_fqdn>/api/content/subscribed-library/<library_id>` HTTP request.

As of VCF 9.1, users are prevented from deleting a content library if it contains associations to another content library. Attempting to delete an in-use content library throws error `NotAllowedInCurrentState`. Users with sufficient privilege can override this guardrail with `forceDelete`.

Working with Library Items

A library item groups multiple files within one logical unit. You can perform various tasks with the items in a content library.

You can upload files to a library item in a local library and update existing items. You can download the content of a library item from a subscribed library and use the item, for example, to deploy a virtual machine. You can remove the content of a library item from a subscribed library to free storage space and keep only the metadata of the library item. When you no longer need local library items, you can delete them and they are removed from the subscribed library when a synchronization task is completed.

You can create a library item from a specific item type, for example, `.ovf` and VM template. The Content Library service must support the library item type to handle the item correctly. If no support is provided for a specified type, the Content Library service handles the library item in the default way, without adding metadata to the library item or guiding the upload process. For information about the supported VM template types, see the *vSphere Virtual Machine Administration* documentation.

Creating an Empty Library Item

You can create as many library items as needed and associate them with a local content library. You can create a library item by using the `POST https://<vcenter_ip_address_or_fqdn>/api/content/library/item` HTTP request.

Querying Library Items

You can perform numerous query operations on library items.

You can retrieve a list of all items in a library, retrieve a library item that has a specific type or name, and find a library item that is not cached on the disk. You can then update the library item content from the subscribed library.

You can use the `GET https://<vcenter_ip_address_or_fqdn>/api/content/library/item?library_id` HTTP request to retrieve a list of all items in a particular library.

You can filter the items contained in a library and retrieve only the items matching specific criteria. For example, you might want to remove or update only specific items in a library. You can use the `POST https://<vcenter_ip_address_or_fqdn>/api/content/library/item?action=find` HTTP request to filter items in a particular library.

Editing the Settings of a Library Item

You can use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/content/library/item/<library_item_id>` HTTP request to edit the properties of a library item such as name, description, and type.

Uploading a File from a Local System to a Library Item

You can upload different types of files from a local system to a library item that you want to use in the VMware Cloud Foundation environment.

To upload a file, you must create an empty library item and verify that you have access to the `UpdateSession` and `File` services.

1. Use the `LibraryItemUpdateSessionModel` data structure to track the changes that you make to the library item.
2. Create an update session by using the `UpdateSession` service.
3. Use the `LibraryItemUpdatesessionFileAddSpec` data structure to describe the upload method and other properties of the file to be uploaded.
4. Create the request for changing the item by using the `File` service.
5. Upload the file that is on the local system.
6. Complete and delete the update session to apply the changes to the library item.

Upload a File from a URL to a Library Item

You can upload different types of files from a URL to a library item that you want to use in the VMware Cloud Foundation environment.

To upload a file, you must first create an empty library item and verify that you have access to the `UpdateSession` and `File` services.

1. Use the `LibraryItemUpdateSessionModel` data structure to track the changes that you make to the library item.
2. Create an update session by using the `UpdateSession` service.
3. Create a file specification to describe the upload method and other properties of the file to be uploaded.
4. Specify the location of the file you want to upload by using the `LibraryItemTransferEndpoint` data structure.
5. Add the file source endpoint to the file specification.
6. Create a request for changing the item by using the configured file specification.
7. Complete the update session to apply the changes to the library item.

Downloading Files to a Local System from a Library Item

You might want to download files to a local system from a library item and then make changes to the files before you use them.

1. Create a download session model to specify the item, which contains the file that you want to download.
2. Access the `File` service and retrieve the file that you want to export to your system within the new download session.
3. Prepare the files that you want to download and wait until the files are in the prepared state.
4. Retrieve the download endpoint URI of the files.
5. Download the files by using an HTTP GET request.
6. Delete the download session after all files are downloaded.

Synchronizing a Library Item in a Subscribed Content Library

The items in a subscribed library have features that are distinct from the items in a local library. Synchronizing the content and the metadata of an item in a subscribed library depends on the synchronization mechanism of the subscribed library. You can use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/content/library/subscribed-item/<library_item_id>?action=sync` HTTP request to force the synchronization of an individual library item in a subscribed library.

Removing the Content of a Library Item

You can remove the content from a library item to free space on your storage.

If you create a subscribed library with the option to synchronize library content on demand, only the metadata for the library items is stored. When you want to use the items in the library, you must force synchronization on the items to download their content. When you no longer need the files in an item, you can remove the cached content of the library item and free storage space. You can use the `POST https://<vcenter_ip_address_or_fqdn>/api/content/library/subscribed-item/<library_item_id>?action=evict` HTTP request to evict the cached content of a library item in a subscribed library.

Deleting a Library Item

You can remove a library item from a local library when you no longer need it.

You can use the `DELETE https://<vcenter_ip_address_or_fqdn>/api/content/library/item/<library_item_id>` HTTP request to remove a library item from a library. The item content is asynchronously removed from the storage.

You cannot remove items from a subscribed library. If you remove an item from a local library, the item is removed from the subscribed library when you perform a synchronization task on the subscribed library item.

Content Library Support for OVF and OVA Packages

You can use the objects and methods provided by the Content Library API to manage OVF and OVA packages.

Open Virtualization Format (OVF) is an industry standard that describes metadata about a virtual machine image in an XML format. An OVF package includes an XML descriptor file and optionally disk images, resource files (such as ISO files), manifest files, and certificate files.

An OVA package is a single file that contains all OVF package files in an archived form. After you upload an OVA package to a content library, the OVA file is converted to the standard OVF package.

When you try to upload signed content to a content library, you might receive preview warnings. Signed content can be either OVF or OVA packages that contain manifest and certificate files. If you do not respond to the preview warnings, the upload fails. To complete an upload operation successfully, you must ignore any preview warnings by using the `WarningBehavior` class`LibraryItemUpdatesessionWarningBehavior` data structure.

With the VMware Cloud Foundation API, you can use the OVF package in a content library to deploy virtual machines and vApps on hosts, resource pools, and clusters. You can also use the API to create OVF packages in content libraries from vApps and virtual machines on hosts, resource pools, and clusters.

When you create library items to store OVF packages, you must set the item type to `ovf`. To comply with the specific standards of the OVF packages, the VMware Cloud Foundation API provides the `LibraryItem` class`Item` service.

Working with OVF and OVA Packages in a Content Library

You can upload an OVF or OVA package to a library item by using the `UpdateSession`interfaceservice. You can also download an OVF and OVA packages from a content library to your local file system.

In case you want to upload an OVF package, the location of the content determines whether you can pull the content from a URL or push the content directly to a content library. For information about uploading content to library items, see Upload a File from a Local System to a Library ItemUploading a File from a Local System to a Library Item and Upload a File from a URL to a Library ItemUpload a File from a URL to a Library Item.

To download the files that are included in an OVF or OVA package to your local file system, use the `DownloadSession`interfaceservice. For more information, see Download Files to a Local System from a Library Item Downloading Files to a Local System from a Library Item.

Upload an OVF or an OVA Package from a URL to a Library Item

You can upload an OVF or an OVA package from a Web server to a library item.

- Create a new local content library or retrieve the desired existing content library.
- Required privileges: **Content library** > **Add library** item and **Content library** > **Update files** on the library.

> **Note:**
> If you try to upload a signed OVF package and it returns preview warnings, you must ignore the preview warnings to complete the upload.

1. Create an empty library item.
2. Create an update session object.
3. Create an `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to describe the properties and the upload location of the OVF descriptor file or of the OVA package file.
4. Link the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to the update session.
   All files that are included in the OVF package are automatically uploaded.
5. Complete the asynchronous transfer.

Upload an OVF or OVA Package from a Local File System to a Library Item

You can upload an OVF or OVA package from a local file system. This procedure describes how to use the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure after you have created a library item and initiated an update session.

- Create a new local content library or retrieve the desired existing content library.
- Required privileges: **Content library** > **Add library item** and **Content library** > **Update files** on the library.

> **Note:**
> If you try to upload a signed OVF package and it returns preview warnings, you must ignore the preview warnings to complete the upload.

1. Create a library item.
2. Create an update session.
3. Create an `AddSpec` objecta `LibraryItemUpdatesessionFileAddSpec` data structure to describe the properties and the upload locations of the OVF descriptor file or of the OVA package file.
4. Link the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to the update session.
5. Optional: Create an `AddSpec` objecta `LibraryItemUpdatesessionFileAddSpec` data structure for each VMDK file included in the OVF package.
6. Add all `AddSpec` objects`LibraryItemUpdatesessionFileAddSpec` data structures to the update session.

   If you upload an OVF package and it has a VMDK file included, you must repeat steps 5 and 6. If you are uploading a signed OVF package, steps 5 and 6 must also be repeated for the manifest and certificate files included in the OVF package.
7. Initiate the upload operation.
8. Complete the update session.
9. Delete the session.

Creating Virtual Machines and vApps from Templates in a Content Library

You can create VM and OVF templates from virtual machines and vApps in your inventory. You can then deploy virtual machines and vApps from the templates that are stored in a content library.

Create a VM Template in a Content Library from a Virtual Machine
By using the VMware Cloud FoundationREST API, you can create a VM template in a content library from an existing virtual machine in your vCenter inventory.

- Verify that you have administrative privileges on your vCenter instance.
- Verify that you created a VMware Cloud Foundation session to your vCenter.
- Verify that you created a local library by using the vSphere Client or the VMware Cloud Foundation APIs.

When you call the `create` function of the `com.vmware.vcenter.vm_template.LibraryItems` service, a VM template is created as a library item in your local content library. If the operation is successful, the `LibraryItems` service returns the ID of the newly created library item.

To create a library item that contains a VM template, you can use the `ctreate` function of the `LibraryItems` interface`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request. You can review the information about a VM template by using the `get` function of the `LibraryItems` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>` HTTP request. For information about how to create a VM template by using the vSphere Client, see the *vSphere Virtual Machine Administration* documentation.

For information about the available and mandatory parameters, see the *API Reference* documentation.

1. Get the ID of your ESX host on which you want to store the VM template.

   You can use the `list` function of the `com.vmware.vcenter.Host` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request.
2. Get the ID of the datastore on which you want to store the VM template files.

   You can use the `list` function of the `com.vmware.vcenter_client.Datastore` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore` HTTP request.
3. Get the ID of the virtual machine that you want to save as a VM template.

   You can use the `list` method of the `com.vmware.vcenter_client.VM` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm` HTTP request.
4. Get the ID of your local library.

   You can get the list of the local libraries in your vCenter and review the information about each library by using the `list` and `get(library_id)` functions of the `com.vmware.content_client.LocalLibrary` class`GET`

`https://<vcenter_ip_address_or_fqdn>/api/com/vmware/content/local-library` and `GET https://<vcenter_ip_address_or_fqdn>/api/com/vmware/content/library/id:<library_id>` HTTP requests.

5. Create a library item specification for the VM template.

   You can use the `com.vmware.vcenter.vm_template.LibraryItems.CreateSpec` classbody of the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request.

   a) Specify the local library, source virtual machine, and the name of the library item by using the `library`, `source_vm`, and `name` parameters. You must use the IDs of the local library and source virtual machine.
   b) Specify the placement information for your VM template.

      You can use the `LibraryItems.CreatePlacementSpec` class`placement` parameters in the body of the HTTP request. To specify the host, resource pool, cluster, and folder, you must use their IDs.
   c) Specify the datastore on which you want to store the log, configuration, and disk files of your VM template.

      To specify the storage backing for the VM template, you can use the `com.vmware.vcenter.vm_template.LibraryItems.CreateSpecVmHomeStorage` and `com.vmware.vcenter.vm_template.LibraryItems.CreateSpecDiskStorage` classes`vm_home_storage` and `disk_storage` parameters in the body of the HTTP request. You must use the ID of the datastore.
   d) Include the placement and storage specifications in the library item specification.

6. Create a library item for storing the VM template.

   You can use the `create(spec)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request.

If the operation is successful, the `LibraryItems` service returns the ID of the library item that contains the VM template. For information about the available responses, see the *API Reference* documentation.

- Review the information stored in the library item by using the `get(VM_template_item_ID)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<library_item_id>` HTTP request. If you did not save the ID of the library item holding the VM template, you can check the UUID by using the vSphere Client. The URN ends with the ID of the library item and has the following format: `urn:vapi:com.vmware.content.library.Item:<VMTemplateItemID>`.

Create an OVF Template in a Content Library from a Virtual Machine or vApp

You can create library items from existing virtual machines or vApp. Use those library items later to deploy virtual machines and vApps on hosts and clusters in your vCenter environment.

1. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.DeployableIdentityLibraryItem.DeployableIdentity` instance to specify the source virtual machine or vApp to be captured in an OVF template.
2. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.CreateTargetLibraryItem.CreateTarget` instance to identify the content library where the OVF template is stored.
3. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.CreateSpecLibraryItem.CreateSpec` instance to specify the properties of the OVF template.
4. Initiate a synchronous create operation by invoking the `create` function of the `com.vmware.vcenter.ovf.LibraryItem` serviceusing the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/ovf/library-item` HTTP request.
5. Verify the outcome of the create operation.

Deploy a Virtual Machine from a VM Template in a Content Library

By using the VMware Cloud Foundation APIs, you can deploy a virtual machine from a VM template stored in a content library.

- Verify that you have administrative privileges on your vCenter instance.

- Verify that you created a VMware Cloud Foundation session to your vCenter instance.

To deploy a virtual machine from a VM template in a content library, call the `deploy` function of the `com.vmware.vcenter.vm_template.LibraryItems` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request. You can specify the power state and customize the guest operation system prior to the virtual machine deployment.

For information about the available and mandatory parameters, see the *API Reference* documentation.

1. Review the information stored in the VM template library item.

   You can use the `get(VM_template_item_ID)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface and pass the ID of your VM template item`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>` HTTP request. If you did not save the ID of your item, you can select the UUID of your VM template item by using the vSphere Client. The URN ends with the ID of the item and has the following format: `urn:vapi:com.vmware.content.library.Item:<VMTemplateItemID>`.

2. Get the ID of the host on which you want to deploy the virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Host` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request.

3. Get the ID of the resource pool to which you want to add your virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.ResourcePool` interface`https://<vcenter_ip_address_or_fqdn>/api/vcenter/resource-pool` HTTP request.

4. Get the ID of the `VIRTUAL_MACHINE` folder to which you want to add your virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Folder``GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/folder` HTTP request.

5. Get the ID of the datastore on which you want to store log, configuration, and disk files of the virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Datastore` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore` HTTP request.

6. Create a deployment specification.

   You can use the `com.vmware.vcenter.vm_template.LibraryItems.DeploySpec` classbody of the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request.

   a) Specify a name and description of the virtual machine that you want to deploy.
   b) Specify the place in your inventory on which you want to deploy the virtual machine such as an ESX host, resource pool, and VM folder.

      You can use the `com.vmware.vcenter.vm_template.LibraryItems.DeployPlacementSpec` class`placement` parameter in the body of the request. You must use the IDs of your inventory objects.
   c) Specify the datastore on which you want to store the log, configuration, and disk files of the virtual machine. You must use the ID of the datastore.

      You can use the `DeploySpecVmHomeStorage` and `DeploySpecDiskStorage` classes`vm_home_storage`, and `disk_storage` parameters in the body of the request.
   d) Optional: Specify the guest operating system and hardware customization specifications that you want to apply to the virtual machine during the deployment process. Add this information to the deployment specification.

      You can use the `GuestCustomizationSpec` and `HardwareCustomizationSpec` classes`guest_customization` and `hardware_customization` parameters in the body of the request. You can get a list of the guest operating system customization specifications that are available in your vCenter by using the `list` function of the `com.vmware.vcenter.guest.CustomizationSpecs` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/guest/customization-specs` HTTP request.
   e) Include the placement and storage specifications in the deployment specification.

7. Deploy a virtual machine from your VM template.

   You can use the `deploy(template_library_item, spec)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface by passing the library item ID where the VM

template is stores and the deployment specification`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request.

If the operation is successful, the ID of the deployed virtual machine is returned. For information about the possible exceptions, see the *API Reference* documentation.

Deploy a Virtual Machine or vApp from an OVF Template in a Content Library
You can use the `com.vmware.vcenter.ovf.LibraryItemLibraryItem` service to deploy a virtual machine or vApp on a host, cluster, or resource pool from a library item.

1. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.DeploymentTargetLibraryItem.DeploymentTarget` instance to specify the deployment location of the virtual machine or vApp.
2. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.ResourcePoolDeploymentSpecLibraryItem.ResourcePoolDepl` instance to define all necessary parameters for the deployment operation.
   For example, you can assign a name for the deployed virtual machine or vApp, and accept the End User License Agreements (EULAs) to complete the deployment successfully.
3. Optional: Retrieve information from the descriptor file of the OVF template and use the information during the OVF template deployment.
4. Call the `deploy` method on the `LibraryItem` service Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/ovf/library-item/<ovf_library_item_id>?action=deploy` HTTP request.
5. Verify the outcome of the deployment operation.

vSphere Tag Service

The vSphere Tag Service supports the definition of tags that you can associate with vSphere objects or resources. Tags simplify discovery, filtering, and sorting of inventory objects across your vCenter environment.

The vCenter Automation APIs provide programmatic access to creating and managing vSphere tags in your vCenter inventory.

Broadcom provides programmatic access to tag management through the CIS tagging APIs. VMware Cloud Foundation 9.0 extends this surface with three additional REST endpoints under `/vcenter/tagging/` that support filtered, paginated listing and are the foundation for unified vCenter tag management.

For example, to tag your virtual machines by guest operating system type, you can create a category that is called `operating system`, restrict it to virtual machines with `SINGLE` cardinality, then create tags such as `Windows`, `Linux`, and `Mac OS`. You attach the appropriate tag to each VM to enable filtering and policy-driven workflows across your inventory.

vCenter Automation APIs introduce two parallel namespaces: `/cis/tagging/` and `/vcenter/tagging/`. Use the CIS API for all create, update, and delete operations. Use the `/vcenter/tagging/` REST endpoints when you need filtered or paginated listing.

Creating vSphere Tags

You can use the vSphere Automation API to create tags and attach them as metadata to vSphere inventory. Tags are organized into categories, each tag must belong to exactly one category.

Tags and categories can span multiple vCenter instances.

- With Enhanced Linked Mode, tags and tag categories replicate across all linked vCenter instances.
- With Hybrid Linked Mode, tags and tag categories are maintained across your linked domain. That means the on-premises SDDC and the VMware Cloud on AWS SDDC share tags and tag attributes. For more information about Hybrid Linked Mode, see "Hybrid Linked Mode" in the *VMware Cloud on AWS Product Documentation*.

The workflow for creating tags is:

1. Create a tag category that defines the cardinality and the resource types the tags apply to.
2. Create one or more tags within that category.

Creating a Tag Category

A tag category defines the context in which a set of tags operates. Create a category before you create tags. You can restrict a category to specific resource types or can leave it unrestricted so that its tags apply to any object in the inventory.

A tag category has the following properties:

- `name` - Display name for the category. Must be unique within the vCenter instances.
- `description` - Optional free-text description.
- `cardinality` -
  - `SINGLE` — At most one tag from this category per object.
  - `MULTIPLE` — Multiple tags from this category can be attached to one object.
- `associable_types` - The set of resource types whose objects can receive tags from this category. An empty set means that the category applies to all resource types.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

The category `create()` function returns an identifier that you use when you create a tag for that category. The empty set for the associable types indicates that any object type can be associated with a tag in this category.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
import java.util.Set;
import java.util.HashSet;
import com.vmware.cis.tagging.Category;
import com.vmware.cis.tagging.CategoryTypes;
import com.vmware.cis.tagging.CategoryModel;

// Initialize the client factory — replace null with a trust store in production
VcenterClientFactory factory = new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));

try (VcenterClient client = factory.createClient(username, password, null)) {
    Category categoryService = client.createStub(Category.class);

    CategoryTypes.CreateSpec createSpec = new CategoryTypes.CreateSpec();
    createSpec.setName("favorites");
    createSpec.setDescription("My favorite virtual machines.");
    createSpec.setCardinality(CategoryModel.Cardinality.MULTIPLE);

    // Empty set — category applies to all resource types.
    // To restrict to virtual machines only:
 createSpec.setAssociableTypes(Set.of("VirtualMachine"))
    Set<String> associableTypes = new HashSet<>();
    createSpec.setAssociableTypes(associableTypes);

    // Returns the unique identifier used in all subsequent tag and association calls
    String categoryId = categoryService.create(createSpec);

    System.out.println("Successfully created category with ID: " + categoryId);
}
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

The category `create()` function returns an identifier that you use when you create a tag for that category. The empty set for the associable types indicates that any object type can be associated with a tag in this category.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
import requests
import urllib3
from com.vmware.cis.tagging_client import CategoryModel
from vmware.vapi.vsphere.client import create_vsphere_client

# Disable SSL verification warnings for clean console outputs
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# 1. Setup the secure HTTP session context
session = requests.session()
session.verify = False  # Disable SSL verification ONLY in non-production environments

# 2. Initialize the high-level vSphere Automation Client gateway
vsphere_client = create_vsphere_client(
    server=VC_HOST,
    username=VC_USER,
    password=VC_PASSWORD,
    session=session
)

# 3. Construct the creation payload specification
tc_create_spec = vsphere_client.tagging.Category.CreateSpec(
    name="favorites",
    description="My favorite virtual machines.",
    cardinality=CategoryModel.Cardinality.MULTIPLE,
    associable_types=set()  # Empty set allows the category to apply to all resource
 types universally
)

# 4. Trigger creation via the client stub and capture the unique identifier
category_id = vsphere_client.tagging.Category.create(tc_create_spec)

print(f"Successfully created Category ID: {category_id}")
```

REST

This example demonstrates how to create a tag category.

```
CATEGORY_BODY='{
  "create_spec": {
    "name": "favorites",
    "description": "My favorite virtual machines.",
    "cardinality": "MULTIPLE",
    "associable_types": ["VirtualMachine"]
  }
}'

CATEGORY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/category" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${CATEGORY_BODY}" | jq -r '.')

echo "Created Category ID: $CATEGORY_ID"
```

Note:

To restrict the category to a specific resource type — for example, virtual machines only — set `associable_types` to `["VirtualMachine"]`. Attempting to attach a tag from this category to a resource of a different type returns an error.

Create one or more tags within this category. See Creating a Tag.

Creating a Tag

After you create a tag category, create one or more tags within that category and assign them to inventory objects. Each tag belongs to exactly one category and must have a name that is unique within that category.

A tag has the following properties:

- `name` - Display name for the tag. Must be unique within the category.
- `description` - Optional free-text description.
- `category_id` - The identifier of the parent category.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

This example creates a tag specification and then uses it to create the tag. The tag specification references the category identifier that was returned from the category create operation. Use the returned tag identifier for subsequent operations on the tag.

This example is based on the information that is provided in Creating a Tag.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
Tag taggingService = client.createStub(Tag.class);
TagTypes.CreateSpec spec = new TagTypes.CreateSpec();
spec.setName("red");
spec.setDescription("My favorite color");
spec.setCategoryId(categoryId); // identifier returned from the category create call
String tagId = taggingService.create(spec);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

This example creates a tag specification and then uses it to create the tag. The tag specification references the category identifier that was returned from the category create operation. Use the returned tag identifier for subsequent operations on the tag.

This example is based on the information that is provided in Creating a Tag.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
tag_create_spec = vsphere_client.tagging.Tag.CreateSpec(
    name="red",
    description="My favorite color",
    category_id=category_id,  # identifier returned from the category create call
)
tag_id = vsphere_client.tagging.Tag.create(tag_create_spec)
```

REST

This example demonstrates how to create a tag.

```
# Option A: Single-quote wrap (Recommended for clean JSON strings)
TAG_BODY='{
  "create_spec": {
    "name": "red",
    "description": "My favorite color",
    "category_id": "'"${CATEGORY_ID}"'"
  }
}'
```

```
TAG_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/tag" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${TAG_BODY}" | jq -r '.')

echo "Created Tag ID: $TAG_ID"
```

Attach the tag to a vSphere inventory object. See Creating Tag Associations.

Creating Tag Associations

After you create a tag category and a tag within the category, you can associate the tag with a vSphere managed object or resource.

An association is a link between a tag and an inventory object. It contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object. Tag associations are local to a vCenter instance — a listing request returns only the associations that are stored by that vCenter.

When you associate a tag with an object, the object's type must match one of the associable_types defined for the tag's category. If the category's associable_types is empty, the tag can be attached to any resource type.

An association is a simple link that contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object.

Tag associations are local to a vCenter instance. When you request a list of tag associations from a vCenter system, it enumerates only the associations that it has stored.

When you associate a tag with an object, the object's type must match one of the `associable_types` defined for the tag's category. If the category's `associable_types` is empty, you can attach the tag to any resource type.

Attach a Tag to a Content Library

You can attach a tag to a Content Library using the `DynamicID` to provide the library identifier in the format the CIS tagging API requires.

- Tag and tag category created. See Creating vSphere Tags.
- Content library identifier
- `InventoryService.Tagging.AttachTag` privilege on the target Content Library

1. Construct a dynamic object identifier for the library.

   The CIS tagging API identifies inventory objects using a `DynamicID` — a pair of object type String and `id` String that is derived from the `ManagedObjectReference`. To attach a tag to a vSphere inventory object, you must supply its `DynamicID`.

2. Attach the tag to the content library.

Java

```
TagAssociation tagAssociation = client.createStub(TagAssociation.class);

// Library.RESOURCE_TYPE provides the correct type string for a Content
 Library object
DynamicID libraryDynamicId =
    new DynamicID(Library.RESOURCE_TYPE, myLibrary.getId());
tagAssociation.attach(tagId, libraryDynamicId);
```

Python

```
from com.vmware.vapi.std_client import DynamicID
from com.vmware.content_client import Library

# 1. Construct the DynamicID from the Content Library resource type and
 identifier
library_dynamic_id = DynamicID(
    type=Library.RESOURCE_TYPE,
```

```
    id=my_library.id
)

tag_association_stub = vsphere_client.tagging.TagAssociation()

# 2. Attach the tag using the initialized stub instance
tag_association_stub.attach(
    tag_id=tag_id,
    object_id=library_dynamic_id
)
```

REST

```
# Option A: Using single quotes for the outer JSON string (Recommended)
ATTACH_BODY='{
  "object_id": {
    "type": "com.vmware.content.Library",
    "id": "'"${LIBRARY_ID}"'"
  }
}'

# Option B: Escaping inner double quotes if you prefer one continuous string
curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/tag-association/${TAG_ID}?action=attach"
 \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
```

Attach a Tag to a Cluster

After you create a tag, you can attach the tag to a vSphere managed object. Tags make the inventory objects in your virtual environment more sortable and searchable.

- Tag and tag category created. See Creating vSphere Tags.
- Cluster name or obtain the managed object identifier for the specified cluster.
  To get the managed object identifier of the `ClusterComputeResource`, you must access vCenter by using the Virtual Infrastructure JSON APIs.
- `InventoryService.Tagging.AttachTag` privilege on the target cluster
- Tag category `associable_types` includes `ClusterComputeResource` or is empty (unrestricted)

This procedure describes the steps for applying a tag to a cluster object in your inventory.

1. Construct a dynamic object identifier for the cluster.

   The `DynamicID` includes the type and ID of the `ManagedObjectReference`.

2. Attach the tag to the cluster.

Java

```
// Resolve the cluster ManagedObjectReference by name using PropertyCollector
ManagedObjectReference clusterMoRef =
    propertyCollectorHelper.getMoRefByName(clusterName,
 CLUSTER_COMPUTE_RESOURCE);

if (clusterMoRef == null) {
    throw new RuntimeException("Cluster not found: " + clusterName);
}

// Convert to DynamicID — required for all CIS tagging API calls
DynamicID clusterDynamicId = new DynamicID(clusterMoRef.getType(),
 clusterMoRef.getValue());

// Confirm the tag is attachable before calling attach()
```

```
List<String> attachableTags =
 tagAssociation.listAttachableTags(clusterDynamicId);
if (!attachableTags.contains(tagId)) {
    throw new RuntimeException(
        "Tag is not attachable — verify category associable_types and
 Tag.Assign privilege: " + tagId
    );
}

tagAssociation.attach(tagId, clusterDynamicId);

// Verify that the tag is now listed as attached
if (!tagAssociation.listAttachedTags(clusterDynamicId).contains(tagId)) {
    throw new RuntimeException("Tag attachment verification failed: " +
 tagId);
}
```

Python

```
from com.vmware.vapi.std_client import DynamicID

# 1. Retrieve the cluster identifier from the vCenter Cluster list
clusters = vsphere_client.vcenter.Cluster.list()
cluster = next((c for c in clusters if c.name == CLUSTER_NAME), None)

if cluster is None:
    raise ValueError(f"Cluster not found: {CLUSTER_NAME}")

# cluster.cluster is the ManagedObjectReference value for
 ClusterComputeResource
cluster_dynamic_id = DynamicID(
    type='ClusterComputeResource',
    id=cluster.cluster
)

attachable = vsphere_client.tagging.TagAssociation.list_attachable_tags(
    object_id=cluster_dynamic_id
)

if tag_id not in attachable:
    raise RuntimeError(
        f"Tag is not attachable — verify category associable_types and
 Tag.Assign privilege: {tag_id}"
    )

# 2. Attach the tag securely using proper keyword bindings
vsphere_client.tagging.TagAssociation.attach(
    tag_id=tag_id,
    object_id=cluster_dynamic_id
)
```

REST

```
# 1. Retrieve the cluster identifier by name
CLUSTER_ID=$(curl -s -k \
  "https://${VC_HOST}/api/vcenter/cluster?names=${CLUSTER_NAME}" \
  -H "vmware-api-session-id: ${SESSION}" | jq -r '.[0].cluster')

ATTACH_BODY='{
  "object_id": {"type": "ClusterComputeResource", "id": "'"${CLUSTER_ID}"'"}
}'

# 2. Verify that the tag is attachable before calling attach
```

```
curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/tag-association?action=list-attachable-
tags" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}" | jq -r ".[] | select(. == \"${TAG_ID}\")"

# 3. Attach the tag to the cluster
curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/tag-association/${TAG_ID}?action=attach"
 \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
```

To enumerate all associations across the inventory, see Retrieve Tag Associations at Scale.

Updating a Tag

To update a tag, you must create an update specification for the tag.

In the update spec, you set values for the fields to be changed, and omit values for the other fields. The update operation modifies only the fields that carry values in the spec and leaves all other fields unchanged.

A common use case is updating a tag description with a timestamp that records when a resource was last reconfigured.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

This example adds a timestamp in a tag description to identify when a resource was last reconfigured. The tag description is updated with the timestamp after the resources is reconfigured.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
String newDateTime = DateFormat.getDateInstance().format(new Date());
String newDescription = String.format("Server tag updated at (%s).", newDateTime);
TagTypes.UpdateSpec updateSpec = new TagTypes.UpdateSpec();
updateSpec.setDescription(newDescription); // only description is set — name is
 unchanged
taggingService.update(myTagId, updateSpec);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

This example adds a timestamp in a tag description to identify when a resource was last reconfigured. The tag description is updated with the timestamp after the resources is reconfigured.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
import time


# 1 - Format the current time.
date_time = time.strftime('%d/%m/%Y %H:%M:%S')
description = 'Server tag updated at ' + date_time


# 2 - Set up a tag update spec.
tag_update_spec = vsphere_client.tagging.Tag.UpdateSpec()
tag_update_spec.description = description # only description is set — name is
 unchanged
```

```
# 3 - Apply the update spec to change the tag description.
vsphere_client.tagging.Tag.update(
    tag_id=tag_id,
    tag_update_spec=tag_update_spec
)
```

REST

```
UPDATE_BODY='{
 "update_spec": {
 "description": "Server tag updated at '"$(date '+%d/%m/%Y %H:%M:%S')_"'."
 }
}'

# PATCH updates only the fields present in update_spec
curl -s -k -X PATCH \
  "https://${VC_HOST}/api/cis/tagging/tag/${TAG_ID}" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${UPDATE_BODY}"
```

Note:

`UpdateSpec` performs a partial update. Omitting a field preserves its current value. To clear a field, set it to an empty String. The `category_id` and `cardinality` fields cannot be changed after you create a tag or category.

Using Tags to Create and Manage Compute Policies

A compute policy uses a tag to identify the vSphere objects on which vCenter enforces the policy.

Create a Compute Policy

You can create a compute policy and check the compliance status of the policy for a specific virtual machine.

1. Retrieve the object ID of a virtual machine.
2. Retrieve the object ID of the host you want the virtual machine to run on.

   Note:
   The host should be different from the host the virtual machine is already running on.
3. Tag the virtual machine.
   For example, use the *`tag-1`* tag.
4. Tag the host.
   For example, use the *`tag-2`* tag.
5. Create a compute policy.
   For example, use the *`tag-1`* and *`tag-2`* tags.
6. Check the compliance status of the policy on this virtual machine.

   Note:
   The compliance status can take up to 3 minutes to update.

vCenter creates the compute policy and places the virtual machine on the tagged host.

Java

This example follows the same client setup and capability-discovery pattern as `ListComputePolicy.java` in the vcf-sdk-java GitHub repository.

It creates a VM-host affinity policy for a virtual machine and host that are already tagged.

```
package com.vmware.sdk.samples.vcenter.compute.policies;
```

```
import static
 com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.CreateSpec;

/** Demonstrates how to create a VM-Host affinity compute policy. */
public class CreateVmHostAffinityPolicy {
    private static final Logger log =
 LoggerFactory.getLogger(CreateVmHostAffinityPolicy.class);

    /** REQUIRED: vCenter FQDN or IP address. */
    public static String serverAddress = "vcenter1.mycompany.com";
    /** REQUIRED: Username to log in to the vCenter. */
    public static String username = "username";
    /** REQUIRED: Password to log in to the vCenter. */
    public static String password = "password";
    /** OPTIONAL: Absolute path to a file of trusted server certificates. Leave null
 to disable TLS verification. */
    public static String trustStorePath = null;
    /** REQUIRED: Identifier of a tag already attached to the virtual machine. */
    public static String vmTagId = "urn:vmomi:InventoryServiceTag:5b3564f4-
a762-4480-9145-0f87d28aea78:GLOBAL";
    /** REQUIRED: Identifier of a tag already attached to the host. */
    public static String hostTagId = "urn:vmomi:InventoryServiceTag:a8d0d278-
edd1-4a79-86ac-fa9384595062:GLOBAL";

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(CreateVmHostAffinityPolicy.class, args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));

        try (VcenterClient client = factory.createClient(username, password, null)) {
            Policies policies = client.createStub(Policies.class);

            CreateSpec createSpec = new CreateSpec();
            createSpec.setName("rack_a_policy");
            createSpec.setDescription("rack_a_policy description");
            createSpec.setVmTag(vmTagId);
            createSpec.setHostTag(hostTagId);

            String policyId = policies.create(createSpec);
            log.info("Created compute policy with ID: {}", policyId);
        }
    }
}
```

Python

This example is based on code in the `compute_policy_vcls_workflow.py` sample file. It tags the virtual machine and the host, creates a VM-host affinity policy, powers on the virtual machine, and checks the resulting compliance status.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
from com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity_client \
     import CreateSpec
...
# Tag the virtual machine and the host.
attach_tag(self.client, self.vm_id, "VirtualMachine", vm_tag)
attach_tag(self.client, host_id, "HostSystem", host_tag)

# Create a VM-host affinity policy.
create_spec = CreateSpec(vm_tag=vm_tag.id, host_tag=host_tag.id,
                         name=self.policy_name,
                         description=self.policy_desc)
self.policy_id = self.client.vcenter.compute.Policies.create(create_spec)

# Power on the virtual machine.
self.client.vcenter.vm.Power.start(self.vm_id)

# Check the compliance status of the policy on this virtual machine.
status = self.client.vcenter.vm.compute.Policies.get(self.vm_id,
                                                     self.policy_id)
print("The compliance status of policy {} for virtual machine "
      "{} is {}".format(self.policy_id, self.vm_id, status.status))
```

REST

```
POLICY_BODY='{
  "spec": {
    "@class":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec",
    "name": "rack_a_policy",
    "description": "rack_a_policy description",
    "vm_tag": "'"${VM_TAG_ID}"'",
    "host_tag": "'"${HOST_TAG_ID}"'"
  }
}'
POLICY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${POLICY_BODY}" | jq -r '.')
echo "Created Policy ID: $POLICY_ID"
```

Managing Compute Policy Workflows

You can manage compute policy capabilities, create tags and associations, create, list, and use policies.

The following sequence represents a typical workflow of discovering all compute policy capabilities available on a vCenter instance and creating policies based on those capabilities.

- Manage Compute Policy Capabilities
- Create Tags and Associations
- Create, List, and Use Policies

Manage Compute Policy Capabilities

You can list capabilities, retrieve details about a specific capability, and use the details to create a compute policy.

The `Capabilities` service provides operations to manage compute policy capabilities. The description of the capability provides information about the intent of a policy based on this capability. A capability provides a type to create a policy. A capability also provides a type that describes the information returned when retrieving information about a policy.

1. List all available compute policy capabilities.
2. Retrieve details about the `vm_host_affinity` capability.
3. Use the details to identify the fields of the `CreateSpec` structure for that capability.

Java

```
package com.vmware.sdk.samples.vcenter.compute.policies;

import static
 com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.policies.Capabilities;
import com.vmware.vcenter.compute.policies.CapabilitiesTypes;

/** Demonstrates how to discover compute policy capabilities. */
public class ListComputePolicyCapabilities {
    private static final Logger log =
 LoggerFactory.getLogger(ListComputePolicyCapabilities.class);

    public static String serverAddress = "vcenter1.mycompany.com";
    public static String username = "username";
    public static String password = "password";
    public static String trustStorePath = null;

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(ListComputePolicyCapabilities.class,
 args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));

        try (VcenterClient client = factory.createClient(username, password,
 null)) {
            Capabilities capabilities = client.createStub(Capabilities.class);

            // List all available compute policy capabilities.
            List<CapabilitiesTypes.Summary> summaries = capabilities.list();
            for (CapabilitiesTypes.Summary summary : summaries) {
                log.info("capability={}, name={}", summary.getCapability(),
 summary.getName());
            }

            // Retrieve details about the vm_host_affinity capability.
            CapabilitiesTypes.Info info = capabilities.get(

 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity");
            log.info("createSpecType={}, infoType={}",
 info.getCreateSpecType(), info.getInfoType());
        }
    }
```

```
}
```

Python

```
# List all available compute policy capabilities.
summaries = vsphere_client.vcenter.compute.policies.Capabilities.list()
for summary in summaries:
    print(f"capability={summary.capability}, name={summary.name}")

# Retrieve details about the vm_host_affinity capability.
info = vsphere_client.vcenter.compute.policies.Capabilities.get(
    "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity")
print(f"create_spec_type={info.create_spec_type}, info_type={info.info_type}")
```

REST

```
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities" \
  -H "vmware-api-session-id: ${SESSION}"
```

The following example response contains details about the `vm_host_affinity` capability.

```
[
    {
        "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity",
        "name": "VM-Host Affinity",
        "description": "Virtual machines with the specified tag are placed on
 hosts with the specified tag."
    }
]
```

```
curl -s -k \  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities/
com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity" \
  -H "vmware-api-session-id: ${SESSION}"
```

```
{
    "info_type":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.info",
    "name": "VM-Host Affinity",
    "description": "Virtual machines with the specified tag are placed on
 hosts with the specified tag.",
    "create_spec_type":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec"
}
```

You can create tags to identify inventory objects on which a compute policy should be enforced. See Create Tags and Associations.

Create Tags and Associations

You must create a category and specify the associable types and cardinality for the tags that make up the category. After you create a category, you can create tags for the category.

1. Create a tagging category, for example, a category restricted to hosts.
2. Create a tag for the tagging category.
3. List all tags on a vCenter instance.
4. List all virtual machines in the inventory, to identify the virtual machine to place on the tagged host.
5. Attach the tag to a host.
6. List all tags attached to a host.

Java

This example follows the same category and tag creation pattern already shown in Creating a Tag Category and Creating a Tag, restricted to the `HostSystem` associable type.

```
mport java.util.HashSet;
import java.util.List;
import java.util.Set;
```

```
import com.vmware.cis.tagging.Category;
import com.vmware.cis.tagging.CategoryModel;
import com.vmware.cis.tagging.CategoryTypes;
import com.vmware.cis.tagging.Tag;
import com.vmware.cis.tagging.TagAssociation;
import com.vmware.cis.tagging.TagTypes;
import com.vmware.vapi.std_client.DynamicID;
import com.vmware.vcenter.Host;
import com.vmware.vcenter.HostTypes;

Category categoryService = client.createStub(Category.class);
Tag taggingService = client.createStub(Tag.class);
TagAssociation tagAssociation = client.createStub(TagAssociation.class);
Host hostService = client.createStub(Host.class);

// Create a tagging category, restricted to hosts.
CategoryTypes.CreateSpec categorySpec = new CategoryTypes.CreateSpec();
categorySpec.setName("cat-4");
categorySpec.setDescription("cat-4-desc");
categorySpec.setCardinality(CategoryModel.Cardinality.SINGLE);
Set<String> associableTypes = new HashSet<>();
associableTypes.add("HostSystem");
categorySpec.setAssociableTypes(associableTypes);
String categoryId = categoryService.create(categorySpec);

// Create a tag for the tagging category.
TagTypes.CreateSpec tagSpec = new TagTypes.CreateSpec();
tagSpec.setName("tag-1");
tagSpec.setDescription("tag-1-desc");
tagSpec.setCategoryId(categoryId);
String tagId = taggingService.create(tagSpec);

// List all virtual machines in the inventory. See ListComputePolicy.java for
 the
// equivalent pattern applied to compute policies.
List<HostTypes.Summary> hosts = hostService.list(new HostTypes.FilterSpec());
String hostId = hosts.get(0).getHost();

// Attach the tag to a host.
DynamicID hostDynamicId = new DynamicID("HostSystem", hostId);
tagAssociation.attach(tagId, hostDynamicId);

// List all tags attached to the host.
List<String> attachedTags = tagAssociation.listAttachedTags(hostDynamicId);
```

Python

```
# Create a tagging category, restricted to hosts.
category_spec = vsphere_client.tagging.Category.CreateSpec(
    name="cat-4",
    description="cat-4-desc",
    cardinality=CategoryModel.Cardinality.SINGLE,
    associable_types={"HostSystem"}
)
category_id = vsphere_client.tagging.Category.create(category_spec)

# Create a tag for the tagging category.
tag_spec = vsphere_client.tagging.Tag.CreateSpec(
    name="tag-1",
```

```
    description="tag-1-desc",
    category_id=category_id
)
tag_id = vsphere_client.tagging.Tag.create(tag_spec)

# List all virtual machines in the inventory, to identify the virtual machine
 to
# place on the tagged host.
vms = vsphere_client.vcenter.VM.list()

# List hosts, and attach the tag to the first one.
hosts = vsphere_client.vcenter.Host.list()
host_dynamic_id = DynamicID(type='HostSystem', id=hosts[0].host)
vsphere_client.tagging.TagAssociation.attach(tag_id=tag_id,
 object_id=host_dynamic_id)

# List all tags attached to the host.
attached_tags = vsphere_client.tagging.TagAssociation.list_attached_tags(
    object_id=host_dynamic_id)
```

REST

The following example `create_spec` specifies the associable types and cardinality for the tags that make up the category.

```
CATEGORY_BODY='{
  "create_spec": {
    "associable_types": ["HostSystem"],
    "cardinality": "SINGLE",
    "description": "cat-4-desc",
    "name": "cat-4"
  }
}'
CATEGORY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/category" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${CATEGORY_BODY}" | jq -r '.')
echo "Created Category ID: $CATEGORY_ID"
TAG_BODY='{
  "create_spec": {
    "category_id": "'"${CATEGORY_ID}"'",
    "description": "tag-1-desc",
    "name": "tag-1"
  }
}'
TAG_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${TAG_BODY}" | jq -r '.')
echo "Created Tag ID: $TAG_ID"
curl -s -k \
  "https://${VC_HOST}/api/vcenter/vm" \
  -H "vmware-api-session-id: ${SESSION}"
# Resolve the host identifier by name.
HOST_ID=$(curl -s -k \
  "https://${VC_HOST}/api/vcenter/host?names=${HOST_NAME}" \
  -H "vmware-api-session-id: ${SESSION}" | jq -r '.[0].host')
```

```
ATTACH_BODY='{
  "object_id": {
    "id": "'"${HOST_ID}"'",
    "type": "HostSystem"
  }
}'
curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag-association/${TAG_ID}?
~action=attach" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag-association?~action=list-
attached-tags" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
```

You can create and manage compute policies. See Create, List, and Use Policies.

Create, List, and Use Policies

You can create various types of policies, list all policies on a vCenter instance, and list or filter the tags that policies use.

1. Create a `vm_host_affinity` policy.
2. List all policies on a vCenter instance.
3. List tags used by one or all policies defined on a vCenter instance.
4. Filter tags used by one or all policies defined on a vCenter instance by policy ID, tag UUID, and tag type.

Java

This example builds on `CreateVmHostAffinityPolicy.java`, shown earlier in Create a Compute Policy, and adds the list and tag-usage operations.

```
import java.util.List;
import java.util.Set;

import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.PoliciesTypes;
import com.vmware.vcenter.compute.policies.TagUsage;
import com.vmware.vcenter.compute.policies.TagUsageTypes;
import
 com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.CreateSpec;

Policies policies = client.createStub(Policies.class);
TagUsage tagUsage = client.createStub(TagUsage.class);

// Create a vm_host_affinity policy.
CreateSpec createSpec = new CreateSpec();
createSpec.setName("rack_a_policy");
createSpec.setDescription("rack_a_policy description");
createSpec.setVmTag(vmTagId);
createSpec.setHostTag(hostTagId);
String policyId = policies.create(createSpec);

// List all policies on this vCenter instance.
List<PoliciesTypes.Summary> policySummaries = policies.list();
for (PoliciesTypes.Summary summary : policySummaries) {
    log.info("policy={}, name={}", summary.getPolicy(), summary.getName());
}
```

```
// List tags used by one or all policies.
List<TagUsageTypes.Info> tagUsageInfo = tagUsage.list(null);
for (TagUsageTypes.Info usage : tagUsageInfo) {
    log.info("categoryName={}, tagName={}, policy={}",
            usage.getCategoryName(), usage.getTagName(), usage.getPolicy());
}

// Filter tags used by one or all policies by policy ID, tag UUID, tag type,
// policy name, or capability.
TagUsageTypes.FilterSpec filterSpec = new TagUsageTypes.FilterSpec();
filterSpec.setPolicies(Set.of(policyId));
filterSpec.setTags(Set.of(hostTagId));
filterSpec.setTagTypes(Set.of("com.vmware.cis.tagging.Tag:HostSystem"));
filterSpec.setPolicyNames(Set.of("rack_a_policy"));
filterSpec.setCapabilities(Set.of("com.vmware.vcenter.compute.policies.capabilities.vm_host_aff
List<TagUsageTypes.Info> filteredUsage = tagUsage.list(filterSpec);
```

Python

```
# Create a vm_host_affinity policy.
create_spec =
 vsphere_client.vcenter.compute.policies.capabilities.vm_host_affinity_client.CreateSpec(
    name="rack_a_policy",
    description="rack_a_policy description",
    vm_tag=vm_tag_id,
    host_tag=host_tag_id
)
policy_id = vsphere_client.vcenter.compute.Policies.create(create_spec)

# List all policies on this vCenter instance.
policy_summaries = vsphere_client.vcenter.compute.Policies.list()
for summary in policy_summaries:
    print(f"policy={summary.policy}, name={summary.name}")

# List tags used by one or all policies.
tag_usage_info = vsphere_client.vcenter.compute.policies.TagUsage.list()
for usage in tag_usage_info:
    print(f"category_name={usage.category_name}, tag_name={usage.tag_name},
 policy={usage.policy}")

# Filter tags used by one or all policies by policy ID, tag UUID, tag type,
# policy name, or capability.
filter_spec = vsphere_client.vcenter.compute.policies.TagUsage.FilterSpec(
    policies={policy_id},
    tags={host_tag_id},
    tag_types={"com.vmware.cis.tagging.Tag:HostSystem"},
    policy_names={"rack_a_policy"},

 capabilities={"com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity"}
)
filtered_usage =
 vsphere_client.vcenter.compute.policies.TagUsage.list(filter_spec)
```

REST

```
POLICY_BODY='{
  "spec": {
    "@class":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec",
    "name": "rack_a_policy",
    "description": "rack_a_policy description",
    "vm_tag": "'"${VM_TAG_ID}"'",
```

```
    "host_tag": "'"${HOST_TAG_ID}"'"
  }
}'
POLICY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${POLICY_BODY}" | jq -r '.')
echo "Created Policy ID: $POLICY_ID"
```

The following example response contains the policy ID.

```
{
    "value": "178aaf62-06e7-4958-bff9-841674633a3f"
}
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}"
{
    "value": [
        {
            "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity",
            "name": "rack_a_policy",
            "description": "rack_a_policy description",
            "policy": "178aaf62-06e7-4958-bff9-841674633a3f"
        }
    ]
}
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies/tag-usage" \
  -H "vmware-api-session-id: ${SESSION}"
{
    "value": [
        {
            "category_name": "cat-4",
            "tag_type": "com.vmware.cis.tagging.Tag:HostSystem",
            "tag_name": "tag-1",
            "tag": "urn:vmomi:InventoryServiceTag:a8d0d278-edd1-4a79-86ac-
fa9384595062:GLOBAL",
            "policy": "178aaf62-06e7-4958-bff9-841674633a3f",
            "policy_name": "rack_a_policy",
            "policy_description": "rack_a_policy description",
            "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity"
        }
    ]
}
```

Note:

The `policy_name`, `policy_description`, and `capability` fields were added in VCF 9.1.0.0.

```
curl -s -k -G \
  "https://${VC_HOST}/api/vcenter/compute/policies/tag-usage" \
  -H "vmware-api-session-id: ${SESSION}" \
  --data-urlencode "policies=${POLICY_ID}" \
  --data-urlencode "tags=${HOST_TAG_ID}" \
```

```
--data-urlencode "tag_types=com.vmware.cis.tagging.Tag:HostSystem"
```

Note:

You can also filter by `policy_names` or by `capabilities`, both added in VCF 9.1.0.0.

VM-VM Affinity and Anti-Affinity Policies

vCenter provides a pair of VM-VM placement capabilities, in addition to the VM-host affinity capability described earlier in this chapter. Use these capabilities to keep a group of tagged virtual machines together, or apart, regardless of which host or hosts run them.

- Use the `com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity_v2` and `com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity` capability to keep virtual machines that share a tag together.
- Use the `com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2` and `com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity` capability to keep virtual machines that share a tag apart.

The VM-host affinity capability uses two separate tags to select virtual machines and hosts. Each VM-VM capability instead uses a single tag and an object selector to select every virtual machine to place together or apart.

Discover the VM-VM Capabilities

1. List all available compute policy capabilities.

```
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities" \
  -H "vmware-api-session-id: ${SESSION}"
```

   The response now also includes the two VM-VM capabilities, alongside `vm_host_affinity` and any other capabilities already available on the vCenter instance. The list includes both `com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity_v2` and `com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity`.

```
[
    {
        "capability": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity_v2",
        "name": "VM-VM Affinity v2",
        "description": "Virtual machines that share the specified tag are placed on hosts together."
    },
    {
        "capability": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2",
        "name": "VM-VM Anti-Affinity v2",
        "description": "Virtual machines that share the specified tag are placed on separate hosts."
    }
{
        "capability": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_affinity",
        "name": "VM-VM Affinity v1",
        "description": "Virtual machines that share the specified tag are placed on hosts together."
    },
    {
        "capability": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity",
        "name": "VM-VM Anti-Affinity v1",
        "description": "Virtual machines that share the specified tag are placed on separate hosts."
    }
]
```

2. Retrieve details about the `vm_vm_anti_affinity_v2` capability.

```
curl -s -k \  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities/
com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2" \
  -H "vmware-api-session-id: ${SESSION}"
```

The `vm_vm_affinity_v2` capability returns the equivalent detail, with `vm_vm_affinity_v2` in place of `vm_vm_anti_affinity_v2` throughout.

```
{
    "info_type": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2.info",
    "name": "VM-VM Anti-Affinity",
    "description": "Virtual machines that share the specified tag are placed on separate hosts.",
    "create_spec_type":
 "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2.create_spec_v2"
}
```

3. Use the details to identify the fields of the `create_spec_v2` structure. The following table applies to both VM-VM capabilities. Only the capability identifier differs between them.

Field	Type	Description
`capability`	ID	Identifier of the capability. Inherited from the common `CreateSpec` structure.
`name`	String	Name of the policy. Inherited from the common `CreateSpec` structure. If unset, vCenter generates a name. See Understanding Automatically Generated Policy Names.
`description`	String	Description of the policy. Inherited from the common `CreateSpec` structure.
`managed_by`	ManagedBy	The entity that manages the policy. Inherited from the common `CreateSpec` structure.
`vm_selector`	ObjectSelectorSpec	Selector that identifies the virtual machines that the policy applies to. Currently supports tag-based selection only.

Note:

vCenter accepts a `create_spec_v2` specification only through the batch operation that is described in Create Multiple Tags and Policies in a Single Request. The single-policies create operation that is described earlier in this chapter does not accept a `create_spec_v2` specification.

Create Multiple Tags and Policies in a Single Request

Create one or more compute policies, together with their tags and tag categories, in a single batch request. If a policy specification refers to a tag or category that does not already exist, vCenter creates the tag or category automatically.

1. Send a batch create request that specifies one or more policies. The following example request creates a `vm_vm_anti_affinity_v2` policy for a tag that does not yet exist. vCenter creates the `anti-affinity-pods` category and the `pod-a` tag automatically.

```
BATCH_CREATE_BODY='{
  "policies": [
    {
      "spec": {
        "@class":
 "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2.create_spec_v2",
        "description": "Keep pod-a virtual machines on separate hosts.",
        "vm_selector": {
          "tag": {
            "tag_name_spec": {
              "category_name": "anti-affinity-pods",
              "tag_name": "pod-a"
            }
          }
        }
      }
    }
  ]
}'
curl -s -k -X POST \
```

```
  "https://${VC_HOST}/api/vcenter/compute/policies?action=create-tags-and-policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${BATCH_CREATE_BODY}"
```

Note:

Attach the `pod-a` tag to every virtual machine that must remain on a separate host. See Create Tags and Associations for the general tag and association steps. A VM-VM anti-affinity policy requires at least two tagged virtual machines to take effect.

2. Review the response. The response lists the successfully created policies by name, together with the identifier and information for each policy, and the errors for any policy that vCenter did not create.

```
{
    "policies": {
        "AutoGenPolicy-anti-affinity-
pods-978dfd366c8b9529e2c6056d8a406a0ccca6974ae2c577d6c330c28de9fc849": {
            "policy": "178aaf62-06e7-4958-bff9-841674633a3f",
            "info": {
                "@class": "com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity_v2.info",
                "name": "AutoGenPolicy-anti-affinity-
pods-978dfd366c8b9529e2c6056d8a406a0ccca6974ae2c577d6c330c28de9fc849",
                "description": "Keep pod-a virtual machines on separate hosts.",
                "vm_selector": {
                    "tag": {
                        "tag_name_spec": {
                            "category_name": "anti-affinity-pods",
                            "tag_name": "pod-a"
                        }
                    }
                }
            }
        }
    }
}
```

Note:

When you call the `createTagsAndPolicies` API as a user or API client, you must explicitly specify a policy name. vCenter automatically creates policies and generates names with the `AutoGenPolicy` prefix only when VM Service (`VmSvc`) specifies placement policies (`VmPlacementPolicies`) in the VM `ConfigSpec` during VM creation. For details on how vCenter names these system-generated policies, see Understanding Automatically Generated Policy Names.

Java Example

```
package com.vmware.sdk.samples.vcenter.compute.policies;

import static com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.policies.CreateItem;
import com.vmware.vcenter.compute.policies.CreateResult;
```

```
import com.vmware.vcenter.compute.policies.CreateTagsAndPoliciesSpec;
import com.vmware.vcenter.compute.policies.ObjectSelectorSpec;
import com.vmware.vcenter.compute.policies.TagNameSpec;
import com.vmware.vcenter.compute.policies.TagSpec;
import com.vmware.vcenter.compute.policies.capabilities.vm_vm_anti_affinity.CreateSpecV2;

/** Demonstrates how to create a VM-VM anti-affinity compute policy in a batch request. */
public class CreateVmVmAntiAffinityPolicy {
    private static final Logger log = LoggerFactory.getLogger(CreateVmVmAntiAffinityPolicy.class);

    public static String serverAddress = "vcenter1.mycompany.com";
    public static String username = "username";
    public static String password = "password";
    public static String trustStorePath = null;
    /** REQUIRED: Name of an existing or new tag category. */
    public static String categoryName = "anti-affinity-pods";
    /** REQUIRED: Name of an existing or new tag. */
    public static String tagName = "pod-a";

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(CreateVmVmAntiAffinityPolicy.class, args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress, loadKeystoreOrCreateEmpty(trustStorePath));

        try (VcenterClient client = factory.createClient(username, password, null)) {
            Policies policies = client.createStub(Policies.class);

            TagNameSpec tagNameSpec = new TagNameSpec();
            tagNameSpec.setCategoryName(categoryName);
            tagNameSpec.setTagName(tagName);

            TagSpec tagSpec = new TagSpec();
            tagSpec.setTagNameSpec(tagNameSpec);

            ObjectSelectorSpec vmSelector = new ObjectSelectorSpec();
            vmSelector.setTag(tagSpec);

            CreateSpecV2 createSpecV2 = new CreateSpecV2();
            createSpecV2.setDescription("Keep pod-a virtual machines on separate hosts.");
            createSpecV2.setVmSelector(vmSelector);

            CreateItem item = new CreateItem();
            item.setSpec(createSpecV2);

            CreateTagsAndPoliciesSpec spec = new CreateTagsAndPoliciesSpec();
            spec.setPolicies(List.of(item));

            CreateResult result = policies.createTagsAndPolicies(spec);
            log.info("Created policies: {}", result.getPolicies());
            log.info("Errors: {}", result.getErrors());
        }
    }
}
```

Understanding Automatically Generated Policy Names

When VM Service (`VmSvc`) workloads specify placement policies in the VM `ConfigSpec` during deployment, vCenter automatically creates the underlying compute placement policy on behalf of VM Service.

Because VM Service does not explicitly specify a policy name, vCenter automatically generates a unique policy name using the following format:

```
AutoGenPolicy-<category_name>-<64-character SHA-256 hash>
```

vCenter derives the hash from the tag name, category name, topology, strictness, and capability of the policy. This hash guarantees name uniqueness and a fixed length across the cluster. To view human-readable details about an automatically generated policy, review its policy description in vCenter.

> **Note:**
>
> The `AutoGenPolicy` name prefix is reserved for policies that vCenter creates automatically (such as those requested by VM Service). If you specify a policy name that starts with `AutoGenPolicy` through the API or SDK, vCenter rejects the request with a `PolicyNameReservedPrefix` error.

Delete Multiple Compute Policies

Delete one or more compute policies in a single batch request. When you delete a policy that vCenter created together with an automatically created tag, vCenter also deletes that tag, provided that no other policy still references it. Furthermore, vCenter deletes the parent category of the tag in case the category has no other child tags.

1. Send a batch delete request that specifies one or more policy identifiers.

```
DELETE_BODY='{
  "policies": [
    "'"${POLICY_ID}"'"
  ]
}'
curl -s -k -X POST \
  "https://${VC_HOST}/api/vcenter/compute/policies?action=delete-tags-and-policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${DELETE_BODY}"
```

2. Review the response. If vCenter deletes every specified policy successfully, the response contains no errors. If a policy fails to delete, the response identifies the policy and the associated error.

```
{
    "errors": {}
}
```

Java Example

```
package com.vmware.sdk.samples.vcenter.compute.policies;

import static com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import java.util.Set;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.policies.DeleteResult;
import com.vmware.vcenter.compute.policies.DeleteTagsAndPoliciesSpec;
```

```
/** Demonstrates how to delete one or more compute policies in a batch request. */
public class DeleteComputePolicies {
    private static final Logger log = LoggerFactory.getLogger(DeleteComputePolicies.class);

    public static String serverAddress = "vcenter1.mycompany.com";
    public static String username = "username";
    public static String password = "password";
    public static String trustStorePath = null;
    /** REQUIRED: Identifier of the policy to delete, returned by createTagsAndPolicies. */
    public static String policyId = "178aaf62-06e7-4958-bff9-841674633a3f";

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(DeleteComputePolicies.class, args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress, loadKeystoreOrCreateEmpty(trustStorePath));

        try (VcenterClient client = factory.createClient(username, password, null)) {
            Policies policies = client.createStub(Policies.class);

            DeleteTagsAndPoliciesSpec spec = new DeleteTagsAndPoliciesSpec();
            spec.setPolicies(Set.of(policyId));

            DeleteResult result = policies.deleteTagsAndPolicies(spec);
            log.info("Errors: {}", result.getErrors());
        }
    }
}
```

Automatic Cleanup of Unreferenced Tags and Policies

On every 3 minutes, vCenter removes unreferenced, automatically created tags and policies with the `AutoGenPolicy` name prefix. Only the policies which have no association with any virtual machines are deleted by this background process. Use Delete Multiple Compute Policies to remove a policy and its automatically created tag immediately.

If you remove a virtual machine without first deleting its associated policy, a background cleanup process removes the orphaned policy and tag later. Whenever you delete a virtual machine that has automatically created policies attached, and that virtual machine is the last one associated with those policies, vCenter deletes the policies as well.

The background cleanup process does not remove a policy that vSphere Supervisor manages.

vSphere Security

This chapter provides information about securing your vSphere environment for vCenter and ESX.

Managing Certificates

You can use the VMware Cloud Foundation API to manage certificates in your vSphere environment. You can not only refresh default certificates that are issued by the VMware Certificate Authority (VMCA) but also add third-party or custom-made certificates to your environment.

Certificate Management Operations
Use the VMware Cloud Foundation API to manage trusted root certificate chains, VMware Certificate Authority (VMCA) root certificates, machine SSL (TLS) certificates, and Security Token Service (STS) signing certificates.

With the VMware Cloud Foundation API, you can refresh the VMCA-issued certificates but also add external and third-party certificates to your vSphere environment. For more information on vSphere certificate management, see the *vSphere Authentication* guide.

Certificate Management Interfaces

You can use the following interfaces to manage certificates with the VMware Cloud Foundation API:

- `Trusted Root Chains`
- `VMCA Root`
- `TLS CSR`
- `TLS`
- `SigningCertificate`

Starting with VCF 9.1.1, the `SigningCertificate` interface supports generating a CSR and importing a signed certificate chain without transmitting the private key over the network. This secure workflow supplements the existing replace and refresh operations.

Certificate Management Operations

You can use the operations listed in the following table to manage certificates.

Operation	Interface	HTTP Request	Description	Introduced in
List trusted root certificates	`Trusted Root Chains`	`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`	You can retrieve the identifiers of all trusted root certificates that are present in vCenter.	vSphere 6.7 U2
Get trusted root certificate information	`Trusted Root Chains`	`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains/<chain>`	You can retrieve a root certificate chain by providing its identifier. You can retrieve the identifier by using the List trusted root certificates operation.	vSphere 6.7 U2
Add a trusted root certificate	`Trusted Root Chains`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`	Use this operation to add a trusted root certificate chain to your vCenter system.	vSphere 6.7 U2
Delete a trusted root certificate	`Trusted Root Chains`	`DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains/<chain>`	You can delete a root certificate by providing its unique identifier. You can retrieve the identifier by using the List trusted root certificates operation.	vSphere 6.7 U2
Replace the VMCA root certificate	`VMCA Root`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/vmca-root`	You can replace the VMCA root certificate with a new VMCA-signed certificate. The operation triggers a restart of the services that are using this certificate.	vSphere 7.0
Generate a CSR	`TLS CSR`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls-csr`	You can generate a CSR and use it to issue a custom certificate. If the operation is successful, you receive a CSR in PEM format.	vSphere 6.7 U2
Get the Machine SSL certificate	`TLS`	`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls`	You can retrieve the Machine SSL certificate of your vCenter system.	vSphere 6.7 U2
Renew the Machine SSL certificate	`TLS`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls?action=renew`	You can renew the validity of the machine SSL certificate for a specified period in days. The duration must be less than or equal to 730 days.	vSphere 6.7 U2

Operation	Interface	HTTP Request	Description	Introduced in
Replace the Machine SSL certificate with a custom signed certificate	`TLS`	`PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls`	You can replace the vCenter Machine SSL certificate with a custom certificate signed by an external Certificate Authority (CA).	vSphere 6.7 U2
Replace the Machine SSL certificate with a VMCA-signed certificate	`TLS`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls?action=replace-vmca-signed`	You can replace the vCenter Machine SSL certificate with a VMCA-signed certificate.	vSphere 7.0
Retrieve the STS signing certificate chains	`Signing Certificate`	`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate`	You can retrieve the STS signing certificate chains, which are used for validating tokens signed by vCenter.	vSphere 7.0 U3
Replace the STS signing certificate	`Signing Certificate`	`PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate`	You can replace the current STS signing certificate with a certificate of your choice. The accepted file format is PEM.	vSphere 7.0 U3
Generate a CSR for the STS signing certificate	`SigningCertificate`	`POST https://<vcenter-fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate`	You can generate a Certificate Signing Request (CSR) for the STS signing certificate and stage the private key on the vCenter system. The private key is not returned to the caller.	VCF 9.1.1
Import a signed STS signing certificate	`SigningCertificate`	`PATCH https://<vcenter-fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate`	You can import an externally signed STS signing certificate chain without including the private key in the request body. The operation uses the private key that the appliance staged during CSR generation.	VCF 9.1.1
Refresh the STS signing certificate	`Signing Certificate`	`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate?action=refresh`	You can replace the current STS signing certificate with a new VMCA-signed certificate. The newly generated certificate is set as the active STS signing certificate for the vCenter token service.	vSphere 7.0 U3

Add a Root Certificate to vCenter

You can use the `Certificate Management vCenter Trusted Root Chains` interface to add, delete and read trusted root certificate chains.

- Verify that you are connected to a VMware Cloud Foundation API server.
- Verify that the root certificate or certificate chain you want to add is available on your machine.
- Verify that you have the required privileges: **CertificateManagement** > **Manage** and **CertificateManagement** > **Administer**.

If you want to use an enterprise or third-party certificate authority (CA) for certificate management of your vSphere environment, you must first establish trust with that CA. You can do this by adding the root certificate of the external CA to the trusted root store of your vCenter system.

Adding a root certificate or certificate chain to the vCenter trusted certificate store establishes trust with an enterprise or third-party certificate authority. You can add a root certificate to vCenter as a prerequisite for other scenarios such as setting a third-party or enterprise machine SSL certificate.

1. Optional: Retrieve the root certificates on your vCenter system.

   `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`

2. Populate the `TrustedRootChains.CreateSpec` data structure.

Parameter	Type	Description
`cert_chain`	String	Certificate or certificate chain in base64 encoding. The input must be JSON string escaped for newline (\n).

3. Add the certificate or certificate chain.

   `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`

   If the operation is successful, the system returns the unique identifier of the trusted root certificate you added.

Delete a Root Certificate from vCenter

You can use the `Certificate Management vCenter Trusted Root Chains` interface to add, delete and read trusted root certificate chains. This use case demonstrates how to delete a root certificate or certificate chain from the trusted root store of your vCenter system.

- Verify that you are connected to a VMware Cloud Foundation API server.
- Verify that you have the required privileges for executing the method: **CertificateManagement** > **Administer** and **CertificateManagement** > **Manage**.

Deleting certificates is not available through the vSphere Client and you can only do this by using the VMware Cloud Foundation API or the CLI tools.

> **CAUTION**:
> Deleting a root certificate or certificate chain that is in use might cause breakage of your systems. Proceed to delete a root certificate only if you are sure it is not in use by your vCenter or any connected systems.

1. Optional: Retrieve the root certificates from your vCenter system.

   `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`

   The system lists the unique identifiers (chains) of the certificates in the trusted root store.

2. Retrieve the certificate you want to delete.

   To retrieve and verify a root certificate, use its unique identifier (chain).

   `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains/<chain>`

3. Delete the certificate by providing its unique identifier (chain).

   `DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains/<chain>`

   The system returns a 204 error, which means that the request was processed but no content is returned.

4. Optional: To verify you deleted the certificate, retrieve the root certificates from your vCenter system once again.

   `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/trusted-root-chains`

Change the Machine SSL Certificate of vCenter

You can change the machine SSL certificate of a vCenter system by using the TLS and the TLS CSR interfaces of the VMware Cloud Foundation API.

- Verify that you are connected to a VMware Cloud Foundation API server.

- Verify that the root certificate of the CA you are going to use is available on your machine.
- Verify that you have the required privileges: **CertificateManagement** > **Administer** and **CertificateManagement** > **Manage**.

The machine SSL certificate is used for server verification and for secure communication such as HTTPS or LDAPS. The machine certificates are the human-facing certificates in vSphere. They are used to create an SSL socket on the server side to which SSL clients can then connect.

Changing the machine SSL certificate with one issued by an official or enterprise certificate authority is an essential part of the Hybrid Mode of vSphere certificate management. In this mode, you replace the machine SSL certificate and you leave the VMCA to manage all other certificates automatically. The VMCA is a just-enough internal certificate authority that comes integral with your vSphere deployment. It has been purpose-built to serve the certificate needs of your vSphere environment. For more information on vSphere certificate management, see the *vSphere Authentication* guide.

1. Optional: Retrieve the current Machine SSL certificate of your vCenter system.

   `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls`

2. Generate a certificate signing request (CSR) by using the `TLS CSR` functionality.
   a) Populate the `CertificateManagementVcenterTlsCsrSpec` data structure.

Parameter	Type	Description
`country`	String	Specifies the country in the certificate subject.
`state_or_province`	String	Specifies the state or province in the certificate subject.
`locality`	String	Specifies the locality in the certificate subject.
`organization`	String	Specifies the organization in the certificate subject.
`organization_unit`	String	Specifies the organization unit in the certificate subject.
`email_address`	String	Specifies the email address in the certificate subject.

   b) Make the request.

   `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls-csr`

   The system returns the CSR in PEM format.

3. Save the CSR to your machine.
4. Send the CSR to the certificate authority of your choice.

   Note:

   The private key corresponding to the public key generated by the CSR is stored in the vCenter keystore and does not exit your system.

5. Save the issued Machine SSL certificate to your machine.
6. Set the new custom certificate to your vCenter system by using the `TLS` functionality.
   a) Populate the `CertificateManagementVcenterTlsSpec` data structure.

Parameter	Type	Description
`cert`	String	The Machine SSL certificate in PEM format. You must also paste the intermediate CA certificate, if you have one. The certificates must be JSON string escaped for newline (\n).
`root_cert`	String	The third-party root CA certificate in PEM format. You must also paste the intermediate CA certificate, if you have one. The certificates must be JSON string escaped for newline (\n).

Note:

You must not provide the private key as it was generated with the CSR and is already saved to your system.

   b) Make the request.

```
PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/tls
```

The system returns a 204 error, which means that the request was processed successfully but no content is returned.

Note:

Restart of vCenter services after the certificate change is not necessary. The certificate replacement is completed seamlessly and all your sessions remain active.

Refresh the vCenter STS Signing Certificate with a VMCA-Issued Certificate

You can refresh the vCenter Security Token Service (STS) signing certificate by using the `Certificate Management vCenter Signing Certificate` interface. The STS is an internal entity that issues and verifies tokens so that vSphere services can communicate with and trust each other.

- Verify that you are connected to a VMware Cloud Foundation API server.
- Verify that you have the **CertificateManagement** > **Administer** privilege.

You can refresh the current STS signing certificate of your vCenter system with a new VMCA-issued certificate.

There are two valid reasons for refreshing your STS signing certificate or certificate chain.

- If it is close to expiry. The standard lifespan of the vCenter STS signing certificate is 10 years. Your vCenter system will notify you in advance of STS certificate expiry. An alarm is triggered once per week when your STS certificate is 90 days away from expiry, and then daily when seven days away.
- If you already replaced your signing certificate with a third-party or enterprise one and now want to revert back to a default VMCA-issued certificate. This procedure replaces the custom or third-party STS signing certificates you added.

1. Optional: Retrieve the current vCenter STS signing certificate chain.

```
GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate
```

2. Refresh the STS signing certificate.
   a) Optional: Populate the `SigningCertificateRefreshRequestBody` data structure.

Parameter	Type	Description
`force`	boolean	The default is `false`. You can use this parameter to force a signing certificate refresh in environments that would otherwise prevent refresh from occurring such as mixed-version environments. Use `force` only when it is understood why the refresh fails or if you are instructed to do so by VMware® customer support.

   b) Request the refresh.

```
POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/
vcenter/signing-certificate?action=refresh
```

If successful, the system returns the x509 certificate chain issued in accordance with the vCenter policies.

CAUTION:

If you used a forced refresh, you must restart your vCenter and all linked services.

Java

This example shows how to refresh the vCenter STS signing certificate. The example is based on the code in the `RefreshSigningCertificate.java` sample file.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
/**
 * Sample code to refresh the STS Signing Certificate for the vCenter.
 * Use the force option to attempt to force the refresh in environments
 * that would otherwise fail such as mixed-version environments. On success,
 * the new signing certificates will be printed.
 */
public class RefreshSigningCertificate extends SamplesAbstractBase {
    private SigningCertificate certService;
    protected boolean force;

    @Override
    protected void parseArgs(String args[]) {
        Option forceOption = Option.builder()
            .required(false)
            .argName("FORCE")
            .longOpt("force")
            .desc("Attempt to force refresh")
            .build();
        List<Option> optionList = Arrays.asList(forceOption);
        super.parseArgs(optionList, args);

        this.force = parsedOptions.get("force") != null;
    }

    @Override
    protected void setup() throws Exception {
        this.certService =
            vapiAuthHelper.getStubFactory().createStub(SigningCertificate.class,
                                                       sessionStubConfig);
    }
```

```
    @Override
    protected void run() throws Exception {
        X509CertChain newCert= certService.refresh(this.force);
        if(newCert == null ) {
            System.out.println("ERROR: refresh signing certificate did not return a
certificate");
        } else {
            System.out.println("New vCenter signing certificate
\n"+newCert.toString());
        }
    }

    public static void main(String[] args) throws Exception {
        RefreshSigningCertificate get = new RefreshSigningCertificate();
        get.execute(args);
    }

    @Override
    protected void cleanup() throws Exception {
        // No cleanup required
    }
}
```

Replace the vCenter STS Signing Certificate

Replace the vCenter Security Token Service (STS) signing certificate with a custom or third-party certificate by using the VMware Cloud Foundation API. For vCenter 9.1.1 and later, generate the certificate signing request (CSR) on the vCenter instance and import only the signed certificate chain, so the private key never leaves vCenter.

- Verify that you are connected to a vCenter instance.
- Verify that you have access to a certificate authority (CA) that can sign a certificate request.
- Verify that you have the **CertificateManagement** > **Administer** privilege.
- Verify that the vCenter instance is version 9.1.1 or later. For earlier releases, use the procedure in Certificate Replacement Using the Legacy Method instead.

The STS is an internal entity that issues and verifies tokens so that vSphere services can communicate with and can trust each other. The STS signing certificate is not an external-facing certificate. A fresh vCenter 8.0 or later installation includes a signing certificate with a default duration of 10 years. Replace the STS signing certificate with a custom or third-party certificate only if your company security policy requires you to do so.

Before vCenter 9.1.1, the only supported method for this task was the `set` operation that is described in Certificate Replacement Using the Legacy Method, which requires that the caller send the unencrypted private key to vCenter in the request. Starting with vCenter 9.1.1, the `SigningCertificate` interface also provides a two-step CSR workflow: vCenter generates the key pair and stages the private key internally, and the caller submits only the CA-signed certificate chain. Use this method on vCenter 9.1.1 and later so that the private key remains within the vCenter appliance throughout the certificate rotation. The `set` method remains available on vCenter 9.1.1 and later for backward compatibility, but Broadcom recommends the CSR-based method for all new automation.

1. Generate a CSR and stage a new private key on the vCenter appliance. Send a `POST` request to the signing-certificate resource with the distinguished name fields, the subject alternative names, and the key size for the new key pair:

   All request fields are optional. The following table describes each field and the default that applies when you omit it:

Field	Type	Description
`common_name`	string	Common name for the certificate subject. If omitted, vCenter uses the primary network identifier (PNID).
`organization`	string	Organization name for the certificate subject. If omitted, vCenter supplies an appliance-appropriate default.
`organization_unit`	string	Organizational unit for the certificate subject. Leave this field unset: the CA/Browser Forum prohibits CAs from including an organizational unit, and some CAs reject a CSR that specifies one. If omitted, vCenter excludes this field from the certificate subject.
`locality`	string	City or locality for the certificate subject. If omitted, vCenter excludes this field from the certificate subject.
`state_or_province`	string	State or province for the certificate subject. If omitted, vCenter excludes this field from the certificate subject.
`country`	string	Two-letter country code for the certificate subject. If omitted, vCenter excludes this field from the certificate subject.
`email_address`	string	Contact email address for the certificate subject. Consider leaving this field unset: many current certificate profiles and CAs discourage or reject an email address in the subject. If omitted, vCenter excludes this field from the certificate subject.
`subject_alt_name`	array of strings	Subject alternative names, such as additional hostnames and IP addresses, to include in the CSR. If omitted, vCenter generates the CSR with no subject alternative names.
`key_size`	integer	The size, in bits, of the RSA key pair that vCenter generates and stages for this request, for example, 2048. If omitted, vCenter uses a secure default.

```
curl -ks -X POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate \
  -H "Content-Type: application/json" \
  -H "vmware-api-session-id: ${session}" \
  -d '{
    "common_name": "ssoserverSign",
    "organization": "Broadcom Inc.",
    "organization_unit": "VCF",
    "locality": "Bangalore",
    "state_or_province": "Karnataka",
    "country": "IN",
    "email_address": "admin@broadcom.com",
    "subject_alt_name": [
      "lvn-dvm-10-161-137-38",
```

```
        "10.161.137.38"
      ],
      "key_size": 2048
    }'
```

vCenter returns the CSR in the response body:

```
{
  "csr": "-----BEGIN CERTIFICATE REQUEST-----\nMII...\n-----END CERTIFICATE REQUEST-----"
}
```

2. Submit the CSR to your CA and obtain the signed leaf certificate and the CA root certificate.
3. Import the CA-signed certificate chain.

   Send a `PATCH` request with the certificate chain ordered leaf certificate first and root certificate last. Do not include the private key: vCenter matches the chain against the private key that is staged in step 1.

```
payload=$(jq -n '{signing_cert_chain: {cert_chain: [$leaf, $root]}}' \
  --rawfile leaf signing-certificate.crt --rawfile root ca.crt)

curl -ik https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-
certificate \
  -H "Content-Type: application/json" \
  -H "vmware-api-session-id: ${session}" \
  -X PATCH \
  -d "${payload}"
```

   vCenter returns status `204 No Content` and activates the new signing certificate.

vCenter uses the imported certificate chain for STS token signing.

Restart vCenter and all linked services after the certificate change. This restart is required because the STS caches its signing certificate, regardless of which method you use to replace it.

Certificate Replacement Using the Legacy Method
The `PUT` operation on the signing-certificate resource remains available on vCenter 9.1.1 and later for backward compatibility. It requires that you supply both the certificate chain and the unencrypted private key in a single request, so use the CSR-based procedure instead unless an existing integration depends on this method.

1. Verify that you have the custom certificate chain and the corresponding private key available.
2. Build a request body that contains the following fields:
   - An unencrypted PKCS#8 private key, base64-encoded in PEM format and escaped as a JSON string.
   - A certificate chain, base64-encoded in PEM format, ordered leaf certificate first and root certificate last.
3. Send a `PUT` request to the signing-certificate resource:

```
PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/certificate-management/vcenter/signing-certificate
```

   vCenter returns status `204 No Content`.
4. Restart vCenter and all linked services.

Managing vSphere Native Key Provider

You can use the VMware Cloud FoundationREST APISDK to perform VMware vSphere®Native Key Provider™ operations.

In vSphere 7.0 Update 2 and later, you can use the built-in vSphere Native Key Provider to enable encryption technologies, such as virtual TPMs (vTPM).

vSphere Native Key Provider is included in all vSphere editions and does not require an external key server (also called a Key Management Server (KMS) in the industry). You can also use vSphere Native Key Provider for vSphere Virtual Machine Encryption, but you must purchase the VMware vSphere®Enterprise Plus Edition™.

For details about vSphere Native Key Provider, see the *vSphere Security* documentation.

The procedures in this chapter are based on the REST API. For details, see the *vSphere Automation REST API Reference*.

The procedures in this chapter are based on the Java API. For details, see the *vSphere Automation Java API Reference*.

Retrieve vSphere Native Key Providers

You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to list vSphere Native Key Providers and retrieve details about a specific vSphere Native Key Provider.

1. Retrieve a list of vSphere Native Key Providers by calling the `list(ProvidersTypes.FilterSpec filterSpec)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers
   ```

   In the response body, you receive information about the health status, ID, and type of each provider.

2. Retrieve information about a vSphere Native Key Provider by calling the `get(java.lang.String provider)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

   In the response body, you receive detailed information about the health status, the type of the provider, and any constraints on the provider.

Manage vSphere Native Key Providers

You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to perform vSphere Native Key Provider management operations.

You can add, update, and remove vSphere Native Key Providers. Some operations require you to specify parameters in the body of the HTTP request according to your environment. For details about the syntax of each HTTP request body, see the *API Reference* documentation.

1. Add a vSphere Native Key Provider by specifying the provider ID in the request body by calling the `create(ProvidersTypes.CreateSpec spec)` method of the `Providers` interface.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers
   ```

   The following request body contains example syntax.

   ```
   {
       "native_spec": {
       },
       "provider": Test_Provider
   }
   ```

2. Update a vSphere Native Key Provider by specifying the new provider configuration in the request body by calling the `update(java.lang.String provider, ProvidersTypes.UpdateSpec spec)` method of the `Providers` interface.

   ```
   PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

3. Remove a vSphere Native Key Provider by calling the `delete(java.lang.String provider)` method of the `Providers` interface.

   ```
   DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

Export and Import vSphere Native Key Provider Configuration

You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to export and import vSphere Native Key Provider configuration.

You can set a vSphere Native Key Provider as the default for a vCenter instance by using the `SetDefaultKmsCluster` method of the `CryptoManagerKmip` managed object from the vSphere ?anagement SDK.

The operations require you to specify parameters in the body of the HTTP request according to your environment. For details about the syntax of each HTTP request body, see the *API Reference* documentation.

The code snippets in the following procedure are from the `ManageNativeKeyProviders.java` sample file.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

1. Export a vSphere Native Key Provider configuration by specifying the provider you want to export in the request body by calling the `export(ProvidersTypes.ExportSpec spec)` method of the `Providers` interface.

   Note:
   You must export the vSphere Native Key Provider configuration at least once before you can use a vSphere Native Key Provider.

   a) Initiate the export by providing the name identifier of the key provider and a password to encrypt the export data in the request body.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers?action=export
   ```

   The following request body contains example syntax.

   ```
   {
       "password": <your_password>,
       "provider": Test_Provider
   }
   ```

   In the response body, you receive the URL of the exported configuration and an authentication token. The following response body contains example syntax.

   ```
   {
       "location": {
           "download_token": {
               "expiry": 2023-09-20T21:04:50.000Z,
               "token": eyJhbGciOiJIUzI1NjKXn21y01CM
           },
           "url": https://vc8.home/cryptomanager/kms/test
       },
       "type": LOCATION
   }
   ```

   b) Send an HTTP request to the URL with the authentication token to fetch the exported PKCS12 data by providing the URL and token values received in the response body of the previous step.

   ```
   POST <url>
   Authorization: Bearer <token>
   ```

   The following example HTTP request contains the values from the example response in the previous step.

   ```
   POST https://vc8.home/cryptomanager/kms/test
   Authorization: Bearer eyJhbGciOiJIUzI1NjKXn21y01CM
   ```

   a) Back up vSphere Native Key Provider data.

   This is a two-stage process. First, you request a backup. In the second step, you download the backup by using a token and address that are returned from the first step.

   ```
   private byte[] backupKeyProvider(String name, char[] pwd) {
       // Step 1: request backup
       ExportSpec spec = new ExportSpec.Builder(name)
           .setPassword(pwd)
           .build();
       ExportResult res = nativeKeyProviders.export(spec);
       log("Backup step one: export result is {0}", res);

       // Step 2: download the backup
       URI url = res.getLocation().getUrl();
       char[] token = res.getLocation().getDownloadToken().getToken();
       return downloadBackupData(url, token);
   ```

```
}
```

b) Download the backup data from an online location.

This method uses the Java Apache HTTP client to download the back up data. The download is performed by making a POST request to the URL with an authorization bearer header carrying the supplied token.

```
private byte[] downloadBackupData(URI url, char[] token) {
    HttpPost request = new HttpPost(url);
    request.addHeader("Authorization", MessageFormat.format("Bearer {0}", new String(token)));

    log("Backup request {0}", request);
    try (CloseableHttpClient client = createHttpClient();
            CloseableHttpResponse resp = client.execute(request)) {
        int statusCode = resp.getStatusLine().getStatusCode();
        if (statusCode != 200) {
            log("Backup failed. HTTP status code {0}", statusCode);
            throw new RuntimeException("Cannot backup");
        }
        HttpEntity body = resp.getEntity();

        byte[] backup = new byte[body.getContent().available()];
        body.getContent().read(backup);
        log("Backup received {0} bytes. Backup completed.", backup.length);
        return backup;
    } catch (IOException e) {
        log("IO Exception during backup: {0}", e);
        throw new RuntimeException(e);
    }
}
```

2. Import a vSphere Native Key Provider configuration by specifying the configuration you want to import in the request body by calling the `importProvider(ProvidersTypes.ImportSpec spec)` method of the `Providers` interface.

```
POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers?action=import
```

The following request body contains example syntax.

Note:
You must provide a Base64-encoded string that is a representation of the PKCS12 data exported in the previous step as the value of the `config` parameter. The other parameters are optional.

```
{
    "config": <string>,
    "constraints": {
        "tpm_required": false
    },
    "dry_run": false,
    "password": <your_password>
}
```

```
private void restoreKeyProvider(byte[] backup, char[] pwd) {
    ImportSpec spec = new ImportSpec.Builder().setConfig(backup).setPassword(pwd).build();
    ImportResult res = nativeKeyProviders.importProvider(spec);
    log("Restored Native Key Provider {0}", res);
}
```

In the response body, you receive information about the imported provider and the time when that provider was exported.

Retrieve vSphere Native Key Providers on a Host

You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.hosts.kms` package to list vSphere Native Key Providers on a host and retrieve details about a specific vSphere Native Key Provider.

1. Retrieve a list of vSphere Native Key Providers on a host by calling the `list(java.lang.String host, ProvidersTypes.FilterSpec filterSpec)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/hosts/<host>/kms/providers
   ```

   In the response body, you receive information about the health status, ID, and type of each provider.

2. Retrieve information about a vSphere Native Key Provider on a host by calling the `get(java.lang.String host, java.lang.String provider)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/hosts/<host>/kms/providers/<provider>
   ```

   In the response body, you receive detailed information about the health status and the type of the provider.

Managing Host Hardware Security

You can use the VMware Cloud FoundationREST APISDK to perform host hardware management operations.

You can retrieve host hardware TPM information and manage host hardware TPM endorsement keys. You can also register SGX hosts.

The procedures in this chapter are based on the REST API. For details, see the *vSphere Automation REST API Reference*.

The procedures in this chapter are based on the Java API. For details, see the *vSphere Automation Java API Reference*.

Retrieve Host Hardware TPM Information

You can use HTTP requeststhe `Tpm` interface from the `com.vmware.vcenter.trusted_infrastructure.hosts.hardware` package to retrieve a list of configured TPM devices on a host and information about each TPM device.

1. List configured TPM devices on a host by calling the `list(java.lang.String host, TpmTypes.FilterSpec filter)` method of the `Tpm` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm
   ```

   You receive the results in the response body. You can use the retrieved information to review the basic information of the available TPM devices.

2. Retrieve detailed information about a specific TPM device, including the manufacturer, model, and firmware version by calling the `get(java.lang.String host, java.lang.String tpm)` method of the `Tpm` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>
   ```

   You receive the results in the response body. You can use the retrieved information to review the detailed information of the TPM device.

Manage Host Hardware TPM Endorsement Keys

You can use HTTP requestsinterfaces from the `com.vmware.vcenter.trusted_infrastructure.hosts.hardware.tpm` package to retrieve a list of configured TPM endorsement keys on a host and information about each endorsement key. You can also retrieve the TPM event log and unseal a secret that is bound to an endorsement key.

You can retrieve the TPM event log for different purposes, such as configuring firmware trust with an attestation service or validating the boot time TPM measurements. You can unseal a secret that is bound to an endorsement key to verify reported measurements. For example, you can verify measurements from the TPM event log. For details about the unsealing operation, see the *REST API Reference* documentation.

1. List configured TPM endorsement keys on a host by calling the `list(java.lang.String host, java.lang.String tpm, EndorsementKeysTypes.FilterSpec filter)` method of the `EndorsementKeys` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/<tpm>/endorsement-keys
   ```

   You receive the results in the response body. You can use the retrieved information to review the basic information of the available TPM endorsement keys.

2. Retrieve detailed information about a specific TPM endorsement key by calling the `get(java.lang.String host, java.lang.String tpm, java.lang.String key)` method of the `EndorsementKeys` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/<tpm>/endorsement-keys/<key>
   ```

   You receive the results in the response body. You can use the retrieved information to review the detailed information of the TPM endorsement key.

3. Retrieve the event log associated with a TPM device by calling the `get(java.lang.String host, java.lang.String tpm)` method of the `EventLog` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/<tpm>/event-log
   ```

   You receive the information in the response body.

4. Unseal a secret that is bound to an endorsement key by calling the `unseal(java.lang.String host, java.lang.String tpm, java.lang.String key, EndorsementKeysTypes.UnsealSpec spec)` method of the `EndorsementKeys` interface.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/<tpm>/endorsement-keys/<key>?action=unseal
   ```

   You receive a string that contains the unsealed secret.

Register an SGX Host

You can use HTTP requeststhe `Hosts` interface from the `com.vmware.vcenter.confidential_computing.sgx` package to register hosts by stored Intel Software Guard Extensions (SGX) configuration.

Register a host by stored SGX configuration by calling the `register_Task(HostsTypes.RegisterSpec spec)` method of the `Hosts` interface.

```
POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/confidential-computing/sgx/hosts?action=register&vmw-task=true
```

You receive the task ID in the response body. You can use the task ID to check the status of the task by running the following HTTP request.

```
GET https://<vcenter_ip_address_or_fqdn>/api/cis/tasks/<task_ID>
```

vSphere Supervisor Configuration and Management

Starting with vSphere 7.0, you can use the VMware Cloud FoundationREST APIs to enable and configure vSphere Supervisor on a vSphere cluster. Then you can run Kubernetes workloads directly on ESX hosts.

You can use the VMware Cloud Foundation to automate the enabling and disabling of vSphere Supervisor on a vSphere cluster, creating and managing namespaces.

The information in this chapter is intended for vSphere administrators who want to use the VMware Cloud FoundationREST APIs to configure their environment to run Kubernetes workloads in vSphere. To take fully advantage of these VMware Cloud FoundationREST APIs, you must have basic knowledge about the Kubernetes technology and containers.

For more information about how to configure and manage vSphere Supervisor through the vSphere Client, see the *Installing and Configuring vSphere Supervisor* and *vSphere with Tanzu Configuration and Management* documentation.

vSphere Supervisor Terminology

You must understand the basic terminology in this chapter to be able use the vSphere Supervisor automation APIs effectively.

vSphere Supervisor Basic Terms

Term	Description
Supervisor	A vSphere cluster that has the vSphere Supervisor enabled.
VKS cluster	An upstream Kubernetes cluster provisioned and managed by using the vSphere Kubernetes Service. A VKS resides in a vSphere Namespace. You can deploy workloads and services to such clusters in the same way as you do with standard Superviso r.
vSphere Namespace	A namespace that is created within a Supervisor. Each namespace sets the resource boundaries for CPU, memory, storage, and also the number of Kubernetes objects that can run within the namespace. After a namespace is configured, you can run Kubernetes workloads within the namespace.
vSphere Pod	A virtual machine with a small footprint that runs one or more Linux containers. A vSphere Pod is equivalent to a Kubernetes pod. vSphere Pods are compatible with the Open Container Initiative (OCI) and can run OCI compatible containers regardless the operating system.
Spherelet	A spherelet is an implementation of the kubelet functionality ported natively on each host in the Supervisor.
Kubernetes Workload	Workloads are applications that are deployed in one of the following ways: • As containers running inside vSphere Pods. • Workloads provisioned through the VM service. • clusters deployed by using the . • Applications running inside the clusters.
Supervisor control plane	vSphere Supervisor creates a Kubernetes control plane directly on the hypervisor layer. The control plane manages the worker nodes and the vSphere Pods in the Supervisor.
Supervisor worker nodes	ESX hosts that are part of a Supervisor are considered as worker nodes. You run your Kubernetes workloads on the worker nodes.
Container Runtime Executive (CRX)	CRX is an isolated Linux execution environment similar to a VM that works together with ESX.
VM Service	The VM Service functionality allows DevOps engineers to deploy and manage virtual machines in their Kubernetes environment through standard Kubernetes APIs. vSphere administrators are responsible for providing VM Classes and VM Images for the DevOps engineers to choose from, as well as managing resource allocations to self-service provisioned VMs.
Self-Service Namespace	vSphere administrators can activate the Self-Service Namespace service on a Supervisor and create namespace templates for DevOps engineers to create a vSphere Namespace themselves.

Term	Description
vSphere Zones	vSphere Zones provide high availability against clusters-level failures to workloads deployed on vSphere Supervisor. You can configure a three-zone Supervisor mapped to three vSphere clusters or a one-zone Supervisor mapped to a single vSphere cluster. In a single cluster deployment, the high availability is provided by vSphere HA and is only on a host level.

How to Configure and Manage a Supervisor

You use the `Clusters` service to enable and disable a Supervisor, or edit the configuration of an existing Supervisor. The `Clusters` service is provided within the `namespace_management` package.

You can enable a vSphere cluster to manage Kubernetes workload objects, only after you enable vSphere DRS in a fully automated mode and enable HA on the cluster.

Before you enable a vSphere Supervisor on a vSphere cluster, you must prepare your environment to meet the specific networking, storage, and infrastructure requirements. See the *Installing and Configuring vSphere Supervisor* documentation.

For more information about how to configure the storage settings to meet the requirements of vSphere Supervisor, see Creating Storage Policies for vSphere Supervisor.

For more information about how to configure the networking settings for Supervisors that are configured with the VMware NSX-T™ Data Center as the networking stack, see Configuring NSX for vSphere Supervisor.

Starting with vSphere 7.0 Update 1, you can enable a Supervisor with vSphere networking or NSX-T Data Center, to provide connectivity between control planes, services, and workloads. A Supervisor that is configured with vSphere networking uses a vSphere Distributed Switch to provide connectivity to Kubernetes workloads and control planes. The cluster also requires a third-party load balancer that provides connectivity to DevOps users and external services. You can install in your vSphere environment the HAProxy load balancer implementation that VMware provides. See Configuring the vSphere Networking Stack for vSphere Supervisor and Installing and Configuring the HAProxy Load Balancer.

Staring with vSphere 7.0 Update 2, if you are using vSphere networking, you can use the VMware Avi Load Balancer to support VKS clusters provisioned by the . See Using the Avi Load Balancer with vSphere Networking.

Persistent Storage in vSphere Supervisor

Some Kubernetes workloads use persistent storage to store data permanently. vSphere Supervisor integrates with Cloud Native Storage (CNS) to provision persistent storage.

To understand how vSphere Supervisor uses persistent storage, refer to the *vSphere Supervisor Concepts and Planning* and *vSphere Storage* documentations.

Creating Storage Policies for vSphere Supervisor

Before you enable vSphere Supervisor, you must set up the storage to provision the Kubernetes infrastructure. You achieve this task by creating storage policies to be used in the Supervisor and namespaces.

To automate the creation of a tag-based storage policy, use the VMware® vSphere Management SDK. For more information about how to create a tag-based storage policy through the Web Services API, see the *VMware Storage Policy SDK Programming Guide* and *vSphere Web Services SDK Programming Guide* documentations.

Optionally, you can use the VMware Cloud FoundationREST APIs to create and add a tag to the datastore. See the vSphere Tag Service chapter. Currently, you can create a tag-based storage policy only through the Web Services APIs.

Use the VMware Cloud FoundationREST APIs to retrieve the default storage policy of a specific datastore by calling the `get(datastore_ID)` function of the `com.vmware.vcenter.datastore.DefaultPolicy` serviceusing the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore/datastore_id/default-policy` request and submitting the ID of the datastore as a path parameter. You can also retrieve commonly used information about the storage policies available in the vCenter instance by calling the `list()` function of the `Policies` service. by using the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/compute/policies` request.

You can use the storage policies retrieved through the VMware Cloud Foundation APIs to perform the following tasks:

- Assign the storage policies to the Supervisor. The storage policies set within the Supervisor enable specification ensure that the Supervisor control plane, the ephemeral disks of all vSphere Pods, and the container images are placed on the datastores that the policies represent. See Configuring NSX for vSphere Supervisor.
- Assign the storage policies to the vSphere Namespace. The storage policies associated with a namespace determine which datastores the namespace can access and use for persistent volumes for the vSphere Pod and the pods inside a VKS cluster. See Create a vSphere Namespace.

Enabling ReadWriteMany Support

You can enable the ReadWriteMany support in vSphere Supervisor and allow multiple pods and applications to mount simultaneously a single persistent volume.

In vSphere 7.0 Update 3, only VKS clusters support persistent volumes in ReadWriteMany mode. When you enable file volume support for vSphere Supervisor, be aware of the potential security weaknesses:

- The volumes are mounted without encryption. The unencrypted data might be accessed while the data transits the network.
- Access Control List (ACL) is used for the file shares to isolate file share access within a supervisor namespace. It might have risk of IP spoofing.

Follow these guidelines for networking:

- Make sure the vSAN File Services is routable from the Workload network.
- Use common DNS server for vSAN File Services and the vSphere cluster.
- If your vSphere Supervisor has NSX networking, use the SNAT IP of the Supervisor namespace and the SNAT IP of the VKS cluster for ACL configuration.
- If you have vSphere Supervisor with vSphere Distributed Switch (VDS) networking, use the VKS cluster VM IP or the IP of the Supervisor namespace for ACL configuration.

Before you activate the file volume support on a Supervisor, you must set up a vSAN cluster with enabled vSAN File Service. To configure a vSAN cluster with enabled vSAN File Service in the vSphere Client, see the *Configure File Services* topic in the *Administering VMware vSAN* documentation. For more information about how to programmatically achieve this task, see the *vSAN SDKs Programming Guide* documentation.

You activate the ReadWriteMany support on a cluster when you enable vSphere Supervisor on it, or reconfigure an existing Supervisor. See Enable vSphere Supervisor on a Cluster with NSX as the Networking StackEnable vSphere Supervisor on a Cluster with NSX as the Networking Stack, Enable vSphere Supervisor on a Cluster with the vSphere Networking StackEnable vSphere Supervisor on a Cluster with the vSphere Networking Stack, and Reconfiguring a Supervisor. Pass the list of vSAN clusters to be used for provisioning file volumes by using the `setCnsFileConfig(CNSFileConfig cnsFileConfig)` Java methodthe `cns_file_config` property of respective data structure. Currently, you can use only the current vSphere cluster for provisioning file volumes if it is a vSAN cluster with enabled vSAN File Service.

To deactivate the persistent volumes support on a Supervisor, pass an empty list when you set the Cloud Native Storage persistent storage support for the cluster. After that existing ReadWriteMany persistent volumes provisioned in the cluster remain unaffected and usable.

Supervisor Networking

You can enable a Supervisor with vSphere networking or NSX to provision connectivity to Kubernetes control planes, services, and workloads.

A Supervisor that uses the vSphere networking stack is backed by a vSphere Distributed Switch and requires a load balancer to provide connectivity to DevOps users and external services. The Avi Load Balancer and the HAProxy load balancers are supported for vSphere 7.0 Update 2.

A Supervisor that is configured with NSX, uses the software-based networks of the solution and an NSX Edge load balancer to provide connectivity to external services and DevOps users.

Configuring NSX for vSphere Supervisor

vSphere Supervisor requires specific networking configuration to allow you to connect to the Supervisors, vSphere Namespaces, and all objects that run inside the namespaces.

Follow the instructions for installing and configuring the NSX for managing Kubernetes workloads documented in the *Installing and Configuring vSphere Supervisor* guide.

First, you need to create a vSphere Distributed Switch and a distributed port group for each NSX Edge uplink. To automate this step, use the Web Services APIs as described in the *vSphere Web Services SDK Programming Guide*. Then, you can use the NSX REST APIs to add a compute manager, create transport zones, and perform other steps required for configuring the NSX for vSphere Supervisor.

Configuring the vSphere Networking Stack for vSphere Supervisor

To configure a Supervisor with the vSphere networking stack, you must connect all hosts from the cluster to a vSphere Distributed Switch. Depending on your topology, you must create one or more distributed port groups on the switch and configure them as workload networks to the vSphere Namespaces on the cluster.

Workload networks provide connectivity to the nodes of VKS clusters and to the Supervisor control planes. The workload network that provides connectivity to Supervisor control planes is called primary workload network. Each Supervisor must have one primary workload network represented by a distributed port group.

The Supervisor control planes on the cluster use three IP addresses from the IP address range that is assigned to the primary workload network. Each node of a VKS cluster has a separate IP address assigned from the address range of the workload network that is configured with the namespace where the VKS cluster runs.

To create a vSphere Distributed Switch and port groups for configuring the vSphere networking stack of a Supervisor, you can use the vSphere Web Services APIs as described in the *vSphere Web Services SDK Programming Guide* documentation. When you create a distributed virtual switch, vCenter automatically creates one distributed virtual port group. You can use this port group as the primary workload network and use it to handle the traffic for the Supervisor control planes. Then you can create as many distributed port groups for the workload networks as your topology requires. For a topology with one isolated workload network, create one distributed port group that you will use as a network for all namespaces on the Supervisor. For a topology with isolated networks for each vSphere Namespace, create the same number of distributed port groups as the number of namespaces.

To list all workload networks available for a Supervisor and retrieve information about the configuration of a specific workload network, use the `Networks` service from the VMware Cloud FoundationREST APIs. To associate a vSphere Distributed port group to a workload network, set the necessary information through the `setVsphereNetwork(NetworksTypes.VsphereDVPGNetworkSetSpec vsphereNetwork)` parameter of the workload network `SetSpec` object.`PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/cluster_id/networks/network_id` request and submit a `NamespaceManagementNetworksSetSpec` JSON object in the request body. The `NamespaceManagementNetworksSetSpec` data structure holds the `NamespaceManagementNetworksVsphereDVPGNetworkSetSpec` structure through which you define the parameters of the existing vSphere Distributed port group.Use the `NetworksTypes.VsphereDVPGNetworkSetSpec` class to describe the configuration or retrieve information about the current configuration of the vSphere Distributed port group of a specific workload network.

If you want to retrieve a list of the distributed switches compatible with vSphere Supervisor on a vCenter system, use the `DistributedSwitchCompatibility` service and filter the available switches by using `VSPHERE_NETWORK` as the networking provider.

If you want to retrieve a list of the distributed switches compatible with vSphere Supervisor on a vCenter system, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/distributed-switch-compatibility` request and pass the `VSPHERE_NETWORK` value for the `network_provider` query parameter.

Installing and Configuring the HAProxy Load Balancer

You can use the VMware Cloud Foundation APIs to customize the HAProxy control plane VM after you install the HAProxy in your vSphere Supervisor environment.

If you use the vSphere networking stack in your vSphere Supervisor environment, you need to supply your own load balancer. You can use the open source implementation of the HAProxy load balancer that VMware provides.

For more information about the prerequisites for installation and the deployment procedure through the vSphere Client, see the *Installing and Configuring vSphere Supervisor* documentation.

You can use the VMware Cloud FoundationREST APIs to install and configure the HAProxy load balancer. You can download the latest version of the HAProxy OVA file from the VMware-HAProxy site to a content library item. For more information about how to achieve this task, see Upload an OVF or OVA Package from a Local File System to a Library Item . Then you can create a new VM from the OVA template in the content library as described in Deploy a Virtual Machine or vApp from an OVF Template in a Content Library.

To configure the HAProxy load balancer, call the `set(String cluster, String id, LoadBalancersTypes.SetSpec spec)` and pass the configuration through the `LoadBalancersTypes.SetSpec` object. The load balancer specification takes as a setting a `LoadBalancers.HAProxyConfigCreateSpec` instance that captures the runtime configuration of the HAProxy load balancer. You must set the following configuration parameters to the HAProxy load balancer configuration specification: To configure the HAProxy load balancer, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/load-balancers/<loadbalancer_id>` request and submit a `LoadBalancers.SetSpec` JSON object in the request body. You can define the following HAProxy load balancer settings within the `LoadBalancers.HAProxyConfigSetSpec` data structure which is an optional property of the `LoadBalancers.SetSpec` data structure.

Parameter	Description
`servers setServers(List<LoadBalancersTypes.Server> servers)`	A list of `Server`s that represent the endpoints for configuring the HAProxy load balancers. Each endpoint is described by a load balancer IP address and a Data Plane API management port. Each endpoint must be described with the port on the HAProxy VM on which the Data Plane API service listens. The Data Plane API service controls the HAProxy server and runs inside the HAProxy VM. The default port is *5556*. Port *22* is reserved for SSH.
`username setUsername(String username)`	The administrator user name that is configured with the HAProxy OVA file and is used to authenticate to the HAProxy Data Plane API server.
`password setPassword(char[] password)`	The password for the administrator user name.
`certificate_authority_chain setCertificateAuthorityChain(String certificateAuthorityChain)`	The certificate in PEM format that is signed or is a trusted root of the server certificate that the Data Plane API server presents.

Using the Avi Load Balancer with vSphere Networking

If you use the vSphere networking stack for workload management, you can install and configure the Avi Load Balancer, also known as Avi Load Balancer, Essentials Edition, to support the VKS clusters.

For more information about how to install and configure the Avi Load Balancerthrough the vSphere Client, see the *Installing and Configuring vSphere Supervisor* documentation.

You can use the VMware Cloud Foundation APIs to deploy the Avi Controller on your vSphere Management network. You can upload the latest version of the Avi Load Balancer to a library item from your local file system or from a URL. For more information about how to achieve this task, see Upload an OVF or OVA Package from a Local File System to a Library Item . Then you can deploy the Controller VM on your vSphere Management network from the OVA template in the content library as described in Deploy a Virtual Machine or vApp from an OVF Template in a Content Library.

To configure the Avi Load Balancer settings, create a `LoadBalancers.AviConfigCreateSpec` instance and use the following parameters.

To configure the Avi Load Balancer, send a `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/load-balancers/<loadbalancer_id>` request and submit a `LoadBalancers.SetSpec` JSON object in the request body. You can define the following Avi Load

Balancer settings within the `LoadBalancers.AviConfigSetSpec` data structure which is an optional property of the `LoadBalancers.SetSpec` data structure.

Parameter	Description
`server setServer(LoadBalancersTypes.Server server)`	The address of the Avi Controller that is used to configure virtual services.
`username setUsername(java.lang.String username)`	The administrator user name that is used for accessing the Controller VM of the Avi Load Balancer.
`password setPassword(char[] password)`	The password for the administrator user name.
`certificate_authority_chain setCertificateAuthorityChain(java.lang.String certificateAuthorityChain)`	The certificate in PEM format that is used by the Controller. You can use the certificate that you assigned during the configuration of the Avi Load Balancer.

Enable vSphere Supervisor on a Cluster with NSX as the Networking Stack

Through the VMware Cloud Foundation APIs, you can enable a vSphere cluster for managing Kubernetes workloads. A cluster configured with NSX supports running vSphere Pod and VKS clusters.

- Verify that your environment meets the system requirements for enabling vSphere Supervisor on the cluster. For more information about the requirements, see the *vSphere Supervisor Concepts and Planning* documentation.
- Verify that the NSX is installed and configured. See Configuring NSX for vSphere Supervisor.
- Create storage policies for the placement of pod ephemeral disks, container images, and Supervisor control plane cache.
- Verify that DRS is enabled in fully automated mode and HA is also enabled on the cluster.
- Configure shared storage for the cluster. Shared storage is required for vSphere DRS, HA, and storing persistent volumes of containers.
- Verify that the user who you use to access the VMware Cloud Foundation services has the **Modify cluster-wide configuration** privilege on the cluster.
- Create a subscribed content library on the vCenter system to accommodate the VM image that is used for creating the nodes of the VKS clusters.

To enable a vSphere cluster for Kubernetes workload management, you use the services under the `namespace_management` package.

1. Retrieve the IDs of the tag-based storage policies that you configured for vSphere Supervisor.

   Use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/storage/policies` request to retrieve a list of all storage policies and then filter the policies to get the IDs of the policies that you configured for the Supervisor.

2. Retrieve the IDs of the vSphere Distributed Switch and the NSX Edge cluster that you created when configuring the NSX for vSphere Supervisor.

   Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/networks/nsx/distributed-switches?action=check_compatibility` request to list all vSphere Distributed Switches associated with the specific vSphere cluster and then retrieve the ID of the Distributed Switch that you configured to handle overlay networking for the Supervisor.

   Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/networks/nsx/edges?action=check_compatibility` request to retrieve a list of the created NSX Edge clusters for the specific vSphere cluster and associated with the specific vSphere Distributed Switch. Retrieve the ID of the NSX Edge cluster that has the tier-0 gateway that you want to use for the namespaces networking.

3. Retrieve the ID of the port group for the management network that you configured for the management traffic.

   To list the visible networks available on the vCenter instance that match some criteria, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/networks` request and then retrieve the ID of the management network you previously configured.

4. Create a `Clusters.EnableSpec` JSON object and define the parameters of the Supervisor that you want to create.

   You must specify the following required parameters of the enable specification:

   - Storage policies settings and file volume support. The storage policy you set for each of the following parameters ensures that the respective object is placed on the datastore referenced in the storage policy. You can use the same or different storage policy for the different inventory objects.

Property	Description
`ephemeral_storage_policy`	Specify the ID of the storage policy that you created to control the storage placement of the vSphere Pods.
`image_storage`	Set the specification of the storage policy that you created to control the placement of the cache of container images.
`master_storage_policy`	Specify the ID of the storage policy that you created to control the placement of the Supervisor control plane cache.

     Optionally, you can activate the file volume support by using `cns_file_config`. See Enabling ReadWriteMany Support.
   - Management network settings. Configure the management traffic settings for the Supervisor control plane.

Property	Description
`network_provider`	Specify the networking stack that must be used when the Supervisor is created. To use the NSX as the network solution for the cluster, set `NSXT_CONTAINER_PLUGIN`.
`master_management_network`	Enter the cluster network specification for the Supervisor control plane. You must enter values for the following required properties: • `network`- Use the management network ID retrieved in Step 3. • `mode` - Set `STATICRANGE` or `DHCP` for the IPv4 address assignment mode. The `DHCP` mode allows an IPv4 address to be automatically assigned to the Supervisor control plane by a DHCP server. You must also set the floating IP address used by the HA primary cluster by using `floating_IP`. Use the DHCP mode only for test purposes. The `STATICRANGE` mode, allows the Supervisor control plane to have a stable IPv4 address. You can use it in a production environment. • `address_range`- Optionally, you can configure the IPv4 addresses range for one or more interfaces of the management network. Specify the following settings: – The starting IP address that must be used for reserving consecutive IP addresses for the Supervisor control plane. Use up to 5 consecutive IP addresses. – The number of IP addresses in the range. – The IP address of the gateway associated with the specified range. – The subnet mask to be used for the management network.
`master_DNS`	Enter a list of the DNS server addresses that must be used from the Supervisor control plane. If your vCenter instance is registered with an FQDN, you must enter the IP addresses of the DNS servers that you use with the vSphere environment so that the FQDN is resolvable in the Supervisor. The list of DNS addresses must be specified in the order of preference.
`master_DNS_search_domains`	Set a list of domain names that DNS searches when looking up for a host name in the Kubernetes API server. Order the domains in the list by preference.

Property	Description
`master_NTP_servers`	Specify a list of IP addresses or DNS names of the NTP server that you use in your environment, if any. Make sure that you configure the same NTP servers for the vCenter instance, all hosts in the cluster, the NSX, and vSphere Supervisor. If you do not set an NTP server, VMware Tools time synchronization is enabled.

- Workload network settings. Configure the settings for the networks for the namespaces. The namespace network settings provide connectivity to vSphere Pods and namespaces created in the Supervisor.

Property	Description
`ncp_cluster_network_spec`	Set the specification for the Supervisor configured with the NSX networking stack. Specify the following cluster networking configuration parameters for `NCPClusterNetworkEnableSpec`: • `cluster_distributed_switch` - The vSphere Distributed Switch that handles overlay networking for the Supervisor. • `nsx_edge_cluster` - The NSX Edge cluster that has tier-0 gateway that you want to use for namespace networking. • `nsx_tier0_gateway`- The tier-0 gateway that is associated with the cluster tier-1gateway. You can retrieve a list of `NSXTier0Gateway` objects associated with a particular vSphere Distributed Switch and determine the ID of the tier-0 gateway you want to set. • `namespace_subnet_prefix` - The subnet prefix that defines the size of the subnet reserved for namespaces segments. Default is 28. • `routed_mode` - The NAT mode of the workload network. If set to `false`: • The IP addresses of the workloads are directly accessible from outside the tier-o gateway and you do not need to configure the egress CIDRs. • File Volume storage is not supported. Default is `true`. • `egress_cidrs` - The external CIDR blocks from which the NSX Manager assigns IP addresses used for performing source NAT (SNAT) from internal vSphere Pods IP addresses to external IP addresses. Only one egress IP address is assigned for each namespace in the Supervisor. These IP ranges must not overlap with the IP ranges of the vSphere Pods, ingress, Kubernetes services, or other services running in the data center. • `ingress_cidrs`- The external CIDR blocks from which the ingress IP range for the Kubernetes services is determined. These IP ranges are used for load balancer services and Kubernetes ingress. All Kubernetes ingress services in the same namespace share a common IP address. Each load balancer service is assigned a unique IP address. The ingress IP ranges must not overlap with the IP ranges of the vSphere Pods, egress, Kubernetes services, or other services running in the data center. • `pod_cidrs` - The internal CIDR blocks from which the IP ranges for vSphere Pods are determined. The IP ranges must not overlap with the IP ranges of the ingress, egress, Kubernetes services, or other services running in the data center. All vSphere Pods CIDR blocks must be of at least /23 subnet size.
`worker_DNS`	Set a list of the IP addresses of the DNS servers that must be used on the worker nodes. Use different DNS servers than the ones you set for the Supervisor control plane.

Property	Description
service_cidr	Specify the CIDR block from which the IP addresses for Kubernetes services are allocated. The IP range must not overlap with the ranges of the vSphere Pods, ingress, egress, or other services running in the data center. For the Kubernetes services and the vSphere Pods, you can use the default values which are based on the cluster size that you specify.

- Supervisor size. You must set a size to the Supervisor which affects the resources allocated to the Kubernetes infrastructure. The cluster size also determines default maximum values for the IP addresses ranges for the vSphere Pods and Kubernetes services running in the cluster. You can use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/cluster-size-info` calls to retrieve information about the default values associated with each cluster size.
- Optional. Associate the Supervisor with the subscribed content library that you created for provisioning VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.

  To set the library, use `default_kubernetes_service_content_library` and pass the subscribed content library ID.

5. Enable vSphere Supervisor on a specific cluster by passing the cluster enable specification to the `Clusters` service.

A task runs on vCenter for turning the cluster into a Supervisor. Once the task completes, Kubernetes control plane nodes are created on the hosts that are part of the cluster enabled with vSphere Supervisor. Now you can create vSphere Namespaces.

Create and configure namespaces on the Supervisor. See Create a vSphere Namespace.

Enable vSphere Supervisor on a Cluster with the vSphere Networking Stack

Starting with vSphere 7.0 Update 1, you can select between creating a Supervisor with the vSphere networking stack or with NSX as the networking solution. A Supervisor that is configured with the vSphere networking stack only supports VKS clusters. vSphere Pods are not supported.

- Verify that your environment meets the system requirements for enabling vSphere Supervisor on the cluster. For more information about the requirements, see the documentation.
- Verify that DRS is enabled in fully automated mode and HA is also enabled on the cluster.
- Configure shared storage for the cluster. Shared storage is required for vSphere DRS, HA, and storing persistent volumes of containers.
- Create storage policies for the placement of Kubernetes control planes.
- Create a subscribed content library on the vCenter system to accommodate the VM image that is used for creating nodes of VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.
- Add all hosts from the cluster to a vSphere Distributed Switch and create port groups for workload networks. See Configuring the vSphere Networking Stack for vSphere Supervisor.
- Configure an HAProxy load balancer instance that is routable to the vSphere Distributed Switch that is connected to the hosts from the vSphere cluster.
- Verify that the user who you use to access the VMware Cloud Foundation services has the **Namespaces.Manage** privilege on the cluster.

To enable a cluster configured with the vSphere networking stack for Kubernetes workloads management, you must use the services under the `namespace_management` package.

1. Retrieve the ID of the cluster which hosts were added to the vSphere Distributed Switch.

   Use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/cluster-compatibility` request to filter the clusters by using their network providers. To retrieve a list of all clusters in the vCenter system which are configured with the vSphere networking stack, set the network provider in the filter specification to `VSPHERE_NETWORK`.

2. Retrieve the IDs of the tag-based storage policies that you configured for vSphere Supervisor.

   Use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/storage/policies` request to retrieve a list of all storage policies and then filter the policies to get the IDs of the policies that you configured for the Supervisor.

3. Retrieve the ID of the port group for the management network that you configured for the management traffic.

   To list the visible networks available on the vCenter instance that match some criteria and then retrieve the ID of the management network you previously configured, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/networks` request.

4. Create a Supervisor enable specification and define the parameters of the Supervisor that you want to enable.

   You must specify the following required parameters of the enable specification:

   - Supervisor size. You must set a size to the Supervisor which affects the resources allocated to the Kubernetes infrastructure. The cluster size also determines default maximum values for the IP addresses ranges for the vSphere Pod and Kubernetes services running in the cluster. You can use the `ClusterSizeInfo.get()` call `GET https://<server>/api/vcenter/namespace-management/cluster-size-info` request to retrieve information about the default values associated with each cluster size.
   - Storage policy settings and file volume support. To specify the ID of the storage policy that you created to control the placement of the Supervisor control plane cache, use the `master_storage_policy` propertythe `setMasterStoragePolicy(java.lang.String masterStoragePolicy)` method. Optionally, you can activate the file volume support by using the `cns_file_config` propertythe `setCnsFileConfig(CNSFileConfig cnsFileConfig)` method. See Enabling ReadWriteMany Support.
   - Load balancer. To specify the user-provisioned load balancer configuration for the cluster, use the `load_balancer_config_spec` parameter of the enable specification. You must specify the following parameters of the `LoadBalancersTypes.ConfigSpec` specification:

Parameter	Description
`id`	A user-friendly name of the load balancer. The name must be an alphanumeric string with a maximum length of 63 characters which is unique across the namespaces in the vCenter instance.
`provider`	The type of the load balancer that you want to use. In vSphere 7.0 Update 2, you can choose between the HAProxy load balancer and the Avi Load Balancer. Pass as a value to this parameter one of the following constants: `HA_PROXY` or `AVI`.
`address_ranges`	The IP address ranges in CIDR format from which HAProxy allocates the IP addresses for the virtual servers. You must provide at least one IP range which is reserved by HAProxy. The CIDR range specified with this parameter must not overlap with the IPs allocated for the Kubernetes control planes and workloads. The IP range that you configure must be on a separate subnet.
`ha_proxy_config_create_spec`	The HAProxy runtime configuration. See Installing and Configuring the HAProxy Load Balancer.
`avi_config_create_spec`	The Avi Load Balancer configuration. See Using the Avi Load Balancer with vSphere Networking.

- Management network settings. Configure the network parameters for the Kubernetes control planes.

Parameter	Description
`network_provider`	Specify the networking stack that must be used when the Supervisor is created. To use the vSphere network as the solution for the cluster, select `VSPHERE_NETWORK`.
`master_management_network`	Enter the cluster network specification for the Supervisor control plane. You must enter values for the following required properties: • `network`- Use the management network ID retrieved in Step 3. • `mode`- Set `STATICRANGE` or `DHCP` for the IPv4 address assignment mode. The `DHCP` mode allows an IPv4 address to be automatically assigned to the Supervisor control plane by a DHCP server. You must also set the floating IP address used by the HA primary cluster by using `floating_IP`. Use the DHCP mode only for test purposes. The `STATICRANGE` mode, allows the Supervisor control plane to have a stable IPv4 address and can be used in a production environment.
`master_DNS`	Enter a list of the DNS server addresses that must be used from the Supervisor control plane. If your vCenter instance is registered with an FQDN, you must enter the IP addresses of the DNS servers that you use with the vSphere environment so that the FQDN is resolvable in the Supervisor. The list of DNS addresses must be specified in the order of preference.
`master_DNS_search_domains`	Set a list of domain names that DNS searches inside the Kubernetes control plane nodes, so that the DNS server can resolve them. Order the domains in the list by preference.
`master_NTP_servers`	Specify a list of IP addresses or DNS names of the NTP server that you use in your environment, if any. Make sure that you configure the same NTP servers for the vCenter instance, all hosts in the cluster, and vSphere Supervisor. If you do not set an NTP server, VMware Tools time synchronization is enabled.

- Workload network settings. Configure the settings for the network that will handle the networking traffic for Kubernetes workloads running on the Supervisor.

Parameter	Description
`service_cidr`	Specify the CIDR block from which the IP addresses for Kubernetes services are allocated. The IP range must not overlap with the ranges of the vSphere Pods, ingress, egress, or other services running in the data center. For the Kubernetes services and the vSphere Pods, you can use the default values which are based on the cluster size that you specify.
`workload_networks_spec`	Enter the workload networks specifications for the cluster. To configure the primary workload network that is used to expose the Supervisor control plane to DevOps and other workloads, create a `NetworksType.CreateSpec` / `NetworksTypes.CreateSpec` instance. Enter the following parameters of the vSphere Distributed Switch: • `network`. The name of the vSphere Distributed Switch that is associated with the hosts in the cluster. The name must be a unique alphanumeric string that does not exceed 63 characters. • `network_provider`. Pass `VSPHERE_NETWORK` as value to this parameter. • `vsphere_network`. Optionally, you can create a `vsphere_DVPG_network_create_spec` instance to describe the configuration of the namespace network backed by the vSphere Distributed port group. You must define the following parameters for the vSphere Distributed port group specification: – `portgroup`. Specify the port group that serves as the primary network to the Supervisor. – `address_ranges`. Set the IP range for allocating IP addresses for the Kubernetes control planes and workloads. You must use unique IP ranges for each workload network. – `gateway`. Set the gateway for the primary network. – `subnet_mask`. Specify the subnet mask of the network.

- Content library settings. Add the subscribed content library that contains the VM images for deploying the nodes of VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.
  To set the library, use `default_kubernetes_service_content_library` and pass the subscribed content library ID.

5. Enable the Supervisor by passing the enable specification to the `Clusters` service.

A task runs on vCenter for enabling vSphere Supervisor on the cluster. Once the task completes, three Kubernetes control planes are created on the hosts that are part of the cluster.

Create and configure namespaces on the Supervisor.

Upgrading a Supervisor
You can use the vSphere SupervisorREST APIs to upgrade a single or a group of clusters to a specific version.

vSphere Supervisor supports rolling upgrades through the VMware Cloud FoundationREST APIs for Supervisors and for the infrastructure supporting these clusters. This model ensures that there is minimal downtime for the cluster workloads during the upgrade process.

To retrieve a list of all available vSphere Supervisor upgrade versions for a specific vCenter system, use the Cluster Available Versions service. You can get information about the release version, name, description, release date, and release notes for each available upgrade.

You must use the Software Clusters service for upgrading a Supervisor. You can retrieve upgrade information about all Supervisors enabled on a vCenter system by using the `list()` Java method `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/software/clusters` HTTP request. You receive a list of basic upgrade-related information for each cluster, such as the current software version, the date of the last successful upgrade, the upgrade status of the cluster, and so on. In case some of the clusters are in the process of upgrading, you can retrieve also information about their desired upgrade version. If you want to view a more detailed upgrade-related information about a cluster, you must use the `get(cluster_ID)` method`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/software/clusters/<cluster_id>` HTTP request.

After you view the details about the upgrade versions that you can apply on a single or multiple Supervisors, you can create upgrade specifications that define the versions you want to upgrade to. When you upgrade a batch of Supervisor and for some reason one of the clusters fails to upgrade, you receive information about the pre-check exceptions that led to that cluster upgrade failure.

Java

This example upgrades a Supervisor.

The following code snippet is part of the `UpgradeSupervisorCluster.java` sample. Some parts of the original code sample are omitted to save space. You can view the complete and up-to-date version of this sample in the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
@Override
 protected void run() throws Exception {

  System.out.println("We are building the Spec for upgrading vSphere supervisor
 cluster");
  com.vmware.vcenter.namespace_management.software.ClustersTypes.UpgradeSpec spec =
 new UpgradeSpec();
  spec.setIgnorePrecheckWarnings(true);
  spec.setDesiredVersion(this.desiredVersion);
  this.wcpUpdateService.upgrade(clusterId, spec);
  System.out.println(
    "Invocation is successful for updating vSphere supervisor cluster,
                              check H5C, track the status using GET API");
 }
```

Monitoring the Enable and Upgrade Supervisor Operations

When you run the Supervisor enable and upgrade operations, the status of the tasks is not returned. Since these operations might be time-consuming but critical when automating second- and third-party products with vSphere Supervisor, you can write a logic to track the task status.

To monitor the status of the enable and upgrade Supervisor operations, use the `Clusters.Info` data structure`ClustersTypes.Info` instance and query the Kubernetes and configuration status of the cluster. Track the status every two minutes or so, until you receive `READY` for `kubernetes_statusgetKubernetesStatus()` and `RUNNING` for `config_statusgetConfigStatus()`. These statuses indicate that the Supervisor reached the desired configuration status and is ready for running Kubernetes workloads.

Configure Supervisor Log Streaming

As a part of centralized log collection from VCF components into VCF Operations, a new external API is available in VCF 9.1 to help configure log streaming for better vSphere Supervisor observability.

This namespace management API expects a payload with "content" field in a specific format. The prescribed body follows the REST call:

```
PUT /api/vcenter/namespace-management/supervisors/{supervisor}/logs/agent-configuration
{
  "configurations": [ {
    "certificates": [ "<SSLtrustCertificates>" ],
    "content": "<fluentbit-config>",
    "enabled": true,
```

```
    "unittype": "SUPERVISOR"
  } ]
}
```

With unit type SUPERVISOR, the `content` field may be specified with one or more plug-ins (PARSER, INPUT, FILTER, OUTPUT) but the order of plugins should be maintained. Do not specify a SERVICE plug-in.

```
# Parser comment\n[PARSER]\n\t...\n\n# Input comment\n[INPUT]\n\t...\n\n# Filter comment\n[FILTER]\n\t...\n\n#
 Output comment\n[OUTPUT]\n\t...
```

For `...` in the content field, substitute a valid INI configuration for the FluentBit plug-ins (PARSER, INPUT, FILTER, OUTPUT). An example INI file is below. See docs.fluentbit.io for more details.

```
[OUTPUT]
    Name  vcfsuperv
    Match *
    apiKey ${API_KEY}
    maxBufferSize ${BUFFER_SIZE}
    maxRecords ${MAX_RECORDS}
```

Reconfiguring a Supervisor
You can change some or all the predefined settings of a Supervisor through the VMware Cloud FoundationREST APIs.

To update only some of the Supervisor settings, you must create an instance of `UpdateSpec` and pass it to the `update(cluster_ID, update_spec)` methoduse the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_ID>` HTTP request and submit the updated specification in the request body. To reconfigure entirely the Supervisor, you must create an instance of `SetSpec` and pass it to the `set(cluster_ID, set_spec)` methoduse the `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_ID>` HTTP request and submit a `Clusters.SetSpec` data structure in the request body. The specification replaces entirely all existing Supervisor settings.

The settings you can configure with the `UpdateSpec` and `SetSpec` specifications are the same as the ones that you used for enabling the Supervisor. For example, you can change the storage settings on the Supervisor. Note that the changes that you make to the storage settings after the initial cluster configuration, apply only to the newly created Supervisor control planes.

Deactivating a Supervisor
You can programmatically deactivate vSphere Supervisor on a vSphere cluster by using the VMware Cloud FoundationREST APIs.

When you deactivate a Supervisor, the vSphere Kubernetes Service forcefully deletes from the cluster all objects and configurations part of the Kubernetes infrastructure. To deactivate Kubernetes workloads on a cluster, call the `disable(String cluster)` operation of the `Clusters` service and pass as parameter the ID of the cluster on which you want to deactivate Kubernetes.Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>?action=disable` request and pass the cluster ID as a path parameter.

Content Libraries in vSphere Supervisor

vSphere Supervisor uses content libraries as centralized repositories for templates, VM images, VKS release distributions, and other files related to their deployment.

Creating, Securing, and Synchronizing Content Libraries for VKS Releases
VMware Tanzu distributes Kubernetes software versions as VKS releases. To obtain and use these releases on your VKS clusters, you create subscribed or local content libraries.

A VKS release provides the VMware Kubernetes distribution which can be used with VKS clusters. Each VKS release is distributed as an OVA package. The uses the OVA package to deploy the virtual machine nodes for VKS clusters.

A VKS release is supported on Photon OS. The virtual machine nodes that are built from the OVA package have a 16 GB disk size. You specify the CPU and RAM resource reservations when you use a virtual machine class to size the VKS cluster.

Depending on your need for synchronization frequency and on the access to the published content libraries storing the VKS releases, you can use two approaches for storing VKS releases.

> **Note:**
>
> Starting with vSphere 7.0 Update 3, you can protect your content library by a security policy. In such case, make sure that all library items are compliant. If a protected library includes a mix of compliant and non-compliant library items, DevOps engineers are not able to retrieve the list of VM images provided with the library.

Automated Synchronization of VKS Releases

VMware publishes a content library that contains the latest VMware distributions of Kubernetes as an OVA package. If you want to provision VKS clusters, you can create a subscribed content library on the vCenter instance where vSphere Supervisor is enabled. When configuring the content library subscription, use the following subscription URL of the publisher : https://wp-content.vmware.com/v2/latest/lib.json. For more information about how to create a subscribed content library, see Subscribe to a Content LibrarySubscribing to a Content Library.

When you create the subscription, you configure the synchronization mechanism for downloading the content of the published library. You can select between on demand and automatic download of the virtual machine image for the VKS cluster nodes. If you choose to synchronize the subscribed library on demand, only the metadata for the library content is updated and as a result storage space is saved. This approach is an important consideration as more images containing different Kubernetes versions are published. However, the first time you decide to use a new virtual machine image version, you have to wait for it to download.

Starting with vSphere 7.0 Update3, you can secure a subscribed content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you update a subscribed content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

You associate the subscribed content library with the Supervisor on which you want to create a VKS cluster, when you first enable vSphere Supervisor on a cluster. See Enable vSphere Supervisor on a Cluster with NSX as the Networking StackEnable vSphere Supervisor on a Cluster with NSX as the Networking Stack.

The size of the content library can grow over time as new Kubernetes versions and images are published. If the underlying storage runs out of space, you will need to move to a new subscribed content library. After you create a new subscribed content library that has sufficient capacity for the target cluster, update the library association of the Supervisor. See Reconfiguring a Supervisor.

Manual Synchronization of VKS Releases

In an air-gapped network environment, you can use the storing functionality provided by a local content library for the needed VKS releases. You must first create a local content library, then download the OVA package for each VKS release that you want to import to the library. See Create a Local Content LibraryCreating a Local Content Library.

Starting with vSphere 7.0 Update3, you can secure a local content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you update a local content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

You can find the latest versions of the Kubernetes distribution by navigating to the https://wp-content.vmware.com/v2/latest URL. You must download the `photon-ova.ovf` and `photon-ova-disk1.vmdk` for each distribution you want and then upload these files from your local file system to your local content library. See Upload an OVF or OVA Package from a Local File System to a Library Item .

Note:

Make sure that you use as a name for each library item the Photon image version and the Kubernetes version from the directory where you downloaded the files. For example: `photon-3-k8s-v1.20.2---vmware.1-tkg.1.1d4f79a`.

Creating and Managing Content Libraries for VM Provisioning in vSphere Supervisor
To provision new virtual machines in a vSphere Supervisor environment, the DevOps engineers rely on VM templates and images. Your role is to make sure the DevOps engineers have access to these VM templates and images by using the Content Library service.

You can create a local content library and populate it with VM templates in OVF or OVA file format, or other types of files. For more information and a sample of how to create a local content library, see Create a Local Content LibraryCreating a Local Content Library.

You can also create a subscription to download the content of a published local content library as described in the following topic: Subscribe to a Content LibrarySubscribing to a Content Library.

Starting with vSphere 7.0 Update3, you can secure the content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you update a local or subscribed content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

After you create the content library, you must populate it with content either from your local file system or from a Web server. You must use only the VM images available on the VMware Cloud Marketplace web site. For example, download or subscribe to VM Service Image for Ubuntu if you want to enable a DevOps engineer to deploy a VM using this image. For more information about the available ways to populate a content library with content, see How To Use Library ItemsWorking with Library Items.

You must give the DevOps engineers access to the VM templates stored in the content libraries, so that they can use these templates to provision VMs through the VM Service functionality. To give access, you must associate one or more content libraries to the namespace where the VM Service is present. See Associating a Content Library with a Namespace and Virtual Machines in vSphere Supervisor.

Associating a Content Library with a Namespace
You must give access to a source of VM templates, so that the DevOps engineers can use them to provision VMs in a self-service manner. To give access, you associate a content library with VM templates to the namespace used by the DevOps engineers.

You can add multiple content libraries to a namespace that has the VM Service enabled or the same content library to several namespaces. You associate a content library to a namespace when you create a new namespace, update or reconfigure an existing one.

To make the VM Service aware of the content libraries in your environment that the DevOps engineers can use to self-service VMs, you must use a `VMServiceSpec` instance and pass it to the namespace configurationuse the `NamespacesInstancesVMServiceSpec` data structure which is part of the namespace create specification. The instance contains a list of content libraries that will be used by the VM Service. You can specify this list with the `content_libraries` propertyby calling the `setContentLibraries(java.util.Set<java.lang.String> contentLibraries)` method of the VM Service specification.

You can also associate one or more VM classes with the namespace. See Associating a VM Class with a vSphere Namespace.

Managing Namespaces on a Supervisor

You can use the VMware Cloud Foundation APIs to create namespaces on a Supervisor and configure them with resource limits and permissions for the DevOps users.

To create and configure a namespace, use the `Instances` service from the `namespaces` package. You can configure the access control to the objects in a namespace by using the `Access` service.

Create a vSphere Namespace

You can use the vSphere Supervisor automation REST APIs to create namespaces on a Supervisor. You can set resource quotas, storage, as well as permissions for the DevOps users.

- Enable vSphere Supervisor on a vSphere cluster.
- Create users and groups for the DevOps engineers who will use the namespace. For more information about how to create users and groups through the Web Services APIs, see the *vSphere Web Services SDK Programming Guide*.
- Create storage policies for persistent storage used by the vSphere Pods and the pods inside a VKS cluster.
- Create VM Classes and content libraries for DevOps provisioned VMs. See Create a VM Class in vSphere Supervisor and Creating and Managing Content Libraries for VM Provisioning in vSphere Supervisor.
- Required privileges on the Supervisor:
  - **Namespaces** > **Modify cluster-wide configuration**
  - **Namespaces** > **Modify namespace configuration**
  - **Virtual Machine Classes** > **Manage Virtual Machine Classes**

1. Retrieve the Supervisor ID by filtering the clusters available in the vCenter system.

   Call the list method of the `Clusters` service from the `com.vmware.vcenter.namespace_management` package and retrieve the ID of the cluster on which you want to create a namespace from the returned cluster summary objects.

   Use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters` request and retrieve the ID of the cluster on which you want to create a namespace from the list of cluster summary JSON objects.
2. Retrieve the ID of the storage policy that you configured for placement of the persistent volumes from vSphere Pods and VKS clusters.
3. Configure the access control to the objects in the namespace.

   Create an instance of the `com.vmware.vcenter.namespaces.InstancesTypes.Access` class and specify the following access information:

   Populate the properties of the `Instances.Access` data structure with appropriate values:

Parameter Property	Description
`domain setDomain(domain)`	Set the domain name of the vCenter system on which the namespace is created.
`subject_type setSubjectType(subjectType)`	Set the type of the user accounts that are associated with the specific role for the namespace. You must select between the `USER` and `GROUP` options.
`subject setSubject(subject)`	Set the name of the user or group that have permissions to access the namespace objects.
`role setRole(role)`	Set the role that is associated with the predefined set of privileges that you want to grant the specific user or group. You can select between the `EDIT`, `VIEW` and `OWNER` roles. The owner role is introduced in vSphere 7.0 Update 2a. When a DevOps engineer creates a namespace in a self-service manner, the Namespace Self-Service grants the owner role to the namespace creator. See Self-Service Namespace Management.

4. Create a `CreateSpec` instance that holds the namespaces specification. Populate the `Instances.CreateSpec` data structure with the appropriate namespace specification information.

   The namespace specification can contain the following information:

Parameter Property	Description
`cluster setCluster(cluster)`	Set the ID of the Supervisor on which the namespace is created.
`namespace setNamespace(namespace)`	Set a name of the namespace following the DNS label standard defined in RFC 1123. The name must be unique across all namespaces in the current vCenter system.
`networks setNetworks(java.util.List<java.lang.String> networks)`	Optional. You can set the workload networks used by the vSphere Namespace. Pass `null` as a value of this parameter, if the Su pervisor is configured to use NSX as networking solution. The workload networking support for such namespaces is provisioned by NSX. If the Supervisor uses the vSphere networking stack, pass the workload network to be associated with the namespace. If you pass `null` as a value of this parameter, the vSphere Namespaces on the cluster are automatically associated with the cluster primary workload network. See Configuring the vSphere Networking Stack for vSphere Supervisor.
`description setDescription(description)`	Optional. You can set a description of the namespace.
`access_list setAccessList(accessList)`	Optional. You can set the access control that is associated with the namespace in Step 3.
`storage_specs setStorageSpecs(storageSpecs)`	Optional. You can set the amount of storage dedicated to each storage policy associated with the namespace and the maximum amount of storage that is used by the namespace. Use the `StorageSpec` specification to configure the storage quotas on the namespace.
`resource_spec setResourceSpec(resourceSpec)`	Optional. You can set resource limitations to the namespace. You can limit the CPU, memory, the maximum number of pods that can exist on the namespace, and so on.
`creator setCreator(InstancesTypes.Principal creator)`	Optional. The Namespace Self-Service populates this parameter with information about the DevOps user who created the namespace with `cubectl`. The user name and domain of the namespace creator are stored with this parameter.
`vm_service_spec setVmServiceSpec(InstancesTypes.VMServiceSpec vmServiceSpec)`	Optional. The VM Service specification for the Dev-Ops provisioned virtual machines.

5. Create a namespace object on the Supervisor by using the namespace create specification.

   Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances` request and submit an `Instances.CreateSpec` JSON object in the request body.

Share the namespace with DevOps engineers and provide them with the user or group configured for accessing the namespace.

Java

This example creates a vSphere Namespace on a Supervisor.

The following code snippet is part of the `CreateNameSpace.java` sample. Some parts of the original code sample are omitted to save space. You can view the complete and up-to-date version of this sample in the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
(...)
  @Override
  protected void run() throws Exception {
```

```
    InstancesTypes.CreateSpec spec =new InstancesTypes.CreateSpec();
    spec.setCluster(this.clusterId);
    spec.setDescription("My first namespace, WOW");
    spec.setNamespace(this.namespaceName);
    InstancesTypes.StorageSpec storageSpec=new InstancesTypes.StorageSpec();
    storageSpec.setLimit(Long.valueOf(this.storageLimit).longValue());
    storageSpec.setPolicy(this.storagePolicyId);
    List<InstancesTypes.StorageSpec> storageSpecs = new
ArrayList<InstancesTypes.StorageSpec>();
    storageSpecs.add(storageSpec);
    spec.setStorageSpecs(storageSpecs);
    InstancesTypes.Access accessList= new InstancesTypes.Access();
    accessList.setDomain(this.domainName);
    if(this.roleName.equalsIgnoreCase("EDIT")) {
    accessList.setRole(AccessTypes.Role.EDIT);
    } else{
     accessList.setRole(AccessTypes.Role.VIEW);
    }
    accessList.setSubject( this.subjectName); //Default is Administrator
    if(this.subjectType.equalsIgnoreCase("USER")) {
     accessList.setSubjectType( AccessTypes.SubjectType.USER);
    } else{
     accessList.setSubjectType( AccessTypes.SubjectType.GROUP);
    }

    List<InstancesTypes.Access> accessLists = new ArrayList<InstancesTypes.Access>();
    accessLists.add(accessList);
    spec.setAccessList(accessLists);
    this.namespaceService.create(spec);
    System.out.println("Invocation is successful for creating supervisor namespace,
check H5C or call GET API to get status");


   }
  (...)
```

Updating the Namespace Configuration

You can change the whole namespace configuration or only some of the namespace settings.

To change the configuration of an existing namespace, you must have the **Namespaces.Configure** privilege on the Supervisor.

> **Note:**
>
> Before deleting a storage policy from vCenter or a vSphere Namespace, or changing the storage policy assignment, make sure that no persistent volume claim with the corresponding storage class runs in the namespace. Also, ensure that no VKS cluster is using the storage class.

To patch a namespace configuration, create an `UpdateSpec` specification and set new values only to the configuration settings that you want to change. The parameters of the update specification are the same as the ones you configured during the namespace creation. When you call the update operation, only the settings that you configured in the update specification are applied, the other settings are left as they are.

To patch a namespace configuration, use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/<namespace_id>` request and submit an `UpdateSpec` JSON object in the request body. You set new values only to the configuration settings that you want to change. The parameters of the update specification are the same as the ones you configured during the namespace creation. When you issue the update request only the settings that you configured in the update specification are applied, the other settings are left as they are.

To reconfigure a namespace entirely, you must create an instance of the `SetSpec` class. You can change the description, access controls, storage settings, and resource limitations of the specific namespace.

To reconfigure a namespace entirely, you must issue a `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/<namespace_id>` request and submit a `SetSpec` JSON object in the request body. You can change the description, access controls, storage settings, and resource limitations of the specific namespace.

Configuring the Access to a Namespace

You can use the vSphere Supervisor APIs to grant access permissions to DevOps engineers on the vSphere Namespaces.

Use the `Access` service to retrieve information about the access control of the DevOps engineers on a specific namespace. You can also set up or remove an access control for a specific user or group on a specific namespace, and add another access control on the namespace. You set up each access control to allow a user or group to access a namespace in a specific vCenter system. You can grant access to a DevOps engineer to more than one namespace.

You must have the **Namespaces** > **Configure** > privilege to grant permissions to a user. You assign the view and edit access role on the namespace for the user or group.

Starting with vSphere 7.0 Update 2?, you can also assign the owner role to a DevOps engineer. These roles allow the user to deploy workloads, share the namespace with other DevOps engineers, and delete it when it is no longer needed.

Self-Service Namespace Management

You can use the vSphere Supervisor automation REST APIs to create a vSphere Namespace with specific resource quotas, set permissions, and assign storage policies. DevOps engineers can then use the namespace as a template for self-provisioning namespaces on the cluster.

Starting with vSphere 7.0 Update 2a, the Namespace Self-Service feature is available in vSphere Supervisor. The service enables Kubernetes users to create vSphere Namespaces from templates configured through the automation REST APIs or vSphere Client. To activate the Namespace Self-Service on a cluster, use one of the following options:

- Create a self-service namespace template and then activate the Namespace Self-Service on the cluster.
- Create or update a self-service namespace template simultaneously with activating the Namespace Self-Service on the cluster.

Currently, only one namespace self-service template is allowed per vSphere Namespace. After a DevOps engineer creates a namespace from the template, the namespace can also be deleted through `kubectl`. You can verify whether a namespace is created from a template by retrieving the value of the`self_service_namespace` property of the `Instances.Info` JSON object that you receive in the request body of the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/namespace_id` request.`getSelfServiceNamespace()` flag of the `com.vmware.vcenter.namespaces.InstancesTypes.Info` object that you receive when you call the `get(String namespace)` method of the `Instances` interface.

To create a template for a self-service namespace, call the `create(String cluster, NamespaceTemplatesTypes.CreateSpec spec)` method of the `NamespaceTemplates` interface. use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-templates/clusters/cluster_id` HTTP request. You use as path parameterparameters the cluster ID and submit in the request body the namespace template create specification.

You define the following configuration settings and resource limitations of the template:

ParameterProperty	Description
`templatesetTemplate(String template)`	The identifier of the namespace template must be a unique name across all clusters on the vCenter instance. The name must be compliant with DNS.
`resource_specsetResourceSpec(Structure resourceSpec)`	The resource quotas, such as CPU and memory, that are reserved for the namespace on the vCenter instance. The CPU limit is set in MHz and the minimum value is 10 MHz. The memory and the storage limits are set in MiB. For more options to configure resource limits for the namespace, see the `ResourceQuotaOptionsV1` class in the API Reference documentation.
`storage_spec ssetStorageSpecs(List<InstancesTypes.StorageSpec> storageSpecs)`	The amount of storage in MiB utilized for each storage policy that you associate with the namespace. You must specify at least one policy.
`networkssetNetworks(List<java.lang.String> networks)`	Optional. The networks associated with the namespace. Currently, you can set only one network for the namespace. Pass `null` as argument if the Supervisor is configured with NSX-T Data Center support. If you pass `null` for a namespace template on a cluster configured with a vSphere networking stack, the namespace is automatically associated with the Supervisor management workload network.
`permission ssetPermissions(List<NamespaceTemplatesTypes.PermissionSpec> permissions)`	Optional. The permissions that allow DevOps engineers to use the template to self-provision namespaces through `kubectl`. If set to `null`unset, only users with the Administrator role can use the template.

Once you have the template created, you can activate the Namespace Self-Service on the cluster by calling the `activate(java.lang.String cluster)` method of the `NamespaceSelfService` interface.by issuing the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-self-service/<cluster_id>?action=activate` HTTP request. If you want to restrict DevOps users to use the namespace template on a cluster, you can deactivate the Namespace Self-Service feature. Then users are able to delete only the namespaces already created from the template.

You can activate the Namespace Self-Service on the cluster after configuring the namespace template by using the `NamespaceSelfService` service. You call the `activateWithTemplate(java.lang.String cluster, NamespaceSelfServiceTypes.ActivateTemplateSpec spec)` method of the `NamespaceSelfService` interface.You issue the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-self-service/<cluster_id>?action=activateWithTemplate` HTTP request and submit the `NamespaceSelfService.ActivateTemplateSpec` JSON object in the request body. Depending on the availability of a template on the cluster, this method either creates a namespace template or activates the deactivated service and at the same time updates the existing template.

Virtual Machines in vSphere Supervisor

vSphere Supervisor offers the VM Service functionality to enable DevOps engineers to provision and manage VMs on a namespace in a self-service manner. You use the vSphere Supervisor automation APIs to create VM classes that specify the deployment policy and resource reservations of such VMs.

Starting with vSphere 7.0 Update 2a, DevOps engineers can use the VM Service functionality to deploy and run VMs on a namespace through the `kubectl` commands. You can use the vSphere Supervisor automation APIs to manage the two VM Service components: VM classes and content libraries. For more information about managing content libraries in the context of vSphere Supervisor, see Content Libraries in vSphere Supervisor.

You can use the automation APIs to create and manage VM classes. A VM class specification defines the number of CPUs, memory capacity, and resource reservation settings of the desired virtual machine. vSphere Supervisor currently

offers twelve ready-to-use VM classes (T-shirt sizes) that are derived from the most popular VMs in Kubernetes. Based on the resource reservation that a VM specification requests, each predefined VM class has two editions: guaranteed and best effort. The guaranteed VM class fully reserves the configured resources. A best effort VM class does not guarantee any resource reservations and allows their overcommitment.

You associate a VM class with a specific namespace to make it available to the DevOps engineers who have access to that namespace. You can assign any number of existing VM classes or create a custom one. Note that VMs deployed by the DevOps engineers through the VM Service can only be managed with the `kubectl` commands. A VM provisioned by DevOps engineers shares the same resources in a namespace as containers.

Use the `VirtualMachineClasses` interface to create and manage a specification of a VM class object. Through these objects you predefine the number of CPUs, memory capacity, and reservation settings. See Create a VM Class in vSphere Supervisor. To make a VM class available to the DevOps engineers for self-service VM deployment, you must associate it with a specific namespace. See Associating a VM Class with a vSphere Namespace.

Create a VM Class in vSphere Supervisor

You can use the VMware Cloud Foundation Kubernetes REST APIs to create custom VM classes to be used for VM deployment in vSphere Supervisor.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

A VM class specifies the CPU, memory, and resource reservations for a VM. vSphere Supervisor offers several preconfigured VM classes which you can use as is, edit, or delete. You can also create a custom VM class in your vCenter instance and it will be available to all Supervisors and the namespaces created in these clusters. Note that even though a VM class is available to all namespaces, a DevOps user can only use the VM classes associated with the namespaces that he/she can access.

1. Create the specification of the VM class object by defining the following options.

Option	Description
`id setId(String id)`	The identifier of the VM class must follow these DNS requirements: • A unique name in the current vCenter instance. • An alphanumeric name with maximum 63 characters. • No uppercase letters of spaces. • A dash can be used anywhere except as a first or last character. Note that after a VM class is created, you cannot edit its ID.
`cpu_count setCpuCount(long cpuCount)`	The number of virtual CPUs (vCPUs) configured for a VM that are deployed with this VM class.
`memory_MB setMemoryMB(long memoryMB)`	The memory in MB configured for a VM that are deployed with this VM class. The value must be between 4 MB and 24 TB and a multiple of 4.
`description setDescription(String description)`	Optional. The description of the VM class.
`cpu_reservation setCpuReservation(Long cpuReservation)`	Optional. The percentage of total available CPU resources reserved for the VM deployed with the VM class. The percentage you specify with this attribute is multiplied by the minimum CPU available among all cluster nodes to get the CPU resources guaranteed by vSphere for a VM. The resulting value is in MHz.
`memory_reservation setMemoryReservation(Long memoryReservation)`	Optional. The percentage of available memory that is reserved for a VM deployed with this VM class. The value can be from 0 through 100%.

2. Create the VM class object.

   Call the `create(VirtualMachineClassesTypes.CreateSpec spec)` method of the `VirtualMachineClasses` interface and pass as argument the created VM class specification. Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes` HTTP request and pass the created VM class specification in the request body.

After you create the custom VM class, you can edit its parameters or delete it from your environment. See Editing or Removing a VM Class from Your Environment.

You can make your VM class available to DevOps engineers by associating it with a namespace. See Associating a VM Class with a vSphere Namespace.

Editing or Removing a VM Class from Your Environment

You can use the automation REST APIs to edit the configuration of a VM class that you created or a predefined VM classes that vSphere Supervisor offers. When you no longer need an existing VM class, you can remove it from your vCenter instance.

Note that editing a VM class specification does not affect the VMs that are already deployed by the DevOps engineers from this class. Only newly deployed VMs will use the reconfigured VM class.

Deleting a VM class results in its removal from all related namespaces. All VMs deployed with this VM class remain unchanged but DevOps engineers can no longer use it.

You can list all VM classes available for a vCenter instance by calling the `list` method of the `VirtualMachineClasses` interface. by using the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes` HTTP request. You receive a list of all VM classes and a detailed information about each one of them. Based on the detailed information, you can narrow the list and retrieve the IDs of the VM classes that you want to edit or delete.

To edit a VM class configuration, call the `update(String vmClass, VirtualMachineClassesTypes.UpdateSpec spec)` method of the `VirtualMachineClasses` interface and pass as arguments the VM class ID and the update specification.use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes/vm_class_id` HTTP request and pass the update specification in the request body. You can edit all VM class attributes except the ID of the class. Only the attributes modified within the update specification will be edited.

To delete a VM class from your environment, call the `delete(java.lang.String vmClass)` method of the `VirtualMachineClasses` interface and pass the ID of the class as argument.use the `DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes/vm_class_id` HTTP request.

Associating a VM Class with a vSphere Namespace

You must associate a VM class with a namespace to make it available for DevOps engineers to deploy VMs in a self-service manner.

You can associate one or more VM classes with a single namespace or you can add one VM class to several namespaces. You can use the predefined VM classes that vSphere Supervisor provides or you can create custom ones. See Create a VM Class in vSphere Supervisor.

Since VM Service is the feature responsible for handling VM classes, you must make that service aware of the VM classes available to the engineers using a specific namespace. You can achieve this when you create a new namespace or edit an existing one. See Managing Namespaces on a Supervisor.

When you create the VM Service specification set the list of VM classes that must be used to create VMs on the specific namespace. You achieve this by calling the `setVmClasses(java.util.Set<java.lang.String> vmClasses)` method of the `VirtualMachineClasses` object.using the `vm_classes` property of the `Instances.VMServiceSpec` data structure.

You can also associate one or more content libraries with a namespace that has the VM Service enabled. See Associating a Content Library with a Namespace.

vCenter Management

You can use the API to configure, monitor, maintain, and update vCenter.

Authorization Model for Administration of vCenter

There are three types of authorization levels in vCenter.

Table 1859: Authorization Levels

Authorization Level	Description
operator	A user has read access to configuration settings.
administrator	A user has read and write access to configuration settings, but cannot manage user accounts.
super administrator	A user has all the capabilities of the other roles, and has the additional capabilities of creating local user accounts and accessing the local Bash shell.

This model applies to the API and all other interfaces to vCenter except when you use SSH and log in by using a local account.

Authorization Model Mapping to the vCenter Single Sign-On Domain

The three-level authorization model of vCenter maps to local roles and to vCenter Single Sign-On groups, depending on how the user authenticated. This model allows consistent security control regardless of operational context.

The authorization levels map to group and role.

Table 1860: Authorization Mapping

Authorization Level	vCenter Single Sign-On Group	vCenter Local Role
operator	SystemConfiguration.Administrators	operator
administrator	SystemConfiguration.Administrators	admin
superAdministrator	SystemConfiguration.BashShellAdministrators	superAdmin

When a super administrator adds user accounts, the options available include a choice of the role to assign to the new user.

Using the Operator Role

The **operator** role is the most restricted of the authorization levels available to users who work with vCenter.

Operators are allowed to view information about vCenter. They are not allowed to alter its configuration. The **operator** role is suited for monitoring and reporting functions. For example, the **operator** role provides access to the following methods.

- `resources.system.health.get`
- `resources.storage.stats.list`
- `services.status.get`

Using the Admin Role

The **administrator** role provides an intermediate authorization level for users who manage vCenter.

An **administrator** role is required for users who alter the vCenter configuration, exercise control functions, or other operations that can affect regular users.

For example, the **administrator** role provides access the following methods.

- `networking.ip4v.renew`

- `networking.firewall.addr.inbound.add`
- `services.control`
- `shutdown.reboot`

Using the SuperAdmin Role
The **superAdmin** role is the most expansive authorization level for users who manage vCenter.

The **superAdmin** role allows unrestricted access to vCenter. This role is required for adding or altering user accounts and for using the Bash shell.

Password Expiry and Lockout Policy for the Administrator Account

VCF 9.1.1 provides an API that controls whether fleet password expiry and lockout policies apply to the default Single Sign-On administrator account, which is exempt by default.

The default Single Sign-On (SSO) administrator account, for example `administrator@vsphere.local`, does not expire and does not lock out after failed login attempts, regardless of the authorization level described in Authorization Model for Administration of vCenter. This exemption exists so that applying the fleet password policy does not break existing automation, integrations, and daily operations that depend on this account.

Starting with VCF 9.1.1, you can opt the administrator account in to the fleet expiry policy, the fleet lockout policy, or both, by using the `PasswordPolicy` API. The account remains opted out of both policies until you explicitly opt it in. When you opt the account in to a policy, the fleet-wide settings that already apply to other local accounts, such as the maximum password age or the failed-login threshold, also apply to the administrator account.

> **Note:**
>
> In VCF 9.1.1, this setting is available only through the API. VMware Cloud Foundation Operations does not expose a user interface control for this setting in this release.

Password Policy Operations

The following table lists the operations that the `PasswordPolicy` resource provides at `/api/identity/admin/administrator/password-policy`. The resource represents a single policy record for the administrator account, so the path does not include an identifier.

Operation	HTTP Method	Description
Retrieve the policy	`GET`	Returns the current opt-in state of the expiry and lockout policies for the administrator account.
Update the policy	`PATCH`	Updates one or both opt-in flags for the administrator account. A field that you omit from the request body retains its current value.

The response and request bodies contain the following fields.

Field	Type	Description
`enable_expiry_policy`	boolean	When set to `true`, the fleet password expiry policy applies to the administrator account. When set to `false`, the administrator password does not expire. The default value is `false`.
`enable_lockout_policy`	boolean	When set to `true`, the fleet lockout policy applies to the administrator account. When set to `false`, the administrator account does not lock out after failed login attempts. The default value is `false`.

Examples of Updating the Password Expiry and Lockout Policy for the Administrator Account

REST

- This example retrieves the current policy for the administrator account.

```
curl -X GET -u administrator@vsphere.local:<password> -k \ 'https://
<vcenter_ip_address_or_fqdn>/api/identity/admin/administrator/password-policy'
```

- This example opts the administrator account in to both the expiry and the lockout policy.

```
curl -X PATCH -u administrator@vsphere.local:<password> -k \ -H "Content-Type:
 application/json" \ -d '{"enable_expiry_policy": true, "enable_lockout_policy":
 true}' \ 'https://<vcenter_ip_address_or_fqdn>/api/identity/admin/administrator/
password-policy'
```

Java

This Java example retrieves the current password expiry and lockout opt-in state for the administrator account, opts the account in to both policies, and confirms the change.

This example uses the `PasswordPolicy` interface from the `com.vmware.identity.admin.administrator` package.

```
import com.vmware.identity.admin.administrator.PasswordPolicy;
import com.vmware.identity.admin.administrator.PasswordPolicyTypes;

private void updateAdministratorPasswordPolicy(
        StubFactory stubFactory, StubConfiguration sessionStubConfig) {

    PasswordPolicy passwordPolicyService =
            stubFactory.createStub(PasswordPolicy.class, sessionStubConfig);

    // Retrieve the current opt-in state for the administrator account.
    PasswordPolicyTypes.Info currentPolicy = passwordPolicyService.get();
    System.out.println("Expiry policy enabled: "
            + currentPolicy.getEnableExpiryPolicy());
    System.out.println("Lockout policy enabled: "
            + currentPolicy.getEnableLockoutPolicy());

    // Opt the administrator account in to both the expiry and lockout policies.
    PasswordPolicyTypes.UpdateSpec updateSpec = new PasswordPolicyTypes.UpdateSpec();
    updateSpec.setEnableExpiryPolicy(true);
    updateSpec.setEnableLockoutPolicy(true);
    passwordPolicyService.update(updateSpec);

    // Retrieve the updated policy to confirm the change.
    PasswordPolicyTypes.Info updatedPolicy = passwordPolicyService.get();
    System.out.println("Expiry policy enabled: "
            + updatedPolicy.getEnableExpiryPolicy());
    System.out.println("Lockout policy enabled: "
            + updatedPolicy.getEnableLockoutPolicy());
}
```

Python

This Python example retrieves the current password expiry and lockout opt-in state for the administrator account, opts the account in to both policies, and confirms the change.

This example uses the `PasswordPolicy` class from the `com.vmware.identity.admin.administrator_client` module.

```
from com.vmware.identity.admin.administrator_client import PasswordPolicy


def update_administrator_password_policy(stub_config):
    """
    Retrieve the current opt-in state for the administrator account,
    opt the account in to the expiry and lockout policies, and confirm
    the change.
    """
    password_policy_svc = PasswordPolicy(stub_config)
```

```
current_policy = password_policy_svc.get()
print('Expiry policy enabled: {}'.format(current_policy.enable_expiry_policy))
print('Lockout policy enabled: {}'.format(current_policy.enable_lockout_policy))

update_spec = PasswordPolicy.UpdateSpec(enable_expiry_policy=True,
                                        enable_lockout_policy=True)
password_policy_svc.update(update_spec)

updated_policy = password_policy_svc.get()
print('Expiry policy enabled: {}'.format(updated_policy.enable_expiry_policy))
print('Lockout policy enabled: {}'.format(updated_policy.enable_lockout_policy))

return updated_policy
```

Performing Privilege Checks Operations

Privilege checks recorder is a feature which allows you to monitor and subsequently query the privileges that were checked. You can use the recordings to create scripts that automatically create roles with minimum required privileges to run specific workflows of operations.

Currently, it is very hard to find out the minimal set of privileges that are required to run a specific workflow of operations. The `PrivilegeChecks`serviceinterface provides operationsmethods for retrieving a list of the latest privilege checks along with the corresponding sessions, users, managed objects, and operation IDs (`opIDs`).

The following table lists the available privilege checks operations.

Operation	Description
List privilege checks	Returns a list of privilege checks that match criteria which you specified.
Get latest privilege check	Returns a marker to the last recorded privilege check. This allows retrieving all privilege checks up to or after a specified moment in time.

When you retrieve a list of privilege checks, the results are returned in pages due to the possibly large number of privilege checks that can be returned. You can control the page size either by using the `AuthorizationPrivilegeChecksIterationSpec` data structure, or with the `config.vpxd.privCheck.pageSize` advanced setting.

When you retrieve a list of privilege checks, the results are returned in pages due to the possibly large number of privilege checks that can be returned. You can control the page size either through a `com.vmware.vcenter.authorization.PrivilegeChecks.IterationSpec` structure, which is passed as an argument to the API call, or with the `config.vpxd.privCheck.pageSize` advanced setting.

Filtering Privilege Checks

When you retrieve lists of privilege checks, you can refine your search by using the `AuthorizationPrivilegeChecksFilterSpecPrivilegeChecksTypes.FilterSpec`data structureclass. The following table lists the available filtering options.

Filtering Option	Description
`Objects`	IDs of the managed objects on which the privilege check was performed. If unset`null`, all objects match.
`OpIDs`	`OpIDs` of the requests for which the check was performed. If unset`null`, all `opIDs` match.

Filtering Option	Description
`Principals`	Principals for which the privilege check was performed. The unset`nullPrivilegeChecks.PrincipalPrincipal` value matches privilege checks for anonymous sessions. If unset`null`, all principals match.
`Privileges`	Privileges that were checked. If unset`null`, all privileges match.
`Sessions`	Sessions for which the check was performed. If unset`null`, all sessions match.

Using Advanced Settings

You can configure advanced settings by using the vSphere Client. Some of the advanced settings are not available in the API, but are required for the privilege checks recorder to function properly. For information about configuring advanced settings, see *vCenter Server Configuration*.

Advanced Setting Name	Description
`config.vpxd.privCheck.pageSize`	Specifies the default page size for privilege checks lists.
`config.vpxd.privCheck.bufferSize`	Specifies the count of privileges to be kept in memory. The default value is 0. If you do not change the default value, the privilege checks recorder does not record any data.
`config.vpxd.privCheck.cleanupInterval`	Specifies the interval on which privilege checks for unused sessions are cleaned up. The default value is 30 minutes.

vCenter Installation and Setup

You can use the API to perform operations related to stage 2 of the installation process. You can also perform backup, restore, and troubleshooting operations.

Install Stage 2

The vCenter API provides methods for performing stage 2 deployment operations on a newly installed vCenter instance.

The vCenter instance is deployed in two stages. With stage 1 of the deployment process, you deploy the OVA file, which is included in the installer. With stage 2 of the deployment process, you set up and start the services of the newly deployed vCenter instance.

To complete stage 1 of the deployment process, you can use the GUI installer or perform a CLI deployment. For details, see *vCenter Installation and Setup*. Alternatively, you can perform a deployment by using the VMware OVF Tool. For details, see *OVF Tool User's Guide*.

Setting Up a Newly Installed vCenter Instance

You can use the API to set up a newly deployed vCenter instance.

After stage 1 of the deployment process completes successfully, the vCenter instance enters in an `INITIALIZED` state. If the instance is not initialized, you cannot run stage 2 of the deployment process. You can get the state of the vCenter instance by using the `vcenter deployment` service. The vCenter instance can enter six states during the deployment process.

Figure 553: Install Stage 2 State Diagram

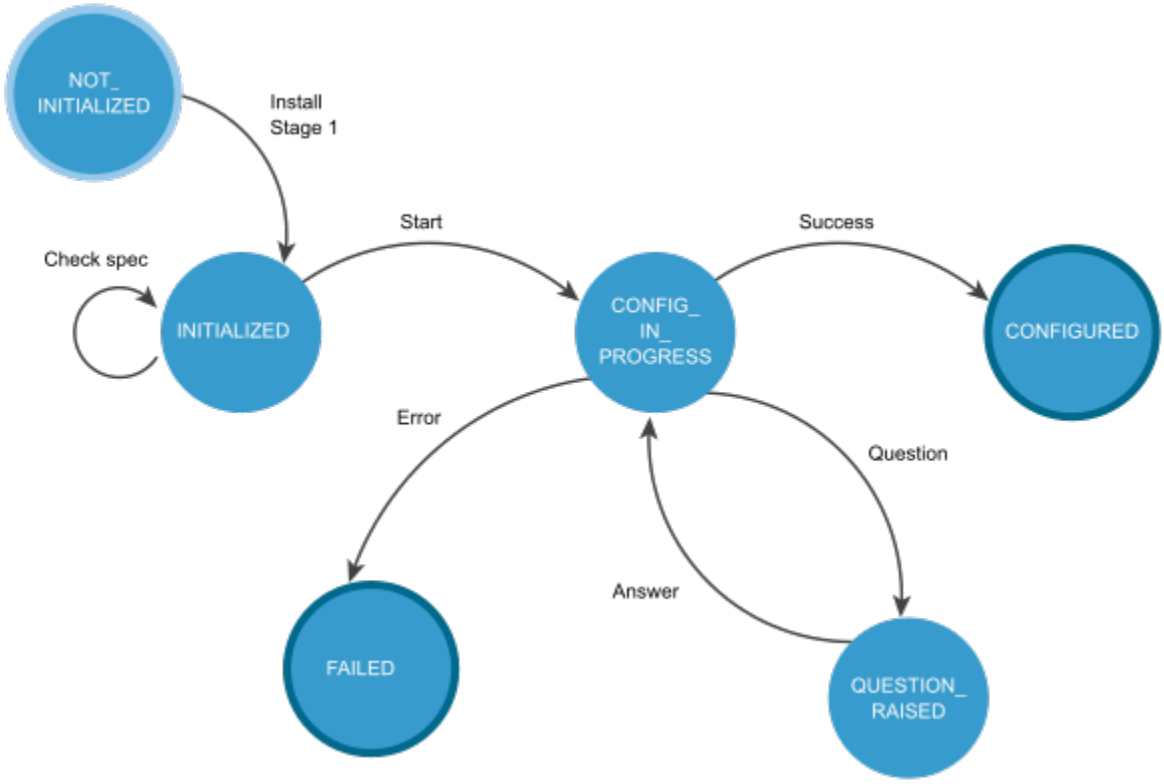


Table 1861: vCenter Instance States During Install Stage 2

State	Description
NOT_INITIALIZED	The install stage 1 phase is in progress, not started, or failed.
INITIALIZED	The vCenter instance is deployed and ready for setup.
CONFIG_IN_PROGRESS	The setup process is in progress.
QUESTION_RAISED	You must answer the question to continue the setup process. The vCenter instance stays in the QUESTION_RAISED state until it receives the correct answer.
FAILED	Errors occurred during the setup process. You can check the errors, warnings, and info data structures.
CONFIGURED	The vCenter instance is installed and configured successfully.

FAILED and CONFIGURED are final states.

Table 1862: User Operations lists operations that you can perform to set up your newly deployed vCenter instance.

Table 1862: User Operations

Operation	Description
Get deployment information	You can retrieve information about the current deployment status. This operation is useful both before initiating stage 2 of the deployment and for monitoring the progress of the setup process.
Validate the configuration document	You can optionally verify whether your install spec is valid before starting the setup process.
Configure the vCenter instance	You can initiate the setup process by providing an install spec that defines the values for the settings that you want to configure.
Get question	You can retrieve a question raised during the setup process.

Operation	Description
Answer question	You can provide an answer to the question raised during the setup process. The available answer values are `YES`, `NO`, `OK`, `CANCEL`, `ABORT`, `RETRY`, and `IGNORE`. The possible answer values depend on the type of the question. **Note:** Each question has a default answer value. If you set questions to receive automatic answers in the install spec and a question is raised during the setup process, the default answer value is automatically provided as the answer to the question.

For information about the HTTP requests that you can use to perform the user operations, see HTTP Requests for Install Stage 2.

Workflows for Install Stage 2

You can use the `vcenter deployment` API to run the install stage 2 process of your vCenter instance.

Figure 554: Install Workflow and Figure 555: Install Stage 2 Workflow show example install workflows.

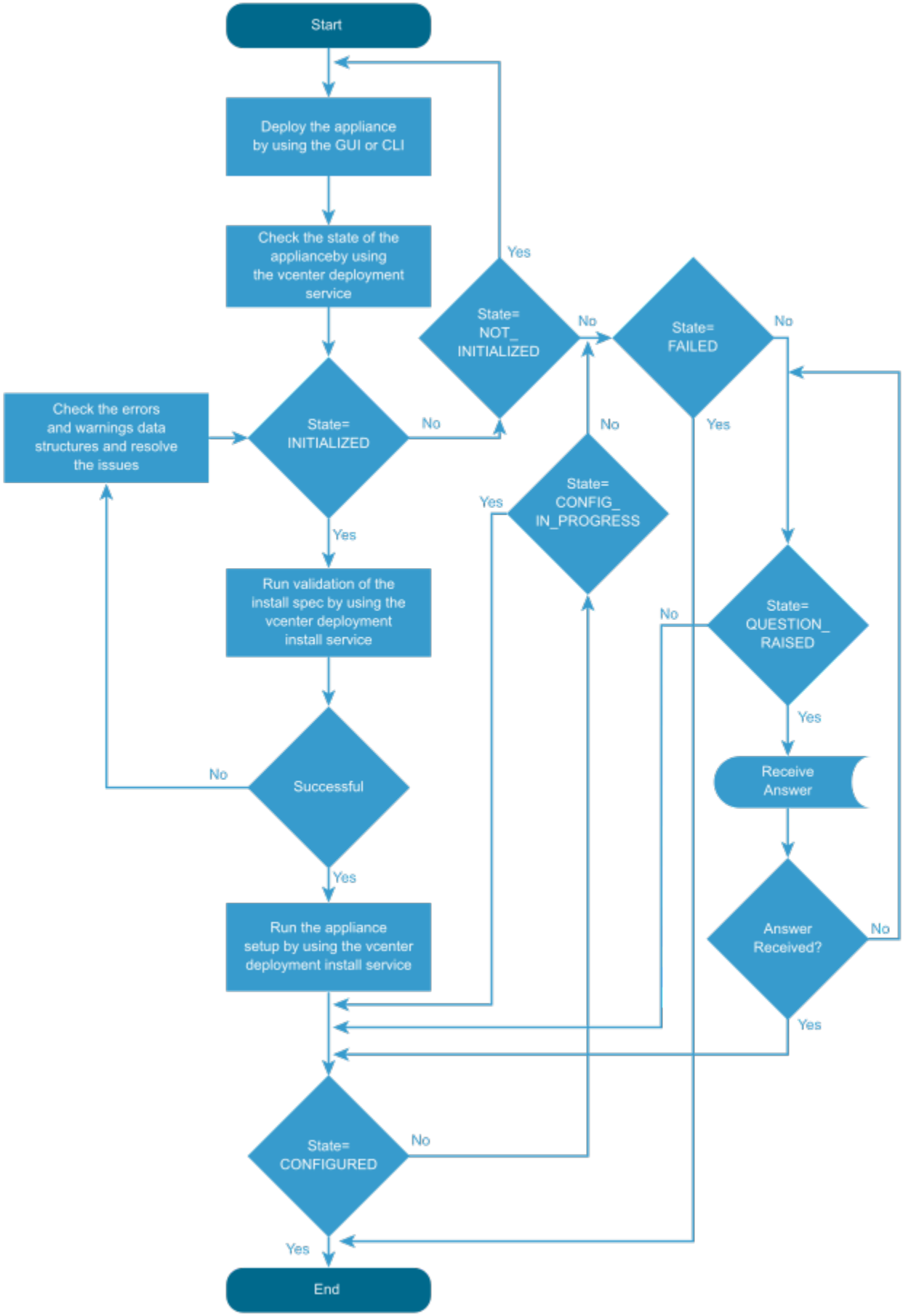
During stage 1, the vCenter instance is in a `NOT_INITIALIZED` state. After a successful deployment, the vCenter instance enters in an `INITIALIZED` state. If there are errors during stage 1, the vCenter instance stays in a `NOT_INITIALIZED` state and you must redeploy it.

You can check the state of the vCenter instance before, during, and after the setup process. You can run the install stage 2 process if the vCenter instance is initialized. You can check the setup configuration before you initiate stage 2 by running pre-checks. If errors or warnings appear during the validation of the install specification, you must remove the causes and correct the specification.

During the setup process, the regular vCenter instance state is `CONFIG_IN_PROGRESS`. The vCenter instance can also enter in a `FAILED` or `QUESTION_RASED` state. If a question appears during the setup, the vCenter instance enters in a `QUESTION_RAISED` state and stays in it until you provide an answer. You can set questions to receive automatic answers in the install spec and if a question is raised during the setup process, the default answer value is automatically provided as the answer to the question.

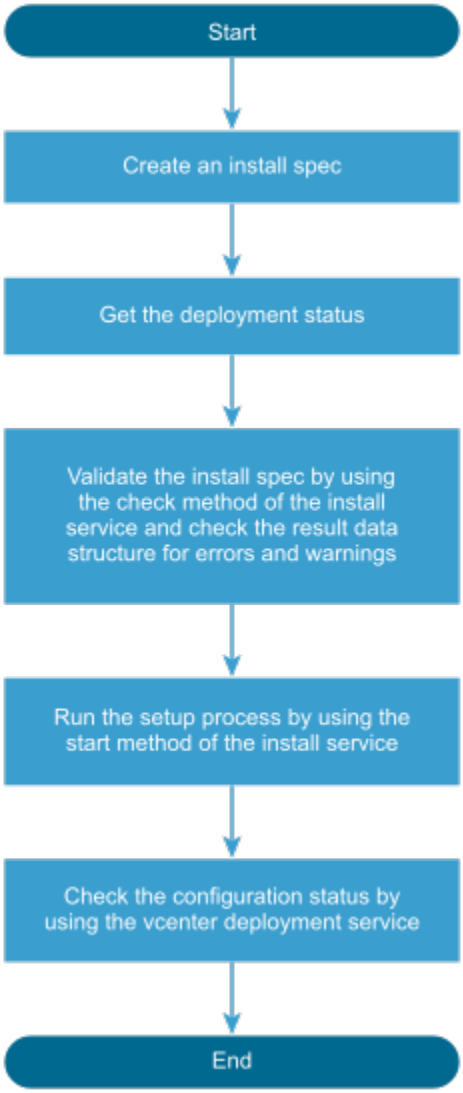
If errors occur during the setup process, the vCenter instance enters in a `FAILED` state and you must restart the setup after the causes are removed. If the setup is successful, the vCenter instance enters in a `CONFIGURED` state.

Figure 554: Install Workflow



For information about the states of the vCenter instance and available operations, see Setting Up a Newly Installed vCenter Instance.

Figure 555: Install Stage 2 Workflow



You can run the setup pre-checks and the install stage 2 process by creating and passing an `InstallSpec`. In `InstallSpec`, you define the setup configuration. See Figure 555: Install Stage 2 Workflow . You can run the setup in silent mode by setting the `InstallSpec.auto_answer` to `true`. The default value of `InstallSpec.auto_answer` is `false` and the setup is in interactive mode, in which you must provide answers to the raised questions.

For information about the classes, variables, and default values, see the *API reference* documentation.

HTTP Requests for Install Stage 2

You can use HTTP requests to set up a newly deployed vCenter instance.

HTTP Requests

After stage 1 of the deployment process completes successfully, you can perform setup by sending HTTP requests.

> **Note:**
> When you send the requests, you must authenticate with vCenter**root** credentials.

The following HTTP requests show the syntax that you can use to perform the available user operations.

- Get deployment information

```
GET https://<vcenter_ip_address_or_fqdn>:5480/rest/vcenter/deployment
```

- Validate the install spec

```
POST https://<vcenter_ip_address_or_fqdn>:5480/rest/vcenter/deployment/install?action=check
```

- Configure the vCenter instance

```
POST https://<vcenter_ip_address_or_fqdn>:5480/rest/vcenter/deployment/install?action=start
```

- Get question

```
GET https://<vcenter_ip_address_or_fqdn>:5480/rest/vcenter/deployment/question
```

- Answer question

```
POST https://<vcenter_ip_address_or_fqdn>:5480/rest/vcenter/deployment/question?action=answer
```

For information about the content and syntax of the HTTP request body, see the *API reference* documentation.

HTTP Status Codes and Errors

Table 1863: HTTP Status Codes and Errors lists the status codes that you can receive when you send HTTP requests.

Table 1863: HTTP Status Codes and Errors

HTTP Status Code	Description	Operations that Return the Status Code
200	The operation is successful.	All operations. You can check the returned data in the results data structure.
400	You cannot perform the operation because the vCenter instance is in the current state. For information about the states of the vCenter instance, see Setting Up a Newly Installed vCenter Instance.	• Validate the install spec • Start the setup • Get the install spec • Get the raised question • Answer the question • Get the state of the vCenter instance
401	You use an invalid user name or password, or authentication has failed.	• Validate the install spec • Start the setup • Get the install spec • Get the raised question • Answer the question • Get the state of the vCenter instance
404	The state of the vCenter instance cannot be determined .	Get the vCenter instance state.
500	There is a `vapi std` error. For information about the types of `vapi std` errors, see `vapi.std.errors` in the *API reference* documentation.	Get the raised question.

If errors occur during the setup process, you can check the `results` data structure, the API log file, and download the vCenter instance support bundle from `https://<vcenter_ip_address_or_fqdn>:443/appliance/support-bundle`.

Use HTTP Requests to Set Up a Newly Deployed vCenter Instance

You can send HTTP requests to complete stage 2 of the deployment process of a newly deployed vCenter instance.

- Verify that the newly deployed vCenter instance is reachable.
- Verify that you have the correct credentials for sending HTTP requests.

You set up a newly deployed vCenter instance by providing configuration settings in the body of the HTTP request.

1. Check whether the vCenter instance state is set to `INITIALIZED`.

```
GET https://<server>:5480/rest/vcenter/deployment
```

If the vCenter instance is in the correct state, you receive a message body that contains the following line and you can continue with the setup process.

```
...
    "state": "INITIALIZED",
...
```

2. Optional: Validate the configuration settings that you provide in the body of the HTTP request.

   ```
   POST https://<server>:5480/rest/vcenter/deployment/install?action=check
   ```

   The following example shows syntax that you can use in the body of the HTTP request.

   ```
   {
     "spec": {
       "vcsa_embedded": {
           "ceip_enabled": true,
           "standalone": {
           "sso_domain_name": "vsphere.local",
           "sso_admin_password": "<your_password>"
         }
       },
       "auto_answer": true
     }
   }
   ```

   If the input is valid, you receive the following response.

   ```
   {
       "status": "SUCCESS"
   }
   ```

3. Initiate the setup process by providing valid input in the body of the HTTP request.

   ```
   POST https://<server>:5480/rest/vcenter/deployment/install?action=start
   ```

4. Monitor the progress of the setup process.

   ```
   GET https://<server>:5480/rest/vcenter/deployment
   ```

   The following example shows part of the response body when the setup process is ongoing.

   ```
   {
       "progress": {
           "completed": 2,
           "message": {
               "id": "install.ciscommon.component.starting",
               "args": [
                   "VMware Authentication Framework"
               ],
               "default_message": "Starting VMware Authentication Framework..."
           },
           "total": 3
       },
       "status": "RUNNING",
       "state": "CONFIG_IN_PROGRESS",
       "operation": "INSTALL",
   ...
   ```

   The following example shows part of the response body when the setup process has completed successfully.

   ```
   {
       "subtask_order": [
           "rpminstall",
           "validate",
           "firstboot"
       ],
       "cancelable": false,
   ```

```
        "progress": {
            "completed": 3,
            "total": 3,
            "message": {
                "default_message": "Task has completed successfully.",
                "id": "com.vmware.vcenter.deploy.task.complete.success",
                "args": []
            }
        },
        "status": "SUCCEEDED",
        "description": {
            "default_message": "Install vCenter.",
            "id": "com.vmware.vcenter.deploy.task.description.op.install",
            "args": []
        },
        "state": "CONFIGURED",
    ...
```


You successfully configured the newly deployed vCenter instance.

File-Based Backup and Restore of vCenter

You can back up a vCenter instance and later restore the instance from the backup copy.

Backing up vCenter

The vCenterManagement API supports backing up key parts of the vCenter instance. This allows you to protect vCenter data and to minimize the time required to restore data center operations.

The backup process collects key files into a tar bundle and compresses the bundle to reduce network load. To minimize storage impact, the transmission is streamed without caching in the vCenter instance. To reduce total time required to complete the backup operation, the backup process handles the different components in parallel.

You have the option to encrypt the compressed file before transmission to the backup storage location. When you choose encryption, you must supply a password which can be used to decrypt the file during restoration.

The backup operation always includes the vCenter database and system configuration files, so that a restore operation has all the data needed to re-create an operational vCenter instance. Current Alarms are included as well. You also have the option to specify additional data sets, called parts. In this release, you can specify a data set that includes Statistics, Events, and Tasks.

Backup and Restore Protocols for vCenter

The vCenter backup and restore feature supports a number of plug-in communication protocols.

Choose one of these protocols as the backup location type when you invoke the operation.

- FTP
- FTPS
- SCP
- HTTP
- HTTPS
- NFS
- SMB

The value `PATH` for the location type field indicates a locally mounted volume.

> **Note:**
> If you specify the `SCP` protocol, you must specify an absolute path as the value of the location type field when you create the backup job.

Calculate the Size Needed To Store the Backup File

When you prepare to do a backup of avCenter instance, you can use the API to calculate the storage space needed for the backup file.

- Verify that you have avCenter instance running.
- Verify that you are familiar with authentication methods. See Authentication Mechanisms.

You can do this task when you are choosing a backup storage location or whenever your existing storage location might be approaching full capacity.

1. Authenticate to theVMware Cloud Foundation API endpoint and establish a session.
2. Request a list of backup parts available.
3. For each available backup part, request the size of the backup file.

   The backup process calculates the compressed size of each backup part.
4. Choose which parts to include in the backup, and sum their sizes.

   The backup storage server must have sufficient space to contain the chosen parts.

After you choose which backup parts you will store, and verify that the backup storage server has sufficient free space, you can launch a backup job. For information, see Back up a vCenter Instance by Using the API.

Python

This example shows how to use Python to collect the information you need to calculate the size needed to store a backup image of thevCenter instance.

```
from com.vmware.appliance.recovery.backup_client import Parts

# This example assumes you have previously created a session
# and stored the session ID in my_stub_config.

# Issue a request to list the backup image parts.
Parts_stub = Parts( my_stub_config )
parts = Parts_stub.list()

# Extract IDs of backup image parts.
sizes = {}
total = 0
for part in parts :
   size = Parts_stub.get( part.id )
   sizes[part.id] = size
   total += size

# Show the result.
print( 'Backup image parts:' )
for part_id in sizes.keys() :
   print( '  part {0} = {1}KB'.format( part_id, sizes[part_id] ) )
   print( 'Total size: {0}KB'.format( total ) )
```

Back up a vCenter Instance by Using the API

You can use the Management API to create a backup of the vCenter database and key components of the vCenter instance.

- Verify that the vCenter instance is in a ready state. All processes with start-up type automatic must be running.
- Verify that no other backup or restore jobs are running.
- Verify that the destination storage location is accessible to the backup process.
- Verify that the path to the destination directory exists, as far as the parent directory.
- If the destination directory does not exist, the backup process creates it. If the directory does exist, verify that it is empty.

- Verify that the destination storage device has sufficient space for the backup file. For information about how to calculate the space needed for the backup file, see Calculate the Size Needed To Store the Backup File.

This procedure explains the sequence of operations you use to create a backup file of the vCenter instance. You can do this as part of a regular maintenance schedule.

1. Authenticate to the VMware Cloud Foundation API endpoint and establish a session.
2. Create a backup request object Use the `RecoveryBackupJobBackupRequest` data structure to describe the backup operation.

   The requestdata structure specifies several attributesproperties, especially the backup location, the protocol used to communicate with the storage server, the necessary authorization, and which optional parts of the database you want to back up. The core inventory data and Alarms are always backed up, but you can choose whether or not to back up Statistics, Events, and Tasks. Collectively, this optional part of the backup is referred to as `seat`.
3. Issue a request to start the backup operation.
4. From the response, save the unique job identifier of the backup operation.
5. Monitor the progress of the job until it is complete.
6. Report job completion.

The backup file of the vCenter instance is created.

Python

This example specifies that the backup image should include Statistics, Events, and Tasks as well as the core inventory and alarm data. The value for `req.parts` indicates the optional data part for Statistics, Events, and Tasks.

This example uses the following global variables.

- *`my_storage_server`*
- *`my_backup_folder`*
- *`my_scp_user`*
- *`my_scp_password`*
- *`my_stub_config`*

When you back up the vCenter instance, you need two sets of authentication credentials. The API client needs to authenticate to the vCenter instance, and the backup service needs to authenticate to the backup storage server.

The example assumes that your API client has already authenticated the connection to the vCenter instance, and the security context is stored in *`my_stub_config`*.

In the backup request, you need to specify the folder that will contain the backup image. The folder name must be specified as a path name relative to the home directory of the user that authenticates with the storage server.

```
from com.vmware.appliance.recovery.backup_client import Job
import time

# This example assumes you have previously created a session
# and stored the session ID in my_stub_config.

# Create a backup request object.
req = Job.BackupRequest()
# Include optional backup part for Statistics, Events, and Tasks.
req.parts = ['seat']
req.location_type = Job.LocationType.SCP
req.comment = 'On-demand backup'
req.location = my_storage_server + ':/home/scpuser/' + my_backup_folder \
  + '/' + time.strftime('%Y-%m-%d-%H-%M-%S')
req.location_user = my_scp_user
req.location_password = my_scp_password
```

```
# Issue a request to start the backup operation.
backup_job = Job( my_stub_config )
job_status = backup_job.create( req )
job_id = job_status.id

# Monitor progress of the job until it is complete.
while (job_status.state == Job.BackupRestoreProcessState.INPROGRESS) :
    print( 'Backup job state: {} ({}%)'.format( job_status.state, \
                                                job_status.progress ) )
    time.sleep( 10 )
    job_status = backup_job.get( job_id )

# Report job completion.
print( 'Backup job completion status: {}'.format( job_status.state) )
```

Schedule a Backup Job

You can automate the backup process by creating a schedule that runs backup jobs at specific times.

- Verify that you can access the backup server and you have read and write permissions.
- Verify that you have established a connection to the vAPI services.

You can keep existing backups on the backup server. The retention policy defines the maximum number of backups that the server keeps. You can also specify whether the backup job should run once, or on a recurring basis. The recurrence policy defines the days of the week and specific times at which the backup job is scheduled to run.

1. Create a `Schedules` objectschedule by using the `POST https://<vCenter_Server_IP>/api/appliance/recovery/backup/schedules` HTTP request.
2. Specify the retention and recurrence information.
3. Create a schedule by specifying the backup location, user credentials to access the location, retention, and recurrence information.
4. Create an `UpdateSpec` Use the `RecoveryBackupSchedulesUpdateSpec` data structure and pass the updated information.
5. Get a backup schedule by passing a schedule ID.

Run the backup job by using the schedule.

Python

This example shows how you can schedule a backup job, update the schedule, and get a schedule. The example is based on the `backup_schedule.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...
   # Connect to vAPI services
   self.stub_config = vapiconnect.connect(
                               host=args.server,
                               user=args.username,
                               pwd=args.password,
                               skip_verification=args.skipverification)

   self.schedule_client = Schedules(self.stub_config)
...

   def create_schedule(self):
         retention_info = Schedules.RetentionInfo(self.max_count)
         recurrence_info = Schedules.RecurrenceInfo(
                                    days=self.days,
                                    hour=self.hour,
                                    minute=self.minute)
```

```
            create_spec = Schedules.CreateSpec(
                                        location=self.location,
                                        location_user=self.location_user,
                                        location_password=self.location_password,
                                        recurrence_info=recurrence_info,
                                        retention_info=retention_info)

            self.schedule_client.create(self._schedule_id, create_spec)

        def update_schedule(self):
            retention_info = Schedules.RetentionInfo(self.max_count)
            recurrence_info = Schedules.RecurrenceInfo(
                                        days=self.days,
                                        hour=self.hour,
                                        minute=self.minute)
            update_spec = Schedules.UpdateSpec(
                                        location=self.location,
                                        location_user=self.location_user,
                                        location_password=self.location_password,
                                        recurrence_info=recurrence_info,
                                        retention_info=retention_info)

            self.schedule_client.update(self._schedule_id, update_spec)

        def get_schedule(self):
            self.schedule_client = Schedules(self.stub_config)
            schedule_spec = self.schedule_client.get(self._schedule_id)

            recurrence_info = schedule_spec.recurrence_info
            retention_info = schedule_spec.retention_info

            table = []
            data = [self._schedule_id,
                    "{}:{}".format(recurrence_info.hour, recurrence_info.minute),
                    " ".join(recurrence_info.days),
                    retention_info.max_count]
            table.append(data)
            headers = ["Schedule ID", "Time", "Days", "Retention"]
            print(tabulate(table, headers))

...
```

Restoring vCenter

The vCenterManagement API supports restoring a vCenter instance from a backup copy. The API simplifies the process by unifying the handling of various components of vCenter in a single operation.

The process of restoring a vCenter instance from a backup has two phases.

1. Deploy a new vCenter instance. OVF deployment is described in the *VMware Cloud Foundation SDKs Programming Guide*.
2. Invoke the `restore` operation from the Management API to apply configuration settings and load the vCenter database from the backup file.

> **Note:**
> You cannot specify optional parts for the restore operation. The restore operation includes all optional parts, such as Events and Tasks, that were specified at the time when the backup file was created.

Authentication When Restoring a vCenter Instance

During the process of restoring a vCenter instance from a backup image, you cannot use token-based authentication. You must use basic authentication until the vCenter instance is fully configured.

When you restore your vCenter instance from a backup file, it begins in an unconfigured state. During this time, you must use basic authentication to access the Management API. When you use basic authentication, do not use the VMware Cloud Foundation API endpoint. Instead, you must connect your client to port 5480 of the vCenter instance.

When you use basic authentication, you must pass user name and password credentials with each method invocationrequest. Use credentials that are known to the guest operating system of the vCenter instance.

Availability of Services While Restoring a vCenter Instance

During the process of restoring the vCenter backup file, services in the vCenter instance must restart. While they are restarting, your API client receives an error message.

You can write your client to trap the error, but you have no way to know when the vCenter services are running again. To determine when the restore process is complete, you must retry the API connection until it succeeds, then request the status of the job.

Restore a vCenter Instance by Using the API

You can use the Management API of to restore a vCenter instance from a backup file containing the vCenter database and key components of the vCenter instance.

- Verify that the backed up vCenter instance is powered off.
- A new vCenter instance must be deployed in an unconfigured state, except that it must have a fully qualified domain name or IP address that matches the old one.
- Verify that the new vCenter instance has the same build number as the one in the backup file.
- Verify that the new vCenter instance has a size equal to or greater than the old one. If the old vCenter instance was customized to exceed the largest template size, the new one must be customized to the same size.
- Verify that no other backup or restore jobs are running.
- Verify that the destination storage location is accessible to the vCenter restore process.

1. Create a restore request object Use the `RecoveryRestoreJobRestoreRequest` data structure to describe the restore operation.
2. Issue a request to start the restore operation.
3. Monitor the progress of the job until it is complete.
4. Report job completion.

After the vCenter instance is fully configured by the restore operation, you can resume using the VMware Cloud Foundation API endpoint for subsequent operations.

Python

This example shows how to use Python to restore a vCenter instance. This operation is the second phase of restoring the vCenter instance from a backup image.

This example uses the following global variables.

- *my_vcsa_hostname*
- *my_vcsa_username*
- *my_vcsa_password*
- *my_backup_name*
- *my_storage_server*
- *my_scp_user*
- *my_scp_password*
- *my_backup_folder*

When you restore the vCenter instance from a backup image, you need two sets of authentication credentials. The API client needs to authenticate to the vCenter instance, and the vCenter backup service needs to authenticate to the backup storage server.

The example uses local user name and password authentication for the connection to the vCenter instance because the VMware Cloud Foundation API endpoint is not yet running when you restore the vCenter instance. The client must connect to port `5480` for this operation.

In the restore request, you need to specify the folder that contains the backup image. The folder name is the same name that was specified in the backup request. It must be specified as a path name relative to the home directory of the user that authenticates with the storage server.

This example assumes the backup image is not encrypted.

```
import requests
 from vmware.vapi.lib.connect import get_requests_connector
 from vmware.vapi.security.user_password import create_user_password_security_context
 from vmware.vapi.stdlib.client.factories import StubConfigurationFactory
 from com.vmware.appliance.recovery.restore_client import (Job)
 import time

 # Create a session object in the client.
 session = requests.Session()

 # For development environment only, suppress server certificate checking.
 session.verify = False
 from requests.packages.urllib3 import disable_warnings
 from requests.packages.urllib3.exceptions import InsecureRequestWarning
 disable_warnings(InsecureRequestWarning)

 # Create a connection to port 5480.
 local_url = 'https://%s:5480/api' % my_vcsa_hostname
 connector = get_requests_connector(session=session, url=local_url)

 # Add username/password security context to the connector.
 basic_context = create_user_password_security_context(my_vcsa_username,
 my_vcsa_password)
 connector.set_security_context(basic_context)

 # Create a stub configuration by using the username-password security context.
 local_stub_config = StubConfigurationFactory.new_std_configuration(connector)

 # Create a restore request object.
 req = Job.RestoreRequest()
 req.location_type = Job.LocationType.SCP
 req.location = my_storage_server + ':/home/scpuser/' + my_backup_folder + '/' +
 my_backup_name
 req.location_user = my_scp_user
 req.location_password = my_scp_password

 # Issue a request to start the restore operation.
 restore_job = Job( local_stub_config )
 job_status = restore_job.create( req )

 # Monitor progress of the job until it is complete.
 while (job_status.state == Job.BackupRestoreProcessState.INPROGRESS) :
     print( 'Restore job state: {} ({}%)'.format( job_status.state,
 job_status.progress ) )
     time.sleep( 10 )
     job_status = restore_job.get()

 # Report job completion.
 print( 'Restore job completion status: {}'.format( job_status.state) )
```

Reconcile a vCenter Instance with Nodes in Embedded Linked Mode
You can run the reconciliation process after you successfully restored your vCenter instance. By using the API or HTTP requests, you can reconcile vCenter nodes that work in an embedded linked mode and are connected in a ring or daisy-chain.

- Verify that you successfully restored your node from an image.
- Verify that the replication partners are available.
- Verify that you restored your nodes in the correct order, if you use a daisy-chain topology.
- Verify that you have administrator's credentials to your Single Sign-On domain.
- Verify that there is no running or failed reconciliation job.

Reconciliation is a post-restore process that checks whether the vCenter partners in embedded linked mode are available, synchronizes the vCenter data and services with the partners, and runs the vCenter services. The processes of restore and reconciliation depend on the topology and if there are changes in the topology between the backup and restore, you cannot restore the embedded linked mode. If the replication partners are not available and you try to restore the first node, you must ignore the warnings. In this case, any changes that are made in the topology or infrastructure after the backup will be lost. If you restore a node different from the first one, you must add it to the domain of the first node. If you use a daisy-chain topology, you must first restore the first node, and after that to restore the second, link it to the first one, and apply the same to the following nodes.

You can use the reconciliation API after a file-based and an image-based restore. After an image-based restore, you can run the reconciliation process by using the API or UI. After a file-based restore, you can monitor the reconciliation process by using the `GET https://<vcenter_ip_address_or_fqdn>/api/appliance/recovery/reconciliation/job` HTTP request. For information about how to restore a vCenter instance from an image or a file by using the UI, see the *vCenter Installation and Setup* documentation.

1. Create a `Job.CreateSpec` object Use the `RecoveryReconciliationJobCreateSpec` data structure, specify user name and password of Single Sign-On administrator, and set the `ignore_warnings` field to `true`.

   The default value of `ignore_warnings` is `false`. If you do not set `ignore_warnings` to `true`, the reconciliation fails due to the validation warnings.

2. Run a reconciliation job by using the `create(Job.CreateSpec)` method`POST https://<vcenter_ip_address_or_fqdn>/api/appliance/recovery/reconciliation/job` HTTP request.

   You can check the result of the operation by reading the `Job.Info` object`RecoveryReconciliationJobInfo` data structure. `Job.InfoRecoveryReconciliationJobInfo` contains information about the job such as description, status, progress, error, start and end time.

3. Get the status of the job by calling the `get()` methodusing the `GET https://<vcenter_ip_address_or_fqdn>/api/appliance/recovery/reconciliation/job` HTTP request.

   The possible states are `NONE`, `RUNNING`, `FAILED`, and `SUCCEEDED`.

Troubleshooting for vCenter Installation or Deployment
You can use the API to perform troubleshooting operations related to the installation and deployment of vCenter.

Managing System Logs
You can automate the forwarding of vCenter system log messages to remote logging servers by using the vCenter Management API.

You can configure the syslog forwarding by using the API or user interface. For information about how to manage the syslog by using the user interface, see the *vSphere Monitoring and Performance* documentation.

Configuring Syslog Forwarding
You can use the vCenterManagement API or HTTP requests to configure the forwarding of vCenter syslog messages and test the connection between the vCenter instance and remote servers.

The following table lists operations that you can perform to manage the forwarding of syslog messages to remote logging servers.

Table 1864: User Operations

Operation	Description
Get forwarding configuration	You can retrieve information about the log forwarding configuration. See HTTP Requests for Configuring Syslog Forwarding.
Test forwarding configuration	You can validate the current log forwarding configuration. Optionally, you can send a test diagnostic log message from the vCenter instance to all configured logging servers to allow manual end-to-end validation. See HTTP Requests for Configuring Syslog Forwarding.
Set forwarding configuration	You can change the log forwarding configuration. See HTTP Requests for Configuring Syslog Forwarding.

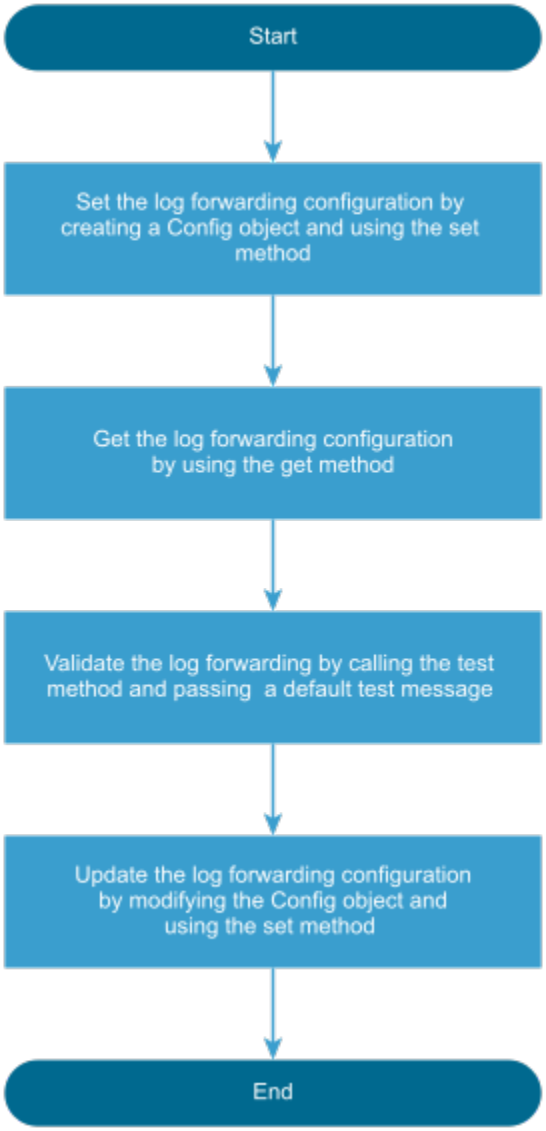
The forwarding configuration includes the IP or FQDN of the remote server, the remote port for receiving syslog information, and the communication protocol. The remote server must be a server with running `rsyslog`, for example, another vCenter instance. The API supports the TCP, UDP, TLS, and RELP protocols. For information about the supported TLS versions, see KB article 2147469. By creating a `Forwarding.Config` object, you specify the connection with a remote server. For information about the `Forwarding` class and its methods, see the *API Reference* documentation and *Example Configuration Workflow*.

You can use several remote servers by creating a list with `Forwarding.Config` objects and passing it to the `set` method. The maximum number of remote servers is three. You can validate the forwarding configuration by using the `test` method. The returned `Forwarding.ConnectionStatus` object shows the status of the connection between the vCenter instance and a remote server. The `State` enumeration shows whether the vCenter instance can reach the remote server. `State` can be `UP`, `DOWN`, or `UNKNOWN`. If the state is `DOWN` or `UNKNOWN`, the vCenter instance cannot access the remote server and you must check the remote server and its settings such as network ports, firewall, supported protocols, and syslog configuration.

Note:

If you use UDP, the connection status is always `UNKNOWN`.

Figure 556: Example Configuration Workflow



REST

By using the API or HTTP requests, you can set and get the forwarding configuration, check the connection with the remote server or servers, and exchange test messages with them.

The following HTTP requests show the syntax that you can use to perform the available user operations.

> **Note:**
> When you send the requests, you must authenticate with vCenter**root** credentials.

```
GET https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/logging/forwarding
POST https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/logging/forwarding?action=test
PUT https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/logging/forwarding
```

For information about the body of each HTTP request, see the *REST API Reference* documentation.

Table 1865: HTTP Status Codes

HTTP Status Code	Description	Operations That Return the Status Code
200	The operation is successful.	All available operations. You can check the returned data in the results data structure.
400	You use an invalid argument. For example, a protocol that it is not supported, invalid port number, or the number of configurations is greater than 3.	Set forwarding configuration.
401	You use invalid user name or password, or authentication is failed.	All available operations.
500	There is a `vapi std` error. For information about the types of `vapi std` errors, see the `vapi.std.errors` API in the *API reference* documentation.	Set forwarding configuration.

HTTP Requests for Configuring Syslog Forwarding

By using the API or HTTP requests, you can set and get the forwarding configuration, check the connection with the remote server or servers, and exchange test messages with them.

The following HTTP requests show the syntax that you can use to perform the available user operations.

Note:
When you send the requests, you must authenticate with vCenter**root** credentials.

```
GET https://<server>:5480/rest/appliance/logging/forwarding
POST https://<server>:5480/rest/appliance/logging/forwarding?action=test
PUT https://<server>:5480/rest/appliance/logging/forwarding
```

For information about the body of each HTTP request, see the *REST API Reference* documentation.

Table 1866: HTTP Status Codes

HTTP Status Code	Description	Operations That Return the Status Code
200	The operation is successful.	All available operations. You can check the returned data in the results data structure.
400	You use an invalid argument. For example, a protocol that it is not supported, invalid port number, or the number of configurations is greater than 3.	Set forwarding configuration.
401	You use invalid user name or password, or authentication is failed.	All available operations.
500	There is a `vapi std` error. For information about the types of `vapi std` errors, see the `vapi.std.errors` API in the *API reference* documentation.	Set forwarding configuration.

Configuring Syslog Forwarding

You can use the vCenterManagement API or HTTP requests to configure the forwarding of vCenter syslog messages and test the connection between the vCenter instance and remote servers.

The table below lists operations that you can perform to manage the forwarding of syslog messages to remote logging servers.

Table 1867: User Operations

Operation	Description
Get forwarding configuration	You retrieve information about the log forwarding configuration. For more information refer to the diagrams below.
Test forwarding configuration	You can validate the current log forwarding configuration. Optionally, you can send a test diagnostic log message from the vCenter instance to all configured logging servers to allow manual end-to-end validation. For more information refer to the diagrams below.
Set forwarding configuration	You can change the log forwarding configuration. For more information refer to the diagrams below.

The forwarding configuration includes the IP or FQDN of the remote server, the remote port for receiving syslog information, and the communication protocol. The remote server must be a server with running `rsyslog`, for example, another vCenter instance. The API supports the TCP, UDP, TLS, and RELP protocols. For information about the supported TLS versions, see KB article 2147469. By creating a `Forwarding.Config` object, you specify the connection with a remote server. For information about the `Forwarding` class and its methods, see the *API Reference* documentation and diagrams below.

You can use several remote servers by creating a list with `Forwarding.Config` objects and passing it to the `set` method. The maximum number of remote servers is three. You can validate the forwarding configuration by using the `test` method. The returned `Forwarding.ConnectionStatus` object shows the status of the connection between the vCenter instance and a remote server. The `State` enumeration shows whether the vCenter instance can reach the remote server. `State` can be `UP`, `DOWN`, or `UNKNOWN`. If the state is `DOWN` or `UNKNOWN`, the vCenter instance cannot access the remote server and you must check the remote server and its settings such as network ports, firewall, supported protocols, and syslog configuration.

Note:
If you use UDP, the connection status is always `UNKNOWN`.

Figure 557: Forwarding Class Diagram for Python

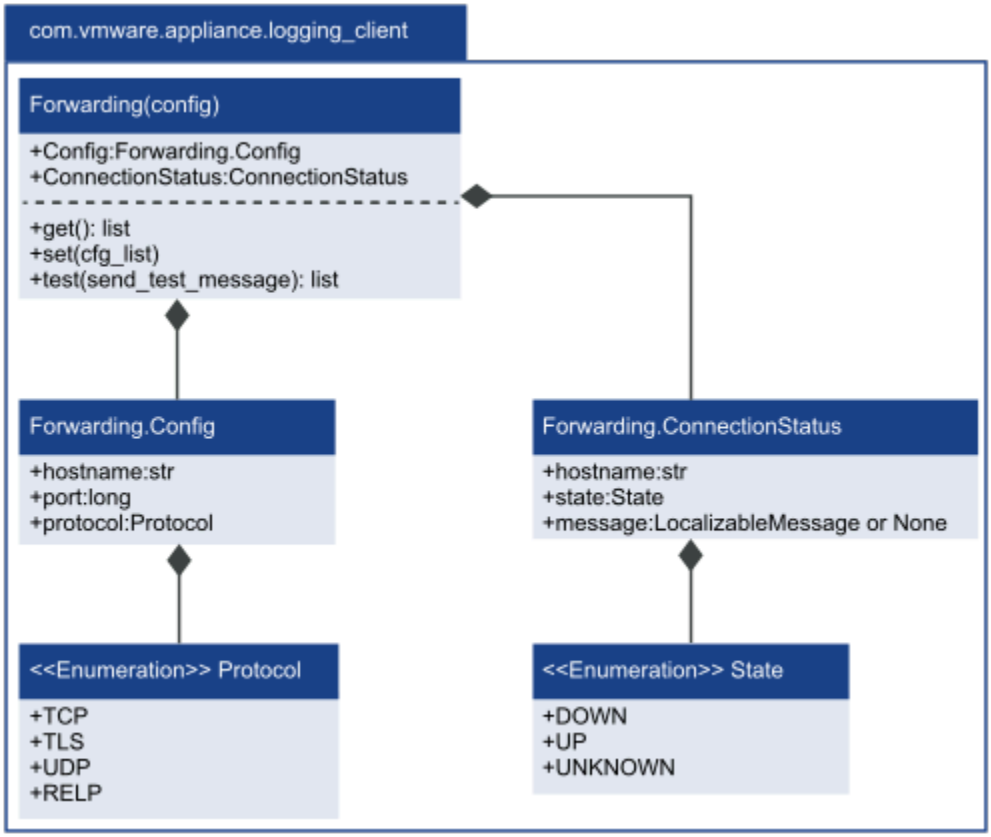
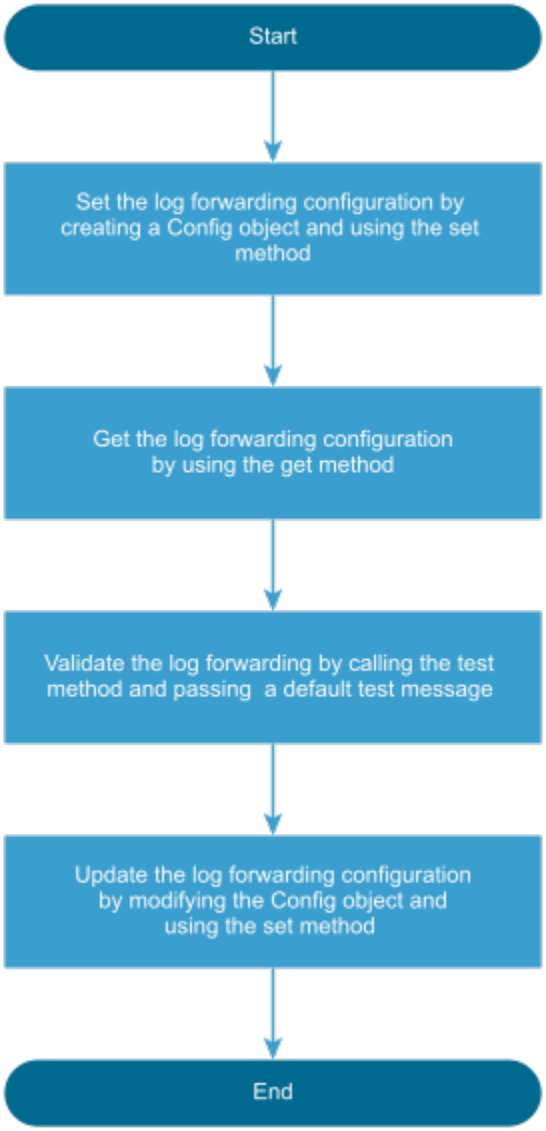


Figure 558: Example Configuration Workflow



Python

This example shows how you can configure and test the forwarding of a vCenter syslog by using the API. The example is based on the `log_forwarding.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

        self.log_forwarding_client = Forwarding(self.stub_config)

...

    def set_log_forwarding(self):
        log_forwarding_config = [Forwarding.Config(hostname=self.loghost,
                                                   port=self.port,
                                                   protocol=self.protocol)]
        self.log_forwarding_client.set(log_forwarding_config)

    def get_log_forwarding(self):
```

```
        configs = self.log_forwarding_client.get()

        print("\nLog forwarding configurations:")
        table = [[cfg.hostname, cfg.port, cfg.protocol] for cfg in configs]
        headers = ["Loghost", "Port", "Protocol"]
        print(tabulate(table, headers))

    def test_log_forwarding(self):
        test_response = self.log_forwarding_client.test(True)

        print("\nLog forwarding test response:")
        table = [[resp.hostname,
                  resp.state,
                  resp.message.default_message if resp.message else None]
                 for resp in test_response]
        headers = ["Loghost", "State", "Message"]
        print(tabulate(table, headers))

    def update_log_forwarding(self):
        # Read log forwarding configuration
        log_forwarding_config = self.log_forwarding_client.get()

        # Delete the newly added configuration
        log_forwarding_config = list(filter(
                                    lambda cfg: cfg.hostname != self.loghost,
                                    log_forwarding_config))

        # Apply the modified log forwarding configuration
        self.log_forwarding_client.set(log_forwarding_config)

...
```

Managing Support Bundles

You can retrieve information about support bundles and create support bundles.

The following table lists the operations that you can perform to manage support bundles.

Table 1868: User Operations

Operation	Description
Get support bundle components	You can retrieve a list of components and manifests included in the support bundle. Manifests specify the files that must be collected for a component as part of the support bundle.
List support bundles	You can retrieve a list of generated support bundles. The list contains details about each support bundle.
Create a support bundle	You can generate a support bundle. Optionally, you can specify the support bundle components and the partition where you want to save the support bundle.

You can run support bundle management operations by using the VMware Cloud Foundation SDK or sending an HTTP request. For information about the HTTP requests that you can use to perform the user operations, see HTTP Requests for Support Bundle Management Operations.You can use the `Components` interface to get support bundle components and the `SupportBundleFactory` class from the `com.vmware.appliance.support_bundle` package to list and create support bundles. For more information, see the *Java API Reference* documentation.

REST

You can use HTTP requests to perform support bundle management operations.

The following HTTP requests show the syntax that you can use to perform the available user operations.

- Get support bundle components

```
GET https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle/
components
```

- List support bundles

  ```
  GET https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle
  ```

- Create a support bundle

  ```
  POST https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle
  ```

For information about the content and syntax of the HTTP request body for each operation, see the *API reference* documentation.

The following examples provide sample input and output values for the support bundle management operations.

- *Example: Get Support Bundle Components*

  This operation does not require input.

  ```
  GET https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle/
  components
  ```

Type	Example

Output	{
	"Group1" : List of manifests under this group,
	"Group2": ...
	}

- *Example: List Support Bundles*

  This operation does not require input, but supports optional query parameters.

  ```
  GET https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle
  ```

Type	Example

Input	"IterationSpec":{
	"size" : 10
	}
	FilterSpec:{
	"creationStatus" : "SUCCEEDED",
	"available" : True/False
	}

Output	{
	"Description": "Reason for generation",
	"Generation time": 2 Mar 2021,
	"URL" : "Download URL",
	"Downloadable" : True/False
	"Expiration time": "URL expiration time"
	}

- *Example: Create a Support Bundle*

  This operation requires description input and supports optional body parameters. As output, you receive a task ID, which you can use to track the progress of the task. When the operation is completed, you receive the download URL in the task response.

```
POST https://<vcenter_ip_address_or_fqdn>:443/api/appliance/support-bundle
```

Type	Example

Input	{
	"Description" : "Reason",
	"Inclusions" : List of components to be included in the bundle
	<Optional> path: partition to be used.
	}

Output	Task ID

HTTP Requests for Support Bundle Management Operations

You can use HTTP requests to perform support bundle management operations.

The following HTTP requests show the syntax that you can use to perform the available user operations.

- Get support bundle components

  ```
  GET https://<server>:443/api/appliance/support-bundle/components
  ```

- List support bundles

  ```
  GET https://<server>:443/api/appliance/support-bundle
  ```

- Create a support bundle

  ```
  POST https://<server>:443/api/appliance/support-bundle
  ```

For information about the content and syntax of the HTTP request body for each operation, see the *API reference* documentation.

Example Input and Output for Support Bundle Management Operations

The following examples provide sample input and output values for the support bundle management operations.

Get Support Bundle Components

This operation does not require input.

```
GET https://<server>:443/api/appliance/support-bundle/components
```

Type	Example

Output	{
	"Group1" : List of manifests under this group,
	"Group2": ...
	}

List Support Bundles

This operation does not require input, but supports optional query parameters.

```
GET https://<server>:443/api/appliance/support-bundle
```

Type	Example

Input	"IterationSpec":{
	"size" : 10
	}
	FilterSpec:{
	"creationStatus" : "SUCCEEDED",
	"available" : True/False
	}

Output	{
	"Description": "Reason for generation",
	"Generation time": 2 Mar 2021,
	"URL" : "Download URL",
	"Downloadable" : True/False
	"Expiration time": "URL expiration time"
	}

Create a Support Bundle

This operation requires description input and supports optional body parameters. As output, you receive a task ID, which you can use to track the progress of the task. When the operation is completed, you receive the download URL in the task response.

```
POST https://<server>:443/api/appliance/support-bundle
```

Type	Example

Input	{
	"Description" : "Reason",
	"Inclusions" : List of components to be
	included in the bundle
	<Optional> path: partition to be used.
	}

Output	Task ID

vCenter Upgrade

You can use the API to perform operations related to stage 2 of the upgrade process. You can also perform historical data transfer operations.

Upgrade Stage 2

You can upgrade your vCenter instance by using the API, CLI, or GUI.

For information about how to upgrade the vCenter instance by using CLI and GUI, see the *vCenter Upgrade* documentation.

Upgrading a vCenter Instance

You can use the API during stage 2 of the vCenter instance upgrade.

By using the API, you can upgrade your vCenter instance. For information about the upgrade process, its stages, supported configurations, upgrade paths, prerequisites for upgrading, and the sequence for upgrading a vSphere environment, see the *vCenter Upgrade* documentation.

After you deploy the vCenter instance on stage 1 by using the GUI or CLI, the instance enters in an `INITIALIZED` state. If the vCenter instance is not initialized, you cannot run stage 2 of the upgrade process. You can get the state of the vCenter instance by using the `vcenter deployment` service. There are six states during the upgrade process.

Figure 559: Upgrade Stage 2 State Diagram

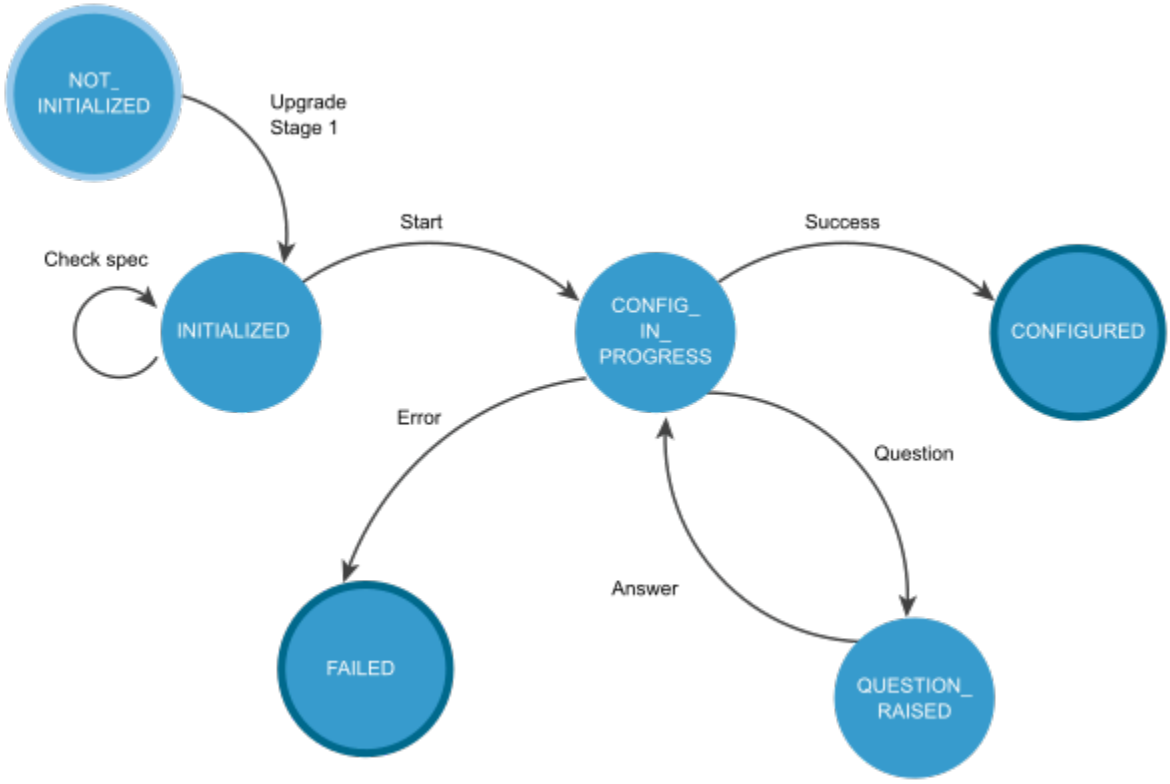


Table 1869: vCenter Instance States During Upgrade Stage 2

State	Description
`NOT_INITIALIZED`	The upgrade stage 1 phase is in progress, not started, or failed.
`INITIALIZED`	The vCenter instance is deployed and ready for upgrading.
`CONFIG_IN_PROGRESS`	The upgrade process is in progress.
`QUESTION_RAISED`	You must answer the question to continue the upgrade process. The vCenter instance stays in the `QUESTION_RAISED` state until it receives the correct answer.
`FAILED`	Errors appeared during the upgrade process. You can check the errors, warnings, and info data structures.
`CONFIGURED`	The vCenter instance is upgraded or configured successfully.

`FAILED` and `CONFIGURED` are final states.

You can roll back a vCenter instance upgrade by using the GUI. For information about how to roll back a vCenter instance, see the *vCenter Upgrade* documentation.

After the upgrade, you can check the vCenter instance type, domain registration, services, their state and health status by using the API. For information about how to verify whether the upgrade of your vCenter instance is successful, see the *vCenter Upgrade* documentation.

Table 1870: User Operations shows operations that you can perform to upgrade your vCenter instance.

Table 1870: User Operations

Operation	Description
Operations for upgrading	
Get the state of the vCenter instance	You can get the state of the vCenterinstance before, during and after the upgrade process. For information about the HTTP requests for diagnostic and deferring the transfer of historical data, see HTTP Requests for Upgrade Stage 2.
Check	You can validate the upgrade spec before you run the upgrade process. If the vCenter instance is in the `INITIALIZED` state, you can run the validation. The operation runs upgrade pre-checks. You can check the errors, warnings, and status data structures before you run the upgrade process. For information about the HTTP requests for upgrading, see HTTP Requests for Upgrade Stage 2.
Start	If the vCenter instance is in an `INITIALIZED` state, you can run the upgrade process. If errors appear during the upgrade, you can download the vCenter support bundle. For information about the HTTP requests for upgrading, see HTTP Requests for Upgrade Stage 2.
Get	If the vCenter instance is in a `CONFIGURED` state, you can get the `spec` that is used for upgrading. For information about the HTTP requests for upgrading, see HTTP Requests for Upgrade Stage 2.
Operations for getting and answering a question	
Get	You can get the raised question. If you set the `Upgrade.auto_answer` to `true`, the upgrade process will be in a `silent` mode and the vCenterinstance does not generate questions. It uses default answers and you should not provide an answer. For information about the HTTP requests for getting and answering a question, see HTTP Requests for Upgrade Stage 2.
Answer	You can provide an answer to the raised question. The available answers for the upgrading are `OK`, `CANCEL`, `YES`, `NO`, `ABORT`, `RETRY`, and `IGNORE`. The answer depends on the type of the question. If you set the `Upgrade.auto_answer` to `true`, the upgrade process will be in a `silent` mode and the vCenterinstance does not generate questions. It uses default answers and you should not provide an answer. For information about the HTTP requests for getting and answering a question, see HTTP Requests for Upgrade Stage 2.

For information about the available operations in the API, see the `vcenter deployment`, `vcenter deployment upgrade`, `vcenter services`, and `vcenter system-config deployment type` services in the *API reference* documentation.

You can upgrade your vCenter instance by using HTTP requests. For information about the HTTP requests, see HTTP Requests for Upgrade Stage 2.

Workflows for Upgrade Stage 2

You can use the `vcenter deployment` API to run the upgrade stage 2 process of your vCenter instance.

Figure 560: Upgrade Workflow and Figure 561: Upgrade Stage 2 Workflow show example upgrade workflows.

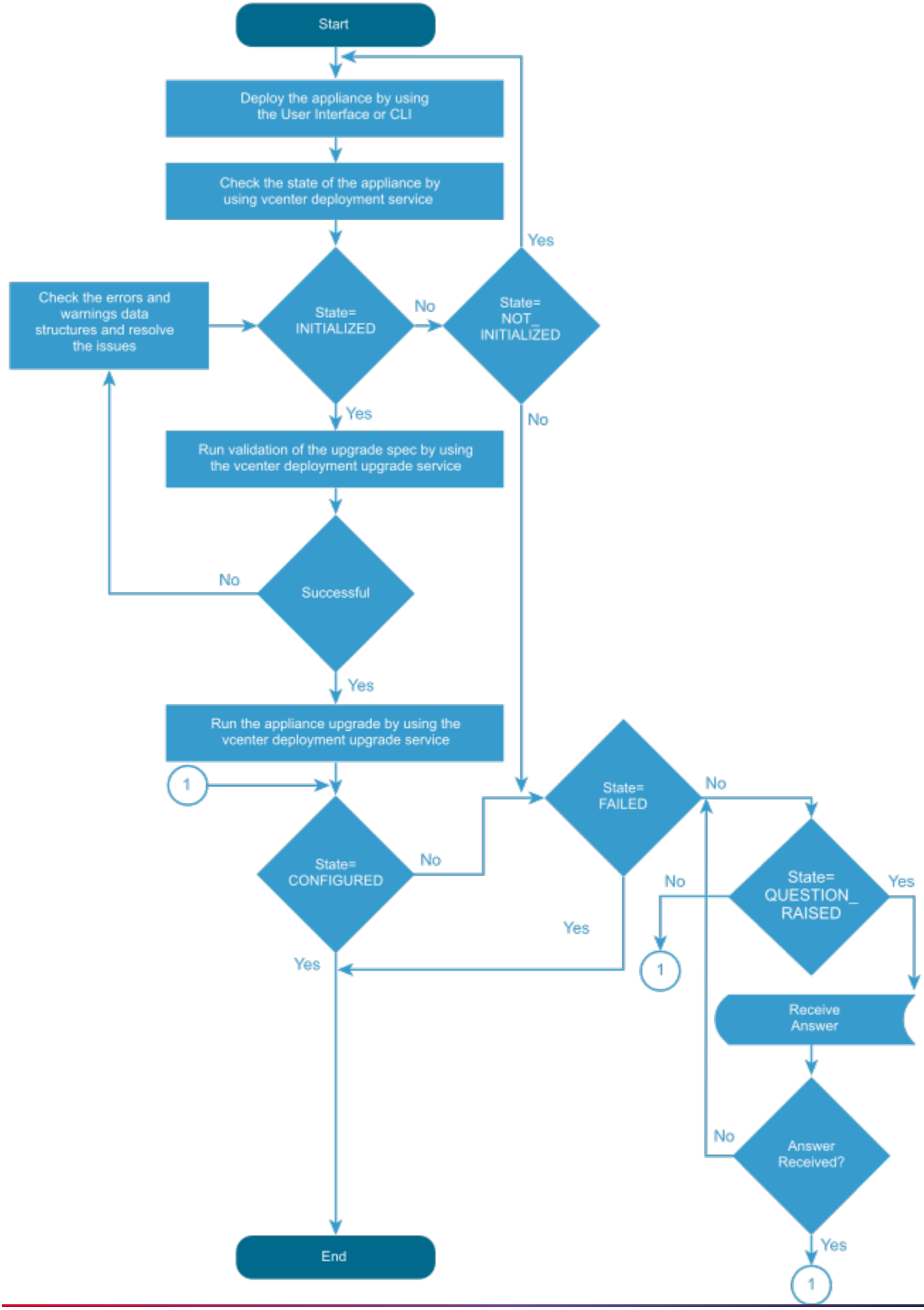
During stage 1, the vCenter instance is in a `NOT_INITIALIZED` state. After a successful deployment, the vCenter instance enters in an `INITIALIZED` state. If there are errors during stage 1, the vCenter instance stays in a `NOT_INITIALIZED` state and you must redeploy it.

You can check the state of the vCenter instance before, during, and after the upgrade process. You can run the upgrade stage 2 process if the vCenter instance is initialized. You can check the upgrade configuration before you run the upgrade by running pre-checks. If errors or warnings appear during the validation of the upgrade specification, you must remove the causes and correct the specification.

During the upgrade process, the vCenter instance can enter in a `FAILED` or `QUESTION_RASED` state. If a question appears during the upgrade, the vCenter instance enters in a `QUESTION_RAISED` state and stays in it until you provide an answer. You can run the upgrade in silent mode, in which the vCenter instance does not generate questions, and uses default answers.

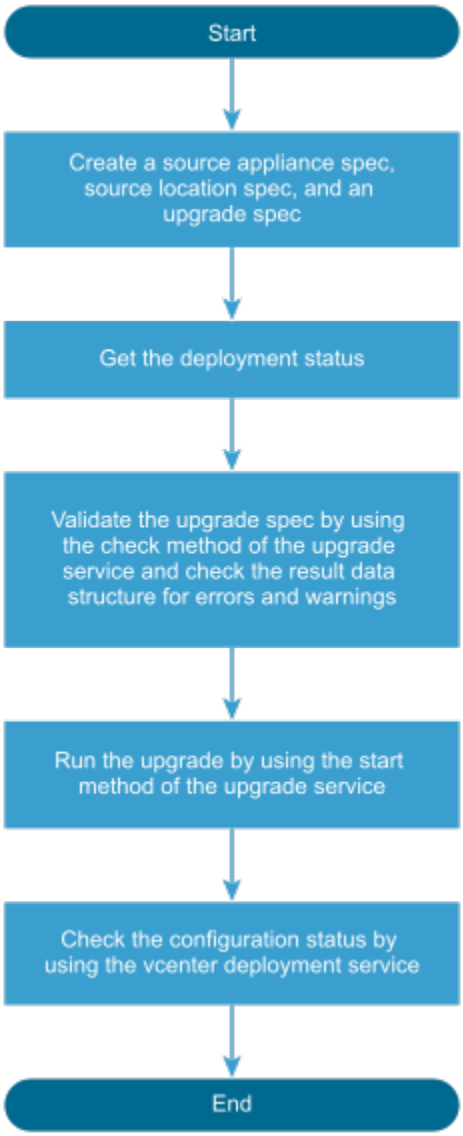
If errors appear during the upgrade, the vCenter instance enters in a `FAILED` state and you must remove the causes, redeploy the instance and restart the upgrade. If stage 2 of the upgrade process is successful, the vCenter instance enters in a `CONFIGURED` state. If the vCenter instance is configured, you can check its services and pause the historical data transfer.

Figure 560: Upgrade Workflow



For information about the states of the vCenter instance and available operations, see Upgrading a vCenter Instance.

Figure 561: Upgrade Stage 2 Workflow



You can run the upgrade pre-checks and the upgrade stage 2 process by creating and passing an `UpgradeSpec`. In `UpgradeSpec`, you define the upgrade configuration and specify the source vCenter instance and the source ESX host in `SourceApplianceSpec` and `LocationSpec`. See Figure 561: Upgrade Stage 2 Workflow . You can run the upgrading in silent mode by setting the `UpgradeSpec.auto_answer` to `true`. The default value of `UpgradeSpec.auto_answer` is `false` and the upgrading is in interactive mode, in which you must provide answers to the raised questions.

For information about the classes, variables, and default values, see the *API reference* documentation.

HTTP Requests for Upgrade Stage 2

You can use HTTP requests or the API to upgrade your vCenter instance.

HTTP Requests

HTTP requests for upgrading

- Get the state of the vCenter instance

  ```
  GET https://<IP_address_upgraded_or_target_instance>:5480/rest/vcenter/deployment
  ```

- Validate the upgrade spec

```
POST https://<IP_address_target_instance>:5480/rest/vcenter/deployment/upgrade?action=check
```

- Run the upgrading

```
POST https://<IP_address_target_instance>:5480/rest/vcenter/deployment/upgrade?action=start
```

- Get the upgrade spec used for upgrading

```
GET https:// https://<IP_address_upgraded_instance>:5480/rest/vcenter/deployment/upgrade
```

HTTP requests for getting and answering a question

- Get the raised question

```
GET https://<IP_address_upgraded_or_target_instance>:5480/rest/vcenter/deployment/question
```

- Answer to the question

```
POST https://<IP_address_upgraded_or_target_instance>:5480/rest/vcenter/deployment/question?action=answer
```

For information about the content and syntax of the HTTP request body, see the *API reference* documentation.

Note:

When you send the requests, you must authenticate with vCenter **root** credentials.

HTTP Status Codes and Errors

Table 1871: HTTP Status Codes and Errors

HTTP Status Code	Description	Operations That Return the Status Code
200	The operation is successful.	All operations. You can check the returned data in the results data structure.
400	You cannot perform the operation because the vCenter instance is in the current state. For information about the states of the vCenter instance, see the Upgrading a vCenter Instance topic.	• Get the state of the vCenter instance • Validate the upgrade spec • Run the upgrade • Get the upgrade spec • Get the raised question • Answer the question
401	You use invalid user name or password, or authentication is failed.	• Get the state of the vCenter instance • Validate the upgrade spec • Run the upgrade • Get the upgrade spec • Get the raised question • Answer the question
404	The state of the vCenter instance cannot be determined	Get the state of the vCenter instance.
500	There is a `vapi std` error. For information about the types of `vapi std` errors, see the `vapi.std.errors` API in the *API reference* documentation.	Get the raised question.

If errors appear during the upgrade process, you can check the `results` data structure, the API log file, and download the vCenter support bundle from `https://<vcenter_ip_address_or_fqdn>:443/appliance/support-bundle`.

Historical Data Transfer

If you migrate vCenter for Windows, you can transfer the historical data of your source vCenter instance together with the core configuration data.

Deferred Import

The deferred import is a process of historical data transfer after the successful migration of a vCenter instance with an external database. The historical data includes statistics, events, and tasks.

By using the deferred import feature, you can postpone the historical data transfer after the migration process completes, so that you manage the downtime of your environment. You can select whether all historical data, or only events and tasks, will be migrated with the core data during the migration. The historical data transfer and deferred import of historical data are deactivated by default. You can enable and configure the historical data transfer by using the API, vCenter Management Interface, vCenter installer, or CLI installer. A vCenter super administrator can run and control the migration and deferred import processes.

If you use the deferred import feature, the historical data is migrated with the core data and the historical data import process starts automatically after a successful migration and when the vCenter instance is running. You can pause the historical data import and resume it later.

For information about how to configure and run the migration and deferred import processes by using the vCenter Management Interface, see the *vCenter Upgrade* documentation.

By using the API, you can configure, control, and monitor the data transfer process. If you use the API to enable the deferred import feature, you must create a history migration spec and set `defer_import` to `true`. For information about how to configure the deferred import by using the API, see the *API reference*.

The data import process has five states that you can check. If the historical data migration and the deferred import are configured, the historical data import starts automatically after a successful migration.

Figure 562: Deferred Import State Diagram

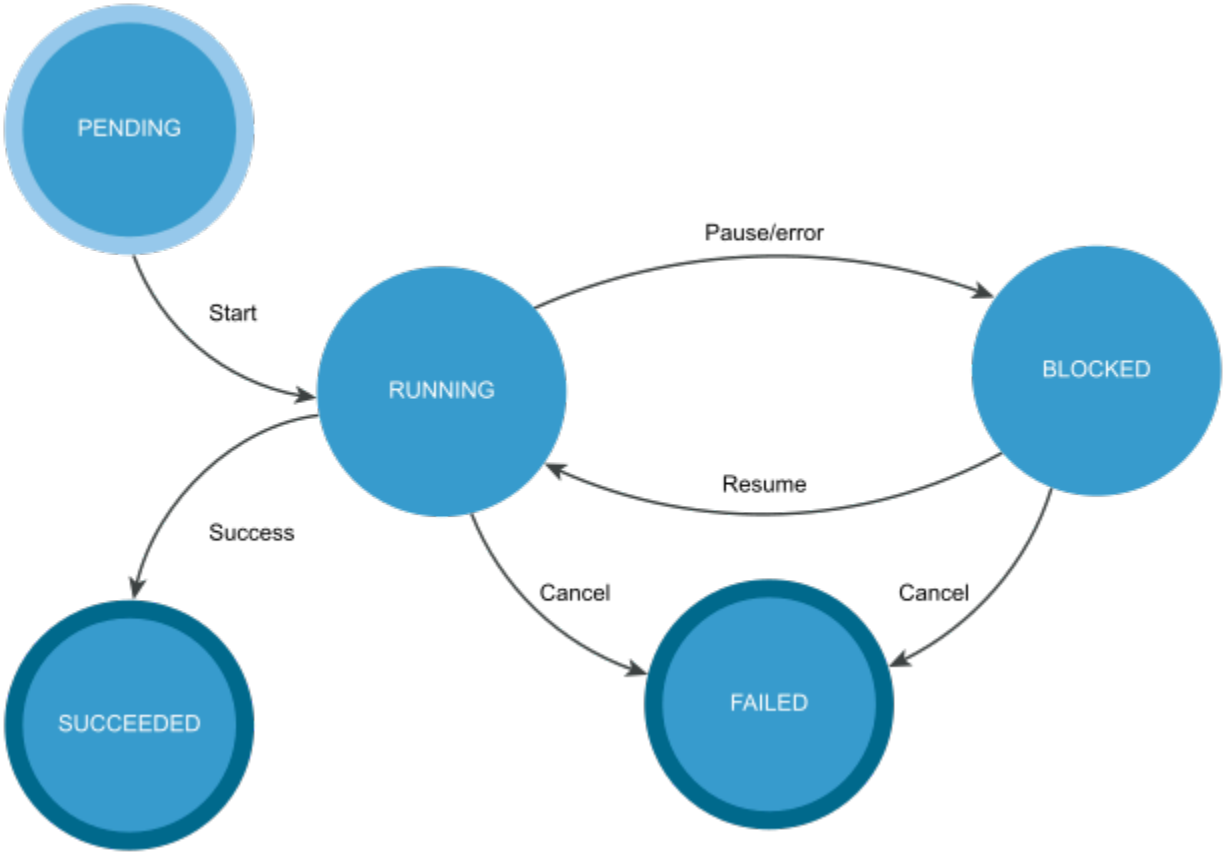


Table 1872: User Operations

Operation	Description
Pause	You can pause the running data transfer process. If the data transfer is paused, you can resume or cancel it.
Cancel	You can cancel the data transfer process if it is in a `RUNNING` or `BLOCKED` state. **Note:** If you cancel the data transfer, the process enters in a final `FAILED` state and you cannot resume the transfer.
Resume	You can resume the stopped data transfer.
Get status	You can retrieve the status of the data transfer process. There are five states. **`PENDING`** The transfer is not started. **`RUNNING`** The transfer is started or resumed. **`BLOCKED`** The transfer is paused or there was a recoverable error, such as not enough disk space, during the import. **`SUCCEEDED`** The transfer is successful. **`FAILED`** The transfer is canceled.

You can run the deferred import operations by using the API or sending an HTTP request.

Note:
When you send the requests, you must use an authentication.

If you send requests to port `5480`, you must authenticate with vCenter**root** credentials. If you send requests to the vCenter reverse proxy port, you must authenticate with vCenter Single Sign-On credentials.

The following HTTP requests show the syntax that you can use to perform the available user operations.

```
https://<IP_of_migrated_instance>:5480/rest/vcenter/deployment/history?action=pause
https://<IP_of_migrated_instance>:5480/rest/vcenter/deployment/history?action=cancel
https://<IP_of_migrated_instance>:5480/rest/vcenter/deployment/history?action=resume
https://<IP_of_migrated_instance>:5480/rest/vcenter/deployment/history
https://<IP_of_migrated_instance>/api/vcenter/deployment/history?action=pause
https://<IP_of_migrated_instance>/api/vcenter/deployment/history?action=cancel
https://<IP_of_migrated_instance>/api/vcenter/deployment/history?action=resume
https://<IP_of_migrated_instance>/api/vcenter/deployment/history
```

For information about the historical data transfer errors, see Historical Data Import Errors.

If you pause the data transfer by using the API or an HTTP request, you can resume or cancel the process by using the API or the vCenter Management Interface.

Important:
If you cancel the transfer process, and want to transfer the historical data later, you must restart the migration process.

Historical Data Import Errors

If an error appears during the data import, the import stops and the process enters in a `BLOCKED` state. You can resume the data import after you eliminate the cause.

You can check the errors, warnings, and info messages by reading the `info`, `status`, and `notifications` data structures.

If the information in the data structures is not enough, you can download the vCenter support bundle from *<vcenter_ip_address_or_fqdn>*`:443/appliance/support-bundle` and check the log files.

Table 1873: Log Files

Log File	Path
API log file	`/var/log/vmware/applmgmt/applmgmt.log`
Backend log file	`/var/log/vmware/upgrade/upgrade-post-import.log`
Upgrade Runner log file	`/var/log/vmware/upgrade/deferredimport-upgrade-runner.log`
Deferred import log file	`/var/log/vmware/upgrade/DeferredImport_com.vmware.vcdb_<date_time>.log`

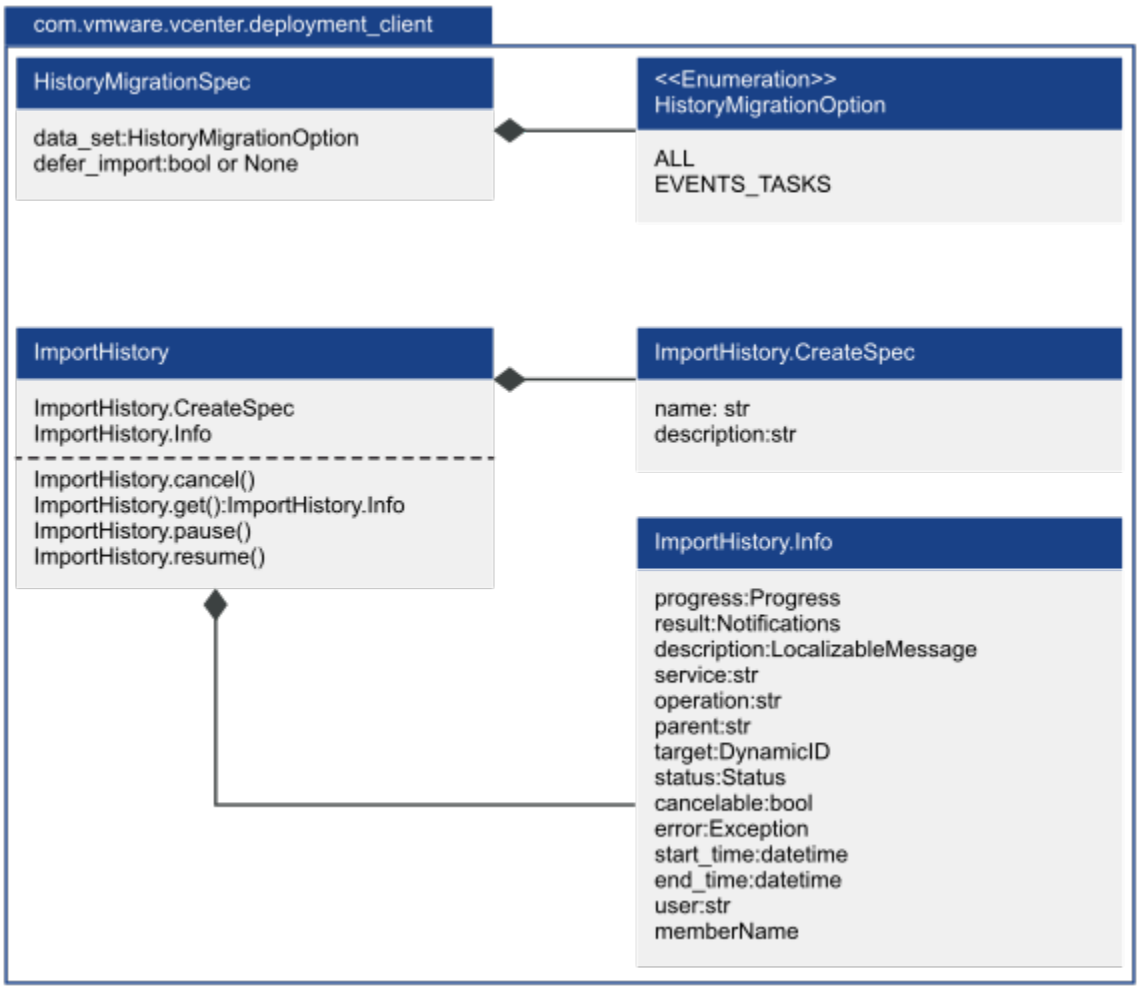
Class Diagrams for Deferred Import

The `vcenter deployment` API provides classes and interfaces that you can use for configuring and controlling the historical data import.

The historical data transfer during the vCenter migration is deactivated by default and only the core data is migrated. You can enable the historical data transfer and the deferred import by creating a history migration spec and setting `defer_import` to `true`. For example, see Figure 563: Python Class Diagrams for Deferred Import . You can change the historical data scope by using the `HistoryMigrationOption` enumeration. By default, the `data_set` is set to `EVENTS_TASKS`.

You can control the deferred import by creating an import history spec and calling the methods of the `ImportHistory` class. Figure 563: Python Class Diagrams for Deferred Import shows the classes that you can use to configure and control the deferred import.

Figure 563: Python Class Diagrams for Deferred Import



Use the Deferred Import Sample

You can use the vCenter Management Interface to migrate your vCenter instance and to run the `vc_import_history_sample.py` sample to pause and resume the historical data import.

- Verify that you cloned or downloaded the vSphere Automation SDK for Python from https://github.com/vmware/vsphere-automation-sdk-python.
- Verify that you set up a test environment. For information about the prerequisites and how to set up a test environment, see the `README.md` file in the `deferhistoryimport` directory and the Quick Start guide for vSphere Automation SDK for Python at https://github.com/vmware/vsphere-automation-sdk-python.
- Verify that you have vCenter root credentials.
- Verify that you opened the vCenter Management Interface of your target vCenter instance.

1. From the getting started page, run the **Migrate** wizard.
2. Select one of the options for **Configuration and historical data**.
   You can import only events and tasks or import all historical data.
3. Select **Import historical data in the background** and complete all steps from the wizard.
4. Run the `vc_import_history_sample.py` sample after successful migration.
   Use the IP address of your source vCenter instance and the vCenter administrator credentials when you run the sample. You can use or skip the verification of the vCenter certificate. For example, you can use the following command.

```
vc_import_history_sample.py --server <IP_of_migrated_instance> --username <admin_username> --password
 <admin_password> --skipverification
```

You import the hystorical data of the selected vCenter instance.

Python

The example shows how you can pause and resume the deferred import process by using the API. The example is based on the `vc_import_history_sample.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...
self.service_manager = ServiceManager(args.server,
                                       args.username,
                                       args.password,
                                       args.skipverification)
self.service_manager.connect()
...
# Using REST API service
import_history = ImportHistory(self.service_manager.stub_config)
...
# Change the status - either pause or resume it
start_status = get_defer_history_import_status(import_history)
if start_status == Status.RUNNING:
 print('Pausing Defer History Data Import.')
 import_history.pause()
 ...
elif start_status == Status.PAUSED:
 print('Resuming Defer History Data Import.')
 import_history.resume()
...
```

vCenter Configuration

You can use the API to perform operations related to health monitoring and capacity monitoring of vCenter. You can also manage the global FIPS compliance and perform infrastructure profile management operations.

Health Monitoring of vCenter

The vCenter API offers health status indicators for several key components of the system. These indicators can be polled periodically to monitor the components for problems.

The health status indicators report graded values from green to red. The general meanings of the grades are as follows.

green	The component is healthy.
yellow	The component is healthy, but may have some problems.
orange	The component is degraded, and may have serious problems.
red	The component is unavailable, or will stop functioning soon.
gray	No health data is available.

Check Overall System Health of vCenter

vCenter provides a composite health indicator that enables you to test a single value that represents the health of all system components. This procedure shows how to test the composite health indicator.

Verify that you have an active authenticated session with vCenter. This procedure assumes that the session ID is present in the security context of a stub configuration.

The value of the overall system health indicator reflects the strongest trouble indication among the vCenter components. If any component has a `red` indicator, the overall system health indicator is `red`, else if any component has an `orange` indicator, the overall system health indicator is `orange`, and so on.

A `gray` value for any component indicates that data for the subsystem is unknown. If one or more components have a `gray` value, but all other subsystems are `green`, then the overall system health indicator is `gray` rather than `green`. However, if any component has a definite trouble indication, the overall system health indicator reflects the strongest trouble indication among the components.

1. Create an interface stub or REST path that uses the stub configuration.
2. Invoke the `health.system` method.
3. Format and display the resulting value.

You retrieved information about the overall health status of your vCenter system.

Python

This example shows the use of Python with the vSphere Automation SDK for Python to request the overall system health indicator for vCenter and the overall health indicator for management services. The example assumes a previously existing session with the VMware Cloud Foundation API endpoint.

This example depends on the following global variables.

- *my_stub_config*

```
from com.vmware.appliance import health_client

# This example assumes you have previously created a session
# and stored the session ID in my_stub_config.

# Issue request for overall system health.
System_stub = health_client.System( my_stub_config )
health = System_stub.get()
print( 'Overall system health: %s' % health )

# Issue request for applmgmt services health.
Applmgmt_stub = health_client.Applmgmt( my_stub_config )
health = Applmgmt_stub.get()
print( 'Applmgmt services health: %s' % health )
```

Capacity Monitoring ofvCenter

vCenter keeps a history of statistics that you can use to monitor resources used by thevCenter instance.

You can use the statistics to spot peak usage demands or to monitor trends for advance warning of potential resource exhaustion.

Frequency and Retention of Statistics Collection invCenter

vCenter collects statistics from the guest operating system at regular intervals and stores them in a database. Users can query the statistics in the database by selecting a time period and a roll-up function thatvCenter applies to the statistics before returning them to the client.

After the monitoring service starts up, it begins requesting statistics from the guest operating system periodically, at a frequency that depends on the type of statistic. The service requests storage statistics once every 10 minutes, while it requests memory, CPU, and networking statistics once per minute. The collection times are fixed relative to the startup time of the monitoring service, rather than to clock time.

The monitoring service retains statistics approximately 13 months, by default. Older statistics are deleted by the service, creating a 13-month moving window within which you can query statistics. You can choose to delete statistics as needed to conserve storage resources.

Nature of Statistics invCenter

vCenter supplies statistics of several types.

The guest operating system computes statistics either as rates, such as CPU cycles per second, or as snapshots of size, such as KB used for storage. Statistics stored as size snapshots are collected at the end of their sample periods. Statistics stored as rates are computed as averages of values sampled frequently during each sample period.

When you query the statistics database, the units are not returned with the data, but you can determine the units for any metric by requesting metadata for the metric with the `get()` method.

Requesting Statistics fromvCenter

To request statistics, you must construct an appropriate request structure to filter statistics from the database.

To request data or metadata for a metric, you must supply the ID of the metric. You can get a list of metric IDs by using the `list()` method, which returns information on all available metrics.

When you query statistics, you provide a list of IDs to specify the metrics in which you are interested. You also supply a start time, an end time, a roll-up interval, and a roll-up function. These values interact as follows to determine the data returned to you.

- The response contains a list of data points for each metric ID you specified in the request.
- The start time and end time control the limits for the data you want in the response. The response contains data points only for statistics that have timestamps between those limits, inclusive of the endpoints. However, the start time is adjusted to a round number, in some cases.
- The roll-up interval enables you to control the granularity of the data points in the response. Rather than a response with a data point for every statistic between the start time and end time, you get a response with a number of data points equal to the number of intervals between the start and end times. Generally, you should specify a time period that is an even multiple of the interval, so that each data point in the response represents the same number of statistics.
- The roll-up function specifies how the response summarizes the statistics that fall within each interval. The resulting data point can be the maximum statistic value within collection interval, or the mean of the statistics values within the interval, and so on.

Statistics Collection Times

The actual time that a statistic was collected is not readily predictable.

The API does not enable you to determine the exact time that a statistic was collected. Furthermore, some statistics, such as those for storage metrics, might take seconds or minutes to collect, so that they are not available immediately at the time a request is made to the guest operating system.

However, because statistics are collected at regular intervals, and roll-up intervals for a request generally all have the same size, each data point in the response represents the same number of statistics as the others.

Statistics Interval Adjustment invCenter

When you make a request for statistics, the monitoring service might adjust the specified roll-up interval times to improve the appearance of statistics graphs in a graphical interface.

The monitoring service adjusts the start time of a data collection request when it is not an exact multiple of the interval length. In these instances, the start time is rounded downward to the previous UTC time that is a multiple of the interval. All subsequent intervals of the data collection are also adjusted to align with the new start time.

For example, if the start time is 10:31 and the interval length is 1 hour, the monitoring service adjusts the start time to 10:00 and the roll-up intervals have the following continuous pattern.

- 10:00 to 10:59:59.999
- 11:00 to 11:59:59.999
- 12:00 to 12:59:59.999

The monitoring service does not adjust the end time of a data collection. Consequently, the response to a statistics query might contain one more data value than expected, or an incomplete final interval might be lengthened.

Empty Data Values

In some instances, you might encounter a response that reports an empty data value, or even a series of empty data values. This might manifest as a list of data values containing some numeric values alternating with empty values.

- Empty data values can happen when the report time period is too short to be certain of containing any statistics. For instance, a time period of 30 seconds is half the length of the sample period for network metrics, so you have only a 50% chance of finding a network statistic during any 30-second reporting period.
- Empty data values can also happen when the interval is shorter than the sample period for a metric you have requested. In this case, some data points are present in the list, while others are empty because no statistic was collected during those intervals. For instance, an interval of 5 minutes is only half the length of the sample period for storage metrics, so every second data value is empty.
- Empty data values can also happen when the monitoring service has not finished collecting and writing the last sample to the database, even if the nominal sample timestamp falls within the report time period. For example, calculation of storage used can delay writing a storage statistic to the database. A request for the statistic during that delay time produces an empty data point in the response.

When a response contains an empty data value, this indicates that no statistics were collected during a collection interval. An appropriate action for the client in such a case depends on how the client is using the data. For example, if you are graphing a resource usage trend, you might choose to interpolate for the missing value to produce a smooth line.

Check Database Usage in vCenter

vCenter contains a database of all objects managed by the vCenter instance. In addition to inventory objects, the database includes vCenter statistics, events, alarms, and tasks. You can calculate the database storage consumption by adding the sizes of all data categories.

This task assumes you have previously authenticated and created a client session.

You need to monitor storage consumption in vCenter.

1. Prepare a request for database usage statistics.

   Include metric IDs both for `vcdb_core_inventory` and `vcdb_seat`. The name `vcdb_seat` refers to Statistics, Events, and Tasks in the vCenter database.

2. Issue the request to the API endpoint.

3. Process the resulting data points as needed.

4. Format and print the results.

The result of this procedure shows the storage used in the vCenter database, which includes storage overhead used for indexes and other optimizations beyond the actual size of the data.

Python

This example shows the use of Python with the vSphere Automation SDK for Python to view recent statistics for the vCenter database usage in vCenter. The example assumes a previously existing session with the VMware Cloud Foundation API endpoint.

This example requests statistics at 30-minute intervals for a recent 2-hour report period. The example requests the storage used by the inventory component and the storage used by the combination of statistics, events, alarms, and tasks. The example adds the two values to calculate the vCenter database usage in each 30-minute roll-up interval, and then reports the maximum size found over the 2-hour report period.

```
from com.vmware import appliance_client
import datetime

# This example assumes you have previously created a session
# and stored the session ID in my_stub_config.

# Issue request for core inventory and 'SEAT' (stats, events, & alarms) usage.
req = appliance_client.Monitoring.MonitoredItemDataRequest()
req.names = ['storage.used.filesystem.vcdb_core_inventory',
             'storage.used.filesystem.vcdb_seat']
req.interval = appliance_client.Monitoring.IntervalType.MINUTES30
req.function = appliance_client.Monitoring.FunctionType.MAX
d_now = datetime.datetime.utcnow()
 req.start_time = d_now - datetime.timedelta( minutes=135 )
 req.end_time = d_now - datetime.timedelta( minutes=15 )
 Monitoring_stub = appliance_client.Monitoring( my_stub_config )
 resp = Monitoring_stub.query( req )

 # Extract resulting arrays.
 core_sizes = resp[0].data
 seat_sizes = resp[1].data
 # Remove empty data points:
 core_sizes = filter( (lambda x: x != ''), core_sizes )
 seat_sizes = filter( (lambda x: x != ''), seat_sizes )

 # Add the usage stats for each interval, and display maximum usage.
 highest = max( map( (lambda a,b: int(a) + int(b)),
                     core_sizes, seat_sizes ) )
 print( 'vCenter database inventory + stats, events, alarms, tasks:' +
        ' (max) size = {0} KB'.format( highest ) )
```

List Storage Consumption By Data Type in vCenter

vCenter provides statistics on several types of storage.

Verify that you have authenticated and created a client session.

For example, you can query statistics about inventory storage, transaction log, and vCenter tasks. Many of these statistics are available both for storage consumed and storage available.

This task provides data for system administrators who need to monitor storage consumption in the guest operating system of vCenter.

1. Prepare a request for database usage statistics.

   Include metric IDs for each data type you want to monitor.

2. Issue the request to the API endpoint.
3. Process the resulting data points as needed.
4. Format and print the results.

You retrieved information about the storage consumption in your vCenter system.

Python

This example shows how to use the Monitoring interface to break down database usage by data type. The example requests the individual data types that you can also query as a composite metric for all storage used by Alarms, Statistics, Events, and Tasks in the vCenter instance.

```
from com.vmware import appliance_client
import datetime

# This example assumes you have previously created a session
# and stored the session ID in my_stub_config.

# Prepare request for chosen data types.
req = appliance_client.Monitoring.MonitoredItemDataRequest()
req.interval = appliance_client.Monitoring.IntervalType.MINUTES30
req.function = appliance_client.Monitoring.FunctionType.MAX
d_now = datetime.datetime.utcnow()
req.start_time = d_now - datetime.timedelta( minutes=30 )
req.end_time = d_now
mon = {'storage.totalsize.directory.vcdb_hourly_stats' :
         'Hourly stats',
       'storage.totalsize.directory.vcdb_daily_stats' :
         'Daily stats',
       'storage.totalsize.directory.vcdb_monthly_stats' :
         'Monthly stats',
       'storage.totalsize.directory.vcdb_yearly_stats' :
         'Yearly stats',
       'storage.totalsize.directory.vcdb_events' :
         'Events',
       'storage.totalsize.directory.vcdb_alarms' :
         'Alarms',
       'storage.totalsize.directory.vcdb_tasks' :
         'Tasks'}

req.names = []
for item in mon.keys() :
   req.names.append( item )

# Issue request.
Monitoring_stub = appliance_client.Monitoring( my_stub_config )
resp = Monitoring_stub.query( req )

# Assemble data from response.
out = {}
for metric in resp :
  # Discard empty data points:
  stat = ''
  while (stat == '') :
    stat = metric.data.pop()
  stat = int(stat)
  out[mon[metric.name]] = stat

# Format and print statistics.
for label in sorted( out.keys() ) :
```

```
print( '{0:15s}: {1:8d} KB'.format( label, out[label] ) )
```

Managing the Global FIPS Compliance

You can retrieve information about the current FIPS (Federal Information Processing Standards) settings of vCenter. You can also enable or deactivate the global FIPS compliance.

FIPS 140-2 is a U.S. and Canadian government standard that specifies security requirements for cryptographic modules. vSphere uses FIPS-validated cryptographic modules to match those specified by the FIPS 140-2 standard. The goal of vSphere FIPS support is to ease the compliance and security activities in various regulated environments.

The following table lists the operations that you can perform to manage the FIPS settings of your vCenter system.

Table 1874: User Operations

Operation	Description
Get FIPS status	You can check whether the global FIPS compliance is currently enabled on the vCenter system.
Manage FIPS status	You can enable or deactivate the global FIPS compliance on the vCenter system.

Note:
When you enable FIPS compliance, some components might present functional constraints. For more information, see *vSphere Security*.

You can run FIPS management operations by using the VMware Cloud Foundation SDK or sending an HTTP request. For information about the HTTP requests that you can use to perform the user operations, see HTTP Requests for Global FIPS Compliance Operations.You can use the `GlobalFips` interface from the `com.vmware.appliance.system.security` package to check the FIPS status and enable or deactivate the global FIPS compliance. For more information, see the *Java API Reference* documentation.

REST

The following HTTP requests show the syntax that you can use to perform the available user operations.

- Get FIPS status

  ```
  GET https://<vcenter_ip_address_or_fqdn>:443/api/appliance/system/global-fips
  ```

- Manage FIPS status

  ```
  PUT https://<vcenter_ip_address_or_fqdn>:443/api/appliance/system/global-fips
  ```

  You can enable the global FIPS compliance by using the following request body.

  ```
  {
      "enabled": true
  }
  ```

  You can deactivate the global FIPS compliance by using the following request body.

  ```
  {
      "enabled": false
  }
  ```

  After you change the global FIPS compliance, the vCenter system reboots to apply the change.

Performing Infrastructure Profile Management Operations

You can export an existing vCenter configuration and import it to other vCenter instances.

You can export multiple configuration profiles at once. The exported data can contain general configuration settings and user content. You can replicate the same configuration across all vCenter instances in your environment by importing the same data package. You can also use the exported data as a backup if you need to revert to the last known good configuration. To avoid configuration issues, you can validate the exported data before importing it to a vCenter instance.

The following table lists the operations that you can perform to manage the configuration profiles in your infrastructure.

Table 1875: User Operations

Operation	Description
List configuration profiles	You can retrieve a list of all configuration profiles that are registered with vCenter.
Export configuration profiles	You can export specific vCenter configuration profiles.
Validate configuration profiles	You can validate the exported vCenter configuration profiles. The validation process examines the configuration file for possible errors and conflicts and returns output. This operation can help avoid configuration issues or loading the wrong configuration file.
Import configuration profiles	You can import specific vCenter configuration profiles into another vCenter instance.

You can run infrastructure profile management operations by using the VMware Cloud Foundation SDK or sending an HTTP request. You can use the `Configs` interface from the `com.vmware.appliance.infraprofile` package to list, export, validate, and import configuration profiles. For more information, see the *Java API Reference* documentation.

Patching and Updating vCenter Deployments

You can use the API to perform operations related to the planning and installing of vCenter software updates.

Planning vCenter Updates

The vCenter API provides operations that can help you plan the life cycle of vCenter instances in your environment.

The vCenter API provides operations to help plan the lifecycle of vCenter instances. You can discover associated VMware products, manage product associations, and generate detailed interoperability and pre-check reports. The reports can contain interoperability or pre-check information.

- Interoperability reports contain information about the interoperability between the associated products and a specific vCenter version.
- Pre-check reports contain information about the compatibility of the current vCenter version with a pending update version.

You can plan to perform vCenter updates which are based on the information that is gathered in the produced reports.

In VCF 9.1, these reports include a quick patch classification and estimated downtime metrics. This allows you to identify security updates that can be applied with minimal disruption to vCenter services. You can also plan maintenance windows which are based on specific time-to-remediate calculations.

Performing Discovery and Planning Operations

You can retrieve information about VMware products associated with vCenter, list available vCenter updates, and produce reports. The findings can help you plan vCenter updates in your environment.

The lifecycle management API provides operations that are grouped in the discovery, update, and reports categories. The discovery functionality of the API consists of the operations in the discovery category. The planning functionality of the API consists of the operations in the update category. Both functionalities can produce reports that you can download by using the reports category.

The discovery category provides operations for listing VMware products that can be associated with vCenter, managing products associations, and creating interoperability reports. The update category provides operations for listing all available updates and upgrades for vCenter and generating a pre-check compatibility report. The reports category provides an operation for downloading reports generated by interoperability and pre-check operations. By using the retrieved information, you can select one of the available patches and can plan an actual patch or upgrade for a specific vCenter version.

The following table lists the operations that are available in the discovery category.

Table 1876: Discovery User Operations

Operation	Description
Get product catalog	You can retrieve a list of all VMware products that can be associated with vCenter.
List associated products	You can retrieve a list of all VMware product deployments in the environment that are associated with vCenter. **Note:** The list contains both product deployments discovered automatically and deployments registered manually through the API.
Get associated product information	You can retrieve detailed information about a product associated with vCenter.
Create product association	You can manually associate a VMware product with vCenter.
Update product association	You can modify a manually added VMware product that is associated with vCenter. **Note:** You cannot modify VMware products that are discovered automatically.
Delete product association	You can delete or dissociate a manually added VMware product that is associated with vCenter. **Note:** You cannot delete or dissociate VMware products that are discovered automatically.
Create interoperability report	You can create an interoperability report between a vCenter release version and all products registered with the vCenter instance.

The following table lists the operations that are available in the update category.

Table 1877: Update User Operations

Operation	Description
List updates	You can retrieve a list of all available vCenter updates. The list can contain minor, in-place updates, and major migration-based, upgrades. In VCF 9.1, the list identifies which updates support quick patches to reduce downtime.
Get update info	You can retrieve detailed vCenter information about a specific update or upgrade. In VCF 9.1, the response includes an `estimated_time_to_install` field and a `services_will_be_stopped` list.
Create a pre-check report	You can create a vCenter pre-update compatibility check report for a pending update version. **Note:** You can export and download the report in CSV format.

The following table lists the operations that are available in the reports category.

Table 1878: Reports User Operations

Operation	Description
Get report	You can download the report generated by the interoperability and pre-check operations. For information about downloading the report, see the *API reference* documentation.

You can run life cycle management operations by using the VMware Cloud Foundation SDK or sending an HTTP request. For information about the HTTP requests that you can use to perform the user operations, see *HTTP Requests for Discovery and Planning Operations* . In addition to sending HTTP requests, you can also run cURL commands to perform operations. See *cURL Examples of Performing Discovery and Planning Operations* . You can use interfaces from the `com.vmware.vcenter.lcm.discovery`, `com.vmware.vcenter.lcm.update`, and `com.vmware.vcenter.lcm` packages to perform discovery, update, and reports operations. For more information, see the *Java API Reference* documentation.

cURL

The following cURL command examples show the syntax for performing life cycle operations such as discovering and managing product associations, listing updates, performing pre-checks, and retrieving reports.

- This example lists all compatible versions of VMware products that can be associated with vCenter.

```
curl -X GET -u
 administrator@vsphere.local:<password>'https://<vcenter_ip_address_or_fqdn>:443/
api/vcenter/lcm/discovery/product-catalog'
```

- This example lists all VMware products associated with vCenter including manually added products.

```
curl -X GET -u
 administrator@vsphere.local:<password>'https://<vcenter_ip_address_or_fqdn>:443/
api/vcenter/lcm/discovery/associated-products'
```

- This example associates a VMware product with vCenter.

```
curl -H "Content-Type: application/json" -X POST -d
 '{"name" :"vRealize Automation", "version": "6.8.9"}' -u
 administrator@vsphere.local:<password>'https://<vcenter_ip_address_or_fqdn>:443/
api/vcenter/lcm/discovery/associated-products' -k -i
```

- This example retrieves details about a product associated with vCenter.

```
curl -X GET -u administrator@vsphere.local:<password>
 'https://<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/discovery/products/
com.vmware.vRA-6.8.9'
```

- This example updates a VMware product associated with vCenter. You can modify only manually added products.

```
curl -H "Content-Type: application/json" -X PATCH -u
 administrator@vsphere.local:<password> 'https://<vcenter_ip_address_or_fqdn>:443/
api/vcenter/lcm/discovery/associated-products/com.vmware.vRealOrche_7.3.1' -k -d
 '{"spec":{"deployments": ["<ip_address>"]}}'
```

- This example deletes or dissociates a VMware product associated with vCenter. You can delete or dissociate only manually added products.

```
curl -X DELETE -u administrator@vsphere.local:<password> 'https://
<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/discovery/products/
com.vmware.vRA-6.8.9'
```

- This example generates an interoperability report for all VMware products associated with vCenter against a specific vCenter version.

```
curl -X POST -u administrator@vsphere.local:<password>
 'https://<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/discovery/interop-
report?vmw-task=true' -k -d '{"spec":{"target_version":"6.7.0.2000"}}' -H "Content-
Type: application/json"
```

- This example lists all available and applicable updates for vCenter versions discovered in your environment. The list can contain minor, in-place, updates and major, migration-based, upgrades. The operation calculates which updates or upgrades are applicable based on the current vCenter version.

```
curl -X GET -u administrator@vsphere.local:<password>
 'https://<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/update/pending'
```

- This example retrieves details about a specific vCenter update. The update is identified by the ID provided in the URL. The update can be either a minor, in-place, update or a major, migration-based, upgrade.

```
curl -X GET -u administrator@vsphere.local:<password>
 'https://<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/update/
pending/7.0.0.20000'
```

- This example performs source update pre-checks and identifies whether the provided update ID corresponds to a minor update or a major upgrade. In case of a minor update, the operation invokes the appropriate update API. In case of a major upgrade, the operation downloads and installs the `requirements.rpm` file for the upgrade. The pre-check can be a long-running task, which you can monitor by using the task ID that the operation returns.

```
curl -X POST -u administrator@vsphere.local:<password> 'https://
<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/update/pending/7.0.0.50000/
precheck-report?vmw-task=true' -k
```

- This example tracks the status, progress, and retrieves the final result of the pre-check operation.

```
curl -X GET -u administrator@vsphere.local:<password>
 'https://<vcenter_ip_address_or_fqdn>:5480/rest/cis/
tasks/3f77223f-1edb-4992-9c48-2d75d7b3b91d:com.vmware.vcenter.lcm.update.precheck_report'
 -k
```

- This example retrieves information about the location of the actual report for downloading.

```
curl -X GET -u administrator@vsphere.local:<password> 'https://
<vcenter_ip_address_or_fqdn>:443/api/vcenter/lcm/reports/abcd123_report.csv'
```

List Available Products and Manage Associated Products

You can automate the management of VMware products associated with vCenter by using the API.

- Verify that you have an active authenticated session with vCenter.

This procedure includes the operations that you can use to manage the product catalog and associated products.

1. Create a stub configuration.
2. Retrieve the product catalog.
3. Retrieve a list of VMware products associated with vCenter.
4. Associate a VMware product with vCenter.
5. Update a VMware product associated with vCenter.
6. Delete a product form the list of VMware products associated with vCenter.

You view the list of available products and can update and delete VMware products associated with the vCenter instance

Python

This example shows how you can retrieve the product catalog, list the associated products, add, update, and delete products associations. The example is based on the `discovery_sample.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...
        # Create a stub configuration
        stub_config = StubConfigurationFactory.new_std_configuration(connector)
        self.product_client = ProductCatalog(stub_config)
        self.associated_products_client = AssociatedProducts(stub_config)

    def run(self):
        # Product catalog
        product_catalog = self.product_client.list()
        print("Product catalog list: \n", product_catalog)

        # Associated products
        associated_products = self.associated_products_client.list()
        print("Associated products list : \n", associated_products)

        # Add product
        spec = {'product_name': 'VMware Identity Manager', 'version': '3.3',
 'deployments': '3'}
        add_associated_product = self.associated_products_client.create(spec)
        print('Added new product. \n', add_associated_product)

        associated_products = self.associated_products_client.list()
        print("Associated products after adding the product: \n", associated_products)
```

```
        # Update product
        update_spec = {'deployments': '9'}
        update_associated_product =
self.associated_products_client.update(add_associated_product, update_spec)
        associated_products = self.associated_products_client.list()
        print("Associated products after updating the product: \n",
associated_products)

        # Delete product
        delete_associated_product =
self.associated_products_client.delete(add_associated_product)
        associated_products = self.associated_products_client.list()
        print("Associated products after deleting the product: \n{0}",
associated_products)
...
```

.

List Available Updates

You can retrieve a list of available vCenter updates, details about the updates, and pre-check information by using the API.

- Verify that you have an active authenticated session with vCenter.

1. Create a stub configuration.
2. Retrieve a list of available vCenter updates.
   If there are available updates, you can retrieve details about the updates.
3. Retrieve pre-check information.

You can review the updates information for the vCenter instance.

Python

This example shows how you can retrieve a list of available vCenter updates, details about the updates, and pre-check information. The example is based on the `update_sample.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...
        # Create a stub configuration
        stub_config = StubConfigurationFactory.new_std_configuration(connector)
        self.pending_client = Pending(stub_config)
        self.precheck_client = PrecheckReport(stub_config)

    def run(self):
        # List updates
        available_updates = self.pending_client.list()
        print("vCenter available updates - ", available_updates)
        if available_updates.updates:
            target_version = available_updates.updates[0].version
            update_details = self.pending_client.get(target_version)
            print("vCenter available update details - ", update_details)

            # Get pre-check result
            precheck_result = self.precheck_client.create_task(target_version)
            print("Pre-upgrade checks task id started with:
\n{0}".format(precheck_result.get_task_id()))
...
```

Retrieve a Report

You can retrieve a report generated by the interoperability and pre-check operations by using the API.

- Verify that you have an active authenticated session with vCenter.

1. Create a stub configuration.
2. Retrieve the report details.

The report is retrieved successfully.

Python This example shows how you can retrieve a report. The example is based on the `lcm_sample.py` sample.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...
        # Create a stub configuration
        stub_config = StubConfigurationFactory.new_std_configuration(connector)
        self.report_client = Reports(stub_config)

    def run(self):
        # Retrieve report
        report_details = self.report_client.retrieve('com.vmware.vcenter.lcm.report')
        print("Report details - ", report_details)
...
```

Updating vCenter

vCenter provides interfaces to perform software updates.

Before applying updates, you must make sure that your environment is prepared for the vCenter software update process.

Applying vCenter Software Updates

You can automate the installation of vCenter software updates to ensure that your system is stable and protected. Software updates can include security fixes, performance optimizations, and new features.

Security patches usually address vulnerabilities in third-party components and do not affect the vCenter functionality. vCenter bug fixes can introduce changes to the functionality without affecting the data format or database schema of the system.

Each update contains metadata with information about the updated content, for example, whether high-priority OS updates are included. The update metadata includes a list of components to be updated, the release date of the update, a list of fixed issues, time and disk space requirements, and information whether a reboot is required. The metadata can also contain a new vCenter version number, including a build number. In addition to the metadata, an update can contain optional components such as update scripts, new versions of vCenter software components, and new versions of OS components.

In VCF 9.1, security patches are categorized to distinguish between standard updates and quick patches that address vulnerabilities in third-party or OS components without a full system reboot. Update metadata in 9.1 now includes a `services_affected` list, detailing which vCenter services will be restarted or disrupted during the application process.

vCenter can obtain software updates from either a URL or an ISO image. The URL can either point to the VMware Web repository or to a custom repository in which you upload the updates in ZIP format. To perform an update by using an ISO image, attach the image to the CD/DVD drive of the vCenter instance.

If you want to prevent issues related to the possibility of update installation failures, you should create a backup or take a snapshot of your vCenter instance before you start the update process. A backup can also be useful when an update is successfully installed. For example, you might decide to revert to the previous version if you encounter any undesired system behavior related to functional changes in the new software version.

Table 1879: User Operations

Operation	Description
Get state information	You can retrieve information about the update state.
Check for update	You can check whether a new update is available.
Get update information	You can retrieve information about the available updates.
Get update requirements	You can retrieve information about the update requirements.
Stage	You can initiate the download of the update. **Note:** The check phase must have completed successfully before you can stage the update.
Get staging status	You can retrieve information about the status of the stage operation. **Note:** You must provide the task ID value that you received as a response when you initiated the stage operation.
Install	You can initiate the installation of the update. **Note:** The update must be staged before you can install it.
Get installation status	You can retrieve information about the status of the install operation. **Note:** You must provide the task ID value that you received as a response when you initiated the install operation. In VCF 9.1, the response now includes `auto_rollback_state`. If an async service update fails, the system automatically attempts to revert to the previous working version.
Stage and install	You can initiate the download of the update and the installation starts when the download completes.

You can run software update operations by using the VMware Cloud Foundation SDK or sending an HTTP request. You can use interfaces from the `com.vmware.appliance.update` package to perform the operations. For more information, see the *Java API Reference* documentation.

Note:
When you send the requests, you must use an authentication.

If you send requests to port `5480`, you must authenticate with vCenter**root** credentials. If you send requests to the vCenter reverse proxy port, you must authenticate with vCenter Single Sign-On credentials.

The following HTTP requests show the syntax that you can use to perform the available user operations.

```
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending?source_type=DEFAULT
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending/<target_version>
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending/<target_version>/requirements
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending/<target_version>?action=stage
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/task/<update_stage_task_id>
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending/<target_version>?action=install
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/task/<update_install_task_id>
https://<vcenter_ip_address_or_fqdn>:5480/rest/appliance/update/pending/<target_version>?action=stage-and-
install
https://<vcenter_ip_address_or_fqdn>/api/appliance/update
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending?source_type=DEFAULT
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending/<target_version>
```

```
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending/<target_version>/requirements
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending/<target_version>?action=stage
https://<vcenter_ip_address_or_fqdn>/api/appliance/task/<update_stage_task_id>
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending/<target_version>?action=install
https://<vcenter_ip_address_or_fqdn>/api/appliance/task/<update_install_task_id>
https://<vcenter_ip_address_or_fqdn>/api/appliance/update/pending/<target_version>?action=stage-and-install
```

In addition to sending HTTP requests, you can also run cURL commands to perform update operations. See cURL Examples of Performing vCenter Software Update Operations.

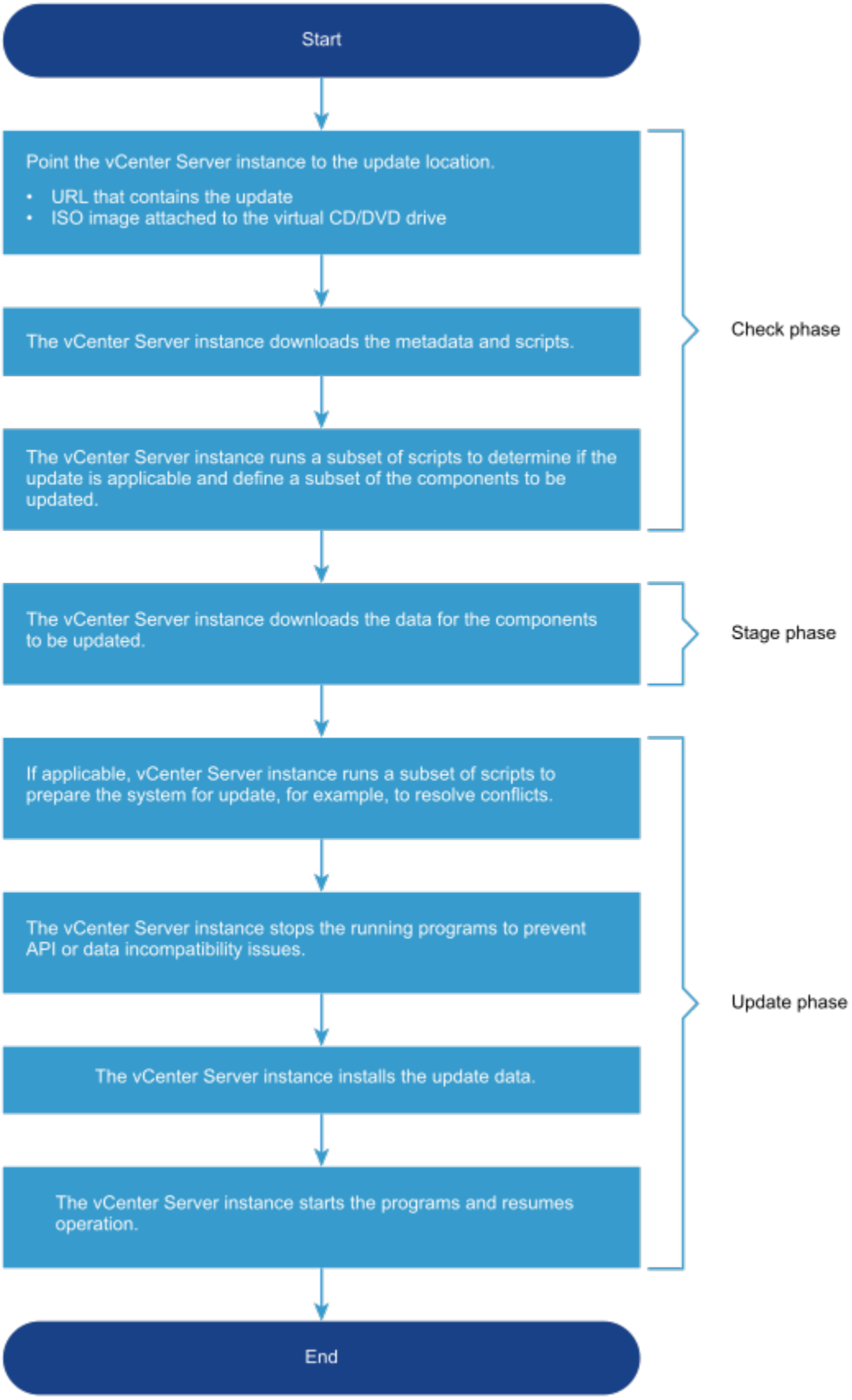
vCenter Software Update Workflow

The vCenter software update process consists of three major phases. In the first phase, the vCenter instance performs various checks, in the second phase it stages the update, and applies the update in the final phase.

To initiate the update process, you must choose whether the vCenter instance should obtain software updates from a URL or an ISO image. If you use an ISO image to update the vCenter instance, the image must remain attached to the CD/DVD drive of the instance during the stage and install operations.

The workflow in Figure 564: Update Process Workflow describes the standard steps of the update process.

Figure 564: Update Process Workflow



You can automate checks for new updates and staging of updates by using an update policy. For example, you can set an update policy to make the vCenter instance perform automatic checks for new updates at midnight every day. If there are new updates available, the vCenter instance can stage them automatically. Using an update policy reduces the waiting time by automating the first two phases and giving you the option to initiate only the update phase manually.

cURL Examples of Performing vCenter Software Update Operations

The following cURL command examples show the syntax for performing update operations such as checking for, staging, and installing updates, as well as retrieving information about update status, and setting update policies.

Check for an Update

This example queries a custom URL for a new update.

```
curl -X GET -k -u root:<root_password> "https://<server>:5480/rest/appliance/update/pending?
source_type=LOCAL_AND_ONLINE&url=https://<custom_url>"
```

Stage an Update

This example initiates the staging of the update.

```
curl -X POST -k -u root:<root_password> -H  "Content-Type: application/json"  -d
 '{"version":"<target_version>"}' https://<server>:5480/rest/appliance/update/pending/
<target_version>?action=stage
```

Install an Update

This example initiates the installation of the update.

```
curl -X POST -k -u root:<root_password> -H "Content-Type: application/json" -d
 '{"version":"<target_version>","user_data":[{"key":"vmdir.password","value":"<sso_password>"}]}'
 https://<server>:5480/rest/appliance/update/pending/<target_version>?action=install
```

Stage and Install an Update

This example downloads the update and installs it when the download completes.

```
curl -X POST -k -u root:<root_password> -H "Content-Type: application/json" -d
 '{"version":"<target_version>","user_data":[{"key":"vmdir.password","value":"<sso_password>"}]}'
 https://<vcenter_fqdn>:5480/rest/appliance/update/pending/<target_version>?action=stage-and-
install
```

Identify Quick Patches

```
curl -X GET -k \
  -u root:<root_password> \
  -H "Content-Type: application/json" \
  'https://<vcenter_fqdn>:5480/rest/appliance/update/pending?quick_patch=true'
```

Retrieve Downtime and Service Impact

```
curl -X GET -k \
  -u root:<root_password> \
  -H "Content-Type: application/json" \
  'https://<vcenter_fqdn>:5480/rest/appliance/update/pending/<target_version>?
fields=estimated_time_to_install,services_will_be_stopped'
```

Retrieve Update Status

This example retrieves information about the update state.

```
curl -X GET -k -u root:<root_password> https://<server>:5480/rest/appliance/update
```

Set an Update Policy

This example sets an update policy to check a custom URL for new updates at specific times every Friday and Saturday, and if a new update is available, it is staged automatically.

```
curl -X PUT -k -u root:<root_password> -H "Content-Type: application/json" -d '{"policy":
{"auto_stage": true,"check_schedule": [{"day": "FRIDAY","hour": 23,"minute": 30},{"day":
 "SATURDAY","hour": 12,"minute": 30}],"custom_URL":"https://123.com"}}'
```

Introduction to the VMware Cloud Foundation SDKs

The VMware Cloud Foundation SDKs bundle client libraries for accessing core management services, including SDDC Manager workload domain orchestration, NSX VPC networking, virtual machine configuration and lifecycle management, Content Libraries, vSphere Tags, Storage Policy-Based Management (SPBM), ESX Agent Manager (EAM), VCF Operations, and VCF Installer.

The VCF SDKs contain sample applications and API reference documentation for the vSphere and VCF services that are accessible via the VCF API endpoint. The VCF SDKs also provide sample code that demonstrates how you can establish a secure connection with vCenter and SDDC Manager endpoints and access the available services for working with virtual infrastructure objects.

The VCF SDK support two programming languages for developing client applications for managing components in your virtual environment by accessing the VCF API services.

- VMware Cloud Foundation Java SDK (vcf-sdk-java-9.1.1.0)
- VMware Cloud Foundation Python SDK (vcf-sdk-python-9.1.1.0)

VMware Cloud Foundation API

Client applications connect directly to well-known service endpoints to authenticate and invoke VMware Cloud Foundation services. To access REST services such as virtual machine management, Content Library, and vSphere Tags, client applications send HTTPS requests directly to the VMware Cloud Foundation REST API endpoint (`https://<vcenter_ip_address_or_fqdn>/api`). To access SOAP-based modules such as core inventory operations (`vim25`), Storage Policy Management (`pbm`), or ESX Agent Manager (`eam`), client applications connect directly to their corresponding well-known HTTPS paths (such as `/sdk`, `/pbm`, and `/eam/sdk`).

Client applications use the VCF SDK client bindings to authenticate with vCenter Single Sign-On (`https://<vcenter_fqdn>/STS/STSService`) or OAuth 2.0 Identity Provider endpoints (`https://<vcenter_fqdn>/api/vcenter/identity/providers`), obtain security tokens or session identifiers, and execute REST operations across core management modules.

API Explorer

The API Explorer is a VMware utility similar to the Managed Object Browser (MOB). Access the API Explorer at `https://<vcenter_ip_address_or_fqdn>/apiexplorer` or from the **API Explorer** tab of the **Developer Center** pane in vSphere Client. Use the utility to connect to the VMware Cloud Foundation API endpoint, browse through the available vSphere REST APIs, and make calls to your vCenter instance.

SDK Samples

The VMware Cloud Foundation SDKs provide code samples that demonstrate how the VMware Cloud Foundation API work. The code snippets included in this documentation are based on the samples in the VMware Cloud Foundation SDKs.

Connecting to Service Endpoints

To access services and resources in the virtual environment, client applications that use the VCF API must know the endpoints of VCF and vSphere services.

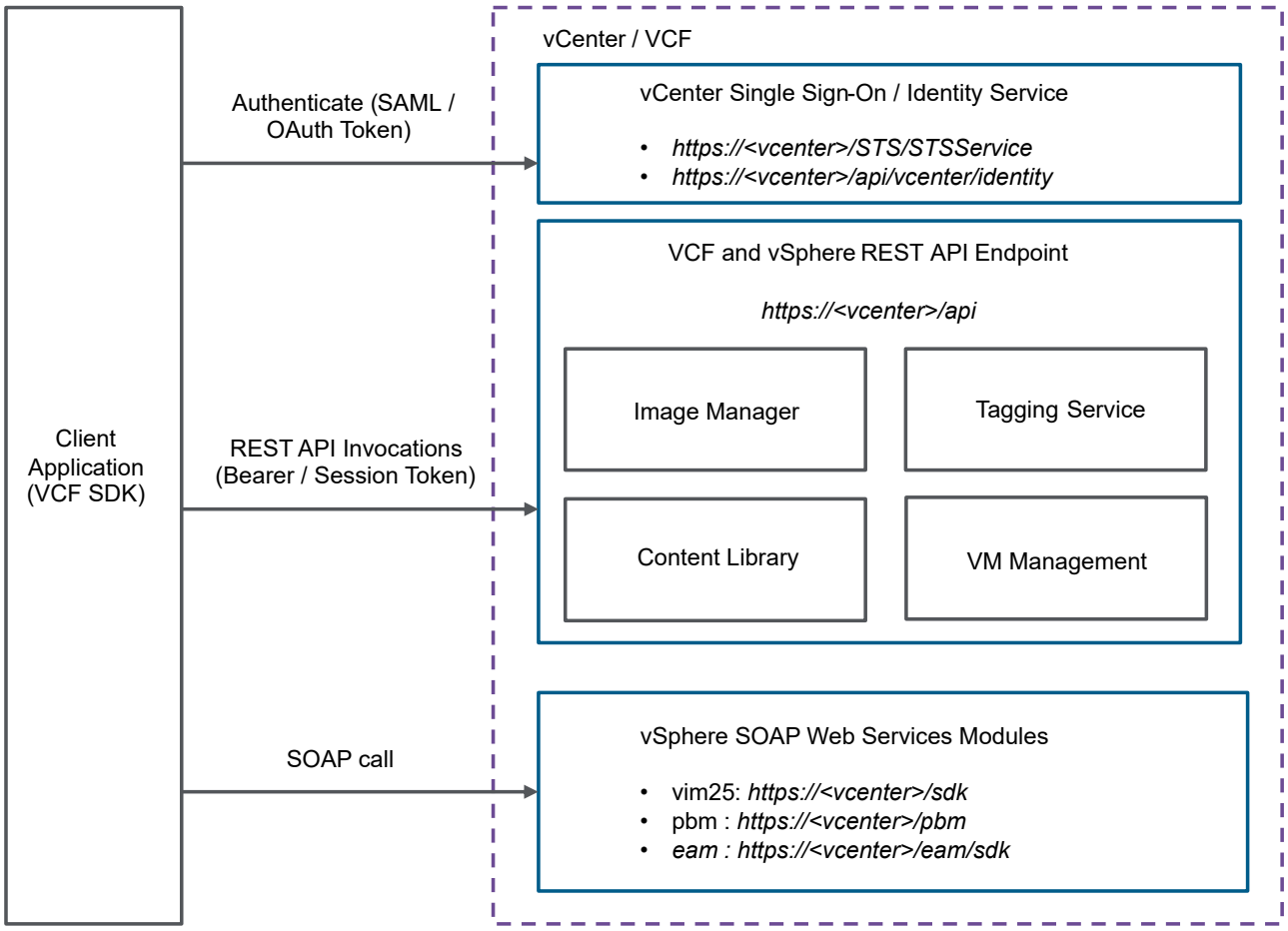
Communicaiton Model

Client applications built with the VCF SDKs issue HTTP/HTTPS requests directly to standard service endpoints hosted by vCenter and SDDC Manager.

The updated communication model simplifies client bootstrap into two straightforward steps:

1. Authenticate
   The client application authenticates with vCenter Single Sign-On or an OAuth 2.0 Identity Provider to obtain a session token or bearer token.
2. Communicate with Services
   The client application issues HTTPS requests directly to standard REST API endpoints (e.g. `/api`) or SOAP Web Services endpoints (e.g. as `/sdk`, `/pbm`, and `/eam/sdk`) using the well-known FQDN of the vCenter or SDDC Manager.

Figure 565: Communication Model Between a Client Application and the VMware Cloud Foundation API



By connecting directly to well-known endpoints, client applications gain the following advantages:

- Eliminate unnecessary network round-trips during initialization.
- Simplify secure TLS handshake configuration.
- Ensure compatibility with VCF 9.x unified client bindings.

Standard Service Endpoints

All service endpoints that are hosted by vCenter and SDDC Manager use standard, deterministic URI patterns over HTTPS on port 443. The following table specifies the default endpoint URLs for all core vSphere and VCF services.

Service Name	API Protocol / Format	Standard Endpoint URL Path	Description
VCF Endpoint	HTTPS / REST (JSON)	`https://<vcenter_fqdn>/api`	Accessing VCF REST services - Virtual Machine management, Content Library, Tagging, Networking, Compute Policies, and Appliance Configuration.
vCenter Single Sign-On (STS)	WS-Trust / SAML 2.0	`https://<vcenter_fqdn>/STS/STSService`	Acquiring SAML security tokens (Holder-of-Key or Bearer) for federated authentication.
OAuth 2.0 Identity Provider	HTTPS / REST	`https://<vcenter_fqdn>/api/vcenter/identity/providers`	Querying external identity providers and retrieving OAuth 2.0 token endpoints in VCF 9.0.
vSphere Web Services (vim25)	HTTPS / SOAP (XML)	`https://<vcenter_fqdn>/sdk`	Core vSphere inventory management and ESX host operations.
Storage Policy Management (pbm)	HTTPS / SOAP (XML)	`https://<vcenter_fqdn>/pbm`	Storage policy management and compliance checking.
ESX Agent Manager (eam)	HTTPS / SOAP (XML)	`https://<vcenter_fqdn>/eam/sdk`	ESX host solution agency lifecycle management.
API Explorer and Developer Center	Web browser UI	`https://<vcenter_fqdn>/apiexplorer`	Interactive API discovery, inline Swagger documentation, and manual REST request testing.
SDDC Manager Fleet API	HTTPS / REST	`https://<sddc_manager_fqdn>:443`	Accessing VCF Fleet APIs (workload domains, clusters, hosts).

Custom HTTP and HTTPS Ports

If the environment customizes HTTP reverse proxy ports, append the custom port to the hostname. For example, when the `rhttpproxy.ext.port2` setting changes the port, use `https://vcenter.domain.com:8443/api`. In standard production deployments, all endpoints route through port `443`.

Java Example of Connecting to Service Endpoints

This example is based on the code in the `VcenterCredentialsAuthenticationExample.java` sample file in the VCF Java SDK (vcf-sdk-java-9.1.1.0). A single authenticated client instance provides direct access to both REST service stubs (`createStub`) and SOAP service bindings (such as `getVimPort()` for `vim25`.

For a complete and up-to-date version of the Java sample code, see the `vmware/vcf-sdk-java` repository at GitHub.

```
package com.vmware.sdk.samples.vcenter.authentication;

import static com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;
import static com.vmware.sdk.vsphere.utils.VimClient.getVimServiceInstanceRef;

import java.text.SimpleDateFormat;
import javax.xml.datatype.XMLGregorianCalendar;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.appliance.system.Version;
import com.vmware.sdk.vsphere.utils.VcenterClient;
```

```
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vim25.VimPortType;

public class DirectServiceEndpointsExample {

    private static final Logger log = LoggerFactory.getLogger(DirectServiceEndpointsExample.class);

    public static String serverAddress = "vcenter.mycompany.com";
    public static String username = "Administrator@vsphere.local";
    public static String password = "SecretPassword123!";
    public static String trustStorePath = "/path/to/truststore.jks";

    public static void main(String[] args) throws Exception {
        // 1. Initialize client factory directly with vCenter FQDN
        VcenterClientFactory factory = new VcenterClientFactory(
                serverAddress,
                loadKeystoreOrCreateEmpty(trustStorePath)
        );

        // 2. Create authenticated client (authenticates directly with /STS/STSService)
        try (VcenterClient client = factory.createClient(username, password, null)) {

            // 3. Access SOAP Web Services API (vim25 at https:///sdk)
            VimPortType vimPort = client.getVimPort();
            XMLGregorianCalendar time = vimPort.currentTime(getVimServiceInstanceRef());
            SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd 'T' HH:mm:ss.SSSZ");
            log.info("Server Current Time (vim25 SOAP): {}",
 sdf.format(time.toGregorianCalendar().getTime()));

            // 4. Access REST Services API (https:///api)
            Version versionStub = client.createStub(Version.class);
            log.info("Appliance Version (VCF REST): {}", versionStub.get().getVersion());
        }
    }
}
```

Tip:

Always instantiate `VcenterClient` inside a try-with-resources block or call `client.close()` explicitly to ensure that underlying HTTPS sessions and STS token references are cleanly released.

Python Example of Connecting to Service Endpoints

This example is based on the code in the `list_vms.py` sample file in the VCF Python SDK (`vcf-sdk-python-9.1.1.0`). For SOAP services (such as `vim25` or `pbm`), applications connect directly to well-known HTTPS paths using standard SOAP bindings (such as `pyVmomi`).

For a complete and up-to-date version of the sample code, see the vmware/vcf-sdk-python repository at GitHub.

```
#!/usr/bin/env python
from pprint import pprint
import requests
from vmware.vapi.vsphere.client import create_vsphere_client

server = "vcenter.mycompany.com"
username = "Administrator@vsphere.local"
password = "SecretPassword123!"

# Configure HTTPS transport session
```

```
session = requests.Session()
session.verify = False  # Set to True or pass CA bundle path in production

# Direct client binding to well-known REST endpoint (https:///api)
client = create_vsphere_client(
    server=server,
    username=username,
    password=password,
    session=session
)

# List Virtual Machines via VCF REST API
print("Listing Virtual Machines via VCF REST API:")
vms = client.vcenter.VM.list()
pprint(vms)
```

Connecting in Multi-vCenter Topologies (Enhanced Linked Mode)

In vSphere 9.x and VCF 9.x, targeting a specific instance in a multi-vCenter or Linked Mode topology is simplified.

To connect to a specific vCenter instance in an Enhanced Linked Mode group or multi-vCenter environment, specify the FQDN or IP address of the target vCenter instance when instantiating the SDK client. For example, `create_vsphere_client(server="vc-finance.mycompany.com", ...))`.

To discover linked vCenter instances across an SSO domain dynamically, query the vCenter Deployment REST API endpoint - `GET /api/vcenter/deployment`.

Authentication Mechanisms

To perform operations in the vSphere environment, principals must authenticate to the VMware Cloud Foundation services. You can use the default vCenter Single Sign-On identity source or federate your vCenter to an external identity provider such as Microsoft Active Directory Federation Services, Okta, or Azure AD.

As a result from authenticating to the VMware Cloud Foundation services, client applications and users receive a session identifier. You can choose from several supported authentication mechanisms to obtain a session identifier that grants you access to the VMware Cloud Foundation services in the virtual environment.

Choosing Your Identity Source

First, you must decide whether to authenticate to vCenter by using the default identity provider, vCenter Single Sign-On, or leverage your enterprise configuration to an external identity provider.

Default or External Identity Provider

vCenter acts as the default identity provider, by using its built-in vCenter Single Sign-On service to authenticate users and service accounts.

In vSphere 7.0 and later, you can delegate vSphere identity management to an external identity provider and take advantage of increased security and multifactor authentication. In this scenario, vCenter Single Sign-On is replaced by an external identity provider as the entity that performs principals' authentication.

Identity provider federation enhances the security and compliance of your applications and provides access to flexible benefits such as multifactor authentication (MFA) and automatic account synchronization.

You can federate vCenter authentication to:

- AD FS (starting in vSphere 7.0)
- Okta (starting in vSphere 8.0 Update 1)
- Azure AD (starting in vSphere 8.0 Update 2)

For more information, see Identity Provider Federation for vCenter.

Basic or Token-Based Authentication

You can use basic and token-based authentication to authenticate principals to the VMware Cloud Foundation services. VMware encourages you to use token-based authentication as it provides more security and flexibility for your applications and users.

Basic Authentication

Basic authentication passes a user name and password to vCenter.

By default, principals can use the vCenter Single Sign-On user name and password to connect to the VMware Cloud Foundation endpoint. The VMware Cloud Foundation endpoint checks whether the user name and password are present in the VMware Directory Service. On success, the system returns a session identifier for the VMware Cloud Foundation endpoint.

If your vCenter is federated to an external identity provider, you can use basic authentication through the OAuth 2.0 Password grant type.

> **Important:**
>
> VMware encourages you to refrain from using basic authentication and adopt token-based authentication, as it is more secure and provides more flexibility.

SAML Token-Based Authentication

Token-based authentication involves the use of an encrypted token that provides authentication and authorization data to the VMware Cloud Foundation endpoint. The vCenter token complies with the Security Assertion Markup Language (SAML) specification, an XML-based schema for communicating authentication data.

To acquire a SAML token, client applications issue a token request to vCenter Single Sign-On. Client applications then send the SAML token to the VMware Cloud Foundation API endpoint in exchange for a session identifier.

Starting with vSphere 7.0, you can use token-based authentication for your federated vCenter through a 3-step process:

1. Principals use the various OAuth 2.0 grant types with the external identity provider to obtain identity and access tokens in JSON Web Token (JWT) format.
2. Principals exchange the JWT tokens for a vCenter Single Sign-On SAML token.
3. Principals use the SAML token to obtain a session identifier from the VMware Cloud Foundation endpoint.

Authentication Terminology

To use the vSphere programming features effectively, you must understand a set of specific authentication terms and concepts.

Table 1880: vCenter Single Sign-On Glossary

Term	Definition
Principal	An entity that can be authenticated, such as a user.
Identity Provider	A service that manages identity sources and authenticates principals. Examples: Microsoft Active Directory Federation Services (AD FS) and vCenter Single Sign-On.
Identity Source (Directory Service)	Stores and manages principals. Principals consist of a collection of attributes about a user or a service account such as name, address, email, and group membership. Examples: Microsoft Active Directory and VMware Directory Service (vmdir).
Authentication	The means of determining whether someone or something is, in fact, who or what it declares itself to be. For example, users are authenticated when they provide their credentials, such as smart cards, user name and correct password, and so on.

Term	Definition
Authorization	The process of verifying what objects principals have access to.
Token	A signed collection of data comprising the identity information for a given principal. A token might include not only basic information about the principal such as email address and full name, but also, depending on the token type, the principal's groups and roles.
vmdir	VMware Directory Service. The internal (local) LDAP repository in vCenter that contains user identities, groups, and configuration data.
OAuth 2.0	An open authorization standard that enables the exchange of information among principals and web services without exposing principals' credentials.
OpenID Connect (OIDC)	Authentication protocol based on OAuth 2.0 that augments OAuth with user-identifying information. It is represented by the ID token that the authorization server returns together with the access token during OAuth authentication. vCenter uses OIDC capabilities when interacting with Active Directory Federation Services (AD FS), Okta, Microsoft Entra ID, and PingFederate.
System for Cross-domain Identity Management (SCIM)	The standard for automating the exchange of user identity information between identity domains or IT systems.
VMware Identity Services	Starting in version 8.0 Update 1, VMware Identity Services is a built-in container within vCenter that you can use for identity federation to external identity providers. It serves as an independent identity broker within vCenter and comes with its own set of APIs. Currently, VMware Identity Services supports Okta, Microsoft Entra ID, and PingFederate as external identity providers.
Tenant	A VMware Identity Services concept. A tenant provides a logical separation of data from other tenants' data in the same virtual environment.
JSON Web Token (JWT)	A token format defined by the OAuth 2.0 specification. A JWT token carries authentication and authorization information about a principal.
Relying party	A relying party "relies" on the authorization server, VMware Identity Services, or AD FS, for identity management. For example, through federation, vCenter establishes relying party trust to VMware Identity Services or AD FS.
Security Assertion Markup Language (SAML)	An XML-based open standard for exchanging authentication and authorization data between parties that is used by vCenter. Principals obtain a SAML token from vCenter Single Sign-On and then send it to the vSphere Automation API endpoint for a session identifier.

Create a VMware Cloud Foundation Session with User Credentials

You can use your vCenter Single Sign-On credentials with basic authentication to establish a session with a vCenter system.

- Retrieve the VMware Cloud Foundation endpoint URL from the Lookup Service.
- Verify that you have valid user credentials for the vCenter Single Sign-On identity store.

The single-factor basic authentication is the default method to authenticate to vCenter. You connect to the VMware Cloud Foundation SDKs with your vCenter Single Sign-On user name and password. The VMware Cloud Foundation endpoint checks whether the user credentials are present in vmdir, a vCenter Single Sign-On component. On success, the system returns a session identifier that you can use for subsequent API calls.

1. Create a connection stub by specifying the VMware Cloud Foundation endpoint URL and the message protocol to be used for the connection.
2. Create a stub configuration instance and set the specific security context to be used.

   The security context object uses the user name and password that are used for authenticating to the vCenter Single Sign-On service.
3. Create a `Session` stub that uses the stub configuration instance.
4. Call the create operation on the `Session` stub to create an authenticated session to the VMware Cloud Foundation endpoint.

   The operation returns a session identifier.
5. Create a security context instance and add the session ID to it.
6. Update the stub configuration instance with the session security context.

   Important:

   VMware encourages you to refrain from using basic authentication and adopt token-based authentication, as it is more secure and provides more flexibility.

You can use the authenticated session to access VMware Cloud Foundation services. For more information about creating stubs to the VMware Cloud Foundation services, see Accessing VMware Cloud Foundation Services.

Java Example of Creating a VMware Cloud Foundation API Session with User Credentials
This example is based on the code in the `VapiAuthenticationHelper.java` sample.

This example uses the steps that are described in the Create a VMware Cloud Foundation Session with User Credentials procedure

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

this.stubFactory = createApiStubFactory(server, httpConfig);

// Create a security context for username/password authentication
SecurityContext securityContext =
        SecurityContextFactory.createUserPassSecurityContext(
            username, password.toCharArray());

// Create a stub configuration with username/password security context
StubConfiguration stubConfig = new StubConfiguration(securityContext);

// Create a session stub using the stub configuration.
Session session =
        this.stubFactory.createStub(Session.class, stubConfig);

// Login and create a session
char[] sessionId = session.create();

// Initialize a session security context from the generated session id
SessionSecurityContext sessionSecurityContext =
        new SessionSecurityContext(sessionId);

// Update the stub configuration to use the session id
stubConfig.setSecurityContext(sessionSecurityContext);

/*
```

```
 * Create a stub for the session service using the authenticated
 * session
 */
this.sessionSvc =
        this.stubFactory.createStub(Session.class, stubConfig);

VM vmService = this.stubFactory.createStub(VM.class, stubConfig);
```

Python Example of Creating a VMware Cloud Foundation Session with Username and Password
This example illustrates how to create an authenticated session to the VMware Cloud Foundation API by using basic authentication in Python.

```
import requests
import urllib3
from vmware.vapi.vsphere.client import create_vsphere_client
session = requests.session()
session.verify = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
vsphere_client = create_vsphere_client(hostname, username, password, session=session)
# Interface factory object:
stub_config = vsphere_client._stub_config
```

Retrieve a SAML Token

The vCenter Single Sign-On service provides authentication mechanisms for securing the operations that your client application performs in the virtual environment. Client applications use SAML security tokens for authentication.

Verify that you have the vCenter Single Sign-On URL. You can use the Lookup Service on vCenter to obtain the endpoint URL. For information about retrieving service endpoints, see Connecting to Service Endpoints.

Client applications use the vCenter Single Sign-On service to retrieve SAML tokens. For more information about how to acquire a SAML security token, see the *vCenter Single Sign-On Programming Guide* documentation.

1. Create a connection object to communicate with the vCenter Single Sign-On service.
   Pass the vCenter Single Sign-On endpoint URL, which you can get from the Lookup Service.
2. Issue a security token request by sending valid user credentials to the vCenter Single Sign-On service on vCenter.

The vCenter Single Sign-On service returns a SAML token.

You can present the SAML token to the VMware Cloud Foundation API endpoint or other endpoints, such as the vSphere Web Services endpoint. The endpoint returns a session ID and establishes a persistent session with that endpoint. Each endpoint that you connect to uses your SAML token to create its own session.

Java Example of Retrieving a SAML Token
The example is based on the code in the `ExternalPscSsoWorkflow.java` sample.

This example uses the steps that are described in the Retrieve a SAML Token procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

LookupServiceHelper lookupServiceHelper = new LookupServiceHelper(
            this.lookupServiceUrl);

System.out.println("\nStep 2: Discover the Single Sign-On service URL"
                   + " from lookup service.");
String ssoUrl = lookupServiceHelper.findSsoUrl();
```

```
System.out.println("\nStep 3: Connect to the Single Sign-On URL and "
                   + "retrieve the SAML bearer token.");
SamlToken samlBearerToken = SsoHelper.getSamlBearerToken(ssoUrl,
    username,
    password);
```

Python Example of Retrieving a SAML Token

This example is based on the code in the `external_psc_sso_workflow.py` sample file.

This example uses the steps that are described in the Retrieve a SAML Token procedure.

This example uses the following global variables.

- *my_vapi_hostname*
- *my_sso_username*
- *my_sso_password*

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
from vsphere.samples.common import sso

# Use the SsoAuthenticator utility class to retrieve
# a bearer SAML token from the vCenter Single Sign-On service.
sso_url = 'https://' + my_vapi_hostname + ':7444/ims/STSService'
authenticator = sso.SsoAuthenticator(sso_url)
saml_token = authenticator.get_bearer_saml_assertion(my_sso_username,
                                                     my_sso_password,
                                                     delegatable=True)
```

Create a VMware Cloud Foundation Session with a SAML Token

To establish a VMware Cloud Foundation session, you create a connection to the VMware Cloud Foundation API endpoint and then you authenticate with a SAML token to create a session for the connection.

- Retrieve the VMware Cloud Foundation endpoint URL from the Lookup Service.
- Obtain a SAML token from the vCenter Single Sign-On service.

1. Create a connection by specifying the VMware Cloud Foundation API endpoint URL and the message protocol to be used for the connection.

   > **Note:**
   > To transmit your requests securely, use `https` for the VMware Cloud Foundation API endpoint URL.

2. Create the request options or stub configuration and set the security context to be used.

   The security context object contains the SAML token retrieved from the vCenter Single Sign-On service. Optionally, the security context might contain the private key of the client application.

3. Create an interface stub or a REST path that uses the stub configuration instance.

   The interface stub corresponds to the interface containing the method to be invoked.

4. Invoke the session `create` method.

   The service creates an authenticated session and returns a session identification cookie to the client.

5. Create a security context instance and add the session ID to it.

6. Update the stub configuration instance with the session security context.

Use the updated stub configuration with the session ID to create a stub for the interface that you want to use. Method calls on the new stub use the session ID to authenticate.

Java Example of Creating a VMware Cloud Foundation API Session with a SAML Token

This example is based on the code in the `ExternalPscSsoWorkflow.java` sample.

This example uses the steps that are described in the Create a VMware Cloud Foundation Session with a SAML Token

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

this.stubFactory = createApiStubFactory(server, httpConfig);

// Create a SAML security context using SAML bearer token
SecurityContext samlSecurityContext =
        SecurityContextFactory.createSamlSecurityContext(
            samlBearerToken, null);

// Create a stub configuration with SAML security context
StubConfiguration stubConfig =
        new StubConfiguration(samlSecurityContext);

// Create a session stub using the stub configuration.
Session session =
        this.stubFactory.createStub(Session.class, stubConfig);

// Log in and create a session
char[] sessionId = session.create();

// Initialize a session security context from the generated session id
SessionSecurityContext sessionSecurityContext =
        new SessionSecurityContext(sessionId);

// Update the stub configuration to use the session id
stubConfig.setSecurityContext(sessionSecurityContext);

/*
 * Create a stub for the session service using the authenticated
 * session
 */
this.sessionSvc =
        this.stubFactory.createStub(Session.class, stubConfig);

VM vmService = this.stubFactory.createStub(VM.class, stubConfig);
```

Python Example of Creating a VMware Cloud Foundation API Session with a SAML Token

This example is based on code in the `external_psc_sso_workflow.py` sample file.

This example uses the steps that are described in the Create a VMware Cloud Foundation Session with a SAML Token

This example uses the following global variables.

- *my_vapi_hostname*
- *my_stub_config*
- *saml_token*

The example assumes that you previously obtained a VMware Cloud Foundation API URL from the Lookup Service, and a SAML token from the vCenter Single Sign-On Service.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a session object in the client.
session = requests.Session()

# For development environment only, suppress server certificate checking.
session.verify = False
from requests.packages.urllib3 import disable_warnings
from requests.packages.urllib3.exceptions import InsecureRequestWarning
disable_warnings(InsecureRequestWarning)

# Create a connection for the session.
vapi_url = 'https://' + my_vapi_hostname + '/api'
connector = get_requests_connector(session=session, url=vapi_url)

# Add SAML token security context to the connector.
saml_token_context = create_saml_bearer_security_context(saml_token)
connector.set_security_context(saml_token_context)

# Create a stub configuration by using the SAML token security context.
my_stub_config = StubConfigurationFactory.new_std_configuration(connector)

# Create a Session stub with SAML token security context.
session_stub = Session(my_stub_config)

# Use the create operation to create an authenticated session.
session_id = session_stub.create()

# Create a session ID security context.
session_id_context = create_session_security_context(session_id)

# Update the stub configuration with the session ID security context.
my_stub_config.connector.set_security_context(session_id_context)
```

Create a Web Services Session

To develop a complex workflow, you might need to send requests to vSphere Web Services running in your virtual environment. To achieve this, you access the vSphere Web Services API by using the Web Services endpoint.

- Retrieve the vSphere Web Services endpoint URL from the Lookup Service.
- Obtain a SAML token from the vCenter Single Sign-On service.

The vSphere Web Services API also supports session-based access. To establish an authenticated session, you can send the SAML token retrieved from the vCenter Single Sign-On service to a vSphere Web Service. In return, you receive a session identifier that you can use to access the service. For more information about accessing Web Services and additional examples, see the *vSphere Web Services SDK Programming Guide* documentation.

The vSphere Automation SDK for Python supports a simplified way of creating connections to the Web Services API by using the `pyVim` library.

1. Connect to the vSphere Web Services endpoint.
2. Send the SAML token to a specific Web service to create an authenticated session.
3. Add the retrieved session ID to the service content object.

   The Service Content object gives you access to several server-side managed objects that represent vSphere services and components.

Java Example of Creating a vSphere Web Services Session

This example is based on the code in the `VapiAuthenticationHelper.java` and `VimUtil.java` samples.

This example uses the steps that are described in the Create a Web Services Session procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
... // Log in to the vSphere Web Services endpoint and retrieve a session identifier.
 SamlTokenElement tokenElement = ssoConnection.getSamlBearerTokenElement(); String sessionId =
 LoginByTokenSample.LoginUsingSAMLToken(tokenElement, vimUrl, null, null); // Use the VimPortType
 and VimService objects from // the vSphere Web Services API for accessing Web Services and //
 retrieve the request context. VimService vimService = new VimService(); vimPortType vimPort =
 vimService.getVimPort(); // Add the retrieved session ID to the request context. Map<String, Object> ctxt
 = ((BindingProvider) vimPort).getRequestContext(); ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY,
 vimUrl); ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true); Map<String, List<String>> headers
 = (Map<String, List<String>>) ctxt.getMessageContext.HTTP_REQUEST_HEADERS); if (headers == null)
 { headers = new HashMap<String, List<String>>(); } headers.put("Cookie", Arrays.asList(vcSessionId);
 ctxt.put(MessageContext.HTTP_REQUEST_HEADERS, headers); // Use the session ID context when retrieving the
 ServiceContent object. // The ServiceContent object gives you access to a number of // server-side managed
 objects that represent vSphere services and components. // For more information about the vSphere Web
 Services, // see the vSphere Web Services SDK Programming Guide documentation.
ServiceContent serviceContent = VimUtil.getServiceContent(vimPort);
```

Python Example of Creating a Web Services Session

This example is based on code in the `service_manager.py` sample file.

This example uses the steps that are described in the Create a Web Services Session procedure.

This example uses the following global variables.

- *my_ws_url*
- *my_sso_username*
- *my_sso_password*

The *my_ws_url* variable represents the URL of the vCenter Web Services API endpoint. You can retrieve the endpoint URL from the Lookup Service.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Extract the hostname from the endpoint URL.
url_scheme, url_host, url_path, url_params, url_query, url_fragment = \
      urlparse(my_ws_url)
pattern = '(?P<host>[^:/ ]+).?(?P<port>[0-9]*)'
match = re.search(pattern, url_host)
host_name = match.group('host')

# Invoke the SmartConnect() method by supplying
# the host name, user name, and password.
```

```
service_instance_stub = SmartConnect(host=host_name,
                                     user=my_sso_username,
                                     pwd=my_sso_password)

# Retrieve the service content.
service_content = service_instance_stub.RetrieveContent()
```

Identity Provider Federation for vCenter

Starting with vSphere 7.0, you can replace through federation your built-in vCenter identity source with an external identity source. Depending on your vCenter version, you can use external identity providers such as AD FS, Azure AD, or Okta.

With identity federation, you can use the same corporate identity source for your vCenter that you use for your other federated desktop and cloud applications.

vCenter Identity Provider Federation Basics

In vSphere 7.0 and later, vCenter supports federated authentication. In this scenario, when a user logs in to vCenter, vCenter redirects the user login to the external identity provider. The user credentials are no longer provided to vCenter directly. Instead, the user provides credentials to the external identity provider. vCenter trusts the external identity provider to perform the authentication. In the federation model, users never provide credentials directly to any service or application but only to the identity provider. As a result, you "federate" your applications and services, such as vCenter, with your identity provider.

Why Is Identity Provider Federation Useful

Federating vCenter to an enterprise identity provider alleviates the burden of identity management and provides flexible options such as multifactor authentication (MFA), automatic registration and termination of users across services, and many more. Identity provider federation uses token-based authentication and minimizes the risk of bad actors acquiring protected credentials such as user names and passwords. Identity provider federation also helps your organization with compliance as various international standards require MFA to guarantee data security. In addition, with identity provider federation, you can automate vCenter user management because you utilize the users and groups from your enterprise identity source.

vCenter Server External Identity Provider Support

vCenter supports the following external identity providers:

- AD FS (vSphere 7.0 and later)
- Okta (vSphere 8.0 Update 1 and later)
- Microsoft Entra ID, formerly called Azure AD (vSphere 8.0 Update 2 and later)
- PingFederate (starting in vSphere 8.0 Update 3)

Federation to Microsoft Active Directory Federation Services (AD FS)

In vSphere 7.0 and later, you can activate identity federation to Microsoft Active Directory Federation Services (AD FS). In this scenario, vCenter federates directly to the enterprise identity provider, AD FS, without the use of an authentication intermediary service.

You can configure federation to AD FS with the help of the VMware Cloud Foundation API. For more information, see Federate vCenter to Microsoft Active Directory Federation Services (AD FS).

Federation to Okta and Azure AD Through VMware Identity Broker - vCenter

Starting from vSphere 8.0 Update 1, you can activate federation to Okta as the identity provider. Starting from vSphere 8.0 Update 2, you can activate federation to Azure AD as the identity provider. Both configurations use VMware Identity Broker - vCenter, an authentication intermediary that functions as a built-in container within vCenter. With VMware Identity Broker - vCenter, you can configure principals to authenticate to vCenter by using an external identity provider.

vCenter can support the default `vsphere.local` identity source and only one configured external identity provider. You cannot use multiple external identity providers.

Once you set up your federated authentication, you can use OAuth for login to vCenter.

> **Note:**
>
> You can federate vCenter to Okta or Azure AD by using the vSphere Client. For more information, see `Configure vCenter Identity Provider Federation for Okta` and `Configure vCenter Identity Provider Federation for Microsoft Entra ID` in the *vSphere Authentication Guide*.

Authentication to a Federated vCenter

Once you have configured vCenter to point to AD FS, Okta, or Azure AD, you can use the VMware Cloud Foundation and the VMware Identity Broker - vCenter APIs to authenticate principals to your vCenter. Depending on the type of user or application you want to authenticate, you can choose among different token-based authentication options. vSphere supports the various OAuth 2.0 grant types.

Once you have configured vCenter to point to AD FS, you can use the VMware Cloud Foundation API to authenticate principals to your vCenter. Depending on the type of user or application you want to authenticate, you can choose among different token-based authentication options. vSphere supports the various OAuth 2.0 grant types.

What is OAuth 2.0

OAuth 2.0 is an open authorization framework that allows information exchanges among web services without exposing sensitive data such as user names and passwords. OAuth 2.0 boosts security and prevents malicious actors from stealing users' credentials over the Internet. OAuth 2.0 is an open framework and can be customized according to the organizational requirements.

You can use different OAuth 2.0 grant types to authenticate your applications to vCenter. The OAuth grant types represent different methods by which a principal can obtain an access token from an authorization server. In the vSphere implementation, the access token is used to obtain a SAML token and authenticate principals to vCenter.

OAuth 2.0 Grant Types

Password

In the Password grant type, the client application exchanges the user's credentials, user name and password, for an access token, and optionally an ID token (JWT tokens), from the authorization server. OAuth 2.0 has been designed to prevent the exchange of user credentials, so the Password grant type must be avoided whenever possible, especially for third-party application authentication. The latest *OAuth 2.0 Security Best Current Practice* recommends avoiding the Password grant type altogether. As a single-factor method, the Password grant type is incompatible with the requirements of multifactor authentication (MFA). For more information, see Authenticate to an AD FS-Federated vCenter by Using the Password Grant Type.

> **Important:**
> The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.

Authorization Code

In the Authorization Code grant type, the client application uses an authorization code to obtain JWT tokens from the authorization server. This grant type requires a user to verify an authentication request through a web browser. The Authorization Code grant type is compatible with multi-factor authentication and guarantees a substantial degree of security. This grant type is appropriate for regular web applications running on a server. For more information, see Authenticate to an AD FS - Federated vCenter by Using the Authorization Code Grant Type.

Client Credentials

The Client Credentials grant type permits web services to authenticate to other web services without the involvement of a user. This grant type is useful for applications that are resource owners and can take advantage of headless, or backend automation, when a physical user is not present. For example, a client application might use this grant type to authenticate and fetch details about a virtual environment.

Refresh Token

The Refresh Token grant type is used by clients to exchange an expiring access token for a refresh token that can be used to receive a new access token. With this grant type, a client application can fetch a valid access token without asking the user to log in again. It allows client applications to provide a seamless user experience without further interaction with the user after timeouts.

What Is OpenID Connect (OIDC)

The vSphere implementation of OAuth 2.0 uses the OpenID Connect (OIDC) authentication protocol. With OIDC, the authorization server returns an ID token containing data about who the users are and what permissions they have on the resource server. As a result, vCenter can identify users and grant them the respective permissions that are defined on the authorization server.

AD FS Federation Workflows

You can federate your vCenter to Microsoft Active Directory Federation Services (AD FS) and use the OAuth 2.0 grant types to authenticate.

Federate vCenter to Microsoft Active Directory Federation Services (AD FS)

You can federate vCenter to Microsoft Active Directory Federation Services (AD FS) as an external identity provider by using the vCenter `Identity Providers` service.

> **Note:**
> This process of configuring an AD FS identity provider requires that you have administrative access to both your vCenter and your AD FS server. During the configuration process, you enter information first in your vCenter, then in your AD FS server, then in your vCenter.

Active Directory Federation Services requirements:

- AD FS for Windows Server 2016 or later must already be deployed.
- AD FS must be connected to Active Directory.
- An Application Group for vCenter must be created in AD FS as part of the configuration process. See the VMware knowledge base article at https://kb.vmware.com/s/article/78029.
- An AD FS server certificate (or a CA or intermediate certificate that signed the AD FS server certificate) that you add to the Trusted Root Certificates Store.
- You have created a vCenter administrators group in AD FS that contains the users you want to grant vCenter administrator privileges to.

For more information about configuring AD FS, see the Microsoft documentation.

vCenter and other requirements:

- vSphere 7.0 or later
- vCenter must be able to connect to the AD FS discovery endpoint, and the authorization, token, logout, JWKS, and any other endpoints advertised in the discovery endpoint metadata.
- You need the **VcIdentityProviders** > **Manage** privilege to create, update, or delete a vCenter Identity Provider that is required for federated authentication. To limit a user to view the Identity Provider configuration information only, assign the **VcIdentityProviders** > **Read** privilege.

1. Authenticate to the VMware Cloud Foundation API endpoint and establish a session.
2. Add your AD FS root CA certificate to the Trusted Root Certificates Store.

   See Managing Certificates.
3. Create a `ProvidersTypes.OidcCreateSpec` object by using the Application Group configuration from AD FS.

Method	Description
`setDiscoveryEndpoint`	The OpenID address of the AD FS server.
`setClientId`	The client identifier of the AD FS Application Group.
`setClientSecret`	The secret shared between the client and the provider.
`setClaimMap`	This parameter is required but not applicable to AD FS. Use an empty array `[]`.

4. Create an object of type `ProvidersTypes.ActiveDirectoryOverLdap`.

Method	Description
`setUserName`	The user name of a user in the domain who has a minimum of read-only access to base Distinguished Name (DN) for users and groups.
`setPassword`	The password of a user in the domain who has a minimum of read-only access to base DN for users and groups.
`setUsersBaseDn`	The base DN for users in the Active Directory environment connected to AD FS that you want to be able to federate with vCenter.
`setGroupsBaseDn`	The base DN for groups in the Active Directory environment connected to AD FS that you want to be able to federate with vCenter.
`setServerEndpoints`	Active directory server endpoints. At least one Active Directory server endpoint must be set. Use the format `ldap://<hostname>:<port>` or `ldaps://<hostname>:<port>`. The port is typically 389 for LDAP connections and 636 for LDAPS connections. For Active Directory multi-domain controller deployments, the port is typically 3268 for LDAP and 3269 for LDAPS.
`setCertChain`	The SSL certificate chain in base64 encoding. You can skip this parameter only if all the Active Directory server endpoints use the LDAP (and not the LDAPS) protocol.

5. Add the identity provider by using the `Identity ProvidersIdentity Providers` service.
   a) Create an object of type `ProvidersTypes.CreateSpec`.

Method	Description
`setConfigTag`	The configuration type of the identity provider. The possible values are *`Oauth2`* and *`Oidc`*. For AD FS federation, use *`Oidc`*.
`setName`	The user-friendly name for the identity provider. You must use the exact string *`Microsoft ADFS`* for proper configuration.
`setUpnClaim`	The name of the claim in the AD FS JWT token that contains the user principal name of the user that is logging in. You must use the same value that you used when you set up the AD FS Application Group. The procedure from the article in the prerequisites uses *`upn`*. If unset, the default value is *`acct`*.
`setGroupsClaim`	The name of the claim in the AD FS JWT token that contains the group membership of the user that is logging in. You must use the same value that you used when you set up the AD FS Application Group. The procedure from the article in the prerequisites uses *`group`*. If unset, the groups for the subject consist of the groups in `group_names` and `group_ids` claims.
`setIsDefault`	Set to *`true`*. Specifies whether the provider is the default provider. Setting to *`true`* makes all other providers non-default. If unset: • In case it is the first created provider, it is set as the default provider. • In case it is not the first created provider, it is not set as the default provider.
`setOidc`	Use the `ProvidersTypes.OidcCreateSpec` object.

Method	Description
`setIdmProtocol`	The communication protocol used to connect to AD FS to search for users and groups when assigning permissions in vCenter. You must use *`LDAP`*. If unset, no communication protocol is configured for the users and groups search.
`setActiveDirectoryOverLdap`	Use the `ProvidersTypes.ActiveDirectoryOverLdap` object.

b) To add the provider, call the `create(ProvidersTypes.CreateSpec)` method.

The operation returns the ID of the provider you added.

6. Configure vCenter permissions for Active Directory users or groups in your AD FS environment.

   You can do this in two ways:

   - Add a user from your AD FS environment to a group in vCenter.
   - Configure Global Permissions for an AD FS user.

   Note:

   In vSphere 8.0 and later, you cannot configure permissions through the VMware Cloud Foundation API. Instead, you use either the vSphere Client or the vSphere Web Services API. For more information, see the *vSphere Authentication Guide* or the *vSphere Web Services SDK Programming Guide*.

7. Optional: Copy the two redirect URIs from the Identity Provider Configuration page in the vSphere Client and add them to your AD FS Application Group.

   Note:

   You must do this step to enable logging in to vCenter through AD FS by using the vSphere Client.

You configured vCenter to use AD FS as the identity provider.

Authenticate to an AD FS-Federated vCenter by Using the Password Grant Type

If your vCenter is federated to AD FS, you can authenticate with the OAuth 2.0 Password grant type.

- Verify that your vCenter is federated to AD FS. For more information, see *Federate vCenter to Active Directory Federation Services (AD FS).*
- You must have an AD FS account and user credentials with the necessary permissions to view and manage vCenter.
- You must register an OAuth client for your application on the authorization server (AD FS).

You can use the Password grant type to exchange user credentials for an access token and an ID token from the authorization server. From the user's perspective, the password grant type functions exactly as basic authentication with the only difference that you use your AD FS, and not your local vCenter Single Sign-On credentials.

The Password grant type is possible among native apps but is not recommended for authentication to third-party apps. Modern security best practices require that primary user credentials such as passwords do not leave the native API environment. Therefore, it is more secure to use other OAuth 2.0 grant types for authentication to third-party apps.

To use the Password grant type with AD FS, you provide your AD FS user name and password to the `Common Infrastructure Services (CIS) Session` endpoint.

In the background, the client application calls the authorization server (AD FS) and obtains an access token and an ID token in JWT format. vCenter Single Sign-On converts the JWT tokens into a SAML token which is used to obtain a session identifier and authenticate your application to vCenter.

Important:
The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.

1. Call the `Session` interface with the base-64 encoded value of your AD FS user name and password separated by a colon (`username:password`) in the authorization header.

```
POST https://<vcenter_server_ip_address_or_fqdn>/api/session
```

```
Authorization: Basic <username:password>
```

On success (status code 201), the system returns an authentication session identifier.

2. To authenticate subsequent API calls, pass the session identifier as a request header.

```
vmware-api-session-id: <session_ID>
```

Java Example of Creating a VMware Cloud Foundation API Session with User Credentials
This example is based on the code in the `VapiAuthenticationHelper.java` sample.

This example uses the steps that are described in the Create a VMware Cloud Foundation Session with User Credentials procedure

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

this.stubFactory = createApiStubFactory(server, httpConfig);

// Create a security context for username/password authentication
SecurityContext securityContext =
        SecurityContextFactory.createUserPassSecurityContext(
            username, password.toCharArray());

// Create a stub configuration with username/password security context
StubConfiguration stubConfig = new StubConfiguration(securityContext);

// Create a session stub using the stub configuration.
Session session =
        this.stubFactory.createStub(Session.class, stubConfig);

// Login and create a session
char[] sessionId = session.create();

// Initialize a session security context from the generated session id
SessionSecurityContext sessionSecurityContext =
        new SessionSecurityContext(sessionId);

// Update the stub configuration to use the session id
stubConfig.setSecurityContext(sessionSecurityContext);

/*
 * Create a stub for the session service using the authenticated
 * session
 */
this.sessionSvc =
        this.stubFactory.createStub(Session.class, stubConfig);

VM vmService = this.stubFactory.createStub(VM.class, stubConfig);
```

Python Example of Creating a VMware Cloud Foundation Session with Username and Password
This example illustrates how to create an authenticated session to the VMware Cloud Foundation API by using basic authentication in Python.

```
import requests
import urllib3
from vmware.vapi.vsphere.client import create_vsphere_client
session = requests.session()
```

```
session.verify = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
vsphere_client = create_vsphere_client(hostname, username, password, session=session)
# Interface factory object:
stub_config = vsphere_client._stub_config
```

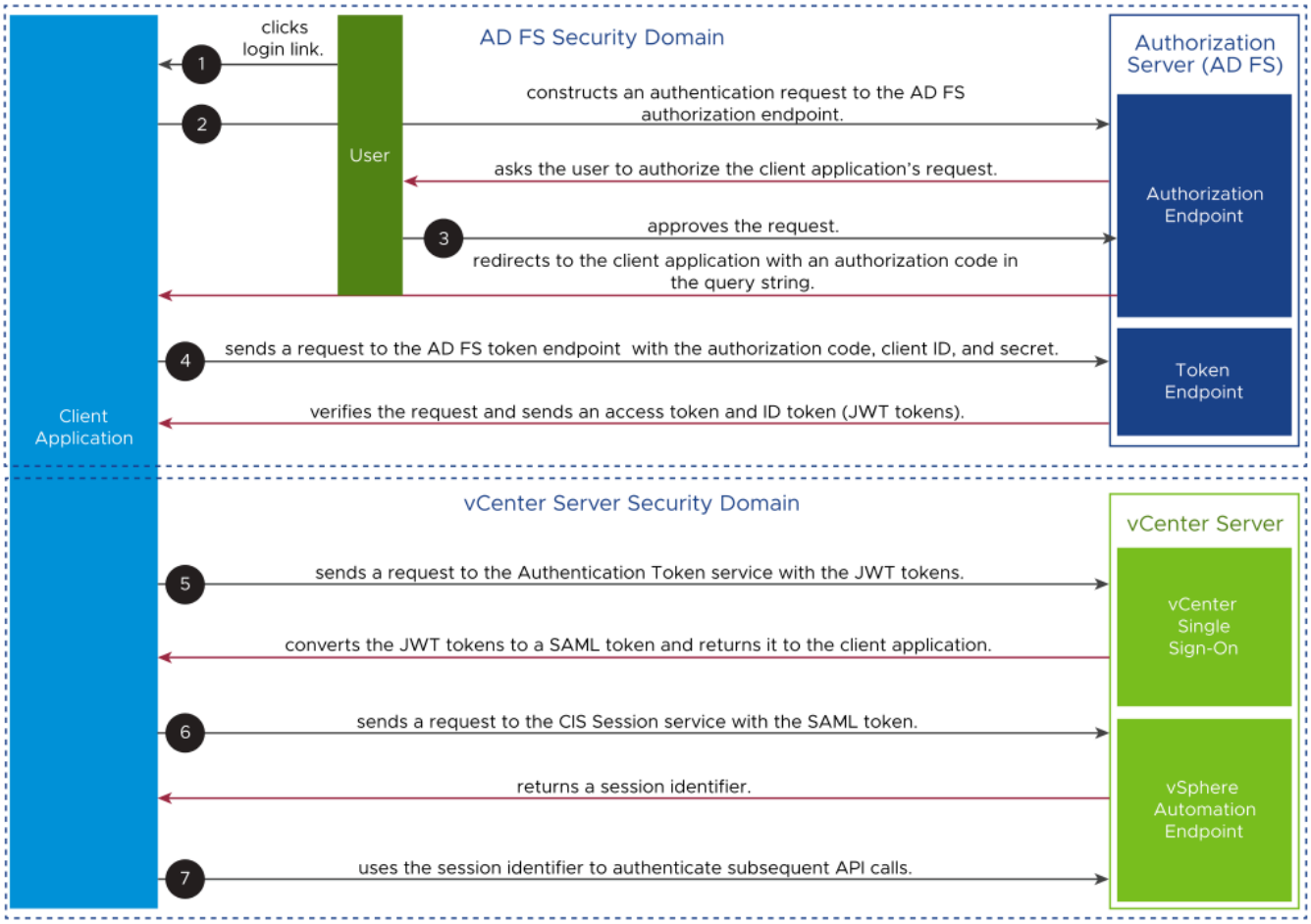
Authenticate to an AD FS-Federated vCenter by Using the Authorization Code Grant Type

If your vCenter is federated to an enterprise identity provider, such as Active Directory Federation Services (AD FS), you can authenticate your client applications by using the OAuth 2.0 Authorization Code grant type.

- Verify that your vCenter is federated to AD FS. For more information, see *Federate vCenter to Active Directory Federation Services (AD FS).*
- You must have an AD FS account and user credentials with the necessary permissions to view and manage vCenter.
- You must register an OAuth client for your application on the authorization server (AD FS).

You can use the Authorization Code grant type if you want to authenticate client apps to vCenter through multifactor authentication (MFA). This grant type is applicable for regular web applications that involve a human user interacting with a server.

Figure 566: Authentication to an AD FS-Federated vCenter Through the Authorization Code Grant Type



A principal authenticates to vCenter through the Authorization Code grant type as follows.

1. The user clicks **Login** within the client application.
2. The client application constructs a request to the AD FS authorization endpoint.

   To retrieve the authorization endpoint, you must query the AD FS discovery endpoint at, for example, `https://<your_adfs_server.com>/adfs/.well-known/openid-configuration`.

The client application's request must include the application's client ID and a redirect URI that tells the authorization server where to redirect the user after approving the request.

As a result, the authorization server opens a prompt page asking the user to authorize the client application's request.

3. The user confirms the request.

   The browser is redirected back to the client application with an authorization code in the query string.

4. The client application sends a request to the AD FS token endpoint with the authorization code, client ID, and secret.

   The authorization server verifies the request and sends an access token and an ID token (JWT tokens).

5. The client application sends the JWT tokens to vCenter Single Sign-On by using the `Token` interface of the `com.vmware.vcenter.authentication` packageAuthentication Token service.
   a) Create an object of type `TokenTypes.IssueSpec` with the following modifiers.

Method	Value
setSubjectTokenType	*urn:ietf:params:oauth:token-type:access_token*
setSubjectToken	*<ACCESS_TOKEN>*
setActorTokenType	*urn:ietf:params:oauth:token-type:id_token*
setActorToken	*<ID_TOKEN>*
setGrantType	*urn:ietf:params:oauth:grant-type:token-exchange*
setRequestedTokenType	*urn:ietf:params:oauth:token-type:saml2*

   b) Call the `issue(TokenTypes.IssueSpec)` method

   For example,

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/authentication/
token' \
--header 'Authorization: Bearer <ACCESS_TOKEN>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'subject_token_type=urn:ietf:params:oauth:token-type:access_token' \
--data-urlencode "subject_token=<ACCESS_TOKEN>" \
--data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
--data-urlencode "actor_token=<ID_TOKEN>" \
--data-urlencode 'grant_type=urn:ietf:params:oauth:grant-type:token-exchange' \
--data-urlencode 'requested_token_type=urn:ietf:params:oauth:token-type:saml2'
```

   vCenter Single Sign-On returns a SAML token.

6. The client application calls the `Common Infrastructure Services (CIS) Session` service.

   By using the `CIS Session` service, principals can manage session tokens - including creation and deletion of session identifiers, and obtaining information about sessions.

   a) Base-64 decode the SAML token, compress it, and then Base-64 encode it again.
   b) Call the `session.create()` methodSession service with the SAML token.

      For example,

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/session' \
--header 'Authorization: SIGN token=<COMP_TOKEN>'
```

   On success (status code 201), the VMware Cloud Foundation endpoint returns a session identifier in the response.

7. To authenticate, the client application uses the session identifier in subsequent API calls to the VMware Cloud Foundation API endpoints.

   For example,

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/cis/tagging/category' \
```

```
--header 'vmware-api-session-id: <session_ID>'
```

VMware Identity Broker - vCenter Workflows

You can federate your vCenter to an external identity provider through VMware Identity Broker - vCenter. It is a built-in container within vCenter that enables federation to an external identity provider such as Okta or Azure AD.

What Is VMware Identity Broker - vCenter

VMware Identity Broker - vCenter is VMware's specialized authentication solution that allows you to federate your apps to external identity providers such as Okta or Azure AD. VMware Identity Broker - vCenter functions as a container built-in within your vCenter. VMware Identity Broker - vCenter comes with its own VMware Identity Broker - vCenter API that is separate from the VMware Cloud Foundation API and the vSphere Web Services API.

You can use VMware Identity Broker - vCenter to federate to:

- Okta (starting in vSphere 8.0 Update 1)
- Azure AD (starting in vSphere 8.0 Update 2)

Configure vCenter Identity Provider Federation to Okta or Azure AD

To configure your vCenter to point to Okta or Azure AD as the identity provider, you must use the Okta or Azure AD interface and the vSphere Client. This integration uses VMware Identity Broker - vCenter. For more information, see Configure vCenter Identity Provider Federation for Okta and Configure vCenter Identity Provider Federation for Microsoft Entra ID from the *vSphere Authentication Guide*.

OAuth Authentication to Your Federated vCenter, App, or Script

Once configured, you can use the VMware Cloud Foundation and the VMware Identity Broker - vCenter APIs to authenticate to your federated vCenter, application, or script.

You can use the following OAuth 2.0 grant types:

- Password (not recommended)
- Authorization Code
- Client Credentials
- Refresh Token

Federate Apps or Scripts to an External Identity Provider by Using Your VMware Identity Broker - vCenter Configuration

Configure a client application or script for authentication by using an already existing vCenter identity provider configuration on VMware Identity Broker - vCenter .

Okta and Azure AD requirements:

- You are a customer of Okta or Microsoft and have a dedicated Okta domain space or an Azure AD account.
- You have identified the Okta or Azure AD users and groups that you want to share with your app.

vCenter and other requirements:

- vSphere 8.0 Update 1 or later, with the VMware Identity Broker - vCenter activated (they are activated by default).
- A vCenter that is federated to Okta or Azure AD. (You must know the IP address or the FQDN of the vCenter.)
- The users and groups from the identity provider are provisioned in your vCenter.
- You must know the tenant that you want to use for your client app or script. The default tenant that is created during vCenter federation is called *CUSTOMER*.
- You must have the **VcIdentityProviders** > **Manage** privilege.

You can leverage the identity provider configuration you created for vCenter for other apps or scripts that you want to federate to Okta or Azure AD.

For this automation scenario you must use services from the VMware Cloud Foundation API and the VMware Identity Broker - vCenter API. The VMware Identity Broker - vCenter API endpoints are located also on your vCenter but are taxonomically separated from the VMware Cloud Foundation API endpoints.

1. Get a session identifier for the VMware Cloud Foundation API.

   For more information see Authentication Mechanisms.

2. Get a tenant admin client token.

   To authenticate to the VMware Identity Broker - vCenter endpoints, you must obtain a tenant admin client token by using the `Tenant Admin Client` service of the VMware Cloud Foundation API.

   To authenticate to VMware Identity Broker - vCenter, you must obtain a tenant admin client token by calling the `get(tenant)` method of the `AdminClient` interface from the `com.vmware.vcenter.identity.broker.tenants` package.

   The default `tenant` that is created during the vCenter federation is called *`CUSTOMER`*. You can use this tenant for your app or script.

   For authentication, use the session identifier that you obtained in the previous step.

   Example request:

   ```
   curl --location --request GET 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/identity/broker/
   tenants/<tenant>/admin-client' \
   --header 'vmware-api-session-id: <session_id>' \
   --header 'Content-Type: application/json'
   ```

   In the response, you get an access token in JWT format that you use to authenticate to VMware Identity Broker - vCenter.

3. Create an OAuth 2.0 client for your app.

   To authenticate to the external identity provider, you must create an OAuth 2.0 client for your app on VMware Identity Broker - vCenter. You can do this by using the `Create Broker OAuth 2 Client` service of the VMware Identity Broker - vCenter API.

   For authentication, you must use the tenant admin client token obtained in the previous step.

   Example request:

   ```
   curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>
   /broker/oauth2-clients' \
   --header 'Content-Type: application/
   vnd.vmware.horizon.manager.accesscontrol.broker.oauth2client.with.rule.sets+json' \
   --header 'Accept: application/vnd.vmware.horizon.manager.accesscontrol.broker.oauth2client.with.rule.sets
   +json' \
   --header 'Authorization: HZN <my_admin_client_token>' \
   --data-raw '{
       "client_id": "<my_app_client_id>",
       "grant_types": [
           "refresh_token",
           "client_credentials",
           "password",
           "authorization_code"
       ],
       "scope": [
           "email",
           "profile",
           "user",
           "openid",
           "group"
       ],
       "secret": "<my_app_client_secret>",
       "redirect_uris": [
           "https://<my_app_redirect_uri1>",
           "https://<my_app_redirect_uri2>"
       ]
   }'
   ```

   On success (status code 201), the operation returns the data about the new OAuth 2.0 client.

Authenticate your app or script to the external identity provider by using the OAuth 2.0 grant types.

Obtain JWT Tokens from VMware Identity Broker - vCenter by Using the Password Grant Type

If your vCenter or other application is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access and ID token in JWT format by using the OAuth Password grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.
- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id, client secret, and the redirect URIs for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *CUSTOMER*.
- You must have an Okta or Azure AD account and user credentials with the necessary permissions to view and manage vCenter.

You can use the Password grant type to exchange user credentials for an access token and an ID token from the authorization server. From the user's perspective, the password grant type functions similarly to local domain authentication with the difference that principals use their external identity provider, and not their local vCenter Single Sign-On credentials, to authenticate.

The Password grant type involves the client collecting the principal's user name and password directly and exchanging them for an access token from the authorization server. From a security perspective, handling user credentials directly increases the risk of exposure and compromise. From a user experience perspective, users might be reluctant to expose their credentials to third-party apps, as it goes against the principle of separating authentication from authorization. Therefore, using the OAuth 2.0 Password grant type is not recommended.

> **Important:**
> The Password grant type is disallowed by the latest *OAuth 2.0 Security Best Current Practice*. The Password grant is excluded entirely from OAuth 2.1.

Obtain an access and ID token.

Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint with the required grant type, your app's client ID and secret, and the user name and password of a registered user.

In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (`client_id:secret`).

The content type must be *x-www-form-urlencoded*.

Example request:

```
curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'username=<my_user_name>' \
--data-urlencode 'password=<my_password>'
```

The authorization server returns the JWT tokens corresponding to the grant type: access token , ID token, and refresh token.

Exchange the JWT tokens for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Obtain JWT Tokens from VMware Identity Broker - vCenter by Using the Authorization Code Grant Type

If your vCenter or other application is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access and ID token in JWT format by using the OAuth 2.0 Authorization Code grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.

- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id, client secret, and the redirect URIs for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *CUSTOMER*.
- You must have an Okta or Azure AD account and user credentials with the necessary permissions to view and manage vCenter.

The Authorization Code grant type is appropriate for regular web applications that involve a human user interacting with a server, especially if you want to activate multifactor authentication (MFA). The Authorization Code grant type protects user credentials as they do not leave the API environment.

1. Obtain an authorization code.
   a) Construct a query request in the browser with the following parameters.

client_id	The client identifier you obtained when you registered the OAuth 2.0 client on VMware Identity Services.
redirect_uri	The callback endpoint in your application that must receive the authorization code. It must be one of the `redirect_uris` defined in your OAuth 2.0 client registration. When sending the `redirect_uri` as a URL parameter it must be URL-encoded.
state	A random string that your application generates and that is sent back as a parameter during the URI redirect. Serves as a security mechanism as per the OAuth 2.0 specification.
response_type	Specifies what must be included in the authorization response. For the authorization code grant type, use *code*.

   b) Send the query request to the VMware Identity Broker - vCenter`Authorize` endpoint.

      For example:

```
https://<vcenter_server_fqdn>/acs/t/<tenant>/authorize?
client_id=<my_app_client_id>&redirect_uri=https://
<my_app_redirect_uri1>&state=<generated_random_string>&response_type=code
```

      VMware Identity Broker - vCenter opens a dialog box asking the user to authenticate.
   c) The user authenticates to the identity provider.

      The authentication mechanism depends on the specific user configuration on the extern?l identity provider website. It could be basic authentication, biometrics, multifactor authentication, and so on.

      The authorization server directs the user to the client app's redirect URI with the authorization code in the query string.

2. Obtain an access and ID token.

   Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint containing the required grant type, the authorization code, the redirect URI, and your app's client ID and secret.

   In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (*my_app_client_id*:*my_app_client_secret*).

   The content type must be *x-www-form-urlencoded*.

   Example request:

```
curl -- location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=authorization_code' \
--data-urlencode 'code=<my_app_authorization_code>' \
--data-urlencode 'redirect_uri=https://<my_app_redirect_uri1>'
```

   The authorization server returns the JWT tokens corresponding to the grant type: access token , ID token, and refresh token.

Exchange the JWT tokens for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Obtain JWT Token from VMware Identity Broker - vCenter by Using the Client Credentials Grant Type

If your vCenter, app, or script is federated to an external identity provider, such as Okta or Azure AD, through VMware Identity Broker - vCenter, you can obtain an access token in JWT format by using the OAuth Client Credentials grant type.

- Verify that your vCenter or client app is federated to an external identity provider through VMware Identity Broker - vCenter.
- You must create an OAuth client for your vCenter or client app on VMware Identity Broker - vCenter. You must know the client id and secret for the OAuth client.
- You must know the tenant name for the configuration you want to use. The default tenant that is created during vCenter federation is called *CUSTOMER*.

You can use the Client Credentials grant type if you want to authenticate vCenter or another application to your external identity provider without the presence of a human user. The Client Credentials grant type is appropriate for headless automation when you want a web service to authenticate automatically against an identity provider.

Obtain an access token for your app.

Send a request to the VMware Identity Broker - vCenter`Get Access Token` endpoint with the required grant type.

In the authorization header, use the base-64 encoded value of your app's client ID and secret separated by a colon (`client_id:secret`).

The content type must be *x-www-form-urlencoded*.

Example request:

```
curl --location --request POST 'https://<vcenter_server_fqdn>/acs/t/<tenant>/token' \
--header 'Authorization: Basic <my_app_client_id>:<my_app_client_secret>' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=client_credentials'
```

The authorization server returns the JWT token corresponding to the grant type: access token.

Exchange the JWT token for a vCenter SAML token that you can use to establish an authentication session. For more information, see Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier.

Exchange JWT Tokens for a SAML Token and Obtain a Session Identifier

You can exchange the JWT tokens that you obtained from the authorization server for a vCenter SAML token and create an authenticated session.

- You must have a valid access and ID token (in JWT format) from the authorization server (AD FS or VMware Identity Broker - vCenter) for the user or application you want to authenticate.

1. Call the `Authentication Token` service with the JWT token(s) you obtained.

   For example,

   ```
   curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/vcenter/authentication/
   token' \
   --header 'Authorization: Bearer <ACCESS_TOKEN>' \
   --header 'Content-Type: application/x-www-form-urlencoded' \
   --data-urlencode 'subject_token_type=urn:ietf:params:oauth:token-type:access_token' \
   --data-urlencode "subject_token=<ACCESS_TOKEN>" \
   --data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
   --data-urlencode "actor_token=<ID_TOKEN>" \
   --data-urlencode 'grant_type=urn:ietf:params:oauth:grant-type:token-exchange' \
   --data-urlencode 'requested_token_type=urn:ietf:params:oauth:token-type:saml2'
   ```

   On success (code 200), vCenter Single Sign-On returns an `Oauth2TokenInfo` data structure containing the SAML token.

2. Send the JWT tokens to vCenter Single Sign-On by using the `Token` interface of the `com.vmware.vcenter.authentication` package.
   a) Create an object of type `TokenTypes.IssueSpec` with the following modifiers.

Method	Value
`setSubjectTokenType`	*urn:ietf:params:oauth:token-type:access_token*
`setSubjectToken`	<*ACCESS_TOKEN*>
`setActorTokenType`	*urn:ietf:params:oauth:token-type:id_token*
`setActorToken`	<*ID_TOKEN*>
`setGrantType`	*urn:ietf:params:oauth:grant-type:token-exchange*
`setRequestedTokenType`	*urn:ietf:params:oauth:token-type:saml2*

   b) Call the `issue(TokenTypes.IssueSpec)` method

   On success, vCenter Single Sign-On returns a `TokenInfo` object containing the SAML token.

3. Prepare the SAML token for the API call.

   Base-64 decode the SAML token, compress it, and then Base-64 encode it again.

4. To obtain a session identifier, call the `session.create()` methodSession service with the SAML token.

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/session' \
--header 'Authorization: SIGN token=<COMP_TOKEN>'
```

   On success (code 201), the VMware Cloud Foundation endpoint returns a session identifier in the response. On success, the VMware Cloud Foundation API returns a session identifier.

5. To authenticate, use the session identifier in subsequent API calls to the VMware Cloud Foundation API endpoints. To authenticate, add the session identifier to your security context instance.

   For example:

```
curl --location --request POST 'https://<vcenter_server_ip_address_or_fqdn>/api/cis/tagging/category' \
--header 'vmware-api-session-id: <session_ID>'
```

You created an authenticated session and now you can access and use the VMware Cloud Foundation services.

Bash Shell

This Bash script illustrates the use case where you get JWT tokens from the authorization server (AD FS or VMware Identity Broker - vCenter), exchange them for a vCenter SAML token, and obtain an authentication session identifier from the VMware Cloud Foundation endpoint.

This script consists of three parts:

1. Obtain JWT tokens from the authorization server (AD FS or VMware Identity Broker - vCenter) by using the OAuth 2.0 Password grant type.
2. Exchange the JWT tokens for a vCenter SAML token.
3. Use the SAML token to obtain a session identifier for the VMware Cloud Foundation API.

```
#!/bin/bash

: '
Variable definitions:
$vcip = The IP address or FQDN of your vCenter
$ACCESS_TOKEN = The access token in JWT format that you received from the
 authentication server.
$ID_TOKEN = The ID token in JWT format that you received from the authentication
 server.
'
if [ -z "$1" ] || [ -z "$2" ] || [ -z "$3" ] || [ -z "$4" ] || [ -z "$5" ] || [ -z
 "$6" ]; then
  echo "Usage: <vc-ip> <pwgrant-userid> <pwgrant-password> <client-id> <client-secret>
 <token-endpoint>"
```

```
  exit 0
fi

vcip="$1"
userid="$2"
password="$3"
clientid="$4"
clientsecret="$5"
tokenendpoint="$6"

echo "Obtaining JWT access and ID tokens for user $userid ..."

PWGRANT_OUTPUT=$(curl -k --silent --location -u "$clientid:$clientsecret" --request
 POST "$tokenendpoint" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=password" \
  --data-urlencode "username=$userid" \
  --data-urlencode "password=$password")
ACCESS_TOKEN=$(echo $PWGRANT_OUTPUT | jq -r '.access_token')
ID_TOKEN=$(echo $PWGRANT_OUTPUT | jq -r '.id_token')

echo
echo "Access token: $ACCESS_TOKEN"
echo
echo "ID token: $ID_TOKEN"
echo

echo "Exchanging JWT tokens for vCenter SAML token ..."

TOKEN_EXCHANGE_OUTPUT=$(curl -k --silent --location --request POST "https://$vcip/api/
vcenter/authentication/token" \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --header "Authorization: Bearer $ACCESS_TOKEN" \
  --data-urlencode "subject_token_type=urn:ietf:params:oauth:token-type:access_token"
 \
  --data-urlencode "subject_token=$ACCESS_TOKEN" \
  --data-urlencode "actor_token_type=urn:ietf:params:oauth:token-type:id_token" \
  --data-urlencode "actor_token=$ID_TOKEN" \
  --data-urlencode "grant_type=urn:ietf:params:oauth:grant-type:token-exchange" \
  --data-urlencode "requested_token_type=urn:ietf:params:oauth:token-type:saml2")

echo
echo "$TOKEN_EXCHANGE_OUTPUT"
echo

SAML_TOKEN=$(echo $TOKEN_EXCHANGE_OUTPUT | jq -r '.access_token')

echo "vCenter SAML token: $SAML_TOKEN"
echo

echo "Establishing vCenter session with SAML token ${SAML_TOKEN::7}..."
echo
COMP_TOKEN=$(echo $SAML_TOKEN | base64 -d | gzip | base64 -w0)

SESSION_OUTPUT=$(curl -k --silent --location --request POST "https://$vcip/api/
session" \
  --header "Authorization: SIGN token=\"$COMP_TOKEN\"")
```

```
echo "Create Session Response: $SESSION_OUTPUT"

SESSION_ID=$(echo "$SESSION_OUTPUT" | tr -d '"')

echo "Tagging categories:"
curl -k "https://$vcip/api/cis/tagging/category" --header "vmware-api-session-id:
 $SESSION_ID"

echo
echo "Done!"
```

Accessing VMware Cloud Foundation Services

VMware Cloud Foundation SDK provides mechanisms for creating remote stubs to give clients access to VMware Cloud Foundation services.

The sequence of tasks you must follow to create a remote stub starts with creating a `ProtocolFactory`. You use the protocol factory object to create a `ProtocolConnection`. Connection objects provide the basis for creating stub interfaces toVMware Cloud Foundation services.

When you establish a connection to the VMware Cloud Foundation endpoint, you can create a `StubFactory` object and a `StubConfiguration` object. With these objects, you can create the remote stub for the VMware Cloud Foundation service that you want to access.

The complete connection sequence also includes SSL truststore support and a temporary `StubConfiguration` that you use for SAML token authentication and session creation.

SSL Handshake

The VMware Cloud Foundation endpoint (`https://host/api`) is an SSL-enabled service that requires client authentication during login. The SSL connection relies on certificate verification supported by the Java security architecture. The Java security architecture defines truststores for SSL connections. A truststore contains vCenter Single Sign-On credentials. You use a truststore to verify credentials from a vCenter instance.

The vSphere Automation SDK for Java includes an SSL utility sample code that supports the creation of a truststore for the HTTP connection, `com.vmware.vcloud.suite.samples.common.SslUtil`.

Note:
The vSphere Automation SDK for Java SSL utility creates an instance of the Java security certificate class `X509TrustManager`. This instance declares an override client-side method, `checkServerTrusted`, that accepts all HTTPS certificates. This method is suitable only for development environments. For a production environment, do not use the `X509TrustManager` override methods. Instead, set up a truststore for use by the default `X509TrustManager` implementation.

For greater security, use an external utility to create a certificate store:

```
keytool -import -noprompt -trustcacerts \
-alias <alias name> \
-file <certificate file> \
-keystore <truststore filename> \
-storepass <truststore password>
```

Access a VMware Cloud Foundation Service

To access a VMware Cloud Foundation service, you must have a valid session connection. The sequence for accessing a VMware Cloud Foundation service includes creating a protocol connection object and using it to create the service stub.

Establish a connection to the VMware Cloud Foundation endpoint URL. For more information about the authentication mechanisms that you can use, see Authentication Mechanisms.

1. Create a protocol factory object.
2. Create a protocol connection object to access an API provider.

   The VMware Cloud Foundation API clients use `ApiProvider` instances to invoke operations on services running in the virtual environment. To invoke an operation, you must specify the target service and operation, input parameters, and execution context.
3. Create a `StubFactory` object by using the `ApiProvider` instance.
4. Create a `StubConfiguration` instance and set the security context to be used for the service stub.
5. Create the stub for the VMware Cloud Foundation service interface by calling the create method of the `StubFactory` instance. Pass the service class and the `StubConfiguration` instance as arguments.

A connection to the VMware Cloud Foundation endpoint is established and you can access all avaiable VMware Cloud Foundation services.

Java

The example is based on the code in the `LibraryCrud.java` sample.

This example shows the steps for creating an authenticated session to the VMware Cloud Foundation endpoint and creating the service stub for the Content Library API provider.

This example uses the steps described in the Access a VMware Cloud Foundation Service procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Log in by using username/password
   this.stubFactory = createApiStubFactory(server, httpConfig);

// Create a security context for username/password authentication
SecurityContext securityContext =
        SecurityContextFactory.createUserPassSecurityContext(
            username, password.toCharArray());

// Create a stub configuration with username/password security context
StubConfiguration stubConfig = new StubConfiguration(securityContext);

// Create a session stub by using the stub configuration.
Session session =
        this.stubFactory.createStub(Session.class, stubConfig);

// Log in and create a session
char[] sessionId = session.create();

// Initialize a session security context from the generated session id
SessionSecurityContext sessionSecurityContext =
        new SessionSecurityContext(sessionId);

// Update the stub configuration to use the session id
stubConfig.setSecurityContext(sessionSecurityContext);

/*
 * Create a stub for the session service using the authenticated
 * session
 */
this.sessionSvc =
        this.stubFactory.createStub(Session.class, stubConfig);
```

```
// Create service stubs for the Content Library service.
Library libraryService = stubFactory.createStub(Library.class, stubConfig);

// Invoke an operation of the Content Library service.
List<String> listContentLibraries = libraryService.list();
```

ESX Hosts

Use the VMware Cloud Foundation APIs to run general operations on the ESX hosts in your vSphere environment.

You can retrieve information about the hosts, create a standalone host, disconnect, and reconnect an ESX host to a vCenter system.

How to Retrieve Information About ESX Hosts

You retrieve information about the ESX hosts running in a vCenter instance by listing only the ESX hosts that you are interested in.

To filter the ESX hosts on a vCenter instance and get only the ones you want, call the `list` function and pass your criteria with a `HostTypes.FilterSpec` instance.use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request and specify your filter criteria through the query parameters. Combine several filter criteria by including one or more of the following parameters:

- The name or unique identifier of the host.
- Clusters, data centers, or folders that contain the host.
- Connection state of the host which can be one of the following: `CONNECTED`, `DISCONNECTED`, or `NOT_RESPONDING`.
- Power state of the host which can be one of the following: `POWERED_OFF`, `POWERED_ON`, or `STANDBY`.

The function returns a list of `com.vmware.vcenter.HostTypes.Summary` objects that contain information about up to 2500 hosts that match all specified criteria. You can use the list to retrieve information about the returned ESX hosts.

The request returns a `HostSummary` JSON object for each host that matches your filter. You can view information for up to 2500 hosts.

How to Add a Standalone ESX Host to vCenter

You can use the VMware Cloud FoundationREST APIs to add a standalone host to a vCenter instance.

Add a single ESX host to a vCenter instance by calling the `create` function and passing a `com.vmware.vcenter.HostTypes.CreateSpec` instance as parameter. When you create the host specification, make sure that you set the IP address or the DNS resolvable host name and the administrator credentials.

Add a single ESX host to a vCenter instance by using the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` request and submitting a `HostCreateSpec` JSON object in the request body. When you construct the host object, make sure that you set values for the IP address or the DNS resolvable host name and the administrator credentials.

How to Disconnect and Reconnect ESX Hosts

You can use the VMware Cloud Foundation APIs to connect ESX hosts to a vCenter instance and make the hosts managed. You can temporarily disconnect a managed host from a vCenter instance and reconnect the host, for example, to refresh the ESX agents on the host.

When you add a host to a vCenter instance, the host is connected to vCenter and becomes a managed host. To disconnect a managed host from a vCenter instance, call the `disconnect` function and pass the host identifier as a parameteruse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>?action=disconnect` request and submit the host identifier as a path parameter. The managed host and its associated virtual machines remain in the inventory but vCenter temporarily stops managing and monitoring them.

To reconnect a managed host to a vCenter instance, call the `connect` function and pass the host identifier as a parameteruse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>?action=connect` request and submit the host identifier as a path parameter. As a result, the connection status of the host changes, and vCenter resumes managing the host and its associated virtual machines.

If you want to delete a host and all its associated virtual machines from the inventory, you can remove the host from the vCenter instance. To delete a disconnected host from a vCenter instance, call the `delete` function and pass the host identifier as a parameteruse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>` and submit the host identifier as a path parameter.

How to Configure ESX Entropy

You can use the VMware Cloud Foundation API to feed external entropy data to an ESX host in your inventory. You can also query the current entropy levels on the host and add external entropy data when needed.

Entropy is a measure of the randomness or diversity of a data-generating function. In releases prior to vSphere 8.0 Update 1, ESX supported only CPU-based entropy data generated from interrupts or manufacturer provided interfaces, such as RDSEED and RDRAND. High-quality entropy is important for the proper functioning of security-related operations such as generating encryption keys for secure communication over the network. Starting with vSphere 8.0 Update 1, you can add external entropy sources to an ESX host and in this way ensure the high quality of the entropy data on that host. You provide external entropy data by using devices such as hardware security modules (HSMs) which are FIPS 140-3 and EAL4 certified.

You can configure the ESX entropy sources by using the VMkernel boot options. To use external entropy sources, set the `entropySources` value to more than or equal to `8`. For more information about how to set the desired entropy sources by using the VMkernel boot options, see Controlling ESX Entropy in the *vSphere Security* documentation.

You can also configure external entropy sources in the kickstart file for the ESX scripted installation. See Configuring External Entropy Sources During Scripted Installation.

> **Note:**
>
> If a host is configured to use only external entropy sources, that is, `entropySources` is set to `8`, you must keep supplying the external entropy data through the VMware Cloud Foundation API. In case the entropy in the host gets exhausted, the host becomes unresponsive and might require a hard reboot or re-installation to recover the host from this situation.

Querying Entropy Data on a Host

To retrieve details about the external entropy available on an ESX host, use the `ExternalPool` service. You must have the **Host.Entropy.Read** privilege.

You can check whether an external entropy source is added to a host by calling the `get(host_id)` method of the `com.vmware.vcenter.host.entropy.ExternalPool` interface and passing the host ID as parameter. The method returns an `ExternalPool.Info` instance that contains detailed information about the external entropy data on the hostusing the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>/entropy/external-pool` HTTP request and passing the host ID as a path parameter. The operation returns an `ExternalPool.Info` JSON object in the response body that contains detailed information about the external entropy data on the host.

Table 1881: Details for the External Entropy Data on a Host

External Entropy Data Detail Property	Description
`getStatus() status`	Indicates whether an external entropy source is added for a host.
`getCapacity() capacity`	Shows the maximum capacity of external entropy data in bytes that a host can store in the VMkernel entropy pool. When you feed the host with additional external entropy data, make sure that you do not exceed this maximum capacity. Otherwise, all extra entropy data is discarded.

External Entropy Data Detail Property	Description
`getCurrentlyAvailable() currently_available`	Indicates the current amount of entropy data in bytes available in the VMkernel entropy pool on the host.
`getInactiveSourceTimeout() inactive_source_timeout`	Indicates the amount of time in seconds that can elapse without any activity between the host and the external entropy source. Your application must check periodically the levels of entropy data on the host and send entropy data from the external source when required. When this timeout exceeds an `esx.audit.entropy.external.source.disconnected` VMkernel Observation (VOB) is logged. This system event suggests possible loss of connection to the external entropy source. For the full list of available entropy VOBs in vSphere 8.0, see the VMware knowledge base article at https://kb.vmware.com/s/article/89074.
`getLowWatermark() low_watermark`	Indicates the threshold in bits sufficient for the in-memory cache to operate successfully. When the in-memory entropy cache is running low and the threshold is reached, an `esx.audit.entropy.available.low` VOB is logged. For more information about the entropy VOBs, see the VMware knowledge base article at https://kb.vmware.com/s/article/89074.

Adding External Entropy Data to a Host

To add entropy data from an external entropy source to a host, you must have the **Host.Entropy.Write** privilege. Use the `ExternalPool` service and call the `add(host_id, add_spec)` method. Pass the host ID and an `ExternalPool.AddSpec` instance as method parameters. The `ExternalPool.AddSpec` instance contains the Base64 encoded external entropy data. You must convert the binary data coming from the external entropy source to Base64 format so that the host can consume it. As a result, the method returns an instance of the `ExternalPool.AddResult` class.

To add entropy data from an external entropy source to a host, you must have the **Host.Entropy.Write** privilege. Use the `ExternalPool` service and send the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/host/<host_id>/entropy/external-pool?action=add` request. Pass the host ID as a path parameter and an `ExternalPool.AddSpec` data structure in the request body. The `ExternalPool.AddSpec` JSON object contains the Base64 encoded external entropy data. You must convert the binary data coming from the external entropy source to Base64 format so that the host can consume it. The request returns an `ExternalPool.AddResult` JSON object in the response body, if the request succeeds.

When the external entropy data reaches the host, the entropy daemon stores it first in the in-memory cache and the storage cache. Then the external entropy data is pushed to the VMkernel entropy pool from which it is fed to the applications in need.

The entropy data in the storage cache persists on the host disk and is only consumed during the ESX booting. The default storage file size is 4MiB and can be configured through the ESXCLI commands. See the *ESXCLI Command Reference* documentation.

Managing the Life Cycle of Hosts and Clusters

You can manage the life cycle of ESX hosts collectively by using the vSphere Lifecycle Manager feature through the VMware Cloud Foundation API. Starting with vSphere 8.0, you can manage the lifecycle of a standalone ESX host by using an image only through the vSphere Lifecycle Manager automation API.

You can automate the life cycle management of a cluster or standalone host by performing the following operations:

- Retrieve information about the current state of the cluster or the standalone host.
- Create a desired state that includes a specific version of the ESX host. You can also add some compatible partner software and firmware components and add-ons.
- Validate the desired state to detect any discrepancies between the desired state and the host hardware.
- Check the compliance of a cluster or host against the desired state and determine whether some additional steps must be taken to ensure the success of the cluster or host remediation.

- Apply the desired state on a cluster or a standalone host.

You can use the vSphere Lifecycle Manager to manage the life cycle of hosts in a cluster that meet the following requirements:

- Hosts must be of version 7.0 and later.
- Hosts must be stateful.
- All hosts in the cluster must be from the same vendor and with identical hardware.
- The hosts must include only integrated solutions, such as VMware vSAN™, vSphere Supervisor, NSX and VMware vSphere® High Availability.

A standalone host is a host that is added to a vCenter instance but is not part of any cluster. For more information about how to add, connect, and disconnect standalone host, see ESX Hosts. You can manage the life cycle of a standalone host by performing almost all vSphere Lifecycle Manager operations that you can perform on a cluster level. The only limitation for managing the life cycle of a standalone host through the VMware Cloud Foundation API, is that you can't update the firmware of the host.

vSphere Lifecycle Manager Terms

You must understand the basic terminology that is used within this chapter to be able to use the vSphere Lifecycle Manager functionality efficiently.

vSphere Lifecycle Manager Terminology

Term	Definition
Upgrade, update, and patch	You can upgrade to another major version of the software running on an ESX host, and install patches and updates that include smaller changes, bug fixes, or other small improvements.
Depot	A depot is a well-defined folder structure that is used for distributing payloads and their metadata. Depots are consumed by different products and features such as the vSphere Lifecycle Manager and ESXCLI. The vSphere Lifecycle Manager works with three types of depots: online, offline, and UMDS. See Software Depots.
Component	A component is the smallest unit that the vSphere Lifecycle Manager uses during the installation and update processes. Software vendors use components to encapsulate a group of payloads that can be managed together.
Base image	A base image is a collection of components that shape the bootable ESX used for the installation or upgrade process. Base images are currently distributed only by VMware and support x86 servers. VMware provides new versions of the base image for each upgrade, update, and patch release of the ESX. Base images are hosted at the VMware online depot that is available by default to the vSphere Lifecycle Manager. Furthermore, you can download a different base image version, in the form of an offline ZIP bundle, from https://my.vmware.com/web/vmware/downloads.

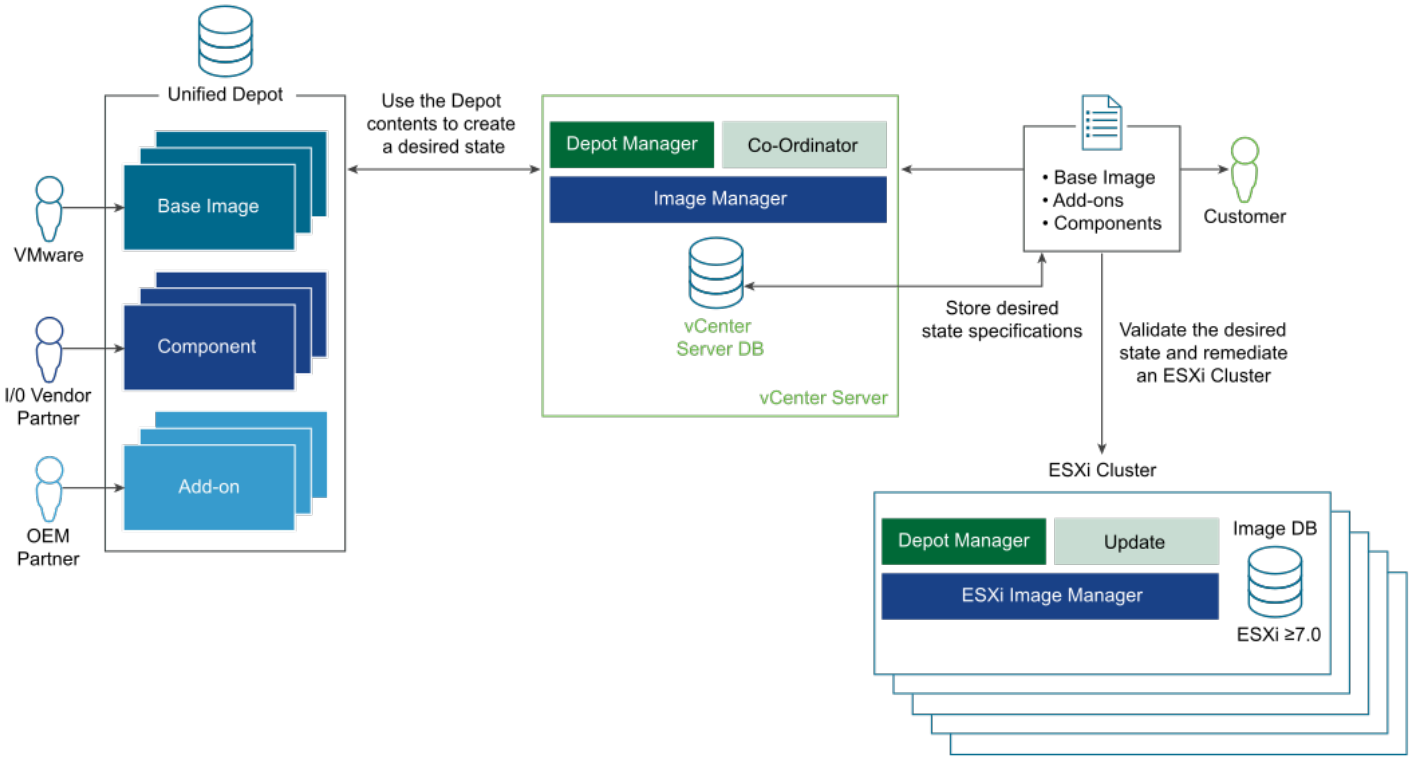
Term	Definition
Vendor add-on	An add-on is a collection of components that different OEMs provide on top of a base image. Vendors use an add-on to group some customizations for a family of servers. Unlike base images, add-ons are not complete and are not sufficient to boot an ESX. Each add-on must have a unique name and version. An add-on allows vendors to add, remove, or update components that are part of the VMware base image, if there are no unresolved dependencies and conflicts between the components.
Solution	A solution contains one or more components, and provides information about its constraints and compatibility with the different ESX versions. For example, from the perspective of the vSphere Lifecycle Manager solutions are VMware NSX, VMware vSphere® High Availability, vSAN.
Desired state	A desired state of a cluster is represented with a software specification. The desired state defines a set of components that a user wants to install on a single ESX host or on a cluster of hosts.
OEMs	Original Equipment Manufacturers. VMware partners enrolled in the VMware Partner Connect application, such as Dell, Inc., HP Inc., Lenovo Group Ltd., and so on.
IOVP	I/O Vendor Partner. Qualified VMware partners providing certified I/O device drivers for network and storage host bus adapters.
Third-party software providers	Providers of I/O filters, device drivers, CIM modules, and so on, that are not part of VMware partner programs.
vSphere Configuration Profiles	With vSphere Configuration Profiles, you can manage the configuration of all hosts in a cluster collectively. This ensures consistency in host configuration at a cluster level. You can enable vSphere Configuration Profiles only on clusters that have the vSphere Lifecycle Manager enabled.

vSphere Lifecycle Manager Overview

The vSphere Lifecycle Manager feature provides means for managing the life cycle of hosts on a cluster level. The functionality of this feature is achieved through several major services that are running on both vCenter and ESX.

After you install vSphere 8.0 Update 1, you can access the feature through several major services available on vCenter and on each ESX host.

Figure 567: vSphere Lifecycle Manager System Architecture



Depot Manager

For each upgrade and patch release, VMware, OEMs, and other third-party companies make the software updates of their products available to the customers. Software updates are distributed to different locations and in different formats depending on the way they are accessed, downloaded, and used. Depot Manager allows these different software resources to be presented to the vSphere customers in a unified format and as a result, makes them easier to use.

Depending on your environment and specific use case, Depot Manager gives you access to the software updates within three different types of depots: online, offline, and UMDS. See Software Depots.

Depot Manager that runs on the vCenter instance achieves the following goals:

- Represents the contents of all depots in a unified way.
- Caches the payloads and their metadata locally prior to their use.
- Enables depot overrides on a cluster or host level for Remote Office/Branch Office (ROBO) environments, or Edge computing environments.

Depot Manager that runs on the hosts serves as a proxy to the vCenter Depot Manager. If ROBO cluster or ROBO host, or Edge computing environments are enabled, Depot Manager that runs on the ESX hosts serves as a proxy to the nearby vSphere Lifecycle Manager compatible depot.

Image Manager

Image Manager allows you to create a desired state that you can apply on a cluster or a standalone host. The specification describes all components, add-ons, and the base image that you can use to update or upgrade the hosts in your environment. Image Manager supports validation of the desired state and the detecting drifts from the desired state.

Coordinator and ESX Updater Managers

Coordinator Manager runs on the vCenter instance and makes sure that the desired state is applied to all hosts in the cluster. This module also runs pre-checks to evaluate how each host in the cluster is affected by the remediation and

whether you must take some additional steps to ensure the success of the procedure. Coordinator Manager also allows you to query the status of the remediation operation for the cluster and for each host part of the cluster.

The ESX Updater Manager takes care of the actual remediation happening on each host.

Options for Managing the ESX Life Cycle

Based on your needs and environment setup, you can choose from several methods for managing the life cycle of the ESX hosts. The vSphere Lifecycle Manager provides means for updating all hosts in a cluster or a standalone host with a desired software state.

To manage the life cycle of the hosts in your environment, you can use the vSphere Lifecycle Manager through the vSphere Client. See Managing Host and Cluster Lifecycle.

This chapter of the VMware Cloud Foundation SDKsREST API Programming Guide discusses how you can access and use the functionality provided by the vSphere Lifecycle Manager Automation API.

vSphere Lifecycle Manager Features

You can use the VMware Cloud Foundation API to manage the life cycle of all hosts in a cluster or of standalone hosts by using a vSphere Lifecycle Manager image. You can access and use the following vSphere Lifecycle Manager functionality:

- Depot management. You can add, remove, explore the contents of different types of depots. See Software Depots. The content of the depots is provided by VMware and VMware partners. Partners can use the ESX Packaging Kit (EPK) to assemble a custom bootable ESX image. The custom image can then be shared to other third-party customers and used through the Depot Manager service. For more information about how to create custom ESX images, see *ESX Packaging Kit (EPK) Development Guide*.
- Desired software state. You can create, edit, and delete a desired software state for a cluster or a standalone host on which the vSphere Lifecycle Manager is enabled. A desired software state must contain at least a single ESX image provided by VMware. You can also set an add-on provided by OEMs, and one or more components by different software vendors. Furthermore, during the process of creating the desired software state, you can check the validity of the specification and compare the current state of the hosts in the cluster or the standalone host with the desired software state.
- Cluster remediation. You can apply the desired state on each of the hosts in a cluster which current state is different from the desired specification. Applying a desired state on a cluster level has the following prerequisites:
  - The cluster must have the vSphere Lifecycle Manager enabled.
  - All hosts in the cluster must store their data on a local or remote disk, or on a USB drive.
  - All hosts in the cluster must be of version 7.0 or higher.
  - All hosts must contain only components that the vSphere Lifecycle Manager can recognize and maintain. If a host contains some old content that the vSphere Lifecycle Manager does not recognize, the content is removed from the host during remediation.
- Standalone host remediation. You can apply the desired software state on a standalone host that is managed with baselines or on a standalone host that is managed with images but its current state differs from the desired state.

Required Privileges for Using vSphere Lifecycle Manager and vSphere Configuration Profiles

To configure vSphere Lifecycle Manager settings and to use successfully vSphere Lifecycle Manager baselines and images, you must have the proper privileges. Similarly, you need the proper privileges to use vSphere Configuration Profiles.

You can assign vSphere Lifecycle Manager and vSphere Configuration Profiles privileges to different roles in the vSphere Client.

vSphere Lifecycle Manager Privileges For Using Images

When you use vSphere Lifecycle Manager images, you need a different set of privileges for each task.

Table 1882: VMware vSphere vSphere Lifecycle Manager Privileges For Using Images

Task	Privilege in the vSphere Client	Privilege in the API
Set Up Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Import Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Upload File > Upload upgrade images and offline bundles**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read` • `VcIntegrity.FileUpload.com.vmware.vcIntegrity.ImportFile`
Export Image	**VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read**	`VcIntegrity.lifecycleSoftwareSpecification.Read`
Edit Image	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Work with Recommendations	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Image Privileges > Write** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Read`
Work with Depot	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Write**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write`
Manage Depot Overrides	• **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Read** • **VMware vSphere Lifecycle Manager > Lifecycle Manager: Settings Privileges > Write**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write`

Task	Privilege in the vSphere Client	Privilege in the API
Check Compliance	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Read**	`VcIntegrity.lifecycleSoftwareSpecification.Read`
Run Remediation Pre-Check	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: General Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **ESXi Health Perspectives** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Read**	• `VcIntegrity.lifecycleGeneral.Read` • `VcIntegrity.lifecycleHealth.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Read`
Remediate Against an Image	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: General Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **ESXi Health Perspectives** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Write** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read**	• `VcIntegrity.lifecycleGeneral.Read` • `VcIntegrity.lifecycleHealth.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Read` • `VcIntegrity.lifecycleSoftwareRemediation.Write` • `VcIntegrity.lifecycleSettings.Read`
Edit Remediation Settings	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Write**	• `VcIntegrity.lifecycleSettings.Read` • `VcIntegrity.lifecycleSettings.Write`
Update Firmware	• **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Read** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Privileges** > **Write** • **VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Settings Privileges** > **Read**	• `VcIntegrity.lifecycleSoftwareSpecification.Read` • `VcIntegrity.lifecycleSoftwareSpecification.Write` • `VcIntegrity.lifecycleSettings.Write`
View Hardware Compatibility	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Hardware Compatibility Privileges** > **Access Hardware Compatibility**	`VcIntegrity.HardwareCompatibility.Read`
Stage an Image	**VMware vSphere Lifecycle Manager** > **Lifecycle Manager: Image Remediation Privileges** > **Write**	`VcIntegrity.lifecycleSoftwareRemediation.Write`

For more information about managing users, groups, roles, and permissions, see the *vSphere Security* documentation.

Required Privileges for Using vSphere Configuration Profiles

When you use vSphere Configuration Profiles, you need a different set of privileges for each task.

Table 1883: Required Privileges For Using vSphere Configuration Profiles

Task	Privilege in the vSphere Client	Privilege in the API	Description
Create Cluster with vSphere Configuration Profiles enabled	**Host** > **Inventory** > **CreateCluster**	`Host.Inventory.CreateCluster`	Create a cluster along with its initial configuration.
View Host Settings in the Cluster Configuration	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Read-only access to desired configuration management platform**	`VcIntegrity.ClusterConfiguration.View`	View the draft configuration document, current configuration settings, compliance reports, and pre-check reports.
Check Cluster Compliance			
View Compliance Results			
Run Remediation Pre-Check			
View Remediation Pre-Check Results			
View Draft Pre-Check Results			
Remediate a Cluster Against the Desired Cluster Configuration	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Remediate cluster to the desired configuration.**	`VcIntegrity.ClusterConfiguration.Remediate`	Remediate a cluster against the draft configuration or transition a cluster to the desired configuration.
Transition to vSphere Configuration Profiles			
Export a Configuration or Configuration Schema	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Export desired cluster configuration**	`VcIntegrity.ClusterConfiguration.Export`	Export the desired cluster configuration document.
Extract a Configuration Document from a Reference Host	**VMware vSphere Lifecycle Manager** > **Desired Configuration Management Privileges** > **Modify desired cluster configuration.**	`VcIntegrity.ClusterConfiguration.Modify`	Modify the desired cluster configuration by crating a draft configuration, extracting a configuration document from a reference host, or importing a configuration.
Create a Draft Configuration for a Cluster			
Import a Configuration			

Software Depots

A software depot represents a well-defined file structure used for storing and hosting the ESX software updates, patches, and upgrades that VMware, partners, and third-party vendors provide. You can use the VMware Cloud Foundation APIs to manage the life cycle of the hosts in your environment by applying software updates hosted on different depots.

Software depots are managed by the Depot Manager which is part of the vSphere Lifecycle Manager. Software depots contain the actual payloads and the metadata of the software updates. Depending on the way you access the software updates, the Depot Manager recognizes three types of software depots: online, offline, and UMDS.

The following section of the documentation explains the concept of a software depot in terms of the vSphere Lifecycle Manager feature. You can also find common use cases available through the APIs for working with the different types of depots and their content.

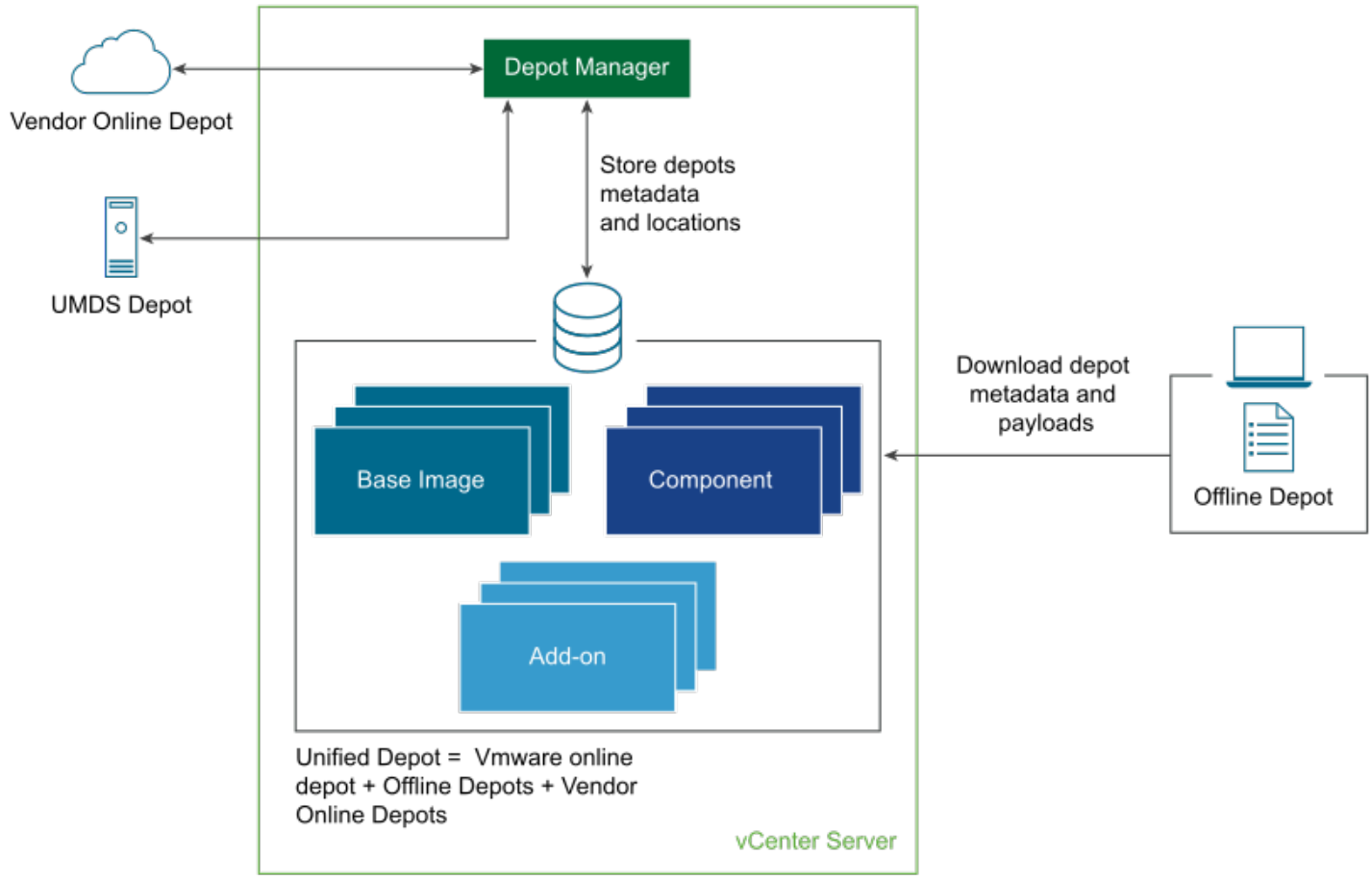
Types of Software Depots

Depot Manager works with the software updates provided by three different types of software depots: online, offline, and UMDS.

Regardless of the different way in which you access each type of software depot, all depots have the same structure. The same depot structure allows content from different vendors to be uploaded to one depot. Partners and third-party customers can use the ESX Packaging Kit to build and distribute software updates in the form of offline or online depots.

You can access their content by adding the online vendor depot to the vSphere Lifecycle Manager or by downloading the content of the offline depot to the vCenter instance.

Figure 568: Types of Software Depots



Online Depot

VMware and partners upload software updates to the VMware online depot at `https://hostupdate.vmware.com/software/VUM/PRODUCTION/...` or to a custom online depot. Software updates can be patches of the ESX base image, different versions of the partner add-ons, the IOVP drivers certified by VMware, and the VMware Tools™ updates. Online depots are accessible through a URL. By default, you can see the base images, add-ons, and components provided within the VMware online depot at the following locations:

- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/addon-main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/iovp-main/vmw-depot-index.xml`
- `https://hostupdate.vmware.com/software/VUM/PRODUCTION/vmtools-main/vmw-depot-index.xml`

When you deploy the vCenter, the vSphere Lifecycle Manager is configured to access the VMware online depot, by default. You can use the VMware Cloud Foundation APIs to add a custom online depot to be managed by Depot Manager. The metadata of the newly added online depot is not synchronized immediately. To synchronize the metadata, you can run a synchronization operation or wait for the scheduled synchronization to take place.

The Depot Manager stores in the vCenter database only the metadata of the software updates and the location of the added online depots. You can create a schedule to synchronize the software updates metadata stored in the vCenter with the metadata available in the accessible depots. The payloads of the software updates are downloaded only during the cluster remediation process.

To add, remove, list, and retrieve information about the online depots, you can use the `com.vmware.esx.settings.depots.Online` interface service. See Working with Online Depots.

Offline Depot

The offline depot is also called an offline bundle and is distributed as a downloadable ZIP file. Offline depots contain both the metadata and the payloads of the software update. Partners and third-party customers can use the ESX Packaging Kit to build and distribute offline bundles. You can download offline bundles from the VMware website or from the websites of third-party vendors. When you add an offline depot to the vSphere Lifecycle Manager depot, the software updates are downloaded to the vCenter database.

To manage offline depots, you can use the `com.vmware.esx.settings.depots.Offline` interface service. See Working with Offline Depots.

UMDS Depot

In case, the vCenter instance is in an air-gapped environment and has no access to any wire or wireless network, you can use a UMDS depot. The Update Manager Download Service (UMDS) is available as a `VMware-UMDS-8.0.1.-build_number.tar.gz` file within the ISO image of the vCenter appliance 8.0 UMDS is a 64-bit application and requires a 64-bit Linux-based system. Install UMDS on a machine that has Internet access and is different from the machine on which the vSphere Lifecycle Manager is running. For further information about how to install and configure the UMDS module, see the *Managing Host and Cluster Lifecycle*.

You can set up a synchronization schedule for downloading specific software updates from online vendor depots to the UMDS depot. Then use these updates to create desired software state for the clusters in your environment.

To manage UMDS depots through the VMware Cloud Foundation API, you can use the `com.vmware.esx.settings.depots.Umds` interface `UMDS` service.

Working with Online Depots

You can use the VMware Cloud Foundation APIs to add online depots to the list of currently configured online software depots.

Use online depots to add new content over time to the management scope of Depot Manager. Depot Manager periodically updates the software depots metadata stored on the vCenter instance. In case, new software updates are uploaded to the online depots, the Depot Manager makes sure that the metadata stored on the vCenter database is updated accordingly.

To add an online depot to the Depot Manager, you must first create the online depot specification by using the `com.vmware.esx.settings.depots.OnlineTypes.CreateSpec` class`Online.CreateSpec` data structure. To specify the URI to the `vendor-index.xml` file of the online depot, use the `setLocation(location)` method of the `OnlineTypes.CreateSpec` classuse the `location` property of the `Online.CreateSpec` data structure. Optionally, you can add a description and enable the depot. By default, when you add an online depot to the Depot Manager, the depot is enabled and its metadata is synchronized following the defined schedule. If you want to synchronize the added online depot immediately, call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request. When you complete the depot specification, call the `create(spec)` method of the `com.vmware.esx.settings.depots.Online` interface to add the depotuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online` request and submit the `Online.CreateSpec` JSON object to add the depot.

You can edit the depot description and deactivate the depot by creating an `com.vmware.esx.settings.depots.OnlineTypes.UpdateSpec` object and pass it to the `update( depot, update_spec)` method of the `com.vmware.esx.settings.depots.Online` interfaceusing the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>` request and submitting the depot ID as path parameter and an `Online.UpdateSpec` JSON object in the request body.

You can remove an online depot from the list of currently configured depots by using the `delete(depotID)` method of the `com.vmware.esx.settings.depots.Online` interface`DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>?vmw-task=true` request and submitting the depot ID as a pat parameter. This operation does not remove the already downloaded metadata and payloads from the deleted depot. You cannot delete the default VMware online depot, you can only deactivate it.

To retrieve a list of currently configured online depots, call the `list()` method of the `Online` interfaceuse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online` request. You can also retrieve

information about a currently configured online depot by using the `get(depotID)` method of the `Online` interface`GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/online/<depot_id>` request and submitting the depot ID as path parameter.

Working with UMDS Depots

In an air-gapped vCenter environment, you can use the VMware Cloud FoundationREST API to add a UMDS depot to the depots managed by Depot Manager.

After you install and configure the Update Manager Download Service (UMDS) on a physical machine with Internet access, you can add the UMDS depot to Depot Manager. Only one UMDS depot can be added at a time to Depot Manager. When you add a UMDS depot, its content is not immediately synchronized. To synchronize the content of the UMDS depots, you must call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request or wait for the scheduled synchronization to take place.

To add a UMDS depot, call the `set(set_spec)` method of the `com.vmware.esx.settings.depots.Umds` interface and pass a `com.vmware.esx.settings.depots.UmdsTypes.SetSpec` object as an argumentuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request and submit a `Umds.SetSpec` JSON object in the request body.. The UMDS specification must contain the URI location to the `index.xml` file of the depot. Optionally, you can set a description and indicate whether the depot must be enabled. By enabling the UMDS depot, you instruct Depot Manager to synchronize only the content that is available on that depot.

You can always edit the initial UMDS depot settings, by calling the `update(update_spec)` method of the `Umds` interface and passing an `com.vmware.esx.settings.depots.UmdsTypes.UpdateSpec` object.using the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request and submit a `Umds.UpdateSpec` JSON object in the request body.

To retrieve information about the currently configured UMDS depot, use the `get()` method of the `Umds` interface`GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds` request. You can remove a currently configured UMDS depot and all its downloaded content from Depot Manager by calling the `delete_Task()` method of the `com.vmware.esx.settings.depots.Umds` interface.using the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/umds?vmw-task=true` request.

Synchronizing Software Depots

The VMware online depot, the vendor online depots, and the UMDS depot must be synchronized regularly if you want to have the most recent software updates delivered by VMware, partners, and other third-party vendors. Use the VMware Cloud Foundation APIs to create a synchronization schedule or to synchronize the added depot immediately.

The Depot Manager does not synchronize immediately the metadata of the newly added online and UMDS depots. If you want to force the synchronization and not wait for the scheduled synchronization to take place, call the `sync_Task()` method of the `com.vmware.esx.settings.Depots` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots?action=sync&vmw-task=true` request. . You can also define a custom schedule to sync the metadata from the currently configured online or UMDS depots.

To create a custom schedule for checking for new software updates, you must first define the schedule parameters by using the `com.vmware.esx.settings.depots.SyncScheduleTypes.Schedule` class`SyncSchedule.Schedule` data structure.. Then you can add the schedule to the schedule specification by using the `setSchedule(schedule)` method of the `com.vmware.esx.settings.depots.SyncScheduleTypes.Spec` class`SyncSchedule.Spec` data structure.. Optionally, you can use the schedule specification to add an email to which notifications will be sent and define whether updates will be downloaded automatically. To apply the custom schedule, call the `set(spec)` method of the `com.vmware.esx.settings.depots.SyncSchedule` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/sync-schedule` request and submit the `SyncSchedule.Spec` JSON object in the request body.

The default schedule is set to update the metadata daily at a random time. To reset the schedule to the default settings, call the `set(spec)` of the `SyncSchedule` interface and pass `null` as an argumentuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/sync-schedule` and leave the `SyncSchedule.Spec` data structure unset..

Working with Offline Depots

An offline depot is a ZIP file that contains the metadata and payloads of software updates and follows the same structure as the online depot. Use the VMware Cloud Foundation APIs to import the content of an offline depot to the vCenter database.

To add an offline depot to the depots managed by the Depot Manager, you must create an offline depot specification by using the `com.vmware.esx.settings.depots.OfflineTypes.CreateSpec` class`Offline.CreateSpec` data structure. When you define the offline depot parameters, call the `create_Task(create_spec)` method of the `com.vmware.esx.settings.depots.Offline` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depots/offline?vmw-task=true` request and submit the `Offline.CreateSpec` JSON object. Depending on the location of the offline depot, when you create the offline depot specification, you must provide either the URI location or the file ID returned by the Jetty Web server embedded in the vSphere Lifecycle Manager. You set the type of the source from which the offline depot is downloaded by using the `setSourceType(sourceType)` method of the `OfflineTypes.CreateSpec` class`source_type` property of the `Offline.CreateSpec` data structure.

Pull Depot Content from a URI

To indicate that the offline depot resides on a URI location, call the `setLocation(java.net.URI location)` method of the `OfflineTypes.CreateSpec` class. You can pass as an argument the depot location in one of the following URI schemes: `http`, `https`, or `file`set the depot location as a value to the `location` property of the `Offline.CreateSpec` data structure. You can use one of the following URI schemes: `http`, `https`, or `file`. If you provide an HTTPS location to the offline depot, make sure you also provide a certificate trusted by the VMware Certificate Authority (VMCA) or a custom certificate from the VMware Endpoint Certificate Store (VECS). For detailed information about how to manage certificates, see the *vSphere Authentication* documentation.

Managing Depot Overrides

In case, you have a smaller Remote Office/ Branch Office (ROBO) cluster environment, or Edge computing environment, your clusters have no, or limited access to the Internet and limited connection to a vCenter instance. In such an environment, you can use Depot Manager to fetch the metadata and payloads of a desired software state from a local to the cluster depot.

To remediate a ROBO cluster, you must have access to a local software depot that hosts the components of the desired state. You can use Depot Manager to either export the whole vSphere Lifecycle Manager depot to an offline bundle, or export only the content of the desired state required for remediating the ROBO cluster.

To export the desired state image from the vSphere Lifecycle Manager depot, call the `export(cluster,export_spec)` method of the `com.vmware.esx.settings.clusters.Software` interface. The method returns the URI of the offline bundle that is hosted on the vSphere Lifecycle Manager Jetty Web serveruse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_ID>/software?action=export` request and submit the cluster ID as path parameter and an `Software.ExportSpec` JSON object. You receive the URI of the offline bundle that is hosted on the vSphere Lifecycle Manager Jetty Web server. To move the content of the offline bundle to the ROBO location, you must physically copy the ZIP file, unarchive, and mount its content to an HTTP server inside the ROBO environment.

To redirect a ROBO cluster to download software updates from a local repository and not from the vSphere Lifecycle Manager depot on the vCenter instance, the VMware Cloud FoundationREST API offer the cluster `DepotOverrides` service. To add a depot override location to a ROBO cluster, call the `add(cluster,override_depot)` method of the `DepotOverrides` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_ID>/depot-overrides?action=add` request. Specify the URI location of the local depot with the `setLocation(location)` method of the `com.vmware.esx.settings.clusters.DepotOverridesTypes.Depot` class`location` property of the `DepotOverrides.Depot` data structure which you submit to the request body. During the ROBO cluster remediation, Depot Manager instructs the hosts in the ROBO cluster to download the software updates from the configured local depots within the ROBO cluster.

Inspecting Depot Contents

You can use the VMware Cloud FoundationREST API to inspect the contents of the already synchronized and imported depots. You can list the available base images, add-ons, and components, or retrieve some detailed information about a specific software update.

To retrieve a list of the base images available on a vCenter instance, call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.BaseImages` interface. To narrow the list of returned base images and retrieve only items matching to specific criteria, you must pass a `com.vmware.esx.settings.depot_content.BaseImagesTypes.FilterSpec` instance as an argument to the list method. Use the retrieved list to get some information about each base image, including their display name and version, release date, and their category. You can also get some detailed information about a single base image by using the `get(version)` method of the `com.vmware.esx.settings.depot_content.base_images.Versions` interface. The information includes a list of the components present in this base image.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/base-images` request. You receive in the response body, a list of `BaseImages.Summary` JSON objects for each available base image. To get more information about a single base image, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/base-images/versions/<version>` request. You also get information about the components present in this base image.

To retrieve a list of all currently available add-ons in the vSphere Lifecycle Manager depot, call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.AddOns` interface and pass `null` as an argument. You can filter the available add-ons by using some specific criteria such as the add-on vendor, name, versions, or minimum version. You can retrieve some detailed information about a single add-on version, by calling the `get(name,version)` method of the `com.vmware.esx.settings.depot_content.add_ons.Versions` interface. The information includes the list of components part of the add-on, and the list of components that were removed by this add-on version.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/add-ons` request and submit some of the query parameters to filter the list of retrieved add-ons. To retrieve some detailed information about an add-on, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/add-ons/<name>/versions/<version>` request and submit the name and version of the add-on.

To retrieve a list of all components currently available in the vSphere Lifecycle Manager depot, you can call the `list(filter_spec)` method of the `com.vmware.esx.settings.depot_content.Components` interface and pass `null` as an argument. To retrieve a list of components that matches some specific criteria, define your preferences with a `com.vmware.esx.settings.depot_content.ComponentsTypes.FilterSpec` instance and pass it as an argument to the list method. Use the retrieved list to get some information regarding each component.use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/components` request and submit some of the query parameters to filter the list of retrieved components. To retrieve detailed information about a component, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/depot-content/components/<name>/versions/<version>` request and submit the name and version of the component.

Enabling a Cluster to Use a Software Specification

If you want to use the vSphere Lifecycle Manager to manage the life cycle of clusters in your environment, you have two options for enabling this feature. You can enable the vSphere Lifecycle Manager when you create the cluster. You can also turn an already created cluster into one managed by the vSphere Lifecycle Manager.

Creating a Cluster with Enabled vSphere Lifecycle Manager

You can use only the vSphere Web Services API or the Virtual Infrastructure Management API to create a cluster in your virtual environment and specify the initial desired state of the cluster. Use the VMware Cloud FoundationREST API to specify the detailed desired state after the cluster creation.

Starting with vSphere 8.0 Update 1, you can use the Virtual Infrastructure Management API which introduced a new way of accessing and using the vSphere Web Services. That is, you can now use the JSON data format over HTTP and create REST-like requests to the vSphere Web Services. Use the Virtual Infrastructure API or the vSphere Web Services API to create a cluster. For more information about how to authenticate and use the Virtual Infrastructure Management API, see the *vSphere Web Services SDK Programming Guide*, *An Introduction to vSphere APIs*, and *Getting Started with vSphere APIs* documentation. You can also check the *Virtual Infrastructure JSON API* reference documentation at https://developer.vmware.com/apis/vi-json/latest/.

You can use the `POST https://<vcenter_ip_address_or_fqdn> /sdk/vim25/<release_number>/Folder/<moID>/CreateClusterEx` request and submit the name of the cluster and a `ClusterConfigSpecEx` data object in the request body. In the data object, among other properties, you can specify the desired state for the cluster. The `desiredSoftwareSpec` property in the `ComputeResourceConfigSpec` data object contains the desired software specification for the cluster. This property is available for applications using the vSphere Web Services API of version 7.0 and later. You can create a `DesiredSoftwareSpec` data object and specify the base image that must be applied on the cluster with the `baseImageSpec` property. Optionally, you can specify a vendor add-on to be added to the software specification with the `vendorAddOnSpec` property.

Starting with vSphere 8.0, you can also enable the cluster to use vSphere Configuration Profiles by setting the `enableConfigManager` property of the `ComputeResourceConfigSpec` data object to `true`.

Enabling an Existing Cluster to Use vSphere Lifecycle Manager

If you want to manage the life cycle of a cluster by using a single software specification, you must first enable vSphere Lifecycle Manager on that cluster. You can use the VMware Cloud FoundationREST API to enable a cluster to use the vSphere Lifecycle Manager feature.

Before you enable vSphere Lifecycle Manager on a cluster, you can check whether the cluster meets all prerequisites. vSphere Lifecycle Manager can be enabled for a cluster only if the following requirements are met:

- All hosts in the cluster are of version 7.0 or later.
- All hosts in the cluster are stateful.
- All hosts in the cluster include only components that belong to integrated solutions, such as VMware vSAN™ and VMware vSphere® High Availability.
- None of the hosts in the cluster are in the process of active remediation through the VMware vSphere®Update Manager™.
- The cluster has a desired state already created for it.

If you want to run a preliminary check about whether all hosts in the cluster meet these requirements, call the `check_Task(cluster_ID,check_spec)` method of the `com.vmware.esx.settings.clusters.enablement.Software` interface. Pass as arguments the cluster ID, and optionally, a `com.vmware.esx.settings.clusters.enablement.SoftwareTypes.CheckSpec` instanceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software?action=check&vmw-task=true` request and submit the cluster ID and optionally a `Software.CheckSpec` JSON object.

The cluster ID represents the unique identifier for a cluster resource. To retrieve commonly used information about clusters including their IDs, call the `list(filter_spec)` method of the `com.vmware.vcenter.Cluster` interface and pass as argument a `com.vmware.vcenter.ClusterTypes.FilterSpec` instance to list the clusters that match specific criteria. You receive a list of `com.vmware.vcenter.ClusterTypes.Summary` objects which you can use to get the cluster ID.To retrieve a list of clusters available on your vCenter instance, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/cluster` request and filter the list of clusters by using the query parameters. You receive an array of `Cluster.Summary` JSON objects that contain the cluster ID among the other commonly used information.

You pass a `CkeckSpec` instance You submit a `Software.CheckSpec` structure in the request body to specify which checks can be skipped during the cluster preliminary check. Though you can skip some checks with this operation, the Image Manager runs all checks during the enablement operation. If you leave the check specification empty, all checks are run for each host in the cluster. You can select among the following checks to be skipped when running a pre-check operation:

- `SOFTWARE`. Checks whether there are any orphaned vSphere Installation Bundles (VIBs) and any software that cannot co-exist with vSphere Lifecycle Manager.
- `VERSION`. Checks whether all hosts in the cluster are of version greater than a predefined one.
- `STATELESSNESS`. Checks whether there are any stateless hosts in the cluster. vSphere Lifecycle Manager can be enabled only if the cluster does not contain stateless hosts.
- `VUM_REMEDIATION`. Checks whether any of the hosts in the cluster are currently remediated through the VMware vSphere®Update Manager™.

- SOFTWARE_SPECIFICATION_EXISTENCE. Checks whether there is a software specification already associated with this cluster. In case, this check reports that the cluster does not have a software specification, you must first create a draft software specification for this cluster and then commit the draft.
- VSAN_WITNESS_ELIGIBILITY. Checks whether the software specification can be used on any vSAN witness hosts in the cluster. For information about how you can manage a vSAN cluster by using vSphere Lifecycle Manager, see *vSAN Clusters and vSphere Lifecycle Manager* chapter in the *Managing Host and Cluster Lifecycle* documentation.

To enable a cluster to be managed with vSphere Lifecycle Manager, call the `enable_Task(cluster_ID,enable_spec)` method of the `com.vmware.esx.settings.clusters.enablement.Software` interface. Pass as arguments the cluster ID and optionally, a `com.vmware.esx.settings.clusters.enablement.SoftwareTypes.EnableSpec` instance. To specify checks that you want to be skipped during the enablement process, pass the `EnableSpec` instance. Currently, you can only skip the `SoftwareTypes.CheckType.SOFTWARE` check.

To enable a cluster to be managed with vSphere Lifecycle Manager, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software?vmw-task=true` request and submit the cluster ID as path parameter. To specify which check can be skipped during the enablement process, submit a `Software.EnableSpec` JSON object. Currently, you can only skip the software checks.

You can also get information about which clusters in your environment are managed with a single software specification. Call the `get(cluster_ID)` method and pass the cluster ID as an argument. Use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/enablement/software` request and submit the cluster ID as a path parameter.

Enabling a Standalone Host to Use a Software Specification

Starting with vSphere 8.0, you can use the vSphere Lifecycle Manager to manage the life cycle of standalone hosts in your vCenter system.

You can add a standalone host to a vCenter instance under a data center object or into a folder. See ESX Hosts. If you remove a host from a cluster but leave it in the data center, the host is also considered a standalone host.

To manage a standalone host with a software specification, you must first enable the vSphere Lifecycle Manager on that host. You can run a preliminary check to establish whether the standalone host meets all requirements for enabling the vSphere Lifecycle Manager. Call the `check_Task(host_ID, check_spec)` method of the `com.vmware.esx.settings.hosts.enablement.Software` interface and pass as arguments the host identifier and a `CheckSpec` instanceUse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/enablement/software?action=check&vmw-task=true` request and submit the host ID as path parameter. The pre-checks that you can skip before enabling the vSphere Lifecycle Manager on a standalone host are the same as for a cluster.

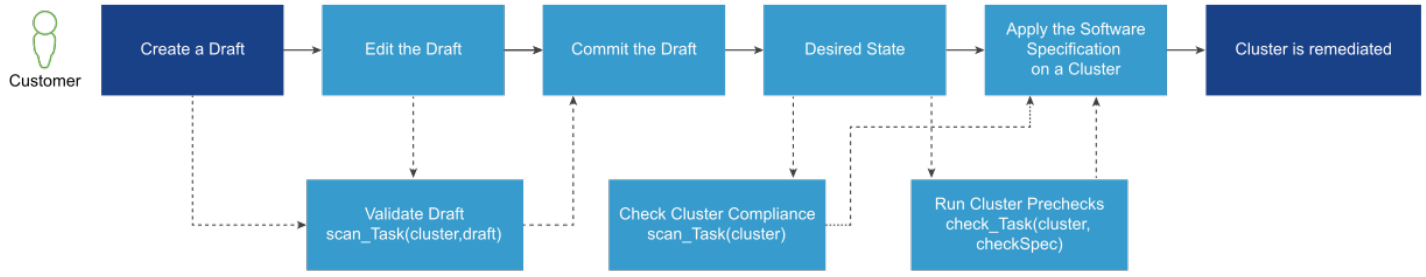
To enable vSphere Lifecycle Manager on a standalone host, call the `enable_Task(host_ID, check_spec)` method of the `com.vmware.esx.settings.hosts.enablement.Software` interface. Pass as arguments the host unique identifier and optionally, a `com.vmware.esx.settings.hosts.enablement.SoftwareTypes.EnableSpec` instance to define whether any checks can be skipped during the feature enablement operationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/enablement/software?vmw-task=true` request and submit the host ID as path parameter. Optionally, to define whether you want to skip any checks during the feature enablement, you can add a `Software.EnableSpec` data structure to the request body. Currently, you can skip only the check for orphaned vSphere Installation Bundles (VIBs).

Working with Draft Software Specifications

You create a draft software specification to describe the components of the desired state that you want to apply on a cluster or a standalone host.

The draft software specification is the working copy of the desired software state. Only one user at a time is allowed to edit a single draft for a cluster or a standalone host. Before saving the changes to the edited draft version, you can validate the content of the draft.

Figure 569: Drafts Workflow for a Cluster



A typical workflow for working with draft software specifications starts with creating a draft software specification for a specific cluster or a standalone host. If the cluster or the standalone host already have a software specification defined, this action takes the latest committed draft and you can edit its contents according to your needs. If the cluster or the standalone host have no software specification created yet, this method creates an empty draft. The only mandatory item for a draft is a base image of a specific version.

After you complete adding components, an add-on, and a base image, you can save your changes by committing the draft. This operation results in setting the committed draft as the current desired state of the cluster or the standalone host. Before committing the draft software specification, you can validate the contents of the draft or check whether all hosts in the cluster or the single standalone host are compliant with the draft.

If the commit operation is successful, the draft becomes the desired state for the cluster or the standalone host. You can now export the software specification and use it, for example, in a ROBO cluster scenario or another standalone host. You can also validate the compliance of the hosts in the cluster or the standalone host against the desired state and then apply the software specification, if feasible.

Creating a Draft Software Specification

To describe the components of a desired state for a cluster or a standalone host, create a draft software specification and save the desired state when ready.

To edit an existing desired state or to create an empty draft software specification:

- For a cluster, call the `create(cluster_ID)` method of the `com.vmware.esx.settings.clusters.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts` request.
- For a standalone host, call the `create(host_ID)` method of the `com.vmware.esx.settings.hosts.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts` request.

Pass the cluster ID or the host ID as an argument to the respective method. Submit the cluster ID or the host ID as path parameters to the respective request. As a result, you receive a draft ID which you can use to add a base image, an add-on, or some components to the draft.

Editing a Draft Software Specification

After you have created a draft software specification, use the VMware Cloud FoundationREST API to edit its items.

To set a base image to a draft software specification:

- For a cluster, call the `set(cluster_ID, draft_ID, base_image_spec)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.BaseImage` interface and pass as arguments the cluster and the draft IDs, and a base image specificationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/base-image` request and submit the cluster and draft IDs as path parameters and a base image specification in the request body.
- For a standalone host, call the `set(host_ID, draft_ID, base_image_spec)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.BaseImage` interface and

pass as arguments the standalone host and draft ID, and a base image specificationuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/base-image` request and submit the host and draft IDs as path parameters and a base image specification in the request body.

If the draft contains a base image, this method overwrites the existing image. The base image specification contains the version of the bootable ESX that must be included in the desired state. To retrieve details about the base image that is currently present in a draft, call the `get(cluster_ID,draft_ID)` or `get(host_ID, draft_ID)` methods of the `com.vmware.esx.settings.clusters.software.drafts.software.BaseImage` interface. Pass as arguments the cluster or the standalone host ID and the draft ID. Use the returned `com.vmware.esx.settings.BaseImageInfo` object to query the version, display name and version, and the release date of the ESX hostuse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/base-image` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/base-image` requests and submit as path parameters the cluster or the standalone host ID, and the draft ID. You receive a `BaseImageInfo` JSON object in the request body that holds the details about the version, display name and version, and the release date of the ESX host.

To add an OEM add-on to a draft software specification:

- For a cluster, call the `set(cluster_ID,draft_ID,addon_spec)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.AddOn` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/add-on` request.
- For a standalone host, call the `set(host_ID,draft_ID,addon_spec)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.AddOn` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/add-on` request.

Pass as arguments to these methods the cluster or the standalone host ID, the draft ID, and the add-on specification. If you want to remove an add-on from a draft, call the `delete(cluster_ID,draft_ID)` or `delete(host_ID,draft_ID)` method of the respective `AddOn` interface and pass as arguments the cluster or the standalone host ID, and the draft IDuse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/add-on` and `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/add-on` and submit as path parameters the cluster or the standalone host ID, and the draft ID.

You can add a component, change the version, or delete an existing component from a draft software specification. To change the version of a component included in a draft:

- For a cluster, call the `set(cluster_ID,draft_ID,component_ID,version)` method of the `com.vmware.esx.settings.clusters.software.drafts.software.Components` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components/<component_id>` request.
- For a standalone host, call the `set(host_ID, draft_ID, component_ID, version)` method of the `com.vmware.esx.settings.hosts.software.drafts.software.Components` interfaceuse the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components/<component_id>` request.

As a result, you add the component specified with the `component_ID` and `version` arguments to the draft, if it is missing. To remove a component from a draft, call the `delete(cluster_ID,draft_ID,component_ID)` or `delete(host_ID,draft_ID,component_ID)` method of the respective `Components` interfaceuse the `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components/<component_id>` and `DELETE https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components/<component_id>` requests. You can change multiple components in a draft by calling the `update(cluster_ID,draft_ID,update_spec)` or `update(host_ID,draft_ID,update_spec)` method of the respective `Components` interface. To specify the components you want to remove, add, or update for a draft, pass a cluster or a host `ComponentsTypes.UpdateSpec` instance as an argument to the update methodusing the `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/software/components`

and `PATCH https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>/software/components` requests. To specify the components you want to remove, add, or update for a given draft, submit a cluster or a host `Components.UpdateSpec` JSON object to the request body of the respective request.

To retrieve information about all components present in a draft, or a single component, you can use the list methods of the respective `Components` interfaceuse the list requests of the respective `Components` service.

Validating the Draft Software Specification

Before saving a draft and turning it into a desired state for a cluster or a standalone host, you can check whether the specification is complete and valid. You can also check whether the draft specification drifts in any way from the current state of the cluster or the host.

The `Drafts` service offers two methods for validating the draft software specification. To check whether a draft is complete and there are no conflicts between the draft components, or unresolved dependencies, call the `validate_Task(cluster_ID,draft_ID)` or `validate_Task(host_ID,draft_ID)` method of the respective `Drafts` interface. Pass as arguments to the method the cluster or the standalone host ID, and the draft IDuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=validate&vmw-task=true` or `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=validate&vmw-task=true` request depending on the targeted vSphere inventory object. Submit as path parameters the cluster ID or the standalone host ID, and the draft ID. You validate whether there were any other drafts committed for this cluster or standalone host, which can make the current commit operation invalid. The method also validates whether all components defined in the software specification are available in the depot metadata. This method does not run compliance checks against the cluster or the standalone host.

To check whether all hosts in the cluster or the standalone host are compliant with the draft software specification, call the `scan_Task(cluster_ID,draft_ID)` or `scan_Task(host_ID,draft_ID)` method of the respective `Drafts` interface. Pass as arguments to this method the cluster or the standalone host ID, and the draft IDuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=scan&vmw-task=truePOST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=scan&vmw-task=true` request depending on the targeted vSphere inventory object. Submit as path parameters the cluster ID or the standalone host ID, and the draft ID. This method results in running a comparison between the draft specification and the current state of each host in the cluster or the current state of the standalone host.

Committing the Draft Software Specification

When you commit a draft software specification, it becomes the desired state for the cluster or the standalone host.

To save the draft that you created for a cluster or a standalone host, call the `commit_Task(cluster_ID,draft_ID,draft_commit_spec)` or `commit_Task(host_ID, draft_ID, draft_commit_spec)` method of the respective `Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>?action=commit&vmw-task=true` and `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts/<draft_id>?action=commit&vmw-task=true` request depending on the targeted vSphere inventory object. The Image Manager component runs a validation check before the draft gets saved to the database. This method returns an identifier of the commit operation. You can use the ID to retrieve information about a specific commit such as the author of the commit operation, the time when the draft was committed, and so on.

Working with Desired Software States

When you commit a draft software specification, you make the committed draft the desired state for that cluster or standalone host. If all hosts in the cluster are compliant with the desired state, you can remediate the cluster. If the single standalone host is compliant with the desired state, you can update the ESX host to that state.

You can use the methods provided with the `com.vmware.esx.settings.clusters.Software` or `com.vmware.esx.settings.hosts.Software` interfacethe cluster and host `Software` services to manage a

desired state for a cluster or a standalone host. Before you apply a desired state on a cluster or a standalone host, you can run pre-checks to ensure that all hosts in the cluster or the single host are in a good state to be remediated. The pre-checks verify whether any of the hosts in the cluster or the single standalone host must be rebooted or are in maintenance mode. You can also check the compliance of the cluster or the standalone host against the desired state. See Checking the Compliance Against the Desired State.

You can export a software specification created for a cluster or a standalone host by using one of the following formats:

- An offline bundle in a ZIP file format.
- An ISO image.
- A JSON file.

Use the vSphere Lifecycle Manager API to import a software specification as a draft and then edit it. You have several options for running the import operation depending on the location and format of the desired software state.

Exporting and Importing a Desired State
Use the VMware Cloud FoundationREST API to export the desired software state of a cluster or a standalone host. Then you can import the desired state to a different cluster or host in the same or a different vCenter instance.

Exporting a Desired State

To export a desired state, you can use one of the following methods:

- For a cluster, call the `export(cluster_ID, export_spec)` method of the `com.vmware.esx.settings.clusters.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=export` request.
- For a standalone host, call the `export(host_ID, export_spec)` method of the `com.vmware.esx.settings.hosts.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=export` request.

This method does not export any information about the solutions available on the cluster or the standalone host since the constraints set by these solutions might not be applicable for another cluster or host. Pass as parameters the cluster or the standalone host ID, and an instance of the respective `SoftwareTypes.ExportSpec` classSubmit the cluster or the standalone host ID, and a `Software.ExportSpec` JSON object in the request body.

You can choose how to export the desired software specification.

- Export as an ISO image. Call the `setExportIsoImage(exportIsoImage)` method of the `ExportSpec` instance and pass `true` as an argument.Set `true` to the `export_iso_image` property of the `Software.ExportSpec` JSON object that you submit in the request body. Use the exported ISO image for performing clean installs and for bootstrapping purposes. You can upload the ISO file into the Jetty Web server on the target vCenter instance but you cannot use ISO files to manage the life cycle of clusters or a standalone host through the vSphere Lifecycle Manager feature.
- Export as an offline bundle in a ZIP file format. Call the `getExportOfflineBundle()` method of the `ExportSpec` instance. Set `true` to the `export_offline_bundle` property of the `Software.ExportSpec` JSON object that you submit in the request body. You can use the exported offline bundle to create a depot and add its components to the resources managed by the Depot Manager module.
- Export as a JSON file holding the desired state specification. Call the `setExportSoftwareSpec(exportSoftwareSpec)` method of the `ExportSpec` instance and pass `true` as argument.Set `true` to the `export_software_spec` property of the `Software.ExportSpec` JSON object that you submit to the request body. You can then reuse the JSON file to apply the desired state that it contains to another cluster or standalone host in the same or in a different vCenter instance. Note that the JSON file holds only the description of the desired state. You must check whether all components described in the JSON file are available in the depot for the target cluster. See Importing a Desired State Specification for information about how you can use a desired state specification for another cluster or standalone host.

Importing a Desired State Specification

To import a desired state of a cluster or a standalone host and assign it to another cluster or standalone host in the same or different vCenter instance:

- For a cluster, call the `importSoftwareSpec(cluster_ID, drafts_import_spec)` method of the `com.vmware.esx.settings.clusters.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts?action=import-software-spec` request.
- For a standalone host, call the `importSoftwareSpec(host_ID, drafts_import_spec)` method of the `com.vmware.esx.settings.hosts.software.Drafts` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/drafts?action=import-software-spec` request.

Pass as parameters the cluster or the standalone host ID, and an instance of the respective `DraftsTypes.ImportSpec` class. Submit as pat parameters the cluster or the standalone host ID, and a `Drafts.ImportSpec` JSON object in the request body. Use the instance of the import specification to describe the download source and the source type of the imported software specification. Depending on the location and the source type of the exported desired state, you can choose from the following import options:

- Import a file from the vCenter or your local file system. Call the `setFileId(fileId)` method of the `ImportSpec` instance. Pass as argument the file ID of the software specification which was previously uploaded on the Jetty Web server running on the vCenter at `https://<vcenter_FQDN>:9087/vum-fileupload` URL.Use the `file_id` property of the `Drafts.ImportSpec` data structure to set the file ID of the software specification which was previously uploaded on the Jetty Web server running on the vCenter at `https://<vcenter_FQDN>:9087/vum-fileupload` URL. You can also use this option to import a specification file that resides on your local file system. Make sure you set the source type of the import specification to `PUSH` through the `setSourceType(sourceType)` method of the `ImportSpec` instance`source_type` property of the `Drafts.ImportSpec` data structure.
- Import a file that resides on a URI location. Call the `setLocation(location)` method of the `ImportSpec` instance. Pass as argument the URI location of the software specification file. The software specification can be pulled from a URI location with one of the following schemes: `file`, `http`, or `https`. You can use this import mechanism only if you set the source type to `SourceType.PULL`Use the `location` property of the `Drafts.ImportSpec` data structure to submit the URI location of the software specification file. The software specification can be pulled from a URI location with one of the following schemes: `file`, `http`, or `https`. You can use this import mechanism only if you set the `source_type` property to `PULL`.
- Import a desired state as a JSON string. Call the `setSoftwareSpec(softwareSpec)` method of the `ImportSpec` instance. Pass as argument the JSON string representing the software specification you want to import. Use this mechanism only if you set the source type to `SourceType.JSON_STRING`.Use the `software_spec` property of the `Drafts.ImportSpec` data structure to set the JSON string representing the software specification you want to import. Use this method only if you set the `source_type` property to `JSON_STRING`.

Checking the Compliance Against the Desired State

Before applying a desired state on a cluster, you can scan all hosts in the cluster against the desired state and check the cluster compliance against the desired state. Before applying a desired state on a standalone host, you can scan the host and check its compliance against the desired state.

To check the compliance of all hosts in a cluster or of the single standalone host:

- For a cluster, call the `scan_Task(cluster_ID)` method of the `com.vmware.esx.settings.clusters.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=scan&vmw-task=true` request.
- For a standalone host, call `scan_Task(host_ID)` method of the `com.vmware.esx.settings.hosts.Software` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=scan&vmw-task=true`request.

Pass as an parameter the cluster or the host identifier. The method compares the desired state against the current state of each host in the cluster or of the single host and as a result calculates the compliance.

You can retrieve the cluster or the standalone host compliance status by calling the `get(cluster_ID)` or `get(host_ID)` method of the respective `Compliance` interface and passing as argument the cluster or the standalone host ID. As a result you receive a `com.vmware.esx.settings.ClusterCompliance` or a `com.vmware.esx.settings.HostCompliance` object. You can use the `ClusterCompliance` or the `HostCompliance` instance to retrieve the following informationusing the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/compliance` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software/compliance` requests and passing the cluster or standalone host ID as path parameter. As a result, you receive a `ClusterComliance` or `HostCompliance` JSON object that contains the following information:

- The overall cluster compliance status regarding the target version of the components described within the cluster desired state.
- The impact of applying the desired state on the cluster in case the cluster is non-compliant.
- A list of all compliant hosts in the cluster.
- A list of the incompatible hosts in the cluster.
- A list of the non-compliant hosts in the cluster.
- A list of the unavailable hosts which cannot be checked for compliance against the desired state.
- The compliance status of each host in the cluster.
- The notifications returned by the compliance check operation.
- The time that the compliance check takes.
- The ID of the committed draft for which the compliance check is performed. If the `getCommit()` method of the `ClusterCompliance` or the `HostCompliance` instance returns `null`If the `commit` property of the cluster or host compliance data structure is unset, the compliance check is run against a draft software specification.
- The compliance overall stage status of the cluster or the standalone host and is relevant when the compliance status of the cluster or the standalone host is `NON_COMPLIANT`.
- The compliance of the OEM add-on on the standalone host with respect to the add-on in the desired state.
- The compliance of the base image on the standalone host with respect to the base image in the desired state.
- The compliance of all effective components and all components present on the standalone host in respect to the components in the desired state.
- The compliance of all DPU devices on the standalone host with respect to the desired state.
- The compliance of the hardware support on the standalone host with respect to the hardware support defined in the target state.

A cluster or a standalone host can have one of the following compliance statuses regarding the target versions:

- `COMPLIANT`. The target versions of the components described in the desired state of the cluster or the standalone host are the same as the versions of the components currently present on the hosts in the cluster or on the standalone host.
- `NON_COMPLIANT`. The desired state of the cluster or the standalone host describes components with higher versions than the versions of the components currently present on the hosts in the cluster or on the standalone host. Non-compliant clusters or hosts are those clusters or hosts which have orphaned VIBs, or components on the hosts that are not present in the desired state specification.
- `INCOMPATIBLE`. One or more hosts in the cluster or in the standalone host have components with higher versions than the components described in the desired state specification.
- `UNAVAILABLE`. The current state of one or more hosts in the cluster or of the standalone host cannot be retrieved and as a result the compliance check cannot be performed.

You can check the compliance impact of applying the desired state on a non-compliant cluster or a standalone host by calling the `getImpact()` method of the `ClusterCompliance` or the `HostCompliance` instance. A `com.vmware.esx.settings.ComplianceImpact` object is returnedusing the `impact` property of the respective compliance data structure which contains the `ComplianceImpact` data structure. Use it to retrieve information about the steps you must take to remediate the cluster or the standalone host successfully. You might need to reboot a host or put a host into maintenance mode to remediate the cluster or the standalone host successfully.

Hardware Compatibility Data

The hardware compatibility data contains information about the compatibility between ESX hosts and ESX versions.

You can use the `com.vmware.esx.hcl.CompatibilityDataCompatibilityData` service to retrieve information about the compatibility data or to update the local compatibility data on a vCenter instance. To retrieve information about the compatibility data stored on your vCenter instance, call the `get()` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/compatibility-data/status` request. To update the local compatibility data with the latest version available from the official VMware source, call the `update_Task()` methoduse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/compatibility-data?action=download&vmw-task=true` request.

Checking the Hardware Compatibility of an ESX Host

You can query the hardware compatibility for a host before upgrading to a new ESX version. You can also download the information generated by the hardware compatibility report.

To use services form the `com.vmware.esx.hcl.hostsesx hcl` package, you must verify that you have accepted to participate in the CEIP and there is available compatibility data.

You can use the `com.vmware.esx.hcl.hosts.CompatibilityReleases` interface`CompatibilityReleases` service to list available releases for generating a compatibility report for a specific ESX host. To list the locally available ESX releases for the host that can be used to generate a compatibility report, call the `list(host_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-releases` request. The list includes only major and update releases. Patch releases are not listed.

You can use the `com.vmware.esx.hcl.hosts.CompatibilityReport` interface`CompatibilityReport` service to generate a hardware compatibility report for an ESX host against a specific ESX release. To return the last generated hardware compatibility report for a specific host, call the `get(host_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-report` request. To generate a hardware compatibility report for a specific host against specific ESX release, call the `create_Task(host_ID,spec)` methoduse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/hosts/<host_id>/compatibility-report?vmw-task=true` request.

You can use the `com.vmware.esx.hcl.Reports` interface`Reports` service to download information generated by the hardware compatibility report. To retrieve the URI location for downloading a compatibility report, call the `get(report_ID)` methoduse the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/hcl/reports/<report_id>` request.

Configuring Remediation Settings

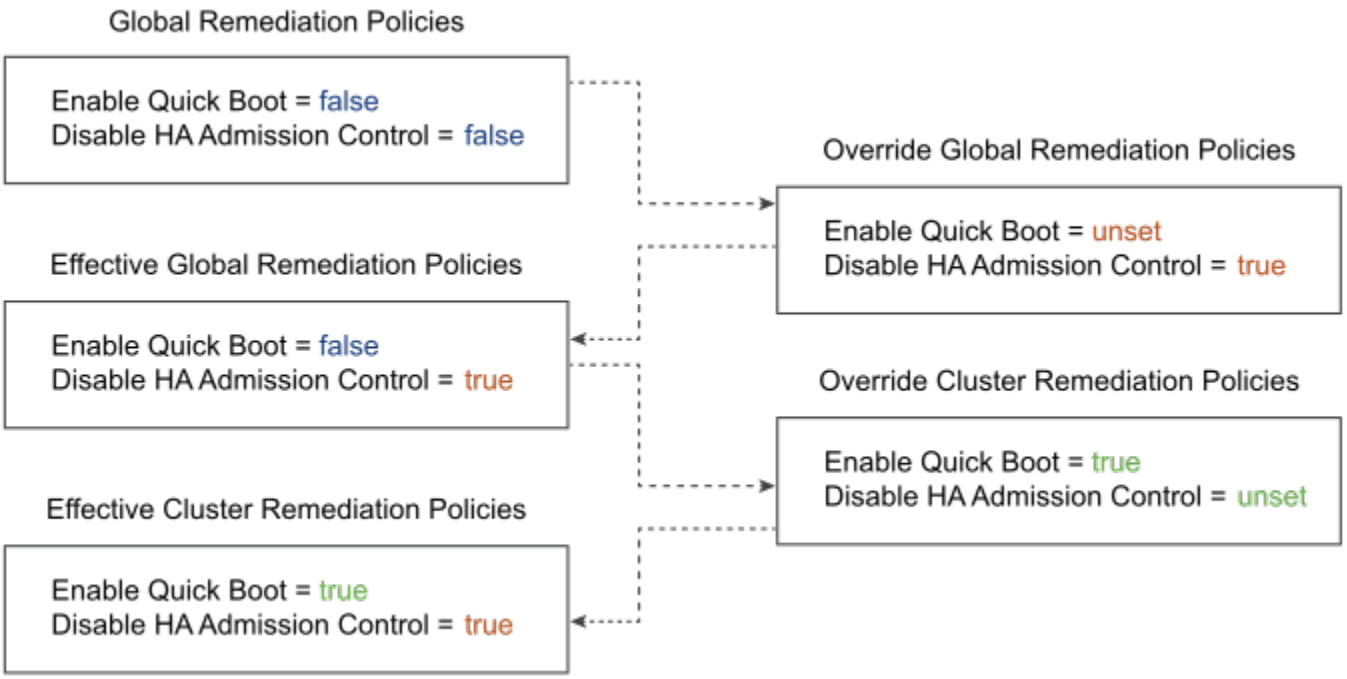
You can control the behavior of the ESX hosts and virtual machines during the remediation process. You can create a global remediation policy that applies to all clusters and standalone hosts in a vCenter instance. You can also set a remediation policy to a specific cluster and a standalone host.

When you run cluster and standalone host compliance checks, the Coordinator module runs a series of checks on each host to determine their state and whether some additional actions must be taken to ensure the success of the remediation operation. In case one or more hosts in the cluster and any of the standalone hosts are evaluated as non-compliant, additional checks are run on those hosts to evaluate whether they must be rebooted or put into maintenance mode. Currently, VMware provides a set of behavior controls (remediation policies) regarding the virtual machines and the hosts in a cluster or a standalone host. This set of remediation policies might change with the next vSphere release.

How Remediation Policies Overrides Work

The vSphere Lifecycle Manager provides a default global policy configuration that must be applied on each cluster and standalone host during remediation. Through the VMware Cloud FoundationREST API, you can change the global policies and create some cluster-and host-specific policies. Before remediating a cluster and standalone host, you can use the API to determine the effective global and cluster-and host-specific remediation policies. The following graphic describes how the mechanism of the policy overrides works.

Figure 570: How Remediation Policies Work



All clusters and standalone hosts in a vCenter instance inherit the default or the overridden global policy settings unless the global policy is explicitly overridden on a cluster and host level.

Editing Global or Cluster- and Host-Specific Remediation Policies

To view the currently set global remediation policy, call the `get(cluster_ID)` or `get(host_id)` method of the `com.vmware.esx.settings.defaults.clusters.policies.Apply` or `com.vmware.esx.settings.defaults.hosts.policies.Apply` interface. You receive a default `ApplyTypes.ConfiguredPolicySpec` instance that contains the configuration settings of the global remediation policy. To edit a global remediation policy, call the `set(policy_spec)` method of the respective `com.vmware.esx.settings.defaults.clusters.policies.Apply` or `com.vmware.esx.settings.defaults.hosts.policies.Apply`interface. Pass as an argument an `ApplyTypes.ConfiguredPolicySpec` instance and define new values to the global policy settings. To view the effective global remediation policy settings for a cluster and host, call the `get()` method of the respective `com.vmware.esx.settings.defaults.clusters.policies.apply.Effective` or `com.vmware.esx.settings.defaults.hosts.policies.apply.Effective` interface. The method returns an `EffectivePolicySpec` instance that contains the effective global policies applicable for all clusters and hosts in your vCenter environment.

To view the currently set global remediation policy, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply` request. You receive a `Apply.ConfiguredPolicySpec` data structure that contains the settings of the global remediation policy. To edit a global remediation policy, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply` or `PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply` request and submit the respective `Apply.ConfiguredPolicySpec` data structure to define new values to the global policy settings. To view the effective global remediation policy settings for a cluster and host, use the `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/clusters/policies/apply/effective` or `GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/defaults/hosts/policies/apply/effective` request. You receive the respective `Effective.EffectivePolicySpec` JSON object that contains the effective global policies applicable for all clusters and hosts in your vCenter environment.

To view the cluster- and host-specific remediation policies, call the `get(cluster_ID)` or `get(host_id)` method of the respective `com.vmware.esx.settings.clusters.policies.Apply` or `com.vmware.esx.settings.hosts.policies.Apply` interface. The method returns an instance of

the com.vmware.esx.settings.clusters.policies.ApplyTypes.ConfiguredPolicySpec or com.vmware.esx.settings.hosts.policies.ApplyTypes.ConfiguredPolicySpec class which contain the cluster- and host-specific policies to be applied during remediation. To change the cluster- and host-specific policy, call the set(cluster_ID,policy_spec) or set(host_ID,policy_spec) method of the respective Apply interface. Pass as argument an ApplyTypes.ConfiguredPolicySpec instance and describe the cluster- and host-specific remediation policies. To view the effective cluster- and host-specific policies, call the get(cluster_ID) or get(host_id) method of the com.vmware.esx.settings.clusters.policies.apply.Effective or com.vmware.esx.settings.hosts.policies.apply.Effective interface. The method returns an EffectivePolicySpec instance that describes the effective cluster- and host-specific policies.

To view the cluster- and host-specific remediation policies, use the GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply or GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply request. You receive an Apply.ConfiguredPolicySpec JSON object that contain the cluster- and host-specific policies to be applied during remediation. To change the cluster- and host-specific policy, use the PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply or PUT https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply request and submit anApply.ConfiguredPolicySpec data structure to describe the cluster- and host-specific remediation policies. To view the effective cluster- and host-specific policies, use the GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/clusters/<cluster_id>/policies/apply/effective or GET https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/policies/apply/effective request. You receive the respective Effective.EffectivePolicySpec data structure that describes the effective cluster- and host-specific policies.

Remediation Policy Options for Clusters

To describe a global or cluster-specific remediation policy, use the com.vmware.esx.settings.defaults.clusters.policies.ApplyTypes.ConfiguredPolicySpec and com.vmware.esx.settings.clusters.policies.ApplyTypes.ConfiguredPolicySpec classesthe respective

cluster `Apply.ConfiguredPolicySpec` data structure. For the vSphere 8.0 release, VMware provides the following methods to configure a global or cluster-specific policy.

MethodProperty	Description
`setDisableDpm(disableDpm)disable_dpm`	Deactivate the VMware Distributed Power Management (DPM) feature for all clusters or for a specific cluster. DPM monitors the resource consumption of the virtual machines in a cluster. If the total available resource capacity of the hosts in a cluster is exceeded, DPM powers off (or recommends powering off) one or more hosts after migrating their virtual machines. When resources are considered underutilized and capacity is needed, DPM powers on (or recommends powering on) hosts. Virtual machines are migrated back to these hosts. During the cluster remediation, the vSphere Lifecycle Manager cannot wake up and remediate hosts that are automatically put into a stand-by mode by DPM. These hosts stay non-compliant when DPM turns them on. The vSphere Distributed Resource Scheduler (DRS) is unable to migrate virtual machines to the hosts which are not remediated with the desired state for the cluster. To deactivate DPM during the cluster remediation, call the `setDisableDpm(disableDpm)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `disable_dpm` property. By default, the vSphere Lifecycle Manager temporarily deactivates DPM and turns on the hosts to complete the remediation. DPM is enabled again when the cluster remediation finishes.
`setDisableHac(disableHac)disable_hac`	Deactivate the vSphere HA admission control. vSphere HA uses admission control to ensure that a cluster has sufficient resources to guarantee the virtual machines recovery when a host fails. If vSphere HA admission control is enabled during remediation, putting a cluster into maintenance mode fails because vMotion cannot migrate virtual machines within the cluster for capacity reasons. To allow the vSphere Lifecycle Manager to temporary deactivate vSphere HA admission control, call the `setDisableHac(disableHac)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `disable_hac` property. By default, the vSphere HA admission control is enabled because DRS should be able to detect issues with the admission control and deactivate it to allow the remediation to complete.
`setEvacuateOfflineVms(evacuateOfflineVms)evacuate_offline_vms`	Migrate the suspended and powered off virtual machines from the hosts that must enter maintenance mode to other hosts in the cluster. To enable this remediation policy, call the `setEvacuateOfflineVms(evacuateOfflineVms)` method of the `ConfiguredPolicySpec` instance and pass as argument `true`set `true` to the `evacuate_offline_vms` property. By default, this setting is deactivated in the global remediation policy.

MethodProperty	Description
`setFailureAction(failureAction)failure_action`	Specify what actions vSphere Lifecycle Manager must take if a host fails to enter maintenance mode during the remediation. To configure this policy on a global or cluster-specific level, call the `setFailureAction(failureAction)` method of the `ConfiguredPolicySpec` instance. Pass as argument an `ApplyTypes.FailureAction` instanceset the properties of the `Apply.FailureAction` data structure. You can set the number of times that vSphere Lifecycle Manager tries to put a host into maintenance mode and the delay between the tries. When the threshold is reached and the host failed to enter maintenance mode, the cluster remediation fails. By default, vSphere Lifecycle Manager tries to put a host into maintenance mode three times with a five minute delay between each try before the cluster remediation fails.
`setEnforceHclValidation(enforceHclValidation)enforce_hcl_validation`	Prevents the remediation of vSAN clusters if vSphere Lifecycle Manager reports hardware compatibility issues during the hardware compatibility check performed as part of the remediation pre-check or the remediation tasks of the cluster. If you pass `null` as a parameterIf you leave the `enforce_hcl_validation` property unset, detected hardware issues are reported as warnings and do not prevent the remediation of the vSAN cluster.
`setParallelRemediationAction(parallelRemediationAction)parallel_remediation_action`	Enable simultaneous remediation of all hosts that are in maintenance mode with in the cluster. Pass an instance of the `com.vmware.esx.settings.defaults.clusters.policies.ApplyTypes.ParallelRemediationAction` classSubmit the `Apply.ParallelRemediationAction` data structure to indicate the maximum number of hosts that can be remediated in parallel. **Note:** If the hosts have NSX virtual distributed switches that are ready to be migrated to vSphere Distributed Switches, you must manually set the maximum number of parallel remediations to no more than 4. In cases when host switch migration is needed, if more than 4 hosts are remediated in parallel, the remediation might fail, because the host switch migration takes more time than the time vSphere Lifecycle Manager needs to complete the parallel remediation.

MethodProperty	Description
setPreRemediationPowerAction (preRemediationPowerAction)pre_remediation_power_action	Specify how the power state of the virtual machines must change before the host enters maintenance mode. If DRS is not enabled on a cluster or the automation level of a DRS cluster is not set to fully automated, the Coordinator module fails to remediate the cluster if the remediation requires a reboot or maintenance mode. You can set a policy that powers off or suspends the virtual machines on hosts that must be rebooted or must enter maintenance mode during remediation. The DRS takes care of changing the power state of the virtual machines when the host enters and exits maintenance mode. To set a policy for the power state of the virtual machines during the remediation, call the setPreRemediationPowerAction(preRemediationPowerAction) method of the ConfiguredPolicySpec instance. Pass as a parameter an instance of the PreRemediationPowerAction enumeration class and specifysubmit the Apply.ConfiguredPolicySpec.PreRemediationPowerAction enumerated type and set one of the following values: • DO_NOT_CHANGE_VMS_POWER_STATE. Indicates that the power state of the virtual machines must not be changes. • POWER_OFF_VMS. Indicates that the virtual machines must be powered off before the hosts enter maintenance mode. • SUSPEND_VMS. Indicates that the virtual machines must be suspended before the hosts enter maintenance mode. • SUSPEND_VMS_TO_MEMORY. Indicates that the virtual machines must be suspended before the hosts enter maintenance mode. By default, the Coordinator must leave the power state of the virtual machines unchanged.
setEnableQuickBoot(enableQuickBoot)enable_quick_boot	Reduce the reboot time of an ESX host by skipping all the hardware initialization processes and restarting only the hypervisor. This policy is applicable only if the host platform supports the Quick Boot feature. To enable the Quick Boot feature on the hosts during the remediation, call the setEnableQuickBoot(enableQuickBoot) method of the ConfiguredPolicySpec instance and pass as argument trueset true to the enable_quick_boot property. By default, this policy is deactivated.

Remediation Policy Options for Standalone Hosts

To describe a global or cluster-specific remediation policy, use the com.vmware.esx.settings.defaults.hosts.policies.ApplyTypes.ConfiguredPolicySpec and com.vmware.esx.settings.hosts.policies.ApplyTypes.ConfiguredPolicySpec classesrespective host

`Apply.ConfiguredPolicySpec` data structure. For the vSphere 8.0 release, VMware provides the following methods to configure a global or host-specific policy.

MethodProperty	Description
`setEnableQuickBoot(enableQuickBoot)enable_quick_boot`	Optimize the host patching and upgrade operations by reducing the reboot time of an ESX host. Since the patching and upgrading operations do not affect the hardware of the host, the hardware initialization processes can be skipped. This policy is applicable only if the host platform supports the Quick Boot feature. For more information about which hosts are Quick Boot compatible, see the following KB article https://kb.vmware.com/s/article/52477.
`setPreRemediationPowerAction (preRemediationPowerAction)pre_remediation_power_action`	Specify how the power state of the virtual machines must change before the standalone host enters maintenance mode. You can choose between the following power actions: • `DO_NOT_CHANGE_VMS_POWER_STATE`. Indicates that the power state of the virtual machines must not be changed. • `POWER_OFF_VMS`. Indicates that the virtual machines must be powered off before the standalone host enters maintenance mode. • `SUSPEND_VMS`. Indicates that the virtual machines must be suspended before the standalone host enters maintenance mode. • `SUSPEND_VMS_TO_MEMORY`. Indicates that the virtual machines must be suspended before the standalone host enters maintenance mode.
`setFailureAction(failureAction)failure_action`	Specify what actions vSphere Lifecycle Manager must take if a standalone host fails to enter maintenance mode during the remediation. You can set the number of times that vSphere Lifecycle Manager tries to put a standalone host into maintenance mode and the delay between the tries. When the threshold is reached and the standalone host failed to enter maintenance mode, the host remediation fails.

Remediating an ESX Cluster and a Standalone Host

You can use the VMware Cloud Foundation API to initiate the remediation of a cluster or a standalone host through the vSphere Lifecycle Manager.

To remediate a cluster with a desired state, call the `apply_Task(cluster_ID, apply_spec)` method of the `com.vmware.esx.settings.clusters.Software` interface. Pass as argument a `SoftwareTypes.ApplySpec` instance and specify whether the VMware End User License Agreement (EULA) must be acceptedeuse the `POST https://<vcenter_ip_address_or_fqdn>api/esx/settings/clusters/<cluster_id>/software?action=apply&vmw-task=true` request and submit a `Software.ApplySpec` JSON object in the request body. Specify whether the VMware End User License Agreement (EULA) must be accepted. You can also set the minimum commit ID of the draft software specification that must be used for remediating the cluster. Upon successful completion of the remediation task, all hosts in the cluster have the same software state.

To remediate a standalone host with a desired state, call `apply_Task(host_ID, apply_spec)` method of the `com.vmware.esx.settings.hosts.Software` interface and pass as arguments the standalone host ID and an instance of the `SoftwareTypes.ApplySpec` classuse the `POST https://<vcenter_ip_address_or_fqdn>/api/esx/settings/hosts/<host_id>/software?action=apply&vmw-task=true` request and submit a `Software.ApplySpec` JSON object in the request body. Use the software apply specification to set the minimum commit identifier of the draft software specification. See Committing the Draft Software Specification. You can also specify whether the VMware End User License Agreement (EULA) must be accepted.

Integrate Third-Party Solutions with vSphere Lifecycle Manager

A solution is an ESX software package that extends the functionality and capabilities of a host and integrates with the vCenter system. To be able to manage the life cycle of your third-party solutions on a cluster managed with images, you must integrate the solutions with vSphere Lifecycle Manager.

Examples of VMware integrated solutions are VMware NSX, vSAN, and vSphere Supervisor. For more information about the VMware integrated solutions, see *vSphere Lifecycle Manager Images and Other VMware Products and Solutions* section in the *Managing Host and Cluster Lifecycle* documentation.

Third-party software providers can use the vSphere APIs, VMware vSphere APIs for I/O Filtering (VAIO), VMware Daemon Software Development Kit, and others to develop third-party solutions for their vSphere platforms.

You can use the vSphere Lifecycle Manager automation APIs to manage the life cycle of third-party solutions on a cluster managed with a single image. First you need to package and upload your solution components to the vSphere Lifecycle Manager depot. Use the depot to store and manage the software updates for your third-party solutions. To make a solution available on a cluster, create a software specification that contains the solution and remediate all hosts in the cluster with that image. vSphere Lifecycle Manager manages the life cycle of the solution components by consuming the software updates from the vSphere Lifecycle Manager depot.

> **Note:**
> If you export an image from the cluster where your third-party solution is running, the solution components are not part of the exported image.

Prerequisites

To enable vSphere Lifecycle Manager to manage your third-party solutions, you must use the ESX Packaging Kit (EPK) to create installable packages. As of the vSphere 7.0 release, partner development kits generate components as installation packages. For more information about how to use the EPK to create components, assemble components into add-ons, then merge the add-ons with the base image to author a depot, see the *ES Packaging Kit (EPK) Development Guide* documentation.

Integrate a Third-Party Solution to Work with vSphere Lifecycle Manager

1. Create an online or offline depot to host your third-party solutions. See Working with Online Depots and Working with Offline Depots.
   - The following example creates an online depot that can be accessed through the `http://my_online_depot.com` URL.

     ```
     POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/depots/online
      {
         "description" : "My online depot adds the My Solution component to the ESX 7.0U3d base image",
         "ownerdata" : "ACME Company",
         "location" : "http://my_online_depot.com",
         "enabled" : true
     }
     ```

   - The following example creates an offline depot, also called an offline bundle, that can be imported to the vSphere Lifecycle Manager depot.

     ```
     POST https://{server}/api/esx/settings/depots/offline?vmw-task=true

     {
         "file_id" : "string",
         "description" : "string",
         "ownerdata" : "string",
         "source_type" : "PULL",
         "location" : "http://myurl.com"
     }
     ```

2. Synchronize the online depot to download the depot metadata and make the vSphere Lifecycle Manager aware of your solutions. See Synchronizing Software Depots.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/depots/online?vmw-task=true&action=sync
```

3. Create a draft software specification. See Creating a Draft Software Specification.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts
```

4. Add your solution to the created draft software specification. See Editing a Draft Software Specification.

```
PUT https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/<draft_id>/
software/components/<component_id>
```

5. Save the created draft software specification to make it the desired state for the cluster. See Committing the Draft Software Specification.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software/drafts/
<draft_id>?action=commit&vmw-task=true
```

6. Remediate the ESX cluster with the desired state that contains your solution to apply the desired state on all hosts in that cluster. See Remediating an ESX Cluster and a Standalone Host.

```
POST https://<vcenter_server_ip_or_fqdn>/api/esx/settings/clusters/<cluster_id>/software?action=apply&vmw-
task=true
{
    "hosts" : [
        "obj-103",
        "obj-103"
    ],
    "commit" : "obj-103",
    "accept_eula" : true
}
```

Results

You set up an image for the cluster which includes your third-party solution. You now manage all hosts in the cluster collectively with a single image. Upon remediation, the image is installed on all hosts in the cluster.

What to do next

You can update, delete or add new solutions to the draft software specification and then commit the changes to make the draft the desired state for the cluster.

Enable vSphere Lifecycle Manager on a Cluster Managed with Baselines

For more information about how to convert a cluster to use vSphere Lifecycle Manager images instead of baselines, see Enabling an Existing Cluster to Use vSphere Lifecycle Manager.

vSphere Lifecycle Manager APIs Equivalent to the vSphere Host Patch Manager APIs

To manage the life cycle of a single host with baselines, you use the vSphere Host Patch Manager APIs which are part of the vSphere Web Services API. To manage the life cycle of all hosts in a cluster collectively with an image, you use the vSphere Lifecycle Manager APIs which are part of the VMware Cloud Foundation APIs. The following table compares the APIs for life cycle management of the hosts and clusters in your environment.

Operation	vSphere Lifecycle ManagerAPI	vSphere Host Patch Manager API
Check whether hosts can be remediated	`com.vmware.esx.settings.clusters.Software.check`	`vim.host.PatchManager.CheckHostPatch_Task`
Check the compliance of the cluster or host with the desired state	`com.vmware.esx.settings.clusters.Software.scan`	`vim.host.PatchManager.ScanHostPatchV2_Task`
Remediate the cluster or host with the desired state	`com.vmware.esx.settings.clusters.Software.apply`	`vim.host.PatchManager.InstallHostPatchV2_Task`

Operation	vSphere Lifecycle ManagerAPI	vSphere Host Patch Manager API
Retrieve information about the cluster or host that you want to remediate	• `com.vmware.esx.settings.clusters.Software.get` - View information about the current desired state. • `com.vmware.esx.settings.clusters.software.Components.list` - View all components in the current desired state. • `com.vmware.esx.settings.clusters.software.Components.get` - View detailed information about a specific component.	`vim.host.PatchManager.QueryHostPatch_Task` - View information about the bulletins installed on an ESX host.
Stage a desired state on a cluster or host	`com.vmware.esx.settings.clusters.Software.stage`	`vim.host.PatchManager.StageHostPatch_Task`
Uninstall a component	`com.vmware.esx.settings.clusters.Software.apply` - When a software component is removed from the desired software specification and then the cluster is remediated, the component will be uninstalled from all hosts in that cluster.	`vim.host.PatchManager.UninstallHostPatch_Task`

Virtual Machine Configuration and Management

A virtual machine is a software computer that, like a physical computer, runs an operating system and applications.

The virtual machine consists of a set of specification and configuration files and is backed by the physical resources of a host. Each virtual machine encapsulates a complete computing environment and runs independently of the underlying hardware.

Starting with vSphere 6.5, you can create virtual machines, configure virtual machine settings, and perform power operations on the virtual machines through the VMware Cloud Foundation APIs.

Starting with vSphere 7.0, you can use the VMware Cloud Foundation APIs to perform various virtual machine management operations. For example, you can deploy virtual machines by using several approaches, clone an existing virtual machine, create an instant clone of a running virtual machine. You can also install VMware Tools which enables you to manage the life cycle and customize the networking and identity settings of the guest operating system installed on the virtual machine.

Creating Virtual Machines

You can use the VMware Cloud Foundation APIs to create virtual machines depending on your needs and infrastructure setup.

You can create a basic virtual machine and then configure it according to your needs. You can also create a more comprehensive virtual machine and then edit its settings. To create a virtual machine, you must specify the datastore, resource pool, folder, or host where the virtual machine is placed. Later, you can customize the virtual machine by specifying the boot options, number of CPUs, the guest OS, and virtual NIC. See Creating a Virtual Machine Without a Clone or Template and Configuring Virtual Machines.

You can use a turned off virtual machine to create a VM template from which to deploy other virtual machines. See Create a VM Template in a Content Library from a Virtual Machine. You can also mark a virtual machine as template by calling the `VirtualMachine.MarkAsTemplate` method from the vSphere Web Services APIs. See *vSphere Web Services SDK Programming Guide*.

You can capture a virtual machine or a vApp in an OVF template and store the template in a content library. Then you can use the OVF template to deploy a virtual machine or a vApp in your environment. See Creating Virtual Machines and vApps from Templates in a Content Library.

Creating a Virtual Machine Without a Clone or Template

You can create a virtual machine by using the `VM.create` method. The method takes as parameter a `CreateSpec` instance that describes the details of the virtual machine.

When you create a virtual machine without a template or clone, you can configure the virtual hardware, including processors, hard disc, memory. To create a virtual machine, you must specify the virtual machine attributes by using the `CreateSpec` class. For example, you can specify a name, boot options, networking, and memory for the new virtual machine.

All attributes are optional except the virtual machine placement information that you must provide by using the `PlacementSpec` class. Use the virtual machine placement specification to set the datastore, cluster, folder, host, or resource pool of the created virtual machine. You must make sure that all these vSphere objects are located in the same data center in a vCenter instance.

For more information, refer to the *API Reference* documentation inside the SDK.

Java Example of Creating a Basic Virtual Machine

This example is based on the code in the `CreateBasicVM.java` sample file.

This example uses the information provided in Creating a Virtual Machine Without a Clone or Template.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private void createBasicVM(
        VMTypes.PlacementSpec vmPlacementSpec, String standardNetworkBacking) {
        // Create the scsi disk as a boot disk
        DiskTypes.CreateSpec bootDiskCreateSpec =
                new DiskTypes.CreateSpec.Builder().setType(
                    DiskTypes.HostBusAdapterType.SCSI)
                    .setScsi(new ScsiAddressSpec.Builder(0l).setUnit(0l)
                        .build())
                    .setNewVmdk(new DiskTypes.VmdkCreateSpec())
                    .build();

        // Create a data disk
        DiskTypes.CreateSpec dataDiskCreateSpec =
                new DiskTypes.CreateSpec.Builder().setNewVmdk(
                    new DiskTypes.VmdkCreateSpec()).build();
        List<DiskTypes.CreateSpec> disks = Arrays.asList(bootDiskCreateSpec,
            dataDiskCreateSpec);

        // Create a nic with standard network backing
        EthernetTypes.BackingSpec nicBackingSpec =
                new EthernetTypes.BackingSpec.Builder(
                    BackingType.STANDARD_PORTGROUP).setNetwork(
                        standardNetworkBacking).build();
        EthernetTypes.CreateSpec nicCreateSpec =
                new EthernetTypes.CreateSpec.Builder().setStartConnected(true)
                    .setBacking(nicBackingSpec)
                    .build();
        List<EthernetTypes.CreateSpec> nics = Collections.singletonList(
            nicCreateSpec);

        // Specify the boot order
        List<DeviceTypes.EntryCreateSpec> bootDevices = Arrays.asList(
            new DeviceTypes.EntryCreateSpec.Builder(DeviceTypes.Type.ETHERNET)
```

```
                .build(),
            new DeviceTypes.EntryCreateSpec.Builder(DeviceTypes.Type.DISK)
                .build());
      VMTypes.CreateSpec vmCreateSpec = new VMTypes.CreateSpec.Builder(
            this.vmGuestOS).setName(BASIC_VM_NAME)
                .setBootDevices(bootDevices)
                .setPlacement(vmPlacementSpec)
                .setNics(nics)
                .setDisks(disks)
                .build();
        System.out.println("\n\n#### Example: Creating Basic VM with spec:\n"
                           + vmCreateSpec);
        this.basicVMId = vmService.create(vmCreateSpec);

...
```

Python Example of Creating a Basic Virtual Machine

This example is based on the code in the `create_basic_vm.py` sample file.

This example uses the information provided in Creating a Virtual Machine Without a Clone or Template.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

def create_basic_vm(stub_config, placement_spec, standard_network):
    """
    Create a basic VM.

    Using the provided PlacementSpec, create a VM with a selected Guest OS
    and provided name.

    Create a VM with the following configuration:
    * Create 2 disks and specify one of them on scsi0:0 since it's the boot disk
    * Specify 1 ethernet adapter using a Standard Portgroup backing
    * Setup for PXE install by selecting network as first boot device

    Use guest and system provided defaults for most configuration settings.
    """
    guest_os = testbed.config['VM_GUESTOS']

    boot_disk = Disk.CreateSpec(type=Disk.HostBusAdapterType.SCSI,
                                scsi=ScsiAddressSpec(bus=0, unit=0),
                                new_vmdk=Disk.VmdkCreateSpec())
    data_disk = Disk.CreateSpec(new_vmdk=Disk.VmdkCreateSpec())

    nic = Ethernet.CreateSpec(
        start_connected=True,
        backing=Ethernet.BackingSpec(
            type=Ethernet.BackingType.STANDARD_PORTGROUP,
            network=standard_network))

    boot_device_order = [BootDevice.EntryCreateSpec(BootDevice.Type.ETHERNET),
                         BootDevice.EntryCreateSpec(BootDevice.Type.DISK)]

    vm_create_spec = VM.CreateSpec(name=vm_name,
```

```
                                    guest_os=guest_os,
                                    placement=placement_spec,
                                    disks=[boot_disk, data_disk],
                                    nics=[nic],
                                    boot_devices=boot_device_order)
    print('\n# Example: create_basic_vm: Creating a VM using spec\n-----')
    print(pp(vm_create_spec))
    print('-----')

    vm_svc = VM(stub_config)
    vm = vm_svc.create(vm_create_spec)

    print("create_basic_vm: Created VM '{}' ({})".format(vm_name, vm))

    vm_info = vm_svc.get(vm)
    print('vm.get({}) -> {}'.format(vm, pp(vm_info)))

    return vm

...
```

Configuring Virtual Machines

You configure a virtual machine in the process of creation by using `com.vmware.vcenter.CteareSpec`. You can later view and edit virtual machine settings by adding or changing the type of the storage controllers, configure the virtual disks, boot options, CPU and memory information, or networks. Virtual machine settings can be configured when cloning, registering and relocating an existing virtual machine.

Name and Location
You specify the display name and the location of the virtual machine by using the `CreateSpec` and `PlacementSpec` classes.

When you create your virtual machine, use the `setName` method of the `CreateSpec` class to pass as argument the display name of the virtual machine.

You must create also a `PlacementSpec` instance that describes the location of the virtual machine in regards to the resources of a given vCenter instance. Use the `setPlacement(PlacementSpec placement)` method of the `CreateSpec` class to set the placement information for the virtual machine. You can set one or all of the following vSphere resources: datastore, cluster, folder, host, and resource pool.

Java Example of Configuring the Name and Placement of a Virtual Machine
This example is based on the code in the `CreateDefaultVM.java` and `PlacementHelper.java` sample files.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private static final String DEFAULT_VM_NAME = "Sample-Default-VM";
    private VM vmService;
    private GuestOS vmGuestOS = GuestOS.WINDOWS_9_64;
    private String defaultVMId;

              ...

    public VMTypes.PlacementSpec getPlacementSpecForCluster(
            StubFactory stubFactory, StubConfiguration sessionStubConfig,
            String datacenterName, String clusterName,
```

```
            String vmFolderName, String datastoreName) {

            String clusterId =
                    ClusterHelper.getCluster(stubFactory,
                        sessionStubConfig,
                        datacenterName,
                        clusterName);
            System.out.println("Selecting cluster " + clusterName + "(id="
                            + clusterId + ")");

            String vmFolderId =
                    FolderHelper.getFolder(stubFactory,
                        sessionStubConfig,
                        datacenterName,
                        vmFolderName);
            System.out.println("Selecting folder " + vmFolderName + "id=("
                            + vmFolderId + ")");

            String datastoreId =
                    DatastoreHelper.getDatastore(stubFactory,
                        sessionStubConfig,
                        datacenterName,
                        datastoreName);
            System.out.println("Selecting datastore " + datastoreName + "(id="
                            + datastoreId + ")");

            /*
             *  Create the virtual machine placement spec with the datastore, resource pool,
             *  cluster and vm folder
             */
            VMTypes.PlacementSpec vmPlacementSpec = new VMTypes.PlacementSpec();
            vmPlacementSpec.setDatastore(datastoreId);
            vmPlacementSpec.setCluster(clusterId);
            vmPlacementSpec.setFolder(vmFolderId);

            return vmPlacementSpec;
        }

    private void createDefaultVM() {
        VMTypes.PlacementSpec vmPlacementSpec =
                    this.getPlacementSpecForCluster(
                    this.vapiAuthHelper.getStubFactory(),
                    this.sessionStubConfig,
                    this.datacenterName,
                    this.clusterName,
                    this.vmFolderName,
                    this.datastoreName);


        VMTypes.CreateSpec vmCreateSpec =
                new VMTypes.CreateSpec.Builder(this.vmGuestOS)
                    .setName(DEFAULT_VM_NAME)
                    .setPlacement(vmPlacementSpec)
                    .build();

                ...
    }
```

Python Example of Configuring the Placement of a Virtual Machine
This example is based on the code in the `vm_placement_helper.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

def get_placement_spec_for_resource_pool(stub_config,
                                         datacenter_name,
                                         vm_folder_name,
                                         datastore_name):
    """
    Returns a VM placement spec for a resourcepool. Ensures that the
    vm folder and datastore are all in the same datacenter which is specified.
    """
    resource_pool = resource_pool_helper.get_resource_pool(stub_config,
                                                           datacenter_name)

    folder = folder_helper.get_folder(stub_config,
                                      datacenter_name,
                                      vm_folder_name)

    datastore = datastore_helper.get_datastore(stub_config,
                                               datacenter_name,
                                               datastore_name)

    # Create the vm placement spec with the datastore, resource pool and vm
    # folder
    placement_spec = VM.PlacementSpec(folder=folder,
                                      resource_pool=resource_pool,
                                      datastore=datastore)

    print("get_placement_spec_for_resource_pool: Result is '{}'".
          format(placement_spec))
    return placement_spec
```

Hardware Version

The hardware version of a virtual machine reflects the virtual hardware features supported by a virtual machine. These features depend on the physical hardware available on the ESX host on which the virtual machine is running.

Virtual hardware features include the BIOS and Extensible Firmware Interface (EFI), the maximum number of CPUs, the maximum memory configuration, and other hardware characteristics.

When you create a virtual machine, the default hardware version of the virtual machine is the most recent version available on the host where the virtual machine is created. For information about the latest VMware products and virtual hardware versions, see Virtual machine hardware versions (1003746).

To set a different than the default hardware version, call the `setHardwareVersion(hardwareVersion)` function of the `com.vmware.vcenter.VMTypes.CreateSpec` class. Use the `HardwareTypes.Version` class to define a valid hardware version for a virtual machine. For information about the hardware features available for the virtual hardware versions, see Hardware features available with virtual machine compatibility settings (2051652).

You can set a lower virtual hardware version of a virtual machine than the highest supported by the ESX host on which the virtual machine is running. Setting a lower hardware version can provide flexibility and is useful in the following cases:

- To help you standardize testing and deployment in your environment.
- In case you do not need the hardware features of the latest hardware version of the host.
- To maintain compatibility with hosts with a lower hardware version.

Boot Options

You can configure the boot options of a virtual machine by using the `setBoot(CreateSpec boot)` method of the `CreateSpec` class.

The method takes as argument the `BootTypes.CreateSpec` class. You can select one of the following settings when booting the virtual machine:

- Delay - Indicates a delay in milliseconds before starting the firmware boot process when the virtual machine is powered on.
- Retry - Indicates whether the virtual machine automatically retries to boot after a failure.
- Retry delay - Indicates a delay in milliseconds before retrying the boot process after a failure.
- Enter setup mode - If set to `true`, indicates that the firmware boot process automatically enters BIOS setup mode the next time the virtual machine boots. The virtual machine resets this flag to `false` once it enters setup mode.
- EFI legacy boot - If set to `true`, indicates that the EFI legacy boot mode is used.

Java Example of Configuring the Boot Options of a Virtual Machine

This example is based on the code in the `BootConfiguration.java` sample file.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private String vmName;
    private String vmId;
    private BootTypes.Info originalBootInfo;
    private Boot bootService;

                  ...

       this.bootService = vapiAuthHelper.getStubFactory().createStub(Boot.class,
                     this.sessionStubConfig);

       System.out.println("\n\n#### Setup: Get the virtual machine id");
       this.vmId = VmHelper.getVM(vapiAuthHelper.getStubFactory(),
             sessionStubConfig,
             vmName);
        // Print the current boot configuration
        System.out.println("\n\n#### Print the original Boot Info");
        BootTypes.Info bootInfo = this.bootService.get(this.vmId);
        System.out.println(bootInfo);

        // Save the current boot info to verify that we have cleaned up properly
        this.originalBootInfo = bootInfo;

        System.out.println(
            "\n\n#### Example: Update firmware to EFI for boot configuration.");
        BootTypes.UpdateSpec bootUpdateSpec = new BootTypes.UpdateSpec.Builder()
            .setType(BootTypes.Type.EFI)
            .build();
        this.bootService.update(this.vmId, bootUpdateSpec);
        System.out.println(bootUpdateSpec);
        bootInfo = this.bootService.get(this.vmId);
        System.out.println(bootInfo);

        System.out.println(
            "\n\n#### Example: Update boot firmware to tell it to enter setup"
```

```
            + " mode on next boot.");
        bootUpdateSpec = new BootTypes.UpdateSpec.Builder()
            .setEnterSetupMode(true)
            .build();
        this.bootService.update(this.vmId, bootUpdateSpec);
        System.out.println(bootUpdateSpec);
        bootInfo = this.bootService.get(this.vmId);
        System.out.println(bootInfo);

        System.out.println(
            "\n\n#### Example: Update firmware to introduce a delay in boot "
            + "process and automatically reboot after a failure to boot, "
            + "retry delay = 30000 ms.");
        bootUpdateSpec = new BootTypes.UpdateSpec.Builder()
            .setDelay(10000l)
            .setRetry(true)
            .setRetryDelay(30000l)
            .build();
        this.bootService.update(this.vmId, bootUpdateSpec);
        bootInfo = this.bootService.get(this.vmId);
        System.out.println(bootInfo);

...
```

Python Example of Configuring the Boot Options

The following example is based on the code of the `boot.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

"""
Demonstrates how to configure the settings used when booting a virtual machine.

Sample Prerequisites:
The sample needs an existing VM.
"""

vm = None
vm_name = None
stub_config = None
boot_svc = None
cleardata = False
orig_boot_info = None

...

def run():
    global vm
    vm = get_vm(stub_config, vm_name)
    if not vm:
        exit('Sample requires an existing vm with name ({}). '
             'Please create the vm first.'.format(vm_name))
    print("Using VM '{}' ({}) for Boot Sample".format(vm_name, vm))

    # Create Boot stub used for making requests
```

```
    global boot_svc
    boot_svc = Boot(stub_config)

    print('\n# Example: Get current Boot configuration')
    boot_info = boot_svc.get(vm)
    print('vm.hardware.Boot.get({}) -> {}'.format(vm, pp(boot_info)))

    # Save current Boot info to verify that we have cleaned up properly
    global orig_boot_info
    orig_boot_info = boot_info

    print('\n# Example: Update firmware to EFI for Boot configuration')
    update_spec = Boot.UpdateSpec(type=Boot.Type.EFI)
    print('vm.hardware.Boot.update({}, {})'.format(vm, update_spec))
    boot_svc.update(vm, update_spec)
    boot_info = boot_svc.get(vm)
    print('vm.hardware.Boot.get({}) -> {}'.format(vm, pp(boot_info)))

    print('\n# Example: Update boot firmware to tell it to enter setup mode on '
          'next boot')
    update_spec = Boot.UpdateSpec(enter_setup_mode=True)
    print('vm.hardware.Boot.update({}, {})'.format(vm, update_spec))
    boot_svc.update(vm, update_spec)
    boot_info = boot_svc.get(vm)
    print('vm.hardware.Boot.get({}) -> {}'.format(vm, pp(boot_info)))

    print('\n# Example: Update boot firmware to introduce a delay in boot'
          ' process and to reboot')
    print('# automatically after a failure to boot. '
          '(delay=10000 ms, retry=True,')
    print('# retry_delay=30000 ms')
    update_spec = Boot.UpdateSpec(delay=10000,
                                  retry=True,
                                  retry_delay=30000)
    print('vm.hardware.Boot.update({}, {})'.format(vm, update_spec))
    boot_svc.update(vm, update_spec)
    boot_info = boot_svc.get(vm)
    print('vm.hardware.Boot.get({}) -> {}'.format(vm, pp(boot_info)))

...
```

Guest Operating System

The guest operating system that you specify affects the supported devices and available number of virtual CPUs.

When you create a virtual machine, you specify the guest operating system by using the `setGuestOS(GuestOS guestOS)` method of the `VMTypes.CreateSpec` class. The `GuestOS` class defines the valid guest OS types that you can choose from for configuring a virtual machine.

After the create operation finishes successfully, you can install the guest operating system on the mew virtual machine in the same way as you install it on a physical machine. For further information on installing a guest operating system, refer to the *Guest Operating System Installation Guide* at http://partnerweb.vmware.com/GOSIG/home.html and the *vSphere Virtual Machine Administration* guide.

Starting with vSphere 7.0, you can use the VMware Cloud Foundation APIs to install the VMware Tool on the quest operating system and perform some guest OS customizations. See Installing VMware Tools.

CPU and Memory

The `CreateSpec` class allows you to specify the CPU and memory configuration of a virtual machine.

To change the CPU and memory configuration settings, use the `CpuTypes.UpdateSpec` and `MemoryTypes.UpdateSpec` classes.

CPU Configuration

You can set the number of CPU cores in the virtual machine by using the `setCount` method of the `CpuTypes.UpdateSpec` class. The supported range of CPU cores depends on the guest operating system and virtual hardware version of the virtual machine. If you set `CpuTypes.Info.getHotAddEnabled()` and `CpuTypes.Info.getHotRemoveEnabled()` to `true`, you allow virtual processors to be added or removed from the virtual machine at runtime.

Memory Configuration

You can set the memory size of a virtual machine by using the `setSizeMiB` method of the `MemoryTypes.UpdateSpec` class. The supported range of memory sizes depends on the configured guest operating system and virtual hardware version of the virtual machine. If you set `MemoryTypes.UpdateSpec.setHotAddEnabled()` to `true` while the virtual machine is not powered on, you enable adding memory while the virtual machine is running.

Java Example of Configuring the CPU and Memory of a Virtual Machine
This example is based on the code in the `CpuConfiguration.java` and `MemoryConfiguration.java` sample files.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private String vmName;
    private String vmId;
    private Memory memoryService;
    private Cpu cpuService;
...

        this.memoryService = vapiAuthHelper.getStubFactory().createStub(Memory.class,
                    this.sessionStubConfig);

        this.vmId = VmHelper.getVM(vapiAuthHelper.getStubFactory(), sessionStubConfig, vmName);

        // Update the memory size of the virtual machine
        MemoryTypes.UpdateSpec memoryUpdateSpec = new MemoryTypes.UpdateSpec.Builder().setSizeMiB(8 *
 1024l).build();
        memoryService.update(this.vmId, memoryUpdateSpec);
        memoryInfo = memoryService.get(this.vmId);

        // Enable adding memory while the virtual machine is running
        memoryUpdateSpec = new MemoryTypes.UpdateSpec.Builder().setHotAddEnabled(true).build();
        memoryService.update(this.vmId, memoryUpdateSpec);
             ...

        this.cpuService = vapiAuthHelper.getStubFactory().createStub(Cpu.class,
            this.sessionStubConfig);

          // Get the current CPU information
        CpuTypes.Info cpuInfo = cpuService.get(this.vmId);

         // Update the number of CPU cores
        CpuTypes.UpdateSpec cpuUpdateSpec = new CpuTypes.UpdateSpec.Builder()
            .setCount(2l).build();
```

```
        cpuService.update(this.vmId, cpuUpdateSpec);
        cpuInfo = cpuService.get(this.vmId);

         // Update the number of cores per socket in the virtual machine and
         // allow CPU cores to be added to the virtual machine while it is running
        cpuUpdateSpec = new
 CpuTypes.UpdateSpec.Builder().setCoresPerSocket(2l).setHotAddEnabled(true).build();
        cpuService.update(this.vmId, cpuUpdateSpec);
...
```

Python Example of Configuring the CPU and Memory of a Virtual Machine

These examples are based on the code in the `cpu.py` and `memory.py` sample files.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

The following example shows how you can update the CPU configuration of a virtual machine.

```
...

vm = None
vm_name = None
stub_config = None
cpu_svc = None
cleardata = False
orig_cpu_info = None

...
        server, username, password, cleardata, skip_verification, vm_name = \
            parse_cli_args_vm(testbed.config['VM_NAME_DEFAULT'])
        stub_config = vapiconnect.connect(server,
                                          username,
                                          password,
                                          skip_verification)
...
def run():
    global vm
    vm = get_vm(stub_config, vm_name)
    if not vm:
        exit('Sample requires an existing vm with name ({}). '
             'Please create the vm first.'.format(vm_name))
    print("Using VM '{}' ({}) for Cpu Sample".format(vm_name, vm))

    # Create CPU stub used for making requests
    global cpu_svc
    cpu_svc = Cpu(stub_config)

    # Get the current CPU configuration
    cpu_info = cpu_svc.get(vm)
    print('vm.hardware.Cpu.get({}) -> {}'.format(vm, pp(cpu_info)))

    # Save current CPU info to verify that we have cleaned up properly
    global orig_cpu_info
    orig_cpu_info = cpu_info

    # Update the number of CPU cores of the virtual machine
    update_spec = Cpu.UpdateSpec(count=2)
```

```
    print('vm.hardware.Cpu.update({}, {})'.format(vm, update_spec))
    cpu_svc.update(vm, update_spec)

    # Get the new CPU configuration
    cpu_info = cpu_svc.get(vm)
    print('vm.hardware.Cpu.get({}) -> {}'.format(vm, pp(cpu_info)))

    # Update the number of cores per socket and
    # enable adding CPUs while the virtual machine is running
    update_spec = Cpu.UpdateSpec(cores_per_socket=2, hot_add_enabled=True)
    print('vm.hardware.Cpu.update({}, {})'.format(vm, update_spec))
    cpu_svc.update(vm, update_spec)
...
```

The following example demonstrates how you can add memory to a running virtual machine.

```
...

vm = None
vm_name = None
stub_config = None
memory_svc = None
cleardata = False
orig_memory_info = None

...
    server, username, password, cleardata, skip_verification, vm_name = \
            parse_cli_args_vm(testbed.config['VM_NAME_DEFAULT'])
    stub_config = vapiconnect.connect(server,
                                      username,
                                      password,
                                      skip_verification)

...

    global vm
    vm = get_vm(stub_config, vm_name)
    if not vm:
        exit('Sample requires an existing vm with name ({}). '
             'Please create the vm first.'.format(vm_name))
    print("Using VM '{}' ({}) for Memory Sample".format(vm_name, vm))

    # Create Memory stub used for making requests
    global memory_svc
    memory_svc = Memory(stub_config)

    # Get the current Memory configuration
    memory_info = memory_svc.get(vm)
    print('vm.hardware.Memory.get({}) -> {}'.format(vm, pp(memory_info)))

    # Update the memory size of the virtual machine
    update_spec = Memory.UpdateSpec(size_mib=8 * 1024)
    print('vm.hardware.Memory.update({}, {})'.format(vm, update_spec))
    memory_svc.update(vm, update_spec)

    # Get the new Memory configuration
    memory_info = memory_svc.get(vm)
    print('vm.hardware.Memory.get({}) -> {}'.format(vm, pp(memory_info)))
```

```
    # Enable adding memory while the virtual machine is running
    update_spec = Memory.UpdateSpec(hot_add_enabled=True)
    print('vm.hardware.Memory.update({}, {})'.format(vm, update_spec))
    memory_svc.update(vm, update_spec)

...
```

Networks

You configure network settings so that a virtual machine can communicate with the host and with other virtual machines. When you configure a virtual machine, you can add network adapters (NICs) and specify the adapter type.

You can add virtual Ethernet adapters to a virtual machine by using the `VMTypes.CreateSpec.setNics` method. Pass as argument a `List` of `EthernetTypes.CreateSpec` objects that provide the configuration information of the created virtual Ethernet adapters. You can set the MAC address type to `EthernetTypes.MacAddressType.MANUAL`, `EthernetTypes.MacAddressType.GENERATED`, or `EthernetTypes.MacAddressType.ASSIGNED`. Select `MANUAL` to specify the MAC address explicitly.

You can specify also the physical resources that back a virtual Ethernet adapter by using the `EthernetTypes.BackingSpec.setType` method. The method takes as argument one of the following types: `EthernetTypes.BackingType.STANDARD_PORTGROUP`, `HOST_DEVICE`, `DISTRIBUTED_PORTGROUP`, or `OPAQUE_NETWORK`.

Java Example of Configuring the Virtual Machine Network

This example is based on the code in the `EthernetConfiguration.java` sample file.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private String vmName;
    private String datacenterName;
    private String stdPortgroupName;
    private String distPortgroupName;
    private String vmId;
    private List<String> createdNics = new ArrayList<String>();
    private Power powerService;
    private Ethernet ethernetService;

...
        this.ethernetService = vapiAuthHelper.getStubFactory().createStub(
            Ethernet.class, this.sessionStubConfig);

        // Get the virtual machine ID
        this.vmId = VmHelper.getVM(vapiAuthHelper.getStubFactory(),
            sessionStubConfig,
            vmName);

        // List all Ethernet adapters of the virtual machine
        List<EthernetTypes.Summary> nicSummaries = this.ethernetService.list(
            this.vmId);
        System.out.println("\n\n#### List of all Ethernet NICS on the VM:\n"
                           + nicSummaries);

        // Get info for each Ethernet adapter on the VM
        System.out.println("\n\n####Print info for each Ethernet NIC on the"
                                   + " vm.");
        for (EthernetTypes.Summary ethSummary : nicSummaries) {
```

```
            EthernetTypes.Info ethInfo = this.ethernetService.get(vmId,
                ethSummary.getNic());
            System.out.println(ethInfo);
        }

        // Create Ethernet NIC by using STANDARD_PORTGROUP with default settings
        String stdNetworkId = NetworkHelper.getStandardNetworkBacking(
            this.vapiAuthHelper.getStubFactory(), sessionStubConfig,
            this.datacenterName, this.stdPortgroupName);
        EthernetTypes.CreateSpec nicCreateSpec =
                new EthernetTypes.CreateSpec.Builder().setBacking(
                    new EthernetTypes.BackingSpec.Builder(
                        EthernetTypes.BackingType.STANDARD_PORTGROUP)
                            .setNetwork(stdNetworkId).build()).build();
        String nicId = this.ethernetService.create(this.vmId, nicCreateSpec);
        this.createdNics.add(nicId);
        EthernetTypes.Info nicInfo = this.ethernetService.get(this.vmId, nicId);

       // Update the Ethernet NIC with a different backing
        EthernetTypes.UpdateSpec nicUpdateSpec = new EthernetTypes.UpdateSpec.Builder().setBacking(
                    new EthernetTypes.BackingSpec.Builder(EthernetTypes.BackingType.STANDARD_PORTGROUP)
                            .setNetwork(stdNetworkId).build()).build();
        this.ethernetService.update(this.vmId, lastNicId, nicUpdateSpec);
        nicInfo = this.ethernetService.get(this.vmId, lastNicId);

        // Update the Ethernet NIC configuration
        nicUpdateSpec = new EthernetTypes.UpdateSpec.Builder()
            .setAllowGuestControl(false)
            .setStartConnected(false)
            .setWakeOnLanEnabled(false)
            .build();
        this.ethernetService.update(this.vmId, lastNicId, nicUpdateSpec);
        nicInfo = this.ethernetService.get(this.vmId, lastNicId);

        // Powering on the VM to connect the virtual Ethernet adapter to its backing
        this.powerService.start(this.vmId);
        nicInfo = this.ethernetService.get(this.vmId, lastNicId);

        // Connect Ethernet NIC after powering on the VM
        this.ethernetService.connect(this.vmId, lastNicId);
        nicInfo = this.ethernetService.get(this.vmId, lastNicId);

        // Disconnect Ethernet NIC after powering on VM
        this.ethernetService.disconnect(this.vmId, lastNicId);
        nicInfo = this.ethernetService.get(this.vmId, lastNicId);
...
```

Python Example of Configuring the Virtual Machine Network

This example is based on the code in the `ethernet.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

vm = None
vm_name = None
```

```
stub_config = None
ethernet_svc = None
cleardata = False
nics_to_delete = []
orig_nic_summaries = None
...

        server, username, password, cleardata, skip_verification, vm_name = \
            parse_cli_args_vm(testbed.config['VM_NAME_DEFAULT'])
        stub_config = vapiconnect.connect(server,
                                          username,
                                          password,
                                          skip_verification)

    global vm
    vm = get_vm(stub_config, vm_name)
    if not vm:
        exit('Sample requires an existing vm with name ({}). '
             'Please create the vm first.'.format(vm_name))
    print("Using VM '{}' ({}) for Disk Sample".format(vm_name, vm))

    # Get standard portgroup to use as backing for sample
    standard_network = network_helper.get_standard_network_backing(
        stub_config,
        testbed.config['STDPORTGROUP_NAME'],
        testbed.config['VM_DATACENTER_NAME'])

    # Create Ethernet stub used for making requests
    global ethernet_svc
    ethernet_svc = Ethernet(stub_config)
    vm_power_svc = Power(stub_config)
    nic_summaries = ethernet_svc.list(vm=vm)

    # Save current list of Ethernet adapters to verify that we have cleaned
    # up properly
    global orig_nic_summaries
    orig_nic_summaries = nic_summaries

    global nics_to_delete

    # Create Ethernet Nic using STANDARD_PORTGROUP with the default settings
    nic_create_spec = Ethernet.CreateSpec(
        backing=Ethernet.BackingSpec(
            type=Ethernet.BackingType.STANDARD_PORTGROUP,
            network=standard_network))
    nic = ethernet_svc.create(vm, nic_create_spec)
    nics_to_delete.append(nic)
    nic_info = ethernet_svc.get(vm, nic)

    # Create Ethernet Nic by using STANDARD_PORTGROUP
    nic_create_spec = Ethernet.CreateSpec(
        start_connected=True,
        allow_guest_control=True,
        mac_type=Ethernet.MacAddressType.MANUAL,
        mac_address='01:23:45:67:89:10',
        wake_on_lan_enabled=True,
        backing=Ethernet.BackingSpec(
```

```
            type=Ethernet.BackingType.STANDARD_PORTGROUP,
            network=standard_network))
    nic = ethernet_svc.create(vm, nic_create_spec)
    nics_to_delete.append(nic)
    nic_info = ethernet_svc.get(vm, nic)

    # Update the Ethernet NIC with a different backing
    nic_update_spec = Ethernet.UpdateSpec(
        backing=Ethernet.BackingSpec(
            type=Ethernet.BackingType.STANDARD_PORTGROUP,
            network=standard_network))
    ethernet_svc.update(vm, nic, nic_update_spec)
    nic_info = ethernet_svc.get(vm, nic)

    # Update the Ethernet NIC configuration
    nic_update_spec = Ethernet.UpdateSpec(
        wake_on_lan_enabled=False,
        mac_type=Ethernet.MacAddressType.GENERATED,
        start_connected=False,
        allow_guest_control=False)
    ethernet_svc.update(vm, nic, nic_update_spec)
    nic_info = ethernet_svc.get(vm, nic)

    # Powering on the VM to connect the virtual Ethernet adapter to its backing
    vm_power_svc.start(vm)
    nic_info = ethernet_svc.get(vm, nic)

    # Connect the Ethernet NIC after powering on the VM
    ethernet_svc.connect(vm, nic)

    # Disconnect the Ethernet NIC while the VM is powered on
    ethernet_svc.disconnect(vm, nic)
...
```

Managing Virtual Machines

Virtual machines can be configured like physical computers. You can change the guest operating system settings after installing VMware Tools. You can add and remove virtual machines from the vCenter inventory. You can also move virtual machines from one host or storage location to another.

Filtering Virtual Machines

You can retrieve commonly used information about virtual machines that match specific criteria. You can retrieve information for up to 4000 virtual machines in a single vCenter instance.

You can retrieve a list of virtual machines in a single vCenter instance by filtering the results based on a specific requirement. For example, you can use as filter criteria the power state of the virtual machines, or the host, cluster, data center, folder, or resource pool that must contain the virtual machines. In case you specify multiple filter criteria, only virtual machines that match all filter criteria are returned.

To retrieve a list of the virtual machines that match your specific criteria, call the `list` methods of the `VM` service. The method takes as parameter the `VMTypes.FilterSpec` instance that you can use to provide your filter criteria.

Java Example of Filtering Virtual Machines

The code example is based on the `VmHelper.java` sample file.

The following code example shows how you can retrieve the VM ID of a virtual machine with a specific name.

This example uses the information provided in Filtering Virtual Machines.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    public static String getVM(
        StubFactory stubFactory, StubConfiguration sessionStubConfig,
        String vmName) {
        VM vmService = stubFactory.createStub(VM.class, sessionStubConfig);

        // Get summary information about the virtual machine
        VMTypes.FilterSpec vmFilterSpec = new VMTypes.FilterSpec.Builder()
            .setNames(Collections.singleton(vmName)).build();
        List<VMTypes.Summary> vmList = vmService.list(vmFilterSpec);
        assert vmList.size() > 0 && vmList.get(0).getName().equals(
            vmName) : "VM with name " + vmName + " not found";

        return vmList.get(0).getVm();
    }
...
```

Python Example of Filtering Virtual Machines
This example is based on the code in the `vm_helper.py` sample file.

This example uses the information provided in Filtering Virtual Machines.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

def get_vms(stub_config, vm_names):
    """Return identifiers of a list of vms"""
    vm_svc = VM(stub_config)
    vms = vm_svc.list(VM.FilterSpec(names=vm_names))

    if len(vms) == 0:
        print('No vm found')
        return None

    print("Found VMs '{}' ({})".format(vm_names, vms))
    return vms
```

Installing VMware Tools
VMware Tools is a set of drivers and utilities that you install on the guest operating system of a virtual machine to enhance the performance of the guest OS. VMware Tools also improve the management of the virtual machine. For each guest OS, VMware provides a specific binary-compatible version of VMware Tools.

Before you install VMware tools, you must install and boot the guest operating system.

To mount and unmount the VMware Tools installer CD as a CD-ROM for the guest operating system, use the methods of the `com.vmware.vcenter.vm.tools.Installer` interface.

To mount the VMware Tools, call the `connect(vm_ID)` method of the `Installer` interface. On Windows guest operating systems with activated Autorun feature, this method automatically initiates the installation of the VMware Tools and requires a user input to complete. On other guest operating systems, this method only mounts the VMware Tools disk image on the virtual CD/DVD drive and the user is required to do some guest OS-specific actions. For example, for

some Linux distributions, the user is required to extract the contents of the VMware Tools installation archive and run the installer.

To unmount the VMware Tools installer CD image from the virtual CD/DVD drive, call the `disconnect(vm_ID)` method of the `Installer` interface.

To monitor the status of the VMware Tools installation, you can first retrieve the properties of the VMware Tools by calling the `get(vm_VM)` method of the `com.vmware.vcenter.vm.Tools` interface. The method returns a `ToolsTypes.Info` object which you can use to call the `getVersionStatus()` and `getRunState()` methods.

To upgrade the VMware Tools, call the `upgrade(vm_ID, commandLineOptions)` method of the `Tools` interface. The method takes as arguments the ID of the virtual machine on which the VMware Tools is installed and running. Use the `commandLineOptions` argument to specify the command-line options that you want to pass to the installer to modify the tools installation procedure. You can monitor the upgrade operation in the same way you monitor the installation operation.

To update the properties of the VMware Tools, call the `update(vm_ID, update_spec)` method of the `Tools` interface. Pass as argument a `ToolsTypes.UpdateSpec` object and define the tools upgrade policy settings for the virtual machine by calling the `setUpgradePolicy(upgradePolicy)` method.

Performing Virtual Machine Power Operations

You can start, stop, reboot, and suspend virtual machines by using the methods of the `Power` class.

A virtual machine can have one of the following power states:

- `PowerTypes.State.POWERED_ON` - Indicates that the virtual machine is running. If a guest operating system is not currently installed, you can perform the guest OS installation in the same way as for a physical machine.
- `PowerTypes.State.POWERED_OFF` - Indicates that the virtual machine is not running. You can still update the software on the physical disk of the virtual machine, which is impossible for physical machines.
- `PowerTypes.State.SUSPENDED` - Indicates that the virtual machine is paused and can be resumed. This state is the same as when a physical machine is in standby or hibernate state.

To perform a power operation on a virtual machine, you can use one of the methods of the `Power` class. Before you call one of the methods to change the power state of a virtual machine, you must first check the current state of the virtual machine by using the `Power.get` method. Pass as argument the virtual machine identifier.

Following is a list if the power operations:

- `Power.start` - Powers on a powered off or suspended virtual machine. The method takes as argument the virtual machine identifier.
- `Power.stop` - Powers off a powered on or suspended virtual machine. The method takes as argument the virtual machine identifier.
- `Power.suspend` - Pauses all virtual machine activity for a powered on virtual machine. The method takes as argument the virtual machine identifier.
- `Power.reset` - Shuts down and restarts the guest operating system without powering off the virtual machine. Although this method functions as a `stop` method that is followed by a `start` method, the two operations are atomic with respect to other clients, meaning that other power operations cannot be performed until the `reset` method completes.

Java Example of Powering On a Virtual Machine

This example is based on the code in the `EthernetConfiguration.java` sample file.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

    private String vmName;
    private String vmId;
    private Power powerService;
```

```
...
      this.powerService = vapiAuthHelper.getStubFactory().createStub(
            Power.class, this.sessionStubConfig);
      this.vmId = VmHelper.getVM(vapiAuthHelper.getStubFactory(),
            sessionStubConfig,
            vmName);
      this.powerService.start(this.vmId);

...
```

Python Example of Powering On a Virtual Machine

This example is based on the code in the `ethernet.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

   vm = None
   vm_name = None
   stub_config = None

   server, username, password, cleardata, skip_verification, vm_name = \
            parse_cli_args_vm(testbed.config['VM_NAME_DEFAULT'])
    stub_config = vapiconnect.connect(server, username, password, skip_verification)

    # Get the virtual machine ID
    vm = get_vm(stub_config, vm_name)

    # Create the Power stub  for running power operations on virtual machines
    vm_power_svc = Power(stub_config)
    vm_power_svc.start(vm)
```

Registering and Unregistering Virtual Machines

When you create a virtual machine, it becomes part of the vCenter inventory and is registered to the host and vCenter. If you remove a virtual machine from the vCenter inventory, it becomes unusable. Virtual machine files remain in the same datastore but you cannot power on the virtual machine when it is not registered in the inventory.

You can temporarily remove a virtual machine from vCenter by unregistering it. Virtual machine files are not deleted from the datastore. Though all high-level information about the virtual machine such as statistics, resource pool association, permissions, and alarms, is removed from the host and the vCenter instance. To remove a virtual machine from the inventory, call the `unregister(vm_ID)` method of the `VM` service and pass as argument the ID of the virtual machine.

To restore a virtual machine to the vCenter inventory, and make it usable again, call the `register(register_spec)` method of the `VM` service. You pass as argument a `VMTypes.RegisterSpec` instance that contains information about the current location of the virtual machine files on the datastore. You can also define the location within the vCenter inventory, for example, the cluster, folder, or the host, where you want to register the virtual machine. After registration, the virtual machine takes its resources (CPU, memory, and so on) from the resource pool or host to which it is registered.

If you no longer need a virtual machine and you want to free up datastore space, you can permanently delete a virtual machine from the inventory. Call the `delete(vm_ID)` method of the `VM` service and pass as argument the ID of the virtual machine. Upon a successful completion of the operation, the virtual machine files are removed from the datastore, including the configuration file and the virtual disk files.

Virtual Machine Guest Operations

The VMware Cloud Foundation APIs enable you to run operations on the guest operating system such as running scripts, performing power operations, and customizing the network and identity settings.

Upload and Run a Script on a Guest Operating System

You can create a directory, create files, copy a script, and run the script on a guest operating system.

Before you perform operations on the guest operating system, you must prepare the environment. You must create the `Process.CreateSpec` for initiating processes in the guest and create the `Transfer.CreateSpec` for the file transfer to or from the guest. The content of the optional `FileAttributeCreateSpec` associated with the `Transfer.CreateSpec` establishes the direction of the transfer and controls guest operating system specific file attributes. You must set up equivalent `_download` and `_upload` functions for URL management of the file transfer to or from the guest. You can also create an argument parser for standard inputs, such as server, user name, and password, and you can add custom input arguments.

1. Find the virtual machine on which the guest operating system runs, verify that VMware Tools is running, and provide credentials.
2. Create a temporary directory from which to run the script and capture any output.
3. Create temporary files for `stdout` and `stderr`.
4. Optional: Copy the script that you want to run.
5. Start the script and capture `stdout` and `stderr` in the temporary files that you created earlier.
6. Create a loop to handle processes that run longer.
7. Create a download URL and copy the results.
8. Clean up the temporary files and directories on the guest operating system.

Python Example of Uploading and Running a Script on a Guest Operating System

This example is based on the code in the `guest_ops.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

    # Create the Process.CreateSpec for initiating processes in the guest
    def _process_create_spec(self, path, args=None, dir=None, env={}):
        return Processes.CreateSpec(path=path,
                                    arguments=args,
                                    working_directory=dir,
                                    environment_variables=env)

    # Create the Transfer.CreateSpec for the file transfer to/from the guest
    def _create_transfer_spec(self,
                              path,
                              attributes=None):
        return Transfers.CreateSpec(attributes=attributes,
                                    path=path)

    # Create a FileAttributeCreateSpec for a generic (non-OS specific) guest
    def _fileAttributeCreateSpec_Plain(self,
                                       size,
                                       overwrite=None,
                                       last_modified=None,
                                       last_accessed=None):
        return Transfers.FileCreationAttributes(size,
                                                overwrite=overwrite,
```

```
                                          last_modified=last_modified,
                                          last_accessed=last_accessed)

    # Create a FileAttributeCreateSpec for a linux (Posix) guest
    def _fileAttributeCreateSpec_Linux(self,
                                       size,
                                       overwrite=None,
                                       last_modified=None,
                                       last_accessed=None,
                                       owner_id=None,
                                       group_id=None,
                                       permissions=None):
        posix = Transfers.PosixFileAttributesCreateSpec(owner_id=owner_id,
                                                        group_id=group_id,
                                                        permissions=permissions)
        return Transfers.FileCreationAttributes(size,
                                          overwrite=overwrite,
                                          last_modified=last_modified,
                                          last_accessed=last_accessed,
                                          posix=posix)

    def _download(self,
                  url,
                  expectedLen=None):
        urloptions = urlparse(url)
        # Skip server cert verification.
        # This is not recommended in production code.
        conn = httpclient.HTTPSConnection(urloptions.netloc,
                                          context=ssl._create_unverified_context())

        conn.request("GET", urloptions.path + "?" + urloptions.query)
        res = conn.getresponse()
        if res.status != 200:
            print("GET request failed with errorcode : %s" % res.status)
            raise HTTPError(res.status, res.reason)
        body = res.read().decode()

        return body

    def _upload(self, url, body):
        urloptions = urlparse(url)
        conn = httpclient.HTTPSConnection(urloptions.netloc,
                                          context=ssl._create_unverified_context())

        headers = {"Content-Length": len(body)}
        # Skip server cert verification.
        # This is not recommended in production code.
        conn.request("PUT", urloptions.path + "?" + urloptions.query,
                     body,
                     headers)
        res = conn.getresponse()
        if res.status != 200:
            print("PUT request failed with errorcode : %s" % res.status)
            raise HTTPError(res.status, res.reason)

        return res
```

```
    def __init__(self):
        # Create argument parser for standard inputs:
        # server, username, password, cleanup and skipverification
        parser = sample_cli.build_arg_parser()

        # Add your custom input arguments
        parser.add_argument('--vm_name',
                            action='store',
                            help='Name of the testing vm')
        parser.add_argument('--root_user',
                            action='store',
                            help='Administrator account user name')
        parser.add_argument('--root_passwd',
                            action='store',
                            help='Administrator account password')

        args = sample_util.process_cli_args(parser.parse_args())
        self.vm_name = args.vm_name
        self.root_user = args.root_user
        self.root_passwd = args.root_passwd

        self.cleardata = args.cleardata

        # Skip server cert verification if needed.
        # This is not recommended in production code.
        session = get_unverified_session() if args.skipverification else None

        # Connect to vSphere client
        self.client = create_vsphere_client(server=args.server,
                                            username=args.username,
                                            password=args.password,
                                            session=session)

    def run(self):
        # Using vAPI to find VM.
        filter_spec = VM.FilterSpec(names=set([self.vm_name]))
        vms = self.client.vcenter.VM.list(filter_spec)
        if len(vms) != 1:
            raise Exception('Could not locate the required VM with name ' +
                            self.vm_name + '. Please create the vm first.')
        if vms[0].power_state != 'POWERED_ON':
            raise Exception('VM is not powered on: ' + vms[0].power_state)
        vm_id = vms[0].vm

        # Check that vmtools svc (non-interactive user) is running.
        info = self.client.vcenter.vm.guest.Operations.get(vm_id)
        if info.guest_operations_ready is not True:
            raise Exception('VMware Tools/open-vm-tools is not running as required.')

        # Establish the user credentials that will be needed for all Guest Ops APIs.
        creds = Credentials(interactive_session=False,
                            user_name=self.root_user,
                            password=self.root_passwd,
                            type=Credentials.Type.USERNAME_PASSWORD)

        # Step 2 - Create a temporary directory from which to run the command and capture any output
        tempDir = self.client.vcenter.vm.guest.filesystem.Directories.create_temporary(
```

```
                vm_id, creds, '', '', parent_path=None)

        # Step 3 - Create temproary files to reveive stdout and stderr as needed.
        stdout = self.client.vcenter.vm.guest.filesystem.Files.create_temporary(
                  vm_id, creds, '', '.stdout', parent_path=tempDir)
        stderr = self.client.vcenter.vm.guest.filesystem.Files.create_temporary(
                  vm_id, creds, '', '.stderr', parent_path=tempDir)

        # Step 4 - (Optional) Copy the script to be run.
        scriptPath = self.client.vcenter.vm.guest.filesystem.Files.create_temporary(
                      vm_id, creds, '', '.sh', tempDir)

        # Create script contents and transfer to the guest.
        baseFN = os.path.basename(scriptPath)
        script = ('#! /bin/bash\n'
                  '#    ' +
                  baseFN + '\n'
                  '\n'
                  'sleep 5    # Adding a little length to the process.\n'
                  'ps -ef\n'
                  'echo\n'
                  'rpm -qa | sort\n'
                  '\n')
        print(script)
        attr = self._fileAttributeCreateSpec_Linux(size=len(script),
                                                   overwrite=True,
                                                   permissions='0755')
        spec = self._create_transfer_spec(path=scriptPath,
                                          attributes=attr)
        toURL = self.client.vcenter.vm.guest.filesystem.Transfers.create(vm_id,
                                                                         creds,
                                                                         spec)
        res = self._upload(toURL, script)

        # Check that the uploaded file size is correct.
        info = self.client.vcenter.vm.guest.filesystem.Files.get(vm_id,
                                                                 creds,
                                                                 scriptPath)
        if info.size != len(script):
            raise Exception('Uploaded file size not as epected.')

        # Step 5 - Start the program on the guest, capturing stdout and stderr in the separate temp files
obtained earlier.
        options = (" > " + stdout + " 2> " + stderr)

        spec = self._process_create_spec(scriptPath,
                                         args=options,
                                         dir=tempDir)
        pid = self.client.vcenter.vm.guest.Processes.create(vm_id, creds, spec)
        print('process created with pid: %s\n' % pid)

        # Step 6 - Need a loop to wait for the process to finish to handle longer running processes.
        while True:
            time.sleep(1.0)
            try:
                # List the single process for pid.
                result = self.client.vcenter.vm.guest.Processes.get(vm_id,
```

```
                                                                creds,
                                                                pid)
            if result.exit_code is not None:
                print('Command: ' + result.command)
                print('Exit code: %s\n' % result.exit_code)
                break
            if result.finished is None:
                print('Process with pid %s is still running.' % pid)
                continue
        except Exception as e:
            raise e

    # Step 7 Copy out the results (stdout).
    spec = self._create_transfer_spec(path=stdout)
    # Create the download URL
    fromURL = self.client.vcenter.vm.guest.filesystem.Transfers.create(vm_id,
                                                                       creds,
                                                                       spec)
    body = self._download(fromURL, expectedLen=info.size)
    print("-----------  stdout  ------------------")
    print(body)
    print("---------------------------------------")

    # Optionally the contents of "stderr" could be downloaded.

    # And finally, clean up the temporary files and directories on the
    # Linux guest.  Deleting the temporary diretory and its contents.
    self.client.vcenter.vm.guest.filesystem.Directories.delete(vm_id,
                                                               creds,
                                                               tempDir,
                                                               recursive=True)

...
```

Managing Data Sets

You can use data sets to share information between a virtual machine and its guest operating system.

Data sets provide a communication infrastructure that you can use to share information between the host and guest. This information can then be used by guest agents or applications to configure the guest itself or guest applications. The information is grouped into data sets, each of which contains key-value entries comprising the data. Each application that uses the service should have at least one unique data set in which to store its data to avoid conflict with other applications. Each data set has attributes that define its access control and interoperability configuration.

> **Note:**
> You should not store sensitive data, such as passwords or private keys, in plain text.

To perform data set operations, the virtual machine must be running virtual hardware version 20 or later.

You should modify each data set by using the application that originally created that data set. If your application modifies a data set owned by another application, the other application might stop working.

For more information about data sets, see *vSphere Virtual Machine Administration* and the *VMware Guest SDK Programming Guide*.

Data Set Operations

You can use the VMware Cloud Foundation API to manipulate entries in data sets.

You can use data set operations to share information between a virtual machine and its guest operating system. The available operations include listing data sets, retrieving data set information, creating, modifying, and deleting a data set. You can also manipulate individual entries in a data set. The available operations include listing entry keys in a data set, retrieving the value of an entry in a data set, modifying and deleting an entry in a data set.

Table 1884: User Operations

Operation	Description
List data sets	You can list all available data sets of a virtual machine.
Get data set	You can retrieve information about a specific data set.
Create data set	You can create a new data set.
Update data set	You can modify the attributes of a data set.
Delete data set	You can delete a data set.
List data set entry keys	You can list all entry keys in a data set.
Get data set entry key value	You can retrieve the value of a specific entry in a data set.
Set data set entry key value	You can create or update an entry in a data set. If the new entry does not contain a key that matches an existing key, the operation creates a new entry. If the new entry contains a key that matches an existing key, the new key overwrites the value of the existing key.
Delete data set entry key value	You can delete an entry in a data set.

Using Content Libraries

Use the VMware Cloud Foundation API to access the Content Library service and use it to store and share different types of content in your vSphere environment. Content libraries are vSphere container objects for storing and sharing OVF and OVA packages, VM templates, vApp templates, and other types of files.

You can use content libraries to share VM and vApp templates, and other types of files, such as ISO images, text files, and so on, across the vCenter instances in the same or different locations. Sharing templates across your virtual environment promotes consistency, compliance, efficiency, and automation in deploying workloads at scale.

You use library items to store and manage content in a content library. A single library item can contain one or more files. For example, an OVF package consists of a `.vmdk` file, manifest file, and others, but it is represented by a single library item.

Starting with vSphere 7.0 Update 3, you can add a security policy to a local or subscribed content library and thus protect your system when you synchronize or download library content from third party providers.

What Is a Content Library

A content library instance represents a container for a set of library items. A content library item instance represents the logical object stored in the content library, which might be one or more usable files.

You create and manage the content of a content library on a single vCenter instance, but you can distribute the content to other vCenter instances. Depending on your needs, you can maintain two types of content libraries: local and subscribed. You can shape the contents of a library item and then combine several library items in a local content library. Furthermore, you can publish the library to make its content available to other users.

Content Library Types

You can create two types of libraries, local and subscribed.

Local library. You can create a local library as the source for content you want to save or share. Create the local library on a single vCenter instance. You can add items to a local library or remove them. You can publish a local

	library and as a result this content library service endpoint can be accessed by other vCenter instances in your virtual environment. When you publish a library, you can configure a password for authentication.
Subscribed library.	You can create a subscribed library and populate its content by synchronizing to a local library. A subscribed library contains copies of the local library files or just the metadata of the library items. The local library can be located on the same vCenter instance as the subscribed library, or the subscribed library can reference a local library on a different vCenter instance. You cannot add library items to a subscribed library. You can only add items to the source library. After synchronization, both libraries will contain the same items.

Content Library Items

Library items are VM templates, vApp templates, or other VMware objects that can be contained in a content library.

VMs and vApps have several files, such as log files, disk files, memory files, and snapshot files that are part of a single library item. You can create library items in a specific local library or remove items from a local library. You can also upload files to an item in a local library so that the libraries subscribed to it can download the files to their NFS or SMB server, or datastore.

For information about the tasks that you can perform by using the content library service, see How to Create and Publish Content LibrariesWorking with Content Libraries.

Content Library Storage

When you create a local library, you can store its contents on a datastore managed by the vCenter instance or on a remote file system.

Depending on the type of storage that you have, you can use Virtual Machine File System (VMFS) or Network File System (NFS) for storing content on a datastore.

For storing content on a remote file system, you can enter the path to the NFS storage that is mounted on the Linux file system of the vCenter instance. For example, you can use the following URI formats: `nfs://<server>/<path>?version=4` and `nfs://<server>/<path>`. If you have a vCenter instance that runs on a Windows machine, you can specify the Server Massage Block (SMB) URI to the Windows shared folders that store the library content. For example, you can use the following URI format: `smb://<unc-server>/<path>`.

Java

This example is based on the code in the `LibraryCrud.java` sample file and shows how to store library content on a datastore.

For more information about storing the contents of a local library, see Content Library Storage.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a StorageBacking instance for storing the library content on a datastore.

    StorageBacking libraryBacking = new StorageBacking();
    libraryBacking.setType(Type.DATASTORE);

/*
 * Pass the value of the datastore ManagedObjectReference.
 * See the vSphere Web Services SDK Programming Guide
 * and the vSphere Web Services SDK samples. In addition, the  vSphere Automation SDK
 for Java provides
 * the VimUtil utility class in the vmware.samples.vim.helpers package.
 * You can use the utility to retrieve the ManagedObjectReference
 * of the datastore entity.
 */
    libraryBacking.setDatastoreId("datastore-123");

// Create a LibraryModel that represents a local library backed on a datastore.
```

```
LibraryModel libraryModel = new LibraryModel();
libraryModel.setName("AcmeLibrary");
libraryModel.setDescription("Local library backed by VC datastore");
libraryModel.setType(LibraryType.LOCAL);
libraryModel.setStorageBackings(Collections.singletonList(libraryBacking));
```

Python

This example is based on the code in the `library_crud.py` sample file and shows how to store library content on a datastore.

For more information about storing the contents of a local library, see Content Library Storage.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a StorageBacking instance of datastore type.
library_backing = library_client.StorageBacking()
library_backing.type = library_client.StorageBacking.Type.DATASTORE

# Pass the value of the datastore managed object reference.
# The  vSphere Automation SDK for Python contains
# the GetDatastoreByName class, which sample resource is located in the
# /client/samples/src/com/vmware/vcloud/suite/sample/vim/helpers/
 directory.
# You can use the utility to retrieve the managed object reference of the datastore
 entity.
library_backing.datastore_id = 'datastore-123'

# Create a LibraryModel that represents a local library backed on a datastore.
library_model = content_client.LibraryModel()
library_model.name = 'AcmeLibrary'
library_model.storage_backings = [library_backing]
```

How to Query Content Libraries

Use the VMware Cloud Foundation API to create queries for libraries that match your criteria. You can also retrieve a list of all libraries or only the libraries of a specific type.

Listing All Content Libraries

You can retrieve a list of all content library IDs in your virtual environment, regardless of their type, by using the `Library` service.

You can use the `list` function to retrieve all local and subscribed libraries in your system.

Java

This example shows how to retrieve a list of all content libraries.

This example uses the steps that are described in Listing All Content Libraries.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Access the Library Service.
   Library libraryService = vapiAuthHelper.getStubFactory().createStub(Library.class,
 sessionStubConfig);

// List all content libraries.
   List<String> allLibraries = libraryService.list();
```

```
   System.out.println("List of all library identifiers: /n");
   for (String cl : allLibraries) {
         System.out.println(cl);
   }
```

Python

This example shows how to retrieve a list of all content libraries.

This example uses the steps that are described in Listing All Content Libraries.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

library_stub = content_client.Library(my_stub_config)
libraries = library_stub.list()
print('List of all library identifiers:')
for library_id in library_ids :
  library = library_stub.get(library_id)
  print('Library ID {}: {}'.format(library_id, library.name))
```

Listing Content Libraries of a Specific Type

You can use the VMware Cloud Foundation API to retrieve content libraries of a specific type. For example, you can list only the local libraries in your virtual environment.

If you want to retrieve only a list of the local libraries, you must retrieve the `LocalLibrary` service and use the `list` function on the `LocalLibrary` service. To list only subscribed libraries, you must retrieve the `SubscribedLibrary` service and call the `list` function on the `SubscribedLibrary` service.

Listing Content Libraries by Using Specific Search Criteria

You can filter the list of content libraries and retrieve only the libraries that match your specific criteria. For example, you might want to publish all local libraries with a specific name.

To a filter with specific search criteria, call the `find (find_spec)` function of the `Library` service. Pass as argument a `LibraryTypes.FindSpec` instance that contains your search criteria. Upon a successful completion of the call, you receive a list of all content libraries that match your search criteria.

Java

This example retrieves a list of all local libraries with the name `AcmeLibrary` that exist in your virtual environment.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a FindSpec instance to set your search criteria.
   FindSpec findSpec = new FindSpec();

// Filter the local content libraries by using a library name.
   findSpec.setName("AcmeLibrary");
   findSpec.setType(LibraryType.LOCAL);
   List<String> ids = libraryService.find(findSpec);
```

Python

This example retrieves a list of all local libraries with the name `AcmeLibrary` that exist in your virtual environment.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a FindSpec object to specify your search criteria.
```

```
find_spec = content_client.Library.FindSpec()
find_spec.name = 'AcmeLibrary'
find_spec.type = content_client.LibraryModel.LibraryType.LOCAL

# Invoke the find() function by using the FindSpec instance.
library_stub = content_client.Library(my_stub_config)
library_ids = library_stub.find(find_spec)
```

How to Create and Publish Content Libraries

The Content Library API provides services that allow you to create and manage content libraries programmatically. You can create a local library and publish it for the entire virtual environment. You can also subscribe to use the contents of a local library and enable automatic synchronization to ensure that you have the latest content.

Create a Local Content Library

You can create a local content library programmatically by using theVMware Cloud Foundation API. The API allows you to populate the content library with OVF and vApp templates. You can use these templates to deploy virtual machines or vApps in your virtual environment.

Required privileges:

- **Content library** > **Create local library** on thevCenter instance where you want to create the library.
- **Datastore** > **Allocate space** on the destination datastore.

1. Access the `LocalLibrary` service that provides support for creating local content libraries.
2. Create a `StorageBacking` instance and define the storage location.
3. Optional: Create a `SecurityPolicies` instance to define the security policy rules for the library. For vSphere 7.0 Update3, you can only define rules for an OVF and OVA templates in the library by using the `OVF_STRICT_VERIFICATION` security rule. When the OVF security policy is configured for a local content library, an OVF or OVA template can be synchronized or imported to the library only after its certificate is validated against a trusted certificate authority.
4. Create a `LibraryModel` instance and set the properties of the new local library.

   If you want to apply a security policy on the local library, you must use the `setSecurityPolicyId(java.lang.String securityPolicyId)` method or the `security_policy_id` parameter of the `LibraryModel` instance.
5. Call the create function on the `LocalLibrary` object and pass the `LibraryModel` as a parameter.

A local content library is created on thevCenter instance and you can edit its contents.
You maintain the contents of the local library by managing its library items. See How To Use Library Items and Content Library Support for OVF and OVA Packages. You can also share the library content by publishing the local library. See Publish an Existing Content Library.

Java

This example is based on the code in the `LibraryCrud.java` sample file.

This example uses the steps that are described in the Create a Local Content Library procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a StorageBacking instance to back the library content on the local file
 system.
    StorageBacking libraryBacking = new StorageBacking();
    libraryBacking.setType(Type.OTHER);
    libraryBacking.setStorageUri(URI.create("file:///tmp"));
    libraryModel.setStorageBackings(Collections.singletonList(libraryBacking));
```

```
// Create a LibraryModel that represents a local library.
    LibraryModel libraryModel = new LibraryModel();
    libraryModel.setType(LibraryModel.LibraryType.LOCAL);
    libraryModel.setName("AcmeLibrary");

// Access the LocalLibrary service by using the  endpoint.
    LocalLibrary localLibraryService =
 this.vapiAuthHelper.getStubFactory().createStub(LocalLibrary.class,
 sessionStubconfig);

// Call the create method of the LocalLibrary service passing as an
// argument the LibraryModel instance.
    String libraryId = localLibraryService.create(UUID.randomUUID().toString(),
 libraryMod
```

Python

This example creates a local library with name `AcmeLibrary` , which is stored on the local file system wherevCenter runs.

This example uses the steps that are described in the Create a Local Content Library procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create a storage backing instance on a local file system.
library_backing = library_client.StorageBacking()
library_backing.type = library_client.StorageBacking.Type.OTHER
library_backing.storage_uri = 'file:///tmp'

# 2 - Create a Library model to specify properties of the new library.
library_model = content_client.LibraryModel()
library_model.type = content_client.LibraryModel.LibraryType.LOCAL
library_model.name = 'AcmeLibrary'
library_model.storage_backings = [library_backing]

# 3 - Call the create() method, passing the library model as a parameter.
idem_token = str(uuid.uuid4())
local_library_stub = content_client.LocalLibrary(my_stub_config)
library_id = local_library_stub.create(create_spec=library_model,
                                       client_token=idem_token)
```

Specifying a Storage Policy for a Content Library

As of VCF 9.1, storage location can be either a Datastore, Storage Policy, or Other (URI) type. If the storage location of a content library is specified as a Storage Policy, a compatible storage backing will be selected based on policy compatibility.

When creating a content library, you can specify a storage policy without explicitly specifying a datastore. You may still specify a datastore, but not both datastore and storage policy. You can view a content library, given its storage policy and the datastore it resides on. You can migrate a policy-governed content library to a different datastore with a compatible storage policy.

If multiple compatible datastores for a given storage policy exist, the content library will be placed on the datastore with the most free space. Storage backing for a content library might differ from backing for a contained library item.

The following REST example shows a local content library with storage policy requesting vSAN backing.

```
curl -X POST 'https://IP.addr.of.VC/api/content/local-library'
 -H 'Cookie: vmware-api-session-id=cookie' -H 'Content-Type: application/json' \
 --data '{
   "name": "lib-with-storage-policy",
```

```
    "storage_backings": [
       {
          "type": "STORAGEPOLICY",
          "storage_policy": {
             "policy":"vSAN Default storage policy"
          }
       }
    ],
    "type": "LOCAL",
    "optimization_info": {
       "optimize_remote_publishing": false
    }
 }'
```

Publish an Existing Content Library

To make the library content available for other vCenter instances across the VMware Cloud Foundation environment, you must publish the library. Depending on your workflow, select a method for publishing the local library. You can publish a local library that already exists in your VMware Cloud Foundation environment.

1. Retrieve a reference to the `LocalLibrary` service.
2. Retrieve an existing local library by using the library ID.
3. Create a `PublishInfo` instance to define how the library is published.
4. Specify the authentication method to be used by a subscribed library to authenticate to the local library. You can enable either basic authentication or no authentication. Basic authentication requires a user name and password.
5. Optional: If you publish the library with basic authentication, you must specify a user name and password for the `PublishInfo` instance, which must be used for authentication.

   Important:
   Use the predefined user name `vcsp` or leave the user name undefined. You must set only a password to protect the library.
6. Set the local library to published.
7. Use the retrieved local library to configure it with the `PublishInfo` instance.
8. Update the properties of the `LibraryModel` object returned for the local library.

Java

This example is based on the code in the `LibraryPublishSubscribe.java` sample file.

This example uses the steps that are described in the Publish an Existing Content Library procedure.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Access the LocalLibrary service.
      LocalLibrary localLibraryService =
 this.vapiAuthHelper.getStubFactory().createStub(LocalLibrary.class,
 sessionStubconfig);

// Retrieve an existing local library.
      LibraryModel libraryModel = localLibraryService.get(libraryId);
      PublishInfo publishInfo = new PublishInfo();

// Configure how the local library is published by using BASIC authentication.
      publishInfo.setUserName("vcsp");
```

```
        publishInfo.setPassword("password".toCharArray());
        publishInfo.setAuthenticationMethod(AuthenticationMethod.BASIC);

    // Set the local library to published and update the library instance.
        publishInfo.setPublished(true);
        libraryModel.setPublishInfo(publishInfo);
        localLibraryService.update(libraryModel.getId(), libraryModel);
```

Python

This example is based on the code in the `library_publish_subscribe.py` sample file.

This example uses the steps that are described in the Publish an Existing Content Library procedure.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Retrieve an existing local library.
local_library_stub = content_client.LocalLibrary(my_stub_config)
local_library = local_library_stub.get(my_library_id)

# Specify how the local library is published, using BASIC authentication.
publish_info = library_client.PublishInfo()
publish_info.user_name = 'vcsp' # Can omit this value; if specified, it must
 be 'vcsp'.
publish_info.password = 'password'
publish_info.authentication_method =
 library_client.PublishInfo.AuthenticationMethod.BASIC
publish_info.published = True

# Update the LibraryModel object retieved in step 1
# and configure it with the PublishInfo object.
local_library.publish_info = publish_info

# Use the LibraryModel object to update the library instance.
local_library_stub.update(library_id=my_library_id,
update_spec=local_library)
```

Publish a Library at the Time of Creation

You can publish a local library at the time of creation to enable other libraries to subscribe and use the library content.

1. Retrieve the `LocalLibrary` service.
2. Create a `PublishInfo` instance to define how the library is published.
3. Specify the authentication method to be used by a subscribed library to authenticate to the local library.

   You can enable either basic authentication or no authentication on the library. Basic authentication requires a user name and password.
4. Optional: If you publish the library with basic authentication, you must specify a user name and password for the `PublishInfo` instance, which must be used for authentication.

   > **Important:**
   > Use the predefined user name `vcsp` or leave the user name undefined. You must set only a password to protect the library.
5. Optional: Create a `SecurityPolicies` instance to define the security policy rules for the library. For vSphere 7.0 Update3, you can only define rules for an OVF and OVA templates in the library by using the `OVF_STRICT_VERIFICATION` security rule. When the OVF security policy is configured for a local content library,

an OVF or OVA template can be synchronized or imported to the library only after its certificate is validated against a trusted certificate authority.

6. Create a `LibraryModel` instance and configure the instance.

   If you want to apply a security policy on the local library, you must use the `setSecurityPolicyId(java.lang.String securityPolicyId)` method or the `security_policy_id` parameter of the `LibraryModel` instance.

7. Set the library type to local and use the configured `PublishInfo` instance to set the library to published.
8. Define where the content of the local library is stored by using the `StorageBacking` class.
9. Create a published local library.

Subscribe to a Content Library

You can subscribe to local content libraries. When you subscribe to a library, you must specify the backing storage for the library content. If the library requires basic authentication, you must also provide the correct user name and password.

Required privileges:

- **Content library** > **Create subscribed library** on the vCenter instance where you want to create the library.
- **Datastore** > **Allocate space** on the destination datastore.

> **Note:**
> If you subscribe to libraries created with basic authentication on a vCenter instance, make sure that you pass `vcsp` as an argument for the user name.

1. Create a `StorageBacking` instance to define the location where the content of the subscribed library is stored.
2. Create a `SubscriptionInfo` instance to define the subscription behavior of the library.
   a) Provide the mechanism to be used by the subscribed library to authenticate to the published library.

      You can select between no authentication and basic authentication depending on the settings of the published library you subscribe to. If the library is published with basic authentication, you must set the basic authentication in the `SubscriptionInfo` instance. To match the credentials of the published library, set the user name and the password of the `SubscriptionInfo` instance.
   b) Provide the URL to the endpoint where the metadata of the published library is stored.
   c) Define the synchronization mechanism of the subscribed library.

      You can select between two synchronization modes: automatic and on demand. If you enable automatic synchronization for a subscribed library, both the content and the metadata are synchronized with the published library. To save storage space, you can synchronize the subscribed library on demand and update only the metadata for the published library content.
   d) Set the thumbprint that is used for validating the certificate of the published library.
3. Create a `LibraryModel` instance and set the library type to subscribed (`LibraryModel.LibraryType.SUBSCRIBED`).
4. Access the `SubscribedLibrary` service and create the subscribed library instance.

Java

This example is based on the code in the `LibraryPublishSubscribe.java` sample file.

This example uses the steps that are described in the Subscribe to a Content Library procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a StorageBacking instance to store
// the contents of the subscribed library on the local file system.
       StorageBacking libraryBacking = new StorageBacking();
       libraryBacking.setType(StorageBacking.Type.OTHER);
       libraryBacking.setStorageUri(URI.create("/mnt/nfs/cls-root"));
```

```
// Create a new SubscriptionInfo object to define the subscription
// behavior of the library.
       SubscriptionInfo subscriptionInfo = new SubscriptionInfo();
       subscriptionInfo.setAuthenticationMethod

 (com.vmware.content.library.SubscriptionInfo.AuthenticationMethod.BASIC);
       subscriptionInfo.setUserName("libraryUser");
       subscriptionInfo.setPassword("password".toCharArray());
       subscriptionInfo.setSubscriptionUrl(URI.create("https://
www.acmecompary.com/library_inventory/lib.json"));

// Specify that the content of the subscribed library will be downloaded
 immediately.
       subscriptionInfo.setAutomaticSyncEnabled(true);

// Set an SHA-1 hash of the SSL certificate of the remote endpoint.

 subscriptionInfo.setSslThumbprint("9B:00:3F:C4:4E:B1:F3:F9:0D:70:47:48:E7:0B:D1:A7:0E:DE:60:A5

// Create a new LibraryModel object for the subscribed library.
       LibraryModel libraryModel = new LibraryModel();
       libraryModel.setType(LibraryModel.LibraryType.SUBSCRIBED);
       libraryModel.setName("SubscrLibrary");

// Attach the storage backing and the subscription info to the library model.

 libraryModel.setStorageBackings(Collections.singletonList(libraryBacking));
       libraryModel.setSubscriptionInfo(subscriptionInfo);

// Create the new subscribed library.
       String clientToken = UUID.randomUUID().toString();
       SubscribedLibrary subscribedLibService =
 this.vapiAuthHelper.getStubFactory().createStub(SubscribedLibrary.class,
 sessionStubconfig);
       String subscribedLibId = subscribedLibService.create(clientToken,
 libraryModel);
```

Python

This example is based on the code in the `library_publish_subscribe.py` sample file.

This example uses the steps that are described in the Subscribe to a Content Library procedure.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a StorageBacking instance on a local file system.
library_backing = library_client.StorageBacking()
library_backing.type = library_client.StorageBacking.Type.OTHER
library_backing.storage_uri = '/mnt/nfs/cls-root'

# Create a new SubscriptionInfo object to describe the subscription behavior.
subscription_info = library_client.SubscriptionInfo()
subscription_info.authentication_method =
 library_client.SubscriptionInfo.AuthenticationMethod.BASIC
subscription_info.user_name = 'libraryUser'
subscription_info.password = 'password'
subscription_info.subscription_url = 'https://www.acmecompary.com/
library_inventory/lib.json'
```

```
subscription_info.automatic_sync_enabled = True
subscription_info.ssl_thumbprint =
 '9B:00:3F:C4:4E:B1:F3:F9:0D:70:47:48:E7:0B:D1:A7:0E:DE:60:A5'

# Create a new LibraryModel object for the subscribed library.
library_model = content_client.LibraryModel()
library_model.type = content_client.LibraryModel.LibraryType.SUBSCRIBED
library_model.name = 'subscrLibrary'

# Attach the storage backing and the subscription info to the library model.
library_model.storage_backings = [library_backing]
library_model.subscription_info = subscription_info

# Create the new library instance.
idem_token = str(uuid.uuid4())
local_library_stub = content_client.LocalLibrary(my_stub_config)
library_id =
 local_library_stub.create(create_spec=library_model,client_token=idem_token)
```

Synchronize a Subscribed Content Library

When you subscribe to a published library, you can configure the settings for downloading and updating the library content.

- You can enable automatic synchronization of the subscribed library and download a copy of the content of the local library immediately.
- You can save storage space and download only the metadata for the items that are part of the local library.

To ensure that your subscribed library contains the most recent published library content, you can force a synchronization task.

1. Access the `SubscribedLibrary`VMware Cloud Foundation service.
2. Retrieve the subscribed library ID from the `SubscribedLibrary` service.
3. Force the synchronization of the subscribed library.

The synchronization operation depends on the update settings of the subscribed library. If the subscribed library is configured to update only on demand, only the metadata of the library items will be synchronized.

Editing the Settings of a Content Library

You can update the settings of content library types in your virtual environment by using the vSphere Automation API.

Table 1885: Options for Updating a Content Library

Content Library Types	Option
Local content library	You can change the settings of a local library before calling the update function on the `LocalLibrary` object: Before a library is published, you can edit the name, description, version, and so on. After a library is published, you must first retrieve the `PublishInfo` instance of the published library you want. You can use the instance to configure the following settings. • Unpublish the local library. • Change the authentication method of the library. • Change the password that must be used for authentication.
Subscribed content library	You can edit the settings of a subscribed library if you retrieve the `SubscriptionInfo` instance associated with it. To apply the changes, you must update the library by using the `SubscribedLibrary` object. You can configure the following settings: • The authentication method required by the local library • The user name and password of the subscribed library for authentication to the local library • The method for synchronizing the metadata and the content of the subscribed library • The thumbprint used for validating the SSL certificate of the local library

Note:
Starting with vSphere 7.0 Update 3, you can use the Security Policies service to secure a content library. You can set, update, or remove a security policy when you update a local or subscribed content library.

- To update or set a security policy, pass `false` to the `unset_security_policy_id` / `setUnsetSecurityPolicyId(java.lang.Boolean unsetSecurityPolicyId)` parameter and then use `security_policy_id` / `setSecurityPolicyId(java.lang.String securityPolicyId)` to pass the security policy identifier.
- To remove a security policy, pass `true` to the `unset_security_policy_id` / `setUnsetSecurityPolicyId(java.lang.Boolean unsetSecurityPolicyId)` parameter of the `LibraryModel` update specification.

Convert Subscribed Content Library to Published Local Library

With VCF 9.1, to avoid loss of content in case the original published local library is not available, a subscribed content library can be converted to a published local content library.

For example, when you decommission a vCenter instance that hosts a published library serving subscribed libraries, this feature allows a subscribed library on another vCenter to convert into a published library, becoming the new content distributor and maintaining content availability.

Content library conversion is a feature available in both vCenter UI and REST API, and requiring maintenance mode. After content library conversion, other subscribers must manually update the old URL to the converted one, using either the vCenter UI or the `Update Library` API.

Implementation involves two new keywords for the post action, similar to the pre-existing action=migrate keyword. The `enter-maintenance` keyword shuts down access to the content library so it can be reconfigured as a published local library, including possible transfer of content library items. The `exit-maintenance` keyword resumes operations.

```
POST /api/content/library/{library_id}?action=enter-maintenance
POST /api/content/library/{library_id}?action=exit-maintenance
```

VCF Automation provides access to this feature, for durability of tenant access to content libraries. Conversion is an implicit operation when VCF decommissions a vCenter that is hosting a publisher library. Required privileges include ContentLibrary.EnterMaintenance, ContentLibrary.ConvertLibrary, and ContentLibrary.ExitMaintenance. Here is a Python code snippet showing an API for the conversion:

```
try:
   conversion_spec = Library.ConversionSpec(Library.ConversionSpec.ConversionType.TO_PUBLISHED_LIBRARY)
   self.library.convert(subscribed_library_id, conversion_spec)
except NotAllowedInCurrentState as e:
   validate_exception(e, ClExceptionId.LIBRARY_ITEM_OUT_OF_SYNC)
```

Removing the Content of a Subscribed Library

You can free storage space in your virtual environment by removing the subscribed library content that you no longer need.

You can create a subscribed library with the option to download the library content on demand. As a result, only the metadata for the library items is stored in the associated with the subscribed library storage. When you want to deploy a virtual machine from a VM temple in the subscribed library, you must synchronize the subscribed library to download the entire published library content. When you no longer need the VM template, you can call the `evict` function on the `SubscribedLibrary` service. You must provide the subscribed library ID to this function. As a result, the subscribed library content that is cached on the backing storage is deleted.

If the subscribed library is not configured to synchronize on demand, an exception is thrown. In this case the subscribed library always attempts to have the most recent published library content.

Delete a Content Library

When you no longer need a content library, you can invoke the `delete` method on either the `LocalLibrary` or the `SubscribedLibrary` service depending on the library type.

1. Access the `SubscribedLibrary` or the `LocalLibrary` service by using the VMware Cloud Foundation Endpoint.
2. Retrieve the library ID you want to delete.
3. Call the `delete` function on the library service and pass the library ID as argument.

   All library items cashed on the storage backing are removed asynchronously. If this operation fails, you must manually remove the content of the library.

How To Use Library Items

A library item groups multiple files within one logical unit. Use the VMware Cloud Foundation API to perform various tasks with the items in a content library.

You can upload files to a library item in a local library and update existing items. You can download the content of a library item from a subscribed library and use the item, for example, to deploy a virtual machine. You can remove the content of a library item from a subscribed library to free storage space and keep only the metadata of the library item. When you no longer need local library items, you can delete them and they are removed from the subscribed library when a synchronization task is completed.

You can create a library item from a specific item type, for example `.ovf` and VM template. The Content Library service must support the library item type to handle the item correctly. If no support is provided for a specified type, the Content Library service handles the library item in the default way, without adding metadata to the library item or guiding the upload process. For information about the supported VM template types, see the *vSphere Virtual Machine Administration* documentation.

Create an Empty Library Item

You can create as many library items as needed and associate them with a local content library.

1. Access the `Item` service by using the VMware Cloud Foundation endpoint.
2. Instantiate the `ItemModel` class.
3. Define the settings of the new library item.
4. Associate the library item with an existing local library.
5. Invoke the `create` function on the `Item` object to pass the library item specification and the unique client token.

Upload content to the new library item. See Upload a File from a Local System to a Library Item and Upload a File from a URL to a Library Item.

Java

This example shows how to create an empty library item that stores an ISO image file.

This example uses the steps that are described in the Create an Empty Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create an instance of the ItemModel class and specify the item settings.
ItemModel libItemSpec = new ItemModel();
libItemSpec.setName("ESX ISO image");
libItemSpec.setDescription("ISO image with the latest security patches for ESX");
libItemSpec.setType("iso");

// Associate the item with an existing content library.
libItemSpec.setLibraryId("<content_library_ID>");

// Create the new Item instance, using the specified model.
Item libItemService = this.vapiAuthHelper.getStubFactory().createStub(Item.class,
 sessionStubconfig);
String itemID = UUID.randomUUID().toString();
String newItem = libItemService.create(itemID, libItemSpec);
```

Python

This example shows how to create an empty library item that stores an ISO image file.

This example uses the steps that are described in the Create an Empty Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create an instance of the ItemModel class and specify the item settings.
item_model = library_client.ItemModel()
item_model.name = 'ESX ISO image'
item_model.description = 'ISO image with the latest security patches for ESX'
item_model.type = 'iso'

# 2 - Associate the new item with an existing library.
item_model.library_id = my_library_id

# 3 - Create the new instance of the Item class, using the specified model.
idem_token = str(uuid.uuid4())
item_stub = library_client.Item(my_stub_config)
item_id = item_stub.create(create_spec=item_model, client_token=idem_token)
```

Querying Library Items

You can perform numerous query operations on library items.

You can retrieve a list of all items in a library, retrieve a library item that has a specific type or name, and find a library item that is not cached on the disk. You can then update the library item content from the subscribed library.

List Library Items

You can use the `list` method of the `Item` object to retrieve a list of all items in a particular library.

Verify that you have access to the `Item` service.

1. Retrieve the ID of the content library whose items you want to list.
2. List the items of the specific library.
3. Retrieve a list of the files that belong to a library item.

   You can see an example query operation in the code example for Edit the Settings of a Library Item. The beginning of the example lists the items of a published library and prints a list with the names and size of each file in the listed items.

List Library Items That Match Specific Criteria

You can filter the items contained in a library and retrieve only the items matching specific criteria. For example, you might want to remove or update only specific items in a library.

Verify that you have access to the `Item` service.

1. Create an instance in the `FindSpec` class.
2. Specify the filter properties by using the `FindSpec` instance.
3. List the items matching the specified filter.

A list of items matching the filter criteria is created as a result.

Edit the Settings of a Library Item

You can edit the name, description, and type of a library item.

Verify that you have access to the `Item` service.

1. Retrieve the item that you want to update.
2. Create an `ItemModel` instance.
3. Change the human-readable name and description of the library item.
4. Update the library item with the configured item model.

The settings of the library item are sucessfully edited.

Java

This example shows how to find an item by using the item name and then how to change the name and description of the retrieved item.

This example uses the steps that are described in the Edit the Settings of a Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// List the items in a published library
     Item libItemService = this.vapiAuthHelper.getStubFactory().createStub(Item.class,
 sessionStubconfig);
     List<String> itemIds = libItemService.list(libraryId.getId());
     for (String itemId : itemIds) {
         ItemModel singleItem = libItemService.get(itemId);

// List the files uploaded to each library item and print their names and size
```

```
        com.vmware.content.library.item.File itemFilesService =
 this.vapiAuthHelper.getStubFactory().createStub(com.vmware.content.library.item.File.class,
 sessionStubconfig);
        List<com.vmware.content.library.item.FileTypes.Info> fileInfos =
               itemFilesService.list(itemId);
            for (com.vmware.content.library.item.FileTypes.Info singleFile :
 fileInfos) {
                System.out.println("Library item with name "+ singleFile.getName() +
 " has size
                                        " + singleFile.getSize());
            }

// Change the name and description of the library item with the specified name
               if (singleItem.getName().equals("simpleVmTemplate")) {
                    ItemModel libItemUpdated = new ItemModel();
                    libItemUpdated.setName("newItemName");
                    libItemUpdated.setDescription("Description of the newItemName");
                    libItemService.update(singleItem.getId(), libItemUpdated);
                }
        }
```

Python

This example shows how to find an item by using the item name and then how to change the name and description of the retrieved item.

This example uses the steps that are described in the Edit the Settings of a Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - List the items in a published library.
item_stub = library_client.Item(my_stub_config)
item_ids = item_stub.list(my_library_id)

# 2 - List the files uploaded to each library item and print their names and sizes.
file_stub = item_client.File(my_stub_config)
for item_id in item_ids :
  item = item_stub.get(item_id)
  file_infos = file_stub.list(item_id)
  for file_info in file_infos :
    print('Library item {} has file {} with size {}'.format(item.name, file_info.name,
 file_info.size))

# 3 - For a library item with a specified name,
#     create an ItemModel to change the name and description of the library item.
if item.name == 'simpleVmTemplate' :
  print('Library item {} with description {}'.format(item.name, item.description))
  item_model = library_client.ItemModel()
  item_model.name = 'newItemName'
  item_model.description = 'Description of the newItemName'
  item_stub.update(library_item_id=item_id,
                   update_spec=item_model)
  print('has been changed to:')
  print('library item {} with description {}'.format(item_model.name,
 item_model.description))
```

Upload a File from a Local System to a Library Item

You can upload different types of files from a local system to a library item that you want to use in the VMware Cloud Foundation environment.

- Create an empty library item. See Create an Empty Library Item.
- Verify that you have access to the `UpdateSession` and `File` services.

1. Create an `UpdateSessionModel` instance to track the changes that you make to the library item.
2. Create an update session by using the `UpdateSession` service.
3. Create an `AddSpec` instance to describe the upload method and other properties of the file to be uploaded.
4. Create the request for changing the item by using the `File` service.
5. Upload the file that is on the local system.
6. Complete and delete the update session to apply the changes to the library item.

The file is uploaded to the library item.

Java

This example shows how to upload an ISO image file from a local system to a library item.

This example uses the steps that are described in the Upload a File from a Local System to a Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Access the com.vmware.content.library.item.updatesession.File.
// and the UpdateSession services by using the vSphere Automation Endpoint.
    File uploadFileService =
 this.vapiAuthHelper.getStubFactory().createStub(File.class, sessionStubconfig);
    UpdateSession uploadService=
 this.vapiAuthHelper.getStubFactory().createStub(UpdateSession.class,
 sessionStubconfig);

// Create an UpdateSessionModel instance to track the changes you make to the item.
    UpdateSessionModel updateSessionModel = new UpdateSessionModel();
    updateSessionModel.setLibraryItemId(newItem);

// Create a new update session.
    String clientToken = UUID.randomUUID().toString();
    String sessionId = uploadService.create(clientToken, updateSessionModel);

// Create an instance of the HttpClient class which is part of the
// com.vmware.vcloud.suite.samples.common package.
    try {
       HttpClient httpClient = new HttpClient(true);

// Create a new AddSpec instance to describe the properties of the file to be
 uploaded.
       FileTypes.AddSpec fileSpec = new FileTypes.AddSpec();
       fileSpec.setName("ESX patch");
       fileSpec.setSourceType(FileTypes.SourceType.PUSH);

// Link the ISO file specification to the update session.
       FileTypes.Info fileInfo = uploadFileService.add(sessionId, fileSpec);

// Use the HTTP library to upload the file to the library item.
```

```
        URI uploadUri = fileInfo.getUploadEndpoint().getUri();
        java.io.File file = new java.io.File("/updates/esx/esx_patch.iso");
        String transferUrl = uploadUri.toURL().toString();
        httpClient.upload(file, transferUrl);

// Mark the upload session as completed.
        uploadService.complete(sessionId);
    } finally {
         uploadService.delete(sessionId);
    }
```

Python

This example shows how to upload an ISO image file from the local system to a library item.

This example uses the steps that are described in the Upload a File from a Local System to a Library Item procedure.

> **Note:**
> For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create an instance of the ItemModel class and specify the item settings.
item_model = library_client.ItemModel()
item_model.name = 'ESX patches'
item_model.description = 'ESX security patches'
item_model.type = 'iso'
item_model.library_id = my_library_id
idem_token = str(uuid.uuid4())
item_stub = library_client.Item(my_stub_config)
item_id = item_stub.create(create_spec=item_model,
client_token=idem_token)

# 2 - Create an UpdateSessionModel instance to track the changes you make to the item.
update_session_model = item_client.UpdateSessionModel()
update_session_model.library_item_id = item_id

# 3 - Create an update session from the model.
idem_token = str(uuid.uuid4())
update_session_stub = update_session_client.UpdateSession(my_stub_config)
session_id = update_session_stub.create(create_spec=update_session_model
client_token=idem_token)

try :
  # 4 - Create a new AddSpec instance to describe the properties of the file to be
 uploaded.
  file_spec = update_session_client.AddSpec()
  file_spec.name = 'ESX patch'
  file_spec.source_type = update_session_client.File.SourceType.PUSH

  # 5 - Link the ISO file spec to the update session.
  update_file_stub = update_session_stub.File(my_stub_config)
  file_info = update_file_stub.File.add(update_session_id=session_id,
                                        file_spec=file_spec)
  # 6 - Use HTTP library to upload the file to the library item.
  upload_uri = file_info.upload_endpoint.uri
  file_name = "/updates/esx/esx_patch.iso"
  host = urlparse.urlsplit(upload_uri)
  connection = httplib.HTTPConnection(host.netloc)
  with open(file_name, "rb") as f :
```

```
connection.request("PUT", upload_uri, f)

# 7 - Commit the updates.
library_item_service.UpdateSession.complete(session_id)

finally :
# 8 - Delete the session.
library_item_service.UpdateSession.delete(session_id)
```

Upload a File from a URL to a Library Item

You can upload different types of files from a local system to a library item that you want to use in the VMware Cloud Foundationenvironment.

- Create an empty library item. See Create an Empty Library Item.
- Verify that you have access to the `UpdateSession` and `File` services.

1. Create an `UpdateSessionModel` instance to track the changes that you make to the library item.
2. Create an update session by using the `UpdateSession` service.
3. Create a file specification to describe the upload method and other properties of the file to be uploaded.
4. Specify the location of the file to be uploaded by creating a `TransferEndpoint` instance.
5. Add the file source endpoint to the file specification.
6. Create a request for changing the item by using the configured file specification.
7. Complete the update session to apply the changes to the library item.

The files are uploaded to the library item.

Java

This example shows how to upload a file from a URL to a library item. The example is based on the code in the `ItemUploadHelper.java` sample file.

This example uses the steps that are described in the Upload a File from a URL to a Library Item procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a new library item. See Create an Empty Library Item.

...

// Access the com.vmware.content.library.item.updatesession.File
// and the UpdateSession services by using the vSphere Automation Endpoint.
   File uploadFileService =
 this.vapiAuthHelper.getStubFactory().createStub(File.class, sessionStubConfig);
   UpdateSession uploadService =
 this.vapiAuthHelper.getStubFactory().createStub(UpdateSession.class,
 sessionStubConfig);

// Create an UpdateSessionModel instance to track the changes you make to the item.
        UpdateSessionModel updateSessionModel = new UpdateSessionModel();
        updateSessionModel.setLibraryItemId(newItem);

// Create a new update session.
        String clientToken = UUID.randomUUID().toString();
        String sessionId = uploadService.create(clientToken, updateSessionModel);
```

```
// Create a new AddSpec instance to describe the properties of the file to be
 uploaded.
         FileTypes.AddSpec fileSpec = new AddSpec();
         fileSpec.setName("ESX patch");
         fileSpec.setSourceType(SourceType.PULL);

// Specify the location from which the file is uploaded to the library item.
         TransferEndpoint endpoint = new TransferEndpoint();
         endpoint.setUri(URI.create("http://www.acme.com/patches_ESX65/
ESX_patch.iso"));
         fileSpec.setSourceEndpoint(endpoint);
         uploadFileService.add(sessionId, fileSpec);

// Mark the session as completed.
         uploadService.complete(sessionId);
```

Python

This example shows how to upload a file from a URL to a library item. The example is based on the code in the `cls_api_helper.py` sample file.

This example uses the steps that are described in the Upload a File from a URL to a Library Item procedure.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create a new library item to hold the uploaded file.
item_model = library_client.ItemModel()
item_model.name = 'ESX patches'
item_model.description = 'ESX security patches'
item_model.type = 'iso'
item_model.library_id = my_library_id
idem_token = str(uuid.uuid4())
item_stub = library_client.Item(my_stub_config)
item_id = item_stub.create(create_spec=item_model, client_token=idem_token)

# 2 - Create an UpdateSessionModel instance to track the changes you make to the item.
update_session_model = item_client.UpdateSessionModel()
update_session_model.library_item_id = item_id

# 3 - Create an update session from the model.
idem_token = str(uuid.uuid4())
update_session_stub = update_session_client.UpdateSession(my_stub_config)
session_id = update_session_stub.create(create_spec=update_session_model,
 client_token=idem_token)

try :
  # 4 - Create a new AddSpec instance to describe the properties of the file to be
 uploaded.
  file_spec = update_session_client.AddSpec()
  file_spec.name = 'ESX patch'
  file_spec.source_type = update_session_client.File.SourceType.PULL

  # 5 - Specify the location from which the file is to be uploadod to the library
 item.
  endpoint = item_client.TransferEndpoint()
  endpoint.uri = 'http://www.example.com/patches_ESX65/ESX_patch.iso'
  file_spec.source_endpoint = endpoint
```

```
# 6 - Link the file specification to the update session.
update_file_stub = update_session_client.File(my_stub_config)
update_file_stub.File.add(update_session_id=session_id, file_spec=file_spec)

# 7 - Mark session as completed, to initiate the asynchronous transfer.
update_session_stub.complete(session_id)
```

Download Files to a Local System from a Library Item

You might want to download files to a local system from a library item and then make changes to the files before you use them.

1. Create a download session model to specify the item, which contains the file that you want to download.
2. Access the `File` service and retrieve the file that you want to export to your system within the new download session.
3. Prepare the files that you want to download and wait until the files are in the prepared state.
4. Retrieve the download endpoint URI of the files.
5. Download the files with an HTTP GET request.
6. Delete the download session after all files are downloaded.

The files are downloaded from the library item to the local system.

Java

This example is based on the code in the `ItemDownloadHelper.java` sample file.

This example uses the steps that are described in the Download Files to a Local System from a Library Item procedure.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Access the DownloadSession service.
     DownloadSession downloadSessionService =
 vapiAuthHelper.getStubFactory().createStub(DownloadSession.class, sessionStubConfig);

// Create a new download session model.
     DownloadSessionModel downloadSessionModel = new DownloadSessionModel();
     downloadSessionModel.setLibraryItemId(libItem.getId());
     String downloadSessionId =
 downloadSessionService.create(UUID.randomUUID().toString(), downloadSessionModel);

// Access the File service and retrieve the files you want to export.
     File downloadFileService = vapiAuthHelper.getStubFactory().createStub(File.class,
 sessionStubConfig);
     List<FileTypes.Info> downloadFileInfos =
 downloadFileService.list(downloadSessionId);
     for (FileTypes.Info downloadFileInfo : downloadFileInfos) {

// Make sure all files are in the prepared state before you precede with the
 downloading operation.
         downloadFileService.prepare(downloadSessionId, downloadFileInfo.getName(),
 EndpointType.HTTPS);
         long timeOut = 360;
         Long endTime = System.currentTimeMillis() + timeOut * 1000;
         try {
              Thread.sleep(5000);
         } catch (InterruptedException e) {
```

```
                    System.out.println(e);
            }
            FileTypes.PrepareStatus expectedStatus =
 com.vmware.content.library.item.downloadsession.File.PrepareStatus.PREPARED;
            downloadFileInfo = downloadFileService.get(downloadSessionId,
 downloadFileInfo.getName());
            FileTypes.PrepareStatus currentStatus = downloadFileInfo.getStatus();
            if (currentStatus == expectedStatus) {

// When the files are prepared, you can retrieve the download information for each
 file.
               downloadFileInfo = downloadFileService.get(downloadSessionId,
 downloadFileInfo.getName());
               try {
                    URI downloadUri = downloadFileInfo.getDownloadEndpoint().getUri();
                    String downloadUrl = downloadUri.toURL().toString();

// Run an HTTP GET request and pass the download endpoints of the files.
                    HttpClient httpClient = new HttpClient(true);
                    InputStream inputStream = httpClient.downloadFile(downloadUrl);
                    String fileNameDownload = downloadFileInfo.getName();
                    File tmpDir = new java.io.File("tmp");
                    tmpDir.mkdir();
                    String fullPath = tmpDir.getAbsolutePath() +
                          System.getProperty("file.separator") + fileNameDownload;

// Copy the files to the directory on your machine.
                    Files.copy(inputStream,
 Paths.get(fullPath),StandardCopyOption.REPLACE_EXISTING);
               } catch (MalformedURLException e) {
                  System.out.println("Failed to download due to IOException!" + e);
                  throw new RuntimeException("Failed to download due to IOException!",
 e);
               } catch (IOException e) {
                  System.out.println("IO exception during download" + e);
                  throw new RuntimeException("Failed to download due to IOException!",
 e);

// Delete the download session after all files are downloaded.
               } finally {
                    downloadFileService.delete(downloadSessionId);
               }
           } else {
               while (endTime > System.currentTimeMillis()) {
                   downloadFileInfo = downloadFileService.get(downloadSessionId,
                                downloadFileInfo.getName());
                   currentStatus = downloadFileInfo.getStatus();
                   if (currentStatus == expectedStatus) {
                       return;
                   } else if (currentStatus ==
 com.vmware.content.library.item.downloadsession.File.PrepareStatus.ERROR) {
                     System.out.println("DownloadSession Info : " +
 downloadSessionService.get(downloadSessionId));
                    throw new RuntimeException("Error while waiting for download file
 status to be PREPARED...");
                  }
             }
```

```
        }
    }
```

Python

This example uses the code in the `cls_api_helper.py` sample file.

This example uses the steps that are described in the Download Files to a Local System from a Library Item procedure.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create a new download session model.
download_session_model = item_client.DownloadSessionModel()
download_session_model.library_item_id = my_library_item_id
download_session_stub = item_client.DownloadSession(my_stub_config)
idem_token = str(uuid.uuid4())
download_session_id = download_session_stub.create(create_spec=download_session_model,
client_token=idem_token)

# 2 - Access the File service and retrieve the files you want to export.
download_session_file_stub = download_session_client.File(my_stub_config)
file_infos = download_session_file_stub.list(download_session_id)
for file_info in file_infos :
  download_session_file_stub.prepare(download_session_id, file_info.name)

# 3 - Wait until the file is in the prepared state before downloading.
  download_info = download_session_file_stub.get(download_session_id, file_info.name)
  while (DownloadSessionFile.PrepareStatus.PREPARED != download_info.status) :
  time.sleep(30)

# 4 - Download the file with an HTTP GET request.
  response = urllib2.urlopen(download_info.download_endpoint.uri)
  file_path = os.path.join(my_directory, file_info.name)
  with open(file_path, 'wb') as f :
  f.write(response.read())

# 5 - Delete the download session after all files are downloaded.
download_session_stub.delete(download_session_id)
```

Synchronizing a Library Item in a Subscribed Content Library

The items in a subscribed library have features that are distinct from the items in a local library. Synchronizing the content and the metadata of an item in a subscribed library depends on the synchronization mechanism of the subscribed library.

Table 1886: Options for Synchronizing a Library Item

Synchronization Type of the Subscribed Library	Description
Synchronized on demand	If the subscribed library is synchronized on demand, you can use the `sync` method on the `SubscribedItem` service and pass as arguments the library item ID and `true`. When you perform the task, both the item metadata and the content are synchronized. To synchronize only the metadata of the item, pass the library ID and `false` as arguments to the method.
Not synchronized on demand	If the subscribed library is not synchronized on demand, you can use the `sync` method on the `SubscribedItem` service and pass as argument the item ID. In this case, the content of the item is always synchronized and the Boolean value is ignored when the call is run.

Synchronization Type of the Subscribed Library	Description
Synchronized automatically	If the subscribed library is synchronized automatically, you can also use the `sync` method to force the synchronization of an item. Method execution depends on whether the subscribed library is synchronized on demand.

Removing the Content of a Library Item

You can remove the content from a library item to free space on your storage.

If you create a subscribed library with the option to synchronize library content on demand, only the metadata for the library items is stored. When you want to use the items in the library, you must force synchronization on the items to download their content. When you no longer need the files in an item, you can remove the cached content of the library item and free storage space. To achieve this task use the `evict` function of the `SubscribedItem` object.

Deleting a Library Item

You can remove a library item from a local library when you no longer need it.

To remove a library item from a library, you can call the `delete` method on the `Item` object and pass the library item ID as an argument. The item content is asynchronously removed from the storage.

You cannot remove items from a subscribed library. If you remove an item from a local library, the item is removed from the subscribed library when you perform a synchronization task on the subscribed library item.

Content Library Support for OVF and OVA Packages

You can use the objects and methods provided by the Content Library API to manage OVF and OVA packages.

Open Virtualization Format (OVF) is an industry standard that describes metadata about a virtual machine image in an XML format. An OVF package includes an XML descriptor file and optionally disk images, resource files (such as ISO files), manifest files, and certificate files.

An OVA package is a single file that contains all OVF package files in an archived form. After you upload an OVA package to a content library, the OVA file is converted to the standard OVF package.

When you try to upload signed content to a content library, you might receive preview warnings. Signed content can be either OVF or OVA packages that contain manifest and certificate files. If you do not respond to the preview warnings, the upload fails. To complete an upload operation successfully, you must ignore any preview warnings by using the `WarningBehavior` class`LibraryItemUpdatesessionWarningBehavior` data structure.

With the VMware Cloud Foundation API, you can use the OVF package in a content library to deploy virtual machines and vApps on hosts, resource pools, and clusters. You can also use the API to create OVF packages in content libraries from vApps and virtual machines on hosts, resource pools, and clusters.

When you create library items to store OVF packages, you must set the item type to `ovf`. To comply with the specific standards of the OVF packages, the VMware Cloud Foundation API provides the `LibraryItem` class`Item` service.

Working with OVF and OVA Packages in a Content Library

You can upload an OVF or OVA package to a library item by using the `UpdateSession`interfaceservice. You can also download an OVF and OVA packages from a content library to your local file system.

In case you want to upload an OVF package, the location of the content determines whether you can pull the content from a URL or push the content directly to a content library. For information about uploading content to library items, see Upload a File from a Local System to a Library ItemUploading a File from a Local System to a Library Item and Upload a File from a URL to a Library ItemUpload a File from a URL to a Library Item.

To download the files that are included in an OVF or OVA package to your local file system, use the `DownloadSession`interfaceservice. For more information, see Download Files to a Local System from a Library Item Downloading Files to a Local System from a Library Item.

Upload an OVF or an OVA Package from a URL to a Library Item

You can upload an OVF or an OVA package from a Web server to a library item.

- Create a new local content library or retrieve the desired existing content library.
- Required privileges: **Content library** > **Add library** item and **Content library** > **Update files** on the library.

> **Note:**
> If you try to upload a signed OVF package and it returns preview warnings, you must ignore the preview warnings to complete the upload.

1. Create an empty library item.
2. Create an update session object.
3. Create an `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to describe the properties and the upload location of the OVF descriptor file or of the OVA package file.
4. Link the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to the update session.
   All files that are included in the OVF package are automatically uploaded.
5. Complete the asynchronous transfer.

Python Example of Uploading an OVF Package from a URL to a Library Item
This example is based on the `ovf_import_export.py` sample file.

This example uses the steps that are described in the Upload an OVF or an OVA Package from a URL to a Library Item procedure.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create a empty library item to describe the virtual machine.
item_model = library_client.ItemModel()
item_model.name = "ubuntu-vm"
item_model.description = "ubuntu 7.0"
item_model.library_id = my_library_id
item_model.type = "ovf"
client_token = str(uuid.uuid4())
item_stub = library_client.Item(my_stub_config)
item_id = item_stub.create(create_spec=item_model,
client_token=client_token)

# 2 - Create an update session.
update_session_model = item_client.UpdateSessionModel()
update_session_model.library_item_id = item_id
client_token = str(uuid.uuid4())
update_session_stub = update_session_client.UpdateSession(my_stub_config)
session_id = update_session_stub.create(create_spec=update_session_model,
client_token=client_token)

# 3 - Create a file specification for the OVF envelope file.
file_spec = update_session_client.AddSpec()
file_spec.name = "ubuntu.ovf"
file_spec.source_type = File.SourceType.PULL
endpoint = item_client.TransferEndpoint()
endpoint.uri = "http://www.example.com/images/ubuntu.ovf"
file_spec.source_endpoint = endpoint

# 4 - Link the file specification to the update session.
update_file_stub = update_session_client.File(my_stub_config)
```

```
update_file_stub.File.add(update_session_id=session_id,
file_spec=file_spec)

# 5 - Initiate the asynchronous transfer.
update_session_stub.complete(session_id)
```

Upload an OVF or OVA Package from a Local File System to a Library Item

You can upload an OVF or OVA package from a local file system. This procedure describes how to use the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure after you have created a library item and initiated an update session.

- Create a new local content library or retrieve the desired existing content library.
- Required privileges: **Content library** > **Add library item** and **Content library** > **Update files** on the library.

> **Note:**
> If you try to upload a signed OVF package and it returns preview warnings, you must ignore the preview warnings to complete the upload.

1. Create a library item.
2. Create an update session.
3. Create an `AddSpec` objecta `LibraryItemUpdatesessionFileAddSpec` data structure to describe the properties and the upload locations of the OVF descriptor file or of the OVA package file.
4. Link the `AddSpec` object`LibraryItemUpdatesessionFileAddSpec` data structure to the update session.
5. Optional: Create an `AddSpec` objecta `LibraryItemUpdatesessionFileAddSpec` data structure for each VMDK file included in the OVF package.
6. Add all `AddSpec` objects`LibraryItemUpdatesessionFileAddSpec` data structures to the update session.

   If you upload an OVF package and it has a VMDK file included, you must repeat steps 5 and 6. If you are uploading a signed OVF package, steps 5 and 6 must also be repeated for the manifest and certificate files included in the OVF package.
7. Initiate the upload operation.
8. Complete the update session.
9. Delete the session.

Python Example of Uploading an OVA Package to a Library Item

This example is based on the `signed_ova_import.py` sample file.

This example uses the steps that are described in the Upload an OVF or OVA Package from a Local File System to a Library Item procedure.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Specify the OVA filename and location.
SIGNED_OVA_FILENAME = 'nostalgia-signed.ova'
SIGNED_OVA_RELATIVE_DIR = '../resources/signedOvaWithCertWarning'

# 2 - Create a new library item in the content library for uploading the files.
self.lib_item_id = self.helper.create_library_item(library_id=self.local_lib_id,
                                                   item_name=self.lib_item_name,
                                                   item_description='Sample template from ova file',
                                                   item_type='ovf')

# 3 - Set a pointer to the OVA file you want to upload.
```

```
ova_file_map = self.helper.get_ova_file_map(self.SIGNED_OVA_RELATIVE_DIR,
                                            local_filename=self.SIGNED_OVA_FILENAME)

# 4 - Create a new update session for uploading the files.
session_id = self.client.upload_service.create(
    create_spec=UpdateSessionModel(library_item_id=self.lib_item_id),
    client_token=generate_random_uuid())
self.helper.upload_files_in_session(ova_file_map, session_id)

# 5 - Wait for terminal preview state and obtain preview warnings.
self.wait_for_terminal_preview_state(session_id, AVAILABLE)
session = self.client.upload_service.get(session_id)
preview_info = session.preview_info

# 6 - Ignore preview warnings on session, if any.
ignore_warning_behaviors = []
for warning_type in preview_warning_types:
    warning_behavior = WarningBehavior(type=warning_type, ignored=True)
    ignore_warning_behaviors.append(warning_behavior)
self.client.upload_service.update(session_id, update_spec=UpdateSessionModel(
    warning_behavior=ignore_warning_behaviors))

# 7 - Complete the update session.
self.client.upload_service.complete(session_id)

# 8 - Delete the session.
self.client.upload_service.delete(session_id)
```

Creating Virtual Machines and vApps from Templates in a Content Library

You can create VM and OVF templates from virtual machines and vApps in your inventory. You can then deploy virtual machines and vApps from the templates that are stored in a content library.

Create a VM Template in a Content Library from a Virtual Machine

By using the VMware Cloud FoundationREST API, you can create a VM template in a content library from an existing virtual machine in your vCenter inventory.

- Verify that you have administrative privileges on your vCenter instance.
- Verify that you created a VMware Cloud Foundation session to your vCenter.
- Verify that you created a local library by using the vSphere Client or the VMware Cloud Foundation APIs.

When you call the `create` function of the `com.vmware.vcenter.vm_template.LibraryItems` service, a VM template is created as a library item in your local content library. If the operation is successful, the `LibraryItems` service returns the ID of the newly created library item.

To create a library item that contains a VM template, you can use the `ctreate` function of the `LibraryItems` interface`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request. You can review the information about a VM template by using the `get` function of the `LibraryItems` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>` HTTP request. For information about how to create a VM template by using the vSphere Client, see the *vSphere Virtual Machine Administration* documentation.

For information about the available and mandatory parameters, see the *API Reference* documentation.

1. Get the ID of your ESX host on which you want to store the VM template.

   You can use the `list` function of the `com.vmware.vcenter.Host` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request.

2. Get the ID of the datastore on which you want to store the VM template files.

   You can use the `list` function of the `com.vmware.vcenter_client.Datastore` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore` HTTP request.

3. Get the ID of the virtual machine that you want to save as a VM template.

   You can use the `list` method of the `com.vmware.vcenter_client.VM` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm` HTTP request.

4. Get the ID of your local library.

   You can get the list of the local libraries in your vCenter and review the information about each library by using the `list` and `get(library_id)` functions of the `com.vmware.content_client.LocalLibrary` class`GET https://<vcenter_ip_address_or_fqdn>/api/com/vmware/content/local-library` and `GET https://<vcenter_ip_address_or_fqdn>/api/com/vmware/content/library/id:<library_id>` HTTP requests.

5. Create a library item specification for the VM template.

   You can use the `com.vmware.vcenter.vm_template.LibraryItems.CreateSpec` classbody of the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request.

   a) Specify the local library, source virtual machine, and the name of the library item by using the `library`, `source_vm`, and `name` parameters. You must use the IDs of the local library and source virtual machine.
   b) Specify the placement information for your VM template.

      You can use the `LibraryItems.CreatePlacementSpec` class`placement` parameters in the body of the HTTP request. To specify the host, resource pool, cluster, and folder, you must use their IDs.
   c) Specify the datastore on which you want to store the log, configuration, and disk files of your VM template.

      To specify the storage backing for the VM template, you can use the `com.vmware.vcenter.vm_template.LibraryItems.CreateSpecVmHomeStorage` and `com.vmware.vcenter.vm_template.LibraryItems.CreateSpecDiskStorage` classes`vm_home_storage` and `disk_storage` parameters in the body of the HTTP request. You must use the ID of the datastore.
   d) Include the placement and storage specifications in the library item specification.

6. Create a library item for storing the VM template.

   You can use the `create(spec)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items` HTTP request.

If the operation is successful, the `LibraryItems` service returns the ID of the library item that contains the VM template. For information about the available responses, see the *API Reference* documentation.

- Review the information stored in the library item by using the `get(VM_template_item_ID)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<library_item_id>` HTTP request. If you did not save the ID of the library item holding the VM template, you can check the UUID by using the vSphere Client. The URN ends with the ID of the library item and has the following format: `urn:vapi:com.vmware.content.library.Item:<VMTemplateItemID>`.

Python Example of Creating a VM Template in a Content Library from a Virtual Machine

This example shows how you can create a VM template from a virtual machine and add the template to a content library by using the VMware Cloud Foundation API. The example is based on the `create_vm_template.py` sample.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
class CreateVmTemplate(SampleBase):
 ...
    def _setup(self):
```

```
        # Required arguments
        self.vm_name = self.args.vmname
        self.datacenter_name = self.args.datacentername
        self.resource_pool_name = self.args.resourcepoolname
        self.datastore_name = self.args.datastorename

        # Optional arguments
        self.item_name = (self.args.itemname if self.args.itemname
                          else rand('vmtx-item-'))

        self.servicemanager = self.get_service_manager()
        self.client = ClsApiClient(self.servicemanager)
        self.helper = ClsApiHelper(self.client, self.skip_verification)

        session = get_unverified_session() if self.skip_verification else None
        self.vsphere_client = create_vsphere_client(server=self.server,
                                                    username=self.username,
                                                    password=self.password,
                                                    session=session)

    def _execute(self):
        # Get the identifiers for the virtual machine and resource pool
        vm_id = get_vm(self.vsphere_client, self.vm_name)
        assert vm_id
        resource_pool_id = get_resource_pool(self.vsphere_client,
                                             self.datacenter_name,
                                             self.resource_pool_name)
        assert resource_pool_id

        # Create a local content library
        storage_backings = self.helper.create_storage_backings(self.servicemanager,
                                                               self.datastore_name)
        self.library_id = self.helper.create_local_library(storage_backings,
                                                           self.library_name)

        # Build the library item create specification
        create_spec = VmtxLibraryItem.CreateSpec()
        create_spec.source_vm = vm_id
        create_spec.library = self.library_id
        create_spec.name = self.item_name
        create_spec.placement = VmtxLibraryItem.CreatePlacementSpec(resource_pool=resource_pool_id)

        # Create a new library item from the source virtual machine
        self.item_id = self.client.vmtx_service.create(create_spec)
...
```

Create an OVF Template in a Content Library from a Virtual Machine or vApp

You can create library items from existing virtual machines or vApp. Use those library items later to deploy virtual machines and vApps on hosts and clusters in your vCenter environment.

1. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.DeployableIdentityLibraryItem.DeployableIdentity` instance to specify the source virtual machine or vApp to be captured in an OVF template.
2. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.CreateTargetLibraryItem.CreateTarget` instance to identify the content library where the OVF template is stored.
3. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.CreateSpecLibraryItem.CreateSpec` instance to specify the properties of the OVF template.
4. Initiate a synchronous create operation by invoking the `create` function of the `com.vmware.vcenter.ovf.LibraryItem` serviceusing the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/ovf/library-item` HTTP request.
5. Verify the outcome of the create operation.

Java Example of Creating an OVF Template in a Content Library from a Virtual Machine

This example shows how to capture a virtual machine in an OVF template and store the file in a new library item in a specified library.

This example uses the steps that are described in the Create an OVF Template in a Content Library from a Virtual Machine or vApp procedure.

Note:
For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Specify the resource to be captured.
LibraryItemTypes.DeployableIdentity deployableIdentity = new LibraryItemTypes.DeployableIdentity();
deployableIdentity.setType("VirtualMachine");
deployableIdentity.setId("vm-32");

// Create a target spec to identify a library to hold the new item.
LibraryItemTypes.CreateTarget createTarget = new LibraryItemTypes.CreateTarget();
createTarget.setLibraryId(myLibraryId);

// Specify OVF properties.
LibraryItemTypes.CreateSpec createSpec = new LibraryItemTypes.CreateSpec();
createSpec.setName("snap-32");
createSpec.setDescription("Snapshot of VM-32");

// Initiate synchronous capture operation.
LibraryItem itemStub = myStubFactory.createStub(LibraryItem.class, myStubConfiguration);
String clientToken = UUID.randomUUID().toString();
LibraryItemTypes.CreateResult result = itemStub.create(clientToken, deployableIdentity, createTarget,
 createSpec);

// Verify capture status.
System.out.printf("Resource Type=%s (ID=%s) status:",
                  deployableIdentity.getType(),
                  deployableIdentity.getId());
  if (result.getSucceeded() == true) {
     System.out.println("Resource captured.");

  }else {
     System.out.println("Capture failed.");
  }

  if (result.getError() != null) {
```

```
    for (OvfError error : result.getError().getErrors()) {
        System.out.printf("Error: %s", error.getMessage().toString());
    }

    for (OvfWarning warning : result.getError().getWarnings()) {
        System.out.printf("Warning: %s", warning.getMessage().toString());
    }

    for (OvfInfo info : result.getError().getInformation()) {
        List<LocalizableMessage> messages = info.getMessage();
    }

    for (LocalizableMessage message : messages) {
      System.out.printf("Message: %s", message.toString());
    }
  }
```

Python Example of Creating an OVF Template in a Content Library from a Virtual Machine

This example shows how to capture a virtual machine in an OVF template and store the files in a new library item in a specified library.

This example uses the steps that are described in the Create an OVF Template in a Content Library from a Virtual Machine or vApp procedure.

Note:
For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Specify the resource to be captured.
deployable_identity = ovf_client.LibraryItem.DeployableIdentity();
deployable_identity.type = "VirtualMachine"
deployable_identity.id = "vm-32"

# Create a target spec to identify a library to hold the new item.
create_target = ovf_client.LibraryItem.CreateTarget()
create_target.library_id = my_library_id

# Specify OVF properties.
create_spec = ovf_client.LibraryItem.CreateSpec()
create_spec.name = "snap-32"
create_spec.description = "Snapshot of VM-32"

# Initiate synchronous capture operation.
lib_item_stub = ovf_client.LibraryItem(my_stub_config)
client_token = str(uuid.uuid4())
result = lib_item_stub.create(source=deployable_identity,
                              target=create_target,
                              create_spec=create_spec,
                              client_token=client_token)

# Verify capture status.
print("Resource Type={} (ID={}) status:".format(deployable_identity.type, deployable_identity.id))

if result.succeeded == True :
  print("Resource captured.")
else :
  print("Capture failed.")
```

```
if result.error is not None :
  for error in result.error.errors :
    print("Error {}".format(error.message))
  if len(result.error.warnings) > 0 :
    print("Warnings:")
    for warning in result.error.warnings :
      print("{}".format(warning.message))
  if len(result.error.information) > 0 :
    print("Messages:")
    for info in result.error.information :
      for message in info.messages :
        print("{}".format(message))
```

Deploy a Virtual Machine from a VM Template in a Content Library

By using the VMware Cloud Foundation APIs, you can deploy a virtual machine from a VM template stored in a content library.

- Verify that you have administrative privileges on your vCenter instance.
- Verify that you created a VMware Cloud Foundation session to your vCenter instance.

To deploy a virtual machine from a VM template in a content library, call the `deploy` function of the `com.vmware.vcenter.vm_template.LibraryItems` interfaceuse the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request. You can specify the power state and customize the guest operation system prior to the virtual machine deployment.

For information about the available and mandatory parameters, see the *API Reference* documentation.

1. Review the information stored in the VM template library item.

   You can use the `get(VM_template_item_ID)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface and pass the ID of your VM template item`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>` HTTP request. If you did not save the ID of your item, you can select the UUID of your VM template item by using the vSphere Client. The URN ends with the ID of the item and has the following format: `urn:vapi:com.vmware.content.library.Item:<VMTemplateItemID>`.

2. Get the ID of the host on which you want to deploy the virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Host` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/host` HTTP request.

3. Get the ID of the resource pool to which you want to add your virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.ResourcePool` interface`https://<vcenter_ip_address_or_fqdn>/api/vcenter/resource-pool` HTTP request.

4. Get the ID of the `VIRTUAL_MACHINE` folder to which you want to add your virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Folder`GET `https://<vcenter_ip_address_or_fqdn>/api/vcenter/folder` HTTP request.

5. Get the ID of the datastore on which you want to store log, configuration, and disk files of the virtual machine.

   You can use the `list` function of the `com.vmware.vcenter.Datastore` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore` HTTP request.

6. Create a deployment specification.

   You can use the `com.vmware.vcenter.vm_template.LibraryItems.DeploySpec` classbody of the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request.

a) Specify a name and description of the virtual machine that you want to deploy.
b) Specify the place in your inventory on which you want to deploy the virtual machine such as an ESX host, resource pool, and VM folder.

   You can use the `com.vmware.vcenter.vm_template.LibraryItems.DeployPlacementSpec` class`placement` parameter in the body of the request. You must use the IDs of your inventory objects.
c) Specify the datastore on which you want to store the log, configuration, and disk files of the virtual machine. You must use the ID of the datastore.

   You can use the `DeploySpecVmHomeStorage` and `DeploySpecDiskStorage` classes`vm_home_storage`, and `disk_storage` parameters in the body of the request.
d) Optional: Specify the guest operating system and hardware customization specifications that you want to apply to the virtual machine during the deployment process. Add this information to the deployment specification.

   You can use the `GuestCustomizationSpec` and `HardwareCustomizationSpec` classes`guest_customization` and `hardware_customization` parameters in the body of the request. You can get a list of the guest operating system customization specifications that are available in your vCenter by using the `list` function of the `com.vmware.vcenter.guest.CustomizationSpecs` interface`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/guest/customization-specs` HTTP request.
e) Include the placement and storage specifications in the deployment specification.

7. Deploy a virtual machine from your VM template.

   You can use the `deploy(template_library_item, spec)` function of the `com.vmware.vcenter.vm_template.LibraryItems` interface by passing the library item ID where the VM template is stores and the deployment specification`POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/vm-template/library-items/<vm_template_item_id>?action=deploy` HTTP request.

If the operation is successful, the ID of the deployed virtual machine is returned. For information about the possible exceptions, see the *API Reference* documentation.

Python Example of Deploying a VM from a VM Template Library Item

This example shows how you can deploy a VM from a VM Template library item by using the API. The example is based on the `deploy_vm_template.py` sample.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
class DeployVmTemplate(SampleBase):
 ...
    def _setup(self):
        # Required arguments
        self.item_name = self.args.itemname
        self.datacenter_name = self.args.datacentername
        self.folder_name = self.args.foldername
        self.resource_pool_name = self.args.resourcepoolname
        self.datastore_name = self.args.datastorename

        # Optional arguments
        self.vm_name = self.args.vmname if self.args.vmname else rand('vm-')

        self.servicemanager = self.get_service_manager()
        self.client = ClsApiClient(self.servicemanager)
        self.helper = ClsApiHelper(self.client, self.skip_verification)

        session = get_unverified_session() if self.skip_verification else None
        self.vsphere_client = create_vsphere_client(server=self.server,
                                                    username=self.username,
                                                    password=self.password,
                                                    session=session)
```

```
    def _execute(self):
        # Get the identifiers of the resources used for deployment
        item_id = self.helper.get_item_id_by_name(self.item_name)
        assert item_id
        folder_id = get_folder(self.vsphere_client,
                               self.datacenter_name,
                               self.folder_name)
        assert folder_id
        resource_pool_id = get_resource_pool(self.vsphere_client,
                                             self.datacenter_name,
                                             self.resource_pool_name)
        assert resource_pool_id
        datastore_id = get_datastore_id(self.servicemanager,
                                        self.datastore_name)
        assert datastore_id

        # Build the deployment specification
        placement_spec = VmtxLibraryItem.DeployPlacementSpec(
            folder=folder_id,
            resource_pool=resource_pool_id)
        vm_home_storage_spec = VmtxLibraryItem.DeploySpecVmHomeStorage(
            datastore=datastore_id)
        disk_storage_spec = VmtxLibraryItem.DeploySpecDiskStorage(
            datastore=datastore_id)
        deploy_spec = VmtxLibraryItem.DeploySpec(
            name=self.vm_name,
            placement=placement_spec,
            vm_home_storage=vm_home_storage_spec,
            disk_storage=disk_storage_spec)

        # Deploy a virtual machine from the VM template item
        self.vm_id = self.client.vmtx_service.deploy(item_id, deploy_spec)
        self.vm = get_obj_by_moId(self.servicemanager.content,
                                  [vim.VirtualMachine], self.vm_id)
...
```

Deploy a Virtual Machine or vApp from an OVF Template in a Content Library

You can use the `com.vmware.vcenter.ovf.LibraryItemLibraryItem` service to deploy a virtual machine or vApp on a host, cluster, or resource pool from a library item.

1. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.DeploymentTargetLibraryItem.DeploymentTarget` instance to specify the deployment location of the virtual machine or vApp.
2. Create a `com.vmware.vcenter.ovf.LibraryItemTypes.ResourcePoolDeploymentSpecLibraryItem.ResourcePoolDepl` instance to define all necessary parameters for the deployment operation.
   For example, you can assign a name for the deployed virtual machine or vApp, and accept the End User License Agreements (EULAs) to complete the deployment successfully.

3. Optional: Retrieve information from the descriptor file of the OVF template and use the information during the OVF template deployment.
4. Call the `deploy` method on the `LibraryItem` service Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/ovf/library-item/<ovf_library_item_id>?action=deploy` HTTP request.
5. Verify the outcome of the deployment operation.

Java Example of Deploying a Virtual Machine from a Library Item in a Resource Pool

This example shows how to deploy a virtual machine from a local library item in a resource pool. You can also see how to verify the results of the deployment operation.

This example uses the steps that are described in the Deploy a Virtual Machine or vApp from an OVF Template in a Content Library procedure.

Note:
For related code samples, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// Create a virtual machine deployment specification to accept any network resource.
ResourcePoolDeploymentSpec deploymentSpec = new ResourcePoolDeploymentSpec();
String vmName = "MyVirtualMachine";
deploymentSpec.setName(vmName);
deploymentSpec.setAcceptAllEULA(true);


// Create a deployment target specification to accept any resource pool.
String clusterName = "myCluster";
ManagedObjectReference clusterMoRef = VimUtil.getCluster(this.vimAuthHelper.getVimPort(),
 this.vimAuthHelper.getServiceContent(), clusterName);
DeploymentTarget deploymentTarget = new DeploymentTarget();
deploymentTarget.setResourcePoolId(clusterMoRef.getValue());


// Retrieve the library items OVF information and use it for populating the
// deployment spec instance.
LibraryItem libItemStub = stubFactory.createStub(LibraryItem.class, myStubConfiguration);
OvfSummary ovfSummary = libItemStub.filter(libItemId, deploymentTarget);
deploymentSpec.setAnnotation(ovfSummary.getAnnotation());
String clientToken = UUID.randomUUID().toString();
DeploymentResult result = libItemStub.deploy(clientToken,libItemId,
                                                        deploymentTarget,
                                                        deploymentSpec);

// Verify the status of the resource deployment.
System.out.printf("Resource Type=%s (ID=%s) status: ",
                  result.getResourceId().getType(),
                  result.getResourceId().getId());

  if (result.getSucceeded() == true) {
      System.out.println("Resource instantiated.");

  } else {
      System.out.println("Instantiation failed.");
  }
```

Python Example of Deploying a Virtual Machine from a Library Item on a Resource Pool

This example is based on the `deploy_ovf_template.py` sample file.

This example uses the steps that are described in the Deploy a Virtual Machine or vApp from an OVF Template in a Content Library procedure.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# Create a VM deployment specification to accept any network resource.
deployment_spec = ovf_client.LibraryItem.ResourcePoolDeploymentSpec()
deployment_spec.accept_all_eula = True

# Create deployment target spec to accept any resource pool.
target_spec = ovf_client.LibraryItem.DeploymentTarget()

# Initiate synchronous deployment operation.
item_stub = ovf_client.LibraryItem(my_stub_config)
result = item_stub.deploy(my_library_item_id,
                          target_spec,
                          deployment_spec,
                          client_token=str(uuid.uuid4()))

# Verify deployment status.
print("Resource Type={} (ID={}) status:".format(result.resource_id.type, result.resource_id.id))
if result.succeeded == True :
  print("Resource instantiated.”)
else :
  print("Instantiation failed.")
if result.error is not None :
  for error in result.error.errors :
    print("Error {}".format(error.message))
  if len(result.error.warnings) > 0 :
    print("Warnings:")
    for warning in result.error.warnings :
      print("{}".format(warning.message))
  if len(result.error.information) > 0 :
    print("Messages:")
    for info in result.error.information :
      for message in info.messages :
        print("{}".format(message))
```

vSphere Tag Service

The vSphere Tag Service supports the definition of tags that you can associate with vSphere objects or resources. Tags simplify discovery, filtering, and sorting of inventory objects across your vCenter environment.

The vCenter Automation APIs provide programmatic access to creating and managing vSphere tags in your vCenter inventory.

Broadcom provides programmatic access to tag management through the CIS tagging APIs. VMware Cloud Foundation 9.0 extends this surface with three additional REST endpoints under `/vcenter/tagging/` that support filtered, paginated listing and are the foundation for unified vCenter tag management.

For example, to tag your virtual machines by guest operating system type, you can create a category that is called `operating system`, restrict it to virtual machines with `SINGLE` cardinality, then create tags such as `Windows`, `Linux`, and `Mac OS`. You attach the appropriate tag to each VM to enable filtering and policy-driven workflows across your inventory.

vCenter Automation APIs introduce two parallel namespaces: `/cis/tagging/` and `/vcenter/tagging/`. Use the CIS API for all create, update, and delete operations. Use the `/vcenter/tagging/` REST endpoints when you need filtered or paginated listing.

Creating vSphere Tags

You can use the vSphere Automation API to create tags and attach them as metadata to vSphere inventory. Tags are organized into categories, each tag must belong to exactly one category.

Tags and categories can span multiple vCenter instances.

- With Enhanced Linked Mode, tags and tag categories replicate across all linked vCenter instances.
- With Hybrid Linked Mode, tags and tag categories are maintained across your linked domain. That means the on-premises SDDC and the VMware Cloud on AWS SDDC share tags and tag attributes. For more information about Hybrid Linked Mode, see "Hybrid Linked Mode" in the *VMware Cloud on AWS Product Documentation*.

The workflow for creating tags is:

1. Create a tag category that defines the cardinality and the resource types the tags apply to.
2. Create one or more tags within that category.

Creating a Tag Category

A tag category defines the context in which a set of tags operates. Create a category before you create tags. You can restrict a category to specific resource types or can leave it unrestricted so that its tags apply to any object in the inventory.

A tag category has the following properties:

- `name` - Display name for the category. Must be unique within the vCenter instances.
- `description` - Optional free-text description.
- `cardinality` -
  - `SINGLE` — At most one tag from this category per object.
  - `MULTIPLE` — Multiple tags from this category can be attached to one object.
- `associable_types` - The set of resource types whose objects can receive tags from this category. An empty set means that the category applies to all resource types.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

The category `create()` function returns an identifier that you use when you create a tag for that category. The empty set for the associable types indicates that any object type can be associated with a tag in this category.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
import java.util.Set;
import java.util.HashSet;
import com.vmware.cis.tagging.Category;
import com.vmware.cis.tagging.CategoryTypes;
import com.vmware.cis.tagging.CategoryModel;

// Initialize the client factory — replace null with a trust store in production
VcenterClientFactory factory = new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));

try (VcenterClient client = factory.createClient(username, password, null)) {
    Category categoryService = client.createStub(Category.class);

    CategoryTypes.CreateSpec createSpec = new CategoryTypes.CreateSpec();
    createSpec.setName("favorites");
```

```
    createSpec.setDescription("My favorite virtual machines.");
    createSpec.setCardinality(CategoryModel.Cardinality.MULTIPLE);

    // Empty set — category applies to all resource types.
    // To restrict to virtual machines only:
 createSpec.setAssociableTypes(Set.of("VirtualMachine"))
    Set<String> associableTypes = new HashSet<>();
    createSpec.setAssociableTypes(associableTypes);

    // Returns the unique identifier used in all subsequent tag and association calls
    String categoryId = categoryService.create(createSpec);

    System.out.println("Successfully created category with ID: " + categoryId);
}
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

The category `create()` function returns an identifier that you use when you create a tag for that category. The empty set for the associable types indicates that any object type can be associated with a tag in this category.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
import requests
import urllib3
from com.vmware.cis.tagging_client import CategoryModel
from vmware.vapi.vsphere.client import create_vsphere_client

# Disable SSL verification warnings for clean console outputs
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# 1. Setup the secure HTTP session context
session = requests.session()
session.verify = False  # Disable SSL verification ONLY in non-production environments

# 2. Initialize the high-level vSphere Automation Client gateway
vsphere_client = create_vsphere_client(
    server=VC_HOST,
    username=VC_USER,
    password=VC_PASSWORD,
    session=session
)

# 3. Construct the creation payload specification
tc_create_spec = vsphere_client.tagging.Category.CreateSpec(
    name="favorites",
    description="My favorite virtual machines.",
    cardinality=CategoryModel.Cardinality.MULTIPLE,
    associable_types=set()  # Empty set allows the category to apply to all resource
 types universally
)

# 4. Trigger creation via the client stub and capture the unique identifier
category_id = vsphere_client.tagging.Category.create(tc_create_spec)

print(f"Successfully created Category ID: {category_id}")
```

REST

This example demonstrates how to create a tag category.

```
CATEGORY_BODY='{
```

```
  "create_spec": {
    "name": "favorites",
    "description": "My favorite virtual machines.",
    "cardinality": "MULTIPLE",
    "associable_types": ["VirtualMachine"]
  }
}'

CATEGORY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/category" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${CATEGORY_BODY}" | jq -r '.')

echo "Created Category ID: $CATEGORY_ID"
```

Note:

To restrict the category to a specific resource type — for example, virtual machines only — set `associable_types` to `["VirtualMachine"]`. Attempting to attach a tag from this category to a resource of a different type returns an error.

Create one or more tags within this category. See Creating a Tag.

Creating a Tag

After you create a tag category, create one or more tags within that category and assign them to inventory objects. Each tag belongs to exactly one category and must have a name that is unique within that category.

A tag has the following properties:

- `name` - Display name for the tag. Must be unique within the category.
- `description` - Optional free-text description.
- `category_id` - The identifier of the parent category.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

This example creates a tag specification and then uses it to create the tag. The tag specification references the category identifier that was returned from the category create operation. Use the returned tag identifier for subsequent operations on the tag.

This example is based on the information that is provided in Creating a Tag.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
Tag taggingService = client.createStub(Tag.class);
TagTypes.CreateSpec spec = new TagTypes.CreateSpec();
spec.setName("red");
spec.setDescription("My favorite color");
spec.setCategoryId(categoryId); // identifier returned from the category create call
String tagId = taggingService.create(spec);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

This example creates a tag specification and then uses it to create the tag. The tag specification references the category identifier that was returned from the category create operation. Use the returned tag identifier for subsequent operations on the tag.

This example is based on the information that is provided in Creating a Tag.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
tag_create_spec = vsphere_client.tagging.Tag.CreateSpec(
```

```
    name="red",
    description="My favorite color",
    category_id=category_id,  # identifier returned from the category create call
)
tag_id = vsphere_client.tagging.Tag.create(tag_create_spec)
```

REST

This example demonstrates how to create a tag.

```
# Option A: Single-quote wrap (Recommended for clean JSON strings)
TAG_BODY='{
  "create_spec": {
    "name": "red",
    "description": "My favorite color",
    "category_id": "'"${CATEGORY_ID}"'"
  }
}'

TAG_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/cis/tagging/tag" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${TAG_BODY}" | jq -r '.')

echo "Created Tag ID: $TAG_ID"
```

Attach the tag to a vSphere inventory object. See Creating Tag Associations.

Creating Tag Associations

After you create a tag category and a tag within the category, you can associate the tag with a vSphere managed object or resource.

An association is a link between a tag and an inventory object. It contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object. Tag associations are local to a vCenter instance — a listing request returns only the associations that are stored by that vCenter.

When you associate a tag with an object, the object's type must match one of the associable_types defined for the tag's category. If the category's associable_types is empty, the tag can be attached to any resource type.

An association is a simple link that contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object.

Tag associations are local to a vCenter instance. When you request a list of tag associations from a vCenter system, it enumerates only the associations that it has stored.

When you associate a tag with an object, the object's type must match one of the `associable_types` defined for the tag's category. If the category's `associable_types` is empty, you can attach the tag to any resource type.

Assign a Tag to a Content Library

After you create a tag, you can assign the tag to aVMware Cloud Foundation resource.

1. Construct a dynamic object identifier for the library.
   The dynamic identifier includes the type and ID of the object.
2. Attach the tag to the content library.

The tag is assigned to the content library.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

This example uses the steps that are described in the Assign a Tag to a Content Library procedure.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// 1 - Create a dynamic type ID for the content library.
DynamicID libraryDynamicId = new DynamicID(Library.RESOURCE_TYPE,
                                           myLibrary.getId());

// 2- Attach the tag to the ClusterComputeResource managed object.
TagAssociation tagAssociationStub = myStubFactory.createStub(TagAssociation.class,
                                                             myStubConfig);
tagAssociationStub.attach(myLibrary.getId(),
                          libraryDynamicId);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

This example uses the steps that are described in the Assign a Tag to a Content Library procedure.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Create a dynamic type ID for the content library.
library_dynamic_id = DynamicID(type=Library.RESOURCE_TYPE,
                               id=my_library.id)

# 2- Attach the tag to the ClusterComputeResource managed object.
tag_association_stub = tagging_client.TagAssociationStub(my_stub_config)
tag_association_stub.attach(tag_id,
                            library_dynamic_id)
```

Assign a Tag to a Cluster

After you create a tag, you can assign the tag to a vSphere managed object. Tags make the inventory objects in your virtual environment more sortable and searchable.

Obtain the managed object identifier for the specified cluster.

To get the managed object identifier of the `ClusterComputeResource`, you must accessvCenter by using the vSphere Web Services API. For more information about how to access Web Services, see Create a Web Services Session.

This procedure describes the steps for applying tag a to a cluster object in your inventory.

1. Construct a dynamic object identifier for the cluster.
   The dynamic identifier includes the type and ID of the managed object reference.
2. Attach the tag to the cluster.

The tag is assigned to the cluster.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
...

// 1 - Determine the MOID of the ClusterComputeResource from its name.
ManagedObjectReference clusterMoRef = VimUtil.getCluster(vimPort,
                                                         serviceContent,
                                                         myClusterName);
```

```
// 2 - Create a dynamic type ID for the cluster.
DynamicID clusterDynamicId = DynamicID(clusterMoRef.getType(),
                                       clusterMoRef.getValue());

// 3 - Attach the tag to the ClusterComputeResource managed object.
TagAssociation tagAssociationStub = myStubFactory.createStub(TagAssociation.class,
                                                             myStubConfig);
tagAssociationStub.attach(tagId,
                          clusterDynamicId);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
...

# 1 - Determine the MOID of the ClusterComputeResource from its name.
cluster_object = get_obj(service_content,
                         [vim.ClusterComputeResource],
                         my_cluster_name)
cluster_moid = cluster_obj._GetMoId()

# 2 - Create a dynamic type ID for the cluster.
dynamic_id = DynamicID(type='ClusterComputeResource', id=cluster_moid)

# 3 - Attach the tag to the ClusterComputeResource managed object.
tag_association_stub = tagging_client.TagAssociation(my_stub_config)
tag_association_stub.attach(tag_id=tag_id,
                            object_id=dynamic_id)
```

Updating a Tag

To update a tag, you must create an update specification for the tag.

In the update spec, you set values for the fields to be changed, and omit values for the other fields. The update operation modifies only the fields that carry values in the spec and leaves all other fields unchanged.

A common use case is updating a tag description with a timestamp that records when a resource was last reconfigured.

Java

This example is based on code in the `TaggingWorkflow.java` sample file.

This example adds a timestamp in a tag description to identify when a resource was last reconfigured. The tag description is updated with the timestamp after the resources is reconfigured.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
String newDateTime = DateFormat.getDateInstance().format(new Date());
String newDescription = String.format("Server tag updated at (%s).", newDateTime);
TagTypes.UpdateSpec updateSpec = new TagTypes.UpdateSpec();
updateSpec.setDescription(newDescription); // only description is set — name is
 unchanged
taggingService.update(myTagId, updateSpec);
```

Python

This example is based on code in the `tagging_workflow.py` sample file.

This example adds a timestamp in a tag description to identify when a resource was last reconfigured. The tag description is updated with the timestamp after the resources is reconfigured.

Note:
For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
import time


# 1 - Format the current time.
date_time = time.strftime('%d/%m/%Y %H:%M:%S')
description = 'Server tag updated at ' + date_time


# 2 - Set up a tag update spec.
tag_update_spec = vsphere_client.tagging.Tag.UpdateSpec()
tag_update_spec.description = description # only description is set — name is
 unchanged


# 3 - Apply the update spec to change the tag description.
vsphere_client.tagging.Tag.update(
    tag_id=tag_id,
    tag_update_spec=tag_update_spec
)
```

REST

```
UPDATE_BODY='{
 "update_spec": {
 "description": "Server tag updated at '"$(date '+%d/%m/%Y %H:%M:%S')_"'."
 }
}'

# PATCH updates only the fields present in update_spec
curl -s -k -X PATCH \
  "https://${VC_HOST}/api/cis/tagging/tag/${TAG_ID}" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${UPDATE_BODY}"
```

Note:

`UpdateSpec` performs a partial update. Omitting a field preserves its current value. To clear a field, set it to an empty String. The `category_id` and `cardinality` fields cannot be changed after you create a tag or category.

Using Tags to Create and Manage Compute Policies

A compute policy uses a tag to identify the vSphere objects on which vCenter enforces the policy.

Create a Compute Policy

You can create a compute policy and check the compliance status of the policy for a specific virtual machine.

1. Retrieve the object ID of a virtual machine.
2. Retrieve the object ID of the host you want the virtual machine to run on.

   Note:
   The host should be different from the host the virtual machine is already running on.
3. Tag the virtual machine.
   For example, use the *`tag-1`* tag.

4. Tag the host.
   For example, use the *tag-2* tag.
5. Create a compute policy.
   For example, use the *tag-1* and *tag-2* tags.
6. Check the compliance status of the policy on this virtual machine.

   Note:
   The compliance status can take up to 3 minutes to update.

vCenter creates the compute policy and places the virtual machine on the tagged host.

Java

This example follows the same client setup and capability-discovery pattern as `ListComputePolicy.java` in the vcf-sdk-java GitHub repository.

It creates a VM-host affinity policy for a virtual machine and host that are already tagged.

```
package com.vmware.sdk.samples.vcenter.compute.policies;

import static
 com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.CreateSpec;

/** Demonstrates how to create a VM-Host affinity compute policy. */
public class CreateVmHostAffinityPolicy {
    private static final Logger log =
 LoggerFactory.getLogger(CreateVmHostAffinityPolicy.class);

    /** REQUIRED: vCenter FQDN or IP address. */
    public static String serverAddress = "vcenter1.mycompany.com";
    /** REQUIRED: Username to log in to the vCenter. */
    public static String username = "username";
    /** REQUIRED: Password to log in to the vCenter. */
    public static String password = "password";
    /** OPTIONAL: Absolute path to a file of trusted server certificates. Leave null
 to disable TLS verification. */
    public static String trustStorePath = null;
    /** REQUIRED: Identifier of a tag already attached to the virtual machine. */
    public static String vmTagId = "urn:vmomi:InventoryServiceTag:5b3564f4-
a762-4480-9145-0f87d28aea78:GLOBAL";
    /** REQUIRED: Identifier of a tag already attached to the host. */
    public static String hostTagId = "urn:vmomi:InventoryServiceTag:a8d0d278-
edd1-4a79-86ac-fa9384595062:GLOBAL";

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(CreateVmHostAffinityPolicy.class, args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));
```

```
            try (VcenterClient client = factory.createClient(username, password, null)) {
                Policies policies = client.createStub(Policies.class);

                CreateSpec createSpec = new CreateSpec();
                createSpec.setName("rack_a_policy");
                createSpec.setDescription("rack_a_policy description");
                createSpec.setVmTag(vmTagId);
                createSpec.setHostTag(hostTagId);

                String policyId = policies.create(createSpec);
                log.info("Created compute policy with ID: {}", policyId);
            }
        }
}
```

Python

This example is based on code in the `compute_policy_vcls_workflow.py` sample file. It tags the virtual machine and the host, creates a VM-host affinity policy, powers on the virtual machine, and checks the resulting compliance status.

> **Note:**
> For a complete and up-to-date version of the sample code, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
from com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity_client \
     import CreateSpec
...
# Tag the virtual machine and the host.
attach_tag(self.client, self.vm_id, "VirtualMachine", vm_tag)
attach_tag(self.client, host_id, "HostSystem", host_tag)

# Create a VM-host affinity policy.
create_spec = CreateSpec(vm_tag=vm_tag.id, host_tag=host_tag.id,
                         name=self.policy_name,
                         description=self.policy_desc)
self.policy_id = self.client.vcenter.compute.Policies.create(create_spec)

# Power on the virtual machine.
self.client.vcenter.vm.Power.start(self.vm_id)

# Check the compliance status of the policy on this virtual machine.
status = self.client.vcenter.vm.compute.Policies.get(self.vm_id,
                                                     self.policy_id)
print("The compliance status of policy {} for virtual machine "
      "{} is {}".format(self.policy_id, self.vm_id, status.status))
```

REST

```
POLICY_BODY='{
  "spec": {
    "@class":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec",
    "name": "rack_a_policy",
    "description": "rack_a_policy description",
    "vm_tag": "'"${VM_TAG_ID}"'",
    "host_tag": "'"${HOST_TAG_ID}"'"
  }
}'
POLICY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${POLICY_BODY}" | jq -r '.')
```

```
echo "Created Policy ID: $POLICY_ID"
```

Managing Compute Policy Workflows

You can manage compute policy capabilities, create tags and associations, create, list, and use policies.

The following sequence represents a typical workflow of discovering all compute policy capabilities available on a vCenter instance and creating policies based on those capabilities.

- Manage Compute Policy Capabilities
- Create Tags and Associations
- Create, List, and Use Policies

Manage Compute Policy Capabilities

You can list capabilities, retrieve details about a specific capability, and use the details to create a compute policy.

The `Capabilities` service provides operations to manage compute policy capabilities. The description of the capability provides information about the intent of a policy based on this capability. A capability provides a type to create a policy. A capability also provides a type that describes the information returned when retrieving information about a policy.

1. List all available compute policy capabilities.
2. Retrieve details about the `vm_host_affinity` capability.
3. Use the details to identify the fields of the `CreateSpec` structure for that capability.

Java

```
package com.vmware.sdk.samples.vcenter.compute.policies;

import static
 com.vmware.sdk.samples.utils.ssl.SecurityHelper.loadKeystoreOrCreateEmpty;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.vmware.sdk.samples.utils.SampleCommandLineParser;
import com.vmware.sdk.vsphere.utils.VcenterClient;
import com.vmware.sdk.vsphere.utils.VcenterClientFactory;
import com.vmware.vcenter.compute.policies.Capabilities;
import com.vmware.vcenter.compute.policies.CapabilitiesTypes;

/** Demonstrates how to discover compute policy capabilities. */
public class ListComputePolicyCapabilities {
    private static final Logger log =
 LoggerFactory.getLogger(ListComputePolicyCapabilities.class);

    public static String serverAddress = "vcenter1.mycompany.com";
    public static String username = "username";
    public static String password = "password";
    public static String trustStorePath = null;

    public static void main(String[] args) throws Exception {
        SampleCommandLineParser.load(ListComputePolicyCapabilities.class,
 args);

        VcenterClientFactory factory =
                new VcenterClientFactory(serverAddress,
 loadKeystoreOrCreateEmpty(trustStorePath));
```

```
        try (VcenterClient client = factory.createClient(username, password,
 null)) {
            Capabilities capabilities = client.createStub(Capabilities.class);

            // List all available compute policy capabilities.
            List<CapabilitiesTypes.Summary> summaries = capabilities.list();
            for (CapabilitiesTypes.Summary summary : summaries) {
                log.info("capability={}, name={}", summary.getCapability(),
 summary.getName());
            }

            // Retrieve details about the vm_host_affinity capability.
            CapabilitiesTypes.Info info = capabilities.get(

 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity");
            log.info("createSpecType={}, infoType={}",
 info.getCreateSpecType(), info.getInfoType());
        }
    }
}
```

Python

```
# List all available compute policy capabilities.
summaries = vsphere_client.vcenter.compute.policies.Capabilities.list()
for summary in summaries:
    print(f"capability={summary.capability}, name={summary.name}")

# Retrieve details about the vm_host_affinity capability.
info = vsphere_client.vcenter.compute.policies.Capabilities.get(
    "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity")
print(f"create_spec_type={info.create_spec_type}, info_type={info.info_type}")
```

REST

```
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities" \
  -H "vmware-api-session-id: ${SESSION}"
```

The following example response contains details about the `vm_host_affinity` capability.

```
[
    {
        "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity",
        "name": "VM-Host Affinity",
        "description": "Virtual machines with the specified tag are placed on
 hosts with the specified tag."
    }
]
```

```
curl -s -k \  "https://${VC_HOST}/api/vcenter/compute/policies/capabilities/
com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity" \
  -H "vmware-api-session-id: ${SESSION}"
```

```
{
    "info_type":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.info",
    "name": "VM-Host Affinity",
    "description": "Virtual machines with the specified tag are placed on
 hosts with the specified tag.",
    "create_spec_type":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec"
}
```

You can create tags to identify inventory objects on which a compute policy should be enforced. See Create Tags and Associations.

Create Tags and Associations

You must create a category and specify the associable types and cardinality for the tags that make up the category. After you create a category, you can create tags for the category.

1. Create a tagging category, for example, a category restricted to hosts.
2. Create a tag for the tagging category.
3. List all tags on a vCenter instance.
4. List all virtual machines in the inventory, to identify the virtual machine to place on the tagged host.
5. Attach the tag to a host.
6. List all tags attached to a host.

Java

This example follows the same category and tag creation pattern already shown in Creating a Tag Category and Creating a Tag, restricted to the `HostSystem` associable type.

```
mport java.util.HashSet;
import java.util.List;
import java.util.Set;

import com.vmware.cis.tagging.Category;
import com.vmware.cis.tagging.CategoryModel;
import com.vmware.cis.tagging.CategoryTypes;
import com.vmware.cis.tagging.Tag;
import com.vmware.cis.tagging.TagAssociation;
import com.vmware.cis.tagging.TagTypes;
import com.vmware.vapi.std_client.DynamicID;
import com.vmware.vcenter.Host;
import com.vmware.vcenter.HostTypes;

Category categoryService = client.createStub(Category.class);
Tag taggingService = client.createStub(Tag.class);
TagAssociation tagAssociation = client.createStub(TagAssociation.class);
Host hostService = client.createStub(Host.class);

// Create a tagging category, restricted to hosts.
CategoryTypes.CreateSpec categorySpec = new CategoryTypes.CreateSpec();
categorySpec.setName("cat-4");
categorySpec.setDescription("cat-4-desc");
categorySpec.setCardinality(CategoryModel.Cardinality.SINGLE);
Set<String> associableTypes = new HashSet<>();
associableTypes.add("HostSystem");
categorySpec.setAssociableTypes(associableTypes);
String categoryId = categoryService.create(categorySpec);

// Create a tag for the tagging category.
TagTypes.CreateSpec tagSpec = new TagTypes.CreateSpec();
tagSpec.setName("tag-1");
tagSpec.setDescription("tag-1-desc");
tagSpec.setCategoryId(categoryId);
String tagId = taggingService.create(tagSpec);

// List all virtual machines in the inventory. See ListComputePolicy.java for
 the
// equivalent pattern applied to compute policies.
List<HostTypes.Summary> hosts = hostService.list(new HostTypes.FilterSpec());
```

```
String hostId = hosts.get(0).getHost();

// Attach the tag to a host.
DynamicID hostDynamicId = new DynamicID("HostSystem", hostId);
tagAssociation.attach(tagId, hostDynamicId);

// List all tags attached to the host.
List<String> attachedTags = tagAssociation.listAttachedTags(hostDynamicId);
```

Python

```
# Create a tagging category, restricted to hosts.
category_spec = vsphere_client.tagging.Category.CreateSpec(
    name="cat-4",
    description="cat-4-desc",
    cardinality=CategoryModel.Cardinality.SINGLE,
    associable_types={"HostSystem"}
)
category_id = vsphere_client.tagging.Category.create(category_spec)

# Create a tag for the tagging category.
tag_spec = vsphere_client.tagging.Tag.CreateSpec(
    name="tag-1",
    description="tag-1-desc",
    category_id=category_id
)
tag_id = vsphere_client.tagging.Tag.create(tag_spec)

# List all virtual machines in the inventory, to identify the virtual machine
 to
# place on the tagged host.
vms = vsphere_client.vcenter.VM.list()

# List hosts, and attach the tag to the first one.
hosts = vsphere_client.vcenter.Host.list()
host_dynamic_id = DynamicID(type='HostSystem', id=hosts[0].host)
vsphere_client.tagging.TagAssociation.attach(tag_id=tag_id,
 object_id=host_dynamic_id)

# List all tags attached to the host.
attached_tags = vsphere_client.tagging.TagAssociation.list_attached_tags(
    object_id=host_dynamic_id)
```

REST

The following example `create_spec` specifies the associable types and cardinality for the tags that make up the category.

```
CATEGORY_BODY='{
  "create_spec": {
    "associable_types": ["HostSystem"],
    "cardinality": "SINGLE",
    "description": "cat-4-desc",
    "name": "cat-4"
  }
}'
CATEGORY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/category" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${CATEGORY_BODY}" | jq -r '.')
echo "Created Category ID: $CATEGORY_ID"
TAG_BODY='{
```

```
  "create_spec": {
    "category_id": "'"${CATEGORY_ID}"'",
    "description": "tag-1-desc",
    "name": "tag-1"
  }
}'
TAG_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${TAG_BODY}" | jq -r '.')
echo "Created Tag ID: $TAG_ID"
curl -s -k \
  "https://${VC_HOST}/api/vcenter/vm" \
  -H "vmware-api-session-id: ${SESSION}"
# Resolve the host identifier by name.
HOST_ID=$(curl -s -k \
  "https://${VC_HOST}/api/vcenter/host?names=${HOST_NAME}" \
  -H "vmware-api-session-id: ${SESSION}" | jq -r '.[0].host')

ATTACH_BODY='{
  "object_id": {
    "id": "'"${HOST_ID}"'",
    "type": "HostSystem"
  }
}'
curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag-association/${TAG_ID}?
~action=attach" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
curl -s -k -X POST \
  "https://${VC_HOST}/api/com/vmware/cis/tagging/tag-association?~action=list-
attached-tags" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${ATTACH_BODY}"
```

You can create and manage compute policies. See Create, List, and Use Policies.

Create, List, and Use Policies

You can create various types of policies, list all policies on a vCenter instance, and list or filter the tags that policies use.

1. Create a `vm_host_affinity` policy.
2. List all policies on a vCenter instance.
3. List tags used by one or all policies defined on a vCenter instance.
4. Filter tags used by one or all policies defined on a vCenter instance by policy ID, tag UUID, and tag type.

Java

This example builds on `CreateVmHostAffinityPolicy.java`, shown earlier in Create a Compute Policy, and adds the list and tag-usage operations.

```
import java.util.List;
import java.util.Set;

import com.vmware.vcenter.compute.Policies;
import com.vmware.vcenter.compute.PoliciesTypes;
import com.vmware.vcenter.compute.policies.TagUsage;
```

```
import com.vmware.vcenter.compute.policies.TagUsageTypes;
import
 com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.CreateSpec;

Policies policies = client.createStub(Policies.class);
TagUsage tagUsage = client.createStub(TagUsage.class);

// Create a vm_host_affinity policy.
CreateSpec createSpec = new CreateSpec();
createSpec.setName("rack_a_policy");
createSpec.setDescription("rack_a_policy description");
createSpec.setVmTag(vmTagId);
createSpec.setHostTag(hostTagId);
String policyId = policies.create(createSpec);

// List all policies on this vCenter instance.
List<PoliciesTypes.Summary> policySummaries = policies.list();
for (PoliciesTypes.Summary summary : policySummaries) {
    log.info("policy={}, name={}", summary.getPolicy(), summary.getName());
}

// List tags used by one or all policies.
List<TagUsageTypes.Info> tagUsageInfo = tagUsage.list(null);
for (TagUsageTypes.Info usage : tagUsageInfo) {
    log.info("categoryName={}, tagName={}, policy={}",
            usage.getCategoryName(), usage.getTagName(), usage.getPolicy());
}

// Filter tags used by one or all policies by policy ID, tag UUID, tag type,
// policy name, or capability.
TagUsageTypes.FilterSpec filterSpec = new TagUsageTypes.FilterSpec();
filterSpec.setPolicies(Set.of(policyId));
filterSpec.setTags(Set.of(hostTagId));
filterSpec.setTagTypes(Set.of("com.vmware.cis.tagging.Tag:HostSystem"));
filterSpec.setPolicyNames(Set.of("rack_a_policy"));
filterSpec.setCapabilities(Set.of("com.vmware.vcenter.compute.policies.capabilities.vm_host_aff
List<TagUsageTypes.Info> filteredUsage = tagUsage.list(filterSpec);
```

Python

```
# Create a vm_host_affinity policy.
create_spec =
 vsphere_client.vcenter.compute.policies.capabilities.vm_host_affinity_client.CreateSpec(
    name="rack_a_policy",
    description="rack_a_policy description",
    vm_tag=vm_tag_id,
    host_tag=host_tag_id
)
policy_id = vsphere_client.vcenter.compute.Policies.create(create_spec)

# List all policies on this vCenter instance.
policy_summaries = vsphere_client.vcenter.compute.Policies.list()
for summary in policy_summaries:
    print(f"policy={summary.policy}, name={summary.name}")

# List tags used by one or all policies.
tag_usage_info = vsphere_client.vcenter.compute.policies.TagUsage.list()
for usage in tag_usage_info:
    print(f"category_name={usage.category_name}, tag_name={usage.tag_name},
 policy={usage.policy}")
```

```
# Filter tags used by one or all policies by policy ID, tag UUID, tag type,
# policy name, or capability.
filter_spec = vsphere_client.vcenter.compute.policies.TagUsage.FilterSpec(
    policies={policy_id},
    tags={host_tag_id},
    tag_types={"com.vmware.cis.tagging.Tag:HostSystem"},
    policy_names={"rack_a_policy"},

 capabilities={"com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity"}
)
filtered_usage =
 vsphere_client.vcenter.compute.policies.TagUsage.list(filter_spec)
```

REST

```
POLICY_BODY='{
  "spec": {
    "@class":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity.create_spec",
    "name": "rack_a_policy",
    "description": "rack_a_policy description",
    "vm_tag": "'"${VM_TAG_ID}"'",
    "host_tag": "'"${HOST_TAG_ID}"'"
  }
}'
POLICY_ID=$(curl -s -k -X POST \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}" \
  -H "Content-Type: application/json" \
  -d "${POLICY_BODY}" | jq -r '.')
echo "Created Policy ID: $POLICY_ID"
```

The following example response contains the policy ID.

```
{
    "value": "178aaf62-06e7-4958-bff9-841674633a3f"
}
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies" \
  -H "vmware-api-session-id: ${SESSION}"
{
    "value": [
        {
            "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity",
            "name": "rack_a_policy",
            "description": "rack_a_policy description",
            "policy": "178aaf62-06e7-4958-bff9-841674633a3f"
        }
    ]
}
curl -s -k \
  "https://${VC_HOST}/api/vcenter/compute/policies/tag-usage" \
  -H "vmware-api-session-id: ${SESSION}"
{
    "value": [
        {
            "category_name": "cat-4",
            "tag_type": "com.vmware.cis.tagging.Tag:HostSystem",
```

```
            "tag_name": "tag-1",
            "tag": "urn:vmomi:InventoryServiceTag:a8d0d278-edd1-4a79-86ac-
fa9384595062:GLOBAL",
            "policy": "178aaf62-06e7-4958-bff9-841674633a3f",
            "policy_name": "rack_a_policy",
            "policy_description": "rack_a_policy description",
            "capability":
 "com.vmware.vcenter.compute.policies.capabilities.vm_host_affinity"
        }
    ]
}
```

Note:

The `policy_name`, `policy_description`, and `capability` fields were added in VCF 9.1.0.0.

```
curl -s -k -G \
  "https://${VC_HOST}/api/vcenter/compute/policies/tag-usage" \
  -H "vmware-api-session-id: ${SESSION}" \
  --data-urlencode "policies=${POLICY_ID}" \
  --data-urlencode "tags=${HOST_TAG_ID}" \
  --data-urlencode "tag_types=com.vmware.cis.tagging.Tag:HostSystem"
```

Note:

You can also filter by `policy_names` or by `capabilities`, both added in VCF 9.1.0.0.

vSphere Supervisor Configuration and Management

Starting with vSphere 7.0, you can use the VMware Cloud FoundationREST APIs to enable and configure vSphere Supervisor on a vSphere cluster. Then you can run Kubernetes workloads directly on ESX hosts.

You can use the VMware Cloud Foundation to automate the enabling and disabling of vSphere Supervisor on a vSphere cluster, creating and managing namespaces.

The information in this chapter is intended for vSphere administrators who want to use the VMware Cloud FoundationREST APIs to configure their environment to run Kubernetes workloads in vSphere. To take fully advantage of these VMware Cloud FoundationREST APIs, you must have basic knowledge about the Kubernetes technology and containers.

For more information about how to configure and manage vSphere Supervisor through the vSphere Client, see the *Installing and Configuring vSphere Supervisor* and *vSphere with Tanzu Configuration and Management* documentation.

vSphere Supervisor Terminology

You must understand the basic terminology in this chapter to be able use the vSphere Supervisor automation APIs effectively.

vSphere Supervisor Basic Terms

Term	Description
Supervisor	A vSphere cluster that has the vSphere Supervisor enabled.
VKS cluster	An upstream Kubernetes cluster provisioned and managed by using the vSphere Kubernetes Service. A VKS resides in a vSphere Namespace. You can deploy workloads and services to such clusters in the same way as you do with standard Superviso r.

Term	Description
vSphere Namespace	A namespace that is created within a Supervisor. Each namespace sets the resource boundaries for CPU, memory, storage, and also the number of Kubernetes objects that can run within the namespace. After a namespace is configured, you can run Kubernetes workloads within the namespace.
vSphere Pod	A virtual machine with a small footprint that runs one or more Linux containers. A vSphere Pod is equivalent to a Kubernetes pod. vSphere Pods are compatible with the Open Container Initiative (OCI) and can run OCI compatible containers regardless the operating system.
Spherelet	A spherelet is an implementation of the kubelet functionality ported natively on each host in the Supervisor.
Kubernetes Workload	Workloads are applications that are deployed in one of the following ways: • As containers running inside vSphere Pods. • Workloads provisioned through the VM service. • clusters deployed by using the . • Applications running inside the clusters.
Supervisor control plane	vSphere Supervisor creates a Kubernetes control plane directly on the hypervisor layer. The control plane manages the worker nodes and the vSphere Pods in the Supervisor.
Supervisor worker nodes	ESX hosts that are part of a Supervisor are considered as worker nodes. You run your Kubernetes workloads on the worker nodes.
Container Runtime Executive (CRX)	CRX is an isolated Linux execution environment similar to a VM that works together with ESX.
VM Service	The VM Service functionality allows DevOps engineers to deploy and manage virtual machines in their Kubernetes environment through standard Kubernetes APIs. vSphere administrators are responsible for providing VM Classes and VM Images for the DevOps engineers to choose from, as well as managing resource allocations to self-service provisioned VMs.
Self-Service Namespace	vSphere administrators can activate the Self-Service Namespace service on a Supervisor and create namespace templates for DevOps engineers to create a vSphere Namespace themselves.
vSphere Zones	vSphere Zones provide high availability against clusters-level failures to workloads deployed on vSphere Supervisor. You can configure a three-zone Supervisor mapped to three vSphere clusters or a one-zone Supervisor mapped to a single vSphere cluster. In a single cluster deployment, the high availability is provided by vSphere HA and is only on a host level.

How to Configure and Manage a Supervisor

You use the `Clusters` service to enable and disable a Supervisor, or edit the configuration of an existing Supervisor. The `Clusters` service is provided within the `namespace_management` package.

You can enable a vSphere cluster to manage Kubernetes workload objects, only after you enable vSphere DRS in a fully automated mode and enable HA on the cluster.

Before you enable a vSphere Supervisor on a vSphere cluster, you must prepare your environment to meet the specific networking, storage, and infrastructure requirements. See the *Installing and Configuring vSphere Supervisor* documentation.

For more information about how to configure the storage settings to meet the requirements of vSphere Supervisor, see Creating Storage Policies for vSphere Supervisor.

For more information about how to configure the networking settings for Supervisors that are configured with the VMware NSX-T™ Data Center as the networking stack, see Configuring NSX for vSphere Supervisor.

Starting with vSphere 7.0 Update 1, you can enable a Supervisor with vSphere networking or NSX-T Data Center, to provide connectivity between control planes, services, and workloads. A Supervisor that is configured with vSphere networking uses a vSphere Distributed Switch to provide connectivity to Kubernetes workloads and control planes. The cluster also requires a third-party load balancer that provides connectivity to DevOps users and external services. You can install in your vSphere environment the HAProxy load balancer implementation that VMware provides. See Configuring the vSphere Networking Stack for vSphere Supervisor and Installing and Configuring the HAProxy Load Balancer.

Staring with vSphere 7.0 Update 2, if you are using vSphere networking, you can use the VMware Avi Load Balancer to support VKS clusters provisioned by the . See Using the Avi Load Balancer with vSphere Networking.

Persistent Storage in vSphere Supervisor

Some Kubernetes workloads use persistent storage to store data permanently. vSphere Supervisor integrates with Cloud Native Storage (CNS) to provision persistent storage.

To understand how vSphere Supervisor uses persistent storage, refer to the *vSphere Supervisor Concepts and Planning* and *vSphere Storage* documentations.

Creating Storage Policies for vSphere Supervisor

Before you enable vSphere Supervisor, you must set up the storage to provision the Kubernetes infrastructure. You achieve this task by creating storage policies to be used in the Supervisor and namespaces.

To automate the creation of a tag-based storage policy, use the VMware® vSphere Management SDK. For more information about how to create a tag-based storage policy through the Web Services API, see the *VMware Storage Policy SDK Programming Guide* and *vSphere Web Services SDK Programming Guide* documentations.

Optionally, you can use the VMware Cloud FoundationREST APIs to create and add a tag to the datastore. See the vSphere Tag Service chapter. Currently, you can create a tag-based storage policy only through the Web Services APIs.

Use the VMware Cloud FoundationREST APIs to retrieve the default storage policy of a specific datastore by calling the `get(datastore_ID)` function of the `com.vmware.vcenter.datastore.DefaultPolicy` serviceusing the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/datastore/datastore_id/default-policy` request and submitting the ID of the datastore as a path parameter. You can also retrieve commonly used information about the storage policies available in the vCenter instance by calling the `list()` function of the `Policies` service. by using the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/compute/policies` request.

You can use the storage policies retrieved through the VMware Cloud Foundation APIs to perform the following tasks:

- Assign the storage policies to the Supervisor. The storage policies set within the Supervisor enable specification ensure that the Supervisor control plane, the ephemeral disks of all vSphere Pods, and the container images are placed on the datastores that the policies represent. See Configuring NSX for vSphere Supervisor.
- Assign the storage policies to the vSphere Namespace. The storage policies associated with a namespace determine which datastores the namespace can access and use for persistent volumes for the vSphere Pod and the pods inside a VKS cluster. See Create a vSphere Namespace.

Enabling ReadWriteMany Support

You can enable the ReadWriteMany support in vSphere Supervisor and allow multiple pods and applications to mount simultaneously a single persistent volume.

In vSphere 7.0 Update 3, only VKS clusters support persistent volumes in ReadWriteMany mode. When you enable file volume support for vSphere Supervisor, be aware of the potential security weaknesses:

- The volumes are mounted without encryption. The unencrypted data might be accessed while the data transits the network.
- Access Control List (ACL) is used for the file shares to isolate file share access within a supervisor namespace. It might have risk of IP spoofing.

Follow these guidelines for networking:

- Make sure the vSAN File Services is routable from the Workload network.
- Use common DNS server for vSAN File Services and the vSphere cluster.
- If your vSphere Supervisor has NSX networking, use the SNAT IP of the Supervisor namespace and the SNAT IP of the VKS cluster for ACL configuration.
- If you have vSphere Supervisor with vSphere Distributed Switch (VDS) networking, use the VKS cluster VM IP or the IP of the Supervisor namespace for ACL configuration.

Before you activate the file volume support on a Supervisor, you must set up a vSAN cluster with enabled vSAN File Service. To configure a vSAN cluster with enabled vSAN File Service in the vSphere Client, see the *Configure File Services* topic in the *Administering VMware vSAN* documentation. For more information about how to programmatically achieve this task, see the *vSAN SDKs Programming Guide* documentation.

You activate the ReadWriteMany support on a cluster when you enable vSphere Supervisor on it, or reconfigure an existing Supervisor. See Enable vSphere Supervisor on a Cluster with NSX as the Networking StackEnable vSphere Supervisor on a Cluster with NSX as the Networking Stack, Enable vSphere Supervisor on a Cluster with the vSphere Networking StackEnable vSphere Supervisor on a Cluster with the vSphere Networking Stack, and Reconfiguring a Supervisor. Pass the list of vSAN clusters to be used for provisioning file volumes by using the `setCnsFileConfig(CNSFileConfig cnsFileConfig)` Java methodthe `cns_file_config` property of respective data structure. Currently, you can use only the current vSphere cluster for provisioning file volumes if it is a vSAN cluster with enabled vSAN File Service.

To deactivate the persistent volumes support on a Supervisor, pass an empty list when you set the Cloud Native Storage persistent storage support for the cluster. After that existing ReadWriteMany persistent volumes provisioned in the cluster remain unaffected and usable.

Supervisor Networking

You can enable a Supervisor with vSphere networking or NSX to provision connectivity to Kubernetes control planes, services, and workloads.

A Supervisor that uses the vSphere networking stack is backed by a vSphere Distributed Switch and requires a load balancer to provide connectivity to DevOps users and external services. The Avi Load Balancer and the HAProxy load balancers are supported for vSphere 7.0 Update 2.

A Supervisor that is configured with NSX, uses the software-based networks of the solution and an NSX Edge load balancer to provide connectivity to external services and DevOps users.

Configuring NSX for vSphere Supervisor

vSphere Supervisor requires specific networking configuration to allow you to connect to the Supervisors, vSphere Namespaces, and all objects that run inside the namespaces.

Follow the instructions for installing and configuring the NSX for managing Kubernetes workloads documented in the *Installing and Configuring vSphere Supervisor* guide.

First, you need to create a vSphere Distributed Switch and a distributed port group for each NSX Edge uplink. To automate this step, use the Web Services APIs as described in the *vSphere Web Services SDK Programming Guide*. Then, you can use the NSX REST APIs to add a compute manager, create transport zones, and perform other steps required for configuring the NSX for vSphere Supervisor.

Configuring the vSphere Networking Stack for vSphere Supervisor

To configure a Supervisor with the vSphere networking stack, you must connect all hosts from the cluster to a vSphere Distributed Switch. Depending on your topology, you must create one or more distributed port groups on the switch and configure them as workload networks to the vSphere Namespaces on the cluster.

Workload networks provide connectivity to the nodes of VKS clusters and to the Supervisor control planes. The workload network that provides connectivity to Supervisor control planes is called primary workload network. Each Supervisor must have one primary workload network represented by a distributed port group.

The Supervisor control planes on the cluster use three IP addresses from the IP address range that is assigned to the primary workload network. Each node of a VKS cluster has a separate IP address assigned from the address range of the workload network that is configured with the namespace where the VKS cluster runs.

To create a vSphere Distributed Switch and port groups for configuring the vSphere networking stack of a Supervisor, you can use the vSphere Web Services APIs as described in the *vSphere Web Services SDK Programming Guide* documentation. When you create a distributed virtual switch, vCenter automatically creates one distributed virtual port group. You can use this port group as the primary workload network and use it to handle the traffic for the Supervisor control planes. Then you can create as many distributed port groups for the workload networks as your topology requires. For a topology with one isolated workload network, create one distributed port group that you will use as a network for all namespaces on the Supervisor. For a topology with isolated networks for each vSphere Namespace, create the same number of distributed port groups as the number of namespaces.

To list all workload networks available for a Supervisor and retrieve information about the configuration of a specific workload network, use the `Networks` service from the VMware Cloud FoundationREST APIs. To associate a vSphere Distributed port group to a workload network, set the necessary information through the `setVsphereNetwork(NetworksTypes.VsphereDVPGNetworkSetSpec vsphereNetwork)` parameter of the workload network `SetSpec` object.`PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/cluster_id/networks/network_id` request and submit a `NamespaceManagementNetworksSetSpec` JSON object in the request body. The `NamespaceManagementNetworksSetSpec` data structure holds the `NamespaceManagementNetworksVsphereDVPGNetworkSetSpec` structure through which you define the parameters of the existing vSphere Distributed port group.Use the `NetworksTypes.VsphereDVPGNetworkSetSpec` class to describe the configuration or retrieve information about the current configuration of the vSphere Distributed port group of a specific workload network.

If you want to retrieve a list of the distributed switches compatible with vSphere Supervisor on a vCenter system, use the `DistributedSwitchCompatibility` service and filter the available switches by using `VSPHERE_NETWORK` as the networking provider.

If you want to retrieve a list of the distributed switches compatible with vSphere Supervisor on a vCenter system, use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/distributed-switch-compatibility` request and pass the `VSPHERE_NETWORK` value for the `network_provider` query parameter.

Installing and Configuring the HAProxy Load Balancer

You can use the VMware Cloud Foundation APIs to customize the HAProxy control plane VM after you install the HAProxy in your vSphere Supervisor environment.

If you use the vSphere networking stack in your vSphere Supervisor environment, you need to supply your own load balancer. You can use the open source implementation of the HAProxy load balancer that VMware provides.

For more information about the prerequisites for installation and the deployment procedure through the vSphere Client, see the *Installing and Configuring vSphere Supervisor* documentation.

You can use the VMware Cloud FoundationREST APIs to install and configure the HAProxy load balancer. You can download the latest version of the HAProxy OVA file from the VMware-HAProxy site to a content library item. For more information about how to achieve this task, see Upload an OVF or OVA Package from a Local File System to a Library Item . Then you can create a new VM from the OVA template in the content library as described in Deploy a Virtual Machine or vApp from an OVF Template in a Content Library.

To configure the HAProxy load balancer, call the `set(String cluster, String id, LoadBalancersTypes.SetSpec spec)` and pass the configuration through the `LoadBalancersTypes.SetSpec` object. The load balancer specification takes as a setting a `LoadBalancers.HAProxyConfigCreateSpec` instance that captures the runtime configuration of the HAProxy load balancer. You must set the following configuration parameters to the HAProxy load balancer configuration specification: To configure the HAProxy load balancer, use the `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/load-balancers/<loadbalancer_id>` request and submit a `LoadBalancers.SetSpec` JSON object in the request body. You can define the following HAProxy load balancer

settings within the `LoadBalancers.HAProxyConfigSetSpec` data structure which is an optional property of the `LoadBalancers.SetSpec` data structure.

Parameter	Description
`servers setServers(List<LoadBalancersTypes.Server> servers)`	A list of `Server`s that represent the endpoints for configuring the HAProxy load balancers. Each endpoint is described by a load balancer IP address and a Data Plane API management port. Each endpoint must be described with the port on the HAProxy VM on which the Data Plane API service listens. The Data Plane API service controls the HAProxy server and runs inside the HAProxy VM. The default port is *5556*. Port *22* is reserved for SSH.
`username setUsername(String username)`	The administrator user name that is configured with the HAProxy OVA file and is used to authenticate to the HAProxy Data Plane API server.
`password setPassword(char[] password)`	The password for the administrator user name.
`certificate_authority_chain setCertificateAuthorityChain(String certificateAuthorityChain)`	The certificate in PEM format that is signed or is a trusted root of the server certificate that the Data Plane API server presents.

Using the Avi Load Balancer with vSphere Networking

If you use the vSphere networking stack for workload management, you can install and configure the Avi Load Balancer, also known as Avi Load Balancer, Essentials Edition, to support the VKS clusters.

For more information about how to install and configure the Avi Load Balancerthrough the vSphere Client, see the *Installing and Configuring vSphere Supervisor* documentation.

You can use the VMware Cloud Foundation APIs to deploy the Avi Controller on your vSphere Management network. You can upload the latest version of the Avi Load Balancer to a library item from your local file system or from a URL. For more information about how to achieve this task, see Upload an OVF or OVA Package from a Local File System to a Library Item . Then you can deploy the Controller VM on your vSphere Management network from the OVA template in the content library as described in Deploy a Virtual Machine or vApp from an OVF Template in a Content Library.

To configure the Avi Load Balancer settings, create a `LoadBalancers.AviConfigCreateSpec` instance and use the following parameters.

To configure the Avi Load Balancer, send a `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>/load-balancers/<loadbalancer_id>` request and submit a `LoadBalancers.SetSpec` JSON object in the request body. You can define the following Avi Load Balancer settings within the `LoadBalancers.AviConfigSetSpec` data structure which is an optional property of the `LoadBalancers.SetSpec` data structure.

Parameter	Description
`server setServer(LoadBalancersTypes.Server server)`	The address of the Avi Controller that is used to configure virtual services.
`username setUsername(java.lang.String username)`	The administrator user name that is used for accessing the Controller VM of the Avi Load Balancer.
`password setPassword(char[] password)`	The password for the administrator user name.
`certificate_authority_chain setCertificateAuthorityChain(java.lang.String certificateAuthorityChain)`	The certificate in PEM format that is used by the Controller. You can use the certificate that you assigned during the configuration of the Avi Load Balancer.

Enable vSphere Supervisor on a Cluster with NSX as the Networking Stack

Through the VMware Cloud Foundation APIs, you can enable a vSphere cluster for managing Kubernetes workloads. A cluster configured with NSX supports running vSphere Pod and VKS clusters.

- Verify that your environment meets the system requirements for enabling vSphere Supervisor on the cluster. For more information about the requirements, see the *vSphere Supervisor Concepts and Planning* documentation.
- Verify that the NSX is installed and configured. See Configuring NSX for vSphere Supervisor.
- Create storage policies for the placement of pod ephemeral disks, container images, and Supervisor control plane cache.
- Verify that DRS is enabled in fully automated mode and HA is also enabled on the cluster.
- Configure shared storage for the cluster. Shared storage is required for vSphere DRS, HA, and storing persistent volumes of containers.
- Verify that the user who you use to access the VMware Cloud Foundation services has the **Modify cluster-wide configuration** privilege on the cluster.
- Create a subscribed content library on the vCenter system to accommodate the VM image that is used for creating the nodes of the VKS clusters.

To enable a vSphere cluster for Kubernetes workload management, you use the services under the `namespace_management` package.

1. Retrieve the IDs of the tag-based storage policies that you configured for vSphere Supervisor.

   Use the `Policies` service to retrieve a list of all storage policies and then filter the policies to get the IDs of the policies that you configured for the Supervisor.

2. Retrieve the IDs of the vSphere Distributed Switch and the NSX Edge cluster that you created when configuring the NSX for vSphere Supervisor.

   Use the `DistributedSwitchCompatibility` service to list all vSphere Distributed Switches associated with the specific vSphere cluster and then retrieve the ID of the Distributed Switch that you configured to handle overlay networking for the Supervisor. Use the `EdgeClusterCompatibility` service to retrieve a list of the created NSX Edge clusters for the specific vSphere cluster and associated with the specific vSphere Distributed Switch. Retrieve the ID of the NSX Edge cluster that has the tier-0 gateway that you want to use for the namespaces networking.

3. Retrieve the ID of the port group for the management network that you configured for the management traffic.

   Use the `Networks` service to list the visible networks available on the vCenter instance that match some criteria and then retrieve the ID of the management network you previously configured.

4. Create a `ClustersTypes.EnableSpec` instance and define the parameters of the Supervisor that you want to create.

   You must specify the following required parameters of the enable specification:

   - Storage policies settings and file volume support. The storage policy you set for each of the following parameters ensures that the respective object is placed on the datastore referenced in the storage policy. You can use the same or different storage policy for the different inventory objects.

Parameter	Description
`setEphemeralStoragePolicy(java.lang.String ephemeralStoragePolicy)`	Specify the ID of the storage policy that you created to control the storage placement of the vSphere Pods.
`setImageStorage(ClustersTypes.ImageStorageSpe imageStorage)`	Set the specification of the storage policy that you created to control the placement of the cache of container images.
`setMasterStoragePolicy(java.lang.String masterStoragePolicy)`	Specify the ID of the storage policy that you created to control the placement of the Supervisor control plane cache.

   Optionally, you can activate the file volume support by using `setCnsFileConfig(CNSFileConfig cnsFileConfig)`. See Enabling ReadWriteMany Support.

- Management network settings. Configure the management traffic settings for the Supervisor control plane.

Parameter	Description
`setNetworkProvider(ClustersTypes.NetworkProvider networkProvider)`	Specify the networking stack that must be used when the Supervisor is created. To use the NSX as the network solution for the cluster, select `NSXT_CONTAINER_PLUGIN`.
`setMasterManagementNetwork(ClustersTypes.NetworkSpec masterManagementNetwork)`	Enter the cluster network specification for the Supervisor control plane. You must enter values for the following required properties: • `setNetwork(java.lang.String network)` - Use the management network ID retrieved in Step 3. • `setMode(ClustersTypes.NetworkSpec.Ipv4Mode mode)` - Set `STATICRANGE` or `DHCP` for the IPv4 address assignment mode. The `DHCP` mode allows an IPv4 address to be automatically assigned to the Supervisor control plane by a DHCP server. You must also set the floating IP address used by the HA primary cluster by using `setFloatingIP(java.lang.String floatingIP)`. Use the DHCP mode only for test purposes. The `STATICRANGE` mode, allows the Supervisor control plane to have a stable IPv4 address. You can use it in a production environment. • `setAddressRange(ClustersTypes.Ipv4Range addressRange)` - Optionally, you can configure the IPv4 addresses range for one or more interfaces of the management network. Specify the following settings: – The starting IP address that must be used for reserving consecutive IP addresses for the Supervisor control plane. Use up to 5 consecutive IP addresses. – The number of IP addresses in the range. – The IP address of the gateway associated with the specified range. – The subnet mask to be used for the management network.
`setMasterDNS(java.util.List<java.lang.String> masterDNS)`	Enter a list of the DNS server addresses that must be used from the Supervisor control plane. If your vCenter instance is registered with an FQDN, you must enter the IP addresses of the DNS servers that you use with the vSphere environment so that the FQDN is resolvable in the Supervisor. The list of DNS addresses must be specified in the order of preference.
`setMasterDNSSearchDomains(java.util.List<java.lang.String> masterDNSSearchDomains)`	Set a list of domain names that DNS searches when looking up for a host name in the Kubernetes API server. Order the domains in the list by preference.

Parameter	Description
`setMasterNTPServers(java.util.List<java.lang.String> masterNTPServers)`	Specify a list of IP addresses or DNS names of the NTP server that you use in your environment, if any. Make sure that you configure the same NTP servers for the vCenter instance, all hosts in the cluster, the NSX, and vSphere Supervisor. If you do not set an NTP server, VMware Tools time synchronization is enabled.

- Workload network settings. Configure the settings for the networks for the namespaces. The namespace network settings provide connectivity to vSphere Pods and namespaces created in the Supervisor.

Parameter	Description
`setNcpClusterNetworkSpec(ClustersTypes.NCPClusterNetworkEnableSpec ncpClusterNetworkSpec)`	Set the specification for the Supervisor configured with the NSX networking stack. Specify the following cluster networking configuration parameters for `NCPClusterNetworkEnableSpec`: • `setClusterDistributedSwitch(java.lang.String clusterDistributedSwitch)` - The vSphere Distributed Switch that handles overlay networking for the Supervisor. • `setNsxEdgeCluster(java.lang.String nsxEdgeCluster)` - The NSX Edge cluster that has tier-0 gateway that you want to use for namespace networking. • `setNsxTier0Gateway(java.lang.String nsxTier0Gateway)` - The tier-0 gateway that is associated with the cluster tier-1gateway. You can retrieve a list of `NSXTier0Gateway` objects associated with a particular vSphere Distributed Switch and determine the ID of the tier-0 gateway you want to set. • `setNamespaceSubnetPrefix(java.lang.Long namespaceSubnetPrefix)` - The subnet prefix that defines the size of the subnet reserved for namespaces segments. Default is 28. • `setRoutedMode(java.lang.Boolean routedMode)` - The NAT mode of the workload network. If set to `false`: • The IP addresses of the workloads are directly accessible from outside the tier-o gateway and you do not need to configure the egress CIDRs. • File Volume storage is not supported. Default is `true`. • `setEgressCidrs(java.util.List<Ipv4Cidr> egressCidrs)` - The external CIDR blocks from which the NSX Manager assigns IP addresses used for performing source NAT (SNAT) from internal vSphere Pods IP addresses to external IP addresses. Only one egress IP address is assigned for each namespace in the Supervisor. These IP ranges must not overlap with the IP ranges of the vSphere Pods, ingress, Kubernetes services, or other services running in the data center. • `setIngressCidrs(java.util.List<Ipv4Cidr> ingressCidrs)` - The external CIDR blocks from which the ingress IP range for the Kubernetes services is determined. These IP ranges are used for load balancer services and Kubernetes ingress. All Kubernetes ingress services in the same namespace share a common IP address. Each load balancer service is assigned a unique IP address. The ingress IP ranges must not overlap with the IP ranges of the vSphere Pods, egress, Kubernetes services, or other services running in the data center. • `setPodCidrs(java.util.List<Ipv4Cidr> podCidrs)` - The internal CIDR blocks from which the IP ranges for vSphere Pods are determined. The IP ranges must not overlap with the IP ranges of the ingress, egress, Kubernetes services, or other services running in the data center. All vSphere Pods CIDR blocks must be of at least /23 subnet size.

Parameter	Description
`setServiceCidr(Ipv4Cidr serviceCidr)`	Specify the CIDR block from which the IP addresses for Kubernetes services are allocated. The IP range must not overlap with the ranges of the vSphere Pods, ingress, egress, or other services running in the data center. For the Kubernetes services and the vSphere Pods, you can use the default values which are based on the cluster size that you specify.

- Supervisor size. You must set a size to the Supervisor which affects the resources allocated to the Kubernetes infrastructure. The cluster size also determines default maximum values for the IP addresses ranges for the vSphere Pods and Kubernetes services running in the cluster. You can use the `ClusterSizeInfo.get()` calls to retrieve information about the default values associated with each cluster size.
- Optional. Associate the Supervisor with the subscribed content library that you created for provisioning VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.
  To set the library, use the `setDefaultKubernetesServiceContentLibrary(java.lang.String defaultKubernetesServiceContentLibrary)` method and pass the subscribed content library ID.

5. Enable vSphere Supervisor on a specific cluster by passing the cluster enable specification to the `Clusters` service.

A task runs on vCenter for turning the cluster into a Supervisor. Once the task completes, Kubernetes control plane nodes are created on the hosts that are part of the cluster enabled with vSphere Supervisor. Now you can create vSphere Namespaces.

Create and configure namespaces on the Supervisor. See Create a vSphere Namespace.

Java

This example enables vSphere Supervisor on a cluster that has NSX-T configured as the networking stack.

The following code snippet is part of the `EnableSupervisorCluster.java` sample. Some parts of the original code sample are omitted to save space. You can view the complete and up-to-date version of this sample in the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
      (...)
@Override
  protected void run() throws Exception {

      System.out.println("We are building the Spec for enabling vSphere supervisor
 cluster");

      ClustersTypes.EnableSpec spec = new ClustersTypes.EnableSpec();
      (...)
      spec.setSizeHint(SizingHint.TINY);
      (...)
      spec.setServiceCidr(serCidr);
      spec.setNetworkProvider(ClustersTypes.NetworkProvider.NSXT_CONTAINER_PLUGIN);
      (...)
      spec.setNcpClusterNetworkSpec(NCPSpec);
      (...)
      spec.setMasterManagementNetwork(masterNet);
      (...)
      spec.setMasterDNS(masterDNS);
      (...)
      spec.setWorkerDNS(workerDNS);
      (...)
      spec.setMasterNTPServers(NTPserver);

      spec.setMasterStoragePolicy(this.storagePolicyId); // Storage policy identifier
      spec.setEphemeralStoragePolicy(this.storagePolicyId);// Storage policy
 identifier
      spec.setLoginBanner("This is your first Project pacific cluster");
```

```
        (...)
        spec.setImageStorage(imageSpec);

        this.ppClusterService.enable(clusterId, spec);
        System.out.println("Invocation is successful for enabling vSphere supervisor
cluster, check H5C");

    }
        (...)
```

Enable vSphere Supervisor on a Cluster with the vSphere Networking Stack

Starting with vSphere 7.0 Update 1, you can select between creating a Supervisor with the vSphere networking stack or with NSX as the networking solution. A Supervisor that is configured with the vSphere networking stack only supports VKS clusters. vSphere Pods are not supported.

- Verify that your environment meets the system requirements for enabling vSphere Supervisor on the cluster. For more information about the requirements, see the documentation.
- Verify that DRS is enabled in fully automated mode and HA is also enabled on the cluster.
- Configure shared storage for the cluster. Shared storage is required for vSphere DRS, HA, and storing persistent volumes of containers.
- Create storage policies for the placement of Kubernetes control planes.
- Create a subscribed content library on the vCenter system to accommodate the VM image that is used for creating nodes of VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.
- Add all hosts from the cluster to a vSphere Distributed Switch and create port groups for workload networks. See Configuring the vSphere Networking Stack for vSphere Supervisor.
- Configure an HAProxy load balancer instance that is routable to the vSphere Distributed Switch that is connected to the hosts from the vSphere cluster.
- Verify that the user who you use to access the VMware Cloud Foundation services has the **Namespaces.Manage** privilege on the cluster.

To enable a cluster configured with the vSphere networking stack for Kubernetes workloads management, you must use the services under the `namespace_management` package.

1. Retrieve the ID of the cluster which hosts were added to the vSphere Distributed Switch.

   Use the `ClusterCompatibility` service to filter the clusters by using their network providers. To retrieve a list of all clusters in the vCenter system which are configured with the vSphere networking stack, set the network provider in the filter specification to `VSPHERE_NETWORK`.

2. Retrieve the IDs of the tag-based storage policies that you configured for vSphere Supervisor.

   Use the `Policies` service to retrieve a list of all storage policies and then filter the policies to get the IDs of the policies that you configured for the Supervisor.

3. Retrieve the ID of the port group for the management network that you configured for the management traffic.

   To list the visible networks available on the vCenter instance that match some criteria and then retrieve the ID of the management network you previously configured, use the `Networks` service.

4. Create a Supervisor enable specification and define the parameters of the Supervisor that you want to enable.

   You must specify the following required parameters of the enable specification:

   - Supervisor size. You must set a size to the Supervisor which affects the resources allocated to the Kubernetes infrastructure. The cluster size also determines default maximum values for the IP addresses ranges for the vSphere Pod and Kubernetes services running in the cluster. You can use the `ClusterSizeInfo.get()` call `GET https://<server>/api/vcenter/namespace-management/cluster-size-info` request to retrieve information about the default values associated with each cluster size.
   - Storage policy settings and file volume support. To specify the ID of the storage policy that you created to control the placement of the Supervisor control plane cache, use the `master_storage_policy` propertythe `setMasterStoragePolicy(java.lang.String masterStoragePolicy)` method.

Optionally, you can activate the file volume support by using the `cns_file_config` propertythe `setCnsFileConfig(CNSFileConfig cnsFileConfig)` method. See Enabling ReadWriteMany Support.

- Load balancer. To specify the user-provisioned load balancer configuration for the cluster, use the `setLoadBalancerConfigSpec(LoadBalancersTypes.ConfigSpec loadBalancerConfigSpec)` parameter of the enable specification. You must specify the following parameters of the `LoadBalancersTypes.ConfigSpec` specification:

Parameter	Description
`setId(java.lang.String id)`	A user-friendly name of the load balancer. The name must be an alphanumeric string with a maximum length of 63 characters which is unique across the namespaces in the vCenter instance.
`setProvider(LoadBalancersTypes.Provider provider)`	The type of the load balancer that you want to use. In vSphere 7.0 Update 2, you can choose between the HAProxy load balancer and the Avi Load Balancer. Pass as a value to this parameter one of the following constants: `HA_PROXY` or `AVI`.
`setAddressRanges(java.util.List<IPRange> addressRanges)`	The IP address ranges in CIDR format from which HAProxy allocates the IP addresses for the virtual servers. You must provide at least one IP range which is reserved by HAProxy. The CIDR range specified with this parameter must not overlap with the IPs allocated for the Kubernetes control planes and workloads. The IP range that you configure must be on a separate subnet.
`setHaProxyConfigCreateSpec(LoadBalancersTypes.HaProxyConfigCreateSpec haProxyConfigCreateSpec)`	The HAProxy runtime configuration. See Installing and Configuring the HAProxy Load Balancer.
`setAviConfigCreateSpec(LoadBalancersTypes.AviConfigCreateSpec aviConfigCreateSpec)`	The Avi Load Balancer configuration. See Using the Avi Load Balancer with vSphere Networking.

- Management network settings. Configure the network parameters for the Kubernetes control planes.

Parameter	Description
`setNetworkProvider(ClustersTypes.NetworkProvider networkProvider)`	Specify the networking stack that must be used when the Supervisor is created. To use the vSphere network as the solution for the cluster, select `VSPHERE_NETWORK`.
`setMasterManagementNetwork(ClustersTypes.NetworkSpec masterManagementNetwork)`	Enter the cluster network specification for the Supervisor control plane. You must enter values for the following required properties: • `setNetwork(java.lang.String network)` - Use the management network ID retrieved in Step 3. • `setMode(ClustersTypes.NetworkSpec.Ipv4Mode mode)` - Set `STATICRANGE` or `DHCP` for the IPv4 address assignment mode. The `DHCP` mode allows an IPv4 address to be automatically assigned to the Supervisor control plane by a DHCP server. You must also set the floating IP address used by the HA primary cluster by using `setFloatingIP(java.lang.String floatingIP)`. Use the DHCP mode only for test purposes. The `STATICRANGE` mode, allows the Supervisor control plane to have a stable IPv4 address and can be used in a production environment.
`setMasterDNS(java.util.List<java.lang.String> masterDNS)`	Enter a list of the DNS server addresses that must be used from the Supervisor control plane. If your vCenter instance is registered with an FQDN, you must enter the IP addresses of the DNS servers that you use with the vSphere environment so that the FQDN is resolvable in the Supervisor. The list of DNS addresses must be specified in the order of preference.

Parameter	Description
`setMasterDNSSearchDomains(java.util.List<java.lang.String> masterDNSSearchDomains)`	Set a list of domain names that DNS searches inside the Kubernetes control plane nodes, so that the DNS server can resolve them. Order the domains in the list by preference.
`setMasterNTPServers(java.util.List<java.lang.String> masterNTPServers)`	Specify a list of IP addresses or DNS names of the NTP server that you use in your environment, if any. Make sure that you configure the same NTP servers for the vCenter instance, all hosts in the cluster, and vSphere Supervisor. If you do not set an NTP server, VMware Tools time synchronization is enabled.

- Workload network settings. Configure the settings for the network that will handle the networking traffic for Kubernetes workloads running on the Supervisor.

Parameter	Description
`setServiceCidr(Ipv4Cidr serviceCidr)`	Specify the CIDR block from which the IP addresses for Kubernetes services are allocated. The IP range must not overlap with the ranges of the vSphere Pods, ingress, egress, or other services running in the data center. For the Kubernetes services and the vSphere Pods, you can use the default values which are based on the cluster size that you specify.
`setWorkloadNetworksSpec(ClustersTypes.WorkloadNetworksSpec workloadNetworksSpec)`	Enter the workload networks specifications for the cluster. To configure the primary workload network that is used to expose the Supervisor control plane to DevOps and other workloads, create a `NetworksTypes.CreateSpec` instance. Enter the following parameters of the vSphere Distributed Switch: • `setNetwork(java.lang.String network)`. The name of the vSphere Distributed Switch that is associated with the hosts in the cluster. The name must be a unique alphanumeric string that does not exceed 63 characters. • `setNetworkProvider(ClustersTypes.NetworkProvider networkProvider)`. Pass `VSPHERE_NETWORK` as value to this parameter. • `setVsphereNetwork(NetworksTypes.VsphereDVPGNetwo vsphereNetwork)`. Optionally, you can create a `Network sTypes.VsphereDVPGNetworkCreateSpec` instance to describe the configuration of the namespace network backed by the vSphere Distributed port group. You must define the following parameters for the vSphere Distributed port group specification: – `setPortgroup(java.lang.String portgroup)`. Specify the port group that serves as the primary network to the Supervisor. – `setAddressRanges(java.util.List<IPRange> addressRanges)`. Set the IP range for allocating IP addresses for the Kubernetes control planes and workloads. You must use unique IP ranges for each workload network. – `setGateway(java.lang.String gateway)`. Set the gateway for the primary network. – `setSubnetMask(java.lang.String subnetMask)`. Specify the subnet mask of the network.

- Content library settings. Add the subscribed content library that contains the VM images for deploying the nodes of VKS clusters. See Creating, Securing, and Synchronizing Content Libraries for VKS Releases.
  To set the library, use `setDefaultKubernetesServiceContentLibrary(java.lang.String defaultKubernetesServiceContentLibrary)` and pass the subscribed content library ID.

5. Enable the Supervisor by passing the enable specification to the `Clusters` service.

A task runs on vCenter for enabling vSphere Supervisor on the cluster. Once the task completes, three Kubernetes control planes are created on the hosts that are part of the cluster.

Create and configure namespaces on the Supervisor.

Upgrading a Supervisor
You can use the vSphere SupervisorREST APIs to upgrade a single or a group of clusters to a specific version.

vSphere Supervisor supports rolling upgrades through the VMware Cloud FoundationREST APIs for Supervisors and for the infrastructure supporting these clusters. This model ensures that there is minimal downtime for the cluster workloads during the upgrade process.

To retrieve a list of all available vSphere Supervisor upgrade versions for a specific vCenter system, use the Cluster Available Versions service. You can get information about the release version, name, description, release date, and release notes for each available upgrade.

You must use the Software Clusters service for upgrading a Supervisor. You can retrieve upgrade information about all Supervisors enabled on a vCenter system by using the `list()` Java method `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/software/clusters` HTTP request. You receive a list of basic upgrade-related information for each cluster, such as the current software version, the date of the last successful upgrade, the upgrade status of the cluster, and so on. In case some of the clusters are in the process of upgrading, you can retrieve also information about their desired upgrade version. If you want to view a more detailed upgrade-related information about a cluster, you must use the `get(cluster_ID)` method`GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/software/clusters/<cluster_id>` HTTP request.

After you view the details about the upgrade versions that you can apply on a single or multiple Supervisors, you can create upgrade specifications that define the versions you want to upgrade to. When you upgrade a batch of Supervisor and for some reason one of the clusters fails to upgrade, you receive information about the pre-check exceptions that led to that cluster upgrade failure.

Java

This example upgrades a Supervisor.

The following code snippet is part of the `UpgradeSupervisorCluster.java` sample. Some parts of the original code sample are omitted to save space. You can view the complete and up-to-date version of this sample in the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
@Override
 protected void run() throws Exception {

  System.out.println("We are building the Spec for upgrading vSphere supervisor
 cluster");
  com.vmware.vcenter.namespace_management.software.ClustersTypes.UpgradeSpec spec =
 new UpgradeSpec();
  spec.setIgnorePrecheckWarnings(true);
  spec.setDesiredVersion(this.desiredVersion);
  this.wcpUpdateService.upgrade(clusterId, spec);
  System.out.println(
    "Invocation is successful for updating vSphere supervisor cluster,
                              check H5C, track the status using GET API");
 }
```

Monitoring the Enable and Upgrade Supervisor Operations
When you run the Supervisor enable and upgrade operations, the status of the tasks is not returned. Since these operations might be time-consuming but critical when automating second- and third-party products with vSphere Supervisor, you can write a logic to track the task status.

To monitor the status of the enable and upgrade Supervisor operations, use the `Clusters.Info` data structure`ClustersTypes.Info` instance and query the Kubernetes and configuration status of the cluster. Track the status every two minutes or so, until you receive `READY` for `kubernetes_statusgetKubernetesStatus()` and

`RUNNING` for `config_statusgetConfigStatus()`. These statuses indicate that the Supervisor reached the desired configuration status and is ready for running Kubernetes workloads.

Reconfiguring a Supervisor
You can change some or all the predefined settings of a Supervisor through the VMware Cloud FoundationREST APIs.

To update only some of the Supervisor settings, you must create an instance of `UpdateSpec` and pass it to the `update(cluster_ID, update_spec)` methoduse the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_ID>` HTTP request and submit the updated specification in the request body. To reconfigure entirely the Supervisor, you must create an instance of `SetSpec` and pass it to the `set(cluster_ID, set_spec)` methoduse the `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_ID>` HTTP request and submit a `Clusters.SetSpec` data structure in the request body. The specification replaces entirely all existing Supervisor settings.

The settings you can configure with the `UpdateSpec` and `SetSpec` specifications are the same as the ones that you used for enabling the Supervisor. For example, you can change the storage settings on the Supervisor. Note that the changes that you make to the storage settings after the initial cluster configuration, apply only to the newly created Supervisor control planes.

Deactivating a Supervisor
You can programmatically deactivate vSphere Supervisor on a vSphere cluster by using the VMware Cloud FoundationREST APIs.

When you deactivate a Supervisor, the vSphere Kubernetes Service forcefully deletes from the cluster all objects and configurations part of the Kubernetes infrastructure. To deactivate Kubernetes workloads on a cluster, call the `disable(String cluster)` operation of the `Clusters` service and pass as parameter the ID of the cluster on which you want to deactivate Kubernetes.Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters/<cluster_id>?action=disable` request and pass the cluster ID as a path parameter.

Content Libraries in vSphere Supervisor

vSphere Supervisor uses content libraries as centralized repositories for templates, VM images, VKS release distributions, and other files related to their deployment.

Creating, Securing, and Synchronizing Content Libraries for VKS Releases
VMware Tanzu distributes Kubernetes software versions as VKS releases. To obtain and use these releases on your VKS clusters, you create subscribed or local content libraries.

A VKS release provides the VMware Kubernetes distribution which can be used with VKS clusters. Each VKS release is distributed as an OVA package. The uses the OVA package to deploy the virtual machine nodes for VKS clusters.

A VKS release is supported on Photon OS. The virtual machine nodes that are built from the OVA package have a 16 GB disk size. You specify the CPU and RAM resource reservations when you use a virtual machine class to size the VKS cluster.

Depending on your need for synchronization frequency and on the access to the published content libraries storing the VKS releases, you can use two approaches for storing VKS releases.

> **Note:**
>
> Starting with vSphere 7.0 Update 3, you can protect your content library by a security policy. In such case, make sure that all library items are compliant. If a protected library includes a mix of compliant and non-compliant library items, DevOps engineers are not able to retrieve the list of VM images provided with the library.

Automated Synchronization of VKS Releases

VMware publishes a content library that contains the latest VMware distributions of Kubernetes as an OVA package. If you want to provision VKS clusters, you can create a subscribed content library on the vCenter instance where vSphere Supervisor is enabled. When configuring the content library subscription, use the following subscription URL of the

publisher : https://wp-content.vmware.com/v2/latest/lib.json. For more information about how to create a subscribed content library, see Subscribe to a Content LibrarySubscribing to a Content Library.

When you create the subscription, you configure the synchronization mechanism for downloading the content of the published library. You can select between on demand and automatic download of the virtual machine image for the VKS cluster nodes. If you choose to synchronize the subscribed library on demand, only the metadata for the library content is updated and as a result storage space is saved. This approach is an important consideration as more images containing different Kubernetes versions are published. However, the first time you decide to use a new virtual machine image version, you have to wait for it to download.

Starting with vSphere 7.0 Update3, you can secure a subscribed content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you update a subscribed content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

You associate the subscribed content library with the Supervisor on which you want to create a VKS cluster, when you first enable vSphere Supervisor on a cluster. See Enable vSphere Supervisor on a Cluster with NSX as the Networking StackEnable vSphere Supervisor on a Cluster with NSX as the Networking Stack.

The size of the content library can grow over time as new Kubernetes versions and images are published. If the underlying storage runs out of space, you will need to move to a new subscribed content library. After you create a new subscribed content library that has sufficient capacity for the target cluster, update the library association of the Supervisor. See Reconfiguring a Supervisor.

Manual Synchronization of VKS Releases

In an air-gapped network environment, you can use the storing functionality provided by a local content library for the needed VKS releases. You must first create a local content library, then download the OVA package for each VKS release that you want to import to the library. See Create a Local Content LibraryCreating a Local Content Library.

Starting with vSphere 7.0 Update3, you can secure a local content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you update a local content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

You can find the latest versions of the Kubernetes distribution by navigating to the https://wp-content.vmware.com/v2/latest URL. You must download the `photon-ova.ovf` and `photon-ova-disk1.vmdk` for each distribution you want and then upload these files from your local file system to your local content library. See Upload an OVF or OVA Package from a Local File System to a Library Item .

> **Note:**
>
> Make sure that you use as a name for each library item the Photon image version and the Kubernetes version from the directory where you downloaded the files. For example: `photon-3-k8s-v1.20.2---vmware.1-tkg.1.1d4f79a`.

Creating and Managing Content Libraries for VM Provisioning in vSphere Supervisor

To provision new virtual machines in a vSphere Supervisor environment, the DevOps engineers rely on VM templates and images. Your role is to make sure the DevOps engineers have access to these VM templates and images by using the Content Library service.

You can create a local content library and populate it with VM templates in OVF or OVA file format, or other types of files. For more information and a sample of how to create a local content library, see Create a Local Content LibraryCreating a Local Content Library.

You can also create a subscription to download the content of a published local content library as described in the following topic: Subscribe to a Content LibrarySubscribing to a Content Library.

Starting with vSphere 7.0 Update3, you can secure the content library. The Content Library service verifies the library signing certificate during the synchronization process. If the certificate verification fails, only the library metadata is synchronized and the library content is not downloaded. For more information how to apply a security policy when you

update a local or subscribed content library, see Editing the Settings of a Content LibraryEditing the Settings of a Content Library.

After you create the content library, you must populate it with content either from your local file system or from a Web server. You must use only the VM images available on the VMware Cloud Marketplace web site. For example, download or subscribe to VM Service Image for Ubuntu if you want to enable a DevOps engineer to deploy a VM using this image. For more information about the available ways to populate a content library with content, see How To Use Library ItemsWorking with Library Items.

You must give the DevOps engineers access to the VM templates stored in the content libraries, so that they can use these templates to provision VMs through the VM Service functionality. To give access, you must associate one or more content libraries to the namespace where the VM Service is present. See Associating a Content Library with a Namespace and Virtual Machines in vSphere Supervisor.

Associating a Content Library with a Namespace

You must give access to a source of VM templates, so that the DevOps engineers can use them to provision VMs in a self-service manner. To give access, you associate a content library with VM templates to the namespace used by the DevOps engineers.

You can add multiple content libraries to a namespace that has the VM Service enabled or the same content library to several namespaces. You associate a content library to a namespace when you create a new namespace, update or reconfigure an existing one.

To make the VM Service aware of the content libraries in your environment that the DevOps engineers can use to self-service VMs, you must use a `VMServiceSpec` instance and pass it to the namespace configurationuse the `NamespacesInstancesVMServiceSpec` data structure which is part of the namespace create specification. The instance contains a list of content libraries that will be used by the VM Service. You can specify this list with the `content_libraries` propertyby calling the `setContentLibraries(java.util.Set<java.lang.String> contentLibraries)` method of the VM Service specification.

You can also associate one or more VM classes with the namespace. See Associating a VM Class with a vSphere Namespace.

Managing Namespaces on a Supervisor

You can use the VMware Cloud Foundation APIs to create namespaces on a Supervisor and configure them with resource limits and permissions for the DevOps users.

To create and configure a namespace, use the `Instances` service from the `namespaces` package. You can configure the access control to the objects in a namespace by using the `Access` service.

Create a vSphere Namespace

You can use the vSphere Supervisor automation REST APIs to create namespaces on a Supervisor. You can set resource quotas, storage, as well as permissions for the DevOps users.

- Enable vSphere Supervisor on a vSphere cluster.
- Create users and groups for the DevOps engineers who will use the namespace. For more information about how to create users and groups through the Web Services APIs, see the *vSphere Web Services SDK Programming Guide*.
- Create storage policies for persistent storage used by the vSphere Pods and the pods inside a VKS cluster.
- Create VM Classes and content libraries for DevOps provisioned VMs. See Create a VM Class in vSphere Supervisor and Creating and Managing Content Libraries for VM Provisioning in vSphere Supervisor.
- Required privileges on the Supervisor:
  - **Namespaces** > **Modify cluster-wide configuration**
  - **Namespaces** > **Modify namespace configuration**
  - **Virtual Machine Classes** > **Manage Virtual Machine Classes**

1. Retrieve the Supervisor ID by filtering the clusters available in the vCenter system.

   Call the list method of the `Clusters` service from the `com.vmware.vcenter.namespace_management` package and retrieve the ID of the cluster on which you want to create a namespace from the returned cluster summary objects.

   Use the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/clusters` request and retrieve the ID of the cluster on which you want to create a namespace from the list of cluster summary JSON objects.
2. Retrieve the ID of the storage policy that you configured for placement of the persistent volumes from vSphere Pods and VKS clusters.
3. Configure the access control to the objects in the namespace.

   Create an instance of the `com.vmware.vcenter.namespaces.InstancesTypes.Access` class and specify the following access information:

   Populate the properties of the `Instances.Access` data structure with appropriate values:

Parameter Property	Description
`domain setDomain(domain)`	Set the domain name of the vCenter system on which the namespace is created.
`subject_type setSubjectType(subjectType)`	Set the type of the user accounts that are associated with the specific role for the namespace. You must select between the `USER` and `GROUP` options.
`subject setSubject(subject)`	Set the name of the user or group that have permissions to access the namespace objects.
`role setRole(role)`	Set the role that is associated with the predefined set of privileges that you want to grant the specific user or group. You can select between the `EDIT`, `VIEW` and `OWNER` roles. The owner role is introduced in vSphere 7.0 Update 2a. When a DevOps engineer creates a namespace in a self-service manner, the Namespace Self-Service grants the owner role to the namespace creator. See Self-Service Namespace Management.

4. Create a `CreateSpec` instance that holds the namespaces specification. Populate the `Instances.CreateSpec` data structure with the appropriate namespace specification information.

   The namespace specification can contain the following information:

Parameter Property	Description
`cluster setCluster(cluster)`	Set the ID of the Supervisor on which the namespace is created.
`namespace setNamespace(namespace)`	Set a name of the namespace following the DNS label standard defined in RFC 1123. The name must be unique across all namespaces in the current vCenter system.
`networks setNetworks(java.util.List<java.lang.String> networks)`	Optional. You can set the workload networks used by the vSphere Namespace. Pass `null` as a value of this parameter, if the Su pervisor is configured to use NSX as networking solution. The workload networking support for such namespaces is provisioned by NSX. If the Supervisor uses the vSphere networking stack, pass the workload network to be associated with the namespace. If you pass `null` as a value of this parameter, the vSphere Namespaces on the cluster are automatically associated with the cluster primary workload network. See Configuring the vSphere Networking Stack for vSphere Supervisor.
`description setDescription(description)`	Optional. You can set a description of the namespace.

Parameter Property	Description
access_list setAccessList(accessList)	Optional. You can set the access control that is associated with the namespace in Step 3.
storage_specs setStorageSpecs(storageSpecs)	Optional. You can set the amount of storage dedicated to each storage policy associated with the namespace and the maximum amount of storage that is used by the namespace. Use the StorageSpec specification to configure the storage quotas on the namespace.
resource_spec setResourceSpec(resourceSpec)	Optional. You can set resource limitations to the namespace. You can limit the CPU, memory, the maximum number of pods that can exist on the namespace, and so on.
creator setCreator(InstancesTypes.Principal creator)	Optional. The Namespace Self-Service populates this parameter with information about the DevOps user who created the namespace with cubectl. The user name and domain of the namespace creator are stored with this parameter.
vm_service_spec setVmServiceSpec(InstancesTypes.VMServiceSpec vmServiceSpec)	Optional. The VM Service specification for the Dev-Ops provisioned virtual machines.

5. Create a namespace object on the Supervisor by using the namespace create specification.

   Use the POST https://*<vcenter_ip_address_or_fqdn>*/api/vcenter/namespaces/instances request and submit an Instances.CreateSpec JSON object in the request body.

Share the namespace with DevOps engineers and provide them with the user or group configured for accessing the namespace.

Java

This example creates a vSphere Namespace on a Supervisor.

The following code snippet is part of the CreateNameSpace.java sample. Some parts of the original code sample are omitted to save space. You can view the complete and up-to-date version of this sample in the vsphere-automation-sdk-java VMware repository at GitHub.

```
(...)
  @Override
  protected void run() throws Exception {

   InstancesTypes.CreateSpec spec =new InstancesTypes.CreateSpec();
   spec.setCluster(this.clusterId);
   spec.setDescription("My first namespace, WOW");
   spec.setNamespace(this.namespaceName);
   InstancesTypes.StorageSpec storageSpec=new InstancesTypes.StorageSpec();
   storageSpec.setLimit(Long.valueOf(this.storageLimit).longValue());
   storageSpec.setPolicy(this.storagePolicyId);
   List<InstancesTypes.StorageSpec> storageSpecs = new
ArrayList<InstancesTypes.StorageSpec>();
   storageSpecs.add(storageSpec);
   spec.setStorageSpecs(storageSpecs);
   InstancesTypes.Access accessList= new InstancesTypes.Access();
   accessList.setDomain(this.domainName);
   if(this.roleName.equalsIgnoreCase("EDIT")) {
   accessList.setRole(AccessTypes.Role.EDIT);
   } else{
    accessList.setRole(AccessTypes.Role.VIEW);
   }
   accessList.setSubject( this.subjectName); //Default is Administrator
   if(this.subjectType.equalsIgnoreCase("USER")) {
    accessList.setSubjectType( AccessTypes.SubjectType.USER);
```

```
    } else{
     accessList.setSubjectType( AccessTypes.SubjectType.GROUP);
    }

    List<InstancesTypes.Access> accessLists = new ArrayList<InstancesTypes.Access>();
    accessLists.add(accessList);
    spec.setAccessList(accessLists);
    this.namespaceService.create(spec);
    System.out.println("Invocation is successful for creating supervisor namespace,
check H5C or call GET API to get status");


  }
 (...)
```

Updating the Namespace Configuration

You can change the whole namespace configuration or only some of the namespace settings.

To change the configuration of an existing namespace, you must have the **Namespaces.Configure** privilege on the Supervisor.

> **Note:**
>
> Before deleting a storage policy from vCenter or a vSphere Namespace, or changing the storage policy assignment, make sure that no persistent volume claim with the corresponding storage class runs in the namespace. Also, ensure that no VKS cluster is using the storage class.

To patch a namespace configuration, create an `UpdateSpec` specification and set new values only to the configuration settings that you want to change. The parameters of the update specification are the same as the ones you configured during the namespace creation. When you call the update operation, only the settings that you configured in the update specification are applied, the other settings are left as they are.

To patch a namespace configuration, use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/<namespace_id>` request and submit an `UpdateSpec` JSON object in the request body. You set new values only to the configuration settings that you want to change. The parameters of the update specification are the same as the ones you configured during the namespace creation. When you issue the update request only the settings that you configured in the update specification are applied, the other settings are left as they are.

To reconfigure a namespace entirely, you must create an instance of the `SetSpec` class. You can change the description, access controls, storage settings, and resource limitations of the specific namespace.

To reconfigure a namespace entirely, you must issue a `PUT https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/<namespace_id>` request and submit a `SetSpec` JSON object in the request body. You can change the description, access controls, storage settings, and resource limitations of the specific namespace.

Configuring the Access to a Namespace

You can use the vSphere Supervisor APIs to grant access permissions to DevOps engineers on the vSphere Namespaces.

Use the `Access` service to retrieve information about the access control of the DevOps engineers on a specific namespace. You can also set up or remove an access control for a specific user or group on a specific namespace, and add another access control on the namespace. You set up each access control to allow a user or group to access a namespace in a specific vCenter system. You can grant access to a DevOps engineer to more than one namespace.

You must have the **Namespaces** > **Configure** > privilege to grant permissions to a user. You assign the view and edit access role on the namespace for the user or group.

Starting with vSphere 7.0 Update 2?, you can also assign the owner role to a DevOps engineer. These roles allow the user to deploy workloads, share the namespace with other DevOps engineers, and delete it when it is no longer needed.

Self-Service Namespace Management
You can use the vSphere Supervisor automation REST APIs to create a vSphere Namespace with specific resource quotas, set permissions, and assign storage policies. DevOps engineers can then use the namespace as a template for self-provisioning namespaces on the cluster.

Starting with vSphere 7.0 Update 2a, the Namespace Self-Service feature is available in vSphere Supervisor. The service enables Kubernetes users to create vSphere Namespaces from templates configured through the automation REST APIs or vSphere Client. To activate the Namespace Self-Service on a cluster, use one of the following options:

- Create a self-service namespace template and then activate the Namespace Self-Service on the cluster.
- Create or update a self-service namespace template simultaneously with activating the Namespace Self-Service on the cluster.

Currently, only one namespace self-service template is allowed per vSphere Namespace. After a DevOps engineer creates a namespace from the template, the namespace can also be deleted through `kubectl`. You can verify whether a namespace is created from a template by retrieving the value of the`self_service_namespace` property of the `Instances.Info` JSON object that you receive in the request body of the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/instances/namespace_id` request.`getSelfServiceNamespace()` flag of the `com.vmware.vcenter.namespaces.InstancesTypes.Info` object that you receive when you call the `get(String namespace)` method of the `Instances` interface.

To create a template for a self-service namespace, call the `create(String cluster, NamespaceTemplatesTypes.CreateSpec spec)` method of the `NamespaceTemplates` interface. use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-templates/clusters/cluster_id` HTTP request. You use as path parameterparameters the cluster ID and submit in the request body the namespace template create specification.

You define the following configuration settings and resource limitations of the template:

ParameterProperty	Description
`templatesetTemplate(String template)`	The identifier of the namespace template must be a unique name across all clusters on the vCenter instance. The name must be compliant with DNS.
`resource_specsetResourceSpec(Structure resourceSpec)`	The resource quotas, such as CPU and memory, that are reserved for the namespace on the vCenter instance. The CPU limit is set in MHz and the minimum value is 10 MHz. The memory and the storage limits are set in MiB. For more options to configure resource limits for the namespace, see the `ResourceQuotaOptionsV1` class in the API Reference documentation.
`storage_spec ssetStorageSpecs(List<InstancesTypes.StorageSpec> storageSpecs)`	The amount of storage in MiB utilized for each storage policy that you associate with the namespace. You must specify at least one policy.
`networkssetNetworks(List<java.lang.String> networks)`	Optional. The networks associated with the namespace. Currently, you can set only one network for the namespace. Pass `null` as argument if the Supervisor is configured with NSX-T Data Center support. If you pass `null` for a namespace template on a cluster configured with a vSphere networking stack, the namespace is automatically associated with the Supervisor management workload network.
`permission ssetPermissions(List<NamespaceTemplatesTypes.PermissionSpec> permissions)`	Optional. The permissions that allow DevOps engineers to use the template to self-provision namespaces through `kubectl`. If set to `null`unset, only users with the Administrator role can use the template.

Once you have the template created, you can activate the Namespace Self-Service on the cluster by calling the `activate(java.lang.String cluster)` method of the `NamespaceSelfService` interface.by issuing the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-self-`

`service/<cluster_id>?action=activate` HTTP request. If you want to restrict DevOps users to use the namespace template on a cluster, you can deactivate the Namespace Self-Service feature. Then users are able to delete only the namespaces already created from the template.

You can activate the Namespace Self-Service on the cluster after configuring the namespace template by using the `NamespaceSelfService` service. You call the `activateWithTemplate(java.lang.String cluster, NamespaceSelfServiceTypes.ActivateTemplateSpec spec)` method of the `NamespaceSelfService` interface.You issue the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespaces/namespace-self-service/<cluster_id>?action=activateWithTemplate` HTTP request and submit the `NamespaceSelfService.ActivateTemplateSpec` JSON object in the request body. Depending on the availability of a template on the cluster, this method either creates a namespace template or activates the deactivated service and at the same time updates the existing template.

Virtual Machines in vSphere Supervisor

vSphere Supervisor offers the VM Service functionality to enable DevOps engineers to provision and manage VMs on a namespace in a self-service manner. You use the vSphere Supervisor automation APIs to create VM classes that specify the deployment policy and resource reservations of such VMs.

Starting with vSphere 7.0 Update 2a, DevOps engineers can use the VM Service functionality to deploy and run VMs on a namespace through the `kubectl` commands. You can use the vSphere Supervisor automation APIs to manage the two VM Service components: VM classes and content libraries. For more information about managing content libraries in the context of vSphere Supervisor, see Content Libraries in vSphere Supervisor.

You can use the automation APIs to create and manage VM classes. A VM class specification defines the number of CPUs, memory capacity, and resource reservation settings of the desired virtual machine. vSphere Supervisor currently offers twelve ready-to-use VM classes (T-shirt sizes) that are derived from the most popular VMs in Kubernetes. Based on the resource reservation that a VM specification requests, each predefined VM class has two editions: guaranteed and best effort. The guaranteed VM class fully reserves the configured resources. A best effort VM class does not guarantee any resource reservations and allows their overcommitment.

You associate a VM class with a specific namespace to make it available to the DevOps engineers who have access to that namespace. You can assign any number of existing VM classes or create a custom one. Note that VMs deployed by the DevOps engineers through the VM Service can only be managed with the `kubectl` commands. A VM provisioned by DevOps engineers shares the same resources in a namespace as containers.

Use the `VirtualMachineClasses` interface to create and manage a specification of a VM class object. Through these objects you predefine the number of CPUs, memory capacity, and reservation settings. See Create a VM Class in vSphere Supervisor. To make a VM class available to the DevOps engineers for self-service VM deployment, you must associate it with a specific namespace. See Associating a VM Class with a vSphere Namespace.

Create a VM Class in vSphere Supervisor

You can use the VMware Cloud Foundation Kubernetes REST APIs to create custom VM classes to be used for VM deployment in vSphere Supervisor.

Required privileges:

- **Namespaces** > **Modify cluster-wide configuration**
- **Namespaces** > **Modify namespace configuration**
- **Virtual Machine Classes** > **Manage Virtual Machine Classes**

A VM class specifies the CPU, memory, and resource reservations for a VM. vSphere Supervisor offers several preconfigured VM classes which you can use as is, edit, or delete. You can also create a custom VM class in your vCenter instance and it will be available to all Supervisors and the namespaces created in these clusters. Note that even though a VM class is available to all namespaces, a DevOps user can only use the VM classes associated with the namespaces that he/she can access.

1. Create the specification of the VM class object by defining the following options.

Option	Description
`id setId(String id)`	The identifier of the VM class must follow these DNS requirements: • A unique name in the current vCenter instance. • An alphanumeric name with maximum 63 characters. • No uppercase letters of spaces. • A dash can be used anywhere except as a first or last character. Note that after a VM class is created, you cannot edit its ID.
`cpu_count setCpuCount(long cpuCount)`	The number of virtual CPUs (vCPUs) configured for a VM that are deployed with this VM class.
`memory_MB setMemoryMB(long memoryMB)`	The memory in MB configured for a VM that are deployed with this VM class. The value must be between 4 MB and 24 TB and a multiple of 4.
`description setDescription(String description)`	Optional. The description of the VM class.
`cpu_reservation setCpuReservation(Long cpuReservation)`	Optional. The percentage of total available CPU resources reserved for the VM deployed with the VM class. The percentage you specify with this attribute is multiplied by the minimum CPU available among all cluster nodes to get the CPU resources guaranteed by vSphere for a VM. The resulting value is in MHz.
`memory_reservation setMemoryReservation(Long memoryReservation)`	Optional. The percentage of available memory that is reserved for a VM deployed with this VM class. The value can be from 0 through 100%.

2. Create the VM class object.

   Call the `create(VirtualMachineClassesTypes.CreateSpec spec)` method of the `VirtualMachineClasses` interface and pass as argument the created VM class specification. Use the `POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes` HTTP request and pass the created VM class specification in the request body.

After you create the custom VM class, you can edit its parameters or delete it from your environment. See Editing or Removing a VM Class from Your Environment.

You can make your VM class available to DevOps engineers by associating it with a namespace. See Associating a VM Class with a vSphere Namespace.

Editing or Removing a VM Class from Your Environment

You can use the automation REST APIs to edit the configuration of a VM class that you created or a predefined VM classes that vSphere Supervisor offers. When you no longer need an existing VM class, you can remove it from your vCenter instance.

Note that editing a VM class specification does not affect the VMs that are already deployed by the DevOps engineers from this class. Only newly deployed VMs will use the reconfigured VM class.

Deleting a VM class results in its removal from all related namespaces. All VMs deployed with this VM class remain unchanged but DevOps engineers can no longer use it.

You can list all VM classes available for a vCenter instance by calling the `list` method of the `VirtualMachineClasses` interface. by using the `GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes` HTTP request. You receive a list of all VM classes and a detailed information about each one of them. Based on the detailed information, you can narrow the list and retrieve the IDs of the VM classes that you want to edit or delete.

To edit a VM class configuration, call the `update(String vmClass, VirtualMachineClassesTypes.UpdateSpec spec)` method of the `VirtualMachineClasses`

interface and pass as arguments the VM class ID and the update specification.use the `PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes/vm_class_id` HTTP request and pass the update specification in the request body. You can edit all VM class attributes except the ID of the class. Only the attributes modified within the update specification will be edited.

To delete a VM class from your environment, call the `delete(java.lang.String vmClass)` method of the `VirtualMachineClasses` interface and pass the ID of the class as argument.use the `DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/namespace-management/virtual-machine-classes/vm_class_id` HTTP request.

Associating a VM Class with a vSphere Namespace

You must associate a VM class with a namespace to make it available for DevOps engineers to deploy VMs in a self-service manner.

You can associate one or more VM classes with a single namespace or you can add one VM class to several namespaces. You can use the predefined VM classes that vSphere Supervisor provides or you can create custom ones. See Create a VM Class in vSphere Supervisor.

Since VM Service is the feature responsible for handling VM classes, you must make that service aware of the VM classes available to the engineers using a specific namespace. You can achieve this when you create a new namespace or edit an existing one. See Managing Namespaces on a Supervisor.

When you create the VM Service specification set the list of VM classes that must be used to create VMs on the specific namespace. You achieve this by calling the `setVmClasses(java.util.Set<java.lang.String> vmClasses)` method of the `VirtualMachineClasses` object.using the `vm_classes` property of the `Instances.VMServiceSpec` data structure.

You can also associate one or more content libraries with a namespace that has the VM Service enabled. See Associating a Content Library with a Namespace.

vSphere Security

This chapter provides information about securing your vSphere environment for vCenter and ESX.

Managing Certificates

You can use the VMware Cloud Foundation API to manage certificates in your vSphere environment. You can not only refresh default certificates that are issued by the VMware Certificate Authority (VMCA) but also add third-party or custom-made certificates to your environment.

Certificate Management Operations

Use the VMware Cloud Foundation API to manage trusted root certificate chains, VMware Certificate Authority (VMCA) root certificates, machine SSL (TLS) certificates, and Security Token Service (STS) signing certificates.

With the VMware Cloud Foundation API, you can refresh the VMCA-issued certificates but also add external and third-party certificates to your vSphere environment. For more information on vSphere certificate management, see the *vSphere Authentication* guide.

Location of the Certificate Management Services

You can find the vSphere certificate management services for your automation in the `com.vmware.vcenter.certificate_management.vcenter` Java package and the `com.vmware.vcenter.certificate_management.vcenter_client` Python module.

Certificate Management Interfaces

You can use the following interfaces to manage certificates with the VMware Cloud Foundation API:

- `TrustedRootChains`
- `VMCARoot`
- `TLSCSR`

- `TLS`
- `SigningCertificate`

Starting with VCF 9.1.1, the SigningCertificate interface supports generating a CSR and importing a signed certificate chain without transmitting the private key over the network. This secure workflow supplements the existing replace and refresh operations.

Certificate Management Operations

You can use the operations listed in the following table to manage certificates.

Table 1887: Certificate Management Operations

Operation	Interface	Java / Python Operation	Description	Introduced in
List trusted root certificates	`TrustedRootChains`	`list()`	You can retrieve the identifiers of all trusted root certificates that are present in vCenter.	vSphere 6.7 U2
Get trusted root certificate information	`TrustedRootChains`	`get(chain)`	You can retrieve a root certificate chain by providing its identifier. You can retrieve the identifier by using the List trusted root certificates operation.	vSphere 6.7 U2
Add a trusted root certificate	`TrustedRootChains`	`create(spec)`	You can add a trusted root certificate chain to your vCenter system.	vSphere 6.7 U2
Delete a trusted root certificate	`TrustedRootChains`	`delete(chain)`	You can delete a root certificate by providing its unique identifier. You can retrieve the identifier by using the List trusted root certificates operation.	vSphere 6.7 U2
Replace the VMCA root certificate	`VMCARoot`	`create(spec)`	You can replace the VMCA root certificate with a new VMCA-signed certificate. The operation triggers a restart of the services that are using this certificate.	vSphere 7.0
Generate a CSR	`TLSCSR`	`create(spec)`	You can generate a CSR and use it to issue a custom certificate from the given spec.	vSphere 6.7 U2
Get the Machine SSL certificate	`TLS`	`get()`	You can retrieve the machine SSL certificate of your vCenter system.	vSphere 6.7 U2
Renew the Machine SSL certificate	`TLS`	`renew(duration)`	You can renew the validity of the machine SSL certificate for a specified period in days. The duration must be less than or equal to 730 days. If you pass `null`/`None`, the default duration of 730 days is applied.	vSphere 6.7 U2
Replace the Machine SSL certificate with a custom signed certificate	`TLS`	`set(spec)`	You can replace the vCenter machine SSL certificate with a custom certificate signed by an external or a third-party Certificate Authority (CA).	vSphere 6.7 U2
Replace the Machine SSL certificate with a VMCA-signed certificate	`TLS`	`replaceVmcaSigned(spec)`	You can replace the vCenter machine SSL certificate with a VMCA-signed certificate.	vSphere 7.0

Operation	Interface	Java / Python Operation	Description	Introduced in
Retrieve the STS signing certificate chains	`SigningCertificate`	`get()`	You can retrieve the STS signing certificate chains, which are used for validating tokens signed by vCenter.	vSphere 7.0 U3
Replace the STS signing certificate	`SigningCertificate`	`set(spec)`	You can replace the current STS signing certificate with a certificate of your choice. The accepted file format is PEM.	vSphere 7.0 U3
Generate a CSR for the STS signing certificate	`SigningCertificate`	`create(spec)`	You can generate a Certificate Signing Request (CSR) for the STS signing certificate and stage the private key on the vCenter appliance. The private key is not returned to the caller.	VCF 9.1.1
Import a signed STS signing certificate	`SigningCertificate`	`update(spec)`	You can import an externally signed STS signing certificate chain without including the private key in the request body. The operation uses the private key that the appliance staged during CSR generation.	VCF 9.1.1

Add a Root Certificate to vCenter

You can use the `TrustedRootChains` interface to add, delete and read trusted root certificate chains.

- Verify that the root certificate or certificate chain you want to add is available on your machine.
- Verify that you have the required privileges: **CertificateManagement** > **Manage** and **CertificateManagement** > **Administer**.

If you want to use an enterprise or third-party certificate authority (CA) for certificate management of your vSphere environment, you must first establish trust with that CA. You can do this by adding the root certificate of the external CA to the trusted root store of your vCenter system.

Adding a root certificate or certificate chain to the vCenter trusted certificate store establishes trust with an enterprise or third-party certificate authority. You can add a root certificate to vCenter as a prerequisite for other scenarios such as setting a third-party or enterprise machine SSL certificate.

1. Optional: Retrieve the root certificates on your vCenter system by calling the `list` function of the `TrustedRootChains` interface.
2. Create a `X509CertChain` instance with the root certificate you want to add.
3. Create a specification with the new `X509CertChain` instance.
4. To add the root certificate, call the `create` function of the `TrustedRootChains` interface.

If the operation is successful, the system returns the unique identifier of the trusted root certificate you added.

Python

This example shows how to add a root certificate or certificate chain to your vCenter system. The example is based on the code in the `trusted_root_chains_create.py` sample file.

Note:
For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
"""
Description: Demonstrates the import of the TRUSTED ROOT CHAIN into vCenter
Sample Prerequisites:
- The user invoking the API should have the CertificateManagement.Manage or the
CertificateManagement.Administer privilege
"""

parser = sample_cli.build_arg_parser()
```

```
parser.add_argument('--certchain',
                    required=True,
                    help='The certificate chain to be imported into vCenter.')

args = sample_util.process_cli_args(parser.parse_args())

session = requests.session()
session.verify = False if args.skipverification else True

# Login to vCenter
vsphere_client = create_vsphere_client(server=args.server,
                                       username=args.username,
                                       password=args.password,
                                       session=session)

cert_chain =
 args.certchain.encode(encoding='utf-8').decode('unicode_escape').split(',')

"""
Creation of the spec for input to the API
"""
x509_cert_chain = X509CertChain(cert_chain=cert_chain)
cert_chain = TrustedRootChains.CreateSpec(cert_chain=x509_cert_chain)

print('The alias of the certificate chain successfully imported into vCenter listed
 below ')
print(vsphere_client.vcenter.certificate_management.vcenter.TrustedRootChains.create(cert_chain)
```

Delete a Root Certificate from vCenter

You can use the `TrustedRootChains` interface to add, delete and read trusted root certificate chains. This use case demonstrates how to delete a root certificate or certificate chain from the trusted root store of your vCenter system.

Verify that you have the required privileges for executing the method: **CertificateManagement** > **Administer** and **CertificateManagement** > **Manage**.

Deleting certificates is not available through the vSphere Client and you can only do this by using the VMware Cloud Foundation API or the CLI tools.

> **CAUTION:**
> Deleting a root certificate or certificate chain that is in use might cause breakage of your systems. Proceed to delete a root certificate only if you are sure it is not in use by your vCenter or any connected systems.

1. Optional: Retrieve the root certificates from your vCenter system by calling the `list` function of the `TrustedRootChains` interface.

   The system lists the unique identifiers (chains) of the certificates in the trusted certificate store.
2. Retrieve the certificate you want to delete by calling the `get` function and passing the unique identifier (chain) of the certificate as an argument.
3. Delete the certificate by using the delete function of the `TrustedRootChains` interface and passing the unique identifier (chain) of the certificate as an argument.
4. Optional: To verify you deleted the certificate, list the root certificates from your vCenter system once again.

The root certificate is deleted from your vCenter system.

Python	This example shows how to delete a root certificate or certificate chain from your vCenter system. The example is based on the code in the `trusted_root_chains_delete.py` sample file.

Note:
For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
"""
Description: Demonstrates the deletion of the TRUSTED ROOT CHAIN corresponding to the
 provided alias
Sample Prerequisites:
- The user invoking the API should have the CertificateManagement.Manage or the
CertificateManagement.Administer privilege
"""

parser = sample_cli.build_arg_parser()

parser.add_argument('--certalias',
                    required=True,
                    help='The alias for the certificate chain to be deleted from
 vCenter.')

args = sample_util.process_cli_args(parser.parse_args())

session = requests.session()
session.verify = False if args.skipverification else True

# Login to vCenter
vsphere_client = create_vsphere_client(server=args.server,
                                       username=args.username,
                                       password=args.password,
                                       session=session)

cert_alias = args.certalias

print('Deleting the certificate chain corresponding to the alias ' + cert_alias)
vsphere_client.vcenter.certificate_management.vcenter.TrustedRootChains.delete(cert_alias)
```

Change the Machine SSL Certificate of vCenter

You can change the machine SSL certificate of a vCenter system by using the TLS and the TLS CSR interfaces of the VMware Cloud Foundation API.

- Verify that the root certificate of the CA you are going to use is available on your machine.
- Verify that you have the required privileges: **CertificateManagement** > **Administer** and **CertificateManagement** > **Manage**.

The machine SSL certificate is used for server verification and for secure communication such as HTTPS or LDAPS. The machine certificates are the human-facing certificates in vSphere. They are used to create an SSL socket on the server side to which SSL clients can then connect.

Changing the machine SSL certificate with one issued by an official or enterprise certificate authority is an essential part of the Hybrid Mode of vSphere certificate management. In this mode, you replace the machine SSL certificate and you leave the VMCA to manage all other certificates automatically. The VMCA is a just-enough internal certificate authority that comes integral with your vSphere deployment. It has been purpose-built to serve the certificate needs of your vSphere environment. For more information on vSphere certificate management, see the *vSphere Authentication* guide.

1. Optional: Retrieve the current machine SSL certificate of your vCenter system by calling the `get` function of the `Tls` interface.
2. Generate a certificate signing request (CSR) by using the `TlsCsr` interface.
   a) Create a new object of type `TlsCsrTypes.Spec` by using the following parameters/methods.

Parameter/Method	Description
`setCountry` / `country`	Specify the country in the certificate subject.
`setStateOrProvince` / `state_or_province`	Specify the state or province in the certificate subject.
`setLocality` / `locality`	Specify the locality in the certificate subject.
`setOrganization` / `organization`	Specify the organization in the certificate subject.
`setOrganizationUnit` / `organization_unit`	Specify the organization unit in the certificate subject.
`setEmailAddress` / `email_address`	Specify the email address in the certificate subject.

   b) Create the CSR by calling the `create(TlsCsrTypes.Spec)` method and passing the CSR specification.

   The system returns the CSR in PEM format.
3. Save the CSR to your machine.
4. Send the CSR to the certificate authority of your choice.

   Note:

   The private key corresponding to the public key generated by the CSR is stored in the vCenter keystore and does not leave your system.
5. Save the issued third-party machine SSL certificate to your machine.
6. Set the new custom certificate to your vCenter system by using the `Tls` service.
   a) Create a new object of type `TlsTypes.Spec` by using the following parameters/methods.

Parameter/Method	Description
`setCert` / `cert`	The Machine SSL certificate in PEM format. You must also paste the intermediate CA certificate, if you have one.
`setRootCert` / `root_cert`	The third-party root CA certificate in PEM format. You must also paste the intermediate CA certificate, if you have one.

   You do not provide the private key as it was generated with the CSR and is already present on your system.
   b) Set the new certificate to your vCenter system by calling the `set(TlsTypes.Spec)` method and passing the TLS specification as an argument.

   Note:

   Restart of the vCenter services after the certificate change is not necessary.

The certificate replacement is completed seamlessly and all your sessions remain active.

Python

- *Generating a Certificate Signing Request (CSR) from vCenter*
  This example shows how to generate a certificate signing request (CSR) from vCenter. The generation of a CSR is an important step in the workflow to change the machine SSL certificate of your vCenter system with a third-party or enterprise certificate. The example is based on the code in the `gencsr.py` sample file.

  Note:
  For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
"""
Description: Demonstrates the generation of the Certificate Signing request
for the MACHINE SSL certificate
Sample Prerequisites:
```

```
- The user invoking the API should have the CertificateManagement.Administer or the
CertificateManagement.Manage privilege.
"""

parser = sample_cli.build_arg_parser()

parser.add_argument('--keysize',
                    help='Key size used to generate the private key.'
                         'keysize will take 2048 bits if not modified')

parser.add_argument('--commonname',
                    help='Common name of the certificate subject field.'
                         'common name will take the Primary Network
 Identifier(PNID) if not modified.')

parser.add_argument('--organization',
                    required=True,
                    help='Organization field in certificate subject.')

parser.add_argument('--organizationunit',
                    required=True,
                    help='Organization unit field in certificate subject')

parser.add_argument('--locality',
                    required=True,
                    help='Locality field in the certificate subject')

parser.add_argument('--stateorprovince',
                    required=True,
                    help='State field in certificate subject')

parser.add_argument('--country',
                    required=True,
                    help='Country field in the certificate subject')

parser.add_argument('--emailaddress',
                    required=True,
                    help='Email field in Certificate extensions')

parser.add_argument('--subjectaltname',
                    help='subjectaltname is list of Dns Names and Ip addresses')

args = sample_util.process_cli_args(parser.parse_args())

session = requests.session()
session.verify = False if args.skipverification else True

# Login to vCenter
vsphere_client = create_vsphere_client(server=args.server,
                                       username=args.username,
                                       password=args.password,
                                       session=session)

common_name = args.commonname
organization = args.organization
organization_unit = args.organizationunit
locality = args.locality
```

```
state_or_province = args.stateorprovince
country = args.country
email_address = args.emailaddress

if args.keysize is None:
    key_size = args.keysize
else:
    key_size = int(args.keysize)

if args.subjectaltname is None:
    subject_alt_name = args.subjectaltname
else:
    subject_alt_name = args.subjectaltname.split(',')

"""
Create the spec for input to the API
"""
spec = TlsCsr.Spec(key_size=key_size,
                   common_name=common_name,
                   organization=organization,
                   organization_unit=organization_unit,
                   locality=locality,
                   state_or_province=state_or_province,
                   country=country,
                   email_address=email_address,
                   subject_alt_name=subject_alt_name)

print('Generating the certificate signing request based on the information provided
 in the spec ')
print(vsphere_client.vcenter.certificate_management.vcenter.TlsCsr.create(spec))
```

- *Setting a New Machine SSL Certificate to vCenter*

  This example shows how to set a a third-party or enterprise machine SSL certificate to your vCenter system. You must have already completed the step of generating a certificate signing request (CSR) and obtained the new certificate from an enterprise or third-party certificate authority. This example is based on the code in the `replace_tls_certificate.py` sample file.

  Note:
  For related code samples, see the `vsphere-automation-sdk-python` VMware repository at GitHub.

```
"""
Description: Demonstrates the replacement of the MACHINE SSL certificate with a
 custom
certificate signed by an external third party CA.
Sample Prerequisites:
- The user invoking the API should have the CertificateManagement.Administer
 privilege.
"""

parser = sample_cli.build_arg_parser()

parser.add_argument('--cert',
                    required=True,
                    help='Leaf certificate for replace the MACHINE SSL
 certificate.')

parser.add_argument('--key',
                    help='The private key.'
```

```
                          'Not required if the gencsr api was used to generated the
 certificate signing request.')

parser.add_argument('--rootcert',
                    help='The root certificate and the intermediate root
 certificates '
                         'required to establish the chain of trust.'
                         'Not required if the certificates are already present in
 the vCenter.')

args = sample_util.process_cli_args(parser.parse_args())

session = requests.session()
session.verify = False if args.skipverification else True

# Login to vCenter
vsphere_client = create_vsphere_client(server=args.server,
                                       username=args.username,
                                       password=args.password,
                                       session=session)

cert = args.cert.encode(encoding='utf-8').decode('unicode_escape')

if args.key is not None:
    key = args.encode(encoding='utf-8').key.decode('unicode_escape')
else:
    key = args.key

if args.rootcert is not None:
    root_cert = args.rootcert.encode(encoding='utf-8').decode('unicode_escape')
else:
    root_cert = args.rootcert

"""
Create the spec for input to the API
"""
spec = Tls.Spec(cert=cert,
                key=key,
                root_cert=root_cert)


print('The MACHINE SSL certificate will be replaced with the custom certificate ')
vsphere_client.vcenter.certificate_management.vcenter.Tls.set(spec)
```

Refresh the vCenter STS Signing Certificate with a VMCA-Issued Certificate

You can refresh the vCenter Security Token Service (STS) signing certificate with a new VMCA-issued certificate by using the `SigningCertificate` interface. The STS is an internal entity that issues and verifies tokens so that vSphere services can communicate with and trust each other.

Verify that you have the required privilege: **CertificateManagement** > **Administer**.

You can refresh the current STS signing certificate of your vCenter system with a new VMCA-issued certificate.

There are two valid reasons for refreshing your STS signing certificate or certificate chain.

- If it is close to expiry. The standard lifespan of the vCenter STS signing certificate is 10 years. Your vCenter system will notify you in advance of STS certificate expiry. An alarm is triggered once per week when your STS certificate is 90 days away from expiry, and then daily when seven days away.

- If you already replaced your signing certificate with a third-party or enterprise one and now want to revert back to a default VMCA-issued certificate. This procedure replaces the custom or third-party STS signing certificates you added.

1. Optional: Retrieve the current vCenter signing certificate chain by calling the `get` function of the `SigningCertificate` interface.

   ```
   get()
   ```

2. Refresh the vCenter signing certificate by calling the `refresh` function of the `SigningCertificate` interface.

   Pass `true` as a value to the `force` parameter to force the refresh of the signing certificates in environments that would otherwise prevent the operation from occurring, such as a mixed-version environment. If `null`, the refresh of the vCenter signing certificate chain is not forced.

If successful, the system returns the x509 certificate chain issued in accordance with the vCenter policies.

CAUTION:

If you used a forced refresh, you must restart your vCenter and all linked services.

Java

This example shows how to refresh the vCenter STS signing certificate. The example is based on the code in the `RefreshSigningCertificate.java` sample file.

Note:
For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

```
/**
 * Sample code to refresh the STS Signing Certificate for the vCenter.
 * Use the force option to attempt to force the refresh in environments
 * that would otherwise fail such as mixed-version environments. On success,
 * the new signing certificates will be printed.
 */
public class RefreshSigningCertificate extends SamplesAbstractBase {
    private SigningCertificate certService;
    protected boolean force;

    @Override
    protected void parseArgs(String args[]) {
        Option forceOption = Option.builder()
            .required(false)
            .argName("FORCE")
            .longOpt("force")
            .desc("Attempt to force refresh")
            .build();
        List<Option> optionList = Arrays.asList(forceOption);
        super.parseArgs(optionList, args);

        this.force = parsedOptions.get("force") != null;
    }

    @Override
    protected void setup() throws Exception {
        this.certService =
            vapiAuthHelper.getStubFactory().createStub(SigningCertificate.class,
                                                       sessionStubConfig);
    }

    @Override
    protected void run() throws Exception {
        X509CertChain newCert= certService.refresh(this.force);
        if(newCert == null ) {
```

```
            System.out.println("ERROR: refresh signing certificate did not return a
certificate");
        } else {
            System.out.println("New vCenter signing certificate
\n"+newCert.toString());
        }
    }

    public static void main(String[] args) throws Exception {
        RefreshSigningCertificate get = new RefreshSigningCertificate();
        get.execute(args);
    }

    @Override
    protected void cleanup() throws Exception {
        // No cleanup required
    }
}
```

Replace the vCenter STS Signing Certificate

Import and replace the vCenter STS signing certificate with a custom generated or third-party certificate by using the `SigningCertificate` interface. For vCenter 9.1.1 and later, generate the certificate signing request (CSR) on the appliance and import only the signed certificate chain, so the private key never leaves vCenter.

- Verify that you have access to a certificate authority (CA) that can sign a certificate request.
- Verify that you have the required privilege: **CertificateManagement** > **Administer**.
- Verify that the vCenter instance is version 9.1.1 or later. For earlier releases, use the procedure in Certificate Replacement Using the Legacy Method instead.

The STS is an internal entity that issues and verifies tokens so that vSphere services can communicate with and can trust each other. The STS signing certificate is not an external-facing certificate. A fresh vCenter 8.0 or later installation includes a signing certificate with a default duration of 10 years. Replace the STS signing certificate with a custom or third-party certificate only if your company security policy requires you to do so.

Before vCenter 9.1.1, the only supported method for this task was the `set` operation that is described in Certificate Replacement Using the Legacy Method, which requires that the caller send the unencrypted private key to vCenter in the request. Starting with vCenter 9.1.1, the `SigningCertificate` interface also provides a two-step CSR workflow: vCenter generates the key pair and stages the private key internally, and the caller submits only the CA-signed certificate chain. Use this method on vCenter 9.1.1 and later so that the private key remains within the vCenter appliance throughout the certificate rotation. The `set` method remains available on vCenter 9.1.1 and later for backward compatibility, but Broadcom recommends the CSR-based method for all new automation.

1. Generate a CSR and stage a new private key on the vCenter instance by creating a `CreateSpec` instance and calling `create(spec)` on the `SigningCertificate` interface.

   Java This example generates a CSR for the STS signing certificate.

```
SigningCertificate certService =
    vapiAuthHelper.getStubFactory().createStub(SigningCertificate.class,
 sessionStubConfig);

SigningCertificateTypes.CreateSpec createSpec = new
 SigningCertificateTypes.CreateSpec();
createSpec.setCommonName("ssoserverSign");
createSpec.setOrganization("Broadcom Inc.");
createSpec.setSubjectAltName(Arrays.asList("lvn-dvm-10-161-137-38",
 "10.161.137.38"));
createSpec.setKeySize(2048L);

String csr = certService.create(createSpec);
```

```
System.out.println("CSR:\n" + csr);
```

Python

This example generates a CSR for the STS signing certificate. The example follows the pattern used in the `gencsr.py` sample file (vcf-sdk-python GitHub repository) for the machine SSL certificate.

```
from com.vmware.vcenter.certificate_management.vcenter_client import
 SigningCertificate

signing_cert_service =
 stub_factory.StubFactory(session).stub_config(SigningCertificate)

create_spec = SigningCertificate.CreateSpec(
    common_name='ssoserverSign',
    organization='Broadcom Inc.',
    subject_alt_name=['lvn-dvm-10-161-137-38', '10.161.137.38'],
    key_size=2048
)

csr = signing_cert_service.create(create_spec)
print('CSR:\n' + csr)
```

Java Method / Python Argument	Description
`setCommonName` / `common_name`	Common name for the certificate subject. If unset, vCenter uses the primary network identifier (PNID).
`setOrganization` / `organization`	Organization for the certificate subject. If unset, vCenter supplies an appliance-appropriate default.
`setOrganizationUnit` / `organization_unit`	Organization unit for the certificate subject. Leave this field unset: the CA/Browser Forum prohibits CAs from including an organizational unit, and some CAs reject a CSR that specifies one.
`setLocality` / `locality`	Locality for the certificate subject. If unset, vCenter omits this field from the certificate subject.
`setStateOrProvince` / `state_or_province`	State or province for the certificate subject. If unset, vCenter omits this field from the certificate subject.
`setCountry` / `country`	Country for the certificate subject. If unset, vCenter omits this field from the certificate subject.
`setEmailAddress` / `email_address`	Email address for the certificate subject. Consider leaving this field unset: many current certificate profiles and CAs discourage or reject an email address in the subject.
`setSubjectAltName` / `subject_alt_name`	Subject alternative names, such as additional hostnames and IP addresses, to include in the CSR. If unset, vCenter generates the CSR with no subject alternative names.
`setKeySize` / `key_size`	The size, in bits, of the RSA key pair that vCenter generates and stages for this request, for example, 2048. If unset, vCenter uses a secure default.

The CSR in PEM format is returned.

2. Save the CSR to your machine and send it to the certificate authority of your choice.
3. Save the issued leaf certificate and the CA root certificate to your machine.
4. Import the CA-signed certificate chain by creating an `UpdateSpec` instance and calling `update(spec)` on the `SigningCertificate` interface.

   Order the chain leaf certificate first and root certificate last. Do not include the private key: vCenter matches the chain against the private key that is staged in step 1.

   Java

   ```
   X509CertChain signingCertChain = new X509CertChain();
   ```

```
signingCertChain.setCertChain(Arrays.asList(leafCertificatePem,
 rootCertificatePem));

SigningCertificateTypes.UpdateSpec updateSpec = new
 SigningCertificateTypes.UpdateSpec();
updateSpec.setSigningCertChain(signingCertChain);

certService.update(updateSpec);
```

Python

```
from com.vmware.vcenter_client import X509CertChain

signing_cert_chain = X509CertChain(cert_chain=[leaf_certificate_pem,
 root_certificate_pem])
update_spec = SigningCertificate.UpdateSpec(signing_cert_chain=signing_cert_chain)

signing_cert_service.update(update_spec)
```

The call completes without a return value and activates the new signing certificate.

vCenter uses the imported certificate chain for STS token signing.

> **CAUTION:**
>
> The change of the STS signing certificate might leave systems in the local vCenter domain in a non-functional state. To prevent system failure, restart your vCenter instance and all linked services. This restart is required because the STS caches its signing certificate, regardless of which method you use to replace it.

Certificate Replacement Using the Legacy Method

The `set` method remains available on vCenter 9.1.1 and later for backward compatibility.

It requires that you supply both the certificate chain and the unencrypted private key in a single call, so use the CSR-based procedure instead unless an existing integration depends on this method.

1. Verify that the custom-generated or third-party certificate chain and the corresponding private key are available on your machine.
2. Create an instance of `SigningCertificateTypes.SetSpec` with the corresponding private key and certificate chain.
   a) Set the private key by calling `setPrivateKey(privateKey)` (Java) or passing `private_key` (Python). Pass the corresponding unencrypted PKCS#8 private key in base64-encoded PEM format.
   b) Set the certificate chain by calling `setSigningCertChain(X509CertChain)` (Java) or passing `signing_cert_chain` (Python). The chain must be valid, with the leaf certificate marked for Digital Signature key usage, the leaf certificate first in the sequence, and the root certificate last.
3. Set the STS signing certificate by calling `set(spec)` and passing the signing certificate specification.
4. Restart your vCenter instance and all linked services to prevent system failure, since the change might leave systems in the local vCenter domain in a non-functional state.

Managing vSphere Native Key Provider

You can use the VMware Cloud FoundationREST APISDK to perform VMware vSphere®Native Key Provider™ operations.

In vSphere 7.0 Update 2 and later, you can use the built-in vSphere Native Key Provider to enable encryption technologies, such as virtual TPMs (vTPM).

vSphere Native Key Provider is included in all vSphere editions and does not require an external key server (also called a Key Management Server (KMS) in the industry). You can also use vSphere Native Key Provider for vSphere Virtual Machine Encryption, but you must purchase the VMware vSphere®Enterprise Plus Edition™.

For details about vSphere Native Key Provider, see the *vSphere Security* documentation.

The procedures in this chapter are based on the REST API. For details, see the *vSphere Automation REST API Reference*.

The procedures in this chapter are based on the Java API. For details, see the *vSphere Automation Java API Reference*.

Retrieve vSphere Native Key Providers
You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to list vSphere Native Key Providers and retrieve details about a specific vSphere Native Key Provider.

1. Retrieve a list of vSphere Native Key Providers by calling the `list(ProvidersTypes.FilterSpec filterSpec)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers
   ```

   In the response body, you receive information about the health status, ID, and type of each provider.

2. Retrieve information about a vSphere Native Key Provider by calling the `get(java.lang.String provider)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

   In the response body, you receive detailed information about the health status, the type of the provider, and any constraints on the provider.

Manage vSphere Native Key Providers
You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to perform vSphere Native Key Provider management operations.

You can add, update, and remove vSphere Native Key Providers. Some operations require you to specify parameters in the body of the HTTP request according to your environment. For details about the syntax of each HTTP request body, see the *API Reference* documentation.

1. Add a vSphere Native Key Provider by specifying the provider ID in the request body by calling the `create(ProvidersTypes.CreateSpec spec)` method of the `Providers` interface.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers
   ```

   The following request body contains example syntax.

   ```
   {
       "native_spec": {
       },
       "provider": Test_Provider
   }
   ```

2. Update a vSphere Native Key Provider by specifying the new provider configuration in the request body by calling the `update(java.lang.String provider, ProvidersTypes.UpdateSpec spec)` method of the `Providers` interface.

   ```
   PATCH https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

3. Remove a vSphere Native Key Provider by calling the `delete(java.lang.String provider)` method of the `Providers` interface.

   ```
   DELETE https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers/<provider>
   ```

Export and Import vSphere Native Key Provider Configuration
You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.kms` package to export and import vSphere Native Key Provider configuration.

You can set a vSphere Native Key Provider as the default for a vCenter instance by using the `SetDefaultKmsCluster` method of the `CryptoManagerKmip` managed object from the vSphere ?anagement SDK.

The operations require you to specify parameters in the body of the HTTP request according to your environment. For details about the syntax of each HTTP request body, see the *API Reference* documentation.

The code snippets in the following procedure are from the `ManageNativeKeyProviders.java` sample file.

> **Note:**
> For a complete and up-to-date version of the Java sample code, see the `vsphere-automation-sdk-java` VMware repository at GitHub.

1. Export a vSphere Native Key Provider configuration by specifying the provider you want to export in the request body by calling the `export(ProvidersTypes.ExportSpec spec)` method of the `Providers` interface.

   Note:
   You must export the vSphere Native Key Provider configuration at least once before you can use a vSphere Native Key Provider.

   a) Initiate the export by providing the name identifier of the key provider and a password to encrypt the export data in the request body.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers?action=export
   ```

   The following request body contains example syntax.

   ```
   {
       "password": <your_password>,
       "provider": Test_Provider
   }
   ```

   In the response body, you receive the URL of the exported configuration and an authentication token. The following response body contains example syntax.

   ```
   {
       "location": {
           "download_token": {
               "expiry": 2023-09-20T21:04:50.000Z,
               "token": eyJhbGciOiJIUzI1NjKXn21y01CM
           },
           "url": https://vc8.home/cryptomanager/kms/test
       },
       "type": LOCATION
   }
   ```

   b) Send an HTTP request to the URL with the authentication token to fetch the exported PKCS12 data by providing the URL and token values received in the response body of the previous step.

   ```
   POST <url>
   Authorization: Bearer <token>
   ```

   The following example HTTP request contains the values from the example response in the previous step.

   ```
   POST https://vc8.home/cryptomanager/kms/test
   Authorization: Bearer eyJhbGciOiJIUzI1NjKXn21y01CM
   ```

   a) Back up vSphere Native Key Provider data.

   This is a two-stage process. First, you request a backup. In the second step, you download the backup by using a token and address that are returned from the first step.

   ```
   private byte[] backupKeyProvider(String name, char[] pwd) {
       // Step 1: request backup
       ExportSpec spec = new ExportSpec.Builder(name)
           .setPassword(pwd)
           .build();
       ExportResult res = nativeKeyProviders.export(spec);
       log("Backup step one: export result is {0}", res);

       // Step 2: download the backup
       URI url = res.getLocation().getUrl();
       char[] token = res.getLocation().getDownloadToken().getToken();
       return downloadBackupData(url, token);
   }
   ```

b) Download the backup data from an online location.

This method uses the Java Apache HTTP client to download the back up data. The download is performed by making a POST request to the URL with an authorization bearer header carrying the supplied token.

```
private byte[] downloadBackupData(URI url, char[] token) {
    HttpPost request = new HttpPost(url);
    request.addHeader("Authorization", MessageFormat.format("Bearer {0}", new String(token)));

    log("Backup request {0}", request);
    try (CloseableHttpClient client = createHttpClient();
            CloseableHttpResponse resp = client.execute(request)) {
        int statusCode = resp.getStatusLine().getStatusCode();
        if (statusCode != 200) {
            log("Backup failed. HTTP status code {0}", statusCode);
            throw new RuntimeException("Cannot backup");
        }
        HttpEntity body = resp.getEntity();

        byte[] backup = new byte[body.getContent().available()];
        body.getContent().read(backup);
        log("Backup received {0} bytes. Backup completed.", backup.length);
        return backup;
    } catch (IOException e) {
        log("IO Exception during backup: {0}", e);
        throw new RuntimeException(e);
    }
}
```

2. Import a vSphere Native Key Provider configuration by specifying the configuration you want to import in the request body by calling the `importProvider(ProvidersTypes.ImportSpec spec)` method of the `Providers` interface.

```
POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/kms/providers?action=import
```

The following request body contains example syntax.

Note:
You must provide a Base64-encoded string that is a representation of the PKCS12 data exported in the previous step as the value of the `config` parameter. The other parameters are optional.

```
{
    "config": <string>,
    "constraints": {
        "tpm_required": false
    },
    "dry_run": false,
    "password": <your_password>
}
```

```
private void restoreKeyProvider(byte[] backup, char[] pwd) {
    ImportSpec spec = new ImportSpec.Builder().setConfig(backup).setPassword(pwd).build();
    ImportResult res = nativeKeyProviders.importProvider(spec);
    log("Restored Native Key Provider {0}", res);
}
```

In the response body, you receive information about the imported provider and the time when that provider was exported.

Retrieve vSphere Native Key Providers on a Host

You can use HTTP requeststhe `Providers` interface from the `com.vmware.vcenter.crypto_manager.hosts.kms` package to list vSphere Native Key Providers on a host and retrieve details about a specific vSphere Native Key Provider.

1. Retrieve a list of vSphere Native Key Providers on a host by calling the `list(java.lang.String host, ProvidersTypes.FilterSpec filterSpec)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/hosts/<host>/kms/providers
   ```

   In the response body, you receive information about the health status, ID, and type of each provider.

2. Retrieve information about a vSphere Native Key Provider on a host by calling the `get(java.lang.String host, java.lang.String provider)` method of the `Providers` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/crypto-manager/hosts/<host>/kms/providers/<provider>
   ```

   In the response body, you receive detailed information about the health status and the type of the provider.

Managing Host Hardware Security

You can use the VMware Cloud FoundationREST APISDK to perform host hardware management operations.

You can retrieve host hardware TPM information and manage host hardware TPM endorsement keys. You can also register SGX hosts.

The procedures in this chapter are based on the REST API. For details, see the *vSphere Automation REST API Reference*.

The procedures in this chapter are based on the Java API. For details, see the *vSphere Automation Java API Reference*.

Retrieve Host Hardware TPM Information

You can use HTTP requeststhe `Tpm` interface from the `com.vmware.vcenter.trusted_infrastructure.hosts.hardware` package to retrieve a list of configured TPM devices on a host and information about each TPM device.

1. List configured TPM devices on a host by calling the `list(java.lang.String host, TpmTypes.FilterSpec filter)` method of the `Tpm` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm
   ```

   You receive the results in the response body. You can use the retrieved information to review the basic information of the available TPM devices.

2. Retrieve detailed information about a specific TPM device, including the manufacturer, model, and firmware version by calling the `get(java.lang.String host, java.lang.String tpm)` method of the `Tpm` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>
   ```

   You receive the results in the response body. You can use the retrieved information to review the detailed information of the TPM device.

Manage Host Hardware TPM Endorsement Keys

You can use HTTP requestsinterfaces from the `com.vmware.vcenter.trusted_infrastructure.hosts.hardware.tpm` package to retrieve a list of configured TPM endorsement keys on a host and information about each endorsement key. You can also retrieve the TPM event log and unseal a secret that is bound to an endorsement key.

You can retrieve the TPM event log for different purposes, such as configuring firmware trust with an attestation service or validating the boot time TPM measurements. You can unseal a secret that is bound to an endorsement key to verify reported measurements. For example, you can verify measurements from the TPM event log. For details about the unsealing operation, see the *REST API Reference* documentation.

1. List configured TPM endorsement keys on a host by calling the `list(java.lang.String host, java.lang.String tpm, EndorsementKeysTypes.FilterSpec filter)` method of the `EndorsementKeys` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>/endorsement-keys
   ```

   You receive the results in the response body. You can use the retrieved information to review the basic information of the available TPM endorsement keys.

2. Retrieve detailed information about a specific TPM endorsement key by calling the `get(java.lang.String host, java.lang.String tpm, java.lang.String key)` method of the `EndorsementKeys` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>/endorsement-keys/<key>
   ```

   You receive the results in the response body. You can use the retrieved information to review the detailed information of the TPM endorsement key.

3. Retrieve the event log associated with a TPM device by calling the `get(java.lang.String host, java.lang.String tpm)` method of the `EventLog` interface.

   ```
   GET https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>/event-log
   ```

   You receive the information in the response body.

4. Unseal a secret that is bound to an endorsement key by calling the `unseal(java.lang.String host, java.lang.String tpm, java.lang.String key, EndorsementKeysTypes.UnsealSpec spec)` method of the `EndorsementKeys` interface.

   ```
   POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/trusted-infrastructure/hosts/<host>/hardware/tpm/
   <tpm>/endorsement-keys/<key>?action=unseal
   ```

   You receive a string that contains the unsealed secret.

Register an SGX Host

You can use HTTP requeststhe `Hosts` interface from the `com.vmware.vcenter.confidential_computing.sgx` package to register hosts by stored Intel Software Guard Extensions (SGX) configuration.

Register a host by stored SGX configuration by calling the `register_Task(HostsTypes.RegisterSpec spec)` method of the `Hosts` interface.

```
POST https://<vcenter_ip_address_or_fqdn>/api/vcenter/confidential-computing/sgx/hosts?action=register&vmw-
task=true
```

You receive the task ID in the response body. You can use the task ID to check the status of the task by running the following HTTP request.

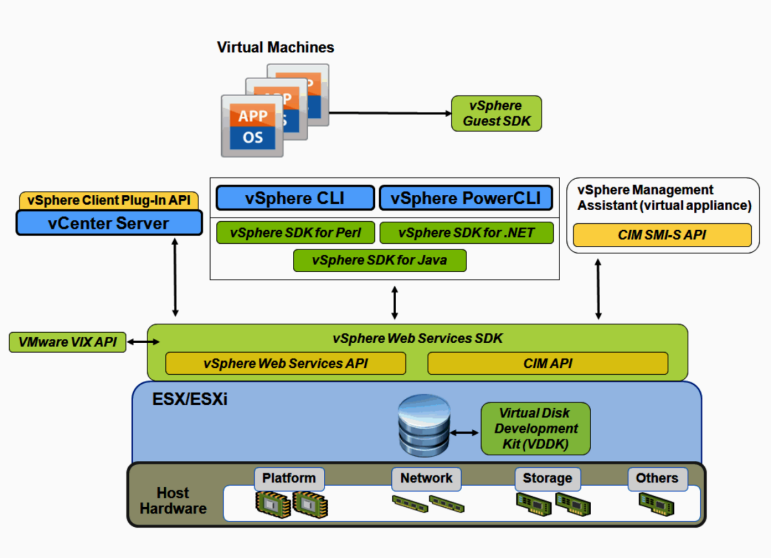
```
GET https://<vcenter_ip_address_or_fqdn>/api/cis/tasks/<task_ID>
```

The vSphere Web Services API

VMware vSphere supports robust, fault-tolerant virtualized applications, networking, and storage. vSphere offers many optional components such as High Availability and vMotion. The vSphere Web Services SDK gives developers programmatic access to most vSphere components.

The Web Services SDK is language neutral. It includes stubs and examples for Java and a comprehensive documentation set including an *API Reference* generated from the source.

Figure 571: vSphere APIs and CLIs

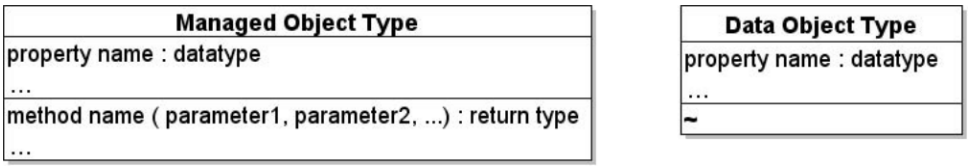


UML Diagrams Used in the Web Services SDK Programming Guide

This guide uses UML (unified modeling language) diagrams to illustrate the API objects and their relationships.

The guide includes class diagrams and instance diagrams. Figure 574: Legend for UML Class Diagrams shows the UML notation used for managed objects and data objects. The diagrams use a tilde (~) if an object has no properties or methods. Ellipses (...) means some properties or methods are omitted.

Figure 572: Legend for UML Class Diagrams

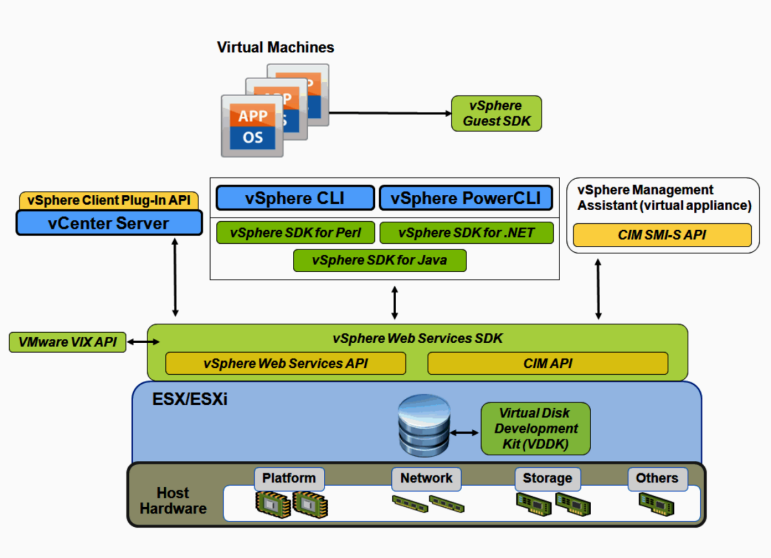


vSphere Web Services SDK

The vSphere Web Services SDK is the most comprehensive of the available management APIs. The SDK works with both ESX and vCenter systems.

The Web Services SDK is language neutral. It includes stubs and examples for Java and a comprehensive documentation set including an *API Reference* generated from the source.

Figure 573: vSphere APIs and CLIs



SDK Developer Setup for the Web Services SDK

Before you can start developing applications with the vSphere Web Services SDK, you must download the software and set up your system. The *vSphere Web Services SDK Developers Setup Guide* has complete instructions for Java client development and discusses a simplified security configuration for development environments.

SDK Samples for the Web Services SDK

The SDK includes a set of Java samples that illustrate many of the SDK features.

The samples include a set of utility applications that are used by the sample code. Example code fragments in this guide are in part based on the Java sample applications, but show code that does not require the utility applications to run.

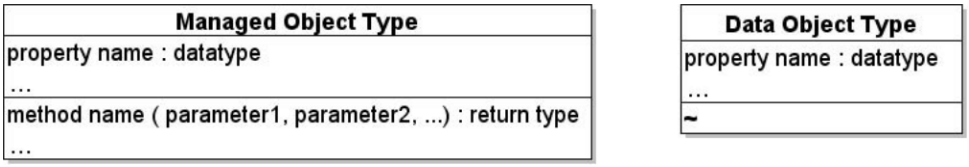
See Sample Program Overview for lists of samples and a brief explanation of what each sample does.

UML Diagrams Used in the Web Services SDK Programming Guide

This guide uses UML (unified modeling language) diagrams to illustrate the API objects and their relationships.

The guide includes class diagrams and instance diagrams. Figure 574: Legend for UML Class Diagrams shows the UML notation used for managed objects and data objects. The diagrams use a tilde (~) if an object has no properties or methods. Ellipses (...) means some properties or methods are omitted.

Figure 574: Legend for UML Class Diagrams



vSphere Web Services Programming Model

The vSphere Web Services API is implemented as a language-neutral Web service. The API is based on a remote procedure call mechanism that client applications use to access services and components on ESX and vCenter systems.

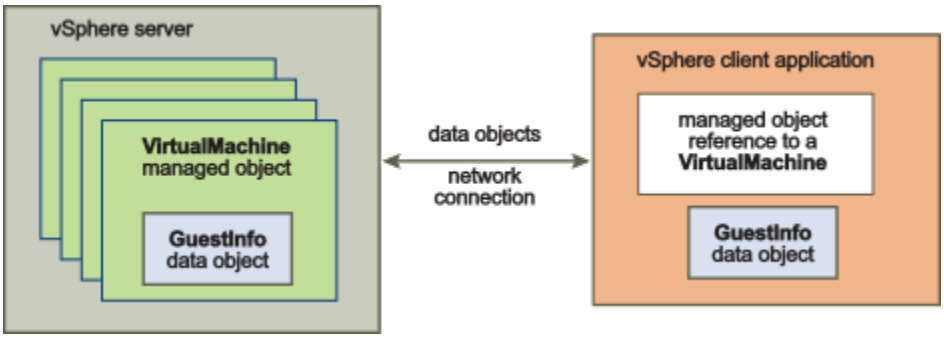
vSphere Client-Server Architecture

VMware vSphere client applications participate in a distributed architecture that uses an asynchronous communications model. This architecture is based on server-side managed objects, client-side managed object references, and data objects.

- Managed objects exist on a vSphere server (ESX or vCenter systems). They represent vSphere services and components. Services include `PropertyCollector`, `SearchIndex`, `PerformanceManager`, and `ViewManager`. Components include inventory objects such as `VirtualMachine`, `Datastore`, and `Folder`.
- Managed object references are client application references to server-side managed objects. Your client application uses `ManagedObjectReference` objects when it invokes operations on a server. A `ManagedObjectReference` is guaranteed to be unique and persistent during an object's lifetime. The reference persists after an object has moved within the inventory, across sessions, and across server restarts. If you remove an object, for example, a virtual machine, from the inventory, and then put it back, the reference changes.
- Data objects contain information about managed objects. Your client application sends data objects to and receives data objects from a vSphere server. Examples are the different specification and capability objects such as `VirtualMachineConfigSpec` and `HostCapability`.

Figure 575: vSphere Server and Client shows a vSphere server and client application. The client has a managed object reference to a virtual machine on the server, and a copy of the `GuestInfo` data object for the virtual machine. A client must maintain its copy of a data object because, depending on the type of client request, a vSphere server might send property data for a data object as a set of name-value pairs associated with a managed object reference. See the description of the `ObjectContent` data object in the *vSphere API Reference*.

Figure 575: vSphere Server and Client



The VMware vSphere application model uses an asynchronous client-server communication model in most cases. Methods are nonblocking and return a reference to a `Task` managed object. For more information about this managed object, see Tasks and Scheduled Tasks.

Web Services API as a Web Service

The vSphere Web Services API is a language-neutral Web service that runs on ESX hosts and vCenter systems.

The API complies with the Web Services Interoperability Organization (WS-I) Basic Profile 1.0. The WS-I Basic Profile 1.0 includes support for the following standards:

- XML Schema 1.0
- SOAP 1.1
- WSDL 1.1

For information about the WS-I Basic Profile 1.0, go to the Web Services Interoperability Organization (WS-I) Web site at http://ws-i.org.

Web services employ **operations**, which are the same as methods in other programming languages. The vSphere API Web Service provides access to most operations necessary for monitoring and managing vSphere components, such as compute resources, virtual machines, networks, storage, and so on.

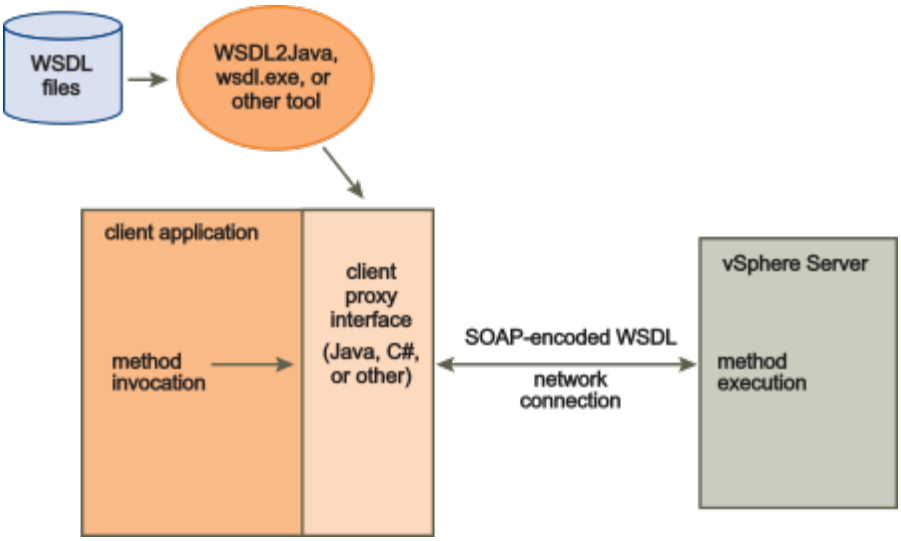
WSDL Files and the Client-Side Proxy Interface

The vSphere Web Services SDK provides a set of WSDL (Web Services Description Language) files that define the vSphere Web Services API. Web-services development tools such as JAX-WS `wsimport` use these WSDL files to generate client-side proxy code (stubs).

The client proxy provides a language-specific interface proxy. The proxy facilitates remote method invocation, organization of object data, and other aspects of distributed, object-oriented, applications programming. Your client application calls proxy interface methods. The client proxy uses SOAP (Simple Object Access Protocol) or JSON (JavaScript Object Notation) to exchange WSDL messages with a vSphere server.

Figure 576: Client-Server Communication Through a Client Proxy Interface is a representation of a client application that uses the client proxy interface to call a method. The client proxy interface is based on the WSDL definitions.

Figure 576: Client-Server Communication Through a Client Proxy Interface



To use the VMware client proxy interface, you must import the vSphere Web Services API client libraries in to your client application using the following Java statement.

```
import com.vmware.vim25.*;
```

> **Important:**
> The vSphere Web Services SDK includes Java client-side proxy code (stubs) generated using the JAX-WS toolkit. If the versions of Java and JAX-WS on your development platform are the same as those used to generate the stubs shipped in the SDK, you do not have to generate stubs from the WSDL yourself. See the *Developer's Setup Guide* for information about how to configure a vSphere Web Services development environment.

Network Access to the vSphere Web Service

Your client application can use the vSphere Web Services API to communicate with vSphere servers over HTTPS (HTTP over an encrypted Secure Sockets Layer connection) at port 443. HTTPS is the default protocol.

You can configure the server to support HTTP. Use HTTP access only for test or development environments, not for production. For information about how to configure the server to support HTTP access, see the *vSphere Web Services SDK Developer's Setup Guide*.

Language-Specific Classes and Methods

The SOAP tools generate language-specific classes and methods that match the WSDL definitions. The tools also produce objects and methods that are not in the WSDL files.

- Generated objects. The additional objects provide access to the vSphere Web Service to establish the client-server connection (`VimService`, `AppUtil`) and declare the methods defined for the vSphere API (`VimPortType`, `VimService`).
- Generated methods. The additional methods are accessor (getter) and mutator (setter) methods for properties. For Java, the method names are constructed by adding `get` and `set` prefixes to a property name, and changing the first character of the property name to upper case.

The following table identifies client proxy definitions for the vSphere Web Services SDK WSDL.

Element Access	Java
Access to vSphere Web service (HTTPS/HTTP)	`VimService` class
Access to vSphere API methods	`VimPortType` class
Access to vSphere API properties	`get` *`PropertyName`* and `set`*`PropertyName`* methods defined for data objects
vSphere API data objects	Data objects in the vSphere API (see the *vSphere API Reference*) defined as objects in the proxy interface

The following code fragments show getter and setter method definitions for the `AfterStartupTaskScheduler.minute` property.

Java

```
public int getMinute() {
     return minute; }
public void setMinute(int minute) {
     this.minute = minute; }
```

You can extrapolate the getter and setter methods that are available in the client proxy interface from the *vSphere API Reference*. For example, the `ScsiLun` data object has a `displayName` property. For the Java API, you can use a `setDisplayName` method to assign a string value to the property, and obtain the string value by using the `getDisplayName` method. The vSphere Web Services SDK includes Java sample code that illustrates how to use the proxy interfaces. See Client Applications for the Web Services API.

Mapping XML Data Types to Java or JSON Data Types

In this guide, the UML class and object diagrams use the primitive data type names such as string and integer, without the XML Schema definition namespace prefix (`xsd:`). The *vSphere API Reference* contains the complete data type name, such as `xsd:string`. The data types map to the primitive data types of the programming language used for the client application.

The following table lists some of the more common XML primitive data type mappings.

XML Schema	Java	Open API
`xsd:base64binary`	`byte[]`	`string` `(format: byte)`
`xsd:boolean`	`boolean`	`boolean`
`xsd:byte`	`byte`	`integer` `(min: -128, max: 127)`
`xsd:dateTime`	`java.util.Calendar`	`string` `(format: date-time)`
`xsd:decimal`	`java.math.BigDecimal`	
`xsd:double`	`double`	`number` `(format: double)`
`xsd:float`	`float`	`number` `(format: float)`
`xsd:int`	`int`	`integer` `(format: int32)`
`xsd:string`	`java.lang.String`	`string`

Access to Managed Objects

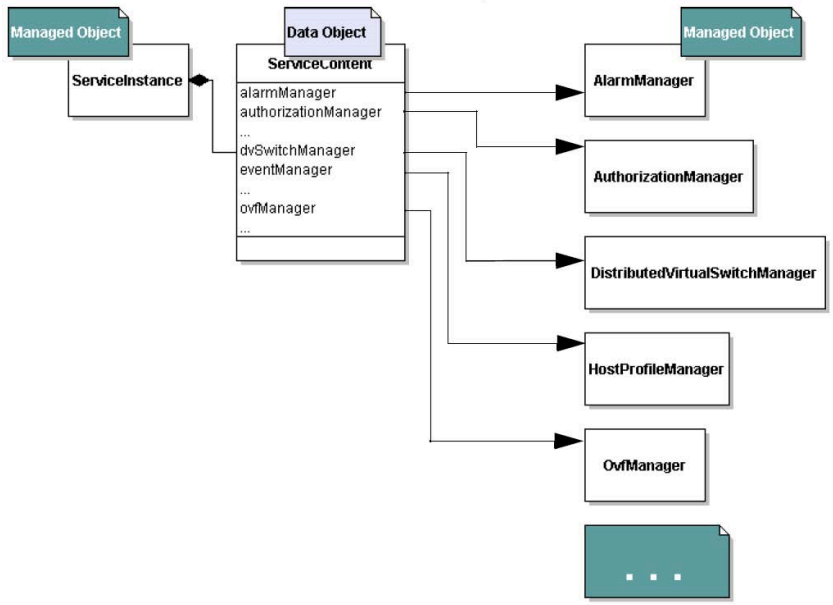
Your client application obtains access to managed objects through the `ServiceInstance` managed object and its associated `ServiceContent` data object. The `ServiceContent` data object contains managed object references to services and manager entities, and to the root folder of the inventory.

The `ServiceInstance` managed object is the root object of the inventory on both ESX and vCenter systems. The server creates the `ServiceInstance`, and creates the manager entities that provide services in the virtual environment. Examples of manager entities are `LicenseManager`, `PerformanceManager`, and `ViewManager`.

The `ServiceInstance` is the primary point of access to the server inventory. Your client application starts by connecting to a server and creating a reference to the `ServiceInstance`. After you have connected to the server, you can call the `ServiceInstance.RetrieveServiceContent` method to a `ServiceContent` data object. `ServiceContent` provides access to the vSphere managed object services. See Overview of a Java Sample Application for the Web Services SDK for an example of connecting to a server and using the `ServiceInstance` reference to retrieve the `ServiceContent` object.

Figure 577: ManagedObjectReference Data Object shows the object model for the `ServiceInstance` and `ServiceContent` objects. The figure shows some of the `ServiceContent` managed object references and the target objects of the references. Each managed object reference identifies a specific managed object on the server with its type and a value. (The `value` property is an opaque string.)

Figure 577: ManagedObjectReference Data Object



Access to vSphere Server Data

To obtain information about the virtual infrastructure, you retrieve managed object properties. Managed object properties can be simple data types, such as integer or string data, or they can be complex types such as data objects that contain sets of properties.

Obtaining Information from a Server

With a reference to a managed object, you can obtain information about the state of the server-side inventory objects and populate client-side data objects based on the values. You can use one of the following approaches:

- Use an accessor (getter) method. The client proxy interface provides accessor methods for each data object property. You can use these accessor methods to obtain the values of the object. See Language-Specific Classes and Methods for information about client proxy interface accessor methods.

- Use a `PropertyCollector` to navigate to a selected point on the server and obtain values from specific properties. See Property Collector for more information about `PropertyCollector`.
- Use the `SearchIndex` managed object to obtain a managed object reference to the managed entity of interest. The `SearchIndex` can return managed object references to specific managed entities—`ComputeResource`, `Datacenter`, `Folder`, `HostSystem`, `ResourcePool`, `VirtualMachine`—given an inventory path, IP address, or DNS name.

  Important:
  You can use API methods to operate on managed objects in the vSphere inventory. A method that updates properties in one managed object may also update properties in other managed objects. The Server performs asynchronous updates to the inventory. There is no guarantee that the inventory will be completely updated when the method returns to the caller. Use the `PropertyCollector` method `WaitForUpdatesEx` to obtain property changes.

Working with Data Structures
Properties contain information about the server-side objects at a given point in time. The value of a property can be of one of the following types:

- Simple data types, such as a string, boolean, or integer (or other numeric) data type. For example, the `ManagedEntity` managed object has a `name` property that takes a string value.
- Arrays of simple data types or data objects. For example, a `HostSystem` managed object contains an array of managed object references (a type of data object) to virtual machines hosted by that physical machine. As another example, the `SessionManager` managed object has a `sessionList` property that is an array of `UserSession` data objects.
- Enumerated types (enumeration, enum) of predefined values. The values can be a collection of simple data types or data objects. For example, a virtual machine's power state can be one of three possible string values—`poweredOn`, `poweredOff`, or `suspended`.
  The type of a property is often a string, but the property actually expects one of the values an enumeration encapsulates. For example, when you set `VirtualMachineConfigSpec.guestid` you can specify one of the elements of the `VirtualMachineGuestOSIdentifier` as a string.
- Complex (or composite) data types. For example, the `HostProfileConfigInfo` object contains data objects, an array of data objects, and an array of strings.

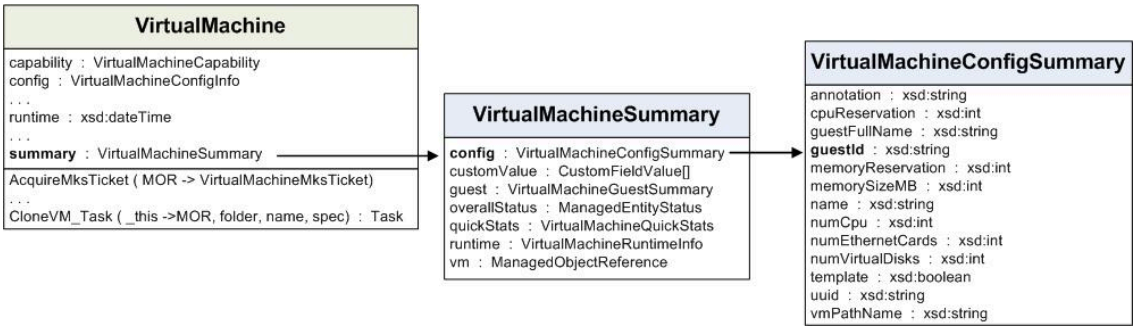
Accessing Property Values
To use the composite data structures and arrays that contain Server data:

- Use dot notation to access nested properties in composite data structures.
- Cast unconstrained property values (`xsd:anyType`) to array types.
- Use keys or index values as appropriate to access array values.

Nested Properties and Property Paths in Composite Data Structures
vSphere Data objects can include properties that are defined as composite data types, such as data objects. The embedded data objects can also contain properties that are data objects. Properties can nest to several levels.

For example, the following figure shows a UML class diagram of the `VirtualMachine` managed object, which has a `runtime` property that is defined as an `xsd:dateTime` data type. `VirtualMachine` also has a `summary` property that is a `VirtualMachineSummary` data object. The `VirtualMachineSummary` data object contains a `config` property that is a `VirtualMachineConfigSummary` data object.

Figure 578: VirtualMachine Managed Object and Nested Properties



To refer to a nested property, use dot notation to separate the object names in the sequence that defines the path to the property. Your code must handle the type referenced at the end of the sequence.

For example, you can compare the property referenced by the path `summary.config.guestId` (a string value) to the property referenced in the path `summary.config` (the complete `VirtualMachineSummary` data object).

The following table shows examples of property references and the corresponding data types for some of the properties of the `VirtualMachine` managed object shown in Figure 578: VirtualMachine Managed Object and Nested Properties.

Reference	Data Type
`summary`	`VirtualMachineSummary` data object
`summary.config`	`VirtualMachineConfigSummary` data object
`summary.config.guestID`	string

xsd:anyType Arrays

The vSphere Web Services API uses `xsd:anyType` unconstrained type declarations. A vSphere client must map values of `xsd:anyType` to explicit data types. An `xsd:anyType` value can represent a single data value or it can represent an array. The WSDL for the vSphere Web Services API defines array types for all of the data values that a vSphere client can send or receive as arrays. The array types use the prefix "`ArrayOf`". An example of an array type is `ArrayOfString` for string values.

When a client sends data to a vSphere Server, the client must use explicit datatypes. For example, a client can define a `MethodAction` for a `ScheduledTask`. The vSphere Web Services API defines arguments to the action (the `MethodActionArgument.value` property) as type `xsd:anyType`. If the action takes an array argument, the client must set the corresponding `MethodAction.argument[].value` to the appropriate ArrayOf... type.

When a client receives `xsd:anyType` data from a vSphere server, it must cast the data to an explicit type. For example, the `PropertyCollector` method `RetrievePropertiesEx` returns a set of `ObjectContent` data objects. The `ObjectContent.propSet` property is a list of `DynamicProperty` objects that contains the requested property values. Each `DynamicProperty` object contains a name-value pair. The value property (`DynamicProperty.val`) is of type `xsd:anyType`. It can represent a single object or an array of objects.

When the returned value is a single object such as an `Event`, `ManagedObjectReference`, or `String`, you can cast it directly to a variable of the appropriate type. However, when the value is an array of objects you cannot cast the `anyType` value directly to an array variable.

When the `PropertyCollector` returns array data, it sends it as an `xsd:anyType` value. The language-specific bindings contain definitions for array objects such as `ArrayOfEvent`, `ArrayOfManagedObjectReference`, and `ArrayOfString`, and corresponding "get" methods. To extract the actual array from a property of type `xsd:anyType`, cast `DynamicProperty.val` to the appropriate array type and use the matching get method – for example, `getEvent()`, `getManagedObjectReference()`, or `getString()`.

The following sections provide some examples of how to cast returned values for a few of the array types.

Cast an xsd:anyType Array to an Event Array

When the `PropertyCollector` returns array data representing an array of `Event` objects, the array has the type `xsd:anyType`. To use the data in your application, you must first cast it to a specialized type for `Event` arrays.

This example code uses the JAX-WS-generated Java bindings for the VMware vSphere Web Services SDK WSDL.

1. Use the `DynamicProperty.getVal()` method to retrieve the anyType property value.
2. Cast the `anyType` value to a value of type ArrayOfEvent.
3. Use the corresponding get method to assign the result of the cast operation to a list variable.

```
/*
* Handling arrays of Event objects.
* Cast the return value to ArrayOfEvent and use getEvent().
*/
List[] eventList = ((ArrayOfEvent) dynamicProp.getVal()).getEvent();
```

Cast an xsd:anyType Array to an Array of Managed Object References

When the `PropertyCollector` returns array data representing an array of managed objects, the array has the type `xsd:anyType`. To use the data in your application, you must first cast it to a specialized type for arrays of managed object references.

This example code uses the JAX-WS-generated Java bindings for the VMware vSphere Web Services SDK WSDL.

1. Use the `DynamicProperty.getVal()` method to retrieve the `anyType` property value.
2. Cast the `anyType` value to a value of type `ArrayOfManagedObjectReference`.
3. Use the corresponding get method to assign the result of the cast operation to a list variable.

```
/*
* Handling arrays of ManagedObjectReference objects.
* Cast the return value to ArrayOfManagedObjectReference and use getManagedObjectReference().
*/
List[] morList =
     ((ArrayOfManagedObjectReference)dynamicProp.getVal()).getManagedObjectReference();
```

Cast an xsd:anyType Array to an Array of String

When the `PropertyCollector` returns array data representing an array of strings, the array has the type `xsd:anyType`. To use the data in your application, you must first cast it to a specialized type for arrays of strings.

This example code uses the JAX-WS-generated Java bindings for the VMware vSphere Web Services SDK WSDL.

1. Use the `DynamicProperty.getVal()` method to retrieve the anyType property value.
2. Cast the `anyType` value to a value of type `ArrayOfString`.
3. Use the corresponding get method to assign the result of the cast operation to a list variable.

```
/*
* Handling arrays of strings.
* Cast the return value to ArrayOfString and use getString().
*/
List[] stringList = ((ArrayOfString) dynamicProp.getVal()).getString();
```

Indexed Array and Key-Based Array Properties

The VMware vSphere data structures include array properties, which can be indexed arrays or key-based arrays.

- **Indexed arrays** are accessed by using an index integer. Indexed arrays are used for arrays of data types whose positions in the array do not change. For example, the `roleList` property of the `AuthorizationManager` managed object is an array of authorization roles. Adding a new role to the array does not change the position of existing elements in the array.
- **Key-based arrays** are used for information whose position is subject to change. A key-based array (same basic concept as a Perl hash or a Python dictionary) uses a unique, unchanging value as a key to access an element's value.

  Typically, the key is a string, but integers can also be used. For example, `Event` arrays use integers as keys.

The vSphere management object model uses key-based arrays to track managed object references. The contents of a key-based array property are accessed by the value of either the key property or, in the case of a managed object reference, its value property. The value of these fields is unique across all of the components of an array.

Nested properties can also refer to entries in a key-based array. For example, `a.b.c["xyz"]` refers to the property `c` that has the key value of `xyz`.

Unset Optional Properties

Many of the Data Objects in the vSphere Web Services SDK have optional properties that may be set by your client application or by a Server process or event. If you retrieve a data object that has an optional property that is unset, the Server will not return a value for the optional property. If you call an accessor function to retrieve the property value, the value returned by the function depends on the programming language that you are using.

For example, if you are programming in Java, the value you will receive for an unset property is `null`.

Figure 579: Data Object - HostFirewallInfo Properties shows part of the Properties table for the `HostFirewallInfo` data object in the *vSphere Web Services SDK API Reference*. When you look at properties in the *vSphere Web Services SDK API Reference*, you can see that optional properties are marked with a red asterisk.

In this example, that the `defaultPolicy` property is always returned, but the `ruleset` property will be returned as a `null` value if it has not been set.

Figure 579: Data Object - HostFirewallInfo Properties

Properties

NAME	TYPE	DESCRIPTION
defaultPolicy	HostFirewallDefaultPolicy	Default firewall policy.
ruleset*	HostFirewallRuleset[]	List of configured rulesets.
Properties inherited from DynamicData		
dynamicProperty, dynamicType		

*Need not be set

Since Data Objects are part of many different constructs, there is no standard scenario for when an optional property should be set, what will happen if an optional property is left unset, or what you should do if a null value is returned.

Escape Character in Name and Path Properties

The percent sign (%) is used as an escape character to embed special characters in strings. For example, %2f (or %2F) is interpreted as the slash (/) character. To include a percent sign as a literal in a string, use %%.The path to the inventory starts from the root folder (`ServiceContent.rootFolder` property), denoted by the slash character.

Character	Description	Representation in URL
%	Percent sign	%25
/	Slash	%2F, %2f
\	Backslash	%5C, %5c
-	Dash	%2D, %2d
.	Dot	%2E, %2e
“	Double quotation mark	%2B, %2b

Client Applications for the Web Services API

Any client application written for the Web Services API must perform certain basic functions. These include making a connection to the server, authenticating and creating a session, and closing the connection.

vCenter Connections

Every vCenter client application must connect to the server and pass user account credentials to authenticate to the server. After the connection has been established, the client application can use vSphere services to access the virtual environment.

vSphere uses SSL certificates, HTTP tokens, and vCenter Single Sign-On tokens to authenticate a client and support a persistent connection between the client and vCenter. The following table provides an overview of these elements.

Security Element	Description
SSL certificates	vSphere Servers use standard X.509 version 3 (X.509v3) certificates to encrypt session information sent over Secure Socket Layer (SSL) protocol connections. In a production environment, client applications verify the vSphere Server certificate during the connection sequence. The examples in this chapter and the examples in the vSphere Web Services SDK accept all certificates.
HTTP tokens	A vSphere Server uses an HTTP token to identify a client session. The Server provides the HTTP token in its response to a client connection request. Subsequent messages between the client and the Server include the HTTP token in the HTTP header.
Client authentication vCenter Single Sign On token	vSphere supports vCenter Single Sign-On. A vCenter client can obtain a vCenter Single Sign-On token from a vCenter Single Sign-On Server and use that token to login to a vCenter.
Client authentication username/password	Username/password authentication for client-server connections. A client can present user credentials either directly to vCenter to establish a session, or to the vCenter Single Sign-On Service in exchange for a SAML token.

Establishing a Single Sign-On Session with a vCenter

vSphere uses single sign-on to provide a single point of authentication for clients. vSphere includes the vCenter Single Sign-On Server. To use vCenter Single Sign-On, your client obtains a SAML token (Security Assertion Markup Language) from the vCenter Single Sign-On Server and passes the token to the vCenter in the login request. The token represents the client and contains claims that support client authentication. Components in the vSphere environment perform operations based on the original authentication. For information about obtaining a vCenter Single Sign-On token from the vCenter Single Sign-On Server, see *vCenter Single Sign On Programming Guide*.

To use single sign on, your client calls the `LoginByToken` method. Your client must send a SAML token to the vCenter by embedding the token in the SOAP header for the `LoginByToken` request. During the login sequence, your client must save and restore the HTTP session cookie. The vCenter Single Sign-On SDK contains sample code that demonstrates how to use the `LoginByToken` method.

The following sections describe examples of using the `LoginByToken` method to establish a vCenter Single Sign On session with a vCenter instance.

LoginByToken to vCenter By Using Java

The following example is based on the `LoginByTokenSample.java` file contained in the vCenter Single Sign On SDK. The SDK contains Java code that supports HTTP and SOAP header manipulation.

vCenter Single Sign-On Session Using Java

After you obtain a SAML token from the vCenter Single Sign-On Server, you can use the vSphere Web Services API method `LoginByToken` to establish a vCenter Single Sign-On session with a vCenter instance. At the beginning of the session, your client is responsible for the following tasks:

- Insert the vCenter Single Sign-On token and a timestamp into the SOAP header of the `LoginByToken` message.

- Maintain the vCenter session cookie. During the login sequence, the Server produces an HTTP session cookie to support the persistent connection. Your client must save this cookie and re-introduce it at the appropriate times.
- If at a later time your client invokes the `LoginByToken` method, or other login method, the Server issues a new session cookie in response. You must have a cookie handler in place to save the cookie for subsequent requests.

The example program uses these general steps:

1. Call the `RetrieveServiceContent` method to establish an HTTP connection with the vCenter and get the Session Manager managed object reference.
2. Call the `LoginByToken` method to authenticate the vCenter session. To send the token to the vCenter, the client uses a handler to embed the token and a time stamp in the SOAP header for the message. The client uses an HTTP header handler method to extract the session cookie from the vCenter response.
3. Restore the session cookie for future requests. To identify the session started with the `LoginByToken` method, the client uses a handler to embed the session cookie in the HTTP header.

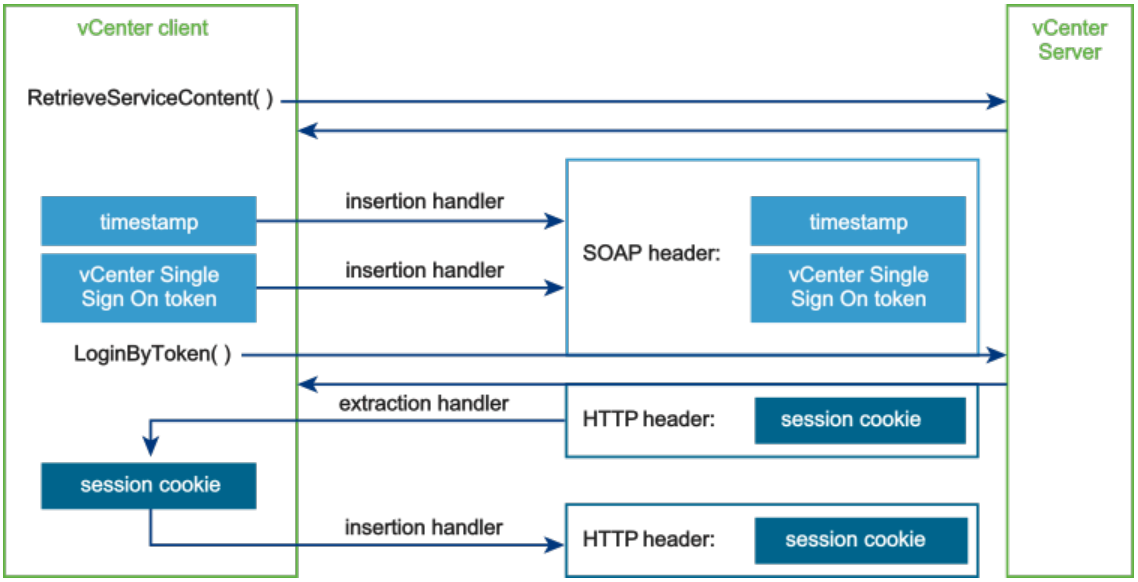
HTTP and SOAP Header Handlers in Java

To use a vCenter Single Sign On token to login to a vCenter, the example uses header handlers to manipulates the HTTP and SOAP header elements of the login request. After establishing a handler, subsequent requests automatically invoke the handler.

- Insertion handlers put the vCenter Single Sign On token and a timestamp into the SOAP header into the HTTP header of the login request.
- An extraction handler obtains the HTTP session cookie provided by the vCenter instance. After setting up the handler, a call to the `LoginByToken` method will invoke the handler to extract the cookie from the Server response.

The following figure shows the use of handlers to manipulate header elements when establishing a vCenter Single Sign On session with a vCenter instance.

Figure 580: Starting a vCenter Session



Important:

Every call to the vCenter will invoke any message handlers that have been established. The overhead involved in using the SOAP and HTTP message handlers is not necessary after the session has been established. The example saves the default message handler before setting up the SOAP and HTTP handlers. After establishing the session, the example will reset the handler chain and restore the default handler.

The example code also uses multiple calls to the `VimPortType.getVimPort method` to manage the request context. The `getVimPort` method clears the HTTP request context. After each call to the `getVimPort` method, the client resets the request context endpoint address to the vCenter URL. After the client has obtained the session cookie, it will restore the cookie in subsequent requests.

Sample Code for a Java Client to the Web Services SDK

The code examples in the following sections show how to use the `LoginByToken` method with a holder-of-key security token. The code examples are based on the sample code contained in the vCenter Single Sign On SDK. The files are located in the Java samples directory (`SDK/ssoclient/java/JAXWS/samples`):

- LoginByToken sample:
  `samples/com/vmware/vsphere/samples/LoginByTokenSample.java`
- Header cookie handlers:
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieHandler.java`
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieExtractionHandler.java`
- SOAP header handlers. These are the same handlers that are used in the vCenter Single Sign-On example in *vCenter Single Sign On Programming Guide*. The SOAP handler files are contained in the vCenter Single Sign-On SDK and are located in the SSO client `soaphandlers` directory:
  `SDK/ssoclient/java/JAXWS/samples/com/vmware/sso/client/soaphandlers`

Creating the HTTP Connection in Java

The code fragment in this section establishes an HTTP session with the vCenterand saves the HTTP session cookie.

The following sequence describes these steps and shows the corresponding objects and methods.

1	Use the `getHandlerResolver` method to save the default message handler. To use the HTTP and SOAP message handlers, you must first save the default message handler so that you can restore it after login. The HTTP and SOAP message handlers impose overhead that is unnecessary after login.	`VimService.getHandlerResolver( )`
2	Get the VIM port. The VIM port provides access to the vSphere API methods, including the `LoginByToken` method.	
3	Set the request context endpoint address to the vCenter URL.	
4	Retrieve the ServiceContent. This method establishes the HTTP connection.	

The following example shows Java code that saves the session cookie.

Saving the vCenter Session Cookie

```
/*
 * The example uses a SAML token (obtained from a vCenter SSO) and vCenter URL.
 * The following declarations indicate the datatypes; the token datatype (Element)
 * corresponds to the token datatype returned by the vCenter Single Sign On Server.
 *
 * Element token;       -- from vCenter Single Sign On Server
 * String vcServerUrl;  -- identifies vCenter
 *
 * First, save the default message handler.
 */

HandlerResolver defaultHandler = vimService.getHandlerResolver();
```

```
/*
 * Create a VIM service object.
 */
vimService = new VimService();

/*
 * Construct a managed object reference for the ServiceInstance.
 */
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");

/*
 * Get the VIM port for access to vSphere API methods. This call clears the request context.
 */
vimPort = vimService.getVimPort();

/*
 * Get the request context and set the connection endpoint.
 */
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Retrieve the ServiceContent. This call establishes the HTTP connection.
 */
serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
```

Using LoginByToken in Java

The code fragment in this section sets up the message handlers and calls the `LoginByToken` method to get the session cookie. The following sequence describes the steps and shows the corresponding objects and methods.

1	Create a new `HeaderHandlerResolver`. Then set the message security handlers for cookie extraction and for inserting the SAML token and credentials in the SOAP header.	
2	Get the VIM port.	
3	Set the connection endpoint in the HTTP request context.	

4	Call the `LoginByToken()` method. The method invocation activates the handlers to insert the elements into the message headers. The method returns a session cookie that identifies the newly created session.	`VimPortType.LoginByToken()`
5	Extract the cookie and save it for later use.	`HeaderCookieExtractionHandler.getCookie()`

The following examples shows Java code that calls the `LoginByToken()` method.

Using LoginByToken

```
/*
 * Create a handler resolver and add the handlers.
 * Create a cookie extraction handler and add it to the handler resolver.
 */
HeaderHandlerResolver handlerResolver = new HeaderHandlerResolver();
HeaderCookieExtractionHandler cookieExtractor = new HeaderCookieExtractionHandler();
handlerResolver.addHandler(cookieExtractor);
handlerResolver.addHandler(new TimeStampHandler());
handlerResolver.addHandler(new SamlTokenHandler(token));
handlerResolver.addHandler(new WsSecuritySignatureAssertionHandler(
                                userCert.getPrivateKey(),
                                userCert.getUserCert(),
                                Utils.getNodeProperty(token, "ID")));
vimService.setHandlerResolver(handlerResolver);

/*
 * Create a handler resolver.
 * Set the VIM service handler resolver.
 */
vimService.setHandlerResolver(handlerResolver);
/*
 *  Get the Vim port; this call clears the request context.
 */
vimPort = vimService.getVimPort();

/*
 * Retrieve the request context and set the server URL.
 */
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Call LoginByToken.
 */
UserSession us = vimPort.loginByToken(serviceContent.getSessionManager(), null);

/*
 * Save the HTTP cookie.
 */
 String cookie = cookieExtractor.getCookie();
```

Restoring the vCenter Session Cookie in a Java Client

After you log in, you must restore the standard vCenter session context. The code fragment in this section restores the default message handler and the session cookie. As the cookie handler has been replaced by the default handler, the

client resets the session cookie by calling request context methods to access the context fields directly. The following sequence describes these steps and shows the corresponding objects and methods.

1	Restore the default message handler. The handlers used for `LoginByToken()` are not used in subsequent calls to the vSphere API.	`VimService.setHandlerResolver()`
2	Get the VIM port.	
3	Set the connection endpoint in the HTTP request context.	
4	Set the HTTP request header (vCenter session cookie).	`RequestContext.get()` `RequestContext.put()`

The following example shows Java code that restores the vCenter session. This code requires the vCenter URL and the cookie and default handler that were retrieved before login. See Sample Code for a Java Client to the Web Services SDK.

Restoring the vCenter Session

```
/*
 * Reset the default handler. This overwrites the existing handlers, effectively removing them.
 */
vimService.setHandlerResolver(defaultHandler);
vimPort = vimService.getVimPort();

/*
 * Restore the connection endpoint in the request context.
 */
// Set the validated session cookie and set it in the header for once,
// JAXWS will maintain that cookie for all the subsequent requests

Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Reset the cookie in the request context.
 */
Map<String, List<String>> headers =
  (Map<String, List<String>>) ctxt.get(MessageContext.HTTP_REQUEST_HEADERS);
if (headers == null) {
   headers = new HashMap<String, List<String>>();
}
headers.put("Cookie", Arrays.asList(cookie));
ctxt.put(MessageContext.HTTP_REQUEST_HEADERS, headers);
```

Establishing a Session with Username and Password Credentials

Although not as secure as `LoginByToken`, you can specify username and password credentials to establish a session with vCenter. Follow these steps:

1. Connect to the vSphere Web service, for example with SSL.
2. Create a managed object referring to `ServiceInstance`. Use this object to retrieve object `ServiceContent` from the server. `ServiceContent` contains a reference to the root folder for the inventory and references to managed objects that provide vSphere services. Create another object for access to vSphere API methods.
3. Log in to the server using appropriate credentials (user account, password, and optionally locale).
4. Access server-side objects to retrieve data and perform management operations.
5. Close the connection.

   For sample Java code, see topics in *Getting Started with vSphere APIs*.

Overview of a Java Sample Application for the Web Services SDK

This section includes an example of a complete client application that demonstrates the basic client capability. The sample client application prints out the product name, server type, and product version to demonstrate that it is connected and able to retrieve information from the server.

While Java Test Client Application is a complete client application that demonstrates the basic client capability, it uses a slightly different format than the Java sample files in the `SDK\` directory. This example, and the Java samples that are included with your vSphere Web Service SDK, have been compiled using JAX-WS bindings.

Most of the vSphere Web Services SDK samples do not handle exceptions, and they accept all security certificates. Use the samples as examples for extracting the types of data you want to view, but do not use these security or exception techniques in your production applications.

Build a Simple vSphere Client Application for the Web Services SDK

This simple client application accepts command-line arguments for the vSphere server name (DNS name or IP address), user name, and password.

To build a simple vSphere client application in Java, use the following steps.

1. Import the vSphere Web Services API libraries:

   ```
   import com.vmware.vim25.*;
   ```

2. Import the necessary Java (and JAX-WS connection, bindings, and SOAP) libraries:

   ```
   import java.util.*;
   import javax.net.ssl.HostnameVerifier;
   import javax.net.ssl.HttpsURLConnection;
   import javax.net.ssl.SSLSession;
   import javax.xml.ws.BindingProvider;
   import javax.xml.ws.soap.SOAPFaultException;
   ```

3. Create the `TestClient` class:

   ```
   public class TestClient {
   ```

4. Include the class variable declarations/definitions. Use a `TrustManager` class to accept all certificates, as shown in Accessing the vSphere Web Services HTTP Endpoint with JAX-WS . This is not appropriate for a production environment. Production code should implement certificate support.
5. Use the vSphere Web Services APIs to create the connection, as shown in Accessing the vSphere Server from a Web Services Client.
6. Retrieve data from the vSphere or vCenter. In this example, we are just going to print out the product name, server type, and product version to prove that the client is connected and working correctly.

   ```
   System.out.println(serviceContent.getAbout().getFullName());
   System.out.println("Server type is " + serviceContent.getAbout().getApiType());
   System.out.println("API version is " + serviceContent.getAbout().getVersion());
   ```

7. Use the `VimPort` object to close the connection, as shown in Closing the Connection from a Web Services Client. Always close your server connections to maintain security.

Java Client Example for the Web Services SDK

This example shows the complete sample client application code, without the explanatory steps. The example opens a connection with the server, retrieves the service content, uses the session manager managed object reference to log in, displays information about the server, and closes the connection.

Java Test Client Application

```
import com.vmware.vim25.*;
import java.util.*;
import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLSession;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.soap.SOAPFaultException;

public class TestClient {

  // Authentication is handled by using a TrustManager and supplying
  // a host name verifier method. (The host name verifier is declared
  // in the main function.)
  private static class TrustAllTrustManager
     implements javax.net.ssl.TrustManager,
                javax.net.ssl.X509TrustManager {

     public java.security.cert.X509Certificate[] getAcceptedIssuers() {
        return null;
     }

     public boolean isServerTrusted(java.security.cert.X509Certificate[] certs) {
        return true;
     }

     public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
        return true;
     }

     public void checkServerTrusted(java.security.cert.X509Certificate[] certs,
                                    String authType)
        throws java.security.cert.CertificateException {
           return;
     }

     public void checkClientTrusted(java.security.cert.X509Certificate[] certs,
                                    String authType)
        throws java.security.cert.CertificateException {
           return;
     }
  }// end class TrustAllTrustManager

  public static void main(String[] args) {
    try {

      // Server URL and credentials.
      String serverName = args[0];
      String userName   = args[1];
      String password   = args[2];
```

```
String url = "https://"+serverName+"/sdk/vimService";
// Variables of the following types  for access to the API methods
// and to the vSphere inventory.
// -- ManagedObjectReference for the ServiceInstance on the Server
// -- VimService for access to the vSphere Web service
// -- VimPortType for access to methods
// -- ServiceContent for access to managed object services
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
VimService vimService;
VimPortType vimPort;
ServiceContent serviceContent;

// Declare a host name verifier that will automatically enable
// the connection. The host name verifier is invoked during
// the SSL handshake.
HostnameVerifier hv = new HostnameVerifier() {
   public boolean verify(String urlHostName, SSLSession session) {
      return true;
   }
};
// Create the trust manager.
javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
trustAllCerts[0] = tm;

// Create the SSL context
javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");

// Create the session context
javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();

// Initialize the contexts; the session context takes the trust manager.
sslsc.setSessionTimeout(0);
sc.init(null, trustAllCerts, null);

// Use the default socket factory to create the socket for the secure connection
javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
// Set the default host name verifier to enable the connection.
HttpsURLConnection.setDefaultHostnameVerifier(hv);

// Set up the manufactured managed object reference for the ServiceInstance
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");

// Create a VimService object to obtain a VimPort binding provider.
// The BindingProvider provides access to the protocol fields
// in request/response messages. Retrieve the request context
// which will be used for processing message requests.
vimService = new VimService();
vimPort = vimService.getVimPort();
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();

// Store the Server URL in the request context and specify true
// to maintain the connection between the client and server.
// The client API will include the Server's HTTP cookie in its
// requests to maintain the session. If you do not set this to true,
// the Server will start a new session with each request.
```

```
            ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
            ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

            // Retrieve the ServiceContent object and login
            serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
            vimPort.login(serviceContent.getSessionManager(),
                          userName,
                          password,
                          null);

            // print out the product name, server type, and product version
            System.out.println(serviceContent.getAbout().getFullName());
            System.out.println("Server type is " + serviceContent.getAbout().getApiType());
            System.out.println("API version is " + serviceContent.getAbout().getVersion());

            // close the connection
            vimPort.logout(serviceContent.getSessionManager());
          } catch (Exception e) {
             System.out.println(" Connect Failed ");
             e.printStackTrace();
          }
       }//end main()
    }// end class TestClient
```

// See Obtaining a Session Token - Code Fragments from VMPromoteDisks.java for more details.

Compiling the Java Test Client in the Web Services SDK

Use the following command to compile the code for the Java Test Client Application, after you have saved it as a `.java` file:

```
c:>javac -classpath path-to-vim25.jar TestClient.java
```

Use the following command to run the compiled class (binary) file:

```
c:>java -classpath path-to-vim25.jar TestClient web-service-url user-name user-password
```

Web Server Session Token

As with other Web services, the vSphere Web service maintains session state for each client connection by using a token in the HTTP header to identify the session. The vSphere server returns a session token to the client in its response to the client connection request. Subsequent messages between client and server automatically include the token.

Each of the stand-alone samples in the `SDK\vsphere-ws\java\JAX-WS\samples\com\vmware\` uses the JAX-WS TrustAllTrustCertificates class, as discussed in Obtaining a Session Token - Code Fragments from VMPromoteDisks.java to ignore certificates, obtain a session token, and then connect to the server.

> **CAUTION:**
> We do not recommend that you trust all certificates in a production environment. Instead, you can look at the sample code to see how the JAX-WS libraries are used when making the connection, but set up an SSL policy that allows connection only with trusted certificates.

The logic for getting a cookie and putting it in the header looks like this:

```
//cookie logic
List cookies = (List) headers.get("Set-cookie");
cookieValue = (String) cookies.get(0);
StringTokenizer tokenizer = new StringTokenizer(cookieValue, ";");
cookieValue = tokenizer.nextToken();
String path = "$" + tokenizer.nextToken();
String cookie = "$Version=\"1\"; " + cookieValue + "; " + path;
```

```
// set the cookie in the new request header
Map map = new HashMap();
map.put("Cookie", Collections.singletonList(cookie));

((BindingProvider) vimPort).getRequestContext().put(
MessageContext.HTTP_REQUEST_HEADERS, map);
```

Accessing the vSphere Web Services HTTP Endpoint with JAX-WS

The steps for accessing any HTTP endpoint with JAX-WS bindings include the vSphere Web Services SDK Server URL, vSphere server object, and variables.

These steps are listed at the beginning of Obtaining a Session Token - Code Fragments from VMPromoteDisks.java.

1. Create a `TrustManager` class to handle certificate checking.

   In this example we use a `TrustManager` class to accept all certificates. This is not appropriate for a production environment. Production code should implement certificate support.

```
private static class TrustAllTrustManager
  implements javax.net.ssl.TrustManager,
  javax.net.ssl.X509TrustManager {

   public java.security.cert.X509Certificate[] getAcceptedIssuers() {
      return null;
   }

   public boolean isServerTrusted(
      java.security.cert.X509Certificate[] certs) {
      return true;
   }

   public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
      return true;
   }

   public void checkServerTrusted(java.security.cert.X509Certificate[] certs,
                                  String authType)
         throws java.security.cert.CertificateException {
      return;
   }

   public void checkClientTrusted(java.security.cert.X509Certificate[] certs,
                                  String authType)
         throws java.security.cert.CertificateException {
      return;
   }
}
```

2. Include the Server URL and credentials as arguments in the main method:

```
public static void main(String[] args) {
  try {
     String serverName = args[0];
     String userName   = args[1];
     String password   = args[2];
     String url = "https://"+serverName+"/sdk/vimService";
```

3. Declare variables of the following types for access to vSphere server objects:

   - `ManagedObjectReference` for the `ServiceInstance`.

- `VimService` object for access to the Web service.
- `VimPortType` object for access to all of the methods defined in the vSphere API.
- `ServiceContent` for access to the managed object services on the server.
  The following Java code fragment shows these variable declarations:

```
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
VimService vimService;
VimPortType vimPort;
ServiceContent serviceContent;
```

4. Declare a host name verifier that will automatically enable the connection. The host name verifier is invoked during the SSL handshake.

```
HostnameVerifier hv = new HostnameVerifier() {
  public boolean verify(String urlHostName, SSLSession session) {
     return true;
  }
};
```

5. Instantiate the trust manager object.

```
// Create the trust manager.
javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
trustAllCerts[0] = tm;
```

6. Create the SSL context

```
javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");
```

7. Create the session context

```
javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();
```

8. Initialize the contexts; the session context takes the trust manager.

```
sslsc.setSessionTimeout(0);
sc.init(null, trustAllCerts, null);
```

9. Use the default socket factory to create the socket for the secure connection

```
javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
```

10. Set the default host name verifier to enable the connection.

```
HttpsURLConnection.setDefaultHostnameVerifier(hv);
```

Accessing the vSphere Server from a Web Services Client

The steps that use the vSphere Web Services API to create the connection are:

1. Create a managed object reference for the `ServiceInstance` object on the server.

```
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");
```

2. Create a `VimService` object to obtain a `VimPort` binding provider. The `BindingProvider` object provides access to the protocol fields in request/response messages. Retrieve the request context which will be used for processing message requests.

   The `VimServiceLocator` and `VimPortType` objects provide access to vSphere servers. The `getVimPort` method returns a `VimPortType` object that provides access to the vSphere API methods.

```
vimService = new VimService();
vimPort = vimService.getVimPort();
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
```

3. Store the Server URL in the request context and specify `true` to maintain the connection between the client and server. The client API will include the server's HTTP cookie in its requests to maintain the session. If you do not set this to true, the server will start a new session with each request.

```
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
```

4. Retrieve the `ServiceInstance` content (the `ServiceContent` data object) and log in to the server.

```
serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
vimPort.login(serviceContent.getSessionManager(),
              userName,
              password,
              null);
isConnected = true;
```

Closing the Connection from a Web Services Client

Use the `VimPort` object again to close the connection. Always close your server connections to maintain security.

```
         vimPort.logout(serviceContent.getSessionManager());
      } catch (Exception e) {
         System.out.println(" Connect Failed ");
         e.printStackTrace();
      }
   }//end main()
}// end class TestClient
```

Using the Java Samples as Reference

The following code fragment from the vSphere Management SDK shows another implementation of the server connection. You can use code similar to this sample in the vSphere Web Services SDK (path below) to get a session token for your client application.

```
SDK/vsphere-ws/java/JAX-WS/samples/com/vmware/vm/VMPromoteDisks.java
```

Obtaining a Session Token - Code Fragments from VMPromoteDisks.java

```
...
private static String cookieValue = "";
private static Map headers = new HashMap();
...
private static void trustAllHttpsCertificates()
   throws Exception {

   javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
   javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
   trustAllCerts[0] = tm;
   javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");
   javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();
   sslsc.setSessionTimeout(0);
   sc.init(null, trustAllCerts, null);
   javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
}
...
private static void connect()
   throws Exception {

   HostnameVerifier hv = new HostnameVerifier() {
      public boolean verify(String urlHostName, SSLSession session) {
```

```
            return true;
         }
      };
      trustAllHttpsCertificates();
      HttpsURLConnection.setDefaultHostnameVerifier(hv);

      SVC_INST_REF.setType(SVC_INST_NAME);
      SVC_INST_REF.setValue(SVC_INST_NAME);

      vimService = new VimService();
      vimPort = vimService.getVimPort();
      Map<String, Object> ctxt =
         ((BindingProvider) vimPort).getRequestContext();

      ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
      ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

      serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
      headers =
         (Map) ((BindingProvider) vimPort).getResponseContext().get(
                 MessageContext.HTTP_RESPONSE_HEADERS);
      vimPort.login(serviceContent.getSessionManager(),
                    userName,
                    password, null);
      isConnected = true;

      propCollectorRef = serviceContent.getPropertyCollector();
      rootRef = serviceContent.getRootFolder();
   }
   ...
```

Multiple Versions of the vSphere API

When a client application connects to a Web service running on a vSphere server (ESX and vCenter system), the server detects the version of the API that was used to develop the client and makes available only those operations supported by the client.

Client applications convey information about the API version used in the SOAP messages that they send to a vSphere server. These SOAP messages include a `versionID` in the `soapAction` attribute. The details are handled transparently by the SOAP toolkit and the client proxy code. The server adjusts its behavior based on the client's version information, exposing the API version that the client supports to the client.

Starting with vSphere 4.0, information about the supported API versions is contained in an XML file, `vimServiceVersions.xml`, located on the server (see Service-Versions File (vimServiceVersions.xml)). You can access this file with the URL https://*server_hostname*`/sdk/vimServiceVersions.xml`.

Service-Versions File (vimServiceVersions.xml)

```
<?xml version="1.0" encoding="UTF-8" ?>
- <!--    Copyright 2008-2010 VMware, Inc.  All rights reserved. -->
- <namespaces version="1.0">
     - <namespace>
            <name>urn:vim25</name>
            <version>5.0</version>
     - <priorVersions>
          <version>2.5u2</version>
          <version>2.5</version>
       </priorVersions>
```

```
    </namespace>
    - <namespace>
        <name>urn:vim2</name>
        <version>2.0</version>
      </namespace>
  </namespaces>
```

If you are developing a client application that must support multiple server versions at the same time (ESX 5.0 and ESX 5.5, for example), you must obtain information about the API versions that are supported on the server and provide logic in your code to use or not use features, based upon the version information.

One approach to targeting multiple versions of the API from the same client application code is to check for the existence of the server versions file on the server. If you do not find a `vimServiceVersions.xml` file on the server, the server is older than ESX/ESX 4.x, ESX and vCenter vCenter 4.x.

Java Sample Applications in the Web Services SDK

The vSphere Web Services SDK includes sample applications, written in Java, that demonstrate features of the vSphere API and object model. Most of the samples do not handle exceptions, and they accept all security certificates. So use the applications as examples for extracting the types of data you want to view, but do not use the helper classes, trust store methods or exception handling techniques in your production environment.

Java Samples in the Web Services SDK
The Java samples in your vSphere Web Services SDK include `.java` files that you can compile and then run using any Java editor or IDE. The samples accept command-line arguments for the vSphere server name (DNS name or IP address), user name, and password.

Client Applications Using the JSON Protocol

You can create client applications for the Web Services API using the JSON data protocol instead of XML. JSON offers functionality similar to the SOAP protocol, but with a REST-like interface and on a separate service endpoint. JSON clients must perform functions such as authenticating a session, retrieving data, and ending the session. This chapter outlines how to code basic functions using Python to drive the JSON protocol.

Client Requirements for the JSON Protocol

A JSON protocol client needs a mid-level capability to build HTTP request messages, send the requests over the network, and manage server responses. Many languages have libraries to abstract TCP/IP layers while also providing access to URLs, headers, and bodies.

For example, some popular libraries include:

Python 3 – `urllib.request` module

Java – `java.net.HttpUrlConnection` or for Java 11 and later, `java.net.HTTP` module

Go – `net/http` package

Rust – `http` module

Perl – `LWP` module

Swift – `URL`, `URLRequest`, `URLSession`, and `URLSessonDataTask` interfaces

C++ – `libcurl` or `curlpp`

Command line and graphical tools for passing HTTP messages include:

- Curl or `wget`
- Postman
- Insomnia

- Hoppscotch

To use the JSON protocol, you build URLs that embed the names of managed objects, properties, and methods. You build request bodies that specify the parameters for method calls. You read response bodies and response headers. You include a session identifier in your request headers.

Outlining a Client for the JSON Protocol

A client to work with the JSON protocol generally contains the following parts, at minimum.

- The basic elements of the URL, including the address of the vCenter instance, the service endpoint specifier, and the version specifier.
- The credentials with which you authenticate your client session.
- Code to GET managed object properties, such as the `ServiceInstance` content, which contains managed object references to singletons, including the `SessionManager`.
- Code to POST requests that invoke methods on managed objects, such as `LoginByToken`.
- A request to fetch the `ServiceInstance` content and extract the `SessionManager` managed object reference.
- A request to authenticate the client session with `SessionManager`.
- A request to end the client session.

Following sections use fragments of Python code to show how you can build these parts of the request.

Building JSON Request URLs

The service endpoint for the JSON API is determined by the vCenter address and the API version that the client prefers for the response. These data form part of the service URL.

All requests in the JSON API specify a managed object. Some requests retrieve properties of a managed object. Other requests invoke a method on a managed object.

The parts of the service URL are as follows:

1. `scheme = 'https://'`
2. `domain =` vCenter *`IP address or FQDN`*
3. The URL path contains the following parts:
   a. `endpoint = '/sdk/vim25'`
   b. `version = '8.0.1.0'`
   c. A managed object reference, generated by the server, which has two subparts, `type` and `value`, where:
      a. `type` is the name of a type of managed object, such as `Folder` or `VirtualMachine` or `ServiceInstance`
      b. `value` is a unique identifier (within this vCenter instance) for a specific object of the type named, such as `group-d1` or `vm-015` or `ServiceInstance`.
      The object notation must be serialized as two strings separated by a slash (`/`). For example: `'Folder/group-d1'`.

```
def serialize_moref_for_request_url(moref):
  return('{}/{}'.format(moref['type'], moref['value']))
```

   d. A property name or method name, such as `childType` or `CreateVM`

An example request URL to retrieve the type or types of managed objects that a folder can contain::

```
>>> scheme = 'https://'
>>> domain = 'vcenter.example.com'
>>> endpoint = '/sdk/vim25'
>>> version = '8.0.1.0'
>>> motype = 'Folder'
>>> moid = 'group-d1'
>>> moref = motype + '/' + moid
>>> property = 'childType'
>>> url = scheme + domain + endpoint + '/' + version + '/' + moref + '/' + property
>>> print(url)
```

```
https://vcenter.example.com/sdk/vim25/8.0.1.0/Folder/group-d1/childType
```

An example request URL to cancel a long-running operation:

```
...
>>> motype = 'Task'
>>> moid = 'task-124'
>>> moref = motype + '/' + moid
>>> method = 'Cancel'
>>> url = scheme + domain + endpoint + '/' + version + '/' + moref + '/' + method
>>> print(url)
https://vcenter.example.com/sdk/vim25/8.0.1.0/Task/task-124/Cancel
```

Building JSON Request Bodies

Methods are invoked with the JSON protocol by sending a POST request with the method name in the URL and the method parameters in the body of the request.

The request body for a method invocation is a single JSON object containing the method parameters as named objects or arrays. For instance, a request body for the `MoveIntoFolder_Task` method might look like this:

```
{'list': [
    {'_typeName': 'ManagedObjectReference',
    'type': 'VirtualMachine',
    'value': 'vm-23'},
    {'_typeName': 'ManagedObjectReference',
    'type': 'VirtualMachine',
    'value': 'vm-96'},
    {'_typeName': 'ManagedObjectReference',
    'type': 'VirtualApp',
    'value': 'resgroup-v44'}
  ]
}
```

The method name itself becomes part of the URL, as described in Building JSON Request URLs.

> **Note:**
> Use the WSDL forms of type names. WSDL names are shown in the *vSphere Web Services API* reference. For example, use the WSDL name `ApplyHostProfileConfigurationSpec`, rather than the package name `vim.profile.host.ProfileManager.ApplyHostConfigSpec`, to build a data object as a parameter to the method `ApplyEntitiesConfig_Task`. Similarly, use the method's WSDL name, `ApplyEntitiesConfig_Task`, rather than the source name, `applyEntitiesConfiguration`.

Retrieving the Service Instance with the JSON Protocol

`ServiceInstance` is a singleton managed object that acts as a gateway to the resources available on the server. You can locate other important singletons such as `SessionManager` and `SearchIndex` from `ServiceInstance` properties. `ServiceInstance` is accessible without authentication, so you can retrieve a reference to the `SessionManager` which you can use to authenticate a session.

You need the IP address or FQDN of a vCenter to which you will send the request.

To retrieve the properties of the `ServiceInstance` managed object, do the following.

1. Synthesize a managed object reference to the `ServiceInstance` managed object. This managed object reference always has a fixed value of `{type: "ServiceInstance", value: "ServiceInstance"}` because it is a prerequisite for access to other managed object references.

   ```
   motype = 'ServiceInstance'
   moid = 'ServiceInstance'
   moref = motype + '/' + moid
   ```

2. Specify the property to retrieve from the managed object.

```
property = 'content'
```

3. Assemble the URL from the pieces previously defined.

```
scheme = 'https://'
domain = 'vc.example.com'
endpoint = '/sdk/vim25'
version = '8.0.3'
url = scheme + domain + endpoint + '/' + version + '/' + moref + '/' + property
print(url)
```

```
href="https://vc.example.com/sdk/vim25/8.0.3/ServiceInstance/ServiceInstance/content"
```

4. Send the URL in an HTTP GET request.

```
from urllib.request import urlopen, Request
import ssl
# Skip certificate verification on test systems only:
unverified_context = ssl._create_unverified_context()
request = Request( url, headers=headers or {} )
with urlopen( request, context=unverified_context ) as response :
  response_headers = response.headers
  response_body = response.read()
```

5. Display the result of the query.

```
from pprint import pprint
json_body = json.loads(response_body)
pprint(json_body)
```

The program displays a number of managed object references for singletons that you can use for additional requests. The following excerpt includes several important managed objects that you will use for basic operations, including authentication.

Excerpt from Service Content

```
...
'propertyCollector': {'_typeName': 'ManagedObjectReference',
                      'type': 'PropertyCollector',
                      'value': 'propertyCollector'},
'rootFolder': {'_typeName': 'ManagedObjectReference',
               'type': 'Folder',
               'value': 'group-d1'},
'scheduledTaskManager': {'_typeName': 'ManagedObjectReference',
                         'type': 'ScheduledTaskManager',
                         'value': 'ScheduledTaskManager'},
'searchIndex': {'_typeName': 'ManagedObjectReference',
                'type': 'SearchIndex',
                'value': 'SearchIndex'},
'serviceManager': {'_typeName': 'ManagedObjectReference',
                   'type': 'ServiceManager',
                   'value': 'ServiceMgr'},
'sessionManager': {'_typeName': 'ManagedObjectReference',
                   'type': 'SessionManager',
                   'value': 'SessionManager'},
...
```

Use the `SessionManager` URL in the Service Content to open an authenticated session with the vCenter instance.

Authenticating a JSON Client with the Session Manager

Most method calls must carry a session ID to authenticate with the server at the time of the call. The session ID is a temporary substitute for username and password, thereby limiting risk to the principal's credentials.

To acquire a session ID, you have these options:

- Invoke the `SessionManager.Login` method with username and password, exchanging the principal's credentials for a session ID token.
- Request a SAML token from the vCenter Single-Sign-On server, and use the token to invoke the `SessionManager.LoginByToken` method, receiving a session ID token in exchange.

After a successful authentication operation with vCenter, you extract the session ID token from the HTTP header of the response message. Include the same header in subsequent API request messages.

For example, the response headers might contain lines similar to the following:

```
content-type: application/json
date: Tue, 14 Aug 2007 13:30:00 GMT
vmware-api-session-id: 0123498765fabdec526374859607l829304100ab
x-envoy-upstream-service-time: 45
```

The value `0123498765fabdec526374859607l829304100ab` is the session ID token. Use it in the headers of your API requests, like this:

```
content-type: application/json
vmware-api-session-id: 0123498765fabdec526374859607l829304100ab
```

Basic Authentication for the JSON Protocol

Basic authentication relies on username and password. These credentials are transported over HTTPS with TLS encryption, but for stricter security consider using SAML token authentication instead.

Basic authentication requires that you have a username and corresponding password that have privileges on the vCenter instance to which you connect. You also need the IP address or fully qualified domain name of the vCenter instance.

To authenticate with username and password, follow these steps.

1. Prepare a JSON serialization of your credentials. This will be the body of your request.

   ```
   body_string = '{"userName": "Administrator@vsphere.local", "password": "betyoucantguess"}'
   ```

2. Get the `SessionManager` managed object reference from the `ServiceInstance` content. Format it for the resource portion of the service URL.

   ```
   moref = si_content['sessionManager']
   url_moref = '{}/{}'.format(moref['type'], moref['value'])
   ```

3. Assemble the complete URL for a request to invoke the Login method of `SessionManager`.

   ```
   scheme = 'https://'
   domain = 'vcenter.example.com'
   endpoint = '/sdk/vim25'
   version = '8.0.1.0'
   method = 'Login'
   url = scheme + domain + endpoint + '/' + version + '/' + url_moref + '/' + method
   print(url)
   ```

   ```
   https://vcenter.example.com/sdk/vim25/8.0.1.0/SessionManager/SessionManager/Login
   ```

4. Set the request header `'Content-Type' = 'application/json'`.

   ```
   request_headers = {}
   request_headers['Content-Type'] = 'application/json'
   ```

5. Send the URL and the body in an HTTP POST request.

   ```
   from urllib.request import urlopen, Request
   import ssl
   # Skip certificate verification on test systems only:
   ```

```
unverified_context = ssl._create_unverified_context()
request = Request(url, headers=request_headers)
with urlopen(request, context=unverified_context) as response :
  response_headers = response.headers
  response_body = response.read()
```

6. Extract the session token from the authentication header in the response.

```
token = response_headers['vmware-api-session-id']
```

The token you extracted is your current session ID and is required to access most managed object properties and methods with the API.

Save the token. Add the token header to all subsequent requests before sending.

```
request_headers['vmware-api-session-id'] = token
```

SAML Token Authentication for a JSON Client

To authenticate by SAML token, you must use SOAP message protocol rather than the JSON protocol. After authentication, you can send authenticated requests over the JSON protocol.

SAML token authentication using the SOAP protocol is described in the *vCenter Single Sign-On Programming Guide*. Use the Single-Sign-On server to exchange the principal's credentials for a SAML token, then invoke the `LoginByToken` method of the `SessionManager` to exchange the SAML token for a session ID token.

The `SessionManager` returns the session ID token in a cookie header. Use the value of the cookie as the value of `vmware-api-session-id` in request message headers, as described in Authenticating a JSON Client with the Session Manager.

Mapping Methods for the JSON Protocol

All methods defined in the WSDL schema for the vSphere Web Services API can be invoked by using a REST-like URL with the POST verb. The method name appends to the URL and the method parameters appear as JSON in the body of the request.

Method names begin with an uppercase letter and match the method name in the vSphere Web Services API Reference. All methods attach to a managed object reference, as in the SOAP API.

For example, the `SessionManager` managed object has a method whose WSDL name is `AcquireLocalTicket`.

The URL format that you use to invoke a method is

```
https://vcenter_fqdn/sdk/vim25/8.0.1.0/mo_ref/method_name
```

For example, you invoke the `AcquireLocalTicket` method with a URL like this:

```
https://vcenter_fqdn/sdk/vim25/8.0.1.0/SessionManager/SessionManager/AcquireLocalTicket
```

When you POST a request to invoke a method on a managed object, the body of the request contains a single anonymous JSON object that specifies the parameter names and values. Unlike the SOAP request protocol, the JSON does not specify a reference to "this" because the managed object reference is in the URL path.

For example, the method `Folder.CreateFolder` requires a name for the new folder. The body for that request would look similar to this:

```
{"name": "spare_VM_folder"}
```

Accessing Managed Object Properties with the JSON Protocol

You retrieve a property of a managed object by appending the property name to the URL, after the managed object reference. Property names always begin with a lowercase letter, matching the name in the vSphere Web Services API Reference.

For example, you can retrieve the string property `HostProfile.validationState` with a URL similar to this:

```
https://vcenter_fqdn/sdk/vim25/8.0.1.0/hostprofile_mo_ref/validationState
```

Note:
You cannot modify properties directly. To modify properties of a managed object, you must invoke a method of the managed object or a related managed object that affects its state on the server.

Retrieving Nested Properties

You can retrieve a top-level property by naming it in the URL, but nested properties are not directly accessible. Many top-level properties are data objects or arrays, containing nested properties of interest. To access nested properties, first retrieve the top-level property, then deserialize the JSON in the response body. After deserialization, you can access the nested properties of the data object or array in the client.

For example, code this request:

```
GET https://vcenter_fqdn/sdk/vim25/8.0.1.0/vm_mo_ref/runtime
```

Then extract and display the nested property like this:

```
data_object = json.loads(response_body)
state = data_object['powerState']
print('VM powerState = ' + state)
```

The nested property displays.

```
VM powerState = poweredOff
```

Retrieving Managed Object Reference Properties

Managed objects often have properties to reference other managed objects. In these cases, you can use a GET request to obtain the managed object reference, but to access the properties or methods of the other managed object you must issue a new GET request, using the managed object reference in the request URL.

If you need to follow a chain of managed object references, you can avoid extra requests by using the `PropertyCollector` managed object. The purpose of the `PropertyCollector` is to delegate the search functions to the server, where they can be done more efficiently.

Using the Property Collector to Retrieve Object Properties

You can use the `PropertyCollector` to retrieve properties from one or more managed objects, whether or not they are related to each other, if you can specify the set of objects in your request. The `PropertyCollector` provides a flexible interface by which you can specify search criteria to execute in the server.

To specify a property retrieval, several approaches are available to you. A few of them are:

- Provide a managed object reference and a set of its properties that you want to retrieve.
- Provide a managed object reference as a starting point for a search that traverses inventory paths to find related managed objects and return their properties.
- Provide a reference to a previous search set as a starting point for a traversal search, often directed to the managed objects contained in the set.

This example shows a hybrid approach where you first locate a `Datacenter` managed object by using the `SearchIndex` method. Then you use that managed object as a starting point to traverse to `VirtualMachine` objects in the data center's virtual machine folder. If there are subfolders within the virtual machine folder, the search specifications traverse the subfolders recursively.

The goal of this example is to report whether the virtual machines are powered on or powered off. That information is in the `VirtualMachine.runtime.powerState` property.

The `powerState` property is a nested property of the virtual machine, so you need to locate the `VirtualMachine` managed objects and specify a property within the `runtime` data object belonging to each `VirtualMachine`. The JSON protocol has the ability to retrieve the entire runtime data object as a single property, but the `PropertyCollector` is able to extract the nested property in a single query.

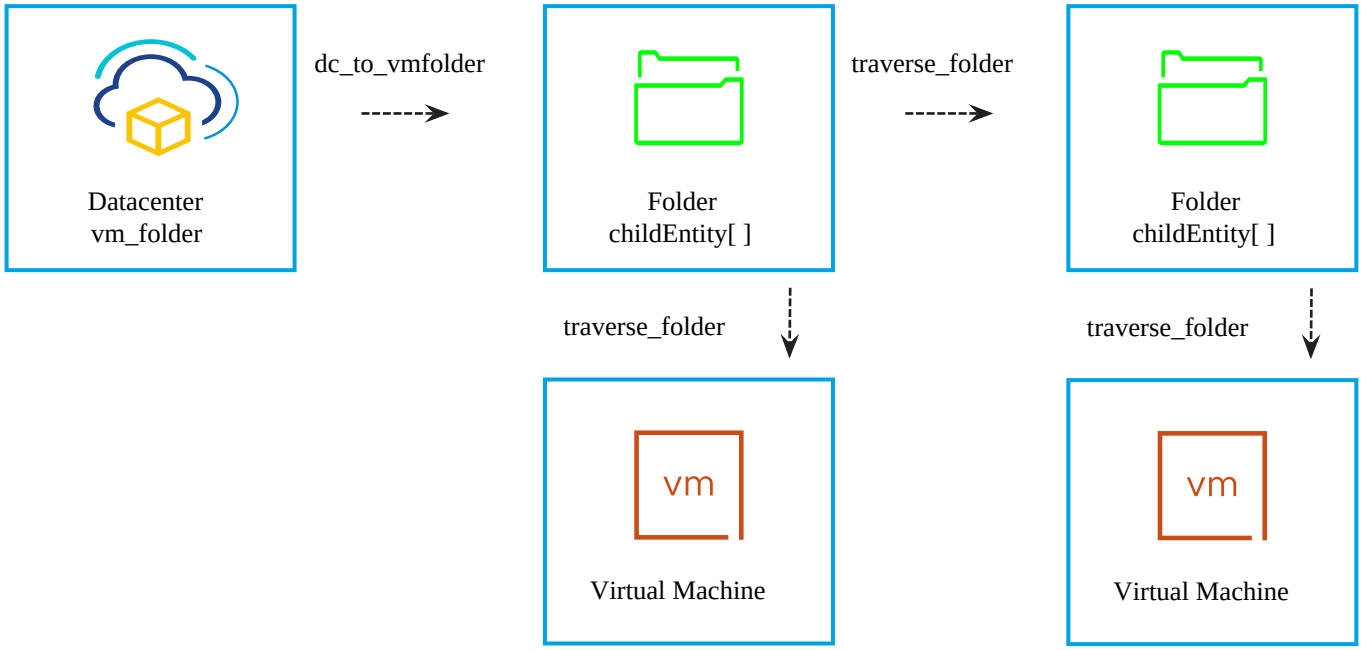
Use the following procedure to build a server-side specification that retrieves the power states of virtual machines in the data center's virtual machine folder.

Note:
For simplicity, this procedure ignores vApps.

1. Build a `PropertySpec` data object to identify the managed object type (`VirtualMachine`) and the nested property name (`runtime.powerState`).

```
propSet = [ {'_typeName': 'PropertySpec',
             'type': 'VirtualMachine', 'pathSet': ['runtime.powerState'], 'all': False}]
```

   The `all` property, if `True`, overrides the `pathSet` property and collects all properties of the managed object. This behavior is useful if you want to maintain a client-side copy in synch with the server-side managed object.
2. (Optional) If it helps to clarify the inventory paths to the virtual machines, sketch `VirtualMachine` objects and their parent managed objects, all the way up the chain to the starting `Datacenter` managed object.



   For each path between managed objects, note the name of the property in the parent object that links to the child object. You must create a `TraversalSpec` (or a `SelectionSpec` reference to a `TraversalSpec`) to follow each of those links to the next managed object. The `VirtualMachine` managed objects whose properties you want to retrieve do not need `TraversalSpec` objects.
3. Working backwards from `VirtualMachine` objects to `Datacenter` object, create a named `TraversalSpec` data object for each path segment.

```
traverse_folder = {'_typeName': 'TraversalSpec'}
traverse_folder['name'] = 'traverse_folder'
traverse_folder['type'] = 'Folder'
traverse_folder['path'] = 'childEntity'
traverse_folder['skip'] = False

dc_to_vmfolder = {'_typeName': 'TraversalSpec'}
dc_to_vmfolder['name'] = 'dc_to_vmfolder'
dc_to_vmfolder['type'] = 'Datacenter'
dc_to_vmfolder['path'] = 'vmFolder'
dc_to_vmfolder['skip'] = True
```

   You do not need a `TraversalSpec` for the `VirtualMachine` object, because it is a leaf object in the traversal tree. You do need a `TraversalSpec` for the `Datacenter` object and the `Folder` object, so the `PropertyCollector` will know how to traverse down the chain to the child objects. The `skip` property is an optimization. It tells the `PropertyCollector` that the next managed object after the traversal can be traversed without checking the `PropertySpec`.

   Note:
   The `TraversalSpec.selectSet[]` property is not yet set. You will use it to link the specs together.

4. If your data center might have nested folders inside its VM folder, create a `SelectionSpec` data object that specifies the name of the `TraversalSpec` for `Folder` objects, so the `PropertyCollector` can follow the `childEntity` path recursively for any nested folders it finds.
```
folder_recursion = {'_typeName': 'SelectionSpec', 'name': 'traverse_folder'}
```
5. Link the `TraversalSpec` objects together by using the `selectSet[]` property to build a managed object traversal chain.
```
traverse_folder['selectSet'] = [folder_recursion]
dc_to_vmfolder['selectSet'] = [traverse_folder]
```
6. Create an `ObjectSpec` data object to specify starting from your `Datacenter` managed object, and to direct the `PropertyCollector` to the `TraversalSpec` that leads to the top-level VM folder.
```
objectSet = [{'_typeName': 'ObjectSpec',
              'obj': dc_moref, 'skip': False,
              'selectSet': [dc_to_vmfolder] }]
```
   The `skip` property in the `ObjectSpec` is different from the `skip` property in a `TraversalSpec`. In the `ObjectSpec`, it tells the `PropertyCollector` that the starting object is not one of the managed objects it is looking for.
7. Combine the `PropertySpec` and the `ObjectSpec` into a `PropertyFilterSpec` that will be a parameter to the `PropertyCollector.RetrievePropertiesEx` method.
```
specSet = [{'_typeName': 'PropertyFilterSpec',
            'propSet': propSet, 'objectSet': objectSet}]
```
8. Specify the `RetrievePropertiiesEx` method parameters in the request body.
```
import json
param_dict = {'options': {'_typeName': 'RetrieveOptions',
                          'maxObjects': 1000}, 'specSet': specSet}
request_body = json.dumps(param_dict).encode('utf-8')
```
9. Send the request, using the POST verb.
```
from urllib.request import urlopen, Request
import ssl
moref = si_content['propertyCollector']
method = 'RetrievePropertiesEx'
url = make_url(domain, mo=moref, m_p=method)
request_headers = {}
request_headers['vmware-api-session-id'] = token
request_headers['Content-Type'] = 'application/json'
headers['Content-Type'] = 'application/json'
# Skip certificate verification on test systems only:
unverified_context = ssl._create_unverified_context()
request = Request(url, headers, data=body)
with urlopen(request, context=unverified_context) as response :
  response_headers = response.headers
  response_body = response.read()
json_body = json.loads(response_body)
print(json_body)
```

The results are returned in a `RetrieveResult` data object, which encloses an array of `ObjectContent` data objects. Each `ObjectContent` contains a managed object reference and a corresponding array of named properties found by the `PropertyCollector` that match your `PropertySpec`. For this example, the managed object references identify `VirtualMachine` objects, and the property name-value pairs show the `powerState` of each `VirtualMachine`.

Using Polymorphic Types with the JSON Protocol

The JSON protocol deals with managed objects whose properties include WSDL-defined data objects, such as configuration objects. Method parameters can also be data objects. Some of these data objects are polymorphic, in the sense that they can inherit from other data objects and can be referenced as the parent type.

The JSON protocol distinguishes the assigned type by adding a `_typename` property to the JSON serialization. For example, a `TaskInfo` data object contains a `result` property that has the XSD type `xsd:anyType`. The actual type of the assigned value is specified with a `_typename` property in `result`.

For example, the following JSON shows how a result data object is returned in the server response and the assigned type is identified by the `_typeName` property.

```
"result": {
    "_typeName": "ManagedObjectReference",
    "_value": {
        "value": "domain-s16",
        "type": "ComputeResource"
    }
}...
```

When you use a polymorphic type in a request, you must add a `_typename` property to identify the assigned type for the server. For example, if you create a `PropertyFilter` for a request to the PropertyCollector, you might include a `TraversalSpec` object, which inherits from the `SelectSet` object. Add the `_typename` property to distinguish a `TraversalSpec` value from a `SelectSet` value:

```
"traverse_vm_folder": {
   "_typeName": "TraversalSpec",
   "type": "Folder",
   "path": "childEntity",
   "skip": false
}
```

Many properties are defined in the WSDL as inheriting from `DynamicData`. For that reason, you will use `_typeName` frequently. A best practice is to add the `_typeName` property to all objects you create for the JSON API.

Python Client Example for the JSON Protocol

This example illustrates several basic operations needed to implement a client that uses the JSON protocol.

```
import json
from pprint import pprint

domain = 'vcenter.example.com'
credentials = {'userName': 'Administrator@vsphere.local', 'password': 'betyoucantguess'}
dc_name = 'DC-1'

def make_url(vc, ver='8.0.1.0', mo='ServiceInstance/ServiceInstance', m_p='content'):
  if isinstance(mo, dict):
    mo = '{}/{}'.format(mo['type'], mo['value'])
  url = 'https://{}/sdk/vim25/{}/{}/{}'.format(vc, ver, mo, m_p)
  return url

def send_request(url, headers=None, body=None):
  from urllib.request import urlopen, Request
  import ssl
  if headers is None: headers = {}
  headers['Content-Type'] = 'application/json'
  # Skip certificate verification on test systems only:
  unverified_context = ssl._create_unverified_context()
  request = Request(url, headers=headers or {}, data=body)
  with urlopen( request, context=unverified_context ) as response :
    response_headers = response.headers
    response_body = response.read()
  return( response_headers, response_body )
```

```
def get_service_content(vc):
  url = make_url(vc)
  (response_headers, response_body) = send_request(url)
  json_body = json.loads( response_body )
  return( json_body )

def basic_auth(domain, credentials):
  si_content = get_service_content(domain)
  moref = si_content['sessionManager']
  method = 'Login'
  url = make_url(domain, mo=moref, m_p=method)
  request_headers = {}
  request_headers['Content-Type'] = 'application/json'
  request_body = json.dumps(credentials).encode('utf-8')
  (response_headers, response_body) = send_request(url, headers=request_headers, body=request_body)
  print('Response:')
  print(response_headers)
  print(json.loads(response_body))
  token = response_headers['vmware-api-session-id']
  return(token)

def logout(domain, session_moref) :
  request_body = {}
  method = 'Logout'
  url = make_url(domain, mo=session_moref, m_p=method)
  request_headers = {}
  request_headers['vmware-api-session-id'] = token
  request_headers['Content-Type'] = 'application/json'
  (response_headers, response_body) = send_request(url, headers=request_headers, body=request_body)
  return(response_headers)

def get_datacenter_moref(domain, dc_name, token):
  si_content = get_service_content(domain)
  moref = si_content['searchIndex']
  method = 'FindByInventoryPath'
  url = make_url(domain, mo=moref, m_p=method)
  body_dict = {'inventoryPath': dc_name}
  request_body = json.dumps(body_dict).encode('utf-8')
  request_headers = {}
  request_headers['vmware-api-session-id'] = token
  request_headers['Content-Type'] = 'application/json'
  (response_headers, response_body) = send_request(url, headers=request_headers, body=request_body)
  json_body = json.loads(response_body)
  return(json_body)

def get_vm_power_states(domain, dc_moref, token):
  # MO type & properties to retrieve.
  propSet = [ {'_typeName': 'PropertySpec',
               'type': 'VirtualMachine', 'pathSet': ['runtime.powerState'], 'all': False}]
  # Work backwards to define the steps in the traversal path.
  # traversalspec = type to traverse, path out, skip next?, (selectSet).
  # Traverse any nested folders inside vmFolder:
  traverse_folder = {'_typeName': 'TraversalSpec'}
  traverse_folder['name'] = 'traverse_folder'
  traverse_folder['type'] = 'Folder'
  traverse_folder['path'] = 'childEntity'
  traverse_folder['skip'] = False
```

```
  traverse_folder['selectSet'] = [{'_typeName': 'SelectionSpec',
                                   'name': 'traverse_folder'}]
  # Traverse Datacenter to vmFolder:
  dc_to_vmfolder = {'_typeName': 'TraversalSpec'}
  dc_to_vmfolder['name'] = 'dc_to_vmfolder'
  dc_to_vmfolder['type'] = 'Datacenter'
  dc_to_vmfolder['path'] = 'vmFolder'
  dc_to_vmfolder['skip'] = True
  dc_to_vmfolder['selectSet'] = [traverse_folder]
  # objectSet = Starting MO + traversal steps to retrieval objects.
  objectSet = [{'_typeName': 'ObjectSpec',
                'obj': dc_moref, 'skip': False,
                'selectSet': [dc_to_vmfolder]}]
  # specSet = total filter specification.
  specSet = [{'_typeName': 'PropertyFilterSpec',
              'propSet': propSet, 'objectSet': objectSet}]
  # Params to RetrievePropertiesEx():
  param_dict = {'options': {'_typeName': 'RetrieveOptions',
                            'maxObjects': 1000}, 'specSet': specSet}
  # Build request message.
  request_body = json.dumps(param_dict).encode('utf-8')
  moref = si_content['propertyCollector']
  method = 'RetrievePropertiesEx'
  url = make_url(domain, mo=moref, m_p=method)
  request_headers = {}
  request_headers['vmware-api-session-id'] = token
  request_headers['Content-Type'] = 'application/json'
  (response_headers, response_body) = send_request(url, headers=request_headers, body=request_body)
  json_body = json.loads(response_body)
  return(json_body)

# MAIN

si_content = get_service_content(domain)
pprint(si_content); print()

token = basic_auth(domain, credentials)
print('Session token: {}'.format(token)); print()

dc_moref = get_datacenter_moref(domain, dc_name, token)
print('{} MOref = {}'.format(dc_name, dc_moref)); print()

response_body = get_vm_power_states(domain, dc_moref, token)
print('Response body:'); pprint(response_body)

moref = si_content['sessionManager']
headers = logout(domain, moref)
print('Logging out:')
print(headers)
```

Datacenter Inventory

The vSphere inventory is a representation of the vSphere datacenter and the objects in the datacenter. Knowing how the objects in the datacenter relate to each other helps you traverse the inventory hierarchy and access the objects you want to manipulate.

Inventory Overview for the Web Services SDK

The vSphere inventory contains the following types of objects:

- Systems in the datacenter: `Host`, `VirtualMachine`, and `VirtualApp`.
- Support components: `ComputeResource`, `Datastore`, `Network`, and virtual devices.
- Organizational components: `Folder` and `Datacenter`

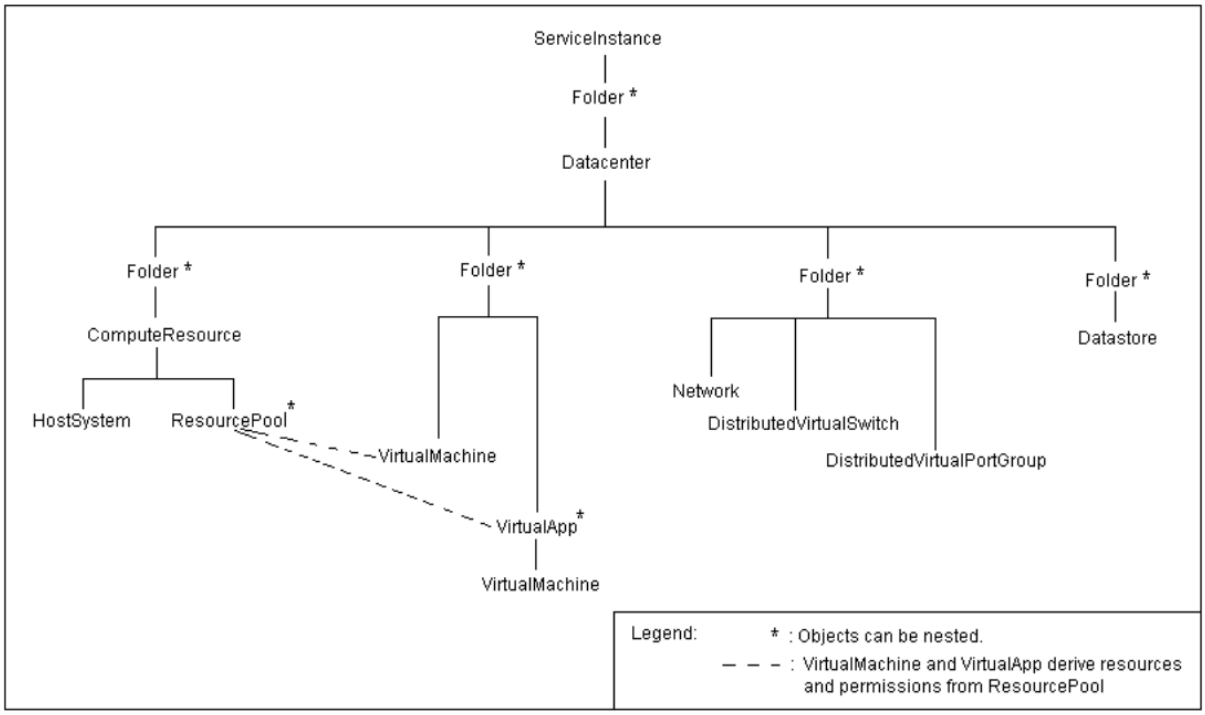
When you manage the virtual infrastructure, you access objects and their properties and methods based on their location in the inventory. Understanding the inventory structure is therefore critical for any programming task. You always start with the `ServiceInstance` associated with a session, which is the root object of the inventory, and traverse the inventory hierarchy from there. See Property Collector. How you access objects depends on whether your client application is connected to an ESX host and a vCenter instance.

Inventory Hierarchies and ServiceInstance

When you start a session, vSphere creates a `ServiceInstance` with one root folder, one `Datacenter`, and four folders that hold the different types of inventory objects.

When you access a vCenter System, the hierarchy shown in the illustration below allows you to traverse the inventory.

Figure 581: vCenter Inventory Hierarchy



CAUTION:

If your ESX hosts are managed by vCenter , you must always access your hosts through vCenter . The vCenter service keeps track of all synchronous and asynchronous operations, and will have the latest status and inventory information about each ESX host. Therefore, connecting directly to a managed host may give you incorrect or incomplete data.

When you have ESX hosts that are not managed by vCenter , your application can connect to each host directly.

Folders in the Hierarchy

If your installation includes a vCenter system, you can create additional datacenters under the root folder. For every `Datacenter` object, the server automatically creates the following `Folder` objects:

- A folder for `VirtualMachine`, template, and `VirtualApp` objects.

- A folder for a `ComputeResource` hierarchy.
- A folder for `Network`, `DistributedVirtualSwitch`, and `DistributedVirtualPortgroup` objects.
- A folder for `Datastore` objects.

In a large deployment, the nested structure allows you to organize the objects in the datacenter into an easily manageable structure by using multiple folders and datacenters.

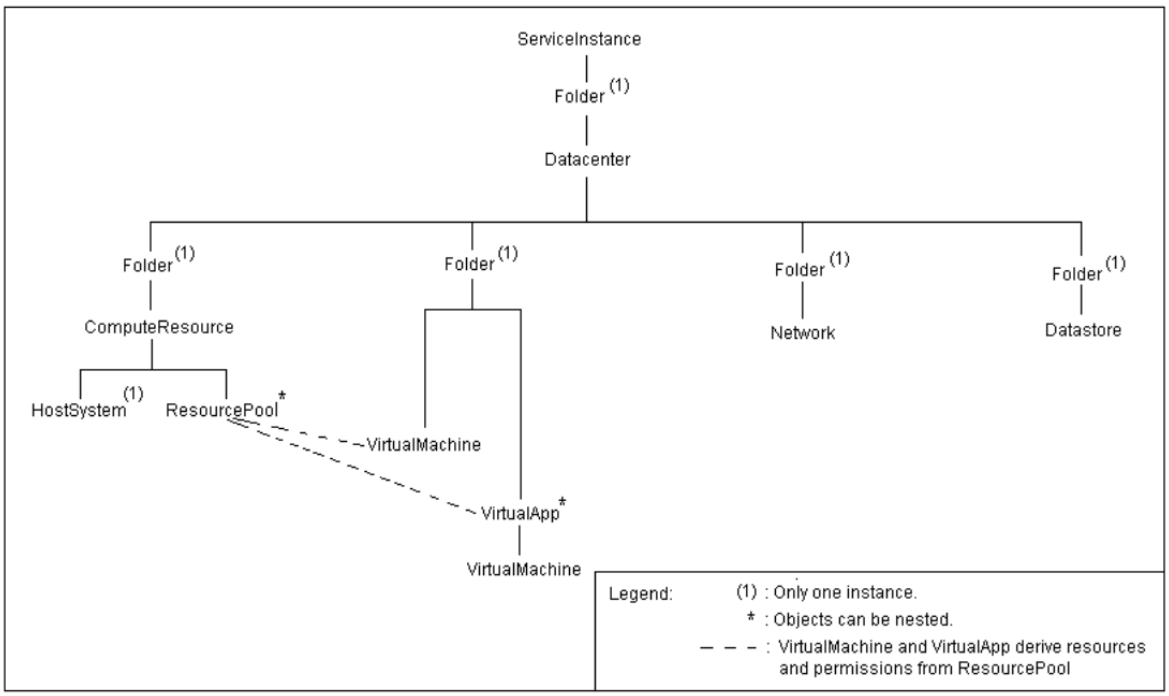
As of VCF 9.1, the maximum length of the `Folder` name was increased for compatibility with NSX network management. This enables common object identifiers between NSX and vCenter. In previous releases, NSX folder names could be too long for vCenter.

ESX Inventory Hierarchy

For a standalone ESX system, only a single datacenter is supported, and the `Folder` managed entity does not support creating additional `Folder` objects or `Datacenter` objects.

When you access an ESX host directly, rather than accessing the host through a vCenter system, the hierarchy shown in the illustration below allows you to traverse the inventory.

Figure 582: ESX Inventory Hierarchy



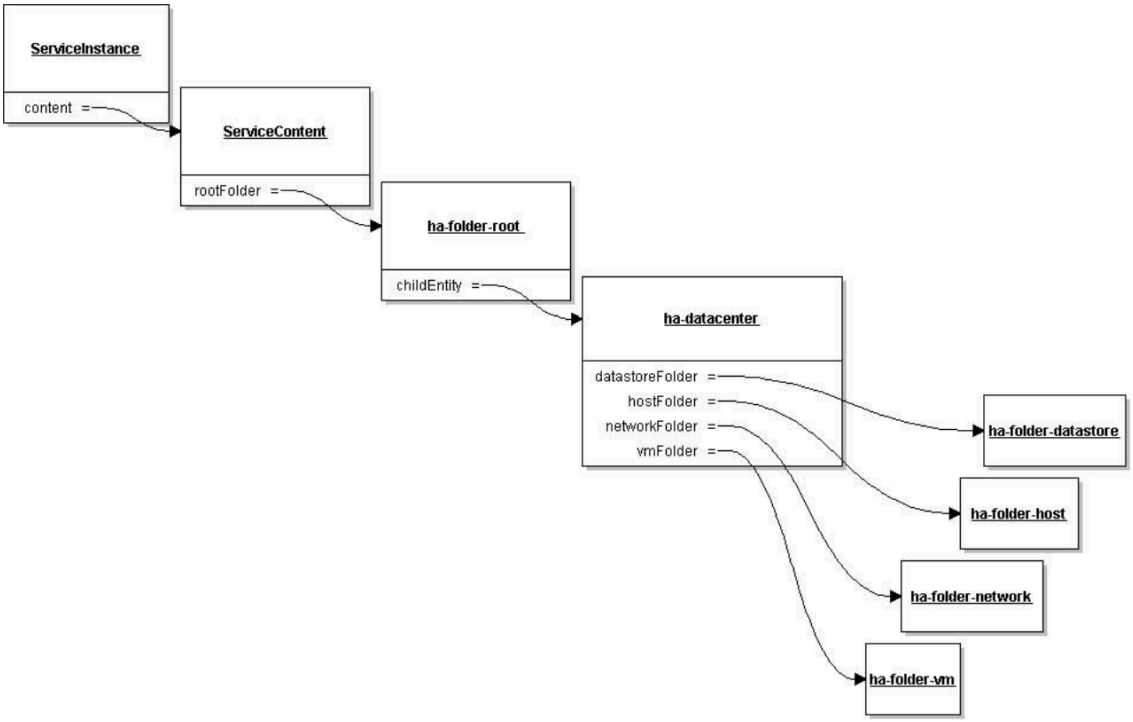
Accessing Inventory Objects

To retrieve information from an inventory object, you start with `ServiceInstance`, the root object of the inventory. You access an object using a `TraversalSpec` in conjunction with a property collector, using the properties that identify an object's position in the hierarchy.

- Every managed entity has a `parent` property that identifies its relative position in the inventory hierarchy.
- The `Folder` managed object has a `childEntity` property that identifies objects in a folder instance.

Figure 583: Instance Diagram of Root Folders in an Inventory shows the `childEntity` and `folder` properties that define the default objects in the inventory of a standalone ESX system. The inventory begins with the `ServiceContent.rootFolder` property. The `rootFolder` has a `childEntity` that consists of a managed object reference to a `Datacenter` managed object.

Figure 583: Instance Diagram of Root Folders in an Inventory



Creating Inventory Objects

The `Folder` managed entity provides methods for creating instances of the following managed entities.

- `Datacenter`
- `DistributedVirtualSwitch`
- `VirtualMachine`
- `Cluster`
- `Folder`

When you create these objects, they appear in the folder you invoked the creation method from.

While some managed entities are created through a method on the `Folder` managed entity, other managed entities are instantiated directly. For example, the `HostDatastoreSystem` has methods for creating datastores such as `CreateNasDatastore` and `CreateVmfsDatastore`.

> **Important:**
> When you create an inventory object, you must stay within the bounds of the host's capabilities, accessible through the `HostSystem.capability` property, which is a `HostCapability` data object. For example, a `HostCapability` object might have the `maxSupportedVMs` property specified.

Privileges Required for Inventory Management

Navigating the inventory requires a user account that can connect to the server and obtain a valid session. The user identity associated with the session is called a principal. When a client application attempts to access an object in the inventory, the server checks the permission object or objects and compares the permissions with the principal's privileges.

For example, creating a virtual machine requires that the principal associated with the session have the following privileges:

- The `VirtualMachine.Inventory.Create` privilege on the folder in which to create the virtual machine.
- The `Resource.AssignVMToPool` privilege on the resource pool from which the virtual machine obtains its allocation of CPU and memory resources.

Reading the `perfCounter` property of the `PerformanceManager` managed object requires the `System.View` privilege on the root folder.

Important:

Some privileges are specific to objects onESX and vCentervCenter or specific to ESX . For example, the `Alarm.Create` privilege associated with `AlarmManager` is available only through vCenter systems.

See Authentication and Authorization for more information on authentication, authorization, roles, and user identity.

Privileges

A privilege is a system-defined requirement associated with a VMware vSphere managed object. Privileges are static and do not change for a version of a product. Privileges for vSphere components are defined as follows:

```
<group>[.<group>].privilege
```

For example:

```
Datacenter.Create
Host.Config.Connection
Host.Config.Snmp
```

Permissions

Permissions are the associations of roles with privileges on a specified managed entity. You use permissions to specify which users can access which managed entity.

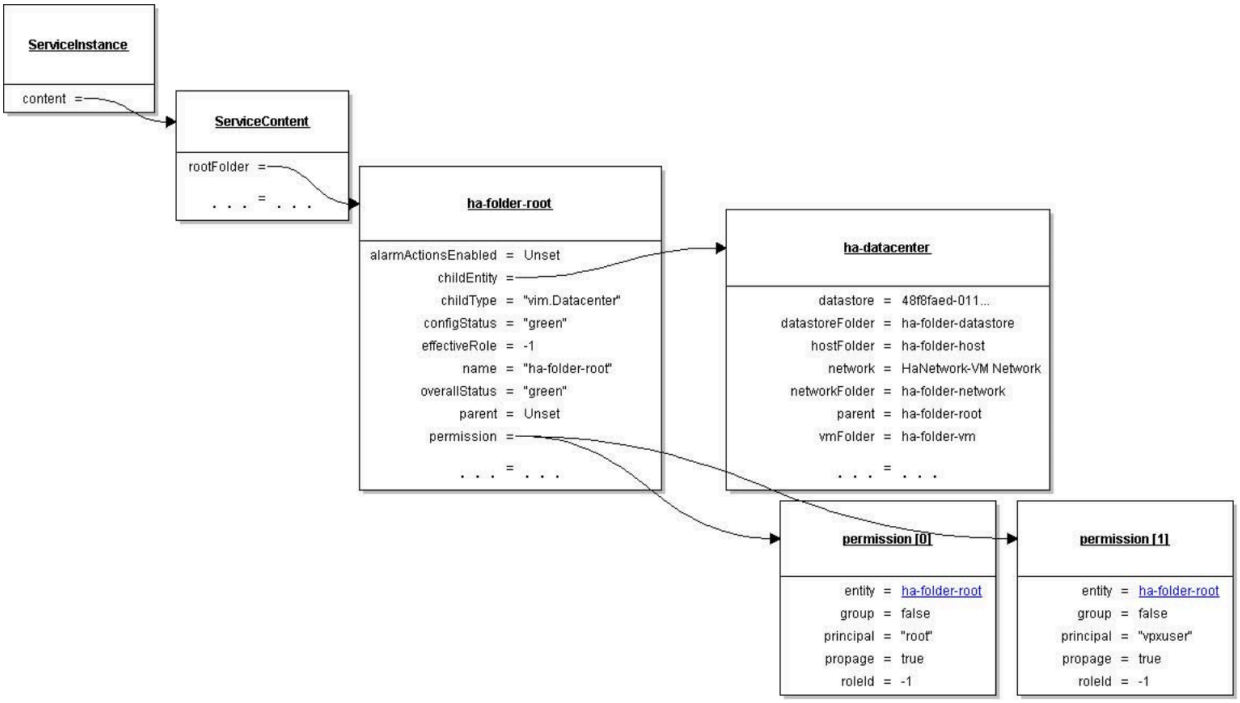
A child entity inherits the permissions of its parent if the parent's `propagate` property is set to true. A permission that is set directly on a child overrides the permission in the parent. To grant permission to all child entities of a `Datacenter` object, assign permissions to the `Datacenter` object and set the `Permission` object's `propagate` property to `true`.

Figure 584: Inventory and Permissions shows that users `root` and `vpxuser` both have permissions on the `rootFolder` of the inventory. The `vpxuser` is the account created on a host by the vCenter system when that host is added to the vCenter system. The vCenter system needs access to the inventory objects of the host systems that it manages, so the `vpxuser` account is granted privileges to the `rootFolder` of each host.

Important:

See Authentication and Authorization for a detailed discussion of privileges, permissions, and user management.

Figure 584: Inventory and Permissions



Managed and Standalone ESX Hosts

You can run ESX as a managed or standalone ESX host.

- Standalone ESX hosts are standalone hosts with limited capabilities. The inventory of a standalone host can support multiple virtual machines and multiple resource pools, but it contains a single default datacenter and a single root folder. The default datacenter and root folder are not visible in the vSphere Client, but they exist in the inventory of a standalone host and they are visible in the MOB.
- Managed ESX hosts have been added to the vCenter inventory. Available features depend on the licenses available for that host. For example, you can configure two or more hosts for VMware DRS resource management or VMware HA failover protection.

The following table summarizes the differences between the number of objects that an inventory can contain. See also Figure 582: ESX Inventory Hierarchy and Figure 581: vCenter Inventory Hierarchy.

ManagedEntity Subtype	ESX Inventory	vCenter Inventory
`ClusterComputeResource`	None.	Multiple instances supported.
`ComputeResource`	Exactly one only.	Multiple instances supported.
`Datacenter`	Exactly one only. Cannot be destroyed. Transparent.	Multiple instances supported.
`Datastore`	Multiple instances supported.	Multiple instances supported.
`DistributedVirtualSwitch`	Multiple instances supported.	Multiple instances supported.
`Folder`	Exactly one only. Cannot be destroyed. Transparent.	Multiple instances supported.
`HostSystem`	Exactly one only.	Multiple instances supported.
`Network`	Multiple instances supported.	Multiple instances supported.
`ResourcePool`	Multiple instances supported.	Multiple instances supported.

ManagedEntity Subtype	ESX Inventory	vCenter Inventory
VirtualApp	None.	Multiple instances supported.
VirtualMachine	Multiple instances supported.	Multiple instances supported.

Property Collector

vSphere provides the PropertyCollector service for accessing data and monitoring changes. Use the RetrievePropertiesEx to obtain values of managed object properties, and WaitForUpdatesEx to monitor and retrieve modified property values.

Introduction to the PropertyCollector

The PropertyCollector service provides a way to monitor and retrieve information about managed objects, such as whether a host in a cluster is offline, or whether a virtual machine is powered on.

The PropertyCollector uses one or more filters to determine the scope of collection and it has methods to retrieve data. A filter uses a set of data objects that specify the following information:

- Starting point for inventory traversal during the collection operation.
- Inventory traversal path.
- Objects and properties from which data will be collected.

A vSphere server creates a default PropertyCollector for every session, and allows you to create multiple, additional PropertyCollector objects. Create additional PropertyCollector objects, using one per thread, to perform concurrent collection operations.

Data Retrieval

PropertyCollector can retrieve data all at once, or incrementally.

- Property retrieval as a single operation uses the RetrievePropertiesEx and ContinueRetrievePropertiesEx methods. These methods perform a single collection operation.
- Incremental property retrieval, also referred to as property monitoring, uses the WaitForUpdatesEx method. The initial call to this method retrieves a baseline set of managed object property values. Subsequent calls retrieve changes in property values since the last retrieval. Use WaitForUpdatesEx to monitor changes to the inventory or any managed object properties.

  Note:
  The PropertyCollector does not guarantee the order of data that it returns in response to a request for data.

Inventory Traversal and Property Selection

To use PropertyCollector, first build a PropertyFilterSpec to guide the collector's actions on the server.

You pass your PropertyFilterSpec to either the RetrievePropertiesEx method or the CreateFilter method. The latter establishes a persistent filter on the server side for use with the WaitForUpdatesEx method.

A PropertyFilterSpec contains:

- The name of at least one managed object class from which you want to collect properties. (PropertySpec.type)
- For each managed object class specified, at least one name of a property to return. (PropertySpec.pathSet).
- If needed, one or more paths through the inventory hierarchy that the PropertyCollector can traverse to reach the managed objects from which you want to collect properties. (ObjectSpec.selectSet)
- For each path, a starting managed object reference from which to traverse that path. (ObjectSpec.obj)

If you do not have a starting managed object reference from a prior API call, you can start with the root folder at the base of the inventory hierarchy. The rootFolder reference is available in the ServiceContent data object. For information about getting the ServiceContent, see Overview of a Java Sample Application for the Web Services SDK.

Gathering Data with a ViewManager Object

You can use a vSphere view object in a `PropertyCollector` filter to simplify access to a subset of inventory objects. For example, `ContainerView` collects information about all VirtualMachine objects in a datacenter, and `ListView` tracks information about `Task` objects while an operation is in progress.

The `ViewManager`'s `ListView` method allows you to customize your view with an input object list, the `ContainerView` method lets you view all objects in a folder, datacenter, resource pool, or other container of managed objects, and the `InventoryView` method lets you monitor the entire inventory.

> **Note:**
> A best practice is to use the smallest view you can create. This will be the most efficient way to retrieve data with the PropertyCollector.

The `ViewManager` has the following property:

- `viewList` – An array of view references. Each array entry is a managed object reference to a view created by this View Manager.

The view acts as a virtual parent object to the set, which provides the following benefits:

- The view can collect objects from different parts of the inventory and make them available under the same parent.
- All objects in the set are accessible by doing a single traversal step, rather than a chain of several steps starting at the root folder.

A best practice when using views is to call the `DestroyView()` method when a view is no longer needed. This practice frees memory on the server.

See Retrieve Properties from a Set of Managed Objects with a View for an example that uses the `ContainerView` method to access Inventory data.

For more information about the vSphere inventory, browse the managed object definitions in the *vSphere Web Services API Reference*.

vSphere Data Objects for Property Collection

`PropertyCollector` data objects are used to specify objects and properties for the collector to retrieve.

Data Object	Description
`PropertyFilterSpec`	Provides access to object and property selection data. A `PropertyFilterSpec` must have at least one `ObjectSpec` and at least one `PropertySpec`.
`ObjectSpec`	Identifies the starting object for property collection, which can be either an inventory object or a view you created. An `ObjectSpec` also identifies the traversal specs used by the property collector.
`PropertySpec`	Identifies object types and their properties for collection.
`TraversalSpec`	Identifies the objects and paths that can be traversed to find the objects and properties identified by the `PropertySpec`.
`SelectionSpec`	Lists the `TraversalSpec` objects available at each step of an inventory traversal. Supports recursive traversal by name.
`View` objects	Use a `view` to form a set of starting objects for traversal and collection.

vSphere Methods for Property Collection

The `PropertyCollector` supports the following approaches to obtaining objects and properties from the server:

- If your client application does not keep a synchronized representation of server state, use the `RetrievePropertiesEx` method. `RetrievePropertiesEx` instantiates a filter, collects the specified objects and properties, and returns

the data to your client application as an `ObjectContent` data object. The server does not add the filter to the `PropertyCollector.filter` array. The server destroys the filter after returning the results to your client.

- If your application maintains a synchronized representation of server state, use the `CreateFilter` and `WaitForUpdatesEx` methods. `WaitForUpdatesEx` returns descriptions of property changes, organized by the filter that identified the properties.

In either case, you create a `PropertyFilterSpec` data object to specify the objects and properties you want to retrieve from the server.

The following table shows the `PropertyCollector` methods organized by the context in which you use them. For more information about these methods, see the *vSphere API Reference*.

Method Context	Method	Description
Monitor properties using different filters	`CreatePropertyCollector`	Creates a new `PropertyCollector` object to monitor properties using different filters. The vSphere server handles requests for a `PropertyCollector` instance independently of any other instances of the `PropertyCollector` on the server.
	`DestroyPropertyCollector`	Destroys an instance of a `PropertyCollector` that was created by a call to `CreatePropertyCollector` from your client application.
Single collection operation	`RetrievePropertiesEx`	Retrieves property data for the managed objects specified by a single-use `PropertyFilterSpec`.
	`ContinueRetrievePropertiesEx`	Retrieves additional property data for an operation started by `RetrievePropertiesEx`.
	`CancelRetrievePropertiesEx`	Cancels a `RetrievePropertiesEx` or `ContinueRetrievePropertiesEx` operation.
Incremental collection or monitoring operation	`WaitForUpdatesEx`	Retrieves changes to property data since the last `WaitForUpdatesEx` cycle. `WaitForUpdatesEx` blocks until one or more changes are detected or until the request times out.
	`CancelWaitForUpdatesEx`	Cancels a `WaitForUpdatesEx` operation.
General	`CreateFilter`	Creates a new instance of a `PropertyFilter` managed object.

Using the PropertyCollector with RetrievePropertiesEx

Two use cases show how to retrieve properties of selected managed objects with the `RetrievePropertiesEx` method.

A simple use case is to retrieve properties from a managed object for which a reference has been supplied. You need only create a `PropertyFilterSpec` that contains a `PropertySpec` to identify the property to retrieve and an `ObjectSpec` to identify the managed object. Pass this `PropertyFilterSpec` to the `RetrievePropertiesEx` method and get the property value from the structured result.

A more complicated use case retrieves properties from a set of managed objects found in the inventory hierarchy. For this use case, you create a `View` managed object that provides a path to all the target managed objects within one traversal step. You need to create a `PropertyFilterSpec` that contains a `PropertySpec` to identify the property to retrieve and an `ObjectSpec` that identifies the `View` object and also contains a `TraversalSpec` to describe the path from the `View` to its children. Pass this `PropertyFilterSpec` to the `RetrievePropertiesEx` method and get the property value from the structured result.

Tip:

Some retrieved information may not be up-to-date. To ensure you retrieve the most current data, perform a refresh or update operation on the relevant object before retrieving data. For example, to get the latest value for datastore free space, call `RefreshDatastore(refresh)` just before `RetrievePropertiesEx`.

Retrieve a Property from a Specified Managed Object

This procedure uses `PropertyCollector` to retrieve a property from a single `VirtualMachine` object with the `RetrievePropertiesEx` method, by obtaining a managed object reference to the parent object of the `VirtualMachine` in the inventory.

For this task you need:

- A virtual machine managed object reference in a variable named `vmRef`.
- An authenticated Web Services session with the vSphere server that manages the virtual machine.
- A `VimPort` binding provider referenced by the variable `methods`, which is attached to the vSphere server connection context.
- A `PropertyCollector` instance referenced by the variable `pCollector`.

Note:
This procedure shows only how to use the `PropertyCollector`. For a description of server connection and getting a reference to the `PropertyCollector`, see Build a Simple vSphere Client Application for the Web Services SDK.

To retrieve a reference to the parent object of a specific virtual machine, you must:

- Prepare a `PropertySpec` to specify the `parent` property and the `parentVApp` property of the `VirtualMachine` class. A virtual machine can have only one parent, but the parent can be either a `Folder` object or a `VirtualApp` object. This example collects both properties, one of which will be null.
- Prepare an `ObjectSpec` to identify the starting object, which is the specified `VirtualMachine`. There is no need for a `TraversalSpec` because the property belongs to the starting object itself. However, the `ObjectSpec.skip` property is set to `false`; a value of `true` would cause the `PropertyCollector` not to collect properties from the starting object.
- Assemble a `PropertyFilterSpec` from the prepared data.
- Invoke the `RetrievePropertiesEx` method.

1. Declare a function that accepts a virtual machine `MOref` and returns a `MOref` to its parent `object.in` a key-value pair.

```
private static DynamicProperty getVMParentProperty(ManagedObjectReference vmRef)
throws Exception {
```

2. Specify the properties for retrieval (`VirtualMachine.parent` and `VirtualMachine.parentVApp`).

   Because the `VirtualMachine` has two mutually exclusive parent properties, depending on the parent type, this code collects both properties.

```
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("parent");
pSpec.getPathSet().add("parentVApp");
```

3. Create an `ObjectSpec` to define the property collection.

   Use the `setObj` method to specify that the `vmRef` is the starting object for this property collection. Set the `skip` property to `false` to indicate that you want to collect properties from the starting object. Omit the `selectSet` property because the property collection does not need to traverse away from the starting object.

```
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(vmRef);
oSpec.setSkip(false);
```

4. Create a `PropertyFilterSpec` and add the object and property specs to it.

```
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
```

```
fSpec.getPropSet().add(pSpec);
```

5. Create a list for the filters and add the property filter spec to it.

```
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);
```

6. Use the filter spec to retrieve the property collection from the server.

```
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(pCollector,fSpecList,ro);
```

7. Unwrap the parent property.

```
if (props != null) {
  for (ObjectContent oc : props.getObjects()) {
    List<DynamicProperty> dps = oc.getPropSet();
    if (dps != null) {
      for (DynamicProperty dp : dps) {
        if (dp.getName().equals("parent") || dp.getName().equals("parentVApp")) {
          return dp;
        }
      }
    }
  }
}
System.out.println("Parent not found.");
throw new Exception();
}
```

The method returns a key-value pair, `name` and `val`, indicating the name of the parent property and a managed object reference to the parent object.

```
DynamicProperty parent = getVMParentProperty( vmRef );
if (parent.getName().equals("parentVApp")) {
  System.out.format("VApp MOref: %s", parent.getVal());
} else {
  System.out.format("Folder MOref: %s", parent.getVal());
}
```

Retrieve Properties from a Set of Managed Objects with a View

This procedure uses `PropertyCollector` with a `ContainerView` to retrieve properties from multiple `VirtualMachine` objects with the `RetrieveProperiesEx` method.

To collect the names of all virtual machines in the inventory, you must:

- Prepare a `ContainerView` that contains only the virtual machines.
- Prepare a `PropertySpec` to specify the `name` property of the `VirtualMachine` class.
- Prepare a `TraversalSpec` to specify the path from the view to its contents.
- Assemble a `PropertyFilterSpec` from the prepared data.
- Invoke the `RetrievePropertiesEx` method.

> **Note:**
> This procedure shows only how to use the `PropertyCollector`. For a description of server connection, see Build a Simple vSphere Client Application for the Web Services SDK.

To collect the names of all virtual machines in the inventory, use the following steps.

1. Get references to the `ViewManager` and the `PropertyCollector`.

   In the example, `sContent` is the variable for the `ServiceContent` data object. `sContent` provides the methods to retrieve the managed object references to the vSphere services.

```
ManagedObjectReference viewMgrRef = sContent.getViewManager();
```

```
ManagedObjectReference propColl = sContent.getPropertyCollector();
```

2. Create a container view for virtual machines.

   `methods` is the variable for the `VimPortType` object. `VimPortType` defines the Java methods that correspond to the vSphere API methods. The `createContainerView` parameters are as follows:

   - `container`: Selection starts within the inventory root folder.
   - `type`: Only `VirtualMachine` type managed objects are selected for the `ContainerView`, starting at the root folder.
   - `recursive`: The value `true` for the last parameter extends the selection beyond the children of the root folder so that the `ViewManager` will follow child folder paths to add virtual machines to the view.

   This container view provides references to all virtual machines in the inventory.

```
List<String> vmList = new ArrayList<String>();
vmList.add("VirtualMachine");

ManagedObjectReference cViewRef = methods.createContainerView(viewMgrRef,
    sContent.getRootFolder(),
    vmList,
    true );
```

3. Create an object specification to define the starting point for inventory navigation.

   The `ObjectSpec.obj` property identifies the starting object (the container view). This example collects only virtual machine data, so the `skip` property is set to `true` to ignore the container view itself during collection.

```
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(cViewRef);
oSpec.setSkip(true);
```

4. Create a traversal specification to identify the path for collection.

   The `TraversalSpec` properties `type` and `path` determine path traversal. `TraversalSpec.type` identifies the type of managed object that this spec can traverse. `TraversalSpec.path` identifies a property that links to other managed objects. The `PropertyCollector` uses the `path` object to select additional objects.

   This example uses a single `TraversalSpec` to walk the list of virtual machines that are available through the container view. The following code fragment specifies the `ContainerView` object for the `TraversalSpec.type` property and the `view` property in the `ContainerView` for the `TraversalSpec.path` property. The `skip` property is set to `false`, so the `PropertyCollector` will collect data from the `path` objects (the virtual machines in the container view).

```
TraversalSpec tSpec = new TraversalSpec();
tSpec.setName("traverseEntities");
tSpec.setPath("view");
tSpec.setSkip(false);
tSpec.setType("ContainerView");
```

5. Add the `TraversalSpec` to the `ObjectSpec.selectSet` array. The `TraversalSpec` tells the PropertyCollector how to traverse from the starting object to other managed objects.

```
oSpec.getSelectSet().add(tSpec);
```

6. Identify a managed object type and the properties to be retrieved from it.

   The example program creates a `PropertySpec` data object to specify the properties to be collected. The `type` property is set to a managed object type present in the container view. The `pathSet` property identifies one or more properties in the `type` object.

   This example specifies the `VirtualMachine.name` property.

```
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("name");
```

7. Add the object and property specifications to the property filter specification.

   A `PropertyFilterSpec` must have at least one `ObjectSpec` and one `PropertySpec`. The `PropertyFilterSpec` specifies what properties to collect, while the `ObjectSpec` tells the PropertyCollector how to find them.

```
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);
```

8. Create a list for the filters and add the `PropertyFilterSpec` to it.

   This example needs only one filter to collect the names of virtual machines.

```
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);
```

9. Retrieve the data.

   To invoke a single property collection operation, call the `RetrievePropertiesEx` method. The example application passes the populated `PropertyFilterSpec` and an empty `options` structure to the method. The default for the `RetrieveOptions.maxObjects` specifies no maximum for the number of objects that can be returned.

   Note:
   The `PropertyCollector` can impose a maximum in some circumstances. If the number of collected objects is greater than the maximum, the `PropertyCollector` returns a `token` value in the `RetrieveResult` data object and this token is used to retrieve the remaining properties using the `ContinueRetrievePropertiesEx` API method. For more information, see Server Data Transmission.

```
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(propColl,fSpecList,ro);
```

10. Print the virtual machine names.

    The following code fragment walks the list of `ObjectContent` objects returned in the `RetrieveResult` object. For each object (`ObjectContent`), the inner loop prints the name-value pairs.

```
if (props != null) {
   for (ObjectContent oc : props.getObjects()) {
   String vmName = null;
   String path = null;
   List<DynamicProperty> dps = oc.getPropSet();
   if (dps != null) {
      for (DynamicProperty dp : dps) {
         vmName = (String) dp.getVal();
         path = dp.getName();
         System.out.println(path + " = " + vmName);
      }
   }
}
```

You can also see these code fragments in the context of an end-to-end example that includes connection and authentication logic, in Build a Simple vSphere Client Application for the Web Services SDK.

Property Collector Program in Java for Web Services SDK

`PropertyCollector` provides a powerful, flexible, and efficient way to collect data from a vCenter or ESX host.

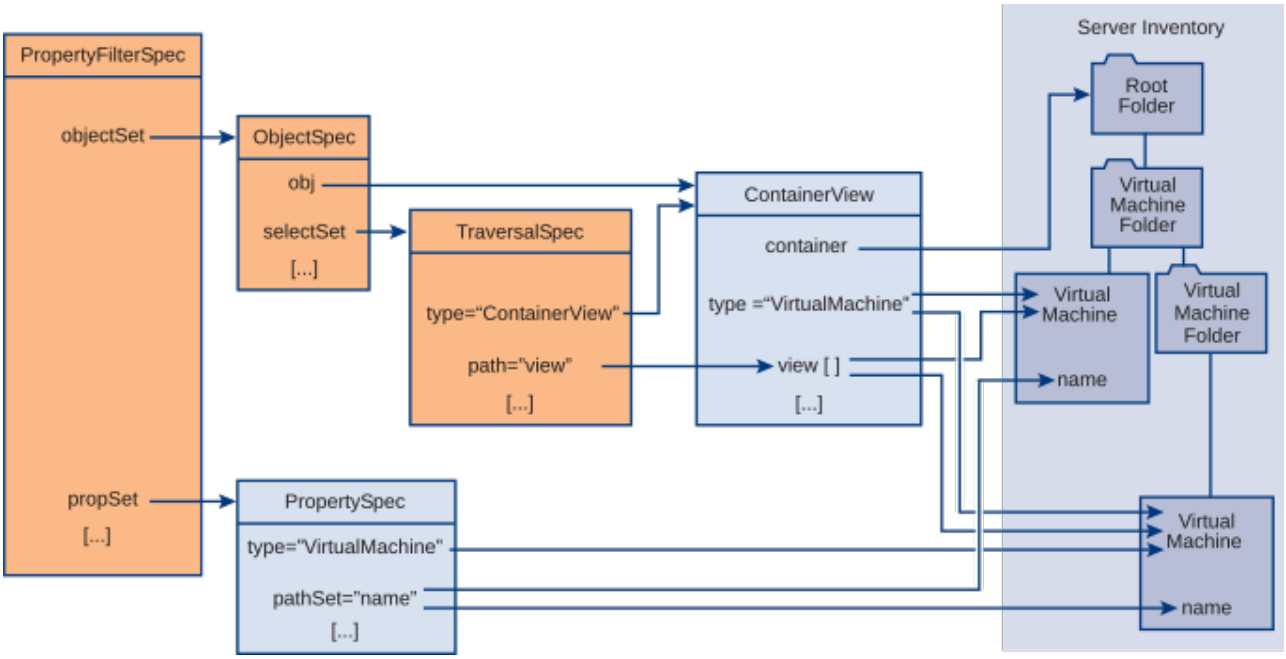
Simple PropertyCollector Example (Java) is a simple `PropertyCollector` example written in Java. The example uses a `ContainerView` for efficient access to the inventory and a `PropertyFilterSpec` that contains one `ObjectSpec`, one `TraversalSpec`, and one `PropertySpec`. The program performs the following tasks:

1. Accepts command line arguments for the vSphere server name (DNS name or IP address), user name, and password.
2. Connects to a vSphere server.
3. Uses a `ContainerView` to create a subset of the inventory; the subset contains only virtual machines.

4. Uses the `RetrievePropertiesEx` method for a single retrieval operation.
5. Collects the names of all of the virtual machines in the inventory and prints the names using the standard output stream.
6. Closes the connection to the server.

Figure 585: Property Filter Specification shows the objects used in Simple PropertyCollector Example (Java). The figure represents properties that identify inventory elements directly or indirectly. It does not show all the properties for the different objects.

Figure 585: Property Filter Specification



Simple PropertyCollector Example (Java)

```
import com.vmware.vim25.*;

import java.util.*;
import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLSession;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.soap.SOAPFaultException;


// PropertyCollector example
// command line input: server name, user name, password

public class PCollector {

  private static void collectProperties(VimPortType methods,
                                        ServiceContent sContent) throws Exception {
    // Get references to the ViewManager and PropertyCollector
    ManagedObjectReference viewMgrRef = sContent.getViewManager();
    ManagedObjectReference propColl = sContent.getPropertyCollector();

    // use a container view for virtual machines to define the traversal
    // - invoke the VimPortType method createContainerView (corresponds
    //   to the ViewManager method) - pass the ViewManager MOR and
    //   the other parameters required for the method invocation
```

```
// - createContainerView takes a string[] for the type parameter;
//   declare an arraylist and add the type string to it
List<String> vmList = new ArrayList<String>();
vmList.add("VirtualMachine");

ManagedObjectReference cViewRef = methods.createContainerView(viewMgrRef,
                                                               sContent.getRootFolder(),
                                                               vmList,
                                                               true);

// create an object spec to define the beginning of the traversal;
// container view is the root object for this traversal
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(cViewRef);
oSpec.setSkip(true);

// create a traversal spec to select all objects in the view
TraversalSpec tSpec = new TraversalSpec();
tSpec.setName("traverseEntities");
tSpec.setPath("view");
tSpec.setSkip(false);
tSpec.setType("ContainerView");

// add the traversal spec to the object spec;
// the accessor method (getSelectSet) returns a reference
// to the mapped XML representation of the list; using this
// reference to add the spec will update the list
oSpec.getSelectSet().add(tSpec);

// specify the property for retrieval (virtual machine name)
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("name");

// create a PropertyFilterSpec and add the object and
// property specs to it; use the getter method to reference
// the mapped XML representation of the lists and add the specs
// directly to the list
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);

// Create a list for the filters and add the spec to it
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);

// get the data from the server
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(propColl,fSpecList,ro);

// go through the returned list and print out the data
if (props != null) {
  for (ObjectContent oc : props.getObjects()) {
    String vmName = null;
    String path = null;
    List<DynamicProperty> dps = oc.getPropSet();
    if (dps != null) {
```

```
        for (DynamicProperty dp : dps) {
          vmName = (String) dp.getVal();
          path = dp.getName();
          System.out.println(path + " = " + vmName);
        }
      }
    }
  }
}//end collectProperties()

// Authentication is handled by using a TrustManager and supplying
// a host name verifier method. (The host name verifier is declared
// in the main function.)
//
// For the purposes of this example, this TrustManager implementation
// will accept all certificates. This is only appropriate for
// a development environment. Production code should implement certificate support.

private static class TrustAllTrustManager
        implements javax.net.ssl.TrustManager,
                   javax.net.ssl.X509TrustManager {
  public java.security.cert.X509Certificate[] getAcceptedIssuers() {
    return null;
  }

  public boolean isServerTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public void checkServerTrusted(java.security.cert.X509Certificate[] certs,
                                 String authType)
         throws java.security.cert.CertificateException {
    return;
  }

  public void checkClientTrusted(java.security.cert.X509Certificate[] certs,
                                 String authType)
         throws java.security.cert.CertificateException {
    return;
  }
}

public static void main(String [] args) throws Exception {

  // arglist variables
  String serverName = args[0];
  String userName = args[1];
  String password = args[2];
  String url = "https://"+serverName+"/sdk/vimService";
  // Variables of the following types  for access to the API methods
  // and to the vSphere inventory.
  // -- ManagedObjectReference for the ServiceInstance on the Server
  // -- VimService for access to the vSphere Web service
```

```
  // -- VimPortType for access to methods
  // -- ServiceContent for access to managed object services
  ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
  VimService vimService;
  VimPortType vimPort;
  ServiceContent serviceContent;

  // Declare a host name verifier that will automatically enable
  // the connection. The host name verifier is invoked during
  // the SSL handshake.
  HostnameVerifier hv = new HostnameVerifier() {
  public boolean verify(String urlHostName, SSLSession session) {
    return true;
  }
};

// Create the trust manager.
javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
trustAllCerts[0] = tm;
// Create the SSL context
javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");

// Create the session context
javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();

// Initialize the contexts; the session context takes the trust manager.
sslsc.setSessionTimeout(0);
sc.init(null, trustAllCerts, null);

// Use the default socket factory to create the socket for the secure connection
javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());

// Set the default host name verifier to enable the connection.
HttpsURLConnection.setDefaultHostnameVerifier(hv);

// Set up the manufactured managed object reference for the ServiceInstance
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");

// Create a VimService object to obtain a VimPort binding provider.
// The BindingProvider provides access to the protocol fields
// in request/response messages. Retrieve the request context
// which will be used for processing message requests.
vimService = new VimService();
vimPort = vimService.getVimPort();
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();

// Store the Server URL in the request context and specify true
// to maintain the connection between the client and server.
// The client API will include the Server's HTTP cookie in its
// requests to maintain the session. If you do not set this to true,
// the Server will start a new session with each request.
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

// Retrieve the ServiceContent object and login
```

```
        serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
        vimPort.login(serviceContent.getSessionManager(),
                      userName,
                      password,
                      null);

        // retrieve data
        collectProperties( vimPort, serviceContent );

        // close the connection
        vimPort.logout(serviceContent.getSessionManager());

        }
    }
```

Inventory Traversal

`PropertyCollector` can traverse the server inventory in arbitrary ways, enabling you to follow links between related objects.

Simple PropertyCollector Example (Java) uses a `ContainerView` to specify the objects that start the collection process. This is the simplest way to set up a filter, using a single reference to a view to provide the `PropertyCollector` with access to a set of objects. To locate objects in the inventory, a filter includes `TraversalSpec` and possibly `SelectionSpec` objects. Use these objects to direct the `PropertyCollector` to the managed objects and their properties named in the `PropertySpec` data objects of the `PropertyFilterSpec`.

TraversalSpec Traversal

Use a `TraversalSpec` object to identify a managed object type and a traversal property in that type. `TraversalSpec` contains type, path, and selection properties.

- `type` – identifies an inventory object type.
- `path` – specifies a managed object reference property in the `type` object. This property provides the traversal path extending from this object.
- `selectSet` – specifies an optional list of selection objects for additional object traversal paths. The `PropertyCollector` tries to apply the `TraversalSpec` objects in the `selectSet` array to the results of the traversal (the targets of `TraversalSpec.path`). If the type of a referenced managed object matches one of the specs in the `SelectSet` array, the `PropertyCollector` follows that `TraversalSpec` next. The `selectSet` array can also contain `SelectionSpec` objects; a `SelectionSpec` is a reference to a named `TraversalSpec`. See SelectionSpec Traversal.
- `skip` – indicates whether to collect properties for managed objects referenced by the `path` property.

During inventory traversal, the `PropertyCollector` visits managed objects referenced by the `TraversalSpec.path` property values. When the `PropertyCollector` visits an object, it collects properties if the object's type matches a `PropertySpec.type` value.

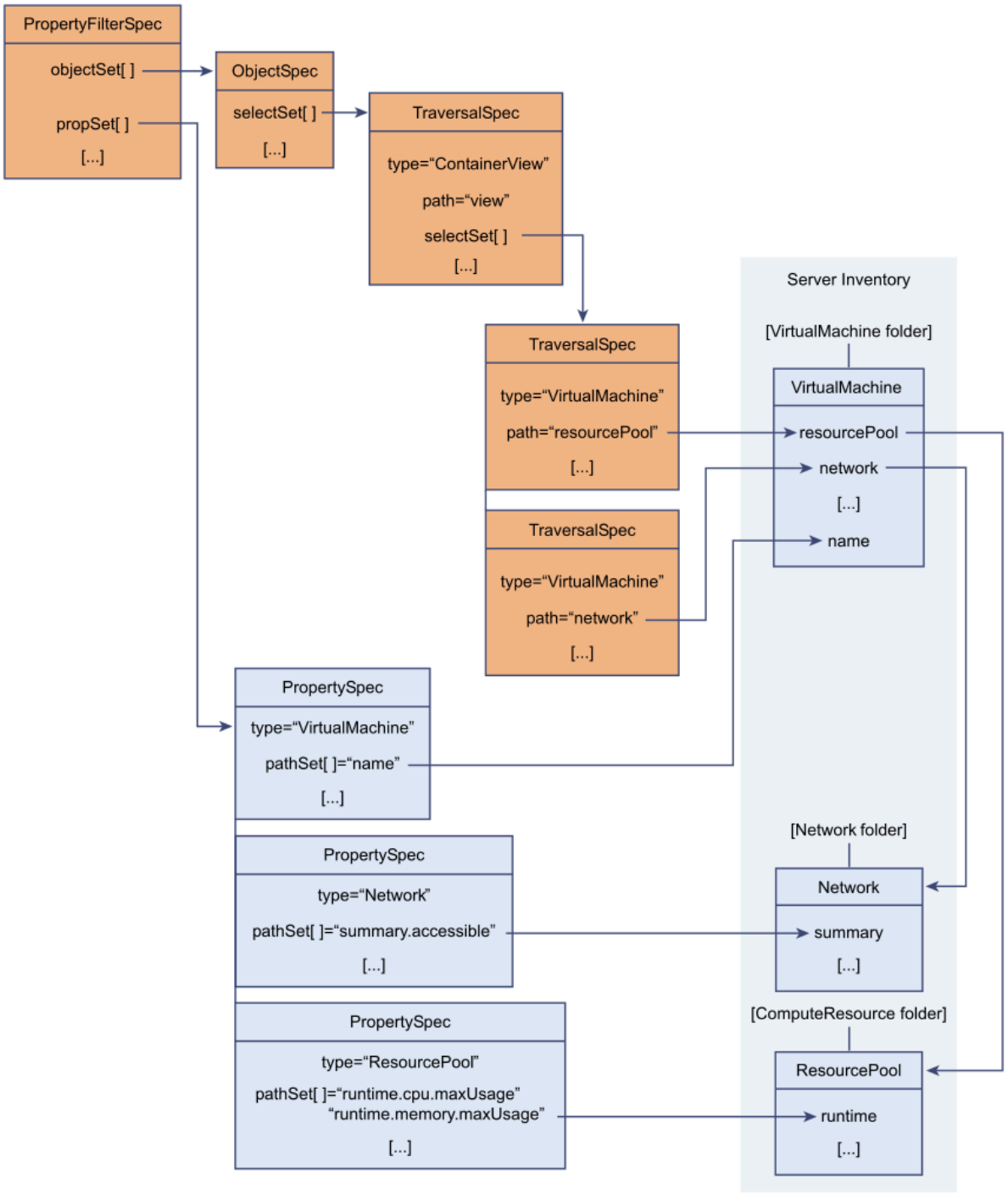
Traversal starts with the managed object referenced by `ObjectSpec.obj`, and the `PropertyCollector` also collects its properties if it matches a `PropertySpec` and if `ObjectSpec.skip` is `false`. Then the `PropertyCollector` follows the `path` property of each applicable `TraversalSpec` in the `ObjectSpec.selectSet` array. If the `TraversalSpec.skip` property is `false`, the `PropertyCollector` can also collect properties from the objects referenced by the `pathSet` property array.

Finally, the `PropertyCollector` checks the `TraversalSpec.selectSet` array for any traversal specs that apply to the referenced objects, and the traversal continues until no more traversal specs apply.

Figure 586: Inventory Navigation is a representation of a `PropertyFilterSpec` that defines traversal of `VirtualMachine` objects to access related `Network` objects.. The filter uses a `ContainerView` as a starting point. The `TraversalSpec` for the `ContainerView` specifies the `view` property for access to the view's virtual machines. The figure shows `TraversalSpec` objects that extend navigation from a `VirtualMachine` object to the associated `Network` and `ResourcePool` objects. The `PropertyCollector` applies these `TraversalSpec` objects to each of the `VirtualMachine`

objects in the view list. The figure also shows the `PropertySpec` objects for collecting data from `VirtualMachine`, `Network`, and `ResourcePool` objects.

Figure 586: Inventory Navigation



Traverse the Inventory By Using the Property Collector

To traverse inventory objects, you define one or more `TraversalSpec` objects. In each `TraversalSpec` object, you specify a property of one managed object that holds a reference to another managed object. The `TraversalSpec` defines a path that the PropertyCollector can follow between objects.

To define inventory traversal, use the following steps.

1. Create a `ContainerView` for virtual machines.
2. Create an `ObjectSpec` that uses the container view as the collection starting point.
3. Create a `TraversalSpec` to be applied to the `ContainerView` to select `VirtualMachine` objects.
4. Create additional `TraversalSpec` objects to select additional objects.

   The `SelectSet` list for the container view `TraversalSpec` has two `TraversalSpec` objects. Both specify a `VirtualMachine` object context. One object uses the `network` property to extend traversal to the `Network` managed object. The other uses the `resourcePool` property to extend traversal to the `ResourcePool` managed object.
5. Create `PropertySpec` objects to retrieve `VirtualMachine`, `Network`, and `ResourcePool` properties.

   To retrieve properties that are embedded in data objects, the `PropertySpec.PathSet` property uses dot notation to specify the property paths.

Inventory Traversal Example in Java

This example shows a complete Java program that collects and prints selected properties from the inventory, using traversal specs with a property collector. The example lists networks, resource pool status, and virtual machines.

Inventory Traversal in Java

```
import com.vmware.vim25.*;

import java.util.*;
import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLSession;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.soap.SOAPFaultException;


// PropertyCollector example
// command line input: server name, user name, password

public class PCollector_traversal {

  private static void collectProperties(VimPortType methods,
                                        ServiceContent sContent) throws Exception {

    // Get references to the ViewManager and PropertyCollector
    ManagedObjectReference viewMgrRef = sContent.getViewManager();
    ManagedObjectReference propColl = sContent.getPropertyCollector();

    // use a container view for virtual machines to define the traversal
    // - invoke the VimPortType method createContainerView (corresponds
    //   to the ViewManager method) - pass the ViewManager MOR and
    //   the other parameters required for the method invocation
    //   (use a List<String> for the type parameter's string[])
    List<String> vmList = new ArrayList<String>();
    vmList.add("VirtualMachine");

    ManagedObjectReference cViewRef =
          methods.createContainerView(viewMgrRef,
                                      sContent.getRootFolder(),
                                      vmList,
                                      true);

    // create an object spec to define the beginning of the traversal;
    // container view is the root object for this traversal
    ObjectSpec oSpec = new ObjectSpec();
```

```
oSpec.setObj(cViewRef);
oSpec.setSkip(true);

// create a traversal spec to select all objects in the view
TraversalSpec tSpec = new TraversalSpec();
tSpec.setName("traverseEntities");
tSpec.setPath("view");
tSpec.setSkip(false);
tSpec.setType("ContainerView");

// add the traversal spec to the object spec;
// the accessor method (getSelectSet) returns a reference
// to the mapped XML representation of the list; using this
// reference to add the spec will update the selectSet list
oSpec.getSelectSet().add(tSpec);

// extend from virtual machine to network
TraversalSpec tSpecVmN = new TraversalSpec();
tSpecVmN.setType("VirtualMachine");
tSpecVmN.setPath("network");
tSpecVmN.setSkip(false);

// extend from virtual machine to resourcepool
TraversalSpec tSpecVmRp = new TraversalSpec();
tSpecVmRp.setType("VirtualMachine");
tSpecVmRp.setPath("resourcePool");
tSpecVmRp.setSkip(false);

// add the network and resource pool traversal specs
// to the virtual machine traversal;
// the accessor method (getSelectSet) returns a reference
// to the mapped XML representation of the list; using this
// reference to add the spec will update the selectSet list
tSpec.getSelectSet().add(tSpecVmN);
tSpec.getSelectSet().add(tSpecVmRp);

// specify the properties for retrieval
// (virtual machine name, network summary accessible, rp runtime props);
// the accessor method (getPathSet) returns a reference to the mapped
// XML representation of the list; using this reference to add the
// property names will update the pathSet list
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("name");

PropertySpec pSpecNs = new PropertySpec();
pSpecNs.setType("Network");
pSpecNs.getPathSet().add("summary.accessible");

PropertySpec pSpecRPr = new PropertySpec();
pSpecRPr.setType("ResourcePool");
pSpecRPr.getPathSet().add("runtime.cpu.maxUsage");
pSpecRPr.getPathSet().add("runtime.memory.maxUsage");
pSpecRPr.getPathSet().add("runtime.overallStatus");

// create a PropertyFilterSpec and add the object and
// property specs to it; use the getter methods to reference
```

```
  // the mapped XML representation of the lists and add the specs
  // directly to the objectSet and propSet lists
  PropertyFilterSpec fSpec = new PropertyFilterSpec();
  fSpec.getObjectSet().add(oSpec);
  fSpec.getPropSet().add(pSpec);
  fSpec.getPropSet().add(pSpecNs);
  fSpec.getPropSet().add(pSpecRPr);

  // Create a list for the filters and add the spec to it
  List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
  fSpecList.add(fSpec);

  // get the data from the server
  RetrieveOptions ro = new RetrieveOptions();
  RetrieveResult props = methods.retrievePropertiesEx(propColl,fSpecList,ro);

  // go through the returned list and print out the data
  if (props != null) {
    for (ObjectContent oc : props.getObjects()) {
      String value = null;
      String path = null;
      List<DynamicProperty> dps = oc.getPropSet();
      if (dps != null) {
        for (DynamicProperty dp : dps) {
          path = dp.getName();
          if (path.equals("name")) {
            value = (String) dp.getVal();
          }
          else if (path.equals("summary.accessible")) {
            // summary.accessible is a boolean
            value = String.valueOf( dp.getVal() );
          }
          else if (path.equals("runtime.cpu.maxUsage")) {
            // runtime.cpu.maxUsage is an xsd:long
            value = String.valueOf( dp.getVal() );
          }
          else if (path.equals("runtime.memory.maxUsage")) {
            // runtime.memory.maxUsage is an xsd:long
            value = String.valueOf( dp.getVal() );
          }
          else if (path.equals("runtime.overallStatus")) {
            // runtime.overallStatus is a ManagedEntityStatus enum
            value = String.valueOf( dp.getVal() );
          }

          System.out.println(path + " = " + value);
        }
      }
    }
  }
}//end collectProperties()

// Authentication is handled by using a TrustManager and supplying
// a host name verifier method. (The host name verifier is declared
// in the main function.)
//
// For the purposes of this example, this TrustManager implementation
```

```
// will accept all certificates. This is only appropriate for
// a development environment. Production code should implement certificate support.
private static class TrustAllTrustManager
    implements javax.net.ssl.TrustManager,
               javax.net.ssl.X509TrustManager {

  public java.security.cert.X509Certificate[] getAcceptedIssuers() {
    return null;
  }

  public boolean isServerTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public void checkServerTrusted(java.security.cert.X509Certificate[] certs, String authType)
      throws java.security.cert.CertificateException {
    return;
  }

  public void checkClientTrusted(java.security.cert.X509Certificate[] certs, String authType)
      throws java.security.cert.CertificateException {
    return;
  }
}

public static void main(String [] args) throws Exception {

  // arglist variables
  String serverName = args[0];
  String userName = args[1];
  String password = args[2];
  String url = "https://"+serverName+"/sdk/vimService";

  // Variables of the following types  for access to the API methods
  // and to the vSphere inventory.
  // -- ManagedObjectReference for the ServiceInstance on the Server
  // -- VimService for access to the vSphere Web service
  // -- VimPortType for access to methods
  // -- ServiceContent for access to managed object services
  ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
  VimService vimService;
  VimPortType vimPort;
  ServiceContent serviceContent;

  // Declare a host name verifier that will automatically enable
  // the connection. The host name verifier is invoked during
  // the SSL handshake.
  HostnameVerifier hv = new HostnameVerifier() {
    public boolean verify(String urlHostName, SSLSession session) {
      return true;
    }
  };
```

```
        // Create the trust manager.
        javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
        javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
        trustAllCerts[0] = tm;

        // Create the SSL context
        javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");

        // Create the session context
        javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();

        // Initialize the contexts; the session context takes the trust manager.
        sslsc.setSessionTimeout(0);
        sc.init(null, trustAllCerts, null);

        // Use the default socket factory to create the socket for the secure connection
        javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());

        // Set the default host name verifier to enable the connection.
        HttpsURLConnection.setDefaultHostnameVerifier(hv);

        // Set up the manufactured managed object reference for the ServiceInstance
        SVC_INST_REF.setType("ServiceInstance");
        SVC_INST_REF.setValue("ServiceInstance");

        // Create a VimService object to obtain a VimPort binding provider.
        // The BindingProvider provides access to the protocol fields
        // in request/response messages. Retrieve the request context
        // which will be used for processing message requests.
        vimService = new VimService();
        vimPort = vimService.getVimPort();
        Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();

        // Store the Server URL in the request context and specify true
        // to maintain the connection between the client and server.
        // The client API will include the Server's HTTP cookie in its
        // requests to maintain the session. If you do not set this to true,
        // the Server will start a new session with each request.
        ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
        ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
        // Retrieve the ServiceContent object and login
        serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
        vimPort.login(serviceContent.getSessionManager(),
                      userName,
                      password,
                      null);

        // retrieve data
        collectProperties( vimPort, serviceContent );

        // close the connection
        vimPort.logout(serviceContent.getSessionManager());

    }
}
```

SelectionSpec Traversal

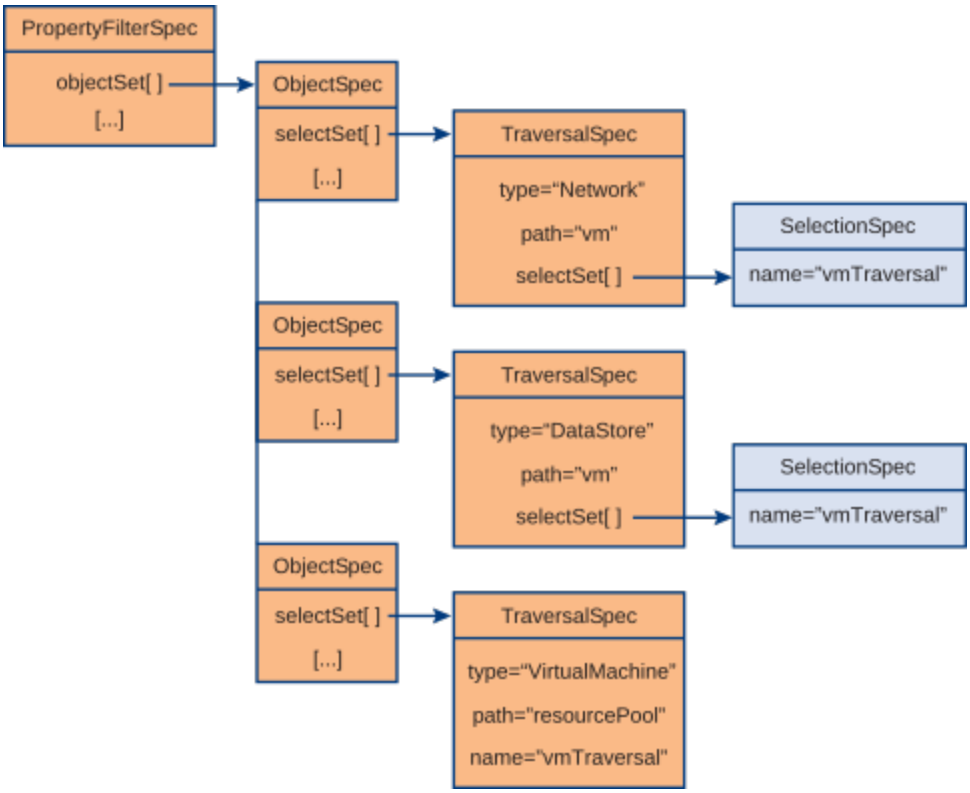
The `selectSet` array in `ObjectSpec` and `TraversalSpec` objects can include `TraversalSpec` objects and `SelectionSpec` objects. `SelectionSpec` is the base class for `TraversalSpec` objects. `SelectionSpec` defines the `name` property. You can use a `SelectionSpec` object in a `selectSet` array as a reference to a named `TraversalSpec` object. By using `SelectionSpec` references, you can reuse a `TraversalSpec` and you can define recursive traversal.

Simple Reference SelectionSpec

Use `SelectionSpec` references to avoid writing duplicate `TraversalSpec` declarations.

The `TraversalSpec` identified in a `SelectionSpec` reference must be within the same `PropertyFilterSpec`. Figure 587: SelectionSpec Reference shows the use of `SelectionSpec` references to a virtual machine `TraversalSpec`. The `SelectionSpec` references are associated with `Network` and `Datastore` traversals.

Figure 587: SelectionSpec Reference



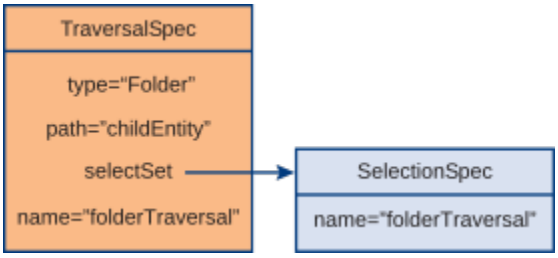
If the `ObjectSpec.selectSet` array contains a `SelectionSpec`, the referenced `TraversalSpec` must identify the same object type. `TraversalSpec.type` must match the type of the object specified in `ObjectSpec.obj`. The `PropertyCollector` applies the `TraversalSpec` to the object and uses the `TraversalSpec.path` property to extend its traversal.

Recursive Traversal

Use a `SelectionSpec` to apply a `TraversalSpec` to the results of its own traversal. To use a recursive filter construction, create a `SelectionSpec` that specifies the name of a `TraversalSpec` and add it to the named `TraversalSpec` selection set. The recursive construction extends inventory traversal beyond the paths directly represented by `TraversalSpec` objects.

You can use recursive traversal on any inventory objects that can be nested. See Inventory Hierarchies and ServiceInstance for a general representation of the structure of an inventory. For example, on a vCenter, folders can nest to arbitrary depths. To describe a traversal path through a succession of folders, you can add a `SelectionSpec` to the `Folder TraversalSpec`. The `SelectionSpec` must reference the `TraversalSpec`. Figure 588: Recursive TraversalSpec and SelectionSpec shows a representation of a `TraversalSpec` and its associated `SelectionSpec` for nested folder traversal.

Figure 588: Recursive TraversalSpec and SelectionSpec



Use Recursive TraversalSpec to Traverse Nested Inventory Folders

To traverse inventory objects that are nested to a variable depth, define one or more recursive `TraversalSpec` objects. In each `TraversalSpec` object, you specify a property of one managed object that holds a reference to another managed object. The `TraversalSpec` defines a path that the property collector can follow between objects.

To define recursive inventory traversal, use the following steps.

1. Use the `SearchIndex` managed object to retrieve the managed object reference for the top-level virtual machine folder.

   This folder is used as the beginning of the inventory traversal. For more information see SearchIndex.
2. Create an `ObjectSpec` object that references the top-level virtual machine folder.
3. Create a `SelectionSpec` object that references the `Folder TraversalSpec` by name.
4. Create a named `TraversalSpec` for `Folder` objects.

   The `TraversalSpec.path` property identifies the `Folder.childEntity` property for traversal to any child objects.
5. Add the `SelectionSpec` to the `TraversalSpec` to create the recursive filter.
6. Add the `TraversalSpec` to the `ObjectSpec`.
7. Create a `PropertySpec` for the `Folder` name.
8. Add the object and property specifications to the `PropertyFilterSpec`.
9. Call the `RetrievePropertiesEx` method.

Nested Folder Traversal in Java

This example shows the use of a recursive `TraversalSpec` with the property collector. The recursive traversal spec enables the property collector to navigate a chain of nested objects of the same type.

Nested Folder Traversal

```
import com.vmware.vim25.*;

import java.util.*;
import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLSession;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.soap.SOAPFaultException;

// PropertyCollector example
// command line input: server name, user name, password

public class nestedTraversal {

  private static void collectProperties(VimPortType methods,
                                        ServiceContent sContent)
                      throws Exception {
```

```
// Get reference to the PropertyCollector
ManagedObjectReference propColl = sContent.getPropertyCollector();

// get the top-level vm folder mor
ManagedObjectReference sIndex = sContent.getSearchIndex();
ManagedObjectReference rootVmFolder =
          methods.findByInventoryPath(sIndex,"datacenter1/vm");
// create an object spec to define the beginning of the traversal;
// root vm folder is the root object for this traversal
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(rootVmFolder);
oSpec.setSkip(true);

// folder traversal reference
SelectionSpec sSpecF = new SelectionSpec();
sSpecF.setName("traverseFolder");

// create a folder traversal spec to select childEntity
TraversalSpec tSpecF = new TraversalSpec();
tSpecF.setType("Folder");
tSpecF.setPath("childEntity");
tSpecF.setSkip(false);
tSpecF.setName("traverseFolder");

// use the SelectionSpec as a reflexive spec for the folder traversal;
// the accessor method (getSelectSet) returns a reference to the
// mapped XML representation of the list; using this reference
// to add the spec will update the list
tSpecF.getSelectSet().add(sSpecF);

// add folder traversal to object spec
oSpec.getSelectSet().add(tSpecF);

// specify the property for retrieval (folder name)
PropertySpec pSpec = new PropertySpec();
pSpec.setType("Folder");
pSpec.getPathSet().add("name");

// create a PropertyFilterSpec and add the object and
// property specs to it; use the getter method to reference
// the mapped XML representation of the lists and add the specs
// directly to the lists
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);

// Create a list for the filter and add the spec to it
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);

// get the data from the server
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(propColl,fSpecList,ro);

// go through the returned list and print out the data
if (props != null) {
  for (ObjectContent oc : props.getObjects()) {
```

```
      String folderName = null;
      String path = null;
      List<DynamicProperty> dps = oc.getPropSet();
      if (dps != null) {
        for (DynamicProperty dp : dps) {
          folderName = (String) dp.getVal();
          path = dp.getName();
          System.out.println(path + " = " + folderName);
        }
      }
    }
  }
}//end collectProperties()

// Authentication is handled by using a TrustManager and supplying
// a host name verifier method. (The host name verifier is declared
// in the main function.)
//
// For the purposes of this example, this TrustManager implementation
// will accept all certificates. This is only appropriate for
// a development environment. Production code should implement certificate support.
private static class TrustAllTrustManager
               implements javax.net.ssl.TrustManager,
               javax.net.ssl.X509TrustManager {

  public java.security.cert.X509Certificate[] getAcceptedIssuers() {
    return null;
  }

  public boolean isServerTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }

  public void checkServerTrusted(java.security.cert.X509Certificate[] certs, String authType)
      throws java.security.cert.CertificateException {
    return;
  }

  public void checkClientTrusted(java.security.cert.X509Certificate[] certs, String authType)
      throws java.security.cert.CertificateException {
    return;
  }
}

public static void main(String [] args) throws Exception {

  // arglist variables
  String serverName = args[0];
  String userName = args[1];
  String password = args[2];
  String url = "https://"+serverName+"/sdk/vimService";
  // Variables of the following types  for access to the API methods
  // and to the vSphere inventory.
```

```
// -- ManagedObjectReference for the ServiceInstance on the Server
// -- VimService for access to the vSphere Web service
// -- VimPortType for access to methods
// -- ServiceContent for access to managed object services
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
VimService vimService;
VimPortType vimPort;
ServiceContent serviceContent;

// Declare a host name verifier that will automatically enable
// the connection. The host name verifier is invoked during
// the SSL handshake.
HostnameVerifier hv = new HostnameVerifier() {
  public boolean verify(String urlHostName, SSLSession session) {
    return true;
  }
};

// Create the trust manager.
javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
trustAllCerts[0] = tm;

// Create the SSL context
javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");

// Create the session context
javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();

// Initialize the contexts; the session context takes the trust manager.
sslsc.setSessionTimeout(0);
sc.init(null, trustAllCerts, null);

// Use the default socket factory to create the socket for the secure connection
javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());

// Set the default host name verifier to enable the connection.
HttpsURLConnection.setDefaultHostnameVerifier(hv);

// Set up the manufactured managed object reference for the ServiceInstance
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");

// Create a VimService object to obtain a VimPort binding provider.
// The BindingProvider provides access to the protocol fields
// in request/response messages. Retrieve the request context
// which will be used for processing message requests.
vimService = new VimService();
vimPort = vimService.getVimPort();
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();

// Store the Server URL in the request context and specify true
// to maintain the connection between the client and server.
// The client API will include the Server's HTTP cookie in its
// requests to maintain the session. If you do not set this to true,
// the Server will start a new session with each request.
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
```

```
        ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

        // Retrieve the ServiceContent object and login
        serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
        vimPort.login(serviceContent.getSessionManager(),
                      userName,
                      password,
                      null);

        // retrieve data
        collectProperties( vimPort, serviceContent );

        // close the connection
        vimPort.logout(serviceContent.getSessionManager());

      }
    }
```

Client Data Synchronization with WaitForUpdatesEx

To maintain a client-side representation of server object state (by monitoring the properties for the inventory), use the `CreateFilter` and `WaitForUpdatesEx` methods. `WaitForUpdatesEx` supports an incremental retrieval model.

When the `WaitForUpdatesEx` call returns an `UpdateSet`, apply the updates as patches to the properties of managed objects in your client-side copy. In some circumstances, the server can return redundant updates. If the client response includes side effects in addition to updating client data, the additional actions should be idempotent.

The `WaitForUpdatesEx` call cannot guarantee synchronization between client and server in the presence of rapid server data changes, but when the server state is static calls to `WaitForUpdatesEx` result in eventual synchronization.

> **Note:**
> If you create additional instances of `PropertyCollector`, each instance uses independent version IDs for data synchronization. You should maintain a separate client copy of server data for each `PropertyCollector` instance when you use `WaitForUpdatesEx`.
>
> **Important:**
> The filters you use for incremental retrieval persist for the duration of the session or until you destroy them.

Property Filtering

A `PropertyCollector` can have one or more associated `PropertyFilter` objects. A `PropertyFilter` object has one associated `PropertyFilterSpec` object.

Some differences apply when filtering properties for `RetrievePropertiesEx` or for `WaitForUpdatesEx`.

When you call `RetrievePropertiesEx`, you supply a `specSet` parameter that can specify more than one `PropertyFilterSpec`. When you call `CreateFilter` you can specify only one `PropertyFilterSpec`.

A call to `CreateFilter` creates a persistent server-side filter belonging to the PropertyCollector. These filters are for use with successive calls to `WaitForUpdatesEx`, made for the same PropertyCollector instance. A call to `RetrievePropertiesEx` creates a set of nonpersistent filters that the server uses to select properties for the `RetrievePropertiesEx` call and for any associated `ContinueRetrievePropertesEx` calls made with the returned `token`.

To create persistent property filter specifications, use the `CreateFilter` method. When you call `CreateFilter`, you pass a `PropertyFilterSpec` object to the method. The method adds the new filter to the `PropertyCollector` associated with the method invocation and returns a reference to the new filter. After you have created the filter, you can add additional `PropertyFilterSpec` objects. You cannot share a filter with a `PropertyCollector` in another session.

A `RetrievePropertiesEx` call returns a change set consisting of the union of results for all the filter specifications in the `specSet` parameter of the call. A `WaitForUpdatesEx` call returns a change set consisting of individual results, one result set for each persistent `PropertyFilter` attached to the PropertyCollector.

You can call WaitForUpdatesEx when the PropertyCollector has no PropertyFilter objects, but the call will block until you create a filter or until the call times out.

WaitForUpdatesEx

The WaitForUpdatesEx method supports a polling mechanism for property collection that is based on a specified wait time.

Specify the following parameters when you call WaitForUpdatesEx:

- Managed object reference to a PropertyCollector instance.
- version value that identifies a sequence value. The first time you call WaitForUpdatesEx, specify an empty string ("") to retrieve a complete set of results for the specified properties. Your subsequent calls should use the version value returned in the previous call. If you don't include the version value, the server returns everything. For more information about data versions, see Server Data Transmission.
- options specifying the amount of data to transmit in a single response (the WaitOptions.maxObjectUpdates property) and the number of seconds the PropertyCollector should wait for updates (the WaitOptions.maxWaitSeconds property).

The value of the WaitOptions.maxWaitSeconds property determines whether the PropertyCollector uses an instant retrieval or a polling model. When you call WaitForUpdatesEx with a wait time of 0, it checks for updates and returns immediately. When you call WaitForUpdatesEx with a wait time greater than 0, the method waits until the specified time or until a change. WaitForUpdatesEx will not return a result until updates occur or until the call times out. The time-out is affected by the maxWaitSeconds value, the amount of time it takes to collect updated property values, and PropertyCollector policy.

If the property collection operation times out, and there are no updates to the requested properties, the PropertyCollector returns null for the WaitForUpdatesEx response.

- maxWaitSeconds is an optional property. If you do not specify a value, the PropertyCollector waits as long as possible for updates. Therefore, if maxWaitSeconds is unset, the waitForUpdatesEx method will cause the server to wait if there are no more changes to report, until the server times out the communication and drops the request. You can use maxWaitSeconds to specify a shorter time-out interval if the client-side firewall times out sooner than the server.
- maxWaitSeconds set to zero specifies an immediate call and response. The PropertyCollector checks for updates for all properties specified by the union of all filters associated with that instance of the PropertyCollector. The PropertyCollector returns any results, or null if there have been no updates.
- maxWaitSeconds greater than zero specifies how long the server should wait before timing out the request. If there are no updates available within maxWaitSeconds, WaitForUpdatesEx returns null instead of a set of updates.

Comparing Usage of MaxWaitSeconds

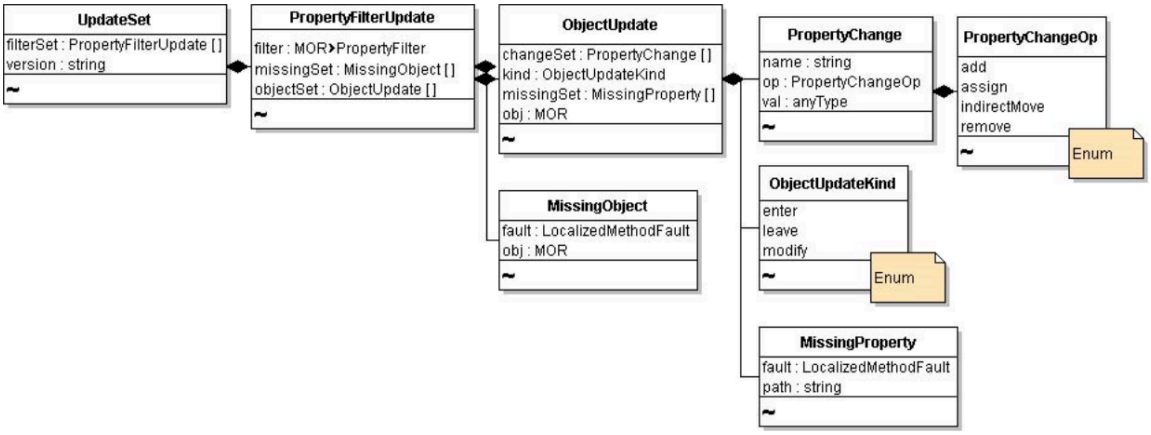
The following table lists some of the advantages and disadvantages of these two operations.

Operation	Advantages	Disadvantages
MaxWaitSeconds=0	Returns only properties that have changed since the version specified. Use this to poll for changes without waiting for changes.	Returns an empty set when nothing has changed on the server. Depending on your client application, this might be inefficient.
MaxWaitSeconds>0	Waits until an update occurs or a time-out occurs. Efficient use of network resources.	This can block a thread until updates occur. However, you can cancel this call while waiting, by using CancelWaitForUpdates.

UpdateSet Data Object from the Property Collector

The WaitForUpdatesEx method returns an UpdateSet data object, the composite data structure shown in the following figure.

Figure 589: UpdateSet Data Object Returned by WaitForUpdates Operations



Server Data Transmission

Property collection can involve the retrieval of large amounts of data, depending on the number of properties implied in the collection request. The server supports segmented data transmission, or chunking, when it sends collected data to a client. If the amount of collected data exceeds the chunk size, the server returns a chunk of data in a single response, and indicates additional data can be retrieved.

- The `WaitForUpdatesEx` method returns an `UpdateSet` data object. The `UpdateSet.truncated` property indicates whether the server has returned all the updates in its reply. The reply size can be limited by `options.maxObjectUpdates` in the call to `WaitForUpdatesEx`, or by server policy. On your first call to `WaitForUpdatesEx`, you can test `UpdateSet.truncated` in the response to see if all properties are present. If not, use `UpdateSet.version` in another call to `WaitForUpdatesEx`, repeating until `UpdateSet.truncated` is false. At that point, you can be sure that your client copy of the filtered properties is complete.
- The `RetrievePropertiesEx` method returns a `RetrieveResult` data object. The `RetrieveResult.token` property indicates whether you must call the `ContinueRetrievePropertiesEx` method to retrieve additional data. If the `token` property has a value, it identifies chunked data. When your client application receives an indication that additional data are available, it must send the returned token in the subsequent call to `ContinueRetrievePropertiexEx` to retrieve the next chunk of data.

Version strings and tokens are sequenced. Your client application must keep track of the sequence of values. If an error interrupts the collection operation, resume the operation by using the version string or token that was submitted before the interruption.

For information about chunk size, see descriptions of properties `RetrieveOptions.maxObjects` and `WaitOptions.maxObjectUpdates` in the *vSphere API Reference*.

> **Note:**
> If you resend a version string with `WaitForUpdatesEx`, you might receive more updates than the first time you used that version string. When you resend, the server repeats the data collection and more changes might have happened in between calls to `WaitForUpdatesEx`.

PropertyCollector Performance

Various factors affect the performance of a `PropertyCollector` for any given session.

- Number of objects
- Number of properties
- Density of property data (composite, nested data objects)
- Frequency of changes to the objects and properties on the server
- Depth of traversal (number of properties traversed)

In addition, a vSphere server is affected by the number of `PropertyCollector` instances and the number of filters each instance is supporting across all sessions on the server.

To minimize `PropertyCollector` overhead and the amount of network traffic for your client application, use `View` objects with the `PropertyCollector`. Simple PropertyCollector Example (Java) illustrates using views with the `PropertyCollector`.

SearchIndex

The `SearchIndex` managed object provides a set of methods to retrieve references to managed objects in the vSphere inventory. You can search by managed objects inventory path, IP address, datastore path, DNS name, and various other identifying attributes.

For example, if you know the IP address of a virtual machine, you can obtain its managed object reference by using the `SearchIndex.FindByIp` method. You can use `SearchIndex` to obtain the reference to a server object, and then use that reference as the starting object for property collection. See the sample applications `SearchIndex.java` and `SearchIndex.cs` for more information about using `SearchIndex`. See the *vSphere API Reference* for more information about `SearchIndex` methods.

Authentication and Authorization

VMware vSphere implements mechanisms to ensure that only valid users can access virtual infrastructure components.Each property and method in the API has an associated privilege requirement, and only uses with corresponding privileges can access the entities. This chapter discusses approaches to securing the system and the related service interfaces. The chapter also discusses the user model, which is different in ESX systems and vCenter systems.

See the *vSphere Datacenter Administration Guide* for a list of required privileges for common tasks and best practices for roles and permissions.

Objects for Authentication and Authorization Management

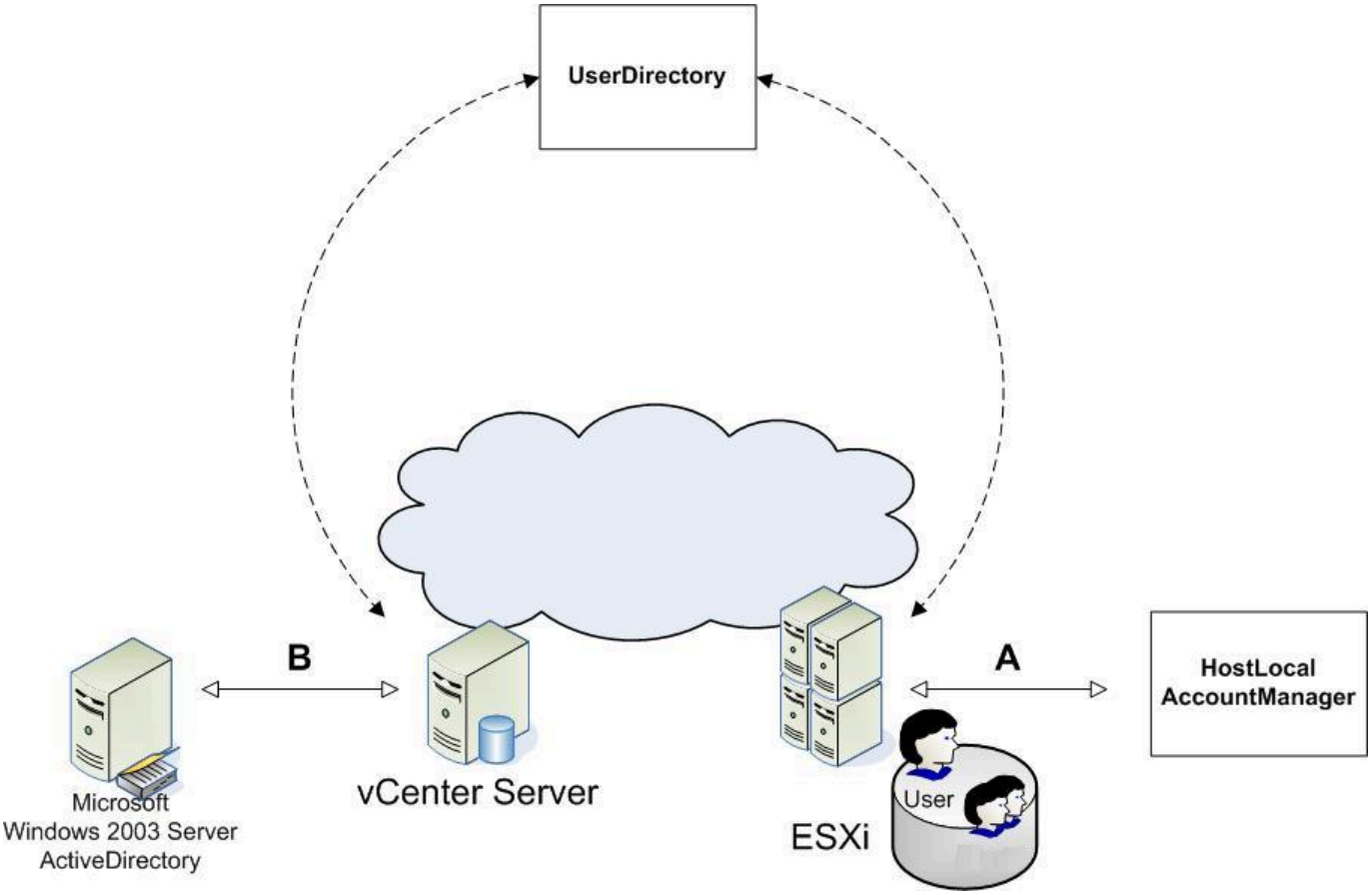
VMware vSphere includes the following interfaces for authenticating users and protecting virtual infrastructure components from unauthorized access:

- `HostLocalAccountManager` is used to create and manage user accounts on ESX systems. Authenticated users can view objects or invoke operations on the server depending on the permissions associated with their account. See Managing ESX Users with HostLocalAccountManager.
- `AuthorizationManager` protects vSphere components from unauthorized access. Access to components is role-based: Users are assigned roles that encompass the privileges needed to view and perform operations on vSphere objects. `AuthorizationManager` has operations for creating new roles, modifying roles, setting permissions on entities, and handling the relationship between managed objects and permissions.
- `UserDirectory` provides a look-up mechanism that returns user-account information to `AuthorizationManager` or to another requestor, such as a client application. See Obtaining User and Group Information from UserDirectory.
- `SessionManager` provides an interface to the authentication infrastructure on the target server system (see Authenticating Users Through SessionManager).
  - For vCenter systems, `SessionManager` supports single sign-on based on SSO tokens obtained from a VMware SSO Server. See Establishing a Single Sign-On Session with a vCenter.
  - For ESX systems, `SessionManager` supports authenticating user accounts as defined on the host system, such as accounts created using vSphere Client or accounts created programmatically through the `HostLocalAccountManager` API.
- Even if a user is authorized to perform operations on a vSphere object, the operation fails if the licenses for the host or the feature have not been assigned. You use `LicenseManager` and `LicenseAssignmentManager` to manage the licenses. See Managing Licenses with LicenseManager.

Authentication and Authorization for ESX and vCenter

Several server-side mechanisms authenticate a human user when a client application, such as the vSphere Client or a vSphere Web Services SDK application, connects to the server. Because ESX uses Linux-based authentication, and vCenter ran on Windows, the two systems can use different approaches for handling user accounts. The following illustration shows two different user management mechanisms associated with the VMware vSphere server.

Figure 590: Managed Objects for Handling User Accounts



These services work together to ensure that only authenticated users can connect to ESX or vCenter systems, and that they can access only those objects—folders, virtual machines, data centers, virtual services, and so on—for which they have the required privileges and which they are authorized to use or to view.

In addition, the vSphere Web Services SDK supports automated login through a credential store. See Using the Credential Store for Automated Login.

ESX User Model

When users enter their user account and credential from a client application, the server consults the appropriate user account store and validates the authenticity of the user account and the associated credential. Currently, the credential consists of a password, but vSphere also supports certificates, such as X.509 certificates. Authenticated users can then access objects they are authorized to use. Authentication succeeds if a user identity exists as a user account on the target system or in a supported directory service.

ESX leverages standard Linux infrastructure, including the Linux pluggable authentication module (PAM) mechanism for user account creation and management. The VMware authentication daemon (`vmware-authd`) is implemented as a PAM module. You can create and manage user accounts on an ESX system by using `HostLocalAccountManager`.

vCenter User Model

The pre-appliance vCenter was a Windows-based service using native Windows facilities and the Windows user model for identification and authentication. The vCenter Web service is associated with the Windows user account that was

logged in to the machine for vCenter installation. This vCenter administrator account was a member of the local Windows Administrator group on the machine.

VMware recommended creating a dedicated Windows user account for installing and managing the vCenter system. Other vCenter users who connect to the Web service also needed a Windows account on the local Administrator group.

> **Important:**
>
> Even if a user with the same name exists on an ESX host and a vCenter system, the two users have different accounts.

Organizations that are using Microsoft Active Directory can use the user identities contained in a Windows Server domain controller or Active Directory service across their virtual infrastructure. Microsoft Active Directory identities are supported for all clients that run vSphere Web Services SDK applications from Windows-based systems.

A vCenter client uses a SAML token to establish a single sign on session with the Server. See Establishing a Single Sign-On Session with a vCenter.

vSphere Security Model

Although the details of authentication and authorization differ between ESX and vCenter, the model itself is the same for both systems. It relies on privileges, roles, and permissions.

Privileges

A privilege is a system-defined requirement associated with a VMware vSphere object. Privileges are defined by VMware. Privileges are static, and do not change for a single version of a product. Each managed object has one or more privileges that a principal (user, group member) must have to invoke an operation or to view a property. For example, managed entities such as `Folder` and `VirtualMachine` require the principal to have the `System.Read` privilege on the entity to view the values of its properties.

The *vSphere API Reference* includes information about privileges required to invoke operations and to view properties on the **Required Privileges** labels on the documentation page for each managed object. Privileges for vSphere components are defined as follows:

```
<group>[.<group>].privilege
```

For example:

```
Datacenter.Create
Host.Config.Connection
Host.Config.Snmp
```

A privilege might be specific to vCenter or to ESX systems. For example, the `Alarm.Create` privilege is defined on vCenter. Setting alarms is done through the `AlarmManager` service interface, which requires a running vCenter system.

Privilege requirements apply to system objects regardless of how a given client application attempts to access server content (vSphere Client, CLI, or SDK). For example, you can use the following URL to access virtual machine datastore files:

```
https://<hostname>/folder[/<path>]/?dcPath=<datacenter_path>[&dsName=<datastore_name>]
```

The URL accesses a `Datastore` object in the inventory. You must have privileges to access each object in the hierarchy, corresponding to the elements of the URL.

Privileges for Datastore Objects in the Web Services API

The following table shows the privileges needed by methods that access datastore objects..

Object Associated with File	URL Element	Required Privileges
Root folder	`/folder`	`System.View`
Datacenter	`?dcPath`	`Datastore.Browse`

Object Associated with File	URL Element	Required Privileges
		Datastore.FileManagement
Datastore	&dsName	Datastore.Browse
		Datastore.FileManagement
Host	/host	Host.Config.AdvancedConfig
	/tmp/	Host.Config.SystemManagement

Roles
A role is a predefined set of privileges. Users are granted privileges to objects through roles.

When you assign user or group permissions, you pair the user or group with a role and associate that pairing with an inventory object. A single user might have different roles for different objects in the inventory.

For example, if you have two resource pools in your inventory, Pool A and Pool B, you might assign a particular user the role Virtual Machine User on Pool A and the role `ReadOnly` on Pool B. These assignments allow that user to turn on virtual machines in Pool A. In Pool B, the user can view the status of virtual machines, but cannot turn on virtual machines.

Permissions
In vSphere, a permission consists of a user or group and an assigned role for an inventory object, such as a virtual machine or ESX host. Permissions grant users the right to perform the activities specified by the role on the object to which the role is assigned.

For example, to configure memory for an ESX host, a user must be granted a role that includes the `Host.Configuration.Memory` privilege. By assigning different roles to users or groups for different objects, you can control the tasks that users can perform in your vSphere environment.

Many tasks require permissions on more than one object.

Setting Up Users, Groups, and Permissions
Setting up users, groups, and permissions consists of these tasks:

1. Get information about privilege requirements and privileges associated with system and sample roles.
   - Find out which operations on vSphere objects require which privileges. See the *API Reference*.
   - Find out which operations the system roles and sample roles can perform. See Description of Roles on vSphere Servers.
2. If necessary, create additional roles (sets of privileges). See Modifying Sample Roles to Create New Roles.
3. Retrieve information about existing users and groups (see Obtaining User and Group Information from UserDirectory) and create additional groups if needed.
4. Associate users or groups with roles using permissions. See Managing Roles and Permissions with AuthorizationManager.

At runtime, use `SessionManager` to log in to the server. vCenter instances support single sign-on sessions. To establish a single sign-on session, use the `SessionManager.LoginByToken` method. To establish a session with a standalone ESX host, use the `SessionManager.Login` method.

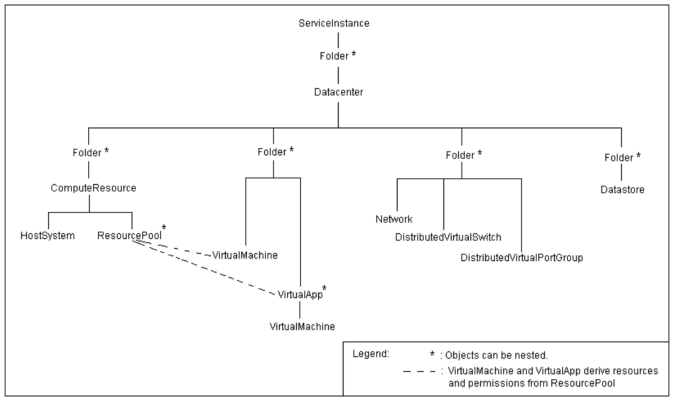
Obtaining User and Group Information from UserDirectory

The `UserDirectory` managed object allows a client application to obtain information about users and groups on a VMware vSphere server. Properties and results vary, depending on whether the server is a vCenter instance or an ESX system.

- vCenter system. Domain controller, Active Directory, or local Windows account repository.
- ESX host. Linux password file in `/etc/passwd` on the host.

For example, vCenter user accounts can be managed in a Windows Active Directory server or domain controller from which the `domainList` property of `UserDirectory` is derived. For ESX systems, the `domainList` property is empty.

Figure 591: UserDirectory Managed Object



RetrieveUserGroups Method

`UserDirectory` allows you to obtain information about users and groups using the `RetrieveUserGroups` method. The method can obtain a list of all user accounts from the host, and can search for specific users or groups based on specific criteria to filter the results. You can search by user name, by group name, for an exact match, or for a partial string (substring).

- ESX does not support local user groups, so this method will not return group information for a host. This method will return information about Active Directory groups.
- For ESX systems, search returns all users from the `passwd` file. If this file contains Network Information System (NIS) or NIS+ users, `RetrieveUserGroups` returns these accounts as well.
- For vCenter search is limited to the specified Windows domain. If the domain is omitted, the search is performed on local users and groups.

  Important:

  Do not configure an ESX system to use NIS or NIS+, unless it is acceptable to have NIS (or NIS+) user information available through the `UserDirectory.RetrieveUserGroups` API.

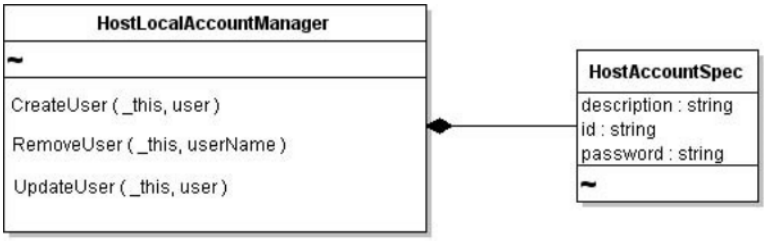
Managing ESX Users with HostLocalAccountManager

The `HostLocalAccountManager` managed object supports user administration tasks. `HostLocalAccountManager` is available only on ESX system.

Important:

vCenter systems use different methods. See vCenter User Model.

Figure 592: HostLocalAccountManager Managed Object



Methods Available for Local Account Management

`HostLocalAccountManager` provides the following methods for local user account management:

- `CreateUser`
- `RemoveUser`
- `UpdateUser`

These methods accept a `HostAccountSpec` data object. Specify the object properties according to the requirements on the target system. Examples of user account requirements are password length requirements and restricted use of dictionary words.

Create a Local User Account on an ESX System

You can use the API to create a user account on an ESX system.

1. Obtain a managed object reference to the `HostLocalAccountManager` of the target system.
2. Create a `HostAccountSpec` data object that defines the properties of the user account, including description and password.

   Define account names and passwords according to the configuration required by your ESX system for user account naming conventions and password requirements, such as minimum length, character set, and other requirements.
3. Call the `HostLocalAccountManager.CreateUserAccount` method, passing in the managed object reference (from step 1) and the `HostAccountSpec` data object (step 2).

   After creating user accounts on the ESX system, you can grant these users access to virtual components by using `AuthorizationManager` methods. See Managing Roles and Permissions with AuthorizationManager.

Managing Roles and Permissions with AuthorizationManager

`AuthorizationManager` is the service interface for handling permissions and roles assigned to the users and groups you define with `HostLocalAccountManager`. `AuthorizationManager` methods allow you to create, modify, and manage roles and permissions, and to obtain information about the roles and permissions defined in the system. If a predefined role does not meet your needs, define a new one that contains only the minimum set of required privileges.

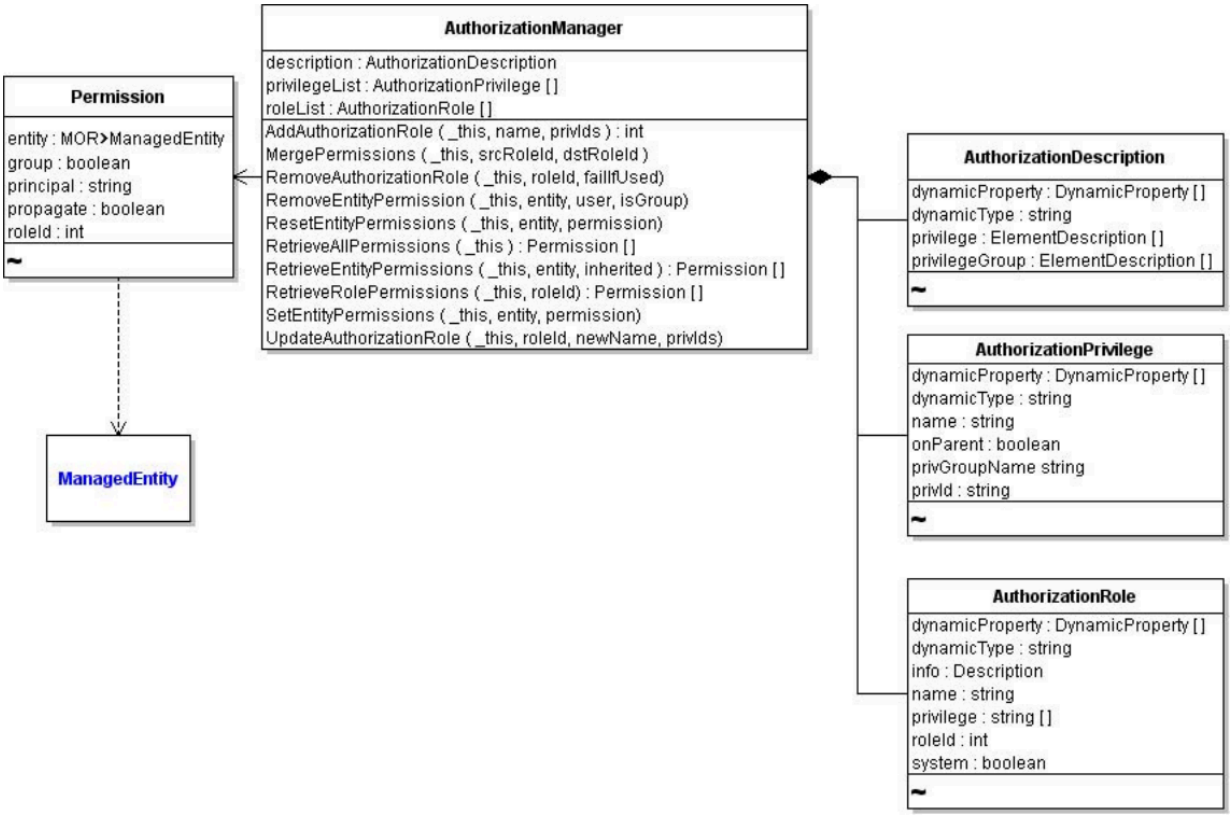
The `AuthorizationManager` also allows access and prevents access to specific server objects based on the permissions associated with the object.

`AuthorizationManager` includes methods for managing roles and for managing permissions:

- Roles Management. `AddAuthorizationRole`, `RemoveAuthorizationRole`, and `UpdateAuthorizationRole`. See Using Roles to Consolidate Sets of Privileges and Modifying Sample Roles to Create New Roles.
- Permissions Management. `MergePermissions`, `RemoveEntityPermission`, `ResetEntityPermissions`, `RetrieveAllPermissions`, `RetrieveEntityPermissions`, `RetrieveRolePermissions`, and `SetEntityPermissions`. See Granting Privileges Through Permissions.

The following diagram shows these methods in a UML diagram for `AuthorizationManager` and some of its associated data objects.

Figure 593: AuthorizationManager Managed Object



`AuthorizationManager` properties allow access to information. For example:

- The `privilegeList` property returns a list of all privileges defined on the system, as an array of `AuthorizationPrivilege` data objects. Privileges are defined by VMware, on the objects and properties contained in the system. These privileges are fixed and cannot be changed by client applications.
- The `roleList` property returns a list of all currently defined roles, including the system-defined roles, as an array of `AuthorizationRole` data objects.

Using Roles to Consolidate Sets of Privileges

A role is a named set of one or more privileges. A role is normally defined for a group of people who have common responsibilities in the system, for example, administrators. Each role can have zero to multiple privileges. ESX defines system roles and user roles.

- System roles. Cannot be modified or deleted.
- User roles. Apply to different user communities or restrict access for add-on tools. Several predefined user roles are included with vCenter and with ESX systems. You can create new roles using these predefined user roles as a starting point.

Description of Roles on vSphere Servers

The following table describes system roles and user roles in more detail and lists currently available roles as examples.

Type	Role name	Role ID	Description
System Roles	Administrator	-1	Superuser access. Encompasses the set of all defined privileges. This role cannot be deleted. By default, the Administrator role is granted to the user or group that owns the root node.
	Anonymous	-4	Cannot be granted. Default access role associated with any user account that has logged in.

Type	Role name	Role ID	Description
	No Access	-5	No access. Explicitly denies access to the user or group with this role. Assigning this role to a user account prevents the user from seeing any objects. Use the `No Access` role to mask subobjects under a higher-level object that has propagated permissions defined.
	Read-Only	-2	Read-only access. Encompasses the set of all nonmutable privileges. (`System.Anonymous`, `System.Read`, and `System.View`). Equivalent to a user role with no permissions. Users with this role can read data or properties and call query methods, but cannot make changes to the system.
	View	-3	Visibility access consisting of `System.Anonymous` and `System.View` privileges. Cannot be granted.
Sample Roles	Virtual Machine Administrator	1	Set of privileges necessary to manage virtual machines and hosts within the system.
	Datacenter Administrator	2	Set of privileges necessary to manage resources, but not interact with virtual machines.
	Virtual Machine Provider	3	Set of privileges necessary to provision resources.
	Virtual Machine Power User	4	Set of privileges for a virtual machine user that can also make configuration changes and create new virtual machines.
	Virtual Machine User	5	Set of privileges necessary to use virtual machines only. Cannot reconfigure virtual machines.
	ResourcePool Administrator	6	Available on vCenter systems only.
	VMware Consolidated Backup Utility	7	Available on vCenter systems only. Set of privileges necessary to run the Consolidated Backup Utility.

Modifying Sample Roles to Create New Roles

The system roles cannot be modified or deleted. However, you can create new roles, or modify the sample roles.

To create new roles using the API, use the following steps.

1. Starting with the `ServiceContent` object in `ServiceInstance.content`, obtain a managed object reference to the `AuthorizationManager` for the server.
2. Invoke the `AddAuthorizationRole` method. Parameters are a reference to `AuthorizationManager`, a name for the role (as a string), and an array of privileges (array of strings) that should be assigned to the role.

   `AddAuthorizationRole` returns an integer (`xsd:int`) value for the `roleId` that the system assigns to the newly defined role.
3. In subsequent code, use the `roleID` to assign the role to specific users or groups.

Granting Privileges Through Permissions

When you use one of the `AuthorizationManager` objects to assign or modify permissions, you use a `Permission` data object. `Permission` associates a principal with a set of privileges. A permission identifies:

- The user or group (`principal`) to which the permission applies.
- The role containing the privileges that should be granted to the user or group.
- The managed object reference to the entity to which the permission applies.

Every managed entity has at least one `Permission` object associated with it. A managed entity can have more than one `Permission` assigned to it, effectively granting different privileges to different users or groups. Permissions are defined for managed entities either explicitly or through inheritance.

Obtaining Information About Permissions

Users with the Administrator role can obtain information about `Permission` objects at different levels of detail.

- For an array of `Permission` objects, call the `AuthorizationManager.RetrieveAllPermissions` method.
- For specific inventory objects, such as managed entities, folders, datacenters, or virtual services, call the `AuthorizationManager.RetrieveEntityPermissions` method.
- For a role defined in the system, call the `AuthorizationManager.RetrieveRolePermissions` method.

See the *vSphere API Reference*.

Setting, Changing, or Deleting Permissions

The `Permission` data object associates the privileges required to perform an action on an object with the principals (user, group). Principals have privileges through their role. To set or update permissions on an object, use the `AuthorizationManager.SetEntityPermissions` method.

To set permissions on an entity, use the following steps.

1. Obtain a reference to the `AuthorizationManager` for the server from the `ServiceContent` object associated with the `ServiceInstance`. For example:

   ```
   ManagedObjectReference hostAuthorizationManager = service.getAuthorizationManager();
   ```

2. Create a `Permission` data object that identifies the user (or group) name, the role, the entity to which the permission should apply, and whether the permission should be applied to the entity's children.

   For example, the following code fragment creates a permission on the root folder of the inventory granting a user Administrator role to the root folder and all its children.

   ```
   Permission per = new Permission();
   per.setGroup(false);
   per.setPrincipal("new_user_name");
   per.setRoleId(-1);
   per.setPropagate(true);
   per.setEntity(rootFolder);
   ```

   Permissions cannot be set directly on children in a complex entity. For complex entities, set permissions on the parent entity and set the `propagate` flag to true to apply permissions to the child entities.

   To replace existing permissions with a new set of permissions, use the `AuthorizationManager.ResetEntityPermissions` method.

Impact of Group Membership on Permissions

Users can be members of multiple groups. The system handles multigroup membership as follows:

- Permissions are applied to inventory objects from the containing object to each of its child entities.
- If a user has no explicit user-level permissions, group-level permissions apply as if granted to the user directly.
- Membership in multiple groups with permissions on the same object results in a union of permissions.
- User-level permissions always take precedence over group-level permissions.

Applying Permission to a Managed Entity

The following example shows some of the code required to create a user account and apply a permission to an entity that grants access to the user account based on a role.

The role with role ID 4, assigned in this example, is defined as a "Virtual Machine Power User." The sample uses `AuthorizationManager` to grant permissions to the user and to associate the permission with the managed entity in the inventory—in this example, the `rootFolder`. The example uses the `apputil` helper classes to access the objects.

Creating a User Account

```
...
ManagedObjectReference _authManRef = _sic.getAuthorizationManager();
public class CreateUser {
private static AppUtil appUtil= null;
private void createUser() throws Exception {
```

```
ManagedObjectReference hostLocalAccountManager =
 appUtil.getConnection().getServiceContent().getAccountManager();
ManagedObjectReference hostAuthorizationManager =
     appUtil.getConnection().getServiceContent().getAuthorizationManager();

// Create a user
HostAccountSpec hostAccountSpec = new HostAccountSpec();
hostAccountSpec.setId(userName);
hostAccountSpec.setPassword(password);
hostAccountSpec.setDescription("my delegated admin auto-agent software");
appUtil.getConnection().getService().createUser(hostLocalAccountManager, hostAccountSpec);
ManagedObjectReference rootFolder = appUtil.getConnection().getServiceContent().getRootFolder();
Permission permission = new Permission();
permission.setGroup(false);
permission.setPrincipal(userName);

// Assign the Virtual Machine Power User role
permission.setRoleId(4);
permission.setPropagate(true);
permission.setEntity(rootFolder);
appUtil.getConnection().getService().setEntityPermissions(hostAuthorizationManager, rootFolder,
     new Permission [] {permission});
...
```

Authenticating Users Through SessionManager

The `SessionManager` managed object controls user access to the server. `SessionManager` includes methods for logging in to the server, obtaining a session, and logging out. The `SessionManager` defines the lifetime and visibility of many objects. Session-specific objects are not visible outside the session in which they are created.

> **Important:**
>
> Each user session uses system resources and creates locks on the server side. Too many concurrent sessions can slow down the server. By default, vCenter terminates a session after 30 minutes.

Upon successful authentication of a user account, `SessionManager` returns a `UserSession` data object to the client application. The session is associated with that user account for the duration of the session. The client application can save the session locally, to a secure file, and reuse the session later to reconnect to the server. You can also configure an ESX or vCenter system to support local sessions, which enable users with credentials on the host to log in based on those privileges.

The `SessionManager` provides these capabilities:

- Log in and log out. Basic operations to log in to ESX or vCenter system, obtain a session, and log out. When a session terminates, all session-specific objects are destroyed.
- Impersonation. One user session adopts the authorization level of another user session. Impersonation is common in Web based scenarios in which a middle-tier application functions as a central account that interacts with other back-end servers or processes. Windows services impersonate a client when accessing resources on behalf of the client. `SesssionManager` supports impersonation through its `ImpersonateUser` method.
- Delegation. A client application that is running on behalf of a local user can call the `SessionManager.AcquireLocalTicket` method to obtain a one-time user name and password for logging in. Delegation is useful for host-based utilities that run in the local console.

If the user account associated with the session does not have the permissions required to perform an action, the `AuthorizationManager` returns a `NoPermission` fault to the client application.

Using VMware Single Sign On for vCenter Sessions

vSphere supports single sign on for a single point of authentication for vCenter clients. To use VMware Single Sign On, your vSphere Web Services SDK client connects to the VMware SSO Server to obtain an SSO token. Your client includes the token in the `SessionManager.LoginByToken` request to start a vSphere session.

For more information about creating a session, see Establishing a Single Sign-On Session with a vCenter.

Using the Credential Store for Automated Login

To facilitate automated login for unattended applications, the vSphere Web Services SDK includes client-side credential store libraries and tools for automating the login process in a more secure manner. The libraries eliminate the need for system administrators to keep passwords in local scripts.

> **Important:**
> These libraries are built on top of the vSphere Web Services SDK.

The credential store has the following components:

- A persistence file (credential store backing file) that stores authentication credentials. Currently, only passwords are supported. The persistence file maps a remote user account from an ESX host to the password for that user on the server.
- Java and Perl libraries for managing the credential store programmatically. See Credential Store Methods for available methods.
- Java and Microsoft PowerShell-based command-line utilities for managing the credential store.

In addition to the libraries listed in Credential Store Libraries, the vSphere Web Services SDK includes the `CredentialStoreAdmin` tool for creating, examining, and managing the credential store. You can use the tool to examine the contents of the credential store, for example, the generated user accounts and passwords.

If you use these credential store client libraries in an application, you must set up the credential store on all client machines that run your application.

Credential Store Libraries

The following table shows the credential management libraries for Java.

Package com.vmware.security.credstore (Java)
CredentialStore.java
CredentialStoreFactory.java

Several of the helper classes provided with the sample applications use the credential store mechanism.

Credential Store Methods

The following table shows the methods used in Java clients for managing credentials.

Java	Description
`addPassword(hostname, username, password)`	Stores the password for the specified host and user. Overwrites any existing password for that user in the credential store. Creates the default credential store backing file in the default location (if it does not exist).
`removePassword(hostname, username)`	Deletes the password for the specified user from the credential store.
`clearPasswords()`	Deletes all passwords from the credential store.

Java	Description
`getPassword(hostname, username)`	Returns the password for the specified host and user from the credential store.
`getHosts()`	Returns the set of hosts contained in the credential store.
`getUsernames(hostname)`	Returns the collection of all user names that have passwords stored for the specified `hostname`.
`close()`	Closes the credential store, preventing further method invocations. Releases associated resources.

Credential Store Backing File

The credential store backing file is an XML file that is saved locally on the client machine for access at runtime. Unless otherwise specified, the backing file is located in the following location:

- Linux. `$HOME/.vmware/credstore/vicredentials.xml`
- Windows Vista and later. `C:\Users\[user_name]\AppData\Roaming\VMware\credstore\vicredentials.xml`
- Windows XP and Windows 2000. `C:\Documents and Settings\[user_name]\Application Data\VMware\credstore\vicredentials.xml`

The credential store persists locally on a per-user basis—each user has his or her own credential store backing file.

> **CAUTION:**
> The credential store backing files use filesystem-level permissions to ensure that passwords remain confidential. Protect the credential store backing file with appropriate file permissions.

Credential Store File Format shows the XML elements that are read and written to the file.

Credential Store File Format

```
<?xml version="1.0" encoding="UTF-8"?>
     <viCredentials>
          <version>1.0</version>
          <passwordEntry>
              <host>mi6.vmware.com</host>
              <username>agent007</username>
              <password>IhWS1saIhtsw2FbIh0w2F2...</password>
          </passwordEntry>
          <passwordEntry>
              ...
          </passwordEntry>
          ...
     </viCredentials>
```

Credential Store Samples

The `CreateUser` and `SimpleAgent` sample applications demonstrate how to use the credential store client libraries.

- The `CreateUser` sample creates a user account and password for the server based on random-number-generation scheme. The sample populates the local credential store backing file with this information. If the backing file does not exist, it is created in the default location.

  When you run `CreateUser`, specify the name of an ESX system, and an administrator user name and password. A user account name and password are created on the server. Specify `--ignorecert` unless your system has a secure connection to the target. Do not use `--ignorecert` in a production environment.

  ```
  java com.vmware.samples.simpleagent.CreateUser --server <servername> --url
  ```

```
https://<servername>/sdk --username <adminuser> --password <pwd> --ignorecert ignorecert
```

CAUTION:
The `CreateUser` sample application is for demonstration purposes only and should not be used as a model for production code. The sample breaks the principle of least privilege by granting the user account the Administrator role (-1). Never do this in a production environment.

- The `SimpleAgent` sample application demonstrates how to use credential store libraries to extract the user account and password at runtime to authenticate a user noninteractively.

```
java com.vmware.samples.simpleagent.SimpleAgent <servername>
```

Specifying Roles and Users with the Credential Store
VMware recommends that you apply the principle of least privilege to any agent-like software or automated application that uses the credential store in a production environment. Give user accounts the minimal number of privileges on the system that they require to do their jobs.

Specify roles and users as follows:

1. For each SDK-based application, use one specific role, newly created or predefined, that has appropriate privileges.

   For example, if you are developing an agent-like application to automatically start the VMware Consolidated Backup utility, you might use the “VMware Consolidated Backup Utility” role (roleID 7).

   If no predefined user role that meets the needs of your application exists, create a role with only those privileges needed for the application. See Using Roles to Consolidate Sets of Privileges for more information about roles.
2. Create a user account for use with the agent or application.
3. Apply the role created in Step 1 to the user account created in Step 2.
4. Store the user account and password in the credential store, using the `CredentialStoreAdministration` tool.

   Never grant administrator privileges to a user account associated with an automated script or software agent, especially one that uses the credential store.

Managing Licenses with LicenseManager

When you want to perform tasks in the vSphere environment, you must have licenses to do so. Licensing applies to ESX hosts, vCenter, and special features such as VMware HA or VMware vMotion.

The *vSphere Datacenter Administration Guide* explains how to manage ESX and vCenter licenses using the vSphere Client, and gives background information about license keys, license inventory, and related topics.

You can also manage licenses using the `LicenseManager` and `LicenseAssignmentManager` managed objects. You use `LicenseManager` to explicitly manage the pool of available licenses on ESX systems released before vSphere 4.0. You use `LicenseAssignmentManager`, available through the `LicenseManager.licenseAssignmentManager` property, to manage assignment of licenses to entities in the vCenter inventory. You can retrieve information, add licenses, and remove licenses.

Retrieve Information

- Retrieve the `LicenseManager.evaluation` and `LicenseManager.licenses` properties to obtain information on evaluation licenses and full licenses.
- Call `LicenseManager.DecodeLicense` to decode license information. The call returns a `LicenseManagerLicenseInfo` data object, which encapsulates information about the license.
- Call `LicenseAssignmentManager.QueryAssignedLicenses` for information about assigned licenses.

Add Licenses

- Call `LicenseManager.AddLicense`, passing in a license key, to add a license to the inventory of available licenses.
- Call `LicenseAssignmentManager.UpdateAssignedLicense`, passing in a license key, to update the licenses for an entity, for example, a host system.

Remove Licenses

- Call `LicenseAssignmentManager.RemoveAssignedLicense` to remove all licenses from an entity, passing in an entity to remove licenses from. You can then assign those licenses to other entities.
- Call `LicenseManager.RemoveLicense`, passing in a license key, to remove a license from the inventory of available licenses.

Hosts

Many of the operations in your vSphere environment involve setting up the ESX hosts on which the virtualization layer runs. You can set up storage and networking, and those settings directly affect the virtual machine. You must also manage other aspects of the host, as discussed in this chapter.

> **Important:**
> See the *ESX Configuration Guide* and the *ESX Configuration Guide* for important information on security considerations, not included here.

Host Management Objects

The vSphere Web Services SDK includes several objects for host management.

The central object is `HostSystem`. Each property of `HostSystem` is a data object that encapsulates some information about the host. For example, the `capability` property is a `HostCapability` object, the `runtime` property is a `HostRuntimeInfo` object. See the *API Reference* for a list of the properties and the corresponding data objects.

`HostSystem` methods allow you to perform certain tasks on ESX hosts. However, many tasks are not performed through `HostSystem` methods, but through methods in managed objects related to `HostSystem`. For example, you manage the host time using the `HostDateTimeSystem` and you manage kernel modules using `HostKernelModuleSystem`.

Retrieving Host Information

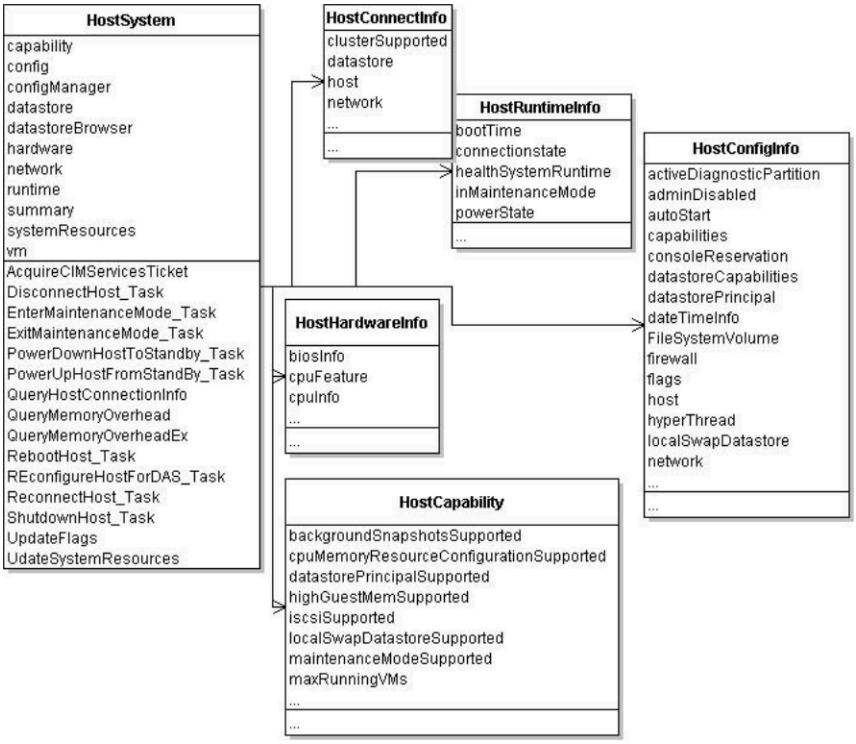
You retrieve information about the host by accessing data objects defined for the `HostSystem`.

- `HostSystem.capability` is a `HostCapability` object. The `HostCapability` properties indicate the features that are supported by the host, for example, `maintenanceModeSupported` or `recursiveResourcePoolsSupported`.
- `HostSystem.runtimeInfo` is a `HostRuntimeInfo` object that contains several data objects with detailed information about the current state of the host. You can, for example, extract the health status as a `HealthSystemRuntime` object or the power state as a `HostPowerState` object.
- `HostSystem.hardware` is a `HostHardwareInfo` object that allows you to retrieve the host's hardware configuration including CPU and NUMA information and memory size.
- `HostSystem.config` is a `HostConfigInfo` object. This data object type encapsulates a typical set of host configuration information that is useful for displaying and configuring a host. You can access the `HostConfigInfo` object only on managed hosts, and only if the host is connected.

`HostSystem` has several additional properties that allow you to directly access the virtual machines, datastores, and networks associated with that system.

The `QueryHostConnectionInfo`, `QueryMemoryOverhead`, and `QueryMemoryOverheadEx` methods are available for information retrieval.

Figure 594: HostSystem and Information Properties



Configuring and Reconfiguring Hosts

When you configure or reconfigure an ESX host, you usually do not use the methods in `HostSystem` directly, but work with managed objects available for configuration of that part of the system. For example, `HostNetworkSystem` allows you to configure the network, and `HostAuthorizationManager` is for managing users, groups, and permissions on a host. The objects and related methods are discussed in the corresponding chapters of this guide.

Some methods are defined locally in `HostSystem`. See the *vSphere API Reference* for details on each method.

- CIM Management – `AcquireCimServicesTicket`. For additional information on using vSphere with CIM, see the VMware CIM APIs documentation.
- Host Lifecycle – `RebootHost_Task`, `ShutdownHost_Task`, `PowerDownHostToStandBy_Task`, `PowerUpHostFromStandBy_Task`, `DisconnectHost_Task`, `ReconnectHost_Task`. See Managing the Host Lifecycle.
- Maintenance Mode – `EnterMaintenanceMode_Task`, `ExitMaintenanceMode_Task`.
- Updates – `UpdateFlags`, `UpdateIpmi`, `UpdateSystemResources`.

Managing the Host Lifecycle

A host's lifecycle depends in part on whether the host is a standalone host or managed by a vCenter system.

Reboot and Shutdown

You can reboot and shut down managed and standalone hosts. The `ShutdownHost_Task` method is not supported on all hosts. Check the host capability `shutdownSupported`.

You can call both methods with a `force` parameter, which specifies whether to reboot hosts even when virtual machines are running or other operations are in progress on the host. If you set the parameter to `false`, hosts are rebooted only when they are in maintenance mode.

- `ShutdownHost_Task` – Shuts down a host. If connected directly to the host, the client never receives an indicator of success in the returned task, but temporarily loses connection to the host. If the method does not succeed, an error is returned.

- `RebootHost_Task` – Reboots a host. If the command is successful, then the host has been rebooted. Clients connected directly to the host do not receive an indication of success in the returned task, but temporarily lose connection to the host. If the method does not succeed, an error is returned.

Using Standby Mode

Standby is a power state in which the host does not support provisioning or power on of virtual machines. VMware power management module might evacuate and put a host in standby mode to save power. The host can be powered up remotely by using `PowerUpHostFromStandBy_Task`.

The following methods support standby mode. Both methods are cancelable.

- `PowerDownHostToStandBy_Task` – Puts the host in standby mode, a mode in which the host is in a standby state from which it can be powered up remotely. The command is only supported on hosts on which the host capability `standbySupported` is `true`.

  While this task is running, no virtual machines can be powered on and no provisioning operations can be performed on the host.

  Calling this method does not directly initiate any operations to evacuate or power down powered-on virtual machines. However, if VMware DRS is enabled, the vCenter migrates powered-off virtual machines or recommends migration to a different host, depending on the automation level. If the host is part of a cluster and the task is issued with a vCenter target with the method's `evacuatePoweredOffVms` parameter set to `true`, the task does not succeed unless all the powered-off virtual machines are reregistered to other hosts.
- `PowerUpHostFromStandBy_Task` – Takes the host out of standby mode. If the command is successful, the host wakes up and starts sending heartbeats. This method might be called automatically by VMware DRS to add capacity to a cluster, if the host is not in maintenance mode.

Disconnecting and Reconnecting Hosts

You can make a host a managed host by adding it to the vCenter system. You can later disconnect and reconnect the host, for example, to refresh the agents.

You can use the following methods, which are only supported if you access the host through a vCenter system.

- `QueryHostConnectionInfo` – Returns a `HostConnectInfo` object, which is the same object that the `Datacenter.QueryConnectionInfo` returns. The information in this object can be used by a connection wizard, like the wizard used in the vSphere Client.
- `DisconnectHost_Task` – Disconnects from a host and instructs the vCenter system to stop sending heartbeats to the host.
- `ReconnectHost_Task` – Reconnects a host to the vCenter system. This process reinstalls agents and reconfigures the host, if it has gotten out of sync with the server. The reconnection process checks for the correct set of licenses and for the number of CPUs on the host, ensures the correct set of agents is installed, and ensures that networks and datastores are discovered and registered with the vCenter system.

  Client applications can change the IP address and port of the host when doing a reconnect operation. This can be useful if the client wants to preserve existing metadata, such as statistics, alarms, and privileges, even though the host is changing its IP address.

Querying and Changing the Host Time

The `HostDateTimeSystem` supports date and time related configuration on a host and supports both Network Time Protocol (NTP) and Precision Time Protocol (PTP) configuration.

The `HostDateTimeSystem.dateTimeInfo` property allows you to retrieve and set date and time information. The `HostDateTimeInfo` data object's properties contain these data objects for date and time management:

- `HostNTPConfig` contains a list of NTP servers for use by the host.
- `HostPTPConfig` contains device and networking information.
- `HostDateTimeSystemTimeZone` specifies the time zone including the GMT offset, identifier for the time zone, and name.

You can also query the host's time information by calling one of the `HostDateTimeSystem` methods.

- `QueryAvailableTimeZones` – Retrieves the list of available timezones on the host. The method uses the public domain `tz` timezone database. The method returns an array of `HostDateTimeSystemTimeZone` objects.
- `QueryDateTime` – Returns the current date and time on the host.

You can modify the host's date and time information by calling one of the following `HostDateTimeSystem` methods:

- `RefreshDateTimeSystem` – Refreshes the date and time related settings to pick up any changes that might have occurred.
- `UpdateDateTime` – Updates the date and time on the host using the date and time passed into the method. Use with caution. Network delays or execution delays can result in time skews.
- `UpdateDateTimeConfig` – Updates the date and time configuration of the host. You call this method with a `HostDateTimeConfig` parameter, which allows you to specify NTP configuration, PTP configuration, and the time zone.

Querying Virtual Machine Memory Overhead

Each virtual machine you power on requires a certain amount of memory for its use. In addition, the host must have some memory overhead available for each virtual machine. To find out about memory overheat, call the `HostSystem.QueryMemoryOverheadEx` method. The method takes a `virtualMachineConfigInfo` data object as an argument, and determines the amount of overhead necessary to power on a virtual machine with those characteristics.

The methods returns the amount of memory required, in bytes.

Storage

A virtual machine uses a virtual disk to store its operating system, program files, and other data. A virtual disk is a large physical file, or a set of files, that can be copied, moved, archived, and backed up like other files. To store and manipulate virtual disk files, a host requires dedicated storage space. ESX supports storage in multiple ways. Hosts that are managed by a vCenter system can share storage.

Any type of network-attached storage requires complete configuration of networking in the VMkernel to support network-based access to the storage media. The VMkernel requires its own IP address. See vSphere Networks.

Storage Management Objects

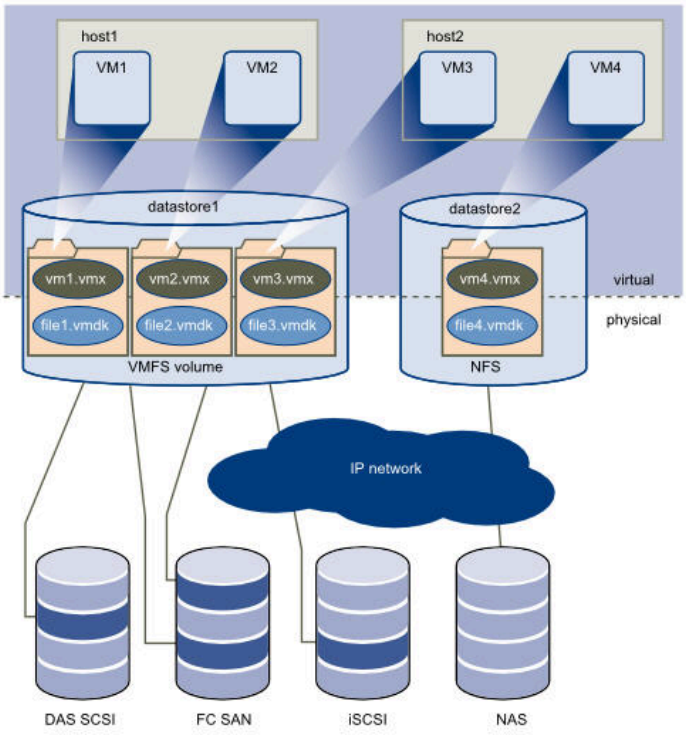
You can access the objects that support storage management through the `HostSystem` managed object.

- `HostStorageSystem` – The `HostSystem.storageSystem` property is a managed object reference to the `HostStorageSystem` of the ESX system. `HostStorageSystem` is a low-level interface that is used mainly for configuring the physical storage. See Configuring Disk Partitions.
- `HostDatastoreSystem` – The `HostSystem.datastoreSystem` property is a managed object reference to a `HostDatastoreSystem` managed object. `HostDatastoreSystem` methods allow you to create, configure, extend, and remove datastores. While `HostStorageSystem` supports access and configuration of physical storage, `HostDatastoreSystem` supports access and configuration of logical storage through the volumes (`Datastore` managed objects) the host can use for virtual machines. See Creating and Managing Datastores.
- `HostDatastoreBrowser` – Provides access to the contents of one or more datastores. The items in a datastore are files that contain configuration, virtual disk, and other data associated with a virtual machine.
- `Datastore` – The `Datastore` managed entity provides methods for mounting datastores, browsing datastores, and obtaining information about the datastores associated with a virtual machine. See Creating and Managing Datastores.
- `HostDiagnosticPartition` – Supports creating and querying diagnostic partitions for your ESX host. See Managing Diagnostic Partitions.

Introduction to Storage

The VMware vSphere storage architecture consists of layers of abstraction that hide and manage the complexity and differences of physical storage subsystems, shown in the following illustration.

Figure 595: Storage Architecture



How Virtual Machines Access Storage

Virtual machines use virtual disks for their operating system, application software, and other data files. A virtual disk is stored as a VMDK file on a datastore. The virtual disk hides the physical storage layer from the virtual machine's operating system. Regardless of the type of storage device that your host uses, the virtual disk always appears to the virtual machine as a local SCSI device. As a result, you can run operating systems that are not certified for specific storage equipment, such as SAN, in the virtual machine.

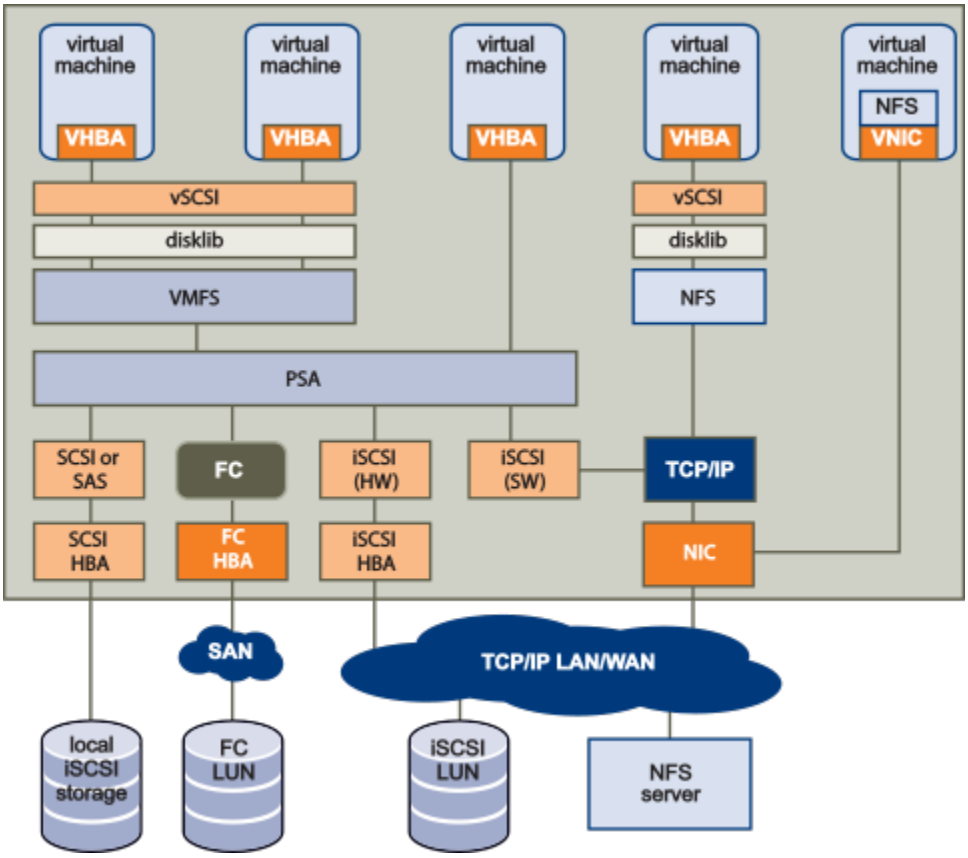
When a virtual machine communicates with its virtual disk stored on a datastore, it issues SCSI commands. Because datastores can exist on different types of physical storage, these commands are encapsulated into other forms, depending on the protocol that the ESX host uses to connect to the physical storage device.

To the applications and guest operating systems running on each virtual machine, the storage subsystem appears as a virtual SCSI controller connected to one or more virtual SCSI disks as shown in the top half of Figure 595: Storage Architecture. These controllers are the only types of SCSI controllers that a virtual machine can see and access, and include the objects that extend `VirtualSCSIController`:

- `ParaVirtualSCSIController`
- `VirtualBusLogicController`
- `VirtualLsiLogicController`
- `VirtualLsiLogicSASController`

How precisely a virtual machine accesses storage depends on the setup of the host. Figure 596: Storage API Architecture gives an overview of the different possibilities.

Figure 596: Storage API Architecture



Datastores

A datastore is a manageable storage entity, usually used as a repository for virtual machine files including log files, scripts, configuration files, virtual disks, and so on. vSphere supports two types of datastores, VMFS and NAS.

- If you want to use a NAS volume, mount it using `CreateNasDatastore` and unmount it using `RemoveDatastore`. The two commands are host specific, you must invoke the create and remove methods on each host on which you want to mount or unmount the datastore.
- To create a VMFS datastore, call `CreateVmfsDatastore`, passing in any existing disk. As a result of the call, the disk is formatted with VMFS and the datastore is automounted on all ESX hosts on which the disk is visible the next time you perform a rescan. When you call `RemoveDatastore` on a VMFS datastore, the datastore is destroyed. After a rescan, the datastore is no longer available to any ESX systems. In contrast to NAS datastores, you do not have to invoke methods for creation and removal of the datastore on each host.

An ESX host automatically discovers the VMFS volume on attached Logical Unit Numbers (LUNs) on startup and after re-scanning the host bus adapter. When you create a VMFS datastore, the datastore label is based on the VMFS volume label. If there is a conflict with an existing datastore, the label is made unique by appending a suffix. The VMFS volume label remains unchanged.

Destroying a VMFS datastore removes the partitions that compose the VMFS volume.

Datastores can span multiple physical storage devices. A single VMFS volume can contain one or more LUNs from a local SCSI disk array on a physical host, a Fibre Channel SAN disk farm, or iSCSI SAN disk farm. The ESX system detects new LUNS that are added to any of the physical storage subsystems. When the user queries for a list of available devices, the newly discovered devices are included. You can extend storage capacity on an existing VMFS volume without powering down physical hosts or storage subsystems.

If any of the LUNs within a VMFS volume fails or becomes unavailable, only virtual machines with data on that LUN are affected. An exception is the LUN that has the first extent of the spanned volume (multi-extent volume). All other virtual machines with virtual disks residing on other LUNs continue to function normally.

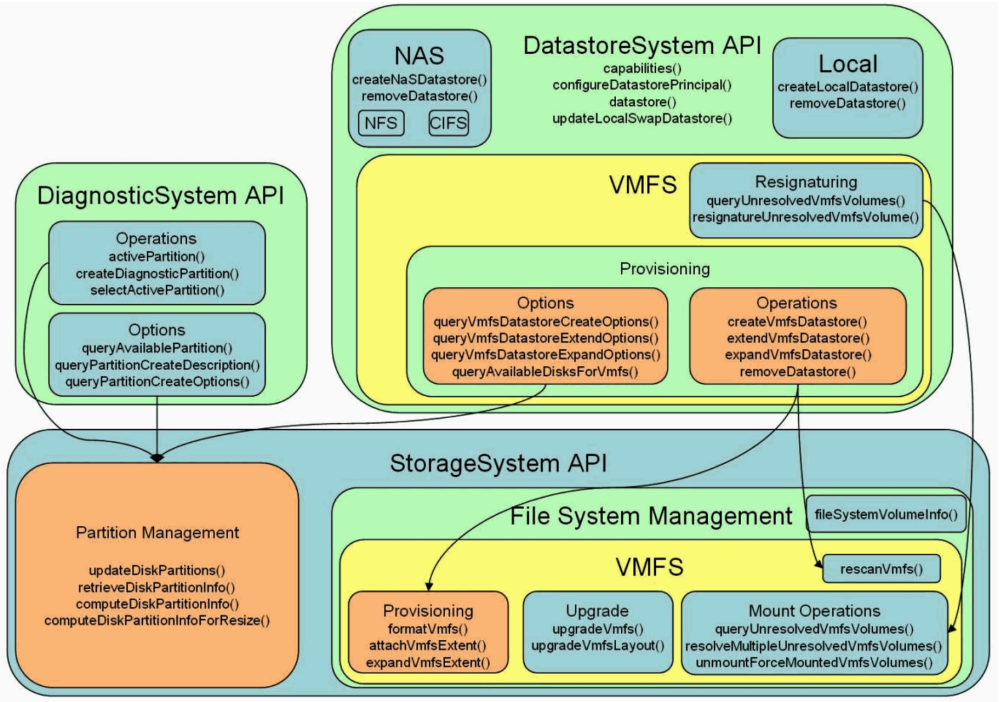
Choosing the Storage API to Use

The `HostStorageSystem` APIs are low-level enough for performing VMFS provisioning operations. They require a knowledge of partitioning details and VMFS extent composition. They do not enforce VMFS best practices like partition alignment and optimum VMFS block sizes, and they allow you to mix extents from different datastores on the same LUN and to add extents even though expansion is preferable in most cases.

The `HostDatastoreSystem` APIs are primarily used for managing VMFS volumes. They don't require an in-depth knowledge of storage systems, and do enforce best practices.

Figure 597: Storage APIs gives an overview of the different APIs. See Managed Objects for Working with Storage to determine which tasks are commonly performed with which API.

Figure 597: Storage APIs



Managed Objects for Working with Storage

The Web Services API provides several managed objects that are designed to work with ESX storage devices.

Managed Object	Task	See
`HostStorageSystem`	Low-level operations associated with individual hosts, such as resizing or updating disk partitions.	Configuring Disk Partitions
`HostStorageSystem`	Multipath management.	Multipath Management
`HostStorageSystem`	iSCSI Storage setup and configuration.	Configuring iSCSI Storage
`HostDatastoreSystem`	Creating and managing VMFS datastores and remote datastores.	Creating and Managing Datastores

Managed Object	Task	See
HostDatastoreSystem HostStorageSystem	Managing VMFS volume copies (resignature or force mount).	Managing VMFS Volume Copies (Resignaturing)
HostDiagnosticSystem	Creating and managing diagnostic partitions.	Managing Diagnostic Partitions

Configuring Disk Partitions

`HostStorageSystem` manages low-level storage components including HBAs, SCSI LUNs, file system volumes, and so on. You can use this API to set up the partitions before creating, extending, or expanding a VMFS file system.

- `ComputeDiskPartitionInfo` – Computes the disk partition information based on the specified disk layout. The server computes a new `HostDiskPartitionInfo` object for a specific disk using the layout that is specified by the `HostDiskPartitionLayout` object. Inside the `HostDiskPartitionLayout` object, you specify the list of block ranges for that partition, and optionally the total number and size of the blocks. You can then use that information inside the `HostDiskPartitionSpec` when updating a disk partition.
- `ComputeDiskPartitionInfoForResize` – Computes the disk partition information to support resizing a given partition. Returns the resized disk partition information as a `HostDiskPartitionInfo` object. You can then use that information inside the `HostDiskPartitionSpec` when resizing the disk partition.
- `RetrieveDiskPartitionInfo` – Allows you to specify an array of device path names that identify disks and returns an array of `HostPartitionInfo` objects for each of those disks.
- `UpdateDiskPartitions` – Changes the partitions on a disk by supplying a partition specification (`HostDiskPartitionSpec`) and device name.

After you have updated the disk partitions for the host, you must perform a rescan by using one of the following methods. Complete rescans might take a long time.

- `RefreshStorageSystem` – Refreshes the storage information and settings to pick up changes, but does not explicitly issue commands to discover new devices.
- `RescanAllHba` – Rescans all host bus adapters for new storage devices. This method might take a long time.
- `RescanHba` – Rescans a specific host bus adapter for new devices.

`HostStorageSystem` methods are also used for setting up iSCSI storage. See Configuring iSCSI Storage.

Multipath Management

The *vSphere Storage* documentation includes information about using multipathing for failover and load balancing. You can manage multipathing using the vSphere Client, the `esxcli` command, or using the following commands. Use the `HostStorageSystem.multipathStateInfo` property to access the `HostMultipathStateInfo` data object that describes runtime information about the state of multipathing on a given host.

- `EnableMultipathPath` – Enables a disabled path for a device. Use the pathname from `HostMultipathStateInfoPath` or `HostMultipathInfoPath`.

- `QueryPathSelectionPolicyOptions` – Obtains the set of path-selection-policy options. These options determine the path that can be used by a device that is managed by native multipathing. A `HostMultipathInfo` data object identifies the devices that are managed through native multipathing.
- `QueryStorageArrayTypePolicyOptions` – Obtains the set of storage-array-type policy options. These options determine the storage-array-type policies that a device that is managed by native multipathing might use. A `HostMultipathInfo` data object identifies the devices that are managed through native multipathing.
- `SetMultipathLunPolicy` – Updates the path selection policy for a LUN. Specify the LUN using the LUN UUID from the `HostMultipathInfoLogicalUnit` object.
- `DisableMultipathPath` – Disables an enabled path for a device. Use the pathname from `HostMultipathStateInfoPath` or `HostMultipathInfoPath`.

Configuring iSCSI Storage

vSphere supports software iSCSI, dependent hardware iSCSI, and independent hardware iSCSI. See *Configuring iSCSI Adapters and Storage* in the *vSphere Storage* documentation for a detailed discussion.

The following `HostStorageSystem` methods are available for iSCSI storage management.

- Add a dynamic or static target.
  - `AddInternetScsiSendTarget` – Adds send target entries to the host bus adapter discovery list if the `DiscoveryProperties.sendTargetsDiscoveryEnabled` flag is set to true.
  - `AddInternetScsiStaticTargets` – Adds static target entries to the host bus adapter discovery list. The `DiscoveryProperty.staticTargetDiscoveryEnabled` flag must be set to true.
- Configure targets.
  - `UpdateInternetScsiAdvancedOptions` – Updates the advanced options that the iSCSI host bus adapter or the discovery addresses and targets associated with it.
  - `UpdateInternetScsiAlias` – Updates the alias of an iSCSI host bus adapter.
  - `UpdateInternetScsiAuthenticationProperties` – Updates the authentication properties for one or more targets or discovery addresses associated with an iSCSI host bus adapter.
  - `UpdateInternetScsiDigestProperties` – Updates the digest properties for the iSCSI host bus adapter or the discovery addresses and targets associated with it.
  - `UpdateInternetScsiDiscoveryProperties` – Updates the discovery properties for an iSCSI host bus adapter.
  - `UpdateInternetScsiIPProperties` – Updates the IP properties for an iSCSI host bus adapter.
  - `UpdateInternetScsiName` – Updates the name of an iSCSI host bus adapter.
  - `UpdateSoftwareInternetScsiEnabled` – Enables and disables software iSCSI in the VMkernel.
- Remove a dynamic or static target.
  - `RemoveInternetScsiSendTargets` – Removes send target entries from the host bus adapter discovery list. The `DiscoveryProperty.sendTargetsDiscoveryEnabled` must be set to true. If any of the targets provided as parameters are not found in the existing list, the other targets are removed and an exception is thrown.
  - `RemoveInternetScsiStaticTargets` – Remove static target entries from the host bus adapter discovery list. The `DiscoveryProperty.staticTargetDiscoveryEnabled` must be set to true. If any of the targets provided as parameters are not found in the existing list, the other targets are removed and an exception is thrown.

iSCSI initiators and targets have unique, permanent iSCSI names and addresses. An iSCSI name correctly identifies a specific iSCSI initiator or target, regardless of physical location. Names must be in EUI or IQN format, as specified by the storage vendor's hardware.

Configure the VMkernel To Support Software iSCSI

Before you can set up iSCSI on a system, you must create a dedicated VMkernel network interface. See. You can then enable the VMkernel to support iSCSI and configure the initiator.

To enable the VMkernel to support software iSCSI, use the following steps.

1. Obtain a managed object reference to the host system's `HostStorageSystem`.
2. Invoke the `UpdateSoftwareInternetScsiEnabled` method, passing the reference to the `HostStorageSystem` and the value `true`.

Configure iSCSI Initiators

After you have configured the VMkernel to support software iSCSI, you can configure the storage initiators for iSCSI.

To configure iSCSI initiators, use the following steps.

1. Access the list of available HBAs on the host system.

   You can do this by creating a property collector with `HostSystem` as the starting point. See Property Collector. From the `HostSystem.config` property, you can obtain the list (array) of host bus adapters by specifying this property path:

   ```
   config.storageDevice.hostBusAdapter
   ```

   The property path returns an array of host bus adapters. For example:

   ```
   hostBusAdapter["key-vim.host.BlockHba-vmhba32"]
   hostBusAdapter["key-vim.host.BlockHba-vmhba33"]
   hostBusAdapter["key-vim.host.BlockHba-vmhba34"]
   hostBusAdapter["key-vim.host.BlockHba-vmhba35"]
   hostBusAdapter["key-vim.host.BlockHba-vmhba1"]
   ...
   ```

2. From the array, select the host bus adapter (instance of `HostHostBusAdapter`) that you want to configure and obtain its `key` property, which is the device name of the host bus adapter as a string.
3. Determine the capabilities of the adapter by retrieving the properties of the `HostHostBusAdapter` object.
4. Configure the initiator.
   - For an independent hardware initiator, configure the IP address.
   - For a software initiator, enable the software initiator in the VMkernel.
5. Configure the iSCSI name by calling `HostStorageSystem.UpdateInternetScsiName` and the alias by running `HostStorageSystem.UpdateInternetScsiAlias`.
6. Configure target discovery by calling `HostStorageSystem.UpdateInternetScsiHbaDiscoveryProperties`.

   The method takes a `HostInternetScsiHbaDiscoveryProperties` data object that you can configure.
7. (Optional) Set the authentication information by calling `HostStorageSystem.UpdateInternetScsiAuthenticationProperties`.

   The `HostInternetScsiHbaAuthenticationProperties` object you pass into that method includes properties for configuring CHAP and Mutual CHAP. See the *vSphere Storage* documentation for information about securing your iSCSI storage array.
8. Configure access to the targets.
9. Rescan the HBAs.

   Rescan enables the HBAs to discover the new storage devices. You can either rescan a single HBA with `HostStorageSystem.RescanHba`, specifying the HBA ID as a parameter, or rescan all HBAs using `HostStorageSystem.RescanAllHba`.

Creating and Managing Datastores

Each datastore is a logical container, analogous to a file system on a logical volume, where the host places virtual disk files and other virtual machine files. Datastores hide specifics of the physical storage device and provide a uniform model for storing virtual machine files.

The `HostDatastoreSystem` managed object provides methods for creating and managing datastores. All `HostDatastoreSystem` methods require a managed object reference to `HostDatastoreSystem`, and return a reference to the `Datastore` object after it is created.

`HostDatastoreSystem` allows you to create and expand, query, and remove or update datastores. `HostDatastoreSystem` also allows you to configure a datastore principal for a host by calling `ConfigureDatastorePrincipal`. All virtual machine-related file I/O is performed under this user.

VMFS provisioning tasks are often performed as follows:

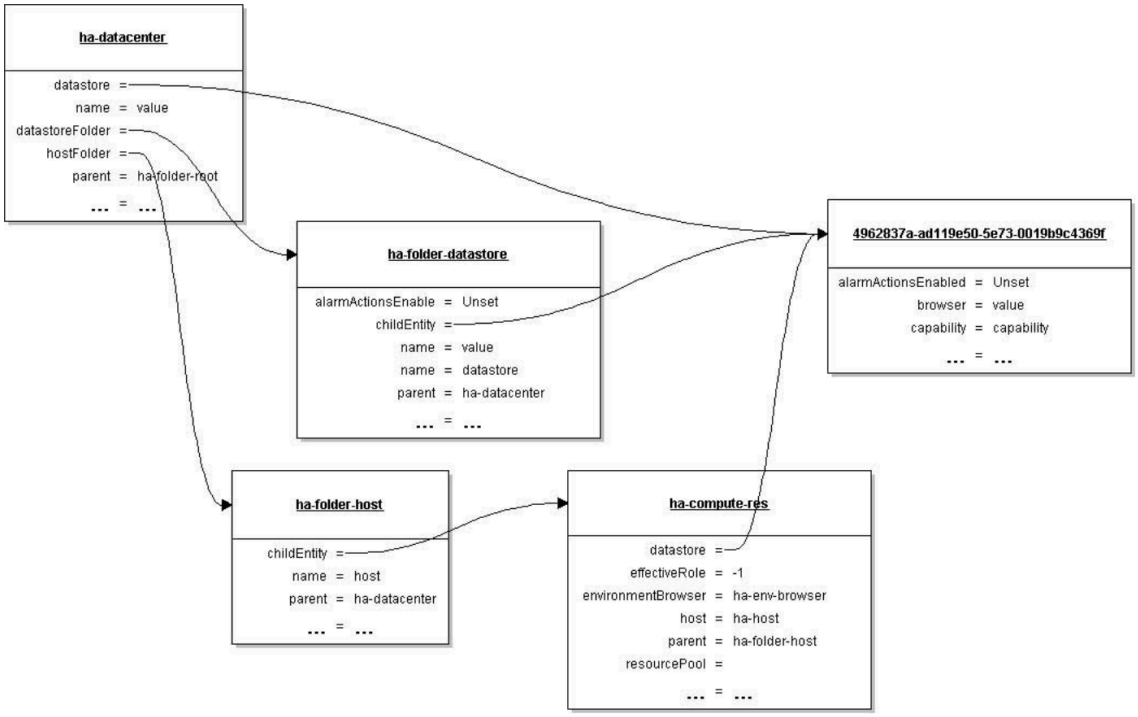
1. Call `QueryAvailableDisksForVmfs` to get the subset of disks that are well suited for holding VMFS datastores. `QueryAvailableDisksForVmfs` obtains a list of disks that can be used to contain VMFS datastore extents. You can provide a datastore name to obtain the list of disks that can contain extents for the specified VMFS datastore. The operation does not return disks currently used by the VMFS datastore, nor does it return management LUNs and disks that are referenced by RDMs. RDM disks are not usable for VMFS datastores.
2. Get information about provisioning options by calling one of the following methods, passing in the selected disk:
   - `QueryVmfsDatastoreCreateOptions` – Obtains information about options for creating a new VMFS datastore on a disk. The method returns an array of `VmfsDatastoreOption` data objects.
   - `QueryVmfsDatastoreExpandOptions` – Obtains information about options for expanding the extents of an existing VMFS datastore.
   - `QueryVmfsDatastoreExtendOptions` – Obtains information about options for extending an existing VMFS datastore for a disk.
3. If required, change the layout by calling `HostStorageSystem.ComputeDiskPartitionInfo` and then `HostStorageSystem.UpdateDiskPartition` to resize the partition.
4. Call `CreateVmfsDatastore`, `ExtendVmfsDatastore`, or `ExpandVmfsDatastore` to complete the VMFS provisioning operation.

Accessing Datastores

The following diagram illustrates how you can access or specify datastores.

- Each `Datacenter` managed object has a `datastore` property that contains an array of datastores.
- Each `Datacenter` managed object has a `datastoreFolder` property that is a reference to the folder (or folder hierarchy) that contains the datastores for this datacenter.
- Each `Datacenter` managed object has a `hostFolder` property that is a reference to the folder (or folder hierarchy) that contains the compute resources, including hosts and clusters, for this datacenter. Each `HostSystem` or `ComputeResource` has a `datastore` property that is an array of `Datastore` managed objects.

Figure 598: Datastore Managed Object



For more information about the hierarchy of managed objects, see Datacenter Inventory.

Creating and Modifying a VMFS Datastore

A datastore is a manageable storage entity, usually used as a repository for virtual machine files including log files, scripts, configuration files, virtual disks, and so on.

VMFS is a proprietary file system VMware designed for virtual machines. VMFS is well suited for storing a small number of large data files like virtual disks. These files are mostly used by a single host. VMFS differs from other filesystem formats like FAT16/FAT32 and so on in that it can be accessed by multiple hosts connected to the same SAN LUN.

You can set up a VMFS datastore on any SCSI-based storage device that the host can access. VMFS volume creation, extension, and expansion requires first partitioning operations and the VMFS volume operations.

Set Up Disk Partitions

The Web Services API enables you to manage disk partitions on ESX host storage devices.

To set up the disk partitions, use the following steps.

1. Call `HostStorageSystem.RetrieveDiskPartitionInfo` to retrieve information about existing partitions.
2. Call `HostStorageSystem.ComputeDiskPartition`, passing in the desired disk layout. The server computes a new partition information object for a specific disk representing the desired layout and returns a `HostDiskPartitionInfo` object that you can use in the `HostDiskPartitionSpec` you pass into `UpdateDiskPartitions`.
3. Call `HostStorageSystem.UpdateDiskPartitions` to update partitions by passing in a `HostDiskPartitionSpec`.

Create the VMFS Datastore

After you create disk partitions on host storage, you can configure VMFS datastores.

To create a VMFS datastore, use the following steps.

1. Configure and install any third-party adapter your storage requires and rescan the adapters by calling `HostStorageSystem.RescanAllHba`.
2. Call `HostDatastoreSystem.QueryAvailableDisksForVmfs` for information about disks that can be used to contain VMFS datastore.

   This method filters out disks that are currently in use by an existing VMFS unless the VMFS using the disk is one being extended. It will also filter out management LUNs and disks that are referenced by RDMs. These disk LUNs are also unsuited for use by a VMFS. The method returns an array of `HostScisiDisk` objects.
3. Call `HostDatastoreSystem.QueryVmfsDatastoreCreateOptions` for information about options for for creating a new VMFS datastore. The call returns an array of `VmfsDatastoreCreateOption` data objects that allow you to access the UUIDs of suitable data stores.
4. (Optional) If no suitable partitions for your VMFS volume exist, you might have to create them. Use the `ComputeDiskPartitionInfo` and `UpdateDiskPartitions` methods in `HostStorageSystem`.
5. Create the datastore.
   - Call `HostDatastoreSystem.CreateVmfsDatastore` to create a VMFS datastore. The method takes a `VmfsDatastoreCreateSpec` data object that consists of a a `partition`, a `HostVmfsSpec`, and an optional `extent`. The `HostVmfsSpec` allows you to specify the block size, extent, major version, and volume name for the VMFS.
   - Call `HostDatastoreSystem.CreateNasDatastore` to create a network-attached storage based datastore.

You can later expand and extend the VMFS datastore by calling one of the following methods.

- Call first `QueryVmfsDatastoreExpandOptions` and then `ExpandVmfsDatastore` to expand an existing VMFS datastore using the specification provided in the `VmfsDatastoreExpandSpec` data object (which contains the name of the extent and partition information). `ExpandVmfsDatastore` increases the size of the datastore up to the full size provisioned for the datastore, if necessary.
- Call first `QueryVmfsDatastoreExtendOptions` and then `ExtendVmfsDatastore` to extend an existing VMFS datastore using the specification provided in the `VmfsDatastoreExtendSpec` data object.

Removing and Updating Datastores

The Web Services API has methods to update and remove datastores.

- `RemoveDatastore` – Removes a datastore from a host.
- `UpdateLocalSwapDatastore` – Choose the `localSwapDatastore` for this host. Any change to this setting affects virtual machines that subsequently power on or resume from a suspended state at this host, or that migrate to this host while powered on. Virtual machines that are currently powered on at this host are not affected.

See the *vSphere API Reference* for more information about the `HostDatastoreSystem` operations, including constraints and limitations.

Managing VMFS Datastores with HostStorageSystem

In most cases, the `Datastore` methods are appropriate for creating and managing VMFS datastores. However, in some cases the following `HostStorageSystem` commands are used instead:

- `AttachVmfsExtent` – Extends a VMFS by attaching a disk partition as an extent.
- `ExpandVmfsExtent` – Expands a VMFS extent as specified by the disk partition specification.
- `FormatVmfs` – Formats a new VMFS on a disk partition based on the `HostVmfsSpec` that you pass in. Returns a `HostVmfsVolume` that represents the new VMFS file system. The `HostVmfsVolume` includes the block size, list of partition names of the disk's VMFS extents, and other information including the UUID.
  This command is a low-level API you can use to partition disks explicitly. In most cases, the `Datastore` VMFS commands are more suitable.
- `RescanVmfs` – Rescans for new VMFS instances.
- `UpgradeVmfs` – Upgrades the VMFS to the current VMFS version.

Update and Upgrade with HostStorageSystem

You can use the HostStorageSystem managed object to alter characteristics of storage units.

- `HostStorageSystem.UpdateScsiLunDisplayName` – Update the mutable display name associated with a SCSI LUN. The SCSI LUN to be updated is identified using the LUN UUID.
- `HostStorageSystem.UpgradeVmLayout` – Iterates over all registered virtual machines. For each virtual machine, upgrades the layout and logs an event. After the method has been called, the information in the `VirtualMachineFileLayout` data object data object is correct.

Managing VMFS Volume Copies (Resignaturing)

By default, ESX hosts mount all VMFS datastores. Each VMFS datastore that is created in a partition on a LUN has a unique UUID that is stored in the file system superblock. In addition, the LUN ID of the source LUN is unique and is stored in the VMFS metadata.

When a LUN is replicated or a copy is made, the resulting LUN copy is identical, byte-for-byte, with the original LUN. As a result, if the original LUN contains a VMFS datastore with UUID X, the LUN copy appears to contain an identical VMFS datastore, or a VMFS datastore copy, with exactly the same UUID X. ESX can determine whether a LUN contains the VMFS datastore copy, and considers the copy unresolved and does not mount it automatically.

To make the data on the LUN copy available, you can either force mount the copy if you are sure the original is not in use, or you can resignature the copy. When you perform datastore resignaturing, consider the following points:

- Datastore resignaturing is irreversible because it overwrites the original VMFS UUID.
- The LUN copy that contains the VMFS datastore that you resignature is no longer treated as a LUN copy, but instead appears as an independent datastore with no relation to the source of the copy.
- A spanned datastore can be resignatured only if all its extents are online.
- The resignaturing process is crash and fault tolerant. If the process is interrupted, you can resume it later.
- You can mount the new VMFS datastore without a risk of its UUID colliding with UUIDs of any other datastore, such as an ancestor or child in a hierarchy of LUN snapshots.

See the *vSphere Storage* documentation for additional information.

Resignaturing Volumes with ResignatureUnresolvedVmfsVolume_Task

The easiest way to resignature unresolved volumes is by using the `HostDatastoreSystem.ResignatureUnresolvedVmfsVolume_Task` method. The method assigns a new `DiskUuid` to a VMFS volume, but keep its contents intact. The method supports safe volume sharing across hosts and is appropriate in most cases.

You can instead use the low-level `HostStorageSystem` methods to find, force mount, or unmount unresolved volumes:

- `HostStorageSystem.QueryUnresolvedVmfsVolume` – Obtains the list of unbound VMFS volumes. For sharing a volume across hosts, a VMFS volume is bound to its underlying block device storage. When a low-level block copy is performed to copy or move the VMFS volume, the copied volume is unbound.
- `HostStorageSystem.ResolveMultipleUnresolvedVmfsVolumes` – Resignatures or force mounts unbound VMFS volumes. This method takes a `HostUnresolvedVmfsResolutionSpec` data object as input. The `HostUnresolvedVmfsResolutionSpec.resolutionSpec` property is an array of `HostUnresolvedVmfsResolutionSpec` data objects that contain a `HostUnresolvedVmfsResolutionSpecVmfsUuidResolution` enumeration. The enumeration is either `forceMount` or `resignature`.
- `UnmountForceMountedVmfsVolume` – Unmounts a force mounted VMFS volume. When a low-level block copy is performed to copy or move the VMFS volume, the copied volume is unresolved. For the VMFS volume to be usable, a resolution operation is applied. As part of resolution operation, you might decide to keep the original VMFS UUID. Once the resolution is applied, the VMFS volume is mounted on the host for its use. This method allows you to unmount the VMFS volume if it is not used by any registered virtual machines.

Managing Diagnostic Partitions

Your host must have a diagnostic partition (dump partition) to store core dumps for debugging and for use by VMware technical support.

For more information about diagnostic bundles, see Generating Diagnostic Bundles. For information about how to collect diagnostic partitions for a purple screen fault in ESX, see the Broadcom knowledge base article at KB 1006796 .

A 100MB diagnostic partition for each host is recommended. If more than one ESX host uses the same LUN as the diagnostic partition, that LUN must be zoned so that all the ESX host can access it. Each host needs 100MB of space, so the size of the LUN determines how many servers can share it. Each ESX host is mapped to a diagnostic slot. VMware recommends at least 16 slots (1600MB) of disk space if servers share a diagnostic partition. You can set up a SAN LUN with FibreChannel or hardware iSCSI. SAN LUNs accessed through a software iSCSI initiator are not supported.

> **CAUTION:**
> If two hosts that share a diagnostic partition fail and save core dumps to the same slot, the core dumps might be lost. To collect core dump data, reboot a host and extract log files immediately after the host fails. If another host fails before you collect the diagnostic data of the first host, the second host does not save the core dump.

Retrieving Diagnostic Partition Information

The `HostDiagnosticSystem` managed object allows you to retrieve information in several ways.

- Retrieve the `HostDiagnosticPartition` object from the `HostDiagnosticSystem.activePartition` property to examine the properties of the active partition.
- Call the `HostDiagnosticPartition.QueryAvailablePartition` method to retrieve a list of available diagnostic partitions, in order of suitability.
- Call the `HostDiagnosticPartition.QueryPartitionCreateOptions` method to retrieve a list of disks with sufficient space to contain a diagnostic partition of the specified type. The choices are returned in order of suitability.

Create a Diagnostic Partition

Creating a diagnostic partition requires that you find a suitable partition using one of the query methods. You can then retrieve a creation specification, and perform the actual creation.

To create a diagnostic partition, use the following steps.

1. Find a suitable partition by calling `HostDiagnosticPartition.QueryAvailablePartition` or `HostDiagnosticPartition.QueryPartitionCreateOptions`.
2. Call `HostDiagnosticPartition.CreateDiagnosticPartition`, passing in a `HostDiagnosticPartitionCreateSpec`, which includes information about the diagnostic type, id, storage type, and so on.

On success, this method creates the partition and makes the partition the active partition if specified in the `active` parameter. On failure, the diagnostic partition might exist, but will not be active even if the partition was supposed to be made active.

Sample Code for Storage

The following table lists the sample applications included with the vSphere Web Services SDK that demonstrate some of the topics discussed in this chapter.

Java (SDK\vsphere-ws\java\JAXWS\samples\com\vmware\)
scsilun\SCSILunName.java
httpfileaccess\GetVMFiles.java

vSphere Networks

Before you add storage and virtual machines to an ESX system, you should have completed networking setup. This chapter describes how to set up virtual switches in the vSphere environment.

Virtual Switches

vSphere supports the use of virtual switches to manage network traffic to and from virtual machines.

- vCenter vCenter supports a distributed network model in which a distributed virtual switch manages ESX host proxy switch configuration. In the distributed network model, a host proxy switch reflects the distributed virtual switch port settings, describes how physical network adapters are bridged to the switch, and performs network I/O.
- On a standalone ESX host, you can use a VMware standard virtual switch to support network traffic to and from virtual machines on the host.

To configure a vSphere network you perform the following operations:

- Set up virtual switches
- Define portgroups
- Configure physical network adapters

Port Groups

Port groups aggregate multiple ports under a common configuration. Each port can connect to a network adapter of a virtual machine, or an uplink adapter on the physical machine.

Each port group is identified by a network label, which is unique to the current host. Network labels make virtual machine configuration portable across hosts. All port groups in a datacenter that are physically connected to the same network (in the sense that each can receive broadcasts from the others) are given the same label. Conversely, if two port groups cannot receive broadcasts from each other, they have distinct labels.

You can use a VLAN ID to restrict port group traffic to a logical Ethernet segment within the physical network. For a port group to reach port groups located on other VLANs, the VLAN ID must be set to 4095. If you use VLAN IDs, you must change the port group labels and VLAN IDs together so that the labels properly represent connectivity.

Virtual Machine Network Interfaces

When you create a virtual machine, you include a `VirtualMachineConfigSpec`, which, in turn, includes a `VirtualDeviceConfigSpec`. The `device` property of `VirtualDeviceConfigSpec` is a `VirtualDevice` data object.

One of the available virtual devices is `VirtualEthernetCard`. You can use one of the subtypes of `VirtualEthernetCard` to specify the virtual card to use and to specify the MAC address and whether wake-on-LAN is enabled for this virtual card. See Adding Devices to Virtual Machines. A limited number of adapters is supported. KB article 1001805 (http://kb.vmware.com/kb/1001805) discusses available network adapters and which adapter is appropriate in which situation.

VMkernel Network Interfaces
The network services that the VMkernel provides (iSCSI, NFS, and VMotion) use a TCP/IP stack in the VMkernel. This stack accesses various networks by attaching to one or more port groups on one or more virtual switches.

The VMware VMkernel TCP/IP networking stack handles iSCSI, NFS, and VMotion in the following ways.

- iSCSI as a virtual machine datastore
- iSCSI for the direct mounting of ISO files, which are presented as CD-ROMs to virtual machines
- NFS as a virtual machine datastore
- NFS for the direct mounting of ISO files, which are presented as CD-ROMs to virtual machines
- Migration with VMotion

If you have two or more physical NICs for iSCSI, you can create multiple paths for the software iSCSI by using port binding. For more information on port binding, see the *iSCSI SAN Configuration Guide*.

A freshly installed ESX system does not include VMkernel network interfaces. When you wish to migrate a virtual machine with VMotion, your VMkernel networking stack must be set up properly. When you want to use storage types that use TCP/IP network communications, such as iSCSI, you must provide a separate VMkernel network interface for that storage device. You must create any VMkernel ports you might need (see Adding a VMkernel Network Interface).

Physical Network Adapter (pnic)
The term pnic refers to the physical network adapters as seen by the primary operating system. When using the vSphere Web Services SDK, you can manipulate the adapter directly. When using the vSphere Client GUI, you manipulate instead the uplink adapter. On an ESX host, each pnic has one associated uplink adapter.

In a vDS environment, you use a DVS uplink instead of an uplink adapter.

Using a Distributed Virtual Switch

A `DistributedVirtualSwitch` managed object is a virtual network switch that is located on a vCenter instance. A distributed virtual switch manages configuration for proxy switches (`HostProxySwitch`). A proxy switch is located on an ESX host that is managed by the vCenter instance and is a member of the switch. A distributed switch also provides virtual port state management so that port state is maintained when vCenter operations move a virtual machine from one host to another.

A proxy switch performs network I/O to support the following network traffic and operations:

- Network traffic between virtual machines on any hosts that are members of the distributed virtual switch.
- Network traffic between a virtual machine that uses a distributed virtual switch and a virtual machine that uses a VMware standard virtual switch.
- Network traffic between a virtual machine and a remote system on a physical network connected to the ESX host.
- vSphere system operations to support capabilities such as VMotion or High Availability.

A `DistributedVirtualSwitch` is the base distributed switch implementation. It supports a VMware distributed virtual switch implementation and it supports third party distributed switch implementations. The base implementation provides the following capabilities (defined in the `DVSFeatureCapability` object):

- NIC teaming
- Network I/O control
- Network resource allocation
- Quality of service tag support
- User-defined resource pools
- I/O passthrough (VMDirectPath Gen2)

A `VmwareDistributedVirtualSwitch` supports the following additional capabilities (defined in the `DVSFeatureCapability` and `VMwareDVSFeatureCapability` objects):

- Backup, restore, and rollback for a VMware distributed virtual switch and its associated portgroups.
- Maximum Transmission Unit (MTU) configuration.
- Health check operations for NIC teaming and VLAN/MTU support.
- Monitoring switch traffic using Internet Protocol Flow Information Export (IPFIX).
- Link Layer Discovery Protocol (LLDP).
- Virtual network segmentation using a Private VLAN (PVLAN).
- VLAN-based SPAN (VSPAN) for virtual distributed port mirroring.
- Link Aggregation Control Protocol (LACP) defined for uplink portgroups.

Distributed Virtual Switch Configuration

To use a distributed virtual switch, you create a switch and portgroups on a vCenter instance, and add hosts as members of the switch.

1. Use the `Folder.CreateDVS_Task` method to create a distributed virtual switch. Use a `DVSConfigSpec` to create a switch for a third-party implementation. Use a `VMwareDVSConfigSpec` to create a VMware distributed virtual switch.
2. Use the `CreateDVPortgroup_Task` method to create portgroups for host and virtual machine network connections and for the connection between proxy switches and physical NICs. A `DistributedVirtualPortgroup` specifies how virtual ports (`DistributedVirtualPort`) will be used. When you create a distributed virtual switch, the vCenter instance automatically creates one uplink portgroup (`config.uplinkPortgroup`). Uplink portgroups are distributed virtual portgroups that support the connection between proxy switches and physical NICs.

   Port creation on a distributed switch is determined by the portgroup type (`DVPortgroupConfigSpec.type`):

   - If a portgroup is early binding (static), then `DVPortgroupConfigSpec.numPorts` determines the number of ports that get created when the portgroup is created. This number can be increased if `DVPortgroupConfigSpec.autoExpand` is true.
   - If a portgroup is ephemeral (dynamic), then `numPorts` is ignored and ports are created as needed.
     You can also specify standalone ports that are not associated with a port group and uplink ports that are created on ESX hosts (`DVSConfigSpec.numStandalonePorts`).
     The `DVPortgroupConfigInfo.numPorts` property is the total number of ports for a distributed virtual switch. This total includes the ports generated by the static and dynamic portgroups and the standalone ports.
3. If you have created additional uplink portgroups, use the `ReconfigureDvs_Task` method to add the portgroup(s) to the `DVSConfigSpec.uplinkPortgroup` array.
4. Retrieve physical NIC device names from the host (`HostSystem.config.network.pnic[].device`).
5. Add host member(s) to the distributed virtual switch. To configure host members:
   - Specify hosts (`DVSConfigSpec.host[]`).
   - For each host, specify one or more physical NIC device names to identify the pNIC(s) for the host proxy connection to the network (`DistributedVirtualSwitchHostMemberConfigSpec.backing.pnicSpec[].pnicDevice`)
   - Use the `DistributedVirtualSwitch.ReconfigureDvs_Task` method to update the switch configuration. When you add a host to a distributed virtual switch (`DistributedVirtualSwitch.config.host`), the host automatically creates a proxy switch. The proxy switch is removed automatically when the host is removed from the distributed virtual switch.

6. Connect hosts and virtual machines to the distributed virtual switch.

Host connection	Specify port or portgroup connections in the host virtual NIC spec (`HostVirtualNicSpec.distributedVirtualPort` or `HostVirtualNicSpec.portgroup`).
Virtual machine connection	Specify port or portgroup connections in the distributed virtual port backing (`VirtualEthernetCardDistributedVirtualPortBackingInfo`) for the virtual Ethernet cards on the virtual machine (`VirtualEthernetCard.backing`).

Backup, Rollback, and Query Operations
If you are using a `VmwareDistributedVirtualSwitch`, you can perform backup and rollback operations on the switch and its associated distributed virtual portgroups.

When you reconfigure a VMware distributed virtual switch (`ReconfigureDvs_Task`), the server saves the current switch configuration before applying the configuration updates. The saved switch configuration includes portgroup configuration data. The server uses the saved switch configuration as a checkpoint for rollback operations. You can rollback the switch or portgroup configuration to the saved configuration, or you can rollback to a backup configuration (`EntityBackupConfig`).

- To backup the switch and portgroup configuration, use the `DistributedVirtualSwitchManager.DVSManagerExportEntity_Task` method. The export method produces a `EntityBackupConfig` object. The backup configuration contains the switch and/or portgroups specified in the `SelectionSet` parameter. To backup the complete configuration you must select the distributed virtual switch and all of its portgroups.
- To rollback the switch configuration, use the `DVSRollback_Task` method to determine if the switch configuration has changed. If it has changed, use the `ReconfigureDvs_Task` method to complete the rollback operation.
- To rollback the portgroup configuration, use the `DistributedVirtualPortgroup.DVPortgroupRollback_Task` method to determine if the portgroup configuration has changed. If it has changed, use the `ReconfigureDVPortgroup_Task` method to complete the rollback operation.

To perform query operations on a distributed virtual switch, use the `DistributedVirtualSwitchManager` methods.

VMware Standard Virtual Switch

Network setup for ESX hosts can consist of several parts:

- Setting up one or more virtual switches. Virtual switches provide the connectivity between virtual machines on the same host or on different hosts. Virtual switches also support VMkernel network access for VMotion, iSCSI, and NFS. You set up virtual switches independently on each host. See Adding a Standard Virtual Switch.
- Adding virtual machine port groups. A virtual machine always accesses the network through a port group. See Adding a Virtual Port Group.
- Specifying the adapter for the virtual machine. This adapter is specified as a virtual device, configured as part of virtual machine setup, and discussed in Configuring a Virtual Machine.
- Adding VMkernel network interfaces, for example, to support iSCSI storage or VMotion. See Adding a VMkernel Network Interface.
- Configuring a physical adapter (pnic), the actual connection from the host to the network. You can configure the pnic through the `HostNetworkSystem.pnic` property, which is a `PhysicalNic` data object. You can specify the set of pnics associated with a virtual switch through the `VirtualSwitch.pnic` property, which takes an array of physical network adapters.
- Network configuration for the host (IP routing, DNS, SNMP). See Adding Networking Services.

Configuring a Standard Virtual Switch
To use a VMware standard virtual switch, you use the following elements to configure the switch on an ESX host.

- `HostNetworkSystem` – Managed object that represents the host's networking configuration. This object's properties point to the networking data objects you can use for network management, including `HostDnsConfig` and `HostIpRouteConfig`.

`HostNetworkSystem` properties allow you to access `HostNetCapabilities` and `HostNetworkInfo` data objects, and access and modify the `HostNetworkConfig` data object.

`HostNetworkSystem` includes methods for retrieving and changing the network configuration. See the *API Reference* for a complete list of methods and the permissions required to run them.

- `HostNetworkConfig` – Allows you to specify the network configuration for the host. You can apply the configuration by running the `HostNetworkSystem.UpdateNetworkConfig` method.
- `Network` – Represents a network accessible by either hosts or virtual machines. This can be a physical network or a logical network, such as a VLAN.

  When you add a host to a vCenter system, or when you add a virtual machine to an ESX host, a `Network` is added automatically.
- `HostSystem.QueryHostConnectionInfo` and `Datacenter.QueryConnectionInfo` both return a `HostConnectInfo` data object, which describes the current network configuration.

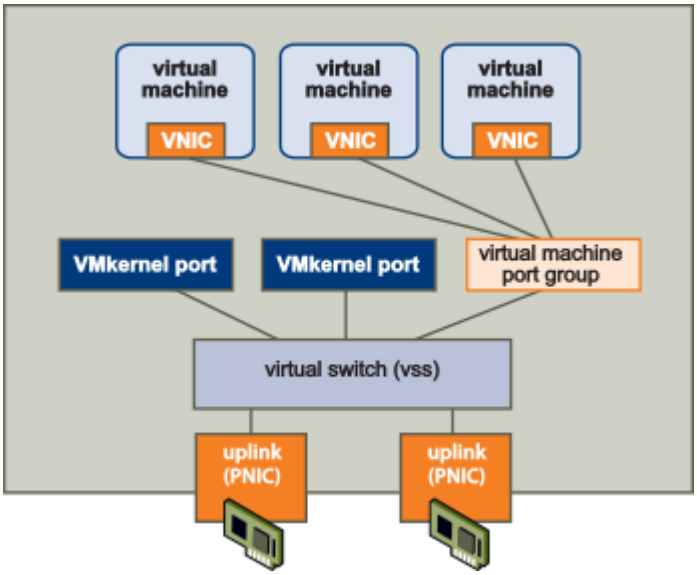
`HostSNMPSystem` – Supports SNMP setup. See Setting Up SNMP.

vNetwork Standard Switch Environment

A vNetwork Standard Switch (vSS) can route traffic internally between virtual machines and can link virtual machines to external networks.

The following diagram shows the elements of a vSS environment.

Figure 599: vSS Environment



Virtual Switches

At the center of networking with vSS is the virtual switch itself. The vSS can send network traffic between virtual machines on the same host (private network) or network traffic to an external network (public network). The public network uses the Ethernet adapter associated with the physical host (uplink adapter).

When two or more virtual machines are connected to the same vSS, network traffic between them is routed locally. If an uplink adapter is attached to the vSS, each virtual machine can access the external network that the adapter is connected to.

Setting Up Networking with vSS

You can use the `HostNetworkSystem` managed object to access and manipulate the elements of an ESX network.

Retrieving Information About the Network Configuration

You can retrieve information about the network configuration as follows:

- The properties of the `HostNetworkConfig` object, which you access through `HostNetworkSystem.networkConfig`, allow you to retrieve configuration information. This information is comprehensive and includes the physical adapters, virtual switches, virtual network interfaces, and so on.

  You can also use `HostNetworkConfig` to make changes to the configuration.
- The properties of the `HostNetworkInfo` object, which you access through `HostNetworkSystem.networkInfo`, allow you to retrieve runtime information.

Adding a Standard Virtual Switch

You call the `HostNetworkSystem.AddVirtualSwitch` method to add one or more virtual switches. Pass in the name of the virtual switch and a `HostVirtualSwitchSpec` data object as parameters.

Inside `HostVirtualSwitchSpec` you can specify the MTU, number of ports, network policy, and bridge specification. The bridge specifies how the virtual switch connects to the physical adapter. The currently supported bond bridge provides network adapter (NIC) teaming capabilities through the use of a list of physical devices and, optionally, a beacon probe to test connectivity with physical adapters.

After you have created the virtual switch, you can connect it to a pnic for connection to the outside, and to a VMkernel port or a port group.

To add a virtual switch, use the following steps.

1. Obtain information about the current networking configuration.

   You can use a property collector to retrieve the `HostNetworkSystem` managed object and several of its properties, such as `networkInfo`.
2. Define a `HostVirtualSwitchSpec` that specifies the attributes of the virtual switch. You can specify the number of ports (56 to 4088 on ESX systems) and the `HostNetworkPolicy`. See “Defining the Host Network Policies” on page 122.
3. Call `HostNetworkSystem.AddVirtualSwitch` to add a virtual switch. Specify a unique name and a `HostVirtualSwitchSpec` that defines the switch attributes.

   The following fragment from `AddVirtualSwitch.java` illustrates this.

   Adding a Virtual Switch

```
vswitchId = vSwitch42;
...
ManagedObjectReference nwSystem = configMgr.getNetworkSystem();
HostVirtualSwitchSpec spec = new HostVirtualSwitchSpec();
spec.setNumPorts(8);
service.addVirtualSwitch(nwSystem, vswitchId, spec);
System.out.println( " : Successful creating : " + vswitchId);
```

Adding a Virtual Port Group

Port groups allow you to differentiate between different kinds of traffic passing through a virtual switch. You can also use port groups as a boundary for communication or for security policy configuration. The default port groups for ESX systems are `Management Network` and `VM Network`.

When you create a port group, you can specify a VLAN ID for it. VLANs are an important part of ESX networking because they allow you to group traffic. For example, you could create separate network segments for VMotion, for management and for development. Using VLANS, you only need to have a separate uplink adapter for each network segment and a single virtual switch connecting to that adapter. That setup can greatly reduce the number of switches you need.

To add a virtual port group, use the following steps.

1. Define a `HostPortgroupSpec`. For each port group, you can specify the network policy, the VLAN ID, and the virtual switch to which the port group belongs.
2. Call `HostNetworkSystem.AddPortGroup`, passing in the `PortGroupSpec`.

Adding a VMkernel Network Interface

VMkernel network interfaces provide the network access for the VMkernel TCP/IP stack. You must create new VMkernel ports for your ESX system if you plan on using VMotion, VMware FT, or iSCSI and NAS storage. A VMkernel port consists of a port on the virtual switch and a VMkernel interface.

To add a VMkernel Network Interface to your ESX system, use the following steps.

1. Create a `HostVirtualNicSpec` data object. Inside the object, you can specify the IP configuration in a `HostIpConfig` data object. For vSS, specify the `portgroup` property. For vDS, specify the `distributedVirtualPort` property.
2. Call `HostNetworkSystem.AddVirtualNic`, passing in the `HostVirtualNicSpec`.
3. You can then use the VMkernel network interface for software iSCSI or NAS, or call the `HostVmotionSystem.SelectVnic` method to use this VMkernel NIC for VMotion.

   Adding a VMkernel Network Interface, a code fragment from the `AddVirtualNic` example, illustrates this. The sample retrieves the IP address from the command line using the `cb.get_option` call.

   Adding a VMkernel Network Interface

```
private HostVirtualNicSpec createVNicSpecification() {
     HostVirtualNicSpec vNicSpec = new HostVirtualNicSpec();
     HostIpConfig ipConfig = new HostIpConfig();
     ipConfig.setDhcp(false);
     ipAddr = cb.get_option("ipaddress");
     ipConfig.setIpAddress(ipAddr);
     ipConfig.setSubnetMask("255.255.255.0");
     vNicSpec.setIp(ipConfig);
     return vNicSpec;
}
     ....

HostVirtualNicSpec vNicSpec = createVNicSpecification();
service.addVirtualNic(nwSystem, portGroup, vNicSpec);
```

Defining the Host Network Policies

When you configure host networks, you can define specific policies for the network. The `HostNetworkPolicy` data object type describes network policies for both virtual switches and port groups. If the settings are not specified for the port group explicitly, the port group inherits policy settings from the virtual switch with which it is associated.

The policies are defined by the following data objects available as properties of `HostNetworkPolicy`.

- `HostNicTeamingPolicy` – Defines the connection to the physical network. This includes failure criteria, active and standby NICs, and failover and load balancing information. See NIC Teaming.
- `HostNetworkSecurityPolicy` – Defines the security policies for the network. See the *ESX Configuration Guide*.
- `HostNetworkTrafficShapingPolicy` – Establishes parameters for three traffic characteristics: average bandwidth, peak bandwidth, and maximum burst size.

You can also specify the VLAN policy by assigning an integer to the `HostPortgroupSpec.vlanid` property. The VMkernel takes care of tagging and untagging the packets as they pass through the virtual switch. See the `HostPortgroupSpec` and `HostNetworkPolicy` data objects in the *API Reference*.

NIC Teaming

Virtual machines connect to the public network through a virtual switch, which, in turn, connects to the physical network interface (pnic). When the physical adapter or the adapter's network connection fails, connectivity for the associated virtual switch and all port groups and virtual machines is lost.

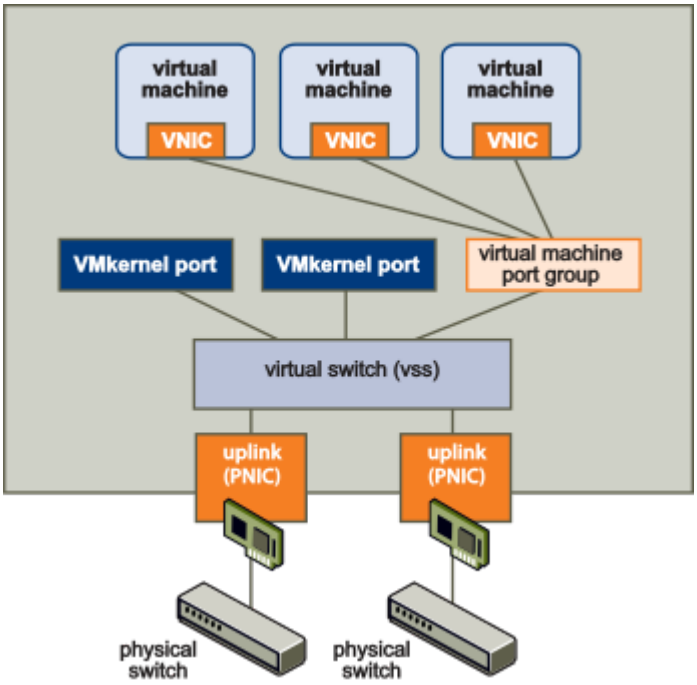
To resolve this issue, you can set up your environment so each virtual switch connects to two uplink adapters. Each uplink adapter connects to two different physical switches. The teams can then either share the load of traffic between physical and virtual networks among some or all of its members, or provide passive failover in the event of a hardware failure or a network outage.

You set up NIC teaming by setting the `HostNetworkPolicy`. The path to the `HostNicTeamingPolicy` is:

`HostConfigSpec.network.vswitch[].spec.policy.nicTeaming`

If you specify NIC teaming for a virtual switch, the `HostVirtualSwitchSpec.bridge` property must be set to `HostVirtualSwitchBondBridge`.

Figure 600: NIC Teaming



Setting Up IPv6 Networking

vSphere supports both Internet Protocol version 4 (IPv4) and Internet Protocol version 6 (IPv6) environments. With IPv6, you can use vSphere features such as NFS in an IPv6 environment.

An IPv6-specific configuration in vSphere involves providing IPv6 addresses, either by entering static addresses or by using DHCP for all relevant vSphere networking interfaces. IPv6 addresses can also be configured using stateless autoconfiguration sent by router advertisement.

You can set up IPv6 networking for a host by changing the `HostIpConfig.ipV6Config` property, which is a `HostIpConfigIpV6AddressConfiguration` data object. `HostIpConfigIpV6AddressConfiguration` allows you to specify whether auto-configuration is enabled, whether DHCP for ipV6 addresses is enabled, and an array of IPv6 addresses (`HostIpConfigIpV6Address` data objects).

`HostIpConfigIpV6Address` allows you to specify all aspects of the IPv6 address including the state of the address, the address (unless DHCP is enabled), life time, operation, origin, and prefix length. See the *API Reference*. The following code fragment illustrates setting the VMkernel NIC to get an automatic IPv6 address from router advertisements and through DHCP. The user provides the IP address on the command line when calling the program from which the fragment is taken. The sample retrieves the address using the `cb.get_option` utility applications call.

IPv6 Setup

```
private HostVirtualNicSpec createVNicSpecification() {
     HostVirtualNicSpec vNicSpec = new HostVirtualNicSpec();
     HostIpConfig ipConfig = new HostIpConfig();

     //setting the vnic to get an automatic ipv6 address from router advertisements
     // and through dhcp

     ipV6Config = new HostIpConfigIpV6AddressConfiguration();
     ipV6Config.setAutoConfigurationEnabled(true);
     ipV6Config.setDhcpV6Enabled(true);
     ipConfig.setIpV6Config(ipV6Config);
     vNicSpec.setIp(ipConfig);
```

```
        return vNicSpec;
    ....
```

Adding Networking Services

You can set up network services for your ESX system by using `HostConfigManager` properties and methods.

Adding an NTP Service

The `HostConfigManager.dateTimeSystem` property contains a `HostDateTimeSystem` data object. This object allows you to perform NTP and date and time related configuration.

- Query and update the date and time information by using one of the methods defined in `HostDateTimeSystem`.
- Modify the `HostDateTimeSystem.dateTimeInfo` property, which contains a `HostDateTimeInfo` object, to set up NTP. The NTP information is stored in the `HostDateTimeInfo.ntpConfig` property, which is a `HostNtpConfig` object. The `HostNtpConfig` objects's `server` property contains a list of time servers, specified by IP address or fully qualified domain name.

  Important:
  You can start and stop the NTP daemon and retrieve information about it by using the `HostServiceSystem` object.

Setting Up the IP Route Configuration

You can use the `HostNetworkSystem.UpdateIPRouteConfig` method to specify the IP route configuration for an ESX system. The method takes a `HostIPRouteConfig` data object as an argument. In this object, you can specify the default gateway address and the IPv6 gateway address.

Setting Up SNMP

Simple Network Management Protocol (SNMP) allows management programs to monitor and control networked devices. vCenter and ESX systems include different SNMP agents:

- The SNMP agent included with vCentercan send traps when the vCenter system is started or when an alarm is triggered on vCenter. The vCentervCenter SNMP agent functions only as a trap emitter and does not support other SNMP operations such as `GET`.
- ESX 4.0 includes an SNMP agent embedded in the host daemon (`hostd`) that can send traps and receive polling requests such as `GET` requests.

Versions of ESX released before ESX 4.0 included a Net-SNMP-based agent. You can continue to use this Net-SNMP-based agent in ESX 4.x with MIBs supplied by your hardware vendor and other third-party management applications. However, to use the VMware MIB files, you must use the embedded SNMP agent. To use the NET-SNMP based agent and embedded SNMP agent at the same time, make one of the agents listen on a nondefault port. By default, both agents use the same port.

The SDK supports SNMP agent configuration through the `HostSnmpSystem` managed object. This object includes two methods, `ReconfigureSnmpAgent` and `SendTestNotification`.

- `HostSnmpSystem.ReconfigureSnmpAgent` allows you to specify agent properties through a `HostSnmpConfigSpec`. That data object allows you to specify the SNMP port, read only communities, and the trap targets in an `HostSnmpDestination` object. The `HostSnmpDestination` object allows you to specify the community, and a host and port listening for notification.
- `HostSnmpSystem.SendTestNotification` allows you to test your configuration.

A `HostSnmpSystemAgentLimits` data object in the `HostSnmpSystem.limits` property specifies limits of the agent.

Sample Code for Networks

The following table lists the sample applications included with the vSphere SDK that demonstrate how to use some of the managed objects discussed in this chapter.

Java
(SDK\vsphere-ws\java\JAXWS\samples \com\vmware\host)
`AddVirtualNic.java`
`AddVirtualSwitch.java`
`AddVirtualSwitchPortGroup.java`
`RemoveVirtualNic.java`
`RemoveVirtualSwitch.java`
`RemoveVirtualSwitchPortGroup.java`

Virtual Machine Configuration

A virtual machine is a software computer that, like a physical computer, runs an operating system and applications. Virtual machines are compatible with all standard x86 computers. Each virtual machine encapsulates a complete computing environment and runs independently of underlying hardware.

VirtualMachine Management Objects and Methods

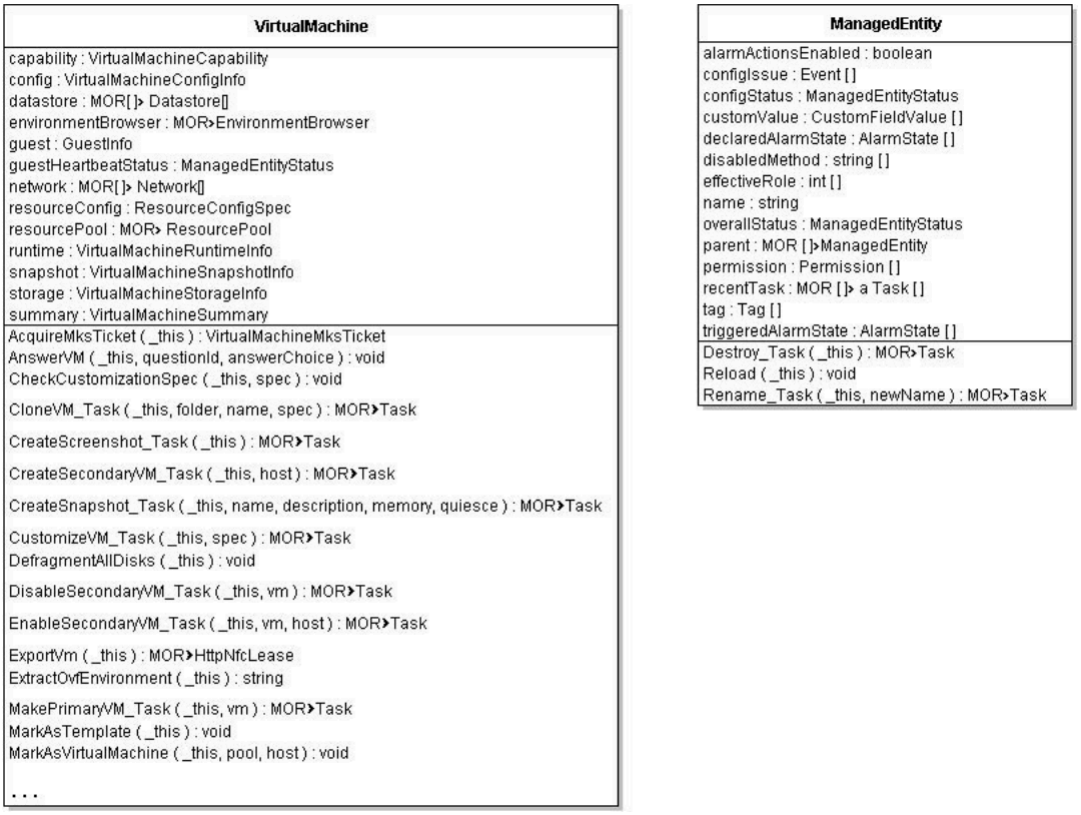
Virtual machines are the central elements of your vSphere environment.

You create a virtual machine by calling `Folder.CreateVM_Task`, and configure the virtual machine by using properties and methods of the `VirtualMachine` managed object. Most of the properties point to data objects that the methods use as input. Figure 601: VirtualMachine Managed Object with Some Properties and Methods shows some of the properties and methods.

Client applications commonly access and manipulate the following virtual machine related objects:

- `VirtualMachine` – Managed object used for most virtual machine manipulation. Includes methods that create templates, clones, or snapshots of a virtual machine, perform power operations and guest OS management, and install VMware Tools.
- `VirtualMachineConfigInfo` – Data object which allows you to retrieve configuration-specific information from a virtual machine.
- `VirtualMachineCloneSpec` – Data object which allows you to specify virtual machine properties for a clone operation. Argument to `VirtualMachine.CloneVM_Task`.

Figure 601: VirtualMachine Managed Object with Some Properties and Methods



Creating Virtual Machines and Virtual Machine Templates

To create a virtual machine, you use the `Folder.CreateVM_Task` method. The method takes a `VirtualMachineConfigSpec` data object as input argument. `VirtualMachineConfigSpec` allows you to specify the attributes of the virtual machine you are creating.

If you need several identical virtual machines, you can convert an existing virtual machine to a template and create multiple copies (clones) from the template. You can also create multiple virtual machines by cloning an existing virtual machine directly.

Creating a Virtual Machine Using VirtualMachineConfigSpec

Use the `Folder.CreateVM_Task` method to create a virtual machine by specifying its attributes. You must specify either a host or a resource pool (or both). The virtual machine uses the CPU and memory resources from the host or resource pool.

Calling the CreateVM_Task Method

Create a virtual machine by calling the `Folder.CreateVM_Task` method with the following arguments:

- `_this` —Folder where you want to place the virtual machine.
- `config`— `VirtualMachineConfigSpec` data object that specifies CPU, memory, networking, and so on. See Specifying Virtual Machine Attributes with VirtualMachineConfigSpec)
- `pool`— Resource pool for the virtual machine to draw resources from.
- `host`— `HostSystem` managed object that represents the target host on which to run the virtual machine. If you invoke this method on a standalone host, omit this parameter. If the target host is part of a VMware DRS cluster, this parameter is optional; if no host is specified, the system selects one.

> **Important:**
> All objects must be located in the same datacenter.

Specifying Virtual Machine Attributes with VirtualMachineConfigSpec

The actual customization of the virtual machine happens through the properties of the `VirtualMachineConfigSpec` that is passed in as an argument to `Folder.CreateVM_Task`. For example, you can specify the name, boot options, number of CPUs, and memory for the virtual machine. All properties of `VirtualMachineConfigSpec` are optional to support incremental changes. See the *API Reference*.

The following example fragment from the `VMCreate` sample program illustrates how to define a `VirtualMachineConfigSpec`.

Defining a VirtualMachineConfigSpec Data Object

```
VirtualMachineConfigSpec vmConfigSpec = new VirtualMachineConfigSpec();
     ...
     vmConfigSpec.setName("MyVM");
     vmConfigSpec.setMemoryMB(new Long(Integer.parseInt 500));
     vmConfigSpec.setNumCPUs(Integer.parseInt 4);
     vmConfigSpec.setGuestId(cb.get_option("guestosid"));
     ...
```

The VMware SDK `SDK/samples/Axis/java/com/vmware/apputils/vim/VMUtils.java` sample defines a more comprehensive virtual machine that also includes a Floppy, CD-ROM, disk, and virtual NIC. See Configuring a Virtual Machine for a discussion of commonly set properties.

When you create a virtual machine, the virtual machine files are added at the virtual machine's storage location. See Table 1888: Default Files.

Additional Configuration Information

The `VirtualMachineConfigInfo` and `VirtualMachineConfigSpec` objects provide the `extraConfig` property for additional configuration information. The `extraConfig` property is an array of key/value pairs that identify configuration options. The Server stores the `extraConfig` options in the `.vmx` file for the virtual machine. As the vSphere API evolves from version to version, an extraConfig option may become a standard configuration property that is part of the defined inventory data model. In this case, you must use the standard data model property for access; you cannot use the `extraConfig` property.

Cloning a Virtual Machine

A clone is a copy of a virtual machine. The main difference between a virtual machine and a clone is that the `VirtualMachine.config.template` property is set to `true`.

You can create a clone in one of the following ways:

- If you no longer need a specific instance of a virtual machine, but you want to use the virtual machine's configuration as a template, use the `VirtualMachine.MarkAsTemplate` method. This method sets the `config.template` property to `true`, and disables the virtual machine.
- If you want to use an existing virtual machine as a template, but keep the virtual machine, call the `VirtualMachine.CloneVM_Task` method to create a duplicate of the virtual machine.

If you use the `VirtualMachine.CloneVM_Task` method, you can customize certain attributes of the clone by specifying them in the `VirtualMachineCloneSpec` data object you pass in when you call the method.

The following code fragment from `VMClone.java` illustrates how you can customize a clone and specify a new location for it.

Cloning a Virtual Machine

```
VirtualMachineCloneSpec cloneSpec = new VirtualMachineCloneSpec();
VirtualMachineRelocateSpec relocSpec = new VirtualMachineRelocateSpec();
cloneSpec.setLocation(relocSpec);
cloneSpec.setPowerOn(false);
cloneSpec.setTemplate(false);

String clonedName = cloneName;
```

```
ManagedObjectReference cloneTask
            = service.cloneVM_Task(vmRef, vmFolderRef, clonedName, cloneSpec);
```

The `VirtualMachine.CloneVM_Task` method takes the source virtual machine, target folder, name, and `VirtualMachineCloneSpec` as arguments.

The `VirtualMachineCloneSpec` data object includes the location, power state, and whether the clone should be a template. The location, in turn, is a `VirtualMachineRelocateSpec` data object that specifies the target location (datastore, disk, and host or resource pool) and any transformation to be performed on the disk.

Tagging a New Virtual Machine

As of vSphere 9.1, programs can assign tags as part of virtual machine creation or cloning. Previously this was a two-step process. One-step makes it easier for infrastructure administrators to assign tags in the VM creation wizard.

Both `CreateVM_Task` and `CloneVM_Task` now take an additional argument to specify tag specifications. Tag association is attempted before longer-running copy file operations. Failure of tag association will prevent VM create or clone from proceeding, and fail the task.

```
/* Optional array of tag specs to be assigned to a VM. */
@optional vim.TagSpec[] tagSpecs;
```

Tags can be identified either with `NameID` (tuple of category and tag names) or `tagUuid` (unique identifier for tag). The Operation type for `tagSpec` should be "add" for tags association.

- For tag association with `CreateVM_Task`, the `tagSpecs` specified in parameter `config(VirtualMachineConfigSpec)` must be filled.
- For tag association with `CloneVM_Task`, the `location.tagSpecs` specified in parameter `spec(VirtualMachineCloneSpec)` must be filled.

Tagging is a vCenter feature, so tagging operations are a no-op when VM create or VM clone operations are directly called on ESX hosts.

Continuous Virtual Machine Clone

Clone is one of the long-running virtual machine operations. It can interfere with or be interrupted by server reboot. As of vSphere 8.0 U3, continuous VM clone promises to streamline maintenance mode operations.

Continuous virtual machine clone ensures that:

- VM operation automatically restarts without customer intervention after vCenter restarts.
- From a user perspective, restarted operation is the same as initially requested. Customers can use existing scripts and interfaces to manage the operation.
- Incoming operations are not affected while the system is recovering interrupted operation.

Going forward, continuous virtual machine clone is the default, but can be deactivated with a `vpxd` configuration option. If instructed by technical support personnel for example, administrators can set `config.provisioning.continuousop.enabled` to `false` in the `vpxd.cfg` configuration file. This can be done with the vSphere Client; see "Configure Advanced Settings" in VMware docs.

Here are the steps for continuous operation. Pre-workflow (1-3) operation was interrupted before the workflow was executed. Mid-workflow (4-5): operation was interrupted in the middle of workflow execution. Post-workflow (6): operation was interrupted when workflow finished but before task was updated.

1. Session preservator saves user session. Task management associates the task with returned UUID corresponding to the preserved session.
2. Session preservator acquires a persistent token (or other equivalents) to preserve the identity.
3. Workflow is journalled in `VPX_JOURNAL` and associated tables. The first action in workflow is `RedoableAction`, which is responsible for restarting the operation during recovery. Transaction manager loads workflow journals from database. For each pending continuous operation, it flags corresponding managed entities in recovery so that the entities do not participate in conflicting operations while recovery is in progress.

4. Restart begins. Recovery long-running operation revives persisted session. The session becomes the current user session for later privilege checks. Recovery cleans up previous operation that was interrupted, publishing any cleanup issues encountered in events or tasks.
5. `RedoableAction` checks if the operation should be restarted. If ye,s it will continue to the rest of recovery, otherwise it sets the task error status. Conditions when an operation should be restarted are an implementation detail, for example, it might not restart if the maximum number of retries was exceeded.
6. If the interruption occurs right before task status is updated, the situation is handled by performing the workflow commit and task update in a single transaction.

As implemented, managed object `RedoableAction` extends `UndoableAction`. In the vSphere Web Services API Reference, see data object `TaskInfo` for details about continuous operations and `VM_CLONE_CONTINUOUS`.

Creating Virtual Machine Templates

Templates allow you to create multiple virtual machines with the same characteristics, such as resources allocated to CPU and memory, or type of virtual hardware. A virtual machine template is a virtual machine that cannot be powered on and that is not associated with a resource pool.

You can convert any powered off virtual machine to a template by calling `VirtualMachine.MarkAsTemplate`. After the conversion, the original virtual machine no longer exists. You can use the template to create multiple clones of the same configuration.

Converting a Template to a Virtual Machine

You can change a template back to an operational virtual machine.

- To convert the template to a virtual machine, call the `MarkAsVirtualMachine` method on the template. You must specify a resource pool and, optionally, a host for the virtual machine. Host and resource pool must be under the same `ComputeResource`. When the operation completes, the template no longer exists.
- To keep the template, clone the template by calling the `CloneVM_Task` method on the template. In the `VirtualMachineCloneSpec` (the `spec` parameter), set the `template` property to `false`.

Accessing Information About a Virtual Machine

After you have created a virtual machine, you can retrieve information about the virtual machine through the `VirtualMachineConfigInfo` properties. See the *API Reference* for a complete list.

Checking Default Files

After you have created a virtual machine, several files are generated and placed in the directory specified in the `VirtualMachineConfigSpec.files` property.

Table 1888: Default Files

File	Usage	File Description	File Format
.vmx	*.vmname*`.vmx`	Virtual machine configuration file.	ASCII
`.vmxf`	*vmname*`.vmxf`	Additional virtual machine configuration files, available, for example, with teamed virtual machines.	ASCII
`.vmdk`	*vmname* `.vmdk`	Virtual disk file.	ASCII
`.flat.vmdk`	*vmname* `.flat.vmdk`	Preallocated virtual disk in binary format.	Binary
`.vswp`	*vmname* `.vswp`	Swap file.	
.nvram	*vmname*`.nvram` or `nvram`	Non-volatile RAM. Stores virtual machine BIOS information.	
.vmss	*vmname* `.vmss`	Virtual machine suspend file.	
.log	vmware.log	Virtual machine log file.	ASCII
#.log	vmware-#.log	Old virtual machine log files. # is a number starting with 1.	ASCII

File	Usage	File Description	File Format
.vmtx	*vmname*.vmtx	Virtual machine template file.	ASCII

If you are using snapshots, the following additional files might be available. See Snapshots.

File Extension	Usage	File Description
`.vmsd`	*vmname.* `vmsd`	Virtual machine snapshot file.
`.vmsn`	*vmname.* `vmsn`	Virtual machine snapshot data file.
`**.delta.vmdk`		Snapshot difference file. A number preceding the extension increases with more snapshots.
`**.vmdk`		Metadata about a snapshot.
-Snapshot#.vmsn		Snapshot of virtual machine memory. Snapshot size is equal to the size of you virtual machine's maximum memory.

Checking Default Devices

When you create a virtual machine, you are also creating a set of default devices, based on the hardware version associated with your SDK. You can see these devices using the `EnvironmentBrowser.QueryConfigOption` method. For example, the IDE controllers are created by default. Many of these default devices contain properties that you cannot change.

However, you can add the following optional devices to the default set: `VirtualSerialPort`, `VirtualParallelPort`, `VirtualFloppy`, `VirtualCdrom`, `VirtualUSB`, `VirtualEthernetCard`, `VirtualDisk`, and `VirtualSCSIPassthrough`. See the VirtualDevice Data Object in the *API Reference* for more information about each of these optional devices.

> **CAUTION:**
> Do not try to change default device properties using the `VirtualMachineConfigSpec.deviceChange` method discussed in Adding Devices to Virtual Machines, because the `deviceChange` method is not applicable to default device properties.

Configuring a Virtual Machine

You can configure a virtual machine during creation (`Folder.CreateVM_Task`) or cloning (`VirtualMachine.CloneVM_Task`). You can also reconfigure a virtual machine using `VirtualMachine.ReconfigVM_Task`.

In vSphere 5.5 and later, the `ReconfigVM_Task` method will throw an error when it attempts to change certain virtual machine properties while the virtual machine is powered on. In previous releases, the server would modify the properties in the configuration specification and the changes would take effect after the virtual machine resets, reboots, or performs a fast suspend and resume.

Since vSphere 5.5, a reconfigure operation modifies the virtual machine in real time, so that the virtual machine properties have been changed by the time the method returns control to the client.

You cannot reconfigure a powered on virtual machine successfully if you specify any of the following properties in the `VirtualMachineConfigSpec` when you call the `ReconfigVM_Task` method.

Property	VirtualMachineConfigSpec Path
VirtualDevice.unitNumber	deviceChange.device.unitNumber
VirtualDevicePciBusSlotInfo.pciSlotNumber	deviceChange.device.slotInfo.pciSlotNumber

Property	VirtualMachineConfigSpec Path
VirtualDiskFlatVer1BackingInfo.diskMode VirtualDiskFlatVer2BackingInfo.diskMode VirtualDiskRawDiskMappingVer1BackingInfo.diskMode VirtualDiskSeSparseBackingInfo.diskMode VirtualDiskSparseVer1BackingInfo.diskMode VirtualDiskSparseVer2BackingInfo.diskMode	deviceChange.device.backing.diskMode
VirtualDiskFlatVer2BackingInfo.digestEnabled VirtualDiskSeSparseBackingInfo.digestEnabled	deviceChange.device.backing.digestEnabled
VirtualMachineConfigSpec.changeTrackingEnabled	changeTrackingEnabled
VirtualEthernetCard.addressType	deviceChange.device.addressType
VirtualEthernetCard.macAddress	deviceChange.device.macAddress
VirtualEthernetCard.wakeOnLanEnabled	deviceChange.device.wakeOnLanEnabled
VirtualSCSIController.sharedBus	deviceChange.device.sharedBus
VirtualSerialPort.yieldOnPoll	deviceChange.device.yieldOnPoll
VirtualUSBController.autoConnectDevices VirtualUSBXHCIController.autoConnectDevices	deviceChange.device.autoConnectDevices
VirtualUSBController.ehciEnabled	deviceChange.device.ehciEnabled
VirtualMachineVideoCard.useAutoDetect	deviceChange.device.useAutoDetect
VirtualMachineVideoCard.videoRamSizeInKB	deviceChange.device.videoRamSizeInKB
VirtualMachineVideoCard.numDisplays	deviceChange.device.numDisplays
VirtualMachineVideoCard.use3dRendererSupported	deviceChange.device.use3dRendererSupported

The *API Reference* lists all properties and includes information about required permissions for these configuration methods. The following sections describe some commonly specified attributes.

Name and Location

You can specify the display name for the virtual machine by setting the `VirtualMachineConfigSpec.name` property. Any % (percent) character used in this `name` parameter must be escaped, unless it is used to start an escape sequence. Clients can also escape any other characters in this name parameter.

Use the `annotation` field to provide a description of the virtual machine. To remove an existing description, specify the empty string as the value of `annotation`.

The location of the virtual machine is determined implicitly during creation because you call a `Folder.CreateVM_Task` method and specify resource pool and optional target host the virtual machine should belong to. See Resource Management for a discussion of resource pools and virtual machine location.

Hardware Version

The hardware version of a virtual machine indicates the lower-level virtual hardware features a virtual machine supports, such as BIOS, number of virtual slots, maximum number of CPUs, maximum memory configuration, and other hardware characteristics.

For a newly created virtual machine, the default hardware version is the most recent version available on the host where the virtual machine is created. To increase compatibility, you might want to create a virtual machine with a hardware version older than the highest supported version. You can do so by specifying the `VirtualMachineConfigSpec.version` property during virtual machine creation. For existing virtual machines, call the `VirtualMachine.UpgradeVM_Task` method.

The hardware version of a virtual machine can be lower than the highest version supported by the ESX host it is running on under the following conditions:

- You migrate a virtual machine to a newer version of ESX after it was created on a host that was running an earlier version of ESX.
- You create a virtual machine on a newer version of ESX by using an existing virtual disk that was created on a host that was running an earlier version of ESX.
- You add a virtual disk created on a host that was running an earlier version of ESX to a virtual machine created on a newer version of ESX.

Virtual machines with hardware versions lower than 4 can run on ESX 4.x hosts but have reduced performance and capabilities. In particular, you cannot add or remove virtual devices on virtual machines with hardware versions lower than 4 when they reside on an ESX 4.x host. To make full use of these virtual machines, upgrade the virtual hardware.

Boot Options

You can control a virtual machine's boot behavior by setting the `VirtualMachineConfigSpec.bootOptions` property. The `VirtualMachineBootOptions` data object in that property allows you to specify the following properties:

- `bootDelay` – Delay before starting the boot sequence, in milliseconds.
- `bootRetryDelay` – Delay before a boot retry, in milliseconds. This property is only considered if the `bootRetryEnabled` property is set to `true`.
- `bootRetryEnabled` – If set to `true`, a virtual machine that fails to boot tries again after the `bootRetryDelay` time period has elapsed.
- `enterBIOSSetup` – If set to `true`, the virtual machine enters BIOS setup the next time it boots. The virtual machine resets this flag to `false` so subsequent boots proceed normally.

Operating System

The guest operating system that you specify affects the supported devices and available number of virtual CPUs. You specify the guest operating system in the following two properties:

- `guestosid` – Specify one of the constants in the `VirtualMachineGuestOsIdentifier` as a string.
- `alternateGuestName` – Full name for the guest operating system. Use this property if `guestosid` is one of the values of `VirtualMachineGuestOsIdentifier` starting with `other*`.

CPU and Memory Information

To manage compute and memory resources at the virtual machine level, you use several properties of the `VirtualMachineConfigSpec` data object. These properties specify the CPU and memory configuration for the virtual machine. You can also specify the operational boundaries for resource allocation, and the priority of the virtual machine in case of resource contention.

Configuring Virtual CPUs and Memory

To configure the capabilities of a virtual machine, you use properties of the `VirtualMachineConfigSpec` data object to specify the machine characteristics available to the guest operating system. The guest uses these capabilities in the same way as resources on a physical machine.

Configuring Virtual CPUs

Set the number of virtual CPU cores for the virtual machine with the `VirtualMachineConfigSpec.numCPUs` property. Legal values for this property change depending on the `guestosid` value you specify. If you use `VirtualMachineConfigSpec` to update the virtual machine properties, you can omit this property to leave it unchanged.

The guest operating system acts as if it had `numCPUs` cores available at all times, but the host's physical resources are shared by all its virtual machines. The host allocates physical cores in time slices aa backing for virtual cores. For information about how to specify guidance for resource allocation, see Configuring Resource Allocation Constraints for Virtual Machines.

Configuring Multi-Core CPUs

Set the number of cores per CPU chip with the `VirtualMachineConfigSpec.numCoresPerSocket` property. The value must be an integral divisor of `VirtualMachineConfigSpec.numCPUs`. The default value is `1` if the property is

omitted for virtual machine creation. If you use `VirtualMachineConfigSpec` to update the virtual machine properties, you can omit this property to leave it unchanged.

Configuring Memory

Set the RAM size for a virtual machine with the `VirtualMachineConfigSpec.memoryMB` property. If you use `VirtualMachineConfigSpec` to update the virtual machine properties, you can omit this property to leave it unchanged.

The guest operating system acts as if it had `memoryMB` available at all times, but the host's physical resources are shared by all its virtual machines. The amount of physical memory available as backing for virtual machines can vary over time, and it can affect virtual machine performance. For information about how to specify guidance for resource allocation, see Configuring Resource Allocation Constraints for Virtual Machines.

CPU Processors and Memory Affinity

If your virtual machine is on an ESX system, and if you have a license that supports Symmetric Multiprocessors (SMP), you can configure the virtual machine to have multiple virtual CPUs by setting `cpuAffinity` and `memoryAffinity`. You define a set of integers that represents the processors (for CPU) and NUMA nodes (for memory). If you are reconfiguring the affinity setting and leave the array empty, any existing affinity is removed. See the *Resource Management Guide* for a discussion of NUMA nodes and affinity.

CPU Features

You can use the `VirtualMachineConfigSpec.cpuFeatureMask[].info` property to represent the CPU features requirements for a virtual machine or guest operating system. See the `HostCpuIdInfo` data object discussion in the *API Reference* for a detailed discussion.

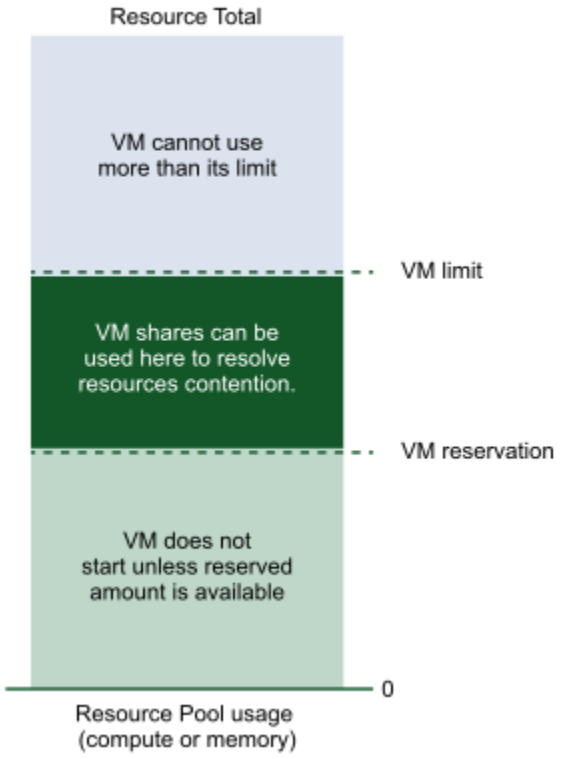
CPU and Memory Modification for Running Virtual Machines

Set `CpuHotAddEnabled` and `CpuHotRemoveEnabled` to specify whether virtual processors can be added to or removed from a virtual machine while the virtual machine is running. Set `MemoryHotAddEnabled` to specify whether memory can be added while the virtual machine is running.

Configuring Resource Allocation Constraints for Virtual Machines

The host hypervisor allocates physical resources as backing for the virtual resources needed by guest operating systems. You can specify resource allocation preferences that guide and constrain the hypervisor when allocating resources to running virtual machines.

To specify allocation preferences for CPU and memory, use the `cpuAllocation` and `memoryAllocation` properties of `VirtualMachineConfigSpec`. Both properties are data objects of type `ResourceAllocationInfo` objects. The `ResourceAllocationInfo` object has properties to specify upper and lower allocation limits, and properties to specify relative priorities when virtual machines contend for resources.



To specify boundaries for resource allocation, use these properties of `ResourceAllocationInfo`:

- `reservation` – Lower limit for resource allocation to the virtual machine. If the hypervisor cannot reserve this amount, it does not start the virtual machine. If the running virtual machine uses less than its reserved amount, other running virtual machines can use a part of the reserved resources temporarily.
- `limit` – Upper limit for CPU or memory resources assigned to this virtual machine. The virtual machine does not exceed this limit, even if unused resources are available. This property is typically used to limit the impact of the virtual machine on other running virtual machines. If the property is set to -1, the configured virtual memory size or the configured number of virtual cores limits the resource allocation.

  Note:
  Although the number of virtual CPU cores limits the compute resource allocated, the virtual bandwidth allocated also depends on the speed of the physical CPU cores assigned as backing.

To specify preferences for resource allocation in case of contention, use the `shares` property of `ResourceAllocationInfo`. The `shares` property is a nested data object of type `SharesInfo`, which specifies a relative priority for resolving resource contention between virtual machines. The `SharesInfo` data object has two properties, `level` and `shares`:

- `level` – An enum type with four potential values: `high`, `normal`, `low`, and `custom`. If you specify the value `custom`, you must also specify the `shares` property. If you specify any other value for `level`, the `shares` property is ignored.
- `shares` – In a situation of resource contention, the `shares` value is compared against the `shares` values of other virtual machines, and resources are allocated first to the virtual machine with the highest `shares` value.

The most direct way to specify resource priority is to set the `shares` values manually. This is most useful in environments where all virtual machines use custom `shares` values. In a mixed environment, the configuration settings do not compare directly, which can lead to unexpected results.

A more sophisticated way of specifying priority is to specify `high`, `normal`, or `low` for the `level` property. These settings cause the host to calculate the shares of a virtual machine in a way that factors in the configured memory size or the configured virtual CPU cores of the virtual machine. The result gives more priority shares to virtual machines with larger configurations.

Note:
Both the custom shares values and the calculated shares values prioritize virtual machines only among their siblings running in the same resource pool. For a system of virtual machines running only in the default root resource pool, determining priority is fairly simple. For data centers with nested resource pools, a virtual machine's priority in the larger context of all running virtual machines depends also on the priority of its parent resource pool.

For a detailed examination of resource pools and how they affect priority shares calculations, see Resource Management. The *Resource Management Guide* includes a detailed discussion of resource allocation in the vSphere environment.

Networks
You configure network settings so that a virtual machine can communicate with the host and with other virtual machines.

Virtual Network Interfaces
You can add a virtual network interface to a virtual machine using a subclass of `VirtualEthernetCard`, you can set the `addressType` to `Manual`, `Generated`, or `Assigned`. If you choose `Assigned`, you can specify a MAC address explicitly.

For more information about virtual devices in general, see Adding Devices to Virtual Machines.

The number of virtual network interfaces depends on the hardware version you specify for a virtual machine. Hardware version 7 virtual machines support up to ten virtual NICs. Hardware version 4 virtual machines support up to four virtual NICs.

Virtual Machine MAC Address
Upon virtual machine creation, ESX or vCenter systems assign each virtual network interface its own unique MAC address. The first three bytes of the MAC address that is generated for each virtual network adapter consists of a manufacturer-specific Organizationally Unique Identifier (OUI). The MAC address-generation algorithm produces the other three bytes. vSphere generates MAC addresses that are checked for conflicts. After the MAC address has been generated, it does not change unless the virtual machine is moved to a different location.

All MAC addresses that have been assigned to virtual network interfaces of running and suspended virtual machines on a given physical machine are tracked. The MAC address of a powered off virtual machine is not checked against those of running or suspended virtual machines. It is possible that a virtual machine acquires a different MAC address after a move.

The *ESX Configuration Guide* discusses virtual machine MAC addresses in detail.

File Locations
File locations for a virtual machine are specified in the following properties:

- `VirtualMachineConfigSpec.files` is a `VirtualMachineFileInfo` data object that allows you to specify the log directory, snapshot directory, suspend directory, and configuration file location. Most locations have a default that you can change as needed.
- `VirtualMachineConfigSpec.locationID` is a 128-bit hash based on the virtual machine’s configuration file location and the UUID of the host the virtual machine is assigned to. This property is not usually set by developers; however, clearing this property by setting it to an empty string is recommended if you move the virtual machine.

If a virtual machine’s `VirtualMachineCapability.swapPlacementSupported` property is `true` for a virtual machine, you can specify a value for the `VirtualMachineConfigSpec.swapPlacement` property. The value must be one of the values of the `VirtualMachineConfigInfoSwapPlacementType` enumeration, as a string.

Adding Devices to Virtual Machines

You can add devices to a virtual machine during creation using the `VirtualMachineConfigSpec.deviceChange` property, which is a `VirtualDeviceSpec`. You specify the host device that the virtual device should map to by using a backing object. A backing object represents the host device associated with a virtual device.

- Backing option objects – You can find out which devices the host supports by extracting the relevant backing option object.

- Backing information object – The backing information object allows you to supply data for virtual device configuration. You access a `VirtualDeviceBackinInfo` object as follows:
  `VirtualMachineConfigSpec.deviceChange[].device.backing`

To add a device to a virtual machine, you must first find out which devices are supported on the corresponding ESX host, and then specify a `VirtualDevice` object. Perform these tasks to add a device to a virtual machine:

1. Find out which devices your ESX system supports by calling the `QueryConfigOption` method, which you can access through the `VirtualMachine.environmentBrowser` property. The method returns a `VirtualMachineConfigOption` data object that specifies what the ESX ESX supports. For example, `VirtualMachineConfigOption.hardwareOptions` includes information about supported CPU and memory and an array of `VirtualDeviceOption` data objects.

   Note:

   You cannot use the `QueryConfigOption` method to create another instance of a default device. If you attempt to add a default device, such as an IDE controller, the server ignores the operation.

2. Specify the backing information object for the device. The actual process for defining the object differs for different objects. For example, for a CD-ROM passthrough device, you use a `VirtualCdromPassthroughBackingInfo` device. The `VirtualDevice.backing` property is a `VirtualDeviceBackingInfo` object which is extended by devices.

   The following code fragment adds a CD-ROM passthrough device:

   ```
   VirtualCdromPassthroughBackingInfo vcpbi = new VirtualCdromPassthroughBackingInfo();
   // Does the virtual device have exclusive access to the CD-ROM device?
   vcpbi.setExclusive(false);
   // Specifies the device name.
   vcpbi.setDeviceName('cdrom0');
   ```

3. Specify connection information for the device.

   The `VirtualDevice.connectable` property is a `VirtualDeviceConnectInfo` data object. This object provides information about restrictions on removing the device while a virtual machine is running. This property is `null` if the device is not removable.

   ```
   VirtualDeviceConnectInfo vdci = new VirtualDeviceConnectInfo();
   // Allow the guest to control whether the virtual device is connected?
   vdci.setAllowGuestControl(false);
   // Is the device currently connected?
   vdci.setConnected(true);
   // Connect the device when the virtual machine starts?
   vdci.setStartConnected(true);
   ```

4. Define the controller key, the virtual device key, and the unit number.

   You define these items with the integer properties: `controllerKey`, `key`, and `unitNumber`. See the `VirtualDevice` data object in the *API Reference*.

5. Specify device Information.

   The `deviceInfo` property is a `Description` data object that has a `name` property and a `summary` property. You can supply a string value for each, describing the device.

   ```
   Description vddesc = new Description();
   vddesc.setLabel('CD-ROM Device cdrom0');
   vddesc.setSummary('The CD-ROM device for this virtual machine.');
   ```

6. Specify the virtual device as the `device` property of a `VirtualDeviceConfigSpec`.

7. Specify the `VirtualDeviceConfigSpec` as the `deviceChange` property to the `VirtualMachineConfigSpec` that you pass in to a `Folder.CreateVM_Task` or `VirtualMachine.ReconfigVM_Task` method.

   Here's the complete code fragment for a CD-ROM passthrough device:

```
VirtualDevice vd = new VirtualDevice();
vd.setBacking(vcpbi);
vd.setConnectable(vdci);
vd.setControllerKey(257);
vd.setDeviceInfo(vddesc);
vd.setKey(2);
vd.setUnitNumber(25);
```

Performing Virtual Machine Power Operations

Just like physical machines, virtual machines have power states.

- Powered on – The virtual machine is running. If no OS has been installed, you can perform OS installation as you would for a physical machine.
- Powered off – The virtual machine is not running. You can still update the software on the virtual machine's physical disk, which is impossible for physical machines.
- Suspended – The virtual machine is paused and can be resumed; like a physical machine in standby or hibernate state.

> **Important:**
> Before you power on a virtual machine, you must make sure that the host has sufficient resources. You must have enough memory for the virtual machine, and some memory overhead. See Querying Virtual Machine Memory Overhead.

`VirtualMachine` power operations allow you to change the power state. Each operation is sensitive to the current power state, for example, powering on a powered off virtual machine has the desired result while powering on a powered on virtual machine results in an error. You must check the current state before you run one of these tasks.

- `PowerOnVM_Task` – Powers on a virtual machine. If the virtual machine is suspended, this method resumes execution from the suspend point.
- `PowerOffVM_Task` – Powers off a virtual machine.
- `ResetVM_Task` – Resets power on this virtual machine. If the current state is `poweredOn`, `ResetVM_Task` first performs a hard `powerOff` operation. After the power state is poweredOff, `ResetVM_Task` performs a `powerOn` operation. Although this method functions as a `powerOff` followed by a `powerOn`, the two operations are atomic with respect to other clients, meaning that other power operations cannot be performed until the reset method completes.
- `SuspendVM_Task` – Suspends the virtual machine. You can later power on the suspended virtual machine to the same state.

Virtual machines are often configured to start up the guest operating system when they are started, and try to shut down the guest operating system when being shut down. However, starting and stopping a virtual machine differs from starting and stopping the guest operating system (see Customizing the Guest Operating System).

> **Important:**
> Power operations might affect other virtual machines that are participating in a DRS cluster or VMware HA. See Resource Management for information about DRS clusters and VMware HA.

You can use the `Datacenter.PowerOnMultiVM_Task` to power on multiple virtual machines in a datacenter. Pass an array of `VirtualMachine` managed object references and an array of option values to the method. If any of the virtual machines in the list is manually managed by VMware DRS, the system generates a DRS recommendation that the user needs to apply manually. Standalone or DRS disabled virtual machines are powered on for the current host. Virtual machines managed by DRS, to be placed by DRS, are powered on for the recommended host.

Registering and Unregistering Virtual Machines

When you create a virtual machine, it becomes part of the inventory (inside the folder from which you called the creation method by default), and it is registered. If you copy virtual machine files to relocate the virtual machine, or if you remove the files from the inventory using the vSphere Client, it becomes unregistered and unusable. You cannot power on a virtual machine that is not part of the inventory.

To restore the virtual machine to the inventory, and make it usable again, you can use the `RegisterVM_Task` method, defined in the `Folder` managed object. You can register the virtual machine to a host or to a resource pool. You can register the virtual machine as a template if you want to use it to clone other virtual machines from.

The `ColdMigration.java` sample illustrates both registering and reconfiguring a virtual machine. At the heart of the sample is the following call, which registers the virtual machine. Arguments include the virtual machine's current folder, datastore path, and name, whether to register as a template, and the resource pool or host to register the machine in.

```
ManagedObjectReference taskmor = cb.getConnection().getService().registerVM_Task(
                  vmFolderMor,vmxPath,getVmName(),false,resourcePool,host);
```

After registration, the virtual machine takes its resources (CPU, memory, and so on) from the resource pool or host to which it is registered.

The `RemoveManagedObject.java` sample illustrates unregistering a virtual machine.

Customizing the Guest Operating System

You install the guest operating system on the virtual machine just as you would install it on a physical machine. Afterwards, you can use the Web Services API to retrieve information and perform some customization if VMware Tools is installed on top of the guest operating system.

`VirtualMachine` includes the following methods for managing the guest operating system:

- `ShutdownGuest` and `RebootGuest` shut down and reboot the guest OS, and `StandbyGuest` puts the guest in hybernate mode. In each case, you perform the action on the guest OS. For example, you might shut down Windows but leave the virtual machine running.
- `ResetGuestInformation` clears cached guest information. Guest information can be cleared only if the virtual machine is powered off. Use this method if stale information is cached, preventing reuse of an IP address or MAC address.
- `SetScreenResolution` sets the console screen size of the guest operating system. When you call this method, the change is reflected immediately the virtual machine console you can access in the vSphere Client.

You can customize the identity and network settings of the guest OS with the `CustomizationSpec` data object that is a parameter to `VirtualMachine.CustomizeVM_Task`. The `CustomizationSpec` is also a property of the `VirtualMachineCloneSpec` you pass in when cloning a virtual machine.

The settings you customize with this method are primarily virtual machine settings, but because the virtual machine and the guest OS share the information, you are also customizing the guest OS with this method.

The `CustomizationSpec` allows you to set the following properties:

- `encryptionKey` – Array of bytes that can be used as the public key for encrypting passwords of administrators.
- `globalIPSettings` – Contains a `CustomizationGlobalIPSettings` data object which specifies a list of DNS servers and a list of name resolution suffixes for the virtual network adapter.
- `identity` – Allows you to specify the network identity and settings, similar to the Microsoft Sysprep tool.
- `nicSettingMap` – Custom IP settings that are specific to a particular virtual network adapter.
- `options` – Optional operations (either LinuxOptions or WinOptions).

Installing VMware Tools

VMware Tools is a suite of utilities that enhances the performance of a virtual machine's guest operating system and improves virtual machine management. For each guest OS, VMware provides a specific binary-compatible version of VMware Tools. The SDK requires that you install VMware Tools, or some operations related to the guest operating system fail.

> **Important:**
> You must install the guest operating system before you install VMware Tools.

With VMware Tools installed on the guest OS, the virtual machine obtains its DNS (domain name server) name and an IP address and is therefore reachable over the network.

`VirtualMachine` includes three methods for automating installation and upgrade of VMware Tools.

- `MountToolsInstaller` – Mounts the VMware Tools CD installer as a CD-ROM for the guest operating system. To monitor the status of the tools installallation, check `GuestInfo.toolsStatus`. Check `GuestInfo.toolsVersionStatus` and `GuestInfo.toolsRunningStatus` for related information.
- `UnmountToolsInstaller` – Unmounts the VMware Tools installer CD.
- `UpgradeToolsTask` – Performs an upgrade of VMware Tools. This method assumes VMware Tools has been installed and is running. The method takes one argument, `InstallerOptions`, which allows you to specify command-line options passed to the installer to modify the installation procedure for tools.

Use the`ToolsConfigInfo` data object in `VirtualMachineConfigSpec.toolsInfo` property to specify the settings for the VMware Tools software running on the guest operating system.

As of VCF 9.1, three new `GuestRebootStatus` fields are available in `GuestInfo` to streamline the operation of VMware Tools. This information is not available with old versions of VMware Tools, nor in some cases for uninstall requests.

- `rebootRequested` – Indicates that a VMware Tools install, upgrade, or uninstall requires reboot of the guest operating system.
- `requestingComponents` – List of tools and/or system components that requested the reboot.
- `requestTimestamp` – Date and time when the reboot was requested.

Upgrading a Virtual Machine

You can upgrade virtual machine hardware by running the `VirtualMachine.UpgradeVM_Task` method. The method upgrades this virtual machine's virtual hardware to the latest revision that is supported by the virtual machine's current host. You can specify the version number as an argument. This method is useful if you want to run your virtual machine on a newer hypervisor that supports newer versions of the hardware.

Virtual Machine Management

Virtual machines can perform like physical computers and can be configured like physical computers. Virtual machines also support special features that physical computers do not support. This chapter discusses some of these features: migrating virtual machines, using snapshots, and using linked virtual machines.

For more information about configuring virtual machines, see the chapter Configuring a Virtual Machine.

Virtual Machine Migration

Migration is the process of moving a virtual machine from one host or storage location to another. Copying a virtual machine creates a new virtual machine. It is not a form of migration. vSphere supports the following migration types:

Migration Type	Description
Cold migration	Moves a powered-off virtual machine to a new host. Optionally, you can relocate configuration and disk files to new storage locations.
Migration of a suspended virtual machine	Moves a suspended virtual machine to a new host. Optionally, you can relocate configuration and disk files to new storage location.
Migration with vMotion	Moves a powered-on virtual machine to a new host. Migration with vMotion allows you to move a virtual machine to a new host without interruption in the availability of the virtual machine.
Migration with Storage vMotion	Moves the virtual disks or configuration file of a powered-on virtual machine to a new datastore. Migration with Storage vMotion allows you to move a virtual machine’s storage without interruption in the availability of the virtual machine.

Migration of a suspended virtual machine and migration with vMotion are both sometimes called hot migration, because they allow migration of a virtual machine without powering it off.

You can move virtual machines manually or set up a scheduled task to perform the cold migration.

Cold Migration
If a virtual machine is shut down, you can move it to a different cluster, resource pool, or host by copying all virtual machine files to a different directory. The `ColdMigration` example illustrates this.

Migration with vMotion
VMware vMotion supports the live migration of running virtual machines from one physical server to another with no downtime. The source and destination physical servers may be in the same datacenter or in different datacenters.

When calling the `VirtualMachine` object's `MigrateVM_Task` method, you can specify a host or resource pool to migrate to, and optionally the task priority and power state of the virtual machine. The `VMotion` example makes the following queries, and performs the migration if possible:

- Uses `QueryVMotionCompatibility_Task` to check that two hosts are compatible.
- Uses `CheckMigrate_Task` to check whether migration is feasible. If two hosts are not compatible, virtual machines cannot be migrated from one to the other.
- Uses `CheckRelocation_Task` to check whether relocation is possible.

`MigrateVM_Task` was deprecated in vSphere 6.5 and can be replaced with `RelocateVM_Task`.

Using Storage vMotion
Storage vMotion allows you to move a running virtual machine from one storage cluster to another. Taking the virtual machine or its associated storage offline is not required. All datastore types are supported, including local storage, VMFS, NAS (network attached storage), and VVols (virtual volumes).

You can place the virtual machine and all its disks in a single location, or select separate locations for the virtual machine configuration file and each virtual disk. The virtual machine remains on the same host during Storage vMotion.

To perform Storage vMotion, call the `VirtualMachine` object's `RelocateVM_Task` method. The `RelocateVMSpec` passed in to the method allows you to specify the target datastore and target host or resource pool.

As of vSphere 6.5, you should call `RelocateVM_Task` for all types of vMotion:

- cold relocate, whether VM storage moves or not
- vMotion within a cluster
- Storage vMotion
- cross-datacenter vMotion (XvMotion)
- Folder moves not involving VM moves

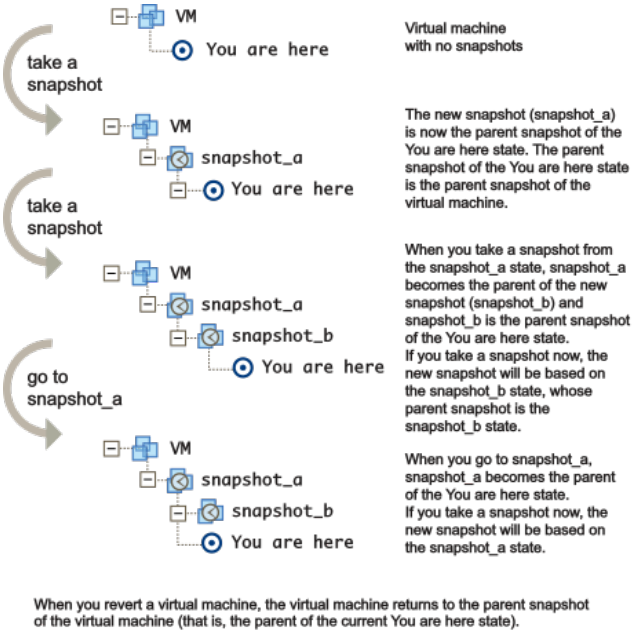
Snapshots

A snapshot is reproduction of the virtual machine just as it was when you took the snapshot. The snapshot includes the state of the data on all virtual machine disks and the virtual machine power state (on, off, or suspended). You can take a snapshot when a virtual machine is powered on, powered off, or suspended.

When you create a snapshot, the system creates a delta disk file for that snapshot in the datastore and writes any changes to that delta disk. You can later revert to the previous state of the virtual machine.

The `VirtualMachine` object has methods for creating snapshots, reverting to any snapshot in the tree, and removing snapshots.

Figure 602: Virtual Machine Snapshots



Snapshot hierarchies can become fairly complex. For example, assume that, in the example in Figure 602: Virtual Machine Snapshots, you revert to `snapshot_a`. You might then work with and make changes to the `snapshot_a` virtual machine, and create a new snapshot, creating, in effect, a branching tree.

Creating a Snapshot

The `VirtualMachine.CreateSnapshot_Task` method creates a new snapshot of a virtual machine. As a side effect, the current snapshot becomes the parent of the new snapshot.

The method allows you to specify a name for the snapshot and also requires you set the `memory` and `quiesce` properties:

- `memory` – If `true`, a dump of the internal state of the virtual machine (basically a memory dump) is included in the snapshot. Memory snapshots consume time and resources, and take a while to create. When set to `false`, the power state of the snapshot is set to powered off.
- `quiesce` – If `true` and the virtual machine is powered on when the snapshot is taken, VMware Tools is used to quiesce the file system in the virtual machine. This ensures that a disk snapshot represents a consistent state of the guest file systems. If the virtual machine is powered off or VMware Tools is not available, the quiesce flag is ignored.

The `VMSnapshot.java` example calls this method as follows:

```
ManagedObjectReference taskMor = service.createSnapshot_Task(
                                        vmMor, snapshotName, desc, false, false);
```

The method returns MOR to a `Task` object with which to monitor the operation. The `info.result` property in the `Task` contains the newly created `VirtualMachineSnapshot` upon success.

Reverting to a Snapshot

When you revert to a snapshot, you restore a virtual machine to the state it was in when the snapshot was taken. The `VirtualMachine.RevertToSnapshot_Task` allows you to specify a target host and whether the virtual machine should be powered on.

If the virtual machine was running when the snapshot was taken, and you restore it, you must either specify the host to restore the snapshot to, or set the `SupressPowerOn` flag to `true`.

Deleting a Snapshot

You can delete all snapshots by calling `VirtualMachine.RemoveAllSnapshots` or by calling the `VirtualMachineSnapshot.RemoveSnapshot_Task` method. The `VirtualMachineSnapshot` object was previously returned in the task returned by the `CreateSnapshot_Task` method.

Linked Virtual Machines

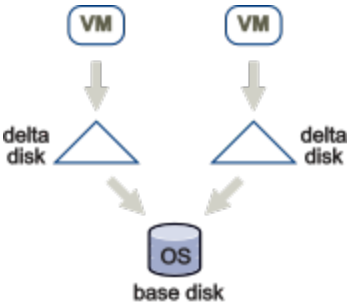
Linked virtual machines are two or more virtual machines that share storage and support efficient sharing of duplicated data.

Linked Virtual Machines and Disk Backings

In its simplest form, shared storage is achieved through the use of delta disk backings. A delta disk backing is a virtual disk file that sits on top of a standard virtual disk backing file. Each time the guest operating system on a virtual machine writes to disk, the data is written to the delta disk. Each time the guest operating system on a virtual machine reads from disk, the virtual machine first targets the disk block in the delta disk. If the data is not on the delta disk, the virtual machine looks for it on the base disk.

Linked virtual machines can be created from a snapshot or from the current running point. After you create a set of linked virtual machines, they share the base disk backing and each virtual machine has its own delta disk backing, as shown in Figure 603: Linked Virtual Machines with Shared Base Disk Backing and Separate Delta Disk Backing.

Figure 603: Linked Virtual Machines with Shared Base Disk Backing and Separate Delta Disk Backing



CAUTION:
We recommend a limit of up to eight host virtual machines accessing the same base disk in a linked virtual machine group. However, you can have an unlimited number of linked virtual machines within each host virtual machine in the group.

Limitation for HA Clusters

Virtual machines in a linked clone group can be part of a VMware HA (high availability) cluster. The number of hosts in a cluster might affect HA's ability to restart a failed virtual machine.

- Clusters that contain ESX 5.0 or earlier hosts – If a cluster has eight or fewer hosts, then linked virtual machines restart properly. However, if the cluster has more than eight hosts and any of the hosts are ESX 5.0 or earlier, HA might not be able to restart a virtual machine after it fails. HA is not aware that virtual machines in the linked clone group are subject to the eight host limit. In this case, when HA responds to a failure, it might try to restart the virtual machine on a host that cannot participate in the group due to the maximum host limit. HA will attempt failover five times to different hosts. Thus, in clusters with 13 or more hosts, it is possible that HA will never try a host that is associated with the linked clone group.
- Clusters that contain only ESXi 5.1 or later hosts – The maximum host limit for a linked clone group is the maximum number of hosts allowed in a cluster. In this case, the number of hosts in the cluster does not affect the ability to restart failed virtual machines.

Creating a Linked Virtual Machine

You can create linked virtual machines in one of two ways:

- Clone the virtual machine from a snapshot.
- Clone the virtual machine from the current virtual machine state. This state might differ from the snapshot point.

Creating a Linked Virtual Machine From a Snapshot

You first create a snapshot, and then create the linked virtual machine from the snapshot.

1. To create the snapshot, call the `CreateSnapshot_Task` method for the virtual machine. The virtual machine can be in any power state. The following pseudo code creates a snapshot named `snap1`. The code does not include a memory dump. VMware Tools is used to quiesce the file system in the virtual machine if the virtual machine is powered on.

```
myVm.CreateSnapshot("snap1", "snapshot for creating linked virtual machines", False, True)
```

2. To create the linked virtual machine, specify the snapshot you created and use a `VirtualMachineRelocateDiskMoveOptions.diskMoveType` of `createNewDeltaDiskBacking`, as illustrated in Creating a Linked Virtual Machine from a Snapshot. Creating linked virtual machines from a snapshot works with virtual machines in any power state.

Creating a Linked Virtual Machine from a Snapshot

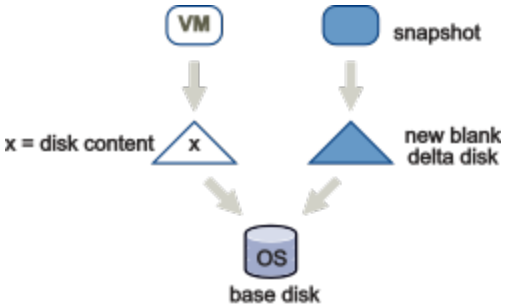
```
relSpec = new VirtualMachineRelocateSpec()
relSpec.diskMoveType = VirtualMachineRelocateDiskMoveOptions.createNewChildDiskBacking

cloneSpec = new VirtualMachineCloneSpec()
cloneSpec.powerOn = False
cloneSpec.template = False
cloneSpec.location = relSpec
cloneSpec.snapshot = myVm.snapshot.currentSnapshot

myVm.Clone(myVm.parent, myVm.name + "-clone", cloneSpec)
```

The result is a virtual machine with the same base disk as the original, but a new delta disk backing.

Figure 604: Creating a Linked Virtual Machine from a Snapshot



Creating a Linked Virtual Machine From the Current Running Point

To create a virtual machine from the current running point, clone the virtual machine, as in , but use a `diskMoveType` of `moveChildMostDiskBacking`. The virtual machine can be in any power state.

For more information about cloning a virtual machine, see Creating a Linked Virtual Machine from a Snapshot.

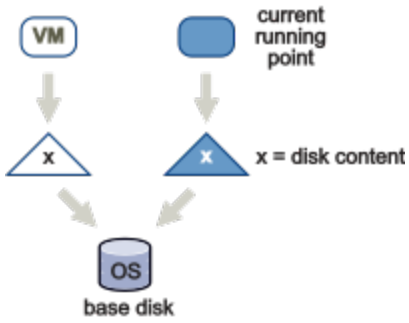
Creating a Linked Virtual Machine from the Current Running Point

```
relSpec = new VirtualMachineRelocateSpec()
relSpec.diskMoveType = VirtualMachineRelocateDiskMoveOptions.moveChildMostDiskBacking

cloneSpec = new VirtualMachineCloneSpec()
cloneSpec.powerOn = False
cloneSpec.template = False
cloneSpec.location = relSpec

myVm.Clone(myVm.parent, myVm.name + "-clone", cloneSpec)
```

Figure 605: Creating a Linked Virtual Machine from the Current Running Point



Linked Clone First Class Disk

As of VCF 9.1, you can make linked clones of first class disk (FCD). These can be used for fast provisioning of container storage volumes in VMware Kubernetes Service (VKS).

First class disk (FCD) is virtual disk that is not associated with a virtual machine. A common use case for FCD is to provide backing for container storage, such as Kubernetes persistent volumes. FCD linked clones are supported only when the VMFS volume allocation policy is "Conserve space when possible" to request thin provisioning.

After relocation of a linked clone FCD, the user or application must call `ConsolidateVMDisks_Task` to recalculate quotas and consolidate snapshots.

Instant Clone Virtual Machines

The instant clone operation creates a running virtual machine that continues running from the current state of the source virtual machine. There is no delay to power on the cloned virtual machine.

An instant clone virtual machine is a kind of linked clone with these characteristics:

- The clone starts out powered on and running.
- The clone replicates the running state of the source virtual machine.
- The clone shares memory pages with the source virtual machine as long as they remain unmodified.

The instant clone operation facilitates rapid deployment of virtual machines for low-latency response to peak computing demand. It also uses memory and disk space as efficiently as possible.

Because the clone replicates the running state of the source virtual machine, it usually requires customization of some network properties in the guest operating system to eliminate conflicts with the source. An instant clone virtual machine may also require new backings for some dedicated virtual devices, such as ISO files that back virtual optical drives.

Instant Clone Terminology

The instant clone feature has its own set of terminology, which uses some terms in ways specific to the feature.

stun	To stop the clock of a virtual machine temporarily, preventing guest operations during an instant clone operation.
freeze	To stop the clock of a virtual machine indefinitely, allowing a series of instant clone operations that reproduce the exact process state of the source virtual machine.
suspend	Save the state of a virtual machine so applications can resume, and put it to sleep.
resume	Continue running the virtual machine that was suspended, including its applications.
unstun	To release a stunned source virtual machine, allowing I/O operations to proceed.
source	A virtual machine that is cloned during an instant clone operation.
instant clone	1. An operation that quickly creates a new virtual machine that is an exact copy of a source virtual machine. 2. A new virtual machine created by an instant clone operation.

Run State of the Instant Clone Source

An instant clone can be created either from a source virtual machine in a frozen state, or from the current running point of a source virtual machine. Certain characteristics of the process and of the result can affect your choice of frozen virtual machine or current running point for the source virtual machine.

Advantages of Cloning from the Current Running Point

An instant Clone created from the current running point of a source virtual machine does not require an agent running in the guest operating system to invoke the frozen state.

If you create an instant clone from the current running point, the source virtual machine continues running from that point after a very brief delay, whereas a frozen source virtual machine requires intervention by the host to thaw it.

Advantages of Cloning from a Frozen Virtual Machine

A source virtual machine in the frozen state allows you to deploy any number of instant clones in exactly the same state, because the guest operating system retains its state after the clone operation.

When you create instant clones from the current running point of a source virtual machine, each cloning operation adds another delta disk to the source virtual machine. Each delta disk adds a small performance penalty, but a long chain of delta disks causes a noticeable performance penalty for virtual disk accesses.

The Instant Clone Process from a Frozen Source

The process to create an instant clone from a frozen source virtual machine requires one-time initial preparation, but after that it proceeds faster than creating an instant clone from the current running point. The source virtual machine must be booted and running, in a state that you want to replicate to other virtual machines. VMware Tools must be installed and running in the guest operating system so that you can run the command to freeze the source virtual machine .

Freezing the Source Virtual Machine for an Instant Clone Operation

Before you start the instant clone operation, you issue a VMware Tools command in the guest operating system, which freezes the source virtual machine. This is only needed one time, prior to the first instant clone operation. After that, the source virtual machine remains frozen. When VMware Tools receives the command to freeze the virtual machine, it passes the command through a back door to the host hypervisor.

The command to freeze the virtual machine depends on the guest operating system.

- For a Linux guest operating system, either of the following commands will freeze the virtual machine:
  - `vmware-rpctool "instantclone.freeze"`
  - `vmtoolsd -cmd "instantclone.freeze"`
- For a Windows guest operating system, the following command will freeze the virtual machine:
  - `C:\Program Files\VMware\VMware Tools\vmtoolsd.exe --cmd "instantclone.freeze"`

When a virtual machine is frozen, the hypervisor prevents future context switches to the guest operating system, which in effect stops the virtual machine's clock, although the virtual machine status continues to indicate that it is running. You can determine whether the virtual machine is frozen by examining the `runtimeinfo.InstantCloneFrozen` flag.

Instant Clone API Methods

The vSphere Web Services API offers several methods that implement different parts of the instant clone operation. By dividing the functionality into separate methods, the API provides flexibility in the timing and customization of the operation.

You can think of the instant clone operation as having three divisions: preparation, cloning, and customization.

Preparation includes these options:

- Installing the guest customization engine. The engine assists with customization of the guest network from the Web Services API. For more information, see Installing the Guest Customization Engine.
- Disconnecting virtual NICs. You have the option to disconnect the NICs in the source virtual machine before you start the clone operation, to avoid network collisions between the source and its clones. For more information, see Avoiding Network Collisions after Instant Clone Operations.
- Freezing the source virtual machine. If you need to create a large number of clones, consider cloning from a frozen source. For more information, see Run State of the Instant Clone Source.

Creating a clone of the source virtual machine requires a single method call. The `VirtualMachine.InstantClone_Task` method takes a parameter of type `InstantCloneSpec`, which enables you to give the clone a new name and configuration values. By using the key-value pairs in the `config` array, you can specify that the virtual NICs of the clone will start in a disconnected state, which avoids network conflicts with the source virtual machine. The `location` property of `InstantCloneSpec` is ignored.

Customization of a clone virtual machine takes place after the `InstantClone_Task` method completes. To customize, you invoke methods of the `VirtualMachineGuestCustomizationManager` managed object, as well as optional steps for application-dependent customization. For more information, see Guest Customization for Instant Clone Virtual Machines.

How the Instant Clone Operation Works

The instant clone operation involves the following sequence of steps.

Stunning the Instant Clone Source Virtual Machine

When you issue the API command to initiate an instant clone operation, the host begins by doing a stun operation on the source virtual machine. The stunned condition lasts a very short time, and is not directly visible to the API user.

The stunned condition blocks all I/O operations of the source virtual machine while the instant clone is created. The stun is redundant in the case of a frozen virtual machine. During the time the source virtual machine is stunned, the host creates a delta disk for the source virtual machine, which remains empty as long as the source virtual machine is stunned or frozen.

Copying Virtual Disks During the Instant Clone Operation

Then the host creates an empty delta disk for the instant clone virtual machine. This delta disk links to the base disk of the source virtual machine. More precisely, if the source virtual machine already had one or more delta disks, the instant clone delta disk links to the second delta disk in the disk chain of the source. The second delta disk is the delta disk that was the topmost delta disk when the stun operation was invoked.

Copying Memory During the Instant Clone Operation

The host creates a new virtual memory paging file for the instant clone virtual machine, whose page tables link to the paging file of the source virtual machine. When the instant clone virtual machine is complete and is allowed to run instructions, all its memory pages will be shared with the source virtual machine until either virtual machine does a write to memory.

Configuring the Instant Clone Virtual Machine

Then the host creates a virtual machine configuration file for the instant clone, which links to its delta disk and memory paging file. At this point, the instant clone virtual machine is capable of running instructions, and the host unstuns it. Both the memory and the virtual disks are shared with the source virtual machine, so the instant clone is identical to the source virtual machine, from the guest operating system point of view.

When you create virtual machines with identical guest operating systems, the network settings conflict. Usually you will want to customize these settings and reset the network stack for each instance of the guest operating system.

Limitations of Instant Clones

The instant clone feature provides good performance and scalability, but it has a few limitations:

- The instant clone feature is not available when you connect directly to ESX. Instant clone depends on functionality available only in vCenter.
- Delta disks created during the instant clone operation are not visible in the snapshot trees of the virtual machines. They cannot be managed with the VirtualMachineSnapshot managed object. Instead, you must manage the delta disks using the `VirtualMachine.Reconfigure()` method.
- The instant clone operation is partly parallel and partly serialized. You can start several operations concurrently, but part of the process is exclusive and requires serialization of requests.

- The instant clone operation responds to errors by backing out changes, such as delta disks, to restore the previous state of the file system and the source virtual machine. However, there is a small window of failure at the end of the operation where the cloned virtual machine is independent and cannot be backed out to recover from an error. This is a low risk situation.
- Instant clones create a delta disk for both the source and generated virtual machine, so resources can run out after vSphere raises a warning. The source virtual machine may be frozen to avoid excess growth. For current limits and ways to deal with failures, see the KB article *Instant Clone fails due to exceeding maximum number of delta disks*.
- The instant clone operation does not allow you to specify a different host for the clone. The clone must be on the same host as the source virtual machine because they share memory and virtual disk files.

Guest Customization of Instant Clone Virtual Machines

The instant clone operation produces a virtual machine whose memory is exactly the same as the source virtual machine. This can cause conflicts in network addressing, if two different virtual machines respond concurrently with the same network identity. The conflicting network attributes are usually the host name, IP address, and MAC address.

In the case of an instant clone derived from a frozen source virtual machine, a conflict might not immediately arise, because the source virtual machine is not active on the network. The instant clone simply assumes the identity of the source virtual machine when the host unstuns it. However, a second clone derived from the same frozen virtual machine could create a network conflict by duplicating the same identity as the first instant clone.

For an instant clone derived from the current running point of the source virtual machine, the problem is more acute because the source virtual machine resumes operation simultaneously with the instant clone.

To avoid network conflicts, you customize the instant clone to adopt a new identity, before allowing both the source virtual machine and the instant clone to run concurrently. In a situation where temporary network disruption is tolerable, you can ignore the conflict while you customize the network settings in the guest operating system of the instant clone.

Otherwise, you have two options to avoid the race condition that results when both virtual machines resume running from the same point. You can either disable the virtual NIC before the instant clone operation, or else you can specify that the virtual NIC in the instant clone will be disabled at the time when it resumes running after the operation. You specify the latter by setting the `config` property of the `spec` parameter when you invoke the `InstantClone_Task` method.

Avoiding Network Identity Collisions after Instant Clone Operations

To avoid a network conflict between a source virtual machine and its instant clone, you must disable at least one of the two conflicting virtual NICs until you complete network customization in the guest operating system of the instant clone. Here are some common approaches:

If the source virtual machine is frozen, its virtual NIC is effectively disabled, and you can reconfigure the guest network settings of the instant clone at any time before the source virtual machine is thawed.

If you derive the instant clone from the current running point of the source virtual machine, you can set the `connected` property of the virtual NIC backing to `false`, which disables it in the source before you start the instant clone operation. Then you re-enable the virtual NIC after the instant clone operation completes. After you customize the network settings in the guest operating system of the instant clone, you must re-enable the virtual NIC in the instant clone as well. This approach requires that you tolerate a network interruption until the instant clone operation is complete.

To avoid a network interruption, you can issue the instant clone operation with an `InstantCloneSpec` that specifies that the virtual NIC in the instant clone will be disconnected. The virtual NIC in the source virtual machine is not affected. After you complete guest operating system customization in the instant clone, you set the `connected` property of its virtual NIC back to `true`, and the instant clone operates under its new network identity.

In any case where you change the network identity settings in the guest operating system of the instant clone, you must reset the network stack for the virtual NIC to cause it to adopt the new settings. You can do this by invoking the `VirtalMachineGuestCustomizationManager.StartGuestNetwork_Task` method after you customize the clone's network settings, or you do the reset from the guest operating system, as with a physical machine. For more information, see Restarting the Guest Network After Customization and Resetting the Network Stack in a Virtual Machine.

Removing Snapshots and Deleting Linked Virtual Machines

After you have created a group of linked virtual machines, you can remove a snapshot that was the basis for a linked virtual machine, or delete a virtual machine. Those actions affect disks in the linked virtual machine group. Perform the actions when connected to a vCenter system for disk consolidation or deletion.

- Snapshot removal – During snapshot removal, the snapshot metadata is also removed, and the virtual machine from which the snapshot was taken is no longer shown as having snapshots. If you remove a snapshot while connected to the ESX host directly, shared disks are not consolidated and unnecessary levels of delta disks might result. If you remove a snapshot while connected to a vCenter system, shared disks are not consolidated, but unshared disks are consolidated.
- Virtual machine deletion – When you delete a virtual machine by directly connecting to the ESX host, shared disks are not deleted. When you delete a virtual machine by connecting to a vCenter system, shared disks are not deleted, but unshared disks are deleted.

  CAUTION:
  Delete all linked virtual machines before deleting the source virtual machine from which they were created, so that you don't have orphaned or corrupt disk files on your file system.

Relocating a Virtual Machine in a Linked Virtual Machine Group

You can move the virtual machines in a linked virtual machine group between datastores and save storage. The contents of the delta disk might not be as important as the contents of the base, and you can save storage by removing the delta disk.

For more information, see Figure 604: Creating a Linked Virtual Machine from a Snapshot.

Relocating a Linked Virtual Machine

```
relSpec = new VirtualMachineRelocateSpec()
relSpec.diskMoveType = VirtualMachineRelocateDiskMoveOptions.moveChildMostDiskBacking
relSpec.datastore = localDatastore
myVm.Relocate(relSpec)
```

You can relocate multiple linked virtual machines to a new datastore, but keep all shared storage during the relocation. To achieve the relocation, relocate the desired virtual machines one by one, giving the option to allow reattaching to an existing disk, as shown in Relocating Multiple Linked Virtual Machines.

Relocating Multiple Linked Virtual Machines

```
relSpec = new VirtualMachineRelocateSpec()
relSpec.diskMoveType = VirtualMachineRelocateDiskMoveOptions.moveAllDiskBackingsAndAllowSharing
relSpec.datastore = targetDatastore
myVm.Relocate(relSpec)
```

Promoting a Virtual Machine's Disk

Promoting a virtual machine's disk improves performance.

Important:

You can use the `PromoteDisks` API only when connected to a vCenter system.

You can use `PromoteDisks` to copy disk backings or to consolidate disk backings.

- Copy – If the `unlink` parameter is `true`, any disk backing that is shared by multiple virtual machines is copied so that this virtual machine has its own unshared version. Files are copied into the home directory of the virtual machine. This setting results in improved read performance, but higher space requirements. The following call copies and shares disks, and then collapses all unnecessary disks.
  ```
  myVm.PromoteDisks(True, [])
  ```
- Consolidate – If the `unlink` parameter is `false`, any disk backing that is not shared between multiple virtual machines and not associated with a snapshot is consolidated with its child backing. The net effect is improved read performance at the cost of inhibiting future sharing. The following call eliminates any unnecessary disks:
  ```
  myVm.PromoteDisks(False, [])
  ```

Promoting a virtual machine's disk might also be useful if you end up with disk backings that are not needed for snapshots or for sharing with other virtual machines.

Both uses of `PromoteDisks` take an optional second argument, which allows you to apply the method to only a subset of disks. For example, you can unshare and consolidate only the virtual disk with key 2001 as follows:

```
for any of my VMs in dev
   if (dev.key == 2001)
      disk2001 = dev

myVm.PromoteDisks(True, [disk2001])
```

Performing Advanced Manipulation of Delta Disks

For advanced manipulation of delta disks, you can use `VirtualDeviceConfigSpec` methods such as `VirtualDeviceConfigSpec.create` and `VirtualDeviceConfigSpec.add`.

Together `add` and `create` allow you to create a blank delta disk on top of an existing disk. For the `VirtualDeviceConfigSpec` you specify `add` to prepare the VM for a new delta disk, then `create` to initiate the file operation for the delta disk whose `parent` property is an existing disk. These methods create a new delta disk whose parent is the pre-existing disk.

One use case is adding a delta disk on top of an existing virtual disk in a virtual machine without creating a snapshot. Adding a Delta Disk Backing illustrates how to add the delta disk for the first virtual disk in the virtual machine.

Adding a Delta Disk Backing

```
disk = None
for any of my VMs in dev
   if (VirtualDisk.isinstance == dev):
      disk = dev

# Remove the disk
removeDev = new VirtualDeviceConfigSpec()
removeDev.operation = "remove"
removeDev.device = disk

# Create a new delta disk which has the
# original disk as its parent disk
addDev = new VirtualDeviceConfigSpec()
addDev.operation = "add"
addDev.fileOperation = "create"
addDev.device = copy.copy(disk)
addDev.device.backing = copy.copy(disk.backing)
addDev.device.backing.fileName = "[" + disk.backing.datastore.name + "]"
addDev.device.backing.parent = disk.backing

spec = new VirtualMachineConfigSpec()
spec.deviceChange = [removeDev, addDev]
```

For working code with a similar use case, see the JAXWS sample program `VMDeltaDisk.java` in the SDK under `vsphere-ws/java`.

Virtual Machine Guest Operations

The vSphere Web Services SDK enables you to do operations inside a virtual machine, by communicating with the guest operating system.

Authenticating with the Guest Operating System

Authentication is required for most guest operations. Before you invoke a method that affects the guest, you must establish your credentials with the guest operating system and acquire a `GuestAuthentication` object that you pass to the method.

The `GuestAuthentication` type has several subclasses that hold guest credentials acquired by different methods. You can choose from several ways to authenticate:

- NamePasswordAuthentication
- SAMLTokenAuthentication
- SSPIAuthentication
- TicketedSessionAuthentication

Running Guest OS Operations

Guest OS operations manipulate processes, files, folders, and environment variables in a virtual machine's guest operating system.

The vSphere API offers the following managed object types for guest operations:

- GuestAuthManager – authenticate to acquire credentials in the guest OS.
- GuestFileManager – manipulate files, directories, and remote copying in the guest OS.
- GuestProcessManager – manipulate processes in the guest OS.
- GuestAliasManager – support single sign-on for guest operations; create and delete user aliases.
- GuestWindowsRegistryManager – manipulate keys and values in the Windows registry.
- VirtualMachineGuestCustomizationManager – customize guest settings, especially for instant clone virtual machines.

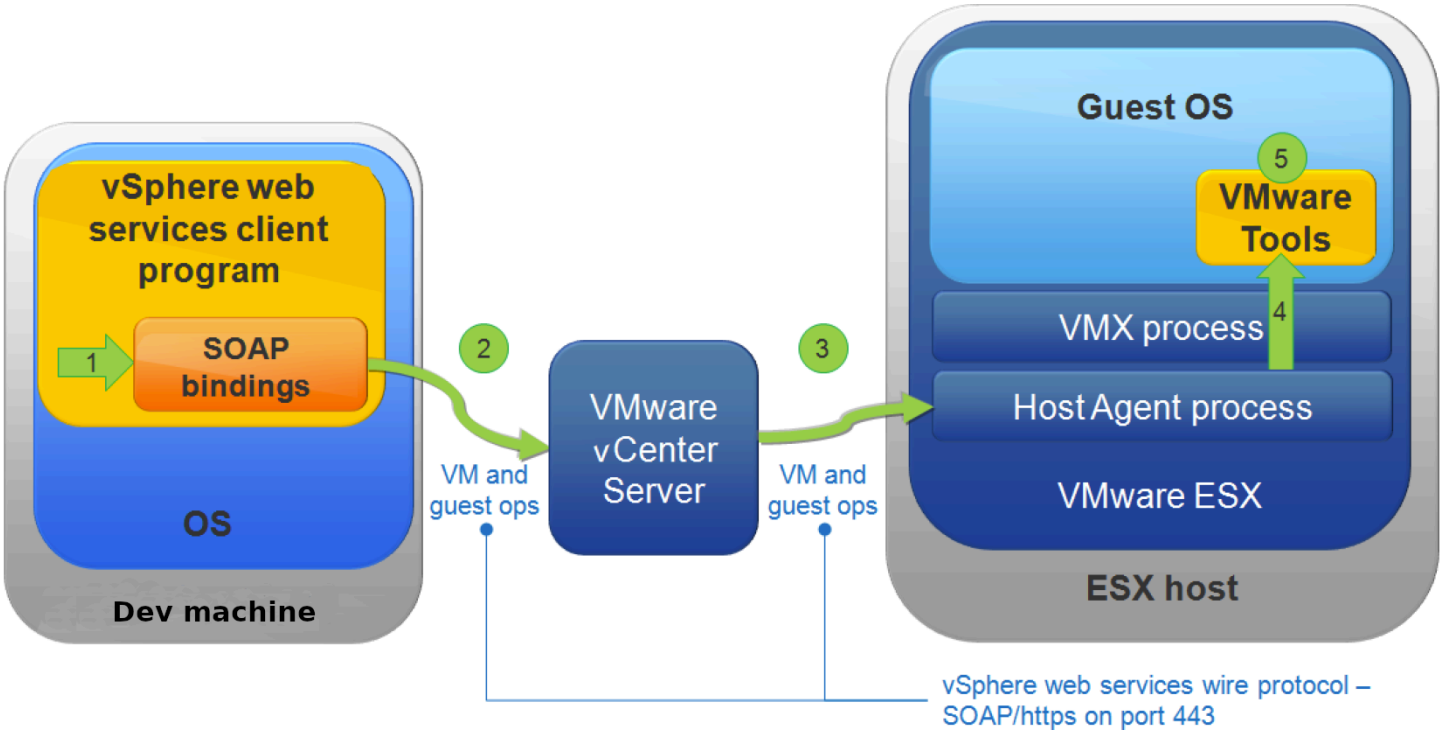
All the above managed objects are subclasses that inherit properties from GuestOperationsManager. In the vSphere API, the VirtualMachine and GuestInfo managed objects contain information about what guest operations might be running and relevant virtual machine state:

```
vim.VirtualMachine.guest()
vim.vm.GuestInfo.guestOperationsReady
vim.vm.GuestInfo.interactiveGuestOperationsReady
```

VMware Tools must be present to run guest operations. To perform interactive guest operations, the user must be logged into the console, for example through the vSphere Client. Steps involved are:

1. Java program translates to SOAP bindings.
2. Directives of vSphere API pass through vCenter.
3. Directives are relayed to ESX host agent process.
4. Virtual machine executable passes guest operations to VMware Tools.
5. Guest OS performs guest operations.

Figure 606: Guest operations control flow



The following table summarizes the methods available for guest operations.

Table 1889: GuestOperationsManager methods

Managed Object	Methods	Description
GuestAliasManager	AddGuestAlias	define alias for guest account
	ListGuestAliases	list guest aliases for specified user
	ListGuestMappedAliases	list alias map for in-guest user
	RemoveGuestAliasByCert	remove certificate associated aliases
GuestAuthManager	AcquireCredentialsInGuest	authenticate, return session object
	ReleaseCredentialsInGuest	release session object
	ValidateCredentialsInGuest	check authentication data or timeout
GuestFileManager	ChangeFileAttributesInGuest	change attributes of file in guest
	CreateTemporaryDirectoryInGuest	make a temporary directory
	CreateTemporaryFileInGuest	create a temporary file
	DeleteDirectoryInGuest	remote directory in guest OS
	DeleteFileInGuest	remove file in guest OS
	InitiateFileTransferFromGuest	start file transfer from guest OS
	InitiateFileTransferToGuest	start file transfer to guest OS
	ListFilesInGuest	list files or directories in guest
	MakeDirectoryInGuest	make a directory in guest
	MoveDirectoryInGuest	move or rename a directory in guest
	MoveFileInGuest	rename a file in guest
GuestWindowsRegistryManager	CreateRegistryKeyInGuest	create a registry key
	DeleteRegistryKeyInGuest	delete a registry key
	DeleteRegistryValueInGuest	delete a registry value

Managed Object	Methods	Description
	ListRegistryKeysInGuest	list registry subkeys for a given key
	ListRegistryValuesInGuest	list registry values for a given key
	SetRegistryValueInGuest	set or create a registry value
GuestProcessManager	ListProcessesInGuest	list processes running in guest OS
	ReadEnvironmentVariableInGuest	read environment variable in guest
	StartProgramInGuest	start running program in guest
	TerminateProcessInGuest	stop a running process in guest

`InitiateFileTransferFromGuest` and `InitiateFileTransferToGuest` are useful for transferring small files between the host and guest. For large file transfers, virtual machines should be connected to the network because networking transfers are much faster.

Java Source Code Samples

Four Java code samples based on JAX-WS are available in the vSphere SDK for Web services, in this directory:

```
SDK/vsphere-ws/java/JAXWS/samples/com/vmware/guest
```

`CreateTemporaryFile.java` creates a temporary file inside a virtual machine, by calling the following method:

```
vimPort.createTemporaryFileInGuest(fileManagerRef, vmMOR, auth, prefix, suffix, directoryPath);
```

`DownloadGuestFile.java` downloads a file from the guest to a specified path on the host where the client is running. The destination, a local file on the client host, is specified on the command line as `--localfilepath`.

```
vimPort.initiateFileTransferFromGuest(fileManagerRef, vmMOR, auth, guestFilePath);
```

`RunProgram.java` runs a specified program inside a guest operating system, with output re-directed to a temporary file, and downloads the resulting output to a file on the local client. The program must already exist on the guest, and is specified on the command line as `--guestprogrampath`. The output file to store on the client host is specified on the command line as `--localoutputfilepath`.

```
vimPort.startProgramInGuest(processManagerRef, vmMOR, auth, spec);
```

`UploadGuestFile.java` uploads a file from the client machine to a specified location inside the guest. The source, a local file on the client host, is specified on the command line as `--localfilepath`.

```
vimPort.initiateFileTransferToGuest(fileManagerRef, vmMOR, auth, guestFilePath,
                          guestFileAttributes, fileSize, optionsmap.containsKey("overwrite"));
```

Guest Operating System Customization

You can use the vSphere Web Services SDK to customize several attributes of the guest operating system. The `CustomizationSpec` data object allows you to specify network and identity settings in the guest operating system.

Use `CustomizationSpec` to specify any of these settings that you choose to modify. You can specify the following areas:

- IP addresses and gateway addresses for virtual NIC devices
- DNS servers
- Virtual hostname
- Domain name
- Time zone
- User scripts (Windows and Linux)
- Password of admin/root user (change or expire)
- Configure IPv6-only networking and avoid IPv4 configuration
- Live network configuration for powered-on VMs

You can customize a guest operating system while the virtual machine is stopped (powered off) by calling `VirtualMachine.CustomizeVM_Task`. See Guest Customization for Stopped Virtual Machines for details. You can customize the guest operating system while the virtual machine is running by calling `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task`. See Guest Customization for Instant Clone Virtual Machines and Flexible Customization for Running Virtual Machines for details.

With VCF 9.1, flexible customization supports network-only customization while a virtual machine is stopped or running.

The two ways to customize a virtual machine are compared in the following table.

Table 1890: Comparing Guest Operating System Customization

	Customization While Stopped	Instant Clone Customization
vSphere Compatibility	• Full customization: vSphere 5 and later • Flexible customization: VCF 9.1 and later • Both vSphere and VMC	• Instant clone customization: vSphere 7 and later • Flexible customization: VCF 9.1 and later • vSphere only (not VMC)

API Method	`VirtualMachine.CustomizeVM_Task`	`VirtualMachineGuestCustomizationManager.CustomizeGuest_Task`
		`VirtualMachineGuestCustomizationManager.StartGuestNetwork_Task`
		`VirtualMachineGuestCustomizationManager.AbortCustomization_Task`

Creating Customization Spec	Create customization specifications in vSphere Client or in API.	Create customization specifications in API.
Virtual Machine State	Must be powered off.	Must be powered on.
`vmtools`	Must be installed.	Must be installed and running.
cloud-init	Must be installed if using cloud-init customization.	Not used.
Customization Engine	Not used.	Required for instant clone customization.
Supported Guests	Both Linux and Windows. For supported versions, see the Guest OS Compatibility Guide.	Instant clone customization: Linux only. Supported distributions are listed in "Guest Customization for Instant Clone Virtual Machines" Flexible customization: both Linux and Windows. Supported distributions are listed in Flexible Customization for Running Virtual Machines
Network Adapter State	For SysPrep or LinuxPrep, `CustomizeVM_Task` disconnects virtual adapters and prepares customization. On next power on, the customization takes effect and `vmtools` will reconnect virtual adapters. For CloudinitPrep customization, `CustomizeVM_Task` does not disconnect virtual adapters.	Instant clone customization requires calling `InstantClone_Task`, which disconnects virtual adapters before creating a clone, to avoid network conflicts. For a Linux VM, use `StartGuestNetwork` to restart virtual adapters after customization. For a Windows VM, it is not necessary to restart the virtual network adapters after customization. Network changes take effect immediately after execution of `CustomizeGuest_Task`.
Rebooting the Guest	`vmtools` reboots the guest operating system after reconnecting virtual network adapters. No reboot needed after Linux cloud-init customization, either LinuxPrep or CloudinitPrep.	No reboot needed. For a Linux VM, `StartGuestNetwork` restarts network services after customization.
Flexibile customization (As of VCF 9.1)	In `CustomizationSpec`, new properties `WindowsFlexPrep` or `LinuxFlexPrep` substitute for `IdentitySettings`.	In `CustomizationSpec`, new properties `WindowsFlexPrep` or `LinuxFlexPrep` substitute for `IdentitySettings`.

Guest Customization for Stopped Virtual Machines

You can customize certain settings in the guest operating system with the help of the vSphere Web Services SDK. Customization is simpler for a virtual machine that is not running.

A guest operating system can be easily customized while it is not running. If stopping the guest will cause a significant service impact, such as in an instant clone virtual machine, see Guest Customization for Instant Clone Virtual Machines instead.

Use this general procedure to customize a guest operating system that you can shut down without a significant service impact. However, to initialize a virtual machine in a cloud environment, by using the cloud-init utility, see Guest Customization Using cloud-init.

1. Shut down the guest operating system by using the `ShutdownGuest` method or the `PowerOffVM_Task` method.
2. Prepare a `CustomizationSpec` data object that specifies the desired global IP settings, virtual NIC settings, and so on.
3. Invoke the `CustomizeVM_Task` method of the `VirtualMachine` managed object.
4. Start the guest operating system by using the `PowerOnVM_Task` method of the `VirtualMachine` managed object.

For more information about customizing the guest operating system, in the context of virtual machine configuration, see Customizing the Guest Operating System.

Flexible Customization for Running Virtual Machines

You can perform flexible customization of a running Linux or Windows virtual machine using the vSphere Web Services SDK. As of VCF 9.1, guest customization supports flexible methods that bypass mandatory configuration requirements. You can selectively customize specific guest settings, such as networking, without providing a full set of customization configuration.

Flexible customization for a running virtual machine involves the following steps:

- Customizing the guest with flexible customization spec, using `WindowsFlexPrep` or `LinuxFlexPrep` instead of `IdentitySettings`.
- Restarting the guest network (mandatory for Linux virtual machines)
- Recovering from guest customization errors
- Application-dependent customization

Customing the Guest with Flexible Customization Spec

You can use a method of the `VirtualMachineGuestCustomizationManager` managed object to apply flexible customization specifications in a guest. As of VCF 9.1, flexible customization supports partial customization for running virtual machines, with initial support focused on network configuration. You can apply new settings for IP address, DNS server, and gateway router of one or more virtual NICs. In Windows, DNS settings can be specific to individual NICs. In Linux, DNS settings are global.

Customization applies the new guest settings that you specify in the spec parameter of the `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task`. The spec parameter is a data object of type `CustomizationSpec`, which contains a `nicSettingMap` property holding an array that maps MAC address to network settings for one or more virtual NICs in the guest. There are two ways to use the `nicSettingsMap` array in the spec parameter:

- Specify each virtual NIC explicitly by its MAC address, in `spec.nicSettingMap[i].macAddress`, and specify the virtual NIC's new settings in the properties of `spec.nicSettingMap[i].adapter`.
- Omit `spec.nicSettingMap[i].macAddress` for all virtual NICs and specify the new settings for each NIC in PCI bus order.

In addition to virtual NIC settings, you can customize these properties of a virtual machine:

- You can set the `CustomizationSpec.identity` property to `WindowsFlexPrep` or `LinuxFlexPrep`. In VCF 9.1, these two objects do not include any identity related settings, which means customization is limited to network configuration only when applied to a running virtual machine.

- You can set the `CustomizationSpec.globalIPSettings` property to a data object of type `CustomizationGlobalIPSettings` in which you specify DNS settings for all virtual NICs. Use this for Linux guest operating systems.

The list may change in subsequent releases, but on VCF 9.1 the `CustomizeGuest_Task` method supports the following guest operating systems: RHEL 6.8 and higher, RHEL 7.4 and higher, CentOS 7.4 and higher, SUSE 11 SP4, SUSE 12 SP3 and higher, Ubuntu LTS 16.04 and higher, Windows 7 and above, Windows Server 2008 R2 and above.

The `toolsd` service must be running and available for this customization step.

The customization data is stored in the Namespace database in a `cust.cfg` format containing name-value pairs. This makes the customization process resilient to vMotion operations. It also means that you can access the data if needed as input for user-supplied customization scripts.

Restarting the Guest Network in Linux

After applying the flexible customization spec, you can restart the guest network manually from within the guest operating system, or by using a method of the `VirtualMachineGuestCustomizationManager` managed object.

To restart the guest network using the vSphere Web Services API, call the `StartGuestNetwork_Task` method. This method should be invoked after `CustomizeGuest_Task` has run. Calling this task restarts the guest network service and ensures the updated network settings take effect within the Linux guest operating system.

The `StartGuestNetwork_Task` method is only mandatory for Linux virtual machines. For Windows virtual machines, due to the operating system's behavior, the new network configuration takes effect immediately after `CustomizeGuest_Task` completes, without requiring an explicit network restart.

Recovering from Guest Customization Errors

Flexible customization for a running virtual machine is a long-running operation with several potential failure points. If you encounter a failure, consider the following steps.

The guest network customization engine holds a lock on the `VirtualMachineGuestCustomizationManager` interface while it applies changes to the guest. A failure during customization or while restarting the network might leave a hung process or an orphaned lock. If any step of the flexible customization returns an error, use the `AbortCustomization_Task` method to stop all running instances of the customization engine and to release the lock.

This method should only be invoked after the `CustomizeGuest_Task` has run**.** To troubleshoot a failure, you can examine the log file used by the `CustomizeGuest_Task` method. The log file is on the host file system at the following locations for Linux and Windows:

```
/var/log/vmware-gosc/instant_clone_customization.log
%SystemRoot%\Temp\vmware-imc\vmware-flexible-customization.log
```

When you abort a guest customization operation, the guest network can be left in an inconsistent state. After the abort operation completes, you should check your customization settings and retry the `CustomizeGuest_Task` method in case of transient error conditions. When you use `VirtualMachineGuestCustomizationManager`, the customization procedure tolerates vMotion without failure.

Application Dependent Customization

After you customize and restart the guest network, the virtual machine might need additional customization steps, depending on the applications that run in the guest operating system.

For example, you might need to signal an application to close and re-open network sockets when you have configured new network IP addresses. To do additional customization steps in the guest, use the `GuestProcessManager` method in the `GuestOperationsManager` managed object.

Guest Customization Using cloud-init

You can customize certain settings in a Linux guest operating system with the help of the vSphere Web Services SDK. The cloud-init utility is a standardized way to initialize a virtual machine when you deploy it in a cloud environment.

Guest operating systems can be customized in one of several ways, depending on certain cloud-init configuration flags and on the subtype of the `identity` property of the `CustomizationSpec` data object. The following table shows the combinations needed to achieve raw data cloud-init customization, LinuxPrep with traditional customization, or LinuxPrep with cloud-init customization. Other combinations are not allowed.

Table 1891: Enabling Guest Customization Types

Flags	CustomizationSpec.identity	Result
`allow_raw_data: true` or absent (default=`true`)	`CloudInitPrep`	VMware Tools will work with cloud-init to customize the guest with the unprocessed `metadata` and `userdata` from `CloudinitPrep`.
`disable_vmware_customization: true` or absent (default=`true`)	`LinuxPrep`	VMware Tools will do traditional customization on the guest.
`disable_vmware_customization: false`	`LinuxPrep`	VMware Tools will work with cloud-init to customize the guest with configuration derived from `LinuxPrep`.

Use the following procedures to apply settings to a virtual machine by passing raw configuration data to the cloud-init utility. You will use the `CustomizeVM_Task` method with a `CustomizationSpec` that contains configuration data compatible with the cloud-init standard.

Prepare a Virtual Machine Template for Raw Data cloud-init Customization

The following steps prepare a virtual machine template for customization with raw cloud-init data.

1. In the guest operating system, install open-vm-tools version 11.3.0 or later. Configure the Linux guest to run open-vm-tools services when the guest operating system starts.
   Example for an Ubuntu 20.04 guest operating system:
   ```
   $ sudo apt update
   $ sudo apt install open-vm-tools
   $ sudo vmtoolsd -v
   $ sudo systemctl is-enabled open-vm-tools.service
   ```
2. In the guest operating system, install cloud-init version 21.1 or later. Configure the Linux guest to run cloud-init services when the guest operating system starts.
   Example for an Ubuntu 20.04 guest operating system:
   ```
   $ sudo apt update
   $ sudo apt install cloud-init
   $ sudo cloud-init -v
   $ sudo systemctl is-enabled cloud-init.service
   $ sudo systemctl is-enabled cloud-init-local.service
   ```
3. Configure cloud-init to accept the OVF data source for VMware raw cloud-init configuration.
   Edit the cloud-init configuration file in `/etc/cloud/cloud.cfg` or in `/etc/cloud/cloud.cfg.d/*`, depending on the Linux distribution. Add `OVF` to the list of datasources accepted by cloud-init.
   Example line for cloud-init version 21.3:
   ```
   datasource_list: [ NoCloud, ConfigDrive, Azure, OVF, OpenStack, Ec2 ]
   ```
4. Configure cloud-init to enable VMware customization with raw cloud-init data.
   Edit the cloud-init configuration file in `/etc/cloud/cloud.cfg` or in `/etc/cloud/cloud.cfg.d/*`, depending on the Linux distribution. Set the flag `disable_vmware_customization` to `true` and set the flag `allow_raw_data` to `true`. If customization time-out is an issue, you can set the value of `vmware_cust_file_max_wait` (default: 15 seconds).
   Example lines for cloud-init version 21.3:
   ```
   disable_vmware_customization: true
   datasource:
     OVF:
   ```

```
      allow_raw_data: true
  vmware_cust_file_max_wait: 25
```

5. Run the cloud-init clean step to remove any artifacts from previous configuration failures.

   ```
   $ sudo cloud-init clean
   ```

6. Power off the virtual machine template.

   ```
   $ sudo poweroff
   ```

Prepare a cloud-init Metadata File

Use the following information to prepare your cloud-init metadata file. The metadata file is required for cloud-init.

- The metadata file is a plain-text file formatted as YAML or JSON.
- The maximum file size is 512KB.
- All standard `v1` keys are accepted, such as:
  - `distro`
  - `local_hostname`
  - `platform`
  - `public_ssh_keys`
- The `instance-id` key is required. All other keys are optional.
- For network configuration, both `v1` and `v2` keys are accepted.

This example shows a metadata file in YAML format that includes a minimal network configuration:

```
instance-id: cloud-vm-example-1
local hostname: cloud-vm
network:
  version: 2
  ethernets:
    nics:
      match:
        name: eth*
      dhcp4: yes
```

This example shows a metadata file in JSON format that specifies a fixed IP network configuration:

```
{
  "instance-id": "cloud-vm-example-2",
  "local-hostname": "cloud-vm",
  "cloud-name": "vSphere",
  "distro": "ubuntu",
  "distro-version": "20.04",
  "distro-release": "focal",
  "network": {
    "version": 2,
    "ethernets": {
      "nics": {
        "match": {
          "name": "eth*"
        },
        "nameservers": {
          "addresses": ["203.0.113.1"],
          "search": ["www.example.com", "example.com"]
        },
        "gateway4": "192.0.2.253",
        "dhcp4": false,
        "addresses": [ "192.0.2.0/24" ]
      }
    }
  }
```

```
}
```

Prepare a cloud-init Userdata File

Use the following information to prepare your cloud-init userdata file. The userdata file is optional for cloud-init.

- The userdata file is a plain-text file in raw cloud-init format, without compression or base64 encoding.
- The maximum file size is 512KB.
- Userdata configuration can be used to accomplish a number of configuration tasks in the guest operating system. For instance, some of the more common tasks include:
  - Add user accounts
  - Install software
  - Run scripts
  - Import SSH keys

This example shows a userdata file that adds a new user to the guest operating system:

```
#cloud-config

users:
  - default
  - name: testuser
    primary_group: testgroup
    sudo: ALL=[ALL] NOPASSWD:ALL
    groups: sudo, wheel
    ssh_import_id: None
    lock_passwd: true
    ssh_authorized_keys:
    - ssh-rsa myRsaPubKey
```

Initiate Raw Data cloud-init Customization with the vSphere Web Services API

Use the following steps to initiate customization with raw cloud-init data by using the vSphere Web Services API.

1. Build a customization specification of type `CustomizationSpec`. Set the following properties in the spec:
   a. The `globalIPSettings` property must be present but need not contain its optional child properties. If the child properties are present, they will be ignored during the method call.
   b. The `identity` property must be present and must have type `CloudinitPrep`.
      - The child property `identity.metadata` is required. It contains the entire text of the metadata file, less any final new line character.
      - The child property `identity.userdata` is optional. If present, it contains the entire text of the userdata file, less any final new line character.

   Example:

   ```
   CloudinitPrep cloudinit = new CloudinitPrep();
   cloudinit.setMetadata(metadataText);
   cloudinit.setUserdata(userdataText);
   CustomizationGlobalIPSettings ipSettings = new CustomizationGlobalIPSettings();
   CustomizationSpec custSpec = new CustomizationSpec();
   custSpec.setIdentity(cloudinit);
   custSpec.setGlobalIPSettings(ipSettings);
   ```

   Note:
   The `nicSettingsMap` property, if present, will be ignored for raw data customization. The cloud-init tool accepts network customization only from the `identity.metadata` property.
2. Invoke the method `VirtualMachine.CustomizeVM_Task`, passing the `customizationSpec` as parameter. The method returns a Task object that you can use to monitor completion of the customization steps.

   Example:

   ```
   ManagedObjectReference custTask = service.CustomizeVM_Task(vmRef, custSpec);
   ```

3. Power on the virtual machine. As it boots, cloud-init applies the metadata configuration settings to the guest operating system. After the guest finishes booting and before it opens a shell window, cloud-init applies the userdata configuration.

Guest Customization for Instant Clone Virtual Machines

You can customize the network settings in the guest operating system with the help of the vSphere Web Services SDK. For instant clone operations, you must customize the clone without stopping the guest operating system.

Guest network customization for a running virtual machine involves several steps:

- Installation of the guest customization engine
- Disconnecting virtual NICs
- Customizing guest network settings
- Reconnecting virtual NICs
- Restarting the guest network
- Recovering from guest customization errors
- Running scripts for application-dependent customization

You can use these steps independently and you can run your own scripts in between steps as needed to customize, reset status, or restart processes in the guest.

> **Note:**
> You must authenticate using credentials accepted by the guest operating system before you can run guest operations.

Installing the Guest Customization Engine

The guest customization engine needs to be installed in a virtual machine before using the `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task` method.You can download the installer from the VMware Customer Connect web site.

Start a web browser in a running virtual machine. For instant clone operations, prepare the source virtual machine by installing the customization engine before creating clones.

The guest customization engine is available for most popular Linux distributions, using either an RPM package or a DEB package. To download and install it within the guest operating system, use the following steps.

1. Login to Broadcom support site and find VMware.
2. Navigate to the Download VMware vSphere page.
3. Click the **Drivers & Tools** tab.
4. Find the entry for the instant clone customization engine.
5. Download the installation package appropriate for your Linux distribution:
   - The `.rpm` file for RHEL or SLES
   - The `.deb` file for Ubuntu
6. Run the command to install the package in the guest:
   - For the `.rpm` package, use this command:
     ```
     sudo /bin/rpm -i --force yourdpkgFilePath >> ./rpm.log 2>&1
     ```
   - For the `.deb` package, use this command:
     ```
     sudo /usr/bin/dpkg -i --force-all yourdpkgFilePath >> ./dpkg.log 2>&1
     ```

After the guest customization engine is installed, the guest is prepared to invoke `VirtualMachine.InstantClone_Task` or `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task`. You can delete the installation package to save virtual disk space.

Disconnecting Virtual NICs

Before you customize the guest network, you should disconnect the virtual NICs to quiesce network traffic during the operation. Disconnecting virtual NICs is a key step in the instant clone process.

You can disconnect a virtual NIC in one of these ways:

- In the `spec` parameter to the `InstantClone_Task` method you can specify that the virtual NIC should be disconnected during the instant clone operation. This way leaves the NIC in the source virtual machine active immediately after the clone operation, while the NIC in the clone is idle and ready for customization.
- In the `spec` parameter to the `ReconfigVM_Task` method you can change the connection state of a virtual NIC. This way is available either with or without an instant clone operation.

Disconnecting a Virtual NIC with InstantClone_Task

To disconnect a virtual NIC during an instant clone operation, prepare a `VirtualMachineInstantCloneSpec` data object that specifies the device should be in the disconnected state after the clone operation completes. You pass the clone specification in the `spec` parameter of `InstantClone_Task`. Assign a value of `disconnect` to the property `spec.location.deviceChange[].device.connectable.migrateConnect`.

Disconnecting a Virtual NIC with ReconfigVM_Task

To disconnect a virtual NIC by reconfiguring the virtual machine, prepare a `VirtualMachineConfigSpec` data object that specifies a disconnected virtual NIC. You pass the config specification in the `spec` parameter of the `ReconfigVM_Task` method. Assign a value of `False` to the property `spec.deviceChange[].device.connectable.connected`.

Customizing Guest Network Settings for Running Virtual Machines

You can use a method of the `VirtualMachineGuestCustomizationManager` managed object to apply new settings to the virtual NICs in the guest. The `VirtualMachineGuestCustomizationManager` uses the guest customization engine designed for instant clone operations to customize the guest while it continues running.

The customization step applies new settings for IP address, DNS server, and gateway server of one or more virtual NICs.

> **Note:**
> DNS settings can be specific to individual NICs in Windows operating systems. DNS settings are global in a Linux operating system.

The guest customization engine must be installed in the virtual machine before you call this method. See Installing the Guest Customization Engine for instructions.

Customization applies the new guest network settings that you specify in the `spec` parameter to the `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task` method. The `spec` parameter is a data object of type `CustomizationSpec`, which contains a `nicSettingMap` property that holds an array that maps MAC address to network settings for one or more virtual NICs in the guest.

There are two ways to use the `nicSettingsMap` array in the `spec` parameter. You can do one of the following:

- Specify each virtual NIC explicitly by its MAC address, in `spec.nicSettingMap[i].macAddress`, and specify the virtual NIC's new settings in the properties of `spec.nicSettingMap[i].adapter`.
- Omit `spec.nicSettingMap[i].macAddress` for all virtual NICs and specify the new settings for each NIC in PCI bus order.

In addition to virtual NIC settings, you can customize these properties of a virtual machine:

- You can set the `CustomizationSpec.identity` property to a data object derived from the type `CustomizationIdentitySettings`. You must choose a subclass of the type that corresponds to the guest operating system installed in the virtual machine. In the `identity` object you can specify a new host name, domain name, and system time settings.
- You can set the `CustomizationSpec.globalIPSettings` property to a data object of type `CustomizationGlobalIPSettings` in which you specify DNS settings for all virtual NICs. Use this for Linux guest operating systems.

The `CustomizeGuest_Task` method supports the following guest operating systems:

- Red Hat Enterprise Linux 6.8 and higher
- Red Hat Enterprise Linux 7.4 and higher
- CentOS 7.4 and higher
- SUSE 11SP4
- SUSE 12SP3 and higher
- Ubuntu 16.04 and higher (LTS distributions)

The toolsd service must be running and available for this customization step.

> **Note:**
> The `CustomizeGuest_Task` method is asynchronous, but a critical part of the operation, which uses the customization engine, is synchronous.

You can pipeline the instant clone and customization operations to minimize the time needed to clone a virtual machine.

> **Note:**
> During an instant clone operation, the toolsd service is briefly offline while it resets connections in the clone. This results in no noticeable delay, but you might need to allow for a retry if you run pre-customization code.

The customization data is stored in the Namespace database in a cust.cfg format containing name-value pairs. This makes the customization process resilient to vMotion operations. It also means that you can access the data if needed as input for user-supplied customization scripts.

Reconnecting Virtual NICs in a Running Virtual Machine

After you customize a guest network in a running virtual machine, you must reconnect the virtual NICs before the guest operating system can use them.

To reconnect a virtual NIC by reconfiguring the virtual machine, prepare a `VirtualMachineConfigSpec` data object that specifies a connected virtual NIC. You pass the config specification in the `spec` parameter of the `ReconfigVM_Task` method. Assign a value of `True` to the property `spec.deviceChange[].device.connectable.connected`.

Restarting the Guest Network After Customization

After customizing guest network settings, you can restart the guest network manually from within the guest operating system, or by using a method of the `VirtualMachineGuestCustomizationManager` managed object. This method is important for instant clone operations, but it can be used for other purposes as well.

To restart the guest network manually, see Resetting the Network Stack in a Running Virtual Machine.

To restart the guest network using the vSphere Web Services API, use the `VirtualMachineGuestCustomizationManager.StartGuestNetwork_Task` method. This method both reconnects the virtual NICs to the network and restarts the guest network service.

The `StartGuestNetwork_Task` method is supported on most popular Linux distributions.

Recovering from Guest Network Customization Errors

Guest network customization is a long-running operation with several potential failure points. If you encounter a failure, consider the following steps.

The guest network customization engine holds a lock on the `VirtualMachineGuestCustomizationManager` interface while it applies changes to the guest. A failure during customization or while restarting the network might leave a hung process or an orphaned lock.

If any step of the network cutsomization returns an error, use the `VirtualMachineGuestCustomizationManager.AbortCustomization_Task` method to stop all running instances of the customization engine and to release the lock.

To troubleshoot a failure, you can examine the log file used by the `VirtualMachineGuestCustomizationManager.CustomizeGuest_Task` method. The log file is on the host file system at `/var/log/vmware-gosc/instant_clone_customization.log` for Linux hosts.

When you abort a guest customization operation, the guest network can be left in an inconsistent state. After the abort operation completes, you should check your customization settings and retry the `CustomizeGuest_Task` method in case of transient error conditions.

> **Note:**
> When you use the `VirtualMachineGuestCustomizationManager`, the customization procedure tolerates vMotion without failure.

Application-Dependent Customization

After you customize and restart the guest network, the virtual machine might need additional customization steps, depending on the applications that run in the guest operatin system. For example, you might need to generate new encryption keys for an instant clone, or signal an application to close and re-open network sockets when you have configured new network IP addresses.

To do additional customization steps in the guest, use the `GuestFileManager` and the `GuestProcessManager` interfaces in the `GuestOperationsManager` managed object.

Resetting the Network Stack in a Running Virtual Machine

In any case where you change the network identity settings in the guest operating system of a running virtual machine, you must reset the network stack for the virtual NIC to cause it to adopt the new settings. You can do this by invoking the `VirtualMachineGuestCustomizationManager.RestartNetwork_Task` method after you customize the network settings, or you can do the reset from the guest operating system, as with a physical machine. The way to reset a NIC differs, depending on the operating system.

For information about the `RestartNetwork_Task` method, see Restarting the Guest Network After Customization.

Resetting the Network Stack in a Linux Virtual Machine

When you change network settings in an instant clone, you must reset the network stack of the virtual NIC to adopt the new settings. For a Linux guest operating system, you must unbind and rebind the network driver so it adopts a new MAC address.

The following shell script rebinds all network drivers:

```
#!/bin/bash
for NETDEV in /sys/class/net/*
do
  DEV_LABEL=`basename \`readlink -f $NETDEV/device\``
  DEV_DRIVR=`basename \`readlink -f $NETDEV/device/driver\``
  echo $DEV_LABEL > /sys/bus/pci/drivers/$DEV_DRIVR/unbind
  echo $DEV_LABEL > /sys/bus/pci/drivers/$DEV_DRIVR/bind
done
```

If the guest operating system was configured to use DHCP, you must also force a DHCP refresh after the MAC addresses refresh has completed.

Tip: Avoid resetting the network stack in the guest operating system while you are working over a network connection, such as SSH. You can run the shell commands to reset the stack as part of a customization script installed in the guest operating system of the source virtual machine before running the instant clone operation.

Resetting the Network Stack in a Windows Virtual Machine

When you change network settings in a Windows virtual machine, you must disable and re-enable the network object. There are several ways to do that in Windows.

Reset the Network Stack in a Windows Virtual Machine Using the Control Panel

When you change network settings in a Windows virtual machine, you must reset the network stack of the virtual NIC to adopt the new settings. For a Windows guest operating system, you must disable and re-enable the network connection so it adopts a new MAC address. You can do this using the Control Panel.

1. In the Control Panel, navigate to **Control Panel** > **Networks and Internet** > **Network Connections**.
2. Right-click the network adapter icon and choose **disable**.
3. Right-click the network adapter icon and choose **enable**.
4. Repeat these steps for each network adapter.

Reset the Network Stack in a Windows Virtual Machine Using PowerShell

When you change network settings in a Windows virtual machine, you must reset the network stack of the virtual NIC to adopt the new settings. For a Windows guest operating system, you must disable and re-enable the network connection so it adopts a new MAC address. To perform this task with PowerShell, use the `Restart-NetAdapter` cmdlet.

The cmdlet is described in more detail at https://technet.microsoft.com/en-us/itpro/powershell/windows/netadapter/restart-netadapter.

1. List the virtual network adapters in the guest operating system.
   ```
   Get-NetAdapter
   ```
   The names of the virtual network adapters appear in the first column of output.
2. Restart the network adapter representing the external interface.
   ```
   Restart-NetAdapter -Name "Ethernet0"
   ```
3. If you have more than one virtual interface, repeat the previous step for each one.

Reset the Network Stack in a Windows Virtual Machine Using netsh

When you change network settings in a Windows virtual machine, you must reset the network stack of the virtual NIC to adopt the new settings. For a Windows guest, you must disable and re-enable the network connection so it adopts a new MAC address. To perform this task, run `netsh` with `interface` command.

Run the following commands in a command prompt (Run > `cmd`).

1. List the network interfaces.
   ```
   netsh interface show interface
   ```
2. Choose the interface that represents the external network connection and disable it.
   ```
   netsh interface set interface name="Ethernet0" admin=DISABLED
   ```
3. Re-enable the interface with a similar command.
   ```
   netsh interface set interface name="Ethernet0" admin=ENABLED
   ```
4. Repeat the previous two commands if you have more than one NIC that needs to be reset.

Virtual Machine Security

Encryption in vSphere protects virtual machines, virtual disks, and related files. First you set up a trusted connection between vCenter and a key management server (KMS), then vCenter can retrieve keys from the KMS as needed. Physical device security and advanced options are additional concerns.

Various aspects of virtual machine encryption are handled differently. You manage setup of the KMS trusted connection and perform most encryption workflows from the vSphere Client. You manage automation of some advanced features using the vSphere Web Services SDK, as discussed in this chapter. You use the `crypto-util` command-line tool directly on ESX hosts for some special cases, for example, to decrypt the core dumps in a vm-support bundle.

How Virtual Machine Encryption Protects a Datacenter

With vSphere virtual machine encryption, you can create encrypted virtual machines and encrypt existing ones. Because all virtual machine files with sensitive information are encrypted, the virtual machine is protected. Only administrators with encryption privileges can perform encryption and decryption tasks.

What Keys are Used
Two types of keys are used for encryption.

- The ESX host generates and uses internal keys to encrypt virtual machines and disks. These keys are used as the disk encryption key (DEK) and are XTS-AES-256 keys.
- The key management server (KMS) sends keys to the vCenter upon request. These keys are used as the key encryption key (KEK) and are AES-256 keys. vCenter stores only the ID of each KEK, but not the key itself.
- ESX hosts use the KEK to encrypt their internal keys, and store only the encrypted internal keys on disk, but not the KEK itself. When an ESX host reboots, vCenter requests the necessary KEKs by sending the corresponding IDs to the KMS, and upon receipt, make the KEKs available to the ESX host, which can then decrypt its internal keys as needed.

What Is Encrypted
Virtual machine encryption supports encrypting virtual machine files, virtual disk files, and core dump files.

Virtual Machine Files
Most virtual machine files, in particular guest data that are not stored in the VMDK file, are encrypted. This set of files includes but is not limited to the NVRAM (memory), VSWP (swap), and VMSN (snapshot) files. The key that vCenter retrieves from the KMS unlocks an encrypted bundle in the VMX file that contains internal keys and other secrets.

If you use the vSphere Client to create an encrypted virtual machine, all virtual disks are encrypted by default. For other encryption tasks, such as encrypting an existing virtual machine, you can encrypt and decrypt virtual disks separate from virtual machine files.

> **Note:**
>
> You cannot associate an encrypted virtual disk with an unencrypted virtual machine.

Virtual Disk Files
Data in an encrypted virtual disk (VMDK) file are never written in cleartext to storage or physical disk, and is never transmitted over the network in cleartext. The VMDK descriptor file is mostly cleartext, but contains a key ID for the KEK and the internal key (DEK) in the encrypted bundle.

You can use the vSphere API to perform either a shallow recrypt operation with a new KEK, or a deep recrypt operation with new internal keys.

Core Dump Files
Core dumps on an ESX host that has encryption mode enabled are always encrypted. You can decrypt and password protect ESX core dumps using the `crypto-util` command-line tool on the ESX host.

> **Note:**
>
> Core dumps on the vCenter (Appliance) are not encrypted. Be sure to protect access to all vCenter systems.

What Is Not Encrypted
Some files that are associated with a virtual machine are not encrypted or partially encrypted.

Log Files
Log files are not encrypted because they should not contain sensitive data.

Virtual Machine Configuration Files
Most of the virtual machine configuration information, stored in the VMX and VMSD files, is not encrypted. Information about the KMS and the key ID is visible in those files.

Virtual Disk Descriptor File
To support disk management without a key, most of the virtual disk descriptor file is not encrypted.

Who Can Perform Cryptographic Operations
Only users who are assigned the Cryptographic Operations privileges can perform cryptographic operations. The privilege set is fine grained; see the *vSphere Security* guide. The default Administrator system role includes all Cryptographic Operations privileges. A new system role, No Cryptography Administrator, supports all Administrator privileges except for the Cryptographic Operations privileges.

You can create additional custom roles, for example, to allow a group of users to encrypt virtual machines but to prevent them from decrypting virtual machines.

For a full list of privileges, see the section “Cryptographic Operations Privileges” in the *vSphere Security* manual.

How Can I Perform Cryptographic Operations

The vSphere Client supports many cryptographic operations. For other tasks, you must use the API.

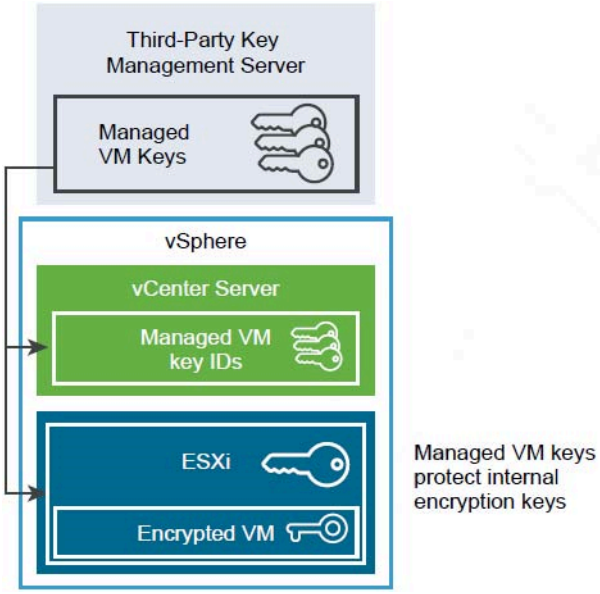
Table 1892: Interfaces for performing cryptographic operations

Interface	Operations	Information
vSphere Client	Create encrypted virtual machines Encrypt and decrypt virtual machines	The *vSphere Security* guide.
vSphere Web Services SDK	Create encrypted virtual machines Encrypt and decrypt virtual machines Perform a deep recrypt of virtual machines (use a different KEK and DEK). Perform a shallow recrypt of virtual machines (use a different KEK).	This chapter.
`crypto-util`	Decrypt encrypted core dumps, check whether files are encrypted, and perform other management tasks directly on the ESX host,	Command-line help and the *vSphere Security* guide.

vSphere Virtual Machine Encryption Components

An external key management server (KMS), the vCenter system, and ESX hosts all contribute to the vSphere virtual machine encryption solution.

Figure 607: vSphere Virtual Machine Encryption Architecture



Key Management Server

The vCenter requests keys from an external KMS. The KMS generates and stores the keys, and passes them to vCenter for distribution.

You can use the vSphere Client or the vSphere API to add a cluster of KMS instances to the vCenter system. If you use multiple KMS instances in a cluster, all instances must be from the same vendor and must replicate keys.

If your environment uses different KMS vendors in different environments, you can add a KMS cluster for each KMS and specify a default KMS cluster. The first cluster that you add becomes the default cluster. You can explicitly change the default later.

As a KMS client, vCenter uses the Key Management Interoperability Protocol (KMIP) and makes it easy to use the KMS of your choice.

vCenter

Only vCenter has credentials for logging in to the KMS. The ESX hosts do not have those credentials. The vCenter obtains keys from the KMS and pushes them to the ESX hosts. The vCenter does not store the KMS keys, it merely keeps a list of key IDs.

The vCenter checks the privileges of users who perform cryptographic operations. You can use the vSphere Client to assign cryptographic privileges or to assign the No cryptography administrator custom role to groups of users. See Prerequisites and Required Privileges for Encryption Tasks.

The vCenter adds cryptography events to the list of events that you can view and export from the vSphere Client Event Console. Each event includes the user, time, key ID, and cryptographic operation.

ESX Hosts

ESX hosts are responsible for several aspects of the encryption workflow.

- Accepting keys and storing them in memory (never on disk). If a host has encryption mode enabled, and the current user's role includes cryptographic operations privileges, vCenter pushes keys to the ESX host upon request. See Prerequisites and Required Privileges for Encryption Tasks.
- Ensuring that guest data for encrypted virtual machines are encrypted when stored on disk.
- Ensuring that guest data for encrypted virtual machines are never sent unencrypted over the network.

The keys that the ESX host generates are called internal keys in this document. These keys are typically act as data encryption keys (DEKs).

Encryption Process Flow

After vCenter is connected to the KMS, users with the required privileges can create encrypted virtual machines and disks. Those users can also perform other encryption tasks such as encrypting existing virtual machines and decrypting encrypted virtual machines. During the encryption process, different components interact as follows.

1. When the user performs an encryption task, for example, creating an encrypted virtual machine, vCenter requests a new key from the default KMS. This key will be used as the KEK.
2. The vCenter stores the key ID and passes the key to an ESX host. If the ESX host is part of a cluster, vCenter sends the KEK to each host in the cluster. The key itself is not stored on the vCenter system. Only the key ID is stored.
3. The ESX host generates internal keys (DEKs) for the virtual machine and its disks, using the KEK that it received from vCenter to encrypt the internal keys. The the internal keys are kept in memory only. Only the encrypted data are stored on disk.
4. The ESX host encrypts the virtual machine and its disks with the encrypted internal key.

ESX hosts that have the KEK and can access the encrypted key file can perform operations on an encrypted virtual machine or disk. Because they come from the KMS, ESX hosts can use the same KEK across reboots.

If you later want to decrypt a virtual machine, you change its storage policy either for the virtual machine or for its disks. If you want to decrypt individual components, first decrypt selected disks, then decrypt the virtual machine by changing the storage policy for VM Home. Both keys are required for decryption of each component, virtual disks or VM Home.

When you encrypt an existing virtual machine, you need at least twice the space that the virtual machine is currently using, in most cases.

Prerequisites and Required Privileges for Encryption Tasks

Users who perform encryption related tasks must have the appropriate privileges. Additional privileges are required if virtual machine encryption tasks require changing the host encryption mode. An extensive number of Cryptographic Operations privileges allow fine-grained control.

Encryption tasks are possibly only in environments that include a vCenter. Additionally, the ESX host must have encryption mode enabled for most encryption tasks. The user who performs an encryption task must have the appropriate privileges. Additional privileges are required if virtual machine encryption tasks require changing the host encryption mode. An extensive number of Cryptographic Operations privileges allow fine-grained control.

Cryptography Privileges and Roles

By default, the user with the vCenter Administrator role has all Cryptographic Operations privileges. You can assign the No cryptography administrator role to all vCenter administrators who do not need cryptographic privileges.

The user with the vCenter Administrator role has all privileges by default. You can assign the No cryptography administrator role to vCenter users who do not need Cryptographic Operations privileges. The No cryptography administrator lacks the following privileges for cryptographic operations:

- Add Cryptographic Operations privileges
- Global.Diagnostics
- Host.Inventory.Add host to cluster
- Host.Inventory.Add standalone host
- Host.Local operations.Manage user groups

To further limit what users can do, you can clone the No cryptography administrator role and create a custom role with only some of the Cryptographic Operations privileges. For example, you can create a role that allows users to encrypt but not to decrypt virtual machines, or that does grant privileges for management operations. See the *vSphere Security* manual for details.

Host Encryption Mode

You can encrypt virtual machines only if host encryption mode is enabled for the ESX host. Host encryption mode is often enabled automatically, but it can be enabled explicitly.

You can check and explicitly set the current host encryption mode from the vSphere Client or by using the vSphere API; see API Methods to Prepare an ESX Host.

After host encryption mode is enabled, it cannot be disabled easily. See the *vSphere Security* guide for details.

Automatic changes occur when encryption operations attempt to enable host encryption mode. For example, suppose that you add an encrypted virtual machine to an ESX host, and host encryption mode is not enabled. If you have the required privileges on the host, encryption mode automatically changes to enabled.

Assume a cluster that includes three ESX hosts, host A, B, and C. You add an encrypted virtual machine to host A. What happens depends on several factors. If all three hosts have encryption enabled, you can create an encrypted virtual machine if you have `Encrypt new` privileges. If none of the hosts has encryption enabled, and you have `Register host` privileges on host A, then the virtual machine creation process enables host encryption on that host; otherwise an error results. The scenario is more complicated if host B or C is not enabled for encryption; see the *vSphere Security* guide for details.

Encrypted vSphere vMotion

vSphere vMotion always tries to use encryption when migrating encrypted virtual machines. You cannot disable encrypted vMotion for encrypted virtual machines in a cluster.

For virtual machines that are not encrypted, you can set encrypted vMotion to `Opportunistic` (use encrypted vMotion if supported) or `Required` (do not migrate if unsupported). See Encrypted vSphere vMotion.

API Methods for vSphere Virtual Machine Encryption

Methods for managing vSphere virtual machine encryption are in the vSphere API, part of the Web Services SDK. In addition, you need a few Storage Policy APIs to create and assign encryption storage policies.

vSphere API Methods for KMS Management

Cryptographic key management interfaces are defined in the following hierarchy.

- CryptoManager – managed object, parent of CryptoManagerKmip. CryptoManager defines methods that are covered in vSphere API Methods for Cryptographic Operations.
  - CryptoManagerKmip – managed object for handing the cryptographic keys.
    - CertificateInfo – basic information of a certificate.
    - ClusterStatus – status of a KMS cluster.
    - ServerCertInfo – information about the KMS certificate.
    - ServerStatus – status of a KMS.

    CryptoManagerKmip defines the following methods:
  - `StringgenerateClientCsr(cluster)` – generate a certificate signing request with its private key.
  - `StringgenerateSelfSignedClientCert(cluster)` – generate self-signed client certificate with its private key.
  - `KmipClusterInfo[]kmipServers()` – get a list of registered KMS, grouped by clusters.
  - `KmipClusterInfo[]listKmipServers(limit)` – get a list of registered KMS.
  - `voidmarkDefault(clusterId)` – set the default KMS cluster.
  - `voidregisterKmipServer(server)` – register a KMS.
  - `voidremoveKmipServer(clusterId,serverName)` – remove a KMS, even if in use.
  - `StringretrieveClientCert(cluster)` – get the client certificate of the KMS cluster.
  - `StringretrieveClientCsr(cluster)` – get the generated client certificate signing request.
  - `ServerCertInforetrieveKmipServerCert(keyProvider,server)` – get the server certificate.
  - `ClusterStatus[]retrieveKmipServersStatus(clusters[])` – get status of the KMS instances.
  - `StringretrieveSelfSignedClientCert(cluster)` – get generated self signed client certificate.
  - `voidupdateKmipServer(server)` – update a KMS.
  - `voidupdateKmsSignedCsrClientCert(cluster,certificate)` – Set KMS signed certificate as the client certificate for a KMS cluster.
  - `voidupdateSelfSignedClientCert(cluster,certificate)` – set a self-signed certificate as the client certificate for the KMS cluster.
  - `voiduploadClientCert(cluster,certificate, privateKey)` – set a client certificate with private key for the KMS cluster.
  - `voiduploadKmipServerCert(cluster,certificate)` – upload a server certificate.
  - `CryptoKeyResultgenerateKey(keyProvider)` – generate a new encryption key. (The `generateKey` method is out of alphabetic order because it is called after the others.)

Certificate Management

Administrators can use the vSphere Client to manage certificates. Some organizations require a third party certificate externally but use VMware Certificate Authority (VMCA) internally. `CertificateManager` is the managed object to reach a certificate authority and refresh or revoke certificates. It includes these methods:

- `refreshCACertificatesAndCRLs` – Fetch certificates and CSR (revocation lists) from trusted certificate authorities and push them to hosts.
- `refreshCertificates` – Get CSRs and certificates signed by VMCA and push them down to hosts.
- `revokeCertificates` – Revokes certificates of specified hosts.

API Methods to Prepare an ESX Host

Encryption interfaces for ESX hosts are additions to a previously existing managed object, `HostSystem`.

- `HostSystem` – managed object providing access to the ESX host, including these additions:

  - `HostCryptoState` – enumeration indicating whether the ESX host is incapable of, prepared for encryption mode, or cryptography safe with its host key already set. Safe means that the ESX host can encrypt virtual machines and will not leak keys.
  - `ConfigureCryptoKey` – vCenter calls this method to set or change the key used for core dump encryption, and to place the host in safe mode. It is like calling `PrepareCrypto` and `EnableCrypto` in quick succession. Not supported if called on an ESX host. As of vSphere 7.0 you can disable encryption on a host, after next reboot, by setting `pendingIncapable` in `HostCryptoState`.
  - `PrepareCrypto` – this method puts the ESX host in prepared mode so it can receive sensitive data, after being enabled.
  - `EnableCrypto` – this method sets or changes the key for core dump encryption and puts the ESX host in safe mode. It must be called in sequence after `PrepareCrypto`.

  The following data objects are used by the `ConfigureCryptoKey` and `EnableCrypto` methods above, and by the CryptoManager methods below in vSphere API Methods for Cryptographic Operations.
  - `CryptoKeyPlain` – data object representing a plain text cryptographic key.
  - `CryptoKeyId` – data object representing a cryptographic key.
  - `CryptoKeyResult` – data object representing the result of a cryptographic key operation.

vSphere API Methods for Cryptographic Operations

Cryptographic operations are defined in the following hierarchy.

- `CryptoManager` – managed object for handling cryptographic keys.

  `CryptoManager` defines the following methods:
  - `void addKey(CryptoKeyPlain key)` – add plain key to the vCenter.
  - `CryptoKeyResult[] addKeys(CryptoKeyPlain[] keys)` – add multiple plain keys to vCenter.
  - `boolean enabled()` – indicate if the encryption feature is enabled.
  - `CryptoKeyId[] listKeys(int limit)` – list keys.
  - `void removeKey(CryptoKeyId key, boolean force)` – remove a key (only its ID is needed).
  - `CryptoKeyResult[] removeKeys(CryptoKeyId[] keys, boolean force)` – multiple keys.
- `VirtualMachineConfigSpec` – previously existing data object passed as parameter to `CreateVM_Task` and `ReconfigVM_Task`. One of its newly added properties is crypto, a CryptoSpec with one of the following options, which is inherited by all virtual disks and virtual machine configuration files (VM home).
  - `CryptoSpecEncrypt` – indicates that the virtual machine should be encrypted.
  - `CryptoSpecDecrypt` – indicates that the virtual machine should be decrypted.
  - `CryptoSpecDeepRecrypt` – indicates that all KEKs and DEKs should be replaced.
  - `CryptoSpecShallowRecrypt` – indicates that only KEKs should be replaced.
  - `CryptoSpecNoOp` – indicates that encryption settings should not be changed.
  - `CryptoSpecRegister` – indicates that the operation should send keys but should not modify the encryption settings of the virtual machine or virtual disk. When an encrypted disk is hot attached, the program must pass `CryptoSpecRegister` with the key ID that encrypted the disk. The key can be obtained from the Datastore Browser.

  These data objects are informational properties of `VMConfigFileInfo` and `VMDiskFileInfo`, respectively. They can be used to check whether the VM home and its virtual disks are encrypted.
  - `VmConfigFileEncryptionInfo` – the encryption information of a virtual machine configuration.
  - `VmDiskFileEncryptionInfo` – the encryption information of a virtual disk.

  The enumeration `EncryptedVMotionModes` controls whether encrypted vMotion is `disabled`, `required`, or `opportunistic` (fall back to unencrypted vMotion if necessary, the default option).

More information about the encryption interfaces is available in vSphere Management SDK Documentation under vSphere Web Services SDK, in the *vSphere API Reference*.

SPBM API Methods for Encryption

The managed object PbmProfileProfileManager (`pbm.profile.ProfileManager`) provides methods to set encryption related storage policies, including create and update.

- `PbmCreate(createSpec)` – create a capability-based storage profile defining storage requirements.

- `PbmUpdate(profileId, createSpec)` – update a storage profile with new capability requirements.

Workflows for vSphere Virtual Machine Encryption

The sections below present vSphere API methods for key acquisition and encryption.

Set Up the Key Management Server Cluster
Before you can start encrypting virtual machines, you must set up the Key Management Server (KMS) cluster. This includes adding the KMS, establishing trust with the KMS, and setting the default KMS cluster. The KMS must support the KMIP (Key Management Interoperability Protocol) 1.1 standard, and it must be capable of acting as a symmetric key foundry and server.

Add Key Management Server(s) to Your Datacenter
Your organization's security administrator may be responsible for this task. This person will need to provide the cluster name (if it already exists), the KMS server name, its IP address, its connection port number, and possibly a proxy address, proxy port number, KMS user name, and corresponding password.

The `CryptoManager.java` code to add a KMS and set the default KMS cluster appears in CryptoManager Java program to add KMS and set default cluster. The user running this program must have Cryptographic operations.Manage key servers privileges.

Establish a Trusted Connection by Exchanging Certificates
There are four ways to establish trust between a KMS and vCenter. Different KMS vendors require different ways.

Java examples for these four ways follow.

1. Upload your Root CA Certificate to the KMS. Obtain it manually and upload with the self-signed call.
2. Upload a self-signed vCenter Certificate to the KMS.
3. Have vCenter generate a certificate signing request (CSR), which the KMS signs and returns.
4. Download a security certificate and private key generated by the KMS to vCenter.

CryptoManager Java program to add KMS and set default cluster

```
package com.vmware.general;
import com.vmware.common.annotations.Action;
import com.vmware.common.annotations.Option;
import com.vmware.common.annotations.Sample;
import com.vmware.connection.ConnectedVimServiceBase;
import com.vmware.vim25.CryptoKeyId;
import com.vmware.vim25.CryptoKeyResult;
import com.vmware.vim25.CryptoManagerKmipServerCertInfo;
import com.vmware.vim25.KeyProviderId;
import com.vmware.vim25.KmipServerInfo;
import com.vmware.vim25.KmipServerSpec;
import com.vmware.vim25.ManagedObjectReference;
import com.vmware.vim25.RuntimeFaultFaultMsg;
/**
 * CryptoManager. Demonstrates uses of the CryptoManager API. Parameters:
 * url      [required] : web service url, for example https://10.9.8.7/sdk
 * username [required] : username for the authentication
 * password [required] : corresponding password
 * Command line to run CryptoManager code:
 * run.bat com.vmware.general.CryptoManager ^
 *         --url webserviceurl --username name --password pass ^
 *         --kmsname kms --kmsip ipaddr --kmsclusterid providerId
 */
@Sample(name = "CryptoManager", description = "Demonstrates uses of the CryptoManager API")
public class CryptoManager extends ConnectedVimServiceBase {
  private ManagedObjectReference cryptoManager = null;
  private KmipServerSpec kmipSpec = null;
```

```
private String kmsName;
private String kmsIp;
private int kmsPort = 5696; // default
private String kmsClusterId;

 public void initCryptoManager() throws RuntimeFaultFaultMsg {
    if (serviceContent != null) {
       cryptoManager = serviceContent.getCryptoManager();
       if (cryptoManager == null) {
          throw new RuntimeFaultFaultMsg("CryptoManager could not be obtained", null);
       }
    } else {
       throw new RuntimeFaultFaultMsg("ServiceContent could not be obtained", null);
    }
 }
 public void registerKmipServer() throws RuntimeFaultFaultMsg {
    KmipServerInfo serverInfo = new KmipServerInfo(); // Create KMS info
    serverInfo.setName(kmsName); // Set the name of your KMS here
    serverInfo.setAddress(kmsIp); // Set the IP addr of your KMS
    serverInfo.setPort(kmsPort); // Set KMS port, if different from default
    KeyProviderId providerId = new KeyProviderId(); // Set the name of KMS cluster here
    providerId.setId("KMScluster");
    kmipSpec = new KmipServerSpec();
    kmipSpec.setInfo(serverInfo); // KMS spec with server and cluster ID
    kmipSpec.setClusterId(providerId);
    vimPort.registerKmipServer(cryptoManager, kmipSpec); // Register server
 }
 public void trustKmip() throws RuntimeFaultFaultMsg { // Get KMS certificate
    CryptoManagerKmipServerCertInfo certInfo = vimPort.retrieveKmipServerCert(
          cryptoManager, kmipSpec.getClusterId(), kmipSpec.getInfo());
    // Upload retrieved certificate to vCenter and trust it
    vimPort.uploadKmipServerCert(
          cryptoManager, kmipSpec.getClusterId(), certInfo.getCertificate());
 }
 public void establishTrust() throws RuntimeFaultFaultMsg {
    // Make KMS trust vCenter by uncommenting and calling one of these lines
    // - establishTrustUsingSelfSignedCert() - see Example 12-2 for source code
    // - establishTrustUsingSignedCsr() - see Example 12-3 for source code
    // - establishTrustUsingCertAndKey() - see Example 12-4 for source code
    trustKmip(); // Now make the vCenter trust KMS
 }
 public void setDefaultKmipCluster() throws RuntimeFaultFaultMsg {
    vimPort.markDefault(cryptoManager, kmipSpec.getClusterId()); // Mark cluster as default
 }
 public void generateNewKey() throws RuntimeFaultFaultMsg {
    CryptoKeyResult keyResult = vimPort.generateKey(cryptoManager, kmipSpec.getClusterId());
    CryptoKeyId keyId = keyResult.getKeyId(); // Generate new key for encryption
 }
 @Action
 public void action() throws RuntimeFaultFaultMsg {
    initCryptoManager();
    registerKmipServer();
    establishTrust();
    setDefaultKmipCluster();
    generateNewKey();
 }
 @Option(name = "kmsname", description = "Name of the KMS", required = true)
```

```
    public void setKMSName(String name) {
       this.kmsName = name;
    }
    @Option(name = "kmsip", description = "IP address of the KMS", required = true)
    public void setKMSIp(String ip) {
       this.kmsIp = ip;
    }
    @Option(name = "kmsport", description = "KMS port", required = false)
    public void setKMSPort(String port) {
       this.kmsPort = Integer.parseInt(port);
    }
    @Option(name = "kmsclusterid", description = "KMS cluster Id", required = true)
    public void setKMSClusterId(String clusterId) {
       this.kmsClusterId = clusterId;
    }
}
```

Trust with self-signed certificate or Root CA certificate

This example method uploads a self-signed vCenter certificate, or the Root CA certificate, to the KMS.

```
public void establishTrustUsingSelfSignedCert() throws RuntimeFaultFaultMsg {
   // Generate self-signed cert, or obtain the Root CA certificate
   String selfSignedCert = vimPort.generateSelfSignedClientCert(
         cryptoManager, kmipSpec.getClusterId());
   // Follow steps for KMS to trust self-signed or Root CA cert, update vCenter to use it
   vimPort.updateSelfSignedClientCert(
         cryptoManager, kmipSpec.getClusterId(), selfSignedCert);
}
```

Trust with CSR then downloading KMS signed certificate

This example method generates a CSR and downloads the KMS signed certificate onto vCenter.

```
public void establishTrustUsingSignedCsr() throws RuntimeFaultFaultMsg {
   // Generate a Certificate Signing Request
   String csr = vimPort.generateClientCsr(cryptoManager, kmipSpec.getClusterId());
   String signedCert = null;
   // Follow steps for your KMS to sign CSR and get the signedCert to update on vCenter
   vimPort.updateKmsSignedCsrClientCert(
         cryptoManager, kmipSpec.getClusterId(), signedCert);
}
```

Trust by downloading KMS certificate and private key

This example method downloads a certificate and private key generated by the KMS to vCenter.

```
public void establishTrustUsingCertAndKey() throws RuntimeFaultFaultMsg {
   String certFromKms = null;
   String privateKeyKms = null;
   // Follow steps for KMS to generate certificate and private key (certFromKms, privateKeyKms)
   vimPort.uploadClientCert(
         cryptoManager, kmipSpec.getClusterId(), certFromKms, privateKeyKms);
}
```

Create an Encryption Storage Policy

Before any virtual machines can be encrypted, vCenter must contain an encryption storage policy. The policy only needs to be created once; it can be assigned to any virtual machines that you want to encrypt.

Java program to set storage policy for encryption

```
package com.vmware.spbm.samples;
```

```
import java.util.ArrayList;
import java.util.List;
import com.vmware.common.annotations.Action;
import com.vmware.common.annotations.Option;
import com.vmware.common.annotations.Sample;
import com.vmware.pbm.InvalidArgumentFaultMsg;
import com.vmware.pbm.PbmCapabilityConstraintInstance;
import com.vmware.pbm.PbmCapabilityInstance;
import com.vmware.pbm.PbmCapabilityMetadata;
import com.vmware.pbm.PbmCapabilityMetadataPerCategory;
import com.vmware.pbm.PbmCapabilityProfileCreateSpec;
import com.vmware.pbm.PbmCapabilityPropertyInstance;
import com.vmware.pbm.PbmCapabilityPropertyMetadata;
import com.vmware.pbm.PbmCapabilitySubProfile;
import com.vmware.pbm.PbmCapabilitySubProfileConstraints;
import com.vmware.pbm.PbmCapabilityVendorNamespaceInfo;
import com.vmware.pbm.PbmCapabilityVendorResourceTypeInfo;
import com.vmware.pbm.PbmDuplicateNameFaultMsg;
import com.vmware.pbm.PbmFaultProfileStorageFaultFaultMsg;
import com.vmware.pbm.PbmProfileId;
import com.vmware.pbm.PbmServiceInstanceContent;
import com.vmware.spbm.connection.ConnectedServiceBase;
import com.vmware.spbm.connection.helpers.PbmUtil;
import com.vmware.vim25.ManagedObjectReference;
import com.vmware.vim25.RuntimeFaultFaultMsg;
/**
 * CreateVMEncryptionProfile
 * Create new Storage Profile with one rule-set based on vmwarevmcrypt capabilities.
 *  Parameters:
 * vcurl       [required] : web service url, for example https://10.9.8.7/sdk
 * username    [required] : username for the authentication
 * password    [required] : corresponding password
 * profilename [required] : name of the storage profile
 * Command Line:
 * run.bat com.vmware.spbm.samples.CreateVMEncryptionProfile --vcurl [webserviceurl] ^
 *         --username [username] --password [password] --profilename [Storage Profile Name]
 */
@Sample(name = "CreateVMEncryptionProfile",
 description = "Create a new storage profile with "
             + "one rule-set based on vmwarevmcrypt capabilities.")
public class CreateVMEncryptionProfile extends ConnectedServiceBase {
   private PbmServiceInstanceContent spbmsc;
   private String profileName;

   // Build capability instance based on capability name associated with vmwarevmcrypt provider
   PbmCapabilityInstance buildCapability(String capabilityName,
                                         List<PbmCapabilityMetadataPerCategory> metadata)
   throws InvalidArgumentFaultMsg {
      // Create Property Instance with capability vmwarevmcrypt
      PbmCapabilityMetadata capabilityMeta =
            PbmUtil.getCapabilityMeta(capabilityName,metadata);
      if (capabilityMeta == null)
         throw new InvalidArgumentFaultMsg("Specified Capability does not exist", null);
      // Create and associate Property Instances with a Rule
      PbmCapabilityConstraintInstance rule = new PbmCapabilityConstraintInstance();
      for (PbmCapabilityPropertyMetadata propMeta : capabilityMeta.getPropertyMetadata()) {
         PbmCapabilityPropertyInstance prop = new PbmCapabilityPropertyInstance();
```

```
      prop.setId(propMeta.getId());
      prop.setValue(propMeta.getDefaultValue());
      rule.getPropertyInstance().add(prop);
   }
   // Associate Rule with a Capability Instance
   PbmCapabilityInstance capability = new PbmCapabilityInstance();
   capability.setId(capabilityMeta.getId());
   capability.getConstraint().add(rule);
   return capability;
}
@Action
public void createProfile() throws RuntimeFaultFaultMsg,
      com.vmware.pbm.RuntimeFaultFaultMsg, InvalidArgumentFaultMsg,
      PbmDuplicateNameFaultMsg, PbmFaultProfileStorageFaultFaultMsg {
   // Get PBM Profile Manager & Associated Capability Metadata
   spbmsc = connection.getPbmServiceContent();
   ManagedObjectReference profileMgr = spbmsc.getProfileManager();
   // Step 1: Check if there is a vmwarevmcrypt Provider
   Boolean encryptionCapable = false;
   List<PbmCapabilityVendorResourceTypeInfo> vendorInfo =
      connection.getPbmPort().pbmFetchVendorInfo(profileMgr, null);
   for (PbmCapabilityVendorResourceTypeInfo vendor : vendorInfo)
      for (PbmCapabilityVendorNamespaceInfo vnsi : vendor .getVendorNamespaceInfo())
         if (vnsi.getNamespaceInfo().getNamespace().equals("vmwarevmcrypt")) {
            encryptionCapable = true;
            break;
         }
   if (!encryptionCapable)
      throw new RuntimeFaultFaultMsg(
            "Cannot create storage profile. 'vmwarevmcrypt' Provider not found.",  null);
   // Step 2: Get PBM Supported Capability Metadata
   List<PbmCapabilityMetadataPerCategory> metadata =
     connection .getPbmPort().pbmFetchCapabilityMetadata(profileMgr,
                                                         PbmUtil.getStorageResourceType(),
                                                         "com.vmware.iofilters");
   // Step 3: Add Provider Specific Capabilities
   List<PbmCapabilityInstance> capabilities = new ArrayList<PbmCapabilityInstance>();
   capabilities.add(buildCapability("vmwarevmcrypt@ENCRYPTION", metadata));
   // Step 4: Add Capabilities to a RuleSet
   PbmCapabilitySubProfile ruleSet = new PbmCapabilitySubProfile();
   ruleSet.getCapability().addAll(capabilities);
   // Step 5: Add Rule-Set to Capability Constraints
   PbmCapabilitySubProfileConstraints constraints = new PbmCapabilitySubProfileConstraints();
   ruleSet.setName("Rule-Set " + (constraints.getSubProfiles().size() + 1));
   constraints.getSubProfiles().add(ruleSet);
   // Step 6: Build Capability-Based Profile
   PbmCapabilityProfileCreateSpec spec = new PbmCapabilityProfileCreateSpec();
   spec.setName(profileName);
   spec.setDescription("Storage Profile in SDK. Rule based on Encryption capability");
   spec.setResourceType(PbmUtil.getStorageResourceType());
   spec.setConstraints(constraints);
   // Step 7: Create Storage Profile
   PbmProfileId profile = connection.getPbmPort().pbmCreate(profileMgr, spec);
   System.out.println("Profile " + profileName + " with ID: " + profile.getUniqueId());
}
@Option(name = "profilename", description = "Name of the storage profile", required = true)
public void setProfileName(String profileName) {
```

```
            this.profileName = profileName;
        }
    }
```

Create an Encrypted Virtual Machine

The Web Services SDK provides Java code to create a virtual machine. The Java code can be modified to create an encrypted virtual machine. Steps below show how to implement virtual machine encryption in the `VMCreate.java` sample program. The `crypto` property in `VirtualMachineConfigSpec` should be set to `CryptoSpecEncrypt`, and an encryption key provided. The storage profile must have been previously set to specify encryption. If the `CryptoSpec` is unset, but a storage profile with encryption is set, vCenter automatically generates new keys and sets `CryptoSpec`, if a default KMS has been configured.

To enhance the `com.vmware.spbm.samples.VMCreate` program for encryption, follow these steps:

1. Import the following classes:
   ```
   import com.vmware.vim25.CryptoKeyId;
   import com.vmware.vim25.CryptoSpecEncrypt;
   import com.vmware.vim25.KeyProviderId;
   import com.vmware.vim25.VirtualDeviceConfigSpecBackingSpec;
   ```
2. Declare the following variables:
   ```
   private CryptoKeyId cryptoKeyId;
   private String keyId;
   private String providerId;
   ```
3. In the `createVirtualDisk()` function, find the following line:
   ```
   diskSpec.setOperation(VirtualDeviceConfigSpecOperation.ADD);
   ```
4. In the `createVirtualDisk()` function, add the following code after the line you found in the previous step:
   ```
   if ((keyId != null) && (providerId != null)) {
      CryptoSpecEncrypt cSpec = new CryptoSpecEncrypt();
      VirtualDeviceConfigSpecBackingSpec backingSpec =
            new VirtualDeviceConfigSpecBackingSpec();
      cSpec.setCryptoKeyId(buildCryptoKeyId());
      backingSpec.setCrypto(cSpec);
      diskSpec.setBacking(backingSpec);
   }
   ```
5. In the `createVmConfigSpec()` function, find the following line:
   ```
   VirtualMachineConfigSpec configSpec = new VirtualMachineConfigSpec();
   ```
6. In the `createVmConfigSpec()` function, add the following code after the line you found in the previous step:
   ```
   if ((keyId != null) && (providerId != null)) {
      CryptoSpecEncrypt cryptoSpecForVMHome = new CryptoSpecEncrypt();
      cryptoSpecForVMHome.setCryptoKeyId(buildCryptoKeyId());
      configSpec.setCrypto(cryptoSpecForVMHome);
   }
   ```
7. Add the following options toward the end of file:
   ```
   @Option(name = "keyid", description = "Key Id", required = false)
   public void setKeyId(String kid) {
      this.keyId = kid;
   }
   @Option(name = "providerid", description = "Cluster/Provider Id", required = false)
   public void setProviderId(String pid) {
      this.providerId = pid;
   }
   ```

The SPBM sample code is in the Storage Policy SDK, not in the vSphere Web Services SDK.

Clone an Encrypted Virtual Machine

Set the `crypto` property in `RelocateSpec` at `cloneSpec.location`. The virtual machine must be powered off, with no existing snapshots. The encrypted virtual machine will be cloned, still encrypted.

Clone an encrypted virtual machine

```
// Step 5: Create Specs
VirtualMachineRelocateSpec relocSpec = new VirtualMachineRelocateSpec();
VirtualMachineCloneSpec cloneSpec = new VirtualMachineCloneSpec();
VirtualMachineConfigSpec configSpec = new VirtualMachineConfigSpec();
// Step 6: Associate Storage Profile
relocSpec.getProfile().add(getVMDefinedProfileSpec(profileName));
cloneSpec.setConfig(configSpec);
cloneSpec.setLocation(relocSpec);
cloneSpec.setPowerOn(false);
cloneSpec.setTemplate(false);
// Step 7: Clone VM
ManagedObjectReference
     cloneTask = connection.getVimPort().cloneVMTask(vmRef, vmFolderRef, cloneName, cloneSpec);
```

Encrypt an Existing Virtual Machine or Disk

First set the `crypto` property in the `VirtualMachineConfigSpec` to `CryptoSpecEncrypt` and provide a key. The virtual machine must be powered off, with no existing snapshots.

Storage profiles must also be set to specify encryption. If the `CryptoSpec` is unset, but a storage profile with encryption is set, the vCenter automatically generates new keys and sets `CryptoSpec`, if the default KMS cluster has been configured.

Encrypt a virtual machine based on storage profile

```
void Encrypt() throws Exception {
   // Create VirtualMachineConfigSpec
   VirtualMachineConfigSpec vmConfigSpec = new VirtualMachineConfigSpec();
   // Create VirtualDeviceConfigSpec
   VirtualDeviceConfigSpec diskSpec = new VirtualDeviceConfigSpec();
   // Get VirtualMachineProfileSpec for new encryption profile and name it encryptionProfile
   VirtualMachineProfileSpec encryptionProfile = new VirtualMachineDefinedProfileSpec();
   // Get VirtualDisk for disk to reconfigure as in com.vmware.vm.VMReconfig, name it disk
   diskSpec.setDevice(disk);
   diskSpec.setOperation(VirtualDeviceConfigSpecOperation.EDIT);
   // Add encryption profile to VirtualDeviceConfigSpec
   diskSpec.getProfile().add(encryptionProfile);
   // Create CryptoSpec for encryption
   // Get Key Id from CryptoManager as keyId
   // See CryptoManager for details on generating or retrieving CryptoKeyId
   CryptoSpecEncrypt cryptoSpec = new CryptoSpecEncrypt();
   cryptoSpec.setCryptoKeyId(keyId);
   // Add CryptoSpecEncrypt to diskSpec backing
   VirtualDeviceConfigSpecBackingSpec backingSpec = new VirtualDeviceConfigSpecBackingSpec();
   backingSpec.setCrypto(cryptoSpec);
   diskSpec.setBacking(backingSpec);
   // When encrypting a VirtualDisk, the VM home needs to be encrypted also.
   // You can choose the same key to encrypt VM home and virtual disk, or use different keys.
   // Set cryptoSpec and profile for encrypting virtual machine home.
   vmConfigSpec.setCrypto(cryptoSpec);
   vmConfigSpec.getVmProfile().add(encryptionProfile);
   // Set the device changes
   vmConfigSpec.getDeviceChange().add(diskSpec);
   // Issue reconfigure - See reConfig() in com.vmware.vm.VMReconfig to reconfigure VM.
}
```

Decrypt an Encrypted Virtual Machine or Disk

This is similar to encrypting an existing virtual machine, but with a different `CryptoSpec`. First set the `crypto` property in the `VirtualMachineConfigSpec` to `CryptoSpecDecrypt`. The virtual machine must be powered off, and the storage profiles must be set not to specify encryption.

If the `CryptoSpec` is unset, but a storage profile without encryption is set, the vCenter sets `CryptoSpecDecrypt`. The `ConfigSpec.crypto` parameter must be set explicitly. Only encryption is deduced from the storage profile.

Decrypt a virtual machine based on storage profile

```
void Decrypt() throws Exception {
   // Create VirtualMachineConfigSpec
   VirtualMachineConfigSpec vmConfigSpec = new VirtualMachineConfigSpec();
   // Create VirtualDeviceConfigSpec
   VirtualDeviceConfigSpec diskSpec = new VirtualDeviceConfigSpec();
   // Create Empty VirtualMachineProfileSpec for decryption
   VirtualMachineProfileSpec emptyProfile = new VirtualMachineEmptyProfileSpec();
   // Get VirtualDisk for disk to reconfigure in com.vmware.vm.VMReconfig, name it disk.
   diskSpec.setDevice(disk);
   diskSpec.setOperation(VirtualDeviceConfigSpecOperation.EDIT);
   // Remove any encryption profile set for the disk
   diskSpec.getProfile().add(emptyProfile);
   // Create CryptoSpec for decryption
   CryptoSpecDecrypt cryptoSpec = new CryptoSpecDecrypt();
   // Add CryptoSpecEncrypt to diskSpec backing
   VirtualDeviceConfigSpecBackingSpec backingSpec = new VirtualDeviceConfigSpecBackingSpec();
   backingSpec.setCrypto(cryptoSpec);
   diskSpec.setBacking(backingSpec);
   // Decrypting virtual machine home is not necessary when decrypting a virtual disk.
   // If no encrypted disk is present on virtual machine after that disk is decrypted,
   // you can choose to decrypt virtual machine home.
   // Set cryptoSpec and profile for decrypting virtual machine home
   vmConfigSpec.setCrypto(cryptoSpec);
   vmConfigSpec.getVmProfile().add(emptyProfile);
   // Set the device changes
   vmConfigSpec.getDeviceChange().add(diskSpec);
   // Issue reconfigure - See reConfig() in com.vmware.vm.VMReconfig to reconfigure VM.
}
```

Encrypt with Different Keys for VM and Disk

This method shows how to use two different keys to encrypt the virtual machine (VM home) and its disk.

Different encryption keys for VM home and virtual disk

```
void EncryptUsingDifferentKeys() throws Exception {
   // Create VirtualMachineConfigSpec and VirtualDeviceConfigSpec
   VirtualMachineConfigSpec vmConfigSpec = new VirtualMachineConfigSpec();
   VirtualDeviceConfigSpec diskSpec = new VirtualDeviceConfigSpec();
   // Get VirtualMachineProfileSpec for encryption profile you created and name it
   VirtualMachineProfileSpec encryptionProfile = new VirtualMachineDefinedProfileSpec();
   // Get VirtualDisk for disk to be reconfigured as shown in VMReconfig and name it
   diskSpec.setDevice(disk);
   diskSpec.setOperation(VirtualDeviceConfigSpecOperation.EDIT);
   // Add encryption profile to VirtualDeviceConfigSpec
   diskSpec.getProfile().add(encryptionProfile);
   // Create CryptoSpec for disk encryption and get KeyId from from CryptoManager
   // See CryptoManager for details on generating or retrieving CryptoKeyId
   CryptoSpecEncrypt cryptoSpecForDisk = new CryptoSpecEncrypt();
   cryptoSpecForDisk.setCryptoKeyId(keyIdForDiskEncryption);
```

```
    // Add CryptoSpecEncrypt to diskSpec backing
    VirtualDeviceConfigSpecBackingSpec backingSpec = new VirtualDeviceConfigSpecBackingSpec();
    backingSpec.setCrypto(cryptoSpecForDisk);
    diskSpec.setBacking(backingSpec);
    // When encrypting a VirtualDisk, the virtual machine home must also be encrypted.
    // You can choose the same key to encrypt VM home and disk, or use different keys.
    // Create CryptoSpec for VM Home encryption and get KeyId from CryptoManager.
    CryptoSpecEncrypt cryptoSpecForVMHome = new CryptoSpecEncrypt();
    cryptoSpecForVMHome.setCryptoKeyId(keyIdForVMHomeEncryption);
    // Set cryptoSpec and profile for encrypting virtual machine home
    vmConfigSpec.setCrypto(cryptoSpecForVMHome);
    vmConfigSpec.getVmProfile().add(encryptionProfile);
    // Set the device changes
    vmConfigSpec.getDeviceChange().add(diskSpec);
}
```

Recrypt (Rekey) Encrypted Virtual Machines

You can recrypt virtual machines using either the vSphere API or the vSphere Client (as of 7.0.x). In the *vSphere Security* manual, recrypt is usually called rekey, or sometimes re-encrypt.

There are two kinds of recryption operations. Deep recrypt replaces all keys, rewriting encrypted data in a powered-off virtual machine and its disks. Shallow recrypt replaces only top-level keys and is comparatively fast.

For details on generating or retrieving the CryptoKeyId, see CryptoManager code in CryptoManager Java program to add KMS and set default cluster.

Recrypt Only Key Encryption Keys

For shallow recrypt, which affects only the key encryption keys (KEKs), set the `crypto` property in the `VirtualMachineConfigSpec` to `CryptoSpecShallowRecrypt` and call the `Reconfigure` method.

Shallow Recrypt

```
void ShallowRecrypt() throws Exception {
    // Shallow recrypt follows the same flow as encrypt. The two differences are:
    // - Instead of using a new encryption profile, just get the previously
    //   applied profile from the virtual machine to be reconfigured and use it.
    // - The type of CryptoSpec object created is CryptoSpecShallowRecrypt()
    // Create CryptoSpec for shallow recrypt
    // Get Key Id from CryptoManager as newKeyId
    CryptoSpecShallowRecrypt cryptoSpec = new CryptoSpecShallowRecrypt();
    cryptoSpec.setNewKeyId(newKeyId);
    // Follow steps from Encrypt()
}
```

Shallow recrypt can be done with the virtual machine powered on. Deep recrypt requires power off.

Recrypt Both Key and Disk Encryption Keys

For deep recrypt, which affects both KEKs and disk encryption keys (DEKs), set the `crypto` property in the `VirtualMachineConfigSpec` to `CryptoSpecDeepRecrypt` and call the `Reconfigure` method.

Deep Recrypt

```
void DeepRecrypt() throws Exception {
    // Deep recrypt follows the same flow as encrypt. The two differences are:
    // - Instead of using a new encryption profile, just get the previously
    //   applied profile from the virtual machine to be reconfigured and use it.
    // - The type of CryptoSpec object created is CryptoSpecDeepRecrypt()
    // Create CryptoSpec for deep recrypt
    // Get Key Id from CryptoManager as newKeyId
    CryptoSpecDeepRecrypt cryptoSpec = new CryptoSpecDeepRecrypt();
```

```
        cryptoSpec.setNewKeyId(newKeyId);
        // Follow steps from Encrypt()
    }
```

Query Crypto Key In-Use Status

Sometimes it is useful to determine key status, for example whether a key is available on vCenter, which virtual machines and ESX hosts are using it, and if third party applications reference it.

As of vSphere 6.7 Update 2, the `queryCryptoKeyStatus` method is available to check use of a KMS key, such as if vCenter can access the key, if the key is being used by some virtual machines, or as a host key. The first parameter is a key ID or an array of key IDs to query. The second parameter is a bitmap defining items to check:

- 0x01 - ask if key data is available to vCentervCenter
- 0x02 - query which virtual machines use this key
- 0x04 - check the ESX hosts using this as a host key
- 0x08 - check third party programs using this key.

Key status results are returned in a `CryptoManagerKmip::CryptoKeyStatus` data object, called `keyStatusArray` in the code below. If your program queried multiple keys, an array of results is returned.

The `queryCryptoKeyStatus` method and its returned data object are new in vSphere 6.7 U2, so it does not work in earlier releases.

The following Java code checks all items above by passing 15, a bitwise OR of the settings above.

```
CryptoManagerKmipCryptoKeyStatus[] keyStatusArray =
   vimPort.queryCryptoKeyStatus(cryptoManager,
   [vmKeyId, diskKeyId, srcHostkey, dstHostkey], 15);
for (CryptoManagerKmipCryptoKeyStatus keyStatus : keyStatusArray[]) {
   System.out.println("keyId: " + keyStatus.getKeyId);
   System.out.println("keyAccessible: " + keyStatus.getKeyAvailable)
   System.out.println("reason: " + keyStatus.getReason)
   System.out.println("encryptedVMs: " + keyStatus.getEncryptedVMs)
   System.out.println("affectedHosts: " + keyStatus.getAffectedHosts)
   System.out.println("referencedByTags: " + keyStatus.getReferencedByTags)
}
```

The `keyAccessible` means the key is available on vCenter. The `reason` is either valid, or indicates why the key is not available. The `encryptedVMs` is an array of virtual machine MoRefs, and `affectedHosts` is an array of MoRefs to ESX hosts. The `referencedByTags` field gives names of third party applications using the key.

Encrypted vSphere vMotion

When virtual machines and data move across the network, all sensitive data remain encrypted.

Enable Encrypted vMotion

The enumeration VirtualMachineConfigSpecEncryptedVMotionModes controls whether encrypted vMotion is disabled, opportunistic (fall back to default unencrypted vMotion if necessary), or required. If encrypted vMotion is required, or opportunistic and available, the vCenter can migrate either encrypted or unencrypted virtual machines within a disk cluster, or from one disk cluster to another using Storage vMotion.

Virtual Disk Manager

The following methods in managed object `VirtualDiskManager` are affected by encryption:

`copyVirtualDisk` – If the source disk is encrypted, its copied disk is encrypted with the same key, regardless of the `crypto` spec. When calling `copyVirtualDisk_Task` on vCenter, do not specify the `destSpec` parameter, which throws a Not Implemented fault; `destSpec` is supported only on ESX hosts.

`createVirtualDisk` – Given a valid `crypto` spec, this task creates an encrypted virtual disk.

`moveVirtualDisk` – Encrypted virtual disk cannot be moved to an unencrypted virtual machine.

`createChildDisk` – The child disk of an encrypted disk remains encrypted with the same key.

`reparentDisks` – Encrypted child disk cannot be reparented to a new disk that is not in the same disk chain.

Best Practices for Virtual Machine Encryption

This section gives tips for optimum use of keys and virtual machine encryption.

Key Lifecycle and Removal

The `removeKey` and `removeKeys` methods delete key(s) from vCenter, but they do not delete keys from the KMS. Key lifecycle is managed entirely from the KMS, where stale keys persist. You can invoke the `listKeys` method to show keys in use on the vCenter, but there is currently no method to query whether a specific key is in use.

Be Careful with Force Remove

The `force` parameter of `removeKey` and `removeKeys` should be used judiciously. With the `force` option, the `removeKey` and `removeKeys` methods delete key(s) from both the vCenter and ESX hosts, even if the key is currently in use. The result can leave virtual machines in a locked state until the key is replaced. The use case for the force option is for customers to prevent a key from being used anywhere, after it was compromised or expired.

Remove Keys on ESX host

If you call `removeKey` on an ESX host, even without the `force` option, the key gets deleted from the host's key cache, and the encrypted virtual machine becomes unusable. ESX hosts do not track which keys are in use. Rebooting the ESX host causes vCenter to push all keys to the host again, but the virtual machine may not be fully recoverable from its failed state.

Carefully Manage Differential Keys

When you encrypt both a virtual machine and its virtual disks from the vSphere Client, the same key is used for both. When you encrypt a virtual machine and its disks using the API, you can set different encryption keys for the virtual machine and each virtual disk. If one of the disk keys is missing, the power on operation may fail. If the missing key is for a non-boot virtual disk, you can remove that disk from the virtual machine and retry the power on operation. Take care when managing the lifecycle of different keys for a virtual machine and its disks.

Rename During Registration

The `registerVM_Task` method can rename a virtual machine at registration time. In vSphere 6.5 this required a two-step workaround for encrypted virtual machines, but as of vSphere 6.7, this operation is supported with encryption. Also as of vSphere 6.7, the `reloadVirtualMachineFromPath_Task` is supported for encrypted virtual machines. The reload API is equivalent to unregistering and registering a virtual machine on a different configuration path.

Encryption on vSAN Datastores

For security on vSAN datastores, you should use vSAN encryption instead of virtual machine encryption. This is because vSAN encryption is designed to be compatible with deduplication and compression, whereas virtual machine encryption causes vSAN deduplication and compression to become less effective.

Unlocking Encrypted Virtual Machines

There are many reasons why an encrypted virtual machine could be, in effect, locked. For solutions, see section "Resolve Missing Key Issues" in the *vSphere Security* manual.

In vSphere 6.7 and later you can unlock locked virtual machines with the `cryptoUnlock_Task` method. A vCenter alarm notifies you when an encrypted virtual machine is in a locked state. To unlock it you must have **Cryptographic operations.RegisterVM** privilege. Before unlocking, it is a good idea to troubleshoot and attempt to fix the cause of the locked virtual machine. The `cryptoUnlock_Task` method takes one parameter, the `MoRef` of a virtual machine.

Configuring Advanced Options

The advanced options settings of an ESX host can affect virtual machine security.

Advanced options include welcome messages, `sshd` prompts, SOAP session timeout, eager zeroing of VM memory, login attempts before lockout, seconds delay after lockout, password history, password duration, password quality, DCUI timeout, shell timeout, and host client session timeout. Eager zeroing of memory can be helpful for security.

Setting advanced options on individual ESX hosts using the UI can be impractical at scale. The vSphere API offers a programmatic interface to manipulate advanced options. The UI is built on top of the API calls.

Advanced options are controlled by the `OptionManager` managed object, a property of `HostConfigManager`. You can show advanced options with `QueryOptions`, and set them with `UpdateOptions`. For details, see the *vSphere API Reference*.

Querying Advanced Options

The full list of advanced options is available in the `OptionManager.supportedOption[]` field, and the list of non-default settings is available in the `OptionManager.setting[]` field. The contents of these arrays are fixed for a particular ESX build and do not change at runtime.

You can use the `QueryOptions` method to get the setting for any supported option.

Setting Advanced Options

To set an advanced option, call the `UpdateOptions` method with the desired key and value. The following pseudo-code sets the advanced option for memory zeroing to "1" for true:

```
MoRef optionMgr = hostSystem.configManager.advancedOption;
opts = new OptionValue[]
opts[0].key = "Mem.MemEagerZero"
opts[0].value = "1"
optionMgr.UpdateOptions(changedValue=opts)
```

Physical Platform Resources

ESX hosts allow direct access to physical devices in limited scenarios.

Many vSphere devices are implemented as virtual devices. Access attempts from the guest are trapped and implemented in software. For example, the keyboard is implemented by trapping I/O port access in the guest and transmitting keycode data obtained using a remote console connection; it is not possible to connect the physical keyboard to any VM.

Several categories of devices can be configured for direct access, including USB, storage, optical disc drives, virtual GPU, and PCI passthrough (DirectPath I/O) devices. Use of physical "passthrough" resources is uncommon, because it prevents vMotion. Some physical resources can be configured for access and still allow vMotion.

Physical device access can be revoked by removing an associated virtual device from its VM configuration.

USB

In its hardware devices field, data object `VirtualMachineConfigInfo` may contain the type `VirtualUSB`. Virtual devices can have a backing field, which specifies access to a particular physical device. For USB connectivity, the VM must have a compatible virtual controller configured for the VM. Programs can set the type of backing object:

- `vim.vm.device.VirtualUSB.USBBackingInfo` – USB device is attached to the ESX host as a passthrough device, which disallows vMotion.
- `vim.vm.device.VirtualUSB.RemoteHostBackingInfo` – USB device is attached to a particular remote ESX host, which remains attached to the device during vMotion.
- `vim.vm.device.VirtualUSB.RemoteClientBackingInfo` – USB device gets forwarded over the network along with the VM's remote console.

See data object `VirtualUSB` in the *vSphere API Reference* for details. You can use the `esxcli` command to find host-attached USB devices for connection to a VM. The following command prints available devices, including VID (Vendor ID) and PID (Product ID), which programs use to configure backing objects types above:

```
esxcli hardware usb passthrough device list
```

You can also attach USB devices to a VM using the Host Client. See procedure "Add a USB Device to a Virtual Machine in the VMware Host Client" on https://docs.vmware.com.

Storage

Passthrough storage devices are known as Raw Device Mappings (RDM) and represent essentially a passthrough of SCSI commands to a particular LUN. RDM can be used with vMotion if LUN IDs are kept consistent across participating ESX hosts.

Configuring an RDM occurs through the `vim.vm.ConfigInfo.hardware.devices[]` array, with instances of type `vim.vm.device.VirtualDisk` that have backing field of type `vim.vm.device.VirtualDisk.RawDiskVer2BackingInfo`.

Physical CD-ROM or DVD

A physical disc drive on a host can be attached directly to a VM. Such drives are automatically detected by the host; no configuration is required. You can configure VMs to access a physical disc drive. See procedure "Add a CD or DVD Drive to a Virtual Machine in the vSphere Client" on https://docs.vmware.com.

Virtual GPU (vGPU)

When used in conjunction with an nVidia graphics card and the nVidia GRID software package, a vGPU can be passed through to one or more VMs for GPU or general purpose GPU (GPGPU) acceleration.

PCI Passthrough

ESX allows specific PCI devices to be configured for passthrough use by a VM instead of by the host. This feature is called DirectPath I/O and is used in limited cases for improved low-latency performance. When operating in this configuration, the PCI device must be behind a VT-d bridge that implements an IOMMU. VMkernel arranges for this IOMMU to be programmed with only pages that the guest is allowed to access. This ensures that the guest can program the PCI device to access only pages that the guest should access. If the VM powers off or becomes unavailable, the PCI device receives a PCI “reset” to clear any pending DMA transactions before detaching the IOMMU domain from the device.

The host configuration that indicates which devices are eligible for assignment to VMs is through the `vim.host.PciPassthruSystem` managed object, accessible using the host’s `ConfigManager.pciPassthruSystem`. The `UpdatePassthruConfig` method on this managed object can be used to enable or disable PCI passthrough for a particular PCI bus address (bus:slot:function). Making a device eligible for passthrough usually requires host reboot. Devices not configured for PCI passthrough on the host are instead claimed by ESX and are thus unavailable to VMs.

The VM configuration uses `vim.vm.ConfigInfo.hardware.devices[]`. When an instance in that array is of type `vim.vm.device.VirtualPCIPassthrough`, the backing field has type `vim.vm.device.VirtualPCIPassthrough.DeviceBackingInfo` and this object describes the exact PCI device that is being passed through.

To enable power-on of a VM, the administrator must specify a PCI device that can be configured for assignment to the VM.

Physical Network

ESX does not allow guests to have direct access to a physical network except when the administrator configures a virtual switch with a network interface attached to specific physical network interfaces. Any VMs expected to have external network access may be configured to have their virtual network interfaces connected to this virtual switch. See Using a Distributed Virtual Switch for information about configuring virtual switches and virtual network interface backings.

Alternatively, the administrator can configure a virtual switch to connect with a specific VLAN on the physical network and then connect VMs to that virtual switch. This provides logical isolation of the VMs from the physical network. See Adding a Virtual Port Group for information about configuration of the management network.

Virtual Applications: the vApp

A virtual application consists of one or more virtual machines, which are deployed, managed, and maintained as a single unit. This chapter explains how to use the vSphere Web Services SDK for building and managing a virtual application.

About Virtual Applications

A virtual application specifies and encapsulates the components of virtual machines and applications, and the operational policies and service levels associated with those components. A virtual application can be as simple as an individual virtual machine with a specific operating system (virtual appliance), or as complex as a complete corporate Web site. Each virtual machine in a virtual application contains a preinstalled, preconfigured operating system and might contain an application stack optimized to provide a specific set of services.

In the vSphere Web Services SDK, the `VirtualApp` managed object represents a virtual application. A `VirtualApp` object extends `ResourcePool` with the following capabilities:

- Store product information such as product name, vendor, properties, and licenses in `vAppConfigInfo`.
- Specify power-on and power-off sequence specification.
- Import and export of `VirtualApp` objects as OVF packages.
- Perform application-level customization using the OVF environment.

Management Overview

You can use the Web Services SDK to create and manage virtual applications by following these steps:

1. Call the `CreateVApp` method to create a virtual application without children. See Creating a VirtualApp.
2. Add child objects. See Managing VirtualApp Children.
3. Export the `VirtualApp` to OVF (`ExportVApp` method) See Exporting a Virtual Application.

   You can then import the OVF to create and customize the virtual application.

Direct and Linked Children

A virtual application consists of one or more child virtual machines or virtual applications. `VirtualApp` children have the following characteristics:

- Each child has exactly one parent `VirtualApp`.
- Each child can participate in power-on and power-off sequences.
- The lifetime of each child is determined by the parent `VirtualApp` object.

`VirtualApp` children are either direct or linked, based on where a child derives its resources.

- Direct Children. A direct child of a virtual application is a virtual machine or virtual application object that you add explicitly. See Managing VirtualApp Children for a list of methods. Direct children share resources with the parent `VirtualApp` object. Both virtual machines and virtual application can be direct children.
- Linked Children. A linked child of a virtual application is a virtual machine or virtual application that you add by calling the `UpdateLinkedChildren` method. Linked children increase the flexibility of the `VirtualApp` by allowing child entities to use different resources from the parent `VirtualApp` object. Linked children can be part of a different clusters, but a virtual application and its children must be in the same `Datacenter`. Both virtual machines and virtual applications can be linked children.

Linked children gives better flexibility. In particular, you can create virtual applications that span clusters. The vSphere Client does not support adding or removing links, though it does show links.

When you add a linked child to a virtual application, the following rules apply:

- An `InvalidArgument` fault is thrown if the `UpdateLinkedChildren` method is called on a link target that is a direct child of another virtual application.

- When you add a virtual machine or virtual application that is already a linked child of another virtual application, the existing link is removed and replaced with the new link.
- The life-time of a linked child is determined by the `destroyWithParent` property on the `VAppEntityConfigInfo` data object. If set to `true`, the child is destroyed when the parent `VirtualApp` is destroyed. Otherwise, the link is removed when the `VirtualApp` is destroyed.

If you add a virtual application that consists of multiple entities, for example multiple virtual machines, the entities are moved sequentially and committed one at a time, as specified in the list. If a failure is detected, the method terminates with an exception.

OVF Packages

Open Virtualization Format (OVF) is a distribution format for virtual applications. vSphere uses the OVF package as a unit of distribution and storage for virtual applications. Because these entities are uploaded, downloaded, and stored in OVF package format, vSphere supports access to and deployment of a wide variety of virtual applications.

A virtual application typically consists of one or more virtual disk files and a configuration file.

- The virtual disk files contain the operating systems and applications that run on the virtual machines in the virtual application.
- The configuration file contains metadata that describes how the virtual application is configured and deployed.

An OVF package might also include certificate and manifest files.

The OVF package contains metadata that describes the capabilities and infrastructure requirements of the virtual application, and contains references to the virtual disks and other files that store the virtual machine state. Most of this information is stored in an XML document called the OVF envelope. When an OVF package is instantiated into either a `VirtualApp` or a `VirtualMachine` object (which depends on metadata in the envelope), then the configuration stored in the OVF envelope is applied to the `VirtualVApp` and the `VirtualMachine` objects.

Some of the information in the OVF file is used unaltered, with entire `ovf:Section_Type` elements included in the `VirtualApp` object body. Other sections are transformed or extended by instantiation. You do not need detailed knowledge of all OVF package elements, but a basic understanding of key parts of the package and how they relate to virtual applications is useful.

See the OVF specification at the DMTF Web site for additional information.

Creating a VirtualApp

You always create a `VirtualApp` without children. The `CreateVApp` method includes the following parameters:

- `resSpec` – Properties you would specify for a `ResourcePool`.
- `configSpec` – `VAppConfigSpec` data object for specifying virtual-application specific information.
- `vmFolder` – Depends on the `VirtualApp` structure:
  - When creating top-level virtual applications, that is, virtual applications with no ancestor virtual applications, you must specify a folder.
  - If the `VirtualApp` has another virtual application in the ancestry chain, the `folder` parameter must be `NULL` when you create the `VirtualApp`.

Managing VirtualApp Children

You can add virtual machines and virtual applications to your virtual application as direct or linked children.

You use different methods for adding or removing direct or linked children, as follows:

- Direct children. Use one of the following methods:
  - `CreateChildVMTask` adds a new virtual machine.
  - `CreateVApp` adds a new virtual application.
  - `MoveIntoResourcePool` adds or removes an existing virtual machine or virtual application
- Linked children. `Use UpdateLinkedChildren` to add or remove virtual machines or virtual applications.

You can call the `UpdateVappConfig` method to specify how each virtual machine fits into the virtual application.

Property	Enumeration
`destroyWithParent`	True if the entity should be removed when the `VirtualApp` is removed.
`key`	Key for the virtual machine or virtual application, a managed object reference to the child.
`startAction`	One of the strings in the `VAppAutoStartAction` enumeration.
`startDelay`	Delay, in seconds, before continuing with the next entity.
`startOrder`	Specifies the start order for this entity. Entities are started from lower numbers to higher-numbers and reverse on shutdown. Multiple entities with the same start order are started in parallel and the order is unspecified. This value must be 0 or higher.
`stopAction`	Defines the stop action for the entity. Can be set to `none`, `powerOff`, `guestShutdown`, or `suspend`. If set to `none`, then the entity does not participate in auto-stop.
`stopDelay`	Delay, in seconds, before continuing with the next entity.
`tag`	Tag for the entity.
`waitingForGuest`	Determines if the virtual machine should start after receiving a heartbeat, from the guest.

For more information about direct children and linked children, see Direct and Linked Children.

Exporting a Virtual Application

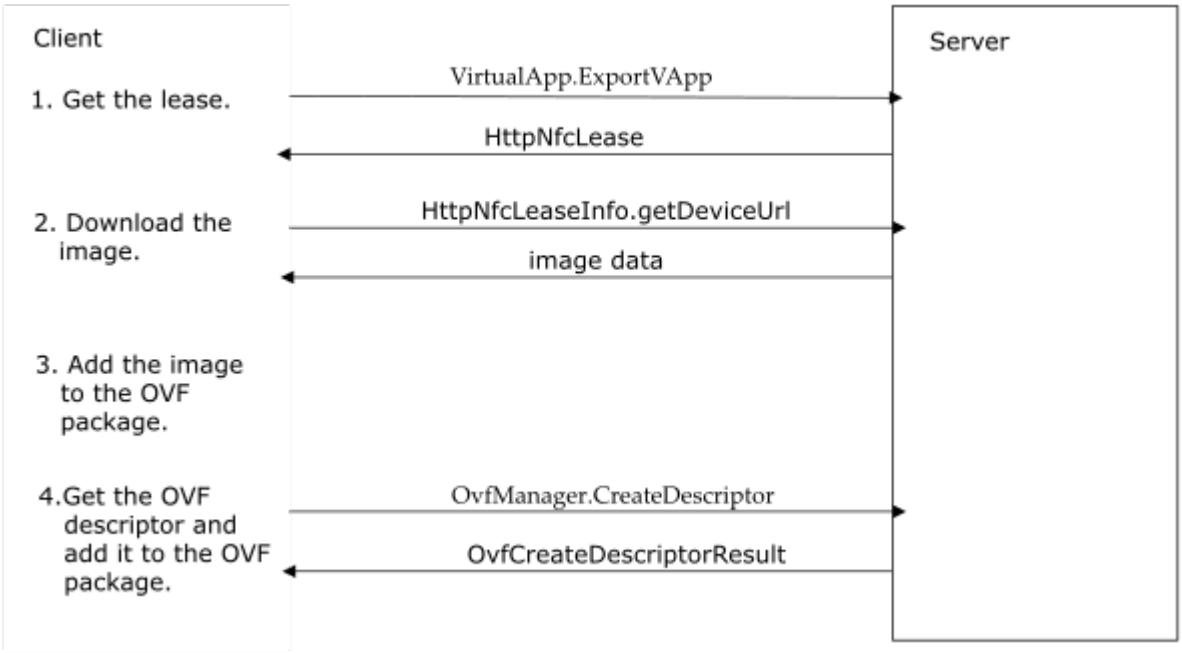
To export a virtual application, you must generate an OVF package. The Web Services API supports the generation of OVF packages. It does not support the generation of OVA files. An OVA file is a tar file that contains an OVF package. The OVF package consists of one of more images and an OVF file descriptor. You can create an OVA file by creating a tar file out of the OVF package for your exported virtual application.

The following steps describe how to use the vSphere `VirtualApp` and `OvfManager` API to generate an OVF package for a virtual application. The steps assume the simplest scenario: downloading one image from one device URL. You use the same steps to download many images from many device URLs. You can also export a VirtualMachine with the same steps, but use `VirtualMachine.ExportVm` rather than `VirtualApp.ExportVApp`.

1. Call the `VirtualApp.ExportVApp` method, which returns `HttpNfcLease`. The `deviceURL` is stored in the `info` property of `HttpNfcLease`.
2. Call the `HttpNfcLeaseInfo.getDeviceUrl` method to access the device URL and download the image data from the device URL.
3. Add the image to the OVF package.
4. Call the `OvfManager.CreateDescriptor` method, which returns `OvfCreateDescriptorResult`. Write the file descriptor to a file with the file extension `.ovf`. Add the `.ovf` file to the OVF package.

   Figure 608: Generating an OVF Package shows the major steps.

Figure 608: Generating an OVF Package



VirtualApp and OvfManager Methods

The following table describes the methods used by the `VirtualApp` and the `OvfManager` API:

Method	Description
CreateDescriptor	Creates an OVF descriptor for the specified `ManagedEntity`, which may be a `VirtualMachine` or a `VirtualApp`. `CreateDescriptor` is a method in the `OvfManager` managed object.
ExportVApp	Obtains an export lease on the virtual application. The export lease contains a list of URLs for the disks of the virtual machines in this virtual application. `ExportVApp` is a method in the `VirtualApp` managed object.
getDeviceUrl	Retrieves the device IDs and URLs from the server. `getDeviceUrl` is an accessor method provided in the generated JAX-WS bindings. It does not appear in the class diagram in VirtualApp Data Structures.

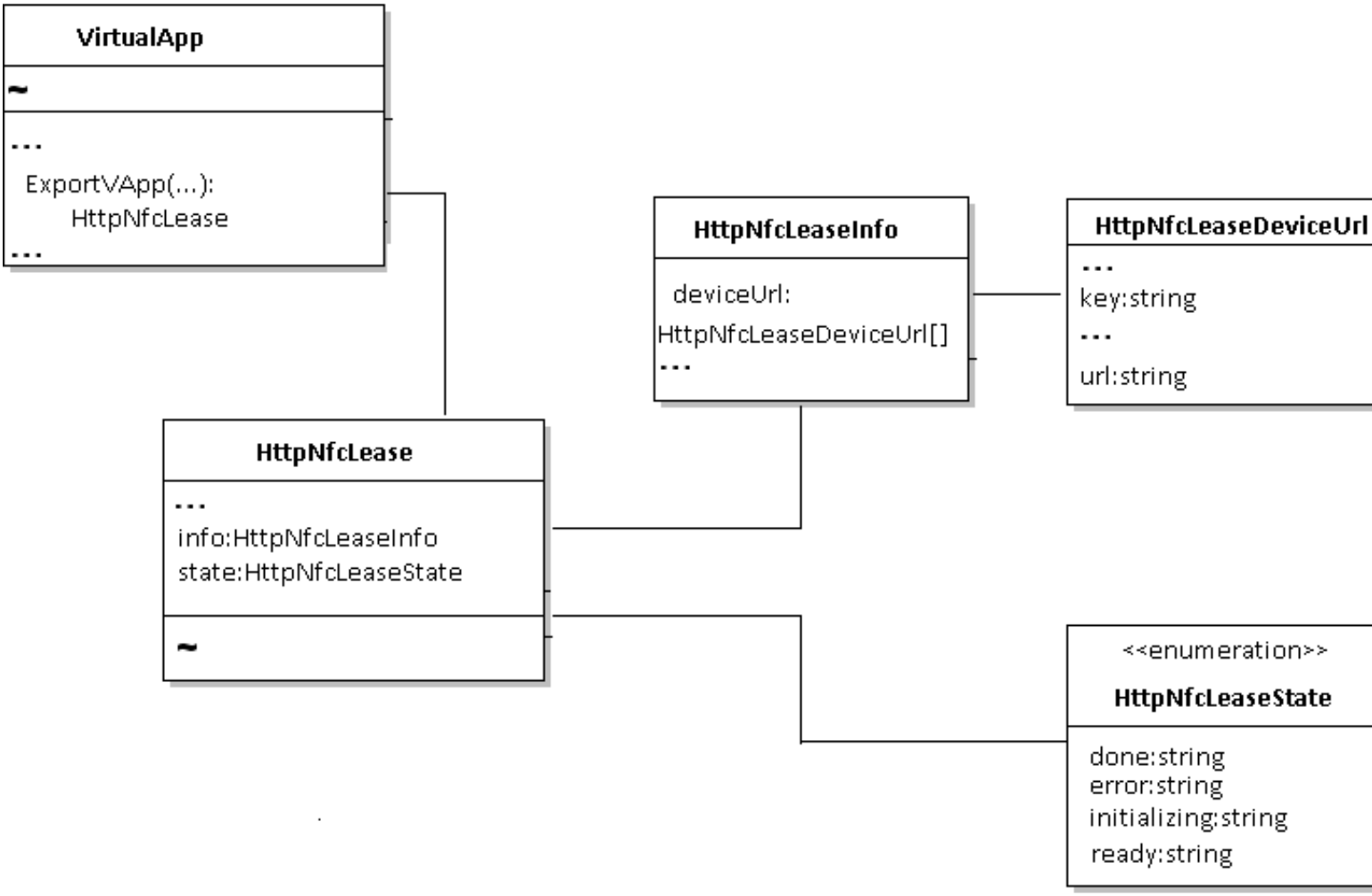
The next two sections deal with the `VirtualApp` and `OvfManager` data structures.

VirtualApp Data Structures

The `VirtualApp` managed object contains the `ExportVApp` method, which returns an `HttpNfcLease`. The `HttpNfcLease` contains the info and state properties, where info is of type `HttpNfcLeaseInfo` and state is of type `HttpNfcLeaseState`. The `HttpNfcLeaseInfo` data object has several properties, one of which is the `deviceUrl` of type `HttpNfcLeaseDeviceUrl[]`. The `HttpNfcLeaseState` has four different states—`done`, `error`, `initializing` and `ready`.

The following class diagram shows the UML representation of the data structures used in the VirtualApp API.

Figure 609: VirtualApp Class Diagram



The VirtualApp API data structures are the following:

- `VirtualApp`—A managed object that is a collection of virtual machines (and potentially other VirtualApp containers) that are operated and monitored as a unit.
- `HttpNfcLease`—A managed object returned when you call `VirtualApp.ExportVApp`. It represents a lease on the virtual application. While you hold the lease, you block the operations that alter the state of the virtual application.
- `HttpNfcLeaseInfo`—A data object that holds information about the lease, such as the virtual application covered by the lease, and the device URLs for up/downloading images.
- `HttpNfcLeaseState`—An enumeration that is a list of possible states of a lease.

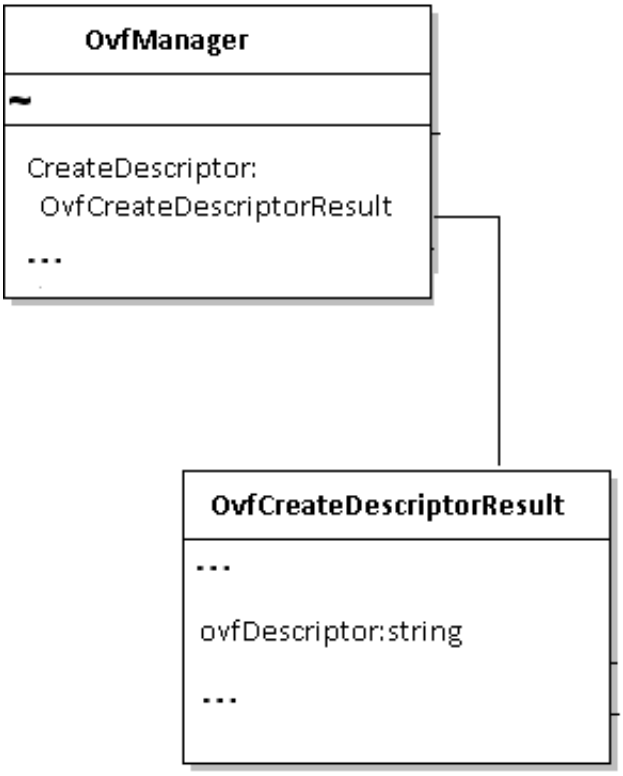
`HttpNfcLeaseDeviceUrl`—A data object that provides a mapping from logical device IDs to upload/download URLs.

OvfManager Data Structures

The OvfManager managed object has a `CreateDescriptor` method that returns an `OvfCreateDescriptorResult`. The `OvfCreateDescriptorResult` has the `ovfDescriptor` string.

Figure 610: OvfManager Class Diagram shows the UML representation of the data structures used in the OvfManager API.

Figure 610: OvfManager Class Diagram



The `OvfManager` data structures are the following:

- `OvfManager`—A managed object that provides a service interface to parse and generate OVF descriptors.
- `OvfCreateDescriptorResult`—A data object that contains the result of creating the OVF descriptor for the virtual application.

Example of Generating an OVF Package

In summary, the steps in generating an OVF package are the following:

1. Get the managed object reference to the `VirtualApp` object. Call the `ExportVApp` method, which returns an `HttpNfcLease` data object. Wait for the state of the lease to turn to `READY`. Get the list of device URLs from the lease and store them in an array.
2. For each of the URLs in the list of device URLs, download the images from that URL to the client.
3. Save the image to the OVF package (directory/folder). Create an `OvfFile` object using the `deviceID`, absolute path of the downloaded image, and the size of the image on the local disk.
4. Call the `OvfManager.CreateDescriptor` method by passing the managed object reference to the `VirtualApp` and the `OvfFile` object wrapped in an `OvfCreateDescriptorParams` object. This method returns `OvfCreateDescriptorResult`, which contains the file descriptor. Write the file descriptor to a file with the file extension `.ovf`. Add the `.ovf` file to the OVF package.

   The following is an example of how to generate an OVF package. The example assumes a more complex scenario: downloading more than one image from more than one device URL. The example is based on the `OVFManagerExportVAAP.java` sample, which is located in the `SDK/vsphere-ws/java/JAXWS/samples/com/vmware/vapp/` directory.

   You can use the `ExportVM` method instead of the `ExportVapp` method when exporting a `VirtualMachine`.

```
package com.vmware.vapp;

import java.io.*;
import java.net.URL;
import java.util.*;

...
  /** 1. Get the MOR of the VirtualApp.
  ManagedObjectReference vAppMoRef = getVAPPByName(vApp);
...
  /**    Call the ExportVApp method, which returns an HttpNfcLease data object. */
  ManagedObjectReference httpNfcLease = vimPort.exportVApp(vAppMoRef);

...
  /**    Wait for the state of the lease to turn to READY. */
  Object[] result = waitForValues.wait(httpNfcLease,
                                       new String[]{"state"},
                                       new String[]{"state"},
                                       new Object[][]{new Object[]{
                                          HttpNfcLeaseState.READY,
                                          HttpNfcLeaseState.ERROR}});
  if (result[0].equals(HttpNfcLeaseState.READY)) {

...
  /**    Get the list of device URLs from the lease. */
     List<HttpNfcLeaseDeviceUrl> deviceUrlArr = httpNfcLeaseInfo.getDeviceUrl();
     if (deviceUrlArr != null) {
...
  /** 2. For each of the URLs in the list of device URLs,
   *     download the images from that URL to the client. */
        for (int i = 0; i < deviceUrlArr.size(); i++) {
           String deviceId = deviceUrlArr.get(i).getKey();
           String deviceUrlStr = deviceUrlArr.get(i).getUrl();
           String absoluteFile = deviceUrlStr.substring(deviceUrlStr.lastIndexOf("/") + 1);

  /** 3. Save the image to the OVF package (directory/folder). Create an OvfFile object using
   * the deviceID, absolute path of the downloaded image, and the size of the image on the
   * local disk.
   */
           long writtenSize = writeVMDKFile(absoluteFile, deviceUrlStr.replace("*", host));
           OvfFile ovfFile = new OvfFile();
           ovfFile.setPath(absoluteFile);
           ovfFile.setDeviceId(deviceId);
           ovfFile.setSize(writtenSize);
           ovfFiles.add(ovfFile);
        }

  /** 4. Call the OvfManager.CreateDescriptor method by passing the managed object reference
   * to the VirtualApp and the OvfFile object wrapped in an OvfCreateDescriptorParams object.
   * This method returns OvfCreateDescriptorResult, which contains the file descriptor.
   * Write the file descriptor to a file with the file extension .ovf. Add the .ovf file to
   * the OVF package.
   */
        ovfCreateDescriptorParams.getOvfFiles().addAll(ovfFiles);
        OvfCreateDescriptorResult ovfCreateDescriptorResult =
              vimPort.createDescriptor( serviceContent.getOvfManager(),
                                        vAppMoRef,
```

```
                                        ovfCreateDescriptorParams);

        String outOVF = localpath + "/" + vApp + ".ovf";
        File outFile = new File(outOVF);
        FileWriter out = new FileWriter(outFile);

        out.write(ovfCreateDescriptorResult.getOvfDescriptor());
        out.close();
```

Importing an OVF Package

To import the virtual application OVF template, you follow a few basic steps. The steps are the same for an OVF package that contains a single virtual machines or an OVF package that contains a more complex virtual application.

1. Parse the OVF descriptor by calling `OvfManager.parseDescriptor`.
2. Validate the target ESX host by calling `OvfManager.validateHost`.
3. Create the `VirtualAppImportSpec` by calling `OvfManager.createImportSpec`.

   This structure contains all the information needed to create the entities on the vCenter, including children. Clients do not have to read or modify `VirtualAppImportSpec` to perform basic OVF operations.
4. Create the vCenter entities by calling `ResourcePool.importVApp`.

   The method uses a parsed OVF descriptor to create `VirtualApp` and `VirtualMachine` objects in the vSphere environment.

The import process itself consists of two steps:

- The server creates the virtual machines and virtual applications.
  You must wait for the server to create all inventory objects. During object creation, the server monitors the `state` property on the `HttpNfcLease` object returned from the `ImportVApp` call. When the server completes object creation, the server changes the lease to `ready` state and you can begin uploading virtual disk contents. If an error occurs while the server is creating inventory objects, the lease changes to the error state, and the import process is terminated.
- The client application uploads virtual disk contents do an HTTP POST request with the content of the disk to the provided URLs. The disk is in the stream-optimized VMDK format. As an alternative, you can use the OVF tool, available for download at the VMware by Broadcom developer site.
  When all inventory objects have been created and the `HttpNfcLease` has changed to ready state, you can upload disk contents by using the URLs provided in the `info` property of the `HttpNfcLease` object. You must call the `HttpNfcLeaseProgress` method on the lease periodically to keep the lease alive and report progress to the server. Failure to do so causes the lease to time out, terminating the import process.

When you are done uploading disks, complete the lease by calling the `HttpNfcLeaseComplete` method. You can terminate the import process by calling the `HttpNfcLeaseAbort` method.

If the import process fails, is terminated, or times out, all created inventory objects are removed, including all virtual disks.

As of VCF 9.1, `OvfManager` has a new method to coordinate with DRS (distributed resource scheduler) – `createConfigSpecWithDRS`. This method returns an `ImportSpec` (`ConfigSpec`) to be used for initial VM creation in a single cluster, or in multiple clusters managed by the same vCenter. Placement is based on host and datastore recommendations generated by DRS. The new method is upward compatible with `createImportSpec` and will be useful for dynamic deployments.

Virtual Application Life Cycle

You can power a virtual application on or off and perform other lifecycle operations.

Powering a Virtual Application On or Off

You can use the `PowerOnVApp_Task` method to power on a `VirtualApp` object. This method starts the virtual machines or child virtual applications in the order specified in the virtual application configuration.

While a virtual application is starting, all power operations performed on subentities are disabled.

If a virtual machine in a virtual application fails to start, an exception is returned and the power-on sequence terminates. In case of a failure, virtual machines that are already started remain powered on.

You can use the `PowerOffVApp_Task` method to power off a virtual application. This method stops the virtual machines or child virtual applications in the order specified in the `VirtualApp` object configuration if `force` is `false`. If `force` is set to `true`, this method stops all virtual machines (in no specific order and possibly in parallel) regardless of the `VirtualApp` object auto-start configuration.

While a virtual application is stopping, all power operations performed on subentities are disabled.

Unregistering a Virtual Application

You can call the `UnregisterVApp_Task` method to remove a `VirtualApp` object from the inventory without removing any of the component virtual machine files on disk. All high-level information stored with the management server (ESX or vCenter system) is removed, including information about `VirtualApp` object configuration, statistics, permissions, and alarms.

Suspending a Virtual Application

You can call the `SuspendVApp_Task` method to suspend all running virtual machines in a virtual application, including virtual machines running in child virtual application. The virtual machines are suspended in the order that is used for a power off operation, which is the reverse of a power on sequence.

While a virtual application is being suspended, all power operations performed on subentities are disabled. If you attempt to perform a power operation, a `TaskInProgress` error results.

Destroying a Virtual Application

When a `VirtualApp` object is destroyed, all of its virtual machines and any child virtual applications are destroyed.

The `VirtualAppVAppState` type defines the set of states a `VirtualApp` object can be in. The transitory state between started and stopped is modeled explicitly, since the starting or stopping of a virtual application might take minutes to complete.

The life-time of a linked child is determined by the `destroyWithParent` property on the `VAppEntityConfigInfo` data object. If set to `true`, the child is destroyed when the parent virtual application is destroyed. Otherwise, only the link is removed when the virtual application is destroyed.

Resource Management

Underlying all virtual components are the actual physical resources of the host system, such as CPU, RAM, storage, network infrastructure, and so on. vSphere supports sharing of resources on an individual host or across hosts using resource pools. vSphere also supports clusters for failover or load balancing.

Resource Management Objects

Central to resource management for all environments is either a `ComputeResource` or a `ClusterComputeResource` managed object.

- The `ComputeResource` managed object represents the set of resources for a set of virtual machines. A `ComputeResource` is always associated with a root `ResourcePool` object, representing the resources of a single host.
- The `ClusterComputeResource` data object aggregates the compute resources of multiple associated `HostSystem` objects into a single compute resource for use by virtual machines. A `ClusterComputeResource` is associated with a root `ResourcePool` representing the combined resources of the cluster. If you plan on using VMware cluster services such as HA (High Availability), DRS (Distributed Resource Scheduling), or on using EVC (Enhanced vMotion Compatibility), use `ClusterComputeResource`.

  Important:
  HA, DRS, and EVC may require feature licenses. If any clustering functionality does not work properly, check whether you have licenses for it.
- The `ResourcePool` managed object represents a set of physical resources of a single host, a subset of a host's resources, or resources spanning multiple hosts. Resource pools can be subdivided by creating child resource pools. Only virtual machines associated with a resource pool can be powered on.

Introduction to Resource Management

An ESX host allocates to each virtual machine a portion of the underlying hardware resources based on several factors:

- Total available resources for the ESX host, resource pool, or cluster to which the virtual machine belongs.
- Number of virtual machines powered on and resource usage by those virtual machines.
- Overhead required to manage the virtualization.
- Constraints defined by the user.

Resource management allows you to dynamically allocate resources to virtual machines so that you can more efficiently use available capacity. You can change resource allocation in the following ways.

- Specify resource allocation for individual virtual machines. See Configuring Resource Allocation Constraints for Virtual Machines.
- Create a hierarchy of resource pools and add the virtual machine to a resource pool with characteristics appropriate for its use. See Resource Pool Hierarchies.
- Add hosts and virtual machines to a cluster so you can take advantage of VMware DRS for recommendations or automatic resource redistribution. See Creating and Configuring Clusters.

Resource Allocation

When you create a virtual machine, you always specify the resource pool that the virtual machine can draw resources from and optionally a host on which the virtual machine should run. You can access the resource pool as follows:

- Standalone host – When you call `Folder.AddStandaloneHost_Task`, the call returns a `Task` object that contains the `ComputeResource`. The `ComputeResource.resourcePool` property is the root resource pool associated with the compute resource (and with the host).
- Cluster – When you call `Folder.CreateClusterEx`, the method returns a managed object reference to a `ClusterComputeResource` instance. Because `ClusterComputeResource` inherits all properties of `ComputeResource`, you can access the root resource pool through the `ClusterComputeResource.resourcePool` property.

Resource Pool Hierarchies

Resource pool hierarchies allow detailed control over which virtual machines are allowed how many resources, by dividing resources between children at each level of the hierarchy. Child resource pools can be configured with reservations and limits, similar to virtual machines. Child resource pools can also be configured with shares settings, which take effect in resource contention situations.

For more information about how reservations and limits apply to resource pool hierarchies, see Configuring Reservation and Limit for Resource Pools. For more information about how shares apply to resource pool hierarchies, see Configuring Priority Shares for Resource Pools.

Cluster Overview

vSphere supports grouping ESX hosts that are managed by the same vCenter system into clusters. Clusters take advantage of features such as VMware DRS and VMware HA.

- VMware HA (VMware High Availability) fails over virtual machines from one host in a cluster to another host, in the event of host failure.
- VMware DRS (VMware Distributed Resource Scheduler) provides dynamic redistribution of resources. DRS also includes support for Distributed Power Management (DPM), which makes recommendations or decisions to power off hosts and power them on again as needed, to save energy.

  You can set up VMware DRS to automatically migrate virtual machines, or to display recommendations if resources are not used efficiently across the cluster.

See Creating and Configuring Clusters and Managing DRS Clusters.

Creating and Configuring Resource Pools

A root resource pool is associated with each `ComputeResource` and with each `ClusterComputeResource`.

You can create a hierarchy of resource pools by calling the `ResourcePool.CreateResourcePool` method and passing in a `ResourceConfigSpec` argument. The `ResourceConfigSpec.cpuAllocation` and `ResourceConfigSpec.memoryAllocation` properties point to `ResourceAllocationInfo` objects that allow you to specify the following information.

- `reservation` – Amount of CPU or memory that is guaranteed available to virtual machines within the resource pool. Reserved resources are not wasted if they are not used. If the utilization is less than the reservation, the resources can be borrowed by virtual machines running within other resource pools.
- `expandableReservation` – In a resource pool with an expandable reservation, the reservation on a resource pool can expand beyond the specified value, if the parent resource pool has unreserved resources. A non-expandable reservation is called a fixed reservation. See Understanding Expandable Reservations.
- `limit` – Upper limit for CPU or memory resources assigned to this resource pool. The resource pool does not allocate more resources to its children, even if resources are available through its parent. This property is typically used to ensure consistent performance by isolating other resource pools from the effects of the running virtual machines within this pool. Set this property to -1 to indicate no fixed upper limit on resource usage.
- `shares` – Relative metric for allocating memory or processing capacity among multiple resource pools in resource contention situations. The shares value indicates resource priority relative to the shares values of sibling resource pools or virtual machines. The `SharesInfo` data object has two properties, `level` and `shares`, that allow you to specify resource allocation.
  - `level` – Choose `high`, `low`, or `normal` to map to a predetermined set of numeric values for shares. See the *API Reference Guide* for the numbers for CPU, memory, and disk shares. Set this property to `custom` to specify an explicit number of shares instead.
  - `shares` – Allows you to specify a custom value for the number of shares you want to allocate to the resource pool. This property is ignored unless the `level` is set to `custom`.

To change the configuration, call the `ResourcePool.UpdateConfig` or `ResourcePool.UpdateChildResourceConfiguration` method and pass in a ResourceConfigSpec that contains values for all fields you want to update.

Configuring Reservation and Limit for Resource Pools

Resource pools are a tool to aggregate physical host resources and channel the aggregated resources to individual virtual machines. You use resource pools to group virtual machines, either to isolate portions of aggregated resources or to prioritize resource allocations between groups of virtual machines.

Resource pools, like individual virtual machines, can be configured with boundaries for resource allocation. Resource pools can also be configured with priority shares, although shares settings act somewhat differently for resource pools than for virtual machines. See Understanding Fixed Shares and Understanding Scalable Shares.

You configure a `reservation` value for a resource pool to help protect its virtual machines from the negative impact of virtual machines in other pools. If you configure a fixed reservation for a resource pool, the host will always make at least that much of the resource available for the resource pool to distribute among its children. See Understanding Fixed Reservations.

> **Note:**
> If a resource pool's virtual machines are not using the resource pool's entire reserved amount of a resource, the host may allocate the unused amount to virtual machines in other pools, on a temporary basis. The host will return the borrowed resource to the resource pool that reserved it, whenever needed to start virtual machines within that resource pool.

You configure a `limit` value for a resource pool to help prevent its virtual machines from negatively impacting performance of virtual machines in other resource pools. The host will never allocate more than that quantity of the resource to any or all of the virtual machines within the pool, whether immediate children or children of a nested pool. After all of the resource is allocated, a resource shortage might cause virtual machines within the pool to fail to start or fail to progress, but it has no effect on virtual machines outside the pool.

If you configure expandable reservations for a resource pool and its siblings, and you configure no reserved amounts for any of the siblings, then they share the parent's `reservation` amount on a 'first come first serve' basis. If any siblings are configured for fixed reservations, those siblings are guaranteed their fixed reservations but no more, while the

remainder of the parent's reservation is available to share among the pools configured for expandable reservations. See Understanding Expandable Reservations.

Understanding Fixed Reservations

A fixed reservation for a resource pool provides a way to guarantee that all virtual machines running simultaneously within the pool have access to a minimum quantity of the resource. The pool's reservation should be large enough to allocate among the virtual machines such that each one has its configured minimum required to start up.

To configure fixed reservations, set the `epandableReservation` property to `false` in `ResourceConfigSpec.ResourceAllocationInfo`. Set the pool's `reservation` amount to an aggregate value that is sufficient to support the maximum number of virtual machines that need to run concurrently within the resource pool. If the pool has nested resource pools that are also configured for fixed reservations, you should determine their `reservation` amounts in the same way, recursively.

When you configure a resource pool hierarchy for fixed reservations, consider these guidelines:

- A resource pool that contains only virtual machines should be configured with a reservation amount at least as great as the sum of the reservations of its children, or as many of them as need to run concurrently. Otherwise, one or more virtual machines will fail to get its minimum resource allocation and the host will not start it.
- A resource pool that contains only nested resource pools configured for fixed reservations must be configured with a reservation amount at least as great as the sum of the reservations of its children. This ensures that its child resource pools can fulfill their obligations to their own children, according to their configured `reservation` amounts. If you cannot configure the parent pool with an adequate amount of the resource, consider configuring the nested resource pools for expandable reservations instead.
- A resource pool that contains both virtual machines and nested resource pools is not a best practice, because it can complicate configuration management. If you configure a resource pool in this way, you should first make sure the virtual machines can receive their own reservation amount, and then configure reservations for the nested pools from the remainder of the parent's reservation after subtracting the virtual machine reservations.

Before you create new child resource pools, check available resources in the parent pool. The `ResourcePool.runtimeInfo` property is a `ResourcePoolRuntimeInfo` data object. The `ResourcePoolRuntimeInfo.cpu` and `ResourcePoolRuntimeInfo.memory` properties are `ResourcePoolResourceUsage` objects with resource usage information, including an `unreservedForPool` property. If the parent pool does not have enough available resources, reconfigure the reservation values of child pools before adding the new pool.

Understanding Expandable Reservations

Expandable reservations enable dynamic allocation of resources to meet the minimum requirements of virtual machines within a number of resource pools. When you set the `expandableReservation` property on sibling resource pools, you enable them to act as a single resource pool for the purpose of providing the minimum of resource that virtual machines need to power on. When a virtual machine starts in any one of the sibling resource pools, it draws its `reservation` amount from the parent's `reservation` amount, rather than from its own resource pool.

If the parent resource pool is also expandable, it can draw resources in turn from its parent, and the sequence can continue in this way until it ends with a parent resource pool that has a fixed reservation.

To configure expandable reservations, set the `expandableReservation` property to `true` in the `ResourceConfigSpec.cpuAllocation` or `ResourceConfigSpec.memoryAllocation` of sibling resource pools to `true`. For maximum flexibility, set the `reservation` amount of each sibling pool `0`. When sibling pools do not reserve any resources, any virtual machine that starts in the sibling pools will take its reservation amount from the parent pool's reservation.

If a sibling pool contains critical virtual machines whose reservations need to be prioritized, set the pool's reservation to the sum of the reservations of the prioritized virtual machines. That reservation amount is saved for the resource pool's virtual machines, and is unavailable for siblings to use.

To illustrate how expandable reservations work, consider the following examples.

Expandable Reservation Example 1

Assume an administrator manages a parent pool P1, and defines two child resource pools, C1 and C2, for two different groups of users. C1 and C2 are self-serve resource pools, allowing users to configure their own virtual machines. The administrator does not know in advance exactly what resources the users will need, so the administrator wants the resource pool configuration to be flexible.

Without expandable reservations, the administrator needs to explicitly allocate fixed amounts of resources to C1 and C2. Such specific allocations can be inflexible, especially in deep resource pool hierarchies, and can complicate setting reservations for C1 and C2. By making the reservations for C1 and C2 expandable, the administrator allows users to more flexibly share and inherit the common reservation for pool P1.

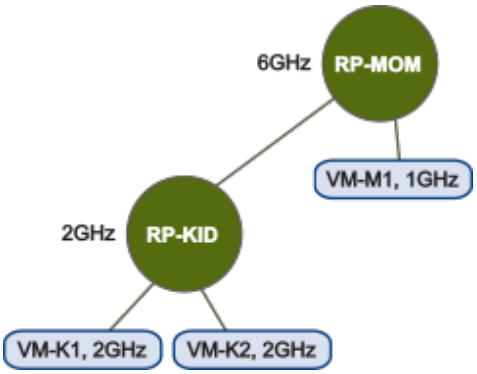
Expandable reservations cause a loss of resource pool isolation in the context of admission control. For example, if C1 and C2 have their reservation amounts set to `0`, then virtual machines in C1 might use all of P1's memory reservation, so that no memory is available to start virtual machines in C2.

Expandable Reservation Example 2

Assume a parent resource pool RP-MOM has a reservation of 6GHz and one running virtual machine VM-M1 that reserves 1GHz. RP-MOM also has a child resource pool RP-KID with a reservation of 2GHz and with **Expandable Reservation** selected. RP-KID contains two virtual machines, VM-K1 and VM-K2, with reservations of 2GHz each.

When a user powers on VM-K1, it can reserve the resources it needs directly from RP-KID (which has 2GHz). When the user tries to power on VM-K2, RP-KID has already allocated its 2GHz reservation to VM-K1, but it has **Expandable Reservation** configured, so it tries to borrow resources from RP-MOM's reservation. RP-MOM has 6GHz minus 1GHz (reserved by VM-M1) minus 2GHz (reserved by RP-KID), which leaves 3GHz of RP-MOM's reservation that is not reserved by other resource pools or virtual machines in RP-MOM. With 3GHz available, VM-K2 is able to power on.

Figure 611: Admission Control with Expandable Resource Pools, Scenario 1

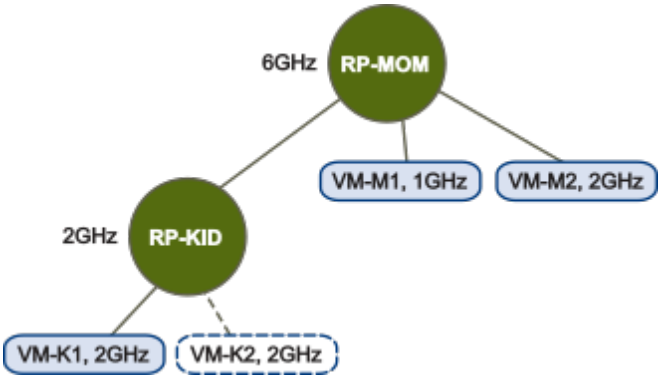


Expandable Reservation Example 3

Assume a parent resource pool RP-MOM has a reservation of 6GHz and two running virtual machines, VM-M1 that reserves 1GHz, and VM-M2 that reserves 2GHz. RP-MOM also has a child resource pool RP-KID with a reservation of 2GHz and with **Expandable Reservation** selected. RP-KID contains two virtual machines, VM-K1 and VM-K2, with reservations of 2GHz each.

When a user powers on VM-K1, it can reserve the resources it needs directly from RP-KID (which has 2GHz). When the user tries to power on VM-K2, RP-KID has already allocated its 2GHz reservation to VM-K1, but it has **Expandable Reservation** configured, so it tries to borrow resources from RP-MOM's reservation. RP-MOM has 6GHz minus 3GHz (reserved by VM-M1 and VM-M2) minus 2GHz (reserved by RP-KID), which leaves only 1GHz of RP-MOM's reservation that is not reserved by other resource pools or virtual machines in RP-MOM. Since VM-K2 requires 2GHz to pass admission control, VM-K2 is not able to power on.

Figure 612: Admission Control with Expandable Resource Pools, Scenario 2



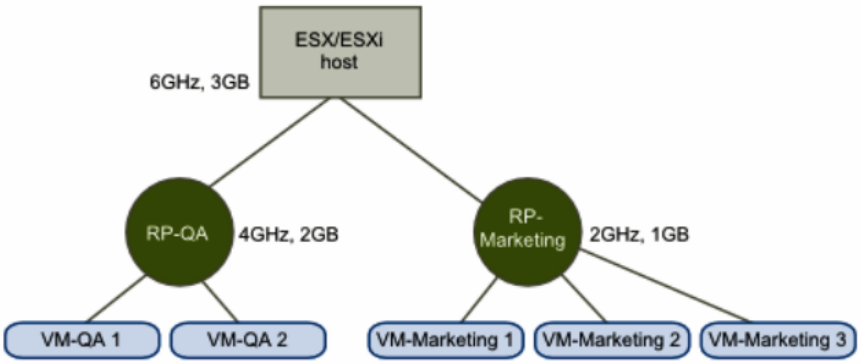
Configuring Priority Shares for Resource Pools

In cases of resource contention, data center hosts need a way to arbitrate between competing virtual machines. You can configure the shares settings of a virtual machine to set its priority relative to its siblings. You can configure the shares settings of a resource pool to affect the priorities of all virtual machines within the resource pool, as a group.

The following illustration shows a standalone host that has several virtual machines. The marketing department uses three of the virtual machines and the QA department uses two virtual machines. Because the QA department needs larger amounts of CPU and memory, the administrator creates one resource pool for each group. The administrator sets CPU Shares to High for the QA department pool and to Normal for the Marketing department pool so that the QA department users can run automated tests. The second resource pool with fewer CPU and memory resources is sufficient for the lighter load of the marketing staff.

Whenever the QA department is not fully using its allocation, the marketing department can use the available resources. When there is resource contention because running virtual machines demand more resources than are available from the host, the resource shares configuration guides arbitration between the competing virtual machines.

Figure 613: Allocating Resources to Resource Pools



You have two options for priority shares configuration that cause the priorities to be applied in different ways. The fixed shares option limits the adverse performance impact of resource contention to a single resource pool and all its descendants. The scalable shares option distributes the performance impact across all resource pools, in proportion to their priority levels.

The chief benefit of configuring fixed shares for a parent resource pool is the predictability of performance for virtual machines within its child resource pools. You can know in advance that its virtual machines can demand a fixed fraction of a virtual resource available from a host or cluster. The drawback of configuring fixed shares is that adding virtual machines to a resource pool impacts the performance of all virtual machines in the same pool and its descendants because the resource pool is not entitled to allocate additional resources.

The chief benefit of configuring scalable shares for a parent resource pool is that resource allocation during contention is adjusted at run time to achieve a fair allocation to virtual machines beyond the boundary of a single resource pool. In effect, the child pool's resource entitlement expands to accommodate more virtual machines as they are added to the

pool. The drawback of configuring scalable shares is that a child resource pool cannot isolate its virtual machines from the demands of virtual machines in other pools that draw from the same scalable parent resource pool.

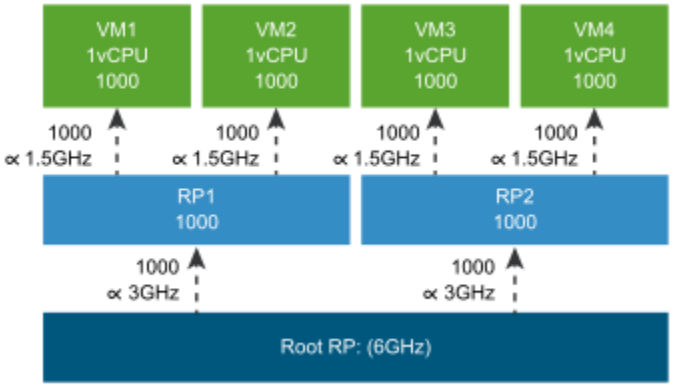
Understanding Fixed Shares

When a parent resource pool is not configured to be scalable, the shares configured for its child resource pools provide a way to prioritize allocation of fixed fractions of the parent resource pool to its children. This serves to isolate users of sibling resource pools so that virtual machines in one child resource pool cannot impact the performance of virtual machines in other child resource pools.

Suppose for example a cluster has 3 physical cores running at 2GHz, for a total of 6GHz. The root resource pool has a total of 6GHz of virtual CPU capacity to divide between its children. If there is no resource contention, all running virtual machines can be allocated their configured amounts of CPU. If there is contention for CPU resources, the 6GHz is divided between child resource pools according to the CPU shares configured for them.

This fictional data center has two child resource pools, each supporting the users in a different division of a business. Division 1 and Division 2 start out about the same size, so the IT department configures RP1 and RP2 with equal values for custom shares, and an equal number of virtual machines.

Figure 614: Example Resource Pool Configuration with Custom Shares



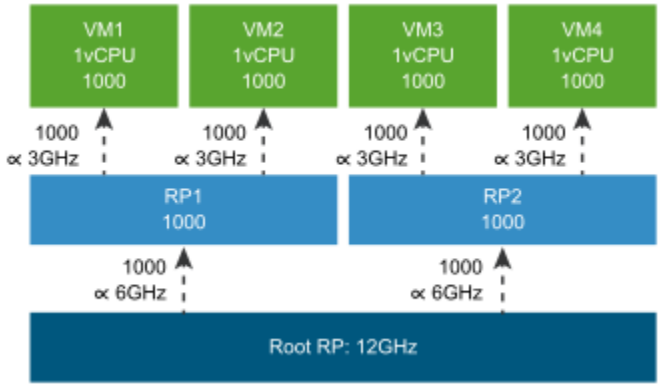
All child resource pools and virtual machines are configured with custom settings of 1000 shares each. Because the two resource pools are configured with the same number of shares, they can allocate the same amount of compute resource to their virtual machines in cases of resource contention. Likewise, all virtual machines are entitled to draw the same fraction of compute resource as their siblings.

> **Note:**
> For simplicity of illustration, these examples deal only with CPU shares. A real configuration would include both CPU and memory resources, which act independently in regard to calculating the absolute resources available to a running virtual machine.

In effect, a non-scalable resource pool guarantees that each of its children gets a fixed fraction of its resources. However, the amount allocated to each child is fixed only in relation to the quantity of resources controlled by the resource pool itself. If a root resource pool gains additional resources due to a hardware upgrade, all its children gain resource entitlements in proportion to their configured shares.

For example, suppose the cluster has been upgraded from 3 to 6 physical cores running at 2GHz, for a total of 12GHz of pooled physical capacity. The root resource pool now has a total of 12GHz of virtual CPU capacity to divide between its children. If there is contention for CPU resources, the 12GHz is divided between child resource pools according to the CPU shares configured for them.

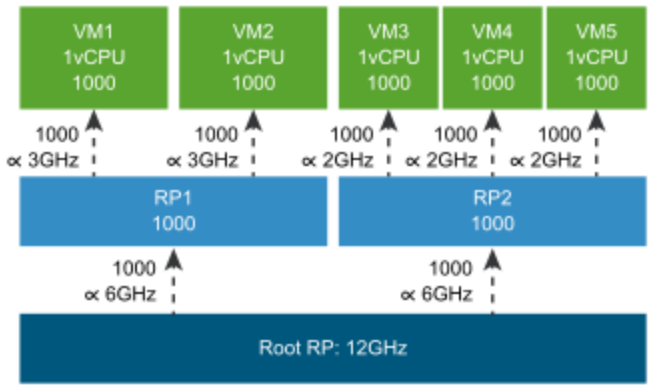
Figure 615: Increased Physical Capcity Provides Increased Virtual Capacity to All Virtual Machines



After the upgrade, each virtual machine is entitled to twice its previous resource allocation, in absolute terms. However, if another child, whether resource pool or virtual machine, is added to a parent pool, previously existing children find their resource allocation reduced, as the pool's resources are shared between more children.

Suppose Division 2 hires some new users, and IT adds an extra virtual machine to RP2 for their use.

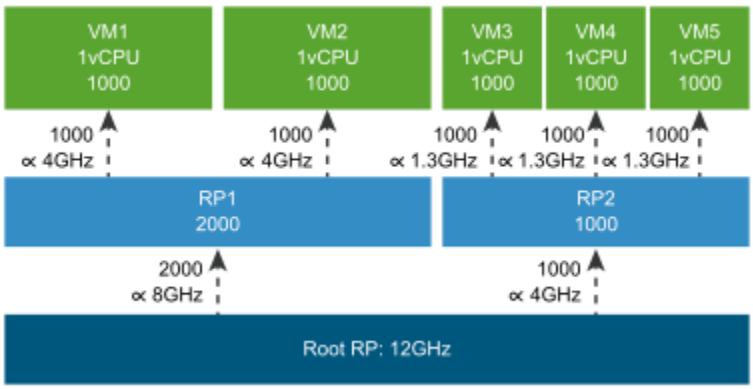
Figure 616: Dilution of Fixed Shares From Adding a Virtual Machine to a Resource Pool



Where previously all virtual machines in the business had the same priority for allocation of scarce compute resource, increasing the virtual machine load in RP2 means that more virtual machines contend for the same quantity of its resource. Consequently, each virtual machine in RP2 might see its performance reduced when all virtual machines are running at full capacity.

To continue from the previous example, some users in Division 1 might be concerned about performance because they plan to hire more employees in the future. Suppose they persuade the IT department that their virtual machines should be in a higher priority resource pool than others, so they will have access to more compute resource if needed. The IT department configures different custom shares values for RP1 and RP2, so that RP1 has twice the priority of RP2.

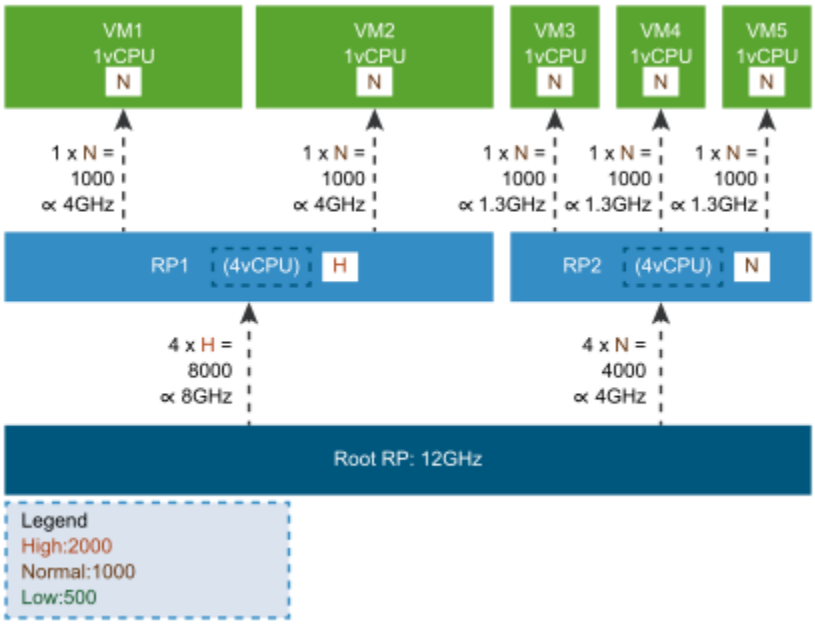
Figure 617: Effects of Reprioritizing a Resource Pool in a Fixed Shares Configurotion



The child resource pool RP1 has 2000 shares configured, and RP2 has 1000 shares configured, so RP1 gets 2/3 of the physical CPU resources to share among its children, and RP2 gets 1/3 of the physical CPU resources to share among its children. You can calculate the amount of compute resource available to RP1 for its children as `12 x 2000 / (2000+1000) = 8`GHz, and the amount of compute resource available to RP2 for its children as `12 x 1000 / (2000+1000) = 4` virtual GHz. The children within each resource pool have identical shares settings, so they share their own pool's resources equally.

Now suppose a new CIO joins the business and decides that from now on priority shares should be specified as levels rather than custom values. The IT department reconfigures all the shares settings to use the enums for shares levels.

Figure 618: Priority Shares Expressed as Levels Instead of Custom Values



When you specify shares levels, you have to make two calculations. First you convert the specified level to a numeric value that corresponds to custom shares settings. This conversion takes into account the number of virtual cores.

> **Note:**
> For memory resources, the conversion takes into account the configured memory size of the virtual machine.

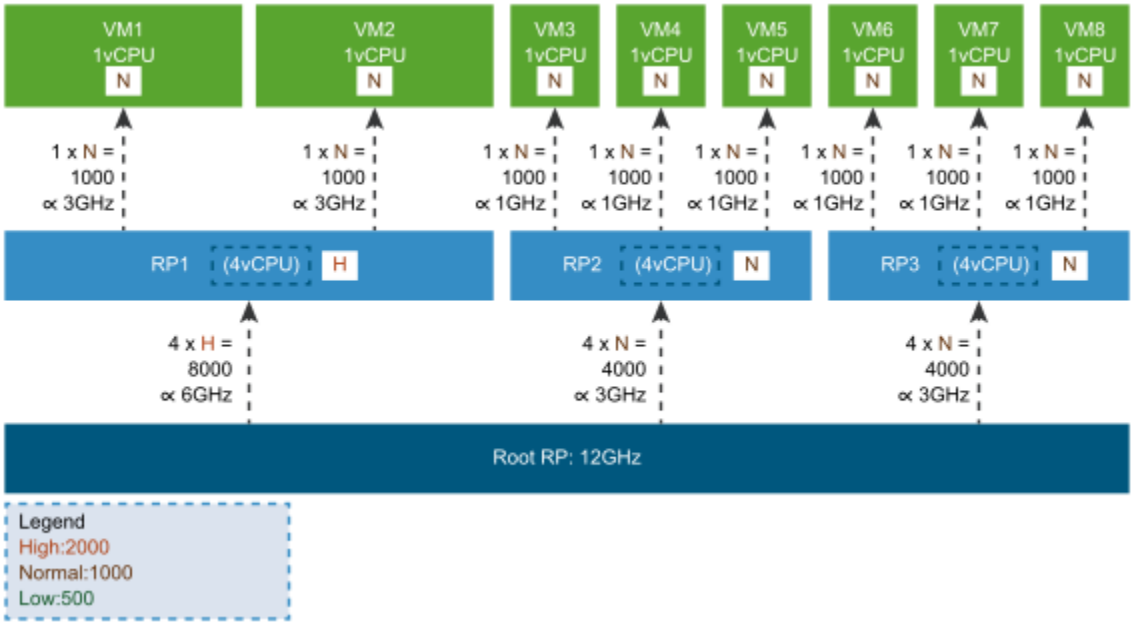
Then you use that numeric value to calculate the proportion of resources to allocate to the child pool, in the same way as with the custom shares settings.

To convert a shares level to a numeric value, multiply the number of vCPUs by a constant that corresponds to the level. For the purpose of this calculation, all resource pools have an implicit size of 4 vCPUs. Use the `VirtualMachine.config.hardware.numCPU` value as the number of vCPUs for virtual machines. The constant for the `high` priority level is 2000, and the constant for the `normal` level is 1000. You can calculate that the numeric shares belonging to RP1 are `4 x 2000 = 8000`. RP2 has a priority level of `normal`, so the numeric shares belonging to RP2 are `4 x 1000 = 4000`. The compute resource that RP1 can allocate is `12 x 8000 / (8000+4000) = 8` GHz, and the compute resource that RP2 can allocate to its children is `12 x 4000 / (8000+4000) = 4` GHz. The new CIO is satisfied that RP1 and RP2 achieve the same result with priority levels as with the previous custom shares values.

> **Note:**
> For memory shares, the constants are different. See the vSphere Web Services API Reference.

But suppose the IT department is asked to support a 3rd division, newly acquired by the business, which is about the same size as the division assigned to RP2. The IT department creates a new child resource pool, RP3, and configures the same number of virtual machines and the same shares levels as RP2. Now the division of resources looks like this:

Figure 619: Resource Dilution from Adding a Resource Pool



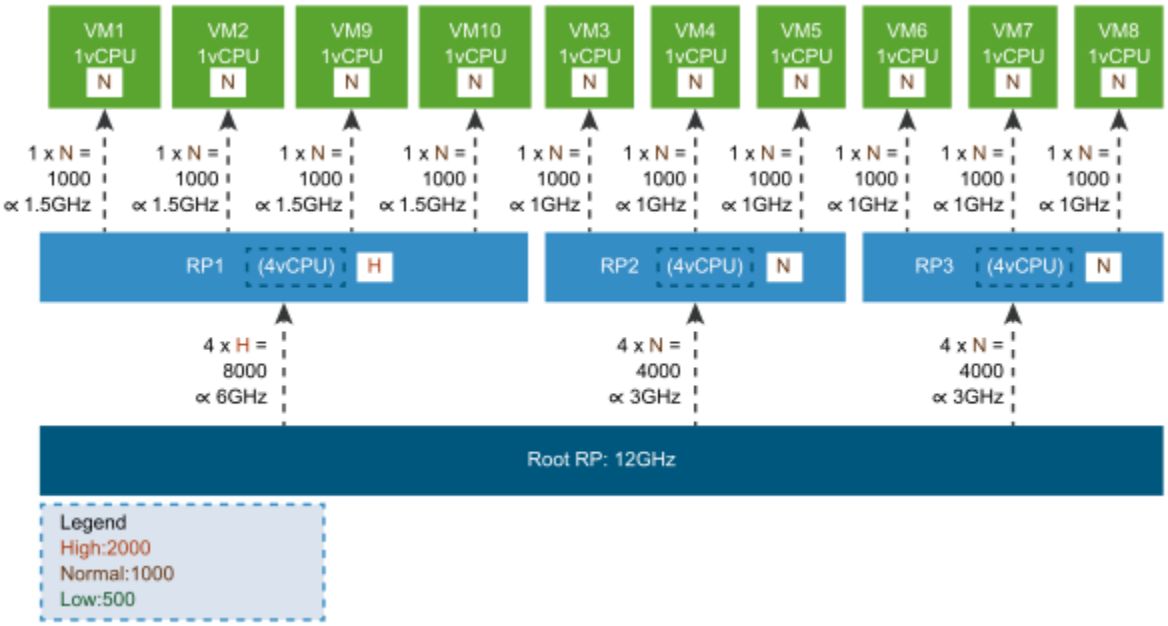
Assume the hardware resources available to the root resource pool have not changed at the time of the acquisition, so the 12GHz of virtual compute resource must be divided into fractions according to the configured shares of the child resource pools. After adding RP3, both RP1 and RP2 get less of the virtual compute resource to divide among their own children.

Virtual machines available to Division 1 now compete for `12 x 8000 / (8000+4000+4000) = 6`GHz in RP1. Virtual machines available to Division 2 compete for `12 x 4000 / (8000+4000+4000) = 3`GHz in RP2. Virtual machines available to Division 3 similarly compete for `12 x 4000 / (8000+4000+4000) = 3`GHz in RP3.

In this example, Division 1 and Division 2 employees might both perceive reduced performance until the IT department's capital budget increases to support the new configuration.

When Division 1 hires a number of new employees, IT adds two more virtual machines to RP1 to accommodate the increased size of Division 1. All four virtual machines in RP1 still benefit from RP1's `high` priority level, but now that RP1's compute resources are spread among a larger number of virtual machines, each of its virtual machines gets a smaller allocation when they are competing for CPU cycles.

Figure 620: Resource Dilution from Adding Virtual Machines to a High Priority Resource Pool

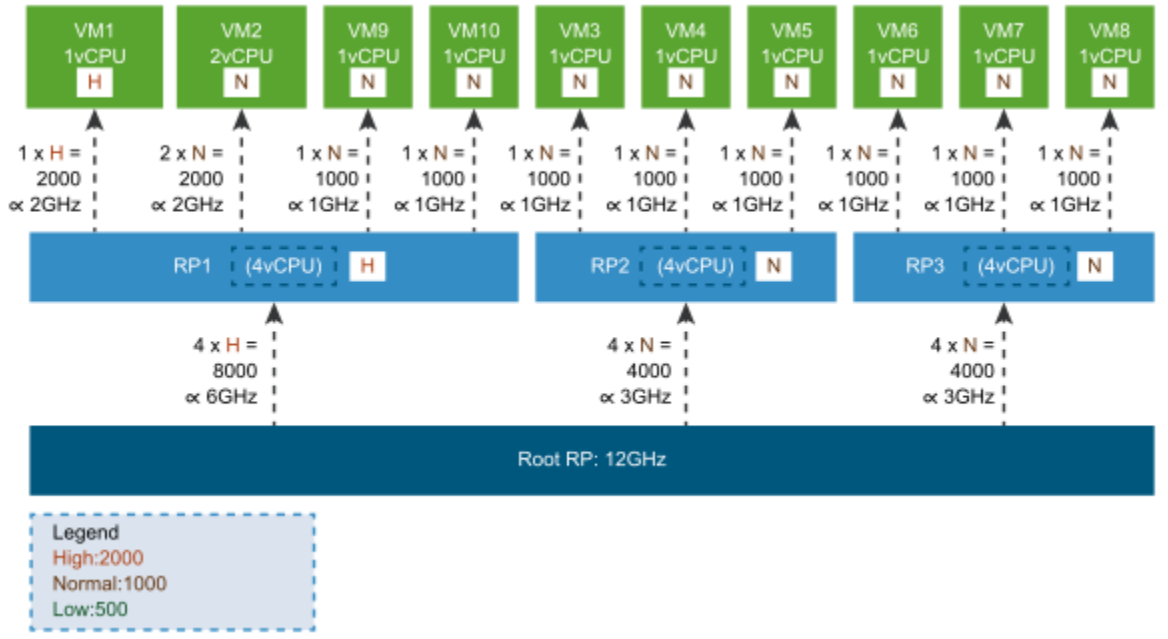


Some virtual machine users who were in Division 1 before the new employees joined are noticing greatly reduced performance. They are only getting 1.5GHz where they used to get 3GHz. When they learn that Division 2 employees are getting 1GHz from their virtual machines in RP2, which has `normal` priority, they might not be happy that they are getting only 1.5GHz from virtual machines in RP1, which has `high` priority.

The disappointed users speak to the IT department, where they learn that this is the way fixed shares for resource pools are intended to work. They isolate Division 1 virtual machine performance from the other divisions, but not from other users of the same resource pool. These Division 1 users learn that there are other ways to protect the performance of high priority virtual machines.

Some Division 1 users try other ways to get more compute resource from RP1's fixed allocation. They request changes to virtual machine configurations, which result in increased priority for some of their virtual machines at the expense of others.

Figure 621: Prioritizing Individual Virtual Machines Within a Resource Pool



After VM1 is reconfigured, you calculate its numeric shares by multiplying the configured `numCPU` (1) by its shares level (`high`), getting `1 x 2000 = 2000`. You calculate its resource allocation by using its numeric shares to prorate the allocation available to its parent resource pool: `6GHz x 2000 / (2x2000+2x1000) = 2GHz`.

You calculate VM2's numeric shares by multiplying the configured `numCPU` (2) by its shares level (`normal`), getting `2 x 1000 = 2000`. You calculate its resource allocation by using its numeric shares to prorate the allocation available to its parent resource pool: `6GHz x 2000 / (2x2000+2x1000) = 2GHz`.

In a similar way, calculate VM9's numeric shares by multiplying its configured `numCPU` (1) by its shares level (`normal`), getting `1 x 1000 = 1000`. You calculate its resource allocation by using its numeric shares to prorate the allocation available to its parent resource pool: `6GHz x 1000 / (2x2000+2x1000) = 1GHz`. VM10's resource configuration and associated calculations are identical to VM9.

When other Division 1 users realize that VM9 and VM10 only get 1GHz of virtual compute resource, they might not be satisfied with this result. These virtual machines are in a `high` priority resource pool, but they get the same resource allocation as Division 2's virtual machines, which are in a `normal` priority resource pool. That doesn't seem fair.

Division 1 users request a meeting with the CIO, who explains that the scalable shares feature is the best solution to their problem.

Understanding Scalable Shares

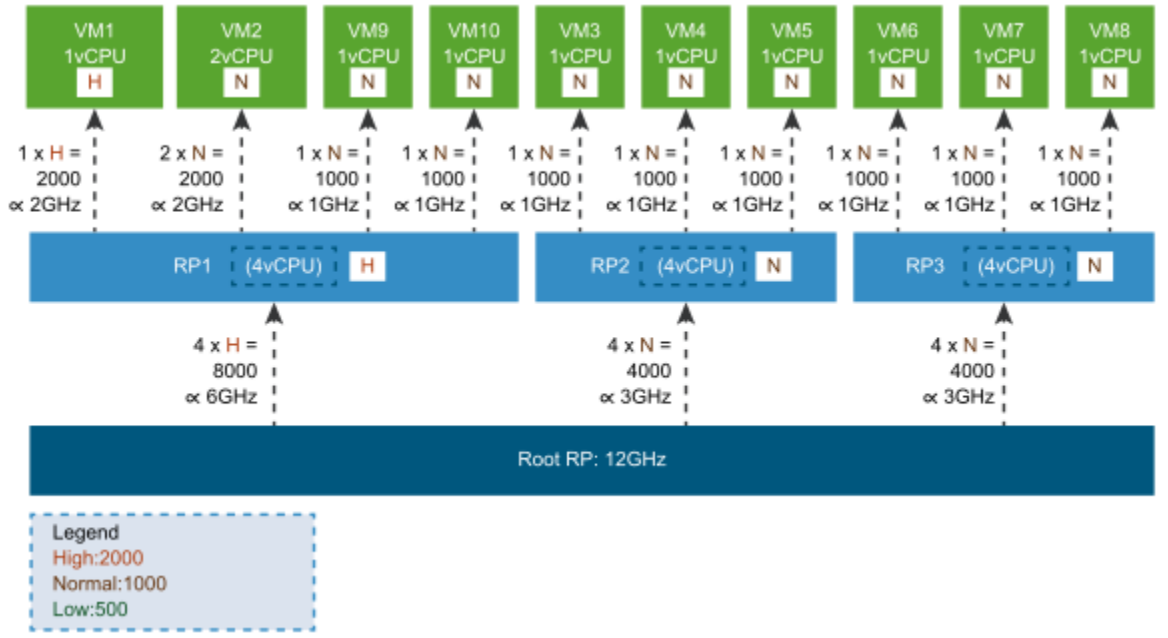
Configuring scalable shares for a parent resource pool means that all its child resource pools are not isolated from each other. This avoids the resource dilution problem within child resource pools, by expanding their resource entitlements when they have more virtual machines running.The effect is to negate resource pool isolation while retaining the

proportions of resource pool priority levels. A `high` priority resource pool is entitled to give twice as much of the available resource to its virtual machines, in comparison with a `normal` priority sibling resource pool, but all virtual machines suffer proportionately when many virtual machines run in the normal priority pool.

To illustrate the use of the scalable shares option, consider how it might apply to a fictional situation in which a data center is configured, with non-scalable shares, to support three divisions of a business. Division 1 is supported by resource pool RP1, which is configured as `high` priority, in an attempt to compensate for the dilution of RP1's resources that resulted from adding new virtual machines to RP1. Some virtual machines in RP1 are individually configured to get higher priority than their siblings.

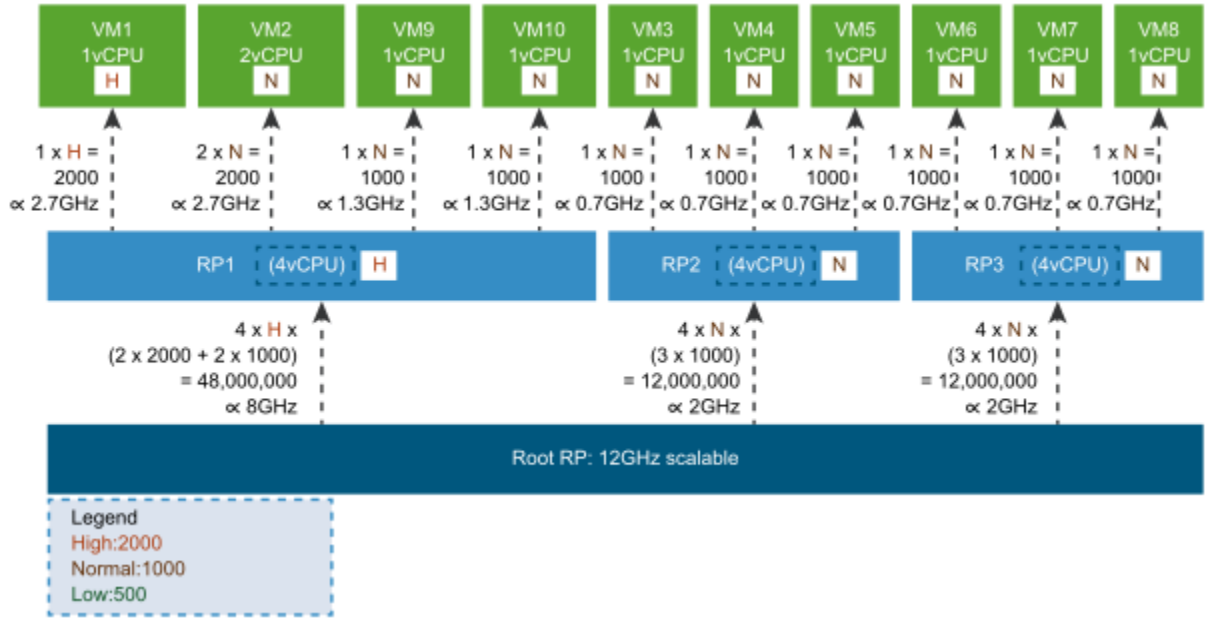
> **Note:**
> For simplicity of illustration, these examples deal only with CPU shares. A real configuration would include both CPU and memory resources, which act independently in regard to calculating the absolute resources available to a running virtual machine.

Figure 622: Example Data Center Configured with Fixed Shares



As a result of resource dilution within RP1 in a fixed shares configuration, some of RP1's virtual machines are entitled to exactly the same resource allocation as RP2's virtual machines, even though RP2 is configured as `normal` priority but RP1 is configured as `high` priority. Some users in Division 1 are disappointed because they expected their virtual machines to receive twice the allotment of RP2's virtual machines. The company's CIO agrees to reconfigure the data center to take advantage of the scalable shares option.

Figure 623: Example Scalable Shares Configuration



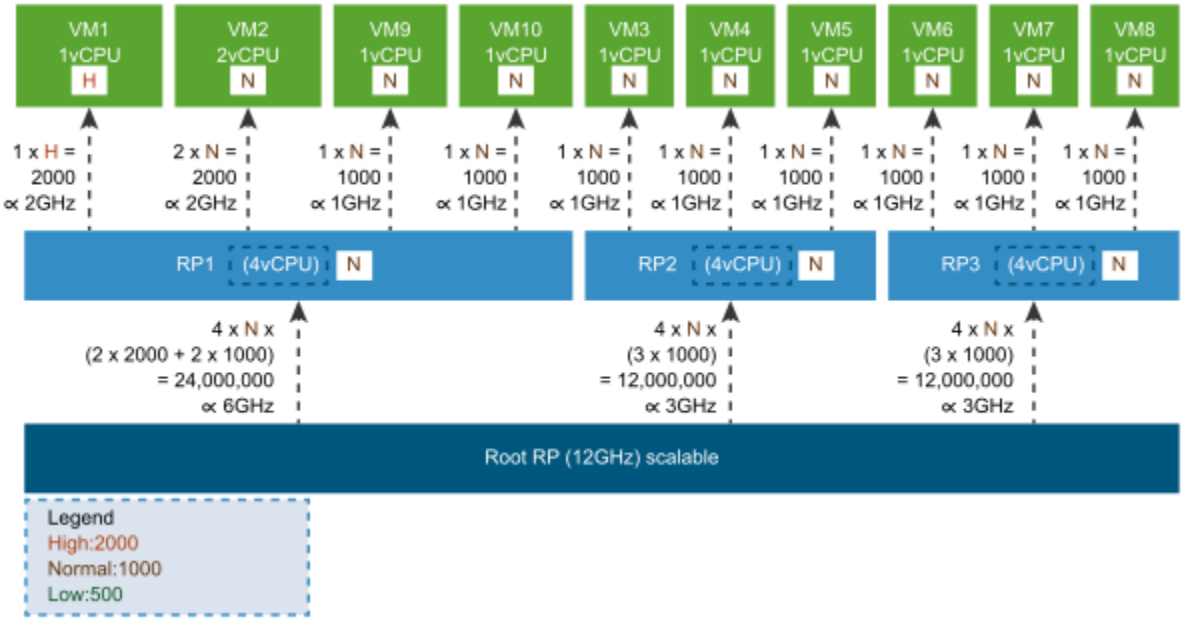
With scalable shares enabled on a parent resource pool, resource entitlements change. Because RP1 has more virtual machines than RP2 or RP3, it is entitled to a larger fraction of the root resource pool's resources during a resource contention situation. This is represented mathematically by multiplying the resource pool's numeric shares by the total of the shares of its children.

Calculating resource pool allocation in a scalable shares configuration is conceptually a six-step recursive process, working from leaf to root of the resource pool tree:

1. Convert the priority levels of its child virtual machines into numeric shares equivalents, in the same way as in a fixed shares configuration: Multiply `VirtualMachine.config.hardware.numCPU` by a constant that depends on the priority level. The constants are `2000`, `1000`, and `500` for `high`, `normal`, and `low` respectively.
2. If the resource pool has child resource pools, convert their priority levels into numeric shares equivalents, recursively.
3. Do the same conversion for the resource pool itself. For the purpose of this conversion, all resource pools are assigned an implicit size of 4 vCPUs.
4. Sum the children's numeric shares and multiply the sum by the parent's numeric shares.
5. Repeat steps 1-4 for all the parent's siblings.
6. Allocate the available resources among all siblings, in proportion to the products from steps 4 and 5.

As a result of the scalable shares configuration, Division 1 users find their virtual machine performance improved, but Divisions 2 and 3 suffer by comparison. To rectify this, the IT department resets the priority levels of RP1, RP2, and RP3 to `normal`. This adjustment leaves virtual machine performance comparable across divisions: `high` priority virtual machines get twice the resource allocation of `normal` priority virtual machines, even across resource pool boundaries.

Figure 624: Scalable Shares Configuration with All Resource Pools Configured at `normal` Level



A best practice when configuring scalable shares is to configure resource pools to the `normal` priority level and apply `high` priority levels or `low` priority levels to specific virtual machines.

Deleting Child Resource Pools

The `ResourcePool.DestroyChildren` method recursively deletes all the child resource pools of a resource pool. The operation takes a single parameter, a reference to the parent `ResourcePool` managed object. Any virtual machines associated with the child resource pool are reassigned to the parent resource pool.

Moving Resource Pools or Virtual Machines Into a Resource Pool

You can move a resource pool and its children within a resource pool hierarchy.

The `ResourcePool.MoveIntoResourcePool` method lets you move virtual machines, virtual applications, or resource pool hierarchies into a new resource pool. You call the method with an array of `ResourcePool` or `VirtualMachine` managed object references that you want to move. The whole resource pool hierarchy, including child resource pools and virtual machines, is moved when you move a resource pool.

Minimum available resources of the immediate children must always be less than or equal to the resources of the immediate parent. The root resource pool cannot be moved.

Introduction to vSphere Clusters

Clusters are useful with VMware DRS (distributed resource scheduler) and VMware HA (high availability). Clusters can be created quickly with VMware HCI. This guide briefly introduces the vSphere interfaces for cluster configuration. Clusters are useful to reduce power use, downtime, and maintenance.

- VMware DRS. *vSphere Resource Management* guide
- VMware HA. *vSphere Availability* guide
- VMware HCI, Hyper-converged infrastructure. In vSphere Client, click **New Cluster**.

VMware DRS

A VMware DRS cluster is a collection of ESX hosts and associated virtual machines with shared resources and a shared management interface. To obtain the benefits of cluster-level resource scheduling you create a DRS cluster.

When you add a host to a DRS cluster, the host's resources become part of the cluster's resources. In addition to aggregation of resources, a DRS cluster supports cluster-wide resource pools and enforces cluster-level resource allocation policies. The following cluster-level resource management capabilities are available.

- Load Balancing. The vCenter monitors distribution and use of CPU and memory resources for all hosts and virtual machines in the cluster. DRS compares these metrics to an ideal resource utilization given the attributes of the cluster's resource pools and virtual machines, the current demand, and the DRS Score, which measures the execution efficiency of the virtual machine. DRS then performs (or recommends) virtual machine migrations. When you first power on a virtual machine in the cluster, DRS attempts to maintain proper load balancing either by placing the virtual machine on an appropriate host or by recommending one.
- Power Management. When the VMware DPM (Distributed Power Management) feature is enabled, DRS compares cluster- and host-level capacity to the demands of the cluster's virtual machines, including recent historical demand. DPM places (or recommends placing) hosts in standby power mode if sufficient excess capacity is found. DPM powers on (or recommends powering on) hosts if capacity is needed. Depending on the resulting host power state recommendations, virtual machines might need to be migrated to and from the hosts.
- Virtual Machine Placement. You can control the placement of virtual machines on hosts within a cluster, by assigning DRS affinity or antiaffinity rules.

See Managing DRS Clusters.

VMware HA

VMware HA supports high availability for virtual machines by pooling the hosts they reside on into a cluster. If one host fails, its virtual machines move to a different host in the cluster.

VMware HA monitors ESX hosts and in the event of host failure, migrates virtual machines to hosts with capacity. When you add new virtual machines to a VMware HA cluster, VMware HA checks whether enough capacity to power on that virtual machine is available on that host or a different host.

See Managing HA Clusters.

VMware HCI

VMware HCI manages the configuration of hyper-converged compute, storage, and network resources for a cluster, as a collection of identically configured ESX hosts. This unified approach simplifies cluster configuration, offering a way to configure all resource types as a group, using a single interface.

VMware HCI clusters are ideal for newly established datacenters. Only conformant hosts are allowed to be a part of the HCI cluster. They must have the same CPU design and contain the minimum number of physical NICs specified in `DvsProfile`. Network settings are wiped to a clean state before configuration. Non-management port groups in standard switches are migrated to a virtual distributed switch.

The `ConfigureHCI_Task` method configures a VMware HCI (hyper converged infrastructure) cluster. The `ExtendHCI_Task` method expands and reconfigures a VMware HCI cluster. Before and after HCI cluster configuration, you can run the `ValidateHCIConfiguration` method to check that the `ClusterComputeResourceHCIConfigSpec` is correct.

The `BatchAddHostsToCluster_Task` method adds more hosts as needed to a VMware HCI cluster, but cannot change cluster configuration.

These configure, extend, and host add methods are composite tasks. That is, they perform multiple operations in a single task. For example, `ConfigureHCI_Task` can:

- Set up cluster services such as DRS, HA, and vMotion
- For an HCI cluster, verify that all hosts have same CPU vendor
- Add new standalone hosts and set state (lockdown)
- Move all hosts into the HCI cluster
- Configure vSAN datastore
- Configure virtual distributed switch and port groups

Automatic remediation of failed hosts is not attempted. The administrator must intervene to correct issues before attempting to place hosts back in the cluster. Remediated hosts must be ready to participate in the vSAN datastore.

Creating and Configuring Clusters

The vSphere Web Services SDK includes objects and methods for all cluster management tasks. Some documentation is available in the *vSphere API Reference*. For additional background and details about the failover and load balancing behavior, see *vSphere Resource Management* and *vSphere Availability*.

In a multi-vCenter cluster, twice as much storage is needed to accomodate a second vCenter. Quota and space-needed calculations are adjusted to account for multiple vCenters.

Creating a Cluster

If your environment includes a vCenter and multiple ESX hosts, you can create a cluster by calling the `Folder.CreateClusterEx` method. You pass in a name for the new cluster and a `ClusterConfigSpecEx` data object. In the data object, you can specify the following properties:

- VMware DRS
  - `drsConfig` property in the `ClusterDrsConfigInfo` data object contains configuration information for the VMware DRS service. Properties in this object specify the cluster-wide (default) behavior for virtual machine and the threshold for generating cluster recommendations. You can enable and disable VMware DRS with the `ClusterDrsConfigInfo.enabled` property.
  - `drsVmConfigSpec` property in the `ClusterDrsVMConfigSpec` data object points to a `ClusterDrsVmConfigInfo` data object that specifies the DRS configuration for a single virtual machine. `ClusterDrsVmConfigInfo` overrides the default DRS configuration for an individual virtual machine and allows you to specify the DRS behavior and whether DRS can perform migration or recommend initial placement for a virtual machine.

    To update a DRS configuration, you call `ComputeResource.ReconfigureComputeResource_Task` and pass in a `ClusterConfigSpecEx` object. In the `ClusterConfigSpecEx.drsVmConfigSpec` property, you can specify an array of `ClusterDrsVMConfigSpec` objects that define the configuration for individual virtual machines.
  - `rulesSpec` property in the `ClusterRuleSpec` data object points a `ClusterRuleInfo` data object that specifies the affinity and antiaffinity rules DRS should use. See the *vSphere API Reference* entry for `ClusterRuleInfo`.
- VMware HA – distributed availability service (DAS)
  - `dasConfig` property in the `ClusterDasConfigInfo` data object specifies HA service on the cluster. Properties in this object determine whether strict admission control is enabled, what the default virtual machine settings in this cluster are, whether VMware HA restarts virtual machines after host failure, and so on. See the *vSphere API Reference*.
  - `dasVMConfigSpec` property in `ClusterDasVMConfigSpec` object, where the `info` property is a `ClusterDasVmConfigInfo` data object that specifies the HA configuration for a single virtual machine. You can apply different settings to different virtual machines, or use the default specified in the `dasConfig` property for all virtual machines in the cluster.
- VMware HCI
  - The `ClusterComputeResourceHCIConfigInfo` data object contains properties to capture the configured hosts, virtual distributed switch settings, host configurations, and HCI workflow state. The HCI cluster includes vSphere facilities for DRS, HA, vMotion, and vSAN.
  - The `ClusterComputeResourceHCIConfigSpec` data object specifies the virtual distributed switch settings, host configurations, enhanced vMotion capability, and vSAN configuration.
  - Once configurations are specified, you create an HCI cluster with the `ConfigureHCI_Task` method, and reconfigure an HCI cluster with the `ExtendHCI_Task` method.

Adding a Host to a Cluster

The methods available for adding hosts to a cluster are useful under different circumstances. Each method returns a managed object reference to a task.

- `ClusterComputeResource.AddHost_Task` adds a host to the cluster. The host may be specified by a numeric IP address or a DNS resolvable name. If the cluster supports nested resource pools and you pass an optional `resourcePool` argument, the host's resource pool hierarchy is imported into the new nested resource pool. If a cluster does not support nested resource pools, the host resource pool hierarchy is discarded and all virtual machines on the host are added to the cluster's root resource pool.

- `ClusterComputeResource.MoveInto_Task` moves a host in the datacenter into a cluster, or from one cluster into another. The `MoveHostInto_Task` method is similar, with extra `resourcePool` parameter.
- `Folder.BatchAddStandaloneHosts_Task` adds a list of `newHosts` to the inventory as standalone hosts. This operation works though the list of hosts and returns, in the task result, a list of hosts that were successfully added.

Reconfiguring a Cluster

You can alter a vSphere cluster by calling the `ComputeResource.ReconfigureComputeResource_Task` method. You can alter an HCI cluster by calling the `ClusterComputeResource.ExtendHCI_Task` method.

To reconfigure a vSphere cluster, for instance to enable or disable VMware DRS or VMware HA, you can change properties in the `ComputeResourceConfigSpec` data object, and pass it in to `ReconfigureComputeResource_`Task. For relevant properties, see Creating a Cluster. To enlarge a vSphere cluster, you can call `AddHost_Task`.

To enlarge an HCI cluster, possibly after adding hosts with `BatchAddStandaloneHosts_Task`, you can call `ClusterComputeResource.ExtendHCI_Task`, passing in the new hosts as parameter `hostInputs`.

To move a datacenter host into a cluster, or from one cluster to another if the host is in maintenance mode, you can call the `ClusterComputeResource.MoveInto_Task` method. The `MoveHostInto_Task` method is similar but allows moving the host into a nested resource pool.

To move hosts out of a cluster, and make them standalone hosts in the datacenter, you can call the `Folder.MoveIntoFolder_Task` method.

To remove a host from a cluster, and from the datacenter, you can call the `Folder.Destroy_Task` method, if you are granted the `Host.Inventory.RemoveHostFromCluster` privilege.

Managing DRS Clusters

The vSphere Client UI allows you to explore DRS cluster behavior, which is also described in the *vSphere Resource Management* guide. When DRS is running, it generates recommendations and associated information that result in a well balanced cluster, including:

- Initial placement of virtual machines
- Virtual machine migration for load balancing. Each migration recommendation has a rating, which you can find in the `ClusterRecommendation.rating` property. Client applications can choose to consider only high-priority migrations or migrations with multiple priority levels.
- Whether or not DRS clusters are valid, and have enough resources to start additional virtual machines.

DRS recommendations are stored in the `ClusterComputeResource.recommendation` property, which is an array of `ClusterRecommendation` data objects. Each `ClusterRecommendation` includes information about the action to perform and information you can use to display information to end users or for logging.

- Client applications can call `ClusterComputeResource.ApplyRecommendation` to apply one or more recommendations.
- For more fine-grained control, client applications can perform individual actions only. The `ClusterRecommendation.action` property is an array of `ClusterAction` objects. Each `ClusterAction` includes a target for the action and the type, which is a string that is one of the values of the `ActionType` enum (`HostPowerV1`, `MigrationV1`, `VmPowerV1`). Client applications can use the `ActionType` information to act on DRS recommendations by powering on hosts, migrating virtual machines, or powering on virtual machines by calling `Datacenter.PowerOnMultiVM_Task`.

Managing HA Clusters

You can add a host to an HA cluster by calling the `AddHost_Task` method, or move a host into a cluster by calling the `MoveHostInto_Task` method or similar.

You might have to call `HostSystem.ReconfigureHostForDAS_Task` to reconfigure the host for HA if the automatic HA configuration fails. See Adding a Host to a Cluster.

Primary and Secondary Hosts

You can add a secondary host to a cluster by calling the `ClusterComputeResource.AddHost_Task` method, which requires that you specify the host name, port, and password for the host to be added as a `HostConnectSpec`.

When you add a host to a VMware HA cluster, an agent is uploaded to the host and configured to communicate with other agents in the cluster. The first five hosts added to the cluster are designated as primary hosts, and all subsequent hosts are designated as secondary hosts. The primary hosts maintain and replicate all cluster state and are used to initiate failover actions. If a primary host is removed from the cluster, VMware HA promotes another host to primary status.

Any host that joins the cluster must communicate with an existing primary host to complete its configuration (except when you are adding the first host to the cluster). At least one primary host must be functional for VMware HA to operate correctly. If all primary hosts are unavailable (not responding), no hosts can be successfully configured for VMware HA.

One of the primary hosts is also designated as the active primary host and its responsibilities include:

- Deciding where to restart virtual machines
- Keeping track of failed restart attempts
- Determining when it is appropriate to keep trying to restart a virtual machine.

Failure Detection and Host Network Isolation

Agents on the different hosts contact and monitor each other through the exchange of heartbeats, by default every second. If a 15-second period elapses without the receipt of heartbeats from a host, and the host cannot be pinged, the host is declared as failed. The virtual machines running on the failed host are restarted on the alternate hosts with the most available unreserved capacity (CPU and memory).

Host network isolation occurs when a host is still running, but it can no longer communicate with other hosts in the cluster. With default settings, if a host stops receiving heartbeats from all other hosts in the cluster for more than 12 seconds, it attempts to ping its isolation addresses. If this also fails, the host declares itself isolated from the network.

When the isolated host's network connection is not restored for 15 seconds or longer, the other hosts in the cluster treat that host as failed and try to fail over its virtual machines. However, when an isolated host retains access to the shared storage it also retains the disk lock on virtual machine files. To avoid potential data corruption, VMFS disk locking prevents simultaneous write operations to the virtual machine disk files. Therefore attempts to fail over the isolated host's virtual machines do not succeed. By default, the isolated host leaves its virtual machines powered on, but you can change HA host isolation response as described in the *vSphere Availability* guide.

Using VMware HA and DRS Together

Using VMware HA with VMware DRS combines automatic failover with load balancing. This combination can result in faster rebalancing of virtual machines after VMware HA has moved virtual machines to different hosts.

When VMware HA performs failover and restarts virtual machines on different hosts, its first priority is the immediate availability of all virtual machines. After the virtual machines have been restarted, those hosts on which they were powered on might be heavily loaded, while other hosts might be comparatively lightly loaded. VMware HA uses the CPU and memory reservation to determine failover, but the actual usage might be higher.

In a cluster using DRS and VMware HA with admission control turned on, virtual machines might not be evacuated from hosts entering maintenance mode because of resources reserved to maintain the failover level. You must manually migrate the virtual machines off of the hosts using VMotion.

When VMware HA admission control is disabled, failover resource constraints are not passed on to DRS and VMware Distributed Power Management (DPM). The constraints are not enforced.

- DRS evacuates virtual machines from hosts and place the hosts in maintenance mode or standby mode regardless of the effect this might have on failover requirements.
- VMware DPM powers off hosts (place them in standby mode) even if doing so violates failover requirements.

The VMware HCI cluster includes VMware DRS and VMware HA features, so all the above remarks apply.

Tasks and Scheduled Tasks

VMware vSphere uses an asynchronous client-server communication model. Methods that end with _Task are non-blocking, returning a reference to a `Task` managed object. You can use `Task` and `ViewManager` managed objects to monitor tasks, cancel certain tasks, and create custom tasks.

If you are using a vCenter system, the `ScheduledTaskManager` allows you to schedule your own tasks for a one-time run or for repeated runs.

Creating Tasks

Each time a vSphere server runs a method, it creates a `Task` and a corresponding `TaskInfo` data object. Some methods run synchronously and return data as the `Task` completes. But methods that end with _Task run asynchronously, and return a reference to a Task that will be created and completed as a processor becomes available. They are created to perform the functions in a non-blocking manner. Therefore, you must use the reference to the `Task` to monitor the status and results of the Task. vSphere operations that include the suffix `_Task` in their names are asynchronous and return `Task` references.

The `Task` object provides information about the status of the invoked operation through its `TaskInfo` data object. An instance of `TaskInfo` populates the `info` property of the `Task` managed object at runtime. By monitoring properties of the `TaskInfo` object, a client application can take appropriate action when the `Task` completes, or can handle errors if the `Task` does not complete successfully.

When a vSphere server creates a `Task`, it also creates a `TaskEvent` object. The `TaskEvent` object contains a copy of the `TaskInfo` object (`TaskEvent.info`). The `TaskEvent` copy of the `TaskInfo` object is a snapshot of the Task state at the time of its creation. It does not change after it is created. To find the current status of the task, use the `Task.info.eventChainId` property.

Session Persistence

A `Task` and its associated objects are session specific, so they will not persist after the session is closed. When your client opens a session, you can only obtain information about the `Task` objects that your client is authorized to view.

Cancelling a Task

To cancel a `Task` that is still running, call the `Task.CancelTask` method, passing in the managed object reference to the `Task` you want to cancel, as shown in this example:

```
my_conn.cancelTask(taskMoRef);
```

You can only cancel a `Task` that has its `cancelable` property set to `true` and its `state` property set to `running`. The operation that initiates the `Task` sets the value of `cancelable` when it creates the `Task`. For example, a `CreateVM_Task` cannot be cancelled. Before attempting to cancel a running `Task`, you can check the values of the `cancelable` property and the `state` property of the `TaskInfo` data object associated with the `Task`.

Using TaskInfo to Determine Task Status

A `Task` object provides information about the status of the invoked operation through its `TaskInfo` data object. An instance of `TaskInfo` populates the `info` property of the `Task` managed object at runtime. By monitoring properties of the `TaskInfo` object, a client application can take appropriate action when the `Task` completes, or can handle errors if the `Task` does not complete successfully.

The `Task.info` property contains a `TaskInfo` data object that contains information about the `Task` the server returns to your client application.

When a `Task` is instantiated by the server, the `TaskInfo.result` property is initialized to `Unset`. Upon successful completion of an operation, the `result` property is populated with the return type specific to the operation. The `result` might be a data object, a reference to a managed object, or any other data structure as defined by the operation.

For example:

1. The `ClusterComputeResource.AddHost_Task` method returns a `Task` object whose `info` property contains a `TaskInfo` data object.
2. At the start of the operation, the `result` property is `Unset`.

3. Upon successful completion of the operation, the `result` property of `TaskInfo` contains the managed object reference of the newly added `HostSystem`.

TaskInfo Values

The following table lists some of the values obtained from a `TaskInfo` data object at the beginning and the end of the `Task` instantiated by the `CreateVM_Task` method.

Property	Datatype	Start of Task Sample Values	End of Task Sample Values
cancelable	boolean	false	false
...			
completeTime	dateTime	Unset	"2009-02-19T22:53:35.015338Z"
progress	int	36	100
queueTime	dateTime	"2009-02-19T22:50:39.111604Z"	"2009-02-19T22:50:39.111604Z"
reason	TaskReason	reason	reason
result	anyType	Unset	64
....			
state	TaskInfoState	"running"	"success"

Monitoring TaskInfo Properties

To monitor the state of a `Task`, use the `PropertyCollector.WaitForUpdatesEx` method.

You can monitor the values of `TaskInfo` properties, which change as the `Task` runs to completion. For example, you can check the values of `startTime`, `queueTime`, `completeTime`, `progress`, `result`, and `state` as the operation progresses. Monitor these properties in your code in a separate thread until the `Task` completes, while the main line of your code continues with other activities.

For more information about monitoring properties, see Client Data Synchronization with WaitForUpdatesEx.

Your code must handle the datatype returned when the `Task` completes (managed object reference, data object, and so on). In addition to `success`, `queued`, and `running`, an operation can enter an `error` state, which your code must handle.

A `Task` object has a lifecycle that is independent of the `TaskManager` that creates it and independent of the entity with which it is associated. It exists to convey status about an operation. You can discard the reference to it when your application no longer needs the information.

The following example shows a code fragment that obtains values for the `info` property from each `Task` object in the array.

Displaying TaskInfoState Values for Tasks in recentTask Array

```
...
private void displayTasks(ObjectContent[] oContents) {
      for(int oci=0; oci<oContents.length; ++oci) {
         System.out.println("Task");
         DynamicProperty[] dps = oContents[oci].getPropSet();
         if(dps!=null) {
            String op="", name="", type="", state="", error="";
            for(int dpi=0; dpi<dps.length; ++dpi) {
               DynamicProperty dp = dps[dpi];
               if("info.entity".equals(dp.getName())) {
                  type = ((ManagedObjectReference)dp.getVal()).getType();
               } else if ("info.entityName".equals(dp.getName())) {
                  name = ((String)dp.getVal());
               } else if ("info.name".equals(dp.getName())) {
```

```
            op = ((String)dp.getVal());
         } else if ("info.state".equals(dp.getName())) {
            TaskInfoState tis = (TaskInfoState)dp.getVal();
            if(TaskInfoState.error.equals(tis)) {
               state = "-Error";
            } else if(TaskInfoState.queued.equals(tis)) {
               state = "-Queued";
            } else if(TaskInfoState.running.equals(tis)) {
               state = "-Running";
            } else if(TaskInfoState.success.equals(tis)) {
               state = "-Success";
            }
         } else if ("info.cancelled".equals(dp.getName())) {
            Boolean b = (Boolean)dp.getVal();
            if(b != null && b.booleanValue()) {
               state += "-Cancelled";
            }
      }...
```

Sample Run of the TaskList Java Application shows output from a run of the program. See the source code listing for `TaskList.java` or for `TaskList.cs` in the vSphere Web Services SDK package for details.

Sample Run of the TaskList Java Application

```
java com.vmware.samples.general.TaskList --url https://srv/sdk --username root --password *******

Started
Task
Operation AcquireCimServicesTicket
Name srv
Type HostSystem
State -Success
Error
======================
Ended TaskList
```

Accessing and Manipulating Multiple Tasks

Use the `ViewManager`'s `ListView` method to identify the set of Tasks you want to monitor.

You can specify a smaller and more efficient data set using one of the `ViewManager` views with the Property Collector. Each view represents objects you have selected on the server. Views are more efficient because you only need a single instance of a `PropertyCollector` object, instead of multiple instances with multiple filter specifications.

Task Monitoring Example Using the Listview Object

You can use the ViewManager's `CreateListView` method to specify a set of tasks that you want to monitor with the PropertyCollector. This example shows how to initiate virtual machine cloning operations and monitor completion by tracking the `Task` objects.

For general task monitoring, a best practice is to use a `ViewManager` to monitor specific tasks. See the *API Reference* for more information about using views.

> **Note:**
>
> For simplicity, the calls to `RetrievePropertiesEx()` and `WaitForUpdatesEx()` in this example ignore the possibility of additional chunks of data returned by the property collector. For more information about chunking in property collector results, see Server Data Transmission.

Create an Authenticated Session with the Server

Use command-line arguments to connect to a vSphere server and log in.

For this task you need:

- The Web Services API WSDL bindings file.
- The hostname of the vSphere server.
- A username and password to authenticate with the vSphere server.
- The name of a virtual machine available to be cloned.

Given the arguments on the command line, this task makes an SSL connection to the vSphere server, retrieves a reference to the SessionManager, and authenticates a session with the credentials passed on the command line.

1. Import the vSphere Web Services API libraries and the necessary Java (and JAX-WS connection, bindings, and SOAP) libraries:

```
import com.vmware.vim25.*;

import java.util.*;
import javax.net.ssl.HostnameVerifier;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLSession;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.soap.SOAPFaultException;
```

2. Create a class to hold data and methods.

```
public class cloneVMTask {
  static ManagedObjectReference pCollector;
  static ManagedObjectReference viewMgr;
  static ServiceContent serviceContent;
  static VimPortType methods;
```

3. Define a TrustManager implementation.

   Authentication is handled by using a TrustManager and supplying a host name verifier method.

   Note:
   For the purposes of this example, this TrustManager implementation will accept all certificates. This is only appropriate for a development environment. Production code should implement certificate validation.

```
private static class TrustAllTrustManager
   implements javax.net.ssl.TrustManager,
              javax.net.ssl.X509TrustManager {
  public java.security.cert.X509Certificate[] getAcceptedIssuers() {
    return null;
  }
  public boolean isServerTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }
  public boolean isClientTrusted(java.security.cert.X509Certificate[] certs) {
    return true;
  }
  public void checkServerTrusted(java.security.cert.X509Certificate[] certs,
                                 String authType)
  throws java.security.cert.CertificateException {
    return;
  }
  public void checkClientTrusted(java.security.cert.X509Certificate[] certs,
                                 String authType)
  throws java.security.cert.CertificateException {
    return;
  }
```

```
} // end TrustAllTrustManager
```

4. Read command line arguments.

```
public static void main(String [] args) throws Exception {
  String serverName = args[0];
  String userName = args[1];
  String password = args[2];
  String vmName = args[3];
  String url = "https://"+serverName+"/sdk/vimService";
```

5. Create SSL context and session context for the connection.

```
javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.TrustManager[1];
javax.net.ssl.TrustManager tm = new TrustAllTrustManager();
trustAllCerts[0] = tm;
// Create the SSL context
javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");
// Create the session context
javax.net.ssl.SSLSessionContext sslsc = sc.getServerSessionContext();
// Initialize the contexts; the session context takes the trust manager.
sslsc.setSessionTimeout(0);
sc.init(null, trustAllCerts, null);
// Use the default socket factory to create the socket for the secure connection
javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
```

6. Set a host name verifier that will enable any connection.

   The host name verifier is invoked during the SSL handshake.

```
HostnameVerifier hv = new HostnameVerifier() {
  public boolean verify(String urlHostName, SSLSession session) {
    return true;
  }
};
HttpsURLConnection.setDefaultHostnameVerifier(hv);
```

7. Create a VimService object to obtain a VimPort binding provider.

   The BindingProvider provides access to the protocol fields in request/response messages. Retrieve the request context which will be used for processing message requests.

```
VimService vimService = new VimService();
methods = vimService.getVimPort();
Map<String, Object> ctxt = ((BindingProvider) methods).getRequestContext();
```

8. Store the Server URL in the request context.

   Specify `true` to maintain the connection between the client and server. The client API will include the Server's HTTP cookie in its requests to maintain the session. If you do not set this to true, the Server will start a new session with each request.

```
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, url);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
```

9. Create a MOref for the ServiceIntsance and retrieve the ServiceContent.

```
// -- ManagedObjectReference for the ServiceInstance on the Server
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");
serviceContent = methods.retrieveServiceContent(SVC_INST_REF);
```

10. Use the SessionManager to log in.

```
methods.login(serviceContent.getSessionManager(),
```

```
                    userName,
                    password,
                    null);
```

11. Get references to the PropertyCollector and the ViewManager.

```
        pCollector = serviceContent.getPropertyCollector();
        viewMgr = serviceContent.getViewManager();
```

With an authenticated session, you can use the PropertyCollector to retrieve the MOref of the virtual machine, then create clones and monitor the `Task` objects:

```
     ManagedObjectReference vmRef = getVmRef( vmName );
     cloneVM( vmRef );
```

A best practice is to close the session when done:

```
     methods.logout(serviceContent.getSessionManager());
   } // end main
 } // end cloneVMTask
```

Find Virtual Machine MOref

Use the PropertyCollector to get a managed object reference for a virtual machine named in the command line.

For this task you need:

- A virtual machine name is referenced by the String variable `vmName`.
- An authenticated Web Services session with the vSphere server that manages the virtual machine.
- A VimPort binding provider is referenced by the variable `methods`, which is attached to the vSphere server connection context.
- The ServiceInstance content is referenced by the variable `serviceContent`.
- The ViewManager object is referenced by the variable `viewMgr`.
- The PropertyCollector object is referenced by the variable `pCollector`.

Given the name of a virtual machine, this function retrieves references to all of the virtual machines in the datacenter and looks for a match to the specified name.

1. Declare a function that accepts a virtual machine name and returns a managed object reference.

```
      private static ManagedObjectReference getVmRef( String vmName )
      throws Exception
      {
        ManagedObjectReference vmRef = null;
```

2. Use a container view to collect references to all virtual machines in the datacenter.

   Start the collection from the root folder at the top of the managed object hierarchy.

```
        List<String> vmList = new ArrayList<String>();
        vmList.add("VirtualMachine");
        ManagedObjectReference cViewRef =
           methods.createContainerView(viewMgr,
                                       serviceContent.getRootFolder(),
                                       vmList,
                                       true);
```

3. Create an `ObjectSpec` to define the beginning of the traversal.

   Use the `setObj` method to specify that the container view is the starting object for this traversal. Set the `skip` property to `true` to indicate that you don't want the PropertyCollector to collect properties from the container.

```
        ObjectSpec oSpec = new ObjectSpec();
        oSpec.setObj(cViewRef);
        oSpec.setSkip(true);
```

4. Create a traversal spec to traverse from the starting object to the objects in the view.

   Set the `skip` property to `false` to indicate that you don't want the PropertyCollector to skip the objects reached by this traversal.

```
TraversalSpec tSpec = new TraversalSpec();
tSpec.setName("traverseEntities");
tSpec.setType("ContainerView");
tSpec.setPath("view");
tSpec.setSkip(false);
```

5. Add the traversal spec to the object spec.

```
oSpec.getSelectSet().add(tSpec);
```

6. Specify the property for retrieval (virtual machine name).

```
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("name");
```

7. Create a PropertyFilterSpec and add the object and property specs to it.

```
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);
```

8. Create a list for the filters and add the spec to it.

```
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);
```

9. Get the data from the server.

```
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(pCollector,fSpecList,ro);
```

10. Go through the returned list and look for a match to the specified `vmName`.

```
if (props != null) {
  for (ObjectContent oc : props.getObjects()) {
    String vmname = null;
    List<DynamicProperty> dps = oc.getPropSet();
    if (dps != null) {
      for (DynamicProperty dp : dps) {
        vmname = (String) dp.getVal();
        // If the name of this virtual machine matches
        // the specified name, save the managed object reference.
        if (vmname.equals(vmName)) {
          vmRef = oc.getObj();
          break;
        }
      }
      if (vmRef != null) { break; }
    }
  }
}
if (vmRef == null) {
  System.out.println("Specified Virtual Machine not found.");
  throw new Exception();
}
return vmRef;
} // end getVmRef
```

Given the MOref of the named virtual machine, you can use the PropertyCollector to retrieve ia MOref to its parent folder, which you will need to clone the virtual machine.

Get_Parent_Property
Given the managed object reference for a virtual machine object, use the PropertyCollector to get the managed object reference for its parent object.

For this task you need:

- A virtual machine managed object reference in a variable named `vmRef`.
- An authenticated Web Services session with the vSphere server that manages the virtual machine.
- A VimPort binding provider referenced by the variable `methods`, which is attached to the vSphere server connection context.
- A PropertyCollector instance referenced by the variable `pCollector`.

Given the MOref of a virtual machine, this function uses the PropertyCollector to retrieve the MOref of the parent of the virtual machine object. The MOref of the parent folder can be used to clone the virtual machine.

A `VirtualMachine` can be a child of either a `Folder` object or a `VirtualApp` object. If it is a child of a `Folder` object, the `parent` property contains the MOref of the `Folder` object and the `parentVApp` property is null. If the `VirtualMachine` is a child of a `VirtualApp` object, the `parent` property is null and the `parentVApp` property contains the MOref of the `VirtualApp` object. The method in this example returns a key-value pair from which the caller can determine which kind of parent the `VirtualMachine` has.

1. Declare a function that accepts a virtual machine MOref and returns a MOref to its parent object.

```
private static DynamicProperty getVMParentProperty(ManagedObjectReference vmRef)
throws Exception {
```

2. Create an `ObjectSpec` to define the property collection. Use the `setObj` method to specify that the vmRef is the starting object for this property collection. Set the `skip` property to `false` to indicate that you want to collect properties from the starting object. Omit the `selectSet` property because the property collection does not need to traverse away from the starting object.

```
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(vmRef);
oSpec.setSkip(false);
```

3. Specify the properties for retrieval (`VirtualMachine.parent` and `VirtualMachine.parentVApp`).

   Because the `VirtualMachine` has two mutually exclusive parent properties, depending on the parent type, this code collects both properties.

```
PropertySpec pSpec = new PropertySpec();
pSpec.setType("VirtualMachine");
pSpec.getPathSet().add("parent");
pSpec.getPathSet().add("parentVApp");
```

4. Create a PropertyFilterSpec and add the object and property specs to it.

```
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);
```

5. Create a list for the filters and add the property filter spec to it.

```
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);
```

6. Use the filter spec to retrieve the property collection from the server.

```
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = methods.retrievePropertiesEx(pCollector,fSpecList,ro);
```

7. Unwrap the parent folder property.

```
if (props != null) {
  for (ObjectContent oc : props.getObjects()) {
```

```
            List<DynamicProperty> dps = oc.getPropSet();
            if (dps != null) {
              for (DynamicProperty dp : dps) {
                if (dp.getName().equals("parent") || dp.getName().equals("parentVApp")) {
                  return dp;
                }
              }
            }
          }
        }
        System.out.println("Parent not found.");
        throw new Exception();
      }
```

Now that you have the reference information for the virtual machine that you specified on the command line (`vmRef`) and a reference for the parent (in the `dp` property), you are ready to extract the type and MOref of the parent.

Create Clones of the Virtual Machine

Create two clones of the virtual machine and build a PropertyFilterSpec that uses a ListView managed object to track the Tasks to completion.

For this task you need:

- A virtual machine managed object reference in a variable named `vmRef`.
- An authenticated Web Services session with the vSphere server that manages the virtual machine.
- A VimPort binding provider referenced by the variable `methods`, which is attached to the vSphere server connection context.
- The ViewManager object is referenced by the variable `viewMgr`.
- A PropertyCollector instance referenced by the variable `pCollector`.

Given the MOref of a virtual machine and the MOref of its parent folder, this function initiates two clone operations, resulting in two `Task` managed objects that you can use to monitor progress of the clone operations. The function uses the `Task` objects to build a `PropertyFilterSpec` that you can use to report status updates for the two `Task` objects.

1. Declare a method that accepts a virtual machine MOref to clone it and monitor the `Task` objects.

   ```
   private static void cloneVM(ManagedObjectReference vmRef) throws Exception {
   ```

2. Use the `getVMParent` method to get a MOref for the parent folder of the virtual machine.

   If the parent object is a `Folder`, the `CloneVMTask` method will create the clones in the same folder that contains the specified virtual machine. If the parent object is a `VirtualApp`, the `VirtualMachine` cannot be cloned.

   ```
   DynamicProperty parent = getVMParentProperty( vmRef );
   if (parent.getName().equals("parentVApp")) {
     System.out.println("Will not clone VM in a VApp.");
     throw new Exception();
   } else {
     ManagedObjectReference folder = (ManagedObjectReference) parent.getVal();
   }
   ```

3. Create a clone specification. Use default values whenever possible.

   ```
     VirtualMachineCloneSpec cloneSpec = new VirtualMachineCloneSpec();
     VirtualMachineRelocateSpec vmrs = new VirtualMachineRelocateSpec();
     cloneSpec.setLocation(vmrs);
     cloneSpec.setPowerOn(true);
     cloneSpec.setTemplate(false);
   ```

4. Create two clone virtual machines.

   ```
     ManagedObjectReference cloneTask = methods.cloneVMTask( vmRef,
                                                             folder,
                                                             "clone__1",
   ```

```
                                                                cloneSpec);
ManagedObjectReference cloneTask2 = methods.cloneVMTask( vmRef,
                                                          folder,
                                                          "clone__2",
                                                          cloneSpec);
```

The `CloneVMTask` method returns a `Task` object from each invocation. You can use PropertyCollector to track task completion.

5. Create a list view to reference the clone tasks.

```
List<ManagedObjectReference> taskList = new ArrayList<ManagedObjectReference>();
taskList.add(cloneTask);
taskList.add(cloneTask2);
ManagedObjectReference cloneTaskList = methods.createListView(viewMgr,
                                                              taskList);
```

6. Create an object spec to describe the property collection.

   Add the list of `Task` objects to the object spec. Set the `skip` property to `true` so the PropertyCollector will ignore the properties of the `ListView` object itself.

```
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(cloneTaskList);
oSpec.setSkip(true);
```

7. Create a traversal spec to tell the PropertyCollector how to move from the `ListView` object to the `Task` objects.

   Set the `skip` property to `false` so the PropertyCollector will try to collect properties from the managed objects at the end of this traversal step.

```
TraversalSpec tSpec = new TraversalSpec();
tSpec.setName("traverseTasks");
tSpec.setPath("view");
tSpec.setSkip(false);
tSpec.setType("ListView");
```

8. Add the traversal spec to the object spec.

```
oSpec.getSelectSet().add(tSpec);
```

9. Create a property spec to collect the `Task.info.state` and `Task.info.result` properties.

```
PropertySpec pSpec = new PropertySpec();
pSpec.setType("Task");
pSpec.setAll(false);
pSpec.getPathSet().add("info.state");
pSpec.getPathSet().add("info.result");
```

10. Create a filter spec from the object spec and the property spec, and use the filter spec to create a `PropertyFilter` object to guide the property collection.

    The filter managed object needs to persist on the server until the PropertyCollector finishes reporting task status.

```
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);
ManagedObjectReference pFilter = methods.createFilter(pCollector,
                                                       fSpec,
                                                       true);
```

Two cloning operations are underway, with status of the operations reflected in `Task` objects. A `PropertyFitler` managed object is created and configured to monitor the task status.

Use the `PropertyCollector.WaitForUpdatesEx` method to monitor the status of the `Task` objects.

Monitor Tasks with WaitForUpdatesEx

Use the `PropertyCollector.WaitForUpdatesEx` method to monitor and report `Task` state changes.

For this task you need:

- An authenticated Web Services session with the vSphere server that manages the virtual machine.
- Managed object references for two `Task` objects in the variables `cloneTask` and `cloneTask2`.
- The ViewManager object is referenced by the variable `viewMgr`.
- The PropertyCollector object is referenced by the variable `pCollector`.

This example uses the `waitForUpdatesEx` method to look for a change in the `info.state` property and the `info.result`.property of `cloneTask` and `cloneTask2`. When the state is `"success"`, `info.result` is the managed object reference of the clone virtual machine.

> **Note:**
> The order of property retrieval is not guaranteed, and it may take more than one call to `waitForUpdatesEx` to retrieve both properties for a task

.

1. Initialize variables for the wait loop.

```
String version = "";
Boolean wait = true;
WaitOptions waitOptions = new WaitOptions();

while ( wait ) {
```

2. Call WaitForUpdatesEx.

   If `waitForUpdatesEx` returns `null`, that indicates that the call has timed out.

```
UpdateSet uSet = methods.waitForUpdatesEx(pCollector,
                                          version,
                                          waitOptions);

if (uSet == null) {
  wait = false;
} else {
```

3. Save the `version` for subsequent calls to `WaitForUpdatesEx`.

```
  version = uSet.getVersion();
```

4. Get the list of property updates.

```
  List<PropertyFilterUpdate> pfUpdates = uSet.getFilterSet();
  for (PropertyFilterUpdate pfu : pfUpdates) {
```

5. Get the list of object updates produced by the filter.

```
    List<ObjectUpdate> oUpdates = pfu.getObjectSet();
    for (ObjectUpdate ou : oUpdates) {
```

6. Look for `ObjectUpdate.kind=="MODIFY"` (property modified).

```
      if (ou.getKind() == ObjectUpdateKind.MODIFY) {
        String name = "";
        TaskInfoState state;
        ManagedObjectReference cloneRef = new ManagedObjectReference();
```

7. Get the changed data.

```
        List<PropertyChange> pChanges = ou.getChangeSet();
```

8. Output the property changes.

   If the task completes successfully, the `result` property contains the MOref of the new clone.

```
                for (PropertyChange pc : pChanges) {
                  name = pc.getName();
                  if (name.equals("info.state")) {
                    state = (TaskInfoState)pc.getVal();
                    System.out.println("State is "+state.value());
                  } else if (name.equals("info.result")) {
                    cloneRef = (ManagedObjectReference)pc.getVal();
                    System.out.println("Clone reference is "+cloneRef.getValue());
                  } // end if name.equals
                } // end for pc
              } // end if ou.getKind
            } // end for ou
          } // end for pfu
        } // end if user
      } // end while wait
    } // end cloneVM
```

This code does not set a time-out (`WaitOptions.maxWaitSeconds` is unset), so after it has retrieved all of the property changes, `waitForUpdatesEx` will block the thread, waiting for the TCP connection with the vSphere Server to time out.

How a client application handles the session depends on the particular context. For example, the client can call `WaitForUpdatesEx` from its own thread, look for specific updates and then stop calling the method. For the preceding example, you might choose to watch for the `Task.info.state` to transition to a value of `success` or `error`.

For more information about `WaitOptions` and the `waitForUpdatesEx` method, see Client Data Synchronization with WaitForUpdatesEx.

Gathering Data with a TaskManager Interface

`TaskManager` is a service interface that you can also use for accessing and manipulating `Task` objects. This approach uses a `PropertyCollector` that includes the `recentTask` property of the `TaskManager` managed object that corresponds to the Recent Tasks pane at the bottom of the vSphere client User Interface.

You can use the following `TaskManager` properties in your client application.

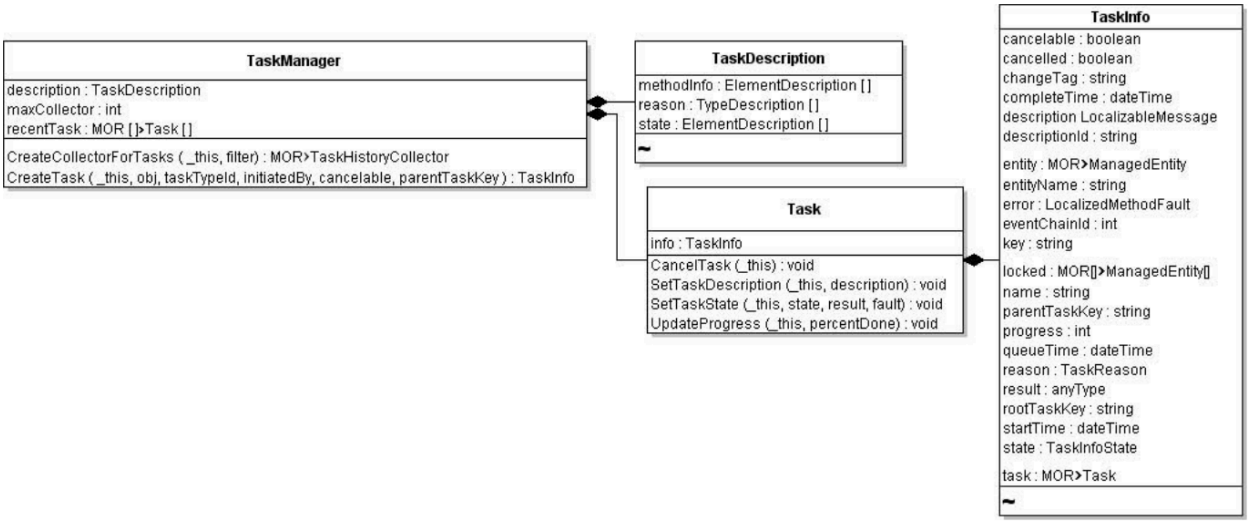
- `description` – `TaskDescription` object that includes a `methodInfo` property. `methodInfo` contains a key-based array that `TaskManager` uses to populate the value of a `TaskInfo` data object's `descriptionId` property with the name of the operation. Examples of two elements from this key-based array are `methodInfo["Folder.createVm"]` and `methodInfo["Folder.createClusterEx"]`.
- `recentTask` – Array of `Task` managed object references that are queued to run, running, or completed within the past 10 minutes. On ESX hosts that are managed by a vCenter, a completed task must also be one of the 200 most recent tasks to be included in the array. A vSphere Client connected to a vSphere Server displays queued, running, and completed tasks in the Recent Tasks pane.

In addition to these properties, `TaskManager` has the following methods:

- `CreateTask` – Used by other methods to create a custom `Task` object. Developers creating extensions can use this method to create custom `Task` objects.
- `CreateCollectorForTasks` – Creates an object that contains all tasks from the vCenter database that meet specific criteria. You cannot run this method against an ESX system. See Using a TaskHistoryCollector.

Figure 625: TaskManager and Task Managed Objects shows a UML class diagram for `TaskManager` and associated objects.

Figure 625: TaskManager and Task Managed Objects



Examining Recent Tasks with TaskManager

To obtain the list of recent tasks, use a `PropertyCollector` to obtain references to the `TaskManager` and to all `Task` objects from the `recentTask` property of the `TaskManager`.

The following example shows an excerpt from the `TaskList.java` sample that creates the `ObjectSpec`, `PropertySpec`, and a `TraversalSpec` to obtain references to all `Task` objects on the server from the `TaskList`. See also Property Collector.

PropertyFilterSpec Definition to Obtain recentTask Property Values

```
private PropertyFilterSpec[] createPFSForRecentTasks(ManagedObjectReference taskManagerRef) {
     PropertySpec pSpec = new PropertySpec();
          pSpec.setAll(Boolean.FALSE);
          pSpec.setType("Task");
          pSpec.setPathSet(new String[] {"info.entity", "info.entityName", "info.name",
              "info.state", "info.cancelled", "info.error"});
     ObjectSpec oSpec = new ObjectSpec();
          oSpec.setObj(taskManagerRef);
          oSpec.setSkip(Boolean.FALSE);
     TraversalSpec tSpec = new TraversalSpec();
          tSpec.setType("TaskManager");
          tSpec.setPath("recentTask");
          tSpec.setSkip(Boolean.FALSE);
     oSpec.setSelectSet(new SelectionSpec[]{tSpec});
     PropertyFilterSpec pfSpec = new PropertyFilterSpec();
          pfSpec.setPropSet(new PropertySpec[]{pSpec});
          pfSpec.setObjectSet(new ObjectSpec[]{oSpec});
     return new PropertyFilterSpec[]{pfSpec};
}
```

For ESX hosts managed by vCenter, use a `TaskHistoryCollector`. See Using a TaskHistoryCollector.

Understanding the ScheduledTaskManager Interface

You can use the `ScheduledTaskManager` to schedule tasks. In the vSphere Client, scheduled tasks display in the Task & Events tab.

You can define actions to occur on vCenter at different times:

- When a vCenter system starts up operations, such as after a reboot

- At a specific time and day
- At hourly, daily, weekly, or monthly intervals

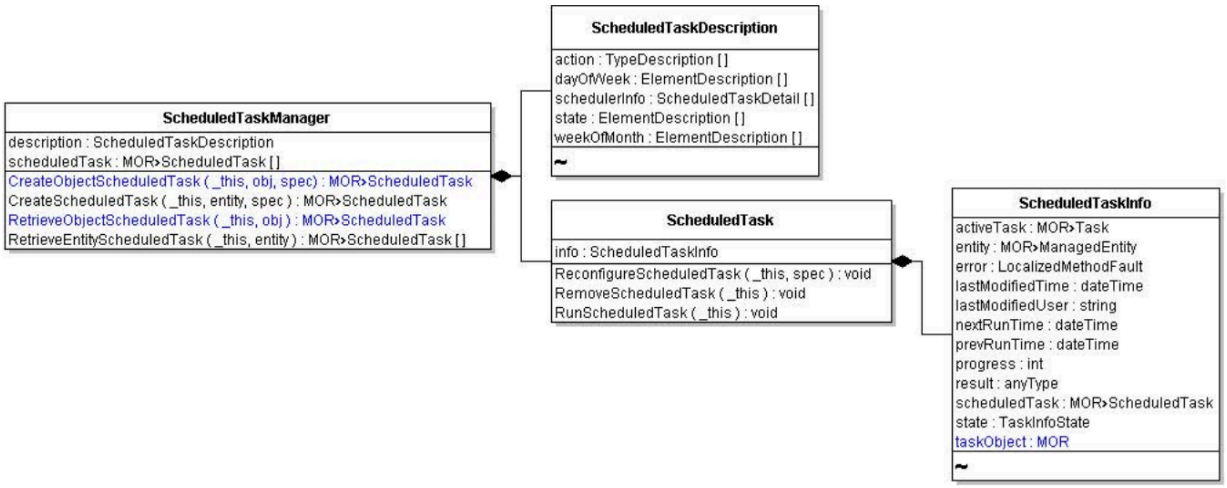
You can schedule scripts to be run or methods to be invoked on the server. You apply the action to an entity in the inventory, such as a virtual machine or a host.

You can perform the following actions with `ScheduledTaskManager`.

- Retrieve scheduled tasks for a specific managed entity by calling the `ScheduledTaskManager.RetrieveEntityScheduledTask` method.
- Create a scheduled task by calling the `ScheduledTaskManager.CreateScheduledTask` method. See Scheduling Tasks.

Figure 626: ScheduledTaskManager and ScheduledTask Managed Objects shows the `ScheduledTaskManager` service interface and associated data objects.

Figure 626: ScheduledTaskManager and ScheduledTask Managed Objects



The `ScheduledTaskManager.scheduledTask` property contains an array of the `ScheduledTask` objects configured for the server. If you have no actions scheduled, this property is empty. For any `ScheduledTask` objects in this array, you can use the `info` property of the `ScheduledTask` object to obtain information about the status of the scheduled action. Information includes the task's `progress`, `state`, previous and next runtimes, and other details contained in the `ScheduledTaskInfo` data object.

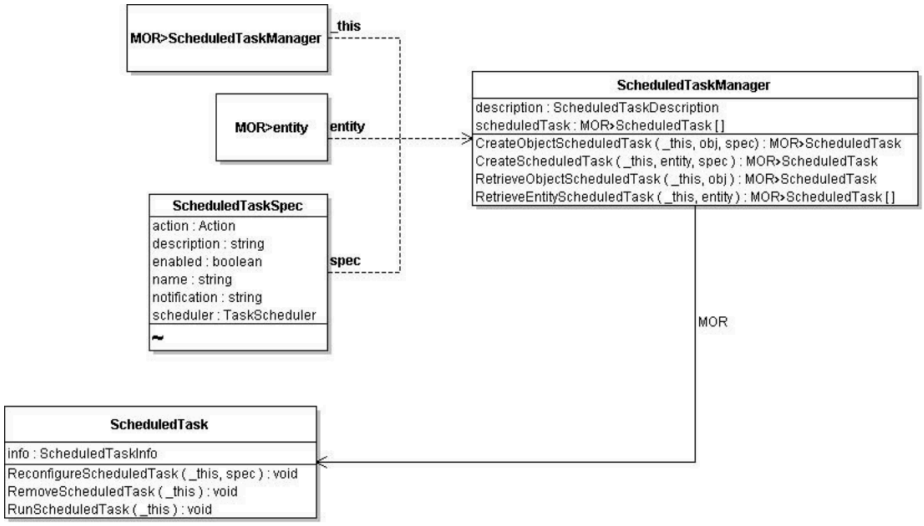
If the action specified for a `ScheduledTask` creates its own `Task` (such as with any of the asynchronous operations), the managed object reference to the `Task` populates the `activeTask` property of `ScheduledTaskInfo`.

Scheduling Tasks

You create a `ScheduledTask` by invoking the `ScheduledTaskManager.CreateScheduledTask` method. When you invoke the method, you include a `ScheduledTaskSpec` object that defines the schedule and specifies the action to take at the specified time. A scheduled action applies to an object based on these rules:

- If you specify a container object as the entity for the scheduled action, the schedule applies to all entities that are direct descendants of the container. You can set a `ScheduledTask` at the `Folder`, `Datacenter`, or `VirtualApp` level and have the scheduled action apply to all entities associated with the `Folder`, `Datacenter`, or `VirtualApp`.
- If you specify a node object in the inventory, such as a virtual machine, the action applies only to the virtual machine.

Figure 627: Using ScheduledTaskManager to Create a ScheduledTask



Defining the Schedule and Action

The `ScheduledTaskSpec` data object contains all the information to create a `ScheduledTask`.

- `action` – Action to take when the `ScheduledTask` runs. Specify an `Action` data object, which is an abstract type that is extended by several specific action types. The `Action` data objects are also used by the `Alarm` infrastructure. See Specifying Alarm Actions.
- `notification` – Specifies the email address for sending notification messages about the `ScheduledTask`. To use notifications, the vCenter system must have an SMTP email gateway configured. By default, `notification` is set to an empty string.
- `scheduler` – Specifies the time, frequency, and other details of the schedule. The `TaskScheduler` data object is the base type for several specific schedule objects. See Scheduling Recurring Operations.

Scheduling Recurring Operations

You can specify the times, days, or frequency of scheduled tasks by creating the appropriate instances of `TaskScheduler` subtypes and setting the `scheduler` property of the `ScheduledTaskSpec`.

The `TaskScheduler` base type has two properties:

- `activeTime` is the time at which the action should occur. If you leave this property unset, it defaults to the time when that specification for the scheduled task was submitted to the server.
- `expireTime` is the time after which the scheduled action should not occur. By default, this property is unset, so the scheduled task does not expire.

The following table provides some usage information about the `TaskScheduler` subtypes. The examples in the table are Java code fragments.

TaskScheduler Subtype	Usage
`AfterStartupTaskScheduler`	Schedule a task to start as soon as the vCenter system is started, or at a defined time after startup. The value must be zero (task triggered at startup) or higher. Example: Schedule a task to run 10 minutes after vCenter startup.
	`AfterStartupTaskScheduler asts = new AfterStartupTaskScheduler(); asts.setMinute(10);`
`OnceTaskScheduler`	Schedule an action to run once only at the specified date and time. Example: Schedule a task to run 30 minutes after the schedule is submitted to the server.

TaskScheduler Subtype	Usage

	`Calendar runTime = Calendar.getInstance();`
	`runtime.add(Calendar.MINUTE, 30);`
	`OnceTaskScheduler ots = new OnceTaskScheduler();`
	`ots.setRunAt(runTime);`

`RecurrentTaskScheduler`	Base type for `HourlyTaskScheduler`, `DailyTaskScheduler`, `WeeklyTaskScheduler`, and `MonthlyTaskScheduler` objects. Set the interval property to define how frequently a task should run. For example, setting the interval property of an hourly task to 4 causes the task to run every 4 hours.
`HourlyTaskScheduler`	Schedule a task to run once every hour (or every specified number of hours) at a specified time. Set the interval property to run the task after a specified number of hours. Example: Schedule a task to run every 4 hours at half-past the hour.

	`HourlyTaskScheduler hts =`
	`  new HourlyTaskScheduler();`
	`hts.setMinute(30);`
	`hts.setInterval(4);`

`DailyTaskScheduler`	Schedule a task to run daily or a specified number of days at a specified time (hour and minutes). Use in conjunction with the interval property to run the task after a specified number of days. Example: Schedule a task to run daily at 9:30 am (EST).

	`DailyTaskScheduler dts =`
	`  new DailyTaskScheduler();`
	`dts.setMinute(30);`
	`dts.setHour(14);`

`WeeklyTaskScheduler`	Schedule a task to run every week (or every specified number of weeks) on a specified day (or days) at a specific time. The hours and minutes are set as UTC values. At least one of the boolean values must be set to `true`. You can also set the interval property to run the task after a specified number of weeks. Example: Schedule a task to run every Tuesday and Sunday at 30 minutes past midnight.

	`WeeklyTaskScheduler wts =`
	`  new WeeklyTaskScheduler();`
	`wts.setMonday(true);`
	`wts.setTuesday(true);`
	`...`
	`wts.setSaturday(false);`
	`wts.setSunday(true);`
	`dts.setMinute(30);`
	`dts.setHour(4);`

`MonthlyByDayTaskScheduler`	Schedule a task to run every month (or every specified number of months) on a specified day at a specified time (hour and minutes). You can also set the interval property to run the task after a specified number of months. Example: Schedule a task to run every 3 months (on the last day of the month) at 12:30 p.m.

	`MonthlyByDayTaskScheduler mbdts =`
	`  new MonthlyByDayTaskScheduler();`
	`mbdts.setDay(31);`
	`mbdts.setInterval(3);`
	`mbdts.setMinute(30);`
	`mbdts.setHour(14);`

TaskScheduler Subtype	Usage
MonthlyByWeekdayTaskScheduler	Schedule a task to run every month (or every specified number of months) on a specified week, weekday, and time (hour: minutes). You can also set the interval property to run the task after a specified number of months. Example: Schedule a task to run on the last Wednesday of each month at 12:30 a.m. `MonthlyByWeekdayTaskScheduler mbwts = new MonthlyByWeekdayTaskScheduler(); mbwts.setOffset(WeekOfMonth.last); mbwts.setWeekday(DayOfWeek.wednesday); mbwts.setHour(4); mbwts.setMinute(30);`

The `hour` and `minute` properties of all objects that extend the `RecurrentTaskSchedule` data object are specified in Coordinated Universal Time (UTC) values rather than the local time of the server. When you define the schedule, convert your local time to a UTC value.

The following code fragment defines a `ScheduledTask` that powers on virtual machines daily at 4:15 a.m., if the server local time is in the Pacific Standard Time (PST) time zone. For a server in the Eastern European Summer Time (EEST) zone, the setting is read by the system as 3:15 pm.

Scheduled Task for Powering-on Virtual Machines

```
...
// Set the schedule using the DailyTaskScheduler subtype.
DailyTaskScheduler dTScheduler = new DailyTaskScheduler();
dTScheduler.setHour(12);
dTScheduler.setMinute(15);
ScheduledTaskSpec tSpec = new ScheduledTaskSpec();
tSpec.setDescription("Start virtual machine as per schedule.");
tSpec.setEnabled(Boolean=TRUE);
tSpec.setName("Power On Virtual Machine");
tSpec.setAction(ma);
tSpec.setScheduler(dTScheduler);
tSpec.setNotification("admin@vmware.com");
my_conn.createScheduledTask(_sic.getScheduledTaskManager, vmRef, tSpec);
...
```

Cancelling a Scheduled Task

You can cancel a scheduled task in several ways.

- To cancel the current run of a scheduled task, call `ScheduledTask.RemoveScheduledTask`. This method does not cancel subsequent runs of the `ScheduledTask`.
- To cancel an upcoming run of a `ScheduledTask`, call `ScheduledTask.ReconfigureScheduledTask` with a new `ScheduledTaskSpec` data object containing the new specifications for the schedule.
- To cancel a `ScheduledTask` that spawns a second task, create a `PropertyCollector` to obtain the reference to the `Tasks` and call its `CancelTask` method. The task must be cancellable.

Using a TaskHistoryCollector

A `TaskHistoryCollector` lets you gather information about tasks. You create a `TaskHistoryCollector` using the `TaskManager.CreateCollectorForTasks` method.

To create a TaskHistoryCollector

1. Identify the type of `Task` objects that you want to collect, and create an instance of a `TaskFilterSpec` data object that specifies your filter criteria.

The TaskFilterSpec includes an taskTypeId property, which you use to limit the set of collected task objects to specific types. You can also provide a time range in the TaskFilterSpec by defining an TaskFilterSpecByTime data object for its time property. See the *vSphere API Reference*.

2. Obtain the managed object reference to the TaskManager on your server instance.
3. Submit the filter and the reference to the server in the CreateTaskHistoryCollector method. The server returns a reference to a TaskHistoryCollector object.

After a HistoryCollector has been created, the server appends new objects that meet the filter criteria to the collection as they occur. The system appends the new object to the collection by placing it in the first position of the latestPage and removes the oldest object from the collection. The latestPage property of the TaskHistoryCollector object has a property that consists of the 1000 most recent objects in the collection. Use a PropertyCollector to obtain the items from the latestPage property.

A HistoryCollector exists only for the duration of the session that instantiated it. Call the HistoryCollector.DestroyCollector method to delete the collector before the session ends.

Creating a TaskHistoryCollector Filter

When you create a TaskHistoryCollector, you can define filters. For example, rather than returning all Task objects associated with virtual machines, you might create a filter to collect only Task objects associated with virtual machines that were started by the backup-administrator between 2:00 and 4:00 a.m. on a specific date.

The TaskFilterSpec object allows you to specify the collection criteria. Most of the properties are optional and can be submitted as null values. The TaskFilterSpec lets you collect tasks based on user name, entity type, time, and state of the Task.

Managing the HistoryCollector

The HistoryCollector managed object provides operations for managing the life-cycle and scrollable view of a collection.

- DestroyCollector – A HistoryCollector exists only for the current session. Invoke the DestroyCollector operation to explicitly destroy the collector before the session ends.
- ResetCollector – Adjusts the starting position for the subset of objects from the collector to the object immediately preceding the current latestPage.
- RewindCollector – Positions the latestPage to the oldest item in the array. When a HistoryCollector is created, this is the default location.
- SetCollectorPageSize – Accepts an integer parameter to set the size of the latestPage property of a HistoryCollector. The default size of a HistoryCollector is an array that consists of at most 1000 objects of the appropriate type (Task, Event). The array is sorted by creation date and time of the objects.

Sample Code for Tasks

The following table lists the sample applications included with the vSphere Web Services SDK that demonstrate Task monitoring.

Java
vsphere-ws\java\JAXWS\samples\com\vmware\general\ TaskList.java
vsphere-ws\java\JAXWS\samples\com\vmware\scheduling\ DeleteOneTimeScheduledTask.java
vsphere-ws\java\JAXWS\samples\com\vmware\scheduling\ OneTimeScheduledTask.java

Java
vsphere-ws\java\JAXWS\samples\com\vmware\scheduling\WeeklyRecurrenceScheduledTask.java

Events and Alarms

Events are sent by vSphere to convey information about things that happen in the system. You can monitor events directly or use an `EventHistoryCollector` to retrieve events from a certain period.

Alarms are sent by vSphere to alert users to problems. You can also create your own alarm to monitor the system and set up follow-up actions. Alarm setup includes specifying the trigger condition and defining the action that should result.

Event and Alarm Management Objects

`EventManager` is the service interface for working with the event infrastructure.

`Event` subtypes define the events that the system generates. See Event Data Objects and Creating Custom Events.

`EventHistoryCollector` allows you to monitor events. You can create a filter to limit the number of events your code retrieves. You can monitor both system events and your own events. See Using an EventHistoryCollector.

The `AlarmManager` is the service interface for creating, setting, and managing alarms. You create an alarm, specifying trigger conditions and the action to take. When the conditions defined for the `Alarm` occur on the system, the `Action` specified for the alarm starts. The alarm also generates an `Event` that you can retrieve with an `EventHistoryCollector`.

Understanding Events

An `Event` is a data object type that contains information about state changes of managed entities and other objects on the server. Events include user actions and system actions that occur on datacenters, datastores, clusters, hosts, resource pools, virtual machines, networks, and distributed virtual switches. For example, these common system activities generate one or more `Event` data objects:

- Powering a virtual machine on or off
- Creating a virtual machine
- Installing VMware Tools on the guest OS of a virtual machine
- Reconfiguring a compute resource
- Adding a newly configured ESX system to a vCenter system

In the vSphere Client, information from `Event` objects generated on a standalone ESX hosts displays in the Events tab. For managed hosts, information from `Event` objects displays in the Tasks & Events tab.

Persistence of `Event` objects depends on the system setup.

- Standalone ESX hosts – `Event` objects are not persistent. Events are retained only for as long as the host system's local memory can contain them. Rebooting a standalone ESX host or powering off a virtual machine removes `Event` objects from local memory.
  A standalone ESX host might keep about 15 minutes worth of `Event` data, but this can vary depending on the processing load of the host, the number of virtual machines, and other factors.
- Managed ESX systems. `Event` objects are persistent. Managed ESX systems send `Event` data to the vCenter system that manages them, and the vCenter system stores the information its database.

You can use the event sample applications included in the SDK package with either managed or standalone ESX systems and with vCenter systems.

Using an `EventHistoryCollector`, you can obtain information about these objects as they are being collected on a specific ESX system, or from a specific historical period from the database. See Using an EventHistoryCollector.

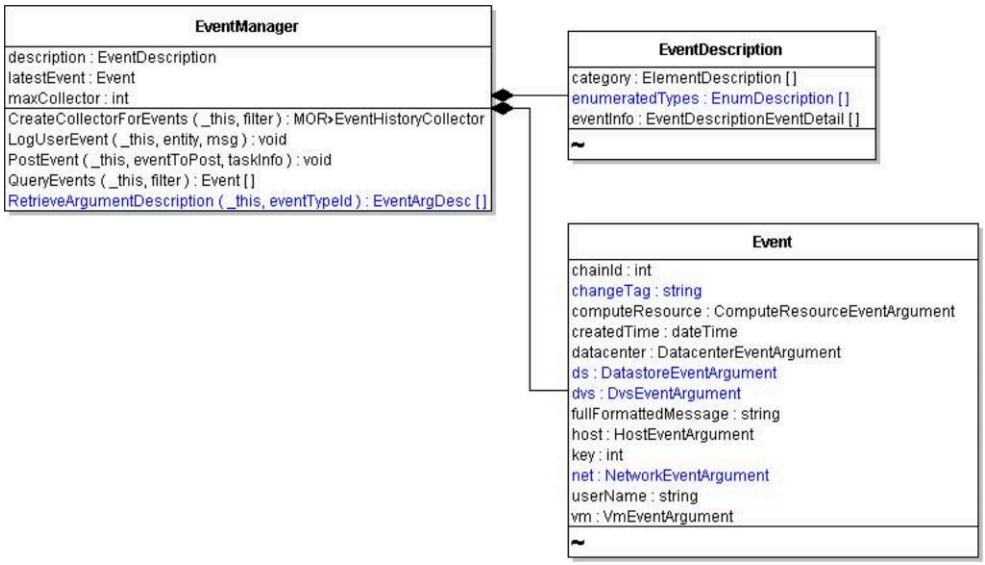
Managing Events with EventManager

`EventManager` is the service interface for working with the event infrastructure.

Figure 628: EventManager Managed Object and Associated Objects shows `EventManager` and related objects. An `EventManager` has these properties:

- A `description` property, defined as an instance of an `EventDescription` data object, which contains an event category and other information.
- A `latestEvent` property that contains the most recent `Event` data object in memory.
- A `maxCollector` property that specifies the number of `EventHistoryCollector` objects per client session that can be created. This value is set by the vCenter system.

Figure 628: EventManager Managed Object and Associated Objects



Event Data Objects

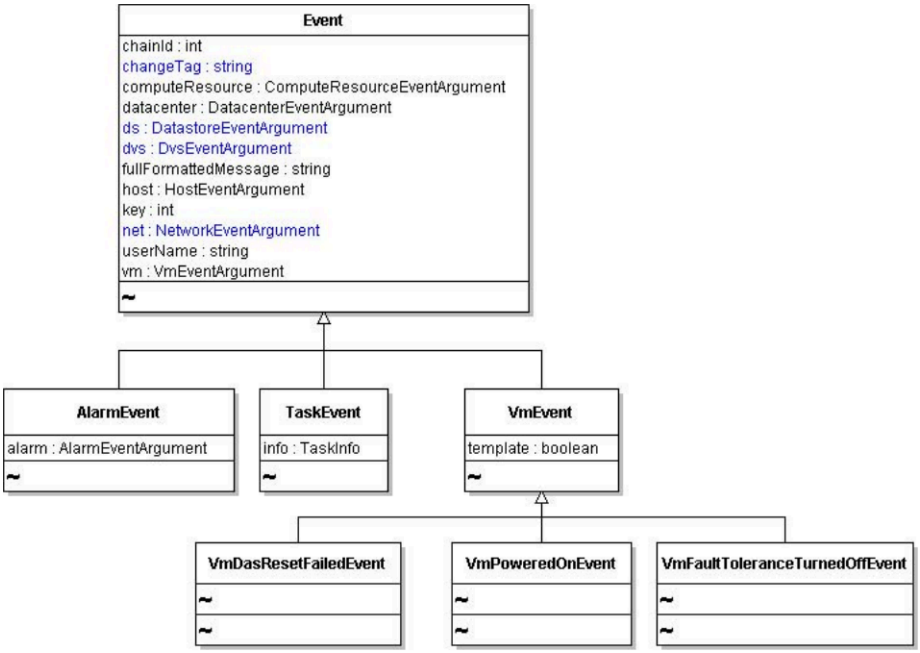
`Event` subtypes define the events that the system generates.

Figure 629: Event Data Object and Sample Subtypes shows only a few of the subtypes that extend the `Event` data object. For example, `TaskEvent` inherits all `Event` properties and includes an `info` property that is an instance of a `TaskInfo` object (see Monitoring TaskInfo Properties).

The following event objects are commonly generated by a console-style client application:

```
com.vmware.vim.VmPoweredOnEvent
com.vmware.vim.VmStartingEvent
com.vmware.vim.VmReconfiguredEvent
com.vmware.vim.VmCreatedEvent
com.vmware.vim.VmBeingCreatedEvent
```

Figure 629: Event Data Object and Sample Subtypes



Formatting Event Message Content

When displayed at the console, `Event` data objects are not formatted and do not provide context information. You can format an `Event` message using the predefined string in the `Event.fullFormattedMessage` property.

You can also format an `Event` message based on contextual information. At runtime, the `Event` data object is populated with values that contain information associated with the source of an event, for example, the `Event` data object's `computeResource`, `datacenter`, `ds`, `dvs`, `host`, `net`, and `vm` properties.

You can use the properties of an `Event` object with the information in the `EventDescriptionEventDetail` in `EventManager.description.eventInfo` to format event messages.

Creating Custom Events

The Web Services API allows you to create custom Event objects that convey information specific to your application. There are two types of custom events, the user log event and the extended event.

Creating User Log Events

The `EventManager.LogUserEvent` method allows you to create user log `Event` objects. You can associate your custom `Event` with any managed entity. User log events are useful for marking actions or status associated with the objects your application deals with.

The following steps show how to create a user log Event.

1. Obtain the managed object reference to the `EventManager`.

```
..
ManagedObjectReference _svcRef = new ManagedObjectReference();
ServiceContent _sic = my_conn.retrieveServiceContent(_svcRef);
ManagedObjectReference eMgrRef = _sic.getEventManager();
...
```

2. Obtain the managed object reference to the entity with which you are associating the `Event`.

   For example, suppose you have a reference to a virtual machine (`myVMRef`) and you want to log a message to record the fact that a virus check completed. You want to use `myVMRef` as a parameter to the `LogUserEvent` method in the next step.

3. Call the `LogUserEvent` method, passing in the `EventManager` and the `Event` reference and a string consisting of the Event message for the `msg` parameter of the operation.

```
LogUserEvent(eMgrRef, myVMRef, "Completed virus check at 1:05 AM on Sunday December 21.");
```

User-defined Event objects display in the vSphere Client among the other events on the system, with the prefix User logged event: followed by the text submitted in your `msg` parameter. In other client applications, such as in the console-based Event sample applications, custom events display as `com.vmware.vim.GeneralUserEvent` objects.

Creating Extended Events

The `EventManager.EventEx` method allows you to create an event that contains an arbitrary dictionary of key-value pairs. This kind of custom event allows greater flexibility to store application data that is not associated with a managed entity, and is not limited to a single string value. The custom event structure also contains more sophisticated metadata than user log events.

The following steps show pseudocode examples of the operations you need to do in your client code.

1. Obtain the managed object reference to the `EventManager`.

```
si = connection.retrieveServiceContent(svc_ref);
em = si.eventManager;
```

2. Choose a severity for the custom event.

```
severity = EventEventSeverity.warning;
```

3. Create a local copy of an extended event with metadata.

```
e = vim.event.EventEx(severity,
                      eventTypeId=”com.example.events.Total_System_Backup”,
                      createdTime = si.CurrentTime(),
                      chainId=0,
                      key=0,
                      userName=local_account_name);
```

4. Create a set of key-value pairs to store the information you want to associate with the event.

```
arg_list = {};
arg1 = vmodl.KeyAnyValue(key=”reason”, value=”Upcoming governance audit”);
arg2 = vmodl.KeyAnyValue(key=”datacenter”, value=”Washington South”);
arg3 = vmodl.KeyAnyValue(key=”organization”, value=”IT Cloud Services”);
arg_list.append(arg1, arg2, arg3);
```

5. Add the set of key-value pairs to the event object.

```
e.arguments = arg_list;
```

6. Use the Event Manager to post the event to the server.

```
em.PostEvent(e);
```

Using an EventHistoryCollector

An `EventHistoryCollector` lets you gather information about events that the server has generated. You create an `EventHistoryCollector` using the `EventManager.CreateCollectorForEvents` method.

The following steps show how to create an `EventHistoryCollector`.

1. Identify the type of `Event` objects that you want to collect, and create an instance of an `EventFilterSpec` data object that specifies your filter criteria. See Creating an EventHistoryCollector Filter.

   The `EventFilterSpec` includes an `eventTypeId` property, which you use to limit the set of collected event objects to specific types. You can also provide a time range in the `EventFilterSpec`, by defining an `EventFilterSpecByTime` data object for its `time` property. See the *vSphere API Reference* for details.

2. Obtain the managed object reference to the `EventManager` on your server instance.
3. Submit the `filter` and the reference to the server in the `CreateEventHistoryCollector` operation. The server returns a reference to an `EventHistoryCollector` object.

After you have created the `HistoryCollector`, the server appends new objects that meet the filter criteria to the collection as they occur. The system appends the new object to the collection by placing it in the first position of the `latestPage` and it removes the oldest object from the collection. The `latestPage` property of the `EventHistoryCollector` object has a property that consists of the 1000 most recent objects in the collection. Use a `PropertyCollector` to obtain the items from the `latestPage` property.

A `HistoryCollector` exists only for the duration of the session that instantiated it. You invoke the `DestroyCollector` operation to explicitly eliminate the collector before the session ends.

Creating an EventHistoryCollector Filter

When you create an `EventHistoryCollector`, you can define filters. For example, rather than returning all `Event` objects associated with virtual machines, you might create a filter to collect only those `Event` objects associated with virtual machines that were started by the backup-administrator between 2:00 and 4:00 a.m. on a specific date.

The `EventFilterSpec` object allows you to specify the collection criteria. Most of the properties are optional and can be submitted as `null` values. The `EventFilterSpec` lets you collect events based on user name, entity type, time, and state of the `Event`.

Managing the HistoryCollector

The `HistoryCollector` managed object provides operations for managing the life-cycle and scrollable view of a collection.

- `DestroyCollector` – A `HistoryCollector` exists only for the current session. Invoke the `DestroyCollector` operation to explicitly destroy the collector before the session ends.
- `ResetCollector` – Adjusts the starting position for the subset of objects from the collector to the object immediately preceding the current `latestPage`.
- `RewindCollector` – Positions the `latestPage` to the oldest item in the array. When a `HistoryCollector` is created, this is the default location.
- `SetCollectorPageSize` – Accepts an integer parameter to set the size of the `latestPage` property of a `HistoryCollector`. The default size of a `HistoryCollector` is an array that consists of at most 1000 objects of the appropriate type (`Task`, `Event`). The array is sorted by creation date and time of the objects.

Using Alarms

The vSphere alarm infrastructure supports automating actions and sending different types of notification in response to certain server conditions. Many `Alarms` exist by default on vCenter systems. You can also create alarms yourself. For example, an Alarm can send an alert email message when CPU usage on a specific virtual machine exceeds 99% for more than 30 minutes.

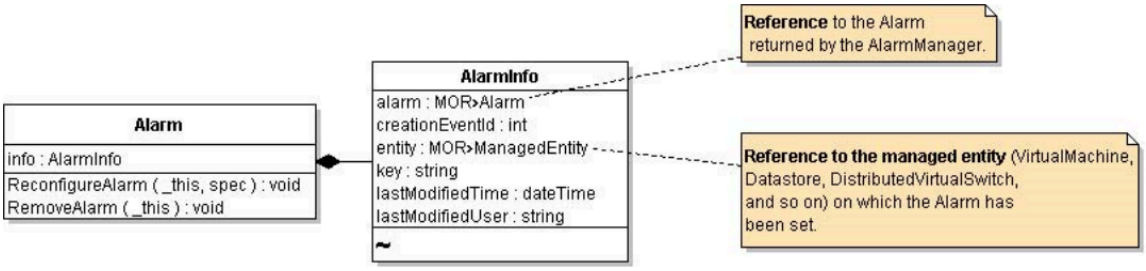
The alarm infrastructure integrates with other server components, such as events and performance counters.

The `AlarmManager` is the service interface for creating, setting, and managing alarms. You create an alarm, specifying the trigger conditions and the action to take. When the conditions defined for the `Alarm` occur on the system, the `Action` specified for the alarm starts. The alarm also generates an `Event` that is posted to the `Event` history database. In addition, the action initiated by the `Alarm` might also post a second `Event` to the database, depending on the `Action` type.

Obtaining a List of Alarms

Use the `AlarmManager.GetAlarm` method to obtain an array of references to all `Alarm` managed objects defined for a specific managed entity. When you call the method, you can pass in an optional reference to a managed entity. Without a reference to a managed entity, the `GetAlarm` operation returns all `Alarm` objects for all entities that are visible to the principal associated with the session invoking the operation.

Figure 630: Alarm Managed Object



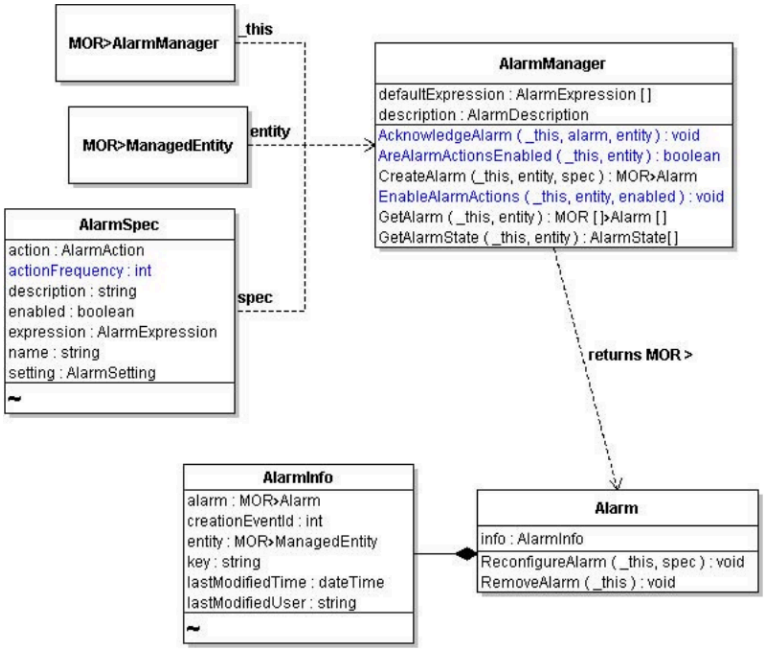
The `Alarm.info` property is an `AlarmInfo` data object. You can obtain information about active `Alarms` by collecting the properties of the `AlarmInfo` data object.

Creating an Alarm

You create an alarm with the `AlarmManager.CreateAlarm` method. In the simplest case, you specify the trigger condition in the `AlarmSpec.expression` property and the action to perform in the `AlarmSpec.action` property. When the expression evaluates to `true`, the alarm performs the action.

Figure 631: CreateAlarm Method Inputs and Outputs shows the `CreateAlarm` method.

Figure 631: CreateAlarm Method Inputs and Outputs



The following steps show how to create an alarm.

1. Obtain a managed object reference to the `AlarmManager` associated with the vCenter.
2. Obtain a managed object reference of the entity on which you want to set the `Alarm`.
3. Create an `AlarmSpec` data object and specify the alarm details in its properties. See Defining Alarms Using the AlarmSpec Data Object.
4. Call `AlarmManager.CreateAlarm`, passing in the references and the `AlarmSpec` data object. The system returns a managed object reference to the `Alarm` (see Figure 631: CreateAlarm Method Inputs and Outputs).

The state of an alarm is contained in an `AlarmState` data object.

Defining Alarms Using the AlarmSpec Data Object

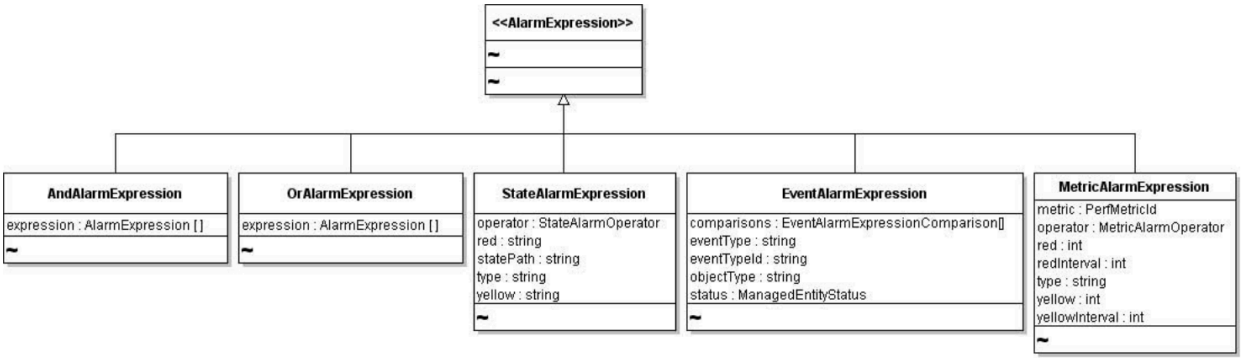
The `AlarmSpec` data object has properties for all aspects of an `Alarm`, including its expression and the action to take when the expression evaluates to true. The following properties define the alarm; see the *API Reference* for a complete list.

- `action` – Action to initiate when the `Alarm` becomes active. Specify one of the `Action` subtypes. See Specifying Alarm Actions.
- `actionFrequency` – Number of seconds that the `Alarm` remains in the state required to initiate the specified action.
- `expression` – One or more `AlarmExpression` data objects combined in a way that evaluates to a true-false expression. See Specifying Alarm Trigger Conditions with AlarmExpression.
- `setting` – Tolerance and frequency limits for the `Alarm` defined in the `AlarmSetting` data object. `AlarmSetting` contains two integer properties:
  - `reportingFrequency`, which specifies the number of seconds between activation of an alarm. Use 0 to specify that the alarm can activate as frequently as required.
  - `toleranceRange`, which specifies the acceptable range (measured in hundredth percentage) above and below the specified value defined in a `MetricAlarmExpression`.

Specifying Alarm Trigger Conditions with AlarmExpression

You use the `AlarmExpression` data object to specify the conditions under which you want the `Alarm` to become active. The `AlarmExpression` data object is an abstract type with several subtypes, which allow you to specify thresholds on objects, state of objects, or specify specific events to monitor.

Figure 632: AlarmExpression and Its Subtypes



AlarmExpression Types

By using the appropriate type of `AlarmExpression`, you can set alarms for different conditions, states, or events.

AlarmExpression	Description	Example
`StateAlarmExpression`	Specifies thresholds that trigger the alarm.	Triggered by a power state change of a virtual machine or state change of a distributed virtual switch.
`MetricAlarmExpression`	Specifies levels at which the alarm changes state. See Using MetricAlarmExpression.	Triggered when resource utilization metrics exceed a specified limit.
`EventAlarmExpression`	Specifies a type of event as the basis for the alarm.	Triggered by power on or power off events of primary or secondary virtual machines in a fault-tolerant cluster.
`EventAlarmComparison`	Specifies the property of the `Event` that should trigger the alarm and the operator to use as the basis for comparison.	
`AndAlarmExpression` `OrAlarmExpression`	Combines one or more instances of the `AndAlarmExpression` and the `OrAlarmExpression` data objects into an expression that evaluates to `true` or `false`.	

Using MetricAlarmExpression

The `MetricAlarmExpression` data object lets you set an alarm to monitor performance metrics. The vSphere Client uses the data object to indicate when hosts or clusters do not have sufficient resources in a DAS or DRS cluster environment. See the *Resource Management Guide*.

You set the `metric` property to the `PerfMetricId` of a performance metric that you want to monitor on the system. Set the `red` or `yellow` properties to identify the level at which the metric value moves from green, to yellow, to red. You must define `red`, `yellow`, or both properties. Use each of these properties with the `isAbove` or `isBelow MetricAlarmOperator` enumerations to complete the definition of the threshold.

In conjunction with `red` and `yellow` properties, you can use the `redInterval` or `yellowInterval` properties. These properties enable you to set the number of seconds that the performance metric must be in `red` or `yellow` state before the expression becomes `true` and triggers the defined action.

Specifying Alarm Actions

You specify the actions that the system should take by setting the `action` property of the `AlarmSpec` data object to the `AlarmAction` data object defined for the purpose.

The `AlarmAction` data object is an abstract type that has two descendent objects.

- The `AlarmTriggeringAction` data object has an `action` property and a `transitionSpecs` property. `AlarmTriggeringActionTransitionSpec` allows you to define a starting state and a final state for the `Alarm`. You can limit the number of `Alarm` objects actually triggered to a single `Alarm` by specifying `false` for the `repeats` property of the `AlarmTriggeringActionTransitionSpec`.
- The `GroupAlarmAction` data object is an array version of the `AlarmAction` base type. You can create a single `AlarmAction` instance or an array of `AlarmAction` instances to take effect when the conditions specified for your alarm are met on the system.

The system can respond to an alarm in several ways:

- Invoking an operation. To invoke an operation, create a `MethodAction` data object.
- Running a Script. To run a script, create an instance of the `RunScriptAction` data object that specifies the fully qualified path to the shell script on the vCenter.
- Send an email message. To send an email message to a system administrator, use the `SendEmailAction` data object.

Figure 633: AlarmAction and Related Objects



For example, you can use the `MethodAction` data object type to invoke an operation on the server. The `MethodAction` data object contains the following properties:

- `name`—Name of the operation that you want to invoke at the scheduled time.
- `argument`—Specifies required parameters, if any, as an array of `MethodArgumentAction` data objects. Depending on the entity associated with the alarm, the `MethodAction.argument` property might not be needed.

Deleting or Disabling an Alarm

An `Alarm` remains active until you delete it or disable it. To delete the alarm, obtain a managed object reference to the `Alarm` and invoke its `RemoveAlarm` operation.

To disable the `Alarm`, obtain managed object references to the `AlarmManager` and to the entity on which the `Alarm` is set. Call `AlarmManager.EnableAlarmActions` operation, passing the value `false` for the `enabled` parameter.

Sample Code for Events and Alarms

The following table lists the sample applications included with the vSphere Web Services SDK that demonstrate some of the topics discussed in this chapter.

Java
`\samples\alarms\MPowerStateAlarm.java`
`\samples\events\EventFormat.java`
`\samples\events\EventHistoryCollectorMonitor.java`
`\samples\events\VMEventHistoryCollectorMonitor.java`

vSphere Performance

VMware vSphere servers use performance counters to track resource use. At runtime, vSphere components generate performance data which the vSphere servers store in performance counters. You can use the `PerformanceManager` interface to retrieve the data.

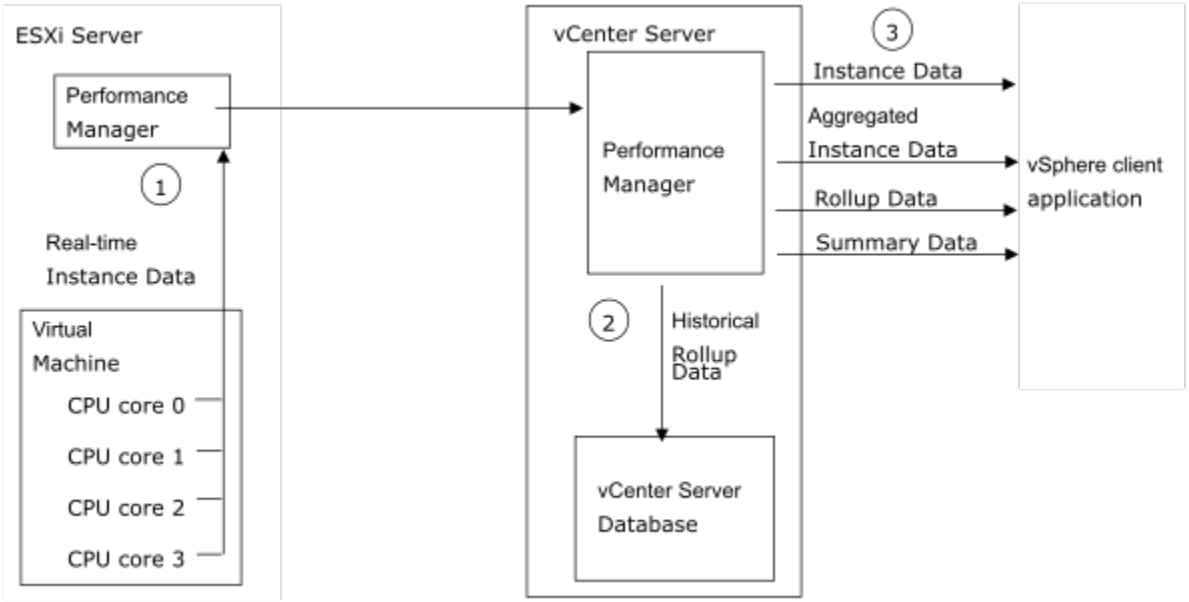
vSphere Performance Data Collection

In a vSphere environment, virtual and physical components generate performance data. To track the use of resources, ESX Servers perform real-time data collection and vCenter instances store the data in the vCenter database. vCenter instances also store a historical rollup of the data according to defined performance intervals.

- Real-time data collection – An ESX Server collects data for each performance counter every 20 seconds and maintains that data for one hour.
- Historical data rollup – A vCenter collects data from all of the hosts that the vCenter manages. The `PerformanceManager` defines performance intervals that specify time periods for performance data rollup, a methodology for combining data values. The server stores the rolled up performance counter data in the vCenter database.

The following figure represents vSphere performance data collection and retrieval.

Figure 634: vSphere Performance Data Collection and Retrieval



1. ESX Servers sample performance counter instances every 20 seconds and maintain the real-time instance data for one hour. For example, the figure shows collection of CPU statistics for four CPU cores.
2. The vCenter retrieves and stores data from the servers that it manages. The Server produces rollup data according to the settings of the historical intervals.
3. vSphere client applications can retrieve real-time instance data, aggregated instance data, historical rollup data, and summary data.

The following table defines terms that are used to describe vSphere performance management.

Term	Definition
performance providers	Performance providers include managed entities, such as hosts, virtual machines, compute resources, resource pools, datastores, and networks.
performance counter	Unit of statistical data collected on a vSphere server. For example, a vCenter collects the average CPU utilization for hosts, virtual machines and clusters (the counter `cpu.usage.average`).
counter ID	System-generated identifier for a performance counter.
instance	An identifier derived from device configuration names. Examples of counter instances are the name of a virtual Ethernet adapter such as “vmnic0:”, or a number that identifies a CPU core, such as 0, 1, 2, or 3. Performance data is retrieved as specific instances of performance counters.
instance data	Performance data collected at 20-second intervals.
metric ID	Combination of a counter ID and an instance. You use metric IDs – `PerfMetricId` objects – when you construct a performance query specification to identify the data to be collected. There are two system-defined instances that you can use to specify aggregate retrieval. See the description of aggregate performance data below. • ″*″ – An asterisk directs the vSphere Server to return all instances plus rollup data. This is not supported for some disk-related counters. • ″″ – A string of length zero directs the vSphere Server to return only aggregated instance data or rollup type data. The vSphere Server returns metric IDs embedded in the data objects that it returns as a response to performance queries.

Term	Definition
performance interval	Data object (`PerfInterval`) which defines the time interval between collection events, the collection level, and the time period that the data will be stored on the Server. • ESX Servers define a built-in performance interval that specifies data collection every 20 seconds for each performance counter. ESX Servers also define a single historical interval (`PerformanceManager.historicalInterval`) that defines aggregate performance data. This system-defined performance interval specifies aggregate data collection every 300 seconds for each counter. You cannot modify the performance intervals on an ESX Server. • vCenter instances define four performance intervals that determine how collected instance data is aggregated and stored. You can modify the system-defined intervals on a vCenter to a limited extent.
collection level	Number between one and four that is assigned to a performance interval (`PerformanceManager.historicalInterval[].level`). The interval collection level corresponds to the level specified for individual performance counters (`PerfCounterInfo.level`). A vCenter uses a performance interval to perform performance data aggregation, using data for the counters with levels that match the performance interval collection level.
rollup type	Methodology for producing a single value from a set of statistical values (`PerformanceManager.perfCounter[].rollupType`). Examples of rollup types are average, latest, and summation.
aggregate performance data	A single value that represents a set of instance data values collected for a performance counter. The single value is derived using one of the rollup types.

PerformanceManager Objects and Methods

`PerformanceManager` provides methods for obtaining statistical data about various aspects of system performance, as generated and maintained by the performance providers. It also defines historical performance intervals and it identifies the set of performance counters that you can use to obtain performance data. The following table shows the `PerformanceManager` properties.

Property	Description
description	Composite object that includes information about the types of counters (`counterType`) and statistics (`statsType`) available on the system.
historicalInterval	Array of system-defined performance intervals (`PerfInterval` data objects). Each object defines the interval between rollup events, the collection level, and the time period that the data is stored on the system. • For an ESX system, the array contains a single performance interval. You cannot modify the ESX performance interval. • For vCenter systems, the `PerfInterval` objects control how ESX performance data are rolled up and stored in the database. You can modify some of the `PerfInterval` properties on a vCenter.
perfCounter	Array of `PerfCounterInfo` data objects. The array identifies all of the performance counters known to the vCenter at the time a client accesses the array. The set of counters may change as ESX hosts are added or removed from vCenter management. Each `PerfCounterInfo` object contains metadata associated with a performance counter.

The `PerformanceManager` methods allow you to retrieve performance statistics and to retrieve metadata that defines the statistics. The following table classifies the methods and describes their purposes.

Method Type	Method	Purpose
Performance data availability	QueryAvailablePerfMetric	Returns `PerfMetricId` objects which identify the counter data available on the specified entity. For example, a virtual machine provides the memory counter `granted`, which indicates the amount of physical memory that is mapped for the virtual machine. The `PerfMetricId` object for the `mem.granted.average` counter specifies the system-defined counter ID. Since this is a memory counter, the `PerfMetricId.instance` property is empty.
Performance data retrieval	QueryPerf	Returns statistics for a specific list of managed entities that provide performance data.
	QueryPerfComposite	Returns statistics for a host and its virtual machines. This method accepts the `refreshRate` for current statistics or the `intervalId` of one of the historical intervals as a parameter. Supported for the `HostSystem` managed entity only.
Performance counter metadata retrieval	QueryPerfCounter	Returns `PerfCounterInfo` data objects for the specified list of counter IDs.
	QueryPerfCounterByLevel	Returns `PerfCounterInfo` data objects for the specified collection level.
Performance provider information	QueryPerfProviderSummary	Returns the `PerfProviderSummary` data object for the specified managed object.
Collection parameters	ResetCounterLevelMapping	Restores a set of performance counters to their default collection levels.
	UpdateCounterLevelMapping	Changes the collection level for a set of performance counters.
	UpdatePerfInterval	Modifies the system-defined performance intervals.

Retrieving vSphere Performance Data

To retrieve collected data, your client application creates a query specification and passes the specification to a performance query method. The query specification is composed of one or more `PerfQuerySpec` objects. Each object identifies the following:

- Performance provider – managed entity for which the Server will return performance data (`PerfQuerySpec.entity`).
- Performance counters – `PerfMetricId` objects that identify performance counter instances (`PerfQuerySpec.metricId`).
- Performance interval – the sampling period that defines the data rollup (`PerfQuerySpec.intervalId`).
- Amount of data to be returned – start and end times (`PerfQuerySpec.startTime`, `PerfQuerySpec.endTime`) and maximum number of values (`PerfQuerySpec.maxSample`) to limit the amount of data to be returned.
- Output data format (`PerfQuerySpec.format`) – one of two kinds:
  - Normal output returned as values contained in data objects.
  - Formatted output returned as strings containing comma-separated values.

The combination of the `entity` and `metricID` properties determine the set of counters for which the server will return performance data. The combination of the `interval`, `startTime`, `endTime` properties produce instance, aggregated instance, rollup, or summarized data. The following table summarizes the different classifications of performance data that you can retrieve from a vCenter.

Performance Data	Description
Instance	ESX Servers sample performance data every 20 seconds. 20-second interval data is called instance data or real-time data. To retrieve instance data, specify a value of 20 seconds for the `PerfQuerySpec.intervalId` property.
Aggregated Instance	A vSphere client can retrieve aggregated instance data. To obtain aggregated instance data, specify the following `PerfQuerySpec` properties. • `intervalId` – Specify 20 seconds to indicate instance data. • `metricId[].instance` – specify a zero-length string ("") for aggregated instance data.
Rollup	The vCenter uses the historical intervals to rollup performance data from the servers that it manages. To retrieve historical performance data, specify the following `PerfQuerySpec` properties. • `intervalId` – Specify a value that corresponds to one of the historical intervals (`PerformanceManager.historicalInterval[].samplingPeriod`). • `startTime`/`endTime` – If specified, use time values that are not within the last 30 minutes of the current time. If you do not specify a starting time, the Server will return values starting with the earliest data. If you do not specify an end time, the Server will return values that include the latest data.
Summary	When you call the `QueryPerf` method and specify a performance interval (`PerfQuerySpec.intervalId`) that does not match one of the historical intervals (`PerformanceManager.historicalInterval[].samplingPeriod`), the Server will attempt to summarize the stored data for the specified interval. In this case, the Server may return values that are different from the values that were stored for the historical intervals.

Performance Counter Example (QueryPerf)
The following code fragments are part of an example that uses the `PerformanceManager.QueryPerf` method to obtain performance statistics for a virtual machine.

The example code in this section does not include server connection code and it does not show the code for obtaining the managed object reference for the virtual machine. See Client Applications for the Web Services API for an example of server connection code.

This example retrieves the following statistics:

- `disk.provisioned.LATEST` – virtual machine storage capacity.
- `mem.granted.AVERAGE` – amount of physical memory mapped for the virtual machine.
- `power.power.AVERAGE` – current power usage.

The example creates a query specification (`PerfQuerySpec`) to identify the data to be retrieved, calls the `QueryPerf` method, and prints out the retrieved performance data and corresponding performance counter metadata. The following sections describe the basic steps involved in retrieving performance statistics.

- Map the performance counters – Mapping Performance Counters (Counter Ids and Metadata).
- Create a performance query specification and call the `QueryPerf` method – Retrieving Statistics.
- Process the returned data – Handling Returned Performance Data.

Mapping Performance Counters (Counter Ids and Metadata)
Performance counters are represented by string names, for example `disk.provisioned.LATEST` or `mem.granted.AVERAGE`. A vSphere server tracks performance counters by using system-generated counter IDs. When you create a performance query, you use counter IDs to specify the statistics to be retrieved, so it is useful to map the names to IDs.

The example must specify counter IDs in the calls to `QueryPerf`, and it will use performance counter metadata when it prints information about the returned data. To obtain performance counter IDs and the corresponding performance counter metadata, the example creates two hash maps. This example maps the entire set of performance counters to support retrieval of any counter.

HashMap Declarations

The following code fragment declares two hash maps.

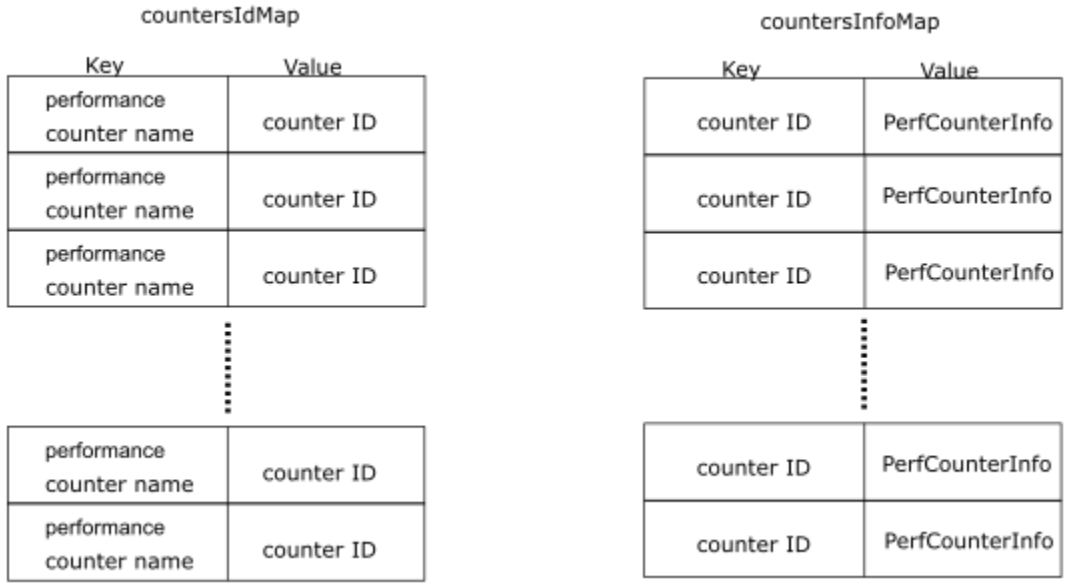
- `countersIdMap` – Uses full counter names to index performance counter IDs. A full counter name is the combination of counter group, name, and rollup type. The example uses this map to obtain counter IDs when it builds the performance query specification.
- `countersInfoMap` – Uses performance counter IDs to index `PerformanceCounterInfo` data objects. The example uses this map to obtain metadata when it prints the returned performance data.

```
/*
 * Map of counter IDs indexed by counter name.
 * The full counter name is the hash key - group.name.ROLLUP-TYPE.
 */
private static HashMap<String, Integer> countersIdMap = new HashMap<String, Integer>();

/*
 * Map of performance counter data (PerfCounterInfo) indexed by counter ID
 * (PerfCounterInfo.key property).
 */
private static HashMap<Integer, PerfCounterInfo> countersInfoMap =
    new HashMap<Integer, PerfCounterInfo>();
```

The following figure shows a representation of the hash maps.

Figure 635: Performance Counter Hash Maps



Creating the Map

The example uses the Property Collector to retrieve the array of performance counters (`PerfCounterInfo`) known to the vCentervCenter (`PerformanceManager.perfCounter[]`). It then uses the data to create the maps. The code fragment uses the variable `apiMethods`, which is a `VimPortType` object that provides access to the vSphere API methods. For information about the `VimPortType` object, see Overview of a Java Sample Application for the Web Services SDK.

The following code fragment performs these steps:

1. Create an `ObjectSpec` to define the property collector context. This example specifies the Performance Manager.
2. Create a `PropertySpec` to identify the property to be retrieved. This example retrieves the `perfCounter` property, which is an array of `PerfCounterInfo` objects.
3. Create a `PropertyFilterSpec` for the call to the `PropertyCollector`. The `PropertyFilterSpec` creates the association between the `ObjectSpec` and `PropertySpec` for the operation.
4. Call the `PropertyCollector.RetrievePropertiesEx` method. This method blocks until the server returns the requested property data.
5. Cast the returned `xsd:anyType` value into the array of `PerfCounterInfo` objects.
6. Cycle through the returned array and load the maps. The counter-name to counter-ID map uses a fully qualified counter name. The qualified name is a path consisting of counter group, counter name, and rollup type – *`group.counter.ROLLUP-TYPE`* The rollup type must be coded in uppercase letters. Examples of qualified names are `disk.provisioned.LATEST` and `mem.granted.AVERAGE`.

```
/*
 * Create an object spec to define the context to retrieve the PerformanceManager property.
 */
ObjectSpec oSpec = new ObjectSpec();
oSpec.setObj(performanceMgrRef);

/*
 * Specify the property for retrieval
 * (PerformanceManager.perfCounter is the list of counters the vCenter is aware of.)
 */
PropertySpec pSpec = new PropertySpec();
pSpec.setType("PerformanceManager");
pSpec.getPathSet().add("perfCounter");

/*
 * Create a PropertyFilterSpec and add the object and property specs to it.
 */
PropertyFilterSpec fSpec = new PropertyFilterSpec();
fSpec.getObjectSet().add(oSpec);
fSpec.getPropSet().add(pSpec);

/*
 * Create a list for the filter and add the spec to it.
 */
List<PropertyFilterSpec> fSpecList = new ArrayList<PropertyFilterSpec>();
fSpecList.add(fSpec);

/*
 * Get the performance counters from the server.
 */
RetrieveOptions ro = new RetrieveOptions();
RetrieveResult props = apiMethods.retrievePropertiesEx(pCollectorRef,fSpecList,ro);

/*
 * Turn the retrieved results into an array of PerfCounterInfo.
 */
List<PerfCounterInfo> perfCounters = new ArrayList<PerfCounterInfo>();
if (props != null) {
   for (ObjectContent oc : props.getObjects()) {
      List<DynamicProperty> dps = oc.getPropSet();
      if (dps != null) {
         for (DynamicProperty dp : dps) {
            /*
```

```
/*
 * Cycle through the PerfCounterInfo objects and load the maps.
 */
             *  DynamicProperty.val is an xsd:anyType value to be cast
             *  to an ArrayOfPerfCounterInfo and assigned to a List<PerfCounterInfo>.
             */
            perfCounters = ((ArrayOfPerfCounterInfo)dp.getVal()).getPerfCounterInfo();
         }
      }
   }
}
for(PerfCounterInfo perfCounter : perfCounters) {
   Integer counterId = new Integer(perfCounter.getKey());

   /*
    * This map uses the counter ID to index performance counter metadata.
    */
   countersInfoMap.put(counterId, perfCounter);

   /*
    * Obtain the name components and construct the full counter name,
    * for example – power.power.AVERAGE.
    * This map uses the full counter name to index counter IDs.
    */
   String counterGroup = perfCounter.getGroupInfo().getKey();
   String counterName = perfCounter.getNameInfo().getKey();
   String counterRollupType = perfCounter.getRollupType().toString();
   String fullCounterName = counterGroup + "." + counterName + "." + counterRollupType;

   /*
    * Store the counter ID in a map indexed by the full counter name.
    */
   countersIdMap.put(fullCounterName, counterId);

}
```

Retrieving Statistics

The following code fragment calls the `QueryPerf` method to retrieve statistics. It performs these tasks:

1. Create a list of qualified performance counter names for retrieval. The name is a path consisting of *group-name.counter-name.ROLLUP-TYPE*, for example `mem.granted.AVERAGE`. The rollup type must be coded in uppercase letters to match the character case of the rollup type in the performance counter metadata (`PerfCounterInfo.rollupType`). See the *vSphere API Reference* for tables of available counters. The *vSphere API Reference* page for the `PerformanceManager` managed object contains links to the tables.
2. Create a list of `PerfMetricId` objects, one for each counter to be retrieved. The metric ID is a combination of the counter ID and the instance. To fill in the `PerfMetricId` properties, the example does the following:
   - Use the `countersIdMap` to translate a full counter name into a counter ID.
   - Specify an asterisk (*) for the `PerfMetricId.instance` property. The asterisk is the system-defined instance specification for combined instance and rollup retrieval.
3. Build a query specification for the method call. This query specifies the following:
   - Virtual machine for which performance data is being retrieved (`entityMor`);
   - Interval ID of 300 to collect 5-minute rollup data.
   - Comma-separated value (CSV) format for the retrieved data.

4. Call the `QueryPerf` method.

```
/*
 * Use <group>.<name>.<ROLLUP-TYPE> path specification to identify counters.
 */
String[] counterNames = new String[] {"disk.provisioned.LATEST",
                                      "mem.granted.AVERAGE",
                                      "power.power.AVERAGE"};

/*
 * Create the list of PerfMetricIds, one for each counter.
 */
List<PerfMetricId> perfMetricIds = new ArrayList<PerfMetricId>();
for(int i = 0; i < counterNames.length; i++) {
   /*
    * Create the PerfMetricId object for the counterName.
    * Use an asterisk to select all metrics associated with counterId (instances and rollup).
    */
   PerfMetricId metricId = new PerfMetricId();
   /* Get the ID for this counter. */
   metricId.setCounterId(countersIdMap.get(counterNames[i]));
   metricId.setInstance("*");
   perfMetricIds.add(metricId);
}

/*
 * Create the query specification for queryPerf().
 * Specify 5 minute rollup interval and CSV output format.
 */
int intervalId = 300;
PerfQuerySpec querySpecification = new PerfQuerySpec();
querySpecification.setEntity(
querySpecification.setIntervalId(intervalId);
querySpecification.setFormat("csv");
querySpecification.getMetricId().addAll(perfMetricIds);

List<PerfQuerySpec> pqsList = new ArrayList<PerfQuerySpec>();
pqsList.add(querySpecification);

/*
 * Call queryPerf()
 *
 * QueryPerf() returns the statistics specified by the provided
 * PerfQuerySpec objects. When specified statistics are unavailable -
 * for example, when the counter doesn't exist on the target
 * ManagedEntity - QueryPerf() returns null for that counter.
 */
List<PerfEntityMetricBase> retrievedStats = apiMethods.queryPerf(performanceMgrRef, pqsList);
```

Performance Data Returned by a vSphere Server

The query methods return sampling information and performance data. The sampling information indicates the collection interval in seconds and the time that the data was collected. When you call performance query methods, you pass in query specifications (`PerfQuerySpec`) to identify the performance data to be retrieved. To indicate the format of the output data, specify either `"normal"` or `"csv"` for the `PerfQuerySpec.format` property.

The query methods return `PerfEntityMetricBase` objects which you must cast into the appropriate type that corresponds to the `PerfQuerySpec.format` value specified in the call to the method.

- The `QueryPerf` method returns a list of `PerfEntityMetricBase` objects.

- The `QueryPerfComposite` method returns a `PerfCompositeMetric` object, which contains `PerfEntityMetricBase` objects.

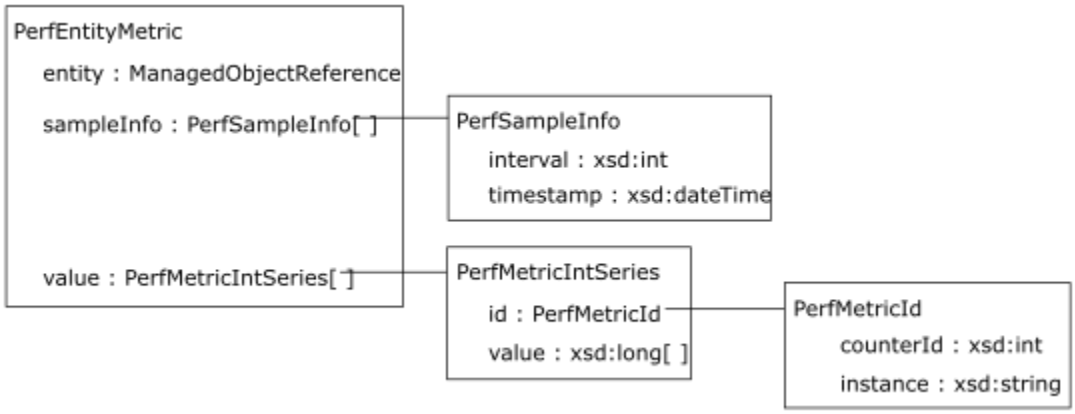
Normal Output Format

When you specify `"normal"` format, you must cast the returned `PerfEntityMetricBase` objects into `PerfEntityMetric` objects. Each `PerfEntityMetric` object contains the following properties:

- `entity` – Reference to the performance provider.
- `sampleInfo` – Array of sample information (`PerfSampleInfo` data objects), encoded as `xsd:int` and `xsd:dateTime` values.
- `value` – Array of data values (`PerfMetricIntSeries` data objects). Each object in the array contains the following properties:
  - `id` – Performance metric ID that identifies the counter instance.
  - `value` – Array of integers that corresponds to the array of sample information (`PerfEntityMetric.sampleInfo`).

The following figure shows a representation of the data object hierarchy returned by the query methods for normal format.

Figure 636: PerfEntityMetric Object Hierarchy



CSV Output Format

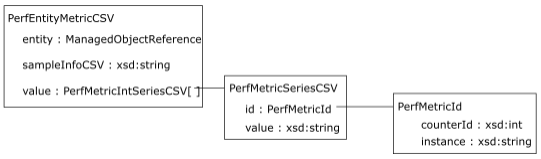
When you specify `"csv"` format, you must cast the returned `PerfEntityMetricBase` objects into `PerfEntityMetricCSV` objects. Both the sampling information and the collected data are encoded as comma-separated values suitable for display in tabular format.

The `PerfEntityMetricCSV` object contains the following properties:

- `entity` – Reference to the performance provider.
- `sampleInfoCSV` – String containing a set of interval and date-time values. The property contains string representations of `PerfSampleInfo xsd:int` and `xsd:dateTime` values. The string values are encoded in the following CSV format: *interval1*, *date1*, *interval2*, *date2*
- value – Array of data values (`PerfMetricSeriesCSV` data objects). Each object in the array contains the following properties:
  - id – Performance metric ID that identifies the counter instance.
  - value – Set of sample values in CSV format, corresponding to the list of sample information (`PerfEntityMetricCSV.sampleInfoCSV`).

The following figure shows a representation of the data object hierarchy returned by the query methods for CSV format.

Figure 637: PerfEntityMetricCSV Object Hierarchy



Handling Returned Performance Data

The following code fragment prints out the returned performance data. This example uses CSV formatted data. The code fragment performs these tasks:

- Loop through the list of `PerfEntityMetricBase` objects returned by the `QueryPerf` method (`retrievedStats`).
  - Cast the `PerfEntityMetricBase` object to a `PerfEntityMetricCSV` object to handle the CSV output specified in the `PerfQuerySpec`.
  - Retrieve the sampled values.
  - Retrieve the interval information (`csvTimeInfoAboutStats`). The `sampleInfoCSV` string (`PerfEntityMetricCSV.sampleInfoCSV`) is `PerfSampleInfo` data formatted as interval,time pairs separated by commas – `interval-1,time-1,interval-2,time-2`. The list of pairs embedded in the string corresponds to the list of sampled values (`PerfEntityMetricCSV.value[]`).
  - Print the time and interval information.
  - Loop through the sampled values (`metricsValues`).
    - Use the counter metadata to print out identifying information about the counter along with the returned sampled value for the counter.
    - Use the `countersInfoMap` to translate the counter ID returned in the `PerfMetricSeriesCSV` object into the corresponding `PerfCounterInfo` object.

```
/*
 * Cycle through the PerfEntityMetricBase objects. Each object contains
 * a set of statistics for a single ManagedEntity.
 */
for(PerfEntityMetricBase singleEntityPerfStats : retrievedStats) {

  /*
   * Cast the base type (PerfEntityMetricBase) to the csv-specific sub-class.
   */
  PerfEntityMetricCSV entityStatsCsv = (PerfEntityMetricCSV)singleEntityPerfStats;

  /* Retrieve the list of sampled values. */
  List<PerfMetricSeriesCSV> metricsValues = entityStatsCsv.getValue();

  if(metricsValues.isEmpty()) {
    System.out.println("No stats retrieved. " +
                       "Check whether the virtual machine is powered on.");
    throw new Exception();
  }

  /*
   * Retrieve time interval information (PerfEntityMetricCSV.sampleInfoCSV).
   */
  String csvTimeInfoAboutStats = entityStatsCsv.getSampleInfoCSV();

  /* Print the time and interval information. */
  System.out.println("Collection: interval (seconds),time (yyyy-mm-ddThh:mm:ssZ)");
  System.out.println(csvTimeInfoAboutStats);

  /*
```

```
     * Cycle through the PerfMetricSeriesCSV objects. Each object contains
     * statistics for a single counter on the ManagedEntity.
     */
    for(PerfMetricSeriesCSV csv : metricsValues) {
      /*
       * Use the counterId to obtain the associated PerfCounterInfo object
       */
      PerfCounterInfo pci = countersInfoMap.get(csv.getId().getCounterId());

      /* Print out the metadata for the counter. */
      System.out.println("---------------------------------------");
      System.out.println(pci.getGroupInfo().getKey() + "."
                         + pci.getNameInfo().getKey() + "."
                         + pci.getRollupType() + " - "
                         + pci.getUnitInfo().getKey());
      System.out.println("Instance: "+csv.getId().getInstance());
      System.out.println("Values: " + csv.getValue());
    }
  }
```

Large-Scale Performance Data Retrieval

The example described in the previous sections shows how to retrieve performance data for a single entity. When you design your application to retrieve performance data on a large scale, take the following information into consideration for more efficient processing.

- Use CSV formatted output. CSV format provides a more compact representation of the output data which can save on meta-data overhead.
- Create query specifications to reference a set of vSphere entities.
  - Using one `QueryPerf` method call per entity is not efficient.
  - Using a single call to `QueryPerf` to retrieve all of the performance data is not efficient.
  - As a general rule, specify between 10 and 50 entities in a single call to the `QueryPerf` method. This is a general recommendation because your system configuration may impose different constraints.
- Do not retrieve statistics more frequently than they are refreshed. For example, when you retrieve 20-second interval data, the data will not change until the next 20-second data collection event.
- Use `QueryAvailablePerfMetric` only when you intend to send a query for a specific counter using a specific performance interval. The method will return `PerfMetricId` objects that you can use for the query.

  In all other cases, create the `PerfMetricId` objects for the query.
  - For the `counterId` property, use the counter IDs from the `PerformanceManager` counter list (`PerformanceManager.perfCounter[].key`).
  - For the instance property, specify an asterisk (“*”) to retrieve instance and aggregate data or a zero-length string (““) to retrieve aggregate data only.

Using the QueryPerf Method as a Raw Data Feed

The `QueryPerf` method can operate as a raw data feed that bypasses the vCenter database and instead retrieves performance data from an ESX host. You can use a raw data feed to obtain real-time instance data associated with 20-second interval collection and aggregate data associated with the 5-minute intervals.

You can use a raw data feed on vCenter 2.5 and later.

Performance Interval	Description
20-second	ESX servers collect data for each performance counter every 20 seconds and maintain that data for an hour. When you specify a 20-second interval in the query specification for the `QueryPerf` method (`PerfQuerySpec.intervalId`), the method operates as a raw data feed. The Server ignores the historical interval collection levels and retrieves data for all of the requested counters from the ESX servers. When you send a query for 20-second instance data, the server returns the most recent data collected for the 20-second interval. The server does not perform additional, unscheduled data collection to satisfy the query.
5-minute	ESX servers aggregate performance data according to the system-defined performance interval which specifies data collection every 300 seconds. To use a raw data feed for this data, specify the following `PerfQuerySpec` properties in the call to the `QueryPerf` method. • `intervalId` – Specify 300 seconds to match the system-defined performance interval. • `startTime`/`endTime` – Specify time values within the last 30 minutes of the current time. The `QueryPerf` method checks the performance interval collection level on the vCenter. The method returns aggregated statistics for performance counters that specify a collection level (`PerfCounterInfo.level`) at or below the vCenter performance interval for the 300 second sampling period (`PerfInterval.level`). For example, if the vCenter performance interval is set to level one, and your query specification requests only performance counters that specify level four, the `QueryPerf` method will not return any data.

Comparison of Query Methods

The following table presents a comparison of performance query methods.

Method	Notes
QueryPerf	• Specify an array of `PerfQuerySpec` objects. • An unset `PerfQuerySpec.metricId` property produces results for all counters defined for `PerfQuerySpec.entity`. • `PerfQuerySpec.maxSample` is ignored for historical statistics. You can use this method to retrieve historical statistics; you can also use it as a raw data feed. For information about retrieving the raw data collected on ESX servers, see Using the QueryPerf Method as a Raw Data Feed.
QueryPerfComposite	• Method works only at the host level. You can use a single call to the `QueryPerfComposite` method to retrieve performance data for a host and its virtual machines. • Specify a single `PerfQuerySpec` object. • You must specify a list of performance metrics to identify the data to be retrieved (`PerfQuerySpec.metricId`). • You cannot specify `PerfQuerySpec.maxSample`. This method is designed for efficient client-server communications. `QueryPerfComposite` usually generates less network traffic than `QueryPerf` because it returns a large-grained object, a `PerfCompositeMetric` data object, that contains all the data.

Retrieving Summary Performance Data

You can obtain near real-time summary information about performance or utilization without using the `PerformanceManager` methods. vSphere servers maintain "quick stats" data objects for hosts (`HostListSummaryQuickStats`), virtual machines (`VirtualMachineQuickStats`), and resource pools (`ResourcePoolQuickStats`). For more information about these objects, see the *vSphere API Reference*.

Performance Counter Metadata

Performance counters are organized by groups of system resources. Examples of performance counter groups are memory, CPU, and disk. The counter groups and specific counters used on any vSphere server depend on the server configuration. The *vSphere API Reference* contains a table for each counter group. The table includes the counter name, type of statistics being collected, unit of measurement, level, and so on. The *vSphere API Reference* page for the `PerformanceManager` managed object contains links to the tables.

PerfCounterInfo

The `PerformanceManager.perfCounter` property is an array of `PerfCounterInfo` data objects. Each object provides metadata for the collected data. A `PerfCounterInfo` object has a unique key, the counter ID. The actual performance data collected at runtime are identified by this counter ID. The following table lists the `PerfCounterInfo` properties.

Property	Description
`groupInfo`	Name of the resource group to which this counter belongs, such as disk, cpu, or memory.
`key`	Unique integer that identifies the counter. Also called the counter ID. The value is unique and it is not static—it might, for example, change between system reboots. The counter key on an ESX system might not be the same as the counter key for the same counter on the vCenter system managing the ESX system. However, the system maps the keys from ESX to vCenter systems automatically.
`level`	Number from 1 to 4 that identifies the level at which data values for this counter are aggregated.
`nameInfo`	Descriptive name for the counter. The name component of a fully qualified counter name, for example `"granted"` is the `nameInfo` property for the `mem.granted.AVERAGE` counter.
`rollupType`	Indicates how multiple samples of a counter are transformed into a single statistical value. Examples of rollup types are average, summation, and minimum. No conversion of values occurs for counters that specify absolute values, such as the total number of seconds that the system has been running continuously since startup. The `PerfSummaryType` is an enumeration containing valid constants for this property.
`statsType`	Type of statistical data that the value represents over the course of the interval, such as an average, a rate, the minimum value, and so on. The `PerfStatsType` is an enumeration containing valid constants for this property.
`unitInfo`	Unit of measure, such as megahertz, kilobytes, kilobytes per second, and so on. The `ElementDescription`'s `key` property is populated using one of the constants available in the `PerformanceManagerUnit` enumeration.

Performance Intervals

The `PerformanceManager` defines performance intervals which specify the period of time between collection events, how much data will be collected, and how long the collected data will be saved.

- An ESX server has a built-in performance interval that produces discrete data values from counter instances sampled every 20 seconds. The server will maintain this instance data for one hour.
- Additional data collection is specified by historical performance intervals which produce data aggregated from counter instances according to the individual intervals.

The `PerformanceManager.historicalInterval` property is an array of `PerfInterval` objects. The following table lists the `PerfInterval` properties.

Property	Description
samplingPeriod	Number of seconds for the interval. You can modify this property on a vCenter only.
length	Period of time for which the server will save the data that it collects. You can modify this property on a vCenter only.
level	Level at which the Server collects data. The interval level corresponds to the performance counter level (`PerfCounterInfo.level`). The Server will collect data for all counters with levels that match `PerfInterval.level`, and for all counters with levels lower than `PerfInterval.level`. You can modify this property on a vCenter only.
enable	Enable/disable performance data collection. You can modify this property on a vCenter only.
key	Unique identifier for the interval. You cannot modify this property.
name	Label for the historical interval; one of the following strings: • `"Past Day"` • `"Past Week"` • `"Past Month"` • `"Past Year"` The `PerformanceManager` uses the `samplingPeriod`, `level`, and `length` properties to determine its collection behavior. It does not interpret the name string. You cannot modify this property.

ESX Server Performance Intervals

An ESX server collects performance data for each performance counter every 20 seconds. The `PerformanceManager.historicalInterval` array for an ESX Server contains a single, readonly `PerfInterval` object that specifies rollup data collection every 5 minutes. You cannot retrieve 5-minute rollup data from an ESX Server directly. You can use a vCenter connection to obtain 5-minute rollup data for an ESX Server. The following table shows the historical interval property values on an ESX server. You cannot modify this performance interval.

Property	Value	Description
`key`	`1`	Numeric identifier for the `PerfInterval`.
`name`	`PastDay`	Name of the `PerfInterval`.
`samplingPeriod`	`300`	Time interval between data sampling events.
`length`	`129600`	Number of seconds that statistics associated with the interval are kept by the vCenter.
`enabled`	`true`	This `PerfInterval` is enabled on the system.
`level`	`null`	Statistics collection level. For an ESX system, this property is `null`. The `PerfInterval` object on an ESX system defines the baseline interval.

vCenter Performance Intervals

A vCenter system aggregates performance data from all ESX systems that it manages. The amount of data aggregated depends on the level setting configured for the vCenter. The level settings are reflected in the `PerformanceManager.historicalInterval` property for the vCenter system. `historicalInterval` is an array of `PerfInterval` data objects that define four different level settings, 1 through 4.

The following table lists the default values for the performance intervals on a vCenter system.

Key	Name	Sampling Period	Length	Enabled	Level
1	Past Day	300	86400	TRUE	1
2	Past Week	1800	604800	TRUE	1
3	Past Month	7200	2592000	TRUE	1
4	Past Year	86400	31536000	TRUE	1

By default, the collection level is set to 1 for each of the four intervals. Using the default level, a vCenter will collect data for all performance counters that specify collection level 1. Using the default length value, a vCenter will save collection data for the following time periods:

- 5-minute samples for the past day
- 30-minute samples for the past week
- 2-hour samples for the past month
- 1-day samples for the past year

Data older than a year is purged from the vCenter database.

vSphere Performance and Data Storage

The following sections provide information about modifying the operation of the `PerformanceManager` and vSphere Server performance data collection and storage.

- Modifying Historical Intervals
- Modifying Performance Counter Collection Levels

Modifying Historical Intervals

Changes to a vCenter performance interval are global and apply to all entities in the system. VMware recommends that you do not modify the historical intervals. The `PerfInterval` data objects in the `PerformanceManager.historicalInterval` array are related. Modifications to a performance interval affects the entire system and may cause problems.

If you must modify a performance interval, use the `PerformanceManager.UpdatePerfInterval` method and follow these guidelines.

- Performance data retention time (`PerfInterval.length`) must be a multiple of the collection interval (`PerfInterval.samplingPeriod`).
- Performance data retention length must increase in each interval compared to its predecessor. The `PerfInterval.length` value for each successive performance interval must be greater than the length property for the previous interval in the historical interval array.
- You cannot modify the value of the `PerfInterval.samplingPeriod` property on ESX systems.

Modifying Performance Counter Collection Levels

The `PerformanceManager` provides the `UpdateCounterLevelMapping` method to change the collection level for individual performance counters (`PerfCounterInfo.level`). Consider carefully the performance and storage consequences of using the `UpdateCounterLevelMapping` method. If you use this method, you may cause a significant increase in data collection and storage, along with a corresponding decrease in performance. vCenter performance and database storage requirements depend on the collection levels defined for the performance intervals (`PerformanceManager.historicalInterval`) and the collection levels specified for individual performance counters (`PerfCounterInfo.level`).

Performance Counter Data Collection

vSphere defines four levels of data collection for performance counters. Each performance counter specifies a level for collection. The historical performance intervals (`PerformanceManger.historicalInterval`) define the sampling period and length for a particular collection level.

The amount of data collected for a performance counter depends on the performance interval and on the type of entity for which the counter is defined. For example, a datastore counter such as `datastoreIops` (the aggregate number of IO operations on the datastore) will generate a data set that corresponds to the number of datastores on a host. If a vCenter manages a large number of hosts with a large number of datastores, the vCenter will collect a large amount of data.

There are other counters for which the vCenter collects a relatively smaller amount of data. For example, memory counters are collected as a single counter per virtual machine and a single counter per host.

Performance Counter Data Storage

The performance interval collection level (`PerfInterval.level`) defines the set of counters for which the vCenter stores performance data. The Server will store data for counters at the specified level and for counters at all lower levels.

By default, all the performance intervals specify collection level one. Using these defaults, the vCenter stores performance counter data in the vCenter database for all counters that specify collection level one. It does not store data for counters that specify collection levels two through four.

Performance Manager Method Interaction

You can use the `UpdateCounterLevelMapping` method to change the collection level for individual counters. You can also use the `UpdatePerfLevel` method to change the collection level for the system-defined performance intervals. These methods can cause a significant increase in the amount of data collected and stored in the vCenter database.

- By default the system-defined performance intervals use collection level one, storing data for all counters that specify collection level one. If you use the `UpdateCounterLevelMapping` method to change the collection level of performance counters to level one, you will increase the amount of stored performance data.
- If you use the `UpdatePerfLevel` method to increase the collection level for the system-defined performance intervals, you will increase the amount of stored performance data.

To restore counter levels to default settings use the `ResetCounterLevelMapping` method.

vSphere Client Management of Performance Statistics

The vSphere Client displays the Performance Manager historical interval collection levels in the vCenter management statistics display. The vSphere Client also displays an estimate of the amount of storage that is required for data collection at the displayed levels. If individual counter levels are modified through the vSphere API (the `UpdateCounterLevelMapping` method), the vSphere Client will show a modified estimate. However, the vSphere Client cannot detect that the method has been called and it cannot display the current levels for individual counters. If you see a significantly increased estimate for storage, be aware that someone may have used the vSphere API to modify data collection.

Sample Code for Performance

The following table lists the sample applications included with the vSphere Web Services SDK that demonstrate some of the topics discussed in this chapter.

Java (SDK\vsphere-ws\java\JAX-WS\samples\com\vmware\performance)
`Basics.java`
`History.java`
`PrintCounters.java`
`RealTime.java`
`VITop.java`
`VIUsage.java`

Diagnostics and Troubleshooting

vSphere includes several logs, which you can access and customize. You can also use the `DiagnosticManager` service interface for troubleshooting.

Troubleshooting Best Practices

Approach troubleshooting and problem-solving systematically, and take notes so you can trace your steps. Follow these guidelines to resolve issues with your client application.

- Do not change more than one thing at a time, and document each change and its result. Try to isolate the problem: Does it seem to be local, to the client? An error message generated from the server? A network problem between client and server?
- Use the logging facilities for your programming language to capture runtime information for the client application.
- Use the following VMware tools for analysis and to facilitate debugging.
  - vSphere Web Services API. The `DiagnosticManager` service interface allows you to obtain information from the server log files, and to create a diagnostic bundle that contains all system log files and all server configuration information. The vSphere Client and the MOB provide graphical and Web based access to the `DiagnosticManager`. `PerformanceManager` supports exploration of bottlenecks. See vSphere Performance.
  - Managed Object Browser (MOB). The MOB provides direct access to live runtime server-side objects. You can use the MOB to explore the object hierarchy, obtain property values, and invoke methods. See Managed Object Browser.
  - VMware vSphere Client GUI. The vSphere Client allows you to examine log files for ESX , vCenter, and virtual machines, and to change log level settings. Use vSphere Client menu commands to create reports that summarize configuration information, performance, and other details, and to export diagnostic bundles. The vSphere Client maintains its own local log files.

Overview of Configuration Files and Log Files

ESX and vCenter configuration files control the behavior of the system. Most configuration file settings are set during installation, but can be modified after installation. Log files capture messages generated by the kernel and different subsystems and services. ESX and vCenter services maintain separate log files. The following table lists log files or reports, their locations and associated configuration files.

Description	Log Location	Filename or Names	Configuration File
ESX service log	`/var/log/vmware/`	`hostd.log [hostd-0.log, ...hostd-9.log]`	See ConfigStore
vCenter agent log	`/var/log/vmware/vpx/`	`vpxa.log`	See ConfigStore
Virtual machine kernel core file	`/root/`	`vmkernel-core.<date> vmkernel-log.<date>`	See ConfigStore
syslogd log	`/var/log/`	`messages [messages.1,... messages.4]`	See ConfigStore
Service console availability report	`/var/log/`	`vmkernel [vmkernel.1, ... vmkernel.8]`	See ConfigStore
VMkernel messages, alerts, and availability reports	`/var/log/vmkernel`		See ConfigStore
VMkernel warning	`/var/log/`	`vmkwarning [vmkwarning.1 ... 4 for history]`	See ConfigStore
Virtual machine log file	`vmfs/volume/<vm_name>`	`vmware.log`	`<vm_name>/<vm_name>.vmx`

For developers, the following files are most relevant:

- `hostd.log` – Host daemon log, see ESX Log File. Can be used as a SOAP monitor when set to trivia log level as in Generating Logs.
- `vpxa.log` – Agent log file found on each managed ESX system.
- `vmware.log` – Virtual machine log. See Virtual Machine Log Files.

Note:
Many configuration settings can no longer be edited directly with a web browser. To modify configuration settings in ConfigStore, use `/bin/configstorecli` or `esxcli`.

In addition to viewing log files in real time you can also generate reports and complete diagnostic bundles. See Generating Diagnostic Bundles.

ESX Log File

The ESX log (`hostd.log`) captures information of varying specificity and detail, depending on the log level. Each request to the server is logged.

You can view the file using the vSphere Client. The raw text form of an ESX (`hostd`) log file is shown in Sample ESX Log (hostd.log) Data.

Sample ESX Log (`hostd.log`) Data

```
...
[2008-05-07 09:50:04.857 'SOAP' 2260 trivia] Received soap response from
 [TCP:myservername.vmware.com:443]: GetInterfaceVersion
[2008-05-07 09:50:04.857 'ClientConnection' 2260 info] UFAD interface version is vmware-
converter-4.0.0
[2008-05-07 09:50:04.857 'SOAP' 2260 trivia] Sending soap request to
 [TCP:myservername.eng.vmware.com:443]: logout
[2008-05-07 09:50:04.857 'ProxySvc Req00588' 3136 trivia] Client HTTP stream read error
[2008-05-07 09:50:04.872 'ProxySvc Req00612' 3136 trivia] Request header:
POST /vmc/sdk HTTP/1.1
User-Agent: VMware-client
Content-Length: 435
Content-Type: text/xml; charset=utf-8
Cookie: vmware_soap_session="F127B435-56C7-4580-BAC4-3034DA1E67B6"; $Path=/
Host: myservername.vmware.com


[2008-05-07 09:50:04.872 'ProxySvc Req00588' 3816 trivia] Closed
[2008-05-07 09:50:08.450 'App' 3560 verbose] [VpxdHeartbeat] Invalid heartbeat from 10.17.218.46
[2008-05-07 09:50:10.013 'App' 3560 verbose] [VpxdHeartbeat] Queuing 10.17.218.45:829 (host-55)
[2008-05-07 09:50:10.013 'App' 1928 verbose] [HeartbeatHandler] 50208862-2752-d94c-2a73-
fa2ec9e38ecc:829 (host-55)
```

Virtual Machine Log Files

Each running virtual machine has its own log file, `vmware.log`, stored on the VMFS volume. By default, the log file is rotated whenever the virtual machine is powered on, but file rotation is configurable.

- ESX maintains six log files that rotate at each power-cycle (the default) or at a configured file size.
- ESX can be configured to maintain a specific number of log files. When the limit is reached, the oldest file is deleted.
- VMware recommends a log file size of 500 KB.
- Messages that are generated by VMware Tools are logged separately.

VMkernel Availability Report

```
Availability Report for <servername>
Feb 27, 2008 - May  7, 2008

Availability: 99.949%
```

```
     Total time: 69 days, 15 hours
     Uptime: 69 days, 14 hours
     Downtime: 51 minutes

Note: Downtime is any time the system isn't capable of running
Virtual Machines.  This includes reboots, crashes, configuration and running linux

Downtime Analysis:
     0.1% (51 minutes) downtime caused by:
     13.1% (6 minutes) scheduled downtime
     86.9% (44 minutes) unscheduled downtime

Reasons for scheduled downtime:
     84.9% server rebooting (1 instance)
     9.4% VMkernel unloaded (1 instance)
     5.7% server booting (3 instances)

Reasons for unscheduled downtime:
100.0% unknown (powerfail / reset?) (1 instance)

Stats:
     Current uptime: 8 days, 11 hours
     Longest uptime: 61 days, 2 hours
     Shortest uptime: 38 minutes
     Average uptime: 23 days, 4 hours
     Longest downtime: 44 minutes
     Shortest downtime: 7 seconds
     Average downtime: 8 minutes
     Maximum VMs Sampled:  1
     Average VMs Sampled: 0.94

Server Information:   Number of CPUs:  4 logical 4 cores
      2 packages,  Intel(R) Xeon(R) CPU            5150  @ 2.66GHz
        Installed Memory:     2096416 kB
        Current Build:  78591
Report generated Wed May  7 04:02:04 PDT 2008
```

vCenter Log Files

vCenter log files are located by default in the Documents and Settings subdirectory of the Windows account used to install the software. For example:

```
C:\Documents and Settings\Administrator\Local Settings\Application Data\VMware\
```

Important:

VMware recommends creating a user account especially for vCenter installation.

By default, the log files are hidden files. See the procedure for your Windows operating system to make the files visible.

Modifying the Log Level to Obtain Detailed Information

The amount of information captured in the log files varies, depending on the level setting.

Log Level Setting	Description
None	Disables logging.
Error	Logging limited to error messages.

Log Level Setting	Description
Warning	Error messages plus warning messages are logged.
Info	Default setting on ESX and vCenter systems. Errors, warnings, plus informational messages about normal operations are logged. Acceptable for production environments.
Verbose	Can facilitate troubleshooting and debugging. Not recommended for production environments.
Trivia	Extended verbose logging. Provides complete detail, including content of all SOAP messages between client and server. Use for debugging and to facilitate client application development only. Not recommended for production environments.

For example, the `hostd` service running on ESX systems has a default log level setting of `info`. The vCenter logs are controlled by settings through the vSphere Client.

Setting the Log Level on ESX Systems
The default log level setting for the ESX Host Agent is `info`. If you run into issues during development, you can set the log level to `verbose`, or to `trivia` to obtain SOAP message content to use in debugging.

To change the log level for hostd on an ESX system, use the following steps.

1. Connect to the ESX system using the vSphere Client.
2. On the **Host** > **Manage** screen, select the **System** tab.
3. On the **System** tab screen, select **Advanced Settings**.
4. Scroll to find the `Config.HostAgent.log.level` setting.
   By default this setting is `info`.
5. Select the `Config.HostAgent.log.level` setting and click **Edit option**.
   A dialog box opens, where you can enter a new setting.
6. Use the drop-down menu to select a new setting, such as **Verbose**, and click **Save**.
   The **Recent Tasks** pane updates to confirm that the change completed.

After the service restarts, the new log level is in effect.

Generating Logs
If you are connected to ESX by SSH, you can use the `tail` command to explicitly create a log file that captures detail about actions that follow. For example, you can use the vSphere Client to create a new virtual machine and then use the content from the log as a model for how to create your own code.

To start the logging process and capture content to a file, use the following steps.

1. Navigate to the location of the `hostd.log` file:

   ```
   cd /var/log/vmware
   ```

2. Run the `tail` command, passing a filename in which to capture output:

   ```
   tail -f hostd.log > yourfilenamehere
   ```

3. Use the vSphere Client to perform whatever action you are having difficulty modeling in your own code. For example, create a new virtual machine and stop the `tail` process with Ctrl-C when the operation completes.

The file contains the SOAP message content and other log messages sent and received by `hostd` during the execution.

Setting the Log Level on vCenter Systems
To change log-level settings on vCenter, you must use the vSphere Client.

To set logging level for vCenter using the VMware vSphere Client, use the following steps.

1. Log in to the vSphere Client and connect to the vCenter instance.
2. Choose **Administration** and click **Server Settings** > **Logging Options**.
3. Choose **Trivia** from the pop-up menu and click **OK**.

Using DiagnosticManager

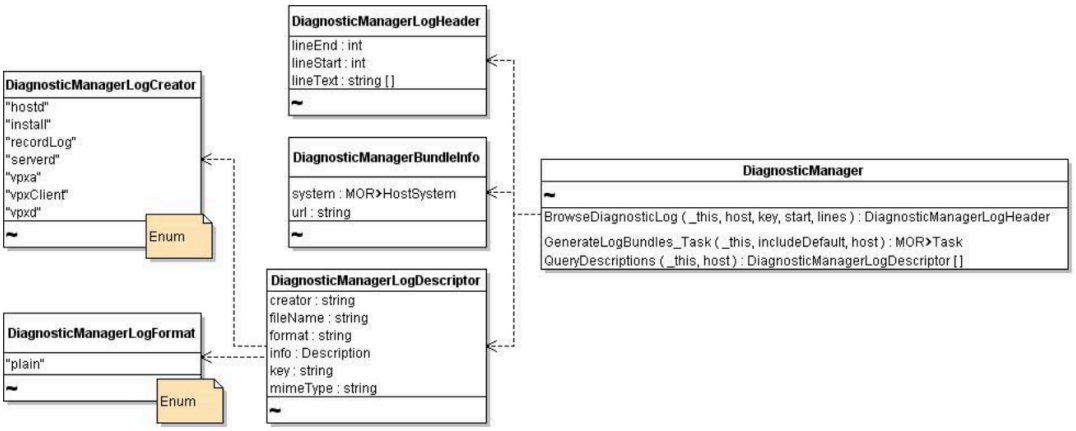
The vSphere API provides access to the `DiagnosticManager`, the service interface for obtaining information from the log files and for generating diagnostic bundles. The logs are populated based on configuration settings, such as `info`, `trivia`, and so on.

The `DiagnosticManager` is a managed object that works service-wide, rather than on a per-session basis. The `DiagnosticManager` has no properties, but provides operations for these tasks:

- Obtaining information about the logs and how they have been defined.
- Generating a diagnostic bundle that can be sent to VMware support for analysis.

Figure 638: DiagnosticManager Managed Object and Associated Data Objects shows a UML class diagram for `DiagnosticManager`, which is available on ESX and vCenter systems.

Figure 638: DiagnosticManager Managed Object and Associated Data Objects



As shown in Figure 638: DiagnosticManager Managed Object and Associated Data Objects, `DiagnosticManager` supports these methods:

- `BrowseDiagnosticLog`
- `GenerateLogBundleTask`
- `QueryDescriptions`

The `DiagnosticManagerLogDescriptor.creator` property contains the creator of the log, which is the system or subsystem that controls a specific log.

The `creator` value is populated from the `DiagnosticManagerLogCreator` enumeration. The following table lists all string values currently available from the `DiagnosticManagerLogCreator` enumeration that can populate the `creator` property of the `DiagnosticManagerLogDescriptor` data object.

Name	Description
`hostd`	Host daemon
`install`	Installation
`recordLog`	System record log
`serverd`	Host server agent
`vpxa`	vCenter agent
`vpxClient`	vSphere Client

Name	Description
vpxd	vCenter service

Creator	File Name	Format	Info.label	Info.summary	Key	Mime Type
hostd	/var/log/vmware/hostd.log	plain	ESX Log	ESX log in plain format	hostd	text/plain
hostd	/var/log/messages	plain	ESX Log	ESX log in plain format	messages	text/plain
hostd	/var/log/vmkernel	plain	ESX Log	ESX log in plain format	vmkernel	text/plain
hostd	/var/log/vmksummary.txt	plain	ESX Log	ESX log in plain format	vmksummary	text/plain
hostd	/var/log/vmkwarning	plain	ESX Log	ESX log in plain format	vmkwarning	text/plain
vpxa	/var/log/vmware/vpx/vpxa.log	plain	vCenter Agent Log	vCenter agent log in plain format	vpxa	text/plain

Using the MOB to Explore the DiagnosticManager

The Managed Object Browser (MOB) is a graphical interface that allows you to navigate the objects on a server and to invoke methods. You can access the `DiagnosticManager` using the MOB.

The following steps show how to explore `DiagnosticManager`.

1. Start the mob by typing the MOB URL (`https://hostname.yourcompany.com/mob`) into a Web browser.
2. In the `ServiceContent` data object, click the link (`ha-diagnosticmanager` or `DiagMgr`) in the `Value` column for the `diagnosticManager` property, to navigate to the `DiagnosticManager` for the system.
   - For ESX , `ha-diagnosticsmanager` is the managed object ID.
   - For vCenter, `DiagMgr` is typically the managed object ID.
3. Click the link to the reference to display the managed object reference to the `DiagnosticManager` in the MOB.

   `DiagnosticManager` provides three operations that allow you to obtain information about the descriptions currently available in the log file and log file content.

   Because `DiagnosticManager` can track multiple ESX systems, you can use the `QueryDescriptions` operation to return the names of keys used for all hosts. From this array, select the key for the host from which you want to obtain the log file.
4. On `QueryDescriptions`, click the **Invoke Method** link.

   The vCenter system returns the contents of the log file for the selected host as a string array for the `lineText` property of `DiagnosticManagerLogHeader`.

The string array returned through the MOB in this way is the content of the log file. The content contained in the log file is the same content that is available through the following other mechanisms:

- Displayed in the vSphere Client
- Included in a diagnostic bundle created through the `DiagnosticManager.GenerateLogBundles_Task` method.
- Available in the `hostd.log` file
- Returned to a client application that you write

For more information about using the DiagnosticManager, see Managed Object Browser.

Generating Diagnostic Bundles

Typically, customers create diagnostic bundles at the request of VMware technical support. Diagnostic bundles also allow developers to quickly obtain all configuration files and log files in a complete package.

The generated compressed files are packaged in a file having the following pattern:

```
<fqdn-hostname>-esxsupport-yyyy-mm-dd@hh-mm-ss.tgz
```

Export Diagnostic Data By Using the vSphere Client

You can use the vSphere Client interface to generate diagnostic bundles.

1. Connect to the ESX system using the vSphere Client.
2. On the **Host** > **Monitor** screen, select the **Logs** tab.

   The **Recent Tasks** pane updates to indicate that the system is generating a diagnostic bundle.

When the task completes, the vSphere Client opens a dialog box that allows you to download the bundle.

Managed Object Browser

The Managed Object Browser (MOB) is a graphical interface that allows you to navigate the objects on a server and to invoke methods. Any changes you make through the MOB take effect on the server.

This appendix explains how to use the MOB. The examples invoke `PerformanceManager` query methods to demonstrate how to pass primitive data types, arrays, and complex data types (data objects, including managed object references) using the MOB.

Using the MOB to Explore the Object Model

The Managed Object Browser, or MOB, is a Web-based server application available for all ESX and vCenter systems. The MOB lets you examine the objects that exist on the server and navigate through the hierarchy of live objects by clicking on links. The MOB populates the browser with actual runtime information, for example, the names of properties.

> **CAUTION:**
> Despite the word "browser" in its name, the MOB is not a read-only mechanism. The MOB allows you to make changes on the server by clicking the **InvokeMethod** link associated with methods.

Accessing the MOB

The MOB runs in a web browser and is accessed by using the fully-qualified domain name or IP address for the ESX or vCenter system.

To access the MOB, use the following steps.

1. Start a Web browser.
2. Enter the fully-qualified domain name (or the IP address) for the ESX or vCenter system:

   ```
   https://hostname.yourcompany.com/mob
   ```
3. Enter the user account and password for the system.

   If warning messages regarding the SSL certificate appear, you can disregard them and continue to log in to the MOB, if VMware is the certificate authority and you are not in a production environment.

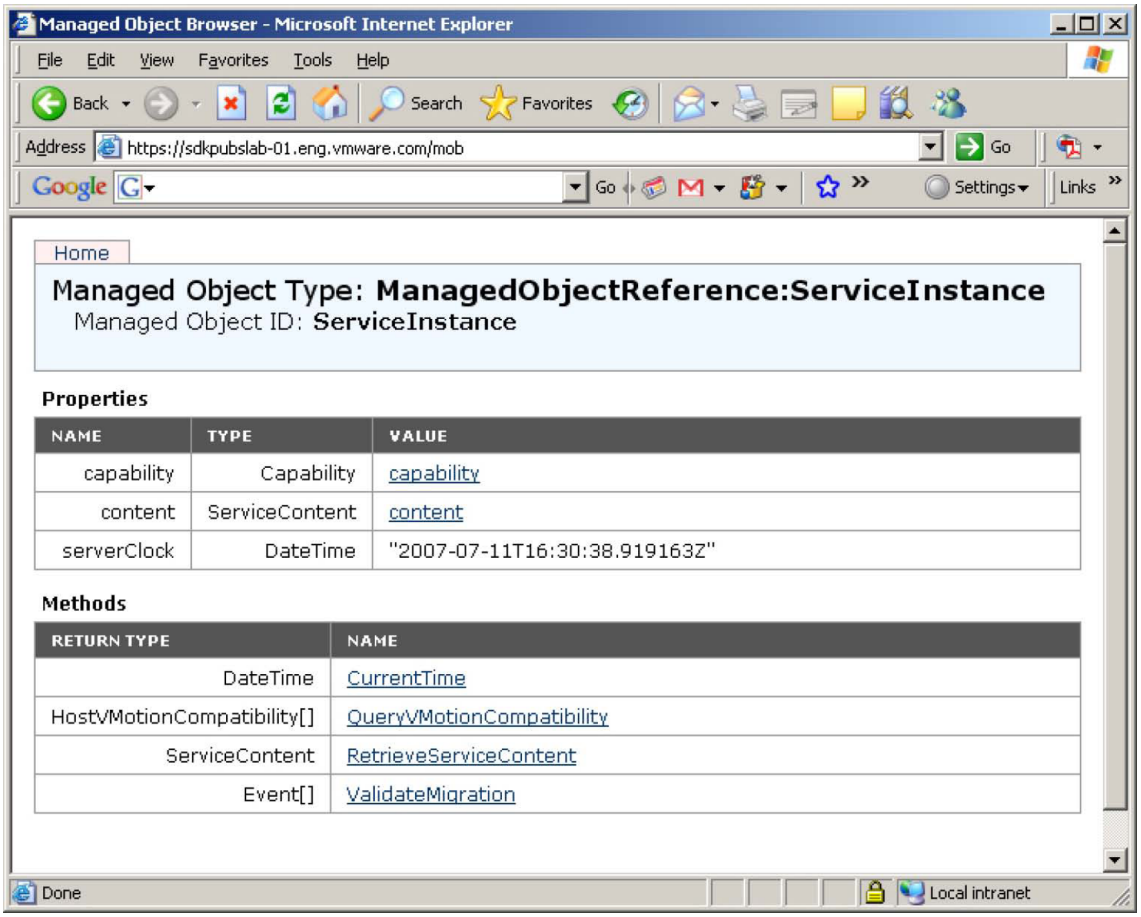
The MOB reveals the underlying structures of the object model. Seeing the structure in conjunction with the *API Reference Guide*, can help with understanding the model.

Using the MOB to Navigate the vSphere Object Model

Upon successful connection to the MOB, the browser displays the managed object reference for `ServiceInstance`. Client applications do not use managed objects directly, but interact with server-side managed objects by reference, using instances of the `ManagedObjectReference` data created for this purpose.

The page lists the properties and methods available through a `ServiceInstance` object. The `ServiceInstance` methods and properties provide access to the entire set of services and inventory objects available on the server. See Datacenter Inventory.

Figure 639: Managed Object Browser Connected to a vCenter



The MOB lets you examine the relationships among objects by looking at the properties and their values, and then drilling down into the objects. To explore the objects on the server, click the links in the **Value** column to navigate to the page that displays the object.

For example, to find out more about `ServiceContent`, click the **content** link to display the `ServiceContent` data object instance.

Using the MOB to Invoke Methods

You can use the MOB to invoke methods as follows:

1. In the display of the object in which the method lives, click the name of the method.

   A browser window displays information about the parameter name and type and allows you to specify parameter values.

2. Specify parameter values, using the method appropriate for the type, and click **Invoke Method**.

   The rest of this section discusses how to pass different types of parameters to the MOB.

Passing Primitive Datatypes to Method

vSphere Web Services SDK data types are defined in the WSDL using XML Schema markup. The primitive data types are specified using the `xsd` namespace. For example, a string value for a property is defined as data type `xsd:string`. Enter a primitive value in the MOB as plain text, without quotation marks or other markup. For example, to enter an integer value of 10, type `10` in the field.

To obtain information about the available performance counters at level 4 on the server, enter a `4` in the `level` field of the `PerformanceManager.QueryPerfCounterByLevel` method. (This method is available only on the vCenter `PerformanceManager` API, not from an ESX system.)

In response to the query, the array of `PerfCounterInfo` data objects and nested objects, with populated values from the server, displays in the Web browser.

Passing Arrays of Primitives to Methods

For an array, use the name of the parameter as the name of the property. For example, the `PerformanceManager.QueryPerfCounter` method requires an array of integers for the `counterId` parameter, as follows:

```
<counterId>58</counterId><counterId>65603</counterId><counterId>65604</counterId>
```

Even if you want to submit a single value for a single array element, you must wrap the parameter name around the value in this way.

Passing Complex Structures to Methods

For complex datatypes, enter the value as defined by the XML Schema in the WSDL. You can obtain the WSDL definition from the *vSphere API Reference* using the **Show WSDL type definition** links. Each data object type has an associated link.

Simple Content

The data object type `ManagedObjectReference` is one of the most commonly required parameters to be passed to the server. For example, the MOB for the `PerformanceManager.QueryPerfProviderSummary` method shows that the method requires a single parameter, the managed object reference (an instance of `ManagedObjectReference`) of the entity for which you want to obtain the `PerfProviderSummary` object.

Using the *vSphere API Reference* for `ManagedObjectReference` type, you can obtain the schema information from the **Show WSDL type definition** link at the bottom of the documentation page for `ManagedObjectReference`.

XML Schema Definition of ManagedObjectReference Data Object

```
<complexType xmlns="http://www.w3.org/2001/XMLSchema" xmlns:vim25="urn:vim25"
 name="ManagedObjectReference">
     <simpleContent>
          <extension base="xsd:string">
              <attribute name="type" type="xsd:string"/>
          </extension>
     </simpleContent>
</complexType>
```

XML Schema Definition of ManagedObjectReference Data Object shows that a managed object reference is defined as a `<SimpleContent>` element that consists of a string that specifies the attribute `type` with its associated value, also as string. Use this information to construct the appropriate structure by replacing `type` with the parameter name from the MOB, setting the value as needed, and submitting in the entry field of the MOB. (The value for the `Datacenter` is displayed in the MOB.)

```
<entity type="Datacenter">datacenter-21</entity>
```

Figure 640: Using the MOB to Pass Complex Types to a Method shows the result of using the definition listed in XML Schema Definition of ManagedObjectReference Data Object to specify the managed object reference for a target datacenter to the `PerformanceManager.QueryPerfProviderSummary` method.

Figure 640: Using the MOB to Pass Complex Types to a Method



As another example, one of the parameters required by the `VirtualMachine.CloneVM_Task` method is a `folder`. In this case, the parameter is defined as a managed object reference to a specific `Folder` object. Using the same definition shown in XML Schema Definition of ManagedObjectReference Data Object, the result is as follows:

```
<folder type="Folder">folder-87</folder>
```

Although both examples submit a `ManagedObjectReference` to the MOB, each is specific to the parameter name required by the method (`entity` type for `PerformanceManager.QueryPerfProviderSummary` method, `folder` type for the `VirtualMachine.CloneVM_Task` method).

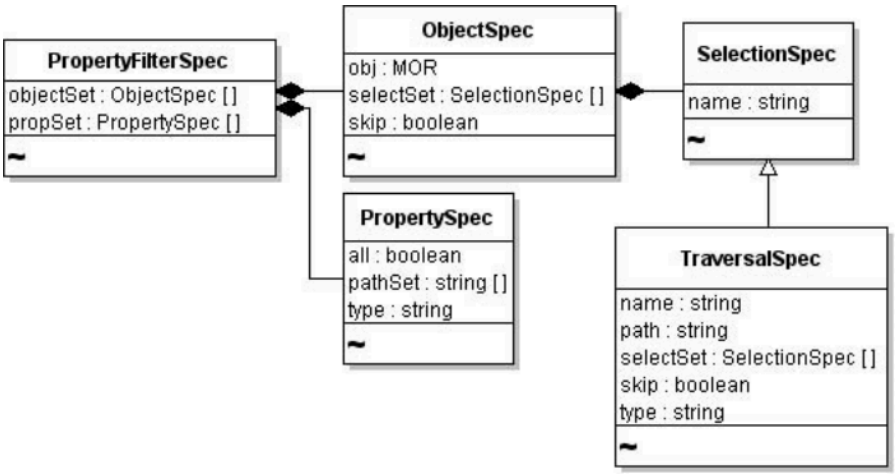
Complex Content

Many of the data objects required for method invocation consist of XML Schema elements defined as `<complexContent>` that can encompass many other elements.

For example, the `PropertyCollector.CreateFilter` method has a `spec` parameter that must be defined before method invocation. The `spec` parameter is defined as an instance of a `PropertyFilterSpec`.

Figure 641: PropertyFilterSpec and Associated Data Objects shows the relationships among several data objects that `PropertyFilterSpec` consists of.

Figure 641: PropertyFilterSpec and Associated Data Objects



To submit complex data structures such as this to the MOB, start by navigating the *vSphere API Reference*. Find the `PropertyFilterSpec` data object. Find the **Show WSDL type definition** link, and click it to display the XML Schema definition (see XML Schema Definition of PropertyFilterSpec Data Object Type).

XML Schema Definition of PropertyFilterSpec Data Object Type shows that the `PropertyFilterSpec` data object is a `<complexContent>` element that extends the `DynamicData` class with a sequence of two additional properties `propSet` (of type `PropertySpec)` and `objectSet` (of type `ObjectSpec`).

XML Schema Definition of PropertyFilterSpec Data Object Type

```
<complexType xmlns="http://www.w3.org/2001/XMLSchema" xmlns:vim25="urn:vim25"
 name="PropertyFilterSpec">
     <complexContent>
          <extension base="vim25:DynamicData">
              <sequence>
                  <element name="propSet" type="vim25:PropertySpec" maxOccurs="unbounded"/>
                  <element name="objectSet" type="vim25:ObjectSpec" maxOccurs="unbounded"/>
              </sequence>
          </extension>
     </complexContent>
</complexType>
```

Because both elements are defined as a sequence, they must exist in the order listed. To obtain the definitions of `propSet` and `objectSet`, you must navigate further into the *vSphere API Reference*. XML Schema

Extract for PropertySpec shows only the relevant parts of the XML Schema definition for `PropertySpec`. The `minOccurs="0"` attribute means that the element does not have to exist. The `maxOccurs="unbounded"` attribute means that the element can be populated as an array of any size. (When `minOccurs` is not set, but `maxOccurs` is set, the default for `minOccurs` defaults to 1, meaning one instance is required.)

XML Schema Extract for PropertySpec

```
<sequence>
     <element name="type" type="xsd:string"/><element name="all"
 type="xsd:boolean" minOccurs="0"/><element name="pathSet" type="xsd:string"
 minOccurs="0" maxOccurs="unbounded"/>
</sequence>
```

Navigate through the *vSphere API Reference* to the `ObjectSpec` definition. ObjectSpec Definition as XML Schema shows the excerpt.

ObjectSpec Definition as XML Schema

```
...
<sequence>
     <element name="obj" type="vim25:ManagedObjectReference"/>
     <element name="skip" type="xsd:boolean" minOccurs="0"/>
     <element name="selectSet" type="vim25:SelectionSpec" minOccurs="0" maxOccurs="unbounded"/>
</sequence>
...
```

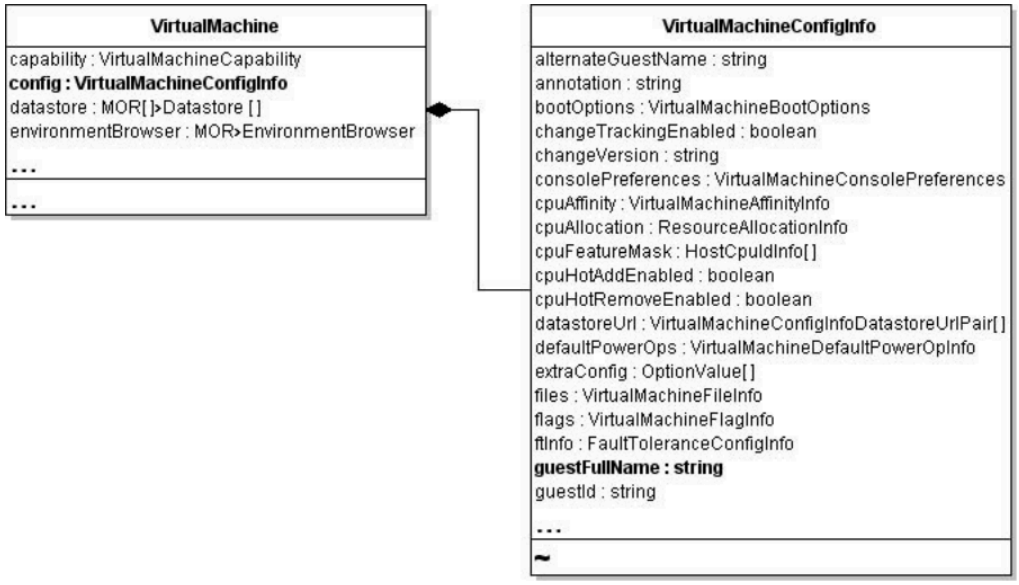
Extrapolating from the WSDL definitions shown in XML Schema Definition of PropertyFilterSpec Data Object Type, XML Schema Extract for PropertySpec, and ObjectSpec Definition as XML Schema might produce results similar to those shown in CreateFilter Spec Property Entry.

CreateFilter Spec Property Entry

```
<spec>
     <propSet>
          <type>VirtualMachine</type>
          <all>false</all>
          <pathSet>config.guestFullName</pathSet>
     </propSet>
     <objectSet>
          <obj type="Folder">group-v4</obj>
          <skip>true</skip>
     </objectSet>
</spec>
```

In this example, the `<spec>` element identifies the `spec` parameter of the `CreateFilter` method. The order of the element tags is as defined in the XML Schema for the property (XML Schema Definition of PropertyFilterSpec Data Object Type). The `pathSet` property defines the full path to the nested data object of interest. In CreateFilter Spec Property Entry, the `pathSet` property defines the path to the `guestFullName` property of the target virtual machine. ObjectSpec Definition as XML Schema shows the UML of these nested data objects.

Figure 642: Nested Data Objects



All of these details are available in the *vSphere API Reference*. By examining the WSDL definition, you can construct the strings needed to submit parameters through the MOB.

Using the MOB Along With the API Reference

The following table provides a brief summary of the steps involved when you use the MOB and the *vSphere API Reference* together.

Datatype	How to Input Values for Methods
Primitive	Enter the value as plain text regardless of its data type (`int`, `string`, `boolean`). Do not use quotes or other markup.
Array	Use the name of the parameter as the name of the element, wrap the values in a series of opening and closing tags for each array element.
Complex	Obtain XML Schema format information from the *vSphere API Reference* for the type (from the **Show WSDL type definition** link). Use the schema definition to construct the sequence of tags around the value (or values) you want to pass to the MOB.

HTTP Access to vSphere Server Files

In most cases, client applications interact with vSphere servers by using the vSphere Web Services SDK. In some cases, direct access to configuration files, log files, and other data on an ESX or vCenter systems is more efficient.

Introduction to HTTP Access

ESX and vCenter systems support file access using HTTP and secure HTTP. You can use HTTP/HTTPS for the following kinds of access.

- Datastore access on ESX and vCenter systems.
- ESX configuration and log file access on ESX systems.
- Update bundle access on ESX systems.

You can use the HTTP methods `GET`, `HEAD`, `PUT`, and `DELETE` to access files. The URL of the HTTP/HTTPS request must contain an embedded keyword that specifies the type of access. The following table shows the server access types with the corresponding URL keyword and HTTP methods.

Server Access	URL Keyword	HTTP Method or Methods
Datastore	folder	`GET`, `HEAD`, `PUT`, `DELETE`
ESX configuration file	host	`GET`, `HEAD`, `PUT` (See Host File Access to vSphere Server Files (/host) for the specific methods supported for each file type.)
Update bundle	tmp	`PUT`

Use the `PUT` method to create new files or overwrite existing files. You can create a subdirectory by using a URL that is consistent with the supported top-level directories. You cannot create datastores or data centers because the URL must refer to a valid data center or datastore.

You can use a Web browser to browse and download files. You cannot use a Web browser to post or delete files.

URL Syntax for HTTP Access

The URL specification in an HTTP request to a vSphere server includes one of the following keywords, which determines the type of access.

- Datastore Access (/folder)
- Host File Access to vSphere Server Files (/host)
- Update Package Access (/tmp)

Datastore Access (/folder)

An HTTP request for datastore access uses the following syntax:

```
http-method http[s]://server/folder[[/path]?dcPath=path[&dsName=name]]
```

http-method	One of the methods `GET`, `HEAD`, `PUT`, or `DELETE`.
http:// or https://	Access protocol (standard access or secure access).
`server`	ESX or vCenter target system. The server value can be an IP address or a DNS name.
/`folder`	Specifies datastore access on an ESX or vCenter system. The datastore URL can include the following optional elements: • *path* – Path to a file or directory in the datastore, relative to the root of the datastore. • `dcPath` – Inventory path to a datacenter. Specify the datacenter path as a name-value pair in the request. For a datacenter named `Datacenter` located in the root folder, the `dcPath` value is `MyDatacenter`. For a datacenter named `YourDatacenter` located in the folder `NorthAmerica` which is located in the root folder, the `cdPath` value is `NorthAmerica/YourDatacenter`. • `dsName` – Datastore associated with the datacenter. Specify the datastore name as a name-value pair in the request.

The following examples illustrate the syntax. If the target server is an ESX system, `dcPath=DCPATH&` is optional and defaults to `dcPath=ha-datacenter`.

Example	Description
`/folder`	Directory listing of known datacenters on this server.
`/folder?dcPath=` `path`	Directory listing of all datastores available at the specified datacenter.

`/folder?dcPath=`
`path`
`&dsName=`
`name`
	Top-level directory listing of the datastore.

`/folder/`
`path`
`?dcPath=`
`path`
`&dsName=`
`name`
	Directory listing of all files in a datastore directory.

`/folder/`
`path`
`/disk-flat.vmdk?dcPa`
`th=`
`path`
`&dsName=`
`name`
	Access individual files.

Host File Access to vSphere Server Files (/host)

An HTTP request for access to ESX configuration files uses the following syntax:

```
GET http[s]://my_system/host
http-method http[s]://my_system/host/file
```

Syntax Element	Description
http-method	One of `GET`, `HEAD`, or `PUT`, depending on the type of configuration file (see the following table).
http:// or https://	Access protocol (standard access or secure access).
`esx-server`	IP address or a DNS name.
`/host`	List of configuration files that you can access. (Use `/host` to retrieve the list.)
`/host/file`	A specific ESX configuration file.

The following table shows ESX host configuration files and the corresponding HTTP/HTTPS methods for access. The set of files might change from version to version.

Configuration File	HTTP Access Method(s)	Configuration File	HTTP Access Method(s)
esx.conf	GET, HEAD, PUT	ipmi0_sel.raw	GET, HEAD
openwsman.conf	GET, HEAD, PUT	ipmi0_sel	GET, HEAD
proxy.xml	GET, HEAD, PUT	ipmi0_sdr_content.raw	GET, HEAD
ssl_cert	GET, HEAD, PUT	ipmi0_sdr_header.raw	GET, HEAD
ssl_key	PUT	ipmi0_sensor_readings.raw	GET, HEAD
vmware_config	GET, HEAD, PUT	ipmi1_sel	GET, HEAD
vmware_configrules	GET, HEAD, PUT	ipmi1_sdr_content.raw	GET, HEAD
		ipmi1_sdr_header.raw	GET, HEAD
		ipmi1_sensor_readings.raw	GET, HEAD
		ipmi2_sel.raw	GET, HEAD
		ipmi2_sel	GET, HEAD
		ipmi2_sdr_content.raw	GET, HEAD
		ipmi2_sdr_header.raw	GET, HEAD
		ipmi2_sensor_readings.raw	GET, HEAD
		ipmi3_sel.raw	GET, HEAD
		ipmi3_sel	GET, HEAD
		ipmi3_sdr_content.raw	GET, HEAD
		ipmi3_sdr_header.raw	GET, HEAD
		ipmi3_sensor_readings.raw	GET, HEAD

Update Package Access (/tmp)

An HTTP request for update package access uses the following syntax:

```
PUT http[s]://esx-server/tmp/file-path
```

http:// or https://	Access protocol.
esx-server	IP address or a DNS name.
/tmp/file-path	Target file on an ESX system.

Privilege Requirements for HTTP Access

HTTP access to a vSphere file is access to a datastore object that is associated with the folder structure in the vSphere inventory. HTTP access requires the same privileges needed to obtain these files using any other mechanism, such as the vSphere Client.

The following table shows the required privileges.

Object Associated with File	Portion of URL	Required Privileges
Root folder	/folder	System.View
Datacenter	?dcPath	Datastore.Browse
		Datastore.FileManagement
Datastore	&dsName	Datastore.Browse
		Datastore.FileManagement
Host	/host	Host.Config.AdvancedConfig
	/tmp/	Host.Config.SystemManagement

Sample Program Overview

The VMware vSphere Web Services SDK includes samples for the Java platforms. This chapter lists the available sample programs and provides some information about each program. The samples have been re-compiled with JAX-WS bindings, and they use JAX-WS credential store classes that allow you to ignore certificates when you connect to a server with the samples.

Java Sample Programs (JAXWS Bindings)

When you download the SDK, you can find the Java sample programs and related files in the following directories.

- SDK\vsphere-ws\java\JAXWS\samples\com\vmware – Top-level directory for Java samples. Details listed in the following table.
- SDK\vsphere-ws\java\JAXWS\samples\com\vmware\security – Credential store utilities
- SDK\vsphere-ws\java\JAXWS\samples\com\vmware\vim25 – Stub directory. The vim25 directory contains stubs for release 2.5 and later.
- SDK\vsphere-ws\java\JAXWS\samples\com\vmware\vm – samples written for a single VM

Directory	Example	Description
alarms	VMPowerStateAlarm	Create an alarm to monitor a virtual machine's power state.
cim	CIMReader	Walks the Common Information Model (CIM) data associated with an ESX host.
connection	BasicConnection (among others)	Shows how to set up a vSphere connection with user name and password.
events	EventFormat	Retrieve and format the last event from the host daemon or vpxd. Includes a function that formats the event message.
	EventHistoryCollectorMonitor	Demonstrates how to create and monitor an EventHistoryCollector. Uses the latestPage property of EventHistoryCollector to filter the events.
	VMEventHistoryCollectorMonitor	Standalone client that demonstrates how to perform the following tasks: (1) Logging into the web service. (2)Creating EventHistoryCollector filtered for a single virtual machine. (3) Monitoring events using the latestPage property of the EventHistoryCollector.
fcd	FcdCreate, FcdDelete	Create and delete a first class disk (FCD) virtual storage object.
	FcdAttachToVM, FcdDetachFromVM	Attach or detach FCD to or from VM.

Directory	Example	Description
	FcdRegisterLegacyDisk	Register legacy disk as a first class disk.
general	Browser	Print all managed entities and for each entity its type, reference value, property name, property value, inner object type, inner reference value and inner property value.
	Connect	Connect to an ESXhost or a vCenter system.
	Create	Create a managed entity such as a folder, datacenter, or cluster.
	Delete	Delete a managed entity from the inventory tree. The managed entity can be a virtual machine, a ClusterComputeResource, or a folder.
	GetCurrentTime	Retrieve the current time from the vSphere Server.
	GetHostName	Retrieve the hostname of the ESX server.
	GetUpdates	Demonstrates how to use the PropertyCollector to monitor one or more properties of one or more managed objects. In particular this sample monitors one or all virtual machines and all hosts or one host for changes to some basic properties.
	LicenseManager	Demonstrates uses of the licensing API using License managed object reference.
	Move	Move a managed entity from its current location in the inventory to a new location in a specified folder
	PropertyCollector	Illustrates the use of the PropertyCollector API.
	RemoveManagedObject	Destroy or unregister a managed inventory object like a Host, VirtualMachine, or Folder.
	Rename	Rename a managed entity object.
	SearchIndex	Illustrates the use of the SearchIndex API.
	SimpleClient	Connect to the server, log in, list the inventory contents (managed entities) at the console, and log out.
	TaskList	Display a list of tasks performed on a specified managed object.
guest	CreateTemporaryFile	Create a temporary file inside a virtual machine.
	DownloadGuestFile	Download a file from the guest to a specified path on the host where the client is running.
	RunProgram	Run a specified program inside a virtual machine. RunProgram re-directs output to a temporary file inside the guest and downloads the output file.
	UploadGuestFile	Upload a file from the client machine to a specified location inside the guest operating system.
hcl	HCIBatchAddHostAndExtendCluster	Call batch-add-host to add four or more new hosts to an HCI cluster.
	HCIComputeOnlyCluster	Call HCI configure to create a simple cluster configuration.
host	AcquireSessionInfo	Acquire session with a vCenter or ESX host and print a CIM service ticket and related session information to a file.

Directory	Example		Description
	AddVirtualNic RemoveVirtualNic		Add a virtual NIC to a port group on a virtual switch. Remove a virtual NIC from a port group.
	AddVirtualSwitch RemoveVirtualSwitch		Add a virtual switch to a host. Remove a virtual switch from the host.
	AddVirtualSwitchPortGroup RemoveVirtualSwitchPortGroup		Add a port group to a virtual switch. Remove a port group from a virtual switch.
	DVSCreate		Create a distributed virtual switch.
	HostProfileManager		Demonstrates the use of HostProfileManager and ProfileComplainceManager.
	NIOCForDVS		Add a network resource pool to a distributed virtual switch.
httpfileaccess	GetVMFiles		Retrieve configuration files, snapshot files, log files, and virtual disk files of a virtual machine; place them on the system on which the program is run.
	PutVMFiles		Put virtual machine files into a specified datacenter and datastore; register and reconfigure the corresponding virtual machine.
performance	Basics		Display available performance counters or other metadata for an ESX host.
	History		Read performance measurements from the current time, or from a specified start time, for a specified duration.
	PrintCounters		Write the available counters of a managed entity into the specified file at the specified location. The managed entity can be a host system, a virtual machine, or a resource pool.
	RealTime		Display performance measurements from the current time at the console.
	VItop		An ESXtop-like sample application that lets administrators specify the CPU and memory counters by name to obtain metrics for a specified host.
	VIUsage		Create a GUI for graphical representation of the counters.
scheduling	DeleteOneTimeScheduledTask		Demonstrates deleting a ScheduledTask.
	OneTimeScheduledTask		Demonstrates creating a ScheduledTask using the ScheduledTaskManager.
	WeeklyRecurrenceScheduledTask		Demonstrates creating a weekly recurrent scheduled task.
scsilun	SCSILunName		Display the CanonicalName,Vendor, Model, Data, Namespace and NamespaceId of the host's SCSI LUN.
security	credstore	Base64	A fast, memory efficient class that encodes and decodes to and from BASE64 in full accordance with RFC 2045.
		CredentialStore	Create an example credential store.

Directory	Example		Description
		CredentialStoreAdmin	A command-line tool that provides complete access to the credential store backing file on the local machine.
		CredentialStoreCipher	This class uses the JVM provided classes in javax.crypto to encrypt and decrypt text.
		CredentialStoreFactory	Factory class providing instances of a credential store.
		CredentialStoreImpl	Implementation class for CredentialStoreAdmin.
		CredentialStoreStorage	This class provides the same functionality as FileInputStream, except that the close() method is overridden so that FileInputStream class' close() method does not get called.
simpleagent	CreateUser		Create a user account and password and store them in the local credential store.
	SimpleAgent		Access the local credential store to obtain a single user account for login to a server.
storage	CreateStorageDRS		Creates storage DRS.
	SDRSRecommendation		Runs storage DRS on an SDRS cluster to obtain SDRS recommendations.
	SDRSRules		Configures rules for an SDRS cluster.
vApp	OVFManagerExportVapp		Demonstrates the OvfManager by exporting VMDKs and OVF Descriptors of all VMs in the vApps.
	OVFManagerExportVMDK		Demonstrates how the OvfManager exports VMDKs from a VM to the localSystem.
	OVFManagerImportLocalVapp		Use this class to import or deploy an OVF Appliance from a local drive.
	OVFManagerImportVAppFromUrl		Use this class to import or deploy an OVF Appliance from a specified URL.
vim25	This directory contains the many Java classes that define JAX-WS bindings to the vSphere API.		
vm	VMApplyEvc		Apply a per-VM extended vMotion compatibility (EVC) setting to an existing virtual machine.
	VMClone		Locate an existing virtual machine on the vCentersystem, make a template from this virtual machine, and deploy instances of the template onto a datacenter.
	VMCreate		Create a virtual machine. Different command-line input creates the virtual machine in different ways.
	VMDeltaDisk		Create a delta disk on top of an existing virtual disk in a virtual machine, and simultaneously removes the original disk using the reconfigure API. Use delta disks in conjunction with linked virtual machines.
	VMDiskCreate		Create a virtual disk.
	VMLinkedClone		Create a linked virtual machine from an existing snapshot.

Directory	Example	Description
	VMManageCD	Configure a CDROM for a virtual machine. Also list information about the CDROMs associated with a virtual machine.
	VMManageFloppy	Configure a floppy drive for a virtual machine. Also list information about the floppy drives associated with a virtual machine.
	VMotion	Check whether migration with VMotion is feasible between two hosts. Perform a migration if the hosts are compatible.
	VMpowerOps	Perform power operations on a virtual machine.
	VMPromoteDisks	Consolidate a linked virtual machine by using the VirtualMachine.PromoteDisks method.
	VMReconfig	Reconfigure a virtual machine. Includes reconfiguring the disk size and disk mode.
	VMRelocate	Relocate a linked virtual machine using disk move type.
	VMSnapshot	Perform virtual machine snapshot operations
	XVCvMotion	Relocate virtual machine across from one vCenter to another.
	XVMotion	Relocate virtual machine to the computing resource recommended by the DRS.

Single Sign-On in the vSphere Environment

A vCenter Single Sign-On client connects to the vCenter Single Sign-On server to obtain a security token that contains authentication claims required for operations in the vSphere environment. The vCenter Single Sign-On client API supports operations to acquire, renew, and validate tokens.

vCenter Single Sign-On Overview

To support the requirements for secure software environments, software components require authorization to perform operations on behalf of a user. In a single sign-on environment, a user provides credentials once, and components in the environment perform operations based on the original authentication. vCenter Single Sign-On authentication can use the following identity store technologies:

- Windows Active Directory
- OpenLDAP (Lightweight Directory Access Protocol)
- Local user accounts (vCenter Single Sign-On server resident on the vCenter machine)
- vCenter Single Sign-On user accounts

For information about configuring identity store support, see *vSphere Installation and Setup* and *vSphere Security* in the VMware Documentation Center.

In the context of single sign-on, the vSphere environment is a collection of services and solutions, each of which potentially requires authentication of clients that use the service or solution. Examples of solutions that might support single sign-on include vShield, SRM (Site Recovery Manager), and vCO (vCenter Orchestrator). Because a service can use another service, single sign-on provides a convenient mechanism to broker authentication during a sequence of vSphere operations.

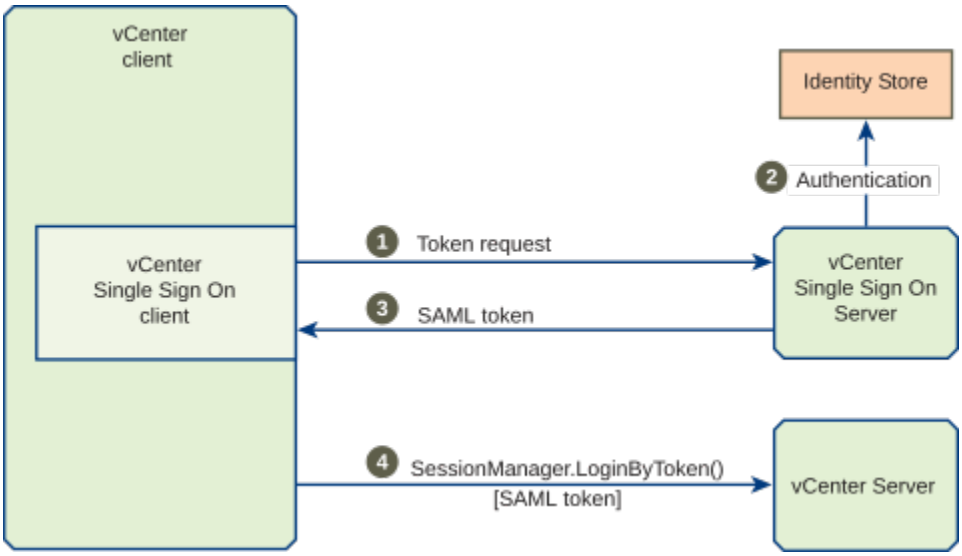
A vCenter Single Sign-On client connects to the vCenter Single Sign-On server to obtain a token that represents the client. The vCenter Single Sign-On server provides a Security Token Service (STS). A token uses the Security Assertion Markup Language (SAML), which is an XML encoding of authentication data. It contains a collection of statements or claims that support client authentication. Examples of token claims include name, key, and group.

vCenter Single Sign-On supports two types of tokens.

- Holder-of-key tokens provide authentication based on security artifacts embedded in the token. Holder-of-key tokens can be used for delegation. A client can obtain a holder-of-key token and delegate that token for use by another entity. The token contains the claims to identify the originator and the delegate. In the vSphere environment, a vCenter obtains delegated tokens on a user's behalf and uses those tokens to perform operations.
- Bearer tokens provide authentication based only on possession of the token. Bearer tokens are intended for short-term, single-operation use. A bearer token does not verify the identity of the user (or entity) sending the request. It is possible to use bearer tokens in the vSphere environment, however there are potential limitations:
  - The vCenter Single Sign-On server may impose limitations on the token lifetime, which would require you to acquire new tokens frequently.
  - Future versions of vSphere might require the use of holder-of-key tokens.

The following figure shows a vCenter client that uses a SAML token to establish a session with a vCenter.

Figure 643: Single Sign-On in the vSphere Environment – vCenter LoginByToken



The vCenter client also operates as a vCenter Single Sign-On client. The vCenter Single Sign-On client component handles communication with the vCenter Single Sign-On server.

1. The vCenter Single Sign-On client sends a token request to the vCenter Single Sign-On server. The request contains information that identifies the principal. The principal has an identity in the identity store. The principal may be a user or it may be a software component. In this scenario, the principal is the user that controls the vCenter client.
2. The vCenter Single Sign-On server uses the identity store to authenticate the principal.
3. The vCenter Single Sign-On server sends a response to the token request. If authentication is successful, the response includes a SAML token.
4. The vCenter client connects to the vCenter and calls the `SessionManager`.`LoginByToken` method. The login request contains the SAML token.

The previous figure shows the vCenter, vCenter Single Sign-On server, and identity store as components running on separate machines. You can use different vCenter Single Sign-On configurations.

- A vCenter Single Sign-On server can operate as an independent component running on its own machine. The vCenter Single Sign-On server can use a remote identity store or it can manage user accounts in its own internal identity store.
- A vCenter Single Sign-On server can operate as an embedded component running on the vCenter machine. In this configuration, the vCenter Single Sign-On server can use a remote identity store, its own internal identity store, or it can access user accounts on the vCentervCenter machine.

For information about installing and configuring the vCenter Single Sign-On server, see *vSphere Installation and Setup* and *vSphere Security* in the VMware Documentation Center.

vCenter Single Sign-On Client API

The vCenter Single Sign-On client API is described in the WSDL (Web Service Definition Language) file that is included in the vCenter Single Sign-On SDK. This API defines a set of request operations that correspond to the WS-Trust 1.4 bindings. The set of operations includes `Issue`, `Renew`, `Validate`, and `Challenge` requests.

- `Issue` – Obtains a token from a vCenter Single Sign-On server.
- `Renew` – Renews an existing token.
- `Validate` – Validates an existing token.
- `Challenge` – Part of a negotiation with a vCenter Single Sign-On server to obtain a token.

The vCenter Single Sign-On SDK includes Java bindings for the vCenter Single Sign-On WSDL. The SDK also contains sample code that demonstrates client-side support for the WS-SecurityPolicy standard. Security policies specify the elements that provide SOAP message security. To secure SOAP messages, a client inserts digital signatures, certificates, and SAML tokens into the SOAP headers for vCenter Single Sign-On requests.

The Java sample includes a JAX-WS implementation of SOAP header methods that support the vCenter Single Sign-On security policies.

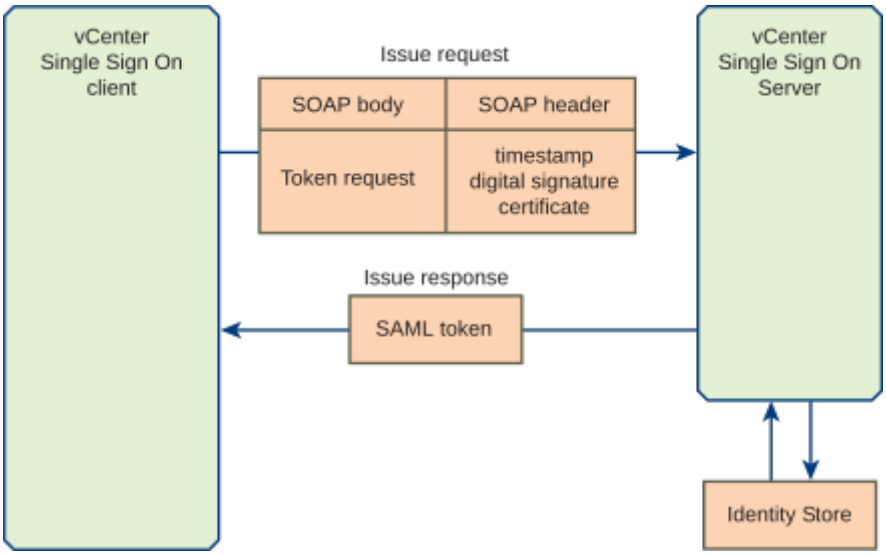
See vCenter Single Sign-On Security Policies and vCenter Single Sign-On SDK.

Acquiring a SAML Token from a vCenter Single Sign-On Server

To obtain a security token from a vCenter Single Sign-On server, the vCenter Single Sign-On client calls the `Issue` method, which sends a SOAP message that contains a token request and authentication data. This section describes a token request that uses a certificate to obtain a holder-of-key token. When the client creates the token request, it also inserts timestamp, signature, and certificate data into the SOAP security header.

The following figure represents the content of an `Issue` request and the response containing a SAML token.

Figure 644: Issue - vCenter Single Sign-On Token Request and Response



The vCenter Single Sign-On SDK provides Java packages that support SOAP header manipulation.

When the vCenter Single Sign-On server receives the issue request, it performs the following operations to generate a token:

- Uses the timestamp to validate the request.
- Validates the certificate.
- Uses the certificate to validate the digital signature.
- Uses the certificate subject to authenticate the request. Authentication is obtained from the identity store that is registered with the vCenter Single Sign-On server.
- Generates a token that specifies the principal – the vCenter Single Sign-On client – as the token subject.

vCenter Single Sign-On Security Policies

Web service security policies define the requirements for secure communication between a Web service and a client. vCenter Single Sign-On security policies are based on the WS-Policy framework and WS-SecurityPolicy specifications. A policy identifies specific elements for token requests. Based on the policy requirements, a vCenter Single Sign-On client will insert data into the SOAP security header for the token request.

vCenter Single Sign-On defines security policies for end user access, solution access, and for token exchange. The policies stipulate the following elements:

- Security certificates (x509V3, x509PKIPathV1, x509PKCS7, or WssSamlV20Token11)
- Message timestamps
- Security binding (transport)
- Encryption algorithm (Basic256Sha256)

vCenter Single Sign-On security policies specify that the body of the SOAP message for a holder-of-key token must be signed. Bearer tokens require only the username and timestamp tokens.

> **Note:**
> The vCenter Single Sign-On server issues SAML tokens to represent client authentication. The standards documentation also uses the term “token” to refer to claims and certificate data that is inserted into SOAP security headers.

The following table shows the vCenter Single Sign-On policies and identifies the requirements for each policy. The vCenter Single Sign-On WSDL defines these policies for use with the vCenter Single Sign-On methods.

Table 1893: vCenter Single Sign-On Policies

Policy	Description
STSSecPolicy	Defines the transport policy and algorithm suite for all communication with the vCenter Single Sign-On server: • Certificate-based server-side SSL authentication. • HTTPS transport binding using NIST (National Institute of Standards and Technology) Basic256Sha256 encryption algorithm. The HTTPS token is used to generate the message signature. • Request security header must contain a timestamp.
IssueRequestPolicy	Defines the security policy for Issue token requests. IssueRequestPolicy specifies either username token (signed), username token (plaintext password), X509 certificate, or holder-of-key token authentication. You specify username/password or X509 certificate credentials to obtain a vCenter Single Sign-On token. If you obtain a holder-of-key token, you can use that token for subsequent Issue requests. Username token (signed) authentication: • X509 endorsing supporting token (WssX509V3Token11, WssX509PkiPathV1Token11, or WssX509Pkcs7Token10) • WssUsernameToken11 signed supporting token Username token (plaintext password) authentication: • WssUsernameToken11 signed supporting token X509 certificate authentication: • X509 endorsing supporting token (WssX509V3Token11, WssX509PkiPathV1Token11, or WssX509Pkcs7Token10) Holder-of-Key token authentication: • WssSamlV20Token11 assertion referenced by a KeyIdentifier • Token must be used to sign the SOAP message body.

Policy	Description
RenewRequestPolicy	Defines the security policy for Renew token requests. The request must contain one of the following endorsing supporting tokens. The SOAP message body must be included in the signature generated with the token. • WssX509V3Token11 • WssX509PkiPathV1Token11 • WssX509Pkcs7Token10

vCenter Single Sign-On SDK Support for vCenter Single Sign-On Security Policies

The vCenter Single Sign-On SDK provides Java utilities that support the vCenter Single Sign-On security policies. Your vCenter Single Sign-On client can use these utilities to create digital signatures and supporting tokens, and insert them into SOAP headers as required by the policies. The SOAP header utilities are defined in files that are located in the samples directory:

`SDK\sso\java\JAXWS\samples\com\vmware\sso\client\soaphandlers`

See Using Handler Methods for SOAP Headers.

Connecting to a vCenter Single Sign-On Server

When a vCenter Single Sign-On client connects to a vCenter Single Sign-On server, it must specify the server URL as the endpoint for the token request message. The endpoint specification uses the following format:

`https://hostname|IPaddress/STS/STSService`

The path suffix (`STS/STSService`) is required. See Sending a Request for a Security Token for an example of setting the endpoint for a token request.

vCenter Single Sign-On Token Delegation

Holder-of-key tokens can be delegated to services in the vSphere environment. A service that uses a delegated token performs the service on behalf of the principle that provided the token. A token request specifies a `DelegateTo` identity. The `DelegateTo` value can either be a solution token or a reference to a solution token.

Components in the vSphere environment can use delegated tokens. vSphere clients that use the `LoginByToken` method to connect to a vCenter do not use delegated tokens. The vCenter will use a vSphere client's token to obtain a delegated token. The vCenter will use the delegated token to perform operations on behalf of the user after the user's vCenter session has ended. For example, a user may schedule operations to occur over an extended period of time. The vCenter will use a delegated token to support these operations.

vCenter Single Sign-On Token Lifetime - Clock Tolerance

A SAML token contains information about the lifetime of a token. A SAML token uses the `NotBefore` and `NotOnOrAfter` attributes of the SAML `Conditions` element to define the token lifetime.

<saml2:Conditions NotBefore="2011-10-04T21:39:17.731Z" NotOnOrAfter="2011-10-04T21:39:47.731Z">

During a token's lifetime, the vCenter Single Sign-On server considers any request containing that token to be valid and the server will perform renewal and validation operations on the token. The lifetime of a token is affected by a clock tolerance value that the vCenter Single Sign-On server applies to token requests. The clock tolerance value accounts for differences between time values generated by different systems in the vSphere environment. The clock tolerance is 10 minutes.

vCenter Single Sign-On Challenge (SSPI)

The vCenter Single Sign-On server supports the use of SSPI (Security Support Provider Interface) for client authentication. SSPI authentication requires that both the client and server use security providers to perform authentication.

At the beginning of a vCenter Single Sign-On server session, the vCenter Single Sign-On client and vCenter Single Sign-On server exchange data. Each participant will use its security provider to authenticate the data it receives. The authentication exchange continues until both security providers authenticate the data.

The vCenter Single Sign-On client API provides a `challenge` request for client participation in SSPI authentication. The following sequence describes the challenge protocol.

1. vCenter Single Sign-On client sends an `issue` request to the vCenter Single Sign-On server. The request contains the client credentials.
2. vCenter Single Sign-On server uses its security provider to authenticate the client. The server returns a `RequestSecurityTokenResponseType` object in response to the `issue` request. The response contains a challenge.
3. vCenter Single Sign-On client uses its security provider to authenticate the vCenter Single Sign-On server response. To continue the authentication exchange, the client sends a `challenge` request to the vCenter Single Sign-On server. The request contains the resolution to the server's challenge and it can also contain a challenge from the vCenter Single Sign-On client.
4. vCenter Single Sign-On server uses its security provider to authenticate the client's response. If there are still problems, the server can continue the authentication exchange by returning a response with an embedded challenge. If authentication is successful, the vCenter Single Sign-On server returns a SAML token to complete the original `issue` request.

To exchange challenge data, the vCenter single Sign-On client and vCenter Single Sign-On server use the following elements defined for both `RequestSecurityTokenType` and `RequestSecurityTokenResponseType` objects.

- Context attribute
- BinaryExchange element

vCenter Single Sign-On SOAP Message Structure

The requirements listed in the following table apply to the SOAP message structure in vCenter Single Sign-On message exchange.

Table 1894: vCenter Single Sign-On SOAP Message Structure

Element	Message Requirements
SOAP envelope	All <wst:RequestSecurityToken>, <wst:RequestSecurityTokenResponse>, and <wst:RequestSecurityTokenResponseCollection> elements must be sent as the single direct child of the body of a SOAP 1.1 <S11:Envelope> element.
	Use HTTP POST to send all vCenter Single Sign-On SOAP messages over an SSL/TLS-protected channel. Set the SOAPAction HTTP header field to the appropriate message binding.
	The <wsse:Security> header in a vCenter Single Sign-On request must contain a <wsu:Timestamp> element.
SOAP message signature	If a signature is applied to a request then it must include: • Either the <S11:Body>, or the WS-Trust element as a direct child of the <S11:Body> • The <wsu:Timestamp>, if present, in the <S11:Header>.
	Exclusive canonicalization without comments (xml-exc-c14n) must be used prior to signature generation.
	The signature certificate must either be carried either within a <wsse:BinarySecurityToken> or a <saml:Assertion> within <wsse:Security> header of the <S11:Header>.
	The signature must contain a <wsse:SecurityTokenReference> that uses an internal direct reference to the <wsse:BinarySecurityToken>.

vCenter Single Sign-On SDK

The vCenter Single Sign-On SDK is distributed as part of the VMware vSphere Management SDK. When you extract the contents of the distribution kit, the vCenter Single Sign-On SDK is located in the `ssoclient` sub-directory:

```
VMware-vSphere-SDK-build-num
      eam
      sms-sdk
      ssoclient
           docs
           dotnet
               cs
                  samples
           java
               JAXWS
                  lib
                  samples
           wsdl
      vsphere-ws
```

The following table shows the locations of the contents of the vCenter Single Sign-On SDK.

Table 1895: vCenter Single Sign-On SDK Contents

vCenter Single Sign-On SDK Component	Location
C# vCenter Single Sign-On samples	ssoclient/dotnet/cs/samples
JAX-WS vCenter Single Sign-On client binding	ssoclient/java/JAXWS/lib
Java samples	ssoclient/java/JAXWS/samples/com/vmware/sso/client/samples
VMware SOAP header utilities	ssoclient/java/JAXWS/samples/com/vmware/sso/client/soaphandlers
General utilities for samples	ssoclient/java/JAXWS/samples/com/vmware/sso/client/utils
WS-Security utilities for samples	ssoclient/java/JAXWS/samples/com/vmware/sso/client/wssecurity
vCenter LoginByToken sample	ssoclient/java/JAXWS/samples/com/vmware/vsphere/samples
VMware SOAP header utilties for LoginByToken example	ssoclient/java/JAXWS/samples/com/vmware/vsphere/soaphandlers
Documentation for example code	ssoclient/docs/java/JAXWS/samples/javadoc/index.html
WSDL files	ssoclient/wsdl

vCenter Single Sign-On SDK Examples

The vCenter Single Sign-On SDK contains Java examples that show how to acquire, validate, and renew tokens. This manual describes examples that show how to obtain a holder-of-key token from a vCenter Single Sign-On server and how to use that token to login to a vCenter.

This manual describes two of the Java examples provided by the VMware SSO Client SDK:

- vCenter Single Sign-On Client Example (JAX-WS). This example shows how to obtain a holder-of-key token from the vCenter Single Sign-On server.
- LoginByToken Example (JAX-WS). This example shows how to use the token to login to vCenter.

The following table lists the sample files in the SDK:

Table 1896: VMware SSO Client SDK Sample Files – Java

Location	Examples	Description
SDK/ssoclient/java/JAXWS/samples/com/vmware/sso/client/samples/		
	AcquireBearerTokenByUserCredentialSample.java	Demonstrates how to use username and password credentials to obtain a bearer token.
	AcquireHoKTokenByHoKTokenSample.java	Demonstrates how to exchange one holder-of-key token for another.
	AcquireHoKTokenBySolutionCertificateSample.java	Demonstrates how a solution uses its private key and certificate to acquire a holder-of-key token.
	AcquireHoKTokenByUserCredentialSample.java	Demonstrates how to use username, password, and certificate credentials to obtain a holder-of-key token. See vCenter Single Sign-On Client Example (JAX-WS).
	RenewTokenSample.java	Demonstrates how to renew a holder-of-key token.
	ValidateTokenSample.java	Demonstrates how to validate a token.
SDK/ssoclient/java/JAXWS/samples/com/vmware/sso/client/soaphandlers/		
	HeaderHandlerResolver.java	Provides methods to manage the set of header handlers.
	SamlTokenExtractionHandler.java	Extracts a SAML token from the vCenter Single Sign-On server response.
	SamlTokenHandler.java	Adds a SAML token to a SOAP security header.
	SSOHeaderhandler.java	Base class for header handler classes.
	TimeStampHandler.java	Adds a timestamp element to a SOAP security header.
	UserCredentialHandler.java	Adds a username token to a SOAP security header.
	WsSecuritySignatureAssertionHandler.java	Uses SAML token assertion ID, private key, and certificate to sign a SOAP message. For use when using an existing token to acquire a new token.
	WsSecurityUserCertificateSignatureHandler.java	Uses a private key and certificate to sign a SOAP message.
SDK/ssoclient/java/JAXWS/samples/com/vmware/vsphere/samples/		
	LoginByTokenSample.java	Demonstrates how to use a SAML token to login to a vCenter. See LoginByToken Example (JAX-WS).
SDK/ssoclient/java/JAXWS/samples/com/vmware/vsphere/soaphandlers/		
	HeaderCookieExtractionHandler.java	Extracts the vCenter HTTP session cookie from the response to a connection request.
	HeaderCookieHandler.java	Inserts an HTTP cookie into a request.

vCenter Single Sign-On API Reference

This chapter contains descriptions of the methods and data structures defined for the vCenter Single Sign-On client API.

vCenter Single Sign-On Client API Methods

The vCenter Single Sign-On client API consists of the following methods:

- `Issue`
- `Renew`
- `Validate`
- `Challenge`

Issue

Sends a security token request to a vCenter Single Sign-On server.

Method Signature

```
Issue (requestSecurityToken : RequestSecurityTokenType)
          returns RequestSecurityTokenResponseCollectionType
```

Parameter

`requestSecurityToken` : see [R]RequestSecurityTokenType – The following `RequestSecurityTokenType` elements are required for an Issue request; the remaining elements are optional.

- `RequestType` – Must be the URL “http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue”.
- `Sig` attribute (`UseKey` element) – Specifies a security token that contains an existing certificate key for subject confirmation.
- `Context` attribute – Required if you include a `BinaryExchangeType` element for SSPI authentication.

Return Value

`RequestSecurityTokenResponseCollectionType` – Set of `RequestSecurityTokenResponseType`. A response contains a SAML token or a challenge requiring additional authentication data.

Comments

Sends a token request to a vCenter Single Sign-On server. The request message must contain security artifacts as determined by the vCenter Single Sign-On policy used for the request. The vCenter Single Sign-On server will authenticate the user credentials in the request. For information about configuring user directory support for authentication, see *vSphere Installation and Setup* and *vSphere Security* in the VMware Documentation Center. If the vCenter Single Sign-On server requires information during SSPI authentication, it will negotiate with the vCenter Single Sign-On client by embedding a challenge in the response.

Renew

Renews an existing SAML token.

Method Signature

```
Renew (token : RequestSecurityTokenType) returns RequestSecurityTokenResponseType
```

Parameter

token : `RequestSecurityTokenType` – Security token request containing a SAML token previously obtained from a vCenter Single Sign-On server. The token must be valid (not expired). The following `RequestSecurityTokenType` elements are required for a `Renew` request; the remaining elements are optional.

- `RequestType` – Must be the URL “`http://docs.oasis-open.org/ws-sx/ws-trust/200512/Renew`”.

- `RenewTarget` – Identifies the SAML token to be renewed.
- `Sig` attribute (`UseKey` element) – Specifies a security token that contains an existing certificate key for subject confirmation.
- `Context` attribute – Required if you include a `BinaryExchangeType` element for SSPI authentication.

Return Value

`RequestSecurityTokenResponseType` – Response containing the renewed token.

Comments

You can renew holder-of-key tokens only. In addition to the the required token request elements shown above, the `Renew` request SOAP header must contain security elements according to the security policy.

Validate
Validates an existing SAML token.

Method Signature

```
Validate (token : RequestSecurityTokenType) returns RequestSecurityTokenResponseType
```

Parameter

token : `RequestSecurityTokenType` – Security token request containing a SAML token previously obtained from a vCenter Single Sign-On server. The following `RequestSecurityTokenType` elements are required for a `Validate` request; the remaining elements are optional.

- `RequestType` – Must specify the URL "`http://docs.oasis-open.org/ws-sx/ws-trust/200512/Validate`".
- `ValidateTarget` – Identifies the SAML token to be validated.
- `Sig` attribute (`UseKey` element) – Specifies a security token that contains an existing certificate key.
- `Context` attribute – Required if you include a `BinaryExchangeType` element for SSPI authentication.

Return Value

`RequestSecurityTokenResponseType` – Response containing the validated token.

Comments

Performs validation of the token and its subject. It includes but is not limited to validations of the following elements:

- Token signature
- Token lifetime
- Token subject
- Token delegates
- Group(s) to which the subject belongs

Challenge
Extends a token request to verify elements in the request.

Method Signature

```
Challenge (response : RequestSecurityTokenResponseType) returns RequestSecurityTokenResponseType
```

Parameter

response : `RequestSecurityTokenResponseType` – Contains SSPI data in the `BinaryExchange` element.

Return Value

`RequestSecurityTokenResponseType` – Response containing the validated token.

Comments

Part of a negotiation with a vCenter Single Sign-On server to resolve issues related to SSPI authentication.

vCenter Single Sign-On API Data Structures

Use the following objects for the vCenter Single Sign-On methods.?

RequestSecurityTokenType	ParticipantsType
RequestSecurityTokenResponseCollectionType	ParticipantType
RequestSecurityTokenResponseType	EndpointReference
LifetimeType	BinaryExchangeType
RenewingType	AdviceType
KeyTypeOpenEnum	AttributeType
UseKeyType	

RequestSecurityTokenType

Defines a set of token characteristics requested by the vCenter Single Sign-On client. The vCenter Single Sign-On client specifies this data object in a call to the `Issue`, `Renew`, and `Validate` methods. The vCenter Single Sign-On server may satisfy a request for a particular characteristic or it may use a different value in the issued token. The response to the token request contains the actual token values. See RequestSecurityTokenResponseType.

The vCenter Single Sign-On API supports a subset of the `RequestSecurityTokenType` elements defined in the WS-Trust specification. The following table shows the supported elements and attributes. An item in the table is defined as an element in the WSDL unless explicitly identified as an attribute.

Table 1897: RequestSecurityTokenType Elements (vCenter Single Sign-On)

Element	Datatype	Description
Context	string	`RequestSecurityToken` attribute specifying a URI (Uniform Resource Identifier) that identifies the original request. If you include this in a request, the vCenter Single Sign-On server will include the context identifier in the response. This attribute is required when the request includes a `BinaryExchange` property.
TokenType	string	Identifies the requested token type, specified as a URI (Uniform Resource Identifier). The following list shows the valid token types: • urn:oasis:names:tc:SAML:2.0:assertion – for issue and renew requests. • http://docs.oasis-open.org/ws-sx/ws-trust/200512/RSTR/Status – for validation requests.
RequestType	string	Identifies the request type, specified as a URI. The `RequestType` property is required. The following list shows the valid request types: • http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue • http://docs.oasis-open.org/ws-sx/ws-trust/200512/Renew • http://docs.oasis-open.org/ws-sx/ws-trust/200512/Validate

Element	Datatype	Description
Lifetime	`LifetimeType`	Time period during which a token is valid. The vCenter Single Sign-On server can ignore the requested lifetime and assign a different lifetime to the token. The lifetime specifies creation and expiration values. This property is optional – used with `Issue` and `Renew` requests.
ValidateTarget		Specifies the token to be validated. This property can contain either a reference to the token or it can contain the token itself. The property is required for and used only with the `Validate` method.
RenewTarget		Specifies the token to be renewed. This property can contain either a reference to the token or it can contain the token itself. This property is required for and used only with the `Renew` method.
Renewing	`RenewingType`	Specifies a request for a renewable token. This property is optional. If you do not specify the `Renewing` property, the vCenter Single Sign-On server will issue a renewable token. This property is optional.
DelegateTo		Specifies a security token or token reference for an identity to which the requested token will be delegated. The `DelegateTo` value must identify a solution.
Delegatable	xs:boolean	Indicates whether the requested token can be delegated to an identity. Use this property together with the `DelegateTo` property. The default value for the `Delegatable` property is false.
UseKey	`UseKeyType`	References a token for subject confirmation. Required for `Issue`, `Renew`, and `Validate` methods.
KeyType	string	String value corresponding to a `KeyTypeOpenEnum` value. The value is a URI (Uniform Resource Identifier) that specifies the requested key cryptography type. This property is optional.
SignatureAlgorithm	string	Specifies a URI (Uniform Resource Identifier) for an algorithm that produces a digital signature for the token. The following list shows the valid values: • http://www.w3.org/2000/09/xmldsig#rsa-sha1 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha256 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha384 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha512
BinaryExchange	`BinaryExchangeType`	Contains data for challenge negotation between the vCenter Single Sign-On client and vCenter Single Sign-On server.
Participants	ParticipantsType	Specifies the identities of participants that are authorized to use the token.
AdviceSet	AdviceSetType	List of `AdviceType`.

RequestSecurityTokenResponseCollectionType

Returned by the `Issue` method. This type contains a response to the request or the requested token. .

Table 1898: RequestSecurityTokenResponseCollectionType

Element	Datatype	Description
RequestSecurityTokenResponse	`RequestSecurityTokenResponseType[]`	List of token request response objects. The current architecture supports a single token response only

RequestSecurityTokenResponseType
Describes a single token.

Table 1899: RequestSecurityTokenResponseType Properties (vCenter Single Sign-On)

Element	Datatype		Description
Context	string		
	`RequestSecurityTokenResponse` attribute specifying a URI (Uniform Resource Identifier) that identifies the original request. This attribute is included in the response if it was specified in the request.		
TokenType	string		
	Identifies the type of token in the response. TokenType is specified as a URI (Uniform Resource Identifier), one of the following: • urn:oasis:names:tc:SAML:2.0:assertion – for issue and renew operations. • http://docs.oasis-open.org/ws-sx/ws-trust/200512/RSTR/Status – for validation operations.		
Lifetime	`LifetimeType`		
	Time period during which a token is valid. The lifetime in the token response is the actual lifetime assigned by the vCenter Single Sign-On server. The lifetime specifies creation and expiration values.		
RequestedSecurityToken	RequestedSecurityTokenType		
	SAML token.		
Renewing	`RenewingType`		
	Indicates whether or not the token can be renewed. By default, the vCenter Single Sign-On server will issue a renewable token.		
BinaryExchange	`BinaryExchangeType`		
	Contains data for challenge negotiation between vCenter Single Sign-On client and vCenter Single Sign-On server.		
KeyType	string		
	Indicates whether or not key cryptography is used. The KeyType is a string value corresponding to an enumerated type value. See `KeyTypeOpenEnum`. The value is a URI (Uniform Resource Identifier) that specifies the key type.		
SignatureAlgorithm	string		
	Indicates a URI (Uniform Resource Identifier) for an algorithm that produces a digital signature for the token. The following list shows the valid values: • http://www.w3.org/2000/09/xmldsig#rsa-sha1 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha256 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha384 • http://www.w3.org/2001/04/xmldsig-more#rsa-sha512		
Delegatable	xs:boolean		
	Indicates whether the requested token can be delegated to an identity.		
Status	StatusType		
	Indicates the status of the request. The property specifies `Code` and `Reason` values.		

LifetimeType

Specifies the token lifetime. Used in `RequestSecurityTokenType` and `RequestSecurityTokenResponseType`.

Table 1900: LifetimeType Properties

Property	Datatype	Description
created	wsu:AttributedDateTime	Creation time of the token. XML date and time, expressed as a standard time value (Gregorian calendar).
expires	wsu:AttributedDateTime	Time interval during which the token is valid, starting at the `created` time. The time interval is an absolute value specified in seconds.

RenewingType

Specifies token renewal.

Table 1901: RenewingType Properties

Property	DataType	Description
Allow	xsd:boolean	Specifies a request for a token for which the lifetime can be extended. This property is optional. The default value is `true`.
OK	xsd:boolean	Indicates that the vCenter Single Sign-On client will accept a token that can be renewed after it has expired. This property is optional. The default value is `false`. If you specify this property, you must specify a value of `false`. A token that can be renewed after expiration does not provide adequate security.

KeyTypeOpenEnum

Specifies a set of enumerated type values that identify the supported types of key cryptography used for security tokens. The values are URIs (Universal Resource Identifiers).

Table 1902: KeyType Properties

Enumerated type value	Description
http://docs.oasis-open.org/ws-sx/ws-trust/200512/PublicKey	Specifies asymmetric key cryptography using a combination of public and private keys. Use this key type for holder-of-key tokens.
http://docs.oasis-open.org/wssx/wstrust/200512/Bearer	Indicates a bearer token, which does not require a key to authenticate the token.

UseKeyType

Specifies the URI for an existing key.

Table 1903: UseKeyType Properties

Property	Datatype	Description
Sig	string	URI (Universal Resource Identifer) that refers to a security token which contains an existing key. If specified, the vCenter Single Sign-On server will use the associated certificate for subject confirmation.

ParticipantsType

Identifies users and services who are allowed to use the token.

Table 1904: ParticipantsType Properties

Property	Datatype	Description
Primary	`ParticipantType`	Primary user of the token.
Participant	`ParticipantType`	List of participants who are allowed to use the token.

ParticipantType
ParticipantType is an end point reference.

Table 1905: ParticipantType Property

Property	Datatype	Description
	`EndpointReference`	Specifies a participant represented as a URI.

EndpointReference
Participant identification. The ReferenceParameters, `Metadata`, and `any` elements are not used.

Table 1906: EndpointReference Property

Property	Datatype	Description
name	tns:AttributedURIType	URI that identifies a participant allowed to use a token.

BinaryExchangeType
Specifies a blob (binary large object) that contains data for negotation between the vCenter Single Sign-On client and server.

Table 1907: BinaryExchangeType Attributes

Attribute	Datatype	Description
ValueType	xsd:anyURI	Identifies the type of negotiation.
EncodingType	xsd:anyURI	Identifies the encoding format of the blob.

AdviceType

Specifies additional informational attributes to be included in the issued token. The vCenter Single Sign-On client can ignore this data. Advice data will be copied to delegate tokens. This type is used in `RequestSecurityTokenType`.

Table 1908: AdviceType Properties

Element/Attribute	Datatype	Description
Advicesource	string	AdviceType attribute specifying a URI representing the identity that provides the advice Attribute elements. This attribute is required.
Attribute	`AttributeType`	Advice data.

AttributeType

Attribute providing advice data. Used in `AdviceType`.

Table 1909: AttributeType Properties

Element/Attribute	Datatype	Description
Name	string	`AttributeType` attribute specifying a URI that is the unique name of the attribute. This attribute is required.
FriendlyName	string	`AttributeType` attribute specifying a human-readable form of the name. This attribute is optional.
AttributeValue	string	List of values associated with the attribute. The `AttributeValue` structure depends on the following criteria: • If the attribute has one or more values, the `AttributeType` contains one `AttributeValue` for each value. Empty attribute values are represented by empty `AttributeValue` elements. • If the attribute does not have a value, the `AttributeType` does not contain an `AttributeValue`.

vCenter Single Sign-On Client Example (JAX-WS)

This chapter describes a Java example of acquiring a vCenter Single Sign-On security token.

vCenter Single Sign-On Token Request Overview

The code examples in the following sections show how to use the `Issue` method to acquire a holder-of-key security token. To see an example of using the token to login to a vvCenter, see LoginByToken Example (JAX-WS). The code examples in this chapter are based on the following sample file located in the vCenter Single Sign-On SDK JAX-WS client `samples` directory:

`.../JAXWS/samples/com/vmware/sso/client/samples/AcquireHoKTokenByUserCredentialSample.java`

The `AcquireHoKTokenByUserCredentialSample` program creates a token request and calls the issue method to send the request to a vCenter Single Sign-On server. The program uses a sample implementation of Web services message handlers to modify the SOAP security header for the request message.

This example uses the username-password security policy (`STSSecPolicy_UserPwd`). This policy requires that the SOAP security header include a timestamp, username and password, and a digital signature and certificate. The sample message handlers embed these elements in the message.

The example performs the following operations:

1. Create a security token service client object (`STSService_Service`). This object manages the vCenter Single Sign-On header handlers and it provides access to the vCenter Single Sign-On client API methods. This example uses the issue method.
2. Create a vCenter Single Sign-On header handler resolver object (`HeaderHandlerResolver`). This object acts as a container for the different handlers.
3. Add the handlers for timestamp, user credentials, certificate, and token extraction to the handler resolver.
4. Add the handler resolver to the security token service.
5. Retrieve the STS port (`STS_Service`) from the security token service object.
6. Create a security token request.
7. Set the request fields.
8. Set the endpoint in the request context. The endpoint identifies the vCenter Single Sign-On server.
9. Call the issue method, passing the token request.
10. Handle the response from the vCenter Single Sign-On server.

Using Handler Methods for SOAP Headers in Java

The VMware vCenter Single Sign-On SDK provides sample code that is an extension of the JAX-WS XML Web services message handler (`javax.xml.ws.handler`). The sample code consists of a set of SOAP header handler methods and a header handler resolver, to which you add the handler methods. The handler methods insert timestamp, user credential, and message signature data into the SOAP security header for the request. A handler method extracts the SAML token from the vCenter Single Sign-On server response.

The VMware vCenter Single Sign-On client SOAP header handler files are located in the `soaphandlers` directory:

`SDK/sso/java/JAXWS/samples/com/vmware/sso/client/soaphandlers`

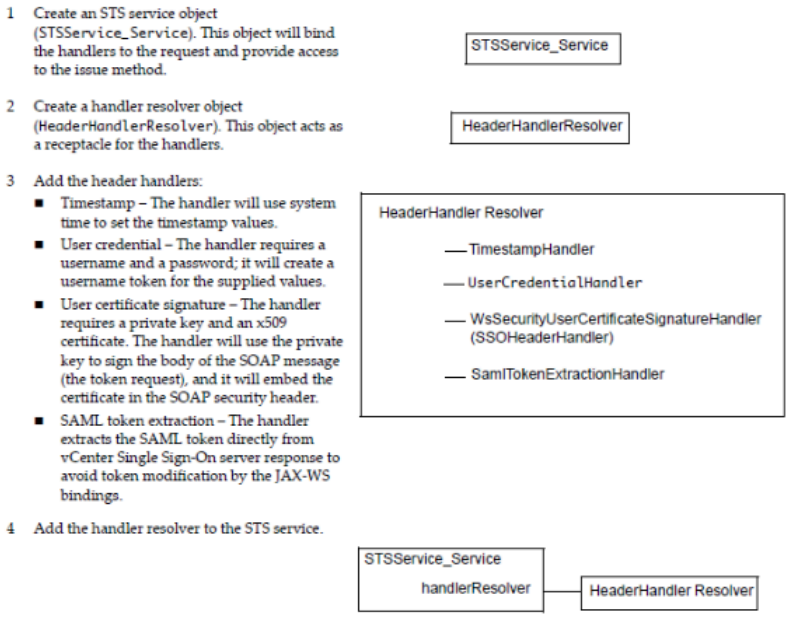
To access the SOAP handler implementation, the example code contains the following import statements:

```
import com.vmware.sso.client.soaphandlers.HeaderHandlerResolver;
import com.vmware.sso.client.soaphandlers.SSOHeaderHandler;
import com.vmware.sso.client.soaphandlers.SamlTokenExtractionHandler
import com.vmware.sso.client.soaphandlers.TimeStampHandler;
import com.vmware.sso.client.soaphandlers.UserCredentialHandler;
import com.vmware.sso.client.soaphandlers.WsSecurityUserCertificateSignatureHandler;
```

This example uses the following handler elements:

- HeaderHandlerResolver
- SamlTokenExtractionHandler
- TimestampHandler
- UserCredentialHandler
- WsSecurityUserCertificateSignatureHandler (SSOHeaderHandler)

The following sequence shows the operations and corresponding Java elements for message security.



The following example creates a handler resolver and adds the handler methods to the handler resolver. After the handlers have been established, the client creates a token request and calls the `Issue` method. See Sending a Request for a Security Token in Java.

> **Important:**
> You must perform these steps for message security before retrieving the STS service port. An example of retrieving the STS service port is shown in Sending a Request for a Security Token in Java.

Acquiring a vCenter Single Sign-On Token – Soap Handlers

```
/*
 * Instantiate the STS Service
 */
 STSService_Service stsService = new STSService_Service();

/*
 * Instantiate the HeaderHandlerResolver.
 */
 HeaderHandlerResolver headerResolver = new HeaderHandlerResolver();

/*
 * Add handlers to insert a timestamp and username token into the SOAP security header
 * and sign the message.
 *
 * -- Timestamp contains the creation and expiration time for the request
 * -- UsernameToken contains the username/password
 * -- Sign the SOAP message using the combination of private key and user certificate.
 *
 * Add the TimeStampHandler
 */
 headerResolver.addHandler(new TimeStampHandler());

/*
 * Add the UserCredentialHandler. arg[1] is the username; arg[2] is the password.
 */
 UserCredentialHandler ucHandler = new UserCredentialHandler(args[1], args[2]);
 headerResolver.addHandler(ucHandler);

/*
 * Add the message signature handler (WsSecurityUserCertificateSignatureHandler);
 * The client is responsible for supplying the private key and certificate.
 */
 SSOHeaderHandler ssoHandler =
    new WsSecurityUserCertificateSignatureHandler(privateKey, userCert);
 headerResolver.addHandler(ssoHandler);

/*
 * Add the token extraction handler (SamlTokenExtractionHandler).
 */
 SamlTokenExtractionHandler sbHandler = new SamlTokenExtractionHandler;
 headerResolver.addHandler(sbHandler);

/*
 * Set the handlerResolver for the STSService to the HeaderHandlerResolver created above.
 */
 stsService.setHandlerResolver(headerResolver);
```

Sending a Request for a Security Token in Java

After setting up the SOAP header handlers, the example creates a token request and calls the issue method. The following sequence shows the operations and corresponding Java elements.

5 Retrieve the STS service port (`STSService`). The service port provides access to the vCenter Single Sign-On client API methods. The vCenter Single Sign-On handler resolver must be associated with the STS service before you retrieve the service port. See "Using Handler Methods for SOAP Headers" on page 34.



6 Create a token request (`RequestSecurityTokenType`). Your vCenter Single Sign-On client will pass the token request to the `Issue` method. The `Issue` method will send the token request in the body of the SOAP message. This example sets the token request fields as appropriate for a holder-of-key token request.

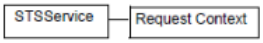


7 Set the token request fields.

- lifetime – Creation and expiration times.
- token type – urn:oasis:names:tc:SAML:2.0:assertion
- request type – http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue
- key type – http://docs.oasis-open.org/ws-sx/ws-trust/200512/PublicKey (for holder-of-key token type)
- signature algorithm – http://www.w3.org/2001/04/xmldsig-more#rsa-sha256
- renewable status



8 Set the endpoint address for the token request.



9 Call the Issue method.



10 Handle the response from the vCenter Single Sign-On server.



The following code example shows Java code that performs these operations.

Acquiring a vCenter Single Sign-On Token – Sending the Request

```
/*
 * Retrieve the STSServicePort from the STSService_Service object.
 */
 STSService stsPort = stsService.getSTSServicePort();

/*
 * Create a token request object.
 */
 RequestSecurityTokenType tokenType = new RequestSecurityTokenType();

/*
 *  Create a LifetimeType object.
 */
 LifetimeType lifetime = new LifetimeType();

/*
 * Derive the token creation date and time.
 * Use a GregorianCalendar to establish the current time,
 * then use a DatatypeFactory to map the time data to XML.
 */
 DatatypeFactory dtFactory = DatatypeFactory.newInstance();
 GregorianCalendar cal = new GregorianCalendar(TimeZone.getTimeZone("GMT"));
 XMLGregorianCalendar xmlCalendar = dtFactory.newXMLGregorianCalendar(cal);
 AttributedDateTime created = new AttributedDateTime();
 created.setValue(xmlCalendar.toXMLFormat());

/*
 * Specify a time interval for token expiration (specified in milliseconds).
 */
 AttributedDateTime expires = new AttributedDateTime();
 xmlCalendar.add(dtFactory.newDuration(30 * 60 * 1000));
```

```
 expires.setValue(xmlCalendar.toXMLFormat());

/*
 * Set the created and expires fields in the lifetime object.
 */
 lifetime.setCreated(created);
 lifetime.setExpires(expires);

/*
 * Set the token request fields.
 */
 tokenType.setTokenType("urn:oasis:names:tc:SAML:2.0:assertion");
 tokenType.setRequestType("http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue");
 tokenType.setLifetime(lifetime);
 tokenType.setKeyType("http://docs.oasis-open.org/ws-sx/ws-trust/200512/PublicKey");
 tokenType.setSignatureAlgorithm("http://www.w3.org/2001/04/xmldsig-more#rsa-sha256");

/*
 * Specify a token that can be renewed.
 */
 RenewingType renewing = new RenewingType();
 renewing.setAllow(Boolean.TRUE);
 renewing.setOK(Boolean.FALSE); // WS-Trust Profile: MUST be set to false
 tokenType.setRenewing(renewing);

/* Get the request context and set the endpoint address. */
 Map<String, Object> reqContext = ((BindingProvider) stsPort).getRequestContext();
 reqContext.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, args[0]);

/*
 * Use the STS port to invoke the "issue" method to acquire the token
 * from the vCenter Single Sign-On server.
 */
 RequestSecurityTokenResponseCollectionType issueResponse = stsPort.issue(tokenType);

/*
 * Handle the response - extract the SAML token from the response. The response type
 * contains the token type (SAML token type urn:oasis:names:tc:SAML:2.0:assertion).
 */
 RequestSecurityTokenResponseType rstResponse = issueResponse.getRequestSecurityTokenResponse();
 RequestedSecurityTokenType requestedSecurityToken = rstResponse.getRequestedSecurityToken();

/*
 * Extract the SAML token from the RequestedSecurityTokenType object.
 * The generic token type (Element) corresponds to the type required
 * for the SAML token handler that supports the call to LoginByToken.
 */
 Element token = requestedSecurityToken.getAny();
```

LoginByToken Example (JAX-WS)

This chapter describes a Java example of using the `LoginByToken` method. A vCenter session begins with the call to the LoginByToken method after receiving the token from the SSO Server.

vCenter Single Sign-On Session

After you obtain a SAML token from the vCenter Single Sign-On server, you can use the vSphere Web Services API method `LoginByToken` to establish a single sign-on session with a vCenter. See vCenter Single Sign-On Client Example (JAX-WS) for an example of obtaining a vCenter Single Sign-On token.

At the beginning of a vCenter Single Sign-On session, your client is responsible for the following tasks:

- Insert the vCenter Single Sign-On token and a timestamp into the SOAP header of the `LoginByToken` message.
- Maintain the vCenter session cookie. During the login sequence, the server produces an HTTP session cookie to support the persistent connection. Your client must save this cookie and re-introduce it at the appropriate times.
- If at a later time your client invokes the `LoginByToken` method, or other login method, the Server issues a new session cookie in response. You must have a cookie handler in place to save the cookie for subsequent requests.

The example program uses these general steps:

1. Call the `RetrieveServiceContent` method. The method establishes the connection with the vCenter and provides access to the `SessionManager` managed object.
2. Call the `LoginByToken` method to authenticate the vCenter session. To send the token to the vCenter, the client uses a handler to embed the token and a time stamp in the SOAP header for the message. The client uses an HTTP header handler method to extract the cookie from the vCenter response.
3. Restore the session cookie for future requests. To identify the session started with the `LoginByToken` method, the client uses a handler to embed the session cookie in the HTTP header.

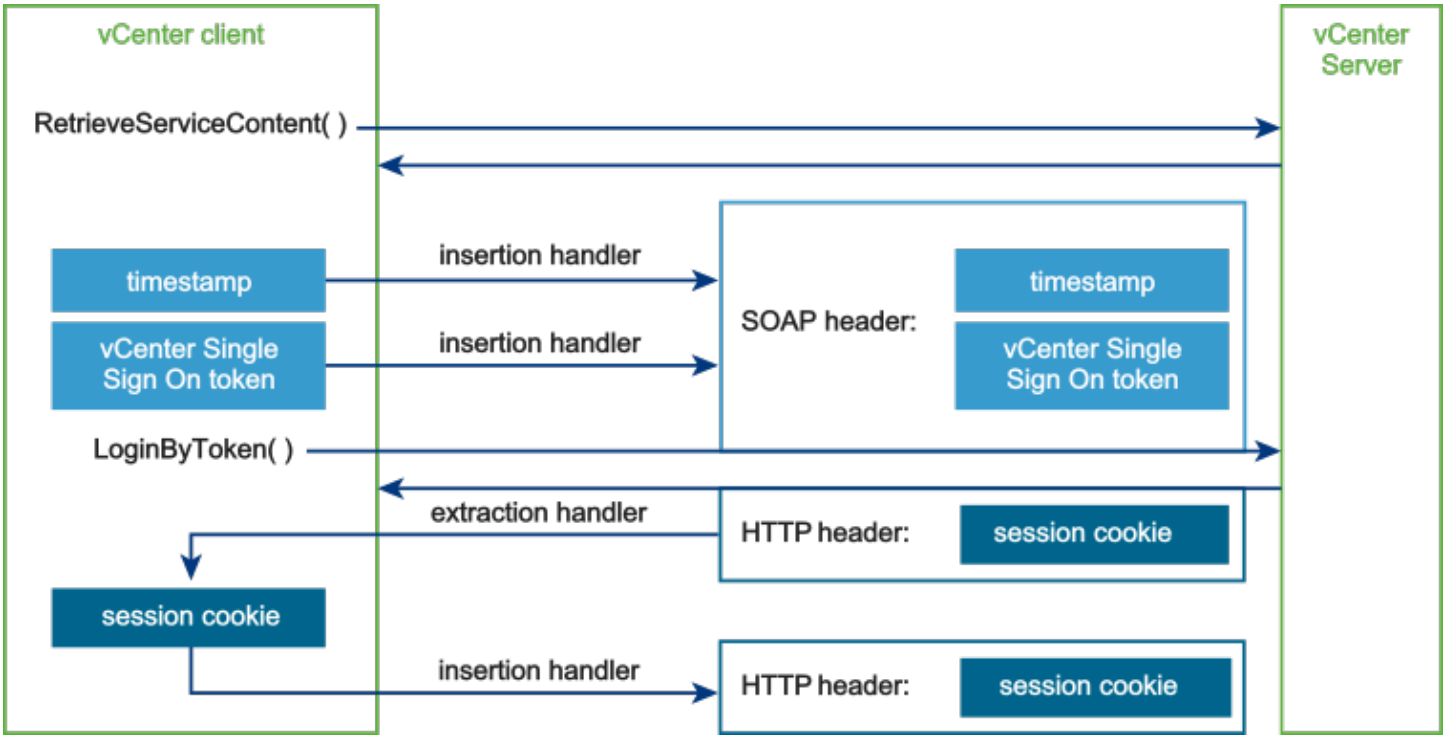
HTTP and SOAP Header Handlers in Java

To use a vCenter Single Sign-On token to login to a vCenter, the example uses header handlers to manipulates the HTTP and SOAP header elements of the login request. After establishing a handler, subsequent requests automatically invoke the handler.

- Insertion handlers put the vCenter Single Sign On token and a timestamp into the SOAP header into the HTTP header of the login request.
- An extraction handler obtains the HTTP session cookie provided by the vCenter. After setting up the handler, a call to the `LoginByToken` method will invoke the handler to extract the cookie from the Server response.

The following figure shows the use of handlers to manipulate header elements when establishing a vCenter Single Sign On session with a vCenter.

Figure 645: Starting a vCenter Session



Important:

Every call to the vCenter will invoke any message handlers that have been established. The overhead involved in using the SOAP and HTTP message handlers is not necessary after the session has been established. The example saves the default message handler before setting up the SOAP and HTTP handlers. After establishing the session, the example will reset the handler chain and restore the default handler.

The example code also uses multiple calls to the `VimPortType.getVimPort method` to manage the request context. The `getVimPort` method clears the HTTP request context. After each call to the `getVimPort` method, the client resets the request context endpoint address to the vCenter URL. After the client has obtained the session cookie, it will restore the cookie in subsequent requests.

LoginByToken Sample Code in Java

The code examples in the following sections show how to use the `LoginByToken` method with a holder-of-key security token. The code examples are based on the sample code contained in the vCenter Single Sign-On SDK. The files are located in the Java samples directory (`SDK/ssoclient/java/JAXWS/samples`):

- LoginByToken sample:
  `samples/com/vmware/vsphere/samples/LoginByTokenSample.java`
- Header cookie handlers:
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieHandler.java`
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieExtractionHandler.java`
- SOAP header handlers. These are the same handlers that are used in vCenter Single Sign-On Client Example (JAX-WS). The SOAP handler files are located in the vCenter Single Sign-On client `soaphandlers` directory:
  `samples/com/vmware/sso/client/soaphandlers`

Steps to Connect with a vSphere Server

For a connection with a vSphere server, use the following steps:

1. Get the `VimPort` object for access to the vSphere services.
2. Call the `RetrieveServiceContent` method.
3. Call the `LoginByToken` method; insert a SAML token into the request header..
4. Extract the session cookie from the response.
5. Use the session cookie to set the request context for future requests.

The following sections describe these steps.

Import the Necessary Java Packages for LoginByToken

The following example describes the packages that you need to import into your application to use the new LoginByToken method:

Import the Packages that Are Needed

```
package com.vmware.vsphere.samples;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import javax.xml.ws.BindingProvider;
import javax.xml.ws.handler.HandlerResolver;
import javax.xml.ws.handler.MessageContext;
import org.w3c.dom.Element;
import com.vmware.sso.client.soaphandlers.HeaderHandlerResolver;
import com.vmware.sso.client.soaphandlers.SamlTokenHandler;
import com.vmware.sso.client.soaphandlers.TimeStampHandler;
import com.vmware.vim25.InvalidLocaleFaultMsg;
```

```
import com.vmware.vim25.InvalidLoginFaultMsg;
import com.vmware.vim25.ManagedObjectReference;
import com.vmware.vim25.RuntimeFaultFaultMsg;
import com.vmware.vim25.ServiceContent;
import com.vmware.vim25.VimPortType;
import com.vmware.vim25.VimService;
import com.vmware.vsphere.soaphandlers.HeaderCookieExtractionHandler;
```

Get the VimPort for LoginByToken in Java

The following example shows the code to get the VimPort:

Getting the VimPort

```
package com.vmware.vsphere.samples;

/**
* Sample program demonstrating the usage of the new loginByToken method,
* introduced to authenticate the client using the SAML token obtained from the
* SSO server.
*
* @author VMware, Inc.
*/
public class LoginByTokenExample {
/**
* This method invokes the loginByToken method for authentication. Once this
* method is called the established session is authenticated and operations
* can be performed on the connected vCenter
*
* @param token
* {@link Element} representing the SAML token that needs to be
* used for the authentication
* @param vcServerUrl
* The vCenter URL that needs to be connected
* @return String authenticated session cookie used by the connection
* @throws RuntimeFaultFaultMsg
* @throws InvalidLocaleFaultMsg
*/
public static String loginUsingSAMLToken(Element token, String vcServerUrl)
throws RuntimeFaultFaultMsg, InvalidLocaleFaultMsg,
InvalidLoginFaultMsg {
VimService vimService = new VimService();
HandlerResolver defaultHandler = vimService.getHandlerResolver();

// Step 1 Get the VimPort object
vimService = new VimService();
VimPortType vimPort = vimService.getVimPort();
```

Retrieve the Service Content in Java

The following example shows the call to `retrieveServiceContent`.

Call `retrieveServiceContent`

```
// Step 2 Retrieve the service content
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, false);
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
```

```
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");
ServiceContent serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);
```

Invoke the `loginByToken` method in Java

The following example shows the `loginByToken` call.

Call the `loginByToken` Method

```
// Step 3 Invoking the loginByToken method
HeaderHandlerResolver handlerResolver = new HeaderHandlerResolver();
handlerResolver.addHandler(new TimeStampHandler());
handlerResolver.addHandler(new SamlTokenHandler(token));
HeaderCookieExtractionHandler cookieExtractor = new HeaderCookieExtractionHandler();
handlerResolver.addHandler(cookieExtractor);
vimService.setHandlerResolver(handlerResolver);
vimPort = vimService.getVimPort();
Map<String, Object> ctxt2 = ((BindingProvider) vimPort).getRequestContext();
ctxt2.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt2.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
vimPort.loginByToken(serviceContent.getSessionManager(), null);
```

Extract the Session Cookie in Java

After calling the `loginByToken`, you extract the session cookie, as shown in the following example.

Extract the Session Cookie

```
// Step 4 Extract the session cookie that is established in the previous
// call to the server.
String cookie = cookieExtractor.getCookie();
// Clear the HandlerResolver chain setup in step 3 as that is
// required only one time for the invocation of the LoginByToken method.
// After login we do not need this handler thus reverting to the
// original (default) handler.
vimService.setHandlerResolver(defaultHandler);
vimPort = vimService.getVimPort();
```

Inject the Session Cookie Back Into the Request in Java

Before making another call to the server, inject the session cookie back into the request as shown in the following example.

Inject the Session Cookie Back Into the Request

```
// Step 5 Inject the session cookie back into the request once before
// making another call to the server. JAXWS will maintain that cookie
// for all subsequent requests.
Map<String, Object> ctxt3 = ((BindingProvider) vimPort).getRequestContext();
ctxt3.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt3.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
// One time setting of the cookie
@SuppressWarnings("unchecked")
Map<String, List<String>> headers =
  (Map<String, List<String>>) ctxt3.get(MessageContext.HTTP_REQUEST_HEADERS);
if (headers == null) {
   headers = new HashMap<String, List<String>>();
}
headers.put("Cookie", Arrays.asList(cookie));
```

```
ctxt3.put(MessageContext.HTTP_REQUEST_HEADERS, headers);
// Authentication complete. Proceed with rest of the API calls
// that are required for your functionality.
return cookie;
}
}
```

Additional Information

Please refer to the `LoginByTokenSample` Java sample source in the vSphere Management SDK for more information. Also refer to the VMware *vSphere API Reference Documentation* and the *vSphere Web Services SDK Developer's Setup Guide*.

Using the vSAN Management APIs

The vSAN Management APIs are offered for two programming languages: Java and Python. They deliver similar functionality. OpenAPI bindings are available for developing bindings to other programming languages.

You can download the VMware Cloud Foundation SDK from developer.broadcom.com or from Github.

> **Note:**
>
> Most examples related to vSAN APIs below are in Python language, not Java.

vSAN Management Java API

The vSAN Management API for Java provides WSDL files, sample code, and API reference for developing custom Java clients against the vSAN Management API. The vSAN Management API for Java depends on the vSphere Web Services API of similar level. You use the vSphere Web Services API for logging in to vCenter and for retrieving vSphere managed objects.

API Reference

The vSAN API reference documentation is included in the `/docs` directory. To view the API Reference, open `index.html` with a Web browser.

WSDL Files and vSAN Java Bindings

The vSAN Management API for Java includes the `vsan.wsdl` and `vsanService.wsdl` files in the `bindings/wsdl` directory. You can use the WSDL definitions to build Java bindings for accessing the vSAN Management API. You can build Java bindings using the `build.py` script.

Running the Sample Applications

The vSAN Management API for Java includes sample applications, `build` and `run` scripts, and dependent libraries. They are located under the `samplecode` directory in the SDK.

You can use the sample code to get vSAN managed objects on vCenter or ESX hosts.

Before running the sample applications, make sure that you have the vSphere Web Services API on your development environment, with the following directory structure:

```
VMware-vSphere-SDK-<version number>-build
 SDK
  vsphere-ws
```

Then copy the `vsan-sdk-java` directory at the same level as the `vsphere-vs` directory in the vSphere Web Services SDK:

```
VMware-vSphere-SDK-<version number>-build
 SDK
  vsphere-ws
```

```
vsan-sdk-java
```

Build the sample applications by running the `build.py` command.

Run the sample applications using the `run.sh` script on Linux, or the `run.bat` script on Windows:

```
./run.sh com.vmware.vsan.samples.<sample_name>
 --url https://<vCenter or host address>/sdk
 --username <username>
 --password <password>
```

To get information about the parameter usage, use `-h` or `--help`.

vSAN Management SDK for Python

The vSAN Management SDK for Python provides language bindings, sample code, and API reference for developing custom Python clients against the vSAN Management API. The vSAN Management SDK for Python depends on `pyVmomi` of similar release level, which is the Python SDK for the vSphere API. You use `pyVmomi` for logging in to vCenter and for retrieving vSphere managed objects.

> **Note:**
>
> You can download `pyVmomi` from GitHub.

API Reference

The vSAN API reference documentation is included in the `/docs` directory. To view the API Reference, open `index.html` with a Web browser.

Python Bindings

You can access the vSAN Management API by using the Python `vsanmgmtObjects.py` script under the `bindings` directory.

To use the Python bindings, place `vsanmgmtObjects.py` on a path where your Python applications import.

Running the Sample Applications

The vSAN Management SDK for Python provides sample applications, which you can find under the `samplecode` directory.

You can use the sample code to get vSAN managed objects on vCenteror ESX hosts. The code automatically identifies the target server type.

The `vsaniscsisamples.py` and `vsaniscsisamples.py` depend on the `vsanapiutis.py`, which provides utility libraries for retrieving vSAN managed objects.

To run the sample applications, use the following commands:

```
python vsanapisamples.py -s <host or vCenter  address> -u <username> -p <password>
    --cluster <cluster name>
python vsaniscsisamples.py -s <host or vCenter  address> -u <username> -p <password>
    --cluster <cluster name>
```

To view information about the parameter usage, use `-h` or `--help`.

Setting Up a vSAN Cluster

This chapter introduces use of the vSAN Management API to automate configuration of a vSAN cluster. You can also configure multiple clusters by repeating the API.

Examples show code in Python.

Connecting to vCenter and Selecting a Cluster for vSAN

Before configuring a vSAN cluster by using the vSAN Management API, you must establish a secure connection with vCenter and filter the clusters on which you want to enable vSAN.

In the following example, a secure connection is established with the vCenter using the user name and password authentication passed as arguments to the Python script. Later, the `getClusterInstance` function is called by passing the cluster name as an argument.

```
if sys.version_info[:3] > (2, 7, 8):
    context = ssl.create_default_context()
    context.check_hostname = False
    context.verify_mode = ssl.CERT_NONE

# Connect to vCenter
si = SmartConnect(
    host=args.host,
    user=args.user,
    pwd=password,
    port=int(args.port),
    sslContext=context)

# Disconnect from vSAN upon exit
atexit.register(Disconnect, si)

# Connect to the cluster passed as an argument
cluster = getClusterInstance(args.clusterName, si)
```

After establishing a secure connection with vCenter and identifying the cluster, a connection is made to that cluster. The `getClusterInstance` function can be reused across the client application to connect to the clusters on which you want to configure vSAN.

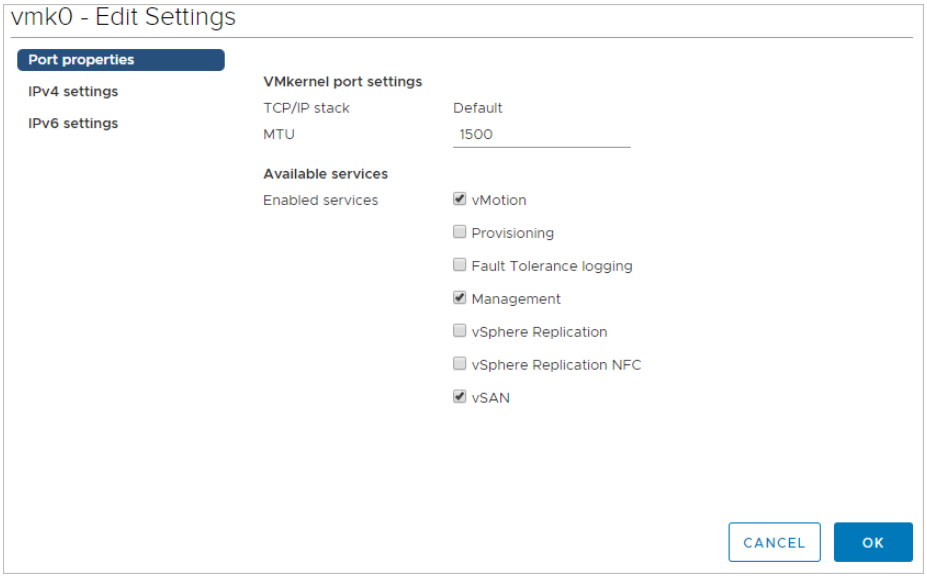
```
def getClusterInstance(clusterName, serviceInstance):
    content = serviceInstance.RetrieveContent()
    searchIndex = content.searchIndex
    datacenters = content.rootFolder.childEntity

# Look for the cluster in each datacenter attached to vCenter
for datacenter in datacenters:
    cluster = searchIndex.FindChild(datacenter.hostFolder, clusterName)
    if cluster is not None:
        return Cluster
    else:
        return None
```

Configuring VMkernel Networking for vSAN

You must configure every host that is part of the vSAN cluster with a VMkernel network adapter that is tagged for vSAN. .

In the vSphere Client, you configure VMkernel networking for vSAN on each host by using a standard switch. You can also do this by using a vSphere Distributed Switch for easier and consistent configuration. In both cases, before configuring the cluster for vSAN, you must configure the hosts with VMkernel network adapters for vSAN.



When you configure vSAN on a cluster, the Configure vSAN wizard validates the networking configuration on the hosts. If some of the hosts do not have the VMkernel network adapter enabled for vSAN, then you must suspend the configuration of the cluster, and set up the host networking for the vSAN traffic.

In your client applications, you can set up preselected VMkernel network adapters for the vSAN traffic.

```
# Update configuration spec for VMkernel networking
configInfo = vim.vsan.host.ConfigInfo(
    networkInfo=vim.vsan.host.ConfigInfo.NetworkInfo(port=[
        vim.vsan.host.ConfigInfo.NetworkInfo.PortConfig(device=args.vmknic)
    ]))

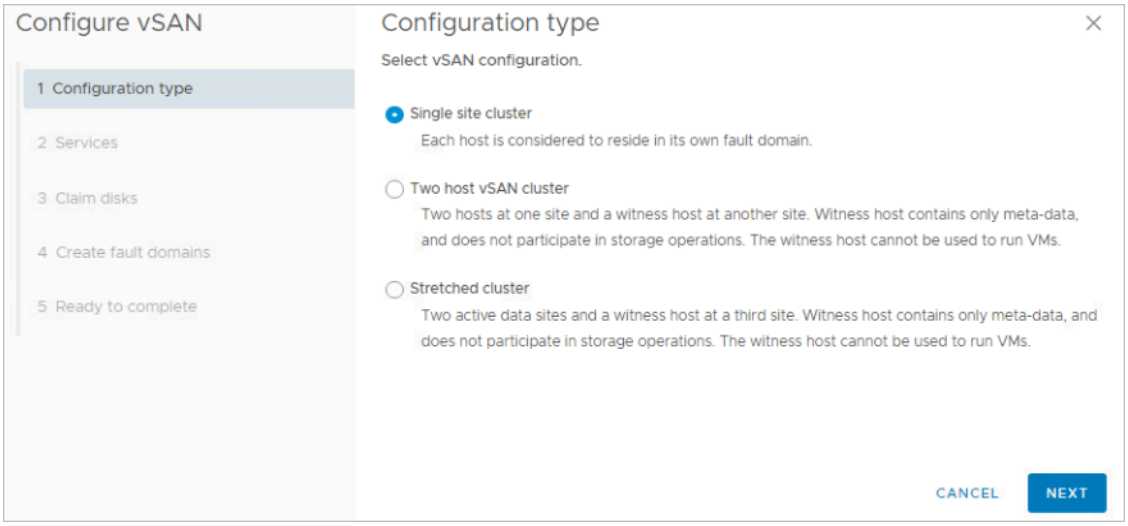
# Enumerate the selected VMkernel adapter for each host and add it to the list of tasks
for hosts in hosts:
    print 'Enable vSAN traffic on host {} with {}'.format(
        hostProps[host]['name'], args.vmknic)
task = hostProps[host]['configManager.vsanSystem'].UpdateVsan_Task(configInfo)
tasks.append(task)

# Execute the tasks
vsanapiutils.WaitForTasks(tasks, si)
```

Enabling vSAN Original Storage Architecture on a Cluster

After filtering the clusters that you want to configure for vSAN, the next step is to enable vSAN on these clusters.

In the VSphere Client, you use the **Configure vSAN** wizard to configure individual clusters for vSAN.



To enable vSAN in your vSAN Management API client applications, build an object of type `VimVsanReconfigSpec` by passing a `VsanClusterConfigInfo` parameter with the property `enable` set to `true`.

```
# Build vsanReconfigSpec step by step.  It takes effect only after calling the VsanClusterReconfig method
clusterConfig = vim.VsanClusterConfigInfo(enabled=True)
vsanReconfigSpec = vim.VimVsanReconfigSpec(modify=True, vsanClusterConfig=clusterConfig)
```

Enabling vSAN Express Storage Architecture on a Cluster

If your cluster is compatible, you can enable vSAN Express Storage Architecture (ESA).

vSAN Express Storage Architecture is optimized for high-performance flash storage devices that provide greater performance and efficiency.

To enable vSAN ESA in your vSAN Management API client applications, build an object of type `VimVsanReconfigSpec` by passing a `VsanClusterConfigInfo` parameter with the properties `enable` set to `true` and `vsanEsaEnabled` set to `true`.

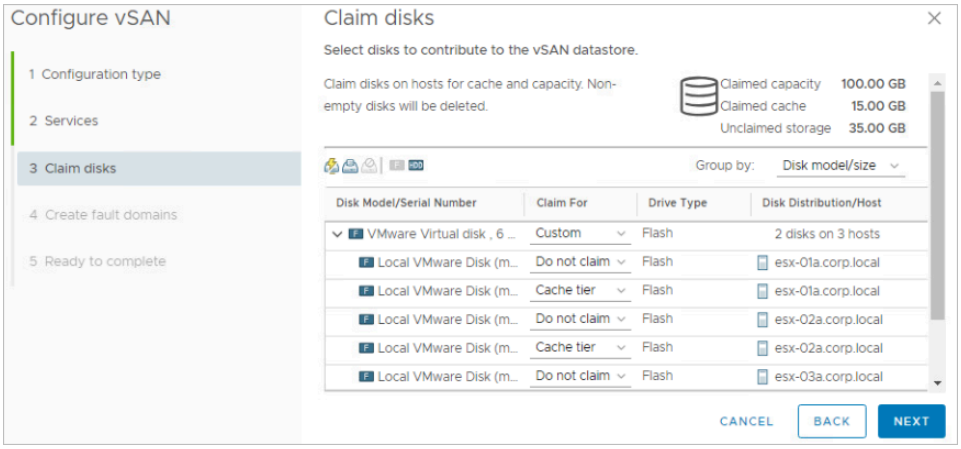
```
clusterConfig = vim.VsanClusterConfigInfo(enabled=True,vsanEsaEnabled=True)
vsanReconfigSpec = vim.VimVsanReconfigSpec(modify=True, vsanClusterConfig=clusterConfig)
```

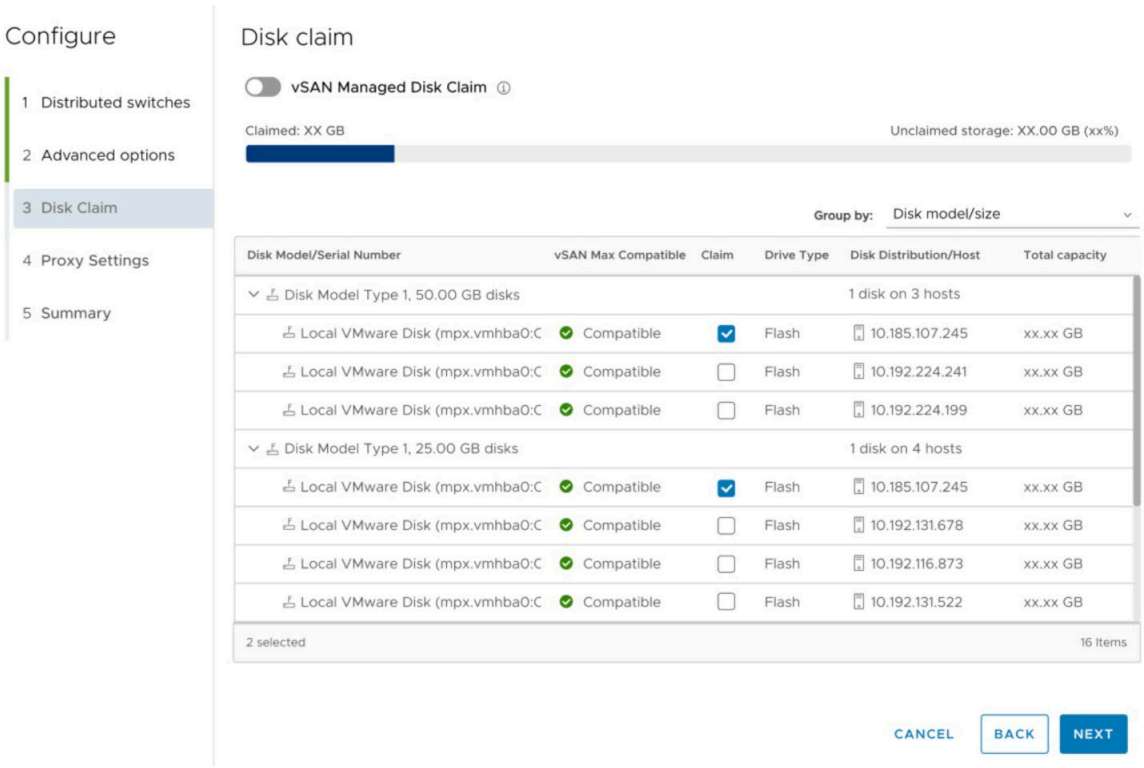
Claiming and Managing Disks

You can add disks to the vSAN cluster during the initial configuration, or you can add them later on.

When claiming disks by using the **Configure vSAN** wizard in the vSphere Client, you can only see the disks that are eligible, meaning they do not have existing vSAN partitions. vCenter filters out the non-eligible disks and these disks are not exposed for adding to the vSAN cluster.

For both hybrid and all-flash vSAN clusters, you assign the devices for cache and capacity tiers.





While claiming disks using the client applications, do the following:

Query for ineligible disks, and if required, clear the existing vSAN partitions on them.

```
# Enumerate the ineligible disks
for host in hosts:
    disks = [
        result.disk
        for result in hostProps[host]['configManager.vsanSystem']
        .QueryDisksForVsan() if result.state == 'ineligible'
    ]
    print 'Find ineligible disks {} in host {}'.format(
        [disk.displayName for disk in disks], hostProps[host]['name'])

# For each disk, interactively ask admin whether to individually wipe ineligible disks or not
for disk in disks:
    if yes('Do you want to wipe disk {}?\nPlease Always check the partition table and the data'
```

```
            ' stored on those disks before doing any wipe! (yes/no)?'.format(
                disk.displayName)):
        hostProps[host]['configManager.storageSystem'].UpdateDiskPartitions(
            disk.deviceName, vim.HostDiskPartitionSpec())
```

If the vSAN cluster is all-flash configuration, then segregate the devices as small and large. If the vSAN cluster is hybrid configuration then divide the devices as flash and HDD.

```
diskmap = {host: {'cache': [], 'capacity': []} for host in hosts}
cacheDisks = []
capacityDisks = []

# For all flash architectures
if isallFlash:
    for host in hosts:
        ssds = [result.disk for result in hostProps[host]
        ['configManager.vsanSystem'].QueryDisksForVsan() if
                result.state == 'eligible' and result.disk.ssd]
smallerSize = min([disk.capacity.block * disk.capacity.blockSize for disk in ssds])
for ssd in ssds:
    size = ssd.capacity.block * ssd.capacity.blockSize
if size == smallerSize:
    diskmap[host]['cache'].append(ssd)
cacheDisks.append((ssd.displayName, sizeof_fmt(size), hostProps[host]['name']))
else:
diskmap[host]['capacity'].append(ssd)
capacityDisks.append((ssd.displayName, sizeof_fmt(size), hostProps[host]['name']))
else:

# For hybrid architectures
for host in hosts:
    disks = [result.disk for result in hostProps[host]
    ['configManager.vsanSystem'].QueryDisksForVsan() if
             result.state == 'eligible']
ssds = [disk for disk in disks if disk.ssd]
hdds = [disk for disk in disks if not disk.ssd]
for disk in ssds:
    diskmap[host]['cache'].append(disk)
size = disk.capacity.block * disk.capacity.blockSize
cacheDisks.append((disk.displayName, sizeof_fmt(size), hostProps[host]['name']))
for disk in hdds:
    diskmap[host]['capacity'].append(disk)
size = disk.capacity.block * disk.capacity.blockSize
capacityDisks.append((disk.displayName, sizeof_fmt(size), hostProps[host]['name']))
for host, disks in diskmap.iteritems():
    if disks['cache'] and disks['capacity']:
        dm = vim.VimVsanHostDiskMappingCreationSpec(
            cacheDisks=disks['cache'], capacityDisks=disks['capacity'],
            creationType='allFlash' if isallFlash else 'hybrid',
            host=host)

# Execute the task
task = vsanVcDiskManagementSystem.InitializeDiskMappings(dm)
tasks.append(task)
```

If you use a vSAN Express Storage Architecture, you can claim disks either manually or automatically (vSAN managed disk claim). In the manual method, you can select a group of storage devices to be claimed. In automatic disk claim, vSAN automatically claims the compatible disks available in those hosts and adds the storage to the vSAN datastore.

```
"""
Demonstrates AddStoragePoolDisks API
Add disks to Storage Pool
If the task of disk addition fails, any exception will be logged.

Args:
   cluster (vim.ClusterComputeResource): vSAN cluster instance.
   vdms: vsan-disk-management-system MO instance.
   spec (vim.vsan.host.AddStoragePoolDiskSpec): Specifies the data evacuation mode.

Returns:
   None.
"""
def addDiskToStoragePool(cluster, vdms, spec):
    try:
        tsk = vdms.AddStoragePoolDisks([spec])
        addDiskTask = vim.Task(tsk._moId, cluster._stub)
        task.WaitForTask(addDiskTask)
        print("AddDisk to storage pool operation completed")
    except Exception as e:
        print("AddDisk to storage pool operation failed: %s" % e)
```

Enabling Deduplication and Compression on All-Flash Clusters

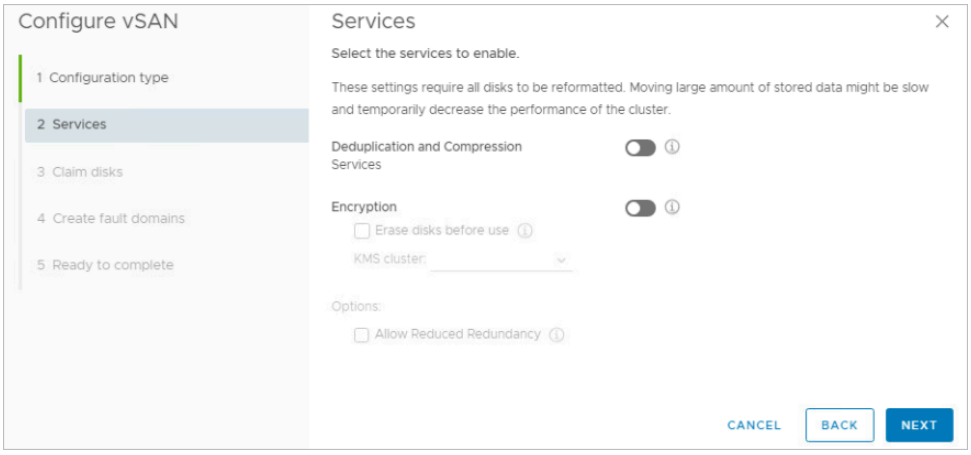
It is often advantageous to enable Deduplication and Compression on All-Flash vSAN deployments. Reduced capacity utilization can often make All-Flash vSAN more cost effective than Hybrid vSAN deployments.

If you are using the Configure vSAN wizard, the Enable Deduplication and Compression option can be selected while creating the vSAN cluster. However, after creating vSAN cluster, the process of enabling or disabling the Deduplication and Compression option using the Configure vSAN wizard can be time consuming. In some scenarios it might lead to reduced availability.

> **Note:**
> If you use the vSAN Express Storage Architecture (ESA), compression is enabled by default on the cluster. To disable compression, you can use storage policies.

By using the vSAN Management API, the process of enabling the Deduplication and Compression is simple.

In the VSphere Client, you enable deduplication and compression in the **Configure vSAN** wizard, before you claim any disks for the cluster.



To enable deduplication and compression with the vSAN Management API, you set the `dataEfficiencyConfig` property of the `vsanReconfigSpec` object with an object of type `VsanDataEfficiencyConfig`.

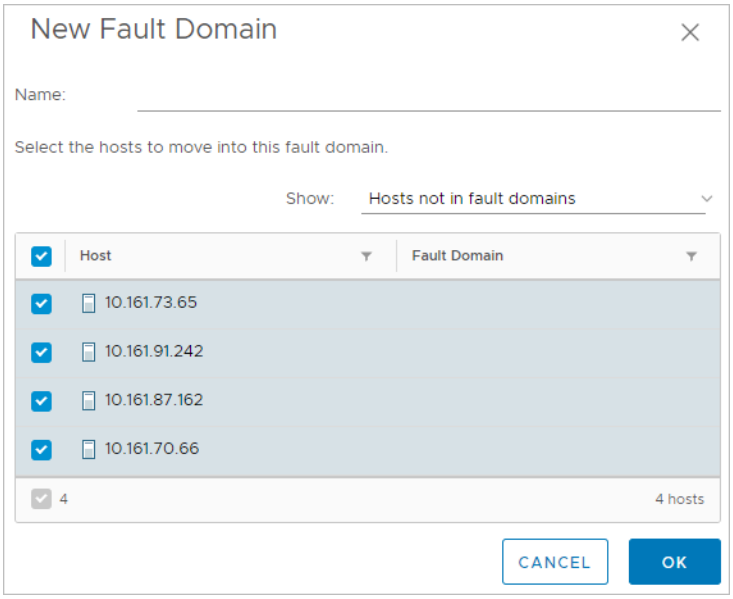
```
if isallFlash:
    print 'Enable deduplication and compression for VSAN'
vsanReconfigSpec.dataEfficiencyConfig = vim.VsanDataEfficiencyConfig(
    compressionEnabled=args.enabledc, dedupEnabled=args.enabledc)

# Enable/disable deduplication and compression
task = vsanClusterSystem.VsanClusterReconfig(cluster, vsanReconfigSpec)
vsanapiutils.WaitForTasks([task], si)
```

Configuring Fault Domains

If your vSAN cluster spans across multiple racks or blade server chassis, you can logically group the hosts in fault domains to protect them against rack or chassis failure. You can separate the vSAN hosts in the same manner that they are physically separated.

In the vSphere Client, you can group hosts in fault domains during the initial configuration of the vSAN cluster or later.



Following is an example of how to configure fault domains by using the vSAN Management API:

```
# Perform these tasks if fault domains are passed as an argument
if args.faultdomains:
    print 'Add fault domains in vsan'
faultDomains = []
# args.faultdomains is a string like f1:host1,host2 f2:host3,host4
for faultdomain in args.faultdomains.split():
    fname, hostnames = faultdomain.split(':')
domainSpec = vim.cluster.VsanFaultDomainSpec(
    name=fname,
    hosts=[
        host for host in hosts
        if hostProps[host]['name'] in hostnames.split(',')
    ])
faultDomains.append(domainSpec)

# Apply domain specification to vSAN Config
vsanReconfigSpec.faultDomainsSpec = vim.VimClusterVsanFaultDomainsConfigSpec(
    faultDomains=faultDomains)

# Configure fault domains
```

```
task = vsanClusterSystem.VsanClusterReconfig(cluster, vsanReconfigSpec)
vsanapiutils.WaitForTasks([task], si)
```

Assigning the vSAN License

To continue using the vSAN cluster after the 60-day evaluation period, you must assign a vSAN license.

Using the vSAN Management API, you can automate the license assignment on the vSAN clusters in your environment. This helps you handle license upgrades and renewal more efficiently.

```
if args.vsanlicense:
    print 'Assign VSAN license'
lm = si.content.licenseManager
lam = lm.licenseAssignmentManager
lam.UpdateAssignedLicense(entity=cluster._moId, licenseKey=args.vsanlicense)
```

Configuring Stretched and Two-Host Clusters

You can automate the configuration of stretched and two-host vSAN clusters using the vSAN Management API.

Examples are in Python.

Deploying the vSAN Witness Appliance

Deploying the vSAN Witness Appliance is an alternative to using a physical host to serve as the witness host in your stretched cluster configuration. Unlike a physical host, the appliance does not require a dedicated license or physical disks to store vSAN data.

You can download the vSAN Witness Appliance from the VMware website as a standard OVA file. Then you can install it by using the vSphere Client just like any other OVA file.

You can also upload the vSAN Witness Appliance OVA file through a script. Start with uploading each of the consisting OVA files separately in vCenter. First, create a function that uploads a single file in vCenter.

```
def uploadFile(srcURL, dstURL, create, lease, minProgress, progressIncrement, vmName=None, log=None):


    '''
    This function will upload vmdk file to vc by using http protocol
    @param srcURL: source url
    @param dstURL: destnate url
    @param create: http request method
    @param lease: HttpNfcLease object
    @param minProgress: file upload progress initial value
    @param progressIncrement: file upload progress update value
    @param vmName: imported virtual machine name
    @param log: log object @return:
    '''
srcData = urllib2.urlopen(srcURL)
length = int(srcData.headers['content-length'])
ssl._create_default_https_context = ssl._create_unverified_context
protocol, hostPort, reqStr = splitURL(dstURL)
dstHttpConn = createHttpConn(protocol, hostPort)
reqType = create and 'PUT' or 'POST'
dstHttpConn.putrequest(reqType, reqStr)
dstHttpConn.putheader('Content-Length', length)
dstHttpConn.endheaders()
bufSize = 1048768  # 1 MB
total = 0
```

```
progress = minProgress
if log:
# If args.log is available, then log to it
log = log.info
else
log = sys.stdout.write
log("%s: %s: Start: srcURL=%s dstURL=%s\n" % (time.asctime(time.localtime()), vmName, srcURL,
                                              dstURL))
log("%s: %s: progress=%d total=%d length=%d\n" % (time.asctime(time.localtime()), vmName, progress,
                                                  total, length))
while True:
    data = srcData.read(bufSize)
if lease.state != vim.HttpNfcLease.State.ready:
    break
dstHttpConn.send(data)
total = total + len(data)
progress = (int)(total * (progressIncrement) / length)
progress += minProgress
lease.Progress(progress)
if len(data) == 0:
    break
log("%s: %s: Finished: srcURL=%s dstURL=%s\n" % (time.asctime(time.localtime()), vmName, srcURL,
                                                 dstURL))
log("%s: %s: progress=%d total=%d length=%d\n" % \ (time.asctime(time.localtime()), vmName,
                                                    progress, total, length))
log("%s: %s: Lease State: %s\n" % \
    (time.asctime(time.localtime()), vmName, lease.state))
if lease.state == vim.HttpNfcLease.State.error:
    raise lease.error
dstHttpConn.getresponse()
return progress
```

Once you have a function for deploying a single file, create another one for uploading multiple files.

```
def uploadFiles(fileItems, lease, ovfURL, vmName=None, log=None):
    '''
    Upload witness vm's vmdk files to vCenter by using the HTTP protocol
    @param fileItems: the source vmdks read from ovf file
    @param lease: Represents a lease on a VM or a vApp, which can be used to import or export disks for
    the entity
    @param ovfURL: witness vApp ovf url
    @param vmName: The name of witness vm @param log: @return:
    '''



uploadUrlMap = {}
for kv in lease.info.deviceUrl:
    uploadUrlMap[kv.importKey] = (kv.key, kv.url)
progress = 5
increment = (int)(90 / len(fileItems))
for file in fileItems:
    ovfDevId = file.deviceId
srcDiskURL = urlparse.urljoin(ovfURL, file.path)
(viDevId, url) = uploadUrlMap[ovfDevId]
if lease.state == vim.HttpNfcLease.State.error:
    raise lease.error
elif lease.state != vim.HttpNfcLease.State.ready:
    raise Exception("%s: file upload aborted, lease state=%s" % (vmName,
```

```
                                                                    lease.state))
progress = uploadFile(srcDiskURL, url, file.create, lease, progress, increment,
                      vmName, log)
```

The next step is to define and implement a function that configures the networking settings, the supplied password as a vApp option, and the placement of the appliance on a specific host or resource pool.

The `DeployWitnessOVF` function, defined in the following example, configures the networking settings, the supplied password as a vApp option, and the placement of the appliance on a specific host or resource pool. This function parses only the contents of the OVF and not the entire vSAN Witness Appliance OVA. Password is the only additional argument required by the vSAN Witness Appliance. You must extract the contents of the witness OVA file to a folder containing the OVF and other required files. The OVA file is a `.tar` archive, that you can extract by using a wide variety of tools.

```
"""
Deploying  witness VM to vCenter.
The import process consists of the following steps:
1>Creating the VMs and/or vApps that make up the entity.
2>Uploading the virtual disk contents.
@param ovfURL: ovf source url
@param si: Managed Object ServiceInstance
@param host: HostSystem on which the VM located
@param vmName: VM name
@param dsRef: Datastore on which the VM located
@param vmFolder: Folder to which the VM belong to
@param vmPassword: Password for the VM
@param network: Managed Object Network of the VM
@return: Witness VM entity
"""


def DeployWitnessOVF(ovfURL, si, host, vmName, dsRef, vmFolder, vmPassword=None, network=None):
    rp = host.parent.resourcePool
    params = vim.OvfManager.CreateImportSpecParams()
    params.entityName = vmName
    params.hostSystem = host
    params.diskProvisioning = 'thin'

    f = urllib.urlopen(ovfURL)
    ovfData = f.read()

    import xml.etree.ElementTree as ET

    params.networkMapping = []
    if vmPassword:
        params.propertyMapping = [vim.KeyValue(key='vsan.witness.root.passwd', value=vmPassword)]
    ovf_tree = ET.fromstring(ovfData)

    for nwt in ovf_tree.findall('NetworkSection/Network'):
        nm = vim.OvfManager.NetworkMapping()
        nm.name = nwt.attrib['name']
        if network != None:
            nm.network = network
        else:
            nm.network = host.parent.network[0]
        params.networkMapping.append(nm)

    res = si.content.ovfManager.CreateImportSpec(ovfDescriptor=ovfData,
                                                 resourcePool=rp, datastore=dsRef, cisp=params)
    if isinstance(res, vim.MethodFault):
```

```
    raise res
if res.error and len(res.error) & gt; 0:
    raise res.error[0]
if not res.importSpec:
    raise Exception("CreateImportSpec raised no errors, but importSpec is not set")

lease = rp.ImportVApp(spec=res.importSpec, folder=vmFolder, host=host)
while lease.state == vim.HttpNfcLease.State.initializing:
    time.sleep(1)

if lease.state == vim.HttpNfcLease.State.error:
    raise lease.error

# Upload files
uploadUrlMap = {}
for kv in lease.info.deviceUrl:
    uploadUrlMap[kv.importKey] = (kv.key, kv.url)

progress = 5
increment = (int)(90 / len(res.fileItem))
for file in res.fileItem:
    ovfDevId = file.deviceId
    srcDiskURL = urlparse.urljoin(ovfURL, file.path)
    (viDevId, url) = uploadUrlMap[ovfDevId]
    if lease.state == vim.HttpNfcLease.State.error:
        raise lease.error
    elif lease.state != vim.HttpNfcLease.State.ready:
        raise Exception("%s: file upload aborted, lease state=%s" % \
                        (vmName, lease.state))
    srcData = urllib2.urlopen(srcDiskURL)
    length = int(srcData.headers['content-length'])
    result = urlparse.urlparse(url)
    protocol, hostPort, reqStr = result.scheme, result.netloc, result.path
    if protocol == 'https':
        dstHttpConn = httplib.HTTPSConnection(hostPort)
    else:
        dstHttpConn = httplib.HTTPConnection(hostPort)
    reqType = file.create and 'PUT' or 'POST'
    dstHttpConn.putrequest(reqType, reqStr)
    dstHttpConn.putheader('Content-Length', length)
    dstHttpConn.endheaders()

    bufSize = 1048768  # 1 MB
    total = 0
    currProgress = progress
    while True:
        data = srcData.read(bufSize)
        if lease.state != vim.HttpNfcLease.State.ready:
            break
        dstHttpConn.send(data)
        total = total + len(data)
        currProgress += (int)(total * (increment) / length)
        progress += minProgress
        lease.Progress(progress)
        if len(data) == 0:
            break
    if lease.state == vim.HttpNfcLease.State.error:
```

```
            raise lease.error

        dstHttpConn.getresponse()
        progress = currProgress
    lease.Complete()

    return lease.info.entity
```

Adding the vSAN Witness Appliance to vCenter

After you deploy the vSAN Witness Appliance, you must add it to vCenter to serve as the witness host in your stretched cluster or two-host configuration. The witness host must not be part of the vSAN cluster.

You can use the vSphere Client to add the vSAN Witness Appliance as a host to vCenter.

To add the host programmatically, first create a function that adds the vSAN Witness Appliance as a host in vCenter.

```
def AddHost(host, user='root', pwd=None, dcRef=None, si=None, sslThumbprint=None, port=443):
    ''' Add a host to a data center Returns a host system '''


cnxSpec = vim.HostConnectSpec(
    force=True, hostName=host, port=port, userName=user, password=pwd, vmFolder=dcRef.vmFolder)
if sslThumbprint:
    cnxSpec.sslThumbprint = sslThumbprint
hostParent = dcRef.hostFolder
try:
    task = hostParent.AddStandaloneHost(addConnected=True, spec=cnxSpec)
vsanapiutils.WaitForTasks([task], si)
return getHostSystem(host, dcRef, si)
except vim.SSLVerifyFault as e:
# By catching this exception, you do not need to input the host's thumb print of the SSL certificate
# The following script does this automatically.
cnxSpec.sslThumbprint = e.thumbprint
task = hostParent.AddStandaloneHost(addConnected=True, spec=cnxSpec)
vsanapiutils.WaitForTasks([task], si)
return getHostSystem(host, dcRef, si)
except vim.DuplicateName as e:
raise Exception("AddHost: ESX host %s has already been added to VC." % host)
```

Then add the host by calling the function.

```
print 'Add witness host {} to datacenter {}'.format(witnessVm.name,
                                                    args.witnessdc)
dcRef = searchIndex.FindChild(
    entity=si.content.rootFolder, name=args.witnessdc)
witnessHost = AddHost(
    witnessVm.guest.ipAddress, pwd=args.vmpassword, dcRef=dcRef, si=si)
```

Configuring a vSAN Stretched Cluster or Two-Host Cluster

You can configure a vSAN stretched cluster or two-host cluster.

Following is the process for configuring an existing vSAN cluster as a stretched cluster:

1. Select the hosts that participate in the preferred fault domain.
2. Select the hosts that participate in the secondary fault domain.
3. Select the witness host and configure cache and capacity disks for it.
4. Finish the configuration.

To configure a stretched cluster or two-host setup by using the vSAN Management API, enumerate the hosts in the cluster, select which hosts to add to each fault domain, and then save this data to an array.

```
preferedFd = args.preferdomain
secondaryFd = args.seconddomain
firstFdHosts = []
secondFdHosts = []
for host in hosts:
    if yes('Add host {} to preferred fault domain ? (yes/no)'.format(hostProps[host]['name'])):
    firstFdHosts.append(host)
for host in set(hosts) - set(firstFdHosts):
    if yes('Add host {} to second fault domain ? (yes/no)'.format(hostProps[host]['name'])):
    secondFdHosts.append(host)
faultDomainConfig = vim.VimClusterVSANStretchedClusterFaultDomainConfig(
    firstFdHosts=firstFdHosts,
    firstFdName=preferedFd,
    secondFdHosts=secondFdHosts,
    secondFdName=secondaryFd)
```

The next step is to define the eligible disks for the witness host.

```
disks = [result.disk for result in witnessHost.configManager.vsanSystem.QueryDisksForVsan() if
         result.state == 'eligible']
diskMapping = None
if disks:
    ssds = [disk for disk in disks if disk.ssd]
nonSsds = [disk for disk in disks if not disk.ssd]
# host with hybrid disks
if len(ssds) > 0 and len(nonSsds) > 0:
    diskMapping = vim.VsanHostDiskMapping(
        ssd=ssds[0],
        nonSsd=nonSsds
    )
# host with all-flash disks,choose the ssd with smaller capacity for cache layer.
if len(ssds) > 0 and len(nonSsds) == 0:
    smallerSize = min([disk.capacity.block * disk.capacity.blockSize for disk in ssds])
smallSsds = []
biggerSsds = []
for ssd in ssds:
    size = ssd.capacity.block * ssd.capacity.blockSize
if size == smallerSize:
    smallSsds.append(ssd)
biggerSsds.append(ssd)
diskMapping = vim.VsanHostDiskMapping(
    ssd=smallSsds[0]
nonSsd = biggerSsds
)
```

After adding the host to the fault domain arrays and defining the eligible disks for the witness host, configure the stretched cluster.

```
print 'start to create stretched cluster'
task = vsanScSystem.VSANVcConvertToStretchedCluster(
    cluster=cluster,
    faultDomainConfig=faultDomainConfig,
    witnessHost=witnessHost, preferredFd=preferedFd,
    diskMapping=diskMapping)
vsanapiutils.WaitForTasks([task], si)
```

Converting Two-Host vSAN Clusters to ROBO Cluster

You can convert two-host vSAN clusters to ROBO clusters with a single shared witness.

Here is an example for converting in batches, one or more regular two-host vSAN clusters to Remote Office/Branch Office (ROBO) clusters with a single shared witness.

```
def convertToRoboClusterInBatch(si, vscs, witness, clusterRefs):
   """ Convert multiple two-node vsan clusters to robo clusters
       that share the same witness in batch.

   Requirements:
      1. The candidate cluster must be a two-node cluster with vsan enable.
      2. There is no network isolation between witness and the multiple
      clusters given.
   """
   checkCompatibility(si, vscs, clusterRefs, witness)
   for cluster in clusterRefs:
      if len(vscs.GetWitnessHosts(cluster)) != 0:
         msg = "ERROR: cluster %s is not a regular vSAN cluster" % cluster.name
         sys.exit(msg)
   print("Converting normal vSAN clusters(2 nodes) '%s' to robo clusters" \
         " with shared witness %s" % \
         ([cluster.name for cluster in clusterRefs], witness.name))
   spec = vim.vsan.VsanVcStretchedClusterConfigSpec(
      witnessHost = witness,
      clusters = [vim.cluster.VsanStretchedClusterConfig(
         cluster = cluster,
         preferredFdName= 'fd1',
         faultDomainConfig=
         vim.cluster.VSANStretchedClusterFaultDomainConfig(
            firstFdName = 'fd1',
            firstFdHosts = [cluster.host[0]],
            secondFdName = 'fd2',
            secondFdHosts = [cluster.host[1]],
         ),
      ) for cluster in clusterRefs]
   )
   addWitnessTask = vscs.AddWitnessHostForClusters(spec)
   vsanapiutils.WaitForTasks([addWitnessTask], si)
```

Replacing Witnesses of Multiple ROBO Clusters with a Single Shared Witness

You can replace witnesses of multiple ROBO clusters with a single shared witness.

Here is an example for replacing in batches, multiple witnesses of ROBO clusters with a single shared witness:

```
def replaceWitnessInBatch(si, vscs, witness, clusterRefs):
   """ Replace witness with the same Shared witness in batches for
       for multiple robo clusters in one operation.

   Requirements:
      1. The candidate cluster must be vSAN robo cluster.
      2. There is no network isolation between witness and the multiple
      clusters given.
   """
   checkCompatibility(si, vscs, clusterRefs, witness)
   for cluster in clusterRefs:
```

```
      if len(vscs.GetWitnessHosts(cluster)) != 1:
         msg = "ERROR: cluster %s is not a robo cluster" % cluster.name
         sys.exit(msg)
   print("Replacing the old witness(es) with shared witness %s" \
         " for clusters: %s" % (witness.name,
         [cluster.name for cluster in clusterRefs]))
   spec = vim.vsan.VsanVcStretchedClusterConfigSpec(
      witnessHost = witness,
      clusters = [vim.cluster.VsanStretchedClusterConfig(
         cluster = cluster
      ) for cluster in clusterRefs]
   )
   replaceWitnessTask = vscs.ReplaceWitnessHostForClusters(spec)
   vsanapiutils.WaitForTasks([replaceWitnessTask], si)

def removeWitnessForClusters(si, vscs, witness, clusterRefs):
   totalTasks = []
   for cluster in clusterRefs:
      print("Removing witness %s from cluster %s" % \
            (witness.name, cluster.name))
      removeTask = vscs.RemoveWitnessHost(cluster, witness)
      totalTasks.append(vsanapiutils.ConvertVsanTaskToVcTask(
                        removeTask, si._stub))
   vsanapiutils.WaitForTasks(totalTasks, si)

def getWitnessClusters(si, vscs, witness):
   clusterNames = []
   getWitnessClustrs = vscs.QueryWitnessHostClusterInfo(witness)
   for cluster in getWitnessClustrs:
      clusterMo = vim.ClusterComputeResource(cluster.cluster._moId, si._stub)
      clusterNames.append(clusterMo.name)
   return clusterNames

class LogWitnessStatus(object):
   def __init__(self, si, vscs, witness):
      self.si = si
      self.vscs = vscs
      self.witness = witness

   def __enter__(self):
      print("Before Ops: shared witness %s has joined the following clusters:"
            " %s" % (self.witness.name,
            getWitnessClusters(self.si, self.vscs, self.witness)))

   def __exit__(self, *a):
      print("After Ops: Now shared witness %s has joined the following"
            " clusters: %s" % (self.witness.name,
            getWitnessClusters(self.si, self.vscs, self.witness)))

def main():
   args = GetArgs()
   if args.password:
      password = args.password
   else:
      password = getpass.getpass(prompt='Enter password for host %s and '
                                        'user %s: ' % (args.host,args.user))
   # For python 2.7.9 and later, the default SSL context has more strict
```

```
# connection handshaking rule. We may need turn off the hostname checking
# and client side cert verification.
context = None
if sys.version_info[:3] > (2,7,8):
   context = ssl.create_default_context()
   context.check_hostname = False
   context.verify_mode = ssl.CERT_NONE

si = SmartConnect(host=args.host,
                  user=args.user,
                  pwd=password,
                  port=int(args.port),
                  sslContext=context)
atexit.register(Disconnect, si)

# Detecting whether the host is vCenter or ESX.
aboutInfo = si.content.about
apiVersion = vsanapiutils.GetLatestVmodlVersion(args.host)

if aboutInfo.apiType == 'VirtualCenter':
   majorApiVersion = aboutInfo.apiVersion
   if LooseVersion(majorApiVersion) < LooseVersion('7.0.1'):
      msg = "The Virtual Center with version %s (lower than 7.0U1) is not "\
            "supported." % aboutInfo.apiVersion
      sys.exit(msg)
   # Get vSAN health system from the vCenter Managed Object references.
   vcMos = vsanapiutils.GetVsanVcMos(
         si._stub, context=context, version=apiVersion)
   vscs = vcMos['vsan-stretched-cluster-system']

   witness = getComputeInstance(args.witness, si)
   if not witness:
      msg = 'Given witness host %s is not found in %s' % \
            (args.witness, args.vc)
      sys.exit(msg)
   witness = witness.host[0]
   allClusters= []
   if args.roboClusters:
      roboClusters = [clusterName.strip() for clusterName \
                      in args.roboClusters.split(',')]
      roboClusters = getClusterInstances(roboClusters, si)
      allClusters.extend(roboClusters)
      with LogWitnessStatus(si, vscs, witness):
         replaceWitnessInBatch(si, vscs, witness, roboClusters)

   if args.normalClusters:
      twoNodesClusters = [clusterName.strip() for clusterName \
                          in args.normalClusters.split(',')]
      twoNodesClusters = getClusterInstances(twoNodesClusters, si)
      allClusters.extend(twoNodesClusters)
      with LogWitnessStatus(si, vscs, witness):
         convertToRoboClusterInBatch(si, vscs, witness, twoNodesClusters)

   with LogWitnessStatus(si, vscs, witness):
      removeWitnessForClusters(si, vscs, witness, allClusters)
else:
   print('Remote host should be a Virtual Center ')
```

```
      return -1
```

Sharing Remote Datastores with HCI Mesh

vSAN HCI Mesh allows you to mount a remote vSAN and non-vSAN datastore to a vSAN cluster. By using datastore sharing you can now mount the other datastore and use it as if it is a local datastore. You can provision VMs running on the local cluster using storage space on the remote datastore.

Mounting and Unmounting a Remote vSAN Datastore Into a vSAN Client Cluster

The HCI Mesh feature enables vSAN clusters to share their datastores with other vSAN clusters. The following are examples of programmatically running precheck before mounting the remote datastore, validating the precheck result, mounting the remote datastore, and unmounting the remote datastore:

Performing Precheck

Example for programatically performing precheck before mounting a remote vSAN datastore into a vSAN client cluster.

```
vrds = vcMos['vsan-remote-datastore-system']
   # Get local vSAN datastore from the server cluster
   localDatastore = getLocalVsanDatastore(serverCluster)

   # Run MountPrecheck API and verify the result for failures
   if len(localDatastore) > 0:
      print('Running MountPrecheck on cluster: %s' % clientCluster.name)
      result = vrds.MountPrecheck(clientCluster, localDatastore[0])
```

Validating Precheck

Example for programatically validating the precheck.

```
def verifyPrecheckFailedResult(result):
      """
      For checking the MountPrecheck failed result in detail
      E.g. Some connectivity issue in a cluster Like, cluster partition, etc.
      Red: Indicates severe warnings
      Yellow: Indicates light warnings
      Green: Indicates no warnings
      """
      status = True
      for precheckItem in result.result:
         if precheckItem.status == "red":
            print('Precheck Item failed: %s' % precheckItem.type)
            print(precheckItem.reason)
            status = False
      return status
```

Mounting a Remote vSAN Datastore

Example for programatically mounting a remote vSAN datastore into a vSAN client cluster.

```
# Get local vSAN datastore from the server cluster
   localDatastore = getLocalVsanDatastore(serverCluster)

   # Run MountPrecheck API and verify the result for failures
   if len(localDatastore) > 0:
      vsccs = vcMos['vsan-cluster-config-system']
      vsanConfig = vim.vsan.cluster.ConfigInfo(enabled=None)

      # Mounting a remote datastore
      print('Mounting remote datastore on cluster: %s'
```

```
            % clientCluster.name)
      dsConfig = vim.vsan.AdvancedDatastoreConfig(
                  remoteDatastores=localDatastore)
      mountUnmountDatastore(si, vsccs, clientCluster, localDatastore,
                            vsanConfig, dsConfig)

   def mountUnmountDatastore(si, vsccs, cluster, dsList, vsanConfig, dsConfig):
      spec = vim.vsan.ReconfigSpec(vsanClusterConfig=vsanConfig,
                                   datastoreConfig=dsConfig)
      vsanTask = vsccs.ReconfigureEx(cluster, spec)
      vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
      vsanapiutils.WaitForTasks([vcTask], si)
      if vcTask.info.state != 'success':
         print('Failed to (un)mount remote datastore with error: %s'
               % vcTask.info.error)
         return -1
      print('Successfully (un)mounted remote vSAN datastore %s on cluster %s'
            % (dsList[0].name, cluster.name))
```

Unmounting a Remote vSAN Datastore

Example for programatically unmounting a remote vSAN datastore into a vSAN client cluster.

```
vsccs = vcMos['vsan-cluster-config-system']
   vsanConfig = vim.vsan.cluster.ConfigInfo(enabled=None)

   # Unmounting a remote datastore
   print('Unmounting remote datastore from cluster: %s'
         % clientCluster.name)
   dsConfig = vim.vsan.AdvancedDatastoreConfig(remoteDatastores=[])
   mountUnmountDatastore(si, vsccs, clientCluster, localDatastore,
                         vsanConfig, dsConfig)

   def mountUnmountDatastore(si, vsccs, cluster, dsList, vsanConfig, dsConfig):
      spec = vim.vsan.ReconfigSpec(vsanClusterConfig=vsanConfig,
                                   datastoreConfig=dsConfig)
      vsanTask = vsccs.ReconfigureEx(cluster, spec)
      vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
      vsanapiutils.WaitForTasks([vcTask], si)
      if vcTask.info.state != 'success':
         print('Failed to (un)mount remote datastore with error: %s'
               % vcTask.info.error)
         return -1
      print('Successfully (un)mounted remote vSAN datastore %s on cluster %s'
            % (dsList[0].name, cluster.name))
```

Mounting and Unmounting a Remote vSAN Datastore Into a Non-vSAN Client Cluster

vSAN 7.0 Update 2 and later enables you to configure a non-vSAN cluster as an HCI Mesh client. You can configure this non-vSAN cluster to compute-only mode. The hosts in an HCI Mesh compute-only client cluster do not need local storage. They can mount remote datastores from a vSAN cluster located within the same data center. The following are examples of programmatically configuring the non-vSAN client to compute-only mode, running precheck before mounting the remote datastore, validating the precheck result, mounting the remote datastore, and unmounting the remote datastore:

Configuring a Non-vSAN Client Cluster to Compute-Only Mode

Example for programatically configuring a non-vSAN client cluster to compute-only mode.

```
# Make the 'client' - non-vSAN cluster as Compute mode
   vccs = vcMos['vsan-cluster-config-system']
   vccs.GetConfigInfoEx(clientCluster)
   rs = vim.Vsan.ReconfigSpec(mode=vim.vsan.Mode.Mode_Compute)
   tsk = vccs.ReconfigureEx(clientCluster, rs)
   tsk = vim.Task(tsk._moId, clientCluster._stub)
   vsanapiutils.WaitForTasks([tsk], si)
```

Performing Precheck

Example for programatically performing precheck before mounting a remote vSAN datastore into a non-vSAN client cluster.

```
vrds = vcMos['vsan-remote-datastore-system']
   # Get remote datastore list from the server cluster
   vsanDatastore = getRemoteDatastores(serverCluster)

   # Run MountPrecheck API and verify the result for failures
   if vsanDatastore is not None:
      remoteVsanDatastore = vsanDatastore[0]
      print('Running MountPrecheck on cluster: %s' % clientCluster.name)
      result = vrds.MountPrecheck(clientCluster, remoteVsanDatastore)
```

Validating Precheck

Example for programatically validating the precheck.

```
def verifyPrecheckFailedResult(result):
      """
      For checking the MountPrecheck failed result in detail
      E.g. Some connectivity issue in a cluster Like, cluster partition, etc.
      Red: Indicates severe warnings
      Yellow: Indicates light warnings
      Green: Indicates no warnings
      """
      status = True
      for precheckItem in result.result:
         if precheckItem.status == "red":
            print('Precheck Item failed: %s' % precheckItem.type)
            print(precheckItem.reason)
            status = False
      return status
```

Mounting a Remote vSAN Datastore

Example for programatically mounting a remote vSAN datastore into a non-vSAN client cluster.

```
# Get remote datastore list from the server cluster
   vsanDatastore = getRemoteDatastores(serverCluster)

   # Run MountPrecheck API and verify the result for failures
   if vsanDatastore is not None:
      vsccs = vcMos['vsan-cluster-config-system']
      vsanConfig = vim.vsan.cluster.ConfigInfo(enabled=None)

      # Mounting a remote datastore
      print('Mounting remote datastore on cluster: %s'
            % clientCluster.name)
      dsConfig = vim.vsan.AdvancedDatastoreConfig(
                  remoteDatastores=vsanDatastore)
```

```
      mountUnmountDatastore(si, vsccs, clientCluster, vsanDatastore,
                            vsanConfig, dsConfig)

   def mountUnmountDatastore(si, vsccs, cluster, dsList, vsanConfig, dsConfig):
      spec = vim.vsan.ReconfigSpec(vsanClusterConfig=vsanConfig,
                                   datastoreConfig=dsConfig)
      vsanTask = vsccs.ReconfigureEx(cluster, spec)
      vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
      vsanapiutils.WaitForTasks([vcTask], si)
      if vcTask.info.state != 'success':
         print('Failed to (un)mount remote datastore with error: %s'
               % vcTask.info.error)
         return -1
      print('Successfully (un)mounted remote vSAN datastore %s on cluster %s'
            % (dsList[0].name, cluster.name))
```

Unmounting a Remote vSAN Datastore

Example for programatically unmounting a remote vSAN datastore into a non-vSAN client cluster.

```
vsccs = vcMos['vsan-cluster-config-system']
   vsanConfig = vim.vsan.cluster.ConfigInfo(enabled=None)

   # Unmounting a remote datastore
   print('Unmounting remote datastore from cluster: %s'
         % clientCluster.name)
   dsConfig = vim.vsan.AdvancedDatastoreConfig(remoteDatastores=[])
   mountUnmountDatastore(si, vsccs, clientCluster, vsanDatastore,
                         vsanConfig, dsConfig)

   def mountUnmountDatastore(si, vsccs, cluster, dsList, vsanConfig, dsConfig):
      spec = vim.vsan.ReconfigSpec(vsanClusterConfig=vsanConfig,
                                   datastoreConfig=dsConfig)
      vsanTask = vsccs.ReconfigureEx(cluster, spec)
      vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
      vsanapiutils.WaitForTasks([vcTask], si)
      if vcTask.info.state != 'success':
         print('Failed to (un)mount remote datastore with error: %s'
               % vcTask.info.error)
         return -1
      print('Successfully (un)mounted remote vSAN datastore %s on cluster %s'
            % (dsList[0].name, cluster.name))
```

Using Encryption in a vSAN Cluster

vSAN can encrypt data at rest, or data in transit across hosts in the cluster .

Setting Data in Transit Encryption

vSAN can encrypt data in transit, as it moves across hosts in your vSAN cluster. vSAN can also encrypt data in transit across hosts in the vSAN cluster. When you enable data-in-transit encryption, vSAN encrypts all data and metadata traffic between hosts.

Here is an example for programmatically setting the data in transit encryption in a vSAN cluster using the vSAN Management API.

```
# Import the vSAN API python bindings and utilities.
import vsanmgmtObjects
import vsanapiutils
```

```
def GetArgs():
   """
   Supports the command-line arguments listed below.
   """
   parser = argparse.ArgumentParser(
       description='Process args for vSAN SDK sample application')
   parser.add_argument('-s', '--host', required=True, action='store',
                       help='Remote host to connect to')
   parser.add_argument('-o', '--port', type=int, default=443, action='store',
                       help='Port to connect on')
   parser.add_argument('-u', '--user', required=True, action='store',
                       help='User name to use when connecting to host')
   parser.add_argument('-p', '--password', required=False, action='store',
                       help='Password to use when connecting to host')
   parser.add_argument('--cluster', dest='clusterName', metavar="CLUSTER",
                      default='VSAN-Cluster')
   args = parser.parse_args()
   return args

def getClusterInstance(clusterName, serviceInstance):
   content = serviceInstance.RetrieveContent()
   searchIndex = content.searchIndex
   datacenters = content.rootFolder.childEntity
   for datacenter in datacenters:
      cluster = searchIndex.FindChild(datacenter.hostFolder, clusterName)
      if cluster is not None:
         return cluster
   return None

def main():
   args = GetArgs()
   if args.password:
      password = args.password
   else:
      password = getpass.getpass(prompt='Enter password for host %s and '
                                        'user %s: ' % (args.host,args.user))

   # For python 2.7.9 and later, the default SSL context has more strict
   # connection handshaking rule. We may need turn off the hostname checking
   # and client side cert verification.
   context = None
   if sys.version_info[:3] > (2,7,8):
      context = ssl.create_default_context()
      context.check_hostname = False
      context.verify_mode = ssl.CERT_NONE

   si = SmartConnect(host=args.host,
                     user=args.user,
                     pwd=password,
                     port=int(args.port),
                     sslContext=context)

   atexit.register(Disconnect, si)

   # Detecting whether the host is vCenter or ESX.
   aboutInfo = si.content.about
```

```
apiVersion = vsanapiutils.GetLatestVmodlVersion(args.host)

if aboutInfo.apiType == 'VirtualCenter':
   majorApiVersion = aboutInfo.apiVersion
   if LooseVersion(majorApiVersion) < LooseVersion('6.7.1'):
      print('The Virtual Center with version %s (lower than 6.7U3) is not '
            'supported.' % aboutInfo.apiVersion)
      return -1

   # Get vSAN health system from the vCenter Managed Object references.
   vcMos = vsanapiutils.GetVsanVcMos(
         si._stub, context=context, version=apiVersion)
   vccs = vcMos['vsan-cluster-config-system']

   cluster = getClusterInstance(args.clusterName, si)

   if cluster is None:
      print('Cluster %s is not found for %s' % (args.clusterName, args.host))
      return -1

   clusterReconfigSpec = vim.vsan.ReconfigSpec()
   clusterReconfigSpec.dataInTransitEncryptionConfig = \
       vim.vsan.DataInTransitEncryptionConfig()

   #Set to True to enable and False to disable data-in-transit encryption.
   #If the value is left unset, it will leave current state unchanged.
   clusterReconfigSpec.dataInTransitEncryptionConfig.enabled = True

   #Periodical rekeying interval in minutes.
   #Default interval is 1440, i.e. 24 hours.
   #For release build, minimal interval is 30 minutes
   #and maximum is 10080, i.e. 7 days.
   clusterReconfigSpec.dataInTransitEncryptionConfig.rekeyInterval = 30

   ditEncryptionConfigTask = vccs.ReconfigureEx(cluster, clusterReconfigSpec)

   ditEncryptionConfigVcTask = vsanapiutils.ConvertVsanTaskToVcTask(
                        ditEncryptionConfigTask, si._stub)
   vsanapiutils.WaitForTasks([ditEncryptionConfigVcTask], si)

   print('Set vSAN data-in-transit encryption finished with '
         'status: %s' % ditEncryptionConfigVcTask.info.state)
else:
   print('Remove host should be a Virtual Center ')
   return -1
```

Managing iSCSI Service

vSAN iSCSI target service enables hosts and physical workloads that reside outside the vSAN cluster to access the vSAN datastore.

This service enables an iSCSI initiator on a remote host to transport block-level data to an iSCSI target on a storage device in the vSAN cluster. vSAN 6.7 and later releases support Windows Server Failover Clustering (WSFC), so WSFC nodes can access vSAN iSCSI targets.

After configuring the vSAN iSCSI target service, you can discover the vSAN iSCSI targets from a remote host. To discover vSAN iSCSI targets, use the IP address of any host in the vSAN cluster, and the TCP port of the iSCSI target.

Examples are in Python.

Enabling vSAN iSCSI Service

To enable iSCSI target service using the vSphere Client, navigate to the vSAN cluster and then click the **Configure** tab. Under **vSAN**, select the **Enable vSAN iSCSI target service** check box.

Here is an example of how to enable iSCSI target service using the vSAN Management API. This example is based on the code in the `vsaniscsisamples.py` sample file located under the `samplecode` directory.

```
# Enable iSCSI service using the vSAN Cluster Reconfiguration API on vCenter, and
# The config port is set to 3260 by default. However, this can be customized.
def EnableIscsi(vsanStoragePolicy, si, context, apiVersion, vcMos, cluster):
    defaultVsanConfigSpec = vim.cluster.VsanIscsiTargetServiceDefaultConfigSpec(
        networkInterface="vmk0",
        port=2300)
    vitEnableSpec = vim.cluster.VsanIscsiTargetServiceSpec(
        homeObjectStoragePolicy=vsanStoragePolicy,
        defaultConfig=defaultVsanConfigSpec,
        enabled=True)
    vccs = vcMos['vsan-cluster-config-system']
    clusterReconfigSpec = vim.vsan.ReconfigSpec(iscsiSpec=vitEnableSpec)
    vitEnableVsanTask = vccs.ReconfigureEx(cluster, clusterReconfigSpec)
    vitEnableVcTask = vsanapiutils.ConvertVsanTaskToVcTask(
        vitEnableVsanTask, si._stub)
    vsanapiutils.WaitForTasks([vitEnableVcTask], si)
    print('Enable vSAN iSCSI service task finished with status: %s' %
          vitEnableVcTask.info.state)
```

Creating iSCSI Targets and LUNs

To create an iSCSI target and its associated LUN using the vSphere Client , navigate to the vSAN cluster and then click the **Configure** tab. Under **vSAN**, click **iSCSI Targets** and then click the **Add a new iSCSI target** icon.

Here is an example of how to create iSCSI targets and LUNs using the vSAN Management API. This example is based on the code in the `vsaniscsisamples.py` sample file located under the `samplecode` directory.

```
# Creating vSAN iSCSI targets and an associated LUN of 1GB size.
def CreateIscsiTargetAndLun(cluster, si):
    targetAlias = "sampleTarget"
    targetSpec = vim.cluster.VsanIscsiTargetSpec(
        alias=targetAlias,
        iqn='iqn.2015-08.com.vmware:vit.target1')
    vsanTask = vits.AddIscsiTarget(cluster, targetSpec)
    vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
    vsanapiutils.WaitForTasks([vcTask], si)
    print('Create vSAN iSCSI target task finished with status: %s' %
          vcTask.info.state)

    lunSize = 1 * 1024 * 1024 * 1024  # 1GB
    lunSpec = vim.cluster.VsanIscsiLUNSpec(
        lunId=0,
        lunSize=lunSize,
        storagePolicy=vsanStoragePolicy)
    vsanTask = vits.AddIscsiLUN(cluster, targetAlias, lunSpec)
    vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
    vsanapiutils.WaitForTasks([vcTask], si)
    print('Create vSAN iSCSI LUN task finished with status: %s' %
          cTask.info.state)
```

Disabling iSCSI Service

To disable iSCSI target service using the vSphere Client, navigate to the vSAN cluster and click the **Configure** tab. Under **vSAN**, deselect the **Enable vSAN iSCSI target service** check box.

Here is an example of how to disable iSCSI target service using the vSAN Management API. This example is based on the code in the `vsaniscsisamples.py` sample file located under the `samplecode` directory.

```
# Disable iSCSI service through vSAN iSCSI API on vCenter.
def DisableIscsi(vitDisableSpec, si, context, apiVersion, vcMos):
    vitDisableSpec = vim.cluster.VsanIscsiTargetServiceSpec(enabled=False)
    clusterReconfigSpec = vim.vsan.ReconfigSpec(iscsiSpec=vitDisableSpec)
    vccs = vcMos['vsan-cluster-config-system']
    vitDisableVsanTask = vccs.ReconfigureEx(cluster, clusterReconfigSpec)
    vitDisableVcTask = vsanapiutils.ConvertVsanTaskToVcTask(
        vitDisableVsanTask, si._stub)
    vsanapiutils.WaitForTasks([vitDisableVcTask], si)
    print('Disable vSAN iSCSI service task finished with status: %s' %
          vitDisableVcTask.info.state)
```

Managing vSAN File Service

vSAN File Service is a layer that sits on top of vSAN to provide file shares. It currently supports NFSv3 and NFSv4.1 file shares. vSAN File Service includes vSAN Distributed File System (vDFS) which provides the underlying scalable file system by aggregating vSAN objects, a Storage Services Platform that provides resilient file server end points, and a control plane for deployment, management, and monitoring. File shares are integrated into the existing vSAN Storage Policy Based Management, and on a per-share basis. vSAN file service brings in capability to host NFS shares directly on the vSAN cluster.

When you configure vSAN file service, vSAN creates a single VDFS distributed filesystem for the cluster which will be used internally for management purposes. A file service VM (FSVM) is placed on each host. The FSVMs manage file shares in the vSAN datastore. Each FSVM contains an NFS file server.

A static IP address pool should be provided as an input while enabling file service workflow. One of the IP addresses is designated as the primary IP address. The primary IP address can be used for accessing all the shares in the file services cluster with the help of NFSv4 referrals. An NFS server is started for every IP address provided in the IP pool. An NFS share is exported by only one NFS server. However, the NFS shares are evenly distributed across all the NFS servers. To provide computing resources that help manage access requests, the number of IP addresses must be equal to the number of hosts in the vSAN cluster.

Downloading File Service OVF

You can download the OVF using the vSphere Client during the vSAN File Service configuration process. You can also download the OVF file while upgrading the vSAN File Service using the vSphere Client.

Here is an example of how to download the compatible OVF from the VMware server using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Download compatible OVF from VMware server
   #Find OVF download url
   print("Finding OVF download url ...")
   ovfUrl = vcfs.FindOvfDownloadUrl(cluster)
   if not ovfUrl:
      print("Failed to find the OVF download url.")
      return -1
   print("Found OVF download url: %s" % ovfUrl)

   # Download FSVM OVF files to vCenter
   print("Downloading ovf files from %s to vCenter ..." % ovfUrl)
```

```
    vsanTask = vcfs.DownloadFileServiceOvf(downloadUrl=ovfUrl)
    vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
    vsanapiutils.WaitForTasks([vcTask], si)
    if vcTask.info.state != 'success':
       print("Failed to download ovf files with error: %s"
             % vcTask.infor.error)
       return -1
    print("Downloaded ovf files to vCenter successfully")
```

Note:
You can also use the API `DownloadFileServiceOvf()` directly with a specific URL that points to the .ovf file on your own HTTP server. You should have all the following files under the same folder with the .ovf file:

- VMware-vSAN-File-Services-Appliance-x.x.x.x-x_OVF10.mf
- VMware-vSAN-File-Services-Appliance-x.x.x.x-x-x_OVF10.cert
- VMware-vSAN-File-Services-Appliance-x.x.x.x-x-x-system.vmdk
- VMware-vSAN-File-Services-Appliance-x.x.x.x-x-cloud-components.vmdk
- VMware-vSAN-File-Services-Appliance-x.x.x.x-x-log.vmdk
- VMware-vSAN-File-Services-Appliance-x.x.x.x-x_OVF10.ovf

Enabling File Service

You can enable vSAN File Service using the vSphere Client. Navigate to the vSAN cluster and click **Configure** > **vSAN** > **Services**. On the File Service row, click **Enable**.

Here is an example of how to enable vSAN File Service using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Enable file service
   print("Enabling the file service")
   network = cluster.host[0].network[0]
   fileServiceConfig = vim.vsan.FileServiceConfig(
         enabled=True,
         network=network,
         domains=[],
   )
   clusterSpec = vim.vsan.ReconfigSpec(fileServiceConfig=fileServiceConfig)
   vsanTask = vccs.ReconfigureEx(cluster, clusterSpec)
   vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
   vsanapiutils.WaitForTasks([vcTask], si)
   if vcTask.info.state != 'success':
      print("Failed to enable file service with error: %s"
            % vcTask.info.error)
      return -1
   print("Enabled file service successfully")
```

Creating File Service Domain

You can create file service domain using the vSphere Client during the vSAN File Service configuration process. To begin the configuration process, navigate to the vSAN cluster and click **Configure** > **vSAN** > **Services**. On the File Service row, click **Enable**. Follow the wizard to configure the vSAN File Service including creating the file service domain.

Here is an example of how to create file service domain using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Create file service domain
   fsDomainConfig = getFileServiceDomainConfig()
   domainName = fsDomainConfig.name
```

```
   print("Creating file service domain")
   vsanTask = vcfs.CreateFileServiceDomain(fsDomainConfig, cluster)
   vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
   vsanapiutils.WaitForTasks([vcTask], si)
   if vcTask.info.state != 'success':
      print("Failed to create file service domain with error: %s"
            % vcTask.info.error)
      return -1
   print("Created file service domain %s successfully"
         % domainName)
```

Creating a File Share

You can create a file share using the vSphere Client. Navigate to the vSAN cluster and then click **Configure** > **vSAN** > **File Service Shares** > **Add**.

vSAN File Service does not support using these file shares as NFS datastores on ESX.

Here is an example of how to create file shares using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Create a file share
   fileShareConfig = getFileShareConfig(si._stub, context, domainName)
   if not fileShareConfig:
      print("Failed to get file share config")
      return -1

   fileShareName = fileShareConfig.name
   print("Creating a file share: %s" % fileShareName)
   vsanTask = vcfs.CreateFileShare(fileShareConfig, cluster)
   vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
   vsanapiutils.WaitForTasks([vcTask], si)
   if vcTask.info.state != 'success':
      print("Failed to create a file share with error: %s"
            % vcTask.info.error)
      return -1
   print("Created file share %s successfully" % fileShareName)
```

Querying File Share Information

You can query the file share information using the vSAN Management API.

Here is an example based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Query file share information
   print("Query file share information: %s" % fileShareName)
   fileShareQuerySpec = vim.vsan.FileShareQuerySpec()
   fileShareQuerySpec.domainName = domainName
   fileShareQuerySpec.names = [fileShareName]
   QueryResult = vcfs.QueryFileShares(fileShareQuerySpec, cluster)
```

Querying File Service Domain Information

You can query the file service domain information using the vSAN Management API.

Here is an example based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Query file service domain information
   fsDomainQuerySpec = vim.vsan.FileServiceDomainQuerySpec()
   result = vcfs.QueryFileServiceDomains(fsDomainQuerySpec, cluster)
```

Removing a File Share

You can remove a file share using the vSphere Client. Navigate to the vSAN cluster and then click **Configure** > **vSAN** > **File Service Shares**. Select the file share that you want to remove and then click **DELETE**.

Here is an example of how to remove a file share using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Remove a file share
   print("Removing file share: %s" % fileShareName)
   fileShareQuerySpec = vim.vsan.FileShareQuerySpec()
   fileShareQuerySpec.domainName = domainName
   fileShareQuerySpec.names = [fileShareName]
   QueryResult = vcfs.QueryFileShares(fileShareQuerySpec, cluster)
   result = QueryResult.fileShares
   vsanTask = vcfs.RemoveFileShare(result[0].uuid, cluster)
   vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
   vsanapiutils.WaitForTasks([vcTask], si)
   if vcTask.info.state != 'success':
      print("Failed to remove a file share with error: %s"
            % vcTask.info.error)
      return -1
   print("Removed file share %s successfully"
         % result[0].config.name)
```

Removing File Service Domain

You can remove the file service domain using the vSphere Client by removing all the file shares and then disabling the vSAN File Service.

Here is an example of how to remove a file service domain using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Remove file service domain
   fsDomainQuerySpec = vim.vsan.FileServiceDomainQuerySpec()
   result = vcfs.QueryFileServiceDomains(fsDomainQuerySpec, cluster)
   print("Removing file service domain: %s" % result[0].config.name)
   vsanTask = vcfs.RemoveFileServiceDomain(result[0].uuid, cluster)
   vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
   vsanapiutils.WaitForTasks([vcTask], si)
   if vcTask.info.state != 'success':
      print("Failed to remove file service domain with error: %s"
            % vcTask.info.error)
      return -1
   print("Removed file service domain %s successfully"
         % result[0].config.name)
```

Disabling File Service

You can disable vSAN File Service using the vSphere Client. Navigate to the vSAN cluster and click **Configure** > **vSAN** > **Services**. On the File Service row, click **Disable**.

Here is an example of how to disable vSAN File Service using the vSAN Management API. This example is based on the code in the vsanfssamples.py sample file located under the Python sample code directory.

```
# Disable file service
  print("Disabling file service")
  fileServiceConfig = vim.vsan.FileServiceConfig(enabled=False)
  clusterSpec = vim.vsan.ReconfigSpec(fileServiceConfig=fileServiceConfig)
  vsanTask = vccs.ReconfigureEx(cluster, clusterSpec)
  vcTask = vsanapiutils.ConvertVsanTaskToVcTask(vsanTask, si._stub)
  vsanapiutils.WaitForTasks([vcTask], si)
  if vcTask.info.state != 'success':
     print("Failed to disable file service with error: %s"
           % vcTask.info.error)
     return -1
  print("Disabled file service successfully")
```

Monitoring vSAN

You can obtain statistical data about various aspects of vSAN performance, as generated and maintained by the vSAN performance service of the cluster. You can also view vSAN cluster health information.

Examples are in Python.

Viewing vSAN Health Check Status

To view the vSAN health using the vSphere Client , navigate to the vSAN cluster, click the **Monitor** tab, and then click **vSAN**. Select **Health** to review the vSAN health check categories.

Here is an example of how to the vSAN health using the vSAN Management API.

```
# Caching vSAN cluster health summary at vCenter.
def GetClusterHealthSummary(cluster, vcMos):
    fetchFromCache = True
    vhs = vcMos['vsan-cluster-health-system']
    healthSummary = vhs.QueryClusterHealthSummary(
        cluster=cluster, includeObjUuids=True, fetchFromCache=fetchFromCache)
    clusterStatus = healthSummary.clusterStatus
    return clusterStatus
```

Monitoring vSAN Performance

You can use the vSAN performance service to monitor the performance of your vSAN cluster, and investigate potential problems.

The performance service collects and analyzes performance statistics and displays the data. You can use the performance charts to manage your workload and determine the root cause of problems.

Enabling the Performance Service

The performance service is disabled by default upon the creation of the vSAN cluster. You can enable the performance service after you configure the vSAN cluster to monitor the performance of the cluster, the participating hosts, disks, and VMs.

In the vSphere Client, you can enable the performance service from Health and Performance settings on the cluster.

To enable the vSAN performance service using the vSphere Client, navigate to the vSAN cluster, click the **Monitor** tab, and then click **vSAN**. Click **Performance** and then click **Enable**.

Following is an example of how to enable the performance service by using the vSAN Management API. :

```
print 'Enable perf service on this cluster'
# Apply the Performance Service to the VSAN config
vsanPerfSystem = vcMos['vsan-performance-manager']
```

```
# Apply the config update
task = vsanPerfSystem.CreateStatsObjectTask(cluster)
vsanapiutils.WaitForTasks([task], si)
```

Viewing vSAN Cluster Performance

To view the vSAN cluster performance, using the vSphere Client , navigate to the vSAN cluster, click the **Monitor** tab, and then click **Performance**. Select **vSAN - Virtual Machine Consumption** with a time range for your query.

Here is an example of how to view the vSAN cluster performance using the vSAN Management API.

```
# Get vSAN cluster performance
def getClusterPerformance(cluster, vsanPerfSystem):
    spec = vim.cluster.VsanPerfQuerySpec()
    spec.entityRefId = "cluster-domclient:5287a00e-e90d-dbdc-1909-bf952fdaad3a"
    endTime = datetime.datetime.utcnow()
    startTime = endTime - datetime.timedelta(hours=1)
    spec.startTime = startTime
    spec.endTime = endTime
    result = vsanPerfSystem.VsanPerfQueryPerf(querySpecs=[spec], cluster=clusterMoID)
    return result
```

Viewing vSAN Host Performance

To view the vSAN host performance, using the vSphere Client, navigate to the vSAN cluster and select a host. Click the **Monitor** tab and then click **Performance**. Select **vSAN - Virtual Machine Consumption** with a time range for your query.

The sample code for getting vSAN host performance is similar to the sample code for getting vSAN cluster performance above, except for the following:

- Instead of `spec.entityRefId`, specify `host-domclient`.
- In place of `cluster-domclient`, specify `host-UUID`.

Viewing vSAN VM Performance

To view the vSAN VM performance, using the vSphere Client , navigate to the vSAN cluster and select a VM. Click the **Monitor** tab, and then click **Performance**. Select **vSAN - Virtual Machine Consumption** with a time range for your query. Now, select **vSAN - Virtual Disk** with a time range for your query.

The sample code for getting vSAN VM performance is similar to the sample code for getting vSAN cluster performance above, except for the following:

- Instead of `spec.entityRefId`, specify `virtual-machine`.
- In place of `cluster-domclient`, specify `VM-UUID`.

Data Protection and vSAN Snapshot Service

Using native snapshots stored locally on a vSAN Express Storage Architecture (ESA) cluster, vSAN Data Protection can help to quickly recover virtual machines from operational failure or ransomware attacks. This vSAN facility is also called Snapshot Service.

In VCF 9.0 and 9.1, and back to vSphere 8.0 U3, VMware includes support for vSAN ESA native snapshots. You can manage VM protection groups and native vSAN snapshots with a REST API. Python and Java SDKs are available as well. Original vSAN OSA does not support snapshot service.

Commercial storage arrays often provide fast and efficient hardware snapshots using internal methods that reduce I/O and minimize sector copying. The vSAN native snapshot service is similar to hardware snapshots offered by network attached storage (NAS) and storage area network (SAN) vendors. Using vSAN ESA snapshots instead of virtual machine snapshots will reduce the duration of snapshot operations, improve performance, and potentially achieve better RPO (recovery point objective).

For an API reference of REST calls for vSAN ESA snapshot service in this release, see vSAN Data Protection > Snapshot Appliance API on the Broadcom developer site.

Data protection with the vSAN snapshot service requires deployment of a virtual appliance to manage vSAN ESA snapshots.

The stand-alone vSAN Data Protection SDK for Python is available on the Broadcom developer site under SDKs > Storage & Availability.

The stand-alone vSAN Data Protection SDK for Java is also available on the Broadcom developer site under SDKs > Storage & Availability. The Java code is well commented.

For information about vSAN ESA snapshots, and a sample screen in the vSphere Client, see "Using vSAN Data Protection" and related topics in the manual *Administering VMware vSAN*.

About vSAN ESA Snapshot Service

Native snapshots in vSAN Express Storage Architecture (ESA) are much faster than traditional vSphere snapshots, allowing more frequent backups and faster recovery when needed.

Early editions of vSAN used a modified redo-log mechanism with `vsanSparse` snapshots to conserve storage space. vSAN ESA offers native snapshots with B-tree instead of chain structure. Snapshot creation is more efficient, and snapshot deletion is up to 100x faster.

Using native snapshots stored locally on the vSAN cluster, vSAN data protection can help you quickly recover VMs from operational failure or ransomware attacks. The native Snapshot Service is supported by vSAN Express Storage Architecture (ESA) but not by vSAN Original Storage Architecture (OSA).

vSAN data protection requires the Snapshot Service appliance to manage vSAN snapshots. After you deploy the appliance, vSAN data protection becomes visible in the vSphere Client.

Using the vSphere Client or an API, you can create VM protection groups, schedule snapshots, delete vSAN snapshots, restore a VM to an earlier state from a vSAN snapshot, and clone a VM for development or testing.

You can also enable immutability mode on a protection group for additional security. An immutable snapshot is a read-only copy of data that cannot be modified or deleted, even by an attacker with administrative privileges. Once immutability mode is enabled on a protection group, it cannot be disabled by an administrator.

Use Cases for vSAN Snapshot Service

With the REST API, the following use cases are possible in this release:

- Site pairing – match replication source and target, including any storage type to vSAN.
- Cluster pairing – replicate source cluster to target including multi-site clusters.
- Protection Group management – seeding, grandfather-father-son, and tag based membership.
- VM operations – create immutable snapshots for resistance to tampering and malware.
- Datastore mapping – select specific datastores at local and remote sites for snapshot service.
- Snapshot management – set policies for retention, schedule, and recovery point objective.
- Capability labels – designate datastores that can do snapshot and replication.
- Isolated recovery environment – for ransomware recovery with Live (Cyber) Recovery.
- Statistics reporting – monitor protection groups, VMs, clusters, datastores, and sites.

Snapshot Service REST API Summary

The following table summarizes the available APIs. Create, revert, and restore operations are provided. Snapshot Service runs on a schedule defined in a protection group, as established in the user interface.

Table 1910: REST operations in vSAN Snapshot Service

	Operation	Request	Description
protection-groups	create	POST	Create a protection group for the cluster.

	Operation	Request	Description
	list	GET	List protection groups for the cluster.
	get	GET	Get details about the protection group.
	update	PATCH	Update configuration or seeding of a protection group.
	pause	POST	Pause snapshots of protection group.
	resume	POST	Resume snapshots of protection group.
	delete	DELETE	Delete the specified protection group.
snapshots	list	GET	List snapshots of protection groups in the cluster.
	get	GET	Get details about the specified snapshot in a protection group.
	create	POST	Create snapshot for the protection group.
	delete	DELETE	Delete the specified protection group snapshot.
virtual machines	list	GET	List the virtual machines for the cluster.
	get	GET	Get information about snapshots of a virtual machine.
	update	PATCH	Update configuration of VM traits.
datastores	list	GET	List datastores on local or remote sites.
clusters	list	GET	List clusters, return available datastore MOIDs.
tags	list	GET	List protection groups and VMs with tagging.
	create	POST	Establish tag IDs when creating protection group.
virtual machine snapshots	list	GET	List snapshots for the virtual machine.
	get	GET	Get information about a virtual machine snapshot.
	restore	POST	Restore a deleted virtual machine to snapshot.
	revert	POST	Revert a virtual machine to specified snapshot point.
	linked clone	POST	Create a linked clone VM from the snapshot.
info about	get	POST	Return information about Snapshot Service.
snap service tasks	list	GET	Return tasks matching a filter specification.
	get	GET	Return information about the identified task.

See vSAN Data Protection > Snapshot Appliance API for the latest reference information.

Curl Examples of Snapshot API

Sections below demonstrate Curl commands running Snapshot Service functions.

Snapshot Service provides REST APIs so users can configure protection groups and snapshot operations. All these APIs require a SAML bearer token acquired from vCenter SSO and passed in the API request header.

Acquire SAML Token from vCenter

The following sample python script shows how to acquire the token and prepare it for the request. The SAML token comes from vCenter's secure token service (STS). It must be compressed with `gzip` and base64 encoded.

```
# To be executed on vCenter
import os
import ssl

# Add python site-packages with pyVim to path
import sys; sys.path.insert(0, "/usr/lib/vmware/site-packages")
from pyVim import sso

if (not os.environ.get('PYTHONHTTPSVERIFY', '') and
        getattr(ssl, '_create_unverified_context', None)):
```

```
    ssl._create_default_https_context = ssl._create_unverified_context

def get_saml_token():
    """Acquire SAML bearer token from STS using admin credentials"""
    sso_path = "/sts/STSService/vsphere.local"
    vcUrl = "https://10.78.116.77"
    sso_url = vcUrl + sso_path
    authenticator = sso.SsoAuthenticator(sso_url)
    saml_token = authenticator.get_bearer_saml_assertion("administrator@vsphere.local", "SLTZ1i_Qj_sn9nEf",
 delegatable=True)
    return saml_token

# Acquire SAML token from VC
saml_token = get_saml_token()

# Compress the SAML token obtained with gzip
import gzip
saml_token_bytes = bytes(saml_token, 'utf-8')
zipped = gzip.compress(saml_token_bytes)

# Base64 encode the gzipped saml token
import base64
encoded = base64.b64encode(zipped)
print(encoded)

# The encoded saml_token printed is the token to be used
# in the request headers for Snapshot Service API requests.
```

In the Curl examples below, items enclosed in { } braces are variables. For instance, `{api_host}` is the FQDN or IP address of the snapshot server appliance, and "`encodedSAMLtoken`" is the compressed and encoded SAML token obtained by Python code above, protected by double quotes.

Get Information about Snapshot Service

```
curl --request GET --location https://{api_host}/api/snapservice/info/about \
  --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response

```
{
    "value" : {
        "version" : "1.0"
    }
}
```

Create Protection Group for a Cluster

```
curl --request POST \
 --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups?vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "locked": false,
    "snapshot_policies": [
        {
            "name": "snapshot_policy_01",
            "schedule": {
                "unit": "MINUTE",
                "interval": 30
            },
```

```
            "retention": {
                "duration": 6,
                "unit": "HOUR"
            }
        }
    ],
    "name": "protection_group_01",
    "target_entities": {
        "vm_name_patterns": [
            "*VM*"
        ],
        "vms": [
            "vm-1"
        ]
    }
}'
```

Sample Response

```
c4598712-48ec-4458-b805-2f202ec00f50:com.vmware.snapservice.clusters.protection_groups
```

List Protection Groups for a Cluster

```
curl --request GET --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response with info (including name and policies), snapshots, status, vms, and pg ID.

```
{
   "items":[
      {
         "info":{
            "last_snapshot_time":"2024-06-29T14:20:23.079Z",
            "locked":false,
            "name":"protection_group_01",
            "oldest_snapshot_time":"2024-06-29T14:16:23.134Z",
            "snapshot_policies":[
               {
                  "name":"snapshot_policy_01",
                  "retention":{
                     "duration":6,
                     "unit":"HOUR"
                  },
                  "schedule":{
                     "interval":30,
                     "unit":"MINUTE"
                  }
               }
            ],
            "snapshots":[
               "8868246b-2348-4833-8477-9f0ef5a204d6",
               "b038428b-8c05-4238-a238-a7ee4f8fa193"
            ],
            "status":"ACTIVE",
            "target_entities":{
               "vm_name_patterns":[
                  "*VM*"
               ],
               "vms":[
```

```
                    "vm-1"
                ]
            },
            "vms":[
                "vm-1"
            ]
         },
         "pg":"517ae2bd-40a4-4794-abc7-37329d3c0aa1"
      }
   ]
}
```

Get Protection Group

```
curl --request GET --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/{pg}' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response

```
{
    "last_snapshot_time": "2024-06-29T14:32:23.145Z",
    "locked": false,
    "name": "protection_group_01",
    "oldest_snapshot_time": "2024-06-29T14:28:23.179Z",
    "snapshot_policies": [
        {
            "name": "snapshot_policy_01",
            "retention": {
                "duration": 6,
                "unit": "HOUR"
            },
            "schedule": {
                "interval": 30,
                "unit": "MINUTE"
            }
        }
    ],
    "snapshots": [
        "54d3c981-82f0-423b-87f3-67039b925b99",
        "60f4bd07-35c2-4237-a19c-b2814d4a3cc0"
    ],
    "status": "ACTIVE",
    "target_entities": {
        "vm_name_patterns": [
            "*VM*"
        ],
        "vms": [
            "vm-1"
        ]
    },
    "vms": [
        "vm-1"
    ]
}
```

Edit Protection Group

```
curl --request PATCH --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups?vmw-
task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "snapshot_policies": [
        {
            "name": "updated_snapshot_policy_02",
            "retention": {
                "duration": 6,
                "unit": "HOUR"
            },
            "schedule": {
                "interval": 1,
                "unit": "HOUR"
            }
        }
    ],
    "name": "updated_protection_group_02",
    "target_entities": {
        "vm_name_patterns": [
            "TestVM_[12]$"
        ],
        "vms": [
            "vm-2"
        ]
    },
    "locked": false
}'
```

Sample Response

```
c4598712-48ec-4458-b805-2f202ec00f50:com.vmware.snapservice.clusters.protection_groups
```

Pause Protection Group

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/{pg}?
action=pause&vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response

```
fde61934-3da5-42e1-9d2d-e08113135c23:com.vmware.snapservice.clusters.protection_groups
```

Resume Protection Group

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/{pg}?
action=resume&vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response

```
43d519a4-5d08-49bd-80b1-b61778090926:com.vmware.snapservice.clusters.protection_groups
```

Delete Protection Group

```
curl --request DELETE --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/
{pg}?vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response (deletes protection group but snapshots are retained until expiration)

```
c4e64f8a-46e4-4893-bd8e-62f52c498479:com.vmware.snapservice.clusters.protection_groups
```

Delete Protection Group and Snapshots

```
curl --request DELETE --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/
{pg}?force=true&vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response (snapshots are also deleted immediately)

```
c4e64f8a-46e4-4893-bd8e-62f52c498479:com.vmware.snapservice.clusters.protection_groups
```

Create Protection Group Snapshot

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/{pg}/
snapshots?vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "name": "manual_protection_greoup_snapshot_01",
    "retention": {
        "duration": 10,
        "unit": "MINUTE"
    }
}'
```

Sample Response

```
c4e64f8a-46e4-4893-bd8e-62f52c498479:com.vmware.snapservice.clusters.protection_groups
```

List Snapshots in a Protection Group

```
curl --request GET --location 'http://{api_host}}/api/snapservice/clusters/{cluster}}/protection-groups/{pg}/
snapshots' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response

```
{
    "snapshots": [
        {
            "info": {
                "end_time": "2024-06-29T17:01:09.889Z",
                "expires_at": "2024-06-29T17:11:09.889Z",
                "name": "manual_protection_greoup_snapshot_01",
                "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
                "snapshot_type": "ONE_TIME",
                "start_time": "2024-06-29T17:01:09.816Z",
                "vm_snapshots": [
                    {
                        "created_at": "2024-06-29T17:01:09.865Z",
                        "expires_at": "2024-06-29T17:11:09.865Z",
                        "name": "manual_protection_greoup_snapshot_01",
                        "snapshot": "b1999d52-3fb2-459a-aa06-4998733e66d0",
                        "vm": "vm-1"
                    }
                ]
            },
            "snapshot": "38be53c7-fbd3-4524-a99f-50f3e05ff9dd"
```

```
        },
        {
            "info": {
                "end_time": "2024-06-29T17:02:57.070Z",
                "expires_at": "2024-06-29T17:12:57.070Z",
                "name": "protection_group_01-31703134-4647-4c9b-b333-af0c0c7b8e9a-1719705777",
                "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
                "snapshot_type": "SCHEDULED",
                "start_time": "2024-06-29T17:02:57.029Z",
                "vm_snapshots": [
                    {
                        "created_at": "2024-06-29T17:02:57.054Z",
                        "expires_at": "2024-06-29T17:12:57.054Z",
                        "name": "protection_group_01-31703134-4647-4c9b-b333-af0c0c7b8e9a-1719705777",
                        "snapshot": "610337e2-2c29-4659-83a8-8a6dae1719fe",
                        "vm": "vm-1"
                    }
                ]
            },
            "snapshot": "31703134-4647-4c9b-b333-af0c0c7b8e9a"
        }
    ]
}
```


Get Snapshot in Protection Group

```
curl --request GET --location 'http://{api_host}}/api/snapservice/clusters/{cluster}}/protection-groups/{pg}/
snapshots/{snapshot}' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response


```
{
    "end_time": "2024-06-29T17:02:57.070Z",
    "expires_at": "2024-06-29T17:12:57.070Z",
    "name": "protection_group_01-31703134-4647-4c9b-b333-af0c0c7b8e9a-1719705777",
    "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
    "snapshot_type": "SCHEDULED",
    "start_time": "2024-06-29T17:02:57.029Z",
    "vm_snapshots": [
        {
            "created_at": "2024-06-29T17:02:57.054Z",
            "expires_at": "2024-06-29T17:12:57.054Z",
            "name": "protection_group_01-31703134-4647-4c9b-b333-af0c0c7b8e9a-1719705777",
            "snapshot": "610337e2-2c29-4659-83a8-8a6dae1719fe",
            "vm": "vm-1"
        }
    ]
}
```


Delete Protection Group Snapshot

```
curl --location --request DELETE 'http://{api_host}/api/snapservice/clusters/{cluster}/protection-groups/{pg}/
snapshots/{snapshot}' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response

```
c4e64f8a-46e4-4893-bd8e-62f52c498479:com.vmware.snapservice.clusters.snapshot
```

List Virtual Machines in Cluster

```
curl --request GET --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines' \
 --header 'Authorization: SIGN token=<encode saml_token>'
```

Sample Response

```
 {
    "items": [
        {
            "info": {
                "in_cluster": true,
                "local_protection_enabled": true,
                "name": "TestVM_1",
                "protection_groups": [
                    "8b4b251e-4716-4c06-a43f-d583788790e3"
                ],
                "snapshot_count": 0
            },
            "vm": "vm-1"
        },
        {
            "info": {
                "in_cluster": true,
                "local_protection_enabled": false,
                "name": "TestVM_2",
                "protection_groups": [],
                "snapshot_count": 0
            },
            "vm": "vm-2"
        }
    ]
}
```

List Virtual Machine Snapshots

```
curl --request GET --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines/{vm}/
snapshots' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response

```
{
    "items": [
        {
            "info": {
                "created_at": "2024-06-29T17:20:09.593Z",
                "disk_snapshots": [
                    {
                        "disk_key": 2001,
                        "epoch": 1,
                        "sequence_uuid": "bd936018-93e4-45b6-bc8c-7da6ee432b6a",
                        "uri": "b92c0012-6476-4e7f-9890-978189c70160::290"
                    },
                    {
                        "disk_key": 2002,
                        "epoch": 2,
                        "sequence_uuid": "319c4fb6-04fb-4155-8a74-2f9996416616",
                        "uri": "6f50b20e-7d61-4193-9442-3007aa59e88e::290"
```

```
                    },
                    {
                        "disk_key": 2003,
                        "epoch": 3,
                        "sequence_uuid": "5192bd64-6098-4853-a253-02c53bfe6e0a",
                        "uri": "42eccb7c-5b38-468f-8a2c-d0173a1acb77::290"
                    }
                ],
                "expires_at": "2024-06-29T17:30:09.593Z",
                "name": "manual_protection_greoup_snapshot_01",
                "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
                "snapshot_type": "ONE_TIME",
                "vm": "vm-1"
            },
            "snapshot": "e2e81470-fb06-48a5-bb0b-56b473cd0661"
        },
        {
            "info": {
                "created_at": "2024-06-29T17:20:21.139Z",
                "disk_snapshots": [
                    {
                        "disk_key": 2001,
                        "epoch": 1,
                        "sequence_uuid": "daeec3bd-24cc-4179-a392-c3e37cc5bf9e",
                        "uri": "b92c0012-6476-4e7f-9890-978189c70160::291"
                    },
                    {
                        "disk_key": 2002,
                        "epoch": 2,
                        "sequence_uuid": "86e4ea26-b27f-495a-aaa9-3d0737b6d3a2",
                        "uri": "6f50b20e-7d61-4193-9442-3007aa59e88e::291"
                    },
                    {
                        "disk_key": 2003,
                        "epoch": 3,
                        "sequence_uuid": "01ed5ba6-6a7b-40e0-8a9a-e80754e9f418",
                        "uri": "42eccb7c-5b38-468f-8a2c-d0173a1acb77::291"
                    }
                ],
                "expires_at": "2024-06-29T17:30:21.139Z",
                "name": "protection_group_01-6994cc79-06ea-4bd7-b1ef-64dd3ab676a7-1719706821",
                "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
                "snapshot_type": "SCHEDULED",
                "vm": "vm-1"
            },
            "snapshot": "6af52f25-e949-4288-b752-1f2001298cf5"
        }
    ]
}
```


Get Virtual Machine Snapshots

```
curl --request GET --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines/{vm}/
snapshots/{snapshot}' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"'
```

Sample Response

```
{
    "created_at": "2024-06-29T17:20:21.139Z",
    "disk_snapshots": [
        {
            "disk_key": 2001,
            "epoch": 1,
            "sequence_uuid": "daeec3bd-24cc-4179-a392-c3e37cc5bf9e",
            "uri": "b92c0012-6476-4e7f-9890-978189c70160::291"
        },
        {
            "disk_key": 2002,
            "epoch": 2,
            "sequence_uuid": "86e4ea26-b27f-495a-aaa9-3d0737b6d3a2",
            "uri": "6f50b20e-7d61-4193-9442-3007aa59e88e::291"
        },
        {
            "disk_key": 2003,
            "epoch": 3,
            "sequence_uuid": "01ed5ba6-6a7b-40e0-8a9a-e80754e9f418",
            "uri": "42eccb7c-5b38-468f-8a2c-d0173a1acb77::291"
        }
    ],
    "expires_at": "2024-06-29T17:30:21.139Z",
    "name": "protection_group_01-6994cc79-06ea-4bd7-b1ef-64dd3ab676a7-1719706821",
    "pg": "8b4b251e-4716-4c06-a43f-d583788790e3",
    "snapshot_type": "SCHEDULED",
    "vm": "vm-1"
}
```


Revert Virtual Machine to Snapshot

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines/{vm}?
action=revert&vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "snapshot" : "6af52f25-e949-4288-b752-1f2001298cf5"
}'
```

Sample Response


```
c56a4fb5-883e-4bdd-a429-3791512c2a81:com.vmware.snapservice.clusters.virtual_machines
```


Restore Deleted Virtual Machine from Snapshot

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines/{vm}?
action=restore&vmw-task=true' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "snapshot" : "6af52f25-e949-4288-b752-1f2001298cf5"
}'
```

Sample Response


```
c56a4fb5-883e-4bdd-a429-3791512c2a81:com.vmware.snapservice.clusters.virtual_machines
```


Create Linked Clone of Virtual Machine from Snapshot

```
curl --request POST --location 'https://{api_host}/api/snapservice/clusters/{cluster}/virtual-machines/{vm}?
action=linked-clone&vmw-task=true' \
```

```
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --header 'Content-Type: application/json' \
 --data '{
    "snapshot" : "c43b377c-5c38-4e5e-a9f6-3b3a83238f6e",
    "name": "VM_linked_clone1",
    "placement": {
        "host":"host-2"
    }
}'
```

Sample Response

```
0d4f710e-e350-492f-abcb-d443fa4ef976:com.vmware.snapservice.clusters.virtual_machines
```

List Tasks

```
curl --location 'https://{api_host}/api/snapservice/tasks' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response


```
{
   "items":[
      {
         "info":{
            "cancelable":false,
            "description":{
               "args":[

               ],
               "default_message":"Create protection group task",
               "id":"com.vmware.vsan.snapservice.CreatePg"
            },
            "end_time":"2024-06-29T21:12:23.625Z",
            "operation":"create",
            "progress":{
               "completed":100,
               "message":{
                  "args":[

                  ],
                  "default_message":"Create protection group task",
                  "id":"com.vmware.vsan.snapservice.CreatePg"
               },
               "total":100
            },
            "result":"517ae2bd-40a4-4794-abc7-37329d3c0aa1",
            "service":"com.vmware.snapservice.clusters.protection_groups",
            "start_time":"2024-06-29T21:12:23.513Z",
            "status":"SUCCEEDED",
            "target":{
               "id":"",
               "type":"com.vmware.vsan.snapservice.protection_group"
            },
            "user":"Administrator@VSPHERE.LOCAL",
            "vc_task":"task-1234"
         },
         "task":"c4598712-48ec-4458-b805-2f202ec00f50:com.vmware.snapservice.clusters.protection_groups"
      }
   ]
```

```
}
```

Get Task Status

```
curl --location 'https://{api_host}/api/snapservice/tasks/c4598712-48ec-4458-
b805-2f202ec00f50:com.vmware.snapservice.clusters.protection_groups' \
 --header 'Authorization: SIGN token="encodedSAMLtoken"' --data ''
```

Sample Response

```
{
    "cancelable": false,
    "description": {
        "args": [],
        "default_message": "Create protection group task",
        "id": "com.vmware.vsan.snapservice.CreatePg"
    },
    "end_time": "2024-06-29T21:12:23.625Z",
    "operation": "create",
    "progress": {
        "completed": 100,
        "message": {
            "args": [],
            "default_message": "Create protection group task",
            "id": "com.vmware.vsan.snapservice.CreatePg"
        },
        "total": 100
    },
    "result": "517ae2bd-40a4-4794-abc7-37329d3c0aa1",
    "service": "com.vmware.snapservice.clusters.protection_groups",
    "start_time": "2024-06-29T21:12:23.513Z",
    "status": "SUCCEEDED",
    "target": {
        "id": "",
        "type": "com.vmware.vsan.snapservice.protection_group"
    },
    "user": "user@domain.com",
    "vc_task": "task-1234"
}
```

How to Develop vSphere Solution

You can add functions to vSphere by developing software applications that you register as vCenter extensions. A vSphere solution is an extension that registers with vCenter and implements some or all of the extension features of the vSphere API.

Developing and Deploying vSphere Solutions, vServices, and ESX Agents uses the terms solution and extension interchangeably.

A vSphere solution is an object or program that you create by extending specific classes in the vSphere Web Services API. After you register your solution with the instance of the `ExtensionManager` managed object associated with your vCenter, you see your solution under **Menu** > **Administration** > **Solutions** > **vCenter Server Extensions** of your vSphere Client.

You can create a vService solution to provide access for a specific application to connect to a service across the network.

You can create an ESX Agent to extend the functions of an ESX host and provide additional services that a vSphere solution requires.

The vService Manager and ESX Agent Manager are pre-built solutions that are part of the vCenter Server Extensions functionality within the vCenter.

The extension features in the vSphere Web Services API include functions so that you can perform the following tasks:

- Register permissions, faults, and events for an extension.
- Identify and prevent manual operations on virtual machines and vApps that extensions deploy.
- Integrate extensions with ExtensionManager.
- Store data about extensions in the vSphere database.
- Provide user interface plug-ins that extend the vSphere Client.

To use the extension functions of the vSphere API, a solution must register itself with the `ExtensionManager` that runs in a vCenter instance. By registering with `ExtensionManager`, a solution can access the extension features of the vSphere extension API.

You can develop solutions that add functions to the standard functions of vCenter. You can deploy a solution as an Open Virtualization Format (OVF) package, with optional VMware vSphere Installation Bundles (VIB). You can also install solutions by using an installer, such as Windows Installer (MSI) or RPM Package Manager. Most of the extension functions in the vSphere API are independent of the technology that you use to deploy a solution. If you deploy a solution by using OVF, you can use the vCenter Extension vService to simplify the registration of the solution with vCenter.

vCenter 8.0 provides built-in solutions.

- vSphere ESX Agent Manager
- vService Manager

vSphere ESX Agent Manager and vService Manager are part of a standard vCenter installation. These solutions appear in vCenter Server Extensions and ExtensionManager with any other solutions that register with ExtensionManager.

Download the vSphere ESX Agent Manager SDK

The ESX Agent Manager SDK is part of the vSphere SDK.

The ESX Agent Manager SDK includes the WSDL file definitions for the vSphere ESX Agent Manager API and the ESX Agent Manager API reference documentation.

1. Download the vSphere Management SDK 8.0 Update 1 bundle, `VMware-vSphere-SDK-8.0.1-`*`build_number`*`.zip`, from the VMware Developer site at https://developer.vmware.com/home.
2. Unzip the vSphere SDK to a convenient location in your development environment.
3. Navigate to the ESX Agent Manager SDK at the following location in the vSphere SDK.

   `VMware-vSphere-SDK-8.0.1-`*`build_number`*`\SDK\eam`
4. Copy the contents of `VMware-vSphere-SDK-8.0.1-`*`build_number`*`\SDK\eam` to a folder where you can modify the files.

   For example, copy the files to *`eam_work_folder`*.

You downloaded and located the ESX Agent Manager SDK, and made a copy of its contents to work on.

Create your solution.

What is in the vSphere Extension APIs

The vSphere SDK provides a set of APIs that you can use to register extensions with vCenter. To develop vSphere extensions, the most important object in the vSphere Extension API is the `ExtensionManager` managed object.

Use the `ExtensionManager` managed object in your vCenter instance to register a new extension. Extensions can add new objects to the vCenter inventory. Extensions define tasks, events, and faults that relate to actions that the solution performs on the objects, the events that occur in the extension, and the problems that the objects encounter. You can also use `ExtensionManager` to add user interface elements to the vSphere Client to allow users to interact with the objects that your extension provides to vCenter.

vCenter extensions can have both a client and a server component, or they can consist of just a client component or just a server component. Each instance of an extension is represented by an `Extension` data object.

You can indicate that an extension manages the virtual machines or vApps that it creates by setting the `managedBy` property in the `ManagedEntityInfo` data object for that virtual machine or vApp. When you register an extension with vCenter, you can define an icon that represents the virtual machines that the extension manages. Virtual machines that an extension manages display the icon that you define in the inventory of virtual machines in the vSphere Client. The vCenter shows a warning if users try to perform manual operations on a virtual machine or vApp that the extension manages.

ExtensionManager adds data objects to the vSphere Extension APIs that integrate extensions as vCenter solutions that you can manage.

For information about how to develop vSphere Client extensions, see the *Developing Remote Plug-ins with the vSphere Client SDK* documentation.

For information about developing vSphere applications, see the *vSphere Web Services SDK Programming Guide* and the *VMware vSphere API Reference* documentation.

How to use the vCenter Server Extensions

The vCenter Server Extensions allows you to monitor and interact with solutions that are registered with a vCenter instance.

The vCenter Server Extensions shows four standard tabs for each running solution. The tabs list the virtual machines that a solution deploys and manages, show the status, name, company URL, and version of the solution.

A solution can also use the **Solutions** portlet on the **Summary** tab to add links to Web pages that provide some functionality specific to the solution. For example, the pages can be used to allow users to configure the solution, or to provide access to the functions of the solution.

Standard Tabs in the vCenter Server Extensions

vCenter Server Extensions displays standard tabs for each solution that is running on a vCenter instance.

Table 1911: Standard tabs in the vCenter Server Extensions

Tab	Description
Summary	General information about the solution, including its name, vendor, and version.
Monitor	Filters all vSphere events and lists only the system activities that are related to the specific solution.
Configure	Displays one or more ESX Agents, the services they contain, and actions you can perform on them.
VMs	Information about the virtual machines and vApps that the solution deploys. You can perform all operations applicable for a VM and vApp.

What are the vSphere ESX agents and agencies

A vSphere ESX agent is a virtual machine and an optional vSphere Installation Bundle (VIB) that extends the functions of an ESX host to provide additional services that a vSphere solution requires.

For example, a solution might require a particular network filter or firewall configuration to function. A solution can use an ESX agent to connect to the vSphere Hypervisor and extend the host with functions specific to that solution. For example, the ESX agent can filter network traffic, act as a firewall, or gather other information about the virtual machines on the host.

ESX agent virtual machines are similar to services in Windows or Linux. They start when the operating system starts and they stop when it shuts down. The behavior of ESX agent virtual machines is transparent to the user. A vSphere host reaches the ready state when the ESX operating system has started and all ESX agent virtual machines have been provisioned and powered on.

To integrate an agent with vSphere ESX Agent Manager and extend the capabilities of an ESX server, an ESX agent must be packaged as an OVF or a VIB module.

ESX agencies act as containers for ESX agents. ESX agencies aggregate information about the agents that they manage. For example, ESX agencies provide an overview of the ESX agents that they contain by aggregating all the issues that relate to the ESX agents.

vSphere ESX Agent Manager

vSphere ESX Agent Manager automates the process of deploying and managing vSphere ESX agents.

The services that ESX Agent Manager provides include out-of-the-box integration of agents with vSphere features such as DRS, `AddHost`, High Availability, DRM, and maintenance mode. Each of these features can be difficult to integrate manually. ESX Agent Manager allows you to monitor the health of ESX agents. You can also block users from performing certain operations on ESX agents that might affect the virtual machines that use them. For example, ESX Agent Manager can prevent an ESX agent virtual machine from being powered off or moved from an ESX host that contains other virtual machines that use that agent.

ESX Agent Manager adds an **ESX Agencies** entry to the table of contents under the **Configure** tab of vSphere ESX Agent Manager. The **ESX Agencies** tab shows information about running agencies, lists any orphaned ESX agents, and logs information about the ESX agents that ESX Agent Manager manages.

What are vServices

A vService is a service that a solution provides to specific applications that run inside virtual machines and vApps. A solution can provide several types of vServices. Virtual machines or vApps can have dependencies on several types of vServices.

A vService is similar to a virtual hardware device upon which virtual machines and vApps can depend. Instead of providing a piece of virtual hardware, vServices typically provide access to a service across a network. By providing a vService, a solution can expose application-aware services to virtual machines and vApps. For example, a vService can provide a backup service or a logging service to virtual machines and vApps.

Virtual machines that use a vService have a vService dependency. If you mark a virtual machine as having a vService dependency, the virtual machine cannot power on unless you bind it to a provider of that vService.

If you deploy virtual machines by using Open Virtualization Format (OVF), you can specify in the OVF descriptor that the virtual machine depends on a vService. If you configure a virtual machine to have dependency on a vService, the solution that provides the vService receives notifications when specific events occur on that virtual machine. The vService can modify the OVF environment and OVF descriptor of the virtual machines and vApps that depend on it. With vServices, you can encode application-specific information in the OVF package about how a virtual machine or vApp interacts with a specific solution. You can also add solution-specific information to code running inside the guest operating system.

vService Manager

vService Manager allows you to manage the set of vServices that extensions provide and to configure virtual machines and vApps to be dependent on these vServices.

You access vService Manager from vCenter Server Extensions. You can add dependencies on vServices to virtual machines and vApps directly in the vSphere Client.

vCenter Extension vService

The vCenter Extension vService is a standard vService that vSphere 8.0 provides. With the vCenter Extension vService you can register virtual machines as vCenter extensions with minimal user interaction.

Typically, you register an extension with vCenter by running an MSI installer or an installation script in which you enter the credentials of a vCenter administrator and the connection parameters of the vCenter system. You can avoid these manual steps by using the vCenter Extension vService to register extensions in vCenter. vCenter Extension vService performs the following functions.

- Provides the virtual machine with the connection parameters and a login token of the vCenter system on which you install the extension.
- Registers the vCenter extension certificate of the solution with vCenter.

To deploy an extension using the vCenter Extension vService, you must deliver the vCenter extension in a virtual appliance that you deploy using OVF.

How do I create vSphere solutions

The vSphere SDK provides a set of APIs that you can use to create extensions for vCenter. The key SDK objects for extension development are `ExtensionManager` and `Extension`.

Use the `ExtensionManager` managed object in your vCenter instance to register a new extension. Extensions can add new objects to the vCenter inventory. Extensions define tasks, events, and faults that relate to actions that the solution performs on the objects, the events that occur in the extension, and the problems that the objects encounter. You can also use `ExtensionManager` to add user interface elements to the vSphere Client to allow users to interact with the objects that your extension provides to vCenter.

ExtensionManager adds data objects to the vSphere Extension APIs that integrate extensions as vCenter solutions that you can manage.

For information about how to develop vSphere Client extensions, see the *Developing Remote Plug-ins with the vSphere Client SDK* documentation.

For information about developing vSphere applications, see the *vSphere Web Services SDK Programming Guide* and the *VMware vSphere API Reference* documentation.

Contents and structure of an extension

A vSphere solution must register with vCenter as an extension. If you register a solution with vCenter, you can mark virtual machines as belonging to that solution, and integrate it with ExtensionManager and ESX Agent Manager.

vCenter extensions can have both a client and a server component, or they can consist of just a client component or just a server component. Each instance of an extension is represented by an `Extension` data object.

An extension can define events, tasks, faults, and privileges for performing operations on the objects that the solution exposes to vCenter, or on existing vCenter objects.

- Events inform users about occurrences in the solution.
- Tasks are operations that the extension performs.
- Faults signal errors in the extension to the end users.
- Privileges define which users or user groups can access the objects in the extension and perform tasks on them.

Solutions typically include Web pages that appear in vSphere Client. You can use the pages to configure the solution and to manage the objects that the solution exposes.

An extension is typically a Web application that runs in an application server or a script that vCenter accesses through a URL. You can implement the Web server features by using any of the following programming languages.

- Java Servlets or Java Server Pages (JSP)
- Microsoft Active Server Pages (ASP.NET)
- Common Gateway Interface (CGI) scripting
- Static or dynamic HTML pages

An extension performs the following types of tasks:

- Establishes the connection to vCenter.

- Registers the extension with `ExtensionManager`.
- Deploys virtual machines or vApps by using Open Virtualization Format (OVF).
- Deploys VMkernel modules or ESX applications as vSphere Installation Bundles (VIB).
- Exposes the functions of the product that the extension integrates into vCenter.
- Provides a user interface to access, configure, and use the extension.

To integrate an extension as a solution with ExtensionManager, the solution must be a valid extension and it must implement the `Extension.shownInSolutionsManager` property. The solution can also provide information about itself to vCenter in the `ExtSolutionManagerInfo` object.

The requirements for integrating a solution with ESX Agent Manager are slightly more restrictive than for integrating with ExtensionManager. For information about requirements, see How do I integrate solutions with vSphere ESX Agent Manager.

For information about how to develop vSphere Client extensions, see the *Developing Remote Plug-ins with the vSphere Client SDK* documentation.

For information about developing vSphere applications, see the *vSphere Web Services SDK Programming Guide* and the *VMware vSphere API Reference* documentation.

Key objects in the vSphere API for extension and solution development

The vSphere API contains objects that are key for developing extensions and for integrating the extensions with ExtensionManager.

Table 1912: Key objects in the vSphere API for extension and solution development

Object	Description
`ExtensionManager`	You call `ExtensionManager.registerExtension()` to register an `Extension` instance with `ExtensionManager`.
`Extension`	Information about extension instances, including properties that you set in an `ExtSolutionManagerInfo` object to integrate an extension as a solution in ExtensionManager.
`ExtensionClientInfo`	Information about the client side of an extension.
`ExtensionEventTypeInfo`	Information about the types of events that occur on the objects in the extension.
`ExtensionFaultTypeInfo`	Information about the types of problems that the objects in the extension encounter.
`ExtensionHealthInfo`	Information about the health of a solution.
`ExtensionPrivilegeInfo`	Information about access privileges for the extension.
`ExtensionResourceInfo`	Information about resource files that contain localizable user interface text and messages that appear in vSphere Client in different locales.
`ExtensionServerInfo`	Information about the server side of an extension.
`ExtensionTaskTypeInfo`	Information about the types of tasks that the objects perform in the extension.
`ExtExtendedProductInfo`	Information about the product that the extension exposes to vCenter.
`ExtManagedEntityInfo`	Information about the objects that the extension manages.
`ExtSolutionManagerInfo`	Information that the solution exposes about itself.
`ExtSolutionManagerInfoTabInfo`	Information about the tabs that a solution adds to its Summary page.

How do I register an extension with vCenter

You must provide information about an extension when you register it with vCenter.

- A description of the extension
- A unique identifier for the extension
- Version information
- URLs to any server or client interfaces that the extension exposes
- An email address for the extension administrator
- Any additional metadata that the extension defines, for example, URLs to the the company and product Web sites, the managed object reference (MoRef) of the virtual machine or vApp in which the solution is running, and so on
- Definitions of the tasks, events, faults, and privileges that the extension adds to vCenter
- Localization data for the task, event, fault, and privilege descriptions
- A public key which vCenter uses to register the extension in the registry of the vCenter system on which it is running.

Create the program that manages the extension
An extension communicates with vCenter across a network, so you can use any programming language to create the program that manages an extension.

Download the vSphere ESX Agent Manager SDK.

The product that you are exposing as an extension to vCenter determines the way you create the program that will manage your extension. The managing program must implement `Extension` to provide the information that vCenter requires to register the extension.

For example, you can create a solution that defines its server side in a `MyManager` class.

The `MyManager.java` can performs the following tasks for your solution.

- Sets up your solution by obtaining the values for the extension key, vCenter connection, IP addresses, and port configuration from the `mysolution.properties` file.
- Secures the connection to vCenter by using TLS.
- Defines methods to construct the URLs through which to access the server side of the extension.
- Implements the `Extension` object to register your solution as an extension with vCenter.
- Integrates your solution as a solution with ExtensionManager.
- Connects the solution to the ESX Agent Manager.
- Instantiates the class that defines the ESX agents that the solution deploys.
- Starts the solution in vCenter.
- Defines a task to unregister the solution from vCenter.

1. Create a program to manage your extension using the programming language of your choice.

   For example, you can create a `MyManager` class to manage the extension. `MyManager` can implement the Spring Framework API.

   ```
   public class MyManager implements InitializingBean {
   }
   ```

2. Create an instance of the `Extension` data object with which to register the extension with vCenter.
   The `MyManager` class can define an internal method that instantiates an `Extension`.

   ```
   public class MyManager implements InitializingBean {

     [...]
     private Extension createExtensionObject() {
       Extension extension = new Extension();
     [...]
     }
   }
   ```

Connect the extension to vCenter.

Connect the extension to vCenter

You must provide information about the vCenter instance to which you connect an extension. Set the details of the connection to vCenter in the client-side stub of the extension.

- Download the vSphere ESX Agent Manager SDK.

> **Note:**
>
> If you deploy your extension using the Open Virtualization Format (OVF), you can integrate it with the vCenter Extension vService. The vCenter Extension vService automates the process of registering extension with vCenter, so you do not need to provide any connection parameters. See How do I integrate an extension with the vCenter Extension vService.

To connect an extension to vCenter, provide the following information to the client-side stub of the connection.

- A username and password for a vCenter administrator account, if you do not use the vCenter Extension vService
- The extension key for the extension
- A reference to the `SessionManager` instance in the vCenter

You can define the connection to vCenter in the `MyVimConnection` class.

The `MyManager` class can perform the following functions.

- Implement the `MyVimConnection` class to establish the connection to vCenter when the My Solution starts.
- Uses the Spring framework to obtain the connection information from the `mysolution.properties` file that you configure when you set up My Solution.
- Passes the connection property values to `MyVimConnection`.

1. Create an instance of the `ManagedObjectReference` data object to define the connection to the extension.

   The `MyVimConnection` class creates a `ManagedObjectReference` object of type `ServiceInstance`, named `_siRef`.

   ```
   public MyVimConnection(String host, int port) {
    [...]
    _siRef = new ManagedObjectReference();
    _siRef.setType("ServiceInstance");
    _siRef.setValue("ServiceInstance");
   }
   ```

2. Define methods to get and set the vCenter host, ports, username, password, connection timeout, and session cookie.

   The `MyVimConnection` constructor defines methods to obtain the host, ports, username, password, connection timeout, and session cookie from the information that you set in the `mysolution.properties` file.

3. Connect to vCenter by obtaining the `SessionManager` managed object for the vCenter.

   `MyVimConnection.java` defines a method named `connect()`. The `connect()` method defines a standard connection to vCenter that uses WSDL. The following segment shows the calls to the `SessionManager.login()` and `SessionManager.loginExtensionByCertificate()` methods that establish the connection to vCenter. The `_stub` variable is an instance of `VimPortType`, `_sc` is a `ServiceContent` object, and `_siRef` is the `ManagedObjectReference` object of type `ServiceInstance`.

   ```
   private synchronized void connect() {
     [...]
     try {
         [...]
         VimService locator = new VimService(wsdlURL, new QName("urn:vim25Service",
                                                                 "VimService"));

         _stub = locator.getVimPort();
         [...]

         _sc = _stub.retrieveServiceContent(_siRef);
   ```

```
        ManagedObjectReference sessionManager = _sc.getSessionManager();

        if (_extensionKey == null) {
          _stub.login(sessionManager, _username, _password, null);
        }
        else {
          _stub.loginExtensionByCertificate(sessionManager, _extensionKey, null);
        }
      [...]
       _connectionStatus = ConnectionStatus.Connected;
      } catch (Exception e) {
          _logger.error(e, e);
      }
    }
```

4. Register the extension by calling the `ExtensionManager.registerExtension()` method.

   `MyVimConnection` defines a `registerExtension()` method that implements `ExtensionManager.registerExtension()`. `MyManager` calls `MyVimConnection.registerExtension()` after it has set the properties for the extension.

```
public void registerExtension(Extension ex) {
  try {
       Extension findExtension = _stub.findExtension(_sc.getExtensionManager(),
                                                     ex.getKey());
       if (findExtension == null) {
           _stub.registerExtension(_sc.getExtensionManager(), ex);
       } else {
           _stub.updateExtension(_sc.getExtensionManager(), ex);
       }
       _stub.setExtensionCertificate(_sc.getExtensionManager(),
                                     ex.getKey(), null);
       } catch (Exception e) {
           _logger.error(e, e);
       }
}
```

You registered an extension with vCenter.

Provide an extension key with which to register the extension with vCenter.

Set the extension key

Every extension that you register with vCenter must have a unique extension key that vCenter uses to identify the extension.

Download the vSphere ESX Agent Manager SDK.

To ensure uniqueness, you can use the Java package-naming convention for the key value, for example `com.yourcompany.yourextension`. Your solution can set the extension key by defining the `extensionKey=com.mycompany.mysolution` property in the `mysolution.properties` file. The `MyManager.java` class picks up this property by calling the `Extension.setKey()` method.

If you change the extension key in the `extensionKey` property in `mysolution.properties`, you can see the changed name in the vSphere Client.

You can see the extensions that you register with `ExtensionManager` in the vCenter Managed Object Browser at `https://<vcenter_server_ip_address>/mob/?moid=ExtensionManager`

Call the `Extension.setKey()` method to set the extension key.
`MyManager.java` sets the extension key to the `extensionKey` variable that the `mysolution.properties` file defines.

```
public class MyManager implements InitializingBean {
  [...]
  public final String EXTENSION_KEY;
  [...]
     EXTENSION_KEY = extensionKey;
  [...]
}
  [...]
  private Extension createExtensionObject() {
    Extension extension = new Extension();
    extension.setKey(EXTENSION_KEY);
  [...]
  }
```

You set the extension key for an extension.

Provide information about the extension to vCenter.

Set the extension product information

You provide product information about an extension by setting properties when you instantiate the `Extension` object in the program that manages the extension. The product information that you set appears in the vSphere Client.

Download the vSphere ESX Agent Manager SDK.

You can instantiate the `Extension` data object in the `MyManager.java` class. Your solution can set the `Extension` product information properties directly in `MyManager.java`, but you can set the property values in configuration files that the program that manages the extension accesses.

1. Provide a description for the extension by creating an instance of the `Description` data object and passing it to `Extension`.

   ```
   private Extension createExtensionObject() {
     Extension extension = new Extension();
     [...]
     Description description = new Description();
     description.setLabel("My Solution");
     description.setSummary("This extension represents my solution.");
     extension.setDescription(description);
     [...]
   }
   ```

2. Provide a version number for the extension by calling the `Extension.setVersion()` method.

   ```
   private Extension createExtensionObject() {
       Extension extension = new Extension();
       [...]
       extension.setVersion("0.1");
       [...]
   }
   ```

3. Optional: Change the value of the version property in `MyManager.java`.
4. Provide information about the vendor of the extension by calling the `Extension.setCompany()` method. You can set the `vendor` property by adding the following line of code to `MyManager.java`.

   ```
   private Extension createExtensionObject() {
       Extension extension = new Extension();
       [...]
       extension.setVersion("0.1");
       extension.setCompany("My Company");
       [...]
   }
   ```

5. Provide URLs to Web pages for the product and for the vendor of the extension by creating an instance of the `ExtExtendedProductInfo` data object and passing it to `Extension`.
   You can set the `companyUrl` property for `ExtExtendedProductInfo` by calling the `ExtExtendedProductInfo.setCompanyUrl()` method in `MyManager.java`:
   ```
   private Extension createExtensionObject() {
       Extension extension = new Extension();
       [...]
       ExtExtendedProductInfo extExtendedProductInfo = new
       ExtExtendedProductInfo();
       extExtendedProductInfo.setCompanyUrl("www.mycompany.com");
       extExtendedProductInfo.setProductUrl("www.mycompany.com/myproduct");
       extension.setExtendedProductInfo(extExtendedProductInfo);
       [...]
     }
   ```
6. Save, build and deploy your solution.
7. Optional: View your changes in the vSphere Client.
   If you edit the values of the `version`, `company`, `companyUrl`, and `productUrl` properties, your changes appear in the **Summary** tab of your solution.

You provided product information about an extension by setting properties in the `Extension` and `ExtExtendedProductInfo` data objects.

Set the extension name and localization information.

Set the extension name and localization resources
When you develop extensions, the vSphere Client can appear in different languages in different locales. You can set the information that appears in the vSphere Client, for example, the extension name, as resources that can be translated.

Download the vSphere ESX Agent Manager SDK.

You provide the information that requires translation in an `ExtensionResourceInfo` data object. You can add an `ExtensionResourceInfo` object for every locale that your extension supports. You set an array of `ExtensionResourceInfo` objects in the `resourceList` property of the `Extension` instance that defines your extension.

You provide onscreen messages and labels to `ExtensionResourceInfo` in a key and value pairing that you add to a `KeyValue` array in the `ExtensionResourceInfodata` property. You can set the values for the `KeyValue` pair directly in the `ExtensionResourceInfo` object, or you can refer to entries in resource files that contain the message text in different languages, according to the locale in which vSphere is running.

You provide a two-character ISO-639 language ID for the `KeyValuelocale` property, and set the `module` property to the type of resource to which this locale applies. For example, you can set the module value to `task`, `event`, `auth`, or `extension`, depending on whether the messages that the resource contains relate to tasks, events, privileges, or extensions.

1. Create an instance of the `ExtensionResourceInfo` data object.
   For example, in `MyManager.java` you can instantiate `ExtensionResourceInfo` in the implementation of `Extension`.
   ```
   private Extension createExtensionObject() {
       Extension extension = new Extension();
       [...]
       ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
       [...]
   }
   ```
2. Set the locale and module properties for the `ExtensionResourceInfo` object.
   For example, in `MyManager.java` you can set the default locale to `en` and apply this locale to the `Extension` instance, `extension`.
   ```
   private Extension createExtensionObject() {
       Extension extension = new Extension();
   ```

```
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    [...]
}
```

3. Provide the data to the `ExtensionResourceInfo` in the form of a `KeyValue` array.

   The `label` property is a property of the `Description` object, that `Extension` implements, and defines the name of the extension as it appears in the vSphere Client.

   For example, in `MyManager.java` you can add the text `My Solution` as the value of the `label` property.

```
private Extension createExtensionObject() {
    Extension extension = new Extension();
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    KeyValue keyValue = new KeyValue();
    keyValue.setKey(EXTENSION_KEY + ".label");
    keyValue.setValue("My Solution");
    [...]
}
```

4. Call the `ExtensionResourceInfo.getData()` method to add the `KeyValue` array that contains the localization data to the `data` property of the `ExtensionResourceInfo` object.

```
private Extension createExtensionObject() {
    Extension extension = new Extension();
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    KeyValue keyValue = new KeyValue();
    keyValue.setKey(EXTENSION_KEY + ".label");
    keyValue.setValue("My Solution");
    extensionResourceInfo.getData().add(keyValue);
    [...]
}
```

5. Optional: Add another `KeyValue` object to the `ExtensionResourceInfodata` property that adds a description of the extension for a given locale.

   For example, you can add the following description to `MyManager.java` in a `KeyValue` object named `keyValue_summary`.

```
private Extension createExtensionObject() {
    Extension extension = new Extension();
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    KeyValue keyValue = new KeyValue();
    keyValue.setKey(EXTENSION_KEY + ".label");
    keyValue.setValue("My Solution");

    KeyValue keyValue_summary = new KeyValue();
    keyValue_summary.setKey(EXTENSION_KEY + ".summary");
    keyValue_summary.setValue("This is a brief description of My Solution.");

    extensionResourceInfo.getData().add(keyValue);
    extensionResourceInfo.getData().add(keyValue_summary);
```

```
    [...]
}
```

6. Call the `Extension.getResourceList()` method to pass the `ExtensionResourceInfo` object to the `Extension` instance.

```
private Extension createExtensionObject() {
    Extension extension = new Extension();
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    KeyValue keyValue = new KeyValue();
    keyValue.setKey(EXTENSION_KEY + ".label");
    keyValue.setValue("My Solution");

    KeyValue keyValue_summary = new KeyValue();
    keyValue_summary.setKey(EXTENSION_KEY + ".summary");
    keyValue_summary.setValue("This is a brief description of My Solution.");

    extensionResourceInfo.getData().add(keyValue);
    extensionResourceInfo.getData().add(keyValue_summary);

    extension.getResourceList().add(extensionResourceInfo);
    [...]
}
```

7. Optional: Add more `ExtensionResourceInfo` instances to provide localized text that displays when the extension runs in different locales.
   For example, you can add an `ExtensionResourceInfo` instance to `MyManager.java` to provide a French translation of the extension name.

```
private Extension createExtensionObject() {
    Extension extension = new Extension();
    [...]
    ExtensionResourceInfo extensionResourceInfo = new ExtensionResourceInfo();
    extensionResourceInfo.setLocale("en");
    extensionResourceInfo.setModule("extension");
    KeyValue keyValue = new KeyValue();
    keyValue.setKey(EXTENSION_KEY + ".label");
    keyValue.setValue("My Solution");

    KeyValue keyValue_summary = new KeyValue();
    keyValue_summary.setKey(EXTENSION_KEY + ".summary");
    keyValue_summary.setValue("This is a brief description of My Solution.");

    extensionResourceInfo.getData().add(keyValue);
    extensionResourceInfo.getData().add(keyValue_summary);

    ExtensionResourceInfo extensionResourceInfo_FR = new ExtensionResourceInfo();
    extensionResourceInfo_FR.setLocale("fr");
    extensionResourceInfo_FR.setModule("extension");
    KeyValue keyValue_FR = new KeyValue();
    keyValue_FR.setKey(EXTENSION_KEY + ".label");
    keyValue_FR.setValue("Ceci est une brève description de Ma solution.");

    extensionResourceInfo_FR.getData().add(keyValue_FR);

    extension.getResourceList().add(extensionResourceInfo);
    extension.getResourceList().add(extensionResourceInfo_FR);
```

```
        [...]
    }
```

8. Optional: Save your changes, build, and deploy your solution.

   If you edit the `label` value, the extension appears in vCenter Server Extensions with the new name. If you added an `ExtensionResourceInfo` object for a different locale, the localized text that you added appears in vCenter Server Extensions when you connect your solution to a vCenter instance that runs in that locale.

You added localizable message resources to the extension, so that onscreen messages and labels that your extension provides can appear in different languages in different locales.

Identify the virtual machines or vApps that an extension manages

If an extension manages virtual machines or vApps, you can identify those virtual machines or vApps as being managed by that extension. You can add icons to the objects that the extension manages that appear in the vCenter inventory, and warn users if they try to perform actions on those objects.

You have a vCenter extension that manages virtual machines or vApps.

For example, your solution can create ESX agent virtual machines. The ESX agent virtual machines that it manages appear in the vCenter inventory with an icon that identifies them as ESX agents. A panel appears in the **Summary** tab for those virtual machines that identifies ESX Agent Manager as the solution that manages them. If you attempt to perform an operation directly on an ESX agent virtual machine, you see a warning that instructs you to perform the operation by using ESX Agent Manager, rather than by performing it manually on the virtual machine. For the list of operations that trigger warnings when you try to perform them on a virtual machine or vApp that an extension manages, see What are the operations that trigger warnings from extensions.

You identify a virtual machine or vApp as belonging to an extension by setting the `managedBy` property in the `VirtualMachineConfigSpec` or `VAppConfigSpec` implementations that define the virtual machines or vApps that the extension manages. You set the `managedBy` property to a `ManagedByInfo` object.

You identify the extension that manages a virtual machine or vApp by setting the `extensionKey` property in the `ManagedByInfo` object. You specify different types of virtual machine or vApp that an extension manages by setting the `type` property in `ManagedByInfo`.

If you set the `ManagedByInfotype` property in the virtual machine or vApp definition, you can pass this value to the `ExtManagedEntityInfo` implementation in the extension definition. `ExtManagedEntityInfo` applies descriptions and icons to all the virtual machines or vApps of this type that the extension manages.

For information about configuring and deploying virtual machines programmatically, see the *vSphere Web Services SDK Programming Guide* and the *VMware vSphere API Reference*.

1. In the program that defines the virtual machines or vApps that an extension deploys, create an instance of `VirtualMachineConfigSpec` or `VAppConfigSpec`.
   For example, you can instantiate `VirtualMachineConfigSpec`.
   ```
   VirtualMachineConfigSpec configSpec = new VirtualMachineConfigSpec();
   ```
2. Create an instance of `ManagedByInfo`.
   ```
   ManagedByInfo managedByInfo = new ManagedByInfo();
   ```
3. Set the `ManagedByInfoextensionKey` property to the extension key of the extension that deploys the virtual machines or vApps.

   Use the extension key that you define in the implementation of `Extension` in your extension.
   ```
   managedByInfo.setExtensionKey("com.mycompany.myextension");
   ```
4. Set the `ManagedByInfotype` property to identify the virtual machine or vApp as being of a certain type.

   Set the `type` property to a descriptive name for this type of virtual machine. In the implementation of `Extension` that manages the extension, you can apply icons and descriptions to all virtual machines or vApps of this type that the extension deploys.
   ```
   managedByInfo.setType("my VM type");
   ```

5. Pass the `ManagedByInfo` instance to the `managedBy` property of the `VirtualMachineConfigSpec` or `VAppConfigSpec` implementation.
```
configSpec.setManagedBy(managedByInfo);
```

You set the `managedBy` properties in a virtual machine or vApp definition, to identify the virtual machines or vApps as being of a certain type and as belonging to an extension.

Set the types of virtual machines or vApps that an extension manages by implementing `ExtManagedEntityInfo`.

What are the operations that trigger warnings from extensions
If an extension deploys virtual machines or vApps, you can configure them as being virtual machines or vApps that the extension manages. If users attempt to perform operations on these virtual machines or vApps, they see a warning.

You configure a virtual machine or vApp as belonging to an extension by implementing `ManagedByInfo` in the configuration of that virtual machine or vApp.

The following operations result in a warning if a user attempts to run them from the vSphere Client on a virtual machine or vApp that an extension manages:

- `PowerOn`
- `PowerOff`
- `Suspend`
- `Reset`
- `RebootGuest`
- `ShutdownGuest`
- `StandbyGuest`
- `Edit`
- `Clone`
- `Templatize`
- `ConvertToVirtualMachine`
- `Deploy`
- `CloneTemplateToTemplate`
- `CloneVmToTemplate`
- `TakeSnapshot`
- `GotoSnapshot`
- `StartRecording`
- `Remove`
- `CreateSecondaryVm`
- `DisableFaultTolerance`
- `TurnFaultToleranceOff`
- `MigrateHost`
- `Migrate`
- `RelocateDatastore`
- `Rename`

Set the types of the virtual machines or vApps that the extension manages
You can set properties in the `ExtManagedEntityInfo` data object to define how the vSphere Client displays the different types of virtual machines and vApps that an extension deploys.

Verify that you have set the `managedBy` property in the configuration of the virtual machines or vApps that an extension deploys. See Identify the virtual machines or vApps that an extension manages.

If you set the `ManagedByInfotype` property in a virtual machine or vApp configuration, you can configure an extension to apply certain properties to all the virtual machines or vApps of that type that it deploys. You can implement the `ExtManagedEntityInfo` data object to apply a common description and an icon that appears in the vCenter inventory for all virtual machines of that type that the extension deploys.

You add an array of ExtManagedEntityInfo objects to an extension by setting the ExtensionmanagedEntityInfo property. If your extension deploys different types of virtual machine or vApp, you can add one ExtManagedEntityInfo object to the array for each type of virtual machine or vApp. By creating several instances of ExtManagedEntityInfo with different properties, you can differentiate the different types of virtual machines or vApps that the extension deploys in the vCenter inventory.

1. Create an instance of ExtManagedEntityInfo in the implementation of Extension in the program that manages the extension.
   ```
   Extension extension = new Extension();
   ExtManagedEntityInfo extManagedEntityInfo = new ExtManagedEntityInfo();
   ```
2. Set the ExtManagedEntityInfotype property to the type value that you set in the ManagedByInfo property in the program that defines the virtual machine or vApp to deploy.
   For example, set the type property to the same value as the type property of the ManagedByInfo object.
   ```
   extManagedEntityInfo.setType("my VM type");
   ```
3. Add the ExtManagedEntityInfo instance to the array of ExtManagedEntityInfo instances in the ExtensionmanagedEntityInfo property.
   ```
   extension.getManagedEntityInfo().add(extManagedEntityInfo);
   ```

You set the types of virtual machine or vApp that the extension manages. When you set different types of virtual machine or vApp in an extension, you can change how those types of object appear in the vSphere inventory.

Set a description for the types of object that the extension manages.

Set the description for a type of virtual machine or vApp that a solution manages
If a solution deploys different types of virtual machine or vApp, you can provide a description for each type by setting the ExtManagedEntityInfodescription property.

- Verify that you have set the managedBy property in the configuration of the virtual machines or vApps that an extension deploys. See Identify the virtual machines or vApps that an extension manages.
- Verify that you have created an instance of ExtManagedEntityInfo in the program that defines an extension.

The text that you set in the ExtManagedEntityInfodescription property allows you to add a description to the types of virtual machines or vApps that your solution deploys. If a solution deploys different types of virtual machine, you can create several ExtManagedEntityInfo instances, each with a different description.

Call the ExtManagedEntityInfo.setDescription() method to set the ExtManagedEntityInfodescription property for a type of virtual machine or vApp that the solution deploys.

```
Extension extension = new Extension();
ExtManagedEntityInfo extManagedEntityInfo = new ExtManagedEntityInfo();
extManagedEntityInfo.setType("my VM type");
extManagedEntityInfo.setDescription("Description of this type of virtual machine or vApp.");
extension.getManagedEntityInfo().add(extManagedEntityInfo);
```

You added a description to all virtual machines or vApps of a certain type that your solution deploys. The description appears in the vSphere Client when the solution registers with vCenter.

Unregister the extension from vCenter
When you develop an extension, the best practice is to provide a means of unregistering the extension from vCenter.

Download the vSphere ESX Agent Manager SDK.

You unregister an extension from vCenter by calling the ExtensionManager.unregisterExtension() method. To unregister an extension, you pass to unregisterExtension() the ManagedObjectReference instance that identifies the ExtensionManager for the vCenter and the extension key for the extension.

Define a method that unregisters the extension by calling ExtensionManager.unregisterExtension().

For example, in MyManager.java you can define a method named cleanup() that unregisters your solution from vCenter. In the MyVimConnection.java class you can define the connection to vCenter. MyManager.java creates an instance of MyVimConnection, _myvimConnection, with which it connects to vCenter. The MyVimConnection.getStub() method obtains the port on which the vCenter is running.

```
public void cleanup()
  throws NotFoundFaultMsg, RuntimeFaultFaultMsg {
    [...]
    _myvimConnection.getStub().unregisterExtension(
                                    _myvimConnection.getExtensionManager(),
                                    EXTENSION_KEY);
}
```

You defined a method to unregister an extension from vCenter.

What are the SDK objects for integration with vCenter Server Extensions

You integrate a solution with vCenter Server Extensions by implementing certain properties and data objects from the vSphere Management SDK in a vCenter extension.

vCenter Server Extensions provides a graphical view in the vSphere Client of the list of extensions that the `ExtensionManager` managed object exposes to vCenter. To integrate a solution with vCenter Server Extensions, you set properties in the implementation of the `Extension` data object that defines an instance of a vCenter extension. Some of the properties that you set in the `Extension` implementation refer to data objects in the vSphere Management SDK that are specific to vCenter Server Extensions.

Properties of the `Extension` data object that relate to the vCenter Server Extensions

Certain properties of the `Extension` data object are specific to vCenter Server Extensions.

You can use the properties of the `Extension` data object to integrate a solution with vCenter Server Extensions.

Properties	Type	Description
`shownInSolutionManager`	Boolean	You must set this property to `true` for a solution to appear in vCenter Server Extensions.
`solutionManagerInfo`	`ExtSolutionManagerInfo` data object	Provides the names for tabs and URLs to the Web applications that define the contents of the tabs.
`extendedProductInfo`	`ExtExtendedProductInfo` data object	Provides information about the solution, such as URLs to product and vendor Web pages.
`managedEntityInfo`	`ExtManagedEntityInfo` data object that represent virtual machines and vApps	Provides information about the objects that the solution manages, such as their type, a description of the objects, and an icon to represent the objects in the vCenter inventory.

vCenter Server Extensions data objects

You set properties in the `Extension` implementation to integrate a solution with the vCenter Server Extensions. Some of the properties that you set refer to data objects in the vSphere Management SDK that are specific to the vCenter Server Extensions.

For information about the properties and implementation details of each data object, see the VMware vSphere API Reference documentation.

Table 1913: Data Objects in the vSphere Management SDK that relate to the vCenter Server Extensions

Data Object	Description
`ExtSolutionManagerInfo`	Provides an array of `ExtSolutionManagerInfoTabInfo` objects that implement the tabs that the solution adds to vCenter Server Extensions.
`ExtExtendedProductInfo`	Provides URLs to information about the solution, such as the solution vendor's Web site, a management interface for the solution, and a description of the solution. Also provides links to the virtual machine or vApp that runs the solution.
`ExtManagedEntityInfo`	Provides information about the objects that the solution manages, such as a description, an icon to represent the objects that the solution creates in the vCenter inventory, and the types of the objects.
`ExtSolutionManagerInfoTabInfo`	Provides label for the solution in the Solutions portlet on the Summary page. You can also add URLs to the Web application.
`ManagedByInfo`	Identifies types of virtual machine or vApp as belonging to a solution. You can configure the virtual machines or vApps that a solution deploys so that they appear in the vCenter inventory with special icons and descriptions. You can also warn users about performing operations on virtual machines or vApps that a solution manages.

How do I integrate a solution with vCenter Server Extensions

You can use the vSphere Web Services API to add a solution to vCenter Server Extensions.

Add a solution to vCenter Server Extensions

You add a solution to vCenter Server Extensions by setting the `shownInSolutionManager` property in the implementation of the `Extension` data object that manages the solution.

Download the vSphere ESX Agent Manager SDK.

You can set the `shownInSolutionManager` property in the `MyManager.java` class.

1. Call the `ExtSolutionManagerInfo()` constructor to create an instance of `ExtSolutionManagerInfo` in the class that implements `Extension`.
   You can add the following code to the `MyManager.java` class to create an instance of `ExtSolutionManagerInfo` for your solution.

```
private Extension createExtensionObject() {
  Extension extension = new Extension();

  [...]
  ExtSolutionManagerInfo extSolutionManagerInfo =
    new ExtSolutionManagerInfo();
  [...]

}
```

2. Call the `setSolutionManagerInfo()` method to set the `ExtSolutionManagerInfo` instance in the solution implementation.
   You can add the following code to the `MyManager.java` class to call the `setSolutionManagerInfo()` method.

```
private Extension createExtensionObject() {
  Extension extension = new Extension();

  [...]
```

```
    ExtSolutionManagerInfo extSolutionManagerInfo =
      new ExtSolutionManagerInfo();
    [...]
    extension.setSolutionManagerInfo(extSolutionManagerInfo);

  }
```

3. Set the value of the `shownInSolutionManager` property to true by calling the `setShownInSolutionManager()` method on the implementation of the `Extension` object.
   You can add the following code to the `MyManager.java` class to set the `shownInSolutionManager` property.

```
private Extension createExtensionObject() {
  Extension extension = new Extension();

  [...]
  ExtSolutionManagerInfo extSolutionManagerInfo =
    new ExtSolutionManagerInfo();
  [...]
  extension.setSolutionManagerInfo(extSolutionManagerInfo);
  extension.setShownInSolutionManager(true);
  [...]

}
```

You set the `shownInSolutionManager` property in the implementation of the `Extension` data object that defines the solution. By setting the `shownInSolutionManager` property to `true`, a solution appears in vCenter Server Extensions when it registers with vCenter.

Set the icon for a type of virtual machine or vApp that an extension manages

You can provide an icon for each type of virtual machine or vApp that an extension manages by setting the `smallIconUrl` property of `ExtManagedEntityInfo`.

- Verify that you have set the `managedBy` property in the configuration of the virtual machines or vApps that an extension deploys. See Identify the virtual machines or vApps that an extension manages.
- Verify that you have created an instance of `ExtManagedEntityInfo` in the program that defines an extension.

The icon that you set with the `smallIconUrl` property of `ExtManagedEntityInfo` allows you to identify the types of virtual machines or vApps that your extension deploys. If an extension deploys different types of virtual machines, you can create several `ExtManagedEntityInfo` instances, with a different icon for each type. The virtual machines that the extension deploys appear in the vCenter inventory with the icons that you set.

The icon image must be in the PNG format and must measure 16 by 16 pixels. You must save the icon image to an appropriate location in the Web application that defines the extension. You provide a URL or path to the image to the `ExtManagedEntityInfo` instance for the type of virtual machine or vApp that this icon represents. Extensions access the icon image by using HTTP. Extensions do not support HTTPS URLs to icon images.

1. Create an icon image of type `PNG` and of dimensions 16 by 16 pixels.
2. Save the icon image to an appropriate location in the Web application that defines your extension.
3. Call `ExtManagedEntityInfo.setSmallIconUrl()` to set the `smallIconUrl` property for a type of virtual machine or vApp that the extension deploys.

```
Extension extension = new Extension();
ExtManagedEntityInfo extManagedEntityInfo = new ExtManagedEntityInfo();
extManagedEntityInfo.setType("my_vm_type");
extManagedEntityInfo.setDescription("Description of this type of virtual machine or vApp.");
extManagedEntityInfo.setSmallIconUrl("path_to_PNG_image");
extension.getManagedEntityInfo().add(extManagedEntityInfo);
```

Virtual machines and vApps that your extension deploys appear in the vCenter inventory with the icon that you set.

Add tabs to a solution

When you create a solution, you can add tabs to your solution to allow users to configure the solution and to access the functions of the solution in the vSphere Client.

Download the vSphere ESX Agent Manager SDK.

You define the content of tabs as dynamic Web pages. For example, you can define tabs that allow you to perform the following kinds of actions.

- Configure the solution.
- Show events triggered by the solution events or vCenter.
- Deploy predefined virtual machines or vApps of different types.
- Monitor the solution application or the virtual machines that it deploys.
- Uninstall the solution.

You create tabs by implementing the `ExtSolutionManagerInfoTabInfo` data object in the program that manages the solution. You set properties in `ExtSolutionManagerInfoTabInfo` to provide a label for the tabs and a URL to the dynamic Web pages that provide the content of the tabs. You create one `ExtSolutionManagerInfoTabInfo` instance for each tab that you add to your solution. You pass that instance to `ExtSolutionManagerInfo` in an array.

How you define the content of the tabs depends on the function of the application that the solution adds to vCenter.

You can implement `ExtSolutionManagerInfoTabInfo` in the `MyManager.java` class.

1. Create an instance of `ExtSolutionManagerInfo` in the program that manages the solution.
   Add the following lines in the `MyManager.java` class to create an instance of `ExtSolutionManagerInfo`.
   ```
   private Extension createExtensionObject() {
     Extension extension = new Extension();
     [...]
     ExtSolutionManagerInfo extSolutionManagerInfo =
       new ExtSolutionManagerInfo();
     [...]
   }
   ```
2. Create an instance of `ExtSolutionManagerInfoTabInfo` to contain the name of the page and a link to the Web page that defines its contents.
   ```
   ExtSolutionManagerInfoTabInfo extSolutionManagerInfoTabInfo =
       new ExtSolutionManagerInfoTabInfo();
   ```
3. Call the `ExtSolutionManagerInfoTabInfo.setLabel()` method to provide a name for the page.
   Add the following line to `MyManager.java` to name your solution Configuration page.
   ```
   extSolutionManagerInfoTabInfo.setLabel("Configuration");
   ```
4. Call the `ExtSolutionManagerInfoTabInfo.setUrl()` method to provide a URL to the Web page that defines the contents of your solution Configuration page.
   You can set in `MyManager.java` that your solution uses the `config.html` Web page to define the contents of the configuration page.
   ```
   extSolutionManagerInfoTabInfo.setUrl("/config.html");
   ```
5. Add the tab to the array of `ExtSolutionManagerInfoTabInfo` instances that define the tabs for the solution in the `ExtSolutionManagerInfo` object.
   ```
   extSolutionManagerInfo.getTab().add(extSolutionManagerInfoTabInfo);
   ```

You added solution-specific tabs that appear on the Summary page of your solution.

Set up health monitoring for the solution and the objects that it manages.

Set up health monitoring for a solution

vCenter pulls data from solutions about their health status. Solutions expose health data by providing a URL to an XML file that defines the health model of the solution.

Download the vSphere ESX Agent Manager SDK.

All solutions must provide an XML file that specifies their health model. The health specification XML file for your solution must conform to the VMware Health Service extensible schema definition (XSD). See Solution health XML schema for the complete VMware solution health schema.

Solutions expose health data about themselves to vCenter by publishing XML documents that declare different health statuses, depending on the events that occur in the solution. You provide messages to accompany each health status in the XML document that the solution exposes to vCenter. Solutions can provide the following statuses to vCenter:

- `alert`
- `warning`
- `info`

When you develop a solution, you must include a program or function that generates the health status XML file for the solution. The health status of a solution can be red, yellow, or green.

You must pass a URL to the XML file that defines the health model for a solution to the `ExtensionHealthInfo` object in the program that manages the solution. For example, you can create the following classes as part of your solution:

- `MyHealthProvider.java` define the health statuses for your solution.
- `MyHealthStatusServlet.java` dynamically create the XML file in which the solution exposes health status data.

You can implement the `ExtensionHealthInfo` in the `MyManager.java` class. The `MyManager.java` class can set a URL to a health definition XML file in the `ExtensionHealthInfourl` property. The URL that `MyManager.java` provides must be the path to the `health.xml` file that `MyHealthStatusServlet.java` will create.

1. Create a program that defines the health statuses of the solution.
   You can define the health statuses of your solution in the `MyHealthProvider.java` class.
2. Create a program that creates an XML file that exposes the health status data, conforming to the VMware health service XSD.
   You can create the health data XML file in the `MyHealthStatusServlet.java` class.
   `MyHealthStatusServlet.java` implements `MyHealthProvider` to extract the health status from the solution. The class creates an XML file, `health.xml`, that exposes the health data about the solution according to the health status that `MyHealthProvider` provides.
3. Create an instance of `ExtensionHealthInfo` in the program that manages the solution.
   You can implement `ExtensionHealthInfo` in `MyManager.java`.
   ```
   private Extension createExtensionObject() {
     Extension extension = new Extension();
     [...]
     ExtensionHealthInfo healthInfo = new ExtensionHealthInfo();
     [...]
   }
   ```
4. Call the `ExtensionHealthInfo.setUrl()` method to set the URL at which the solution publishes its health data XML file.
   Your solution can publish an XML file, `health.xml`, that the `MyHealthStatusServlet.java` class generates.
   ```
   healthInfo.setUrl(_url.toString() + "/health/health.xml");
   ```
5. Call the `Extension.setHealthInfo()` method to add the `ExtensionHealthInfo` object to the solution.
   `MyManager.java` can provide a link to the `health.xml` XML file that the `MyHealthStatusServlet.java` class generates.
   ```
   private Extension createExtensionObject() {
     Extension extension = new Extension();
     [...]
     ExtensionHealthInfo healthInfo = new ExtensionHealthInfo();
     healthInfo.setUrl(_url.toString() + "/health/health.xml");
     extension.setHealthInfo(healthInfo);
     [...]
   ```

```
}
```

Contents of a solution health XML file

Your solution can publish health data at `http://<solution_ip_address>:<solution_port>/my_sample/health/health.xml`. The `MyHealthStatusServlet.java` class in your solution can generate this file when the solution starts. This example shows the XML file that `MyHealthStatusServlet.java` generates when your solution is running correctly.

```
<vimhealth schemaVersion="1.0">
  <health id="com.mycompany.mysolution">
    <name>My Solution</name>
    <status>green</status>
    <message id="com.mycompany.mysolution" level="info" time="<current_date_and_time>">Running</
message>
  </health>
</vimhealth>
```

Solution health XML schema

Solutions must provide data about their health status to vCenter in XML documents.

The health status XML documents that a solutions pushes to vCenter must conform to the VMware health schema.

```
<?xml version="1.0" encoding="utf-8"?>
 <xs:schema targetNamespace="http://www.vmware.com/vi/healthservice"
            elementFormDefault="qualified"
            xmlns="http://www.vmware.com/vi/healthservice"
            xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0">

   <xs:complexType name="healthType">
    <xs:sequence>
      <xs:element name="name" type="xs:string" minOccurs="0" maxOccurs="1"/>
      <xs:element name="status" minOccurs="1" maxOccurs="1">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="green" />
            <xs:enumeration value="yellow" />
            <xs:enumeration value="red" />
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="message" minOccurs="0" maxOccurs="unbounded">
        <xs:complexType mixed="true">
          <xs:sequence>
            <xs:element name="param" type="xs:string" minOccurs="0" maxOccurs="unbounded"/>
          </xs:sequence>
          <xs:attribute name="id" type="xs:string" use="required"/>
          <xs:attribute name="level" use="required">
            <xs:simpleType>
              <xs:restriction base="xs:string">
                <xs:enumeration value="info" />
                <xs:enumeration value="warning" />
                <xs:enumeration value="alert" />
              </xs:restriction>
            </xs:simpleType>
          </xs:attribute>
          <xs:attribute name="time" type="xs:dateTime" use="required"/>
        </xs:complexType>
      </xs:element>
      <xs:element name="health" type="healthType" minOccurs="0" maxOccurs="unbounded"/>
```

```
    </xs:sequence>
    <xs:attribute name="id" type="xs:string" use="required"/>
   </xs:complexType>

   <xs:element name="vimhealth">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="health" type="healthType" minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
      <xs:attribute name="schemaVersion" type="xs:decimal" use="required"/>
    </xs:complexType>
   </xs:element>
</xs:schema>
```

How do I integrate solutions with vSphere ESX Agent Manager

vSphere ESX Agent Manager is a standard vCenter solution that allows other solutions to deploy, monitor, and manage ESX agents on ESX hosts.

ESX Agent Manager performs the following functions:

- Provisions ESX agent virtual machines for solutions.
- Monitors changes to the ESX agent virtual machines and their scope in vCenter.
- Reports configuration issues in the ESX agents to the solution.
- Integrates agent virtual machines with vSphere features such as Distributed Resource Scheduler (DRS), Distributed Power Management (DPM), vSphere High Availability (HA), fault tolerance, maintenance mode, and operations such as adding and removing hosts to and from clusters.

A solution can add functions to an ESX host by deploying an ESX agent virtual machine and an optional vSphere Installation Bundle (VIB) that provide this function.

Every vCenter instance contains a running ESX Agent Manager.

Benefits of integrating solutions with ESX Agent Manager

Integrating a solution with ESX Agent Manager has several benefits and eliminates potential problems that can occur when you deploy ESX agent virtual machines.

Monitoring solutions

When you integrate a solution with ESX Agent Manager, users can monitor the health of agent virtual machines in the vSphere Client. The solution can monitor ESX agents by using the ESX Agent Manager API. Integrating a solution with ESX Agent Manager also helps you to add functions to ESX hosts.

User privileges

By integrating a solution with ESX Agent Manager, you can limit certain operations on ESX agent virtual machines to certain types of user. For example, you can block all users from powering off an ESX agent unless they have the **EAM.Modify** privilege.

Integration with vSphere features

When you develop solutions that deploy virtual machines that extend the function of ESX hosts, integrate the solution with ESX Agent Manager. If you do not integrate the solution and ESX Agent Manager, vSphere does not detect that virtual machines are ESX agent virtual machines, Virtual machines that ESX Agent Manager deploys integrate with vSphere features in ways that other virtual machines do not. For example, if an ESX host implements vSphere High Availability (VMware HA) and that host stops running unexpectedly, normally all the virtual machines running on that host migrate to another ESX host. An ESX agent virtual machine should not migrate to another host. Features such as vSphere Distributed Resource Scheduler (DRS), the Add Host to Cluster operation, vSphere Distributed Power Management

(DPM), and maintenance mode can also create problems if they modify or migrate ESX agents. Similarly, if one of these features migrates or modifies an ESX agent that a solution requires, that solution can stop running.

ESX Agent Manager is compatible with the VMware HA, DRS, `AddHost`, DPM, and maintenance mode features. By integrating a solution with ESX Agent Manager, you can deploy ESX agent virtual machines that modify the function of ESX hosts and implement VMware HA, DRS, `AddHost`, DPM, and maintenance mode. ESX Agent Manager ensures that solutions that deploy ESX agents integrate correctly with these vSphere features.

ESX Agent virtual machines and High Availability

If you have a configuration with a very large virtual appliance per host, a best practice is to use an ESX Agent VM as a container for the appliance, because vCenter High Availability (HA) feature gives special priority to ESX Agent VMs.

The HA feature will power on an ESX Agent VM before any other VMs.

If a failover host policy is in use, the HA feature allows Agent VMs to be powered on with the failover hosts.

The HA feature does not include ESX Agent VMs in admission control calculations. Therefore, if you want to use a VM that is not included in the slot size for the 'host failures to tolerate' admission control policy, you can use an Agent VM.

To take advantage of these special priorities, you must include an ESX Agent VM on each host in a cluster.

Table 1914: Interaction between vSphere features and ESX Agent Manager

vSphere Feature	Interaction with ESX Agent Manager
VMware HA	• If a host stops unexpectedly, VMware HA does not start an ESX agent virtual machine on another host, unless a failover policy is in use. • When a host restarts, VMware HA restarts the ESX agent virtual machines before it restarts other virtual machines. • ESX agent virtual machines are not included in slot size for admission control calculations.
Distributed Resource Scheduler (DRS)	• ESX Agent Manager pins ESX agent virtual machines to the hosts on which they are running. DRS does not move ESX agent virtual machines between hosts in a cluster. • ESX Agent Manager blocks DRS from moving virtual machines that are not ESX agents to hosts in clusters where ESX agents that the virtual machines require are not available.
Add Host to Cluster operation	When a user adds a host to a cluster of hosts that require an ESX agent, ESX Agent Manager deploys the required ESX agents on the new host.
vSphere Distributed Power Management (DPM)	DPM can put a host into standby mode even when ESX agent virtual machines are present. DPM powers off ESX agent virtual machines only after it has moved all the other virtual machines to another host and put the host into standby mode.
Maintenance mode	When a host enters maintenance mode, ESX Agent Manager powers off any ESX agent virtual machines and restarts the ESX agents when the host exits maintenance mode.

vSphere ESX Agent Manager architecture

ESX Agent Manager is an intermediary between vCenter and a solution. ESX Agent Manager provides a WSDL based SOAP API that you can use to create solutions that deploy and monitor ESX agent virtual machines on ESX hosts, and integrate the ESX agents with Distributed Resource Scheduler (DRS), Distributed Power Management (DPM), High Availability (HA), and other vSphere features.

For an overview of the architecture of ESX Agent Manager and how it integrates with vCenter, see the introduction to the *ESX Agent Manager API Reference*.

Configuration components of ESX agencies

For a solution to integrate with ESX Agent Manager, it must register with vCenter as an extension, and create an ESX agency that defines the configuration of the solution and the ESX agents that the solution deploys.

ESX agencies encapsulate the ESX agents that they deploy from a solution. When you create an ESX agency, you must provide configuration information that the solution applies to the individual ESX agents that it deploys.

You must set the scope of the ESX agents that the agency defines and the ESX agency goal state. You provide a URL to an Open Virtualization Format (OVF) file that defines the ESX agent virtual machines. You also provide a URL to an optional vSphere installation bundle (VIB) that provides a function that extends an ESX host, for example, a VMkernel module or a custom ESX server application.

ESX agency scope

When you create an ESX agency, you must define the scope of the agency. You define the scope of an ESX agent in terms of compute resources, namely standalone hosts or clusters.

For example, ESX agents can run on standalone ESX hosts or on clusters of ESX hosts.

A solution must obtain from vCenter the set of compute resource instances on which to deploy ESX agents. For example, you can implement a query in your solution that obtains a list of all the hosts that are running in a vCenter instance. Users can then select the hosts on which to deploy ESX agents.

You set the initial ESX agency scope in the ESX agency configuration. As the solution runs, the scope of the ESX agency can change. For example, your solution runs on compute resources that users select from your solution Configuration page. As users select different compute resources on which to run the solution, the scope of the ESX agency changes as it adds the MoRefs of the selected compute resources to the scope.

ESX agency goal state

When you create an ESX agency, you define a goal state for that ESX agency. The goal state of an ESX agency defines the state in which all the ESX agents in that ESX agency should be when the solution is running correctly.

If the ESX agency is not in its goal state, ESX Agent Manager attempts to remediate the issues that it discovers, so that the agency can achieve or return to its goal state.

Table 1915: ESX agent goal states

Goal State	Description
Enabled	Solution has deployed all ESX agents in the ESX agency successfully, and they are running with a heartbeat. For ESX agent virtual machines without VMware Tools installed, which do not return a heartbeat, ESX Agent Manager considers them to be enabled when they are powered on.
Disabled	Solution has deployed all ESX agents in the ESX agency successfully, but they are not running.
Undeployed	Solution has removed all ESX agents in the ESX agency from all the compute resources in the ESX agent scope.

ESX agency status

When you create an agency, you define the goal state of the ESX agents that the agency contains. The status of an ESX agency reflects the situation of the ESX agency in relation to its goal state.

ESX Agent Manager notifies a solution of the status of its ESX agencies. ESX Agent Manager defines three possible statuses for ESX agencies.

Table 1916: ESX agent status

Status	Description
Green	The ESX agency has achieved its goal state.
Yellow	ESX Agent Manager is working to move the ESX agency into its goal state.
Red	The ESX agency cannot achieve its goal state. The solution or the vSphere administrator must actively resolve one or more issues. ESX Agent Manager reports the issues to the solution in the agency runtime. ESX Agent Manager can resolve certain issues if you select the agency, click the vertical ellipsis and click **Resolve All Issues**, for example if an ESX agent is powered off.

ESX Agent Manager sets the ESX agency status to yellow until the ESX agency reaches its goal state. This can either mean that the deployment of the ESX agents is still ongoing, or that ESX Agent Manager is attempting to remediate issues. You define the remediation of issues in the implementation of the solution, or you can leave the remediation of issues to the vSphere administrator, who can resolve the issue by using the ESX Agent Manager user interface.

The criteria on which ESX Agent Manager determines that the status of an ESX agency is green depends on the goal state of that ESX agency.

Table 1917: Criteria for meeting the goal state of an ESX agency

ESX Agency Goal State	Criteria for Green Status
Enabled	ESX Agent Manager has downloaded the ESX agent binaries from the OVF and VIB URLs, deployed the ESX agent virtual machines, installed the VIB, started the ESX agent virtual machines, and received heartbeats from all of them.
Disabled	ESX Agent Manager has downloaded the ESX agent binaries from the OVF and VIB URLs, deployed the ESX agent virtual machines, and installed the VIB.
Undeployed	ESX Agent Manager has powered off and deleted all the ESX agent virtual machines and uninstalled the VIB.

If the ESX agency cannot reach its goal state, ESX Agent Manager reports the issues to the solution and sets the status to red.

> **Note:**
>
> If the scope of an ESX agency is empty, there are no compute resources onto which to deploy ESX agents, so no ESX agents are deployed. In this case, ESX Agent Manager determines that the ESX agency has performed correctly, and sets the status to green.

Status of ESX agency scope changes

When a solution updates its ESX agency scope, ESX Agent Manager provisions new ESX agents on new hosts and removes ESX agents from hosts that are no longer in the scope.

When the scope of an ESX agency changes, vSphere ESX Agent Manager detects the change and displays the status of the ESX agents in the scope and operations that are in progress. You can add functions in the solution definition to remediate the issues, or the solution can report the issues so that administrators can fix them manually.

When a solution adds or removes compute resources from the ESX agency scope, the status of the ESX agency changes as the solution works to achieve the goal state of the ESX agency.

Table 1918: Status of ESX agency scope changes

Scope Change	Current ESX Agency State	ESX Agency Status
Add compute resource.	Undeployed	Unchanged. New ESX agents always start in the undeployed state.
Remove compute resource.	Undeployed	Unchanged. Old ESX agents that the solution has not removed yet are pending undeployment.
Add compute resource.	Disabled	Yellow, until the solution deploys the ESX agents on the new compute resource.
Remove compute resource.	Disabled	Yellow, until the solution undeploys the ESX agents from the compute resource.
Add compute resource.	Enabled	Yellow, until the solution deploys the ESX agents on the new compute resource, starts the ESX agents, and receives heartbeats.
Remove compute resource.	Enabled	Yellow, until the solution stops and undeploys the ESX agents from the compute resource.

Requirements for itegrating a solution with ESX Agent Manager

To integrate with ESX Agent Manager, a solution must meet certain requirements.

To integrate a solution with ESX Agent Manager, you must develop the solution according to the following requirements:

- The solution must be a vCenter extension that implements the `Extension` data object and registers with `ExtensionManager`.
- Use Open Virtualization Format (OVF) to package ESX agent virtual machines or vApps. ESX Agent Manager only supports the deployment of virtual machines using OVF.
- Use HTTP or HTTPS to publish OVF files to ESX Agent Manager.
- Use vSphere installation bundles (VIB) to add functions to ESX hosts, for example to add VMkernel modules or custom ESX Server applications to ESX hosts.
- Use HTTP or HTTPS to publish VIB files to ESX Agent Manager.
- Use vCenter Compute Resources to define the ESX agent scope.

How do I authenticate against ESX Agent Manager

vCenter handles the authentication of ESX Agent Manager clients, so a solution must first log in to vCenter before it can call the methods of the ESX Agent Manager API.

ESX Agent Manager registers itself with the vCenter reverse proxy in the `eam` namespace. You find the ESX Agent Manager API under `eam/sdk`. You must direct all ESX Agent Manager API calls in your solution to *`vcenter_server_ip_address`*`/eam/sdk`.

ESX Agent Manager grants access to clients that are also clients of the vCenter instance in which ESX Agent Manager is an extension. All vCenter extensions that have an active vCenter session have access to ESX Agent Manager. These extensions can create ESX agencies and monitor existing ESX agencies in ESX Agent Manager. The user name that the extension provides is the extension key that the extension sets in the vCenter`ExtensionManager` instance.

Users can access the ESX Agent Manager user interface only if they have the **Eam.View** and **Eam.Modify** privileges. Solutions by definition have all vCenter privileges, but the **Eam.View** and **Eam.Modify** privileges limit what users can do in the vSphere Client.

Table 1919: ESX Agent Manager privileges

Privilege	Permitted Actions
`Eam.View`	Users can monitor all running ESX agencies in ESX Agent Manager.
`Eam.Modify`	Users can modify the ESX agencies, for example by powering off ESX agent virtual machines.

When you make an HTTP request to call an object in the ESX Agent Manager API for the first time, you must set the VMware SOAP session cookie. You set it to the value of the VMware SOAP session cookie of the vCenter HTTP connection. You must add a function in your solution to obtain the SOAP session cookie from vCenter when the solution establishes the connection to vCenterr. The cookie remains set for the duration of the ESX Agent Manager session.

How do I configure the availability of ESX agent virtual machines

You can configure ESX agencies so that they only mark ESX agent virtual machines as available after the client of the agency has performed additional configuration after provisioning or powering on the ESX agent virtual machine.

You configure an ESX agency in the `Agency.ConfigInfo` object. You can set two options in `Agency.ConfigInfo` that set when ESX agents can be marked as available.

Table 1920: ESX agent availability options

Option	Description
Boolean `manuallyMarkAgentVmAvailableAfterProvisioning`	If set to true, the client of this ESX agency must manually mark the agent as ready after the ESX agent virtual machine has been provisioned. This is useful if the client of this solution performs some extra reconfiguration of the ESX agent virtual machine before it is powered on.
Boolean `manuallyMarkAgentVmAvailableAfterPowerOn`	If set to true, the client of this ESX agency must manually mark the agent as ready after the ESX agent virtual machine has been powered on. In this case, DRS will not regard the ESX agent virtual machine as ready until the client has marked the agent as ready.

Monitoring and resolving ESX agent issues

With the ESX Agent Manager API, you can obtain status and state data about all the ESX agencies that a solution creates. The status of an ESX agency reflects the status of all the ESX agents that are running in the ESX agency.

Solutions can only access data about ESX agencies that they create. A solution cannot access the data of ESX agencies that other solutions create on the same host.

You can use ESX Agent Manager to monitor and track many of the issues that can occur when you deploy ESX agent virtual machines. ESX Agent Manager handles updates to the deployment specifications of ESX agent virtual machines. ESX Agent Manager responds to changes in the vCenter instance in which it runs. If ESX Agent Manager encounters a conflict and cannot satisfy the requirements of an update to a deployment specification, it reports the conflict to the solution as an issue and shows the issue in the ESX Agent Manager user interface.

The issues that the ESX Agent Manager monitors relate to the goal state and status of ESX agencies and agents. If ESX Agent Manager encounters an issue, it sets the status of the ESX agency or agent to red and alerts the solution about the issue. ESX Agent Manager can resolve certain issues if you click the vertical ellipsis icon and then select **Resolve Issues** in the ESX Agent Manager user interface.

How does the ESX Agent Manager remediate issues

ESX Agent Manager reports to the solution or to the vSphere administrator issues that can occur during the deployment of ESX agent virtual machines. ESX Agent Manager can resolve certain issues, if the solution or the administrator requests that it does so.

ESX Agent Manager defines standard issues that can occur during the deployment of ESX agent virtual machines. For example, deployment of ESX agents can fail because the network, datastore, or resource pool settings are incorrect.

Certain errors occur only in the certain goal states. Other errors can occur in any ESX agency goal state.

Table 1921: Remediation of issues by ESX Agent Manager (partial list)

Issue	Goal State	Resolved by ESX Agent Manager?
`AgentVmPoweredOffIssue`	Enabled	Yes
`AgentVmWrongDatastoreIssue`	Any	No
`AgentVmWrongNetworkIssue`	Any	No
`AgentVmWrongResourcePoolIssue`	Any	Yes
`AgentVmWrongFolderIssue`	Any	Yes
`InsufficientSpaceOnDatastoreIssue`	Enabled	No

How do I integrate a solution with ESX Agent Manager

You can integrate your solution with ESX Agent Manager.

For more information about the objects that your solution can implement, see the *vSphere ESX Agent Manager API Reference*.

Connect a solution to ESX Agent Manager

To connect a solution to ESX Agent Manager, you must obtain connection information from the vCenter in which you run the solution. You pass a reference to the `EsxAgentManager` instance that is running in vCenter to the solution.

- Download the vSphere ESX Agent Manager SDK.
- Obtain the following information from the vCenter instance on which you run a solution:
  - The vCenter host ID.
  - The HTTPS proxy port for vCenter.
  - The session cookie for the current user session in which the solution is logged in as an extension.
  - The managed object reference (MoRef) of the ESX Agent Manager instance that is running in the vCenter instance.

You pass the connection information to a solution by calling the appropriate methods from the vSphere API.

> **Note:**
>
> If you integrate your solution with the vCenter Extension vService, the vService obtains the vCenter connection information for you. See How do I integrate an extension with the vCenter Extension vService.

You can define the connection to vCenter in the `MyVimConnection.java` class and the connection to ESX Agent Manager in the `MyEamConnection.java` class.

The `MyVimConnection` class defines methods to obtain the vCenter host ID, HTTPS proxy, and the session cookie. The `MyManager.java` class implements `MyVimConnection` to connect to the vCenter on which your solution is running. See Connect the extension to vCenter.

1. Establish a connection to the vCenter instance on which the solution runs.
   Your solution can establish the connection to vCenter in the `MyVimConnection.java` class.
2. Create an instance of the `ManagedObjectReference` object of type `EsxAgentManager`.
   The `MyEamConnection.java` class can define a constructor that creates a MoRef named `_myeamRef` and a method named `getEsxAgentManager()` that returns the MoRef to the solution.

```
public MyEamConnection(String vcHost, int vcHttpsProxyPort, String sessionCookie) {
 [...]

 _myeamRef = new ManagedObjectReference();
 _myeamRef.setType("EsxAgentManager");
 _myeamRef.setValue("EsxAgentManager");
}

 [...]

 public ManagedObjectReference getEsxAgentManager() {
   return _myeamRef;
}
```

3. Connect to the ESX Agent Manager with the URL to the ESX Agent Manager service and the session cookie for the current session in which the solution is logged as an extension.
   The `MyEamConnection` class can define a `connect()` method that connects to the ESX Agent Manager service.

```
public void connect() {
  if (_isConnected) {
    return;
  }

  try {
    [...]

    String myeamUrl = "https://" + _vcHost + ":" + _vcHttpsProxyPort + "/eam/sdk/";
    [...]
    Map<String, List<String>> map = new HashMap<String, List<String>>();
    map.put("Cookie", Collections.singletonList(_sessionCookie));
    ((BindingProvider) _stub).getRequestContext()
            .put(MessageContext.HTTP_REQUEST_HEADERS, map);
    _isConnected = true;
  }
}
```

You obtained the connection details for vCenter and connected a solution to ESX Agent Manager.

Configure ESX agencies and agents to deploy from the solution.

Configure an ESX agency and ESX agents

You must define the configuration of the ESX agencies in the implementation of the solution. You set the configuration for ESX agencies and ESX agents in the `AgencyConfigInfo` and `AgentConfigInfo` data objects.

Download the vSphere ESX Agent Manager SDK.

When you create ESX agencies, you provide the `AgencyConfigInfo` object with an array of `AgentConfigInfo` objects for each version of ESX on which the agency deploys ESX agents. You also define the name of the ESX agency and of the ESX agents and the scope of the ESX agency in the `AgencyConfigInfo` object.

You define the deployment of the ESX agent virtual machines in the `AgentConfigInfo` object. You set the following information in the `AgentConfigInfo` object.

- A URL to the Open Virtualization Format (OVF) file from which to deploy the ESX agent.
- A URL to an optional vSphere Installation Bundle (VIB) that adds function to ESX, for example a VMkernel module or a custom ESX application that you developed.

The URL to the ESX agent virtual machine OVF and the URL to an optional VIB must lead to a server that ESX Agent Manager can access. ESX Agent Manager downloads the ESX agent virtual machine from the URLs that you provide and deploys the virtual machines on the ESX hosts. ESX Agent Manager installs one ESX agent instance per agency per host.

Note:
To install VIBs, all ESX hosts must have configured the firewall so that they can access the HTTP port on the vCenter instance.

Setting the `ovfEnvironment` property allows a solution to provide OVF properties specific to the ESX agent virtual machine. ESX Agent Manager sets the OVF properties when it deploys an ESX agent. A typical use of the `ovfEnvironment` field is to specify the IP address and credentials of the solution so that ESX agents can connect back to the solution when they are running.

1. Create a program to configure and create ESX agencies and agents.

   Your solution can define the configuration and creation of ESX agencies and agents in the `MyAgentHandler.java` class.

```
public MyAgentHandler(String selfUrl,
                      String selfIp,
                      String ovfUrl4x,
                      String ovfUrl50,
                      String vibUrl4x,
                      String vibUrl50,
                      boolean deployVibs,
                      Map<String, String> ovfEnvironment,
                      VcUtils vcUtils) {
   _vcUtils = vcUtils;

   _agentConfigInfo4x = new AgentConfigInfo();
   _agentConfigInfo50 = new AgentConfigInfo();

   [...]
}
```

2. Create `AgentConfigInfo` instances for each type of ESX agent that the ESX agency deploys.
   Your solution can define ESX agents for ESX 6.7 and for ESX 7.0.

```
public AgentHandler([...]) {
   [...]
   _agentConfigInfo67 = new AgentConfigInfo();
   _agentConfigInfo70 = new AgentConfigInfo();
   [...]
}
```

3. Set the URLs to the OVF files from which the solution deploys ESX agent virtual machines by calling the `AgentConfigInfo.setOvfPackageUrl()` method.
   Your solution can construct the URLs to OVF files from information that you set in the `mysolution.properties` file.

```
public MyAgentHandler([...]) {
   [...]

   _agentConfigInfo67.setOvfPackageUrl(urlPrefix + ovfUrl4x);
   _agentConfigInfo70.setOvfPackageUrl(urlPrefix + ovfUrl50);

   [...]
   }
```

4. Optional: Set the URLs to the optional VIB files from which the solution adds functions to ESX by calling the `AgentConfigInfo.setVibUrl()` method.
   Your solution can construct the URLs to VIB files from information that you set in the `mysolution.properties` file.

```
public MyAgentHandler([...]) {
   [...]

   _agentConfigInfo4x.setOvfPackageUrl(urlPrefix + ovfUrl4x);
   _agentConfigInfo50.setOvfPackageUrl(urlPrefix + ovfUrl50);
```

```
    if (deployVibs) {
       _agentConfigInfo67.setVibUrl(urlPrefix + vibUrl4x);
       _agentConfigInfo70.setVibUrl(urlPrefix + vibUrl50);

       [...]
    }
 }
```

5. Set any OVF environment properties that the solution requires by creating an instance of the `AgentOvfEnvironmentInfo` object and passing it to the `AgentConfigInfo` object for each ESX agent. Your solution can set some dummy properties in the `eamri-webapp.xml` file.

```
public MyAgentHandler([...]) {
   [...]
   AgentOvfEnvironmentInfo ovfEnv = new AgentOvfEnvironmentInfo();
   for (final Map.Entry<String, String> entry : ovfEnvironment.entrySet()) {
      ovfEnv.getOvfProperty().add(new AgentOvfEnvironmentInfoOvfProperty() {
         {
            setKey(entry.getKey());
            setValue(entry.getValue());
         }
      });
   }
   _agentConfigInfo67.setOvfEnvironment(ovfEnv);
   _agentConfigInfo70.setOvfEnvironment(ovfEnv);
   [...]
}
```

6. Create an instance of `AgencyConfigInfo` to define the ESX agency that the solution deploys.

```
public MyAgentHandler([...]) {
   [...]

   _agencyConfigInfo = new AgencyConfigInfo();
   [...]
}
```

7. Provide names for the ESX agency and the ESX agents by calling the `AgencyConfigInfo.setAgencyName()` and `setAgentName()` methods.
   Your solution can name the ESX agency and the ESX agents My Service.

```
public MyAgentHandler([...]) {
   [...]

   _agencyConfigInfo = new AgencyConfigInfo();
   _agencyConfigInfo.setAgencyName("My Service");
   _agencyConfigInfo.setAgentName("My Service");
   [...]
}
```

8. Add an array of ESX agent configurations to the ESX agency configuration by calling the `AgencyConfigInfo.getAgentConfig()` method.

```
public MyAgentHandler([...]) {
   [...]

   _agencyConfigInfo = new AgencyConfigInfo();
   _agencyConfigInfo.setAgencyName("My Service");
   _agencyConfigInfo.setAgentName("My Service");
   _agencyConfigInfo.getAgentConfig().add(_agentConfigInfo4x);
   _agencyConfigInfo.getAgentConfig().add(_agentConfigInfo50);
   [...]
```

```
}
```

9. Set the scope of the ESX Agency by calling the `AgencyConfigInfo.setScope()` method.
   Users of your solution can set the scope of the ESX agency by selecting ESX hosts from your solution Configuration page. Consequently, the scope is empty until your solution updates it according to the user interaction.

```
public MyAgentHandler([...]) {
   [...]
   _agencyConfigInfo.setScope(null);
   [...]
}
```

You set the configuration properties for an ESX agency and the ESX agents that it contains.

Call the `createAgency()` method to create the ESX agency.

Create an ESX agency

You create an ESX agency by calling the `EsxAgentManager.createAgency()` method. You must specify ESX agent configurations for each version of ESX on which you deploy ESX agents.

Download the vSphere ESX Agent Manager SDK.

When you call the `EsxAgentManager.createAgency()` method you must pass it an `AgencyConfigInfo` object and a string to define the initial goal state of the ESX agents that the agency deploys. The `initialGoalState` property informs ESX Agent Manager of the state in which to deploy ESX agent virtual machines when the solution first runs.

You can define a function that creates an ESX agency in a `MyAgentHandler.java` class.

1. Establish a connection to the ESX Agent Manager running in vCenter.
   The `MyAgentHandler.java` can implement the `MyEamConnection.java` class to connect to vCenter and ESX Agent Manager.

```
public void setup(MyEamConnection myeamConnection) {
 assert myeamConnection != null;
 _myeamConnection = myeamConnection;

  ManagedObjectReference eamRef = _myeamConnection.getEsxAgentManager();
  EamPortType stub = _myeamConnection.getStub();

  [...]
}
```

2. Create an ESX agency by calling the `EsxAgentManager.createAgency()` method.
   The `MyAgentHandler.java` class checks whether any ESX agencies are already running, and if not calls the `EsxAgentManager.createAgency()` method. `MyAgentHandler.java` passes to the `EsxAgentManager.createAgency()` method the `ManagedObjectReference` object for the ESX Agent Manager instance running in vCenter, `eamRef`. `MyAgentHandler.java` also passes to `createAgency()` the `AgencyConfigInfo` object that defines the configuration of the ESX agency. The `MyAgentHandler.java` class sets the initial goal state of the ESX agency to `ENABLED`.

```
public void setup(MyEamConnection myeamConnection) {
 assert myeamConnection != null;
 _myeamConnection = myeamConnection;

  ManagedObjectReference eamRef = _myeamConnection.getEsxAgentManager();
  EamPortType stub = _myeamConnection.getStub();
  try {
    List<ManagedObjectReference> agencyRefs = stub.queryAgency(eamRef);
    if (agencyRefs != null && agencyRefs.size() > 0) {
         _agency = agencyRefs.get(0);
    } else {
      _agency = stub.createAgency(eamRef,
                                  _agencyConfigInfo,
                                  EamObjectRuntimeInfoGoalState.ENABLED.toString()
```

```
                                    .toLowerCase());
        }
    [...]
    }
```

3. Call the `Agency.queryConfig()` method to verify the configuration of the ESX agency and report any issues with the configuration.

```
public void setup(MyEamConnection myeamConnection) {
    [...]
    _agencyConfigInfo = stub.queryConfig(_agency);
    _isSetup = true;
  } catch (RuntimeFaultFaultMsg e) {
     _log.error(e, e);
  } catch (InvalidAgencyScopeFaultMsg e) {
     _log.error(e, e);
  } catch (InvalidAgentConfigurationFaultMsg e) {
     _log.error(e, e);
  } catch (InvalidUrlFaultMsg e) {
     _log.error(e, e);
  }
}
```

4. Call the `Agency.agencyQueryRuntime()` method to return the status of the ESX agency.

   The `Agency.agencyQueryRuntime()` method returns an `EamObjectRuntimeInfo` object that contains the goal state of the agency, its current status, and a list of any problems that the agency has encountered.

```
public EamObjectRuntimeInfo getRuntime() throws RuntimeFaultFaultMsg {
    waitForSetup();
    return _eamConnection.getStub().agencyQueryRuntime(_agency);
}
```

You created an ESX agency that a solution can deploy on ESX hosts.

Update the Agency scope of a solution

You define the ESX agency scope of a solution by passing the managed object references (MoRefs) of the vSphere compute resources to the solution.

- Download the vSphere ESX Agent Manager SDK.
- Verify that you have set up and started your solution in an application server.

You set the initial ESX agency scope in the `scope` property of the `AgencyConfigInfo` object. You can change the scope when a solution runs by calling the `Agency.update()` method. For example, in your solution, a user can select the ESX hosts on which to run the solution from a list on the solution Configuration page. The solution can update the scope of the sample ESX agency according to the hosts that the user selects.

Your solution can define a function to update the scope of the ESX agency in the `MyAgentHandler.java` class.

1. Write a function that implements the vSphere Web Services API to detect compute resources on which to run the solution.

   Your solution can provide a helper class, `MyVcUtils.java`, that defines functions to obtain the compute resources on which to run the solution. `MyAgentHandler.java` calls the `MyVcUtils.getComputeResources()` method to obtain a list of `ManagedObjectReference` objects for the ESX hosts running in vCenter.

```
public void updateConfig(String[] updates) throws RuntimeFaultFaultMsg {
   waitForSetup();

   boolean changed = false;
   Map<String, ManagedObjectReference> crs = _myvcUtils.getComputeResources();
```

2. Add the `ManagedObjectReference` objects for the compute resources to a `HashSet` that defines the ESX agency scope.
   The `MyAgentHandler.java` class adds the list of `ManagedObjectReference` objects that the `MyVcUtils.getComputeResources()` method returns to the existing scope and updates the list if additional compute resources are present.

```
Set<ManagedObjectReference> newScope = new HashSet<ManagedObjectReference>();

for (String update : updates) {
    String[] kv = update.split("=", 2);
    if (kv[0].equals("scope")) {
      try {
         ManagedObjectReference cr = crs.get(kv[1]);
         if (cr == null) {
           continue;
         }
         ManagedObjectReference moRef = new ManagedObjectReference();
         moRef.setType(cr.getType());
         moRef.setValue(cr.getValue());
         newScope.add(moRef);
      }
       catch (NullPointerException e) {
           // ignore
      }
    }
}
```

3. Create an `AgencyComputeResourceScope` instance to contain the scope `HashSet`.

```
AgencyComputeResourceScope scopeDO = (AgencyComputeResourceScope) _agencyConfigInfo.getScope();
Set<ManagedObjectReference> oldScope = new
HashSet<ManagedObjectReference>(scopeDO.getComputeResource());
```

4. Compare the old scope to the new scope to establish whether any compute resources have been added or removed.
   The `MyAgentHandler.java` class compares the size of the new scope to the initial scope and adds any new compute resources to the `HashSet` of `ManagedObjectReference` objects.

```
if (!oldScope.containsAll(newScope) || oldScope.size() != newScope.size()) {
   AgencyComputeResourceScope scope = new AgencyComputeResourceScope();
   scope.getComputeResource().addAll(newScope);
   agencyConfigInfo.setScope(scope);
   changed = true;
}
```

5. If the new scope differs from the old scope, call `Agency.update()` to add the new scope to the ESX agency.

```
if (changed) {
  assert _agency != null;
  try {
     _eamConnection.getStub().update(_agency, agencyConfigInfo);
  } catch (Exception e) {
   _log.error("Failed to update agency. Reason: " + e.getMessage());
  }
  updateConfiguration();
}
}
```

You defined a function in a solution to detect changes of scope and to update an ESX agency.

Change the goal state of an agency

A solution can change the goal state of its ESX agencies while the solution is running. For example, when a solution starts, the goal state of its ESX agencies can be `ENABLED`. If the solution includes a function to remove ESX agencies, when this function runs, the goal state of the ESX agency changes to `UNDEPLOYED`.

- Download the vSphere ESX Agent Manager SDK.
- Verify that you have set up and started your solution in an application server.

You call the `Agency.enable()`, `Agency.disable()`, and `Agency.uninstall()` methods to change the goal state of an ESX agency. Calling these methods changes the status of the ESX agency to yellow until the agency reaches the desired state, in which case the status changes to green. If the ESX agency cannot achieve the goal state, the status changes to red.

Your solution can define a function to change the goal state of its ESX agencies in the `MyAgentHandler.java` class. Your solution can change the goal state of its ESX agency when users select ESX hosts on which to run the solution, and when they uninstall the solution.

1. Get the current goal state of the ESX agency by calling the `EamObjectRuntimeInfo.getGoalState()` method. Your solution can define a function in the `MyAgentHandler.java` class to obtain the current goal state from the solution.

```
public void updateGoalState(String params) throws RuntimeFaultFaultMsg,
 NotFoundFaultMsg {
 String[] kv = params.split("=", 2);
 assert kv[0].equals("goalState");

 String goalState = kv[1];
 String currentGoalState = getRuntime().getGoalState().toString();
```

2. Call the `Agency.enable()`, `Agency.disable()`, and `Agency.uninstall()` methods to set the ESX agency in the new goal state.
   Your solution can call the appropriate methods to set the ESX agencies to the goal state. If the goal state is `UNINSTALLED`, the solution calls the`cleanup()` method that `MyManager.java` defines to remove the ESX agencies and uninstall the solution.

```
  if (goalState.equals(currentGoalState)) {
      return;
  }

  if (goalState.equals(EamObjectRuntimeInfoGoalState.ENABLED.toString()
                                                          .toLowerCase())) {
      enable();
  } else if (goalState.equals(EamObjectRuntimeInfoGoalState.DISABLED.toString()
                                                                .toLowerCase())) {
      disable();
  } else {
      assert goalState.equals(EamObjectRuntimeInfoGoalState.UNINSTALLED.toString()
                                                                    .toLowerCase());
      _unregistered = true;
      MyManager.getInstance().cleanup();
  }
 }
```

You changed the goal state of an ESX agency while the solution is running.

Delete an ESX agency

A solution can delete an ESX agency by calling the `Agency.destroyAgency()` method on the `Agency` object.

Download the vSphere ESX Agent Manager SDK.

Typically, before deleting an ESX agency, a solution firsts call the `EsxAgentManager.uninstall()` method to put the agency in the uninstalled state. The solution tracks the progress of `EsxAgentManager.uninstall()` and only calls `destroyAgency()` to remove the ESX agency when the status of the ESX agency is green.

If your solution does not need to track the removal of the ESX agency and its ESX agents, you can call `destroyAgency()` without first calling `uninstall()`. ESX Agent Manager removes the ESX agency and all of the ESX agents without tracking the status of the uninstallation process.

Alternatively, disconnecting the solution from vCenter by calling the `ExtensionManager.unregisterExtension()` method removes all ESX agencies and ESX agents.

1. Call the `Agency.disable()` method to disable the ESX agency.

   Calling the `Agency.disable()` method powers off the ESX agent virtual machines, but does not undeploy them.

   Your solution can call the `Agency.disable()` method in the `MyAgentHandler.java` class.

   ```
   public void disable() throws RuntimeFaultFaultMsg {
      waitForSetup();
         _myeamConnection.getStub().disable(_myagency);
   }
   ```

2. Call the `Agency.uninstall()` method to put the ESX agency in the uninstalled state.

   Calling the `Agency.uninstall()` method uninstalls all the ESX agents in the ESX agency.

   Your solution can call the `Agency.uninstall()` method in the `MyAgentHandler.java` class.

   ```
   public void uninstall() throws RuntimeFaultFaultMsg {
      waitForSetup();
         _myeamConnection.getStub().uninstall(_myagency);
   }
   ```

3. Delete the ESX agency and ESX agents.

Option	Description
Call the `Agency.destroyAgency()` method on the `Agency` object.	Deletes the agency and its ESX agents, but the solution remains registered with `ExtensionManager`.
Call the `ExtensionManager.unregisterExtension()` method on the `Extension` object.	Unregisters the solution from vCenter, which uninstalls the solution and deletes the ESX agency and its ESX agents.

   Your solution can delete the ESX agency by calling the `ExtensionManager.unregisterExtension()` method in the `MyManager.java` class.

   ```
   public void cleanup() throws NotFoundFaultMsg, RuntimeFaultFaultMsg {
      if (_myeamConnection != null) {
         _myeamConnection.disconnect();
      }

       _myvimConnection.getStub().unregisterExtension(_myvimConnection.getExtensionManager(),
                                                      EXTENSION_KEY);
   }
   ```

You defined a function to delete an ESX agency from a solution.

Resolve ESX agent issues

ESX Agent Manager can detect issues in the ESX agents that solutions deploy. Solutions can try to resolve issues when the status of an ESX agency or an ESX agent is set to red.

- Download the vSphere ESX Agent Manager SDK.
- Verify that you have set up and started your solution in an application server.

You obtain the issues that affect an ESX agency or agent by using the `EventManager` to listen for issues.

A solution can try to resolve issues by calling the `EamObject.resolve(Issue[])` method on an individual ESX agent, in which case ESX Agent Manager tries to resolve the issues. The solution can also call `EamObject.resolveAll()` on an ESX agency, in which case ESX Agent Manager attempts to resolve all the issues on all the ESX agents that the ESX agency deploys.

1. Obtain the unique identifiers of issues by calling the `Issue.getKey()` method.
   Your solution can define a method that calls the `Issue.getKey()` method in the `MyAgentHandler.java` class.

   ```
   public String getIssueId(Issue issue) {
      return Integer.toString(issue.getKey());
   ```

```
}
```

2. Call the `Agent.getRuntime()` method to obtain an `EamObjectRuntimeInfo` object for a running ESX agent.
   Your solution can implement `Agency.getRuntime()` in a method that checks for a running ESX agency and obtains the runtime information for that ESX agency.

```
public Issue getIssue(String issueId) throws RuntimeFaultFaultMsg {
   waitForSetup();
   EamObjectRuntimeInfo runtime = getRuntime();
   assert runtime != null;
   [...]
}
```

3. Call the `EamObject.queryIssue()` method to obtain the list of issues affecting an ESX agency from the runtime of that ESX agency.
   Your solution can add any issues that it discovers for the ESX agency to a `List` object, and returns the issues with their issue identifiers.

```
public Issue getIssue(String issueId) throws RuntimeFaultFaultMsg {
   waitForSetup();
   EamObjectRuntimeInfo runtime = getRuntime();
   assert runtime != null;
   List<Issue> issues = _myeamConnection.getStub().queryIssue(_myagency, null);
   if (issues == null) {
      return null;
   }
   for (Issue issue : issues) {
      if (getIssueId(issue).equals(issueId)) {
         return issue;
      }
   }
   return null;
}
```

4. Call the `EamObject.resolveAll()` method to resolve all issues with ESX agents running in an ESX agency.
   Your solution can call the `resolveAll()` method on the `Agency` object that the `MyAgentHandler.java` class creates.

```
public void resolveAll() throws RuntimeFaultFaultMsg {
   waitForSetup();
   _myeamConnection.getStub().resolveAll(_myagency);
}
```

5. Call the `EamObject.resolve()` method to resolve a specific issue with an ESX agent.
   Your solution can call the `resolve()` method on the issue identifier that the `getIssue()` method returns, and generates a list of unknown issues that the solution cannot resolve.

```
public void resolve(String issueId) throws NumberFormatException,
   RuntimeFaultFaultMsg {
   waitForSetup();
   List<Integer> unknownIssueIds = _myeamConnection.getStub()
                                                .resolve(_myagency,

 Collections.singletonList(Integer.parseInt(issueId)));
   if (unknownIssueIds != null) {
      _log.error("Failed to resolve issue:" + issueId);
   }
}
```

You called the methods from the ESX Agent Manager API to obtain issues and attempt to resolve them.

How do I integrate an extension with the vCenter Extension vService

The vCenter Extension vService simplifies the installation and deployment of extensions. By integrating an extension with the vCenter Extension vService you can deploy extensions from the vSphere Client without having to enter the connection parameters of the vCenter on which you install the extension, or provide the login credentials for that vCenter instance.

The vCenter Extension vService performs the following functions for solutions that you integrate with it.

- Provides the extension with the connection parameters of the vCenter instance.
- Registers the extension certificate with vCenter.

For example, to deploy your solution you can provide the IP address and login credentials in the `mysolution.properties` file, which requires you to update the solution for every vCenter instance on which you install it. By integrating an extension with the vCenter Extension vService, you avoid this manual step.

Integrate a virtual machine with the vCenter Extension vService

To integrate a virtual machine with the vCenter Extension vService, you must deliver the extension in an Open Virtualization Format (OVF) package, add sections to the OVF descriptor file, and provide in the guest operating system a script that connects to the vCenter Extension vService.

Configure the OVF descriptor file
To integrate an extension with the vCenter Extension vService, you must add a section to the Open Virtualization Format (OVF) descriptor file that contains the virtual machine.

You have developed an extension running in a virtual machine that you deliver using OVF.

Including a `<vServiceDependencySection>` element in the OVF descriptor file of the virtual machine informs vCenter that this virtual machine depends on the vCenter Extension vService.

When you deploy a virtual machine by using OVF, vCenter generates an OVF environment XML document for the virtual machine. The OVF environment is a secure one-way communication channel between vCenter and the guest OS of the virtual machine. To use this feature you must enable OVF environment transport in the OVF descriptor of your virtual machine. The vCenter Extension vService makes the OVF environment available to the virtual machine in an ISO image that it locates in the first CD-ROM drive of the virtual machine.

The vCenter Extension vService uses the OVF environment to pass connection parameters to the guest operating system in the virtual machine. The vCenter Extension vService adds a `<vServiceEnvironmentSection>` element to the OVF environment of the virtual machine. The `<vServiceEnvironmentSection>` element contains the following connection parameters that a script running within the guest operating system requires to register with vCenter as an extension.

- Communication parameters that allow the virtual machine to make a secure connection to the vCenter Extension vService Guest API:
  - HTTPS URL to the vCenter Extension vService Guest API.
  - Authentication token that authenticates the virtual machine with the vCenter Extension vService.
  - SSL thumbprint of an X509 certificate that the vCenter Extension vService uses to establish a secure HTTPS connection with the virtual machine.
- Communication parameters that allow the guest operating system to make a secure connection to vCenter:
  - IP address of the vCenter instance
  - SSL thumbprint of the X509 certificate that vCenter uses to make a secure HTTPS connection to the virtual machine
  - HTTP port on which the vCenter is reachable
  - HTTPS port on which the vCenter is reachable
  - Managed object reference of the virtual machine in vCenter

    Important:
    The `<vServiceEnvironmentSection>` element contains sensitive data. Take special care within the guest operating system to prevent other users from accessing it. When the connection to vCenter is established, the guest operating system no longer needs the OVF environment so you should eject the CD-ROM. Ejecting the CD-ROM deletes the OVF environment from the datastore.

1. Add a `<vServiceDependencySection>` element to the OVF descriptor file for the virtual machine. You nest the `<vServiceDependencySection>` element in the `<VirtualSystem>` element.

```
<Envelope>
 [...]
 <VirtualSystem>
  [...]
  <vmw:vServiceDependencySection>
  </vmw:vServiceDependencySection>
  [...]
 </VirtualSystem>
 [...]
</Envelope>
```

2. Add the URLs to the standard OVF and VMware OVF schemas.

```
<vmw:vServiceDependencySection xmlns:ovf="http://schemas.dmtf.org/ovf/envelope/1"
                                    xmlns:vmw="http://www.vmware.com/schema/ovf">
</vmw:vServiceDependencySection>
```

3. Set the `ovf:required` attribute.

Option	Description
`ovf:required="true"`	Users can only deploy the OVF on vCenter 5.x. Users cannot power on the virtual machine if it is not bound to the vCenter Extension vService.
`ovf:required="false"`	Users can deploy the OVF vCenter 4.x and 5.x, but the virtual machine only integrates with the vCenter Extension vService in vCenter 5.x.

```
<vmw:vServiceDependencySection xmlns:ovf="http://schemas.dmtf.org/ovf/envelope/1"
                                    xmlns:vmw="http://www.vmware.com/schema/ovf"
                                    ovf:required="true"
                                    vmw:id="installation" >
</vmw:vServiceDependencySection>
```

4. Set the type of the dependency to `com.vmware.vservice.extension`, to bind the extension to the vCenter Extension vService.

```
<vmw:vServiceDependencySection xmlns:ovf="http://schemas.dmtf.org/ovf/envelope/1"
                                    xmlns:vmw="http://www.vmware.com/schema/ovf"
                                    ovf:required="true"
                                    vmw:id="installation" >
   <Info>A vService dependency</ovf:Info>
   <vmw:Type>com.vmware.vservice.extension</vmw:Type>
</vmw:vServiceDependencySection>
```

5. Provide a name and a description for the vService dependency, that appears in vService Manager.

```
<vmw:vServiceDependencySection xmlns:ovf="http://schemas.dmtf.org/ovf/envelope/1"
                                    xmlns:vmw="http://www.vmware.com/schema/ovf"
                                    ovf:required="true"
                                    vmw:id="installation" >
   <ovf:Info>A vService dependency</ovf:Info>
   <vmw:Type>com.vmware.vservice.extension</vmw:Type>
   <vmw:Name>dependency_name</vmw:Name>
   <vmw:Description>dependency_description</vmw:Description>
   <vmw:Configuration />
</vmw:vServiceDependencySection>
```

6. Enable OVF environment transport by setting the `ovf:transport` attribute of the `<VirtualHardwareSection>` element to `iso`.

```
<Envelope>
 [...]
```

```
 <VirtualSystem>
  [...]
  <VirtualHardwareSection ovf:transport="iso">
    [...]
  </VirtualHardwareSection>
  [...]
 </VirtualSystem>
 [...]
</Envelope>
```

You configured the OVF file for a extension to bind the extension to the vCenter Extension vService and to make the connection information for the vCenter instance available over OVF transport.

vService dependency section in the OVF descriptor file

The following code extract shows an example of a `<vServiceDependencySection>` element in an OVF descriptor file.

```
<vmw:vServiceDependencySection xmlns:ovf="http://schemas.dmtf.org/ovf/envelope/1"
                                       xmlns:vmw="http://www.vmware.com/schema/ovf"
                                       ovf:required="true"
                                       vmw:id="installation" >
   <Info>A vService dependency</Info>
   <vmw:Type>com.vmware.vservice.extension</vmw:Type>
   <vmw:Name>vCenter Extension Installation</vmw:Name>
   <vmw:Description>
        This appliance requires a binding to the vCenter Extension vService,
        which allows it to register automatically as a vCenter Extension at runtime.
   </vmw:Description>
   <vmw:Configuration />
</vmw:vServiceDependencySection>
```

Provide a script in the Extension virtual machine to register as a vCenter Extension

You must write a script that you run in the virtual machine that contains the extension.

- You have developed an extension running in a virtual machine that you deliver using OVF.
- You have configured the OVF descriptor file of the extension to depend on the vCenter Extension vService and enabled OVF transport.

The script reads the URL of the vCenter Extension vService Guest API, SSL thumbprint, and authentication token in the `<vServiceEnvironmentSection>` element of the Open Virtualization Format (OVF) environment XML file. The script authenticates itself with the vCenter Extension vService using the authentication token from the vService environment, generates a self-signed certificate, and sends the extension key and certificate to the `RegisterExtension` function from the vCenter Extension vService Guest API.

The script that you run in the guest operating system of the virtual machine must perform the following functions.

1. Parse the `<vServiceEnvironmentSection>` section in the OVF environment file.
2. Create a self-signed X509 certificate.
   For example, create the certificate by using OpenSSL.
3. Send an `HTTP POST` request to the vCenter Extension vService.
   The script obtains the URL to the vCenter Extension vService from the `<vServiceEnvironmentSection>`.
4. Include in the `HTTP` header the authentication token value from the `<vServiceEnvironmentSection>` element of the OVF environment file.
   `"evs-token: `*`Token value`*`"`

5. Verify that the SSL thumbprint of the server-side certificate of the HTTPS connection is the same as that of the `X509Thumbprint` value from the `<vServiceEnvironmentSection>` element of the OVF environment file.
6. In the body of the `HTTP POST` request, pass the extension key of the extension and the PEM encoding of the certificate to the `RegisterExtension` function from the vCenter Extension vService Guest API.

```
<RegisterExtension xmlns="http://www.vmware.com/schema/vservice/ExtensionVService">
   <Key>com.mycompany.extensionkey</Key>
   <Certificate>
   -----BEGIN CERTIFICATE-----
   MIICRTCCAa4CCQDC/hX5KA9rSzANBgkqhkiG9w0BAQUFADBnMQswCQYDVQQGEwIu
   LjEKMAgGA1UECBMBLjEKMAgGA1UEBxMBLjEPMA0GA1UEChMGVk13YXJlMQowCAYD
   VQQLEwEuMREwDwYDVQQDEwhFVlMgZGVtbzEQMA4GCSqGSIb3DQEJARYBLjAeFw0x
   MDA3MDYwNzUwNTVaFw0xMDA4MDUwNzUwNTVaMGcxCzAJBgNVBAYTAi4uMQowCAYD
   VQQIEwEuMQowCAYDVQQHEwEuMQ8wDQYDVQQKEwZWTXdhcmUxCjAIBgNVBAsTAS4x
   ETAPBgNVBAMTCEVWUyBkZW1vMRAwDgYJKoZIhvcNAQkBFgEuMIGfMA0GCSqGSIb3
   DQEBAQUAA4GNADCBiQKBgQC4/XVcMhvNixk35iWl3nn1KHVSgUE18TuQBj7spNUc
   y506RmV8BR847jg9fHl7aErShOQ8RT/EuEEUGey4U1dB2pSocoYldtp2r4g/Lcew
   ZuuyQh2+MC0YzeFe+nyxBDHa0BGUId0dQH9nrjyboW/kNIrWfDkXnxxtq6pQAmFw
   +QIDAQABMA0GCSqGSIb3DQEBBQUAA4GBAElC68z59fuicYUa4fGWBuXNxzb+uqWF
   +cnf78lctBY1pr1DcEedhyww2SYbaGh/xGCc1zqO5kqYhIexQbN/2Vxaol9lJc/n
   vRfQRCp+HaIFTJMu4mVZ2GsYSp/tZSGgiBBQAUXqCLxFQr0eQ29b9rj4Q3/1N+7i
   hbVOln67TOBZ
   -----END CERTIFICATE-----
   </Certificate>
</RegisterExtension>
```

The vCenter Extension vService calls the `RegisterExtension` function and registers the extension with vCenter.

`vServiceEnvironmentSection` in the OVF environment XML file

If you add a vService dependency in an OVF descriptor from which you deploy an extension, the vCenter Extension vService generates and adds a `<vServiceEnvironmentSection>` element to the OVF environment of the virtual machine that it deploys.

```
<ve:vServiceEnvironmentSection xmlns:ve="http://www.vmware.com/schema/ovfenv"
                                      xmlns:evs="http://www.vmware.com/schema/vservice/
ExtensionVService"
                                      ve:bound="true" ve:id="installation"
                                      ve:type="com.vmware.vservice.extension">
   <evs:GuestApi>
         <evs:URL>https://192.168.1.42/vsm/extensionService</evs:URL>
         <evs:Token>1efc34a14232f81a245b9e8172f7a383fdeab312</evs:Token>
         <evs:X509Thumbprint>2b:04:e6:7d:8c:7b:73:70:d4:29:32:ed:96:11:2b:ae:b4:a0:28:78</
evs:X509Thumbprint>
   </evs:GuestApi>
   <evs:VCenterApi>
         <evs:IP>192.168.1.42</evs:IP>
         <evs:X509Thumbprint>2b:04:e6:7d:8c:7b:73:70:d4:29:32:ed:96:11:2b:ae:b4:a0:28:78</
evs:X509Thumbprint>
         <evs:HttpPort>80</evs:HttpPort>
         <evs:HttpsPort>443</evs:HttpsPort>
         <evs:SelfMoRef>VirtualMachine:vm-246</evs:SelfMoRef>
   </evs:VCenterApi>
</ve:vServiceEnvironmentSection>
```

Deploy an extension in the vSphere Client by using the vCenter Extension vService

If you configure the virtual machine that runs an extension to use the vCenter Extension vService, you can deploy the extension directly in the vSphere Client.

- You have developed an extension running in a virtual machine that you deliver using OVF.
- You have configured the OVF descriptor file of the extension to depend on the vCenter Extension vService and enabled OVF transport.
- You have provided a script in the virtual machine that runs the extension that reads the OVF environment file to obtain the connection parameters from the vCenter Extension vService and uses them to register as an extension with vCenter.

1. Log in to the vSphere Client.
2. Select **Menu** > **Hosts and Clusters**.
3. Select the host.
4. From the **Actions** pull-down menu, select **Deploy OVF Template**.
5. Type the URL to the OVF file from which to deploy the extension.
6. Select a host, datastore, disk format, and network on which to run the extension.

You deployed an extension from OVF by using the vCenter Extension vService.

vCenter Extension vService XML schema

The vCenter Extension vService XML Schema defines the `<vServiceEnvironmentSection>` in the OVF environment of the virtual machine that contains the extension. It also defines the `RegisterExtension` function that registers the extension with vCenter.

```
<?xml version="1.0"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema"
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns:evs="http://www.vmware.com/schema/vservice/ExtensionVService"
        targetNamespace="http://www.vmware.com/schema/vservice/ExtensionVService"
        elementFormDefault="qualified" attributeFormDefault="qualified">

   <!-- RegisterExtension_Type:
        The command type used in the guest API when registering as a
        vCenter extension.
   -->
   <complexType name="RegisterExtension_Type">
      <sequence>
         <!-- The key of the extension to register -->
         <element name="Key" type="string" />
         <!-- The PEM-encoded certificate of the extension to register -->
         <element name="Certificate" type="string" />
         <any processContents="lax" minOccurs="0" maxOccurs="unbounded" />
      </sequence>
      <anyAttribute namespace="##any" processContents="lax"/>
   </complexType>

   <element name="RegisterExtension" type="evs:RegisterExtension_Type" />

   <!-- GuestApi: Contains information for the guest software about how to connect to the
        Extension vService.
   -->
   <element name="GuestApi">
     <complexType>
      <sequence>
         <!-- The URL of the Extension vService guest API -->
         <element name="URL" type="anyURI" />
         <!-- Authentication token for the Extension vService guest API -->
         <element name="Token" type="string" />
```

```
        <!-- The X509 thumbprint of the Extension vService server
          certificate.
        -->
        <element name="X509Thumbprint" type="string" />
        <any processContents="lax" minOccurs="0" maxOccurs="unbounded" />
      </sequence>
      <anyAttribute namespace="##any" processContents="lax"/>
    </complexType>
  </element>

  <!-- VCenterApi:
       Contains information for the guest software about how to connect
       to vCenter.
  -->

  <element name="VCenterApi">
    <complexType>
     <sequence>
        <!-- The IP address of the vCenter -->
        <element name="IP" type="string" />
        <!-- The X509 thumbprint of the vCenter certificate -->
        <element name="X509Thumbprint" type="string" />
        <!-- The HTTP port of the vCenter -->
        <element name="HttpPort" type="int" />
        <!-- The HTTPS port of the vCenter -->
        <element name="HttpsPort" type="int" />
        <!--
            The managed object reference of this virtual machine
            in the vCenter
        -->
        <element name="SelfMoRef" type="string" />
        <any processContents="lax" minOccurs="0" maxOccurs="unbounded" />
      </sequence>
      <anyAttribute namespace="##any" processContents="lax"/>
    </complexType>
  </element>
</schema>
```

VMware Storage Policies

Storage Policy Based Management (SPBM) helps administrators automate VM provisioning on appropriate storage devices with requested data services. This avoids laborious manual provisioning of storage for individual VMs. For object-based datastores like vSAN and VVol, vendors formulate metadata describing storage traits, and advertise storage capabilities for profile queries.

Storage Capabilities

The storage provider describes capabilities of a storage array, which the storage policy service obtains and presents to the administrator to assist with VM provisioning.

On object-oriented datastores such as vSAN and VVol datastores, storage capabilities originate from software written by the vendor, called a VASA provider. VASA is an abbreviation of vSphere API for storage awareness. The storage policy service can match storage capabilities with VM storage policies formulated in vSphere.

For VVol datastores: storage capabilities may include array type, vendor, RAID level, read latency, write latency, snapshot information, backup frequency, replication, caching, compression, deduplication, and high availability.

For vSAN datastores: storage capabilities may include RAID type, failures to tolerate, disk stripes per object, flash read cache, IOPS limit, encryption, compression, and deduplication.

For VMFS and NFS datastores: administrators can select the VMware encryption storage policy. VM encryption is implemented on the ESX host rather than on the storage array.

For tag-based policies: administrators can use the vSphere Web Client to define storage policy tags.

Virtual Machine Storage

Virtual machine configuration and virtual disk data reside in datastores.

Virtual machine configuration is stored in a file with the `.vmx` file extension. VM home files also include other files for virtual machine operation, such as a log file (`.log`), virtual firmware (`.nvram`), paging file (`.vmem`), and snapshot data files (`.vmsd`).

Virtual machine data is stored in virtual disks, in files with the `.vmdk` file extension.

Storage policies allow you to distinguish between virtual machine configuration and data files and to specify storage locations based on the needs of each.

Storage Capability Profiles

To use a storage capability profile, you define storage requirements and associate a virtual machine with a storage policy. When you create a VM, vCenter correlates its VM storage policy with the storage policy service to determine an appropriate location for VM home and virtual disk files.

A storage policy consists of a set of subprofiles. Each subprofile defines a set of storage capabilities, and corresponds to a numbered Rule Set in the vSphere Web Client. Several storage policies pre-exist in vSphere, including:

- The VM Encryption Policy is a sample storage policy for VM Home and virtual disk encryption.
- The vSAN Default Storage Policy can be easily modified, but is the storage policy used by default for vSAN datastores.
- The VVol No Requirements Policy allows the storage array, through the VASA provider, to determine the best placement strategy for VM Home and virtual disks.

In the vSphere Web Client, all of these storage policies, and ones you create, have a **Check Compliance** button to verify that virtual machines assigned to that storage policy are correctly placed on storage devices.

Storage Policy Operations

The Storage Policy API is implemented with Web Services Description Language (WSDL), as an XML file describing how to communicate with it. The WSDL is usable with toolkit bindings by many programming languages, including Java.

SPBM Managed Objects

The Storage Policy API is structured like the vSphere API, and is likewise suitable for Java programming. Managed object types include the following:

- `PbmProfileProfileManager` supports operations on virtual machine storage profiles.
- `PbmPlacementSolver` identifies locations that support requested capabilities for storing virtual machine files.
- `PbmComplianceManager` verifies compliance of virtual machine and virtual disk requirement profiles.
- `PbmReplicationManager` deals with replication of virtual machine and virtual disk requirement profiles.
- `PbmServiceInstance` is the root object of Storage Policy service, created connecting to SPBM.

Storage Policy API and vSphere API

The following table shows the correspondence between SPBM managed objects and vSphere API methods.

Table 1922: Storage Policy Operations and Virtual Machine Provisioning

SPBM Operation (Storage Policy API)	Virtual Machine Provisioning (vSphere API)
Use the `PbmProfileProfileManager` methods to create and update storage profiles.	Associate storage profiles with virtual machines and virtual disks. See the description of the vSphere API data object properties `VirtualMachineConfigSpec.vmProfile` and `FileBackedVirtualDiskSpec.profile` in the vSphere API Reference. You can also use the vSphere Web Client to associate a storage profile with a virtual machine or virtual disk.
Use the `PbmPlacementSolver` methods to identify candidate datastores for storage locations.	Specify the datastores when you create virtual machines and virtual disks. See the description of the vSphere API data object properties `VirtualMachineFileInfo.vmPathName` and `VirtualDeviceFileBackingInfo.datastore` in the vSphere API Reference.
Use the `PbmComplianceManager` methods to check compliance between storage requirements and capabilities.	After you associate a storage profile with a virtual machine or virtual disk, the Server will identify non-compliance if the datastore does not satisfy the requirements of the profile.

Access to the Storage Policy Server

You can access the VMware Storage Policy Server by connecting to a vCenter system and obtaining a vCenter session cookie.

The Storage Policy client API is described in the pbmService.wsdl file that is included in the VMware Storage Policy SDK. The API defines a set of request operations that you use to manipulate storage profiles. The VMware Storage Policy SDK includes Java bindings for the SPBM service WSDL.

To gain access to the Storage Policy Server, your client connects to a vCenter system and obtains the vCenter session cookie. Then you can use the vCenter session cookie to establish the connection with the Storage Policy Server. See Establish a Connection with the VMware Storage Policy Server.

After you establish a Storage Policy Server connection, your client uses language-specific Web Services access objects and the `PbmServiceInstance` and `PbmServiceInstanceContent` objects to access the Storage Policy managed objects and their methods.

The Storage Policy Web Services access objects are language-specific API binding objects that are generated from the Storage Policy WSDL. The VMware Storage Policy SDK contains JAXWS bindings to the Storage Policy API. The JAXWS bindings include the `PbmService` and `PbmPortType` Web Services access objects.

- `PbmService` – Provides access to the `PbmPortType` object and support for the Storage Policy Service connection.
- `PbmPortType` – Provides access to Storage Policy methods.

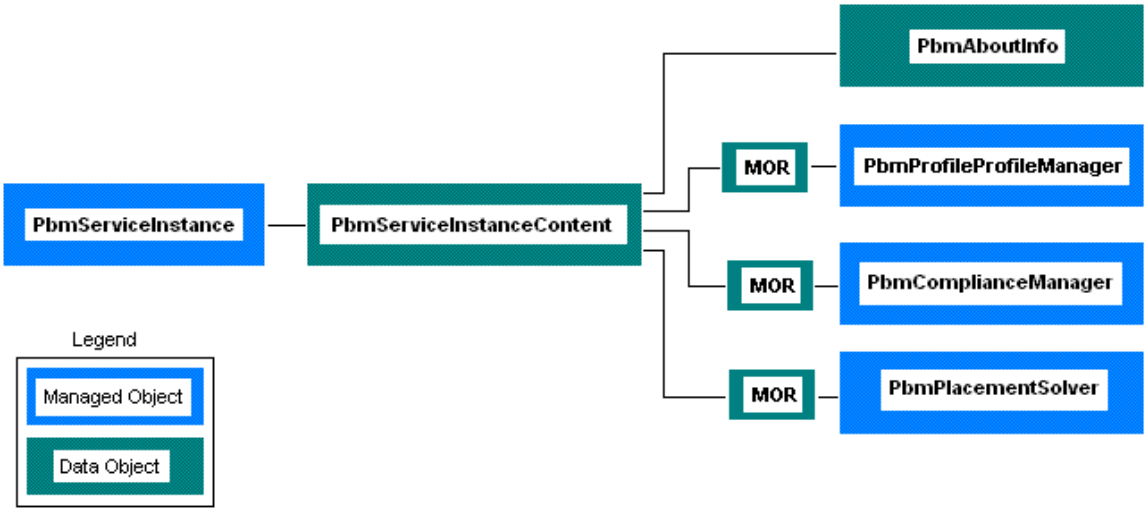
The following code fragment shows the sequence of calls that you use to obtain access to the Storage Policy API methods.

Access to Storage Policy API Methods

```
import com.vmware.pbm.PbmService;
import com.vmware.pbm.PbmPortType;
import com.vmware.pbm.PbmServiceInstanceContent;
[...]
PbmService = new PbmService()
PbmPortType pbmPort = PbmService.getPbmPort()
PbmServiceInstanceContent pbmServiceContent = pbmPort.pbmRetrieveServiceContent
```

The following figure shows the resulting `PbmServiceInstanceContent` data object and the Storage Policy managed objects that provide access to Storage Policy services.?

Figure 646: Storage Policy Service Instance Content



The `PbmServiceInstanceContent` object contains managed object references to the Storage Policy services. The set of Storage Policy services include the profile manager, placement solver, and compliance manager.

Service	Managed Object	Description
Profile Manager	`PbmProfileProfileManager`	Create and update VMware storage profiles to define storage requirements.
Placement Solver	`PbmPlacementSolver`	Identify candidate datastores for storage locations.
Compliance Manager	`PbmComplianceManager`	Check compliance between storage requirements and capabilities.

Storage Profile Queries

The Storage Policy API includes several methods that you can use to query for profiles and vSphere entities, such as datastores, virtual machines, and virtual disks.

The following table provides an overview of these methods. For more information, see the *Storage Policy API Reference*.

Table 1923: Storage Profile API Query Methods

Method	Description
`PbmQueryAssociatedEntities`	Returns the virtual machine and disks that are associated with the given storage policies. With empty parameter, returns all virtual machine and disks that are associated with a storage policy.
`PbmQueryAssociatedEntity`	Returns references to entities associated with the specified profile.
`PbmQueryAssociatedProfile`	Returns profiles associated with the specified entity. The type of profile is determined by the type of entity that you specify. • If you specify a datastore, the method returns one or more capability (resource) profiles. • If you specify a virtual machine or virtual disk, the method returns one or more requirement profiles.

Method	Description
PbmQueryAssociatedProfiles	Returns PbmQueryProfileResult objects. Each result objet identifies an entity and one or more profiles. Profile type is determined by entity type. • If the entity is a datastore, the result object contains one or more capability (resource) profiles. • If the entity is a virtual machine or virtual disk, the result object contains one or more requirement profiles.
PbmQueryByRollupComplianceStatus	Returns all virtual machines for the given rollup compliance status.
PbmQueryDefaultRequirementProfile	Returns the default requirement profile ID for the given datastore. For legacy hubs, returns null.
PbmQueryDefaultRequirementProfiles	Returns the default profiles for the given datastores. For legacy datastores, the defaultProfile is set to null.
PbmQueryMatchingHub (deprecated)	Finds matching placement hubs for the specified requirements profile. Returns only hubs that match the profile. If this method is called for VVolDefaultProfile, then all VVol containers are returned as matching.
PbmQueryMatchingHubWithSpec (deprecated)	Finds matching placement hubs based on a profile creation specification. This method returns only those hubs that match the specification.
PbmQueryProfile	Returns requirement profiles or resource profiles, or both.
PbmQueryReplicationGroups	Returns identifiers for replication groups associated with virtual machines, virtual disks, or virtual machines and all their disks. If the query is performed for a virtual machine and all its disks are virtualMachineAndDisks, an entry per disk and one for the virtual machine configuration are returned.
PbmQuerySpaceStatsForStorageContainer	Retrieves space statistics of a datastore.

VMware Storage Policy SDK

The VMware Storage Policy SDK is distributed as part of the VMware vSphere Management SDK.

When you extract the contents of the distribution, the VMware Storage Policy SDK is located in the spbm sub-directory.

```
VMware-vSphere-SDK-build-num
     eam
     sms-sdk
     spbm
          docs
              java
                  JAXWS/samples/javadoc/index.html
              ReferenceGuide
                  index.html
          java
              JAXWS
                  lib
                  samples
                  build and run scripts
          wsdl
              pbmService.wsdl
              pbm.wsdl
     ssoclient
```

```
vsphere-ws
```

The following table shows the locations of the contents of the VMware Storage Policy SDK.

Table 1924: VMware Storage Policy SDK Contents

VMware Storage Policy SDK Component	Location
JAX-WS VMware Storage Policy client binding	`spbm/java/JAXWS/lib`
Java Storage Policy samples	`spbm/java/JAXWS/samples/com/vmware/spbm/samples/`
Java Storage Policy Server connection sample	`spbm/java/JAXWS/samples/com/vmware/spbm/connection/`
VMware Storage Policy API Reference	`spbm/docs/ReferenceGuide/index.html`
Documentation for example code	`spbm/docs/java/JAXWS/samples/javadoc/index.html`
WSDL files	`spbm/wsdl`

VMware Storage Policy SDK Examples

The VMware Storage Policy SDK contains Java examples that show how to create and use VMware storage policies.

This manual describes examples from the VMware Storage Policy SDK. It also describes examples from the vCenter Single Sign-On SDK that support the client connection to the Storage Policy Server. This manual includes the following single sign-on examples:

- vCenter Single Sign-On Client Example. This example shows how to obtain a holder-of-key token from the ESXvCenter Single Sign-On Server.
- vCenter LoginByToken Example. This example shows how to use the token to login to vCenter.

The following table lists the sample files in the VMware Storage Policy SDK.

Table 1925: VMware Storage Profile SDK Sample File

Location	Examples	Description
`SDK/spbm/java/JAXWS/samples/com/vmware/spbm/samples/`		
	AboutInfo.java	Obtains identifying data about the Storage Policy Server.
	CheckCompliance.java	Checks the compliance of profiles associated with virtual machines and virtual disks.
	CreateProfile.java	Creates a requirement profile.
	CreateVSANProfile.java	Creates a new storage profile with one rule-set based on vSAN capabilities.
	DeleteProfile.java	Deletes a requirement profile.
	EditProfile.java	Adds or deletes subprofiles from a tag-based storage profile.
	FcdAssociateProfile.java	Attaches a first class disk (FCD) to a virtual machine and associates the given storage profile with the FCD.
	ListProfiles.java	Retrieves all of the storage profiles known to the system.

Location	Examples	Description
	ViewProfile.java	Prints the contents of a tag-based storage profile.
	VMClone.java	Deploys multiple instances of a virtual machine template to a datacenter. The clone specification has an associated storage profile.
	VMCreate.java	Creates a virtual machine. The virtual machine configuration specification has an associated storage profile.
	VMRelocate.java	Used to relocate a virtual machine's virtual disks to a datastore compliant with the given storage profile.
`SDK/spbm/java/JAXWS/samples/com/vmware/spbm/connection/`		
	BasicConnection.java	Establishes an authenticated session with a VMware SSO Server, vCenter, and Storage Policy Server.
	ConnectedServiceBase.java	Connection base class for client application implementations.
	Connection.java	Storage Policy sample support; utility class that sets up a Storage Policy Server connection.
	ConnectionException.java	Base exception class for exceptions thrown by connection classes.
	ConnectionMalformedUrlException.java	URL exception.
	KeepAlive.java	Keep-alive utility class; maintains the vCenter connection.
	VcSessionHandler.java	Utility class; inserts vCenter session cookie into SOAP header.

Storage Policy Server Connection

The connection between a Storage Policy client and the Storage Policy Server is based on the client's connection with a vCenter system.

About Storage Policy Server Sessions

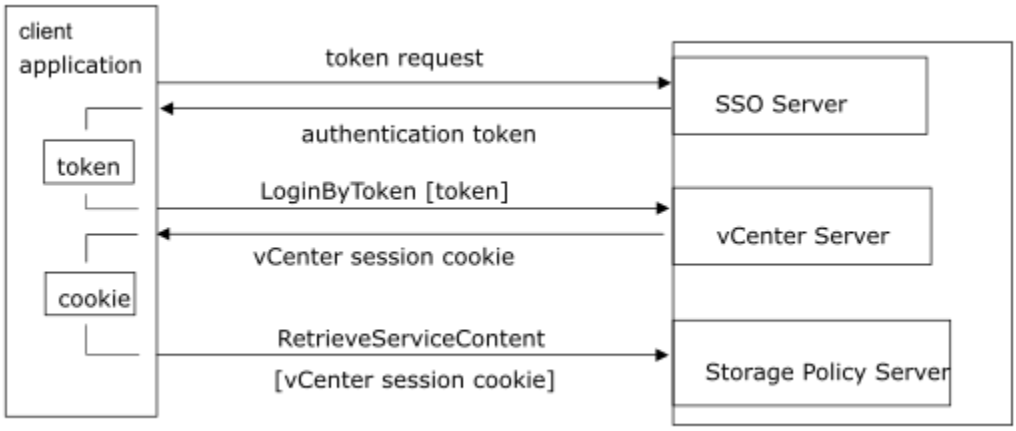
A vCenter client uses an HTTP session cookie to maintain a persistent connection with the Server. A Storage Policy client uses the vCenter session cookie to establish the connection with the Storage Policy Server.

A client performs the following operations to establish vCenter and Storage Policy Server sessions.

- Obtain a SAML token from the VMware SSO Server. ?See vCenter Single Sign-On Client Example.
- Use the SAML token to login to the vCenter.?See vCenter LoginByToken Example.
- Use the `RetrieveServiceContent` method to send the session cookie to the Storage Policy Server and establish the connection with the Server.

The following figure shows a representation of the server connections and operations involved in establishing a Storage Policy Server connection.

Figure 647: Storage Policy Server Connection



Establish a Connection with the VMware Storage Policy Server

Use the session cookie from the vCenter session to establish the Storage Policy session. The session cookie represents the authenticated vCenter session, which is based on the SSO token.

The following code fragments establish connections both with the vCenter and the Storage Policy Server. These examples are based on the `BasicConnection` sample, which is located in the Storage Policy SDK connection sample directory.

`SDK/spbm/java/JAXWS/samples/com/vmware/spbm/connection/BasicConnection.java`

The `BasicConnection` sample uses an instance of the `LoginByTokenSample` class. See vCenter LoginByToken Example. The `LoginByToken` example saves the HTTP cookie produced during the intial connection sequence and then restores the cookie after the vCenter connection has been established. Although the `LoginByToken` example creates a vCenter connection, the `BasicConnection` sample establishes its own connection with the vCenter. A different implementation might integrate those capabilities to reduce the number of vCenter connections.

Server URLs for Basic Connection

The `BasicConnection` sample creates connections to three VMware Servers.

- SSO Server
- vCenter
- Storage Policy Server

In the example configuration, the SSO and Storage Policy Servers are located on the same system as the vCenter instance. In other configurations, the SSO Server might be located on a different server.

Table 1926: VMware Server URLs

VMware Server	URL
vCenter	https://*server-name*|*IPaddress*/sdk/vimService
SSO Server	https://*server-name*|*IPaddress*/sts/STSService
Storage Policy Server	https://*server-name*|*IPaddress*/pbm

Create the Storage Policy Server Connection

The following code fragment uses a vCenter session cookie to create a Storage Policy Server session.

1. Extract the actual cookie value from the `name=value` expression in the cookie string obtained from the vCenter session connection.
2. Create a `PbmService` object.
3. Set up a header handler to support adding the vCenter session cookie to the Storage Policy Server connection.
4. Retrieve the `PbmPort` object for access to the Storage Policy API methods.
5. Retrieve the request context and set the endpoint to the Storage Policy Server URL.
6. Call the `PbmRetrieveServiceContent` method to establish the HTTP connection to the Storage Policy Server.

Storage Policy Server Connection

```
// 1. Set the extracted cookie in the PbmPortType
//
// Need to extract only the cookie value
String[] tokens = cookieVal.split(";");
tokens = tokens[0].split("=");
String extractedCookie = tokens[1];

// 2. Create a PbmService object.
pbmService = new PbmService();

// 3. Setting the header resolver for adding the VC session cookie to the
// requests for authentication
HeaderHandlerResolver headerResolver = new HeaderHandlerResolver();
headerResolver.addHandler(new VcSessionHandler(extractedCookie));
pbmService.setHandlerResolver(headerResolver);

// 4. Retrieve the PbmPort object for access to the Storage Policy API
pbmPort = pbmService.getPbmPort();

// 5. Set the Storage Policy Server endpoint
Map<String, Object> pbmCtxt = ((BindingProvider) pbmPort).getRequestContext();
pbmCtxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
pbmCtxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, spbmurl.toString());

// 6. Retrieve the service content (creates the connection)
pbmServiceContent = pbmPort.pbmRetrieveServiceContent(getPbmServiceInstanceReference());
```

Establish the vCenter Session Connection for the Local Instance

The following code fragment sets up the HTTP connection with the vCenter instance.

1. Retrieve the `VimPort` interface. This provides access to the vSphere API methods.
2. Retrieve the request context and set the vCenter endpoint address in the request context.
3. Set the session cookie in the request context. The cookie (`cookieVal`) is obtained from the vCenter LoginByToken Example.
4. Call the `RetrieveServiceContent` method to establish the HTTP connection with the vCenter instance.

vCenter Connection

```
// 1. Retrieve the VimPort interface.
vimService = new VimService();
vimPort = vimService.getVimPort();

// 2. Retrieve the request context and set the vCenter endpoint.
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcurl.toString());
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);
```

```
// 3. Put the extracted vCenter session cookie into the VimPortType request header.
Map<String, List<String>> headers =
     (Map<String, List<String>>) ctxt.get(MessageContext.HTTP_REQUEST_HEADERS);
if (headers == null) {
     headers = new HashMap<String, List<String>>();
}
headers.put("Cookie", Arrays.asList(cookieVal));
ctxt.put(MessageContext.HTTP_REQUEST_HEADERS, headers);

// 4. Retrieve the vCenter service content. (Establishes the HTTP connection)
vimServiceContent = vimPort.retrieveServiceContent(this.getVimServiceInstanceReference());
```

vVol Based Storage Profiles

Virtual Volumes (vVols) help with granularity in shared storage by providing a way to designate a specific storage policy for each virtual machine or virtual disk. Vendors formulate metadata describing their storage arrays, advertise capability profiles in a VASA provider, so SPBM can query storage capabilities on request. VASA is an abbreviation of vSphere API for storage awareness.

vVol Storage Policy Architecture

Storage Policy Based Management (SPBM) helps vSphere and storage administrators automate the provisioning of virtual machines on shared vVol storage with desired data services.

Storage Management Service (SMS) and SPBM run as part of the Storage Policy Service (SPS), which is a daemon running in vCenter to handle storage subsystems. SPS communicates with vendor-provided VASA providers, which implement about a dozen SPBM related functions to advertise storage capabilities and manage storage requests.

On the storage array, a virtual machine comprises a Config vVol with descriptors, a Swap vVol for memory, and a Data vVol for each virtual disk. Data to and from a storage container flows through a protocol endpoint, which can support either SAN or NAS protocols. Snapshots, space efficiency, replication, and other storage features may be handled natively by the storage array.

Typically storage administration involves a set of rule sets where platinum or gold policy offers the best quality of service, silver policy in the middle, and bronze policy the fewest features. A vSphere administrator can configure these policies using the vSphere Client connected to vCenter. In vSphere there is a predefined No Requirements policy for vVols.

SPBM Calling Sequence for vVols

When managing vVol storage, SPBM uses a specific calling sequence.

After a VASA provider registers with vSphere, SPBM makes the following calls:

1. `queryCapabilityMetadata` to get the capability metadata exposed by the VASA provider, for example RAID level, compression, encryption, caching, or replication.
2. `queryResourceMetadata` to get a list of available storage resources, expressed as capability metadata.
3. `queryMatchingContainer` to get a list of compatible storage containers for a given policy. The response from the VASA provider refers to resources returned by the `queryResourceMetadata` call. There will also be provisioning requests before `queryComplianceResult` is called - not by SPBM, but from the ESX host.
4. `queryComplianceResult` to check compliance with storage policies.
5. For version 3.0 VASA providers, `queryPolicy` is called if required to retrieve the Replication Group ID of the storage device for a vVol. ESX hosts call it to query the policy of objects in the namespace so the policy can be applied to related objects.

SPBM Data Objects

SPBM data objects are numerous, and for replication can be complex, but the inheritance hierarchy is relatively flat. Some important objects are explained below

`CapabilityMetadata` represents the metadata for a single unique setting as defined by the VASA provider. A simple setting has one property, while a complex setting has more than one property.

`CapabilityMetadata` contains the `CapabilityId` (below), name and description of the capability, whether it is mandatory when creating a profile, whether the capability should affect placement and compliance, a key ID for the property, whether multiple constraints are allowed for a capability instance, and metadata for properties that comprise the capability. All contents but the first two and last are optional.

`CapabilityId` contains the unique identifier for a capability within its namespace, and the namespace to which the capability belongs.

`CapabilityInstance` contains the `CapabilityId` (above), an array of constraints on the properties that comprise the capability, and (as of VASA 3.0) the appropriate line of service for the capability.

`ConstraintInstance` contains the property constraints (in `PropertyInstance`) for a single occurrence of this capability. All properties must satisfy their respective constraints to be compliant. `PropertyInstance` includes the unique identifier for the property, and a value for its constraint.

The VASA provider is responsible for sending XML descriptions of capability metadata back to SMS. In the XML capability profiles, the <`subProfiles`> have a <`name`>, followed by a <`capability`> list, with each capability having an <`id`> for the <`namespace`> and a <`constraint`> for the <`propertyInstance`>.

`CapabilityMetadataPerCategory` contains an array of `CapabilityMetadata` (above) and the category to which the metadata belongs. The category is vendor-supplied; well-known terms are recommended.

`CapabilitySchema` describes a capability namespace, as reported by the VASA Provider in response to the `queryCapabilityMetadata` call. `CapabilitySchema` contains vendor information, namespace information, an array of `CapabilityMetadataPerCategory` (above), the line of service, and a schema ID string.

vSAN Based Storage Profiles

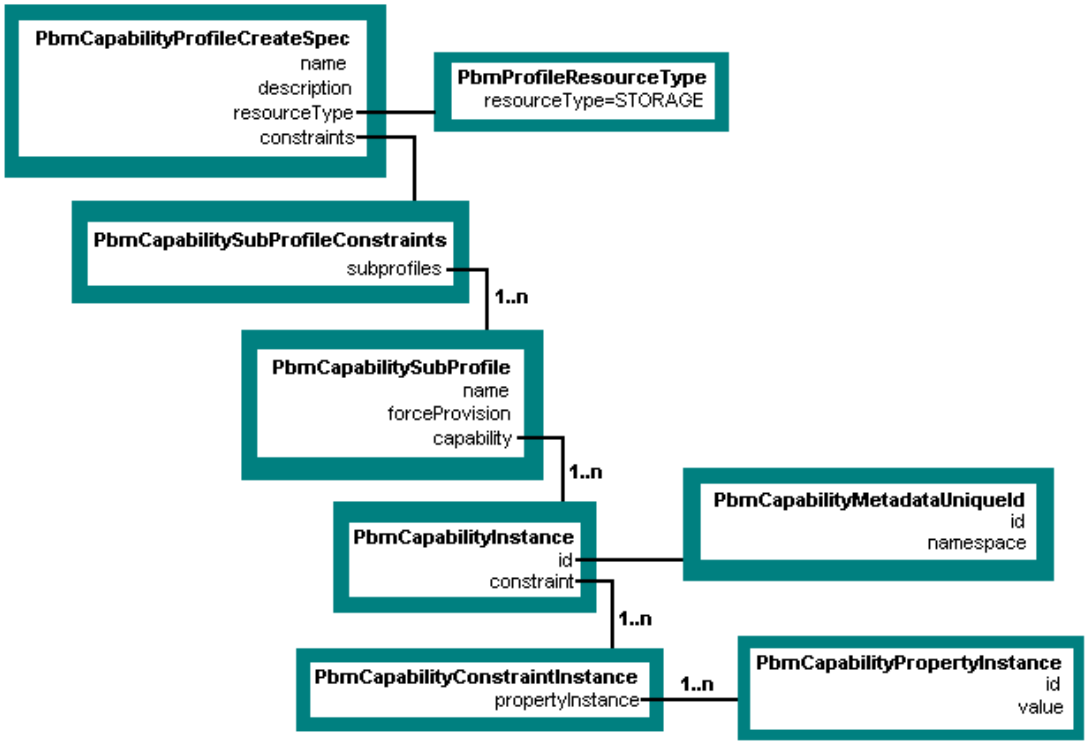
Storage requirements are based on storage capabilities available from storage provider, and vSphere offers vSAN storage capabilities. To create a requirements profile for vSAN capabilities, you retrieve metadata that describes vSAN capabilities and create a subprofile that expresses the storage requirements for virtual machine or virtual disk files. To perform these operations, you connect to the Storage Policy Server

Create a vSAN Requirements Profile

The following example demonstrates how to create a storage requirements profile based on vSphere vSAN storage capabilities. The example creates a requirement profile for vSAN stripe width.

The following figure shows the data objects used for a profile specification.

Figure 648: Storage Profile Specification



The following example is based on the Storage Policy SDK sample file `CreateVSANProfile.java`. This example is divided into two code fragments.

- Create an Individual Storage Requirement – The code fragment is a function that creates a single storage capability instance for a subprofile (rule).
- Create a Storage Profile – The code fragment builds a profile specification and creates the profile.

Create an Individual Storage Requirement

The following example builds a property instance for a capability. The property instance represents a single storage requirement.

The code performs the following steps.

1. Verifies that the capability exists.
2. Creates a property instance for the requirement (`PbmCapabilityPropertyInstance`).
3. Creates a capability constraint for the property instance (`PbmCapabilityConstraintInstance`).
4. Create a capability instance for the constraint and add the subprofile (rule) to the capability.

```
PbmCapabilityInstance buildCapability(String capabilityName, Object value,
         List<PbmCapabilityMetadataPerCategory> metadata)
         throws InvalidArgumentFaultMsg {

// Retrieve the metadata for the capability (stripeWidth)
PbmCapabilityMetadata capabilityMeta = PbmUtil.getCapabilityMeta(capabilityName, metadata);
if (capabilityMeta == null)
throw new InvalidArgumentFaultMsg("Specified Capability does not exist", null);

// Create a New Property Instance based on the Stripe Width Capability
PbmCapabilityPropertyInstance prop = new PbmCapabilityPropertyInstance();
prop.setId(capabilityName);
prop.setValue(value);
```

```
// Associate Property Instance with a Rule (subprofile)
PbmCapabilityConstraintInstance rule = new PbmCapabilityConstraintInstance();
rule.getPropertyInstance().add(prop);

// Associate Rule (subprofile) with a Capability Instance
PbmCapabilityInstance capability = new PbmCapabilityInstance();
capability.setId(capabilityMeta.getId());
capability.getConstraint().add(rule);

return capability;
}
```

Create a Storage Profile

The example performs the following operations.

1. Retrieve a reference to the Storage Policy Profile Manger.
2. Verify that there is vSAN Storage Policy support.
3. Retrieve the vSAN storage capability metadata.
4. Add capabilities to be used as requirements.
5. Add the requirement capabilities to a subprofile. A subprofile corresponds to a rule set in the vSphere Web Client.
6. Specify the subprofile as capability constraints.
7. Build a profile specification.
8. Create the storage profile.

When you create a storage profile, the `PbmCreate` method returns a profile ID (`PbmProfileId`). The Profile Manager maintains a list of profiles. To obtain a profile from the list, use the `PbmQueryProfile` and `PbmRetrieveContent` methods. See Retrieve an Existing Storage Profile from the Server.

vSAN Storage Profile Creation

```
// 1: Get PBM Profile Manager & Associated Capability Metadata
spbmsc = connection.getPbmServiceContent();
ManagedObjectReference profileMgr = spbmsc.getProfileManager();

// 2: Verify that there is vSAN Storage Policy support
Boolean vSanCapabale = false;
List<PbmCapabilityVendorResourceTypeInfo> vendorInfo =
     connection.getPbmPort().pbmFetchVendorInfo(profileMgr, null);
for (PbmCapabilityVendorResourceTypeInfo vendor : vendorInfo)
     for (PbmCapabilityVendorNamespaceInfo vnsi : vendor.getVendorNamespaceInfo())
          if (vnsi.getNamespaceInfo().getNamespace().equals("vSan")) {
              vSanCapabale = true;
              break;
          }

if (!vSanCapabale)
     throw new RuntimeFaultFaultMsg(?"Cannot create storage profile. vSAN Provider not found.",
 null);

// 3: Get PBM Supported Capability Metadata
List<PbmCapabilityMetadataPerCategory> metadata =
     connection.getPbmPort().pbmFetchCapabilityMetadata(profileMgr,
             PbmUtil.getStorageResourceType(), null);
// 4: Add Provider Specific Capabilities
List<PbmCapabilityInstance> capabilities = new ArrayList<PbmCapabilityInstance>();
capabilities.add(buildCapability("stripeWidth", stripeWidth, metadata));
```

```
// 5: Add Capabilities to a RuleSet (subprofile)
PbmCapabilitySubProfile ruleSet = new PbmCapabilitySubProfile();
ruleSet.getCapability().addAll(capabilities);

// 6: Add Rule-Set (subprofile) to Capability Constraints
PbmCapabilitySubProfileConstraints constraints = new PbmCapabilitySubProfileConstraints();
ruleSet.setName("Rule-Set " + (constraints.getSubProfiles().size() + 1));
constraints.getSubProfiles().add(ruleSet);

// 7: Build Capability-Based Profile
PbmCapabilityProfileCreateSpec spec = new PbmCapabilityProfileCreateSpec();
spec.setName(profileName);
spec.setDescription("Storage Profile Created by SDK Samples. Rule based on vSAN capability");
spec.setResourceType(PbmUtil.getStorageResourceType());
spec.setConstraints(constraints);

// 8: Create Storage Profile
PbmProfileId profile = connection.getPbmPort().pbmCreate(profileMgr, spec);
System.out.println("Profile " + profileName + " created with ID: " + profile.getUniqueId());
```

Virtual Machine Storage Profiles

The Storage Policy Server maintains a list of storage profiles. To apply them, you retrieve an existing storage profile from the storage policy server, then associate it with a virtual machine. Code fragments in this chapter are based on sample program `VMCreate.java` in the Storage Policy SDK.

Retrieve an Existing Storage Profile from the Server

You can use a script to retrieve an existing storage profile from the storage policy server.

The following code fragment shows the example function `getPbmProfileSpec` that uses the `PbmQueryProfile` and `PbmRetrieveContent` methods to retrieve storage profiles. In the context of the Storage Policy SDK example `VMCreate.java`, the function returns a `VirtualMachineDefinedProfileSpec` to be used to configure storage for a virtual machine.

The function performs the following operations:

1. Uses the connection to the Storage Policy Server to retrieve a reference to the Profile Manager.
2. Calls the `PbmQueryProfile` method to obtain the list of storage profile identifiers.
3. Calls the `PbmRetrieveContent` method to obtain the list of storage profiles.
4. Finds the profile that matches the specified profile name.
5. Creates a `VirtualMachineDefinedProfileSpec` and assigns the identifier from the named profile to the `VirtualMachineDefinedProfileSpec`. You use the `VirtualMachineDefinedProfileSpec` when you configure the virtual machine. See Apply the Storage Profile to a Virtual Machine.

Retrieving a Storage Profile

```
VirtualMachineDefinedProfileSpec getPbmProfileSpec(String name)
        throws InvalidArgumentFaultMsg, com.vmware.pbm.RuntimeFaultFaultMsg,
        RuntimeFaultFaultMsg {

// 1 Get PBM Profile Manager
        PbmServiceInstanceContent spbmsc = connection.getPbmServiceContent();
        ManagedObjectReference profileMgr = spbmsc.getProfileManager();

// 2 Retrieve the list of profile identifiers.
        List<PbmProfileId> profileIds =
```

```
            connection.getPbmPort().pbmQueryProfile(profileMgr,
                                                    PbmUtil.getStorageResourceType(),
                                                    null);

        if (profileIds == null || profileIds.isEmpty())
           throw new RuntimeFaultFaultMsg("No storage Profiles exist.", null);

// 3 Retrieve the list of storage profiles.
        List<PbmProfile> pbmProfiles =
            connection.getPbmPort().pbmRetrieveContent(profileMgr, profileIds);

// 4,5 Find the named profile and create a VirtualMachineDefinedProfileSpec
//    that will use the same profile identifier.
        for (PbmProfile pbmProfile : pbmProfiles) {
            if (pbmProfile.getName().equals(name)) {
                PbmCapabilityProfile profile = (PbmCapabilityProfile) pbmProfile;
                VirtualMachineDefinedProfileSpec spbmProfile =
                    new VirtualMachineDefinedProfileSpec();

                spbmProfile.setProfileId(profile.getProfileId().getUniqueId());

                return spbmProfile;
            }
        }

// Throw exception if none found
        throw new InvalidArgumentFaultMsg(
                "Specified storage profile name does not exist.", null);
    }
```

Apply the Storage Profile to a Virtual Machine

To use a storage profile for a virtual machine, specify a `VirtualMachineDefinedProfileSpec` object for the `VirtualMachineConfigSpec.vmProfile` property.

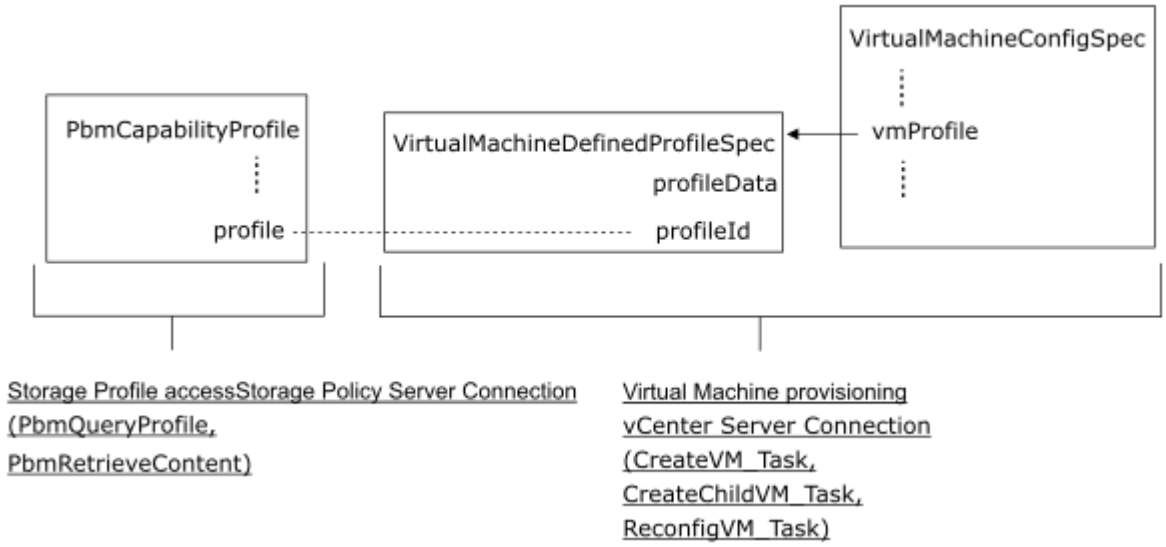
The following code fragment sets the storage profile and creates the virtual machine. The profile (`spbmProfile`) is a `VirtualMachineDefinedProfileSpec`. See Retrieve an Existing Storage Profile from the Server.

Associating a Storage Profile with a Virtual Machine

```
[...]
VirtualMachineConfigSpec configSpec = new VirtualMachineConfigSpec();
// Set SPBM profile
configSpec.getVmProfile().add(spbmProfile);
[...]
ManagedObjectReference taskmor =
     connection.getVimPort().createVMTask(vmFolderMor, vmConfigSpec, resourcepoolmor, hostmor);
```

The following figure shows how a storage profile is integrated into a virtual machine configuration specification. Your client establishes the link between the storage profile (`PbmCapabilityProfile`) and the `VirtualMachineDefinedProfileSpec` by setting the `profileId` property in the `VirtualMachineDefinedProfileSpec`. The Server sets the `profileData` property when it configures the virtual machine.

Figure 649: Using a Storage Profile for Virtual Machine Provisioning



Tag-Based Storage Profiles

To use a tag-based storage profile, you assign a storage policy tag to a data center, define a storage requirement profile based on the tag, and associate the profile with a virtual machine.

Creating a Tag-Based Storage Profile

A storage profile specification has associated tag metadata, which you can use to create a storage profile.

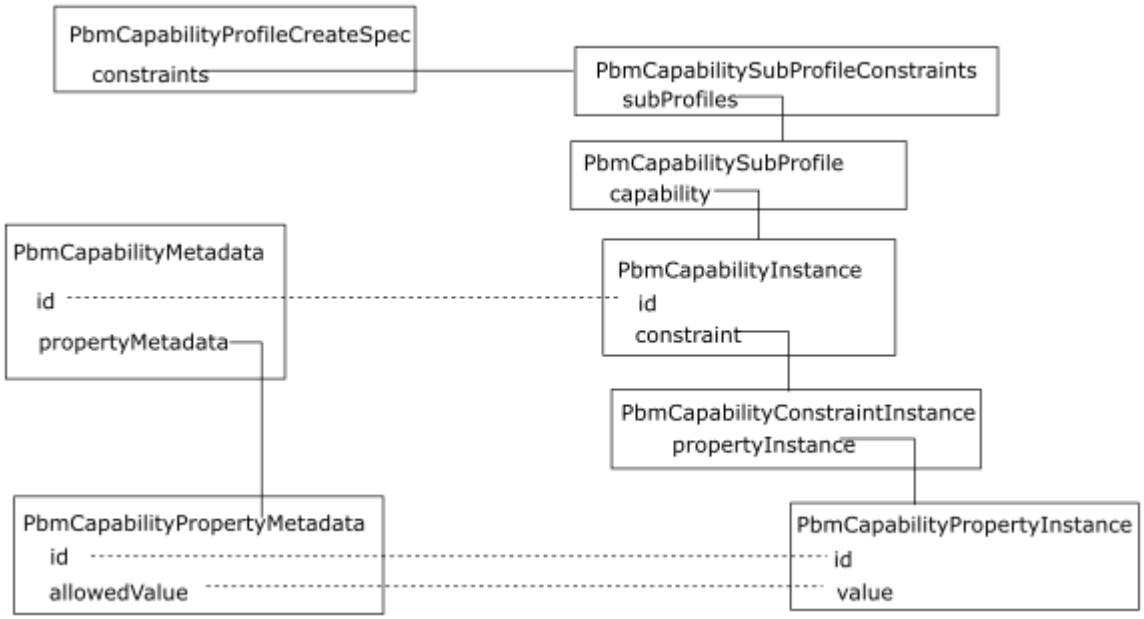
When you create the virtual machine, the vCenter instance will use the Storage Policy Server to resolve the tag reference in the profile and determine a datastore for virtual machine storage.

- To associate a storage policy tag with a datacenter, use the vSphere Web Client.
- To create a tag-based storage requirements profile, you retrieve metadata associated with a storage policy tag and create a storage profile that contains identifiers from the tag metadata.
- To associate the storage profile with a virtual machine, see Apply the Storage Profile to a Virtual Machine.

A tag-based storage profile involves the following metadata:

- The subprofile capability instance identifier (`PbmCapabilityInstance.id`) is set to the storage policy tag metadata identifier (`PbmCapabilityMetadata.id`).
- The capability property instance (`PbmCapabilityPropertyInstance`) specifies both an identifier and a value. Both properties are set to the tag metadata `id` and `allowedValue` properties.

Figure 650: Tag-Based Storage Profile Specification



The following example demonstrates how to create a storage requirements profile based on a storage policy tag. The example is divided into two sections.

- Retrieving Tag Metadata
- Creating a Storage Profile

Creating a Storage Profile

You can create a storage profile based on tag metadata.

The example performs the following operations.

1. Create a property instance with tags from the specified tag category.
2. Associate the property instance with a constraint (rule).
3. Associate the constraint with a capability instance.
4. Add the capability instance to a subprofile (rule set).
5. Add the subprofile to the list of subprofile constraints.
6. Build a profile specification.
7. Create the storage profile.

The following example is based on the Storage Policy SDK sample file `CreateProfile.java`.

Tag-Based Storage Profile Creation

```
// Get PBM Profile Manager and PBM Capability Metadata
spbmsc = connection.getPbmServiceContent();
ManagedObjectReference profileMgr = spbmsc.getProfileManager();

List<PbmCapabilityMetadataPerCategory> metadata =
     connection.getPbmPort().pbmFetchCapabilityMetadata(?
profileMgr,PbmUtil.getStorageResourceType(), null);

// Step 1: Create Property Instance with tags from the specified Category
PbmCapabilityMetadata tagCategoryInfo = PbmUtil.getTagCategoryMeta(tagCategoryName, metadata);

// Fetch Property Metadata of the Tag Category
List<PbmCapabilityPropertyMetadata> propMetaList = tagCategoryInfo.getPropertyMetadata();
```

```
PbmCapabilityPropertyMetadata propMeta = propMetaList.get(0);

// Create a New Property Instance based on the Tag Category ID
PbmCapabilityPropertyInstance prop = new PbmCapabilityPropertyInstance();
prop.setId(propMeta.getId());

// Fetch Allowed Tag Values Metadata; cast the xsd:any property (allowedValue) to a discrete set
PbmCapabilityDiscreteSet tagSetMeta = (PbmCapabilityDiscreteSet) propMeta.getAllowedValue();

// Create a New Discrete Set for holding Tag Values
PbmCapabilityDiscreteSet tagSet = new PbmCapabilityDiscreteSet();
for (Object obj : tagSetMeta.getValues()) {
   tagSet.getValues().add(((PbmCapabilityDescription) obj).getValue());
}
prop.setValue(tagSet);

// Step 2: Associate Property Instance with a Rule
PbmCapabilityConstraintInstance rule = new PbmCapabilityConstraintInstance();
rule.getPropertyInstance().add(prop);

// Step 3: Associate Rule with a Capability Instance
PbmCapabilityInstance capability = new PbmCapabilityInstance();
capability.setId(tagCategoryInfo.getId());
capability.getConstraint().add(rule);

// Step 4: Add Rule to a RuleSet
PbmCapabilitySubProfile ruleSet = new PbmCapabilitySubProfile();
ruleSet.getCapability().add(capability);

// Step 5: Add Rule-Set to Capability Constraints
PbmCapabilitySubProfileConstraints constraints = new PbmCapabilitySubProfileConstraints();
ruleSet.setName("Rule-Set " + (constraints.getSubProfiles().size() + 1));
constraints.getSubProfiles().add(ruleSet);

// Step 6: Build Capability-Based Profile
PbmCapabilityProfileCreateSpec spec = new PbmCapabilityProfileCreateSpec();
spec.setName(profileName);
spec.setDescription("Tag Based Storage Profile Created by SDK Samples. Rule based on tags from
 Category "
            + tagCategoryName);
spec.setResourceType(PbmUtil.getStorageResourceType());
spec.setConstraints(constraints);

// Step 7: Create Storage Profile
PbmProfileId profile = connection.getPbmPort().pbmCreate(profileMgr, spec);
```

Retrieving Tag Metadata

You can retrieve tag metadata for a tag category.

The following example shows a code fragment that retrieves metadata for a tag category. Given the list of metadata obtained from the Storage Policy Server, the function traverses the list and returns the metadata associated with the specified category. This function is defined in the `PbmUtil` package in the Storage Policy SDK.

getTagCategoryMeta (PbmUtil Package)

```
public static PbmCapabilityMetadata getTagCategoryMeta(
     String tagCategoryName, List<PbmCapabilityMetadataPerCategory> schema) {
```

```
            for (PbmCapabilityMetadataPerCategory cat : schema)
                if (cat.getSubCategory().equals("tag"))
                    for (PbmCapabilityMetadata cap : cat.getCapabilityMetadata())
                        if (cap.getId().getId().equals(tagCategoryName))
                           return cap;
            return null;
        }
```

Policy Rules

You can create custom storage policy strings to specify policy rules.

Create a Stand-Alone Virtual Disk with an Attached Storage Encryption Policy

The following example shows how to format an XML string to specify an I/O filter policy for a virtual disk.

- Verify that the name for the policy is user-friendly.
- Verify that the ID string to identify the policy is unique. The ID must be unique among the set of policies known to SPBM.

The format is specific to the current version of the VMware Web Services API. This format might not be compatible with other API versions.

1. Build the policy `<capability>` element, which specifies the filter.

```
<capability>
   <capabilityId>
      <id>vmwarevmcrypt@encryption</id>
      <namespace>IOFILTERS</namespace>
      <constraint></constraint>
   </capabilityId>
</capability>
```

2. Wrap the `<capability>` element singly in a `<subProfiles>` element, adding the rule name.

```
<subProfiles>
   <capability>
      <capabilityId>
         <id>vmwarevmcrypt@encryption</id>
         <namespace>IOFILTERS</namespace>
         <constraint></constraint>
      </capabilityId>
   </capability>
   <name>Rule-Set 1: IOFILTERS</name>
</subProfiles>
```

3. Wrap the `<subProfiles>` element singly in a `<constraints>` element.

```
<constraints>
  <subProfiles>
    <capability>
       <capabilityId>
          <id>vmwarevmcrypt@encryption</id>
          <namespace>IOFILTERS</namespace>
          <constraint></constraint>
       </capabilityId>
    </capability>
    <name>Rule-Set 1: IOFILTERS</name>
  </subProfiles>
</constraints>
```

4. Append profile metadata, such as name and creation date.

```
<constraints>
  <subProfiles>
    <capability>
      <capabilityId>
        <id>vmwarevmcrypt@encryption</id>
        <namespace>IOFILTERS</namespace>
        <constraint></constraint>
      </capabilityId>
    </capability>
    <name>Rule-Set 1: IOFILTERS</name>
  </subProfiles>
</constraints>
<createdBy>client</createdBy>
<creationTime>1999-12-31T23:59:59Z</creationTime>
<lastUpdatedTime>1999-12-31T23:59:59Z</lastUpdatedTime>
<generationId>1</generationId>
<name>IOfilter-Encrypt</name>
```

5. Wrap the `<constraints>` element and metadata in a `<storageProfile>` element, and prefix an XML header.

```
<?xml version='1.0' encoding='UTF-8'?>
<storageProfile xsi:type='StorageProfile'>
  <constraints>
    <subProfiles>
      <capability>
        <capabilityId>
          <id>vmwarevmcrypt@encryption</id>
          <namespace>IOFILTERS</namespace>
          <constraint></constraint>
        </capabilityId>
      </capability>
      <name>Rule-Set 1: IOFILTERS</name>
    </subProfiles>
  </constraints>
  <createdBy>client</createdBy>
  <creationTime>1999-12-31T23:59:59Z</creationTime>
  <lastUpdatedTime>1999-12-31T23:59:59Z</lastUpdatedTime>
  <generationId>1</generationId>
  <name>IOfilter-Encrypt</name>
  <profileId>I am Unique</profileId>
</storageProfile>
```

You can create a `ProfileSpec` to contain the XML data. See the *vSphere Web Services SDK Programming Guide*.

Legacy Storage Profiles

vSphere 5.0/5.1 systems support limited storage capability and requirement profiles.

In vSphere 5.5 and later, the Storage Policy Server supports more complex storage profiles. vSphere 5.5 and later upgrade legacy capability and requirement profiles for Storage Policy Server operations.

VASA 1.0 Storage Capability Upgrade

A Storage Policy Server can obtain storage capability data from VASA providers.

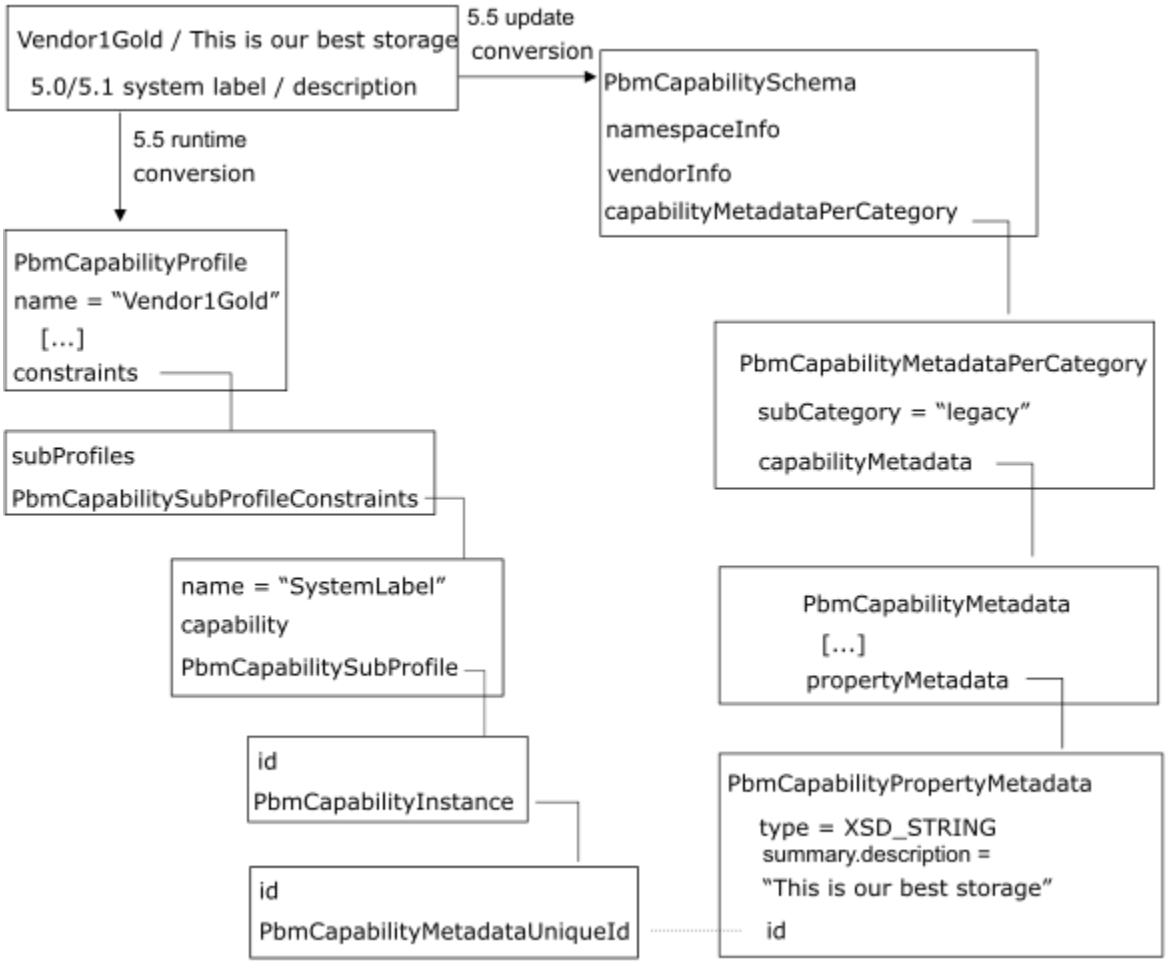
In vSphere 5.5, this generally implies VMware vSAN storage capabilities. A Storage Policy Server can also obtain capability data from a VASA provider that was implemented for the vSphere 5.0/5.1 environment.

The early architecture (vSphere 5.0/5.1) supports a simple expression of storage capability. A VASA 1.0 provider can advertise one system label per datastore. A system label has an associated description.

The Storage Policy Server performs a runtime conversion of VASA 1.0 system labels. The Storage Policy API presents the system label as a storage capability profile. The Server also generates a capability schema for the storage label. The generated storage capability profile references the generated schema.

The following figure shows the Storage Policy data objects that are generated from a vSphere 5.0/5.1 legacy profile.

Figure 651: Converted Legacy Capability Profile



vSphere Web Client User Label Conversion

A vSphere 5.0/5.1 storage profile can reference user labels displayed in the vSphere Web Client.

Users can associate a datastore with a user label. When you upgrade to vSphere 5.5, any existing 5.0/5.1 user labels will be converted into datastore tags.

A converted policy profile contains one subprofile for each label referenced by the original vSphere 5.0/5.1 profile.

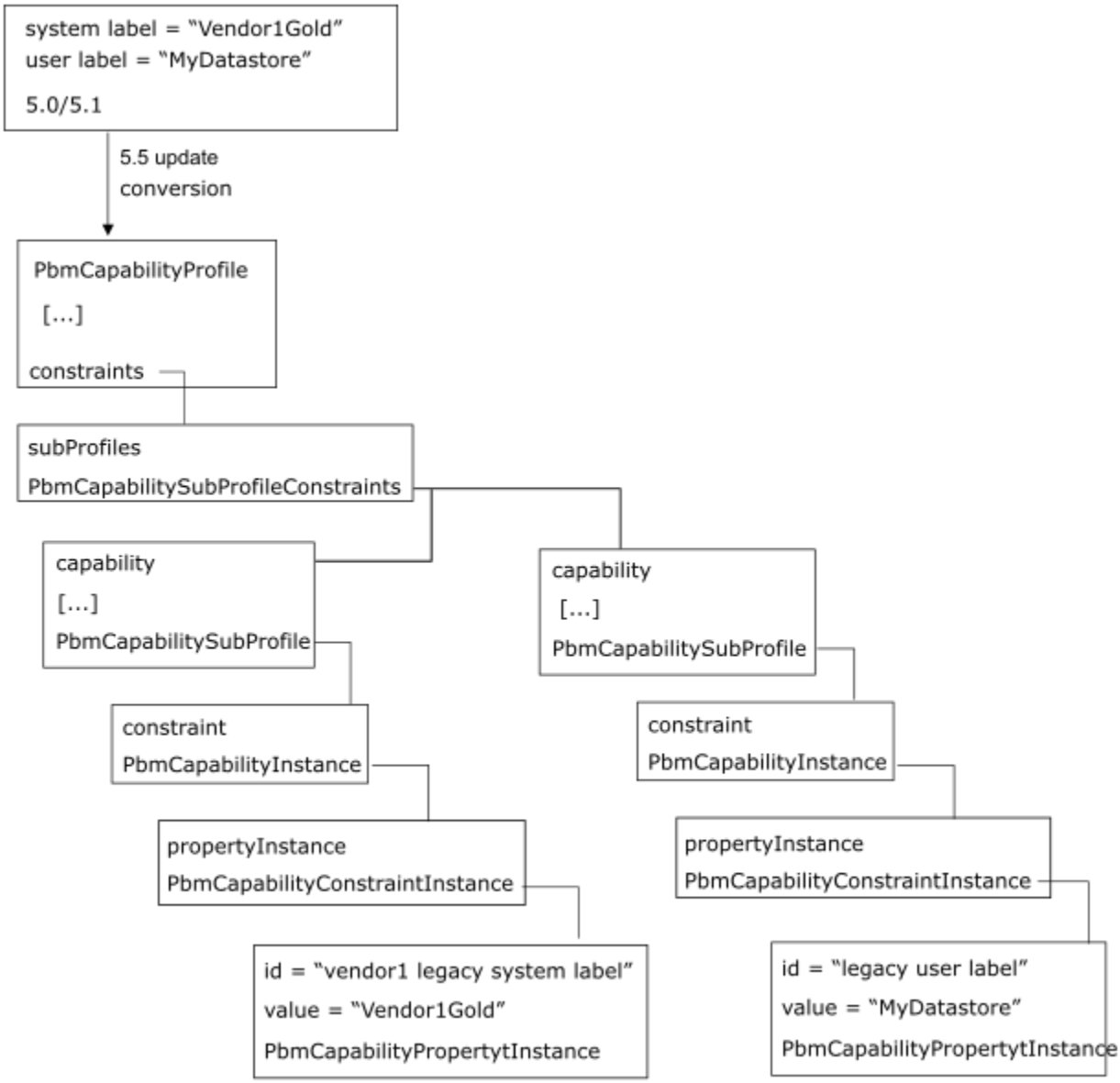
- A system label reference is converted to a reference to the appropriate vendor-specific “legacy system label” capability generated from that label.
- A user label reference is converted to a reference to the appropriate datastore tag generated from that label.

The following figure shows the conversion of a vSphere 5.0/5.1 profile that references a system label “Vendor1Gold” and a user label “MyDatastores”. When you upgrade to vSphere 5.5, the system converts the profile into Storage Policy API elements.

- The legacy system label identifies the original source as a VASA 1.0 provider.

- The legacy user label identifies the original source as a 5.0/5.1 user label in the vSphere Web Client.

Figure 652: Converted Legacy Requirement Profile



vCenter Single Sign-On Client Example

This chapter describes a Java example of acquiring a vCenter Single Sign-On security token.

vCenter Single Sign-On Token Request Overview

The code examples in the following sections show how to use the `Issue` method to acquire a holder-of-key security token.

To see an example of using the token to login to a vCenter, see vCenter LoginByToken Example. The code examples in this chapter are based on the following sample file located in the vCenter Single Sign-On SDK JAX-WS client `samples` directory.

`.../JAXWS/samples/com/vmware/sso/client/samples/AcquireHoKTokenByUserCredentialSample.java`

The `AcquireHoKTokenByUserCredentialSample` program creates a token request and calls the issue method to send the request to a vCenter Single Sign-On Server. The program uses a sample implementation of Web services message handlers to modify the SOAP security header for the request message.

This example uses the username-password security policy (`STSSecPolicy_UserPwd`). This policy requires that the SOAP security header include a timestamp, username and password, and a digital signature and certificate. The sample message handlers embed these elements in the message.

The example performs the following operations.

1. Create a security token service client object (`STSService_Service`). This object manages the vCenter Single Sign-On header handlers and it provides access to the vCenter Single Sign-On client API methods. This example uses the issue method.
2. Create a vCenter Single Sign-On header handler resolver object (`HeaderHandlerResolver`). This object acts as a container for the different handlers.
3. Add the handlers for timestamp, user credentials, certificate, and token extraction to the handler resolver.
4. Add the handler resolver to the security token service.
5. Retrieve the STS port (`STS_Service`) from the security token service object.
6. Create a security token request.
7. Set the request fields.
8. Set the endpoint in the request context. The endpoint identifies the vCenter Single Sign-On Server.
9. Call the issue method, passing the token request.
10. Handle the response from the vCenter Single Sign-On server.

Using Handler Methods for SOAP Headers

The VMware vCenter Single Sign-On SDK provides sample code that is an extension of the JAX-WS XML Web services message handler (`javax.xml.ws.handler`).

The sample code consists of a set of SOAP header handler methods and a header handler resolver, to which you add the handler methods. The handler methods insert timestamp, user credential, and message signature data into the SOAP security header for the request. A handler method extracts the SAML token from the vCenter Single Sign-On Server response.

The VMware vCenter Single Sign-On client SOAP header handler files are located in the `soaphandlers` directory.

`SDK/sso/java/JAXWS/samples/com/vmware/sso/client/soaphandlers`

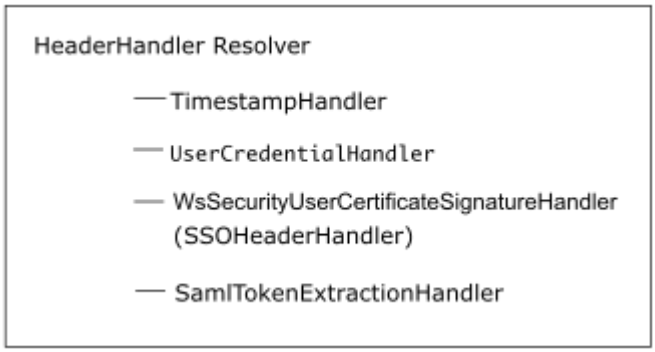
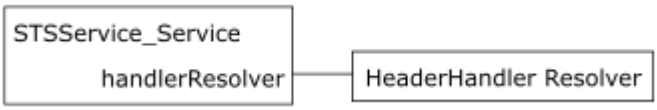
To access the SOAP handler implementation, the example code contains the following import statements.

```
import com.vmware.sso.client.soaphandlers.HeaderHandlerResolver;
import com.vmware.sso.client.soaphandlers.SSOHeaderHandler;
import com.vmware.sso.client.soaphandlers.SamlTokenExtractionHandler
import com.vmware.sso.client.soaphandlers.TimeStampHandler;
import com.vmware.sso.client.soaphandlers.UserCredentialHandler;
import com.vmware.sso.client.soaphandlers.WsSecurityUserCertificateSignatureHandler;
```

This example uses the following handler elements.

- `HeaderHandlerResolver`
- `SamlTokenExtractionHandler`
- `TimestampHandler`
- `UserCredentialHandler`
- `WsSecurityUserCertificateSignatureHandler` (`SSOHeaderHandler`)

The following sequence shows the operations and corresponding Java elements for message security.

Create an STS service object (`STSService_Service`). This object will bind the handlers to the request and provide access to the issue method.	STSService_Service
Create a handler resolver object (`HeaderHandlerResolver`). This object acts as a receptacle for the handlers.	HeaderHandlerResolver
Add the header handlers: • Timestamp – The handler will use system time to set the timestamp values. • User credential – The handler requires a username and a password; it will create a username token for the supplied values. • User certificate signature – The handler requires a private key and an x509 certificate. The handler will use the private key to sign the body of the SOAP message (the token request), and it will embed the certificate in the SOAP security header. • SAML token extraction – The handler extracts the SAML token directly from vCenter Single Sign-On Server response to avoid token modification by the JAX-WS bindings.	
Add the handler resolver to the STS service.	

The following code fragment creates a handler resolver and adds the handler methods to the handler resolver. After the handlers have been established, the client creates a token request and calls the `Issue` method. See Sending a Request for a Security Token.

Important:
You must perform these steps for message security before retrieving the STS service port. An example of retrieving the STS service port is shown in Sending a Request for a Security Token.

```
/*
 * Instantiate the STS Service
 */
STSService_Service stsService = new STSService_Service();

/*
 * Instantiate the HeaderHandlerResolver.
 */
HeaderHandlerResolver headerResolver = new HeaderHandlerResolver();

/*
 * Add handlers to insert a timestamp and username token into the SOAP security header
 * and sign the message.
 *
 * -- Timestamp contains the creation and expiration time for the request
 * -- UsernameToken contains the username/password
 * -- Sign the SOAP message using the combination of private key and user certificate.
 *
 * Add the TimeStampHandler
 */
```

```
headerResolver.addHandler(new TimeStampHandler());

/*
 * Add the UserCredentialHandler. arg[1] is the username; arg[2] is the password.
 */
UserCredentialHandler ucHandler = new UserCredentialHandler(args[1],args[2]);
headerResolver.addHandler(ucHandler);

/*
 * Add the message signature handler (WsSecurityUserCertificateSignatureHandler);
 * The client is responsible for supplying the private key and certificate.
 */
SSOHeaderHandler ssoHandler = new WsSecurityUserCertificateSignatureHandler(privateKey, userCert);
headerResolver.addHandler(ssoHandler);

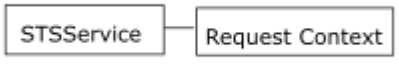
/*
 * Add the token extraction handler (SamlTokenExtractionHandler).
 */
SamlTokenExtractionHandler sbHandler = new SamlTokenExtractionHandler;
headerResolver.addHandler(sbHandler);

/*
 * Set the handlerResolver for the STSService to the HeaderHandlerResolver created above.
 */
stsService.setHandlerResolver(headerResolver);
```

Sending a Request for a Security Token

After setting up the SOAP header handlers, the example creates a token request and calls the `Issue` method.

The following sequence shows the operations and corresponding Java elements.

Retrieve the STS service port (`STSService`). The service port provides access to the vCenter Single Sign-On client API methods. The vCenter Single Sign-On handler resolver must be associated with the STS service before you retrieve the service port. See Using Handler Methods for SOAP Headers.	
Create a token request (`RequestSecurityTokenType`). Your vCenter Single Sign-On client will pass the token request to the `Issue` method. The `Issue` method will send the token request in the body of the SOAP message. This example sets the token request fields as appropriate for a holder-of-key token request.	RequestSecurityTokenType
Set the token request fields. • lifetime – Creation and expiration times. • token type – urn:oasis:names:tc:SAML:2.0:assertion • request type – http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue • key type – http://docs.oasis-open.org/ws-sx/ws-trust/200512/PublicKey (for holder-of-key token type) • signature algorithm – http://www.w3.org/2001/04/xmldsig-more#rsa-sha256 • renewable status	RequestSecurityTokenType tokenType requestType lifetime keyType signatureAlgorithm renewing
Set the endpoint address for the token request.	

Call the `Issue` method.	Issue (RequestSecurityTokenType) STSService
Handle the response from the vCenter Single Sign-On Server.	RequestSecurityTokenResponseType

The following example shows Java code that performs these operations.

Acquiring a vCenter Single Sign-On Token – Sending the Request

```
/*
 * Retrieve the STSServicePort from the STSService_Service object.
 */
STSService stsPort = stsService.getSTSServicePort();

/*
 * Create a token request object.
 */
RequestSecurityTokenType tokenType = new RequestSecurityTokenType();

/*
 *  Create a LifetimeType object.
 */
LifetimeType lifetime = new LifetimeType();

/*
 * Derive the token creation date and time.
 * Use a GregorianCalendar to establish the current time,
 * then use a DatatypeFactory to map the time data to XML.
 */
DatatypeFactory dtFactory = DatatypeFactory.newInstance();
GregorianCalendar cal = new GregorianCalendar(TimeZone.getTimeZone("GMT"));
XMLGregorianCalendar xmlCalendar = dtFactory.newXMLGregorianCalendar(cal);
AttributedDateTime created = new AttributedDateTime();
created.setValue(xmlCalendar.toXMLFormat());

/*
 * Specify a time interval for token expiration (specified in milliseconds).
 */
AttributedDateTime expires = new AttributedDateTime();
xmlCalendar.add(dtFactory.newDuration(30 * 60 * 1000));
expires.setValue(xmlCalendar.toXMLFormat());

/*
 * Set the created and expires fields in the lifetime object.
 */
lifetime.setCreated(created);
lifetime.setExpires(expires);

/*
 * Set the token request fields.
 */
tokenType.setTokenType("urn:oasis:names:tc:SAML:2.0:assertion");
tokenType.setRequestType("http://docs.oasis-open.org/ws-sx/ws-trust/200512/Issue");
tokenType.setLifetime(lifetime);
tokenType.setKeyType("http://docs.oasis-open.org/ws-sx/ws-trust/200512/PublicKey");
tokenType.setSignatureAlgorithm("http://www.w3.org/2001/04/xmldsig-more#rsa-sha256");

/*
 * Specify a token that can be renewed.
```

```
 */
RenewingType renewing = new RenewingType();
renewing.setAllow(Boolean.TRUE);
renewing.setOK(Boolean.FALSE); // WS-Trust Profile: MUST be set to false
tokenType.setRenewing(renewing);

/* Get the request context and set the endpoint address. */
Map<String, Object> reqContext = ((BindingProvider) stsPort).getRequestContext();
reqContext.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, args[0]);

/*
 * Use the STS port to invoke the "issue" method to acquire the token
 * from the vCenter Single Sign-On Server.
 */
RequestSecurityTokenResponseCollectionType issueResponse = stsPort.issue(tokenType);

/*
 * Handle the response - extract the SAML token from the response. The response type
 * contains the token type (SAML token type urn:oasis:names:tc:SAML:2.0:assertion).
 */
RequestSecurityTokenResponseType rstResponse = issueResponse.getRequestSecurityTokenResponse();
RequestedSecurityTokenType requestedSecurityToken = rstResponse.getRequestedSecurityToken();

/*
 * Extract the SAML token from the RequestedSecurityTokenType object.
 * The generic token type (Element) corresponds to the type required
 * for the SAML token handler that supports the call to LoginByToken.
 */
Element token = requestedSecurityToken.getAny();
```

vCenter LoginByToken Example

This chapter describes a Java example of using the `LoginByToken` method.

vCenter Single Sign-On Session

After you obtain a SAML token from the vCenter Single Sign-On Server, you can use the vSphere API method `LoginByToken` to establish a single sign-on session with a vCenter instance.

See vCenter Single Sign-On Client Example for an example of obtaining a vCenter Single Sign-On token.

At the beginning of a vCenter Single Sign-On session, your client is responsible for the following tasks:

- Maintain the vCenter session cookie. The vSphere architecture uses an HTTP cookie to support a persistent connection between a vSphere client and a vCenter instance. During the initial connection, the Server produces a session cookie. Operations during the login sequence will reset the request context so your client must save this cookie and re-introduce it at the appropriate times.
- Insert the vCenter Single Sign-On token and a timestamp into the SOAP header of the `LoginByToken` message.

The example program uses these general steps.

1. Call the `RetrieveServiceContent` method to establish an HTTP connection with the vCenter instance and save the HTTP session cookie. The client uses an HTTP header handler method to extract the cookie from the vCenter response.
2. Call the `LoginByToken` method to authenticate the vCenter session. To send the token to the vCenter instance, the client uses a handler to embed the token and a time stamp in the SOAP header for the message. To identify the session started with the `RetrieveServiceContent` method, the client uses a handler to embed the session cookie in the HTTP header.
3. Restore the session cookie.

HTTP and SOAP Header Handlers

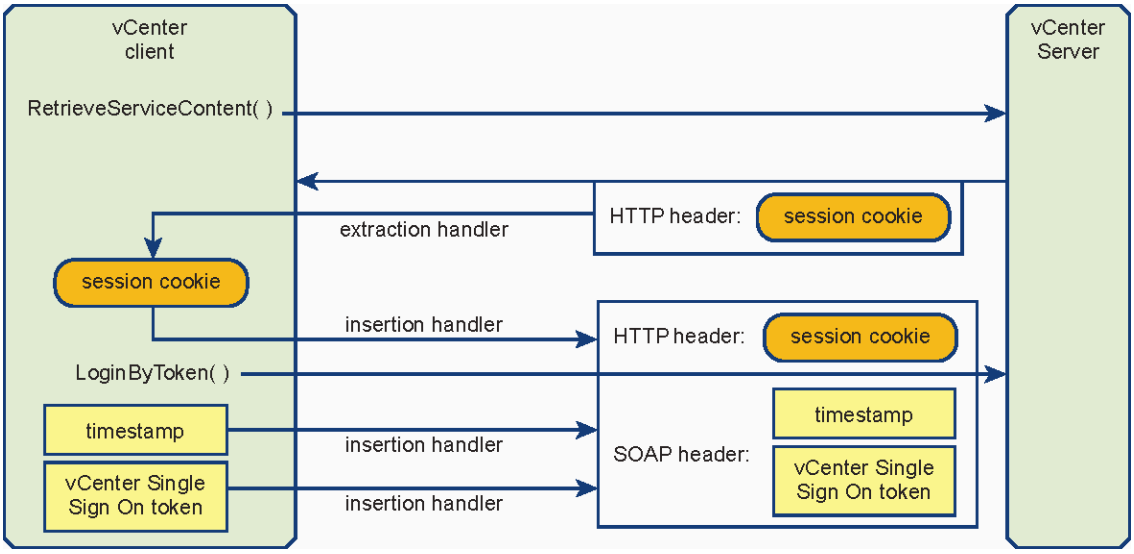
To use a vCenter Single Sign-On token to login to a vCenter instance, the example uses header handlers to manipulates the HTTP and SOAP header elements of the login request.

After establishing a handler, subsequent requests automatically invoke the handler.

- An extraction handler obtains the HTTP session cookie provided by the vCenter instance. After setting up the handler, a call to the `RetrieveServiceContent` method will invoke the handler to extract the cookie from the Server response.
- Insertion handlers put the vCenter Single Sign-On token and a timestamp into the SOAP header and the session cookie into the HTTP header of the login request.

The following figure shows the use of handlers to manipulate header elements when establishing a vCenter Single Sign-On session with a vCenter instance.

Figure 653: Starting a vCenter Session



Important:
Every call to the vCenter instance will invoke any message handlers that have been established. The overhead involved in using the SOAP and HTTP message handlers is not necessary after the session has been established. The example saves the default message handler before setting up the SOAP and HTTP handlers. After establishing the session, the example will reset the handler chain and restore the default handler.

The example code also uses multiple calls to the `VimPortType.getVimPort method` to manage the request context. The `getVimPort` method clears the HTTP request context. After each call to the `getVimPort` method, the client resets the request context endpoint address to the vCenter URL. After the client has obtained the session cookie, it will restore the cookie in subsequent requests.

LoginByToken Sample Code

The code examples in the following sections show how to use the `LoginByToken` method with a holder-of-key security token.

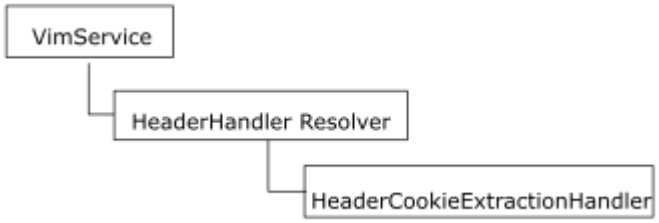
The code examples are based on the sample code contained in the vCenter Single Sign-On SDK. The files are located in the Java samples directory (`SDK/ssoclient/java/JAXWS/samples`).

- LoginByToken sample:
  `samples/com/vmware/vsphere/samples/LoginByTokenSample.java`
- Header cookie handlers:
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieHandler.java`
  `samples/com/vmware/vsphere/soaphandlers/HeaderCookieExtractionHandler.java`
- SOAP header handlers. These are the same handlers that are used in vCenter LoginByToken Example. The SOAP handler files are located in the vCenter Single Sign-On client `soaphandlers` directory:
  `samples/com/vmware/sso/client/soaphandlers`

Saving the vCenter Session Cookie

The code fragment in this section establishes an HTTP session with the vCenter instance and saves the HTTP session cookie.

The following sequence describes these steps and shows the corresponding objects and methods.

Use the `getHandlerResolver` method to save the default message handler. To use the HTTP and SOAP message handlers, you must first save the default message handler so that you can restore it after login. The HTTP and SOAP message handlers impose overhead that is unnecessary after login.	VimService.getHandlerResolver( )
Set the cookie handler. The `HeaderCookieExtractionHandler` method retrieves the HTTP cookie.	
Get the VIM port. The VIM port provides access to the vSphere API methods, including the `LoginByToken` method.	
Set the request context endpoint address to the vCenter URL.	
Retrieve the `ServiceContent`. This method establishes the HTTP connection and sets the session cookie.	
Extract the cookie and save it for later use.	HeaderCookieExtractionHandler.getCookie ( )

The following example shows Java code that saves the session cookie.

Saving the vCenter Session Cookie

```
/*
 * The example uses a SAML token (obtained from a vCenter Single Sign-On Server)
 * and the  vCenter  URL.
 * The following declarations indicate the datatypes; the token datatype (Element) corresponds
 * to the token datatype returned by the vCenter Single Sign-On Server.
 *
 * Element token;       -- from vCenter Single Sign-On Server
 * String vcServerUrl;  -- identifies  vCenter
 *
 * First, save the default message handler.
 */

HandlerResolver defaultHandler = vimService.getHandlerResolver();

/*
 * Create a VIM service object.
 */
vimService = new VimService();
```

```
/*
 * Construct a managed object reference for the ServiceInstance.
 */
ManagedObjectReference SVC_INST_REF = new ManagedObjectReference();
SVC_INST_REF.setType("ServiceInstance");
SVC_INST_REF.setValue("ServiceInstance");

/*
 * Create a handler resolver.
 * Create a cookie extraction handler and add it to the handler resolver.
 * Set the VIM service handler resolver.
 */
HeaderCookieExtractionHandler cookieExtractor = new HeaderCookieExtractionHandler();
HeaderHandlerResolver handlerResolver = new HeaderHandlerResolver();
handlerResolver.addHandler(cookieExtractor);
vimService.setHandlerResolver(handlerResolver);

/*
 * Get the VIM port for access to vSphere API methods. This call clears the request context.
 */
vimPort = vimService.getVimPort();

/*
 * Get the request context and set the connection endpoint.
 */
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Retrieve the ServiceContent. This call establishes the HTTP connection.
 */
serviceContent = vimPort.retrieveServiceContent(SVC_INST_REF);

/*
 * Save the HTTP cookie.
 */
String cookie = cookieExtractor.getCookie();
```

Using LoginByToken

The code fragment in this section sets up the message handlers and calls the `LoginByToken` method.

The following sequence describes the steps and shows the corresponding objects and methods.

Create a new `HeaderHandlerResolver`. Then set the message security handlers for cookie insertion and for inserting the SAML token and credentials in the SOAP header.	
Get the VIM port.	

Set the connection endpoint in the HTTP request context.	
Call the `LoginByToken` method. The method invocation executes the handlers to insert the elements into the message headers. The method authenticates the session referenced by the session cookie.	VimPortType.LoginByToken ( )

The following examples shows Java code that calls the `LoginByToken` method.

Using LoginByToken

```
/*
 * Create a handler resolver and add the handlers.
 */
HeaderHandlerResolver handlerResolver = new HeaderHandlerResolver();
handlerResolver.addHandler(new TimeStampHandler());
handlerResolver.addHandler(new SamlTokenHandler(token));
handlerResolver.addHandler(new HeaderCookieHandler(cookie));
handlerResolver.addHandler(new WsSecuritySignatureAssertionHandler(
                           userCert.getPrivateKey(),
                           userCert.getUserCert(),
                           Utils.getNodeProperty(token, "ID")));
vimService.setHandlerResolver(handlerResolver);

/*
 *  Get the Vim port; this call clears the request context.
 */
vimPort = vimService.getVimPort();

/*
 * Retrieve the request context and set the server URL.
 */
Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Call LoginByToken.
 */
UserSession us = vimPort.loginByToken(serviceContent.getSessionManager(), null);
```

Restoring the vCenter Session Cookie

After you log in, you must restore the standard vCenter session context.

The code fragment in this section restores the default message handler and the session cookie. As the cookie handler has been replaced by the default handler, the client resets the session cookie by calling request context methods to

access the context fields directly. The following sequence describes these steps and shows the corresponding objects and methods.

Restore the default message handler. The handlers used for `LoginByToken` are not used in subsequent calls to the vSphere API.	VimService.setHandlerResolver ( )
Get the VIM port.	
Set the connection endpoint in the HTTP request context.	
Set the HTTP request header (vCenter session cookie).	RequestContext.get () RequestContext.put ( )

The following example shows Java code that restores the vCenter session. This code requires the vCenter URL and the cookie and default handler that were retrieved before login. See LoginByToken Sample Code.

Restoring the vCenter Session

```
/*
 * Reset the default handler. This overwrites the existing handlers, effectively removing them.
 */
vimService.setHandlerResolver(defaultHandler);
vimPort = vimService.getVimPort();

/*
 * Restore the connection endpoint in the request context.
 */
// Set the validated session cookie and set it in the header for once,
// JAXWS will maintain that cookie for all the subsequent requests

Map<String, Object> ctxt = ((BindingProvider) vimPort).getRequestContext();
ctxt.put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY, vcServerUrl);
ctxt.put(BindingProvider.SESSION_MAINTAIN_PROPERTY, true);

/*
 * Reset the cookie in the request context.
 */
Map<String, List<String>> headers = (Map<String, List<String>>)
 ctxt.get(MessageContext.HTTP_REQUEST_HEADERS);
if (headers == null) {
    headers = new HashMap<String, List<String>>();
}
headers.put("Cookie", Arrays.asList(cookie));
ctxt.put(MessageContext.HTTP_REQUEST_HEADERS, headers);
```

Remote Serial Port Access with a Virtual Serial Port Proxy

The VMware vSphere™ API supports the use of virtual serial ports on virtual machines that run on ESX™ hosts. A virtual serial port represents a serial port on a virtual machine. A virtual serial port can operate as a network serial port by using a network socket on the host to provide access to remote systems. A remote system is any system on the network that supports a serial port connection with a virtual machine. This feature supports remote console login for system management using an out-of-band connection that bypasses the vSphere Client.

You can also use a virtual serial port for communication through a software pipe, for output to a file, and for communication through a physical serial port on the host. For information about the vSphere API support for these applications, see the description of the `VirtualSerialPort` object in the *vSphere API Reference*.

In a VMware® data center, a vCenter uses vMotion™ to move virtual machines across hosts as part of resource management. vSphere does not support vMotion operations on a virtual machine that has a direct network serial port connection with a remote system. Network serial ports use the host IP address. If the server attempts to move a virtual machine with a network serial port connection, the host rejects the vMotion request to maintain the connection.

To support vMotion for virtual machines that use network serial ports, you can develop a proxy that operates between a virtual machine and a remote system. The vSphere API supports the use of a third-party virtual serial port concentrator in the data center environment. A virtual serial port concentrator acts as a proxy or an access server between virtual machines and remote systems. The proxy maintains communication with virtual machines that are using network serial ports. When a vMotion event occurs, the virtual machine connects to the proxy from its new host so that the connection with the remote system persists.

Virtual machines use the telnet protocol for virtual serial port proxy connections. VMware defines a telnet extension that includes commands for proxy notification of vMotion events.

See VMware Telnet Extension Commands for Proxy Communication for reference information about the VMware telnet extension commands.

Virtual Serial Port Proxy Capabilities

A virtual serial port proxy operates as an RFC2217-compatible telnet server. A proxy provides the following capabilities in the vSphere environment.

- Accepts telnet connections from virtual machines
- Supports VMware telnet extension commands and responds to VMware telnet extension messages from virtual machines
- Forwards network traffic between virtual machines and remote systems
- Handles extension commands for notification of vMotion events and redirects traffic accordingly

Proxy as a Virtual Serial Port Concentrator

Figure 654: Virtual Serial Port Concentrator shows a proxy that operates as a virtual serial port concentrator. The concentrator has internal connections to several virtual machines in a data center and an external connection to a remote system.

The concentrator provides a remote system with access to multiple virtual machines that act as serial port servers. The virtual machines connect to the concentrator, indicating that they will accept connections from remote systems. See Backing Options and Backing Information for Network Serial Ports for more information about serial port servers. The concentrator presents the choice of virtual machines to the remote system and maintains any subsequent connection. In Figure 654: Virtual Serial Port Concentrator, a solid line represents an active connection between the remote system and a virtual machine, and dashed lines represent potential connections.

Figure 654: Virtual Serial Port Concentrator

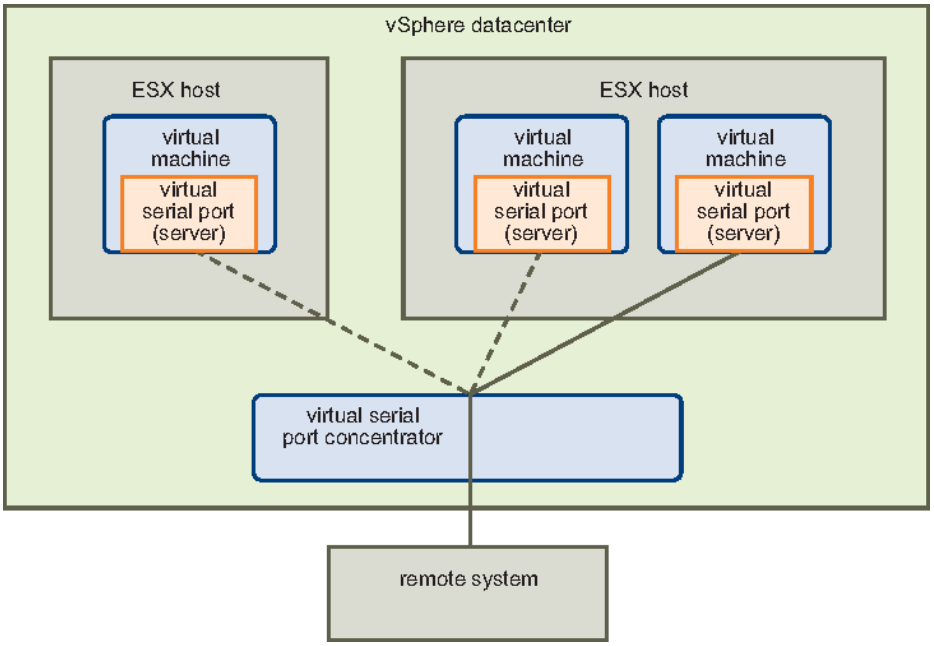


Figure 654: Virtual Serial Port Concentrator shows the concentrator as part of the vSphere data center. For this kind of application, you can deploy the concentrator as a virtual appliance. A virtual appliance is software on a platform of one or more virtual machines (in this case, a single virtual machine). With a virtual appliance implementation, you can take advantage of resource management that vCenter provides. The vCenter uses vMotion technology to migrate the virtual appliance proxy through the data center without interrupting remote serial port service. The virtual appliance uses virtual machine IP addresses that persist across vMotion events.

Although the primary purpose of the proxy is to preserve connections during vMotion events, it also can provide a gateway or firewall to enforce security. The proxy serves as the point of authentication for remote systems that use virtual serial ports to access virtual machines.

Network Serial Ports

A network serial port is a virtual serial port that uses a network socket on the host computer. A network socket is the endpoint of a network connection and is represented by an IP address and a port number. The vSphere API uses Uniform Resource Identifiers (URI) to identify the protocol, IP address, and port number for endpoints of network serial port connections. See RFC3986 for information about Uniform Resource Identifiers.

To use a network serial port, you must verify that the ESX host supports the feature and then create or reconfigure a virtual machine to configure the port. For information about writing an application that uses the vSphere API, see the *vSphere Web Services SDK Programming Guide*. As an alternative, you can use the vSphere Client to configure a network serial port. For more information, see the online help for the VMware vSphere Client.

Backing Options and Backing Information for Network Serial Ports

The vSphere API uses backing objects to represent the host device that is associated with a virtual device. The API offers two types of backing objects for virtual devices: option objects and information objects.

Backing option objects identify host device support for virtual machines. An ESX host defines the set of virtual device backing options that are available for virtual machines. To use a network serial port, the ESX host must provide the virtual serial port URI backing option, which is represented by the `VirtualSerialPortURIBackingOption` object. For information about how to find out if the host supports the URI backing option, see Determining Backing Options.

Backing information objects supply data for virtual device configuration. Use a `VirtualSerialPortURIBackingInfo` object to bind a virtual serial port to a network socket. You can configure a virtual serial port to use URI backing when you

create or reconfigure a virtual machine. For a description of how to configure URI backing for a network serial port, see Create a Virtual Machine with a Network Serial Port.

When you specify URI backing for the network serial port connection, you define these characteristics.

- Direction – defines the role that the virtual machine plays in a connection with a remote system (client or server). When a virtual machine acts as a serial port client, it initiates the connection with the remote system. When a virtual machine acts as a serial port server, it listens for connections that are initiated by remote clients.
- Service URI – identifies the target of connections. When the virtual machine acts as a client, the service URI identifies the remote system. When the virtual machine acts as a server, the service URI identifies proxy access to the host on which the virtual machine runs. See RFC3986 for information about URI syntax.
- Proxy URI – identifies the virtual serial port proxy. The presence of the proxy URI directs the virtual machine to establish a connection with the proxy. The virtual machine always initiates the connection with the proxy.

  You can use either telnet or secure telnet (telnets) for the proxy connection. The proxy URI specification must use one of the following formats. See RFC3986 for information about URI syntax. See the description of the `VirtualSerialPortURIBackingInfo` data object in the *vSphere API Reference* for information about URI schemes and authentication parameters for network serial port configuration.
  - telnet://<host>:<port>[#key[=value][&key[=value] ...]]
  - telnets://<host>:<port>[#key[=value][&key[=value] ...]]

    When the virtual machine performs telnet negotiation with the proxy, it sends the direction and service URI information to the proxy. (See the description of the `DO-PROXY` command in Proxy Operation.)

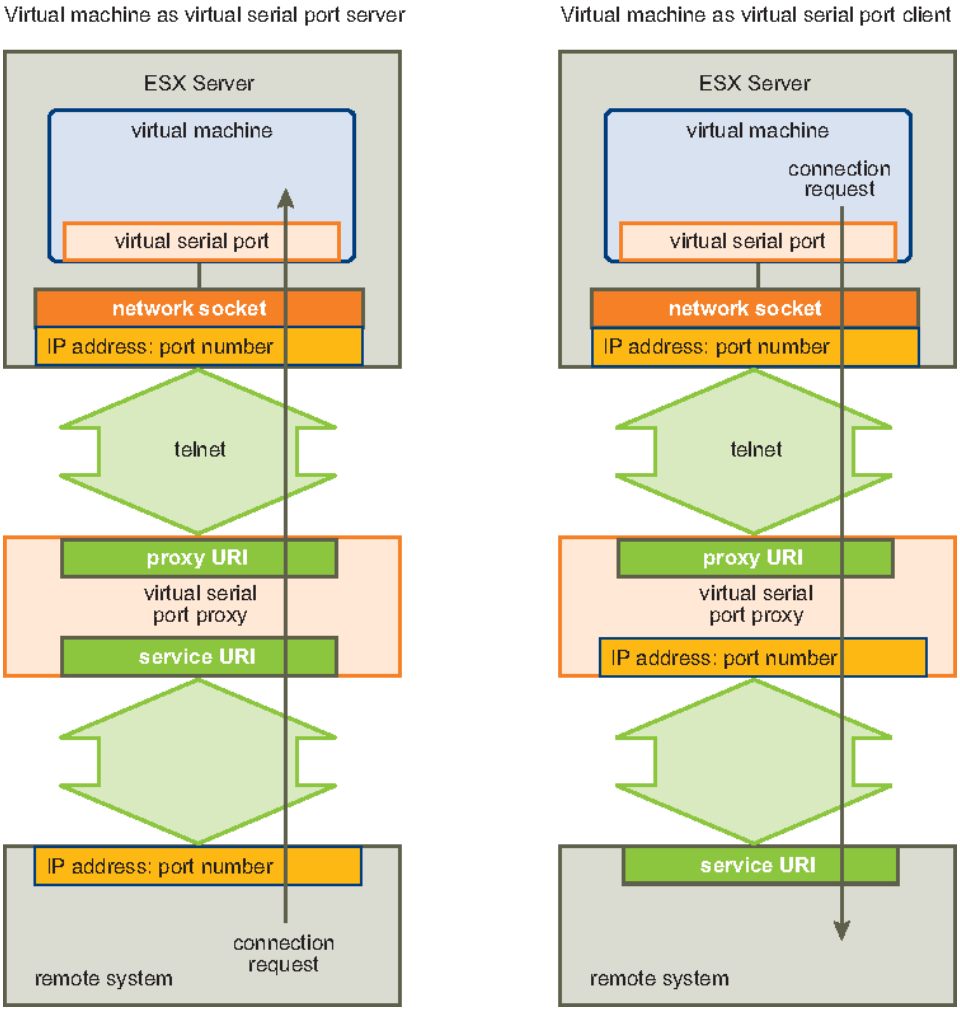
Network Connections Between a Virtual Machine, a Proxy, and a Remote System

Figure 655: Virtual Serial Port Connections to a Remote System Through a Proxy shows two virtual machines. Each machine uses a proxy for its connection to a remote system.

- One virtual machine acts as a virtual serial port server. This virtual machine uses the service URI to identify proxy access to the host machine. The virtual machine initiates a telnet connection with the proxy. The proxy, on behalf of the virtual machine, listens for connection requests from remote systems and forwards the requests to the virtual machine.
- One virtual machine acts as a virtual serial port client. This virtual machine uses the service URI to identify the remote system. The virtual machine initiates a telnet connection with the proxy. The proxy forwards the connection request to the remote system.

The proxy has two network sockets. It uses one socket (proxy URI) for the telnet connections with the virtual machine and the other for the connection with the remote system. The external connection between the proxy and a remote system uses the mutually agreed upon protocol. For a connection initiated by the remote system, the person operating the remote system must obtain the service URI by email or other independent communication.

Figure 655: Virtual Serial Port Connections to a Remote System Through a Proxy



Determining Backing Options

Use the `QueryConfigOption` method to determine which backing options are available on the host on which you will create the virtual machine. The method returns a `VirtualMachineConfigOption` data object. The virtual machine configuration data includes a a list of virtual device options and a list of backing options. The following pseudocode shows the path to the backing options.

```
VirtualMachineConfigOption.hardwareOptions.VirtualDeviceOption[].backingOption[]
```

The virtual device option array must include the `VirtualSerialPortOption` data object. For network backing, the array of serial port backing options must include the `VirtualDeviceURIBackingOption` data object. The `VirtualDeviceURIBackingOption.type` property specifies the name of the backing object you should create when you configure network backing.

Create a Virtual Machine with a Network Serial Port

You can use the `CreateVM_Task` method to create a virtual machine and configure a network serial port. The virtual machine uses the virtual serial port configuration to connect to a proxy and to accept a connection from a remote system.

1. Create a `VirtualMachineConfigSpec` object for the method's `config` parameter.
2. Create a `VirtualDeviceConfigSpec` object for the `deviceChange` array in the `VirtualMachineConfigSpec` object.
3. Create a `VirtualSerialPort` object for the `device` property in the `VirtualDeviceConfigSpec` object.
4. To control proxy connection behavior, create a `VirtualDeviceConnectInfo` data object for the `VirtualSerialPort.connectable` property.

   The following pseudocode shows the resulting path to the connection information.

   ```
   VirtualMachineConfigSpec.deviceChange[].device.connectable
   ```

5. Set the `startConnected` property in the `connectable` object to TRUE.

   The `startConnected` property determines whether the virtual machine will open a connection with the proxy when the virtual machine starts. When the virtual machine powers on, it copies the `startConnected` value to the `VirtualDeviceConnectInfo.connected` property. While the virtual machine is running, you can use the `ReconfigVM_Task` method to set the `connected` property to open or close the proxy connection. If you reconfigure a port with `connected` set to TRUE, the virtual machine closes the existing connection and opens a new one.
6. Create a `VirtualDeviceURIBackingInfo` object for the `backing` property in the `VirtualSerialPort` object.

   The following pseudocode shows the resulting path to the backing information.

   ```
   VirtualMachineConfigSpec.deviceChange[].device.backing
   ```

   Use the backing information object to supply the following information:

   - Set the `direction` property to “server” to direct the virtual machine to accept remote connections.
   - Set the `serviceURI` property to the URI for the host on which the virtual machine runs.
   - Set the `proxyURI` property to the URI for the virtual serial port concentrator. You cannot include a user/password sequence in the proxy URI.

VMware Telnet Session

To establish a VMware telnet session for remote serial port communication, a virtual machine and a proxy participate in a message exchange that follows the model of a standard telnet protocol sequence. The proxy handles messages from the virtual machine and sends the appropriate responses. Before the proxy can forward content, the virtual machine and the proxy participate in the following progression.

1. The virtual machine and the proxy negotiate the use of the VMware telnet extension option.
   Both sides of the connection must agree to use the option before continuing with any additional extension message exchange.
2. The virtual machine and the proxy identify the VMware telnet suboption commands that they support.
   The proxy must support the `KNOWN-SUBOPTIONS-2` and `UNKNOWN-SUBOPTION-RCVD-2` commands. During a session, the proxy can send the `UNKNOWN-SUBOPTION-RCVD-2` command as a valid response to virtual machine messages.
3. The proxy agrees to act as a VMware telnet extension proxy for remote serial port communication.

VMware Telnet Commands

Table lists the VMware telnet extension commands and their brief descriptions. Commands in the table are grouped by purpose: VMware telnet extension and suboptions, proxy, vMotion, and identification. The table uses command symbols. A VMware telnet extension command is a two- or three-byte code embedded in a telnet command sequence.

For more information about commands and the code equivalents, see VMware Telnet Extension Commands for Proxy Communication.

Command	Source	Description
VMWARE-TELNET-EXT	virtual machine, proxy	The VMware telnet extension option command
KNOWN-SUBOPTIONS-1	virtual machine	Suboptions supported by the virtual machine
KNOWN-SUBOPTIONS-2	proxy	Suboptions supported by the proxy
UNKNOWN-SUBOPTION-RCVD-1	virtual machine	Unknown suboption received by the virtual machine
UNKNOWN-SUBOPTION-RCVD-2	proxy	Unknown suboption received by the proxy
DO-PROXY	virtual machine	Virtual machine request for proxy support
WILL-PROXY	proxy	Positive proxy response
WON'T-PROXY	proxy	Negative proxy response
VMOTION-BEGIN	virtual machine	Notification of a pending vMotion operation for a virtual machine
VMOTION-GOAHEAD	proxy	Positive proxy response
VMOTION-NOTNOW	proxy	Negative proxy response
VMOTION-PEER	virtual machine	Virtual machine request identifying the new virtual machine instance
VMOTION-PEER-OK	proxy	Proxy response accepting the new virtual machine instance
VMOTION-COMPLETE	virtual machine	Notification that vMotion operation is complete
VMOTION-ABORT	virtual machine	Notification that vMotion operation was aborted
GET-VM-VC-UUID	proxy	Proxy request for the virtual center representation of virtual machine UUID
VM-VC-UUID	virtual machine	Virtual machine response
GET-VM-NAME	proxy	Proxy request for the virtual machine name
VM-NAME	virtual machine	Virtual machine response
GET-VM-BIOS-UUID	proxy	Proxy request for the virtual machine BIOS UUID
VM-BIOS-UUID	virtual machine	Virtual machine response
GET-VM-LOCATION-UUID	proxy	Proxy request for the virtual machine location UUID
VM-LOCATION-UUID	virtual machine	Virtual machine response

Establish a VMware Telnet Extension Session

If you specify a proxy URI in the virtual serial port configuration, the virtual machine will initiate a telnet connection with the proxy. After the connection is established, the virtual machine initiates a message exchange to establish a VMware telnet extension session. The following progression describes the model for the message exchange and uses symbols to represent the codes. For the code equivalents, see the command descriptions in VMware Telnet Extension Commands for Proxy Communication.

1. Start the VMware telnet session.
   - Virtual Machine – Sends a message that indicates it will send VMware telnet extension commands to the proxy.

```
IAC WILL VMWARE-TELNET-EXT
```

- Proxy – Responds that it will receive VMware telnet extension commands.

```
IAC DO VMWARE-TELNET-EXT
```

2. Identify known suboptions.
   - Virtual Machine – Sends a message that indicates the set of suboption commands that it can send and receive. The suboptions are expressed as a sequence of codes that correspond to suboption commands.

```
IAC SB VMWARE-TELNET-EXT KNOWN-SUBOPTIONS-1 suboptions IAC SE
```

   - Proxy – Responds with a message that indicates the set of suboption commands that it can send and receive.

```
IAC SB VMWARE-TELNET-EXT KNOWN-SUBOPTIONS-2 suboptions IAC SE
```

3. Negotiate proxy support.
   - Virtual Machine – Sends a request for the proxy to act as a VMware telnet extension proxy. The proxy request includes direction and URI information from the network backing for the virtual serial port. (For information about using the API to specify this information, see Create a Virtual Machine with a Network Serial Port.)

```
IAC SB VMWARE-TELNET-EXT DO-PROXY direction uri IAC SE
```

   - Proxy – Responds, indicating that it will perform proxy operations.

```
IAC SB VMWARE-TELNET-EXT WILL-PROXY IAC SE
```

Proxy Failure Detection (TCP keepalive)

The ESX Server uses a TCP keepalive timer to determine if a serial port connection between a virtual machine and a virtual serial port proxy is still active. By default, the virtual serial port keepalive time is 300 seconds.

To configure the keepalive time for a serial port connection, set the `keepIdle` advanced option for the virtual machine. The option name uses the following format:

```
serialn.keepIdle
```

n is an integer in a sequence starting with zero, corresponding to the sequence of serial ports on the virtual machine. Examples of `keepIdle` options include `serial0.keepIdle`, `serial1.keepIdle`, and `serial2.keepIdle`.

The `keepIdle` value is the duration in seconds on an idle connection, after which the ESX Server sends up to five keepalive probes to the virtual serial port proxy. If there is no response after five probes, the connection is considered inactive. The Server uses a keepalive interval of 1/100th of the `keepIdle` value, and a retry value of five. Using the default `keepIdle` value (300 seconds), the Server will detect a proxy failure no later than 315 seconds after the last activity (300 + 5*300/100).

vMotion and Persistent Serial Port Connections

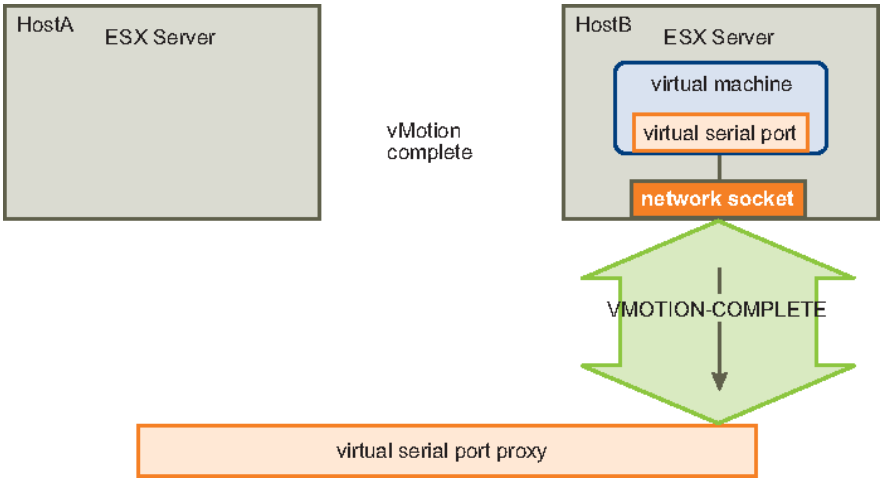
A virtual serial port proxy provides support for persistent remote serial port connections when a vCenter moves a virtual machine to a different host. To support the persistent connection during the vMotion operation, the proxy must support two telnet connections for the virtual machine.

A proxy buffers data while it is forwarding content from the remote system to a virtual machine. A proxy must respond to the `VMOTION-BEGIN` command, which is notification of a pending vMotion operation. ESX Server support for the VMware telnet extension allows the proxy to postpone the vMotion event until it finishes forwarding content. After sending the `VMOTION-GOAHEAD` command in response to `VMOTION-BEGIN`, the proxy must buffer any additional data it receives from the remote system. When the vMotion event is complete, the proxy continues the content transmission to the new instance of the virtual machine.

By default, the proxy must respond to a `VMOTION-BEGIN` command within 5000ms (5 seconds). If the proxy does not respond in time, the vMotion operation is terminated andthe virtual machine remains on the original server. You can specify a different time limit for the vCenter to wait for a `VMOTION-GOAHEAD` response from the proxy. To configure the time limit, set the `serialn.vmotionLimit` advanced option for the virtual machine. The *n* corresponds to the sequence of serial ports on the virtual machine. Specify the `vmotionLimit` value in milliseconds. Examples of `vmotionLimit` options include `serial0.vmotionLimit`, `serial1.vmotionLimit`, and `serial2.vmotionLimit`.

Figure 656: VMOTION-BEGIN and VMOTION-GOAHEAD shows two ESX hosts and a proxy at the start of a vMotion operation. Host A has a virtual machine that will be moved to Host B.

Figure 656: VMOTION-BEGIN and VMOTION-GOAHEAD



- Virtual machine – At the beginning of the vMotion operation, the ESX server (Host A) uses the virtual machine's serial port connection to send a `VMOTION-BEGIN` request to the proxy. To identify the virtual machine, the host provides an opaque `sequence` value for the message.

  ```
  IAC SB VMWARE-TELNET-EXT VMOTION-BEGIN sequence IAC SE
  ```

- Proxy – After sending pending data, the proxy replies by sending a `VMOTION-GOAHEAD` message to indicate that the vMotion operation can continue. The message includes the `VMOTION-BEGINsequence` value and an opaque `secret` value, which the proxy provides.

  ```
  IAC SB VMWARE-TELNET-EXT VMOTION-GOAHEAD sequencesecret IAC SE
  ```

After the proxy replies with a `VMOTION-GOAHEAD` response, the vMotion operation begins. The proxy has a single connection to the virtual machine.

The vCenter creates a second instance of the virtual machine on Host B. When the virtual machine boots on the Host B, it configures network backing for the virtual serial port and establishes a second telnet connection with the proxy. Before continuing with the vMotion operation, the new virtual machine and the proxy renegotiate the telnet `COM-PORT-OPTION`. They do not renegotiate the Com Port configuration. The proxy should be prepared to support the same Com Port configuration that was established for the original telnet connection. See RFC2217 for information about the telnet Com Port control option. The proxy then maintains one telnet connection for each instance of the virtual machine.

To start the VMware telnet extension session for the new connection, the new virtual machine instance negotiates the VMware telnet extension option (`VMWARE-TELNET-EXT`).

Figure 657: VMOTION-PEER and VMOTION-PEER-OK shows the systems during the vMotion operation.

- Virtual Machine: To continue the vMotion operation, Host B sends a `VMOTION-PEER` message to the proxy to identify the new instance as the same virtual machine that started the vMotion operation. The message includes both the `sequence` and `secret` values to identify the virtual machine instance.

  ```
  IAC SB VMWARE-TELNET-EXT VMOTION-PEER sequencesecret IAC SE
  ```

- Proxy: The proxy replies with a `VMOTION-PEER-OK` message to indicate that it accepts the peer connection.

  ```
  IAC SB VMWARE-TELNET-EXT VMOTION-PEER-OK sequence IAC SE
  ```

Figure 657: VMOTION-PEER and VMOTION-PEER-OK

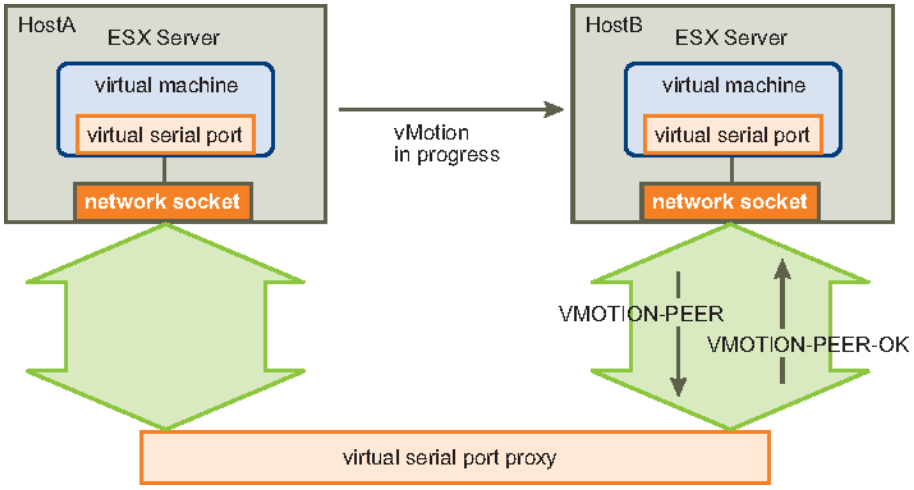
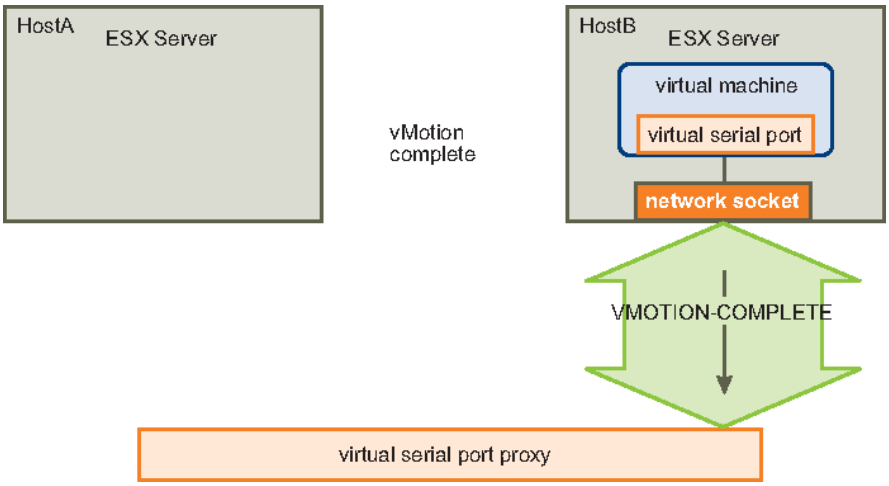


Figure 658: VMOTION-COMPLETE shows the completed vMotion operation. After the proxy accepts the new virtual machine instance as a peer, the ESX server on Host B sends a `VMOTION-COMPLETE` message to the proxy. The message indicates that the proxy should use the new connection for all traffic to the serial port. Now, the proxy can terminate the original telnet connection to Host A.

```
IAC SB VMWARE-TELNET-EXT VMOTION-COMPLETE sequence IAC SE
```

Figure 658: VMOTION-COMPLETE



VMware Telnet Extension Commands for Proxy Communication

Command Syntax

The VMware telnet extension defines a telnet proxy option for sending notification of vSphere vMotion events to a virtual serial port proxy. A VMware telnet extension command sequence uses the following general format:

```
IAC telnet-command VMWARE-TELNET-EXT [ext-command [parameters] IAC SE]
```

Where

- `IAC` is the telnet escape character prefix (Interpret as Command).
- `telnet-command` is one of the following telnet commands: `WILL`, `WON'T`, `DO`, `DON'T`, and `SB`.
- `VMWARE-TELNET-EXT` is the VMware telnet extension command.

- `ext-command` is a VMware telnet extension command that indicates one of the extension suboption commands. A suboption command sequence begins with the `VMWARE-TELNET-EXT` command and is enclosed within `IAC SB ... IAC SE` telnet subnegotiation commands.
- `parameters` indicates one or more parameter values.
- `IAC SE` is the telnet sequence for completing a subnegotiation command sequence.

The following sections contain tables of VMware telnet extension commands. A table entry shows the command, the corresponding integer code, a description, and the command and code sequences. A code sequence for a VMware telnet extension command has the following characteristics.

- The code sequence begins with code 255, the telnet escape character prefix IAC (Interpret as Command).
- VMware telnet extension commands use two- or three-byte code sequences.
- The `VMWARE-TELNET-EXT` command code is 232.
- Subnegotiation command sequences include begin (`SB`) and end (`SE`) codes (250 and 240).

For example, the `DO-PROXY` command requires the following code sequence with embedded arguments for direction (“S” for server) and URI:

```
IAC SB  VMWARE-TELNET-EXT DO-PROXY direction         uri           IAC SE
255 250       232           70       "S" "telnet://example.com" 255 240
[R]VM
```

VMware Telnet Extension Option

Table 1927: lists the commands that establish the VMware telnet extension communication between a virtual machine and a proxy. The virtual machine initiates the connection by sending a `WILL VMWARE-TELNET-EXT` message. If the proxy accepts by responding with a `DO VMWARE-TELNET-EXT` message, the virtual machine and the proxy can continue negotiation.

Table 1927:

Command	Codes

`WILL VMWARE-TELNET-EXT`
A virtual machine request indicating that it is willing to send VMware telnet extension messages to the proxy.?
`IAC WILL VMWARE-TELNET-EXT`
255 251 232
	251 232

`WON’T VMWARE-TELNET-EXT`
A virtual machine notification that it is unwilling to send VMware telnet extension messages to the proxy. The virtual machine can use this command to turn off the extension option. To resume message exchange, the virtual machine and proxy must negotiate to turn on the option again (`WILL VMWARE-TELNET-EXT` and `DO VMWARE-TELNET-EXT`).?
`IAC WON’T VMWARE-TELNET-EXT`
255 252 232
	252 232

`DO VMWARE-TELNET-EXT`
A proxy response that indicates it is willing to receive VMware telnet extension messages from the virtual machine.
`IAC DO VMWARE-TELNET-EXT`
255 253 232
	253 232

Command	Codes
DON'T VMWARE-TELNET-EXT A proxy response that indicates it is unwilling to receive VMware telnet extension messages. The proxy can send this message at any time to stop remote serial port communication. To resume message exchange, the virtual machine and proxy must negotiate to turn on the option again (WILL VMWARE-TELNET-EXT and DO VMWARE-TELNET-EXT). IAC DON'T VMWARE-TELNET-EXT 255 254 232	254 232

Option Subnegotiation

The VMWARE-TELNET-EXT telnet option requires various suboptions to support remote serial port access. The virtual machine and proxy use subnegotiation to indicate the command sets that they can support. Subnegotiation identifies all commands for negotiation as well as commands for proxy support, vMotion notification, and virtual machine identification. Table 2-2 lists the commands for option subnegotiation.

Command	Code
KNOWN-SUBOPTIONS-1 suboptions A virtual machine notification that indicates the VMWARE-TELNET-EXT suboptions that it can generate and accept. *suboptions* is a sequence of one or more codes, each of which corresponds to a VMware telnet extension command. If the virtual machine sends this message more than once during a session, the list of suboptions is always the same. The presence of a command in the list does not imply that the virtual machine will use the command during the session. IAC SB VMWARE-TELNET-EXT KNOWN-SUBOPTIONS-1 suboptions IAC SE 255 250 232 0 suboptions 255 240	0
KNOWN-SUBOPTIONS-2 suboptions A proxy response that indicates the VMWARE-TELNET-EXT suboptions that it can generate and accept. *suboptions* is a sequence of one or more codes, each of which corresponds to a VMware telnet extension command. The proxy must support this command. If the proxy sends this message more than once during a session, the list of suboptions must be the same each time. IAC SB VMWARE-TELNET-EXT KNOWN-SUBOPTIONS-2 suboptions IAC SE 255 250 232 1 suboptions 255 240	1

Unknown Command Response

Table lists the commands to indicate reception of an unknown command. The proxy must send UNKNOWN-SUBOPTION-RCVD-2 any time it receives a command that it does not support.

Command	Code
UNKNOWN-SUBOPTION-RCVD-1 code A virtual machine response to indicate that it received an unknown suboption command from the proxy. *code* identifies the unknown suboption. IAC SB VMWARE-TELNET-EXT UNKNOWN-SUBOPTION-RCVD-1 code IAC SE 255 250 232 2 code 255 240	2
UNKNOWN-SUBOPTION-RCVD-2 code A proxy response to indicate that it received an unknown suboption command from the virtual machine. *code* identifies the unknown suboption. The proxy must support this command. IAC SB VMWARE-TELNET-EXT UNKNOWN-SUBOPTION-RCVD-2 code IAC SE 255 250 232 3 code 255 240	3

Proxy Operation

Table lists the extension commands to negotiate proxy support.

Command	Code
DO-PROXY direction serviceUri A virtual machine request for the proxy to do forwarding operations over the connection. The virtual machine sends a DO-PROXY request before any content is exchanged between the virtual machine and the remote system. direction and serviceURI correspond to the VirtualSerialPortURIBackingInfo properties in the virtual serial port configuration. (See the *vSphere API Reference* for information about the VirtualSerialPortURIBackingInfo data object.) direction is either “C” (client) or “S” (server). If the direction indicates that the virtual machine is acting as a client, serviceUri identifies a remote system and the proxy initiates a connection with the remote system. If the direction indicates that the virtual machine is acting as a server, serviceUri identifies the network socket on the host on which the virtual machine runs. In this case, the virtual machine will accept a connection initiated by a remote system (using the proxy). If possible, the proxy should also handle port control options, as specified by?RFC 2217. IAC SB VMWARE-TELNET-EXT DO-PROXY direction uri IAC SE 255 250 232 70 direction uri 255 240	70
WILL-PROXY A proxy response to indicate that it will do forwarding operations. IAC SB VMWARE-TELNET-EXT WILL-PROXY IAC SE 255 250 232 71 255 240	71
WON’T-PROXY A proxy response to indicate that it will not do forwarding operations. IAC SB VMWARE-TELNET-EXT WON’T-PROXY IAC SE 255 250 232 73 255 240	73

vMotion Notification

Table lists the commands that hosts and proxy use during a vMotion operation. In the following descriptions, the *source* host contains the original virtual machine instance, and the *target* host is the host on which the new instance is created. A host uses the virtual machine serial port connection with the proxy to send vMotion messages.

Command	Code

`VMOTION-BEGIN sequence`
A source host request to indicate that a vMotion operation is pending. The operation requires a VMOTION-GOAHEAD response to start. The intent is to flush all pending data transmission on the connection.
The host provides an opaque `sequence` value. All subsequent vMotion messages related to the connection between the virtual machine and this proxy use this sequence value.
`IAC SB VMWARE-TELNET-EXT VMOTION-BEGIN sequence IAC SE`
`255 250 232 40 sequence 255 240`
	40

`VMOTION-GOAHEAD sequence secret`
A proxy response to indicate that all queued data has been transmitted, and that the vMotion operation can start. The proxy sends this message only in response to a `VMOTION-BEGIN` request.
The proxy returns the sequence value that it received in the `VMOTION-BEGIN` request. The proxy also creates and returns an opaque secret value. The virtual machine uses the sequence and secret to identify itself when it establishes a proxy connection from the new host. See the description of `VMOTION-PEER`.
`IAC SB VMWARE-TELNET-EXT VMOTION-GOAHEAD sequence secret IAC SE`
`255 250 232 41 sequence secret 255 240`
	41

`VMOTION-NOTNOW sequence`
A proxy response to indicate that the vMotion operation should not start at this time. This message includes the sequence value that the proxy obtained in the `VMOTION-BEGIN` request. The proxy can send this message only in response to a `VMOTION-BEGIN` request.
`IAC SB VMWARE-TELNET-EXT VMOTION-NOTNOW sequence IAC SE`
`255 250 232 43 sequence 255 240`
	43

`VMOTION-PEER sequence secret`
A target host request to indicate that this connection belongs to the virtual machine associated with `sequence` and `secret`. The sequence and secret identify the duplicate connections between the proxy and virtual machine instances during a vMotion operation.
The original host generates the sequence value for the `VMOTION-BEGIN` message. The proxy generates the secret value for the `VMOTION-GOAHEAD` message.
`IAC SB VMWARE-TELNET-EXT VMOTION-PEER sequence secret IAC SE`
`255 250 232 44 sequence secret 255 240`
	44

`VMOTION-PEER-OK sequence`
A proxy response to indicate that the secret in the `VMOTION-PEER` message was accepted. The proxy sends this message only in response to a `VMOTION-PEER` request.
`IAC SB VMWARE-TELNET-EXT VMOTION-PEER-OK sequence IAC SE`
`255 250 232 45 sequence 255 240`
	45

`VMOTION-COMPLETE sequence`
A target host notification that the vMotion operation is complete. The proxy must send all subsequent communication for the virtual machine on the connection associated with this `VMOTION-COMPLETE` message. The host sends this message only after sending a `VMOTION-PEER` message.
`IAC SB VMWARE-TELNET-EXT VMOTION-COMPLETE sequence IAC SE`
`255 250 232 46 sequence 255 240`
	46

Command	Code
VMOTION-ABORT A source host notification that the vMotion operation failed. The proxy must send all communication for the virtual machine on the connection associated with this VMOTION-ABORT message. The host can send VMOTION-ABORT only if it has sent VMOTION-BEGIN previously on this connection. IAC SB VMWARE-TELNET-EXT VMOTION-ABORT IAC SE 255 250 232 48 255 240	48

Virtual Machine Identification

Table lists the suboption commands for virtual machine identification.

Command	Code
VM-VC-UUID vm-uuid A virtual machine response that contains its universally unique identifier. IAC SB VMWARE-TELNET-EXT VM-VC-UUID vm-uuid IAC SE 255 250 232 80 vm-uuid 255 240	80
GET-VM-VC-UUID A proxy request for the virtual machine's universally unique identifier. IAC SB VMWARE-TELNET-EXT GET-VM-VC-UUID IAC SE 255 250 232 81 255 240	81
VM-NAME vm-name A virtual machine response that contains its name. IAC SB VMWARE-TELNET-EXT VM-NAME vm-name IAC SE 255 250 232 82 vm-name 255 240	82
GET-VM-NAME A proxy request for the virtual machine's name. IAC SB VMWARE-TELNET-EXT GET-VM-NAME IAC SE 255 250 232 83 255 240	83
VM-BIOS-UUID vm-uuid A virtual machine response that contains its universally unique identifier stored in the virtual machine BIOS. IAC SB VMWARE-TELNET-EXT VM-BIOS-UUID-IS vm-uuid IAC SE 255 250 232 84 vm-uuid 255 240	84
GET-VM-BIOS-UUID A proxy request for the virtual machine's universally unique identifier stored in its BIOS. IAC SB VMWARE-TELNET-EXT GET-VM-BIOS-UUID IAC SE 255 250 232 85 255 240	85
VM-LOCATION-UUID vm-uuid A virtual machine response that contains its universally unique identifier. IAC SB VMWARE-TELNET-EXT VM-LOCATION-UUID vm-uuid IAC SE 255 250 232 86 vm-uuid 255 240	86
GET-VM-LOCATION-UUID A proxy request for the virtual machine's location universally unique identifier. IAC SB VMWARE-TELNET-EXT GET-VM-LOCATION-UUID IAC SE 255 250 232 87 255 240	87

VMware Cloud Foundation PowerCLI

VMware Cloud Foundation PowerCLI is a suite of Microsoft PowerShell cmdlets for automating workflows in vCenter and ESX, as well as VMware Live Site Recovery™, SDDC Manager, vSAN, VMware Cloud Director, Update Manager, VMware Cloud Foundation Operations, VMware Horizon, NSX, VMware HCX, and VMware Cloud on AWS. VMware Cloud Foundation PowerCLI provides access to both high-level product features and low-level granular API access.

Compatibility Matrix for VMware Cloud Foundation PowerCLI

This document lists all changes introduced in the following VMware Cloud Foundation PowerCLI releases:

Compatibility Matrix for VCF PowerCLI 9.1.1

Last Updated on: 25 August 2026

The *Compatibility Matrix for VCF PowerCLI 9.1.1* describe the compatibility between VCF PowerCLI 9.1.1 and platforms, guest operating systems, PowerShell versions, and other VMware solutions. Where compatibility information for VCF PowerCLI exists in the Broadcom Product Interoperability Matrix, the present matrixes describe how to use this tool to find the relevant information. Where the Broadcom Product Interoperability Matrix do not provide information relating to VCF PowerCLI, this information is listed here.

Local operating system support

You can install VCF PowerCLI 9.1.1 on the following operating systems:

OS Type	64-Bit
Server	Windows Server
Workstation	• Windows • Linux • macOS

The local operating system needs to support required .NET and PowerShell versions (check the *Installation Prerequisites* section for more details).

Guest operating system support

You can run VCF PowerCLI 9.1.1 guest cmdlets against virtual machines with installed VMware Tools. Check the release notes of the specific VMware Tools version for a list of supported operating systems.

> **Note:**
> Guest cmdlets are not compatible with IPv6 environments.

Supported PowerShell versions

VCF PowerCLI 9.1.1 is compatible with the following PowerShell versions:

- > **Note:**
  >
  > Windows PowerShell 5.1 is deprecated in VCF PowerCLI 9.0.
- PowerShell 7.0.13 +

Installation Prerequisites

Verify that the following software is present on your system to be able to install and use VCF PowerCLI 9.1.1:

OS Type	PowerShell Version
Windows	**Note:** Windows PowerShell 5.1 is deprecated in VCF PowerCLI 9.0. PowerShell 7.0.13 +
Linux	PowerShell 7.0.13 +
macOS	PowerShell 7.0.13 +

Note:

PowerShell bundles its own .NET runtime. You do not need to install .NET separately. The .NET version is determined by the PowerShell version you install.

Some VCF PowerCLI functionalities that are listed after these have additional prerequisites:

Functionality	Prerequisite
VMware.ImageBuilder module	Python 3.7 -3.14 or newer and the following additional P
Open-VMConsoleWindow cmdlet	VMware Remote Console (VMRC)

Interoperability with VMware Solutions

Refer to the VMware Product Interoperability Matrix for more inforamtion on the interoperability of VCF PowerCLI with vCenter, ESX, vSAN, VMware Live Site Recovery, VMware vSphere Replication, and VMware Cloud Foundation Operations.

1. Go to https://interopmatrix.broadcom.com/Interoperability.
2. Make sure **Interoperability** is selected.
3. From the **Select a Solution** drop-down menu, select **VCF PowerCLI**.
4. From the **Version** drop-down menu, select **9.1.1**.
5. From the **Add Platform/Solution** drop-down menu, select a VMware solution, product, or feature, for example, **VMware vCenter** or **VMware ESX**.
6. If you want to add more solutions, click **Add Another Solution**.

Disclaimer

THIS CONTENT IS PROVIDED "AS-IS," AND TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, VMWARE DISCLAIMS ALL OTHER REPRESENTATIONS AND WARRANTIES, EXPRESS OR IMPLIED, REGARDING THIS CONTENT, INCLUDING THEIR FITNESS FOR A PARTICULAR PURPOSE, THEIR MERCHANTABILITY, OR THEIR NONINFRINGEMENT. VMWARE SHALL NOT BE LIABLE FOR ANY DAMAGES ARISING OUT OF OR IN CONNECTION WITH THE USE OF THIS CONTENT, INCLUDING DIRECT, INDIRECT, CONSEQUENTIAL DAMAGES, LOSS OF BUSINESS PROFITS OR SPECIAL DAMAGES, EVEN IF VMWARE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Updated Information

This section is updated with each release of the product or when necessary.

This table tracks the updated information of the *VCF PowerCLI User's Guide* starting from the 12.4.0 release.

PowerCLI Version / Publish Date	Description
9.1.1.0 / 14 SEP 2026	Release of VMware Cloud Foundation PowerCLI 9.1.1.
9.1.0.0 / 12 MAY 2026	Release of VMware Cloud Foundation PowerCLI 9.1.
9.0.0.0 / 17 JUNE 2025	First release of VMware Cloud Foundation PowerCLI 9.0.
13.3.0 / 1 OCTOBER 2024	Updated the information about how to connect to vCenter federated to Active Directory Federation Services (AD FS) or to another external identity provider managed by VMware Identity Services. See Connecting to a vCenter System
13.2.0 / 27 NOVEMBER 2023	• Added two new topics to Managing VMware vSAN with VMware Cloud Foundation PowerCLI: – Create a vSAN ESA-Enabled Cluster – Mount and Unmount Remote vSAN Datastores
13.2.0 / 08 NOVEMBER 2023	• Added information about non-disruptive Machine SSL certificate renewal. See Change the Machine SSL Certificate of vCenter. • Revamped and reorganized Managing vSphere with VMware Cloud Foundation PowerCLI. • Updated Installing and Configuring Python for VCF PowerCLI and its subtopics. • Added the new modules, `VMware.Sdk.Vcf.*` to VCF PowerCLI Sub-Modules.
13.1.0 / 04 AUGUST 2023	• Added a new chapter: Managing vSphere Lifecycle Manager Images and Baselines with VMware Cloud Foundation PowerCLI. • Optimized the VMware Cloud Foundation PowerCLI chapter.
13.1.0 / 27 JUNE 2023	• Added a new chapter: Managing the VMware Live Site Recovery API with VMware Cloud Foundation PowerCLI. • Added two new modules, `VMware.Sdk.Vr` and `VMware.Sdk.Srm` to VCF PowerCLI Sub-Modules. • Reorganized the chapter structure of the book.
13.1.0 / 22 MAY 2023	• Added a new chapter: Managing the vSphere Replication API with VMware Cloud Foundation PowerCLI.
13.0.0 / 22 NOV 2022	• Added a new chapter: Installing and Configuring Python for VCF PowerCLI. Starting from VCF PowerCLI 13.0, Python is a requirement for the `VMware.ImageBuilder` module.
12.7.0 / 07 NOV 2022	• Added a new chapter: Managing the NSX Policy API with VMware Cloud Foundation PowerCLI.
12.7.0 / 08 AUG 2022	• Added information on how to connect to a vCenter that is federated to Active Directory Federation Services (AD FS). See Connect to a vCenter System Configured to Use AD FS for an External Identity Provider.
12.6.0 / 07 APR 2022	• Removed non-inclusive language terms from the *VCF PowerCLI User's Guide* and replaced them with appropriate alternatives. • Added a new module, `VMware.Sdk.Nsx.Policy`, to VCF PowerCLI Sub-Modules.
12.5.0 / 13 JAN 2022	• Added information on how to manage the certificates of your vSphere environment with VMware Cloud Foundation PowerCLI. See the new chapter Managing Certificates.
12.4.0 / 28 SEP 2021	• Added an Updated Information topic. • Updated the procedure for uninstalling VCF PowerCLI from your system. See Uninstall VCF PowerCLI.

PowerCLI Version / Publish Date	Description
12.4.0 / 16 SEP 2021	• Added information about the new VMware Cloud Foundation PowerCLI modules for the 12.4.0 release. See VCF PowerCLI Sub-Modules. • Updated the chapter on managing VMware Cloud on AWS with VCF PowerCLI and added many new scripts. See Managing VMware Cloud on AWS with VMware Cloud Foundation PowerCLI. • Added information, procedures, and sample scripts for the vSphere Automation SDK for PowerShell, which was introduced in VMware Cloud Foundation PowerCLI 12.4.0. See Managing the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI.

Microsoft PowerShell Basics

VCF PowerCLI is based on Microsoft PowerShell and uses the PowerShell basic syntax and concepts.

Microsoft PowerShell is both a command-line and scripting environment. It uses the .NET object model and provides administrators with system administration and automation capabilities. To work with PowerShell, you run commands, named cmdlets.

PowerShell Command-Line Syntax

PowerShell cmdlets use a consistent verb-noun structure, where the verb represents the action and the noun represents the object to operate on.

PowerShell cmdlets follow consistent naming patterns, ensuring that construction of a command is easy if you know the object that you want to work with.

All command categories take parameters and arguments. A parameter starts with a hyphen and is used to control the behavior of the command. An argument is a data value consumed by the command.

A simple PowerShell command has the following syntax:

```
command -parameter1 -parameter2 argument1, argument2
```

PowerShell Pipelines

A pipeline is a series of commands separated by the pipe operator |.

Each command in the pipeline receives an object from the previous command, performs some operation on it, and then passes it to the next command in the pipeline. Objects are output from the pipeline as soon as they become available.

PowerShell Wildcards

PowerShell has a number of pattern-matching operators named wildcards that you can use to substitute one or more characters in a string, or substitute the complete string.

All wildcard expressions can be used with the VCF PowerCLI cmdlets. For example, you can view a list of all files with a `.txt` extension by running `dir *.txt`. In this case, the asterisk * operator matches any combination of characters.

With wildcard patterns you can indicate character ranges as well. For example, to view all files that start with the letter S or T and have a `.txt` extension, you can run `dir [st]*.txt`.

You can use the question mark ? wildcard to match any single character within a sequence of characters. For example, to view all `.txt` files with names that consist of `string` and one more character at the end, run `dir string?.txt`.

PowerShell Common Parameters

The PowerShell engine retains a set of parameter names, referred to as common parameters. All PowerShell cmdlets, including the VCF PowerCLI cmdlets, support them.

Some of the PowerShell common parameters are `Verbose`, `Debug`, `ErrorAction`, `ErrorVariable`, `OutVariable`, and `OutBuffer`. For a full list of the common parameters and more details on their usage, run `Get-Help about_CommonParameters`.

PowerShell offers two risk mitigation parameters: `WhatIf` and `Confirm`.

WhatIf

Displays the effects of a command without running it.

Confirm

Prompts for confirmation before running a command that stops a program or service, or deletes data.

VCF PowerCLI Concepts

VCF PowerCLI cmdlets are designed to automate VMware environment administration and introduce specific features in addition to PowerShell concepts.

VCF PowerCLI Sub-Modules

VMware Cloud Foundation PowerCLI comprises multiple modules that you can install and use all together or separately, tailored to your specific needs and environments.

The following table lists all official VMware Cloud Foundation PowerCLI modules.

Module	Description
`VMware.PowerCLI`	A root module that third-party modules can depend on. This module ensures the VCF PowerCLI product can be installed, upgraded, and removed as a complete package if needed.
`VMware.VimAutomation.Core`	Provides cmdlets for vSphere environment administration.
`VMware.VimAutomation.Common`	Provides internal utilities. This module is required for other VCF PowerCLI modules to function correctly.
`VMware.VimAutomation.Sdk`	Provides SDK functionality that is needed by all other VCF PowerCLI modules. This module is required for other modules to function correctly.
`VMware.VimAutomation.Vds`	Provides cmdlets for managing vSphere distributed switches and distributed port groups.
`VMware.VimAutomation.Vpc`	Provides cmdlets for managing VPCs.
`VMware.VimAutomation.Cis.Core`	Provides cmdlets for managing vCenterservers through their vSphere Automation API.
`VMware.VimAutomation.Storage`	Provides cmdlets for managing vSAN and vSphere policy-based storage.
`VMware.VimAutomation.StorageUtility`	Provides utility scripts for storage.
`VMware.VimAutomation.License`	Provides the `Get-LicenseDataManager` cmdlet for managing VMware License components.
`VMware.ImageBuilder`	Provides cmdlets for managing depots, image profiles, and VIBs.
`VMware.DeployAutomation`	Provides cmdlets that provide an interface to VMware Auto Deploy for provisioning physical hosts with ESX software.
`VMware.VimAutomation.Cloud`	Provides cmdlets for VMware Cloud Director automation.
`VMware.VumAutomation`	Provides cmdlets for Update Manager automation.
`VMware.VimAutomation.vROps`	Provides cmdlets for VMware Cloud Foundation Operations automation.
`VMware.VimAutomation.Srm`	Provides cmdlets for VMware Live Site Recovery™ management.
`VMware.VimAutomation.HorizonView`	Provides cmdlets for VMware Horizon automation.
`VMware.VimAutomation.Nsxt`	Provides cmdlets for NSX server management.

Module	Description
VMware.VimAutomation.Vmc	Provides cmdlets for VMware Cloud on AWS automation.
VMware.Vim	Provides low-level bindings for the vCenter Management API.
VMware.VimAutomation.Security	Provides cmdlets for managing vSphere Security, including virtual Trusted Platform Module.
VMware.VimAutomation.Hcx	Provides cmdlets for VMware HCX management.
VMware.VimAutomation.WorkloadManagement	Provides cmdlets for managing VMware vSphere Supervisor features.
VMware.Vcf.Sso	Provides cmdlets for managing VCF SSO features.
VMware.CloudServices	Provides cmdlets for VMware Cloud Services management.
VMware.Sdk.vSphere	Provides PowerShell-style bindings for the vCenter Automation API management.
VMware.Sdk.Nsx.Policy.*	Provides PowerShell-style bindings for the VMware NSX server's Policy API management. • VMware.Sdk.Nsx.Policy • VMware.Sdk.Nsx.Policy.GlobalInfra • VMware.Sdk.Nsx.Policy.Infra • VMware.Sdk.Nsx.Policy.Initialize
VMware.Sdk.Vr	Provides PowerShell-style bindings for the the vCenter vSphere Replication API management.
VMware.Sdk.Srm	Provides PowerShell-style bindings for the the vCenter VMware Live Site Recovery API management.
VMware.Sdk.Vcf.*	Provide PowerShell-style bindings for the VMware Cloud Foundation management. • VMware.Sdk.Vcf.SddcManager • VMware.Sdk.Vcf.Installer • VMware.Sdk.Vcf.CloudBuilder • VMware.Sdk.Vcf.Ops

Retrieving vSphere Inventory Objects from Cloud Resources

You can retrieve vSphere inventory objects from cloud resources by using the `RelatedObject` parameter of VCF PowerCLI cmdlets. This interoperability expands your cloud administration, automation, reporting, and troubleshooting options.

You can use the `RelatedObject` parameter of VCF PowerCLI cmdlets to retrieve vSphere inventory objects from VMware Cloud Director objects. Passing the retrieved objects to the cmdlets of the `VMware.VimAutomation.Core` and `VMware.VimAutomation.Vds` modules, extends your administration options.

> **Note:**
> Use of the `VMware.VimAutomation.Core` and `VMware.VimAutomation.Vds` modules to modify the configuration of objects that are managed by VMware Cloud Director might result in unpredictable behavior of the cloud environment.

Table 1928: List of Supported vSphere Inventory Objects You Can Retrieve from Cloud Objects

Cloud Object	Retrieved vSphere Inventory Object	Sample Script for Retrieving the vSphere Inventory Object
ProviderVdc	Datastore	`Get-ProviderVdc -Name 'MyProviderVdc' | Get-Datastore`
CIVM	VirtualMachine	`Get-CIVM -Name 'MyCloudVM' | Get-VM`
NetworkPool	VDSwitch	`Get-NetworkPool -Name 'MyNetworkPool' | Get-VDSwitch`
NetworkPool	VDPortgroup	`Get-NetworkPool -Name 'MyNetworkPool' | Get-VDPortGroup`
ExternalNetwork	VDPortgroup	`Get-ExternalNetwork -Name 'MyExternalNetwork' | Get-VDPortGroup`

Selecting Objects in VCF PowerCLI

You can pass strings and wildcards to all parameters that take inventory objects, datastores, `OSCustomizationSpec` objects, and `VIServer` objects as arguments. VCF PowerCLI then transforms the strings into PowerShell objects. This approach is called Object-by-Name (OBN) selection.

Using OBN

For example, if you pass `Start-VM -VM "myVM"`, VCF PowerCLI trasforms the string parameter, "myVM", into a PowerShell object. This is how OBN works.

For more details about OBN, run `help about_OBN`.

Using pipelines and variables

Instead of assigning an object name to a cmdlet parameter, you can pass the object through a pipeline or a variable. For example, the following three commands are interchangeable:

- `Remove-VM -VM "Win 7 SP1"`
- `Get-VM -Name "Win 7 SP1" | Remove-VM`
- `Remove-VM -VM (Get-VM -Name "Win 7 SP1")`

> **Note:**
> In VCF PowerCLI, passing strings as pipeline input is not supported.

An OBN failure

If you provide a non-existing object name, an OBN failure occurs. In such cases, VCF PowerCLI generates a non-terminating error and runs the cmdlet ignoring the invalid name.

This example illustrates the occurrence of an OBN failure.

```
Set-VM –VM “VM1”, “VM2”, “VM3” –Server $server1, $server2 –MemoryGB 4
```

If the *VM2* virtual machine does not exist on either of the selected servers, VCF PowerCLI generates a non-terminating error and applies the command only on the *VM1* and *VM3* virtual machines.

Processing Non-alphanumeric Characters in VCF PowerCLI

When you provide login credentials in the command prompt or in a script file, a PowerShell limitation might prevent VCF PowerCLI from processing non-alphanumeric characters correctly. To prevent login problems, escape the non-alphanumeric characters in your credentials.

To escape non-alphanumeric characters in VCF PowerCLI, you need to place the expression that contains them in single quotes (').

Note:
When you provide your login credentials in the Specify Credential dialog box, you do not need to escape non-alphanumeric characters.

Connecting to a vCenter System

This example illustrates how to escape non-alphanumeric characters when connecting to a selected vCenter instance with the `Adminis!ra!or` user name and the `pa$$word` password.

```
Connect-VIServer -Server 10.23.112.235 -Protocol https -Username 'Adminis!ra!or'
-Password 'pa$$word'
```

Running VCF PowerCLI Cmdlets Asynchronously

By default, VCF PowerCLI cmdlets return an output only after completion of the requested tasks. If you want a cmdlet to return to the command line immediately, without waiting for the tasks to complete, you can use the `RunAsync` parameter.

When you use the `RunAsync` parameter, the cmdlet returns `Task` objects instead of its usual output. The `Status` property of a returned `Task` object contains a snapshot of the initial state of the task. This state is not updated automatically and has the values Error, Queued, Running, or Success. You can refresh a task state by retrieving the task object with the `Get-Task` cmdlet. If you want to observe the progress of a running task and wait for its completion before running other commands, use the `Wait-Task` cmdlet.

Note:
In VCF PowerCLI, the `RunAsync` parameter affects only the invocation of a cmdlet and does not control whether the initiated tasks run consecutively or in parallel. For example, the `Remove-VM` cmdlet might remove the selected virtual machines simultaneously or consecutively depending on the internal design of VCF PowerCLI. To make sure that tasks initiated by a cmdlet run consecutively, run the cmdlet in a loop, each time applying it to a single object.

Running Remove-VM with and without the RunAsync parameter

```
Remove-VM $vmList
```

The command returns no output when all virtual machines stored in the *$vmList* variable are removed, irrespective of whether they are removed simultaneously.

```
Remove-VM $vmList -RunAsync
```

The command returns an output that consists of one or more `Task` objects immediately.

Managing Default Server Connections in VCF PowerCLI

By default, VCF PowerCLI cmdlets run on the vCenter or VMware Cloud Director systems you are connected to, if no target servers can be determined from the provided parameters. Use the *$DefaultVIServers* and *$DefaultCIServers* variables to manage the default server connections.

When you connect to a vCenter system by using `Connect-VIServer`, the server connection is stored in the *$DefaultVIServers* array variable. This variable contains all connected servers for the current VCF PowerCLI session. To remove a server from the *$DefaultVIServers* variable, you can either use `Disconnect-VIServer` to close all active connections to this server, or modify the value of *$DefaultVIServers* manually.

When you connect to a VMware Cloud Director system by using `Connect-CIServer`, the server connection is stored in the *$DefaultCIServers* array variable. This variable contains all connected servers for the current session. To remove a server from the *$DefaultCIServers* variable, you can either use `Disconnect-CIServer` to close all active connections to this server, or modify the value of *$DefaultCIServers* manually.

Customization Specification Objects in VCF PowerCLI

You can use customization specifications to define the configuration settings and parameters for customizing virtual machines during the deployment or cloning process. VCF PowerCLI provides two types of objects for customization specification: persistent and nonpersistent.

Persistent Customization

Persistent customization specification objects are stored on the vSphere server. All persistent customization specifications created by using vSphere Client or VMware Cloud Foundation PowerCLI 4.1 or later are encrypted. Encrypted customization specifications can be applied only by the server that has encrypted them.

Nonpersistent Customization

Nonpersistent customization specification objects exist only inside the current PowerShell process. Nonpersistent customization specification objects are not encrypted, but cloning them to a vSphere server encrypts them.

Using ESXCLI with VCF PowerCLI

With VCF PowerCLI you can invoke ESXCLI commands within your PowerShell session and manage your ESX hosts.

You can use VCF PowerCLI to access ESXCLI in the following ways:

By Using the Get-ESXCli cmdlet

Use the `Get-ESXCli` cmdlet to invoke ESXCLI commands within your PowerShell console.

For example:

```
$vmHost = Get-VMHost <vmHostIp>
$esxcli = Get-EsxCli -VMHost $vmHost -V2
$esxcli.storage.nmp.device.list.Invoke()
```

For more information, see `Get-EsxCli` in the PowerCLI Reference.

By Using .NET Methods

You can use .NET methods to create managed objects that correspond to specific ESXCLI applications. To access the ESXCLI, you can call methods on these managed objects.

> **Note:**
> To call a method of an ESXCLI object, you must provide values for all parameters. If you want to omit a given parameter, pass *$null* as its argument.

Using the VCF PowerCLI About Articles

Learn about VCF PowerCLI concepts and features from the built-in help articles named *About Articles*. You can access them through a running VCF PowerCLI process.

Running `Help About_*` lists all built-in PowerShell and VMware Cloud Foundation PowerCLI*About Articles*.

Table 1929: Built-In *About Articles* for VCF PowerCLI

Article Title	Command	Article Description
Customer Experience Improvement Program (CEIP)	`Help About_CEIP`	Provides information about VMware's Customer Experience Improvement Program ("CEIP").
Handling Invalid Certificates	`Help About_Invalid_Certificates`	When you connect to a server, VCF PowerCLI checks if the server certificate is valid. If the certificate is not trusted for this server, by default, VCF PowerCLI fails to connect to the server.
Object-by-Name (OBN)	`Help About_OBN`	To help you save time and effort, VCF PowerCLI lets you select objects by their names.
VCF PowerCLI Objects	`Help About_PowerCLI_Objects`	For their input and output, the VCF PowerCLI cmdlets use a set of .NET types that reside in the `VMware.VimAutomation.ViCore.Types` namespace.
Using the RunAsync Parameter	`Help About_RunAsync`	When you set the `RunAsync` parameter, you indicate that you want to run the cmdlet asynchronously.
Authenticating with a vCenter System or a VMware Cloud Director Server	`Help About_Server_Authentication`	To authenticate with vCenter and VMware Cloud Director servers, you can provide a user name and password through the `User` and `Password` parameters, or a `PSCredential` object through the `Credential` parameter.
Unique Identifiers for PowerCLI Objects (UID)	`Help About_UID`	You can uniquely identify a VCF PowerCLI object on a server or across multiple servers by providing its UID.
Datastore Provider (VimDatastore)	`Help About_VimDatastore`	The Datastore Provider (`VimDatastore`) provides a filesystem-style view and access to the contents of datastores.
LicenseDataManager	`Help About_LicenseDataManager`	The `LicenseDataManager` component lets you extend the vCenter inventory with license data.

Installing VMware Cloud Foundation PowerCLI

VMware Cloud Foundation PowerCLI lets you manage, monitor, automate, and handle lifecycle operations on VMware vSphere, VMware Cloud Foundation Operations, vSAN, VMware Cloud Director, Update Manager, NSX, VMware Cloud on AWS, VMware HCX, VMware Horizon, and VMware Site Recovery Manager (SRM) systems. You can install VMware Cloud Foundation PowerCLI modules on all supported operating systems.

After installing the package on your machine, you can connect to your ESX, vCenter, VMware Cloud Foundation Operations, VMware Cloud Director, NSX, VMware Cloud on AWS, VMware Horizon, or VMware Site Recovery Manager (SRM) system by providing valid authentication credentials.

Supported Operating Systems

You can install VCF PowerCLI on supported operating systems. You can run guest cmdlets against virtual machines on which supported guest operating systems are installed.

PowerCLI Local Operating Systems

For a list of operating systems on which you can install VMware Cloud Foundation PowerCLI, see the Compatibility Matrix on the PowerCLI home page.

PowerCLI Guest Operating Systems

You can run VMware Cloud Foundation PowerCLI guest cmdlets against virtual machines with supported guest operating systems. For a list of supported operating systems, see the Compatibility Matrix on the PowerCLI home page.

> **Note:**
> Guest cmdlets are not compatible with IPv6 environments.

Supported VMware Products

You can use the VCF PowerCLI modules to manage all supported VMware products.

For a list of VMware products with which VMware Cloud Foundation PowerCLI is compatible, see VMware Product Interoperability Matrixes.

Supported PowerShell Versions

VCF PowerCLI is compatible with multiple versions of PowerShell.

For a list of PowerShell versions with which VMware Cloud Foundation PowerCLI is compatible, see the Compatibility Matrix on the PowerCLI home page.

Install VCF PowerCLI

You can download and install VCF PowerCLI modules from the PowerShell Gallery. You can install all VCF PowerCLI modules with a single PowerShell command, or install modules individually.

- Verify that your system is compatible with VCF PowerCLI. See the Compatibility Matrix for VMware Cloud Foundation PowerCLI.
- Verify that your system is connected to the Internet.
- Verify that PowerShell is available on your system. For Linux and macOS, you must install PowerShell. See how to install PowerShell on different platforms.

Use the `Install-Module` cmdlet to download and install VCF PowerCLI modules. See the PowerShell Gallery for the complete list of available VCF PowerCLI modules.

1. Open PowerShell on your workstation.
2. To install all VCF PowerCLI modules, run the command.

   ```
   Install-Module VCF.PowerCLI -Scope CurrentUser
   ```

   Alternatively, you can install individual VCF PowerCLI modules by running the `Install-Module` cmdlet with the module name.
3. If you see a warning that you are installing modules from an untrusted repository, press `y` and then press Enter to confirm the installation.
4. Import the `VCF.PowerCLI` module by running the following command.

   ```
   Import-Module VCF.PowerCLI
   ```

Enable execution of local scripts. See Allow Execution of Local Scripts.

Install VCF PowerCLI Offline

You can install all VCF PowerCLI modules in offline mode by using a ZIP file.

- Verify that your system is compatible with VCF PowerCLI. See the Compatibility Matrix for VMware Cloud Foundation PowerCLI.
- Verify that PowerShell is available on your system. For Linux and macOS, you must install PowerShell. See how to install PowerShell on different platforms.
- Download the VCF PowerCLI ZIP file from the PowerCLI home page and transfer the ZIP file to your local machine.

You might need to install VCF PowerCLI on a local machine with no Internet connectivity due to security reasons and deployment restrictions. If you are using such an environment, you can download the VCF PowerCLI ZIP file on a computer with Internet access, transfer the ZIP file to your local machine, and install VCF PowerCLI.

1. Open PowerShell on your local machine.
2. To view the folder paths to which you can extract the VCF PowerCLI ZIP file, run the command.

   ```
   $env:PSModulePath
   ```

3. Extract the contents of the VCF PowerCLI ZIP file to one of the listed folders.
4. For Windows, run the command to unblock the copied files.

   ```
   Get-ChildItem -Path 'folder_path' -Recurse | Unblock-File
   ```

   Replace *folder_path* with the path to the folder where you extracted the contents of the ZIP file.
5. Optional: Verify that the VCF PowerCLI modules have installed successfully.

   ```
   Get-Module VMware* -ListAvailable
   ```

You can now run VCF PowerCLI on your local machine.

Enable execution of local scripts. See Allow Execution of Local Scripts.

Update VCF PowerCLI

You can update VCF PowerCLI when a new version of the product becomes available. See the PowerShell Gallery for the latest VCF PowerCLI versions.

Use the `Update-Module` cmdlet to update VCF PowerCLI modules. You can update all VCF PowerCLI modules with a single command, or update modules individually.

You cannot use the `Update-Module` cmdlet in offline mode. To update VCF PowerCLI in offline mode, you must perform a new installation. See Install PowerCLI Offline.

1. Open PowerShell.
2. To update all VCF PowerCLI modules, run the command.

   ```
   Update-Module VCF.PowerCLI -Scope CurrentUser
   ```

   Alternatively, you can update individual VCF PowerCLI modules by running the `Update-Module` cmdlet with the module name.

Uninstall VCF PowerCLI

You can uninstall VMware Cloud Foundation PowerCLI by deleting the VMware Cloud Foundation PowerCLI modules from your system.

- Close all PowerShell sessions that are running VCF PowerCLI.

To uninstall VMware Cloud Foundation PowerCLI, you must locate your VMware Cloud Foundation PowerCLI modules and delete them.

1. In PowerShell, to see the location of the `VCF.PowerCLI` modules, run the command.

```
Get-Module -ListAvailable VMware* | Select-Object -Property Name, Version, Path
```

2. In your operating system, navigate to the destination folder(s) and delete all modules that start with `VMware`.

You removed VCF PowerCLI from your system.

Configuring VMware Cloud Foundation PowerCLI

Configure how VMware Cloud Foundation PowerCLI responds to invalid server certificates, choose the application settings for different users and user groups, modify the script configuration file, and add custom scripts.

Allow Execution of Local Scripts

If you want to run scripts and load configuration files with VCF PowerCLI, you must set the execution policy of PowerShell to RemoteSigned.

For security reasons, PowerShell supports an execution policy feature. It determines whether scripts are allowed to run and whether they must be digitally signed. By default, the execution policy is set to Restricted, which is the most secure policy. For more information about the execution policy and script digital signing in PowerShell, run `Get-Help About_Signing`.

You can change the execution policy by using the `Set-ExecutionPolicy` cmdlet.

1. Open the PowerShell console.
2. Run `Set-ExecutionPolicy RemoteSigned`.

Configuring VCF PowerCLI Response to Untrusted Certificates

When you try to connect to a server, VCF PowerCLI checks whether the server certificate is valid. You might have to configure the VCF PowerCLI settings to be able to connect to servers with untrusted certificates.

You might not be able to connect to a server because of the default action configured for VCF PowerCLI and untrusted certificates.

By default, VCF PowerCLI is set to deny connections to servers with invalid certificates. You can retrieve the current setting for invalid server certificates, and configure it as required. For more information, see Configure the VCF PowerCLI Response to Untrusted Certificates.

Configure the VCF PowerCLI Response to Untrusted Certificates

You can configure how VCF PowerCLI responds when a server certificate is not valid.

Use the `InvalidCertificateAction` parameter of the `Set-PowerCLIConfiguration` cmdlet to configure how VCF PowerCLI responds to invalid server certificates.

1. Run `Get-PowerCLIConfiguration` and view the current setting of the `InvalidCertificateAction` parameter.
2. To change the setting, run `Set-PowerCLIConfiguration -InvalidCertificateAction` and specify one of the following values.
   - `Unset`. This is the default value and corresponds to `Fail`.
   - `Fail`. VCF PowerCLI does not establish a connection if a certificate is not valid.
   - `Ignore`. VCF PowerCLI ingnores certificate validity and establishes a connection.
   - `Warn`. VCF PowerCLI logs a warning message that a certificate is not valid, displays additional information about the certificate, and establishes the connection.

- `Prompt`. If a server certificate is not valid, VCF PowerCLI prompts you to choose an action before it continues. You can choose one of the following options.

Option	Action
Deny (default)	Cancel the server connection.
Accept Once	Establish the server connection and suppress further warnings for the current PowerShell session.
Accept Permanently	Save the server certificate in the VCF PowerCLI Trusted Certificate Store for the current user and establish the server connection.
Accept For All Users	Save the server certificate both in the current user's Trusted Certificate Store and in the all users' Trusted Certificate Store and establish the server connection.

Note:
For Linux and macOS, only the `Fail` and `Ignore` options are supported.

A warning message prompts you to confirm the operation. Press Y and then press Enter to confirm your choice.

You configured the invalid server certificate action setting. VCF PowerCLI displays a table with the updated configuration settings.

Trusted certificates are saved in the VCF PowerCLI Trusted Certificate Store.

Use VCF PowerCLI to connect to a server system.

VCF PowerCLI Trusted Certificate Store

The VCF PowerCLI trusted certificate store is a CSV file that stores trusted server certificates for the current user or for all users.

The VCF PowerCLI trusted certificate store is named `SslCertificateExceptions.csv` and contains server addresses and certificate thumbprints. You can edit the file with a text editor. You can transfer the CSV file to another machine if you want to use your saved certificates.

Note:
You must permanently accept at least one certificate as trusted for the CSV file to appear on your system.

The certificate store file is saved in a different location depending on the type of user. For more information about the exact file path of the trusted certificate store file, see VCF PowerCLI Configuration Files.

Modify the Timeout Setting for Web Tasks

To avoid unexpected timeouts, you can run `Set-PowerCLIConfiguration` to modify the VCF PowerCLI settings for long-running Web tasks.

Verify that you are connected to a vCenter system.

You might want to change the VCF PowerCLI setting for long-running Web tasks to avoid unexpected timeouts. The default value for the `WebOperationTimeoutSeconds` setting is 300 seconds.

1. Optional: Learn more about what settings you can configure with `Set-PowerCLIConfiguration`.

```
Get-Help
Set-PowerCLIConfiguration
```

2. Store the value of the timeout setting for the current session in the *$initialTimeout* variable.
   ```
   $initialTimeout = (Get-PowerCLIConfiguration -Scope Session).WebOperationTimeoutSeconds
   ```
3. Set the timeout setting for the current session to 30 minutes.
   ```
   Set-PowerCLIConfiguration -Scope Session -WebOperationTimeoutSeconds 1800
   ```
4. Run your Web task.
   - You can run an `esxcli` command to install a software profile.
     ```
     $vmHost = Get-VMHost "vmHostIp"
     $esxcli = Get-EsxCli -VMHost $vmHost -V2
     $arguments = $esxcli.software.profile.install.CreateArgs()
     $arguments.depot = "http://mysite.com/publish/proj/index.xml"
     $arguments.profile = "proj-version"
     $esxcli.software.profile.install.Invoke($arguments)
     ```
   - Alternatively, you can directly specify the arguments hash table in-line.
     ```
     $vmHost = Get-VMHost "vmHostIp"
     $esxcli = Get-EsxCli -VMHost $vmHost -V2
     $esxcli.software.profile.install.Invoke(@{depot="http://mysite.com/publish/proj/index.xml";
      profile="proj-version"})
     ```

     Note:
     The two examples use the ESXCLI V2 interface of PowerCLI.
5. Revert the timeout setting for the current session to the initial value.
   ```
   Set-PowerCLIConfiguration -Scope Session -WebOperationTimeoutSeconds $initialTimeout
   ```

Scoped Settings of VCF PowerCLI

In VCF PowerCLI you can set the scope of the settings to enhance security and personalize the configuration.

Configuring the Scope of the VCF PowerCLI Settings

Scoped configuration enhances system security and prevents nonadministrator users from introducing global changes to the configuration of VCF PowerCLI.

For greater control over the VCF PowerCLI configuration, the `Set-PowerCLIConfiguration` cmdlet provides the `Scope` parameter.

Table 1930: Valid Values for the Scope Parameter

Parameter Value	Description
`Session`	Configures settings for the current VCF PowerCLI session and does not modify any VCF PowerCLI configuration files on your system.
`User`	Configures settings for the current Windows user and modifies some VCF PowerCLI configuration files on your system.
`AllUsers`	Configures settings for all users and modifies some VCF PowerCLI configuration files on your system.

Priority of Settings Scopes in VCF PowerCLI

VCF PowerCLI loads the program configuration based on the scope that you select for each setting.

Table 1931: Scope Impact on the Behavior of VCF PowerCLI

Scope	Priority	Impact
`Session`	High	• When started, VCF PowerCLI tries to load settings with the `Session` scope first. • `Session` settings override `User` and `AllUsers` settings. • `Session` settings are valid for the current VCF PowerCLI session only.
`User`	Medium	• When VCF PowerCLI cannot detect `Session` settings, the program tries to load `User` settings from the VCF PowerCLI configuration files. • `User` settings override `AllUsers` settings. • `User` settings are automatically detected from the VCF PowerCLI configuration files.
`AllUsers`	Low	• When VCF PowerCLI cannot detect `Session` and `User` settings, the program loads `AllUsers` settings. • `AllUsers` settings do not override `Session` and `User` settings. • `AllUsers` settings are automatically detected from the VCF PowerCLI configuration files.

VCF PowerCLI Configuration Files

The copies of the `PowerCLI_settings.xml` file on your system contain `User` and `AllUsers` settings for VCF PowerCLI.

Configuring VCF PowerCLI by running `Set-PowerCLIConfiguration` creates a copy of `PowerCLI_settings.xml` on your system. The location of the `PowerCLI_settings.xml` file depends on the value of the `Scope` parameter.

> **Note:**
> You must have administrator privileges to change the settings for `AllUsers`.

Table 1932: Location of `PowerCLI_settings.xml`

Operating System	Scope	Location	Description
Windows	`User`	`%APPDATA%\VMWare\PowerCLI`	Contains settings for the current Windows user.
	`AllUsers`	`%SYSTEMDRIVE%\ProgramData\VMware\PowerCLI`	Contains settings for all users.
Linux	`User`	`$HOME/.local/share/VMware/PowerCLI`	Contains settings for the current Linux user.
	`AllUsers`	/var/opt/VMware/PowerCLI	Contains settings for all users.
macOS	`User`	`$HOME/Library/Preferences/VMware/PowerCLI`	Contains settings for the current macOS user.
	`AllUsers`	/Library/Preferences/VMware/PowerCLI	Contains settings for all users.

Users with advanced knowledge and understanding of PowerShell and VMware Cloud Foundation PowerCLI can manually modify the contents of `PowerCLI_settings.xml` to change VCF PowerCLI settings. Modifying `PowerCLI_settings.xml` might require administrator privileges.

Note:
If you modify the contents of `PowerCLI_settings.xml` manually while VCF PowerCLI is running, you must restart VCF PowerCLI for the changes to take effect.

Installing and Configuring Python for VCF PowerCLI

Starting from VMware Cloud Foundation PowerCLI 13.0, Python 3.9 or later is a prerequisite for using the `VMware.ImageBuilder` module.

To use the `VMware.ImageBuilder` module, you must install Python 3.9 or later with four additional software packages and configure VCF PowerCLI.

You must install the following additional packages in Python:

- `six`
- `psutil`
- `lxml`
- `pyopenssl`

You must configure VCF PowerCLI by setting the path to your Python installation.

Restart your PowerShell session if you tried to execute a cmdlet from the `VMware.ImageBuilder` module before setting the path to the Python executable.

Install and Configure Python on Windows

On Windows, you must install Python 3.9 or later with the required packages and configure VCF PowerCLI to use the `VMware.ImageBuilder` module.

1. Download Python 3.9 or later from https://www.python.org/downloads and complete the installation on your machine.
2. Save the `get-pip.py` file from https://bootstrap.pypa.io/get-pip.py.
3. Open a terminal and install `pip` package manager.

   ```
   <pythonVer-directory>\python.exe <get-pip-directory>\get-pip.py
   ```

   For example:

   ```
   <python3.10-directory>\python.exe <get-pip-directory>\get-pip.py
   ```

4. In the terminal, install the required Python modules.

   ```
   <pythonVer-directory>\Scripts\pip<Version>.exe install six psutil lxml pyopenssl
   ```

   For example:

   ```
   <python3.10-directory>\Scripts\pip3.10.exe install six psutil lxml pyopenssl
   ```

5. Open PowerShell and set the path to the Python executable.

   ```
   Set-PowerCLIConfiguration -PythonPath <pythonVer-directory>\python.exe -Scope User
   ```

   For example:

   ```
   Set-PowerCLIConfiguration -PythonPath <python3.10-directory>\python.exe -Scope User
   ```

You can now use the VCF PowerCLI`VMware.ImageBuider` module.

Install and Configure Python on macOS

On macOS, you must install Python 3.9 or later with the required packages and configure VCF PowerCLI to use the `VMware.ImageBuilder` module.

Verify that you have OpenSSL version 1.1.1 installed on your machine.

1. Install Python.

Option	Action
By Using the Mac Terminal	Open the Mac Terminal and run `brew install python@<Version>`. For example: `brew install python@3.10`
By Using the Python Installer	Download the Python installer from https://www.python.org/downloads and complete the installation on your machine.

2. Save the `get-pip.py` file from https://bootstrap.pypa.io/get-pip.py.
3. In the Mac Terminal, install `pip` package manager.

   ```
   <pythonVer-directory>/<pythonVer> <get-pip-directory>/get-pip.py
   ```

   For example:

   ```
   <python3.10-directory>/python3.10 <get-pip-directory>/get-pip.py
   ```

4. Install the required Python modules.

   ```
   <pythonVer-directory>/Scripts/pip<Version> install six psutil lxml pyopenssl
   ```

   For example:

   ```
   <python3.10-directory>/Scripts/pip3.10 install six psutil lxml pyopenssl
   ```

5. Open PowerShell and set the path to the Python executable.

   ```
   Set-PowerCLIConfiguration -PythonPath <pythonVer-directory>/<pythonVer> -Scope User
   ```

   For example:

   ```
   Set-PowerCLIConfiguration -PythonPath <python3.10-directory>/python3.10 -Scope User
   ```

You can now use the VCF PowerCLI`VMware.ImageBuider` module.

Install and Configure Python on Linux

On Linux, you must install Python 3.9 or later with the required packages and configure VCF PowerCLI to use the `VMware.ImageBuilder` module.

Verify that you have OpenSSL version 1.1.1 installed on your machine.

1. Open a terminal and install Python by running `apt-get install python<version>`.
   For example:

   ```
   apt-get install python3.10
   ```

2. Save the `get-pip.py` file from https://bootstrap.pypa.io/get-pip.py.
3. In the terminal, install `pip` package manager.

   ```
   <pythonVer-directory>/<pythonVer> <get-pip-directory>/get-pip.py
   ```

   For example:

   ```
   <python3.10-directory>/python3.10 <get-pip-directory>/get-pip.py
   ```

4. Install the required Python modules.

   ```
   <pythonVer-directory>/Scripts/pip<Version> install six psutil lxml pyopenssl
   ```

   For example:

   ```
   <python3.10-directory>/Scripts/pip3.10 install six psutil lxml pyopenssl
   ```

5. Open PowerShell and set the path to the Python executable.

   ```
   Set-PowerCLIConfiguration -PythonPath <pythonVer-directory>/<pythonVer> -Scope User
   ```

   For example:

```
Set-PowerCLIConfiguration -PythonPath <python3.10-directory>/python3.10 -Scope User
```

You can now use the VCF PowerCLI`VMware.ImageBuider` module.

Configuring Customer Experience Improvement Program

You can participate in the Customer Experience Improvement Program (CEIP) to provide anonymous feedback or information to VMware for quality, reliability, and functionality improvments of VMware products and services.

VMware Customer Experience Improvement Program

vCenter participates in VMware's Customer Experience Improvement Program (CEIP).

Details regarding the data collected through CEIP and the purposes for which it is used by VMware are set forth at the Trust & Assurance Center at http://www.vmware.com/trustvmware/ceip.html.

Join the Customer Experience Improvement Program in VCF PowerCLI

You can choose to join the Customer Experience Improvement Program (CEIP), or leave the CEIP at any time.

Run `Set-PowerCLIConfiguration`.

- To join the CEIP, run the following command.
  ```
  Set-PowerCLIConfiguration -ParticipateInCeip $true
  ```
- To leave the CEIP, run the following command.
  ```
  Set-PowerCLIConfiguration -ParticipateInCeip $false
  ```

Managing vSphere with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in vSphere administration.

Connecting to a vCenter System

You can connect to a vCenter system with VCF PowerCLI no matter if you ?re using the default VMware vCenter Single Sign-On identity service, or an external identity provider.

Connect to a vCenter System

To run VCF PowerCLI cmdlets on vSphere and perform administration or monitoring tasks, you must establish a connection to an ESX host or a vCenter system.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see Managing Default Server Connections in VCF PowerCLI.

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see Processing Non-alphanumeric Characters in VCF PowerCLI.

Run `Connect-VIServer` with the server name and valid credentials.

```
Connect-VIServer -Server vc1.example.com -Protocol http -User 'MyAdministratorUser' -Password
'MyPassword'
```

Connect to a vCenter System Configured for an External Identity Provider

If your vCenter is configured with an external identity provider, such as Microsoft Active Directory Federation Services (AD FS) or other external identity provider supported by VMware Identity Service, you can authenticate with VCF PowerCLI by using the OAuth 2.0 Authorization Code grant type.

You can authenticate to a federated vCenter by creating a new OAuth security context and then exchanging it for a SAML security context. You create an OAuth security context for VCF PowerCLI by using the New-OAuthSecurityContext cmdlet. One way to do this is to authenticate through the Authorization Code grant type, which is illustrated by this example. This workflow guarantees a substantial degree of security and can be used with multi-factor authentication.

> **Note:**
>
> You can use VCF PowerCLI to authenticate with the other OAuth 2.0 grant types as well, such as the Client Credentials, Refresh Token, and Password grant types. For more information, run `Get-Help New-OAuthSecurityContext -full`.

Connect to a vCenter System Configured to Use AD FS for an External Identity Provider

If your vCenter is federated to Active Directory Federation Services (AD FS), you can authenticate with VCF PowerCLI by using the OAuth 2.0 Authorization Code grant type.

- Verify that your vCenter system is federted to AD FS. For more information, see *Federate vCenter to Microsoft Active Directory Federation Services (AD FS)* in the *VMware vSphere Automation REST API Programming Guide*.
- Create an OAuth client for VCF PowerCLI on the authentication server (AD FS). Configure the OAuth client to do the same token transformation as your vCenter system . You must configure a redirect URL according to the requirements in the procedure below. Save the Client ID and Client Secret that are generated by the authentication server. For more information, see the Microsoft documentation.

1. Create an OAuth security context object for VCF PowerCLI.

   In the background, VCF PowerCLI sends an authentication request to the authentication server, AD FS. A web browser opens that prompts the user to authorize the client application's request. On confirmation, VCF PowerCLI interacts with the authentication server to obtain the access and ID tokens (JWT tokens) and creates a new OAuth security context.

   ```
   $oauthSecContext = New-OAuthSecurityContext -TokenEndpointUrl "https://<AD FS FQDN>/adfs/oauth2/token/"
    -AuthorizationEndpointUrl "https://<AD FS FQDN>/adfs/oauth2/authorize/"
   -RedirectUrl "http://localhost:8844/auth" -ClientId "powercli-native" -OtherArguments @{ "resource" = "my-
   vcenter" }
   ```

   This script contains the following parameters.

Parameter	Description	Example value
`TokenEndpointUrl`	The base URL where the authentication server listens for requests to issue access tokens. You can see this value in the openid-configuration file of the authentication server under `token_endpoint` key.	https://<AD FS FQDN>/adfs/oauth2/token/
`AuthorizationEndpointUrl`	The base URL at the authentication server where users are redirected in order to authenticate. You can see this value in the openid-configuration file under the `authorization_endpoint` key.	https://<AD FS FQDN>/adfs/oauth2/authorize/

Parameter	Description	Example value
`RedirectUrl`	The URL where the user is redirected after he approves the authentication request. This URL must be on a localhost and a free port on the machine where VCF PowerCLI is running. This URL must use the `http` schema.	http://localhost:8844/auth
`ClientId`	The ID that you received from the authentication server when you registered the client application.	powercli-native
`OtherArguments`	A hashtable of (String, String) pairs that represent arguments to the server-specific parameters. In the following example, it is used for an AD FS Application Group for vCenter with an example ID "my-vcenter".	{ "resource" = "my-vcenter" }

2. Exchange the OAuth security context for an SAML security context.

```
$samlSecContext = New-VISamlSecurityContext -VCenterServer 'myVC' -OAuthSecurityContext
 $oauthSecContext
```

3. Connect to your vCenter system by using the SAML security context.

```
Connect-VIServer -Server 'myVC' -SamlSecurityContext $samlSecContext
```

Connect to a vCenter System Configured to Use External Identity Providers Supported by VMware Identity Services

With vSphere 8.0 Update 1, VMware Identity Services is a built-in container within vCenter that you can use for identity federation to external identity providers. It serves as an independent identity broker within vCenter and currently supports Okta, Microsoft Entra ID, and PingFederate as external identity providers. Starting with vSphere 8.0 Update 3, VCF PowerCLI integrates with VMware Identity Services to support the use of external identity providers to authenticate to vCenter .

- Verify that your vCenter system is federated to an identity provider supported by VMware Identity Service. See the *Configuring vCenter Identity Provider Federation* chapter in the *vSphere Authentication* guide.
- Verify that you have the vCenter Single Sign-On administrator privileges.

1. Log in as an administrator to vCenter.
2. Create a new OAuth client registration with the VMware Identity Services.

```
New-VIOAuth2Client -ClientId "powercli-native" -Name "PowerCLI Client" -Scope @("openid", "user", "group")
 -GrantTypes @("authorization_code", "refresh_token") -RedirectUris @("http://localhost:8844/authcode")
 -AccessTokenTimeToLiveMinutes 30 -RefreshTokenTimeToLiveMinutes 43200 -RefreshTokenIdleTimeToLiveMinutes
 28800 -PkceEnforced $true -Secret "<secret_value>"
```

The New-VIOAuth2Client cmdlet accepts the following parameters:

Parameter	Description	Example
`ClientId`	The identifier used from the client to identify itself during the OAuth exchanges. The client ID must contain only alphanumeric (A-Z, a-z, 0-9), period (.), underscore (_), hyphen (-) and the at sign (@) characters.	`"powercli-native"`
`Name`	The user-friendly name that you set for this OAuth client.	`"PowerCLI Client"`

Parameter	Description	Example
`Scope`	A list of access request scopes that are allowed by this OAuth client. Available scope options are: admin, user, profile, email, openid, and group. • admin - Provides admin level access. • user - Provides user level access. • profile - Provides access to the user's profile (first name, last name, display name, and image) • email - Provides access to the user's email. • openid - Provides access to an OpenID token for the user. • group - Provides access to the user's groups. The minimal set of values is @("openid", "user", "group")	`@("openid", "user", "group")`
`GrantTypes`	A list of OAuth access grant types that are enabled in this OAuth client. Possible values are: `password`, `client_credentials`, `refresh_token`, `authorization_code`, and `token, id_token`. The minimal set of values is `@("authorization_code", "refresh_token")`	`@("authorization_code", "refresh_token")`
`RedirectUris`	A list of absolute URIs of application endpoints that are allowed to receive the authorization code and access token. The redirect URI sent by the application as part of the authorization code grant OAuth flow is verified against this list. A wildcard can be substituted for any string to skip the check for a particular URL section. The field is required if the `GrantTypes` parameter contains an `"authorization_code"` grant type. This URL must point to localhost and use a free port on the machine where VCF PowerCLI is running. The URL must use the HTTP schema.	`@("http://localhost:8844/authcode")`
`AccessTokenTimeToLiveMinutes`	The period of time, in minutes, for which the new access tokens issued to this client should live.	`30`
`RefreshTokenIdleTimeToLiveMinutes`	The period of time, in minutes, for which the new refresh tokens issued to this client can be idle. This parameter is only applicable and mandatory if `GrantTypes` includes `"refresh_token"`. Its value should be less than the value specified by the `RefreshTokenTimeToLiveMinutes` parameter.	`28800`
`RefreshTokenTimeToLiveMinutes`	The period of time, in minutes, for which the new refresh tokens issued to this client should live. Only applicable and mandatory if the `GrantTypes` parameter includes `"refresh_token"`.	`43200`
`PkceEnforced`	Auth client. If not specified, the value is `false`.	`$true`

Parameter	Description	Example
`Secret`	Specifies the OAuth client secret. If a secret string is not provided, an auto-generated secret is returned. For additional security, the stored secret is not be returned by the `Get-VIOAuth2Client` cmdlet. Public clients must not have any secret auto generated for them while confidential clients must always have a client secret. **Note:** The secret is required to connect to the vCenter and must be distributed to the users of this OAuth client..	`"<secret_value>"`

3. Create an OAuth security context object for VCF PowerCLI.

```
$oauthSecContext = New-OAuthSecurityContext -TokenEndpointUrl "https://<vcenter_fqdn>/acs/t/CUSTOMER/
token" -AuthorizationEndpointUrl "https://<vcenter_fqdn>/acs/t/CUSTOMER/authorize" -RedirectUrl "http://
localhost:8844/auth" -ClientId "powercli-native" -ClientSecret "<secret_value>"
```

   The script contains the following parameters:

Parameter	Description	Example
`TokenEndpointUrl`	The base URL used by the VMware Identity Services to listen for access token requests.	`https://<vcenter_fqdn>/acs/t/CUSTOMER/token`
`AuthorizationEndpointUrl`	The VMware Identity Services URL used to redirect users to authenticate.	`https://<vcenter_fqdn>/acs/t/CUSTOMER/authorize`
`RedirectUrl`	The URL where the user is redirected after the approval of the authentication request. This URL must point to localhost and use a free port on the machine where VCF PowerCLI is running. This URL must use the HTTP schema. **Note:** The value of this property must match the value specified with the `RedirectUris` parameter used for the registration of the OAuth client in step 2.	`http://localhost:8844/auth`
`ClientId`	The ID of the VCF PowerCLI OAuth client created in step 2.	`powercli-native`
`ClientSecret`	The client secret of the VCF PowerCLI OAuth client created in step 2	`"<secret_value>"`

   When this script runs, VCF PowerCLI uses VMware Identity Services to send an authentication request to the authentication server. A Web browser opens and prompts the user to authorize the request of the client application. On confirmation, VMware Identity Services interacts with the authentication server to obtain the access and ID tokens, JSON Web Token (JWT). Based on this output VCF PowerCLI creates a new OAuth security context.

4. Exchange the OAuth 2.0 security context for a SAML security context.

```
$samlSecContext = New-VISamlSecurityContext -VCenterServer '<vcenter_fqdn>' -OAuthSecurityContext
 $oauthSecContext
```

5. Connect to your vCenter system by using the SAML security context.

```
Connect-VIServer -Server '<vcenter_fqdn>' -SamlSecurityContext $samlSecContext
```

Unattended Logins to a Federated vCenter System with an External Identiry Provider

Unattended logins to a federated vCenter system represent a security threat and must be allowed with great caution.

When you register the vCenter OAuth client with the OAuth provider, you can create a configuration that allows password grant type (Resource Owner Password Credentials grant type) for some users. Such users can then perform unattended logins to the federated vCenter system.

> **Note:**
> This flow exposes the user credentials to the client application, VCF PowerCLI, and to vCenter. To minimize the security risk, allow access only to service users with limited permissions. For more information, see: https://oauth.net/2/grant-types/password/.

To connect to the vCenter system, you can directly use the `Connect-VIServer` cmdlet in the following way:

```
Connect-VIServer -Server '<vcenter_fqdn>' -User '<user_name>' -Password '<user_password>'
```

If your vCenter system is federated with VCF Single Sign-On, you can perform unattended logins by using a VCF API token. See Unattended Logins to a vCenter System Federated with VCF Single Sign-On.

Create vSphere Inventory Objects

By using VCF PowerCLI cmdlets, you can automate creating different inventory objects on vSphere.

Verify that you are connected to a vCenter system.

1. Get the inventory root folder and create a new folder named `Folder` in it.
   ```
   $folder = Get-Folder -NoRecursion | New-Folder -Name Folder
   ```
2. Create a new data center named `DC` in the `Folder` folder.
   ```
   New-Datacenter -Location $folder -Name DC
   ```
3. Create a folder named `Folder1` under `DC`.
   ```
   Get-Datacenter DC | New-Folder -Name Folder1
   $folder1 = Get-Folder -Name Folder1
   ```
4. Create a new cluster Cluster1 in the `Folder1` folder.
   ```
   New-Cluster -Location $folder1 -Name Cluster1 -DrsEnabled -DrsAutomationLevel FullyAutomated
   ```
   Distributed Resource Scheduler (DRS) is a feature that provides automatic allocation of cluster resources.
5. Add a host in the cluster by using the `Add-VMHost` command, and provide credentials when prompted.
   ```
   $vmhost1 = Add-VMHost -Name 10.23.112.345 -Location (Get-Cluster Cluster1)
   ```
6. Create a resource pool in the root resource pool of the cluster.
   ```
   $myClusterRootRP = Get-Cluster Cluster1 | Get-ResourcePool -Name Resources
   New-ResourcePool -Location $myClusterRootRP -Name MyRP1 -CpuExpandableReservation $true
    -CpuReservationMhz 500 -CpuSharesLevel high -MemExpandableReservation $true -MemReservationGB 1
    -MemSharesLevel high
   ```
7. Create a virtual machine asynchronously.
   ```
   $vmCreationTask = New-VM -Name VM2 -VMHost $vmhost1 -ResourcePool MyRP01 -DiskGB 100 -MemoryGB 2 -RunAsync
   ```
   The `RunAsync` parameter indicates that the command runs asynchronously. This means that in contrast to a synchronous operation, you do not have to wait for the process to complete before supplying the next command at the command line.

vCenter and Host Management

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vCenter and host management.

Get a List of Hosts on a vCenter System and View Their Properties

With VCF PowerCLI, you can get information about all available hosts in a data center and view their properties.

Verify that you are connected to a vCenter system.

1. Get a list of all hosts that are part of a data center.

```
Get-Datacenter DC | Get-VMHost | Format-Custom
```

2. View the properties of the first host in the data center.

```
Get-Datacenter DC | Get-VMHost | Select-Object -First 1 | Format-Custom
```

3. View the `Name` and the `OverallStatus` properties of the hosts in the *`DC`* data center.

```
Get-Datacenter DC | Get-VMHost | Get-View | Format-Table -Property Name, OverallStatus
-AutoSize
```

4. View all hosts and their properties, and save the results to a file.

```
Get-Datacenter DC | Get-VMHost | Format-Custom | Out-File –FilePath hosts.txt
```

5. View a list of the hosts that are in maintenance mode and can be configured for vMotion operations.

```
Get-VMHost -State maintenance | Get-View | Where-Object -FilterScript { $_.capability -ne
$null -and $_.capability.vmotionSupported }
```

Add a Standalone Host to a vCenter System

You can add standalone hosts to a vCenter system by using the `Add-VMHost` cmdlet. After adding the hosts, you will be able to manage them through the vCenter system.

Verify that you are connected to a vCenter system.

1. View all hosts on the vCenter system that you have established a connection with.

```
Get-VMHost
```

2. Add the *`Host`* standalone host.

```
Add-VMHost -Name Host -Location (Get-Datacenter DC) -User root -Password pass
```

Set the License Key for a Host on vCenter

You can set the license key for a host on a vCenter system by using the `LicenseKey` parameter of the `Set-VMHost` cmdlet.

Verify that you are connected to a vCenter system.

1. Save the *`Host`* host object as a variable.

```
$vmhost = Get-VMHost -Name Host
```

2. Set the host to evaluation mode or provide a valid license key.
   - Set the host to evaluation mode by providing the evaluation key.

     ```
     Set-VMHost -VMHost $vmhost -LicenseKey 00000-00000-00000-00000-00000
     ```

   - Provide a valid license key.

     ```
     Set-VMHost -VMHost $vmhost -LicenseKeyYour_license_key
     ```

Activate Maintenance Mode for a Host on vCenter

To complete some specific administration tasks, you might need to activate maintenance mode for a host. On vCenter, you can activate maintenance mode by using the `Set-VMHost` cmdlet.

Verify that you are connected to a vCenter system.

1. Save the *`Host`* host object as a variable.
   ```
   $vmhost = Get-VMHost -Name Host
   ```
2. Get the cluster to which *`Host`* belongs and save the cluster object as a variable.
   ```
   $vmhostCluster = Get-Cluster -VMHost $vmhost
   ```
3. Start a task that activates maintenance mode for the *`Host`* host and save the task object as a variable.
   ```
   $updateHostTask = Set-VMHost -VMHost $vmhost -State "Maintenance" -RunAsync
   ```

   Note:
   If the host is not automated or is partially automated and has powered-on virtual machines running on it, you must use the `RunAsync` parameter and wait until all powered-on virtual machines are relocated or powered off before applying DRS recommendations.
4. Get and apply the recommendations generated by DRS.
   ```
   Get-DrsRecommendation -Cluster $vmhostCluster | where {$_.Reason -eq "Host is entering
   maintenance mode"} | Invoke-DrsRecommendation
   ```
5. Get the task output object and save it as a variable.
   ```
   $myUpdatedHost = Wait-Task $updateHostTask
   ```

Change the Host Advanced Configuration Settings on vCenter

You can modify host configuration, including advanced settings related to virtual machine migration, and apply them to another host.

Verify that you are connected to a vCenter system.

1. Change the migration timeout for the *`ESXHost1`* host.
   ```
   Get-VMHost ESXHost1 | Set-VmHostAdvancedConfiguration -Name Migrate.NetTimeout -Value
   ( [system.int32] 10 )
   ```
2. Enable creation of a checksum of the virtual machines memory during the migration.
   ```
   Get-VMHost ESXHost1 | Set-VmHostAdvancedConfiguration -Name Migrate.MemChksum -Value
   ( [system.int32] 1 )
   ```
3. Get the *`ESXHost1`* host migration settings.
   ```
   $migrationSettings = Get-VMHost ESXHost1 | Get-VmHostAdvancedConfiguration -Name Migrate.*
   ```
4. Apply the migration settings to *`ESXHost2`*.
   ```
   Set-VmHostAdvancedConfiguration -VMHost ESXHost2 -Hashtable $migrationSettings
   ```

Create a Host Profile on a vCenter System

The VMware Host Profiles feature enables you to create standard configurations for ESX hosts. With VCF PowerCLI, you can automate creation and modifying of host profiles.

Verify that you are connected to a host that runs vCenter 4.0 or later.

1. Get the host named *`Host1`* and store it in the *`$vmhost`* variable.
   ```
   $vmhost = Get-VMHost Host1
   ```

2. Create a profile based on the *Host1* host.

```
New-VMHostProfile -Name MyHostProfile01 -Description "This is my test profile based on
Host1." -ReferenceHost $vmhost
```

3. Get the newly created host profile.

```
$hp1 = Get-VMHostProfile -Name MyHostProfile01
```

4. Change the description of the *HostProfile1* host profile.

```
Set-VMHostProfile -Profile $hp1 -Description "This is my old test host profile based on
Host1."
```

Apply a Host Profile to a Host on vCenter

To simplify operational management of large-scale environments, you can apply standard configurations called host profiles to hosts on vCenter. If you want to set up a host to use the same host profile as a reference host, you can attach the host to a profile.

Verify that you are connected to a host that runs vCenter 4.0 or later.

1. Get the *Host2* host.

```
$vmhost2 = Get-VMHost Host2
```

2. Attach the *Host2* host to the *HostProfile1* host profile.

```
Set-VMHost -VMHost $vmhost2 -Profile HostProfile1
```

3. Verify that the *Host2* host is compliant with the *HostProfile1* profile.

```
Test-VMHostProfileCompliance -VMHost $vmhost2
```

   The output of this command contains the incompliant settings of the host, if any.

4. Apply the profile to the *Host2* host.

```
$neededVariables = Invoke-VMHostProfile -Entity $vmhost2 -Profile $hp1 -Confirm:$false
```

   The *$neededVariables* variable contains the names of all required variables and their default or current values, as returned by the server. Otherwise, the *$neededVariables* variable contains the name of the host on which the profile has been applied.

Create a vSphere Role and Assign Permissions to a User

With VCF PowerCLI, you can automate management of vSphere permissions, roles, and privileges.

Verify that you are connected to a vCenter system.

> **Note:**
>
> vSphere permissions determine your level of access to vCenter, and ESX hosts. Privileges define individual rights to perform actions and access object properties. Roles are predefined sets of privileges.

1. Get the privileges of the **Readonly** role.

```
$readOnlyPrivileges = Get-VIPrivilege -Role Readonly
```

2. Create a new role with custom privileges.

```
$role1 = New-VIRole -Privilege $readOnlyPrivileges -Name Role1
```

3. Add the **PowerOn** privileges to the new role.

```
$powerOnPrivileges = Get-VIPrivilege -Name "PowerOn"
$role1 = Set-VIRole -Role $role1 -AddPrivilege $powerOnPrivileges
```

4. Create a permission and apply it to a vSphere root object.

```
$rootFolder = Get-Folder -NoRecursion
```

```
$permission1 = New-VIPermission -Entity $rootFolder -Principal "user" -Role readonly -Propagate
```

The `Principal` parameter accepts both local and domain users and groups if the vCenter system is joined in AD.

5. Update the new permission with the custom role.

```
$permission1 = Set-VIPermission -Permission $permission1 -Role $role1
```

You created a new role and assigned permissions to a user.

Modify the vCenter Email Configuration

You can modify the email configuration settings of a vCenter.

Verify that you are connected to a vCenter system.

1. View the current email configuration settings of the vCenter from the *`$srv`* variable.

```
Get-AdvancedSetting -Entity $srv -Name mail.*
```

2. Update the SMTP server name and port.

```
Get-AdvancedSetting -Entity $srv  -Name mail.smtp.server | Set-AdvancedSetting -Value
 smtp.vmware.com
Get-AdvancedSetting -Entity $srv  -Name mail.smtp.port | Set-AdvancedSetting -Value 25
```

Modify the vCenter SNMP Configuration

To use SNMP, you must first configure the SNMP settings of the vCenter.

Verify that you are connected to a vCenter system.

1. View the current SNMP configuration settings of the vCenter from the *`$srv`* variable.

```
Get-AdvancedSetting -Entity $srv  -Name snmp.*
```

2. Modify the SNMP receiver data.

```
Get-AdvancedSetting -Entity $srv  -Name snmp.receiver.2.community | Set-AdvancedSetting
 -Value public
Get-AdvancedSetting -Entity $srv  -Name snmp.receiver.2.enabled | Set-AdvancedSetting -Value $true
Get-AdvancedSetting -Entity $srv  -Name snmp.receiver.2.name | Set-AdvancedSetting -Value 192.168.1.10
```

Now you can use SNMP with vCenter.

Compute Cluster Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in compute cluster administration.

Create and Modify Advanced Settings for a Cluster

You can customize the behavior of a cluster on a vCenter system by creating and modifying custom advanced settings for it.

Verify that you are connected to a vCenter system.

1. Create a new cluster named *`Cluster`*.

```
$cluster = New-Cluster -Name Cluster -Location (Get-Datacenter Datacenter)
```

2. Create two advanced settings for the new cluster.

```
$setting1 = New-AdvancedSetting -Type "ClusterHA" -Entity $cluster -Name 'das.defaultfailoverhost'
 -Value '192.168.10.1'
```

```
$setting2 = New-AdvancedSetting -Type "ClusterHA" -Entity $cluster -Name 'das.isolationaddress' -Value
 '192.168.10.2'
```

3. Modify the value of the advanced setting stored in the *$setting2* variable.

```
Get-AdvancedSetting -Entity $cluster -Name 'das.isolationaddress' | Set-AdvancedSetting
-Value '192.168.10.3' -Confirm:$false
```

4. Create another advanced setting.

```
New-AdvancedSetting -Entity $cluster -Name 'das.allowNetwork[Service Console]' -Value $true
-Type 'ClusterHA'
```

5. Get the `Service Console` setting and store it in a variable.

```
$setting3 = Get-AdvancedSetting -Entity $entity -Name 'das.allowNetwork`[Service Console`]'
```

The ` character is used to escape the wildcard characters [and] in the advanced setting name.

Create a New VM-VM DRS Rule

You can create a VM-VM DRS affinity rule within a cluster.

- Verify that you are connected to a vCenter system.
- Verify that virtual machines and hosts exist within a cluster with enabled DRS in the vCenter environment.

1. Get the virtual machines for the VM-VM DRS rule.

```
$affinityVMs = Get-VM "VM1", "VM2"
```

2. Get the cluster where you want to create the rule.

```
$cluster = Get-Cluster "MyCluster"
```

3. Create the VM-VM DRS rule within the *MyCluster* cluster.

```
New-DrsRule -Cluster $cluster -Name "AffinityRule1" -KeepTogether $true -VM $affinityVMs
```

Create a New VM-VMHost DRS Rule

You can create a VM-VMHost DRS rule within a cluster after creating a VM DRS cluster group and a VMHost DRS cluster group.

- Verify that you are connected to a vCenter system.
- Verify that virtual machines and hosts exist within a cluster with enabled DRS in the vCenter environment.

1. Get the virtual machines for the VM DRS cluster group.

```
$vms = Get-VM "VM1", "VM2"
```

2. Get the hosts for the VMHost DRS cluster group.

```
$vmHosts = Get-VMHost "hostname1", "hostname2"
```

3. Get the cluster where you want to create the rule.

```
$cluster = Get-Cluster "MyCluster"
```

4. Create a VM DRS cluster group.

```
$vmGroup = New-DrsClusterGroup -Name "MyVmsDrsClusterGroup" -VM $vms -Cluster $cluster
```

5. Create a VMHost DRS cluster group.

```
$vmHostGroup = New-DrsClusterGroup -Name "MyVmHostsDrsClusterGroup" -VMHost $vmHosts -Cluster $cluster
```

6. Create the VM-VMHost DRS rule by using the newly created VM DRS cluster group and VMHost DRS cluster group.

```
New-DrsVMHostRule -Name "MyDrsRule" -Cluster $cluster -VMGroup $vmGroup -VMHostGroup
$vmHostGroup -Type "MustRunOn"
```

Virtual Machine Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in virtual machine administration.

Manage Virtual Machines on vSphere

With VCF PowerCLI, you can automate various administration tasks on virtual machines, for example retrieving information, shutting down and powering off virtual machines.

1. View all virtual machines on the target system.

   ```
   Get-VM
   ```

2. Save the name and the power state properties of the virtual machines in the *ResourcePool* resource pool into a file named `myVMProperties.txt`.

   ```
   $respool = Get-ResourcePool ResourcePool
   Get-VM -Location $respool | Select-Object Name, PowerState > myVMProperties.txt
   ```

3. Start the *VM* virtual machine.

   ```
   Get-VM VM | Start-VM
   ```

4. Get information of the guest OS of the *VM* virtual machine.

   ```
   Get-VMGuest VM | fc
   ```

5. Shut down the OS of the *VM* virtual machine.

   ```
   Stop-VMGuest VM
   ```

6. Power off the *VM* virtual machine.

   ```
   Stop-VM VM
   ```

7. Move the virtual machine *VM* from the *Host01* host to the *Host02* host.

   ```
   Get-VM -Name VM -Location Host01 | Move-VM -Destination Host02
   ```

   Note:
   If the virtual machine you want to move across hosts is powered on, it must be located on a shared storage registered as a datastore on both the original and the new host.

Create Virtual Machines on vCenter Using an XML Specification File

You can use a specification provided in an XML file to automate the creation of virtual machines on vCenter.

Verify that you are connected to a vCenter system.

The `myVM.xml` file must be present with the following content:

```
<CreateVM>
<VM>
<Name>MyVM1</Name>
<HDDCapacity>100</HDDCapacity>
</VM>
<VM>
```

```
<Name>MyVM2</Name>
<HDDCapacity>100</HDDCapacity>
</VM>
</CreateVM>
```

1. Read the content of the `myVM.xml` file.
   ```
   [xml]$s = Get-Content myVM.xml
   ```
2. Create the virtual machines.
   ```
   $s.CreateVM.VM | foreach {New-VM -VMHost $vmHost1 -Name $_.Name -DiskGB $_.HDDCapacity}
   ```

Manage Virtual Machine Templates on vCenter

You can use VCF PowerCLI to create virtual machines templates and convert them to virtual machines on vCenter.

Verify that you are connected to a vCenter system.

> **Note:**
> A virtual machine template is a reusable image created from a virtual machine. The template, as a derivative of the source virtual machine, includes virtual hardware components, an installed guest operating system, and software applications.

1. Create a template from the *VM1* virtual machine.

   ```
   New-Template -VM VM1 -Name VM1Template -Location (Get-Datacenter DC )
   ```
2. Convert the *VM1Template* template for use by a virtual machine named *VM3*.

   ```
   Get-Template VM1Template | Set-Template -ToVM -Name VM3
   ```
3. Create a template from the *VM2* virtual machine.

   ```
   New-Template -VM VM2 -Name VM2Template -Location (Get-Datacenter DC )
   ```
4. Convert the *VM2Template* template to a virtual machine named *VM4*.

   ```
   Get-Template VM2Template | Set-Template -ToVM -Name VM4
   ```
5. Convert the *VM4* virtual machine to a template.

   ```
   Set-VM –VM VM4 –ToTemplate –Name “VM4Template”
   ```
6. Create a template called *VM3Template* by cloning *VM2Template*.

   ```
   Get-Template VM2Template | New-Template -Name VM3Template –VMHost $targetVMHost
   ```

Create and Use Snapshots on vCenter

You can use the `Snapshot` parameter of `Get-VM` to take a snapshot of virtual machines and then revert the states of the virtual machines back to the snapshot.

Verify that you are connected to a vCenter system.

> **Note:**
> A snapshot captures the memory, disk, and settings state of a virtual machine at a particular moment. When you revert to a snapshot, you return all these items to the state they were in at the time you took that snapshot.

1. Take a snapshot of all virtual machines in the *MyRP01* resource pool.

   ```
   Get-ResourcePool MyRP01 | Get-VM | New-Snapshot -Name InitialSnapshot
   ```

The `Location` parameter takes arguments of the `VIContainer` type, on which `Cluster`, `Datacenter`, `Folder`, `ResourcePool`, and `VMHost` object types are based. Therefore, the `Location` parameter can use arguments of all these types.

2. Revert all virtual machines in the *`MyRP01`* resource pool to the *`InitialSnapshot`* snapshot.

```
$VMs = Get-ResourcePool MyRP01 | Get-VM
foreach( $vm in $VMs ) { Set-VM -VM $vm –Snapshot InitialSnapshot }
```

Update the Resource Configuration Settings of a Virtual Machine on vCenter

You can use the `Set-VMResourceConfiguration` cmdlet to modify the resource configuration properties of a virtual machine, including memory, CPU shares, and other settings.

Verify that you are connected to a vCenter system.

1. View the resource configuration for the *`VM1`* virtual machine.

```
Get-VMResourceConfiguration -VM VM1
```

2. View the disk share of the *`VM1`* virtual machine.

```
Get-VMResourceConfiguration -VM VM1 | Format-Custom -Property DiskResourceConfiguration
```

3. Change the memory share of the *`VM1`* virtual machine to low.

```
Get-VM VM1 | Get-VMResourceConfiguration | Set-VMResourceConfiguration -MemSharesLevel
low
```

4. Change the CPU shares of the *`VM1`* virtual machine to high.

```
Get-VM VM1 | Get-VMResourceConfiguration | Set-VMResourceConfiguration -CpuSharesLevel
high
```

5. Change the disk share of the *`VM1`* virtual machine to 100.

```
$vm1 = Get-VM VM1
$vm1disk = Get-HardDisk $vm1
Get-VMResourceConfiguration $vm1 | Set-VMResourceConfiguration -Disk $vm1disk -DiskSharesLevel
 custom -NumDiskShares 100
```

Move a Virtual Machine to a Different Host Using VMware vSphere vMotion

You can migrate a virtual machine between vCenter hosts by using vSphere vMotion.

Verify that you are connected to a vCenter system.

The virtual machine must be stored on a datastore shared by the current and the destination host, and the vMotion interfaces on the two hosts must be configured.

> **Note:**
> You can use vSphere vMotion to move a powered-on virtual machine from one host to another.

Get the *`VM1`* virtual machine and move it to a host named *`ESXHost2`*.

```
Get-VM VM1 | Move-VM -Destination (Get-VMHost ESXHost2)
```

Move a Virtual Machine to a Different Datastore Using VMware vSphere Storage vMotion

You can migrate a virtual machine between datastores using the VMware Storage vMotion feature of vCenter.

Verify that you are connected to a vCenter system.

The host on which the virtual machine is running must have access both to the datastore where the virtual machine is located and to the destination datastore.

Note:
You can use Storage vMotion to move a powered-on virtual machine from one datastore to another.

Get the *VM1* virtual machine and move it to a datastore named *DS2*:

```
Get-VM VM1 | Move-VM -Datastore DS2
```

Move a Virtual Machine to a Different vCenter System

You can migrate a virtual machine from one vCenter system to another by using Cross vCenter vMotion.

You can move virtual machines between vCenter systems of vSphere version 6.0 and later by using the `Move-VM` cmdlet. When you move a virtual machine from one vCenter system to another, only datastores are supported as storage destinations.

1. Connect to the *myVC1* source vCenter system.

```
Connect-VIServer 'myVC1' -Username MyUser1 -Password MyPass1
```

2. Connect to the *myVC2* destination vCenter system.

```
Connect-VIServer 'myVC2' -Username MyUser2 -Password MyPass2
```

3. Store the *MyVM* virtual machine, its network adapters, the destination host, port group, and datastore in variables.

```
$vm = Get-VM 'myVM' -Location 'myVMhostOnVC1'
$destination = Get-VMHost 'MyVMhostOnVc2'
$networkAdapter = Get-NetworkAdapter -VM $vm
$destinationPortGroup = Get-VDPortgroup -VDSwitch 'myVDSwitchOnVC2' -Name 'myPortGroup'
$destinationDatastore = Get-Datastore 'MyDatastoreOnVc2'
```

4. Migrate the virtual machine to the specified destination host and datastore and attach the virtual machine network adapters to the destination port group.

```
Move-VM -VM $vm -Destination $destination -NetworkAdapter $networkAdapter -PortGroup
 $destinationPortGroup -Datastore $destinationDatastore
```

Add Passthrough Devices to a Host and Virtual Machine

You can get information about existing passthrough devices and add new SCSI and PCI devices to virtual machines and hosts.

Verify that you are connected to a vCenter system.

1. Get a list of the PCI passthrough devices of the *VMHost* host

```
$vmhost = Get-VMHost ESXHost
Get-PassthroughDevice -VMHost $vmhost -Type Pci
```

2. Get a list of the SCSI passthrough devices of the *VM* virtual machine

```
$vm = Get-VM VM
Get-PassthroughDevice -VM $vm -Type Scsi
```

3. Add a SCSI passthrough device to the *VM* virtual machine

```
$scsiDeviceList = Get-PassthroughDevice -VMHost ESXHost -Type Scsi
Add-PassthroughDevice -VM $vm -PassthroughDevice $scsiDeviceList[0]
```

Apply a Customization Object to a Cloned Virtual Machine

You can apply a custom configuration to a cloned virtual machine by using a customization object.

Verify that you are connected to a vCenter system.

> **Note:**
> This feature runs only on a 32-bit VCF PowerCLI process.

1. Get the *Spec* customization specification and clone it for temporary use.

```
Get-OSCustomizationSpec Spec | New-OSCustomizationSpec -Type NonPersistent -Name
ClientSpec
```

2. Change the `NamingPrefix` property of the customization object to the name of the virtual machine you want to create.

```
Set-OSCustomizationSpec -Spec ClientSpec -NamingPrefix VM1
```

3. Create a virtual machine named *VM1* by cloning the existing *VM* virtual machine and applying the customization specification.

```
Get-VM VM | New-VM -VMHost Host -Datastore Storage1 -OSCustomizationSpec ClientSpec -Name VM1
```

Modify the Default NIC Mapping Object of a Customization Specification

You can modify the default NIC mapping object of a customization specification and apply the specification on a newly created virtual machine.

1. Create a nonpersistent customization specification for Windows operating systems.

```
New-OSCustomizationSpec -Type NonPersistent -Name Spec -OSType Windows -Workgroup Workgroup
-OrgName Company -Fullname User -ProductKey “valid_key” -ChangeSid -TimeZone "Central European"
-NamingScheme VM
```

2. View the default NIC mapping objects of the *Spec* specification.

```
Get-OSCustomizationNicMapping -Spec Spec | Set-OSCustomizationNicMapping -IpMode
UseStaticIP -IpAddress 172.16.1.30 -SubnetMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1
```

   Each customization specification object has one default NIC mapping object.

3. Modify the default NIC mapping object of the *Spec* customization specification to use static IP.

```
Get-OSCustomizationNicMapping -Spec Spec | Set-OSCustomizationNicMapping  -IpMode
UseStaticIP -IpAddress 172.16.1.30 -SubnetMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1.1
```

4. Create a new virtual machine named *VM1* from a template, and apply the static IP settings.

```
New-VM -Name VM1 -VMHost Host -Datastore Storage1 -OSCustomizationSpec Spec -Template Template
```

Modify Multiple NIC Mapping Objects of a Customization Specification

You can modify multiple NIC mapping objects of a customization specification and apply the specification to an existing virtual machine.

1. Get the network adapters of a virtual machine named *VM*.

```
Get-NetworkAdapter VM
```

When you apply a customization specification, each network adapter of the customized virtual machine must have a corresponding NIC mapping object. You can correlate network adapters and NIC mapping objects either by their position numbers, or by MAC address.

2. Create a customization specification named *Spec*.

```
New-OSCustomizationSpec -Type NonPersistent -Name Spec -OSType Windows -Workgroup Workgroup
-OrgName Company -Fullname User -ProductKey “valid_key” -ChangeSid -TimeZone "Central European"
-NamingScheme VM
```

3. Add a new NIC mapping object that uses a static IP address.

```
New-OSCustomizationNicMapping -Spec Spec -IpMode UseStaticIP -IpAddress 172.16.1.30
-SubnetMask 255.255.255.0 -DefaultGateway 172.16.1.1 -Dns 172.16.1.1
```

4. View the NIC mapping objects and verify that two NIC mapping objects are available.

```
Get-OSCustomizationNicMapping -Spec Spec
```

The default NIC mapping object is DHCP enabled, and the newly added one uses a static IP address.

5. Apply the Spec customization specification to the *VM* virtual machine.

```
Get-VM VM | Set-VM -OSCustomizationSpec -Spec Spec
```

6. Associate a network adapter from the *VMNetwork* network with the NIC mapping object that uses DHCP mode.

```
$netAdapter = Get-NetworkAdapter VM | where { $_.NetworkName -eq 'VMNetwork' }
Get-OSCustomizationNicMapping -Spec Spec | where { $_.IPMode -eq 'UseDHCP' } | Set-
OSCustomizationNicMapping -NetworkAdapterMac $netAdapter.MacAddress
```

Create Multiple Virtual Machines that Use Static IP Addresses

You can deploy multiple virtual machines with a single network adapter and configure the deployed virtual machines to use static IP addresses by applying a customization specification.

Verify that you have defined a list of static IP addresses in a CSV file.

1. Define the naming convention for the virtual machines.

```
$vmNameTemplate = "VM-{0:D3}"
```

2. Save the cluster in which the virtual machines should be created into a variable.

```
$cluster = Get-Cluster MyCluster
```

3. Save the template on which the virtual machines should be based into a variable.

```
$template = Get-Template MyTemplate
```

4. Create the virtual machines.

```
$vmList = @()

for ($i = 1; $i -le 100; $i++) {
    $vmName = $vmNameTemplate -f $i
    $vmList += New-VM -Name $vmName -ResourcePool $cluster -Template $template
}
```

5. Save the static IP addresses from the stored CSV file into a variable.

```
$staticIpList = Import-CSVC:\StaticIPs.csv
```

6. Create the customization specification.

```
$linuxSpec = New-OSCustomizationSpec -Name LinuxCustomization -Domain vmware.com -DomainUsername
 "your_domain_username" -DomainPassword "your_domain_password" -DnsServer "192.168.0.10",
 "192.168.0.20" -NamingScheme VM -OSType Linux
```

7. Clone the customization specification to a nonpersistent type.
```
$specClone = New-OSCustomizationSpec -Spec $linuxSpec -Type NonPersistent
```
8. Apply the customization specification to each virtual machine.
```
for ($i = 0; $i -lt $vmList.Count; $i++) {
    # Acquire a new static IP from the list
    $ip = $staticIpList[$i].IP

    # The specification has a default NIC mapping - retrieve it and update it with the static IP
    $nicMapping = Get-OSCustomizationNicMapping -OSCustomizationSpec $specClone
    $nicMapping | Set-OSCustomizationNicMapping -IpMode UseStaticIP -IpAddress $ip -SubnetMask
 "255.255.252.0" -DefaultGateway "192.168.0.1"

    # Apply the customization
    Set-VM -VM $vmList[$i] -OSCustomizationSpec $specClone -Confirm:$false
}
```

Create Multiple Virtual Machines with Two Network Adapters

You can deploy multiple virtual machines with two network adapters each and configure each adapter to use specific network settings by applying a customization specification.

Verify that you have defined a list of static IP addresses in a CSV file.

You can configure each virtual machine to have one network adapter attached to a public network and one network adapter attached to a private network. You can configure the network adapters on the public network to use static IP addresses and the network adapters on the private network to use DHCP.

1. Define the naming convention for the virtual machines.
```
$vmNameTemplate = "VM-{0:D3}"
```
2. Save the cluster in which the virtual machines should be created into a variable.
```
$cluster = Get-Cluster MyCluster
```
3. Save the template on which the virtual machines should be based into a variable.
```
$template = Get-Template MyTemplate
```
4. Create the virtual machines.
```
$vmList = @()

for ($i = 1; $i -le 100; $i++) {
    $vmName = $vmNameTemplate -f $i
    $vmList += New-VM -Name $vmName -ResourcePool $cluster -Template $template
}
```
5. Save the static IP addresses from the stored CSV file into a variable.
```
$staticIpList = Import-CSVC:\StaticIPs.csv
```
6. Create the customization specification.
```
$linuxSpec = New-OSCustomizationSpec -Name LinuxCustomization -Domain vmware.com -DnsServer
 "192.168.0.10", "192.168.0.20" -NamingScheme VM -OSType Linux -Type NonPersistent
```
7. Apply the customization specification to each virtual machine.
```
for ($i = 0; $i -lt $vmList.Count; $i++) {
    # Acquire a new static IP from the list
    $ip = $staticIpList[$i].IP

    # Remove any NIC mappings from the specification
    $nicMapping = Get-OSCustomizationNicMapping -OSCustomizationSpec $linuxSpec
    Remove-OSCustomizationNicMapping -OSCustomizationNicMapping $nicMapping -Confirm:$false
```

```
    # Retrieve the virtual machine's network adapter attached to the public network named "Public"
    $publicNIC = $vmList[$i] | Get-NetworkAdapter | where {$_.NetworkName -eq "Public"}

    # Retrieve the virtual machine's network adapter attached to the private network named "Private"
    $privateNIC = $vmList[$i] | Get-NetworkAdapter | where {$_.NetworkName -eq "Private"}

    # Create a NIC mapping for the "Public" NIC that should use static IP
    $linuxSpec | New-OSCustomizationNicMapping -IpMode UseStaticIP -IpAddress $ip -SubnetMask
"255.255.252.0" -DefaultGateway "192.168.0.1" -NetworkAdapterMac $publicNIC.MacAddress

    # Create a NIC mapping for the "Private" NIC that should use DHCP
    $linuxSpec | New-OSCustomizationNicMapping -IpMode UseDhcp -NetworkAdapterMac
$privateNIC.MacAddress

    # Apply the customization
    Set-VM -VM $vmList[$i] -OSCustomizationSpec $linuxSpec -Confirm:$false
}
```

vSphere vApp Administration

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere vApp administration.

Use vSphere vApp to package and manage virtual machines as a single entity. vApps are typically used to simplify the deployment and management of multi-tier applications.

Create a vApp on vCenter

With VCF PowerCLI, you can create and manage vApps.

Verify that you are connected to a vCenter system.

1. Create a new vApp named *VApp* on a host.

```
New-VApp -Name VApp -CpuLimitMhz 4000 -CpuReservationMhz 1000 -Location (Get-VMHost Host1)
```

2. Start the new virtual appliance.

```
Start-VApp VApp
```

Modify the Properties of a vApp

With VCF PowerCLI, you can start and stop vApps, and modify their properties.

Verify that you are connected to a vCenter system.

1. Get the vApp named *VApp* and stop it.

```
Get-VApp VApp | Stop-VApp -Confirm:$false
```

2. Change the name and memory reservation for the vApp.

```
Get-VApp VApp | Set-VApp -Name OldVApp -MemReservationGB 2
```

Export or Import vApps

You can import and export vApps to OVA and OVF files.

Verify that you are connected to a vCenter system.

1. Get the vApp you want to export.
```
$oldVApp = Get-VApp OldVApp
```
2. Export the *OldVApp* vApp to a local directory and name the exported appliance *WebApp*.
```
Export-VApp -VApp $oldVApp -Name WebApp -Destination D:\vapps\ -CreateSeparateFolder
```
3. Import the *WebApp* vApp from a local directory to the *Storage2* datastore.
```
Import-VApp -Source D:\vapps\WebApp\WebApp.ovf -VMHost (Get-VMHost Host1) -Datastore (Get-Datastore -VMHost MyHost01 -Name Storage2)
```

Using Tags

The vSphere Tag Service supports the definition of tags that you can associate with different types of vSphere objects, such as virtual machines, resource pools, datastores, and distributed switches. You can use VCF PowerCLI to create and manage tags and tag categories, and to retrieve specified groups of objects.

You create a vSphere tag to add metadata to objects in the vSphere inventory. Tags are grouped in categories and each tag must have at least one category related to it. After you create the tag, you can associate the tag with a vSphere object.

After you create a tag category and create a tag within the category, you can associate the tag with a vSphere managed object. An association is a simple link that contains no data of its own. You can enumerate objects that are attached to a tag or tags that are attached to an object.

Retrieve a Tag and Save It into a Variable

You can retrieve existing tags defined in vSphere and save a specific tag into a variable.

Verify that you are connected to a vCenter system.

1. Get the tag named *MyTag*.
```
Get-Tag -Name 'MyTag'
```
2. Save the tag into a variable.
```
$tag = Get-Tag -Name 'MyTag'
```

Retrieve a Tag Category and Save It into a Variable

You can retrieve existing tag categories defined in vSphere and save a specific tag category into a variable.

Verify that you are connected to a vCenter system.

1. Get the tag category named *MyTagCategory*.
```
Get-TagCategory -Name 'MyTagCategory'
```
2. Save the tag category into a variable.
```
$tagCategory = Get-TagCategory -Name 'MyTagCategory'
```

Create a Tag Category and a Tag

You can create a tag category and add a new tag in that category.

Verify that you are connected to a vCenter system.

1. Create a tag category named *Department*.
```
$departmentTagCategory = New-TagCategory -Name 'Department'
```

2. Create a new tag named *SalesDpt* in the *Department* category.

```
$salesDptTag = New-Tag -Name 'SalesDpt' -Category $departmentTagCategory
```

Assign a Tag to Virtual Machines

You can assign a tag to a group of virtual machines. For example, you can assign a custom tag to all virtual machines that belong to a specific department in your organization.

Verify that you are connected to a vCenter system.

1. Get the virtual machines of a department in your organization.

```
$vms = Get-VM sales-dpt*
```

2. Assign the custom tag to the group of virtual machines.

```
New-TagAssignment -Tag $salesDptTag -Entity $vms
```

Retrieve Objects by Tag

You can retrieve all objects that have a specific tag assigned to them.

Verify that you are connected to a vCenter system.

Get all virtual machines tagged with the *salesDptTag* tag.

```
Get-VM -Tag 'salesDptTag'
```

Note:
You can only specify a tag filter parameter for the `VM`, `VMHost`, `Datastore`, and `VirtualPortGroup` object types.

Generate Tags Automatically by Using a Script

You can use a script to generate tags automatically. For example, you can create a virtual machine owner tag for each user account in a domain.

- Verify that you are connected to a vCenter system.
- Verify that the user accounts and the vCenter system are in the same domain.

You must use the `Get-VIAccount` cmdlet to retrieve user accounts. For more information, see the documentation of the cmdlet.

1. Create a new tag category and specify that tags in this category can only be assigned to entities of type `VirtualMachine`.

```
$ownerTagCategory = New-TagCategory -Name Owner -EntityType VirtualMachine
```

Note:
If you do not specify an entity type, tags from this category can be assigned to all entity types.

2. Retrieve all domain user accounts and save them in a variable.

Note:

You This cmdlet works only with the vCenter built-in identity provider that supports local accounts, Active Directory or OpenLDAP, Integrated Windows Authentication (IWA), and miscellaneous authentication mechanisms (smart card and RSA SecurID).You cannot retrieve user accounts for users who use external identity providers to authenticate with vCenter.

```
$accounts = Get-VIAccount -User -Domain 'DomainName' -Category | select -ExpandProperty Id
```

3. Create a tag for each user account.

```
$accounts | foreach { New-Tag -Category $ownerTagCategory -Name $_ }
```

4. Retrieve a specific tag from the *Owner* category, so that you can later assign it to a specific virtual machine.

```
$ownerTag = Get-Tag -Category $ownerTagCategory -Name 'John_Smith'
```

Add an Entity Type to a Tag Category

You can extend the list of entity types associated with a tag category.

Verify that you are connected to a vCenter system.

Add the *vApp* entity type to the *ownerTagCategory* tag category.

```
$ownerTagCategory | Set-TagCategory -AddEntityType vApp
```

Retrieve Tag Assignments

You can retrieve tag assignments by using category and entity filters.

Verify that you are connected to a vCenter system.

1. Retrieve all virtual machines that have a tag from the *ownerTagCategory* tag category assigned to them.

```
Get-TagAssignment -Category $ownerTagCategory
```

2. Retrieve the owner of the *MyVM* virtual machine.

```
Get-TagAssignment -Category $ownerTagCategory -Entity 'MyVM'
```

Using Content Libraries

Use content libraries with VCF PowerCLI to store and distribute various forms of content within your vSphere environment. Content libraries function as container entities within vSphere, allowing you to save and distribute items such as OVF and OVA packages, virtual machine templates, vApp templates, and various file types.

You can use content libraries to share VM and vApp templates, and other types of files, such as ISO images, text files, and so on, across your vCenter instances. Sharing templates across your virtual environment promotes consistency, compliance, efficiency, and automation in deploying workloads at scale.

A content library instance represents a container for a set of library items. A content library item instance represents the logical object stored in the content library, which might be one or more usable files.

You create and manage the content of a content library on a single vCenter instance, but you can distribute the content to other vCenter instances.

Depending on your needs, you can maintain two types of content libraries: local and subscribed. You can shape the contents of a library item and then combine several library items in a local content library. Furthermore, you can publish the library to make its content available to other users.

Create a Local Content Library

With VCF PowerCLI, you can create a local content library.

Verify that you are connected to a vCenter system.

1. Get the datastore that you want to use to store files for library items in this library.

```
$datastore = Get-Datastore myDatastore
```

2. Create the *My Local Content Library* published local content library that uses the *myDatastore* datastore.

```
New-ContentLibrary -Name 'My Local Content Library' -Description 'Local content library
description.' -Datastore $Datastore1 -Published
```

Create a Subscribed Content Library

With VCF PowerCLI, you can create a subscribed content library.

Verify that you are connected to a vCenter system.

Create the *My Subscribed Content Library* subscribed content library that uses the *myDatastore* datastore.

```
New-ContentLibrary -Name 'My Subscribed Content Library' -Description 'Subscribed content
library description.' -Datastore $Datastore1 -SubscriptionUrl $Url
```

Create a Content Library Item

With VCF PowerCLI, you can create content library items.

Verify that you are connected to a vCenter system.

1. Get the files that you want to upload to the content library.

   ```
   $files = Get-ChildItem -File
   ```

2. Get the content library that you want to upload.

   ```
   $contentLibrary = Get-ContentLibrary -name 'MyContentLibrary'
   ```

3. Create a content library item.

   ```
   New-ContentLibraryItem -ContentLibrary $contentLibrary -name 'New item' -Files $files
   ```

Create a Virtual Machine from a Content Library Item

You can deploy a virtual machine from a content library template.

- Verify that you are connected to a vCenter system.
- Verify that you have a content library with virtual machine templates available.

> **Note:**
> VMware Cloud Foundation PowerCLI cannot distinguish between OVF content library items of type virtual machine template and vApp template. As a result, `New-VM` creates a vApp if you specify a vApp template from the content library by using the `ContentLibraryItem` parameter of the cmdlet. If this happens, `New-VM` returns an error, notifying that the cmdlet produced an inventory item of the wrong type. Avoid creating vApps by using the `New-VM` cmdlet, as this behavior will be deprecated in future releases.

1. Get the virtual machine host.

   ```
   $myVMHost = Get-VMHost myVMHost
   ```

2. Create the *MyVM* virtual machine from the *MyVMContentLibrayItemName* content library item.

   ```
   Get-ContentLibraryItem -Name MyVMContentLibrayItemName | New-VM -Name MyVM -VMHost
   $myVMHost
   ```

Create a vApp from a Content Library Item

You can deploy a vApp from a content library template.

- Verify that you are connected to a vCenter system.
- Verify that you have a content library with vApp templates available.

> **Note:**
> VMware Cloud Foundation PowerCLI cannot distinguish between OVF content library items of type virtual machine template and vApp template. As a result, `New-VApp` creates a virtual machine if you specify a virtual machine template from the content library by using the `ContentLibraryItem` parameter of the cmdlet. If this

happens, `New-VApp` returns an error, notifying that the cmdlet produced an inventory item of the wrong type. Avoid creating virtual machines by using the `New-VApp` cmdlet, as this behavior will be deprecated in future releases.

1. Get the virtual machine host.

```
$myVMHost = Get-VMHost myVMHost
```

2. Create the *MyVApp* vApp from the *MyVAppContentLibrayItemName* content library item.

```
Get-ContentLibraryItem -Name MyVAppContentLibrayItemName | New-VApp -Name MyVApp -VMHost
$myVMHost
```

Export Content Library Item's Files to a Local Machine

With VCF PowerCLI, you can export content library item's files to a local machine.

Verify that you are connected to a vCenter system.

1. Get a content library item.

```
$item = Get-ContentLibraryItem -Name 'myVM'
```

2. Export a content library item to a local machine.

```
Export-ContentLibraryItem -ContentLibraryItem $item -Destination ./myVM-files
```

vSphere Networking

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Networking.

Modify the Settings of the NIC Teaming Policy for a Virtual Switch

You can set the NIC teaming policy on a vSwitch. The NIC teaming policy determines the load balancing and failover settings of a virtual switch and lets you mark NICs as unused.

Verify that you are connected to a vCenter system.

1. Get a list of the physical NIC objects on the host network and store them in a variable.

```
$pn = Get-VMHost 10.23.123.128 | Get-VMHostNetwork | Select -Property physicalnic
```

2. Store the physical NIC objects you want to mark as unused in separate variables.

```
$pn5 = $pn.PhysicalNic[2]
$pn6 = $pn.PhysicalNic[3]
$pn7 = $pn.PhysicalNic[0]
```

3. View the NIC teaming policy of the *VSwitch01* virtual switch.

```
$policy = Get-VirtualSwitch -VMHost 10.23.123.128 -Name VSwitch01 | Get-NicTeamingPolicy
```

4. Change the policy of the switch to indicate that the *$pn5*, *$pn6*, and *$pn7* network adapters are unused.

```
$policy | Set-NicTeamingPolicy -MakeNicUnused $pn5, $pn6, $pn7
```

5. Modify the load balancing and failover settings of the virtual switch NIC teaming policy.

```
$policy | Set-NicTeamingPolicy -BeaconInterval 3 -LoadBalancingPolicy 3 -NetworkFailoverDetectionPolicy
 1 -NotifySwitches $false -FailbackEnabled $false
```

Network Management with vSphere Distributed Switches

The cmdlets provided in the `VMware.VimAutomation.VDS` module let you manage networking with vSphere distributed switches and port groups.

Create a Distributed Switch and Configure Networking

A vSphere distributed switch lets you handle networking traffic for all associated hosts in a data center. After you create a new vSphere distributed switch in VCF PowerCLI, you can add hosts and connect virtual machines to it.

Verify that you are connected to a vCenter system.

1. Get the data center where you want to create the vSphere distributed switch.
   ```
   $myDatacenter = Get-Datacenter -Name "MyDatacenter"
   ```
2. Get all hosts in your data center.
   ```
   $vmHosts = $myDatacenter | Get-VMHost
   ```
3. Create a new vSphere distributed switch.
   ```
   $myVDSwitch = New-VDSwitch -Name "MyVDSwitch" -Location $myDatacenter
   ```
   The distributed switch is created with no port groups.
4. Add the hosts in your data center to the distributed switch.
   ```
   Add-VDSwitchVMHost -VDSwitch $myVDSwitch -VMHost $vmHosts
   ```
5. Get a physical network adapter from your hosts.
   ```
   $hostsPhysicalNic = $vmHosts | Get-VMHostNetworkAdapter -Name "vmnic2"
   ```
6. Add the physical network adapter to the distributed switch that you created.
   ```
   Add-VDSwitchPhysicalNetworkAdapter -VMHostNetworkAdapter $hostsPhysicalNic
   -DistributedSwitch $myVDSwitch
   ```
7. Create a new distributed port group with 1000 ports and add it to the distributed switch.
   ```
   $myVDPortGroup = New-VDPortgroup -Name "MyVMsPortGroup" -VDSwitch $myVDSwitch -NumPorts 1000
   ```
8. Connect all virtual machines running on the hosts in your data center to the distributed port group.
   ```
   $vmHosts | Get-VM | Get-NetworkAdapter | Set-NetworkAdapter -PortGroup $myVDPortGroup
   ```

Adjust the settings of the distributed switch. See Configure a Distributed Switch.

Configure a Distributed Switch

Based on your networking requirements, you can adjust the settings of a newly created or an existing distributed switch.

Verify that you are connected to a vCenter system.

- Modify the maximum MTU size setting for a distributed switch.
  ```
  Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -Mtu 2000
  ```
- Modify the number of uplink ports on a distributed switch.
  ```
  Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -NumUplinkPorts 4
  ```
- Modify the maximum number of ports on a distributed switch.
  ```
  Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -MaxPorts 1000
  ```
- Modify the discovery protocol settings on a vSphere distributed switch.
  ```
  Get-VDSwitch -Name 'MyVDSwitch' | Set-VDSwitch -LinkDiscoveryProtocol LLDP
  -LinkDiscoveryProtocolOperation Both
  ```

Migrate Virtual Machine Networking Configuration from a vSphere Standard Switch to a vSphere Distributed Switch

To manage virtual machine networks on a data center level, you might need to migrate existing networks from vSphere standard switches to vSphere distributed switches.

Verify that you are connected to a vCenter system.

1. Get the source vSphere standard switch from which you want to migrate the virtual machine networking.
```
$virtualSwitch = Get-VirtualSwitch -Name 'MyVirtualSwitch'
```
2. Get the source standard port group to which the virtual machines are connected.
```
$vmsPortGroup = $virtualSwitch | Get-VirtualPortGroup -Name 'VM Network'
```
3. Get the target vSphere distributed switch to which you want to migrate the virtual machine networking.
```
$vdSwitch = Get-VDSwitch -Name 'MyTargetVDSwitch'
```
4. Get the target port group to which you want to connect the virtual machines.
```
$vdPortGroup = Get-VDPortGroup -VDSwitch $vdSwitch -Name 'DPortGroup'
```
5. Get the virtual machine network adapters connected to the source port group.
```
$vmsNetworkAdapters = Get-VM -RelatedObject $vmsPortGroup | Get-NetworkAdapter | where { $_.NetworkName
 -eq $vmsPortGroup.Name }
```
6. Disconnect the retrieved network adapters from the standard port group and connect them to the distributed port group.
```
Set-NetworkAdapter -NetworkAdapter $vmsNetworkAdapters -PortGroup $vdPortGroup
```

Migrate Physical and Virtual NICs to a vSphere Standard Switch

You can migrate both physical and virtual network adapters to a vSphere standard switch simultaneously.

Verify that you are connected to a vCenter system.

1. Get the physical network adapters that you want to migrate.
```
$pNics = Get-VMHostNetworkAdapter -VMHost $vmhost -Physical
```
2. Get the virtual network adapters that you want to migrate.
```
$vNicManagement = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk0
$vNicvMotion = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk1
```
3. Get the vSphere standard switch to which you want to migrate the network adapters.
```
$vSwitch = Get-VirtualSwitch -VMHost $vmhost -Name vSwitch0
```
4. Migrate all network adapters to the vSphere standard switch.
```
Add-VirtualSwitchPhysicalNetworkAdapter -VirtualSwitch $vSwitch -VMHostPhysicalNic $pNics
 -VMHostVirtualNic $vNicManagement,$vNicvMotion
```

Migrate Physical and Virtual NICs to a vSphere Distributed Switch

You can migrate both physical and virtual network adapters to a vSphere distributed switch simultaneously.

Verify that you are connected to a vCenter system.

1. Get the physical network adapters that you want to migrate.
```
$pNics = Get-VMHostNetworkAdapter -VMHost $vmhost -Physical
```
2. Get the virtual network adapters that you want to migrate.
```
$vNicManagement = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk0
$vNicvMotion = Get-VMHostNetworkAdapter -VMHost $vmhost -Name vmk1
```
3. Get the port groups corresponding to the virtual network adapters that you want to migrate to the vSphere distributed switch.
```
$vdPortgroupManagement = Get-VDPortgroup -VDSwitch $vds -Name 'Management Network'
$vdPortgroupvMotion = Get-VDPortgroup -VDSwitch $vds -Name 'vMotion Network'
```
4. Migrate all network adapters to the vSphere distributed switch.

```
Add-VDSwitchPhysicalNetworkAdapter -DistributedSwitch $vds -VMHostPhysicalNic
$pNics -VMHostVirtualNic $vNicManagement,$vNicvMotion -VirtualNicPortGroup $vdPortGroupManagement,
$vdPortGroupvMotion
```

You migrated the *$vNicManagement* network adapter to the Management Network port group and the *$vNicvMotion* network adapter to the vMotion Network port group.

Configure the Traffic Shaping Policy
You can modify the traffic shaping policy of a port group to limit the bandwidth of the incoming traffic and ensure that enough bandwidth is available for other port groups on the same vSphere distributed switch.

Verify that you are connected to a vCenter system.

1. Get the current traffic shaping policy of the port group.

```
$policy = Get-VDTrafficShapingPolicy -Direction In -VDPortGroup $myVDPortGroup
```

2. Set the peak bandwidth to 100 Mbps.

```
Set-VDTrafficShapingPolicy -Policy $policy -PeakBandwidth 104857600
```

Configure the Security Policy
You can modify the security policy of a port group to enable promiscuous mode, which allows monitoring of the traffic generated by virtual machines.

Verify that you are connected to a vCenter system.

1. Get the current security policy of the port group.

```
$policy = Get-VDSecurityPolicy -VDPortGroup $myVDPortGroup
```

2. Enable promiscuous mode for the port group.

```
Set-VDSecurityPolicy $policy -AllowPromiscuous $true
```

Virtual Private Clouds in vCenter

A virtual private cloud (VPC) is a configurable pool of shared network resources allocated within a cloud environment, providing a certain level of isolation between the different organizations using the resources. A subnet is a range of IP addresses in VPC and a workload is connected to a subnet.

VPCs in vCenter

A Virtual Private Cloud (VPC) in vCenter is a software-defined network boundary that groups virtual machine workloads under a shared networking namespace within a VCF environment. Unlike traditional port groups, a VPC enforces IP-address isolation, routing control, and access-mode policies at the network level, so that workloads in one VPC cannot communicate with workloads in another VPC without explicit policy permission.

A VPC in vCenter is not a tenancy construct — it is a network construct that defines networking boundaries for workloads placed within it. Each VPC maps to an NSX VPC object under the NSX Default Project. When vCenter and NSX Manager are paired as a Compute Manager, the VPC objects created from vCenter are created directly in NSX and are reflected back in the vCenter inventory as folders under the datacenter.

Key characteristics of a VPC in vCenter

- A VPC spans all clusters within the vCenter by default. In VCF 9.1, you can restrict the VPC to a specific cluster scope using a Transit Gateway Span.
- Subnets inside a VPC are PortGroup objects with a parent VPC reference. Each subnet has a relationship with all vSphere Distributed Switch (VDS) instances in the scope of the VPC.
- VPCs are projected as folders in the vCenter inventory under the datacenter. Each VPC folder contains its associated subnets (PortGroups).
- The vCenter API for VPC operations is the NSX API. vCenter proxies VPC create, read, update, and delete (CRUD) calls to the paired NSX Manager.

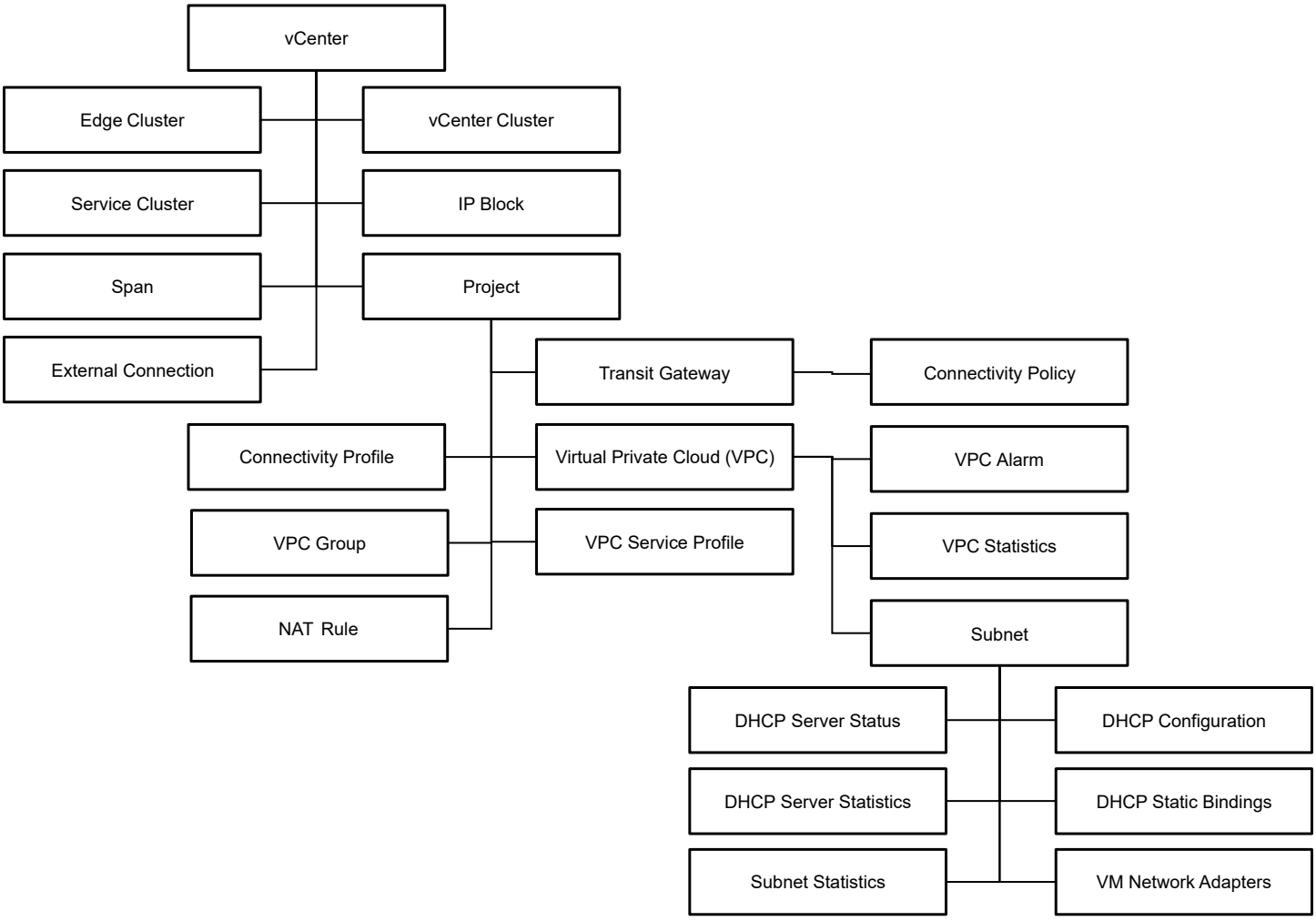
Prerequisites for VPC automation

- vCenter and NSX Manager have been deployed and paired by SDDC Manager.
- SDDC Manager has added vCenter as a Compute Manager of NSX Manager.
- SDDC Manager has created the required NSX objects: Edge Node, Tier-0, and the basic Tier-0 interconnection.
- SDDC Manager has labeled the required resources — Tier-0, Edge Clusters, and IP Blocks — as usable for VPC consumption.
- An NSX Default Project exists in the paired NSX Manager.
- A connectivity profile has been created in NSX and is available to vCenter VPCs.

> **Note:**
>
> VPC configuration options in vCenter are controlled by a flag in SDDC Manager. If VPC creation is not available in your environment, confirm with the VCF Fabric Admin that VPC support has been enabled from Cloud Builder or SDDC Manager.

VPC Object Hierarchy

VPC objects in vCenter follow a strict parent-child hierarchy. Understanding this hierarchy is essential before you automate VPC lifecycle operations with VCF PowerCLI or the VCF SDK.



Object	PowerCLI Noun	Scope	Description
VPC Cluster	`VpcVcCluster`	vCenter	vCenter clusters available for VPC workload placement. Retrieved with `Get-VpcVcCluster`.
Edge Cluster	`VpcEdgeCluster`	vCenter	Edge clusters providing routing and network services for VPCs. Retrieved with `Get-VpcEdgeCluster`.
Service Cluster	`VpcServiceCluster`	vCenter	Service clusters hosting VPC infrastructure services (DHCP relay, DNS). Retrieved with `Get-VpcServiceCluster`.
Span	`VpcSpan`	vCenter	Restricts Transit Gateway networks to specific vCenter clusters. Mark Exclusive to remove clusters from the default scope.
IP Block	`VpcIpBlock`	vCenter	A pool of IP addresses for subnet allocation. Visibility can be External (public-facing) or Private (internal). Referenced by Connectivity Profile; not owned by a VPC.

Object	PowerCLI Noun	Scope	Description
External Connection	`VpcExternalConnection`	vCenter	VLAN-backed distributed uplink for North-South connectivity from a Transit Gateway to external networks. Managed by Project Admin via VCF PowerCLI. **Note:** NOT to be confused with External Attachment (Tier-0 based uplink defined by Enterprise Admin in SDDC Manager - has no VCF PowerCLI cmdlets).
Project	`VpcProject`	vCenter	The administrative boundary for VPC objects. Each Project contains its own set of VPCs, connectivity profiles, service profiles, transit gateways, VPC groups, and NAT rules. Retrieved with `Get-VpcProject`.
Connectivity Profile	`VpcConnectivityProfile`	Project	Defines northbound connectivity settings shared by one or more VPCs: • External IP blocks (from `VpcIpBlock` objects) • Private Transit Gateway IP blocks • Enable Service Gateway (true/false) • Enable Default SNAT (true/false)
Service Profile	`VpcServiceProfile`	Project	Configures shared infrastructure services for VPC subnets - DNS servers, NTP servers, and DHCP relay settings.
Transit Gateway	`VpcTransitGateway`	Project	Centralized routing hub for inter-VPC (E-W) and VPC-to-external (N-S) connectivity. Peer of VPC under the Project — not a parent of VPC. VPCs connect to Transit Gateway by associating a Connectivity Profile that references the Transit Gateway (`Set-VpcConnectivityProfile -TransitGateway`).
VPC Group	`VpcGroup`	Project	Logical grouping of VPCs for policy application and management.

Object	PowerCLI Noun	Scope	Description
NAT Rule	`VpcNatRule`	Project	Read-only view of NAT rules that are applied to VPC subnets. Retrieved with `Get-VpcNatRule`. NAT rule lifecycle management is not exposed through PowerCLI - use NSX REST API or NSX UI.
Virtual Private Cloud (VPC)	`Vpc`	Project	Top-level network boundary for workloads. References one Connectivity Profile. Exposed in vCenter inventory as a folder under the Datacenter.
Connectivity Policy	`VpcConnectivityPolicy`	Transit Gateway	Traffic enforcement policy applied to VPCs associated with a Transit Gateway. Defines whether VPCs are `Isolated`, `Promiscuous`, or `Community`. The rules control both inbound traffic to and outbound traffic from the VPCs the policy is applied to. See Configure a Connectivity Policy.
VPC Alarm	`VpcAlarm`	VPC	Active alarms and health alerts on a VPC. Retrieved with `Get-VpcAlarm`.
VPC Statistics	`VpcStatistics`	VPC	Traffic counters and resource utilization metrics for a VPC. Retrieved with `Get-VpcStatistics`.
Subnet	`VpcSubnet`	VPC	Network segment (PortGroup) within a VPC for connecting VMs. Supports VPC Private, TGW Private, and Public access modes.
DHCP Configuration	-	Subnet	IP lease configuration embedded in subnet parameters. Configured as part of `New-VpcSubnet` or `Set-VpcSubnet`. Helpers can be `New-VpcDhcpGenericOption` and `New-VpcDhcpClass`.
DHCP Static Binding	`VpcDhcpStaticBinding`	Subnet	MAC-to-IP address reservation for a specific VM on a subnet.
DHCP Generic Option	`VpcDhcpGenericOption`	Subnet (helper)	A custom DHCP option that is passed to VMs on lease assignment. Created with `New-VpcDhcpGenericOption`.
DHCP Classless Static Route	`VpcDhcpClass`	Subnet (helper)	Classless static route option distributed through DHCP. Created with `New-VpcDhcpClass`.
VM Network Adapter	-	Subnet	Connects a VM NIC to a VPC subnet. `-Subnet`, `-AutoAssignExternalIp`, `-ExternalIp`.

Object	PowerCLI Noun	Scope	Description
Subnet Status	`VpcSubnetStatus`	Subnet	Operational state of a subnet (`UP`/`DOWN`). Retrieved with `Get-VpcSubnetStatus`.
Subnet Statistics	`VpcSubnetStatistics`	Subnet	Traffic counters per subnet (rx/tx packets, bytes). Retrieved with `Get-VpcSubnetStatistics`.
Subnet DHCP Server Status	`VpcSubnetDhcpServerStatus`	Subnet	Operational state of the DHCP server on a subnet. Retrieved with `Get-VpcSubnetDhcpServerStatus`.
Subnet DHCP Server Statistics	`VpcSubnetDhcpServerStatistics`	Subnet	DHCP lease counters and server metrics per subnet. Retrieved with `Get-VpcSubnetDhcpServerStatistics`.

VPC Networking in VCF 9.1

VCF 9.1 extends the VPC networking model with three new architectural objects: Transit Gateways, Spans, and Connectivity Policies. These objects give admins fine-grained control over inter-VPC routing, cluster scoping, and traffic enforcement, including ransomware isolation scenarios.

Transit Gateway

A Transit Gateway is a distributed routing object that anchors multiple VPCs and acts as a common routing hub. When a VPC is associated with a Transit Gateway, its private subnets can communicate with other VPCs under the same Transit Gateway without NAT, subject to any Connectivity Policies applied.

A Transit Gateway can have:

- One or more Spans that restrict its cluster scope.
- One or more External Connections that provide VLAN-backed uplinks to external networks.

Transit Gateways are visible from vCenter next to VPCs in the networking inventory pane, under a dedicated Transit Gateways folder at the same level as the Virtual Private Cloud folder.

Span

A Span restricts a Transit Gateway — and all VPCs and subnets that are associated with it — to a specific set of clusters in the vCenter instance. This is important for environments where VLAN availability is limited to certain clusters: the span of a Transit Gateway must be smaller than or equal to the span of any VLAN used by an External Connection on that Transit Gateway.

Key Span behaviors:

- The default Transit Gateway Span covers all clusters in the vCenter instance. You can restrict it at creation (day 0) or day 2.
- Spans are visible at the VPC and subnet levels, indicating where those networks are created. The Span listing includes clusters from the local vCenter instance and from Enhanced Linked Mode vCenters.
- If a Span is marked `Exclusive`, those clusters are excluded from the default Span. This prevents user error when assigning VLANs to distributed connections.
- Transit Gateways with a Centralized Connection have no cluster-scope dependency from VLANs and therefore no Span limitation.

Connectivity Policies

A Connectivity Policy is a Project-level object that controls inter-VPC traffic for one or more VPCs. A single Connectivity Policy can be applied to multiple VPCs from the Project level.

Policy types can be `Isolated`, `Community`, and `Promiscuous`. See Configure a Connectivity Policy.

Connectivity Policies address the ransomware recovery requirement: from vCenter, an admin can isolate a set of VMs in a VPC, block or allow connectivity inside the VPC, and block or allow connectivity outside the VPC — including to a shared resources VPC.

VPC Groups

A VPC Group groups VMs within a VPC and is used to scope Connectivity Policy rules to specific workloads rather than to an entire VPC. VPC Groups are managed at the VPC-level by using `New-VpcGroup`, `Set-VpcGroup`, and `Remove-VpcGroup`.

Create a VPC in vCenter

You can create a VPC in vCenter by using the `New-Vpc` cmdlet.

- Verify that you connected to a vCenter instance. See Connect to a System.

1. Create the VPC.

```
New-Vpc -Name clivpc1 -PrivateIp 192.168.130.0/24 -Description "Create a VPC by using VCF
 PowerCLI."
```

2. Verify that the VPC is created correctly.

```
Get-Vpc clivpc1
```

3. Optional: Attach a connectivity profile and connectivity policy to the VPC at creation time or after creation.

```
# Retrieve an existing connectivity profile and policy
$profile = Get-VpcConnectivityProfile -Name "dis-con-profile"
$policy = Get-VpcConnectivityPolicy -TransitGateway "myTgw" -Name "con-policy"
# Attach both to the VPC
Set-Vpc -Vpc clivpc1 -ConnectivityProfile $profile -ConnectivityPolicy $policy
# Verify the update
Get-Vpc -Name "clivpc1" | Format-List
```

Note:

Connectivity policies require a Transit Gateway, which is available in VMware Cloud Foundation PowerCLI 9.1 and later.

A new VPC is created in vCenter.

Modify a VPC in vCenter

You can modify the settings of a VPC you created.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have a VPC created.

1. Modify the VPC settings.

```
Set-Vpc -Vpc clivpc1 -PrivateIps 192.168.131.0/24
```

2. Verify that the VPC private IP address is modified.

```
Get-Vpc clivpc1 | Format-List
```

3. Optional: Update the connectivity profile that is attached to the VPC.

```
$newProfile = Get-VpcConnectivityProfile -Name updated-profile
Set-Vpc -Vpc clivpc1 -ConnectivityProfile $newProfile
```

4. Optional: Update the name or description of the VPC.

```
Set-Vpc -Vpc clivpc1 -Name "clivpc1-renamed" -Description "Updated description."
```

Retrieve Information about a VPC in vCenter

You can view information about a specific VPC in vCenter, their projects, IP blocks, connectivity profiles, service profiles, cluster infrastructure, alarms, and statistics in a vCenter instance.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have at least one VPC created.

1. Retrieve all VPCs, or a specific VPC, in a vCenter instance.

```
# List all VPCs
```

```
Get-Vpc
# Retrieve a specific VPC with full property details
Get-Vpc -Name clivpc1 | Format-List
```

2. Get information about the VPC projects in a vCenter instance.

   For more information about NSX Projects and VPCs, see NSX Projects and VPCs.

   ```
   Get-VpcProject
   ```

3. Get a list all IP blocks for all VPCs in a vCenter instance.

   The cmdlet returns detailed information about all IPs for all VPC projects.

   ```
   Get-VpcIpBlock
   ```

   Note:

   In VCF 9.1 and later, you can create, modify, and remove IP blocks by using `New-VpcIpBlock` , `Set-VpcIpBlock` , and `Remove-VpcIpBlock`. See Manage IP Blocks for a VPC in vCenter.

4. Get the connectivity profiles of all VPCs in a vCenter instance.

   A connectivity profile defines the common northbound connectivity configuration for VPCs.

   The cmdlet returns detailed information about all connectivity profiles of all VPCs in a vCenter instance.

   ```
   Get-VpcConnectivityProfile
   ```

5. Get the service profiles of all VPCs in a vCenter instance.

   A VPC service profile is a collection of services (DHCP and subnet profiles). It simplifies the configuration of common services on multiple VPCs.

   The cmdlet returns detailed information about all service profiles of all VPCs in a vCenter instance.

   ```
   Get-VpcServiceProfile
   ```

6. Get the accumulated statistics for all VPCs in a vCenter instance.

   The cmdlet returns detailed information about the accumulated statistics about all VPCs in a vCenter instance.

   ```
   Get-VpcStatistics
   ```

7. Get the alarm state for a specific VPC.

   ```
   $project = Get-VpcProject -Name "Default"
   Get-VpcAlarm -Project $project -Vpc clivpc1
   ```

8. Retrieve cluster infrastructure associated with VPCs.

   ```
   # Retrieve edge clusters available to VPCs
   Get-VpcEdgeCluster
   # Retrieve vCenter clusters for VPCs
   Get-VpcVcCluster
   # Retrieve service clusters for VPCs
   Get-VpcServiceCluster
   ```

You retrieved detailed information about all the VPCs or a specific VPC in a vCenter instance.

Add a DHCP Subnet for a VPC in vCenter

You can add a subnet to a VPC in vCenter by using the `New-VpcSubnet` and DHCP options cmdlets `New-VpcDhcpGenericOption` and `New-VpcDhcpClasslessStaticRoute`.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have a VPC created.

1. Optional: Create a DHCP configuration that is used by the VPC subnet.

   Note:

   You can configure DHCP only if the DHCP settings are configured in the Service Profile linked to the VPC.

   ```
   $genericOptions = @((New-VpcDhcpGenericOption -Code MTUInterface -Value 1600), (New-VpcDhcpGenericOption -
   Code ARPTimeout -Value 60))
   $staticRoutes = @((New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -NextHop 192.168.133.1),
    (New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -NextHop 192.168.133.2))
   ```

2. Add a subnet for a specific VPC in a vCenter instance.

   ```
   New-VpcSubnet -Vpc clivpc3 -Name clivpc3sub1 -DhcpMode Server -DhcpGenericOption $genericOptions
    -DhcpClasslessStaticRoute $staticRoutes
   ```

3. Optional: Add a subnet without DHCP for an public or private network.

   Use this variant when the subnet type is Private or Public and does not require DHCP.

   ```
   # Add a private subnet with no DHCP
   New-VpcSubnet -Vpc clivpc1 -Name "private-sub1" -AccessMode Private
   # Add an public subnet
   New-VpcSubnet -Vpc clivpc1 -Name "public-sub1" -AccessMode Public
   ```

   Note:

   The `-AccessMode` parameter accepts `Public`, `Private`, or `Private TWG`.

   - `Public` - subnet is routable beyond the Transit Gateway to external networks. VMs can receive external IPs. Subnet definition and IP block ownership require Enterprise Admin authorization.
   - `Private` - Without Transit Gateway (VCF 9.0): traffic is routed within the VPC only (VPC Private). VMs do not receive external IPs. SNAT applies for outbound traffic if enabled on the connectivity profile.
     VCF
   - `Private TWG` - With Transit Gateway (VCF 9.1): subnets defined by the Tenant (Project) Admin can be routed across the Project to other VPCs via the Transit Gateway (Transit Gateway Private). VMs still do not receive external IPs from external networks.

   You cannot change the access mode of a subnet after creation.

4. Verify that the VPC subnet is created.

   ```
   Get-VpcSubnet -Vpc clivpc3
   ```

You added a subnet to a specific VPC in a vCenter instance.

Retrieve Information about the VPC Subnet in vCenter

You can retrieve a detailed information about a subnet of a VPC in a vCenter instance.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have a VPC created and you added a subnet to it.

1. Retrieve all subnets for a specific VPC, or a specific subnet by name.

   ```
   # List all subnets for a VPC
   Get-VpcSubnet -Vpc clivpc3 | Format-List
   # Retrieve a specific subnet
   $subnet = Get-VpcSubnet -Name clivpc3sub1
   ```

2. Get the accumulated statistics for a specific VPC subnet.

   ```
   Get-VpcSubnetStatistics -Subnet $subnet
   ```

3. Get the operational status of a specific VPC subnet.

   ```
   Get-VpcSubnetStatus -Subnet $subnet
   ```

4. Get the accumulated statistics of the DHCP server of the VPC subnet.

```
Get-VpcSubnetDhcpServerStatistics -Subnet $subnet
```

5. Get the operational status of the DHCP server of the VPC subnet.

```
Get-VpcSubnetDhcpServerStatus -Subnet $subnet
```

You retrieved detailed information about the VPC subnet in a vCenter instance.

Modify the Subnet of a VPC in vCenter

You can modify the subnet configuration of a specific VPC in a vCenter instance.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have a VPC created and you added a subnet to it.

1. Optional: Modify the reserved IP address range used by the VPC subnet.

   You can first check the IP address range used by the VPC subnet.

```
Set-VpcSubnet -Subnet clivpc3sub1 -ReservedIpRange 192.168.133.10-192.168.133.20
```

2. Optional: Modify the DHCP classless static routes on the subnet.

```
$updatedRoutes = @(
(New-VpcDhcpClasslessStaticRoute -Network 10.10.0.0/24 -NextHop 192.168.133.1)
)
Set-VpcSubnet -Subnet clivpc3sub1 -DhcpClasslessStaticRoute $updatedRoutes
```

3. Modify custom DHCP options on the subnet.

```
$updatedOptions = @(
(New-VpcDhcpGenericOption -Code MTUInterface -Value 1500)
)
Set-VpcSubnet -Subnet clivpc3sub1 -DhcpGenericOption $updatedOptions
```

4. Verify whether the subnet configuration is modified successfully.

```
Get-VpcSubnet clivpc3sub1 | Format-List
```

The subnet configuration is updated in a vCenter instance.

Create a DHCP Static Bindings Configuration for the VPC Subnet in vCenter

You can create DHCP static bindings for a VPC subnet by using the `New-VpcDhcpStaticBinding` and DHCP options cmdlets `New-VpcDhcpGenericOption` and `New-VpcDhcpClasslessStaticRoute`. A DHCP static binding reserves a specific IP address for a VM NIC based on its MAC address, overriding the DHCP server's dynamic address assignment for that NIC.

- Verify that you connected to a vCenter instance. See Connect to a vCenter System.
- Verify that you have a VPC and a subnet created with DHCP mode set to `Server`. See Add a DHCP Subnet for a VPC in vCenter.
- Verify that DHCP settings are configured in the Service Profile linked to the VPC.
  You can set up a DHCP static bindings configuration only if DHCP is configured in the Service Profile. Use `Get-VpcServiceProfile` to confirm that DHCP is enabled for the VPC.
- Verify that the IP address you want to assign is within the `ReservedIpRange` configured for the subnet. IP addresses outside the reserved range are rejected by the DHCP server.

For more information about DHCP static bindings configuration, see DHCP Static Bindings section in the NSX DHCP Configuration Settings: Reference article.

1. Retrieve existing DHCP static bindings for the subnet to check for conflicts before creating a new binding.

```
Get-VpcDhcpStaticBinding -Subnet clivpc3sub1
```

If no bindings exist, the command returns no results. Note any existing bindings to avoid MAC address or IP address conflicts.

2. Optional: Create a DHCP configuration that is used by the VPC subnet.

   Note:

   You can set up a DHCP configuration only if the DHCP settings are configured in the Service Profile that is linked to the VPC.

   ```
   $bindingGenericOptions = @((New-VpcDhcpGenericOption -Code MTUInterface -Value 1700), (New-VpcDhcpGenericOption -Code ARPTimeout -Value 120))
   $bindingStaticRoutes = @((New-VpcDhcpClasslessStaticRoute -Network 192.168.133.0/28 -NextHop 192.168.133.3))
   ```

3. Create the DHCP static binding of the VPC subnet.

   Note:

   The `-IpAddress` must be within the `ReservedIpRange` of the VPC subnet. The `-MacAddress` must match the MAC address of the VM NIC that will receive the reserved IP. The default lease time is `86400` seconds (24 hours); use `-LeaseTime` to override.

   ```
   New-VpcDhcpStaticBinding -Subnet clivpc3sub1 -Name binding1 -MacAddress 00:A0:C9:12:23:34 -IpAddress 192.168.133.10 -Gateway 192.168.133.1 -HostName binding1.vpc.com -DhcpGenericOption $bindingGenericOptions -DhcpClasslessStaticRoute $bindingStaticRoutes
   ```

4. Verify that the DHCP static binding of the VPC subnet is created.

   ```
   Get-VpcSubnet -Name clivpc3sub1 | Get-VpcDhcpStaticBinding
   ```

   Confirm that the binding `binding1` appears in the results with the correct MAC address, IP address, and gateway. To filter to a specific binding:

   ```
   Get-VpcDhcpStaticBinding -Subnet clivpc3sub1 -Name binding1 | Format-List
   ```

5. Optional: Create a second static binding without custom DHCP options.

   If the VM requires only a reserved IP and default gateway without additional DHCP options, omit the `-DhcpGenericOption` and `-DhcpClasslessStaticRoute` parameters.

   ```
   New-VpcDhcpStaticBinding -Subnet clivpc3sub1 -Name binding2 -MacAddress 00:A0:C9:12:23:35 -IpAddress 192.168.133.11 -Gateway 192.168.133.1 -HostName binding2.vpc.com
   ```

You created DHCP static bindings for the VPC subnet. VMs with the specified MAC addresses receive their reserved IP addresses from the DHCP server when they connect to the subnet.

To modify an existing DHCP static binding, see Modify the Configuration of the DHCP Static Bindings of the VPC Subnet in vCenter.

To remove a binding, use `Remove-VpcDhcpStaticBinding`.

Modify the Configuration of the DHCP Static Bindings of the VPC Subnet in vCenter

You can modify and remove the DHCP static bindings of a VPC subnet by using the `Set-VpcDhcpStaticBinding` and `Remove VpcDhcpStaticBinding`.

- Verify that you connected to a vCenter instance. See Connect to a System.
- Verify that you have a VPC created and you added a subnet to it.
- Verify that you have set the `ReservedIpRange` on the VPC subnet.
- Verify that you configured the DHCP static bindings of the VPC subnet.

1. Optional: Check the current DHCP static binding configuration.

```
Get-VpcDhcpStaticBinding -Name binding1
```

2. Modify the IP address of the DHCP static binding of the VPC subnet.

   The IP address must be within the `ReservedIpRange` of the VPC subnet.

   You can first check the IP address range used by the DHCP static bindings of the VPC subnet.

```
Get-VpcSubnet -Name clivpc3sub1 | Get-VpcDhcpStaticBinding -Name binding1 | Set-VpcDhcpStaticBinding -IpAddress 192.168.133.11
```

3. Verify that your changes are applied correctly.

```
Get-VpcDhcpStaticBinding -Subnet clivpc3sub1 -Name binding1
```

4. Optional: Remove a DHCP static binding that is no longer required.

```
Get-VpcSubnet -Name clivpc3sub1 | Get-VpcDhcpStaticBinding -Name binding1 | Remove-VpcDhcpStaticBinding
```

The DHCP static binding configuration of the VPC subnet is updated.

Connect a Virtual Machine to a VPC Subnet in vCenter

You connect a virtual machine to a VPC subnet in a vCenter instance by using the `New-NetworkAdapter` or `Set-NetworkAdapter` cmdlets.

- Verify that you connected to a vCenter instance. See Connect to a vCenter System.
- Verify that you have a VPC created and you added a subnet to it.

1. Connect a new network adapter to a specific VPC subnet.

```
New-NetworkAdapter -VM UBUNTU-vm1 -Type Vmxnet3 -Subnet clivpc3sub1 -WakeOnLan -StartConnected
```

2. Verify that the virtual machine is connected to the VPC subnet.

```
Get-NetworkAdapter -VM UBUNTU-vm1
```

3. Optional: Connect an existing network adapter to a VPC subnet by using `Set-NetworkAdapter`.

   Use this procedure when the virtual machine already has a network adapter that you want to move to a VPC subnet.

```
# Retrieve the network adapter and target subnet
$nic = Get-NetworkAdapter -VM vm1 -Name "Network adapter 2"
$subnet = Get-VpcSubnet -Name clivpc3sub1

# Connect the adapter to the subnet with automatic external IP assignment
Set-NetworkAdapter -NetworkAdapter $nic -Subnet $subnet -AutoAssignExternalIp:$true -Confirm:$false

# Verify the connection
Get-NetworkAdapter -VM UBUNTU-vm1 | Select-Object Name, MacAddress, ExternalIp
```

4. Optional: Assign a static external IP address to the network adapter.

   Use this procedure when the virtual machine requires a specific external IP address.

```
Set-NetworkAdapter -NetworkAdapter $nic -Subnet $subnet -ExternalIp 142.2.0.10 -Confirm:$false
```

The virtual machine is connected to the VPC subnet.

vSphere Storage

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Storage.

Create an iSCSI Host Storage

For a host, you can enable iSCSI, add iSCSI targets, and create new host storages.

Verify that you are connected to a vCenter system.

1. Enable software iSCSI on a host.
   ```
   $vmhost = Get-VMHost ESXHost1
   Get-VMHostStorage $myHost | Set-VMHostStorage -SoftwareIScsiEnabled $true
   ```
2. Get the iSCSI Host Bus Adapter (HBA) that is on the host.
   ```
   $iscsiHba = Get-VMHostHba -Type iScsi
   ```
3. Add a new iSCSI target for dynamic discovery.
   ```
   $iscsiHba | New-IScsiHbaTarget -Address 192.168.0.1 -Type Send
   ```
4. Rescan the HBAs on the host.
   ```
   Get-VMHostStorage $vmhost -RescanAllHba
   ```
5. Get the path to the SCSI LUN.
   ```
   $lunPath = Get-ScsiLun -VMHost $vmhost -CanonicalName ($iscsiHba.Device + "*") | Get-ScsiLunPath
   ```
   You can provide the LUN path by using its canonical name beginning with the device name of the iSCSI HBA.
6. Create a new host storage.
   ```
   New-Datastore -Vmfs -VMHost $vmhost -Path $lunpath.LunPath -Name iSCSI
   ```

Storage Policy Based Management with VMware Cloud Foundation PowerCLI

Manage your storage policies with VCF PowerCLI. Storage Policy Based Management (SPBM) is a storage policy framework that operates on the virtual machine level and serves as a single unified control panel across a broad range of data services and storage solutions.

Create a Tag-Based Storage Policy

You can create storage policies by using tags from vCenter.

- Verify that you are connected to a vCenter system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a tag named *Tag1* exists in the vCenter environment.

1. Get the *Tag1* tag and store it in the *$tag* variable.
   ```
   $tag = Get-Tag -Name 'Tag1'
   ```
2. Create a rule with the *$tag* tag and store the rule in the *$rule* variable.
   ```
   $rule = New-SpbmRule -AnyOfTags $tag
   ```
3. Create a rule set by using the *$rule* rule and store the rule set in the *$ruleset* variable.
   ```
   $ruleset = New-SpbmRuleSet -AllOfRules $rule
   ```
4. Create a tag-based policy named *Tag-Based-Policy* by using the *$ruleset* rule set and store the policy in the *$policy* variable.
   ```
   $policy = New-SpbmStoragePolicy -Name 'Tag-Based-Policy' -Description 'This policy is created by using a
    tag' -AnyOfRuleSets $ruleset
   ```

Create a Capability-Based Storage Policy

You can create storage policies by using vendor-exposed capabilities.

- Verify that you are connected to a vCenter system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage provider is registered with the vCenter system.

1. Get the *VSAN.hostFailuresToTolerate* capability and store it in the *$cap* variable.

```
$cap = Get-SpbmCapability -Name 'VSAN.hostFailuresToTolerate'
```

2. Create a rule with the *$cap* capability and store the rule in the *$rule* variable.

```
$rule = New-SpbmRule -Capability $cap -value 1
```

3. Create a rule set by using the *$rule* rule and store the rule set in the *$ruleset* variable.

```
$ruleset = New-SpbmRuleSet -AllOfRules $rule
```

4. Create a capability-based policy named *Capability-Based-Policy* by using the *$ruleset* rule set and store the policy in the *$policy* variable.

```
$policy = New-SpbmStoragePolicy -Name 'Capability-Based-Policy' -Description 'This policy is created by
 using capabilities' -AnyOfRuleSets $ruleset
```

Associate a Storage Policy with a Virtual Machine and Its Hard Disk

You can associate a storage policy with a virtual machine and its hard disk and check if they are compliant with the policy.

- Verify that you are connected to a vCenter system.
- Verify that a storage policy named *Str-Policy* exists in the vCenter environment.
- Verify that a virtual machine named *Target-VM* exists in the vCenter environment.

1. Get the *Str-Policy* storage policy and store it in the *$policy* variable.

```
$policy = Get-SpbmStoragePolicy -Name 'Str-Policy'
```

2. Get the *Target-VM* virtual machine and store it in the *$vm* variable.

```
$vm = Get-VM -Name 'Target-VM'
```

3. Get the hard disk associated with the *$vm* virtual machine and store it in the *$hd* variable.

```
$hd = Get-HardDisk -VM $vm
```

4. Assign the *$policy* storage policy to the *$vm* virtual machine and the *$hd* hard disk.

```
Set-SpbmEntityConfiguration $vm, $hd -StoragePolicy $policy
```

5. View the *$policy* storage policy's compliance with the *$vm* virtual machine and the *$hd* hard disk.

```
Get-SpbmEntityConfiguration $vm, $hd
```

Note:
The storage policy can be compliant only if the datastore on which the virtual machine and hard disk are created is compliant with the storage policy.

Disassociate a Storage Policy Associated with a Virtual Machine and Its Hard Disk

You can disassociate a storage policy that is associated with a virtual machine and its hard disk.

- Verify that you are connected to a vCenter system.
- Verify that a virtual machine named *Target-VM* exists in the vCenter environment.
- Verify that a storage policy is associated with the *Target-VM* virtual machine.

1. Get the *Target-VM* virtual machine and store it in the *$vm* variable.

```
$vm = Get-VM -Name 'Target-VM'
```

2. Get the hard disk associated with the *$vm* virtual machine and store it in the *$hd* variable.

```
$hd = Get-HardDisk -VM $vm
```

3. Disassociate all storage policies that are associated with the *$vm* virtual machine and the *$hd* hard disk.

```
Set-SpbmEntityConfiguration $vm, $hd -StoragePolicy $null
```

Remove a Storage Policy
You can disassociate all entities associated with a storage policy and remove the policy completely.

- Verify that you are connected to a vCenter system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage policy named *pol-tag* exists in the vCenter environment.

1. Get the *pol-tag* storage policy and store it in the *$policy* variable.

```
$policy = Get-SpbmStoragePolicy -Name 'pol-tag'
```

2. Disassociate all entities associated with the *$policy* storage policy.

```
Set-SpbmEntityConfiguration (Get-SpbmEntityConfiguration -StoragePolicy $policy)
-StoragePolicy $null
```

3. Remove the *$policy* storage policy.

```
Remove-SpbmStoragePolicy -StoragePolicy $policy
```

Edit a Storage Policy
You can modify a storage policy to replace an existing rule set with a new rule set.

- Verify that you are connected to a vCenter system.
- Verify that you have Profile-driven storage update privileges.
- Verify that a storage provider is registered with the vCenter system.
- Verify that a storage policy named *pol-tag* exists in the vCenter environment.

1. Get the *pol-tag* storage policy and store it in the *$policy* variable.

```
$policy = Get-SpbmStoragePolicy -Name 'pol-tag'
```

2. Create a new rule and store it in the *$newRule* variable.

```
$newRule = New-SpbmRule -Capability (Get-SpbmCapability -Name 'VSAN.hostFailuresToTolerate') -Value 1
```

3. Create a new rule set by using the *$newRule* rule and store it in the *$newRuleset* variable.

```
$newRuleset = New-SpbmRuleSet -AllOfRules $newRule
```

4. Modify the *$policy* storage policy by replacing the existing rule set with the newly created *$newRuleset* rule set.

```
$modPolicy = Set-SpbmStoragePolicy -StoragePolicy $policy -AnyOfRuleSets $newRuleset
```

Export and Import a Storage Policy
You can back up a storage policy by exporting it as a file. You can later import the same storage policy.

- Verify that you are connected to a vCenter system.
- Verify that you have Profile-driven storage update privileges.
- Verify that you have read-write permissions for the directory in which the storage policy is saved.
- Verify that a storage policy named *pol-tag* exists in the vCenter environment.

1. Export the *pol-tag* storage policy.

```
Export-SpbmStoragePolicy -StoragePolicy 'pol-tag' -FilePath 'C:\Policy\pol-tag.xml'
```

2. Import the *pol-tag* storage policy and name it *Imported-Policy*.

```
Import-SpbmStoragePolicy -FilePath 'C:\Policy\pol-tag.xml' -Name 'Imported-Policy'
-Description 'Imported policy description'
```

Create a Virtual Machine in a Datastore Compatible with Storage Policy
You can retrieve a datastore compatible with storage policy and create a virtual machine in the datastore.

- Verify that you are connected to a vCenter system.

- Verify that a tag-based storage policy named *Tag-Policy* exists in the vCenter environment.
- Verify that the tag of the *Tag-Policy* storage policy is associated with one of the available datastores in the vCenter environment.

1. Get the tag-based *Tag-Policy* storage policy and store it in the *$policy* variable.

   ```
   $policy = Get-SpbmStoragePolicy -Name 'Tag-Policy'
   ```

2. Get the tag used in the *Tag-Policy* storage policy and store it in the *$tag* variable.

   ```
   $tag = ($($($policy.AnyOfRulesets).AllOfRules).AnyOfTags)[0]
   ```

3. Get a datastore compatible with the *$policy* storage policy and store it in the *$ds* variable.

   ```
   $ds = Get-SpbmCompatibleStorage -StoragePolicy $policy
   ```

4. Get the virtual machine host that contains the *$ds* datastore and store it in the *$vmhost* variable.

   ```
   $vmHost = Get-VMHost -Datastore $ds
   ```

5. Create a virtual machine named *VM-Tag* in the *$ds* datastore and store the virtual machine object in the *$vm* variable.

   ```
   $vm = New-VM -Name 'VM-Tag' -ResourcePool $vmHost -Datastore $ds -NumCPU 2 -MemoryGB 4 -DiskMB 1
   ```

6. Associate the *$policy* storage policy with the *$vm* virtual machine.

   ```
   Set-SpbmEntityConfiguration $vm -StoragePolicy $policy
   ```

7. Verify that the *$vm* virtual machine is compliant with the *$policy* storage policy.

   ```
   Get-SpbmEntityConfiguration $vm
   ```

   The status should be `Compliant`.

8. Get the `Tag-Assignment` object for the *$ds* datastore and store it in the *$tagAs* variable.

   ```
   $tagAs = Get-TagAssignment -Entity $ds
   ```

9. Remove the *$tag* tag association from the *$ds* datastore.

   ```
   Remove-TagAssignment -TagAssignment $tagAs
   ```

10. Check the compliance of the *$vm* virtual machine with the *$policy* storage policy.

    ```
    Get-SpbmEntityConfiguration $vm
    ```

    The status should be `NonCompliant`.

Add a VASA Provider and Create a Policy

You can add a VASA provider to a vCenter system and create a storage policy.

- Verify that you are connected to a vCenter system.
- Verify that the datastore is mounted to the ESX host.

1. Add a VASA provider to the vCenter system.

   ```
   $script:vasProvider = New-VasaProvider -Name 'name' -Url 'URL' -Username 'user_name' -Password
    'password'  -Description 'description' -Force
   ```

2. Get all SPBM capabilities exposed by the registered VASA provider.

   ```
   Get-SpbmCapability
   ```

3. Create a new SPBM rule with the exposed capabilities of the registered VASA provider.

   ```
   $rule = New-SpbmRule -Capability $capability -Value $value
   ```

4. Create a new SPBM rule set.

```
New-SpbmRuleSet -Name $ruleset -AllofRules @(($rule))
```

5. Create a storage policy.

```
New-SpbmStoragePolicy -Name $storagepolicy -RuleSet $ruleset
```

6. Refresh the VASA provider registered with the vCenter system.

```
$provider = Get-VasaProvider -Name $providername -Refresh
```

7. Verify the VASA storage array.

```
$vasaStorageArray = Get-VasaStorageArray -Provider $vasaProvider -Server $script:vcsrv
```

8. Refresh the VASA provider registered with the vCenter system.

```
$provider = Get-VasaProvider -Name $providername -Refresh
```

9. Get the VASA provider registered with the vCenter system.

```
$vasaProvider = Get-VasaProvider -Name $providername
```

10. Optional: Remove the VASA provider.

```
Remove-VasaProvider -Provider $provider -Confirm:$false
```

11. Optional: Verify that the VASA provider is removed.

```
$provider = Get-VasaProvider -Name $providername
```

Invoke a Planned Failover on a Replication Group and Reverse the Replication

You can invoke a planned failover on a target replication group and recover the devices on the target site. After that, you can reverse the direction of the replication and make the target site the source site.

- Verify that you are connected to the vCenter systems of the source and target sites.
- Verify that you have access to at least one virtual machine host on each site.
- Verify that you have registered a Virtual Volume VASA provider and have access to a Virtual Volume datastore on each site.

1. Create a storage policy with replication capability on the source site.

```
$replicationCapability = Get-SpbmCapability -Name *replication.RPO -Server $srcServer
$persistenceCapability = Get-SpbmCapability -Name *persistence1-readLatency -Server $srcServer
$replicationRule = New-SpbmRule -Capability $ replicationCapability -Value (New-TimeSpan -Hours 4)
$persistenceRule = New-SpbmRule $persistenceCapability -Value 25
$ruleSet = New-SpbmRuleSet -AllOfRules $replicationRule, $persistenceRule
$replicationPolicy = New-SpbmStoragePolicy -Name cokeRep -AnyOfRuleSets $ruleSet -Server $srcServer
```

2. Get a datastore compatible with the created replication storage policy and store it in the *$ds* variable.

```
$ds = Get-SpbmCompatibleStorage -StoragePolicy $replicationPolicy
```

3. Create a virtual machine named *MyVM* with a hard disk in the *$ds* datastore.

```
$vm = New-VM -Name 'MyVM' -VMHost 'Host-Source' -DiskMB 512 -Datastore $ds
$hd = Get-HardDisk -VM $vm
```

4. Get a replication group for the *$ds* datastore and the *$replicationPolicy* storage policy, and store the replication group in the *$rg* variable.

```
$rg = Get-SpbmReplicationGroup –Datastore $ds –StoragePolicy $replicationPolicy
```

5. Associate the *$vm* virtual machine and its hard disk with the *$replicationPolicy* storage policy, and put them in the *$rg* replication group.

```
Set-SpbmEntityConfiguration -Configuration $vm, $hd -StoragePolicy $replicationPolicy
 -ReplicationGroup $rg
```

6. Check the compliance of the *$vm* virtual machine and *$hd* hard disk with the *$replicationPolicy* storage policy.

```
Get-SpbmEntityConfiguration $vm, $hd
```

7. Get the replication pair corresponding to the *$rg* source replication group, and store that pair in the *$rgPair* variable.

```
$rgPair = Get-SpbmReplicationPair -Source $rg
```

8. Synchronize the target replication group.

```
Sync-SpbmReplicationGroup $rgPair.Target
```

9. Power off the *$vm* virtual machine and unregister it.

```
Stop-VM $vm
Remove-VM $vm
```

10. Prepare the failover on the source replication group.

```
Start-SpbmReplicationPrepareFailover $rgPair.Source
```

11. Synchronize the target replication group again, to get the latest state of the source devices.

```
Sync-SpbmReplicationGroup $rgPair.Target
```

12. Invoke the planned failover on the source replication group and store the virtual machine file path on the target site in the *$vmFilePath* variable.

```
$vmFilePath = Start-SpbmReplicationFailover $rgPair.Target
```

13. Register the virtual machine on the *Host-Target* host and power on the virtual machine.

```
$vm = New-VM -VMFilePath $vmFilePath -VMHost 'Host-Target'
Start-VM $vm
```

14. Reverse the direction of the replication.

```
Start-SpbmReplicationReverse $rgPair.Target
```

Attach a Flat VDisk to a Virtual Machine

You can create a flat `VDisk` object and attach it to a virtual machine as a hard disk. After that, you can verify whether the operation was completed successfully.

- Verify that you are connected to a vCenter system.
- Verify that you have access to at least one virtual machine host.
- Verify that there is at least one datastore mounted on the virtual machine host.

1. Mount the datastore on the virtual machine host and store the datastore in the *$ds* variable.

```
$vmHost = Get-VMHost 'Host-A'
$ds = Get-Datastore -RelatedObject $vmHost
```

2. Create a flat, thin-provisioned virtual disk with 2-GB capacity on the *$ds* datastore.

```
$vDisk = New-VDisk -Name 'VirtualDisk' -DiskType Flat -StorageFormat Thin -CapacityGB 2 -Datastore $ds
```

3. Create a virtual machine named *VirtualMachine* with one hard disk and store this virtual machine in the *$vm* variable.

```
$vm = New-VM -Name 'VirtualMachine' -VMHost 'Host-A' -Datastore $ds -DiskMB 512
```

4. Power on the *$vm* virtual machine.

```
Start-VM -VM $vm
```

5. Attach the VDisk object to the *$vm* virtual machine.

```
New-HardDisk -VM $vm -VDisk $vDisk
```

6. Verify that the VDisk object has been attached and the virtual machine now has two hard disks.

```
Get-HardDisk -VM $vm
```

Create an NFS 4.1 Datastore

You can create an NFS 4.1 datastore with Kerberos authentication and multipathing.

- Verify that you are connected to a vCenter system.
- Verify that the remote NFS share supports multipathing and Kerberos authentication.

1. Get the virtual machine host where you want to create the NFS 4.1 datastore.

```
$vmhost = Get-VMHost 'hostname'
```

2. Set NTP servers for the virtual machine host.

```
Add-VMHostNtpServer -VMHost $vmhost -NtpServer 'ntp_server_ip'
```

3. Set a DNS server and search the domain for the virtual machine host.

```
$vmhostnetwork = Get-VMHostNetwork -VMHost $vmhost
Set-VMHostNetwork -Network $vmhostnetwork -DnsFromDhcp $false -DnsAddress 'dns_server_ip' -DomainName
 'domain_name' -SearchDomain 'search_domain'
```

4. Add the virtual machine to the Active Directory domain.

```
$vmhost | Get-VMHostAuthentication | Set-VMHostAuthentication -JoinDomain -Domain 'AD_domain_name'
 -Username 'AD_user_name' -Password 'AD_password'
```

5. Create an NFS user on the virtual machine host for Kerberos-based authentication for the NFS 4.1 datastore.

```
New-NfsUser -VMHost $vmhost -Username 'NFS_user_name' -Password 'password'
```

6. Create an NFS 4.1 datastore with Kerberos authentication and multipathing.

```
New-Datastore -Name 'NFS_datastore_name' -Nfs -FileSystemVersion '4.1' -VMHost $vmhost
 -NfsHost @('remote_host_1_ip', 'remote_host_2_ip') -Path 'NFS_datastore_remote_path' -Kerberos
```

7. Optional: Retrieve the datastore.

```
$ds = Get-Datastore 'NFS_datastore_name'
```

8. Optional: Remove the datastore.

```
Remove-Datastore $ds -VMHost $vmhost
```

9. Optional: Get the NFS user from the virtual machine host.

```
$user = Get-NfsUser -VMHost $vmhost
```

10. Optional: Update the password of the NFS user.

```
$user = Set-NfsUser -NfsUser $user -Password 'new_password'
```

11. Optional: Remove the NFS user.

```
Remove-NfsUser -NfsUser $user
```

Managing Certificates

You can use VMware Cloud Foundation PowerCLI to manage certificates for both vCenter and ESX no matter if you are using Hybrid, Subordinate CA, or Full Custom Mode.

Starting from VMware Cloud Foundation PowerCLI 12.4.0, you can do various certificate management operations in your vSphere environment. The certificate management functionality is included in the `VMware.PowerCLI.VCenter` module.

Add a Root Certificate to vCenter and ESX

You can use VCF PowerCLI to add a root certificate or certificate chain to the trusted root store of vCenter and to the certificate stores of the connected ESX hosts.

- Verify that you are connected to a vCenter system.
- Verify that the root certificate or certificate chain you want to add is available on your machine.

1. Optional: List the trusted root certificates on your vCenter system and the connected ESX hosts.

```
Get-VITrustedCertificate
```

2. Create a variable with the root certificate or certificate chain you want to add.

```
$trustedCertChain = Get-Content "C:\Users\jdoe\Downloads\ca-chain.cert.pem" -Raw
```

3. Add the root certificate or certificate chain to vCenter and to the connected ESX hosts

```
Add-VITrustedCertificate -PemCertificateOrChain $trustedCertChain
```

4. Optional: Verify that your new root certificate or certificate chain has been added to the trusted certificate stores.

```
Get-VITrustedCertificate
```

Change the Machine SSL Certificate of vCenter

You can use VCF PowerCLI to change the Machine SSL certificate of a vCenter system. For a custom certificate, you must generate a certificate signing request (CSR) and send it to the certificate authority (CA) of your choice.

- Verify that you are connected to a vCenter system.
- Verify that the root certificate of the CA you are going to use is added to the trusted root store of vCenter.

1. Optional: Retrieve the current Machine SSL certificate of the vCenter system.

```
Get-VIMachineCertificate -VCenterOnly
```

2. Generate a CSR.

```
$csrParams = @{
    Country="US"
    Email="jdoe@vmware.com"
    Locality="San Francisco"
    Organization="My Company"
```

```
    OrganizationUnit="PowerCLI"
    StateOrProvince="California"
}
$csr = New-VIMachineCertificateSigningRequest @csrParams
```

3. Save the CSR to your system.

```
$csr.CertificateRequestPEM | Out-File "C:\Users\jdoe\Downloads\vc.csr.pem" -Force
```

4. Send the CSR to the CA of your choice.
5. Save the issued custom certificate to your system.
6. Set the new custom certificate to the vCenter system.

```
$vcCert = Get-Content "C:\Users\jdoe\Downloads\vc.cert.jdoe.pem" -RawSet-VIMachineCertificate
 -PemCertificate $vcCert
```

Note:

Starting with vSphere 8.0 Update 2, restart of vCenter services after the certificate change is no longer necessary. The certificate replacement is completed seamlessly and all your sessions remain active.

For older vSphere versions, the change of the Machine SSL certificate triggers a restart of vCenter. Wait for the system to reboot and log in when available.

Change the Machine SSL Certificate of an ESX Host

You can use VCF PowerCLI to change the Machine SSL certificates of one or more ESX hosts in your vSphere environment.

- Verify that the root certificate of the CA you are going to use is added to the trusted root store of vCenter and to the connected ESX hosts.

1. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServer vc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

2. In the vCenter system, retrieve the setting for the ESX host certificate management mode.

```
$certModeSetting = Get-AdvancedSetting "vpxd.certmgmt.mode" -Entity $vCenterConnection
$certModeSetting.Value
```

3. Change the setting to *custom*.

```
Set-AdvancedSetting $certModeSetting -Value "custom"
```

4. For the new certificate management mode to take effect, reboot your vCenter system.
5. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServer vc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

6. Set the ESX host you want to manage to *Maintenance* mode.

```
$vmhost = Get-VMHost 'MyESXHost' `
Set-VMHost -VMHost $vmhost -State Maintenance
```

7. Generate a certificate signing request (CSR) for the ESX host.

```
$esxRequest = New-VIMachineCertificateSigningRequest `
    -VMHost $vmhost `
    -Country "US" `
    -Locality "San Francisco" `
```

```
    -Organization "My Company" `
    -OrganizationUnit "PowerCLI" `
    -StateOrProvince "California" `
    -CommonName <ESX host's FQDN> or <ESX host's IP address>
```

Note:

For `CommonName`, you must use either the ESX host's FQDN or IP address. The common name must match the identifier you use to add the host to the vCenter system.

8. Save the CSR to your system.

```
$esxRequest.CertificateRequestPEM | Out-File "C:\Users\jdoe\Downloads\esx.csr.pem" -Force
```

9. Send the CSR to the CA of your choice.
10. Save the issued custom certificate to your machine.
11. Create a variable with your issued custom certificate.

```
$esxCertificatePem = Get-Content "C:\Users\jdoe\downloads\myesxcert.pem" -Raw
```

12. Remove the ESX host from the vCenter system.

```
Remove-VMHost $vmhost
```

13. Disconnect from the vCenter system.

```
Disconnect-VIServer $vCenterConnection
```

14. Connect directly to the ESX host.

```
$esxConnection = Connect-VIServer $vmhost.Name `
    -User 'My User' `
    -Password 'My Password' `
    -Force
```

15. Set the custom Machine SSL certificate to the ESX host.

```
$targetEsxHost = Get-VMHost $vmhost.Name
Set-VIMachineCertificate -PemCertificate $esxCertificatePem -VMHost $targetEsxHost | Out-Null
```

16. To apply the change, restart the ESX host.

```
Restart-VMHost $targetEsxHost
```

17. Disconnect from the ESX host.

```
Disconnect-VIServer $esxConnection
```

18. Connect to the vCenter system.

```
$vCenterConnection = Connect-VIServervc1.example.com `
    -User 'My User' `
    -Password 'My Password'
```

19. Add the ESX host to the vCenter system.

```
$vmhost = Add-VMHost -Name<ESX host's FQDN> or <ESX host's IP address> `
    -Location (Get-Datacenter "My Datacenter")`
    -User "My User" `
    -Password "My Password"
```

20. Set the ESX host to the *Connected* mode.

```
$vmhost = Set-VMHost -VMHost $vmhost -StateConnected
```

21. Optional: Verify that the Machine SSL certificate of the ESX host is changed.

```
Get-VIMachineCertificate -VMHost $vmhost
```

Using Get-View

Use `Get-View` to access vSphere Management API view objects with VCF PowerCLI.

The `Get-View` cmdlet operates on the low level and binds directly to the vSphere Management API objects. You can use this cmdlet for high-performance automation as it overrides the VCF PowerCLI high-level architecture.

You can use `Get-View` in combination with `Get-VIObjectByVIView` to retrieve the VCF PowerCLI variant of a vSphere Management API view object.

Filter vSphere Objects with Get-View

You can use the `Get-View` cmdlet to filter vSphere objects before performing various actions on them.

Verify that you are connected to a vCenter system.

The filter parameter is a `HashTable` object containing one or more pairs of filter criteria. Each of the criteria consists of a property path and a value that represents a regular expression pattern used to match the property.

1. Create a filter by the power state and the guest operating system name of the virtual machines.
   ```
   $filter = @{"Runtime.PowerState" ="poweredOn"; "Config.GuestFullName" = "Windows XP"}
   ```
2. Get a list of the virtual machines by using the created filter and call the `ShutdownGuest` method for each virtual machine in the list.

   ```
   Get-View -ViewType "VirtualMachine" -Filter $filter | foreach{$_.ShutdownGuest()}
   ```

The filter gets a list of the powered-on virtual machines whose guest OS names contain the string `Windows XP`. The `Get-View` cmdlet then initiates shutdown for each guest operating system in the list.

Populate a View Object with Get-View

To save time and efforts, you can use `Get-View` to retrieve VCF PowerCLI views from previously retrieved view objects.

Verify that you are connected to a vCenter system.

1. Get a view of the *`VM2`* virtual machine by name.
   ```
   $vm2 = Get-View -ViewType VirtualMachine -Filter @{"Name" = "VM2"}
   ```
2. Populate the *`$vmhostView`* object.
   ```
   $vmhostView = Get-View -Id $vm2.Runtime.Host
   ```
3. Retrieve the runtime information for the *`$vmhostView`* object.
   ```
   $vmhostView.Summary.Runtime
   ```

Update the State of a Server-Side Object

You can use the `Get-View` cmdlet to update server-side objects.

Verify that you are connected to a vCenter system.

1. Get the *`VM2`* virtual machine by name.
   ```
   $vm2 = Get-View -ViewType VirtualMachine -Filter @{"Name" = "VM2"}
   $vmhostView = Get-View -Id $vm2.Runtime.Host
   ```
2. View the current power state.
   ```
   $vm2.Runtime.PowerState
   ```

3. Power off the virtual machine.
```
If ($vm2.Runtime.PowerState -ne “PoweredOn”) {
  $vm.PowerOnVM($vm2.Runtime.Host)
} else {
$vm2.PowerOffVM()
}
```
4. View the value of the *$vm2* power state.
```
$vm2.Runtime.PowerState
```
The power state is not updated yet because the virtual machine property values are not updated automatically.
5. Update the view object.
```
$vm2.UpdateViewData()
```
6. Obtain the actual power state of the virtual machine.
```
$vm2.Runtime.PowerState
```

Reboot a Host with Get-View

You can reboot a host by using its corresponding view object.

Verify that you are connected to a vCenter system.

1. Use the `Get-VMHost` cmdlet to get a host by its name, and pass the result to the `Get-View` cmdlet to get the corresponding view object.
```
$vmhostView = Get-VMHost -Name Host | Get-View
```
2. Call the reboot method of the host view object to reboot the host.
```
$vmhostView.RebootHost()
```

Modify the CPU Levels of a Virtual Machine with Get–View and Get–VIObjectByVIView

You can use `Get-View` in combination with `Get-VIObjectByVIView` to invoke a vSphere Management API view object and retrieve its VCF PowerCLI variant.

Verify that you are connected to a vCenter system.

You can modify the CPU levels of a virtual machine by using a combination of the `Get-View` and `Get-VIObjectByVIView` cmdlets.

1. Get the *VM2* virtual machine, shut it down, and pass it to the `Get-View` cmdlet to view the virtual machine view object.
```
$vmView = Get-VM VM2 | Stop-VM | Get-View
```
2. Create a *VirtualMachineConfigSpec* object to modify the virtual machine CPU levels and call the `ReconfigVM` method of the virtual machine view managed object.
```
$spec = New-Object VMware.Vim.VirtualMachineConfigSpec;
$spec.CPUAllocation = New-Object VMware.Vim.ResourceAllocationInfo;
$spec.CpuAllocation.Shares = New-Object VMware.Vim.SharesInfo;
$spec.CpuAllocation.Shares.Level = "normal";
$spec.CpuAllocation.Limit = -1;
$vmView .ReconfigVM_Task($spec)
```
3. Get the virtual machine object by using the `Get-VIObjectByVIView` cmdlet and start the virtual machine.
```
$vm =  Get-VIObjectByVIView $vmView  | Start-VM
```

Creating Custom Properties

You can use VCF PowerCLI to create custom properties for different PowerCLI object types. Then, you can use the custom properties to retrieve objects based on specified criteria.

Create a Custom Property Based on an Extension Data Property

You can create custom properties to add more information to vSphere objects. Custom properties based on extension data properties correspond directly to the property of the corresponding .NET view object.

Verify that you are connected to a vCenter system.

1. Create a new custom property based on the `Guest.ToolsVersion` property.

```
New-VIProperty -ObjectType VirtualMachine -Name ToolsVersion -ValueFromExtensionProperty
'Guest.ToolsVersion'
```

2. View the `ToolsVersion` properties of the available virtual machines.

```
Get-VM | Select Name, ToolsVersion
```

You have created a custom property named `ToolsVersion` for `VirtualMachine` objects.

Create a Script-Based Custom Property for a vSphere Object

You can create a custom property by writing a script and providing a name for the property. The script evaluates when the custom property is called for the first time.

Verify that you are connected to a vCenter system.

1. Create a new custom property named *`NameOfHost`* that stores the name of the host on which a virtual machine resides.

```
New-VIProperty -Name NameOfHost -ObjectType VirtualMachine -Value { return
$args[0].VMHost.Name }
```

2. View the *`NameOfHost`* properties of the available virtual machines.

```
Get-VM | select Name, NameOfHost | Format-Table -AutoSize
```

You created a custom script property named *`NameOfHost`* for `VirtualMachine` objects.

Using the VCF PowerCLI Inventory Provider

Use the VCF PowerCLI Inventory Provider to interact with the VMware vSphere inventory through a file system interface, so that it is easier to navigate, manipulate, and automate tasks.

The VCF PowerCLI Inventory Provider allows navigation and file-style management of the VMware vSphere inventory. By creating a PowerShell drive based on a managed object (such as a data center), you can obtain a view of its contents and the relationships between the items. In addition, you can move, rename, or delete objects by running commands from the PowerShell console.

When you connect to a server with `Connect-VIServer`, the cmdlet builds two default inventory drives: `vi` and `vis`.

To access the inventory of the last connected server, run: `cd vi:\`.

To access the inventory of all vSphere servers connected within the current VCF PowerCLI session, run `cd vis:\`.

Use the `dir` and `cd` commands to navigate the inventories.

Browse the Default Inventory Drive

You can browse the default inventory drive and view its contents.

Verify that you are connected to a vCenter system.

Note:
For more information about the Inventory Provider and the default inventory drive, see Using the VCF PowerCLI Inventory Provider.

1. Navigate to the `vi` inventory drive.
   ```
   cd vi:
   ```
2. View the drive content.
   ```
   dir
   ```

   `dir` is an alias of the `Get-ChildItem` cmdlet.

Create a New Custom Inventory Drive

In addition to the default drive, you can create new custom inventory drives by using the `New-PSDrive` cmdlet.

Verify that you are connected to a vCenter system.

Note:
An alternative to creating an inventory drive is to map an existing inventory path. For example, run: `New-PSDrive -Name myVi -PSProvider VimInventory -Root "vi:\Folder01\Datacenter01"`.

1. Get the root folder of the server.
   ```
   $root = Get-Folder -NoRecursion
   ```
2. Create a PowerShell drive named `myVi` in the server root folder.
   ```
   New-PSDrive -Location $root -Name myVi -PSProvider VimInventory -Root '\'
   ```

   Note:
   You can use the `New-InventoryDrive` cmdlet, which is an alias of `New-PSDrive`. This cmdlet creates a new inventory drive using the `Name` and `Datastore` parameters. For example: `Get-Folder -NoRecursion | New-VIInventoryDrive -Name myVi`.

Manage Inventory Objects Through Inventory Drives

You can use the VCF PowerCLI Inventory Provider to browse, modify, and remove inventory objects from inventory drives.

Verify that you are connected to a vCenter system.

1. Navigate to a host in your server inventory by running the `cd` command with the full path to the host.
   ```
   cd Folder01\DataCenter01\host\Web\Host01
   ```
2. View the content of the host using the `ls` command.
   ```
   ls
   ```

   `ls` is the UNIX style alias of the `Get-ChildItem` cmdlet.
   This command returns the virtual machines and the root resource pool of the host.
3. View only the virtual machines on the host.
   ```
   Get-VM
   ```

   When called within the inventory drive, `Get-VM` gets a list only of the virtual machines on the current drive location.

4. Delete a virtual machine named *VM1*.

```
del VM1
```

5. Rename a virtual machine, for example, from *VM1New* to *VM1*.

```
ren VM1New VM1
```

6. Start all virtual machines with names that start with *VM*.

```
dir VM* | Start-VM
```

Using the VCF PowerCLI Datastore Provider

Use the VCF PowerCLI Datastore Provider to access the contents of one or more datastores through a file system interface, so that it is easier to navigate, manipulate, and automate tasks.

The items in a datastore are files that contain configuration, virtual disk, and other data associated with a virtual machine.

When you connect to a server with `Connect-VIServer`, the cmdlet builds two default datastore drives: `vmstore` and `vmstores`.

To access the datastores of the last connected vCenter or ESX, run: `cd vmstore:\`.

To access the datastores available on all vCenter and ESX systems you are currently connected to, run: `cd vmstores:\`.

Use the `dir` and `cd` commands to navigate the inventories.

Browse the Default Datastore Drives

You can use the VCF PowerCLI Datastore Provider to browse the default datastore drives: `vmstore` and `vmstores`.

Verify that you are connected to a vCenter system.

> **Note:**
> For more information about default datastore drives, see Using the VCF PowerCLI Datastore Provider.

1. Navigate to the `vmstore` drive.

```
cd vmstore:
```

2. View the drive content.

```
dir
```

Create a New Custom Datastore Drive

You can use the VCF PowerCLI Datastore Provider to create custom datastore drives.

Verify that you are connected to a vCenter system.

1. Get a datastore by its name and assign it to the *$datastore* variable.

```
$datastore = Get-Datastore Storage1
```

2. Create a new PowerShell drive `ds:` in *$datastore*.

```
New-PSDrive -Location $datastore -Name ds -PSProvider VimDatastore -Root '\'
```

Note:
You can use the `New-PSDrive` cmdlet, which is an alias of `New-DatastoreDrive`. It creates a new datastore drive using the `Name` and `Datastore` parameters. For example: `Get-Datastore Storage1 | New-DatastoreDrive -Name ds`.

Manage Datastores Through Datastore Drives

You can use the VCF PowerCLI Datastore Provider to browse datastores from datastore drives.

Verify that you are connected to a vCenter system.

1. Navigate to a folder on the `ds:` drive.

   ```
   cd VirtualMachines\XPVirtualMachine
   ```

2. View the files of the folder by running the `ls` command.

   ```
   ls
   ```

   `ls` is the UNIX style alias of the `Get-ChildItem` cmdlet.

3. Rename a file by running the `Rename-Item` cmdlet or its alias `ren`.

   For example, to change the name of the `vmware-3.log` file to `vmware-3old.log`, run:

   ```
   ren vmware-3.log vmware-3old.log
   ```

   All file operations apply only on files in the current folder.

4. Delete a file by running the `Remove-Item` cmdlet or its alias `del`.

   For example, to remove the `vmware-3old.log` file from the `XPVirtualMachine` folder, run:

   ```
   del ds:\VirtualMachines\XPVirtualMachine\vmware-2.log
   ```

5. Copy a file by running the `Copy-Item` cmdlet or its alias `copy`.

   ```
   copy ds:\VirtualMachines\XPVirtualMachine\vmware-3old.log ds:\VirtualMachines\vmware-3.log
   ```

6. Copy a file to another datastore by running the `Copy-Item` cmdlet or its alias `copy`.

   ```
   copy ds:\Datacenter01\Datastore01\XPVirtualMachine\vmware-1.log ds:
   \Datacenter01\Datastore02\XPVirtualMachine02\vmware.log
   ```

7. Create a new folder by running the `New-Item` cmdlet or its alias `mkdir`.

   ```
   mkdir -Path ds:\VirtualMachines -Name Folder01 -Type Folder
   ```

8. Download a file from the datastore drive to the local machine by running the `Copy-DatastoreItem` cmdlet.

   ```
   Copy-DatastoreItem ds:\VirtualMachines\XPVirtualMachine\vmware-3.log C:\Temp\vmware-3.log
   ```

9. Upload a file from the local machine by running the `Copy-DatastoreItem` cmdlet.

   ```
   Copy-DatastoreItem C:\Temp\vmware-3.log  ds:\VirtualMachines\XPVirtualMachine\vmware-3new.log
   ```

Managing vSphere Lifecycle Manager Images and Baselines with VMware Cloud Foundation PowerCLI

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Lifecycle Manager administration.

You can use VCF PowerCLI to manage the lifecycle of all hosts in a cluster collectively or of standalone hosts with vSphere Lifecycle Manager images.

You can use cluster-level and host-level administration starting from the following vSphere and VCF PowerCLI product versions:

Table 1933: vSphere Lifecycle Manager Availability

	Available from vSphere version	Available from PowerCLI version
Cluster-level administration	7.0	12.1.0
Host-level administration	8.0	13.1.0

Understanding vSphere Lifecycle Manager

With VMware vSphere Lifecycle Manager, you can manage ESX clusters and hosts by using images.

> **Important:**
>
> In vSphere Foundation 9.1, you cannot import into your vCenter instance a custom depot ZIP that you exported with `New-OfflineBundle` cmdlet of VCF PowerCLI 9.0 and earlier.

Starting with vSphere 7.0 and VMware PowerCLI 12.1.0, you can use high-level cmdlets for operations on the cluster level as a part of the `VMware.VimAutomation.Core` module. Starting with vSphere 7.0 and VMware PowerCLI 12.4.0, you can also use low-level cmdlets for cluster-level operations.

Starting with vSphere 8.0 and VMware PowerCLI 13.1.0, you can use high-level and low-level cmdlets for vSphere Lifecycle Manager host-level operations.

The low-level cmdlets map directly to the syntax of the `ESX REST APIs` and empower you to use the full API functionality within your PowerShell console.

You can construct low-level SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the `ESX REST API` data structures or operations.

The `ESX REST API` Reference on VMware Developer features ready-to-use vSphere Lifecycle Manager SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.

Lifecycle Management in vSphere

Lifecycle management refers to the process of installing software, maintaining it through updates and upgrades, and decommissioning it.

In the context of maintaining a vSphere environment, your clusters and hosts in particular, lifecycle management refers to tasks such as installing ESX and firmware on new hosts, and updating or upgrading the ESX version and firmware when required.

What Is vSphere Lifecycle Manager

vSphere Lifecycle Manager is a service that operates within vCenter, by using the built-in PostgreSQL database. No additional setup is required to use vSphere Lifecycle Manager. When you deploy the vCenter appliance, the vSphere Lifecycle Manager user interface is automatically activated in the HTML5-based vSphere Client. You can also access vSphere Lifecycle Manager functions through VCF PowerCLI.

vSphere Lifecycle Manager encompasses the functionality of Update Manager from previous vSphere versions and enhances it by introducing new features and options for managing the ESX lifecycle at a cluster and host level.

Up to vSphere 7.0, the Update Manager allowed the use of baselines and baseline groups for patching and upgrading hosts. With vSphere 7.0, vSphere Lifecycle Manager introduced the alternative approach of using vSphere Lifecycle Manager images to manage the life cycle of hosts and clusters in your environment. It also makes possible the upgrade of virtual machine hardware and VMware Tools versions.

vSphere Lifecycle Manager is compatible with an Internet-connected environment, either directly or through a proxy server. It can also operate in a secure network without Internet access. In such cases, you can use the Update Manager Download Service (UMDS) to download updates to the vSphere Lifecycle Manager depot, or import updates manually.

vSphere Lifecycle Manager and VCF PowerCLI

Starting with vSphere 7.0 and VCF PowerCLI 12.1.0, you can create and manage vSphere Lifecycle Manager clusters. For more information, see Creating and Managing vSphere Lifecycle Manager Clusters with VCF PowerCLI.

Starting with vSphere 8.0 and VCF PowerCLI 13.1.0, you can manage standalone ESX hosts with vSphere Lifecycle Manager images. For more information, see Managing vSphere Lifecycle Manager Standalone Hosts with VCF PowerCLI.

Creating and Managing vSphere Lifecycle Manager Clusters with VCF PowerCLI

Starting with vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0, you can create and manage vSphere Lifecycle Manager clusters with VCF PowerCLI.

Cluster-level Functionality Starting from vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0

Starting from vSphere 7.0 and VMware Cloud Foundation PowerCLI 12.1.0, some of the cluster-level operations that you can perform are:

- Create a vSphere Lifecycle Manager cluster with a desired state.
- View desired state elements:
  - ESX version
  - Vendor add-ons
  - Firmware/ drivers
  - Components
- View/ check recommended images.
- Add hosts to a cluster.
- Retrieve a cluster with a vLCM desired state.
- Update a cluster's ESX base image.
- Add, remove, replace, upgrade, or downgrade a vendor add-on on a cluster.
- Update a cluster's vSphere Lifecycle Manager component list.
- Add, remove, replace, upgrade, or downgrade a firmware add-on to a cluster.
- Modify the desired state of a vSphere Lifecycle Manager cluster.
- Convert a Update Manager cluster into a vSphere Lifecycle Manager cluster.
- Check compliance of the ESX hosts on a cluster.
- Apply the desired state on hosts in a cluster, whose current state is different from the desired specification (remediate cluster).
- Export a desired state from a cluster.
- Import a desired state to a cluster.
- View hardware compatibility.

Cluster-level Functionality Starting from vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0

Starting from vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0, some of the new cluster-level operations that you can perform are:

- Update hardware compatibility list.
- View parallel remediation settings.
- Activate/ deactivate parallel remediation.
- Create or modify a cluster by importing an ESX base image.
- Retrieve pre-check results.
- Stage a cluster.

Sample Scripts for vSphere Lifecycle Manager Cluster-Level Operations
The following scripts demonstrate how you can use vSphere Lifecycle Manager on the cluster level with VCF PowerCLI.

Retrieve vSphere Lifecycle Manager (vLCM) Image Profiles

You can retrieve vLCM images from a vCenter system based on different filters by using the `Get-LcmImage` cmdlet. You can pass IDs or a combination of name and version filters to the `Get-LcmImage` parameters.

List all types of vLCM images

```
Get-LcmImage
```

List only `BaseImage` (ESX) vLCM images

```
Get-LCMImage -Type 'BaseImage'
```

List only `VendorAddOn` vLCM images

```
Get-LCMImage -Type 'VendorAddOn'
```

List only `Component` vLCM images

```
Get-LCMImage -Type 'Component'
```

List only `Package` (FirmwareAddOn) vLCM images

```
Get-LCMImage -Type 'Package'
```

Create a New Cluster with a vLCM Desired State

You can create a new vSphere Lifecycle Manager cluster in the following ways:

- By specifying parameters from the VMware depots:

```
$clusterName= Read-Host -Prompt 'Provide the cluster Name'
$vLCMBaseImage = Get-LCMImage -Version '7.0 GA - 15843807' -Type BaseImage
$vendorAddon = Get-LcmImage -Name 'NEC-addon-GEN' -Version '7.0.2-02'
New-Cluster -Location Datacenter -Name $clusterName -BaseImage  $vLCMBaseImage -VendorAddOn $vendorAddon -
HAEnabled -DrsEnabled
```

- By importing a vLCM desired state spec.

```
New-Cluster -Name 'MyCluster' -Location <MyDatacenter> | Import-LcmClusterDesiredState -LocalSpecLocation
 F:\Image\desired-state-spec.json
```

Retrieve a Cluster's Desired State

```
Get-Cluster -Name $clusterName | Select-Object -Property Name, BaseImage, @{n='BaseImageVersion';
 e={$_.BaseImage.Version}}, Components, VendorAddon, FirmwareAddons
```

Update a Cluster's vLCM Base Image

You cannot remove or downgrade an ESX base image - you can only upgrade it. The following example demonstrates an upgrade of a cluster's desired state to ESX 7.0 U2.

```
$vLCMBaseImageu2 = Get-LcmImage -Version '7.0 U2a - 17867351' -Type BaseImage
Get-Cluster -Name $clusterName | Set-Cluster -BaseImage $vLCMBaseImageu2
```

Remove a vLCM Vendor Add-on from a Cluster

```
Get-Cluster -Name $clusterName | Set-Cluster -VendorAddOn $null
```

> **Note:**
> You can add, replace, remove, upgrade, or downgrade a vendor add-on.

Update a Cluster's vLCM Component List

Replace a `Component` list with a new one.

```
$components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>', 'Component-<Component2
 Name>/<Component2 Version>'
Get-Cluster -Name $clusterName | Set-Cluster -Component $components
```

Note:
To remove all components, use `Set-Cluster -Component $null`

Add a vLCM Firmware Add-on

```
$firmwareAddon = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-Cluster -Name $clusterName | Set-Cluster -FirmwareAddOn $firmwareAddon
```

Note:
You can add, replace, remove, upgrade, or downgrade a firmware add-on.

Modify a Cluster's vLCM Desired State

```
$vLCMBaseImage = Get-LCMImage -Version '7.0 U3f - 20036589'
$vendorAddOn = get-lcmimage -type vendoraddon -Version '703.0.0.10.9.0-11'
Set-Cluster 'vlcm-cluster-pcli' -BaseImage $vLCMBaseImage -VendorAddOn $vendorAddOn
```

Convert a Update Manager Cluster to a vSphere Lifecycle Manager Cluster

Convert a Update Manager cluster to a vSphere Lifecycle Manager cluster.

```
$vLCMBaseImage = Get-LCMImage -Version '7.0 U3f - 20036589'
$vendorAddOn = get-lcmimage -type vendoraddon -Version '703.0.0.10.9.0-11'
Set-Cluster 'vum-cluster' -BaseImage $vLCMBaseImage -VendorAddOn $vendorAddOn
```

Check Cluster Compliance

Check the compliance of the hosts on the cluster against the cluster's desired state.

```
Get-Cluster -Name $clusterName | Test-LcmClusterCompliance
```

Remediate a Cluster

Apply the desired state to the hosts in the cluster, whose current state is different from the desired software specification.

```
Get-Cluster -Name $clusterName | Set-Cluster -Remediate -AcceptEULA
```

Export a vLCM Desired State

```
Get-Cluster -Name $clusterName | Export-LcmClusterDesiredState -Destination 'F:\Image' -ExportOfflineBundle -
ExportIsoImage
```

Import a vLCM Desired State

```
Import-LcmClusterDesiredState -Cluster 'ExistingCluster' -LocalSpecLocation F:\Image\desired-state-spec.json -
Verbose
```

View / Check Recommended Images

Generate a desired state recommendation of a vSphere Lifecycle Manager cluster.

- Based on the latest ESX base image version.
  ```
  Get-LcmClusterDesiredStateRecommendation 'lcm-cluster' -Latest
  ```
- Based on the latest patch or update of the currently installed ESX base image version.
  ```
  Get-LcmClusterDesiredStateRecommendation 'lcm-cluster' -Current
  ```

View Hardware Compatibility

```
Get-LcmHardwareCompatibility -Cluster 'lcm-cluster'
```

Update Hardware Compatibility List

```
Invoke-DownloadHclCompatibilityDataAsync
```

View Parallel Remediation Settings

```
Invoke-GetDefaultsClustersPoliciesApply
Invoke-GetClusterPoliciesApply -Cluster $clusterId
```

Activate or Deactivate Parallel Remediation

Activate or deactivate parallel remediation with the maximum number of concurrent remediations.

```
$ParallelRemediationSetting = Initialize-SettingsDefaultsClustersPoliciesParallelRemediationAction -Enabled
 $true -MaxHosts $maxHostCount
$PolicySpec = Initialize-SettingsDefaultsClustersPoliciesApplyConfiguredPolicySpec -ParallelRemediationAction
 $ParallelRemediationSetting
Invoke-SetDefaultsClustersPoliciesApply -Cluster $clusterID -SettingsClustersPoliciesApplyConfiguredPolicySpec
 $PolicySpec
```

Create a Cluster by Importing an Image from a Standalone Host

Extract a software specification from an ESX host and import it into the software depot.

```
$SettingsDepotsOfflineHostCredentials = Initialize-SettingsDepotsOfflineHostCredentials -HostName "MyHostName"
 -UserName "MyUserName" -Password "MyPassword" -Port 0 -SslThumbPrint "MySslThumbPrint"
$SettingsDepotsOfflineConnectionSpec = Initialize-SettingsDepotsOfflineConnectionSpec -AuthType
 "USERNAME_PASSWORD" -HostCredential $SettingsDepotsOfflineHostCredentials
Invoke-CreateFromHostDepotsOfflineAsync -SettingsDepotsOfflineConnectionSpec
 $SettingsDepotsOfflineConnectionSpec
```

Then use `New-Cluster` or `Set-Cluster` to create or modify a cluster's desired state from the returned software spec.

Retrieve Pre-check Results

Return the last remediation pre-check results.

```
$cluster = Get-Cluster -Name <cluster_name>
Invoke-GetClusterSoftwareReportsLastCheckResult -Cluster $cluster.ExtensionData.MoRef.Value
```

Stage a Cluster

Stage a desired software specification on a vSphere Lifecycle Manager cluster.

```
$SettingsClustersSoftwareStageSpec = Initialize-SettingsClustersSoftwareStageSpec -Commit <myCommit> -Hosts
 <MyHosts>
Invoke-StageClusterSoftwareAsync -Cluster <MyCluster> -SettingsClustersSoftwareStageSpec $spec
```

Managing vSphere Lifecycle Manager Standalone Hosts with VCF PowerCLI

Starting with vSphere 8.0 and VMware Cloud Foundation PowerCLI 13.1.0, you can manage vSphere Lifecycle Manager standalone hosts with VCF PowerCLI.

Host-level Functionality in VCF PowerCLI

In VCF PowerCLI 13.1.0 and later, you can use both high-level and low-level cmdlets that bind directly to the ESX REST APIs to manage vSphere Lifecycle Manager hosts.

Starting with VMware Cloud Foundation PowerCLI 13.1.0, some of the host-level operations you can perform are:

- Activate vSphere Lifecycle Manager on a standalone host.
- Retrieve vSphere Lifecycle Manager image profiles:
  - ESX version
  - Vendor add-ons
  - Firmware/ drivers
  - Components
- View/ check recommended images.
- Retrieve a host with a vSphere Lifecycle Manager desired state.
- Modify the desired state of a vSphere Lifecycle Manager host.
- Convert a Update Manager host into a vSphere Lifecycle Manager host.
- Update the ESX base image of a standalone host.
- Add, remove, replace, upgrade, or downgrade a vendor add-on on a standalone host.
- Add remove, replace, upgrade, or downgrade a vSphere Lifecycle Manager firmware add-on on a standalone host.
- Check compliance of a standalone ESX host.
- Pre-check a host.
- Stage a host.
- Apply the desired state on a standalone host, whose current state is different from the desired specification (remediate host).

Sample Scripts for vSphere Lifecycle Manager Host-Level Operations
The following scripts demonstrate how you can use vSphere Lifecycle Manager on the host level with VCF PowerCLI.

Activate vSphere Lifecycle Manager on a Standalone Host

```
$hostName = <host_ip_address_or_fqdn>
$vmhost = Get-Vmhost $hostName
$spec=Initialize-SettingsHostsEnablementSoftwareEnableSpec -SkipSoftwareCheck $false
Invoke-SetHostEnablementSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -
SettingsHostsEnablementSoftwareEnableSpec $spec
```

Retrieve vSphere Lifecycle Manager (vLCM) Image Profiles

You can retrieve vLCM images from a vCenter system based on different filters by using the `Get-LcmImage` cmdlet. You can pass IDs or a combination of name and version filters to the `Get-LcmImage` parameters.

List all types of vLCM images

```
Get-LcmImage
```

List only `BaseImage` (ESX) vLCM images

```
Get-LCMImage -Type 'BaseImage'
```

List Only `VendorAddOn` vLCM images

```
Get-LCMImage -Type 'VendorAddOn'
```

List only `Component` vLCM images

```
Get-LCMImage -Type 'Component'
```

List only `Package` (Firmware) vLCM images

```
Get-LCMImage -Type 'Package'
```

View / Check Recommended Images

```
$vmhost = Get-VMHost -Name $hostName
Invoke-GetHostSoftwareRecommendations -Host $vmhost.ExtensionData.MoRef.Value
```

Retrieve a Host's Desired State

```
Get-VMHost -Name $hostName | Select-Object -Property Name, BaseImage, @{n='BaseImageVersion';
 e={$_.BaseImage.Version}}, Components, VendorAddon, FirmwareAddons
```

Modify a Host's vLCM Desired State

```
$vLCMBaseImageu2 = Get-LCMImage -Version '7.0 GA - 15843807' -Type BaseImage
$vendorAddon = Get-LcmImage -Name 'NEC-addon-GEN' -Version '7.0.2-02'
$components = $components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>'
$firmwareAddon = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-VMHost -Name $hostName | Set-VMHost -BaseImage $vLCMBaseImageu2 -VendorAddOn $vendorAddon -Component
 $components -FirmwareAddOn $firmwareAddon
```

Update a Host's vLCM Base Image

You cannot remove or downgrade an ESX base image - you can only upgrade it. The following example demonstrates an upgrade of a host's desired state to ESX 7.0 U2.

```
$vLCMBaseImageu2= Get-LcmImage -Version '7.0 U2a - 17867351' -Type BaseImage
Get-VMHost -Name $hostName | Set-VMHost -BaseImage $vLCMBaseImageu2
```

Remove a vLCM Vendor Add-on from a Host

```
Get-VMHost -Name $hostName | Set-VMHost -VendorAddOn $null
```

> **Note:**
> You can add, replace, remove, upgrade, or downgrade a vendor add-on.

Update a Host's vLCM Component List

Replace a `Component` list with a new one.

```
$components = Get-LcmImage -Id 'Component-<Component1 Name>/<Component1 Version>', 'Component-<Component2
 Name>/<Component2 Version>'
Get-VMHost -Name $hostName | Set-VMHost -Component $components
```

> **Note:**
> To remove all components, use `Set-Cluster -Component $null`

Add a vLCM Firmware Add-on

```
$firmwareAddon = Get-LcmImage -Id '<FirmwareAddOn_Id>'
Get-VMHost -Name $hostName | Set-VMHost -FirmwareAddOn $firmwareAddon
```

> **Note:**
> You can add, replace, remove, upgrade, or downgrade a firmware add-on.

Check Compliance of a Host

Compare the current image on the standalone host against the desired image that you specified and define the compatibility status of the host.

```
$vmhost = Get-VMHost -Name $hostName
Invoke-ScanHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value
```

Pre-check a Host

Run a remediation pre-check on the host to ensure software and hardware compatibility with the desired software specification.

```
$spec = Initialize-SettingsHostsSoftwareCheckSpec -Commit <MyCommit>
$vmhost = Get-VMHost -Name $hostName
```

```
Invoke-CheckHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareCheckSpec $spec
```

Stage a Host

Stage the desired state on a standalone ESX host.

```
$spec = Initialize-SettingsHostsSoftwareStageSpec -Commit <myCommit>
$vmhost = Get-VMHost -Name $hostName
Invoke-StageHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareStageSpec $spec
```

Remediate a Host

Apply the desired state on a standalone host, whose current state is different from the desired software specification.

```
$spec = Initialize-SettingsHostsSoftwareApplySpec -AcceptEula $true
$vmhost = Get-VMHost -Name $hostName
Invoke-ApplyHostSoftwareAsync -Host $vmhost.ExtensionData.MoRef.Value -SettingsHostsSoftwareApplySpec $spec
```

Managing vSphere Lifecycle Manager Baselines with VMware Cloud Foundation PowerCLI

This section provides sample scripts for VCF PowerCLI with basic and advanced tasks in vSphere Lifecycle Manager.

The vSphere Lifecycle Manager module provides a set of cmdlets for downloading software patches, creating and modifying baselines, and for scanning and remediating virtual machines or hosts.

Create Patch Baselines

You can apply patch baselines to hosts. Depending on the patch criteria you select, patch baselines can be either dynamic or fixed.

Verify that you are connected to a vCenter system.

Patch data in dynamic baselines changes depending on the criteria you specify each time Update Manager downloads new patches. Fixed baselines contain only the patches you have selected, regardless of new patch downloads.

1. Retrieve all host patches released after 1 Jan 2015 for ESX products, and create a fixed baseline named *`Static Baseline`*, containing the retrieved patches.

   ```
   $patches = Get-Patch -After "1 Jan 2015" -Product "ESX"
   $staticBaseline = New-PatchBaseline -Static -Name "Static Baseline" -IncludePatch $patches
   ```

2. Create a critical dynamic baseline named *`Dynamic Baseline`* by using a fetch-all query.

   ```
   $criticalPatchBaseline = New-PatchBaseline -Dynamic -Name "Dynamic Baseline" -SearchPatchSeverity
    Critical
   ```

3. Create an extension baseline that contains all available extensions.

   ```
   $extensions = Get-Patch -BundleType Extension
   New-PatchBaseline -Static -Name "Extension Baseline" -Extension -IncludePatch $extensions
   ```

Attach and Detach Baselines

You can attach baselines to individual objects and to container objects, such as folders, hosts, clusters, and data centers. Attaching a baseline to a container object attaches the baseline to all objects in the container.

Verify that you are connected to a vCenter system.

1. Attach the host patch baselines stored in the provided variables to the host named *`Host`*.

   ```
   Add-EntityBaseline -Baseline $staticBaseline, $criticalPatchBaseline -Entity Host
   ```

2. Detach the two baselines from the host.

   ```
   Remove-EntityBaseline -Baseline $dynamicBaseline, $staticBaseline -Entity Host
   ```

Scan a Virtual Machine

You can scan a virtual machine against the baselines attached to it or inherited by its parent object.

Verify that you are connected to a vCenter system.

1. Initialize scanning on a virtual machine that is named *VM* against baselines containing virtual machine hardware upgrades and VMware Tools upgrades.

   ```
   $task = Test-Compliance -Entity VM -UpdateType VmHardwareUpgrade, VmToolsUpgrade -RunAsync
   ```

   The command initializes a task on the server, returns a snapshot object of the initial state of the task, and saves it in the *$task* variable.

2. View the initial status of the scanning task.

   ```
   $task
   ```

   Note:
   The task object is not updated with the actual state of the task process running on the server. Even after the task is completed, the *$task* variable value is running. To view the actual status of the tasks running on the server, use the `Get-Task` cmdlet.

3. Optional: Run the `Wait-Task` cmdlet to monitor the process progress and wait for the task to complete before running other commands.

   ```
   Wait-Task -Task $task
   ```

Check Virtual Machine Baseline Status

You can check whether a virtual machine has any baselines with unknown compliance status attached to it and start a scan.

Verify that you are connected to a vCenter system.

1. Retrieve the compliance statuses with the value `Unknown` for the baselines attached to the *VM* virtual machine and store them in the *$statuses* variable.

   ```
   $statuses = Get-Compliance -Entity VM -ComplianceStatus Unknown
   ```

2. Check whether the virtual machine has any baselines with unknown compliance status attached to it and start a scan.

   ```
   if ($statuses.Count -gt 0) {
   Test-Compliance -Entity VM -RunAsync"
   }
   ```

Stage Patches

Staging allows you to download patches and extensions from the Update Manager server to the ESX hosts without applying the patches and extensions immediately.

Verify that you are connected to a vCenter system.

Note:
Staging can be performed only for hosts, clusters, and data centers.

1. Retrieve a host and store it in the *$vmHost* variable.

   ```
   $vmHost = Get-VMHost -Name 10.23.112.233
   ```

2. Stage the patches for upgrading the host.

   ```
   Stage-Patch -Entity $vmHost
   ```

Remediate a Virtual Machine

You can retrieve all baselines attached to a virtual machine and remediate the virtual machine.

Verify that you are connected to a vCenter system.

1. Retrieve all baselines attached to the *VM* virtual machine.

```
$baselines = Get-Baseline -Entity VM
```

2. Remediate the virtual machine.

```
Update-Entity -Entity VM -Baseline $baselines
```

Upgrade Virtual Machine Hardware
You can upgrade virtual machine hardware and VMware Tools for all virtual machines in a data center.

Verify that you are connected to a vCenter system.

1. Retrieve all virtual machines in the *Datacenter* data center.

```
$vms = Get-VM -Location Datacenter
```

2. Retrieve all virtual machine upgrade baselines.

```
$ugradeBaselines = Get-Baseline -TargetType VM -BaselineType Upgrade
```

3. Remediate all virtual machines against the virtual machine upgrade baselines.

```
foreach ($vm in $vms) {
Update-Entity -Entity $vm -Baseline $upgradeBaselines
}
```

Remediate a Cluster
You can retrieve all baselines attached to a cluster and remediate the cluster.

Verify that you are connected to a vCenter system.

> **Note:**
> Before remediation, you must temporarily deactivate the Distributed Power Management (DPM), High Availability (HA) admission control, and Fault Tolerance (FT) features of the clusters you want to remediate. After remediation, Update Manager automatically activates the deactivated features.

1. Retrieve all baselines attached to the *Cluster* cluster.

```
$baselines = Get-Baseline -Entity Cluster
```

2. Remediate the cluster.

```
Update-Entity -Entity Cluster -Baseline $baselines -ClusterDisableDistributedPowerManagement
$true -ClusterDisableHighAvailability $true -ClusterDisableFaultTolerance $true
```

Remediate a Host
You can retrieve all baselines attached to a host and remediate the host.

Verify that you are connected to a vCenter system.

> **Note:**
> When remediating a host, you can configure the maintenance mode settings. You can temporarily disconnect any removable media devices that might prevent the host from entering maintenance mode as well.

1. Retrieve all baselines attached to the *Host* host.

```
$baselines = Get-Baseline -Entity Host
```

2. Remediate the host.

```
Update-Entity -Entity Host -Baseline $baselines -HostFailureAction Retry -HostNumberOfRetries
2 -HostDisableMediaDevices $true
```

Download Patches and Scan Objects
You can download patches from a previously defined location.

Verify that you are connected to a vCenter system.

1. Retrieve all entities from the *Datacenter* data center and store the result in a variable.

```
$entities = Get-Inventory -Location Datacenter
```

2. Download all available patches and store the result in a variable.

```
$result = Sync-Patch
```

3. Check whether new patches are downloaded and start scanning the entities in the *Datacenter* data center.

```
if ($result.Count > 0) {
Test-Compliance -Entity $entities
}
```

vSphere Monitoring and Performance

You can monitor your virtual environment with VCF PowerCLI to locate the source of current and potential issues. Some monitoring and performance tools you can use with VCF PowerCLI are the vCenter alarms, and the `esxtop` functionality. You can also use VCF PowerCLI to retrieve valuable statistics about your virtual infrastructure.

Use Esxtop to Get Information on the Virtual CPUs of a Virtual Machine

You can use the `Get-EsxTop` cmdlet to retrieve real-time data for troubleshooting performance problems on your ESX hosts.

Verify that you are connected to a server that runs ESX 4.0, vCenter 5.0 or later.

The `Get-EsxTop` cmdlet exposes the `esxtop` functionality that provides real-time monitoring and performance analysis of an ESX host's resources. You can use `esxtop` to gain insights into the performance and resource utilization of virtual machines and the underlying hardware.

1. Get the group to which the virtual machine belongs and save it as a variable.

```
$group = Get-EsxTop -CounterName SchedGroup | where {$_.VMName -eq $vm.Name}
```

2. Get the IDs of all virtual CPUs of the virtual machine and store them in an array.

```
$gr = Get-EsxTop -TopologyInfo -Topology SchedGroup | %{$_.Entries} | where {$group.GroupID -contains
 $_.GroupId} $group.GroupID
$cpuIds = @()
$gr.CpuClient | %{$cpuIds += $_.CPUClientID}
```

3. Get the CPU statistics for the virtual machine.

```
$cpuStats = Get-EsxTop -CounterName VCPU | where {$cpuIds -contains $_.VCPUID}
```

4. Calculate the used and ready for use percentage by using the `UsedTimeInUsec` and `ReadyTimeInUsec` stats.

```
$result = @()
$cpuStats | %{ `
$row = "" | select VCPUID, Used, Ready; `
$row.VCPUID = $_.VCPUID; `
$row.Used = [math]::Round(([double]$_.UsedTimeInUsec/[double]$_.UpTimeInUsec)*100, 2); `
$row.Ready = [math]::Round(([double]$_.ReadyTimeInUsec/[double]$_.UpTimeInUsec)*100, 2);`
$result += $row
}
```

5. View the used and ready for use percentage for each virtual CPU of the virtual machine.

```
$result | Format-Table –AutoSize
```

Manage Statistics and Statistics Intervals on vCenter

You can use the VCF PowerCLI cmdlets to automate tasks for viewing and managing statistics for vCenter inventory objects.

Verify that you are connected to a vCenter system.

You can modify the properties of a statistics interval and view statistics for a selected cluster.

1. Increase the amount of time for which statistics of the previous day are stored.

```
Set-StatInterval -Interval "past day" -StorageTimeSecs 700000
```

2. View the available memory metric types for the *Cluster1* cluster.

```
$cluster = Get-Cluster Cluster1
$statTypes = Get-StatType -Entity $cluster -Interval "past day" -Name mem.*
```

3. View the cluster statistics collected for the day.

```
Get-Stat -Entity $cluster -Start ([System.DateTime]::Now.AddDays(-1)) -Finish
([System.DateTime]::Now) -Stat $statTypes
```

Alarm Management

You can use a set of preconfigured alarms in vCenter to monitor the operations of vSphere inventory objects. This section provides sample scripts with basic and advanced tasks for vCenter alarm management.

View Actions and Triggers for an Alarm on vCenter

You can see which actions and triggers are configured for an alarm.

Verify that you are connected to a vCenter system.

1. Get all VCF PowerCLI supported alarm actions for the *Host Processor Status* alarm.

```
Get-AlarmDefinition -Name "Host Processor Status" | Get-AlarmAction -ActionType
"ExecuteScript", "SendSNMP", "SendEmail"
```

2. Get all the triggers for the first alarm definition found.

```
Get-AlarmDefinition | select -First 1 | Get-AlarmTrigger
```

Create Alarm Definitions on vCenter

With VCF PowerCLI, you can create vCenter alarm definitions.

Verify that you are connected to a vCenter system.

1. Create an alarm action trigger.

```
$actionTrigger = New-AlarmActionTrigger -StartStatus Green -EndStatus Yellow
```

2. Create an alarm action email.

```
$emailAction = New-AlarmAction -Email -To 'test@vmware.com' -CC @('test1@vmware.com',
 'test2@vmware.com') -Body 'Email text' -Subject 'Email subject' -AlarmActionTrigger $actionTrigger
```

3. Create an event-based alarm trigger.

```
$vmReconfigEvt = Get-EventType | Where-Object {$_.Description -eq "VM reconfigured"}
$eventTrigger = New-AlarmTrigger -EventType $vmReconfigEvt -EntityStatus Yellow -EntityType
 "VirtualMachine"
```

4. Create an alarm definition.

```
$vm = Get-VM | Select-Object -First 1
```

```
New-AlarmDefinition -Name "MyAlarm" -Description "Description" -AlarmTrigger $eventTrigger -AlarmAction
 $emailAction -Entity $vm
```

Modify Alarm Definitions on vCenter

With VCF PowerCLI, you can modify vCenter alarm definitions.

Verify that you are connected to a vCenter system.

1. For all host alarms, modify the interval after the action repeats.

```
Get-AlarmDefinition -Entity (Get-VMHost) | foreach { $_ | Set-AlarmDefinition
-ActionRepeatMinutes ($_.ActionRepeatMinutes + 1)}
```

2. Modify the name and the description of a selected alarm definition, and enable the alarm.

```
Get-AlarmDefinition -Name AlarmDefinition | Set-AlarmDefinition -Name AlarmDefinitionNew
-Description 'Alarm Definition Description' -Enabled:$true
```

Create Alarm Actions and Triggers on vCenter

With VCF PowerCLI, you can create vCenter alarm actions and triggers.

Verify that you are connected to a vCenter system.

1. Create an alarm action `email` for the renamed alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Email -To
'test@vmware.com' -CC @('test1@vmware.com', 'test2@vmware.com') -Body 'Email text' -Subject 'Email
subject'
```

2. Create an `snmp` alarm action.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Snmp
```

3. Create a `script` alarm action.

```
Get-AlarmDefinition -Name AlarmDefinitionNew | New-AlarmAction -Script -ScriptPath 'c:
\test.ps1'
```

4. Create a metric-based alarm trigger.

```
$metric = Get-Metric -MetricGroup CPU -Name Usage | Where-Object {$_.Key -eq 2}
$metricTrigger = New-AlarmTrigger -Metric $metric -Red 9000 -RedIntervalSeconds 3000 -EntityType
 "VirtualMachine" -MetricAlarmOperator Above
```

5. Create a state-based alarm trigger.

```
$stateTrigger = New-AlarmTrigger -StatePath "runtime.powerState" -Value "poweredOff" -EntityStatus Red
 -EntityType "VirtualMachine" -StateAlarmOperator Equal
```

Remove Alarm Definitions and Actions

In some cases, you might want to remove obsolete alarm definitions and actions.

Verify that you are connected to a vCenter system.

1. Remove all the actions for an alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinition | Get-AlarmAction | Remove-AlarmAction
-Confirm:$false
```

2. Remove an alarm definition.

```
Get-AlarmDefinition -Name AlarmDefinition | Remove-AlarmDefinition
```

Managing the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI

You can manage the VMware Cloud Foundation API with VCF PowerCLI by using the vSphere Automation SDK for PowerShell. Starting from VCF PowerCLI version 12.4.0, the VMware Cloud Foundation SDK provides low-level cmdlets for all available VMware Cloud Foundation API services.

Understanding the vSphere Automation SDK for PowerShell

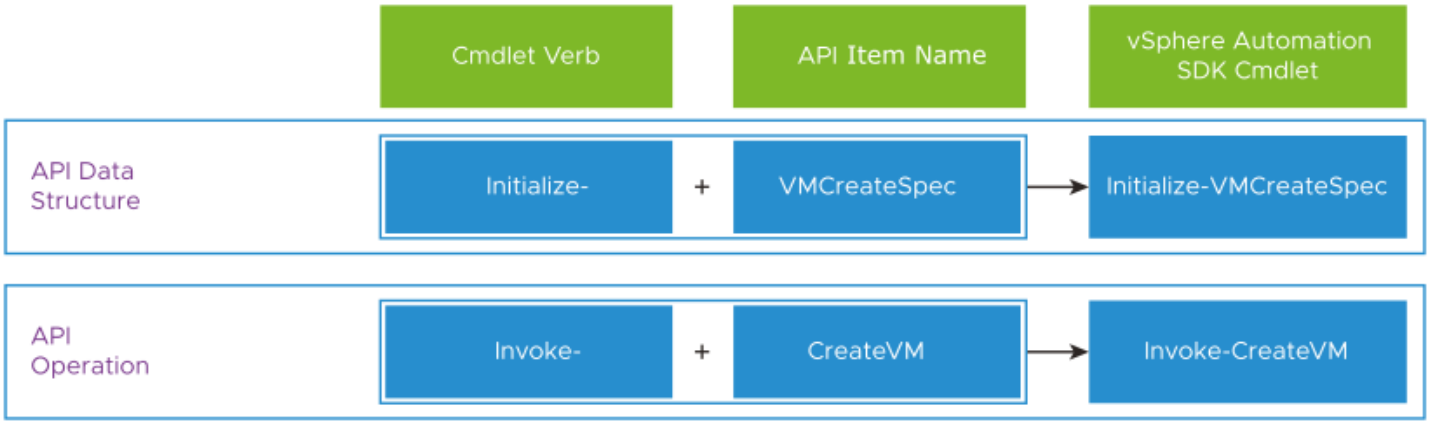
You can manage the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI by using the vSphere Automation SDK for PowerShell.

Starting from VMware Cloud Foundation PowerCLI 12.4.0, you can use the vSphere Automation SDK for PowerShell to communicate with all available vSphere Automation REST APIs.

The modules that start with the `VMware.vSphere.Sdk` prefix contain automatically-generated SDK cmdlets that bind directly to the vSphere Automation REST APIs. The modules function on a low level and communicate directly with the vSphere Automation API server.

You can construct VMware Cloud Foundation SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the vSphere Automation API data structures or operations.

Figure 659: Constructing VMware Cloud Foundation SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the API server.
- Use `Invoke-` cmdlets to call the VMware Cloud Foundation API server and invoke the operations.

Navigating the vSphere Automation SDK for PowerShell

You can navigate the vSphere Automation SDK for PowerShell by using the VMware Developer website, and the `Get-Help` and `Get-Command` cmdlets.

To use the vSphere Automation SDK for PowerShell effectively, you must be familiar with the vSphere Automation API reference documentation .

How to Find a VMware Cloud Foundation SDK Cmdlet

To search for VMware Cloud Foundation SDK cmdlets:

- In your browser, navigate the VMware Cloud Foundation API reference documentation.
  The VMware Cloud Foundation API reference on VMware Developer features ready-to-use VMware Cloud Foundation SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.
- In PowerShell, run `Get-Command -Module VMware.Sdk.vSphere.*` to list all available VMware Cloud Foundation SDK cmdlets.

How to Get Information About a Cmdlet

To get information about a VMware Cloud Foundation SDK cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-CreateVm -full
```

To open the online information in a browser, run `Get-Help` with the cmdlet name and the `Online` parameter. For example:

```
Get-Help
Invoke-CreateVm -Online
```

Connecting to a VMware Cloud Foundation API Server

You can connect to the VMware Cloud Foundation API with VMware Cloud Foundation PowerCLI through the `Connect-VIServer` cmdlet or through a configuration object.

You can access a VMware Cloud Foundation API server in two ways with VCF PowerCLI.

- Through the `Connect-VIServer` cmdlet.
- Through a configuration object.

Connect Through Connect-VIServer

Use the `Connect-VIServer` cmdlet to connect to a vCenter system and access the vSphere Automation API.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see Managing Default Server Connections in VCF PowerCLI.

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see Processing Non-alphanumeric Characters in VCF PowerCLI.

Run `Connect-VIServer` with the server name and valid credentials.

```
Connect-VIServer -Server vc3.example.com -User 'MyUser' -Password 'MyPassword'
```

Connect Through a Configuration Object

You can access the vSphere Automation API by connecting to a vCenter system through a configuration object.

You can create a configuration object and pass information about the vCenter system on which you want to use the vSphere Automation SDK for PowerShell. You can use this connection method if you don't want to use the PowerCLI high-level cmdlets or if you don't have the `VMware.VimAutomation.Core` module installed on your system.

1. Create a server configuration for a vCenter system.

```
$Server = 'vCenter address'
$User = 'username'
$Password = Read-Host -AsSecureString -Prompt "Password:"
```

```
$serverConfig = New-vSphereServerConfiguration -Server $Server -User $User -Password $Password
 -SkipCertificateCheck
```

2. Create a vSphere Automation API session.

```
$apiSession = Invoke-CreateSession -WithHttpInfo
```

3. Update the server configuration with the created API session key.

```
$serverConfig | Set-vSphereServerConfigurationApiKey -SessionResponse $apiSession
```

To learn more about using server configuration objects with the VMware Cloud Foundation SDK, see Managing vCenter Connections Through Configuration Objects.

Managing vCenter Connections Through Configuration Objects
The vSphere Automation SDK for PowerShell features special configuration object cmdlets to manage vCenter connections. ? configuration object is a client-side PowerShell object that holds information about a server that the `Invoke-` cmdlets can interact with.

Table 1934: Configuration Object Cmdlets

Cmdlet	Description
`New-vSphereServerConfiguration`	Creates a vCenter configuration object.
`Get-vSphereServerConfiguration`	Retrieves the existing vCenter configuration objects.
`Set-vSphereServerConfiguration`	Updates a vCenter configuration object.
`Set-vSphereServerConfigurationApiKey`	Updates a vCenter configuration object with an API key for authorization.
`Remove-vSphereServerConfiguration`	Removes a vCenter configuration object.

Working with Configuration Objects

You can create multiple vCenter configuration objects and manage all of them simultaneously with the `Invoke-` cmdlets. For example, `Invoke-DeleteSession` deletes all existing vSphere Automation API sessions.

If you want to apply the `Invoke-` cmdlets to a specific configuration object, you must specify it in the command. For example,

```
Invoke-ListFolder -Server $serverConfig
```

Example Use Cases with vCenter Configuration Objects

- You can create a session and authenticate to a vSphere Automation API server - see Connect Through a Configuration Object.
- You can retrieve the active server configurations.

```
Get-vSphereServerConfiguration
```

- You can remove a specific server configuration.

```
$serverConfig | Remove-vSphereServerConfiguration
```

Create a Local User Account in vCenter

You can create a local user account in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Prepare the data structures for the API operation.

```
$LocalAccountsConfig = Initialize-LocalAccountsConfig -Password '$trongPa$$w0rd' -Roles "superAdmin"
$LocalAccountsCreateRequestBody = Initialize-LocalAccountsCreateRequestBody -Username "MyUsername" -Config
 $LocalAccountsConfig
```

2. Invoke the API operation.

```
Invoke-CreateLocalAccounts -LocalAccountsCreateRequestBody $LocalAccountsCreateRequestBody
```

3. Optional: Retrieve the local account you created.

```
Invoke-GetUsernameLocalAccounts -Username "MyUsername"
```

Update the Local Accounts Global Password Policy in vCenter

You can update the local accounts global password policy in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Retrieve the current local accounts global password policy.

```
Invoke-GetLocalAccountsGlobalPolicy
```

2. Prepare the data structures for the operation.

```
$LocalAccountsPolicyInfo = Initialize-LocalAccountsPolicyInfo -MaxDays 352 -MinDays 1 -WarnDays 7
```

3. Invoke the API operation.

```
Invoke-SetLocalAccountsGlobalPolicy -LocalAccountsPolicyInfo $LocalAccountsPolicyInfo
```

4. Optional: Retrieve the new local accounts global password policy.

```
Invoke-GetLocalAccountsGlobalPolicy
```

Create a Virtual Machine

You can create virtual machines in vCenter by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Retrieve a target datastore ID.

```
$targetDatastore = Invoke-ListDatastore -Names 'local-0'
```

2. Retrieve a target folder ID.

```
$targetFolder = Invoke-ListFolder -Names 'vm'
```

3. Retrieve a target host ID.

```
$targetHost = Invoke-ListHost -Names '10.185.41.87'
```

4. Prepare the virtual machine placement specification.

```
$placementSpec = Initialize-VMPlacementSpec -Folder $targetFolder.folder -Datastore
 $targetDatastore.datastore -VarHost $targetHost.host
```

5. Prepare the virtual machine create specification.

```
$vmCreateSpec = Initialize-VMCreateSpec -Name 'TestVM' -GuestOS WIN_31 -Placement $placementSpec
```

6. Invoke the `CreateVM` API operation.

```
Invoke-CreateVM -VMCreateSpec $vmCreateSpec
```

Create Tag Category, Tag, and Tag Association

You can create a tag category, tag, and tag association by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Create a tag category.

```
$categoryCreateSpec = Initialize-TaggingCategoryCreateSpec -Cardinality 'MULTIPLE' -AssociableTypes
 'VirtualMachine' -Name 'TestCategory' -Description 'TestDescription'
$categoryId = Invoke-CreateCategory -TaggingCategoryCreateSpec $categoryCreateSpec
```

2. Create a tag.

```
$tagCreateSpec = Initialize-TaggingTagCreateSpec -Name 'TestVMTag' -Description 'This is a test tag for
 vms' -CategoryId $categoryId
$tag = Invoke-CreateTag -TaggingTagCreateSpec $tagCreateSpec
```

3. Create a tag association.

```
$vmId = Invoke-ListVM -Names 'MyVM'
$vmDynamicId = Initialize-StdDynamicID -Type 'VirtualMachine' -Id $vmId
$tagAssociationBody = Initialize-TaggingTagAssociationAttachRequestBody -ObjectId $vmDynamicId Invoke-
AttachTagIdTagAssociation -TagId $tag -TaggingTagAssociationAttachRequestBody $tagAssociationBody
```

Create Content Library and Content Library Item

You can create a content library and content library item by using the vSphere Automation SDK for PowerShell.

Verify that you are connected to a VMware Cloud Foundation API server.

1. Specify a datastore for storage backing.

```
$backingDatastore = Invoke-ListDatastore -Names 'local-0'
```

2. Create a content library.

```
$storageBacking = Initialize-LibraryStorageBacking -DatastoreId $backingDatastore.datastore -Type
 'DATASTORE'
$libraryModel = Initialize-LibraryModel -Name 'TestLocalContentLibrary' -StorageBackings $storageBacking
$contentLibraryId = Invoke-CreateContentLocalLibrary -LibraryModel $libraryModel
```

3. Create a content library item.

```
$libraryItemModel = Initialize-LibraryItemModel -Name 'TestContentLibraryItem' -LibraryId
 $contentLibraryId
$libraryItemId = Invoke-CreateContentLibraryItem -LibraryItemModel $libraryItemModel
```

Managing VMware vSAN™ with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware vSAN™ management.

Create a vSAN Datastore

You can create vSAN disk groups on standalone hosts and add the hosts to a vSAN-enabled cluster to form a vSAN datastore. You can then create a virtual machine on the vSAN datastore and assign a storage policy to the virtual machine and its hard disk.

- Verify that you are connected to a vCenter system.
- Verify that you have access to at least three virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD and one HDD.
- Verify that the virtual machine hosts are in maintenance mode.

1. Create a vSAN enabled cluster with manual disk claim mode.

```
$vsanCluster = New-Cluster -Name 'VsanCluster' -Location (Get-Datacenter) -VsanEnabled
 -VsanDiskClaimMode 'Manual'
```

2. Configure a vSAN VMkernel port on each of the three hosts.

```
New-VMHostNetworkAdapter -VMHost 'Host-A' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
 -VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-B' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
 -VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-C' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
 -VsanTrafficEnabled $true
```

3. Create a vSAN disk group on each of the three hosts.

```
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-
CanonicalName' -VMHost 'Host-A'
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-CanonicalName'
 -VMHost 'Host-B'
New-VsanDiskGroup -DataDiskCanonicalName 'HDD1-CanonicalName' -SsdCanonicalName 'SSD1-CanonicalName'
 -VMHost 'Host-C'
```

4. Add each of the three hosts to the vSAN cluster to create a vSAN datastore.

```
Move-VMHost -VMHost 'Host-A' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-B' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-C' -Destination $vsanCluster
```

5. Revert the virtual machine hosts to the Connected state.

```
Set-VMHost -VMHost 'Host-A','Host-B','Host-C' -State 'Connected'
```

6. Create a virtual machine on the vSAN datastore.

```
$vsanDS = Get-Datastore -Name 'vsanDatastore'
$vm = New-VM -Name 'newVM' -DiskMB 1024 -Datastore $vsanDS -VMHost 'Host-A'
```

7. Create a storage policy by using any of the vSAN capabilities.

```
$cap = Get-SpbmCapability -Name vSAN*
$rule = New-SpbmRule $cap[1] $true
$ruleset = New-SpbmRuleSet $rule
$policy = New-SpbmStoragePolicy -Name 'vsan policy' -RuleSet $ruleset -Description 'vSAN-based storage
 policy'
```

8. Assign the storage policy to the virtual machine and its hard disk.

```
$vmHdd = Get-HardDisk -VM $vm
Set-SpbmEntityConfiguration $vm, $vmHdd -StoragePolicy $policy
```

9. Check the compliance of the virtual machine and its hard disk with the storage policy.

```
Get-SpbmEntityConfiguration $vm, $vmHdd
```

   The status should be `Compliant`.

Modify a vSAN Datastore

You can add or remove local disks from existing vSAN disk groups or remove entire vSAN disk groups.

- Verify that you are connected to a vCenter system.
- Verify that at least one vSAN disk group exists in the cluster.

1. Get the vSAN disk group from a cluster.

```
$dgs = Get-VsanDiskGroup -Cluster 'VsanCluster'
```

2. Get all vSAN disks from the vSAN disk group.

```
$dg = $dgs[0]
Get-VsanDisk –VsanDiskGroup $dg
```

3. Add a hard disk to the vSAN disk group.

```
$disk = New-VsanDisk -CanonicalName 'HDD-CanonicalName' -VsanDiskGroup $dg
```

4. Remove a hard disk from the vSAN disk group.

```
Remove-VsanDisk -VsanDisk $disk
```

5. Remove the entire vSAN disk group.

```
Remove-VsanDiskGroup -VsanDiskGroup $dg
```

Create a vSAN Stretched Cluster

You can create a vSAN stretched cluster with a witness node. You can then create a vSAN storage policy and enable performance service on the vSAN cluster.

- Verify that you are connected to a vCenter system.
- Verify that you have access to at least two virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD and one HDD.
- Verify that the virtual machine hosts are in maintenance mode.
- Verify that you have access to an ESX host that can be used as a witness host or deploy a witness appliance on any node. Ensure that the witness host or appliance is outside the vSAN cluster.

1. Configure a vSAN VMkernel port on each of the two hosts.

```
New-VMHostNetworkAdapter -VMHost 'Host-A' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
 -VsanTrafficEnabled $true
New-VMHostNetworkAdapter -VMHost 'Host-B' -PortGroup 'VMkernel' -VirtualSwitch 'vSwitch0'
 -VsanTrafficEnabled $true
```

2. Create a vSAN enabled cluster with automatic disk claim mode.

```
$vsanCluster = New-Cluster -Name 'VsanCluster' -Location (Get-Datacenter) -VsanEnabled
 -VsanDiskClaimMode 'Automatic'
```

3. Add the two hosts to the vSAN cluster to create a vSAN datastore.

```
Move-VMHost -VMHost 'Host-A' -Destination $vsanCluster
Move-VMHost -VMHost 'Host-B' -Destination $vsanCluster
```

4. Revert the virtual machine hosts to the Connected state.

```
Set-VMHost -VMHost 'Host-A','Host-B' -State 'Connected'
```

5. Create two fault domains in the vSAN cluster.

```
$primaryFd = New-VsanFaultDomain -Name 'Primary' -VMHost 'Host-A'
$secondaryFd = New-VsanFaultDomain -Name 'Secondary' -VMHost 'Host-B'
```

6. Enable stretched cluster.

```
            Set-VsanClusterConfiguration -Configuration $vsanCluster -StretchedClusterEnabled $true
-PreferredFaultDomain $primaryFd -WitnessHost 'Witness-Virtual-Appliance-IP'
```

7. Create a storage policy by using any vSAN capability.

```
$cap = Get-SpbmCapability -Name vSAN*
$rule = New-SpbmRule $cap[1] $true
$ruleset = New-SpbmRuleSet $rule
$policy = New-SpbmStoragePolicy -Name 'vsan policy' -RuleSet $ruleset -Description 'vSAN-based storage
 policy'
```

8. Enable performance service on the vSAN cluster.

```
            Set-VsanClusterConfiguration -Configuration $vsanCluster -PerformanceServiceEnabled $true
-StoragePolicy $policy
```

Enable a vSAN File Service

With VCF PowerCLI, you can enable a vSAN file service.

Verify that you are connected to a vCenter system.

1. Add a vSAN file service OVF to the OVF repository of the vCenter system.

```
            Add-VsanFileServiceOvf
```

2. Verify that the vSAN file service OVF is successfully added.

```
            Get-VsanFileServiceOvfInfo
```

3. Increase the VCF PowerCLI timeout for the current session, so that the operation does not stop.

```
            Set-PowerCLIConfiguration -WebOperationTimeoutSeconds 3600 -Scope Session
```

4. Retrieve the network that you want to use for the vSAN file service.

```
$net = Get-VirtualNetwork -Name 'MyNetwork'
```

5. Modify the vSAN cluster configuration that you want to use to enable the vSAN file service.

```
$config = Get-VsanClusterConfigurationSet-VsanClusterConfiguration $config -FileServiceEnabled
 $true -FileServiceNetwork $net
```

Create a vSAN File Service Domain

With VCF PowerCLI, you can create a vSAN file service domain.

- Verify that you are connected to a vCenter system.
- Verify that the vSAN file service is enabled.

1. Configure a vSAN file server IP configuration. You can use the IPs as an IP pool for the file service domain.

```
$ipConfig = New-VsanFileServerIpConfig -Fqdn "h101.vmware.com" -Gateway "192.2.8.3" -IpAddress
 "192.2.8.4" -SubnetMask "255.255.254.0" -IsPrimary
```

2. Create a vSAN file service domain.

```
            New-VsanFileServiceDomain -DnsServerAddress "10.172.199.241" -VsanFileServerIpConfig
$ipConfig -Name "MyFileServiceDomain" -DnsSuffix "vmware.com" -Cluster $config.Cluster
```

Create a vSAN File Share

With VCF PowerCLI, you can create a vSAN file share.

- Verify that you are connected to a vCenter system.
- Verify that the vSAN file service is enabled.
- Verify that the vSAN file service domain is created.

1. Get a vSAN file service domain.

```
$domain = Get-VsanFileServiceDomain -Name “MyFileServiceDomain”
```

2. Create a vSAN file share.

```
New-vsanfileshare -FileServiceDomain $domain -Name "MyVsanFileShare"
```

Create a vSAN ESA-Enabled Cluster

Starting from VMware Cloud Foundation PowerCLI 13.0.0 and vSAN 8.0, you can activate vSAN ESA (Express Storage Architecture) and manage storage pools on your clusters.

- Verify that you are connected to a vCenter 8.0 or later system.
- Verify that you have access to at least three virtual machine hosts.
- Verify that each of the virtual machine hosts has at least one SSD.

Starting from VMware Cloud Foundation PowerCLI 13.0.0 and vSAN 8.0, you can use vSAN ESA and the new storage format called storage pool disks. With vSAN ESA, you can boost performance with more predictable I/O latencies and optimized space efficiency. Each host that contributes storage contains a single storage pool of flash devices. Each flash device provides both caching and capacity to the vSAN ESA cluster. This is different from a disk group, which has dedicated devices in different tiers of cache and capacity.

1. Create a vSAN ESA-enabled cluster.

```
$vsanEsaCluster = New-Cluster -Name 'VsanESACluster' -Location (Get-Datacenter <MyDatacenter>) -
VsanEnabled -VsanEsaEnabled
```

2. Retrieve the available storage pool disks to claim on your cluster.

```
Get-VsanEsaEligibleDisk -Cluster $vsanEsaCluster
```

   The system returns the `ScsiLun` type of the available storage pool disks.

3. Add disks to the storage pool on the specified host.

   For `DiskCanonicalName`, use the canonical name from `ScsiLun`.

```
Add-VsanStoragePoolDisk -VMHost (Get-VMHost <MyVMHost>) -VsanStoragePoolDiskType "singleTier" -
DiskCanonicalName ("mpx.vmhba0:C0:T11:L0","mpx.vmhba0:C0:T12:L0")
```

4. Optional: You can remove a storage pool disk from the specified host.

```
$disks = Get-VsanStoragePoolDisk -VMHost <MyVMHost>
Remove-VsanStoragePoolDisk -VsanStoragePoolDisk $disks -VsanDataMigrationMode "nodatamigration" -Purpose
 "test" -Confirm:$false
```

Mount and Unmount Remote vSAN Datastores

Starting from VMware Cloud Foundation PowerCLI 12.7 and vSAN 7.0 Update 1, you can mount and unmount remote vSAN datastores to your vSAN-enabled clusters.

You can mount a remote datastore:

- From a vSAN or vSAN ESA-enabled cluster on the same vCenter system.
- From a vSAN stretched cluster.
- From a vSAN cluster on a different vCenter system.

Mount and Unmount a Remote vSAN Datastore from the Same vCenter System

Starting from VMware Cloud Foundation PowerCLI 12.7 and vSAN 7.0 Update 1, you can mount a remote vSAN datastore from another vSAN-enabled cluster residing on the same vCenter system.

To mount a datastore:

1. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.
   ```
   $config = Get-VsanClusterConfiguration -Cluster "MyCluster"
   ```
2. Mount the remote datastore.
   ```
   Set-VsanClusterConfiguration -Configuration $config -MountRemoteDatastore (Get-Datastore -Name
    "MyDatastore")
   ```
3. Retrieve the mounted datatore.
   ```
   $newVsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
   $newVsanConfiguration.RemoteDatastore
   ```

To unmount a datastore:

1. Retrieve the new vSAN cluster configuration of the cluster from which you want to unmount the remote datastore.
   ```
   $vsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
   ```
2. Unmount the remote datastore.
   ```
   Set-VsanClusterConfiguration -Configuration $vsanConfiguration -UnMountRemoteDatastore
    $vsanConfiguration.RemoteDatastore
   ```

   Important:
   Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount remote datastores from vSAN ESA-enabled clusters. You can follow the same steps as for vSAN clusters.

Mount and Unmount a Remote vSAN Datastore from a vSAN Stretched Cluster

Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount a remote vSAN datastore to a vSAN cluster from a vSAN stretched cluster.

To mount a datastore:

1. Configure the remote vSAN server cluster configuration.
   It depends on the network topology between the client and server clusters in your HCI Mesh configuration.
   a. In a symmetric HCI Mesh configuration:
      ```
      $serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -networktopology "Symmetric"
      ```
   b. In an asymmetric HCI Mesh configuration with one stretched cluster.
      ```
      $serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -
      networktopology "Asymmetric" -ServerSiteName "Secondary"
      ```
   c. In an asymmetric configuration where both the client and server cluster are stretched clusters.
      ```
      $serverconfig = New-RemoteVsanServerClusterConfig -cluster $clusterServer -
      networktopology "Asymmetric" -ServerSiteName "Secondary" -ClientSiteName "Preferred"
      ```
2. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.
   ```
   $clusterClient = Get-VsanClusterConfiguration -Cluster "MyCluster"
   ```
3. Mount the remote datastore.
   ```
   Set-VsanClusterConfiguration -configuration $clusterClient -MountRemoteDatastore (Get-Datastore -Name
    "MyDatastore") -RemoteVsanServerClusterConfig $serverconfig
   ```

To unmount a datastore:

1. Retrieve the new vSAN cluster configuration of the cluster from which you want to unmount the remote datastore.
   ```
   $newVsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
   ```
2. Unmount the datastore.

```
Set-VsanClusterConfiguration -configuration $newVsanConfiguration -UnmountRemoteDatastore
 $newVsanConfiguration.RemoteDatastore
```

Mount and Unmount a Remote vSAN Datastore from a Different vCenter System

Starting from VMware Cloud Foundation PowerCLI 13.1.0 and vSAN 8.0 Update 1, you can mount a remote vSAN datastore to a vSAN cluster from a vSAN cluster residing on a different vCenter system.

To mount a datastore:

1. Add the different vCenter as a datastore source to your current vCenter system.
   ```
   $dsSource = New-VsanHCIMeshDatastoreSource -VCHost <RemoteVCAddress> -User <MyVCUser>  -Password
    <MyVCPassword>
   ```
2. Retrieve the available remote datastores.
   ```
   $ds = Get-VsanHCIMeshDatastore -VsanHCIMeshDatastoresource $dsSource
   ```
3. Retrieve the vSAN cluster configuration of the cluster to which you want to mount the remote datastore.
   ```
   $clusterClient = Get-VsanClusterConfiguration -Cluster "MyCluster"
   ```
4. Mount the remote datastore.
   ```
   Set-VsanClusterConfiguration -Configuration $clusterClient -MountXVCDatastore $ds
   ```
5. Retrieve the mounted remote datastore.
   ```
   $vsanConfiguration = Get-VsanClusterConfiguration -Cluster "MyCluster"
   $vsanConfiguration.VsanXVCDatastore
   ```

To unmount the datastore:

```
Set-VsanClusterConfiguration -Configuration $vsanConfiguration -UnmountXVCDatastore $ds
```

Managing the vSphere Replication API with VMware Cloud Foundation PowerCLI

Use the vSphere Replication SDK for PowerShell to access the vSphere Replication API with VCF PowerCLI. Starting from VMware Cloud Foundation PowerCLI version 13.1.0, the vSphere Replication SDK provides low-level cmdlets for all available vSphere Replication API operations.

Understanding the vSphere Replication SDK for PowerShell

You can access the vSphere Replication API with VMware Cloud Foundation PowerCLI by using the vSphere Replication SDK for PowerShell.

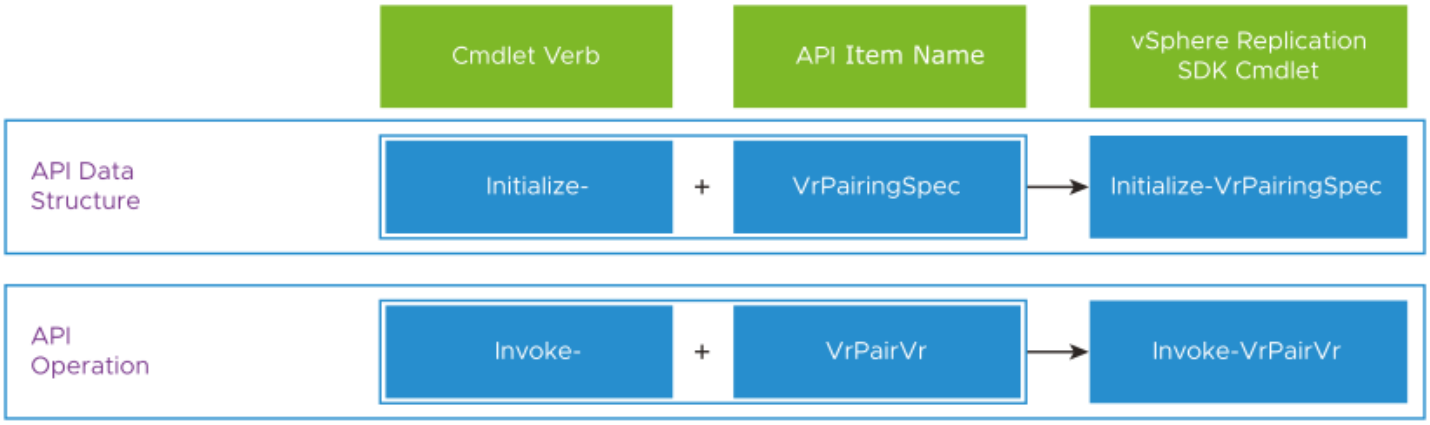
How Does the vSphere Replication SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 13.1.0, you can use the vSphere Replication SDK for PowerShell to communicate with all available vSphere Replication REST APIs.

The `VMware.Sdk.Vr` module contains automatically-generated SDK cmdlets that bind directly to the vSphere Replication REST APIs. The module functions on a low level and communicates directly with the vSphere Replication API server.

You can construct vSphere Replication SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the vSphere Replication API data structures or operations.

Figure 660: Constructing vSphere Replication SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the vSphere Replication API server.
- Use `Invoke-` cmdlets to call the vSphere Replication API server and invoke the operations.

Navigating the vSphere Replication SDK for PowerShell

You can navigate the vSphere Replication SDK for PowerShell by using the VMware Developer website, and the `Get-Help`, `Get-Command`, and `Get-VrOperation` cmdlets.

To use the vSphere Replication SDK for PowerShell effectively, you must be familiar with the vSphere Replication API reference documentation .

How to Find a vSphere Replication Cmdlet

There are several ways to find the vSphere Replication cmdlets you need.

- In your browser, navigate the vSphere Replication API reference documentation.
  The vSphere Replication API Reference on VMware Developer features ready-to-use vSphere Replication SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.
- In PowerShell, run `Get-Command -Module VMware.Sdk.Vr` to list all available vSphere Replication cmdlets.
- If you want to use information from the REST API specification to find a cmdlet, you can run `Get-VrOperation` with the `Method` and `Path` parameters.
  For example:
  ```
  Get-VrOperation -Method GET -Path '/replication-servers'
  ```
  The command returns an object with the operation details.
  ```
  Name                : GetAllVrServers
  CommandInfo         : Invoke-VrGetAllVrServers
  ApiName             : ServerApi
  Path                : /replication-servers
  Tags                : {server}
  RelatedCommandInfos : {}
  Method              : GET
  Parameters          : {filterProperty, filter, sortBy, orderBy...}
  ```
  The cmdlet name is listed in the `CommandInfo` property.

How to Get Information About a Cmdlet

To get information about a vSphere Replication cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-VrConfigureReplication -full
```

Connect to a Local vSphere Replication Server

For replications within the same vCenter system, you must first connect to a local vSphere Replication server. You can do this by using the `Connect-VrServer` cmdlet.

- Verify that a vSphere Replication appliance is deployed at the local vCenter system.
- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Run `Connect-VrServer` and create a variable for the connection. Use the local vSphere Replication server address with the local vCenter user credentials.

   ```
   $vr1 = Connect-VrServer -Server <local_vr_address_or_fqdn> -User 'MyVCUser' -Password 'MyVCPassword'
   ```

2. Optional: Print the connection variable to see the `VrServerConnection` object you created.

   ```
   $vr1
   ```

   The `VrServerConnection` object contains the `ConnectedPairings` property with the pairing you created. You might need this property to extract information required by some vSphere Replication operations.

Connect to a Local and to a Remote vSphere Replication Server

For replications from a source to a target vCenter system, you use the `Connect-VrServer` cmdlet to connect to the local and to the remote vSphere Replication server.

- Verify that vSphere Replication appliances are deployed at the local and at the remote vCenter systems.
- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Run `Connect-VrServer` and create a variable for the connection. Use the local vSphere Replication server address with the local vCenter user credentials. For the remote server connection, use the remote vCenter address with the remote vCenter user credentials.

   ```
   $vr2 = Connect-VrServer -Server <local_vr_address_or_fqdn> -User 'MyVCUser' -Password 'MyVCPassword' -
   RemoteServer <remote_vc_server_name> -RemoteUser 'MyRemoteVCUser' -RemotePassword 'MyRemoteVCPassword'
   ```

2. Optional: Print the connection variable to see the `VrServerConnection` object you created.

   ```
   $vr2
   ```

   The `VrServerConnection` object contains the `ConnectedPairings` property with the pairing you created. You might need this property to extract information required by some vSphere Replication operations.

Use the ConnectedPairings Property

Use the `ConnectedPairings` property to extract connection information that you might need for vSphere Replication SDK operations.

Verify that you have a valid `VrServerConnection`. This example uses the *`$vr2`* connection variable from Connect to a Local and to a Remote vSphere Replication Server.

The `ConnectedPairings` property is a key-value dictionary that contains information about all available vSphere Replication pairings. Use `ConnectedPairings` to extract information such as `PairingID` and `VcServerID`, which is required for some vSphere Replication SDK operations.

1. To list the connected pairings for a vSphere Replication connection, run:

   ```
   $vr2.ConnectedPairings
   ```

   The system lists a key-value map with your connected pairings.

2. To list a pairing, run:

   ```
   $vr2.ConnectedPairings[<vc_server_name>].Pairing
   ```

   The system returns the Pairing object with all its properties. You can copy the `PairingID` property as it is required by some vSphere Replication API operations.

3. To list the `LocalVcServer` object, run:

   ```
   $vr2.ConnectedPairings[<vc_server_name>].Pairing.LocalVcServer
   ```

   The system returns a `ServerInfo` object that contains a server ID that is required by some vSphere Replication API operations.

4. To list the `RemoteVcServer` object, run:

   ```
   $vr2.ConnectedPairings[<vc_server_name>].Pairing.RemoteVcServer
   ```

   The system returns a `ServerInfo` object that contains a server ID that is required by some vSphere Replication API operations.

Replicate a VM to a Datastore on the Local vCenter System

You can use the vSphere Replication SDK for PowerShell to replicate a VM from a source to a target datastore on the local vCenter system.

- Verify that the vSphere Replication appliance is deployed on the vCenter system.
- Verify that you are connected to the vCenter system. This example uses the *`$vr1`* connection variable from Connect to a Local vSphere Replication Server.
- You must have the required privileges to configure replications and manage datastores on the vCenter system.

This example shows how to replicate a single VM from a source to a target datastore on the same vCenter system.

1. Get the local VM that you want to replicate.

   ```
   $pairing = $vr1.ConnectedPairings["<local_vc_address_or_fqdn>"].Pairing
   $replicatedVm = Invoke-VrGetLocalVms -PairingId $pairing.PairingId `
       -VcenterId $pairing.LocalVcServer.Id `
       -FilterProperty "Name" `
       -Filter "<my_vm_name>" `
       -Server $vr1
   ```

2. Get the VM hard disks that you want to replicate.

   ```
   $replicatedHdds = Invoke-VrGetLocalVmDisks -PairingId $pairing.PairingId `
       -VcenterId $pairing.LocalVcServer.Id `
       -VmId $replicatedVm.List[0].Id `
       -Server $vr1
   ```

3. Get the target datastore for the replication.

   ```
   $targetDatastore = Invoke-VrGetVrCapableTargetDatastores -PairingId $pairing.PairingId `
       -VcenterId  $pairing.RemoteVcServer.Id `
       -FilterProperty "Name" `
       -Filter "<target_datastore_name>" `
   ```

```
    -Server $vr1
```

4. Create the configuration for disks replication.

```
$replicationVmDisks = @()
$replicatedHdds.List | ForEach-Object {
   $replicationVmDisks += Initialize-VrConfigureReplicationVmDisk -VmDisk $_ `
     -EnabledForReplication:$true `
     -DestinationDatastoreId $targetDatastore.List[0].Id `
     -DestinationDiskFormat 'SAMEASSOURCE'
```

5. Create the replication spec.

```
$replicationSpec = Initialize-VrConfigureReplicationSpec `
    -Rpo 60 -NetworkCompressionEnabled:$true `
    -MpitEnabled:$true -AutoReplicateNewDisks:$true `
    -LwdEncryptionEnabled:$false `
    -Disks $replicationVmDisks `
    -MpitInstances 1 `
    -MpitDays 1 `
    -TargetVcId $pairing.RemoteVcServer.Id `
    -VmId $replicatedVm.List[0].Id
```

6. Invoke the ConfigureReplication operation.

```
$task = Invoke-VrConfigureReplication -PairingId $pairing.PairingId `
    -ConfigureReplicationSpec $replicationSpec -Server $vr1
```

   vSphere Replication starts replicating the virtual machine files to the target datastore.

7. Monitor the task progress.

```
$t = Invoke-VrGetTaskInfo -TaskId $task.List[0].Id -Server $vr1
$t
```

   Eventually, the task status changes to `SUCCESS`.

8. Optional: Get the replication on the target datastore.

```
$rep1 = Invoke-VrGetAllReplications -PairingId $pairing.PairingId `
-FilterProperty "Name" -Filter <my_vm_name> -Server $vr1
```

9. Sync your virtual machine to the configured replication on the target datastore.

```
Invoke-VrSyncReplication -PairingId $pairing.PairingId `
    -ReplicationId $rep1.List[0].Id -Server $vr1
```

Replicate a VM to a Datastore on a Remote vCenter System

You can use the vSphere Replication SDK for PowerShell to replicate a VM from a source to a target vCenter system.

- Verify that the vSphere Replication appliance is deployed on the source and target sites.
- Verify that your `VrServerConnection` is authenticated against the source and target vCenter systems. This example uses the *`$vr2`* connection variable from Connect to a Local and to a Remote vSphere Replication Server.
- You must have the required privileges to configure replications and manage datastores on the source and target sites.

This example shows how to replicate a single VM from a datastore on the local vCenter system to a datastore on a remote vCenter system.

1. Get the local VM that you want to replicate.

```
$pairing = $vr2.ConnectedPairings["<remote_vc_server_name>"].Pairing
$replicatedVm = Invoke-VrGetLocalVms -PairingId $pairing.PairingId `
    -VcenterId $pairing1.LocalVcServer.Id `
    -FilterProperty "Name" `
    -Filter "<my_vm_name>" `
```

```
    -Server $vr2
```

2. Get the VM hard disks that you want to replicate.

```
$replicatedHdds = Invoke-VrGetLocalVmDisks -PairingId $pairing.PairingId `
    -VmId $replicatedVm.List[0].Id `
    -VcenterId $pairing.LocalVcServer.Id `
    -Server $vr2
```

3. Get the target datastore on the remote site.

```
$targetDatastore = Invoke-VrGetVrCapableTargetDatastores  -PairingId $pairing.PairingId `
    -VcenterId $pairing.RemoteVcServer.Id `
    -FilterProperty "Name" -Filter "<remote_datastore_name>" -Server $vr2
```

4. Create the configuration for disks replication.

```
   $replicatedHdds.List | ForEach-Object {
      $replicationVmDisks += Initialize-VrConfigureReplicationVmDisk -VmDisk $_ `
         -EnabledForReplication:$true `
         -DestinationDatastoreId $targetDatastore.List[0].Id `
         -DestinationDiskFormat 'SAMEASSOURCE'
   }
```

5. Create the replication spec.

```
$replicationSpec = Initialize-VrConfigureReplicationSpec `
    -Rpo 60 -NetworkCompressionEnabled:$true `
    -MpitEnabled:$true -AutoReplicateNewDisks:$true `
    -LwdEncryptionEnabled:$false `
    -Disks $replicationVmDisks `
    -MpitInstances 1 `
    -MpitDays 1 `
    -TargetVcId $pairing.RemoteVcServer.Id `
    -VmId $replicatedVm.List[0].Id
```

6. Invoke the ConfigureReplication operation.

```
$task = Invoke-VrConfigureReplication -PairingId $pairing.PairingId `
    -ConfigureReplicationSpec $replicationSpec -Server $vr2
```

   vSphere Replication starts replicating the virtual machine files to the target site datastore.

7. Monitor the task progress.

```
$t = Invoke-VrGetTaskInfo -TaskId $task.List[0].Id -Server $vr2
$t
```

   Eventually, the task status must change to `SUCCESS`.

8. Optional: Get the replication on the remote site.

```
$rep2 = Invoke-VrGetAllReplications -PairingId $pairing.PairingId `
-FilterProperty "Name" -Filter <my_vm_name> -Server $vr2
```

9. Optional: Sync your virtual machine to the configured replication on the target site.

```
Invoke-VrSyncReplication -PairingId $pairing.PairingId `
    -ReplicationId $rep2.List[0].Id -Server $vr2
```

Managing VMware Live Site Recovery™ with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware Live Site Recovery™ (SRM) administration.

Connect to an SRM Server

To use the SRM API, you must establish a connection to an SRM server.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

Some of the objects returned by the SRM API are objects from the vSphere API. To use those objects in integration with the vSphere API through PowerCLI, you can connect to the vCenter system that the SRM server is registered with.

1. To connect to the vCenter system that the SRM server is registered with, run `Connect-VIServer` with the server name and valid credentials.

   ```
   Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
   ```

2. To connect to the SRM server registered with the connected vCenter system, run `Connect-SrmServer`.

   ```
   $srmConnection = Connect-SrmServer
   ```

   Note:
   If you have previously connected to other vCenter systems configured with SRM server support, this cmdlet invocation establishes a connection to their corresponding SRM servers as well.

3. Optional: To use the SRM API, you can call methods of the root object and instances of the objects that those calls return.

   ```
   $srmApi = $srmConnection.ExtensionData
   ```

   Note:
   The root SRM API object is the `ExtensionData` property of the `SrmServer` object.

Protect a Virtual Machine

You can protect a virtual machine by replicating it to a remote SRM site.

1. Connect to the vCenter system that the SRM server is registered with.

   ```
   Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
   ```

2. Establish a connection to the local SRM server by providing credentials to the remote SRM site.

   ```
   $srmConnection = Connect-SrmServer -RemoteUser 'MyRemoteUser' -RemotePassword 'MyRemotePassword'
   ```

3. List all protection groups associated with the SRM server.

   ```
   $srmApi = $srmConnection.ExtensionData
   $protectionGroups = $srmApi.Protection.ListProtectionGroups()
   ```

4. Associate the *`TestVM`* virtual machine with the *`ProtGroup1`* protection group and enable the protection for that virtual machine.

   ```
   $vmToAdd = Get-VM "TestVM"

   $targetProtectionGroup = $protectionGroups | where {$_.GetInfo().Name -eq "ProtGroup1" }

   $targetProtectionGroup.AssociateVms(@($vmToAdd.ExtensionData.MoRef))

   # Enable protection for that virtual machine
   $protectionSpec = New-Object VMware.VimAutomation.Srm.Views.SrmProtectionGroupVmProtectionSpec
   ```

```
$protectionSpec.Vm = $vmToAdd.ExtensionData.MoRef
$protectTask = $targetProtectionGroup.ProtectVms($protectionSpec)
while(-not $protectTask.IsComplete()) { sleep -Seconds 1 }
```

Create a Report of the Protected Virtual Machines

You can create a simple report containing information about the protected virtual machines associated with an SRM server.

- Verify that you are connected to a vCenter system.
- Verify that you are connected to an SRM server.

1. List all protection groups associated with the SRM server.

```
$srmApi = $srmConnection.ExtensionData
$protectionGroups = $srmApi.Protection.ListProtectionGroups()
```

2. Generate a report of the protected virtual machines.

```
$protectionGroups | % {
    $protectionGroup = $_

    $protectionGroupInfo = $protectionGroup.GetInfo()

    # The following command lists the virtual machines associated with a protection group
    $protectedVms = $protectionGroup.ListProtectedVms()
    # The result of the above call is an array of references to the virtual machines at the vSphere API
    # To populate the data from the vSphere connection, call the UpdateViewData method on each virtual
 machine view object
    $protectedVms | % { $_.Vm.UpdateViewData() }
    # After the data is populated, use it to generate a report
    $protectedVms | %{
        $output = "" | select VmName, PgName
        $output.VmName = $_.Vm.Name
        $output.PgName = $protectionGroupInfo.Name
        $output
    }
} | Format-Table @{Label="VM Name"; Expression={$_.VmName} }, @{Label="Protection group name";
 Expression={$_.PgName} }
```

Create a Report of the Virtual Machines Associated with All Protection Groups

You can create a simple report containing information about the virtual machines associated with all protection groups.

- Verify that you are connected to a vCenter system.
- Verify that you are connected to an SRM server.

1. List all protection groups associated with the SRM server.

```
$srmApi = $srmConnection.ExtensionData
$protectionGroups = $srmApi.Protection.ListProtectionGroups()
```

2. Generate a report of the virtual machines associated with all protection groups.

```
$protectionGroups | % {
    $protectionGroup = $_

    $protectionGroupInfo = $protectionGroup.GetInfo()

    # The following command lists the virtual machines associated with a protection group
    $vms = $protectionGroup.ListAssociatedVms()
    # The result of the above call is an array of references to the virtual machines at the vSphere API
```

```
    # To populate the data from the vSphere connection, call the UpdateViewData method on each virtual
machine view object
    $vms | % { $_.UpdateViewData() }
    # After the data is populated, use it to generate a report
    $vms | %{
        $output = "" | select VmName, PgName
        $output.VmName = $_.Name
        $output.PgName = $protectionGroupInfo.Name
        $output
    }
} | Format-Table @{Label="VM Name"; Expression={$_.VmName} }, @{Label="Protection group name";
 Expression={$_.PgName} }
```

Managing the VMware Live Site Recovery API with VMware Cloud Foundation PowerCLI

Use the VMware Live Site Recovery SDK for PowerShell to access the VMware Live Site Recovery API with VCF PowerCLI. Starting from VMware Cloud Foundation PowerCLI version 13.1.0, the VMware Live Site Recovery SDK provides low-level cmdlets for all available VMware Live Site Recovery API operations.

Understanding the VMware Live Site Recovery SDK for PowerShell

You can access the VMware Live Site Recovery API with VMware Cloud Foundation PowerCLI by using the VMware Live Site Recovery SDK for PowerShell.

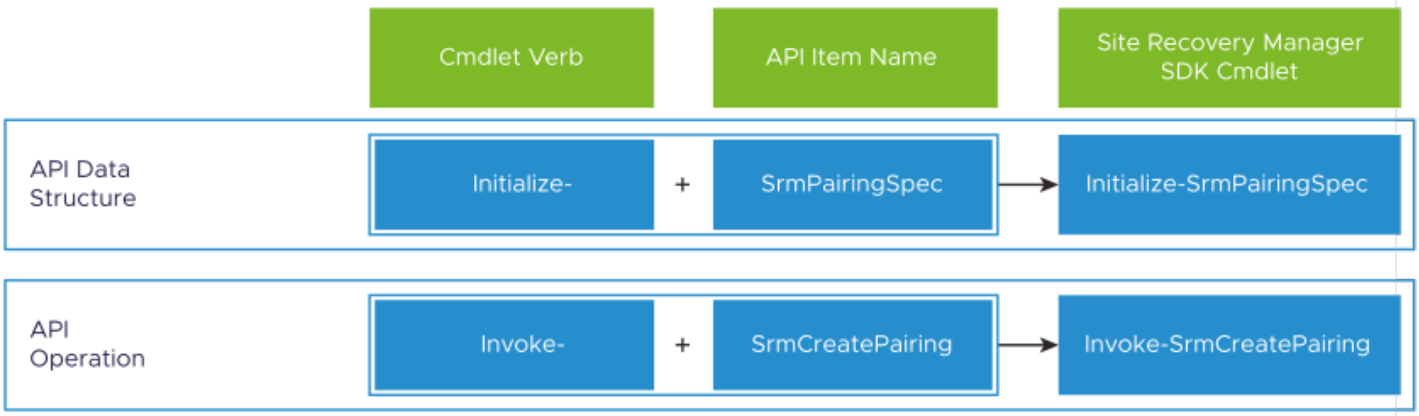
How Does the VMware Live Site Recovery SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 13.1.0, you can use the VMware Live Site Recovery SDK for PowerShell to communicate with the VMware Live Site Recovery REST APIs.

The `VMware.Sdk.Srm` module contains automatically-generated SDK cmdlets that bind directly to the VMware Live Site Recovery REST APIs. The module functions on a low level and communicates directly with the VMware Live Site Recovery API server.

You can construct VMware Live Site Recovery SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the VMware Live Site Recovery API data structures or operations.

Figure 661: Constructing VMware Live Site Recovery SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the VMware Live Site Recovery API server.
- Use `Invoke-` cmdlets to call the VMware Live Site Recovery API server and invoke the operations.

Navigating the VMware Live Site Recovery SDK for PowerShell

You can navigate the VMware Live Site Recovery SDK for PowerShell by using the VMware Developer website, and the `Get-Help`, `Get-Command`, and `Get-SrmOperation` cmdlets.

To use the VMware Live Site Recovery SDK for PowerShell effectively, you must be familiar with the VMware Site Recovery Manager REST API reference documentation .

How to Find a VMware Live Site Recovery SDK Cmdlet

There are several ways to find the VMware Live Site Recovery SDK cmdlets you need.

- In your browser, navigate the VMware Live Site Recovery API reference documentation.
  The VMware Live Site Recovery API Reference on VMware Developer features ready-to-use VMware Live Site Recovery SDK code samples that you can paste directly into PowerShell. You can find a VCF PowerCLI code snippet for each operation under *VCF PowerCLI Client SDK Example* in the *Code Samples* section.
- In PowerShell, run `Get-Command -Module VMware.Sdk.Srm` to list all available vSphere Replication cmdlets.
- If you want to use information from the REST API specification to find a cmdlet, you can run `Get-SrmOperation` with the `Method` and `Path` parameters.
  For example:

  ```
  Get-SrmOperation -Method GET -Path '/pairings/{pairing_id}/protection-management/groups'
  ```

  The command returns an object with the operation details.

  ```
  Name                : GetAllGroups
  CommandInfo         : Invoke-SrmGetAllGroups
  ApiName             : ProtectionApi
  Path                : /pairings/{pairing_id}/protection-management/groups
  Tags                : {protection}
  RelatedCommandInfos : {}
  Method              : GET
  Parameters          : {pairingId, filterProperty, filter, sortBy...}
  ```

  The cmdlet name is listed in the `CommandInfo` property.

How to Get Information About a Cmdlet

To get information about a VMware Live Site Recovery SDK cmdlet, run `Get-Help` with the cmdlet name. For example:

```
Get-Help
Invoke-SrmGetAllGroups -full
```

Connect to a Local and to a Remote VMware Live Site Recovery Server

To invoke operations on the VMware Live Site Recovery API, you must first connect to the VMware Live Site Recovery instances on the local (protected) and the remote (recovery) sites.

- Verify that VMware Live Site Recovery™ is deployed to the local (protected) and to the remote (recovery) vCenter systems.
- Verify that the VMware Live Site Recovery instances on the local and the remote sites are connected.
- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

1. Connect to the local and to the remote VMware Live Site Recovery servers.

```
$localSrm = <local_srm_address_or_fqdn>
$srmConnection = Connect-SrmSdkServer -Server $localSrm `
    -User 'MyLocalVCUser' `
    -Password 'MyLocalVCPassword' `
    -RemoteUser 'MyRemoteVCUser' `
    -RemotePassword 'MyRemoteVCPassword'
```

2. Optional: Print the `ConnectedPairing` property of the connection you created.

```
$srmConnection.ConnectedPairing
```

   The command prints the `Pairing` object that represents the pair between the local and the remote VMware Live Site Recovery sites.

   You will need the `Pairing` object and its properties for various VMware Live Site Recovery API operations.

Host-Based Replication Scenarios

You can use the VMware Live Site Recovery SDK for PowerShell to manage scenarios for host-based replication (HBR) with VMware Cloud Foundation PowerCLI. HBR operates at the hypervisor level and allows you to replicate virtual machine data across different hosts or storage systems.

Create Protection Group for Host-Based Replication

To activate the protection of virtual machines, you first create a protection group for host-based replication (HBR).

- Verify that your SRM connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the *`$srmConnection`* variable from Connect to a Local and to a Remote VMware Live Site Recovery Server.
- The VMs that you want to protect must be replicated by using vSphere Replication.
- You must have the required privileges to create protection groups on the recovery site.

You create protection groups to enable VMware Live Site Recovery to protect virtual machines. This example shows how to create a protection group for host-based replication by using the VMware Live Site Recovery SDK for PowerShell.

1. Get the VM that you want to add to a protection group.

```
$replicatedVm = Invoke-SrmGetReplicatedVms `
    -PairingId $srmConnection.ConnectedPairing.PairingId `
    -VcenterId $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -FilterProperty "Name" `
    -Filter "<myVmName>"
```

2. Prepare the HBR protection group spec.

```
$hbrSpec = Initialize-SrmHbrProtectionGroupSpec -Vms $replicatedVm.List[0].Id
```

3. Prepare the protection group creation spec.

   For the `ReplicationType` parameter, specify *`HBR`*.

```
$protectionGroupSpec = Initialize-SrmProtectionGroupCreateSpec -Name "<myProtectionGroupName>" `
    -ReplicationType HBR `
    -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -HbrSpec $hbrSpec
```

4. Create the protection group task.

```
$task = Invoke-SrmCreateGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
    -ProtectionGroupCreateSpec $protectionGroupSpec
```

5. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

   `$task.Status` eventually changes to `SUCCESS`.

6. Optional: Retrieve the protection group you created.

```
$protectionGroup = Invoke-SrmGetGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
   -GroupId $task.Result
```

Create a recovery plan for your protection group.

Create Recovery Plan

To ensure business continuity and facilitate efficient disaster recovery for your virtual machines, you create a recovery plan for a protection group in a given pairing.

- Verify that your VMware Live Site Recovery connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the *$srmConnection* variable from Connect to a Local and to a Remote VMware Live Site Recovery Server.
- You must have created one or more protection groups on the protected (local) site. This example uses the *$protectionGroup* variable from Create Protection Group for Host-Based Replication.
- You must have the required privileges to create recovery plans on the recovery site.

1. Create the recovery plan spec.

```
$recPlanSpec = Initialize-SrmRecoveryPlanCreateSpec `
    -Name "<myRecoveryPlanName>" `
    -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
    -ProtectionGroups $protectionGroup.Id
```

2. Create the recovery plan task.

```
$task = Invoke-SrmCreatePlan -PairingId $srmConnection.ConnectedPairing.PairingId `
    -RecoveryPlanCreateSpec $recPlanSpec
```

3. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

   Eventually, the task status changes to `SUCCESS`.

4. Optional: Retrieve the recovery plan you created.

```
$recoveryPlan = Invoke-SrmGetRecoveryPlan -PairingId $srmConnection.ConnectedPairing.PairingId `
-PlanId $task.Result
```

Array-Based Replication Scenarios

You can use the VMware Live Site Recovery SDK for PowerShell to manage scenarios for array-based replication (ABR) with VMware Cloud Foundation PowerCLI. With ABR, storage arrays at the protected site replicate data to peer arrays at the recovery site, typically at the block level.

Configure Array Managers for an Array Pair

Before creating a replicated array pair, you must configure array managers for the protected (local) and recovery (remote) SRM sites.

- Verify that your SRM connection is authenticated against the protected (local) and recovery (remote) vCenter systems. This example uses the *$srmConnection* variable from Connect to a Local and to a Remote VMware Live Site Recovery Server.
- Verify that the SRM instances on the protected (local) and recovery (remote) sites are connected. This is known as site pairing.
- Verify that storage replication adapters (SRAs) are installed at both sites.
- You must have the required privileges to create array managers on the protected and recovery sites.

Configure array managers for the protected and recovery sites to enable the creation of an array pair for array-based replication (ABR). Array managers must be configured so that VMware Live Site Recovery can discover replicated devices, compute datastore groups, and initiate storage operations.

1. Retrieve the storage replication adapter (SRA) for the local array manager.

```
$localAdapter = (Invoke-SrmGetStorageAdapters -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -FilterProperty "Name" `
   -Filter "<myAdapterName>").List[0]
```

2. Prepare the connection parameters for the local SRA.
   a) Retrieve the `ConnectionParamGroup` object for the local SRA.

```
$localConnectionParamGroup = (Invoke-SrmGetStorageAdapterConnectionParams `
   -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -StorageAdapterId $localAdapter.Id).List[0]
```

   b) Provide values to the connection parameters, available in the *`$localConnectionParamGroup.ConnectionParams`* list.

   The set of connection parameters is specific to the used SRA. This example uses parameters specific to the NetApp SRA.

```
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "username"}).Value = "<myNetAppUser>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "password"}).Value = "<myNetAppPass>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "VolumeInclusion"}).Value =
 "<myVolume>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "SVM Name"}).Value =
 "<myLocalSvmName>"
($localConnectionParamGroup.ConnectionParams | where {$_.key -eq "mgmtIP"}).Value = "<myNetAppMgmtIP>"
```

3. Prepare the local array manager creation spec.

```
$localArrayManagerSpec = Initialize-SrmArrayManagerSpec -Name "<myLocalArrayManagerName>" `
   -StorageAdapter $localAdapter.Id `
   -ConnectionParams $localConnectionParamGroup
```

4. Create the local array manager.

```
$task = Invoke-SrmCreateArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -ArrayManagerSpec $localArrayManagerSpec
```

5. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

   `$task.Status` eventually changes to `SUCCESS`.

6. Optional: Retrieve the local array manager.

   The `Result` property of the *`$localManagerTask`* object contains the ID of the created array manager.

```
$localManagerTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$localArrayManager = Invoke-SrmGetArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -ArrayManagerId $localManagerTask.Result
```

7. Create the remote array manager by using the same approach.

```
$remoteAdapter = (Invoke-SrmGetStorageAdapters -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
   -FilterProperty "Name" `
   -Filter "<myAdapterName>").List[0]

$remoteConnectionParamGroup = (Invoke-SrmGetStorageAdapterConnectionParams `
   -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
```

```
   -StorageAdapterId $remoteAdapter.Id).List[0]

($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "username"}).Value =
 "<myNetAppUser>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "password"}).Value =
 "<myNetAppPass>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "VolumeInclusion"}).Value =
 "<myVolume>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "SVM Name"}).Value =
 "<myRemoteSvmName>"
($remoteConnectionParamGroup.ConnectionParams | Where-Object {$_.key -eq "mgmtIP"}).Value =
 "myNetAppMgmtIP"

$remoteArrayManagerSpec = Initialize-SrmArrayManagerSpec -Name "<myRemoteArrayManagerName>" `
   -StorageAdapter $remoteAdapter.Id `
   -ConnectionParams $remoteConnectionParamGroup

$task = Invoke-SrmCreateArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
   -ArrayManagerSpec $remoteArrayManagerSpec

#Wait for the create array manager task to complete.
$remoteManagerTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$remoteArrayManager = Invoke-SrmGetArrayManager -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.RemoteSrmServer.Id `
   -ArrayManagerId $remoteManagerTask.Result
```

Create a replicated array pair for array-based replication.

Create Replicated Array Pair

Before you can create protection groups and recovery plans for ABR, you must create a replicated array pair for the protected (local) and recovery (remote) SRM sites.

- Verify that your SRM connection is authenticated against the protected and recovery vCenter systems. This example uses the *`$srmConnection`* variable from Connect to a Local and to a Remote VMware Live Site Recovery Server.
- Verify that you configured array managers for the protected (local) and recovery (remote) SRM sites. This example uses the variables from Configure Array Managers for an Array Pair.
- You must have the required privileges to create replicated array pairs on the protected and recovery sites.

1. Prepare the replicated array pair create spec.

   You must specify the local array manager ID, and the storage array keys of the local (protected) and remote (recovery) array managers.

   ```
   $replicatedArrayPairSpec = Initialize-SrmReplicatedArrayPairSpec -ArrayManagerId $localArrayManager.Id `
      -StorageArrayKey $localArrayManager.StorageArrays[0].StorageArrayKey `
      -PeerStorageArrayKey $remoteArrayManager.StorageArrays[0].StorageArrayKey
   ```

2. Create the replicated array pair.

   ```
   $task = Invoke-SrmCreateReplicatedArrayPair -PairingId $srmConnection.ConnectedPairing.PairingId `
      -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
      -ReplicatedArrayPairSpec $replicatedArrayPairSpec
   ```

3. Monitor the task progress.

   ```
   $task = Invoke-SrmGetTaskInfo -TaskId $task.Id
   ```

   `$task.Status` eventually changes to `SUCCESS`.

4. Optional: Retrieve the replicated array pair.

   The `Result` property of the *`$replicatedArrayPairTask`* object contains the ID of the replicated array pair.

```
$replicatedArrayPairTask = Invoke-SrmGetTaskInfo -TaskId $task.Id

$replicatedArrayPair = Invoke-SrmGetReplicatedArrayPair -PairingId
 $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -ArrayPairId $replicatedArrayPairTask.Result
```

Create protection group and recovery plan by using the replicated array pair.

Create Protection Group and Recovery Plan for ABR

To ensure business continuity and facilitate efficient disaster recovery for your virtual machines with array-based replication, you create protection groups and recovery plans.

- Verify that your SRM connection is authenticated against the protected and recovery vCenter systems. This example uses the *`$srmConnection`* variable from Connect to a Local and to a Remote VMware Live Site Recovery Server.
- Verify that you created a replicated array pair for the protected (local) and recovery (remote) SRM sites. This example uses the variables from Create Replicated Array Pair
- You must have the required privileges to create protection groups and recovery plans on the recovery site.

1. Discover the storage devices of the replicated array pair.

```
$task = Invoke-SrmDiscoverStorageDevices -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -ArrayPairId $replicatedArrayPair.Id
```

2. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

   `$task.Status` eventually changes to `SUCCESS`.

3. Retrieve the storage devices from the replicated array pair.

```
$storageDevices = Invoke-SrmGetStorageDevices -PairingId $srmConnection.ConnectedPairing.PairingId `
   -SrmId $srmConnection.ConnectedPairing.LocalSrmServer.Id `
   -ArrayPairId $replicatedArrayPair.Id
```

4. Prepare the ABR protection group spec.

   This example uses the replicated array pair with a single storage device and a single datastore created in Create Replicated Array Pair.

```
$abrProtectionGroupSpec = Initialize-SrmAbrProtectionGroupSpec `
   -ReplicatedArrayPair $replicatedArrayPair.Id `
   -Datastores $storageDevices.StorageDevicesData[0].Datastores[0].Id
```

5. Prepare the protection group create spec.

   For the `ReplicationType` parameter, specify *`ABR`*.

```
$protectionGroupSpec = Initialize-SrmProtectionGroupCreateSpec -Name "<MyAbrProtectionGroup>" `
   -ReplicationType ABR `
   -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
   -AbrSpec $abrProtectionGroupSpec
```

6. Create the ABR protection group.

```
$task = Invoke-SrmCreateGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
   -ProtectionGroupCreateSpec $protectionGroupSpec
```

7. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

8. Optional: Retrieve the ABR protection group.

```
$protectionGroup = Invoke-SrmGetGroup -PairingId $srmConnection.ConnectedPairing.PairingId `
   -GroupId $task.Result
```

9. Prepare the recovery plan create spec.

```
$recPlanSpec = Initialize-SrmRecoveryPlanCreateSpec `
   -Name "<MyAbrRecoveryPlan>" `
   -ProtectedVcGuid $srmConnection.ConnectedPairing.LocalVcServer.Id `
   -ProtectionGroups @($protectionGroup.Id)
```

10. Create the recovery plan.

```
$task = Invoke-SrmCreatePlan -PairingId $srmConnection.ConnectedPairing.PairingId `
   -RecoveryPlanCreateSpec $recPlanSpec
```

11. Monitor the task progress.

```
$task = Invoke-SrmGetTaskInfo -TaskId $task.Id
```

`$task.Status` eventually changes to `SUCCESS`.

Managing the NSX Policy API with VMware Cloud Foundation PowerCLI

Manage your networking automation with VCF PowerCLI and the NSX Policy SDK for PowerShell. Starting from VCF PowerCLI 12.6, you can use low-level SDK cmdlets to communicate with all available NSX Policy REST APIs.

Understanding the NSX Policy SDK for PowerShell

You can manage the NSX Policy API with VMware Cloud Foundation PowerCLI by using the NSX Policy SDK for PowerShell.

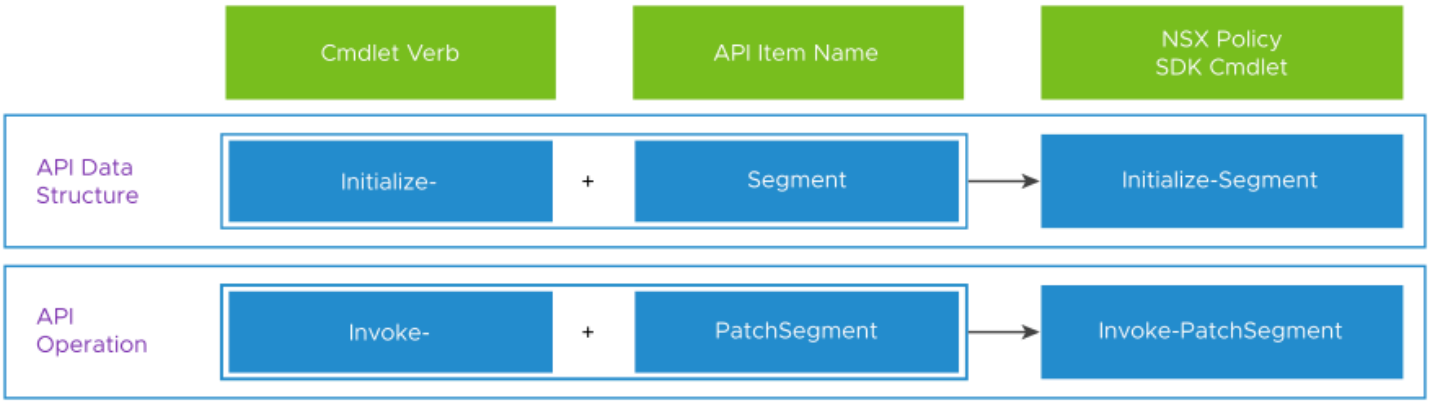
How Does the NSX Policy SDK for PowerShell Work

Starting from VMware Cloud Foundation PowerCLI 12.6, you can use the NSX Policy SDK for PowerShell to communicate with all available NSX Policy REST APIs.

The `VMware.Sdk.Nsx.Policy` module contains SDK cmdlets that are automatically generated and provide PowerShell bindings to the NSX REST APIs. The VCF PowerCLINSX module functions on a low level and communicates directly with the NSX Policy API server.

You can construct NSX Policy SDK cmdlets by using the `Initialize-` and `Invoke-` cmdlet verbs in combination with the names of the API data structures or operations.

Figure 662: Constructing NSX Policy SDK Cmdlets



- Use `Initialize-` cmdlets to prepare the data structures, if any, for your API calls. These cmdlets function on the client side and do not communicate with the API server.
- Use `Invoke-` cmdlets to call the NSX Policy API server and invoke the operations.

Navigate the NSX Policy SDK for PowerShell

You can navigate the NSX Policy SDK for PowerShell by using the `Get-NsxOperation` and `Get-Help` cmdlets.

To navigate the NSX Policy SDK for PowerShell, you must be familiar with the NSX Policy API and its reference documentation.

You can access two NSX REST API references on the VMware Developer website:

- The `NSX REST API reference` that contains all NSX networking services, for both on-prem and VMware Cloud on AWS.
- The `NSX VMC Policy API reference` that contains only the networking services for VMware Cloud on AWS. Here, you can find ready-to-use VCF PowerCLI samples that you can paste directly into your PowerShell console.

The following procedure is based on the NSX REST API reference.

1. In the NSX REST API reference, navigate to the NSX Policy API operation you want to use.

   For example, go to **Segments (Fixed)** > **Create or update a segment**.

2. Copy the method and the path of the operation.
3. In PowerShell, run `Get-NsxOperation` with the `Method` and `Path` parameters.

   For example:

   ```
   Get-NsxOperation -Method PUT -Path '/policy/api/v1/infra/tier-1s/<tier-1-id>/segments/<segment-id>'
   ```

   The command retrieves a custom object with the NSX operation details.

   ```
   Name                : CreateOrReplaceSegment
   CommandInfo         : Invoke-CreateOrReplaceSegment
   ApiName             : PolicyConnectivityApi
   Path                : /infra/tier-1s/{tier-1-id}/segments/{segment-id}
   Tags                : {Policy, Networking, Connectivity, Segments...}
   RelatedCommandInfos : {Initialize-Segment}
   Method              : PUT
   Parameters          : {tier1Id, segmentId, segment}
   ```

4. Copy the VCF PowerCLI cmdlet that corresponds to the API operation.

   It is listed in the `CommandInfo` property.

5. To retrieve information about the cmdlet , run Get-Help.

   For example:

   ```
   Get-Help Invoke-CreateOrReplaceSegment -full
   ```

   You can see the full information about the cmdlet such as its description, parameters, and code examples.

Connect to an On-Prem NSX Server

You can connect to an on-prem NSX server by using the `Connect-NsxServer` cmdlet.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

Run `Connect-NsxServer` with the NSX server IP address or FQDN and valid user credentials.

```
Connect-NsxServer -Server <nsx_server_ip_address_or_fqdn> -User <my_username> -Password
<my_password>
```

Connect to an NSX Server on VMware Cloud on AWS

You can connect to an NSX server on VMware Cloud on AWS by using the `Connect-NsxVmcServer` cmdlet.

- Verify that you have a valid API token from VMware Cloud services.
- Verify that you know the name of the SDDC, in which the server you want to connect to resides.

1. Create a variable with the VMware Cloud on AWS server connection.

   ```
   $vmcServer = Connect-VmcServer -ApiToken $apiToken
   ```

2. Create a variable with the SDDC name.

   ```
   $sddc = Get-VmcSddc -Name $sddcName
   ```

3. Connect to the NSX server on VMware Cloud on AWS.

   ```
   Connect-NsxVmcServer -VmcConnection $vmcServer -VmcSddc $sddc
   ```

Create a Tier-0 Gateway

You can create a Tier-0 gateway with the NSX Policy SDK for PowerShell by using the `Initialize-Tier0` and `Invoke-PatchTier0` cmdlets.

Verify that you are connected to an NSX server system.

1. Prepare the input for the operation.

   ```
   $t0 = Initialize-Tier0 -ArpLimit 5000 -DisplayName $T0GatewayName
   ```

2. Invoke the operation.

   ```
   Invoke-PatchTier0 -Tier0Id $T0GatewayName -Tier0 $t0
   ```

Create a Tier-1 Gateway

You can create a Tier-1 gateway with the NSX Policy SDK for PowerShell by using the `Initialize-Tier1` and `Invoke-PatchTier1` cmdlets.

- Verify that you are connected to an NSX server system.
- Verify that your system has Tier-0 gateway(s).

1. Retrieve the Tier-0 paths.

   ```
   $t0s = Invoke-ListTier0s
   $t0 = $t0s.Results | where {$_.DisplayName -eq $T0GatewayName}
   ```

2. Prepare the input for the operation.

   ```
   $t1 = Initialize-Tier1 -ArpLimit 5000 -DisplayName $T1GatewayName -Tier0Path $t0.Path
   ```

3. Invoke the operation.

   ```
   Invoke-PatchTier1 -Tier1Id $T1GatewayName -Tier1 $t1
   ```

Add an Existing Tier-1 Gateway to a Specific Edge Cluster

You can add an existing Tier-1 gateway to a specific Edge Cluster with the NSX Policy SDK for PowerShell by using the `Invoke-PatchTier1LocaleServices` cmdlet.

- Verify that you are connected to an NSX server system.

- Verify that you know the Edge Cluster name of the Edge Cluster to which you want to add the Tier-1 gateway.

1. Retrieve the Edge Cluster information.

```
$edgeClusters = Invoke-ListEdgeClustersForEnforcementPoint -SiteId default -EnforcementPointId default
$edgeCluster = $edgeClusters.Results | where {$_.Displayname -eq $edgeClusterName}
```

2. Create a client-side object for locale service with the Edge Cluster path.

```
$localeService = Initialize-LocaleServices -EdgeClusterPath $edgeCluster.Path -
DisplayName default -Id default
```

3. Invoke the operation.

```
Invoke-PatchTier1LocaleServices -Tier1Id $T1.Id -LocaleServices $localeService -LocaleServicesId default
```

Create a Segment (On-Prem)

You can create a segment on an on-prem NSX server system by using the `Invoke-PatchInfraSegment` cmdlet.

- Verify that you are connected to an on-prem NSX server system.
- Verify that your system has Tier-1 gateway(s).

To create an on-prem (flexible) segment, you must specify a transport zone and assign a subnet.

1. Retrieve transport zone details.

```
$tzs = Invoke-ListTransportZonesForEnforcementPoint -EnforcementpointId default -SiteId default
$tz = $tzs.Results | where {$_.DisplayName -eq $TransportZone}
```

2. Prepare the input for the segment subnet.

```
$segmentSubnet = Initialize-SegmentSubnet -GatewayAddress $GatewayCIDR
```

3. Retrieve the Tier-1 paths.

```
$t1s = Invoke-ListTier1
$t1 = $t1s.Results | where {$_.DisplayName -eq $T1GatewayName}
```

4. Prepare the input for the segment.

```
$segment = Initialize-Segment -DisplayName $SegmentName -TransportZonePath $tz.Path `
           -Subnets $segmentSubnet -ConnectivityPath $t1.Path
```

5. Invoke the operation.

```
Invoke-PatchInfraSegment -Segment $segment -SegmentId $SegmentName
```

Create a Segment (VMware Cloud on AWS)

You can create a segment for an NSX server system on VMware Cloud on AWS by using the `Invoke-CreateOrReplaceSegment` cmdlet.

- Verify that you are connected to an NSX server system on VMware Cloud on AWS.
- Verify that your system has Tier-1 gateway(s).

1. Prepare the input for the segment subnet.

```
$segmentSubnet = Initialize-SegmentSubnet -GatewayAddress $GatewayCIDR
```

2. Prepare the input for the segment.

```
$segment = Initialize-Segment -DisplayName $SegmentName -Subnets @($segmentSubnet)
```

3. Invoke the operation.

```
Invoke-CreateOrReplaceSegment -Tier1Id $T1GatewayName -SegmentId $SegmentName -Segment $segment
```

Create a Distributed Firewall Policy

You can create a distributed firewall policy for an NSX server system by using the `Invoke-PatchSecurityPolicyForDomain` cmdlet.

- Verify that you are connected to an NSX server system.

1. Create lookup object variables.

```
$serviceList = @("SSH", "HTTP")
$sourceGroups = @("ANY")
$destinationGroups = @("MyGroupName")
```

2. Look up groups and services.

```
$allServices = Invoke-ListServicesForTenant
$ServicePathList = @()
foreach ($serv in $serviceList) {
    $s = $allServices.Results | where {$_.DisplayName -eq $serv}
    $ServicePathList += $s.Path
}

$allGroups = Invoke-ListGroupForDomain -DomainId default
$sourceGroupList = @()
foreach ($gp in $sourceGroups) {
    if ($gp -eq "ANY") {
        $SourceGroupList += "ANY"
    } else {
        $g = $AllGroups.Results | where {$_.DisplayName -eq $gp}
        $SourceGroupList += $g.Path
    }
}

$destinationGroupList = @()
foreach ($gp in $destinationGroups) {
    if ($gp -eq "ANY") {
        $DestinationGroupList += "ANY"
    } else {
        $g = $AllGroups.Results | where {$_.DisplayName -eq $gp}
        $DestinationGroupList += $g.Path
    }
}
```

3. Prepare the input for the policy rule.

```
$rule = Initialize-Rule -DisplayName $ruleName -Id $ruleName -SourceGroups $sourceGroupList -
DestinationGroups $destinationGroupList -Services $servicePathList -Action "ALLOW"
```

4. Prepare the input for the security policy.

```
$securityPolicy = Initialize-SecurityPolicy -DisplayName $policyName -Rules @($rule)
```

5. Invoke the operation.

```
Invoke-PatchSecurityPolicyForDomain -DomainId default -SecurityPolicyId $policyName -SecurityPolicy
 $securityPolicy
```

Managing VMware Cloud Director with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of scripts that illustrate basic and advanced tasks in cloud administration.

Connect to a VMware Cloud Director Server

To run cmdlets on a VMware Cloud Director server and perform administration or monitoring tasks, you must establish a connection to the server.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see Managing Default Server Connections in VCF PowerCLI.

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see Processing Non-alphanumeric Characters in VCF PowerCLI.

Run `Connect-CIServer` with the server name and valid credentials.

```
Connect-CIServer -Server cloud.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
```

Create and Manage Organizations

Organizations provide resources to a group of users and set policies that determine how users can consume those resources. Create and manage organizations for each group of users that requires its own resources, policies, or both.

Verify that you are connected to a VMware Cloud Director server as a provider administrator.

1. Generate a customized report for all organizations on the server.

   ```
   Get-Org | Select Name, Enabled, StoredVMQuota, DeployedVMQuota
   ```

2. Add a new organization on the server and provide a name and a full name for it.

   ```
   New-Org -Name 'MyOrg1' -FullName 'My Organization 1'
   ```

   By default, the new organization is enabled. Enabling the organization lets users log in.

3. Add a description for the new organization.

   ```
   Get-Org -Name 'MyOrg1' | Set-Org -Description "This organization provides resources to John Doe."
   ```

4. Deactivate and remove the new organization.

   ```
   Get-Org -Name 'MyOrg1' | Set-Org -Enabled $false | Remove-Org
   ```

Create and Manage Organization Virtual Data Centers

To allocate resources to an organization, you need to create an organization virtual data center (vDC). When the demands of the organization change, you can modify or remove the organization vDC.

- Verify that you are connected to a VMware Cloud Director server as a provider administrator.
- Verify that at least one enabled provider vDC is available on the server.

1. Create a new organization vDC using the Pay As You Go model for resource allocation.

   ```
   $myOrg = Get-Org -Name 'MyOrg1'
   $myPVdc = Get-ProviderVdc -Name 'MyProvidervDC'
   ```

```
New-OrgVdc -Name 'MyOrgvDC' -AllocationModelPayAsYouGo -Org $myOrg -ProviderVdc $myPVdc -VMCPUCoreMHz
 1000
```

To create the organization vDC, VMware Cloud Foundation Automation Provider Management PowerCLI uses a default configuration based on the selected resource allocation model.

- VMMaxCount is set to 100
- NetworkMaxCount is set to 1024
- The vDC is automatically enabled
- Thin provisioning is deactivated
- Fast provisioning is deactivated
- NicMaxCount is set to $null (unlimited)
- MemoryGuaranteedPercent is set to 100
- CpuGuaranteedPercent is set to 0

2. Modify the vCPU speed setting for the virtual machines in the organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -VMCpuCoreMhz 2000
```

3. Enable fast provisioning for the virtual machines in the organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -UseFastProvisioning $true
```

4. Deactivate and remove the new organization vDC.

```
Get-OrgVdc -Name 'MyOrgVdc' | Set-OrgVdc -Enabled $false | Remove-OrgVdc
```

Filter and Retrieve Organization Virtual Data Center Networks

To generate reports about organization vDC networks, you need to retrieve the respective organization vDC networks. You can use search criteria to filter the results returned by `Get-OrgVdcNetwork`.

Verify that you are connected to a VMware Cloud Director server.

- Get all organization vDC networks for the organization named *`MyOrgVdc`*.

```
Get-OrgVdc -Name 'MyOrgVdc' | Get-OrgVdcNetwork
```

- Get the organization vDC network that is named *`MyOrgVdcNetwork`*.

```
Get-OrgVdc -Name 'MyOrgVdc' | Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
```

Import a vApp Template from the Local Storage

To make an OVF package from your local storage available to other cloud users, you can import the package and save it as a vApp template in a catalog.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the catalog to which you want to add the imported vApp template.

```
$myCatalog = Get-Catalog -Name 'MyCatalog'
```

2. Retrieve the organization virtual data center (vDC) to which you want to add the imported vApp template.

```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```

3. Import a virtual machine from your local storage and save it as a vApp template in the cloud.

```
Import-CIVAppTemplate -SourcePath 'C:\OVFs\WindowsXP\WindowsXP.ovf' -Name
 'MyWindowsXPVAppTemplate' -OrgVdc $myOrgVdc -Catalog $myCatalog
```

Create a vApp Template from a vApp

Creating vApp templates from vApps in the cloud might minimize future efforts for cloning vApps. You can use the templates later to create vApps that are based on the source vApp.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the source vApp for the vApp template that you want to create.
```
$myVApp = Get-CIVApp -Name 'MyVApp'
```
2. If the source vApp is running, stop it.
```
$myVApp = Stop-CIVApp -VApp $myVApp
```
3. Retrieve the catalog to which you want to add the new vApp template.
```
$myCatalog = Get-Catalog -Name 'MyCatalog'
```
4. Retrieve the organization vDC to which you want to add the new vApp template.
```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```
5. Create a vApp template.
```
New-CIVAppTemplate -Name 'MyVAppTemplate' -VApp $myVApp -OrgVdc $myOrgVDC -Catalog
$myCatalog
```
6. Start the source vApp.
```
$myVApp = Start-CIVApp -VApp $myVApp
```

Create a vApp from the template and modify the vApp. See Create and modify a vApp.

Import a vApp from vSphere

To make a virtual machine from the underlying vSphere infrastructure available to your VMware Cloud Director server, you can import it and save it as a vApp.

- Verify that you are connected to a VMware Cloud Director server as a provider administrator.
- Verify that you are connected to a vCenter system.

1. Retrieve the vSphere virtual machine that you want to import.
```
$myVm = Get-VM -Name 'MyVMToImport'
```
2. Retrieve the organization vDC to which you want to import the virtual machine.
```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```
3. Import the virtual machine and store it as a vApp.
```
Import-CIVApp -VM $myVm -OrgVdc $myOrgVdc
```

Create and Modify a vApp

You can use vApp templates to instantiate vApps. After creating the vApp, you can modify its settings to minimize the consumption of computing and storage resources.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the organization vDC to which you want to add the new vApp.
```
$myOrgVdc = Get-OrgVdc -Name 'MyOrgVdc'
```
2. Retrieve the source vApp template for your new vApp.
```
$myVAppTemplate = Get-CIVAppTemplate -Name 'MyVAppTemplate'
```
3. Create your new vApp.
```
$myVApp = New-CIVApp -Name 'MyVApp' -VAppTemplate $myVAppTemplate -OrgVdc $myOrgVDC
```

By default, the vApp is powered off.

4. Renew the runtime lease for the new vApp and set it to 12 hours.

```
Set-CIVApp -VApp $myVApp -RuntimeLease "12:0:0" -RenewLease
```

   To set leases, you can use the *days.hours:minutes:seconds* syntax.

5. Start the new vApp.

```
Start-CIVApp -VApp $myVApp
```

Manage Virtual Machines with vApps

For a large-scale approach to administration, you can start, stop, or restart virtual machines or their guest operating systems by running cmdlets on the associated vApps.

Verify that you are connected to a VMware Cloud Director server.

1. Power on all virtual machines in all vApps with names starting with *MyVApp*.

```
Get-CIVApp -Name 'MyVApp*' | Start-CIVApp
```

2. Suspend all virtual machines in all vApps with names starting with *YourVApp*.

```
Get-CIVApp -Name 'YourVApp*' | Suspend-CIVApp
```

3. Power off all virtual machines in the vApp named *MyVApp1*.

```
Get-CIVApp -Name 'MyVApp1' | Stop-CIVApp
```

4. Shut down the guest operating systems of all virtual machines in the vApp named *MyVApp2*.

```
Get-CIVApp -Name 'MyVApp2' | Stop-CIVAppGuest
```

5. Restart the guest operating systems of all virtual machines in the vApp named *MyVApp3*.

```
Get-CIVApp -Name 'MyVApp3' | Restart-CIVAppGuest
```

6. Reset all virtual machines in the vApp.

```
Get-CIVApp -Name 'MyVApp4' | Restart-CIVApp
```

Manage Virtual Machines and Their Guest Operating Systems

For a targeted approach to administration, you can use the `CIVM` and `CIVMGuest` cmdlets to handle lifecycle operations for one or more virtual machines.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve all virtual machines with names starting with *MyVM* and power them on.

```
Get-CIVM -Name 'MyVM*' | Start-CIVM
```

2. Suspend all virtual machines with names starting with *YourVM*.

```
Get-CIVM -Name 'YourVM*' | Suspend-CIVM
```

3. Power off the virtual machine named *MyVM1*.

```
Get-CIVM -Name 'MyVM1' | Stop-CIVM
```

4. Shut down the guest operating system of the virtual machine named *MyVM2*.

```
Get-CIVM -Name 'MyVM2' | Stop-CIVMGuest
```

5. Restart the guest operating system of the virtual machine named *MyVM3*.

```
Get-CIVM -Name 'MyVM3' | Restart-CIVMGuest
```

6. Reset the nonresponsive virtual machine named *MyVM4*.

```
Get-CIVM -Name 'MyVM4' | Restart-CIVM
```

Retrieve a List of the Internal and External IP Addresses of Virtual Machines in vApps

When managing vApps in the cloud, you might need to obtain information about the NIC settings of the associated virtual machines.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the organization for which you want to generate the report.

```
$myOrg = Get-Org -Name 'MyOrg'
```

2. Retrieve all vApps in the organization.

```
$vApps = Get-CIVApp -Org $myOrg
```

3. Populate an array with the information that you want to report.

```
$vAppNetworkAdapters = @()
foreach ($vApp in $vApps) {
        $vms = Get-CIVM -VApp $vApp
        foreach ($vm in $vms) {
                $networkAdapters = Get-CINetworkAdapter -VM $vm
                foreach ($networkAdapter in $networkAdapters) {
                        $vAppNicInfo = New-Object "PSCustomObject"
                        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name VAppName -Value
 $vApp.Name
                        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name VMName   -Value $vm.Name
                        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name NIC      -Value ("NIC" +
 $networkAdapter.Index)
                        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name ExternalIP -Value
 $networkAdapter.ExternalIpAddress
                        $vAppNicInfo | Add-Member -MemberType NoteProperty -Name InternalIP -Value
 $networkAdapter.IpAddress

                        $vAppNetworkAdapters += $vAppNicInfo
                }
        }
}
```

Running this script retrieves the names of the virtual machines and their associated vApp, the IDs of the NICs of the virtual machines, and external, and internal IP addresses of the NICs.

4. Display the report on the screen.

```
$vAppNetworkAdapters
```

Create and Manage Access Control Rules

By defining access control rules you can assign levels of access to separate users, user groups, or everyone in the organization. You can define access control rules for catalogs and vApps.

Verify that you are connected to a VMware Cloud Director server.

1. Create a new rule for accessing the vApp named *MyVApp*.

```
New-CIAccessControlRule -Entity 'MyVApp' -EveryoneInOrg -AccessLevel "Read"
```

   All users in the organization have read-only access to the vApp.

2. Modify the access rule for a trusted user who needs full control over *MyVApp*.

```
New-CIAccessControlRule -Entity 'MyVApp' -User "MyAdvancedUser" -AccessLevel "FullControl"
```

3. Restrict the full control access of *MyAdvancedUser* to read/write access.

```
$accessRule = Get-CIAccessControlRule -Entity 'MyVApp' -User 'MyAdvancedUser'
$accessRule | Set-CIAccessControlRule -AccessLevel "ReadWrite"
```

4. Remove the custom rule that you created earlier for *MyAdvancedUser*.

```
$accessRule | Remove-CIAccessControlRule
```

Filter and Retrieve vApp Networks

To generate reports about vApp networks, you need to retrieve the respective vApp networks. You can use search criteria to filter the results returned by `Get-CIVAppNetwork`.

Verify that you are connected to a VMware Cloud Director server.

- Get the vApp network named *MyVAppNetwork*.

```
Get-CIVAppNetwork -Name 'VAppNetwork'
```

- Get all vApp networks for the vApp named *MyVApp*.

```
Get-CIVApp -Name 'MyVApp'| Get-CIVAppNetwork
```

- Get all vApp networks of connection type direct and direct fenced.

```
Get-CIVAppNetwork -ConnectionType Direct, DirectFenced
```

- Get all direct vApp networks that connect to the organization vDC network named *MyOrgVdcNetwork*.

```
Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork' | Get-CIVAppNetwork -ConnectionType Direct
```

Create vApp Networks for a Selected vApp

To define how the virtual machines in a vApp connect to each other and access other networks, you need to create a vApp network. When creating the vApp network, you can select the settings for the network, or adopt them from an organization policy.

To address multiple networking scenarios for a vApp, you can create multiple vApp networks.

Create an Isolated vApp Network

When you do not want the virtual machines in a vApp to connect to objects outside the vApp, you must create an isolated vApp network.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the vApp for which you want to create a vApp network.

```
$myVApp = Get-CIVApp -Name 'MyVApp'
```

2. Create the new vApp network with a selected gateway and network mask.

```
New-CIVAppNetwork -VApp $myVApp -Name 'MyVAppInternalNetwork' -Routed -Gateway '192.168.2.1'
-Netmask '255.255.255.0' -ParentOrgVdcNetwork $null
```

By default, the vApp network has an enabled firewall.

Create an NAT Routed vApp Network

To provide a vApp network with DHCP, firewall, NAT, and VPN services, you must create it as an NAT routed vApp network.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the vApp for which you want to create a vApp network.
   ```
   $myVApp = Get-CIVApp -Name 'MyVApp'
   ```
2. Retrieve the organization vDC network to which you want to connect the vApp network.
   ```
   $myOrgVdcNetwork = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
   ```
3. Create the new vApp network with a gateway and network mask, defined pool of static IP addresses, and a deactivated firewall.

   ```
   New-CIVAppNetwork -VApp $myVApp -ParentOrgVdcNetwork $myOrgVdcNetwork -Name
   'MyVAppInternalNetwork' -Routed -Gateway '192.168.2.1' -Netmask '255.255.255.0' -DisableFirewall
   -StaticIPPool "192.168.2.100 - 192.168.2.199"
   ```

   If you do not run `New-CIVAppNetwork` with the `DisableFirewall` parameter, the new vApp network has an active firewall by default.

Create a Direct vApp Network

To establish a network connection between the virtual machines in a vApp and an organization network, you need to create a direct vApp network.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the vApp for which you want to create a vApp network.
   ```
   $myVApp = Get-CIVApp -Name 'MyVApp'
   ```
2. Retrieve the organization vDC network that you want to connect to.
   ```
   $myOrgVdcNetwork = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork'
   ```
3. Create a direct vApp network that connects to the selected organization vDC network.

   ```
   New-CIVAppNetwork -VApp $myVapp -Direct -ParentOrgVdcNetwork $myOrgVdcNetwork
   ```

   By default, the new vApp network has an enabled firewall.

Modify or Remove vApp Networks

Based on the type of the vApp network, you can configure various network settings, such as DNS, static IP pools, and firewalls. If you no longer need a vApp network, you can remove it.

Verify that you are connected to a VMware Cloud Director server.

1. Retrieve the vApp for which you want to modify vApp networks.
   ```
   $myVApp = Get-CIVApp -Name 'MyVApp'
   ```
2. Modify the settings for DNS and static IP pool for the vApp network named *`MyVAppNetwork`*.

   ```
   Get-CIVAppNetwork -VApp $myVApp -Name 'MyVAppNetwork' | Set-CIVAppNetwork -PrimaryDns
   10.17.0.94 -SecondaryDns 10.17.0.95 -DnsSuffix 'my.domain.com' -StaticIPPool "10.151.168.1 -
   10.151.169.240"
   ```

3. Optional: Remove *`MyVAppNetwork`*.
   ```
   $myVApp | Get-CIVAppNetwork -Name 'MyVAppNetwork' | Remove-CIVAppNetwork
   ```

4. Optional: Remove all isolated vApp networks for the vApp named *MyVapp*.
   ```
   $myVApp | Get-CIVAppNetwork -ConnectionType Isolated | Remove-CIVAppNetwork
   ```
5. Retrieve the organization vDC network named *MyOrgVdcNetwork1*.
   ```
   $myOrgVdcNetwork1 = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork1'
   ```
6. Retrieve the organization vDC network named *MyOrgVdcNetwork2*.
   ```
   $myOrVdcgNetwork2 = Get-OrgVdcNetwork -Name 'MyOrgVdcNetwork2'
   ```
7. Redirect all vApp networks that connect to *MyOrgVdcNetwork1* to connect to *MyOrgVdcNetwork2*.

   ```
   Get-CIVAppNetwork -ParentOrgVdcNetwork $myOrgVdcNetwork1 | Set-CIVAppNetwork
   -ParentOrgVdcNetwork $myOrgVdcNetwork2 -NatEnabled $false -FirewallEnabled $false
   ```

   The operation deactivates the firewall and NAT routing for all vApp networks that are connected to *MyOrgVdcNetwork1*.

Managing VMware Cloud Foundation Operations with VMware Cloud Foundation PowerCLI

To help you get started with VMware Cloud Foundation PowerCLI, this documentation provides a set of sample scripts that illustrate basic and advanced tasks in VMware Cloud Foundation Operations.

Connect to a VMware Cloud Foundation Operations Server

To run VMware Cloud Foundation Operations cmdlets, you must establish a connection to a VMware Cloud Foundation Operations server and a vCenter system that is monitored by the VMware Cloud Foundation Operations instance.

- If the certificate of the server you want to connect to is not trusted, verify that your VCF PowerCLI invalid server certificate action settings are configured properly. For more information, see Configuring VCF PowerCLI Response to Untrusted Certificates.
- If you use a proxy server for the connection, verify that it is configured properly, so that the connection is kept alive long enough for tasks to complete running.

  Note:
  If you do not want to use a proxy server for the connection, run `Set-PowerCLIConfiguration -ProxyPolicy NoProxy`.

You can have more than one connection to the same server. For more information, see Managing Default Server Connections in VCF PowerCLI.

If your login credentials contain non-alphanumeric characters, you might need to escape them. For more information, see Processing Non-alphanumeric Characters in VCF PowerCLI.

1. Run `Connect-OMServer` with the server name and valid credentials.

   ```
   Connect-OMServer -Server vrops3.example.com -User 'MyAdministratorUser' -Password
   'MyPassword'
   ```
2. Run `Connect-VIServer` with the server name and valid credentials.

   ```
   Connect-VIServer -Server vc3.example.com -User 'MyAdministratorUser' -Password 'MyPassword'
   ```

Check Memory Waste Levels

You can check the memory waste levels of a virtual machine host for a specific period of time. For example, you can check the memory waste levels in the last month.

- Verify that you are connected to a VMware Cloud Foundation Operations instance.
- Verify that you are connected to the vCenter system that is monitored by the VMware Cloud Foundation Operations instance.

1. Browse the vCenter inventory and select a virtual machine host for which you want to check the memory waste levels.

```
$vmHost = Get-VMHost 'MyHost'
```

2. Get the VMware Cloud Foundation Operations resource that refers to this virtual machine host.

```
$hostResource = $vmHost | Get-OMResource
```

3. Check the defined metrics for this VMware Cloud Foundation Operations resource type.

```
Get-OMStatKey -AdapterKind $hostResource.AdapterKind -ResourceKind $hostResource.ResourceKind
```

4. Get data for a specific metric.

```
$hostResource | Get-OMStat -Key "mem|waste"
```

Note:
This command retrieves all available metric data with the highest available granularity.

5. Get metric data for the last month aggregated on a daily basis.

```
$hostResource | Get-OMStat -Key "mem|waste" -From ([datetime]::Now.AddMonths(-1)) -IntervalType Days
 -IntervalCount 1 -RollupType Avg
```

Get Remediation Recommendations

You can get remediation recommendations for a specific resource, such as a problematic virtual machine.

- Verify that you are connected to a VMware Cloud Foundation Operations instance.
- Verify that you are connected to the vCenter system that is monitored by the VMware Cloud Foundation Operations instance.
- Verify that at least one alert is triggered for the virtual machine.

1. Get the virtual machine you want to check for alerts.

```
$myVm = Get-VM 'MyVM'
```

2. Get the associated VMware Cloud Foundation Operations resource and its associated active alerts.

```
$myVmAlerts = $myVm | Get-OMResource | Get-OMAlert -Status  Active
```

3. List the remediation recommendations for the obtained alerts.

```
$myVmAlerts | Get-OMRecommendation
```

Change Alert Ownership

You can retrieve all active alerts for a specific datastore and assign the alert ownership to your user profile.

- Verify that you are connected to a VMware Cloud Foundation Operations instance.
- Verify that you are connected to the vCenter system that is monitored by the VMware Cloud Foundation Operations instance.

1. Get all active alerts for the datastore.

```
$alerts = Get-Datastore 'shared' | Get-OMResource | Get-OMAlert -Status Active | where
 { ($_.AssignedUser -eq $null) -and ($_.ControlState -eq 'Open') }
```

2. Assign the obtained alerts to the user profile you are currently using.

```
$alerts | Set-OMAlert -TakeOwnership
```

Create a Report for Problematic Hosts

You can create a report for virtual machine hosts that have problematic health status.

- Verify that you are connected to a VMware Cloud Foundation Operations instance.

- Verify that you are connected to the vCenter system that is monitored by the VMware Cloud Foundation Operations instance.

1. Get all problematic host resources in VMware Cloud Foundation Operations that have red or yellow health status.

```
$hosts = Get-OMResource | where { $_.ResourceKind -eq 'HostSystem' -and $_.Health -in ('Red', 'Yellow') }
```

2. Get the virtual machine hosts that cause the problem.

```
$hosts | Get-VmHost
```

Managing VMware Cloud on AWS with VMware Cloud Foundation PowerCLI

To familiarize yourself with VMware Cloud Foundation PowerCLI, you can use a set of scripts that illustrate basic and advanced tasks in the VMware Cloud on AWS administration.

Connecting to VMware Cloud on AWS

You can use VMware Cloud on AWS to provision and manage vCenter virtual data centers in the cloud. VMware Cloud on AWS is a cloud service that belongs to the VMware Cloud services platform.

The VMware Cloud services platform is a web-based application that supports any number of VMware Cloud services. When you sign up for VMware Cloud services, you access your services and manage all your users, organizations, and payment methods through this platform. The VMware Cloud services platform manages common requirements for all cloud services, such as authentication, user roles, and so on.

When connecting to a VMware Cloud on AWS service, you need to authenticate with the corresponding VMware Cloud services platform.

Connect to VMware Cloud on AWS by Using an API Token

To authenticate with VMware Cloud on AWS, you can use an API token from the VMware Cloud service web portal.

Use the VMware Cloud service web portal to generate an API token for authentication with VCF PowerCLI.

- To connect to a VMware Cloud on AWS server by using the API token, run the following command.

```
Connect-Vmc -ApiToken "Your API token"
```

Connect to VMware Cloud on AWS by Using an OAuth Security Context

To connect to VMware Cloud on AWS through an OAuth 2.0 authentication mechanism, you need an OAuth security context for the VMware Cloud services platform.

Use the VMware Cloud service web portal to generate an API token for authentication with VCF PowerCLI.

You can create an OAuth security context by using the `New-VcsOAuthSecurityContext` cmdlet.

1. Create an OAuth security context for the VMware Cloud services platform by using an API token.

```
$oauthSecContext = New-VcsOAuthSecurityContext -ApiToken "Your API token"
```

2. Connect to VMware Cloud on AWS by using an OAuth security context.

```
Connect-Vmc -OAuthSecurityContext $oauthSecContext
```

View the Available Software-Defined Data Centers

With VCF PowerCLI, you can view the available Software-Defined Data Centers (SDDCs) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

View the available SDDCs in your VMware Cloud on AWS organization.

```
Get-VmcSddc
```

Connect to a vCenter on VMware Cloud on AWS

You can use PowerCLI to connect to a vCenter system running on VMware Cloud on AWS.

- Verify that you are connected to VMware Cloud on AWS.
- Configure the VMware Cloud on AWS networking to provide access to the vCenter system running on the cloud.

1. Create a variable with the SDDC you want to connect to.

   ```
   $SDDC = Get-VmcSddc'Your SDDC'
   ```

2. To connect to the vCenter system, provide the vCenter host name and credentials.

   ```
   Connect-VIServer $sddc.VCenterHostName -Credential $sddc.VCenterCredentials
   ```

Connect to a vCenter on VMware Cloud on AWS by Using an OAuth 2.0 Authentication

If the vCenter system has an OpenID authentication enabled, you can authenticate with the vCenter system by using an OAuth security context for the VMware Cloud services platform.

- Verify that you are connected to VMware Cloud on AWS.
- Configure the VMware Cloud on AWS networking to provide access to the vCenter system running on the cloud.

1. Create an OAuth security context for the VMware Cloud service by using an API token.

   ```
   $oauthSecContext = New-VcsOAuthSecurityContext -ApiToken "Your API token"
   ```

2. Create a variable with the SDDC you want to connect to.

   ```
   $SDDC = Get-VmcSddc'Your SDDC'
   ```

3. Exchange the OAuth security context for an SAML security context.

   ```
   $samlSecContext = New-VISamlSecurityContext -VCenterServer $sddc.VCenterHostName -OAuthSecurityContext
    $oauthSecContext
   ```

4. Connect to a vCenter system by using an SAML security context.

   ```
   Connect-VIServer -Server $sddc.VCenterHostName -SamlSecurityContext $samlSecContext
   ```

Create a Software-Defined Data Center

With VCF PowerCLI, you can create a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Get a VMware Cloud on AWS account.

   ```
   $awsAccount = Get-AwsAccount
   ```

2. Get the VMware Cloud on AWS Virtual Private Cloud (VPC) subnets.

   ```
   $awsVpcSubnets = Get-AwsVpcSubnet -AwsAccount $awsAccount -Region "US_EAST_1"
   ```

3. Create an SDDC.

   ```
   New-VmcSddc -Name "MySDDC" -HostCount 3 -Region "US_EAST_1" -AwsAccount $awsAccount
  -AwsVpcSubnet $awsVpcSubnets[0]
   ```

Create a Cluster in a Software-Defined Data Center

With VCF PowerCLI, you can create a cluster in a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Retrieve the SDDC in which you want to create a cluster.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create the cluster.

```
New-VmcSddcCluster -Sddc $sddc -HostCount 3
```

Set the Elastic Distributed Resource Scheduler (EDRS) Policy of a Cluster

With VCF PowerCLI, you can set the Elastic Distributed Resource Scheduler (EDRS) policy of clusters.

Verify that you are connected to VMware Cloud on AWS.

1. Create a variable with the SDDCs of the clusters you want to manage.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create a variable with the clusters you want to manage.

```
$cluster = Get-VmcSddcCluster -Name 'Your Cluster' -Sddc $sddc
```

3. Set the EDRS policy.

```
Set-VmcClusterEdrsPolicy -Cluster $cluster -PolicyType cost -MinHostCount 6 -MaxHostCount 15
```

Remove a Cluster from a Software-Defined Data Center

With VCF PowerCLI, you can remove clusters from Software-Defined Data Centers (SDDCs) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

1. Create a variable with the SDDCs from which you want to remove clusters.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Create a variable with the clusters you want to remove.

```
$cluster = Get-VmcSddcCluster -Name 'Your Cluster' -Sddc $sddc
```

3. Remove the clusters.

```
Remove-VmcSddcCluster -VmcSddcCluster $cluster
```

Add Hosts to a Software-Defined Data Center

With VCF PowerCLI, you can add hosts to a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

Add hosts to an SDDC with VCF PowerCLI. You can use the `Cluster` parameter to indicate in which cluster you want to add the hosts. If you do not specify a cluster, the hosts are added to the default cluster of the SDDC.

1. Create a variable with the SDDCs in which you want to add hosts.

```
$sddc = Get-VmcSddc 'Your SDDC'
```

2. Add the hosts to the default cluster of the specified SDDCs.

```
Add-VmcSddcHost -Sddc$sddc -HostCount2
```

Remove Hosts from a Software-Defined Data Center

With VCF PowerCLI, you can remove hosts from a Software-Defined Data Center (SDDC) in your VMware Cloud on AWS organization.

Verify that you are connected to VMware Cloud on AWS.

Remove hosts from an SDDC with VCF PowerCLI. You can use the `Cluster` parameter to indicate from which cluster you want to remove hosts. If you do not specify a cluster, the hosts are removed from the default cluster of the SDDC.

1. Create a variable with the SDDCs from which you want to remove hosts.

```
$sddc = Get-VmcSddc'Your SDDC'
```

2. Remove hosts from the default cluster of the specified SDDCs.

```
Remove-VmcSddcHost -Sddc$sddc -HostCount2
```

Generate a VCF PowerCLI Support Bundle

Generate a VCF PowerCLI support bundle and send it to Broadcom support to help them resolve your problem.

If you encounter a problem in VCF PowerCLI that you cannot resolve, you can open a support request for Broadcom.

Generate and download a VCF PowerCLI support bundle in the form of a ZIP file and attach it to your Broadcom support request.

1. Create a variable with the script that produced the error.

```
$errorcode = {
    $vm = Get-VM vCLS*
    Start-VM $vm
}
```

2. Generate and download the support bundle by running the `Get-ErrorReport` cmdlet.

```
Get-ErrorReport -ProblemScript $errorcode -Destination .\Downloads -ProblemDescription "VM would not power
 on"
```

   Optional: To limit the size of your support bundle, include the `MaxDataDepth` parameter which specifies the object data depth for the report. For example, to limit object data depth to one level, add `-MaxDataDepth1` to the preceding example.

The ZIP file that contains environmental information about the error is saved at the specified destination.

Attach or upload the ZIP file and submit your Broadcom support request.

Understanding the VMware Cloud Foundation Operations API

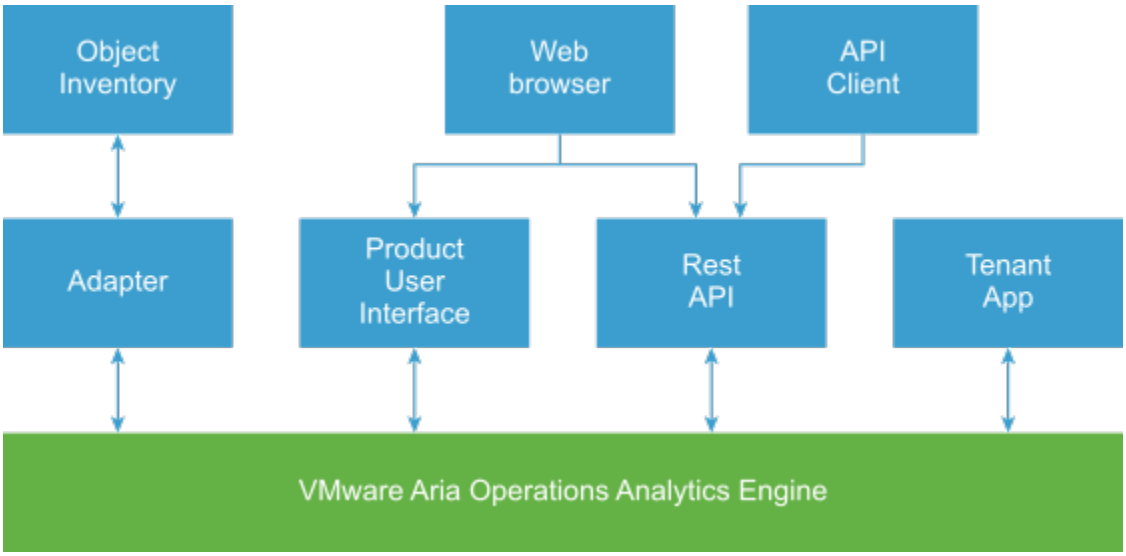
Developers can use the API to build interactive clients of VMware Cloud Foundation Operations .The API follows the REST style and is available to all licensed users.

VCF Operations clients communicate with the server over HTTP, exchanging representations of VCF Operations objects. These representations take the form of JSON or XML elements. You use HTTP GET requests to retrieve the current representation of an object, HTTP POST and PUT requests to create or modify an object, and HTTP DELETE requests to delete an object.

How the VMware Cloud Foundation Operations API Works

You use a Web browser to communicate with the VCF Operations analytics engine, either through the product user interface or through API calls.

Figure 663: VCF Operations Simplified Architecture



The adapter instance collects data from objects in your monitored environment. The VCF Operations analytics engine processes the data and displays the complete model in the graphical interface.

Why Use the VMware Cloud Foundation Operations API

The API is most useful when there is a need to automate a well-defined workflow, such as repeating the same tasks to configure access control for new users. The API is also useful when performing queries on the VCF Operations data repository, such as retrieving data for particular assets in your virtual environment. In addition, you can use the API to extract all data from the VCF Operations data repository and load it into a separate analytics system.

VMware Cloud Foundation Operations API Terminology

The XML syntax you use to describe the objects for an adapter corresponds to the API code syntax but differs from what you find in the user interface. The following terms appear in the user interface. Included with the description of each term is the corresponding XML syntax used in an API call.

Adapter types	Defines the adapter used to discover particular object types. For example, the vCenter adapter discovers objects connected to vSphere datacenters. The AWS adapter discovers AWS services and objects. XML syntax: `adapterkinds`.
Object types	The class of entities that represent objects or information sources. Objects report data to the VCF Operations analytics engine. Virtual machines, datastores, and host systems are examples of object types defined in a vCenter adapter model. XML syntax: `resourcekinds`.

Client Workflow Overview

VCF Operations API clients implement a REST workflow, making HTTP requests to the server and retrieving the information they need from the server's responses.

About REST

REST, an acronym for Representational State Transfer, describes an architectural style characteristic of programs that use the Hypertext Transfer Protocol (HTTP) to exchange serialized representations of objects between a client and a server. In the VCF Operations API, these representations are JSON or XML documents.

In a REST workflow, representations of objects are passed back and forth between a client and a server with the explicit assumption that neither party need know anything about an object other than what is presented in a single request or response. The URLs at which these documents are available often persist beyond the lifetime of the request or response that includes them.

REST API Workflows

Application programs written to use a REST API use HTTP requests that are often executed by a script or other higher-level language to make remote procedure calls that create, retrieve, update, or delete objects that the API defines. In the VCF Operations REST API, these objects are defined by a collection of XML schemas. The operations themselves are HTTP requests, and so are generic to all HTTP clients.

To write a REST API client application, you must understand only the HTTP protocol, and the semantics of JSON or XML, the transfer format that the VCF Operations API uses. To use the API effectively in such a client, you must become familiar with the following concepts.

- The set of objects that the API supports, and what they represent.
- How the API represents these objects.
- How a client refers to an object on which it wants to operate.

The API reference includes a complete list of API requests. See Accessing Swagger Documentation for Schema ReferenceAbout the VCF Operations API Examples.

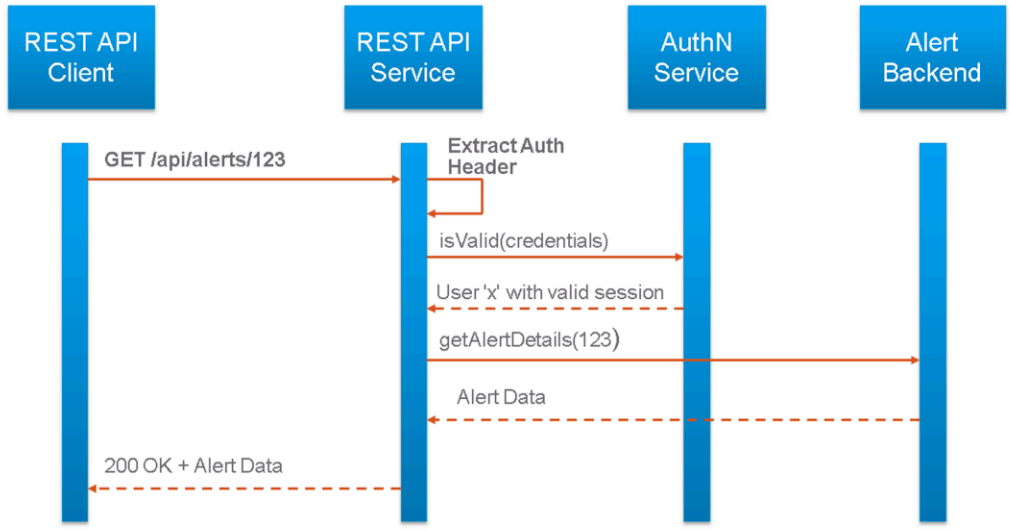
VMware Cloud Foundation Operations API REST Requests

To retrieve object representations, clients make HTTP requests to object references.

Security

The HTTP link between an API client and server is established using SSL. API clients configure token-based authentication to communicate with the server.

Figure 664: Scenario: Provide user credentials to obtain details about alert with ID 123



With token-based authentication, you POST a login request to the VCF Operations API server, supplying valid user credentials to obtain an authentication token. The following example presents a token-based authentication scenario.

1. You obtain valid user credentials for your VCF Operations instance. This is applicable for non-SSO authenticaton.
2. POST a request to the REST endpoint for authentication.
   ```
   https://RESTendpoint.example.com/suite-api/api/auth/token/acquire
   ```
   The request body includes the user name, password, and authentication source.
3. In the response body, the endpoint returns the token, expiry date, and time.
4. For further communication, you include the token object in the Authorization header with the format :
   ```
   Authorization: OpsToken <vROps_token>
   ```
   Note:
   The old format,
   ```
   Authorization: vRealizeOpsToken <vROps_token>
   ```
   continues to be supported in VCF Operations.

   If you acquired the token externally from an SSO source (without using `/suite-api/api/auth/token/acquire API`), the Authorization header is of the format:
   ```
   Authorization: SSO2Token <SSO_SAML_TOKEN>
   ```
5. You can invalidate the token before the expiration date and time by sending a POST request to the logout endpoint.
   ```
   POST https://RESTendpoint.example.com/suite-api/api/auth/token/release
   ```

Request Headers

The following HTTP headers are typically included in API requests:

Accept-Language	To specify the language desired in responses, use the `Accept-Language` request header. Message strings in `ErrorType` responses are localized. To request a response with message strings localized to French, use the following header: `Accept-Language: fr-FR`
Authorization	All requests to create an API session must include an `Authorization` header of the form prescribed by the identity provider that your organization uses
Content-Type	Requests that include a body must include an appropriate HTTP `Content-Type` header. • For a request body in XML, the header must include `Content-Type: application/xml` • For a request body in JSON, the header must include `Content-Type: application/json`
Accept	To specify the desired response format , include the `Accept` request header. • For a response in XML, the header must include `Accept: application/xml` • For a response in JSON, the header must include `Accept: application/json`

Request Bodies in XML

For a request body written in XML, VCF Operations uses a validating XML parser that requires elements in a request body to agree with the schema in order and number. Request bodies are rejected as invalid unless they meet the following criteria:

- XML namespace attributes must be supplied for all namespaces represented by elements in the request.
- If multiple namespaces are represented in the request, XML namespace attributes must include an identifying prefix, and that prefix must be used with all elements from that namespace.
- All required elements must appear in request bodies. All elements that appear in request bodies must appear in the order that the schema establishes, and with content that conforms to the type constraint that the schema specifies.

VCF Operations API REST Responses

All responses include an HTTP status code and, unless the status code is 204 (No Content), an Accept header. Response content depends on the request. Some responses include a document body, some include only a URL, and some are empty.

HTTP Response Codes

An API client can expect a subset of HTTP status codes in a response.

Table 1935: HTTP Status Codes that the API Returns

Status Code	Status Description
`200 OK`	The request is valid and was completed. The response includes a document body.
`201 Created`	The request is valid. The requested object was created and can be found at the URL specified in the Location header.
`202 Accepted`	The request is valid and a task was created to handle it. This response is usually accompanied by a `TaskStatus` element .
`204 No Content`	The request is valid and was completed. The response does not include a body.
`400 Bad Request`	The request body is malformed, incomplete, or otherwise invalid.
`401 Unauthorized`	Login failed or authentication token has expired.
`403 Forbidden`	The user is not authenticated or does not have adequate privileges to access one or more objects specified in the request.
`404 Not Found`	The object specified in the request could not be found.
`405 Method Not Allowed`	The HTTP method specified in the request is not supported for this object.
`406 Not Acceptable`	The resource identified by the request is not capable of generating a response of the type specified in the request's `Accept` header.
`415 Unsupported Media Type`	The resource identified by the request does not support a request of the specified `Content-Type` and HTTP method.
`422 Not Found`	Usually indicates a malformed request URL or request body.
`429 Too Many Requests`	A client has sent too many requests or multiple clients are sending too many simultaneous requests and the server is unable to process them due to rate limits. To work around the problem, try sending the request again later.
`500 Internal Server Error`	The request was received but could not be completed because of an internal error on the server.
`503 Service Unavailable`	The server is currently unable to handle the request due to a temporary condition such as resource exhaustion or server maintenance.
`504 Gateway Timeout`	The server, while acting as a gateway or proxy, did not receive a timely response from the upstream server specified by the request URL.

Using the API with VCF Operations

You can use a browser or an HTTP client program to send requests and receive responses.

REST Client Programs

Any client application that can send HTTPS requests is an appropriate tool for developing REST applications with the VCF Operations API. REST client plug-ins are available for most browsers and many IDEs. The following open-source programs are commonly used:

- cURL. http://curl.haxx.se
- Postman application. http://www.getpostman.com

In addition, VMware provides language-specific client bindings for the VCF Operations API. See Accessing Swagger Documentation for Schema ReferenceAbout the VCF Operations API Examples.

Accessing Swagger Documentation for Schema Reference

The VCF Operations REST API documentation includes reference material for all elements, types, queries, and operations in the VCF Operations API. It also includes the schema definition files.

Swagger based API documentation is available with the product, with the capability of making REST API calls right from the landing page.

To access the API documentation, you must first log into VCF Operations at the URL of your VCF Operations instance.

For example, if the URL of your instance is `https://operations.example.com`, the API reference is available from: `https://operations.example.com/suite-api/doc/swagger-ui.html`

Language-specific client bindings are available from: `https://operations.example.com/suite-api/`

About the VCF Operations API Examples

All examples include HTTP requests and responses. These examples show the workflow and content associated with operations such as creating and querying for information about objects in your monitored environment.

Example request bodies are in JSON. Request headers required by the VCF Operations API are included in example requests that are not fragments of a larger example.

Most example responses show only those elements and attributes that are relevant to the operation being discussed. Ellipses (...) indicate omitted content within response bodies.

Getting Started with the API

API clients and VCF Operations servers communicate over HTTPS, exchanging XML representations of API objects.

This simple example of a REST API workflow shows how to obtain a list of all metrics for a virtual machine object type that is included in the model definition of the vCenter adapter. Using the API, you can obtain the complete list of available metrics for any object type.

Acquire an Authentication Token

VCF Operations requires API requests to be authenticated. The first step in this workflow is to obtain an authentication token.

- Secure a channel between the web browser and the VCF Operations server. Open a browser and enter the URL of a VCF Operations instance such as:

  ```
  https://vrealize.example.com
  ```

  The system warns that your connection is not private. Click through to confirm the security exception and establish an SSL handshake.
- Verify that you can access the APIs. Enter the URL of your VCF Operations instance with `suite-api/docs/rest/index.html` added to the end, such as:

```
https://vrealize.example.com/suite-api/docs/rest/index.html
```

- Verify that you have the login credentials for a user of your VCF Operations instance.

To obtain an authentication token, the login request supplies the user credentials in a form that Basic HTTP authentication requires. In this example, the user is logging in to a VCF Operations instance with URL `https://vrealize.example.com/`.

> **Note:**
> This example uses token-based authentication. For more information regarding authentication mechanisms, see Security.

Using `authSource`, you can import and authenticate users and user group information that reside on another machine. For example, you can authenticate users from LDAP, Active Directory, VMware Identity Manager, Single Sign-On and so on. When you import user account information that resides on another machine, you must define the criteria used to import the user accounts from the source machine.

After creating an auth source you can use it for acquiring a token by specifying the name. The default auth source type is LOCAL.

1. POST a request to the login URL to acquire a token.

   See Login Request and Response.

   1. You obtain valid user credentials for your VCF Operations instance.
   2. POST a request to the REST endpoint for authentication (non-SSO).

      ```
      https://RESTendpoint.example.com/suite-api/api/auth/token/acquire
      ```

      The request body includes the user name, password, and authentication source.
   3. In the response body, the endpoint returns the token, expiry date, and time.
   4. For further communication, you include the token object in the Authorization header with the format :

      ```
      Authorization: OpsToken <vROps_token>
      ```

      > **Note:**
      > The old format,
      >
      > ```
      > Authorization: vRealizeOpsToken <vROps_token>
      > ```
      >
      > continues to be supported in VCF Operations.

      If you acquired the token externally from an SSO source (without using `/suite-api/api/auth/token/acquire API`), the Authorization header is of the format:

      ```
      Authorization: SSO2Token <SSO_SAML_TOKEN>
      ```

2. Examine the response.

   A successful request returns an `ops` authorization token, which you must included in subsequent API requests.

   Login Request and Response

   This example shows a request and response for a user with the login username: `vRealize-user` and password: `vRealize-dummy-password`.

   Request header:

   ```
   POST https://vrealize.example.com/suite-api/api/auth/token/acquire
   Content-Type: application/json
   Accept: application/json
   ```

   Request body in JSON format:

   ```
   {
     "username" : "vRealize-user",
     "password" : "vRealize-dummy-password"
   }
   ```

   Response in JSON:

   ```
   200 OK
   {
     "token": "8f868cca-27cc-43d6-a838-c5467e73ec45::77cea9b2-1e87-490e-b626-e878beeaa23b",
   ```

```
  "validity": 1470421325035,
  "expiresAt": "Friday, August 5, 2016 6:22:05 PM UTC",
  "roles": []
}
```

The response code indicates whether the request succeeded, or how it failed.

- If the request is successful, the server return HTTP response code 200 (OK) and re-usable `ops` authorization token that expires after six hours. This token must be included in each subsequent API request.
- If the authorization header is missing for the request, the server returns HTTP response code 403.
- If the credentials supplied in the Authorization header are invalid, the server returns HTTP response code 401.

Find the Adapter Type and Object Type

Your VCF Operations instance includes multiple adapter types. To find the adapter type for the vCenter adapter, you make a GET request to retrieve a list of all adapter types. The API response includes all the object types that the adapter monitors.

Verify that you are logged in to the VCF Operations instance.

1. Make a GET request for all adapter types.

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds
   ```

2. Examine the response to find the vCenter adapter and list of monitored object types.

   See the response portion of Determine the Adapter Type and Object Types for the vCenter Adapter.

   Determine the Adapter Type and Object Types for the vCenter Adapter

   This example finds the adapter type for the vCenter adapter and all the object types included in the adapter model definition.

   Request header:

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds
   Content-Type: application/json
   Authorization: OpsToken <vROps_token>
   Accept: application/json
   ```

   Note:
   The old format,

   ```
   Authorization: vRealizeOpsToken <vROps_token>
   ```

   continues to be supported in VCF Operations.

   Where vROps_token is the token that you obtained from the response in Acquire an Authentication Token.

   Snippet of the response in JSON for the vCenter Adapter:

   ```
   200 OK
   {
     "key": "VMWARE",
     "name": "vCenter Adapter",
     "description": "Provides the connection information and credentials required...",
     "adapterKindType": "GENERAL",
     "describeVersion": 573,
     "identifiers": [],
     "resourceKinds": [
       "ClusterComputeResource",
       "ComputeResource",
       "CustomDatacenter",
       "Datacenter",
   ```

```
        "Datastore",
        "StoragePod",
        "DatastoreFolder",
        "VM Entity Status",
        "Folder",
        "HostFolder",
        "HostSystem",
        "NetworkFolder",
        "ResourcePool",
        "VMwareAdapter Instance",
        "VirtualMachine",
        "VMFolder",
        "DistributedVirtualPortgroup",
        "VmwareDistributedVirtualSwitch",
        "vSphere World"
      ],
      ...
    }
```

For the vCenter adapter, the `adapter-kind` key is `VMWARE`. The `resourceKinds` are the object types that the vCenter adapter monitors. For virtual machine object type, the `resourceKinds` is `VirtualMachine`.

Generate a List of All Metrics for the Object

To generate a complete list of metrics for any virtual machine defined in the vCenter adapter model, you make a GET request to the URL with the adapter type and the object type.

Verify that the following requirements are met:

- You are logged in to the VCF Operations instance.
- You know the `adapterKind` value for the vCenter adapter and the `resourceKinds` value for the virtual machine. See Determine the Adapter Type and Object Types for the vCenter Adapter.

1. Make a GET request to obtain the metadata for metrics.

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds/VirtualMachine/statkeys
   ```

2. Compare the metrics listed in the response to metrics displayed in the user interface. See Virtual Machine Metrics from the API and in the User Interface.

   Virtual Machine Metrics from the API and in the User Interface

   This example shows how the virtual machine metrics listed in the XML response compare to the metrics displayed in the VCF Operations user interface.

   Request:

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds/VirtualMachine/
   statkeys
   Content-Type: application/json
   Authorization: OpsToken <vROps_token>
   Accept: application/json
   ```

   Note:
   The old format,
   `Authorization: vRealizeOpsToken <vROps_token>`
   continues to be supported in VCF Operations.

   Where:

   - `VMWARE` is the `adapterKindKey`.

- `VirtualMachine` is the `resourceKindKey`.
- *`vROps_token`* is the token that you obtained from the response in Acquire an Authentication Token.

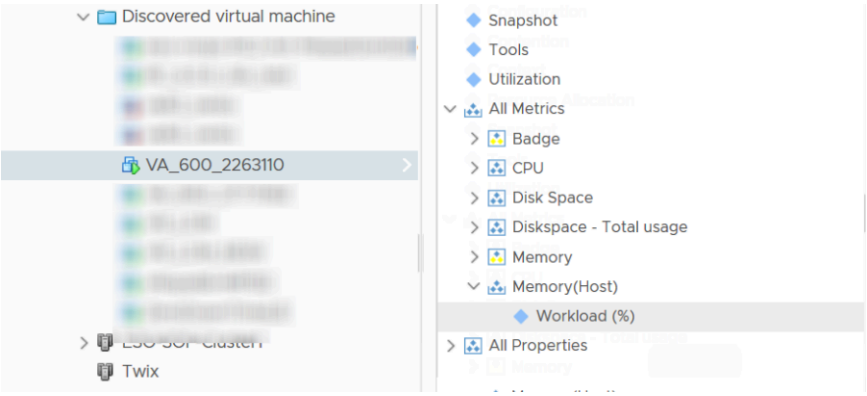
Snippet of the response in JSON:

```
200 OK

{
    "resourceTypeAttributes": [
        ...
        {
            "key": "mem|host_workload",
            "name": "Memory|Host Workload",
            "description": "Host Workload (%)",
            "defaultMonitored": false,
            "rollupType": "AVG",
            "instanceType": "INSTANCED",
            "unit": "%",
            "dataType2": "FLOAT",
            "monitoring": false,
            "property": false
        },
        ...
    ]
}
```

Every `resourceTypeAttribute` in the response is a metric with metadata for a virtual machine object. The `name` corresponds to text displayed in the VCF Operations user interface. In this example, the snippet lists metrics for Memory and Host Workload.

To compare metrics in the response with metrics in the user interface, log in to the VCF Operations interface and navigate to the metrics for a virtual machine. The following example shows where you find metrics for Memory(Host) and Workload.



The example shows how to retrieve metrics for the virtual machine object type. To retrieve metrics for other object types, replace `VirtualMachine` in the GET request with other `resourceKinds`.

Configuring an Adapter Instance

After installing a solution that includes a management pack with an adapter, you must configure an adapter instance to collect data from the objects in the adapter model definition. You can use the VCF Operations API to configure an adapter instance.

This use case example shows how to configure an adapter instance for a vSphere Solution and includes:

- summary of operations with request, request body, and response for each
- specific procedure for each operation

Summary of Configuring an Adapter Instance Requests

You make sequential API requests to configure an adapter instance. Responses from earlier requests yield information required for a subsequent requests.

Table 1936: Summary of Requests

Operation	Request	Request Body	Response
Get all solutions registered with the product and identify the adapter types	GET <*API-URL*>/suite-api/api/solutions	None	`adapterkindkeys`
Get all the object types for a particular adapter type.	GET <*API-URL*>/suite-api/api/adapterkinds/{key}/resourcekinds	None	`resourceIdentifierTypes`
Create an adapter instance object	POST <*API-URL*>/suite-api/api/adapters	Values for `resourceIdentifiers` and `credential`	`adapterid`
Patch an adapter instance to acknowledge the presented certificate	PATCH <*API-URL*>/suite-api/api/adapters	API response of POST <*API-URL*>/suite-api/api/adapters	API response of POST <*API-URL*>/suite-api/api/adapters without `adapter-cerfiticates`
Start adapter monitoring	PUT <*API-URL*>/suite-api/api/adapters/{adapterid}/monitoringstate/start	None	200 OK

Identify the Solution and Its Adapters

Your VCF Operations instance may have several solutions installed. To find the vSphere solution and its adapter types, you make a GET request to retrieve a list of all solutions. The response includes all the adapters included with the solution.

Verify that you can log in to the API URL for a VCF Operations nstance. See Acquire an Authentication Token.

For this example use case, the *API-URL* for the VCF Operations instance is operations.example.com.

1. Make a GET request to list all the solutions.

   ```
   GET https://operations.example.com/suite-api/api/solutions
   ```

2. Examine the response to find the vSphere solution and its adapter types.

   See the response portion of Adapter Types for the vSphere Solution.

   Adapter Types for the vSphere Solution

   This example lists all the installed solutions and the adapter types for each.

   Request header:

   ```
   GET https://operations.example.com/suite-api/api/solutions
   ```

   The response in JSON:

   ```
   200 OK
   {
     "solution":
       {
         "id": "MPforLogInsight",
         "name": "VMware vRealize Operations Management Pack for Log Insight",
         "version": "6.0.3171089",
         "description": "VMware vRealize Operations Management Pack for Log Insight... ",
         "vendor": "VMware Inc.",
         "adapterKindKeys": [
           "LogInsightAdapter"
   ```

```
        ]
      },
      {
        "id": "ep-ops-os-and-availability",
        "name": "Operating Systems / Remote Service Monitoring",
        "version": "1.0.4071095",
        "description": "The End Point Operations Management Solution for Operating... ",
        "vendor": "VMware Inc.",
        "adapterKindKeys": [
          "ep-ops-os-and-availability-kind"
        ]
      },
      {
        "id": "VMware vSphere",
        "name": "VMware vSphere",
        "version": "6.0.7496664",
        "description": "Manages vSphere objects such as Clusters, Hosts...",
        "vendor": "VMware Inc.",
        "adapterKindKeys": [
          "VMWARE",
          "PythonRemediationVcenterAdapter"
        ]
      }
    ]
  }
```


The response shows three solutions installed:

- Management Pack for Log Insight solution
- End Point Operations solution
- vSphere solution

The vSphere solution has two adapter types:

- VMWARE
- PythonRemediationVcenterAdapter

For the vCenter adapter, the adapter type is VMWARE.

Identify the Object Types Required for the Adapter

After you determine that you want to create an instance of the vCenter adapter, you must identify the required object types for that adapter. You make a GET request to retrieve a list of all object types for the vCenter adapter.

Verify that you know the adapter type for the vCenter adapter.

1. Make a GET request to list all the object types for the vCenter adapter.

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds
   ```

2. Examine the response to identify the required object types..

   See the response portion of Object Types Required for the vCenter Adapter.

   Object Types Required for the vCenter Adapter

   This example finds all the object types for the vCenter adapter.

   Request header:

   ```
   GET https://operations.example.com/suite-api/api/adapterkinds/VMWARE/resourcekinds
   ```

   Snippet of the response in JSON:

   ```
   200 OK
   {
   ```

```
    "key": "VMwareAdapter Instance",
    "name": "vCenter",
    "adapterKind": "VMWARE",
    "resourceKindType": "ADAPTER_INSTANCE",
    "resourceKindSubType": "NONE",
    "resourceIdentifierTypes": [
      {
        "name": "AUTODISCOVERY",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      {
        "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      {
        "name": "DV_PORT_GROUP_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      {
        "name": "DVS_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      {
        "name": "PROCESSCHANGEEVENTS",
        "dataType": "STRING",
        "isPartOfUniqueness": false
      },
      {
        "name": "VCURL",
        "dataType": "STRING",
        "isPartOfUniqueness": true
      },
      ...
      {
        "name": "VM_LIMIT",
        "dataType": "INTEGER",
        "isPartOfUniqueness": false
      }
      ],
    ...
  }
```

This snippet shows the Resource Kind with the attribute `"resourceKindType": "ADAPTER_INSTANCE"`. Any object type that has the resource identifier `"isPartOfUniqueness":true` requires a value for that object type with the API request to create the adapter instance.

An adapter instance of the vCenter adapter requires a value for `VCURL` or the URL of the vCenter.

Create the Adapter Instance

After you identify the object types required for the adapter, you provide parameter values for the object types to create an adapter instance. Your POST request includes a request body with the required parameters.

Verify that you have an IP address and credentials for a vCenter.

To create an adapter instance, the VCURL setting is mandatory.

1. Make a POST request to create the adapter instance.

   ```
   POST https://operations.example.com/suite-api/api/adapters
   ```

2. Examine the response to find the name for the vSphere Solution and its adapter types.
   See the response portion of Adapter Instance.

   Adapter Instance

   This example creates the adapter instance for a vCenter with the following parameters:

   - Display Name: VC Adapter Instance
   - Description: A vCenter Adapter Instance for VC 12.345.678.9
   - vCenter IP address: https://12.345.678.9
   - Credential name: VC-Credential-1
   - User Name: administrator@vsphere.local
   - Password: VC-dummy-passwd

   `AUTODISCOVERY` and `PROCESSCHANGEEVENTS` are optional, but are included to show additional examples of resource identifiers in the request body and in the response.

   Request header:

   ```
   POST https://operations.example.com/suite-api/api/adapters
   ```

   Request body in JSON format:

   ```
   {
     "name" : "VC Adapter Instance",
     "description" : "A vCenter Adapter Instance for VC 12.345.678.9",
     "collectorId" : "1",
     "adapterKindKey" : "VMWARE",
     "resourceIdentifiers" : [
       {
         "name" : "AUTODISCOVERY",
         "value" : "true"
       },
       {
         "name" : "PROCESSCHANGEEVENTS",
         "value" : "true"
       },
       {
         "name" : "VCURL",
         "value" : "https://12.345.678.9"
       }
     ],
     "credential" : {
       "id" : null,
       "name" : "VC-Credential-1",
       "adapterKindKey" : "VMWARE",
       "credentialKindKey" : "PRINCIPALCREDENTIAL",
       "fields" : [
         {
           "name" : "USER",
           "value" : "administrator@vsphere.local"
         },
         {
           "name" : "PASSWORD",
           "value" : "VC-dummy-passwd"
         }
   ```

```
    ],
  },
}
```

Snippet of the response in JSON:

```
201 Created
{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DV_PORT_GROUP_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
        "name": "DVS_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "PROCESSCHANGEEVENTS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "VCURL",
          "dataType": "STRING",
          "isPartOfUniqueness": true
```

```
        },
        "value": "https://12.345.678.9"
      },
      {
        "identifierType": {
          "name": "VM_FOLDER_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "VM_LIMIT",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      }
    ]
  },
  "description": "A vCenter Adapter Instance for VC 12.345.678.9",
  "collectorId": 1,
  "collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
  "credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
  "monitoringInterval": 5,
  "adapter-certificates": [
    {
      "thumbprint": "2520fb4351bc91ee7b82ef7cc54a8d88fa893da9",
      "certificateDetails": "[
        Version: V3 Subject: C=US, CN=12.345.678.9
        Signature Algorithm: SHA256withRSA, OID = 1.2.840.113549.1.1.11
        Key: Sun RSA public key, 2048 bits modulus: ...
        Validity: [From: Wed Jul 15 19:26:51 UTC 2015, To: Tue Jul 08 11:26:30 UTC 2025]
        Issuer: O=W12R2UINanduVC, C=US, DC=local, DC=vsphere, CN=CA ...
          ...
      ]"
    }
  ],
  ...
  "id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
}
```

The API creates a new adapter with an internally generated UUID that uniquely identifies the object. The API response includes the certificates that vCenter 12.345.678.9 presents. The value of the adapter instance ID is used to start monitoring and collecting data.

Provide Proof of Certificate Validity

Before VCF Operations can connect to the vCenter and start collecting data, it needs to verify that data sources discovered by the adapter instance are presenting valid certificates. Your PATCH request provides the proof that the certificates are valid by including a request body that is the response from the POST request used to create the adapter.

Verify that you have the response from the POST request used to create the adapter. See the response in Adapter Instance.

Make a PATCH request to notify the system that the user has accepted the certificate presented by the vCenter.

```
PATCH https://operations.example.com/suite-api/api/adapters
```

Certificate Validation

In this example, the request body for the PATCH request is the same as the response from the POST request used to create the adapter instance.

Request header:

```
PATCH https://operations.example.com/suite-api/api/adapters
```

Request body in JSON format:

```
{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "DV_PORT_GROUP_DISABLED",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
        "name": "DVS_DISABLED",
        "dataType": "STRING",
        "isPartOfUniqueness": false
        },
        "value": ""
      },
      {
        "identifierType": {
          "name": "PROCESSCHANGEEVENTS",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
      },
      {
        "identifierType": {
          "name": "VCURL",
          "dataType": "STRING",
```

```
            "isPartOfUniqueness": true
          },
          "value": "https://12.345.678.9"
        },
        {
          "identifierType": {
            "name": "VM_FOLDER_DISABLED",
            "dataType": "STRING",
            "isPartOfUniqueness": false
          },
          "value": ""
        },
        {
          "identifierType": {
            "name": "VM_LIMIT",
            "dataType": "STRING",
            "isPartOfUniqueness": false
          },
          "value": ""
        }
      ]
    },
    "description": "A vCenter Adapter Instance for VC 12.345.678.9",
    "collectorId": 1,
    "collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
    "credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
    "monitoringInterval": 5,
    "adapter-certificates": [
      {
        "thumbprint": "2520fb4351bc91ee7b82ef7cc54a8d88fa893da9",
        "certificateDetails": "[
          Version: V3 Subject: C=US, CN=12.345.678.9
          Signature Algorithm: SHA256withRSA, OID = 1.2.840.113549.1.1.11
          Key: Sun RSA public key, 2048 bits modulus: ...
          Validity: [From: Wed Jul 15 19:26:51 UTC 2015, To: Tue Jul 08 11:26:30 UTC 2025]
          Issuer: O=W12R2UINanduVC, C=US, DC=local, DC=vsphere, CN=CA ...
            ...
        ]"
      }
    ],
    ...
    "id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
}
```


Response in JSON:


```
{
  "resourceKey": {
    "name": "VC Adapter Instance",
    "adapterKindKey": "VMWARE",
    "resourceKindKey": "VMwareAdapter Instance",
    "resourceIdentifiers": [
      {
        "identifierType": {
          "name": "AUTODISCOVERY",
          "dataType": "STRING",
          "isPartOfUniqueness": false
        },
        "value": "true"
```

```
},
{
  "identifierType": {
    "name": "DISABLE_COMPUTATION_BASED_ON_CONSUMERS",
    "dataType": "STRING",
    "isPartOfUniqueness": false
  },
  "value": ""
},
{
  "identifierType": {
    "name": "DV_PORT_GROUP_DISABLED",
    "dataType": "STRING",
    "isPartOfUniqueness": false
  },
  "value": ""
},
{
  "identifierType": {
  "name": "DVS_DISABLED",
  "dataType": "STRING",
  "isPartOfUniqueness": false
  },
  "value": ""
},
{
  "identifierType": {
    "name": "PROCESSCHANGEEVENTS",
    "dataType": "STRING",
    "isPartOfUniqueness": false
  },
  "value": "true"
},
{
  "identifierType": {
    "name": "VCURL",
    "dataType": "STRING",
    "isPartOfUniqueness": true
  },
  "value": "https://12.345.678.9"
},
{
  "identifierType": {
    "name": "VM_FOLDER_DISABLED",
    "dataType": "STRING",
    "isPartOfUniqueness": false
  },
  "value": ""
},
{
  "identifierType": {
    "name": "VM_LIMIT",
    "dataType": "STRING",
    "isPartOfUniqueness": false
  },
  "value": ""
}
```

```
        ]
      },
      "description": "A vCenter Adapter Instance for VC 12.345.678.9",
      "collectorId": 1,
      "collectorGroupId": "909c2fbf-2c2c-4957-9a75-21bf2a887d31",
      "credentialInstanceId": "65081a8d-d462-43b2-b4e0-596eaf3d497e",
      "monitoringInterval": 5,
      ...
      "id": "a97bd204-e3e5-404b-a219-e2b20cf158d2"
    }
```

The response is same as the request body without the `adapter-certificates` section.

Start Monitoring the New Adapter Instance

After the creating the adapter instance and configuring VCF Operations to recognize a valid certificate, start monitoring and collecting data. Your PUT request provides the UUID of the adapter instance used to discover new objects.

Verify that you have the UUID of the newly created adapter instance. See the response in Adapter Instance.

Make a PUT request to start monitoring with the new adapter instance.

```
PUT https://operations.example.com/suite-api/api/adapters/<adapter_UUID>/monitoringstate/start
```

Discover Objects and Collect Data

This example starts the adapter monitoring process using the adapter instance ID from the PUT request that created the adapter instance.

Request header:

```
PUT https://operations.example.com/suite-api/api/adapters/a97bd204-e3e5-404b-a219-e2b20cf158d2/
monitoringstate/start
```

VCF Automation and APIs

Service providers can perform many VCF Automation tasks using an extensive API based on REST calls. After the provider administrator creates a tenant organization, organization administrators and DevOps programmers can automate many tasks using REST calls or Kubernetes commands.

Users who sign into the VCF Automation appliance have a choice of two interfaces, both at the same fully qualified domain name (FQDN).

- Provider UI at `https://FQDN/provider` for the service provider administrator
- Automation UI at `https://FQDN/automation` for tenant organization users

Users can switch from one to the other by clicking Change Organization. From the automation UI, specifying the **system** organization switches to the provider UI. From the provider UI, specifying a valid tenant organization switches to the automation UI.

The service provider administrator sets up regions, organizations, first users, IP spaces, provider gateway, and other networking resources for tenant organizations. The first user of a tenant organization can then act as administrator to create other users, projects, namespaces, policies, and content libraries.

Rather than employ the user interface to set up everything, certain frequently performed tasks can be automated with REST calls or Kubernetes commands.

About VCF Automation Applications

For customers unfamiliar with VMware automation products, this topic offers an introduction to various use cases and configurations of VCF Automation.

VMware Cloud Foundation (VCF) helps customers build a software-defined data center including compute, storage, network, management, and automation of private or hybrid clouds. Based on this infrastructure foundation, VCF

Automation provides multi-cloud infrastructure to configure and automate deployment of traditional and modern applications, with best practices to enforce cost, security, and performance.

Formerly called Aria Automation, VCF Automation for VM Apps is only slightly changed and remains available for all VCF customers. Also, VCF 9.0 introduced VCF Automation for All Apps, with additional support for so-called modern applications (containers and custom resources) and continued support for traditional applications based on virtual machines.

VCF Automation for All Apps provides strong isolation for tenant applications, while VCF Automation for VM Apps implies weak isolation sufficient for guest applications. Many components are isolated from each other to simplify upgrades.

The All Apps model addresses environments where mutual user distrust demands stricter security and minimal resource sharing to prevent data exfiltration and denial of service. The VM Apps model suits applications running in virtual machines, which offer inherent isolation, but is ineffective for microservices running in Kubernetes containers.

REST calls for Service Provider Administration

REST calls for service providers are grouped by similarity into the following categories. Compared to VCF Automation for VM Apps, new categories were established for Instances and Custom Resource Types & Actions.

For more information, see the VCF Automation for All Apps APIs reference documentation.

Category	Description
Access Control	Identity operations for enabling groups, users, and roles.
Aggregator	Approvals, deployment limits, and work items.
Approvals	Enforce policies controlling who must agree to a deployment.
Blueprint	Create, validate, and provision blueprints.
Catalog	Access items and catalog sources for content sharing.
Content Gateway	Manage and synchronize external content sources.
Custom Forms	Define dynamic form rendering in automation services.
Custom Resource Types & Actions	Define form schemas for custom resouces and actions.
Instances	Access instances and platforms or deployed blueprints.
Orchestrator Gateway	Run workflows and actions to automate complex tasks.
Policies	Interact with policies created in the Catalog.
Projects	Provide visibility and isolation of provisioned resources.
Provisioning Service	Enable integration and centralize configuration properties.

Missing categories that are available with VCF Automation for VM Apps are: ABX (action based extensibility), Deployment, Deployment Metrics, Identity, and Onboarding.

After you sign into the VCF Automation user interface, you can browse available REST calls by clicking **admin > API Help Center > Automation APIs** in the upper right corner.

Kubernetes and REST Calls for Organization Management

Kubernetes commands and REST calls for DevOps programmers in tenant organizations are grouped into the following categories. Administrators and developers can act on various resources in the Kubernetes service.

Depending on the kind of resource, administrators and programmers may call the API to perform the following actions.

Resource kind	Administrator action verbs	DevOps action verbs
Project	create, get, update, patch, delete, list	get, list
ProjectRole	get, list	get, list
ProjectRoleBinding	create, get, update, patch, delete, list	get, list

Resource kind	Administrator action verbs	DevOps action verbs
Region	get, list	
SupervisorNamespace	create, get, delete, list	get, list
SupervisorNamespaceClass	create, get, update, patch, delete, list	
SupervisorNamespaceClassConfig	create, get, update, patch, delete, list	

Here is an example of YAML for an organization administrator or DevOps programmer to create a supervisor namespace with `kubectl`, assuming the SupervisorNamespaceClassConfig `e2e-small` contains a Zones section:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
```

Here is a full list of the API resources for DevOps programmers calling `kubectl`. Abbreviations and API version path have been shortened.

```
% kubectl --context cci api-resources
NAME                                 APIVERSION              NAMESPACED  KIND
projectrolebindings                  authorization...v1alpha1   true    ProjectRoleBinding
projectroles                         authorization...v1alpha1   false   ProjectRole
regionmonitortokenrequests           infrastructure...v1alpha1  false   RegionMonitorTokenRequest
regionstorageclassquotas             infrastructure...v1alpha1  false   RegionStorageClassQuota
regionvirtualmachineclassquotas      infrastructure...v1alpha1  false   RegionVirtualMachineClassQuota
regionvirtualmachineclasssummaries   infrastructure...v1alpha1  false   RegionVirtualMachineClassSummary
supervisornamespaceclassconfigs      infrastructure...v1alpha2  false   SupervisorNamespaceClassConfig
supervisornamespaceclasses           infrastructure...v1alpha2  false   SupervisorNamespaceClass
supervisornamespaces                 infrastructure...v1alpha2  true    SupervisorNamespace
virtualmachineremoteconsolerequests  infrastructure...v1alpha1  true    VirtualMachineRemoteConsoleRequest
vkscredentialrequests                infrastructure...v1alpha1  false   VksCredentialRequest
projects                             project...v1alpha2         false   Project
regions                              topology...v1alpha2        false   Region
zones                                topology...v1alpha1        false   Zone
```

In addition to the `kubectl` command, organization administrators and DevOps programmers can use the `vcf` CLI. You can find a command reference for it in the manual *Installing and Using VCF CLI v9.0*, section "VCF CLI Context, Architecture, and Configuration."

APIs for Service Provider Management

The following topics describe a set of REST commands for setting up a service provider, emphasizing tasks that might be repeated frequently.

In the context of VCF Automation, a provider offers services to tenant organizations running virtual machines, developing operations software and microservices, or performing other tasks in a shared computing environment.

The provider interface creates the required organizations and configures underlying infrastructure including regions, IP spaces, gateways, storage, quotas, and users.

When the VCF Automation UI first starts, it displays an inventory of VCF resources, a series of getting-started steps, and Overview menus for Infrastructure, Services, and Administration on the left.

Using REST calls, the provider administrator can automate tasks that might be repeated frequently, or tasks that are easier to accomplish from the command line.

Advantages of Service Provider Accounts

For production environments with virtual machines and container-based microservices, a service account is recommended over a normal user account.

VMware introduced service provider accounts in 2022 as an alternative to user accounts, to enable standalone access for automation plug-ins and third-party applications. Some service provider accounts have system access only through an API, with no access to a user interface. (VCF Automation provider accounts can access a user interface.)

Service provider accounts have several advantages. They cannot be impersonated by anyone else, assuming their credentials are kept secret. Provider management accounts authenticate using bearer access tokens, which rotate after each use. Bearer access tokens typically expire after one hour. They are derived from API refresh tokens, which have a default lifespan of 90 days but can be revoked earlier and optionally rotated on each access.

When authenticating with a service provider account, applications normally cannot perform user management tasks, create API refresh tokens, or manage other provider management accounts.

Provider management accounts are created by a system administrator with privileges View Service Accounts and Manage Service Accounts. System administrators can create provider management accounts either from the VCF Automation user interface, or with the API using a REST call (POST ... register).

Provider management accounts are like user accounts. If allowed, they can create assets such as virtual machines, catalogs, templates, media, and automation sequences. Tasks that are initiated by provider accounts are properly attributed. Provider accounts and user accounts can share assets with each other. System administrators can change provider account attributes, revoke previously granted access, and delete the provider account – ideally after verifying that any assets have been transferred as needed.

Before VCF Automation, provider accounts were used for tenant management and the tenant portal in VMware Cloud Director, which is now discontinued.

Prerequisites for Service Provider Portal

While REST commands exist for attaching a vCenter and registering NSX Managers, these operations are typically performed using VCF Installer or VCF Operations, by specifying the FQDN of connection endpoints.

You can verify vCenter and NSX Manager status using the VCF Automation UI:

1. Click Connections. The vCenter tab appears initially.
2. Name shows the vCenter endpoint, followed by other information such as License Status.
3. Click the tab for NSX Manager. When the tab appears, Name shows the NSX endpoint.
4. On the left, click Supervisors. The default Supervisor was created by vCenter registration.

The Supervisor displays its Region affiliation. Each NSX Manager (or load-balanced set of NSX Managers) is associated with a Region, possibly based on geography.

Before it runs any VCF Automation tasks, the provider automation script must sign in to ensure proper authentication and authorization. See Generating API Tokens for Service Provider Account.

In the following commands, *`vcfa.vcf.example.com`* represents the FQDN of the VCF Automation appliance. All REST commands are assumed to be on a single line, or marked for continuation with backslash (Linux and MacOS shells) or back-quote (PowerShell).

Generating API Tokens for Service Provider Account

VCF Automation uses OAuth to authenticate REST calls with a short-lived bearer access token. First you obtain an API refresh token inside the VCF Automation UI, then you ask to exchange it for a bearer access token on the provider token endpoint.

REST calls require OAuth style HTTP authentication in request headers. The bearer access token is a one-hour expiring token that is used to authenticate all API calls, including calls made from a Provider management account. To acquire a bearer access token, you first generate an API refresh token with the VCF Automation interface, then you generate a bearer access token using the `token` API.

1. Go to the VCF Automation administrator interface at a URL such as vcfa9.example.com/provider. Enter the organization name, such as system, and click **Continue**. If necessary, switch from LDAP to Local authentication.
2. Log in with the service administrator's username and password.
3. In the upper right corner, click ↓ (down arrow) **> My Account**.
4. On the My Account page, click **API Tokens > NEW**.
5. In the Create General Token box, type a name after the tool or function using the API refresh, and click **CREATE**. Make sure to copy and paste the resulting API refresh token, which looks somewhat like this: XhWKaESdLzr0Uo7Nd1JBn2AJZvAC0g07.
6. To obtain a bearer access token, use the `refresh_token` grant type. The response header from this example Curl command includes the long (VCLOUD) bearer access token.

```
curl --location 'https://<vcfa.url>/oauth/provider/token' \
 --header 'Accept: application/*' \
 --header 'Content-Type: application/x-www-form-urlencoded' \
 --data-urlencode 'grant_type=refresh_token' \
 --data-urlencode 'refresh_token=<refreshToken>'
```

7. When you make API calls, provide the bearer access token until it expires. After expiration, you can obtain a new one if needed.

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer"
}
```

Through the UI, API tokens are created without the rotation option. You can use the same API refresh token to get new bearer access tokens until the API refresh token expires.

When token rotation is enabled and you make a `refresh_token` grant-type call, the response body contains both a new bearer access token and a new API refresh token. Save the new refresh token – the previous one is invalidated after the call succeeds.

Create a Region

A Region is a set of one or more supervisors from one or more vCenters, all served by the same NSX Manager, or load balancing NSX Managers. Regions are often geographically based, for example us-west and us-east. If two vCenters are associated with different NSX instances, you cannot add their supervisors in the same region.

This REST command associates `us-west` with an NSX Manager endpoint, including the default Supervisor.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/regions
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"name":"us-west","nsxManager:"https://nsx-west.vcf.example.com",
    "supervisors":["supervisor"], "storagePolicies":["vSAN Default Storage Policy"]}'
```

When creating a Region, you must specify its name as a string, the endpoint URL of an NSX Manager, the endpoint URLs of one or more Supervisors, and one or more storage policies (for example, vSAN ESA Default Storage Policy).

By default, the Region contains a Zone specifying memory and CPU capacity. Zones are aggregated from all supervisors in a Region. The Region inherits a collection of VM Classes, giving a choice of VM sizes and guaranteed or best-effort performance. A Region starts with one Storage Class, but you can add more.

Networking Resources

A Provider does not require Internet connectivity, but in most cases connectivity is useful.

For wide area networking, the Provider needs an Edge Cluster, an IP address Space with CIDR blocks (IPv4 only), and a Provider Gateway associated with the Region. Such a connected Organization is later set to networkingTenancyEnabled.

As a result of associating the Region with NSX, an Edge Cluster is auto-discovered. The Edge Cluster is a logical grouping (often two) of VCF networking Edge transport nodes, which provide network services and north-south routing for workloads. This command lists the nodes. Number of pages and page size are mandatory.

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/vcf/edgeClusters?page=1&pageSize=128
 -H 'Authorization: <value>'
```

You should know the IP address range that tenant organizations can use for Internet connections. The IP Spaces command sets that range for internal scope, and a security address for external reach.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/ipSpaces
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"name:"us-west","externalScopeCidr:"0.0.0.0/0",
    "internalScopeCidrBlocks":[{"id":"us-west","name":"us-west","cidr":"23.215.0.0/16"}]}'
```

Later, you can set quotas for an organization limiting the number of single IPs, the number of CIDRs, and maximum subnet size. Alternatively, you can leave these unlimited.

After the region is given an IP address space, it must get a Provider Gateway. You specify a name for the gateway, a region to which it applies, and an available Tier-0 gateway for physical backing.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/vcf/providerGateways
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"name:"us-west","regionRef":{"name":"us-west","id":"us-west"},
    "backingRef":{"id":"ContainerT0","name":"containerT0"}}'
```

Configure NSX Networking with the vSphere UI

As of VCF 9.1, administrators can manage and operate NSX networking though the vSphere user interface. Virtual public clouds (VPCs) are visible in the vCenter network inventory. Administrators can connect them to each other, or to an external network, using a transit gateway (TGW).

A virtual private cloud (VPC) is a configurable set of resources that are allocated in a cloud environment, providing isolation between different organizations sharing a cluster. A VPC is somewhat like a subnet, but a software construct, and a VPC can itself contain virtual subnets.

A transit gateway (TGW) connects one VPC to another, or to an external network such as the Internet. The external connection can be centralized (CTGW) going through an edge router, or distributed (DTGW) being directly connected to multiple VPCs.

The network span is a logical construct that defines which VPC and non-VPC clusters can connect to each other. By default, all vCenter clusters belong to the default network span. A cluster can belong to multiple network spans. A span can be made exclusive, so that a cluster set can be dedicated to a workload and not shared across the cloud environment.

To configure NSX networking with the vSphere Client user interface:

1. In the network view (globe on upper left) click *Datacenter* > Virtual Private Clouds > ACTIONS > New Transit Gateway. Enter a gateway name for the TGW. Accept the default span, or choose one that has been established. Select either None for internal connections, Centralized for CTGW, or Distributed for DTGW. Click SAVE.
2. To establish a network span, click Virtual Private Clouds > Configure > Settings > Network Spans > ADD NETWORK SPAN. Enter a name, choose Exclusive if desired, and select vSphere clusters in the span. Click SAVE.
3. To configure VPC connectivity policy, click a VPC on the far left, such as Default Transit Gateway. Then click Configure > Routing and Forwarding > Connectivity Policies > ADD POLICY. Enter a Name, choose a policy Type (Community, Isolated, Promiscuous), and select VPC Members that are connected to the TGW. Click SAVE.
4. Once you have configured NSX networking, you can troubleshoot VPC connectivity with live traffic analysis. Click Virtual Private Clouds > Monitor > Troubleshooting > Traceflow.
5. To set up cluster vNetworking, go to the Cluster view (rectangles on upper left). Click *Datacenter* > *Cluster* > Configure > [scroll] vNetworking > Network Configuration. If correctly configured, the cluster is prepared with the default overlay transport zone for VPC readiness.

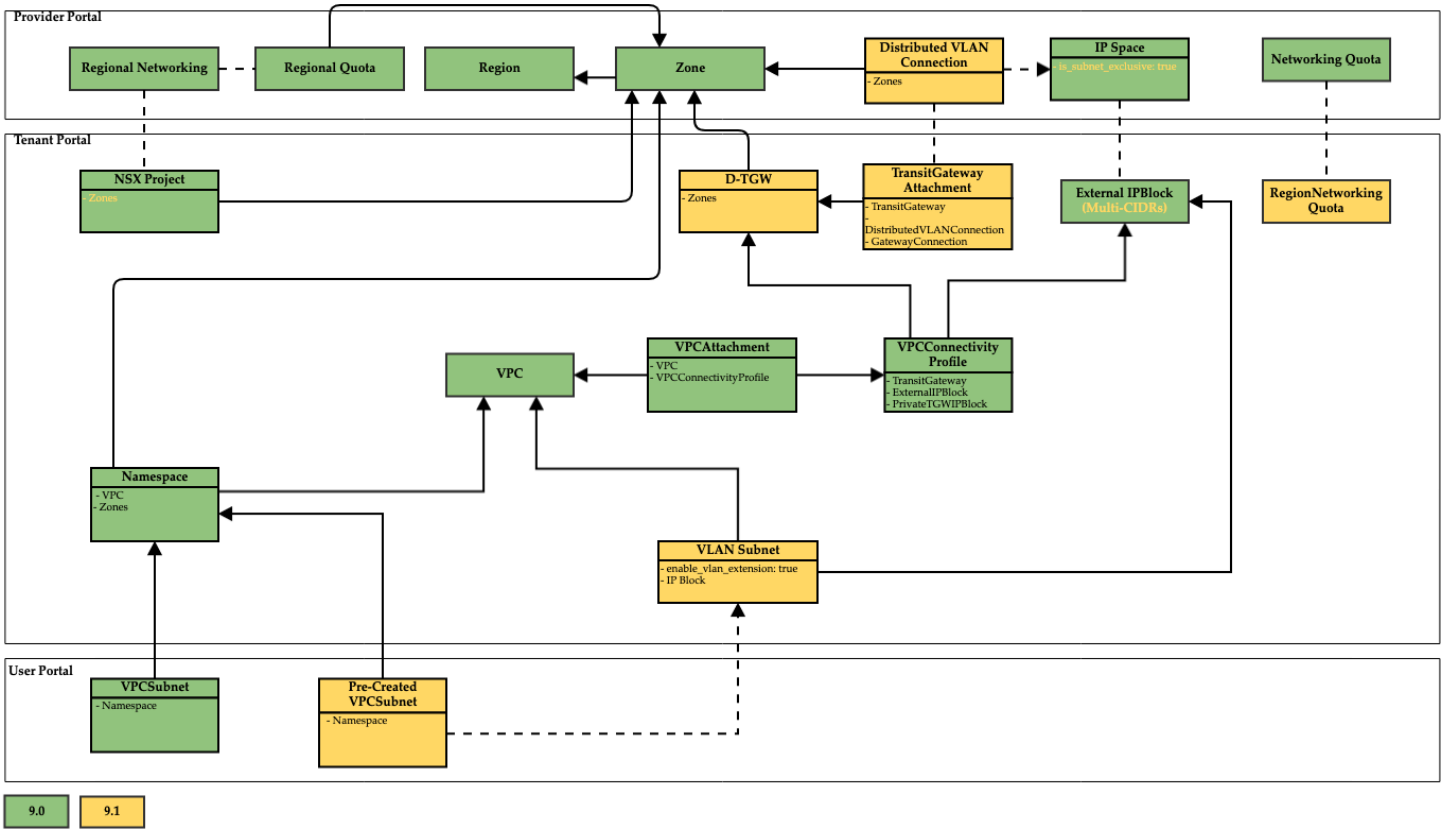
Public cloud adapters (AWS, Azure, GCP) are disabled by default in VCF 9.1.1.

Provisioning a Distributed Transit Gateway

As of VCFA 9.1, organizations that require high-performance distributed routing and VLAN-based connectivity can set up a distributed transit gateway (DTGW). In VCFA 9.0, organizations had to be configured with a centralized transit gateway (CTGW).

The service provider administrator can offer a distributed transit gateway (DTGW) as a networking option for organization tenants. The service provider can assign multiple VLAN connections to the same tenant, ensuring that the connection span overlaps with the zones that are assigned to the tenant's regional quota. Tenant organizations can then create multiple DTGWs.

Figure 665: Network diagram with distributed transit gateway



The distributed VLAN connection defines information about the external VLAN attachment that is used by a tenant's DTGWs, including the external VLAN ID and gateway address.

Regional network settings automatically provision either DTGW or CTGW based on the external connection and networking infrastructure that the provider assigned to the tenant. If virtual network appliance clusters exist in the region, a distributed VLAN connection can be selected as default for regional network settings. The provider can add other ones, but the default distributed VLAN connection cannot be removed or replaced.

The following REST calls manage distributed VLAN connections. JSON describes a VLAN connection.

```
GET, POST          /cloudapi/vcf/distributedVlanConnections
GET, PUT, DELETE   /cloudapi/vcf/distributedVlanConnection/{id}
{
  "status": "REALIZED",
  "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847",
  "name": "dist1",
  "description": "",
  "vlanId": 111,
  "gatewayCidr": "10.1.1.24/24",
  "regionRef": {
    "name": "region2",
```

```
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "zoneRefs": null,
  "ipSpaceRef": {
    "name": "mp-ip1-region2",
    "id": "urn:vcloud:ipSpace:d141d0bd-7048-46ea-824e-c71129244786"
  },
  "subnetExclusive": false,
  "backingId": "dist1",
  "dedicatedOrgRef": null,
  "dedicatedSharedSubnetRef": null
}
```

The following calls view and remove regional network settings. JSON describes regional network settings.

```
GET, PUT, POST, DELETE /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>
{
  "status": "REALIZED",
  "id": "urn:vcloud:regionalNetworkingSetting:8381aebb-ab5a-4637-badd-7ca667bc540f",
  "name": "mp1-STFTestOrgregion2",
  "regionRef": {
    "name": "region2",
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "orgRef": {
    "name": "mp1-STFTestOrg",
    "id": "urn:vcloud:org:c612a1b8-c882-409a-bf71-5d5c7a97b3e5"
  },
  "providerGatewayRef": null,
  "distributedVlanConnectionRef": {
    "name": "dist1",
    "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847"
  },
  "serviceEdgeClusterRef": null,
  "virtualNetworkApplianceClusterRef": {
    "name": "vna-cluster3",
    "id": "urn:vcloud:virtualNetworkApplianceCluster:27433d75-ca0d-4a51-a331-c4160a4c5fd3"
  },
  "defaultVpcPrivateSubnetCidrOverride": null,
  "projectTgwPrivateSubnetCidrOverride": null
}
```

The following calls view and update distributed VLAN connections that are assigned to a region, one of which is default. The `<id>` is the NSX ID of the distributed VLAN connection. JSON describes a distributed VLAN connection.

```
GET, POST   /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>/
distributedVlanConnections
GET, DELETE /vcf/regionalNetworkingSettings/urn:vcloud:regionalNetworkingSetting:<UUID>/
distributedVlanConnections/urn:vcloud:distributedVlanConnections:<UUID>
{
  "status": "REALIZED",
  "id": "dist1",
  "distributedVlanConnectionRef": {
      "name": "dist1",
      "id": "urn:vcloud:distributedVlanConnection:94994a69-cc52-44b1-990d-f434d0d21847"
  },
  "isDefault": true,
  "vlanId": 111,
  "gatewayCidr": "10.1.1.24/24",
```

```
  "ipSpaceRef": {
      "name": "mp-ip1-region2",
      "id": "urn:vcloud:ipSpace:d141d0bd-7048-46ea-824e-c71129244786"
  }
}
```

Virtual network appliance (VNA) clusters are used to support stateful services for DTGWs. In the UI, it is the Virtual Network Appliance Cluster. (Edge clusters function similarly for CTGWs.) The first call is an empty POST to synchronize service clusters in the tenant manager. The GET lists all service clusters in the system, as shown in the sample JSON return.

```
POST /vcf/serviceClusters/sync
GET  /vcf/serviceClusters
{
  "status": null,
  "id": "urn:vcloud:virtualNetworkApplianceCluster:27433d75-ca0d-4a51-a331-c4160a4c5fd3",
  "name": "vna-cluster3",
  "description": null,
  "regionRef": {
    "name": "region2",
    "id": "urn:vcloud:region:6dd82079-aae0-4cbd-98f5-1d635816722f"
  },
  "nodeCount": 2,
  "orgCount": 0,
  "healthStatus": "UP",
  "defaultQosConfig": null,
  "backingRef": {
    "id": "vna-cluster3",
    "name": "vna-cluster3"
  }
}
```

Avi Load Balancer Operations

Service providers can perform day 2 operations for load balancing using REST calls to the Avi load balancer (formerly NSX Advanced load balancer)).

Avi load balancer performs local and global server load balancing, web application firewall security, and ingress services for container-based environments. Avi load balancer was incorporated into NSX networking years ago, and now it is incorporated into VMware Cloud Foundation.

When active, VCF Automation discovers Avi controllers and their associations with NSX Manager. VCF Automation also discovers provider-managed service groups from the Avi controller. Assignment of a service group to a Region is then determined by the load balancer's association with the NSX manager.

The load balancer service engine (SE) is an appliance that requires an IP address in each virtual service network. Service engines are created within a group, which defines how SEs are sized, placed, and made available. A service engine group is a collection of SEs that share properties such as network access and failover. Applications are guaranteed data plane isolation when deployed in separate SE groups.

To query the discovered Avi controllers, the following GET call returns a paged list of Avi controllers:

```
curl --request GET \
  --url https://<vcfa.ip>:8443/cloudapi/1.0.0/loadBalancer/controllers \
  --header 'accept: application/json;version=40.0' \
  --header 'authorization: Bearer <token>' \
  --header 'content-type: application/json;version=40.0'
[response includes]
  "id": "urn:vcloud:loadBalancerController:4887b3c0-7870-492f-8169-7758bc83c76f"
```

Once you have the URN of a particular load balancer, you can call `controllers` again to obtain more information about that controller.

```
GET /1.0.0/loadBalancer/controllers/{aviUrnID}
```

This call adds and registers an Avi load balancer, given a JSON definition.

```
POST /1.0.0/loadBalancer/controllers
```

This call updates an Avi load balancer, given a JSON definition.

```
PUT /1.0.0/loadBalancer/controllers/<aviUrnID>
```

This call deletes an Avi load balancer.

```
DELETE /1.0.0/loadBalancer/controllers/<aviUrnID>
```

As of VCF 9.1, the Service Engine Group name (`segName` field) in SupervisorNamespace.spec can be updated with PATCH, after the namespace is created.

Create an Organization

Organization is the VCF Automation term for tenant organization. You can create an organization with the `Org` API, or with the VCF Automation UI. Under Provider Management, click **Infrastructure > Organizations > Create Organization**.

You can create an organization with the `Org` API, or with the VCF Automation UI – under Provider Management, click **Infrastructure > Organizations > Create Organization**.

To query organizations that already exist, run the following command. The number of pages and page size are mandatory.

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/1.0.0/orgs?page=1&pageSize=128
 -H 'Authorization: <value>'
```

To create an organization, specify an org name and its display name. Other optional data parameters include description and ID, which is auto-filled if not specified.

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/orgs
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"displayName:"Test One","name:"test1"}'
```

As you would expect, there are also REST commands to get organization details, update details, and delete an organization. Organization details include canManageOrgs, creationStatus, directlyManagedOrgCount, isClassicTenant, isEnabled, isProviderConsumptionOrg, managedBy, and maskedEventTaskUsername.

The newly created organization needs a first-user (org admin) with the Organization Administrator role. The first-user is always local, with appliance-specific password rather than identity provider account. For subsequent users, you call a REST command that specifies both organization ID (test1) and role.

```
curl -X POST https://{api_host}/cloudapi/1.0.0/users
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"username":"admin","orgEntityRef":"test1","roleEntityRefs":"OrgAdmin"}'
```

The organization can be associated with a regional network by assigning a quota policy to it, while specifying the quota name and the organization ID.

```
curl -X POST https://{api_host}/cloudapi/1.0.0/quotaPolicies
 -H 'Authorization: <value>' -H 'Content-Type: application/json'
 -d '{"name":"t1q","orgId":"test1","quotaPoolDefinitions":["resourceType":"quota","quota":0]}'
```

Create an Organization for VM Apps

The most convenient way to create a VM Apps organization is with the VCF Automation UI.

In the UI under Provider Management, click **Infrastructure > Organizations > Create Organization > for VM Apps**. For details, see the VMware Cloud Foundation *Provider Management* guide.

You can also use Provider Management APIs to create a VM Apps organization. See Org APIs on the Broadcom developer site.

Before performing the task to create the VM Apps organization, you must obtain a beareraccess token. See Generating API Tokens for Service Provider Account.

The following example shows how to create a VM Apps organization with:

- Name: `vmApp-org-1`
- Display name: `vmApp-org-1 tenant`
- Description: `VM App Org Tenant`

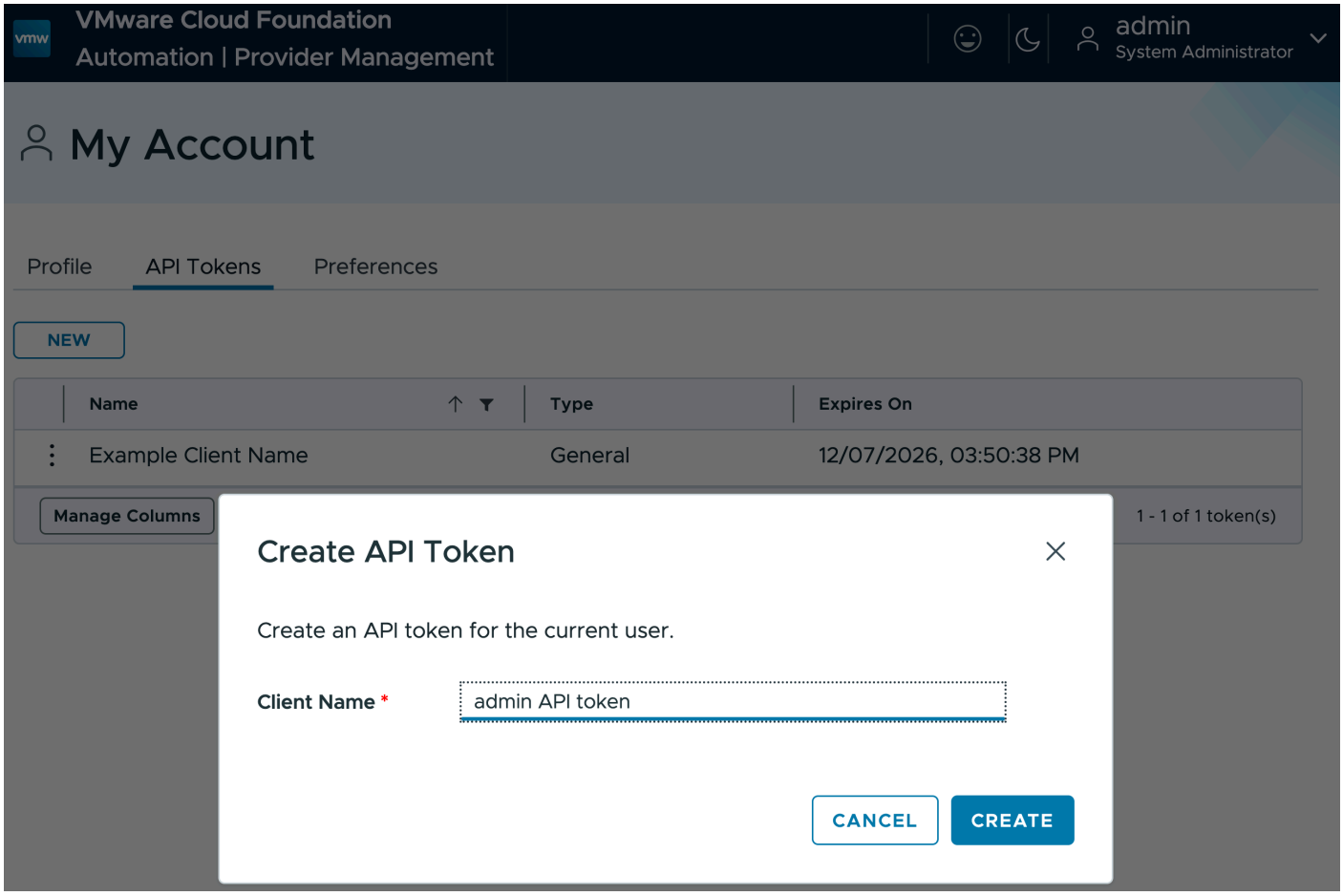
```
curl POST \
  'https://<vcfa.vcf.example.com>/cloudapi/1.0.0/orgs'\
  -H 'accept: application/json;version=10.0.0.0-alpha"
  -H 'authorization: Bearer $access_token'
  -H 'content-type: application/json'
  -d '{
    "name":"vmApp-org-1",
    "displayName":"vmApp-org-1 tenant",
    "description":"VM App Org Tenant",
    "isEnabled":true,
    "isClassicTenant":true
    }'
```

Create Additional Organizations for VM Apps

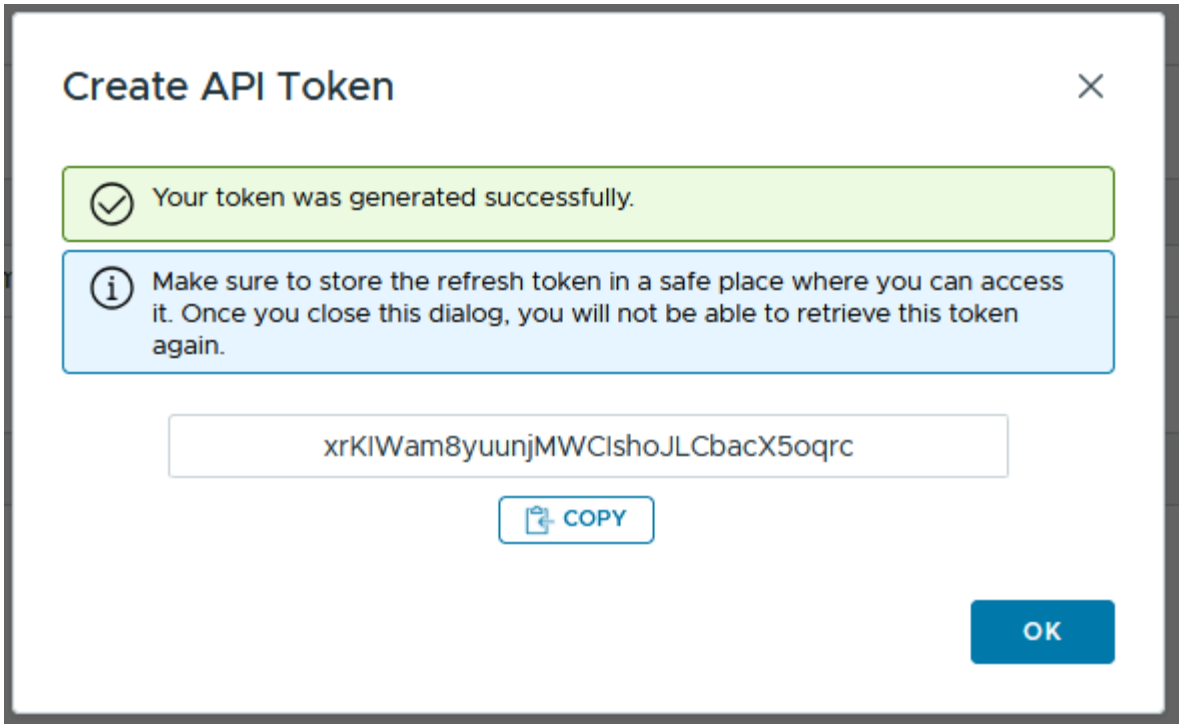
The VCF Automation UI for VM Apps can create only one organization, but you can create another organization using the REST API.

Here are steps to create another VM Apps organization in VCF Automation:

1. Log into the VCF Automation portal, where you can generate an API refresh token.
2. On upper right, select **My Account**. On the My Account page, click **API Tokens > NEW > CREATE**. Type a Client Name for your later reference.



3. Click **COPY** and save the API token. After you click **OK** you cannot retrieve the token again, you can only revoke it. You need thisAPI refresh token to obtain your bearer access token.



4. Run the following command, providing the refresh token. A bearer access token is issued and appears in the response.

```
curl --location --request POST 'https://vcfa.example.com/oauth/provider/token' \
```

```
  --header 'Accept: application/json' \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'grant_type=refresh_token' \
  --data-urlencode 'refresh_token=<Refresh-Token-from-UI>'
```

5. The bearer access token is a long JSON Web Token as in the following example. Copy the bearer access token to paste into the next command.

```
access_token:
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6InNpZ25pbmdfMiJ9.eyJzdWIiOiJ2bXdhcmUuY29tOj......tSQ74_XhszGifZe_gFdxw
```

6. Run the following command with POST request to create the organization. Paste the bearer access token into the Authorization line. Avoid spaces in `name` and `displayName`. Parameter `isClassicTenant=true` indicates a VM Apps organization.

```
curl --location --request POST 'https://vcfa.example.com/cloudapi/1.0.0/orgs' \
 --header 'Accept: application/json;version=40.0' \
 --header 'Authorization: Bearer <access_token>' \
 --header 'Content-Type: application/json' \
 --data-raw '{
   "name": "<vm-app-name>",
   "displayName": "<vm-app-name>",
   "description": "<description>",
   "isEnabled": true,
   "canManageOrgs": false,
   "maskedEventTaskUsername": "system",
   "isClassicTenant": true,
   "isProviderConsumptionOrg": false,
   "creationStatus": "READY"
 }'
```

7. The `curl` command does not send a response. To verify that the VM Apps organization was created, go to the VCF Automation UI and see if the **Create Organization** task was successful. You can also check the **Infrastructure > Organizations** tab.

Infrastructure Compute Policies

Infrastructure policies define requirements and constraints for resource placement and configuration across the regions and zones that are allocated to your organization. At least one compatible policy must be available to each namespace.

Service providers can list, create, and delete policies with the Policies operations. The REST call `Get Assignments` returns a paginated list of policy assignments, which can be filtered by policy IDs, target names, policy types, project ID, or assignment phase. The following `Create Policy` request body sets compute policy for higher performance virtual machines.

```
{
  "name": "High Performance VM Policy",
  "description": "Compute policy for high performance virtual machines",
  "vcComputePolicyName": "HighPerformanceVMPolicy",
  "isMandatory": true,
  "policyRules": [
    { "key": "cpu.cores", "operator": ">=", "values": ["4"] },
    { "key": "vm.class", "operator": "in", "values": ["guaranteed-4xlarge", "guaranteed-8xlarge"] }
   ]
}
```

When a role or object refers to a policy that is subsequently removed with `Delete Policy`, the policy does not go into effect.

Content Library

The Content Library was introduced in vSphere 6. Content Libraries can contain ISO images, OVF files, VM templates, vApps, scripts, applications, and other resources. Content Libraries do not store Kubernetes or Docker containers. Organizations can share Content Libraries, helping reduce storage space and administrator overhead.

Automation is well suited to refreshing and sharing a pre-existing Content Library. To determine what Content Libraries are accessible to an organization, run this command:

```
curl -X GET https://vcfa.vcf.example.com/cloudapi/vcf/contentLibraries?page=1&pageSize=128
  -H 'Authorization: <value>'
```

To refresh the accessible copy of a distributed catalog, run this command, where cat.vcf.example.com is the URN of the accessible catalog:

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/catalogs/cat.vcf.example.com/refresh
  -H 'Authorization: <value>'
```

Consumers of provider services are able to access the refreshed Content Library with the REST command Get Catalog Items. All-apps consumers can also publish, republish, and unpublish existing catalog items.

Administrators are not allowed to upload an image to a Content Library when its status is Partially Ready. Status in the UI is obvious because the **Upload** button is grayed out, whereas the API returns only a `Forbidden` message.

Blueprints

In addition to the content library, tenants can create their own blueprints. If you have a pre-existing organizational blueprint, you can enable it for a tenant.

The VCF Automation user interface offers a drag-and-drop method to create blueprints.

Identity Management

VMware Cloud Foundation 9 has unified identity management that permits single sign-on over a wider range of assets than previously, and offering a decent choice of identity providers.

To help run scheduled updates, the following command begins the LDAP sync task for the identity manager (IDM).

```
curl -X POST https://vcfa.vcf.example.com/cloudapi/1.0.0/ldap/sync
 -H 'Authorization: <value>'
```

Suppose certain users in an organization prefer to use LDAP instead of SAML authentication. This can be set with the `providerType` argument of the **Update User** REST command, or the similar **Update Group** REST command. IDM choices are: Local, LDAP, SAML, and OAuth.

You can choose from modern identity providers – Okta, Ping ID, Entra ID, ADFS, and SAML – or from directory-based identity providers such as Active Directory or Open LDAP.

Custom Branding

For a unique look and feel, interface branding may be applied for all tenants of a Provider service.

The VCF Automation user interface (UI) has a Branding page where you can see resources such as logo, title, and background. Settings can then be assigned as a custom theme to organization pages either shared or private. For details, see topic "Changing the VCF Automation Portal Branding" in the VCF *Provider Management* guide.

An API for `brandingThemes` is also available. To see available themes for an organization, run this command:

```
curl -X GET https://vcfa.example.com/cloudapi/vcf/brandingThemes?page=1&pageSize=128
  -H 'Authorization: <value>'
```

To create a branding theme with the API requires two calls. The first call changes the portal branding name but the branding object remains busy while waiting for the resource Zip to upload. After the second call completes, the branding theme becomes usable again.

```
https://vcfa.example.com/cloudapi/vcf/brandingThemes/urn:vcloud:entity:broadcom:theme:UUID/resources
https://vcfa.example.com/transfer/UUID/resources.zip
```

An easier option is to run a VCF Operations Orchestrator workflow (formerly vRO) to create and install branding themes. Visit this Broadcom developer Tools webpage to download a generic branding workflow that you can modify and run.

To set an organization branding theme, a sample Python script follows the script outline. You will have to change specific values, such as URL and user credentials.

1. Before you run the script, create a `resources.zip` file containing `background.png`, `favIcon.ico`, `styles.css`, and `customLinks.json`, for both light and dark themes.
2. In the Python script, get a bearer access token by calling `cloudapi` sessions/provider.
3. Define the URL of an API endpoint that can receive the `resources.zip` file.
4. Upload the `resources.zip` file using an HTTP PUT request.

```
import requests
import os

# 1. Get an access token
print("__Step 1__")
url = 'https://vcfa.example.com/cloudapi/1.0.0/sessions/provider'
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/json',
}
username = 'admin@system'
password = 'Super!Secret9'
response = requests.post(url, verify=False, auth=(username, password), headers=custom_headers)
print(url)
if response.status_code == 200:
    # Convert the JSON response body to a Python dictionary/list
    user_data = response.json()
    print("Request successful. User data:")
    # Use print for a readable output of the dictionary, print(user_data)
    YourAccessToken = response.headers.get('x-vmware-vcloud-access-token')
    print(YourAccessToken)
else:
    print(f"Error: Request failed with status code {response.status_code}")

# 2. Define the URL of the API endpoint that will receive the file
print("__Step 2__")
url = 'https://vcfa.example.com/cloudapi/vcf/brandingThemes'
file_path = 'resources.zip'
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/json',
    'Authorization': f"Bearer {YourAccessToken}"
}
response = requests.get(url, verify=False, headers=custom_headers)
print(url)
if response.status_code == 200:
    # Convert the JSON response body to a Python dictionary/list
    user_data = response.json()
    print("Request successful. User data:")
    # Use print for a readable output of the dictionary, print(user_data)
else:
    print(f"Error: Request failed with status code {response.status_code}")

# 3. Specify path and contents of your Zip file
```

```
print("__Step 3__")
url = 'https://vcfa.example.com/cloudapi/vcf/brandingThemes/
urn:vcloud:entity:example:theme:5b11f1ca-40e9-4ab9-aad6-cb7c9c62ec4f/resources'
payload = {
    "fileName": "resources.zip",
    "size": 317455,
    "logo": "logo.png",
    "favIcon": "favIcon.ico",
    "background": "background.png",
    "styles": "styles.css",
    "customLinks": "customLinks.json",
    "portalName": "test233"
}
response = requests.put(url, verify=False, headers=custom_headers, json=payload)
print(url)
if response.status_code <= 202:
    # Convert the JSON response body to a Python dictionary/list
    # user_data = response.json()
    Link = response.headers.get('link')
    print("Request successful.");
    # Use print for a readable output of the dictionary
    url = Link.split(";")[0]
    url = url[1:-1]
    print(url)
else:
    print(f"Error: Request failed with status code {response.status_code}")

# 4. Upload Zip file
print("__Step 4__")
print(url)
custom_headers = {
    'Accept': 'application/json;version=41.0.0-alpha',
    'Content-Type': 'application/zip',
    'Authorization': f"Bearer {YourAccessToken}"
    #'Accept-Encoding': 'gzip, deflat, br, zstd'
}
def upload_file_put(url, file_path, content_type='application/octet-stream'):
    """
    Uploads a file using an HTTP PUT request.
    Args:
        url (str): The target URL for the PUT request.
        file_path (str): The path to the file to upload.
        content_type (str): The Content-Type of the file (e.g., 'image/jpeg').
    """
    # Open the file in binary read mode
    with open(file_path, 'rb') as f:
        # Set the headers, specifically the Content-Type
        headers = {'Content-Type': content_type}
        # Send the PUT request with the file data in the 'data' parameter
        response = requests.put(url, verify=False, data=f, headers=headers)
        # Check the response
        if response.status_code in [200, 201, 204]:
            print(f"File uploaded successfully! Status code: {response.status_code}")
            print(response.text)
        else:
            print(f"File upload failed. Status code: {response.status_code}")
            print(response.text)
```

```
file_mime_type = 'application/pdf'
upload_file_put(url, file_path , file_mime_type)
print("Done!")
```

Metadata Management

As of VCF 9.1, administrators can manage metadata for both provider and tenant resources. You can locate resources by searching for related metadata, with role-based access control permissions.

Also, legacy VMware Cloud Director (VCD) entities such as a virtual data center (VDC) can transfer metadata to VCF Automation, as shown in the following table.

VCD to VCFA Resource	Migration Method	Comments
Organization to Organization	Migrate by `VcdOrgId`	Metadata writable by Provider but tenants lack access to Organization.
Provider VDC to Region	Migrate provider VDC	Provider only.
Organization VDC to Region Quota	Migrate organization VDC	Metadata writable by Provider but tenants lack access to Region Quota.
vApp to Namespace	Transfer vApp into Namespace	Provider metadata migrated to RegionQuota. Tenant metadata migrated to Namespace.
VM to VM Service VM	Transfer VM into VM Service	Provider metadata migrated to RegionQuota. Tenant metadata migrated to VM.

The metadata REST calls for the service provider follow the same structure as for defined entities. Reference documentation is available in the API explorer online help.

Retrieve all metadata for the {id} designated entity:

```
GET https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Create a metadata entry in the {id} designated entity:

```
POST https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Replace all metadata entries of the organization with this desired state:

```
PUT https://{vcfa}/1.0.0/orgs/{id}/metadata
```

Retrieve a single metadata entry in the {id} designated entity:

```
GET https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

Update the value of a single key-value metadata entry:

```
PUT https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

Remove a single metadata entry:

```
DELETE https://{vcfa}/1.0.0/orgs/{id}/metadata/{metadataID}
```

APIs for All Apps Organization Management

In VCF Automation, the Provider creates services for tenant organizations. These services allow them to run virtual machines, develop operations software and microservices using containers, and perform other tasks in a shared computing environment.

Kubernetes and REST commands help automate the creation of projects, supervisor namespaces, virtual machines, content library items, and so forth. The following sections describe some actions that can be performed with the `kubectl` command line and REST calls.

Administrators of tenant organizations can assign regions for secure boundaries, namespace class for how to allocate resources, and virtual private clouds (VPC) for network connections that protect regional isolation. Blueprints can be

created so DevOps programmers can easily create an appropriate self-service namespace without having to create to complicated YAML descriptions.

VCF CLI and Kubernetes Setup

The `vcf` command-line interface is required to set up the VCF Automation context, cluster plug-in, and cluster authentication. You can find it for download on the overview page of the VCF Automation UI, or on the Broadcom support site.

To download and install the `vcf` command-line interface, follow this procedure:

1. Open a browser and navigate to your VCF Automation tenant portal:
   ```
   https://<vcfa-url>/tenant/<org>/automation/#/overview
   ```
2. Log in with your tenant credentials.
3. On the Overview page, click **Download VCF CLI**. This navigates to the VCF Consumption CLI download page.
   - Select your system in the OS dropdown menu: Linux, Windows, or Darwin (macOS).
   - Select your CPU architecture in the Arch dropdown menu: AMD64 or ARM64.
   - Click **Download VCF CLI for <OS> (<Arch>)** to download the binary.
   - Optionally, click **Checksum of VCF CLI** to download and verify the SHA256 checksum.
4. Install the binary for your OS and architecture. On MacOS for example:
   ```
   sudo install vcf-cli-darwin_arm64 /usr/local/bin/vcf
   ```

If plug-ins are required to support extended commands, see documentation for VCF Consumption about installing the VCF CLI plug-in bundle.

If you do not already have the Kubernetes command (`kubectl`), it is available using third-party package managers, such as Homebrew for MacOS, Chocolatey for Windows, or RPM for Linux. For example, to download and install `kubectl` on MacOS, type this command:

```
% brew install kubectl-cli
```

As of VCF 9.1, the `vcf` command is no longer available in third-party package managers.

Generating Tokens for Organization APIs

In the VCF Automation UI, log into the tenant organization account to generate an API refresh token. You can then use the API refresh token to generate a bearer access token to authorize API calls.

Prerequisite: The VCF Automation service provider should have created a first user (administrator) account for your tenant organization.

REST calls require OAuth style HTTP authentication in request headers. The bearer access token is a one-hour expiring token that is used to authenticate all API calls, including calls made from a tenant Organization account. To acquire a bearer access token, you first generate an API refresh token with the VCF Automation interface, then you generate a bearer access token using the `token` API.

1. Go to the VCF Automation UI at a URL such as vcfa9.example.com/automation. If necessary, enter your organization name and click **Continue**. Possibly switch from LDAP to local authentication.
2. Log in with the organization administrator's username and password.
3. In the upper right corner, click ↓ (down arrow) > **My Account**.
4. On the My Account page, click **API Tokens > NEW**.
5. In the Create API Token box, type the client name of a tool or function that needs an API refresh token, and click **CREATE**. For example, client name could be *`orgscripts`*.
   Make sure to copy and paste the resulting API refresh token, which looks somewhat like this: TTMrcIeHoqHuYBkhVXGLy1FurFYj9EDK.
6. To exchange the API refresh token for a bearer access token, use the `refresh_token` grant type. You can then use the bearer access token to authorize API calls. Here is an example Curl command for exchange:
   ```
   curl --location 'https://<vcfa.url>/oauth/tenant/<vcfaTenant>/token' \
    --header 'Accept: application/*' \
   ```

```
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=refresh_token' \
--data-urlencode 'refresh_token=<refreshToken>'
```

You can use the same API refresh token to keep getting new bearer access tokens until the API refresh token expires. Then you can obtain a new one from the VCF Automation UI, repeating the steps above.

Initial Setup of Kubernetes Cluster

Access to Kubernetes requires the token obtained above, a certificate for the VCF Automation appliance, and various designations for namespace and project.

Follow these steps to set up the cluster:

1. Fetch the VCF Automation certificate and send it to a PEM file.
   ```
   openssl s_client -connect fqdn:443 -showcerts </dev/null 2>/dev/null | openssl x509 -outform PEM > vcfa-ca-cert.pem
   ```
2. Create the VCF Automation context.
   ```
   vcf context create cci -e fqdn -t cci --api-token <VCF_CLI_API_TOKEN> --tenant-name tenant --ca-certificate vcfa-ca-cert.pem
   Provide Tenant Name: tenant
   ```
3. Use the VCF Automation context.
   ```
   vcf context use cci:namespaceName:projectName
   ```
4. Install the VCF CLI cluster plugin.
   ```
   vcf plugin install cluster
   ```
5. Register the Kubernetes cluster JWT authenticator.
   ```
   vcf cluster register-vcfa-jwt-authenticator <CLUSTER_NAME>
   ```
6. Generate the Kubernetes cluster configuration.
   ```
   vcf cluster kubeconfig get resource.cluster.object.metadata.name --export-file
    resource.cluster.object.metadata.name-kubeconfig
   ```

Follow these steps to access the cluster:

1. Set environment variable VCF_CLI_VCFA_API_TOKEN with the API token to circumvent interactive mode (avoid having to copy-paste it repeatedly).
   ```
   export VCF_CLI_VCFA_API_TOKEN=FlH7L3SXiGAqYZTW784ZcTjQsnvhLlP4
   ```
2. Run the Kubernetes control command.
   ```
   kubectl --kubeconfig resource.cluster.object.metadata.name get namespaces
   ```

> **Note:**
>
> Project name should comply with Kubernetes standards. Administrators can change a project name, while the underlying project ID remains the same.

Creating and Managing Namespaces

This topic shows the YAML for some `kubectl` create commands to manage namespaces. Using the Tenant portal UI, organization users can also create a namespace in the Provider dialog.

Before you create a namespace, a namespace class is required. You can either create a custom namespace class, or use one of the pre-packaged namespace classes: small, medium, or large.

Here is an example that creates a namespace, in the case when the Supervisor Namespace Class Config (e2e-small) contains a Zones array section:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
```

```
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
```

Here is the corresponding namespace class config with the Zones specification. Zones must be set using REST calls, not Kubernetes.

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespaceClassConfig
metadata:
  name: e2e-small
spec:
  defaultLimits:
    cpuLimit: 1000000M
    cpuReservation: 1000M
    memoryLimit: 25000Mi
    memoryReservation: 1000Mi
  storageClasses:
  - limit: 20480Mi
    name: '*'
  vmClasses:
  - name: '*'
  zones:
  - cpuLimit: 100000M
    cpuReservation: 100M
    memoryLimit: 2500Mi
    memoryReservation: 100Mi
    name: vcfa-e2e-wl-vc04-cl1-zone1
```

If there are no Zones in the Supervisor Namespace Class Config, here is an example with override:

```
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespace
metadata:
  generateName: test
  namespace: default-project
spec:
  regionName: e2e-region
  className: e2e-small
  vpcName: e2e-region-default-vpc
  initialClassConfigOverrides:
    zones:
    - cpuLimit: 100000M
      cpuReservation: 100M
      memoryLimit: 2500Mi
      memoryReservation: 100Mi
      name: vcfa-e2e-wl-vc04-cl1-zone1
```

The Region is set by the Service Provider administrator, and is read-only for the organization tenant, who can see it described.

```
% kubectl describe region
Name:         e2e-region
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  topology.cci.vmware.com/v1alpha2
Kind:         Region
```

```
Metadata:
  UID:  f3c633b6-2014-4f65-8736-281ef8cc9a0d
Status:
  Load Balancer Type:  NSX_LB
Events:                <none>
```

Likewise, the Zone is set by the Service Provider administrator, and is read-only for the organization tenant, who can see it described.

```
% kubectl describe zones vcfa-e2e-wl-vc04-cl1-zone1
Name:         dd487fed-b8f1-5696-812a-772a4a607453
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  topology.cci.vmware.com/v1alpha1
Kind:         Zone
Metadata:
  UID:  dd487fed-b8f1-5696-812a-772a4a607453
Spec:
  Cpu Limit:           1000000M
  Cpu Reservation:     10000M
  Memory Limit:        250000Mi
  Memory Reservation:  20000Mi
  Region Name:         e2e-region
  Zone Name:           vcfa-e2e-wl-vc04-cl1-zone1
Status:
  Cpu Reservation Used:     200M
  Cpu Used:                 201000M
  Memory Reservation Used:  200Mi
  Memory Used:              7500Mi
Events:                     <none>
```

Procedure to Create Namespace and Deploy VM

This topic walks you through steps for creating a namespace and deploying a VM with the VCF CLI and Kubernetes CLI, rather than the UI.

Prerequisites: the `vcf` and `kubectl` commands must be installed.

Get API Token. Create a CCI context using the VCFA endpoint, API token, and tenant name. The CLI initializes its plug-in cache on first run, which takes a few seconds.

```
vcf context create vcfa_ctx -e $VCFA_ENDPOINT --api-token $VCFA_API_TOKEN \
  --tenant-name $TENANT_NAME --ca-certificate {path_to_cert_file}
```

Get the Context. List all available contexts to confirm they were saved correctly:

```
vcf context list --wide
```

Set Context to Default. Switch to the top-level CCI context before running project or namespace operations:

```
vcf context use cci
```

The steps below use `kubectl` against the CCI API to create the project, namespace class, and namespace. All manifests follow the standard Kubernetes YAML structure. The CCI-specific API groups handle the underlying VCF orchestration.

Create a New Project. Set a project name and apply the Project manifest:

```
PROJECT_NAME=blr-project
cat <<EOF | kubectl create -f -
apiVersion: project.cci.vmware.com/v1alpha2
kind: Project
```

```
metadata:
  name: ${PROJECT_NAME}
spec:
  description: This is a demo project
EOF
```

Once created, the project appears in the tenant portal and can be listed under `api-resources`.

Create Role Bindings for the Project. Role bindings control which users can access a project and at what permission level. The example below grants admin access to a user named `fritz`:

```
cat <<EOF | kubectl create -f -
apiVersion: authorization.cci.vmware.com/v1alpha1
kind: ProjectRoleBinding
metadata:
  name: cci:user:fritz
  namespace: ${PROJECT_NAME}
roleRef:
  apiGroup: authorization.cci.vmware.com
  kind: ProjectRole
  name: admin
subjects:
- kind: User
  name: fritz
EOF
```

The assigned user appears in the project with the correct role permissions.

Create a Supervisor Namespace Class. The SupervisorNamespaceClass defines a reusable template for namespaces.

```
export SV_NAMESPACE_CLASS=ar-ns-class1
cat <<EOF | kubectl apply -f -
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespaceClass
metadata:
  name: ${SV_NAMESPACE_CLASS}
spec:
  description: ${SV_NAMESPACE_CLASS} supervisor namespace class
EOF
```

Create a Supervisor Namespace Class Config. Before creating the config, confirm your `storageClass` and `contentSource` values, which are environment-specific and must match what is available in your VCF deployment.

```
export CONTENT_SOURCE=blr2-library
export STORAGE_CLASS=wcpglobal_storage_profile
cat <<EOF | kubectl apply -f -
apiVersion: infrastructure.cci.vmware.com/v1alpha2
kind: SupervisorNamespaceClassConfig
metadata:
  name: ${SV_NAMESPACE_CLASS}
spec:
  storageClasses:
    - name: ${STORAGE_CLASS}
      limit: 2048Mi
  vmClasses:
    - name: "*"
  contentSources:
    - name: ${CONTENT_SOURCE}
      type: ContentLibrary
  # Just examples - settings may cause unexpected behavior in namespace
  defaultLimits:
```

```
    cpuLimit: 1000M
    cpuReservation: 1000M
    memoryLimit: 1000Mi
    memoryReservation: 1000Mi
EOF
```

Create a Supervisor Namespace. Before creating a namespace, look up the region, VPC, and zone names to reference in the manifest:

```
kubectl get regions
kubectl get vpc
kubectl get zones
```

With those values in hand, create the namespace. The `className` field references `ar-ns-class1`, the SupervisorNamespaceClass created in the previous step.

```
cat <<EOF | kubectl create -f -
apiVersion: infrastructure.cci.vmware.com/v1alpha3
kind: SupervisorNamespace
metadata:
  generateName: frodo5
  namespace: blr-project
spec:
  className: ar-ns-class1
  description: ''
  regionName: region1
  vpcName: region1-default-vpc
  sharedSubnetNames: []
  classConfigOverrides:
    zones:
      - name: domain-c10
        cpuLimit: 1M
        cpuReservation: 0M
        memoryLimit: 1Mi
        memoryReservation: 0Mi
EOF
```

After creation, the namespace status will show as active in the project page under the Namespaces tab.

Create a VM

Switch Context to the Target Namespace. Switch to the namespace context before deploying the VM. The CLI installs any context-recommended plugins automatically:

```
vcf context use cci:frodo5-pcgyn:blr-project
```

Plug-ins are installed and the inventory is refreshed. If you see a sync error for dev-versioned plug-ins above, run `vcf plugin sync` to resolve before proceeding.

Deploy the VM. Create the VirtualMachine manifest, specifying the VM class, image, and storage class:

```
cat <<EOF | kubectl create -f -
apiVersion: vmoperator.vmware.com/v1alpha5
kind: VirtualMachine
metadata:
  name: my-vm
  namespace: frodo5-pcgyn
spec:
  className:    ar-ns-class1
  imageName:    vmi-feea133c28594bd00
  storageClass: wcpglobal-storage-profile
```

```
EOF
```

Developer Self-Service with Virtual Machine Groups

As organization tenants, DevOps programmers can create a virtual machine group, which is useful to associate machines that can be managed together. For example, a multi-machine application might have a management back end, a dedicated database, and a web interface front end.

This functionality is similar to the traditional vApp in vSphere.

By creating a virtual machine group, you can perform operations on a set of VMs at an appropriate time. For instance, you can define the order of power operations so that your user interface does not start prematurely. You can power off all VMs to synchronize backup of the virtual machine group. You can power off an individual VM without affecting the power state of other VMs in the group.

You may add existing or new VMs to a VM group. You may create a VM group at the same time as its constituent VMs. VM groups may include other VM groups. The virtual machine group must have a namespace, it should contain at least two virtual machines, and the VM Service must be available.

In the YAML definition, a VM group is defined as kind (type) `VirtualMachineGroup`. Affinity and anti-affinity rules are defined in kind `VirtualMachineAffinity`. The snapshot details of a virtual machine group for backup, or other purposes, is defined in kind `VirtualMachineSnapshotSpec`.

A new VIM task `syncVolume` (or `CnsSyncVolume`) was implemented in VCF 9.1 to synchronize the container native storage (CNS) volumes used for storage by the VM group. It gets called behind the scenes during desired state operations.

Region VM Class Quotas

Regions are associated with quotas for resource use (CPU, memory, storage), with VM classes based on size, and with storage classes based on capacity. CPU and memory have both a maximum limit and a minimum reserve allocation.

You can run Kubernetes commands to determine VM class summaries, VM class quotas, and storage class quotas for a Region.

```
% kubectl get regions
NAME
paif-region
% kubectl get regionvirtualmachineclasssummaries --field-selector spec.regionName=paif-region
% kubectl get regionvirtualmachineclassquotas --field-selector spec.regionName=paif-region
% kubectl get regionstorageclassquotas --field-selector spec.regionName=paif-region
```

Networking for All Apps API

Software defined networking with VMware NSX offers a way to increase security and isolation of clients with VCF Automation All Apps tenancy.

VCF Automation involves three contexts: the Console context of VCF, the Supervisor context created with vCenter, and the Kubernetes context for Supervisor clusters. Network static routes and subnets are defined as "kinds" of custom resource definitions (CRD) in the Supervisor context. These are the important ones.

- `networkinfos.crd.nsx.vmware.com`
  NetworkInfo is used to report the network information for a namespace.
- `subnets.crd.nsx.vmware.com`
  Subnet is the resource schema for the subnets API. SubnetPortSpec defines the desired state of SubnetPort. SubnetPortStatus defines the observed state of SubnetPort.
- `subnetsets.crd.nsx.vmware.com`
  SubnetSet is the resource schema for the `subnetsets` API. SubnetSetSpec defines the desired state of SubnetSet. SubnetSetStatus defines the observed state of SubnetSet.
- `staticroutes.crd.nsx.vmware.com`

StaticRoute is the resource schema for the `staticroutes` API. StaticRouteCondition defines the condition of StaticRoute. StaticRouteSpec defines static routes configuration on the virtual private cloud. StaticRouteStatus defines the observed state of StaticRoute.

Virtual private cloud information in VCF is obtained from the NetworkInfo custom resource, which is read-only. This example shows a Kubernetes command retrieving network information so that subnets and static routes can later be applied to this network.

```
# kubectl get networkinfo -o yaml
apiVersion: v1
items:
- apiVersion: crd.nsx.example.com/v1alpha1
  kind: NetworkInfo
  metadata:
    creationTimestamp: "2024-11-04T08:11:13Z"
    generation: 2
    name: svc-tkg-domain-c10
    namespace: svc-tkg-domain-c10
    resourceVersion: "10279"
    uid: 93ee3688-d6f1-477c-9914-18bdab05b2db
  vpcs:
  - defaultSNATIP: 192.168.0.4
    name: vmware-system-supervisor-services-vpc_ef56697a-4a16-4090-8299-6cdb48caab05
    privateIPs:
    - 172.17.0.0/16
```

This example shows a Kubernetes command creating a private subnet that is named `subnet-sample` served by DHCP with 64 nodes.

```
$ cat <<EOF | kubectl create -f -
apiVersion: crd.nsx.example.com/v1alpha1
kind: Subnet
metadata:
  name: subnet-sample
spec:
  accessMode: Private
  ipv4SubnetSize: 64
  subnetDHCPConfig:
    mode: DHCPServer
EOF
```

Once subnets are created, they can be organized into a subnetSet.

This example shows a Kubernetes command creating a private subnet set named `subnetset-sample` with size 64.

```
$ cat <<EOF | kubectl create -f -
apiVersion: crd.nsx.example.com/v1alpha1
kind: SubnetSet
metadata:
  name: subnetset-sample
spec:
  accessMode: Private
  ipv4SubnetSize: 64
EOF
```

This example shows a Kubernetes command creating a 192 private network with static routing through 10.10.0.1 or 10.10.02.

```
$ cat <<EOF | kubectl create -f -
apiVersion: crd.nsx.example.com/v1alpha1
kind: StaticRoute
```

```
metadata:
  name: route1
spec:
  network: 192.168.2.0/24
  nextHops:
  - ipAddress: 10.10.0.2
  - ipAddress: 10.10.0.1
EOF
```

APIs for VM Apps Organization Management

As a VMware Cloud Foundation Automation® (formerly VMware Aria Automation) user or customer, you can perform VCF Automation functions programmatically by using REST API service calls.

API Services for VM Apps

VCF Automation includes the following APIs for the VM Apps Org. API documentation is available with the product. To access all Swagger specifications from a single landing page, log in as admin to the VCF Automation for VM Apps Organization tenant. At the top-right corner of the home page, click the user name and open the **API Help Center**. Then click **Explore Automation APIs**.

API documentation is also available from the Broadcom Developer Portal at https://developer.broadcom.com/xapis/org-management-vm-apps-org/latest/

Note:

- Automation scripts that authenticate against new VM Apps organizations should use only VCF Automation Provider Management Access Control APIs, and not CSP APIs.

Service Name	Description
ABX	Create or manage actions and their versions. Execute actions and flows.
Access Control	Supports identity operations for VCF Automation, enabling management of groups and users.
Approvals	Enforce policies that control required approvals for a deployment or Day 2 action before the request is provisioned.
Blueprint	Create, validate, and provision blueprints.
Catalog	Access catalog items and catalog sources, including content sharing and the request of catalog items.
Content Gateway	Connect to your infrastructure as code content in external content sources such as Source Code Management providers.
Custom Forms	Define dynamic form rendering in Catalog and VCF Automation services.
Deployment	Access deployment objects and platforms or blueprints that have been deployed into the system.
Deployment metric	Aggregated metric values for the deployment objects.
Event Broker	View audit log, events, event topics, and extensibility subscriptions.
Identity	Authenticate and manage the authorization provider.
Onboarding	Define policies and plans to bring existing VMs from any cloud under management.
Orchestrator Gateway	Run workflows and actions to automate complex IT tasks.
Policies	Interact with policies created in Catalog.

Service Name	Description
Projects	Provide visibility and isolation of provisioned resources for users with a project role.
Provisioning Service	Perform infrastructure setup tasks, including validation and provisioning of resources in an iterative manner.

API Deprecations and Changes

CSP APIs are deprecated. For a complete list of Identity API changes, log in as admin to the VCF Automation for VM Apps Organization tenant. At the top-right corner of the home page, click the user name and open the **API Help Center**. At the left, click **API Transition** to see all the changes to the Identity Service functionality that may affect your existing workflows.

As of VCF Automation 9.1, the following APIs have changed:

- The CMX API is no longer available because the CMX service has been removed.
  Formerly CMX service was responsible for Kubernetes integration for deployment and management of Kubernetes clusters and namespaces. In VCF Automation, these capabilities have been moved from VM Apps to All Apps organization types.
- The Pipelines API behind the `/codestream/api/*` endpoint are not available because the feature has been discontinued.
- The Event Broker API behind the `/event-broker/api/*` endpoint is public.
- VCF Automation tenant branding is now controlled from the VCFA Provider Management portal and Provider Management APIs. See the VCF Automation Branding Theme API.
- The Migration Assessment and Migration Service API responsible for orchestrating migrations from vRealize Automation 7.x to Aria Automation 8 has been removed.

VKS integration is available through CCI in the All Apps organization. See VCF Automation and APIs.

The following API endpoints available in Aria Automation have been removed in VCF Automation :

- Catalog Service
  - `/policy/api/policies/limit-usage`
  - `/catalog/api/items/{id}/data/{dataElementType}/{dataElementId}`
  - `/price/api/cloudhealth/perspective-sync`
  - `/catalog/api/about /catalog/api/admin/entitlements`
  - `/catalog/api/admin/sources/limit-usage`
  - `/catalog/api/admin/entitlements/{id}`
- Infrastructure as a Service
  - `/iaas/api/cloud-accounts-nsx-v`
  - `/iaas/api/cloud-accounts-avilb/{id}`
  - `/iaas/api/cloud-accounts-nsx-v/{id}`
  - `/iaas/api/cloud-accounts-avilb`
- Approval Service
  - `/core/authn/logout /access-token /admin/api/upgrade`

How developers use the VCF Automation APIs

To make API service calls, you use a browser application or an HTTP client application to send requests and review responses. The following open-source applications are commonly used:

- cURL. https://curl.se/
- `jq` parser. https://jqlang.org/
- Postman application. https://www.postman.com/
- API Client Bruno. https://www.usebruno.com/

To learn how to use the API, you start by getting an authentication token. Then you can perform steps outlined in the use cases in this guide. The use cases include `curl` commands in request examples. To use the commands, ensure that the

`jq` command-line JSON processor is installed with `curl`. The `jq` parser ensures that responses are formatted for optimum readability. For information about `jq` installation, see https://jqlang.org/download/.

Getting Your Authentication Token

In the REST API, VCF Automation requires an HTTP authentication token in the Authorization request header. Getting your authentication token is a prerequisite for any use case.

You use an API bearer token to authenticate all API calls. The API bearer token is known as the access token in the REST API service call examples. To acquire an authentication token, you must:

- Log in to the VCF Automation Provider Management Portal to find the name of the VM Apps organization.
- Log in to the VM Apps tenant to get the API token for the organization. The API token for the organization is the refresh token that is used to get the access token.
- Use the API token to get an access token either by:
  - Using an API client such as Postman to create an environment for the VM Apps tenant and get an access token that expires in one hour by default.
  - Registering an OAuth Client, then fetching the API token of the registered client to set an expiration on the access token.

The default lifespan of an API token is 129600 minutes or 90 days. The default lifespan of an access token is one hour. If you want to change the default lifespan of the API tokens you create, review the information in Changing the Lifetime Setting of an API Token before obtaining your refresh token.

Get a Refresh Token for the VM Apps Tenant

To get an API token for the VCF Automation for VM Apps API, you must authenticate against the VM Apps tenant. To get the name of the organization for the VM Apps tenant, you use the VCF Automation Provider Management Portal. Then you log in to the VM Apps tenant to get the API token to be used as a refresh token.

> **Important:**
>
> As of VCF 9.0, the Identity Service API functionality has changed and is no longer used to obtain a refresh token programmatically. To get an API token for the VM Apps organization, you use the VCF Automation UI to authenticate against the VM Apps tenant.

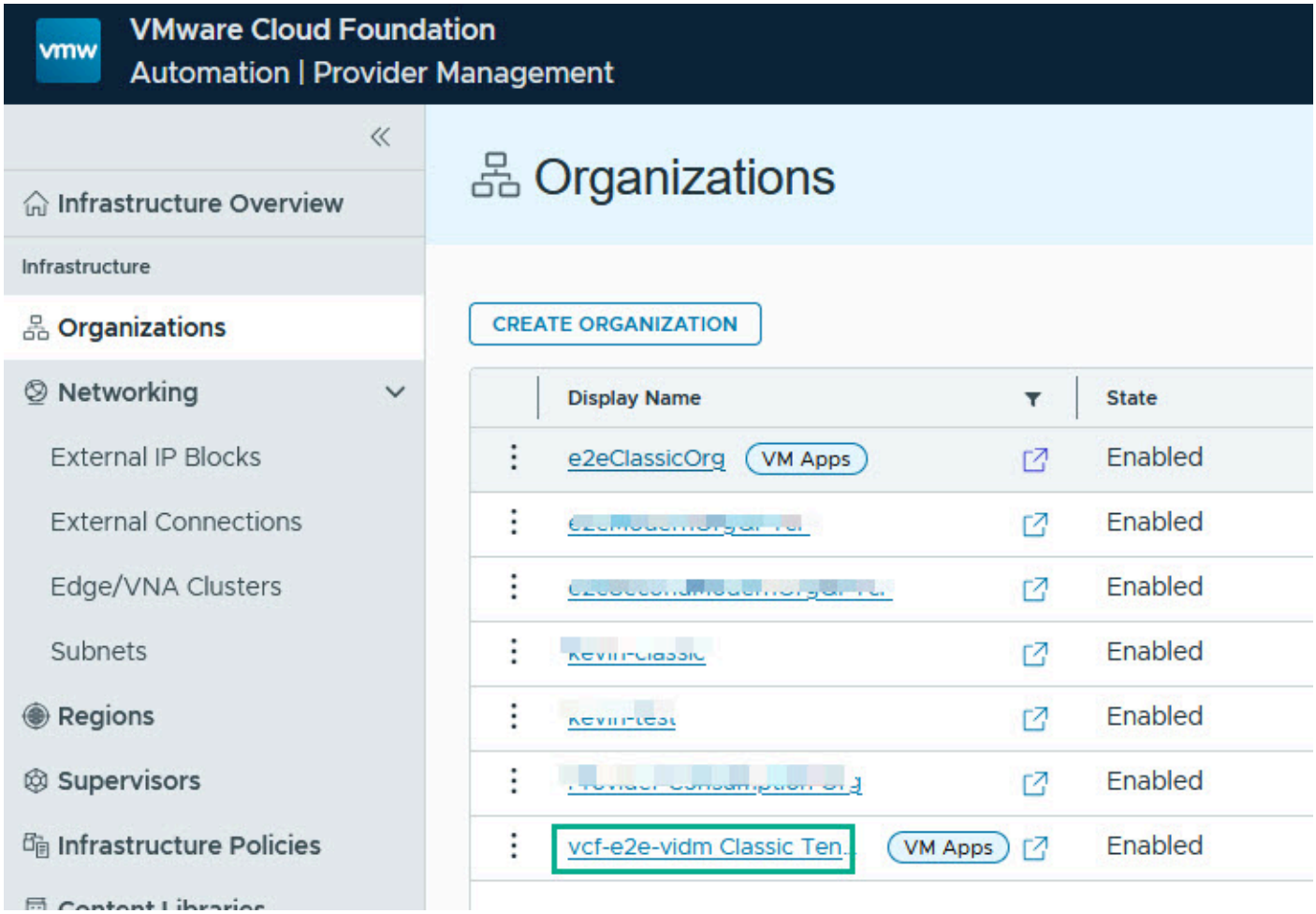
Prerequisites

- Secure a channel between the web browser and the VCF Automation server. Open a browser and enter the URL such as: `https://appliance.domain.com`.
- Verify that you are an authenticated user with the Assembler Administrator or Service Broker Administrator role in the VM Apps organization. See What are the VCF Automation for VM Apps user roles.

Get the Name of the Organization for the VM Apps Tenant

Log in to the Provider Management Portal and perform the following steps to get the name of the organization for the VM Apps tenant.

1. Log in to the Provider Management Portal with the username: `admin` and the password that you used to upgrade from VMware Aria Automation 8.18.1.
2. Click **Organizations** and choose the name of your VM Apps classic tenant.

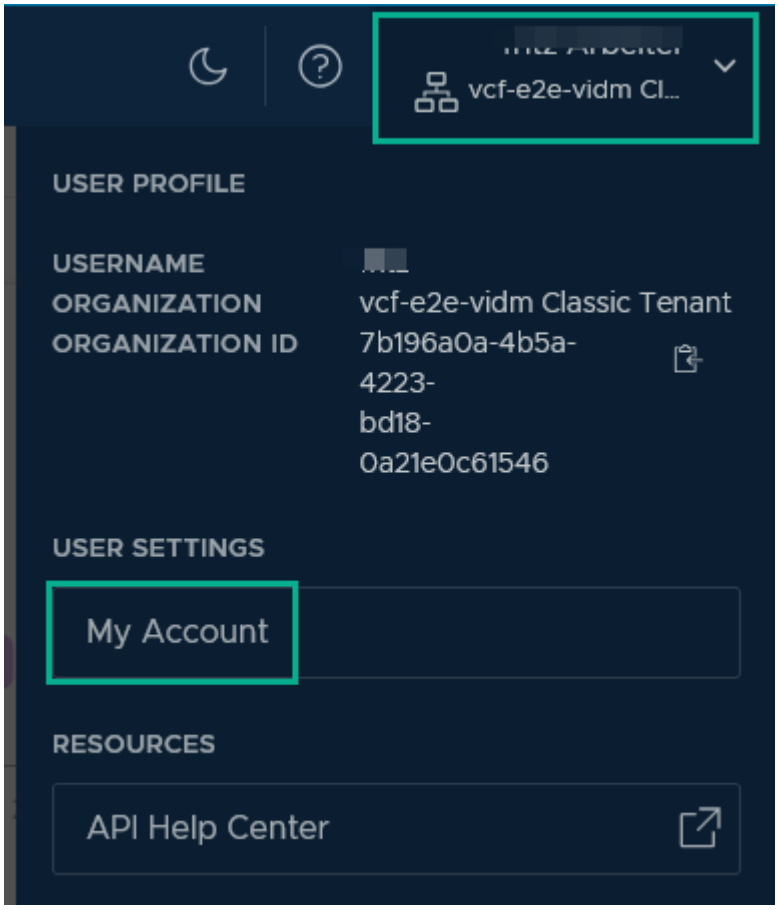


3. When the **Overview** window opens, copy and save:
   - VM Apps tenant name that is displayed as the Name, such as `vcf-e2e-vidm`.
   - VCFA hostname or *`VCFA_FQDN`* that is displayed in the Default organization URL.

Get the API Token for the Organization

Perform the following steps to get the API token for the organization, also known as the refresh token.

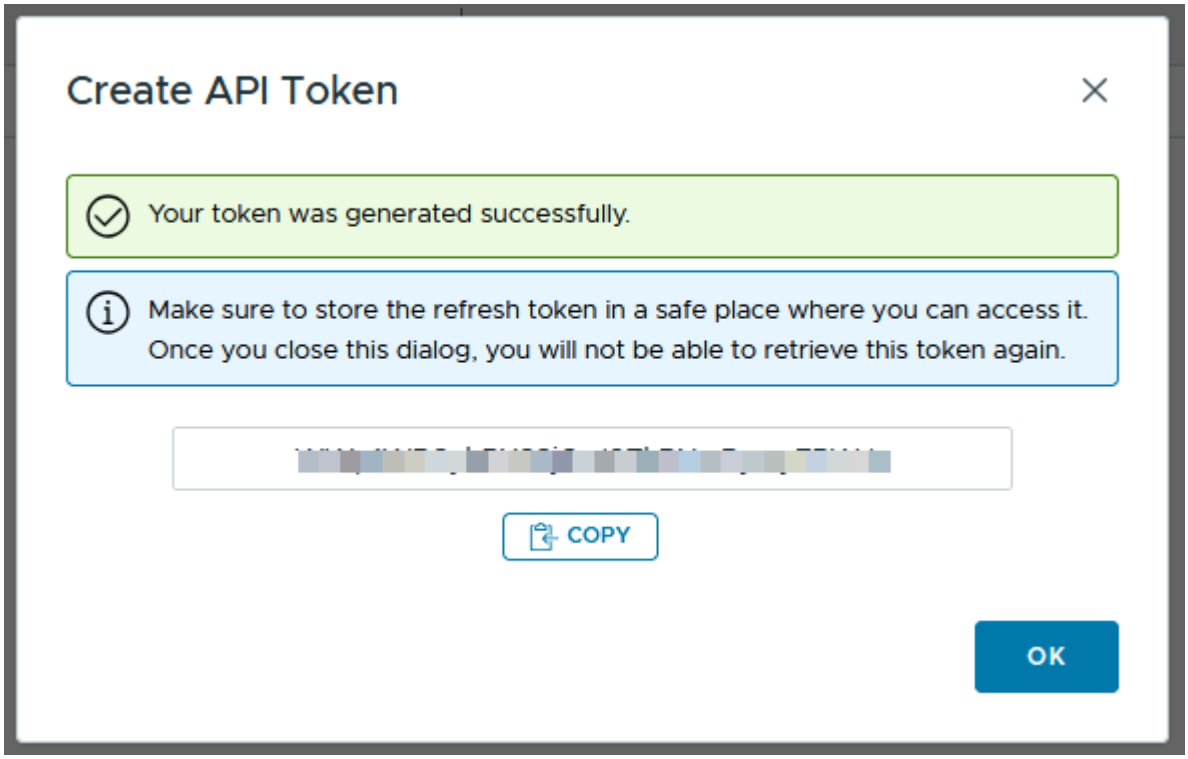
1. Open the VCF Automation UI, and change the organization to the name of the VM Apps tenant, such as `vcf-e2e-vidm`.
2. Log in with your tenant administrator username and password.
3. In the top-right corner, click the name of the tenant and then click the **My Account** link under User Settings.



4. On the My Account page, click the API Tokens tab, then click **New**.
5. On the Create API Token page that appears, enter a name for the token and click **Create**.
6. Click **Copy** and save the API token.

   Important:

   You need this token to obtain your access token.



Get Your Access Token for the VCF Automation API

You use an access token to authenticate your session for a VMware Aria Automation instance that has been upgraded to VCF Automation. To get the access token, you use the API token that you created for the VM Apps tenant organization.

Verify that you have your:

- VCF Automation hostname from the VM Apps organization in the Provider Management Portal.
- VCF Automation tenant name from the VM Apps organization in the Provider Management Portal, such as `vcf-e2e-vidm`.
- API token for the VM Apps tenant organization, also known as the refresh token.

See Get a Refresh Token for the VM Apps Tenant.

> **Note:**
>
> To get a token for a newly deployed instance of VCF Automation that does not include a VM Apps tenant, use the same method that is used for VCF Automation for All Apps. See Generating API Tokens for Service Provider Account.

The following procedure shows how to obtain the access token using the Bruno API Client. The access token expires after one hour. To obtain an access token rather than through an API client, see Getting Your Access Token for the VCF Automation VM Apps API Programmatically.

1. Create and save Global Environment for your VCF Automation VM Apps tenant with the following parameters:

```
vcfaHostname: {VCFA_FQDN}
vcfaTenant: {VM_app_tenant_name}
vcfaAPIToken:{API_token}
```

2. Run a query to obtain the API token.

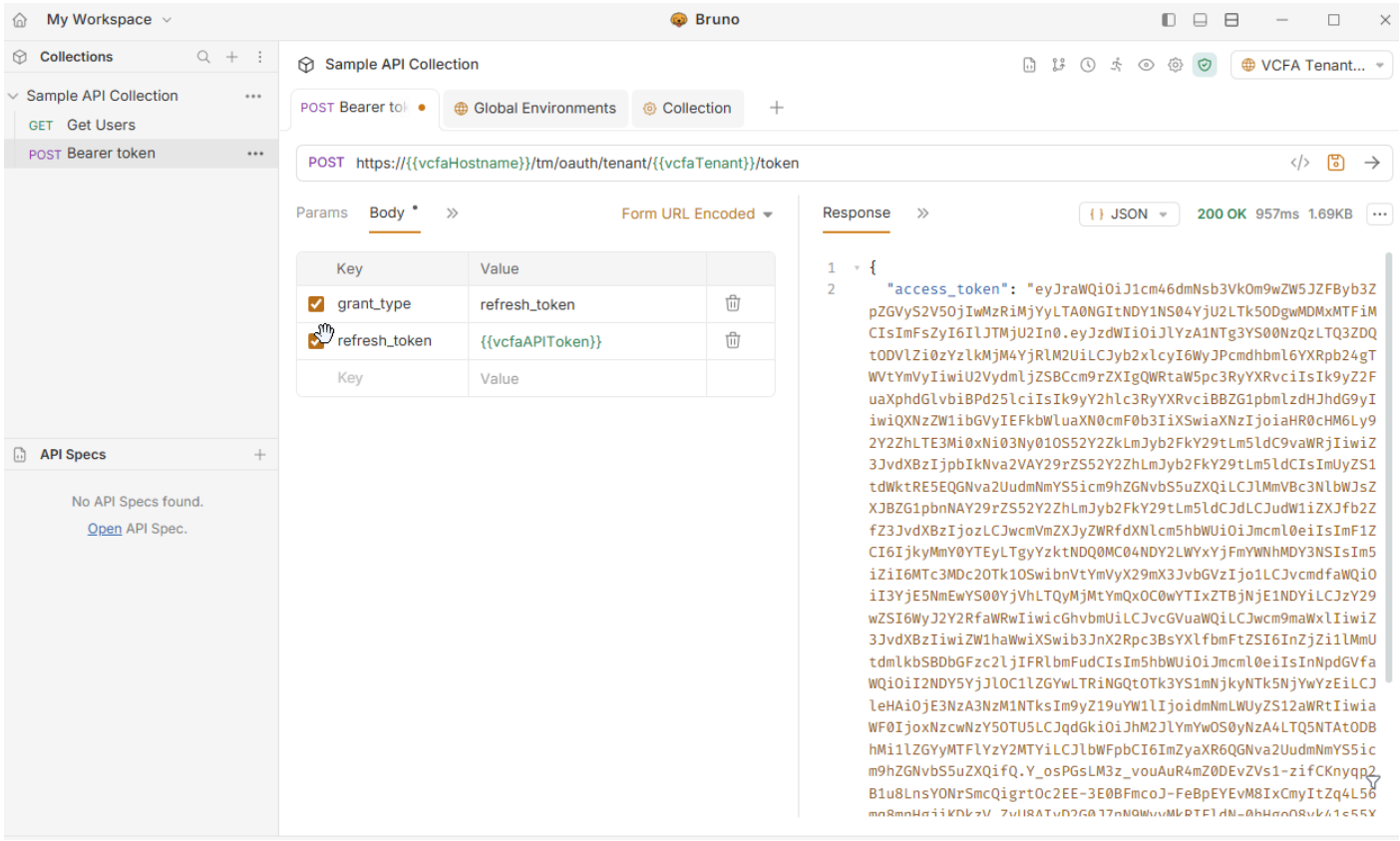
```
POST https://{{vcfaHostname}}/tm/oauth/tenant/{{vcfaTenant}}/token

Headers:
Content-Type: application/x-www-form-urlencoded
```

```
Accept: application/json

Body: Form URL Encoded
grant_type: refresh_token
refresh_token: {{vcfaAPIToken}}
```

The response returns the access token.



3. Copy the text of the access token.

The access token is the API bearer token that is required to access tenant-specific APIs.

Use the access token to verify user roles. See Verify User Roles .

Getting Your Access Token for the VCF Automation VM Apps API Programmatically

With the API token that you created for the VM Apps tenant organization and VCF Automation tenant name from the VM Apps organization, you can obtain your access token programmatically.

Prerequisites

Verify that you have your:

- VCF Automation tenant name from the VM Apps organization in Provider Management Portal, such as `vcf-e2e-vidm`.
- API token for the VM Apps tenant organization, also known as the refresh token.

See Get a Refresh Token for the VM Apps Tenant.

Register a New Client API Token

Use the refresh token to obtain the ID of the registered client, where:

- `org_name: {VM_app_tenant_name}`
- `access_token: {API_token_for_VMApps_org}`

```
POST /oauth/tenant/{org_name}/register
Authorization: Bearer {{access_token}}
Content-Type: application/json

# Body
{"client_name":"my_client"}

# Response
{
    "client_name": "my_client",
    "client_id": "0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a",
    "grant_types": [
        "urn:ietf:params:oauth:grant-type:jwt-bearer",
        "refresh_token"
    ],
    "token_endpoint_auth_method": "none",
    "client_uri": null,
    "software_id": null,
    "software_version": null,
    "scope": null
}
```

Fetch the API Token of a Registered Client

Use the ID of the registered client to fetch the API bearer token. The new API token in the "refresh_token" field is the API bearer token that is required to access tenant-specific APIs.

```
POST /oauth/tenant/org/token
Authorization: Bearer {{access_token}}
Content-Type: application/x-www-form-urlencoded

# Body
# take the client id from the previous response
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
&client_id=0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a
&assertion={{access_token}}
```

```
# Response
# The new API token will be contained in the refresh_token field
{
    "access_token": "new_access_token",
    "token_type": "Bearer",
    "expires_in": 3600,
    "refresh_token": "api_token"
}
```

Changing the Lifetime Setting of an API Token

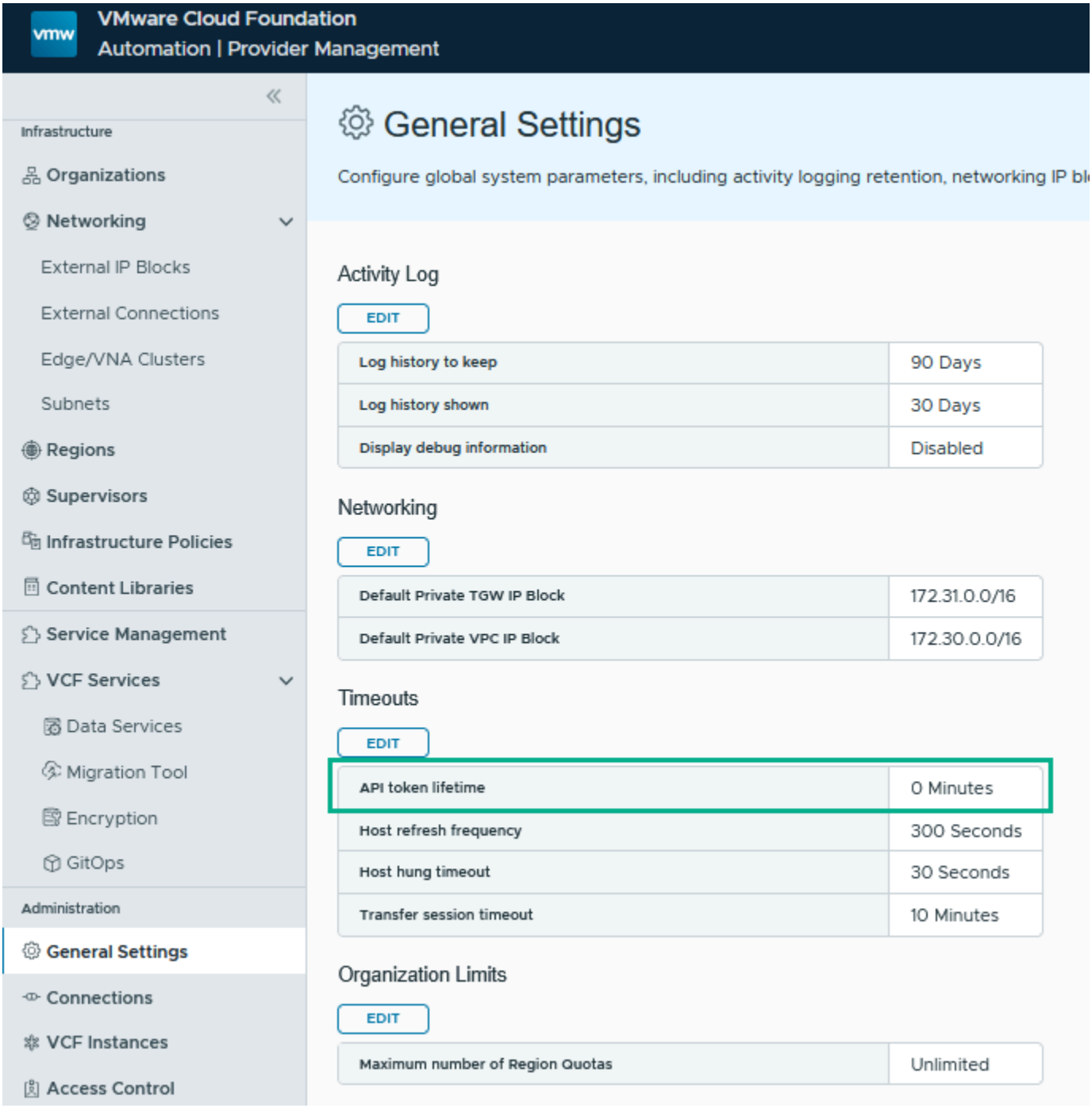
The lifespan of an API token is defined by the API token lifetime setting when you create the token and cannot be changed retroactively. As a best practice, you should change the API token lifetime before creating your token so that you control its expiration period. Or if you created tokens before changing the API token lifetime, you must revoke the old tokens before creating tokens again with the changed lifetime setting.

Change the Lifetime Setting

To change the lifespan of the API Token for your organization, log in to the Provider Management Portal, navigate to **General Settings** > **Timeouts** and enter a value for the API token lifetime. Change this setting before getting your refresh token. See Get a Refresh Token for the VM Apps Tenant.

The default lifespan of the API token is 129600 minutes or 90 days. To change the API token lifetime:

- Enter 0 for an unlimited lifespan.
- Enter an integer value for the lifetime number of minutes.



Revoke an Existing Token

To change the lifespan of an API token that was previously created, you must revoke it and create it again with the changed lifetime setting. If you have multiple tokens, you revoke each token separately. You can revoke tokens either programmatically using the API or from the UI by logging in to the VM Apps or All Apps tenant.

In the following example, the API request deletes the refresh token `0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a`. If you have multiple refresh tokens, insert each token in the command, and run the delete API request again.

```
DELETE /cloudapi/1.0.0/tokens/urn:vcloud:token:0c72befc-f9a1-4b4e-bffb-2fc08ddfbf3a
```

Verify User Roles

To use the API, aVCF Automation user must be an organization member with at least a user service role. You use the access token to verify user roles.

Verify that you have an access token. See Get Your Access Token for the VCF Automation API.

1. Assign values to the variables for the hostname of yourVCF Automation instance, your user name, and your password.

```
url='http://<FQDN>'
username='<your_username>'
password='<your_password>'
```

2. Get your organization ID.

```
curl -X GET \
 "$url/csp/gateway/am/api/loggedin/user/orgs" \
 -H "csp-auth-token: $access_token"
```

3. Examine the response and assign the organization ID variable.

```
org_id='<your_org_id>'
```

4. Get your organization role.

```
curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/roles \
  -H "csp-auth-token: $access_token" | jq "."
```

   The name field displays the organization role and must be either `org_owner` or `org_member`.

5. Get your service role.

```
curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/service-roles \
  -H "csp-auth-token: $access_token" | jq "."
```

   The serviceRolesNames field displays the service role for each service and must be at least `user`.

   Verify User Roles

   Using the access token previously obtained and assigned, verify user roles. See Get Your Access Token for the VCF Automation API.

   Assign variables.

```
# url='https://appliance.company.com'
# username='user@example.local'
# password='example_password'
```

   Get your organization ID.

```
# curl -X GET \
 "$url/csp/gateway/am/api/loggedin/user/orgs" \
 -H "csp-auth-token: $access_token"
```

   The response shows the organization ID.

```
{
 "refLinks": [
   "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30"
 ],
 "items": [
   {
     "name": "DEFAULT-ORG",
     "displayName": "DEFAULT-ORG",
     "refLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30",
     "id": "7f8c518a-65f5-494b-b714-f7e349957a30",
     "metadata": null,
```

```
      "parentRefLink": null
    }
  ]
}
```

Assign the organization ID variable.

```
# org_id='7f8c518a-65f5-494b-b714-f7e349957a30'
```

Verify the organization role.

```
# curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/roles \
  -H "csp-auth-token: $access_token" | jq "."
```

The response shows that the organization role is `org_owner`.

```
{
   "refLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30/
roles/52a6a411-2339-4bc3-91bc-62418977df11",
   "name": "org_owner",
   "displayName": "Organization Owner",
   "organizationLink": "/csp/gateway/am/api/orgs/7f8c518a-65f5-494b-b714-f7e349957a30"
 }
```

Verify the service role.

```
# curl -X GET \
  $url/csp/gateway/am/api/loggedin/user/orgs/$org_id/service-roles \
  -H "csp-auth-token: $access_token" | jq "."
```

A snippet of the response shows the Service Role Names for theVCF Automation service. `cloud_admin` satisfies the minimum service role.

```
...
{
     "serviceDefinitionLink": "/csp/gateway/slc/api/definitions/external/<service_id>",
     "serviceRoleNames": [
       "automationservice:cloud_admin"
     ]
   }
...
```

Prerequisites for API Use Case Examples

Before performing any task using API services, you must review the prerequisites. General prerequisites apply to all services. Prerequisites that are service-specific are common to all endpoints for the service, but may vary depending upon the service role for example, administrator or user.

General prerequisites for all services

Before performing any task for any service, the following prerequisites must be satisfied:

- Verify that you have an active access token. See Getting Your Authentication Token.
- Verify that the URL variable is assigned.

```
url='https://appliance.domain.com'
```

Prerequisites specific to API services

The following table lists prerequisites that are specific to the services with use cases in this guide.

> **Note:**
> Every service includes an API version variable. If you choose not to assign a value to the *`apiVersion`* or to assign it to a different value, review the information in https://techdocs.broadcom.com/us/en/vmware-cis/aria/aria-automation/8-18/aria-automation-api-programming-guide-on-prem-8-18.html.

Table 1937: Prerequisites for VCF Automation Service Use Cases

Product: Service	Service prerequisites	Role-specific prerequisites
VCF Automation: Blueprint service To access the Swagger documentation for the Blueprint API, see `https://<your_FQDN>/blueprint/api/swagger/ui/`.	Verify that the Blueprint API version variable is assigned as in the following example. `api_version='2019-09-12 '`	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Catalog service To access the Swagger documentation for the Catalog API, see `https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=catalog`.	Verify that the Catalog API version variable is assigned as in the following example. `api_version='2020-08-25 '`	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Deployment service To access the Swagger documentation for the Deployment API, see `https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=deployments`. **Note:** For the VCF Automation: Deployment service, use the prerequisites for the VCF Automation: Infrastructure as a Service (IaaS) service.	Verify that the Deployment API version variable is assigned as in the following example. `api_version='2020-08-25 '`	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
VCF Automation: Infrastructure as a Service (IaaS) service To access the Swagger documentation for the IaaS API, see `https://<your_FQDN>/iaas/api/swagger/ui/`.	Verify that the IaaS API version variable is assigned as in the following example. `api_version=' 2021-07-15'` If you do not assign a value to the *`apiVersion`* variable or you assign it to a value other than 2021-07-15 , the IaaS API behavior will default to value of the previous version or 2019-01-15.	• Verify that you are an organization owner in VCF Automation with a VCF Automation administrator service role. • If working with cloud accounts, verify that you have cloud administrator credentials. .
All products: Onboarding service To access the Swagger documentation for the Onboarding API, see `https://<your_FQDN>/relocation/api/swagger/swagger-ui.html`	No version variable needed	Verify that you are at least an organization member in VCF Automation with an VCF Automation administrator service role.
VCF Automation: Pipelines service To access the Swagger documentation for the Pipelines API, see `https://<your_FQDN>/pipeline/api/swagger/swagger-ui.html`.	Verify that the Pipelines API version variable is assigned as in the following example. `api_version='2019-10-17 '`	Verify that you are at least an organization member in VCF Automation with an VCF Automation administrator service role.
VCF Automation: Policies service To access the Swagger documentation for the Policies API, see `https://<your_FQDN>/deployment/api/swagger/swagger-ui.html?urls.primaryName=policies`.	Verify that the Policies API version is assigned as in the following example. `api_version='2020-08-25 '`	Verify that you are at least an organization member in VCF Automation with a VCF Automation administrator service role.
All products: Project service To access the Swagger documentation for the Projects API, see `https://<your_FQDN>/project/api/swagger/swagger-ui.html`.	Verify that the Projects API version variable is assigned as in the following example. `api_version='2019-01-15 '`	Verify that you are at least an organization member in VCF Automation with an administrator service role.

VCF Automation Tutorials

As a VCF Automation administrator, you can use these tutorials to learn how to perform tasks programmatically using the IaaS APIs.

In addition to the steps in the tutorials, there is additional information in the guide. Links are provided to relevant topics.

Working with tags

As an Organization administrator with the Assembler admin service role, you can use the Tags endpoints to create, manage, or delete tags in VCF Automation.

Note:

- Tag filters are case-insensitive.
- There is no way to query for unused tags using the IaaS API. However, you can submit a request to check for tag usage of specific tag IDs.

Prerequisites for working with tags

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

Filtering for tags

You can filter for tags based on a key, value, or origin and the tag filters are case-insensitive.

The following example shows how to filter for a tag with `location` in the key name.

```
curl -X GET \
"$url/iaas/api/tags?$filter=key%20eq%20'location’&apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

A sample response shows `"key": "location"`.

```
{
"content": [
   {
       "key": "location",
       "value": "MUM",
       "id": "9aabbe23-d4bf-3d55-89f7-dc7e67317442"
   }
],
"totalElements": 1,
"numberOfElements": 1
}
```

To filter for the origin of a tag, you specify `USER_DEFINED` or `DISCOVERED` for the origin value. The following example filters for tags with the `DISCOVERED` origin.

```
curl -X GET \
"$url/iaas/api/tags?$filter=origins.item%20eq%20%27DISCOVERED%27&apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

Tag origins can also be both `USER_DEFINED` and `DISCOVERED` as in the following cases:

- If a tag's origin is `USER_DEFINED` and another tag with the same case-sensitive key/value pair is assigned to an endpoint resource from which VCF Automation collects data, VCF Automation adds the `DISCOVERED` origin.
- If a user assigns a tag with the `DISCOVERED` origin to any VCF Automation resource such as a network profile, VCF Automation also assigns the `USER_DEFINED` origin to the tag.

Creating tags

You create a tag with a name and an optional value. The following example shows how to create a tag named `environment` with the value `prod` that you could add to resources in your production environment.

```
curl -X POST \
"$url/iaas/api/tags?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
-d '{
"key" : "environment",
"value" : "prod"
}' | jq "."
```

The response provides the tag ID.

```
{
"key": "environment",
"value": "prod",
"id": "ca0ed5ef-72b5-3d9f-bc6d-0707defff540"
}
```

Listing and deleting tags

To delete a tag, you specify the tag ID. To get the tag ID, you list all tags in your infrastructure.

```
curl -X GET \
"$url/iaas/api/tags?apiVersion=$api_version"
-H 'Content-Type: application/json'
-H "Authorization: Bearer $access_token"
}' | jq "."
```

The response lists all the tag names, values, and IDs as in the following example.

```
{
"content": [
   {
       "key": "location",
       "value": "MUM",
       "id": "9aabbe23-d4bf-3d55-89f7-dc7e67317442"
   }
   {
       "key": "Application",
       "value": "testing",
       "id": "57ead12-523a-3cd2-9447-06e757dcf382"
   }
],
"totalElements": 2,
"numberOfElements": 2
}
```

Before deleting a tag, a best practice is to check the tag usage of the ID that you plan to delete. The following example checks for usage of the tag with ID `9aabbe23-d4bf-3d55-89f7-dc7e67317442`.

```
curl -X POST \
"$url/iaas/api/tags/tags-usage?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"tagIds" : ["9aabbe23-d4bf-3d55-89f7-dc7e67317442"]
}' | jq "."
```

- If the tag is being used, the response shows where it is being used. This sample response shows that the tag is being used in a network profile.

```
{
"documentLinks": [
   "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4"
],
"documents": {
   "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4": {
       "id": "a30929ec-3b06-484e-8785-cc43619b5b01",
       "name": "23360",
       "customProperties": {
           "edgeClusterRouterStateLink": "/resources/routers/2d57f1c4-fbe0-46ce-a061-3a5f1396f37b",
           "tier0LogicalRouterStateLink": "/resources/routers/9c965abe-9d69-40c0-99e0-be5aa7e52097",
           "onDemandNetworkIPAssignmentType": "mixed"
       },
       "tagLinks": [
           "/resources/tags/ef254637-594d-3960-a642-f157f859a830",
           "/resources/tags/9aabbe23-d4bf-3d55-89f7-dc7e67317442"
       ],
       "expandedTags": [
           {
               "tag": "23360\n"
           },
           {
               "tag": "location\nMUM"
           }
       ],
       "documentVersion": 11,
       "documentKind": "com:vmware:admiral:compute:profile:NetworkProfileService:NetworkProfile",
       "documentSelfLink": "/provisioning/resources/network-profiles/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
       "documentUpdateTimeMicros": 1696947968305000,
       "documentExpirationTimeMicros": 0
   }
},
"documentCount": 1,
"documentVersion": 0,
"documentUpdateTimeMicros": 0,
"documentExpirationTimeMicros": 0}
```


- If the tag is not being used, the response shows no resource links.


```
{
"documentLinks": [],
"documents": {},
"documentCount": 0,
"documentVersion": 0,
"documentUpdateTimeMicros": 0,
"documentExpirationTimeMicros": 0
}
```


To delete the unused tag, specify the tag ID. The delete action first checks to ensure that the tag is not associated with a resource before deleting it. The following example deletes the tag with ID `9aabbe23-d4bf-3d55-89f7-dc7e67317442`.

```
curl -X DELETE \
"$url/iaas/api/tags/9aabbe23-d4bf-3d55-89f7-dc7e67317442?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token"  \
}' | jq "."
```

Note:
To delete a tag that is associated with a resource, you can use the `ignoreUsage` query parameter as in the following example that forces the deletion of a tag with ID `9aabbe23-d4bf-3d55-89f7-dc7e67317442`. However, this removes all tag assignments from all the resources associated with the tag.

```
curl -X DELETE \
"$url/iaas/api/tags/9aabbe23-d4bf-3d55-89f7-dc7e67317442?ignoreUsage=true&apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token"  | jq "."
```

Updating tags

You can use the IaaS API to add, remove, or update the tags associated with a resource. Using the ID of the resource, you submit a request get the details about the resource including the tag definitions. Then you submit a request to update the resource with new tag key/value pairs.

The following example shows how to update tags for a vSphere datastore

List all vSphere datastores.

```
curl -X GET \
"$url/iaas/api/fabric-vsphere-datastores?apiVersion=$api_version"
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the ID of the datastore that you want to use and use the ID to get details about the datastore. The following request uses the datastore ID `002e0a62-846d-4fb2-a153-3dcd80e57ba9`.

```
curl -X GET \
"$url/iaas/api/fabric-vsphere-datastores/002e0a62-846d-4fb2-a153-3dcd80e57ba9/?apiVersion=$api_version"
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response lists the tags on the datastore.

```
...
"tags": [
   {
       "key": "Team",
       "value": "E2E"
   },
   {
       "key": "Owner/Requestor",
       "value": "User1"
   },
   {
       "key": "Product",
       "value": "vRA"
   }
],
...
```

Update the tags for the datastore. The following example payload:

- Removes the `Team` and `Owner/Requestor` keys.
- Adds the `environment` key.
- Updates the `Product` key.

```
curl -X PATCH \
"$url/iaas/api/fabric-vsphere-datastore/002e0a62-846d-4fb2-a153-3dcd80e57ba9?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d {
```

```
"tags": [
    {
        "key": "environment",
        "value": "dev"
    },
    {
        "key": "Product",
        "value": "saltstack"
    }
],
}' | jq "."
```

Create a Kubernetes Zone with a Tag

As a VCF Automation admin, you can use APIs to create a new tag and add it to a Kubernetes zone.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for a vSphere cloud account. See Add avSphere Cloud Account .

In the following procedure, you use the IaaS API to create a new tag. Then using the CMX API, you create a Kubernetes zone with a supervisor namespace on vSphere and assign the tag to the zone.

> **Note:**
>
> To create a Kubernetes zone with a vSphere cloud account, the account must be Tanzu-enabled. When you open a Tanzu-enabled vSphere cloud account in the UI, the cloud account appears with the label **Available for Kubernetes deployment**.

1. Create a new tag with a key/value pair.

   ```
   curl -X POST \
     $url/iaas/api/tags?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "key": "'<your_tag_key>'",
       "value": "'<your_tag_value>'"
     }' | jq "."
   ```

   The response includes a tag ID.

2. Assign the tag ID variable.

   ```
   tag_id='<example-tagID-alphanumeric-string>'
   ```

3. Assign your vSphere cloud account ID to the cloud account ID variable.

   ```
   cloud_account_id='<vsphere_cloud_account_ID>'
   ```

4. Use the cloud account ID to create the Kubernetes zone with the tag.

   To associate the Kubernetes zone with a vSphere cloud account you specify `"providerType": "VSPHERE_NAMESPACES"`.

   ```
   curl -X POST \
     "$url/cmx/api/resources/k8s-zones" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name": "<your_k8s_zone_name>",
       "providerId": "'$cloud_account_id'",
       "providerType": "VSPHERE_NAMESPACES",
       "tagIds": [
           "'$tag_id'"
   ```

```
    ]
}' | jq "."
```

Create a Kubernetes Zone with a Tag

Create a new tag with key `example_tag_key` and value `example_tag_value`. Create a Kubernetes zone named `K8s-test-zone` with the tag. To create a Kubernetes zone, you must associate it with a cloud account configured for VCF Automation such as a vSphere cloud account with ID `8d4646bd-d629-4526-9009-10e3d4b66e44`.

Assign variables.

```
url='https://appliance.domain.com'
$ api_version=' '
```

Create a new tag with the key/value.

```
$ curl -X POST \
  "$url/iaas/api/tags?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "key": "example_tag_key",
    "value": "example_tag_value"
  }' | jq "."
```

Examine the response for the tag ID.

```
{
    "key": "example_tag_key",
    "value": "example_tag_value",
    "id": "a6f324e4-101c-33f6-ac51-d9aaaa020123"
}
```

Assign a value to the tag ID variable.

```
$ tag_id1= 'a6f324e4-101c-33f6-ac51-d9aaaa020123'
```

Assign your vSphere cloud account ID to the cloud account ID variable.

```
$ cloud_account_id='8d4646bd-d629-4526-9009-10e3d4b66e44'
```

Create the Kubernetes zone with the tag and the cloud account ID.

```
$ curl -X POST \
  "$url/cmx/api/resources/k8s-zones" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "k8s-test-zone",
    "providerId": "'$cloud_account_id'",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
        "'$tag_id1'"
    ]
}' | jq "."
```

The response shows the Kubernetes zone.

```
{
    "id": "220a81ba-c8ca-4f01-97bc-6f497576d564",
    "createdMillis": 1674847375758,
    "updatedMillis": 1674847375758,
    "orgId": "20451685-2bf6-4ba4-9ea4-87ee61bd32f8",
    "name": "k8s-test-zone",
```

```
    "providerId": "8d4646bd-d629-4526-9009-10e3d4b66e44",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
        "a6f324e4-101c-33f6-ac51-d9aaaa020123"
    ]
}
```

If you want to update the Kubernetes zone, assign the Kubernetes zone ID variable.

```
$ k8s_zone_id='220a81ba-c8ca-4f01-97bc-6f497576d564'
```

If you want to add another tag, assign another tag ID variable.

```
$ tag_id2='21ccbdd6-c849-37f3-a784-1f2452cf802d'
```

Update the Kubernetes zone to add the tag.

Note:
When updating the zone, the update deletes any tags that are not included in the request payload. So when making the request, you must include all the tags that will remain assigned to the zone after the update.

```
$ curl -X PUT \
  "$url/cmx/api/resources/k8s-zones/$k8s_zone_id" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "id": "220a81ba-c8ca-4f01-97bc-6f497576d564",
    "createdMillis": 1674847375758,
    "updatedMillis": 1674847375758,
    "orgId": "20451685-2bf6-4ba4-9ea4-87ee61bd32f8",
    "name": "k8s-test-zone",
    "providerId": "'$cloud_account_id'",
    "providerType": "VSPHERE_NAMESPACES",
    "tagIds": [
        "'$tag_id1'",
        "'$tag_id2'"
    ]
}' | jq "."
```

How do I retrieve provisioning request details

To validate placement scenarios before deploying a blueprint, you use the IaaS API to retrieve the request graph for a simulated provisioning request.

Prerequisites for retrieving provisioning request details

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Assign an API version variable for the Blueprint API.

  ```
  api_version_blueprint='2019-09-12'
  ```

  Note:
  The VCF Automation Infrastructure as a Service (IaaS) service and the VCF Automation Blueprint service have different API version values. You set the API version value for the VCF Automation Infrastructure as a Service (IaaS) service when you satisfied the general prerequisites.
- Verify that you have the ID for the blueprint that you plan to deploy. You can also get the blueprint ID from the UI, by opening the blueprint listed on **Design** > **Templates** and inspecting the ID at the end of the URL.

Retrieving provisioning request details

This example shows how to get request details for a blueprint with the ID `c8197446-d636-4ed9-aa2b-796da98ad10c`. In the following procedure, you use the blueprint API to get the deployment ID and flow ID and provide those values as input for the IaaS API request graph endpoint.

Use the template ID to get the deployment ID and flow ID.

```
curl -X POST \
$url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
    "deploymentId": null,
    "deploymentName": null,
    "description": null,
    "plan": false,
    "blueprintId": "c8197446-d636-4ed9-aa2b-796da98ad10c",
    "content": null,
    "simulate": true
} | jq "."
```

Examine the response to get the deployment ID and the flow ID.

```
{
    "id": "8b918325-21a7-4902-b464-f39f5fb1bb64",
    "createdAt": "2023-11-22T07:12:54.585430Z",
    "createdBy": "user@mycompany.com",
    "updatedAt": "2023-11-22T07:12:54.585430Z",
    "updatedBy": "user@mycompany.com",
    "orgId": "434f6917-4e34-4537-b6c0-3bf3638a71bc",
    "projectId": "267f8448-d26f-4b65-b310-9212adb3c455",
    "projectName": "testing",
    "deploymentId": "924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
    "requestTrackerId": "8b918325-21a7-4902-b464-f39f5fb1bb64",
    "deploymentName": "deployment_924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
    "reason": "Simulate",
    "plan": false,
    "destroy": false,
    "ignoreDeleteFailures": false,
    "simulate": true,
    "blueprintId": "c8197446-d636-4ed9-aa2b-796da98ad10c",
    "inputs": {},
    "status": "STARTED",
    "flowId": "8b918325-21a7-4902-b464-f39f5fb1bb64"
}
```

Use the deployment ID and flow ID to get the provisioning request details.

```
curl -X GET \
$url/iaas/api/request-graph?apiVersion=$api_version \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
    "deploymentId": "924fcbc3-2076-48c9-a5f5-64de7f1cbe3a",
    "flowId": "8b918325-21a7-4902-b464-f39f5fb1bb64",
} | jq "."
```

Using the response to validate placement scenarios

The response provides task and project information for the simulated provisioning request.

The tasks section of the response lists the task ID and stages. To validate a placement scenario, the request must successfully transition through all stages and substages. Any stage that shows an error includes a failure message that you can use to troubleshoot the problem with the placement scenario.

In the following sample response, a snippet shows an error during the reserving stage. To validate the provisioning request, correct errors and rerun the API until all tasks finish successfully.


```
"tasks": [
    {
        "id": "46414006-8f71-46f8-b965-eea0bacbba7e",
        "stages": [
            {
                "transitionSource": {
                    "timestampMicros": 0
                },
                "taskSubStage": "CREATED",
                "taskInfo": {
                    "stage": "STARTED",
                    "isDirect": false
                },
                "resourceLinks": [
                    "/iaas/api/machines/4323fd90-ad4e-4342-b9ca-62a9bb7193c9"
                ],
                "timestampMicros": 1702877437358001
            },
            ...
                },
                "taskSubStage": "ENHANCED",
                "taskInfo": {
                    "stage": "STARTED",
                    "isDirect": false
                },
            ...
                },
                "taskSubStage": "RESOURCE_COUNTED",
                "taskInfo": {
                    "stage": "STARTED",
                    "isDirect": false
                },
            ...
                },
                "taskSubStage": "RESERVING",
                "taskInfo": {
                    "stage": "STARTED",
                    "isDirect": false
                },
            ...
                "taskSubStage": "ERROR",
                "taskInfo": {
                    "stage": "STARTED",
                    "isDirect": false,
                    "failure": {
                        "message": "Cannot find matching image mappings for image: im1",
                        "statusCode": 0,
                        "errorCode": 0,
                        "serverErrorId": "ad4d91ea-4319-4f11-abcf-3601c08f3ecd",
                        "documentKind": "com:vmware:xenon:common:ServiceErrorResponse"
                    }
                },
```

```
            ...
        }
    ]
```

The project section of the response provides reference information for the project associated with the template.

```
"project": {
    "administrators": [],
    "members": [],
    "viewers": [],
    "supervisors": [],
    "zones": [
        {
            "zoneId": "8830680d-71d1-4dcb-8234-f22de4254c6b",
            "priority": 0,
            "maxNumberInstances": 0,
            "allocatedInstancesCount": 0,
            "memoryLimitMB": 0,
            "allocatedMemoryMB": 0,
            "cpuLimit": 0,
            "allocatedCpu": 0,
            "storageLimitGB": 0,
            "allocatedStorageGB": 0.0,
            "id": "8a342007-8e8a-432b-9a12-14ebb51e3c1a-8830680d-71d1-4dcb-8234-f22de4254c6b"
        }
    ],
    "constraints": {},
    "operationTimeout": 0,
    "sharedResources": true,
    "placementPolicy": "DEFAULT",
    "customProperties": {},
    "name": "p",
    "description": "",
    "id": "8a342007-8e8a-432b-9a12-14ebb51e3c1a",
    "orgId": "ce1fb992-7495-48fd-8988-412658094f6b",
    "_links": {
        "self": {
            "href": "/iaas/api/request-graph/8a342007-8e8a-432b-9a12-14ebb51e3c1a"
        }
    }
}
```

How do I list and edit zones associated with a project

To query and edit zones associated with a project, you use Project endpoints in the IaaS API.

Prerequisites for extracting zones associated with a project

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have your project ID. If you do not have the ID, list all projects to find the name and ID of the project with the associated zones that you want to query or edit.

  ```
  curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects?apiVersion=$api_version" | jq "."
  ```

  Examine the response to find your project name and ID as in the following example snippet.

  ```
  ...
      ],
      "name": "project1",
  ```

```
        "description": "",
        "id": "6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9",
        "orgId": "f098d692-e980-41a5-b349-83084fce1ea0",
    ...
```

Querying zones associated with a project

You can retrieve the first 100 cloud zones associated with a project without including a query option. To retrieve information about cloud zones that are not among the first 100 listed, you add query options to your request.

For a complete list of query options, see Querying with the VCF Automation APIs.

This example assumes that you have more than 100 zones associated with the project ID `6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9`. The following procedure shows how to use paging to get the second page of zones associated with the project.

For more information about pagination parameters, see Using Pagination and Count.

Assign the project ID variable.

```
project_id='6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9'
```

Append query options `top=100` and `skip=100` to the request to retrieve cloud zones.

```
curl -X GET \
"$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version&$top=100&skip=100" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
| jq "."
```

The following response lists the second page of zones. Since there are 102 zones associated with the project, the second page lists two zones.

```
"content": [
   {
      "zoneId": "3cf514d6-0dfc-4941-95de-40b01d60e8d3",
      "priority": 0,
      "maxNumberInstances": 0,
      "allocatedInstancesCount": 2,
      "memoryLimitMB": 0,
      "allocatedMemoryMB": 1254,
      "cpuLimit": 0,
      "allocatedCpu": 2,
      "gpuLimit": 0,
      "allocatedGpu": 0,
      "storageLimitGB": 0,
      "allocatedStorageGB": 0.0,
      "id": "0966203d-63f5-41c7-8dcd-7c1833932ec4-3cf514d6-0dfc-4941-95de-40b01d60e8d3",
      "updatedAt": "2021-10-28",
      "orgId": "ce811934-ea1a-4f53-b6ec-465e6ca7d126",
      "_links": {
         "project": { "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9" }
         }
   }   ,
   {
      "zoneId": "e4c56d64-a5bc-4656-bfc6-9f8009af66d3",
      "priority": 0,
      "maxNumberInstances": 0,
      "allocatedInstancesCount": 0,
      "memoryLimitMB": 0,
      "allocatedMemoryMB": 0,
      "cpuLimit": 0,
```

```
      "allocatedCpu": 0,
      "gpuLimit": 0,
      "allocatedGpu": 0,
      "storageLimitGB": 0,
      "allocatedStorageGB": 0.0,
      "id": "0966203d-63f5-41c7-8dcd-7c1833932ec4-e4c56d64-a5bc-4656-bfc6-9f8009af66d3",
      "updatedAt": "2022-01-07",
      "orgId": "ce811934-ea1a-4f53-b6ec-465e6ca7d126",
      "_links": {
         "project":{ "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9" }
      }
   }
],
"totalElements": 2,
"numberOfElements": 2
```

Editing cloud zone assignments

You can edit the following cloud zone assignments in your project:

- Storage limit (GB)
- CPU limit
- Memory limit (MB)
- Maximum number of instances
- Provisioning priority

This example assumes that you have two zones associated with the project ID `094a2fab-7715-4844-94f9-71b45452da27`, one with provisioning priority 0 and one with provisioning priority 1. The following procedure shows how to edit the zone assignments to add a new zone with priority 1 and change an existing zone to priority 2.

Assign the project ID variable.

```
project_id='094a2fab-7715-4844-94f9-71b45452da27'
```

List the zones provisioned in your project.

```
curl -X GET \
"$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version&$top=100&skip=100" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
| jq "."
```

A snippet of the response shows two zone IDs:

- `3c2bbe36-bf8e-4484-9c31-ce552422aaf1`
- `8992bdf0-136f-401c-822a-e22dae67259b`

```
...
  }
     "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
     "priority": 0
     "maxNumberInstances": 0,
     "allocatedInstancesCount":0,
     ...
  },
  {
     "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
     "priority": 1
     "maxNumberInstances": 0,
     "allocatedInstancesCount":0,
```

```
      ...
  }
}
...
```

Note:
When updating project zone assignments, the update deletes any cloud zones that are not included in the request payload. So when making the request, you must include zone IDs for all cloud zones that will remain in the project after the update, even if there is no change to the zone assignment specification.

The zone assignment update includes information for three zones in the project:

- For zone ID `3c2bbe36-bf8e-4484-9c31-ce552422aaf1`, maintain the zone with `"priority": 0`.
- For zone ID `8992bdf0-136f-401c-822a-e22dae67259b`, maintain the zone but change to `"priority": 2`.
- For zone ID `66067958-7e43-47f8-9bc9-0d32594c47e9`, add a new zone with `"priority": 1`.

```
curl -X PUT \
  "$url/iaas/api/projects/$project_id/zones?apiVersion=$api_version"
  -H 'Content-Type: application/json'
  -H "Authorization: Bearer $access_token"
  -d '{
   "zoneAssignmentSpecifications": [
        {
            "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
            "priority": 0
        },
        {
            "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
            "priority": 2
        }.
        {
            "zoneId": "66067958-7e43-47f8-9bc9-0d32594c47e9",
            "priority": 1
        }
    }
  }' | jq "."
```

Examine the response.

```
{
"progress": 0,
"status": "INPROGRESS",
"name": "Project Zones Assignment Task",
"id": "a6241aeb-909e-4689-af5a-940b52d216ff",
"selfLink": "/iaas/api/request-tracker/a6241aeb-909e-4689-af5a-940b52d216ff"
}
```

Assign the selflink variable.

```
selfLink_id='a6241aeb-909e-4689-af5a-940b52d216ff'
```

Use the selfLink for tracking.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The zone assignments are complete when the response includes `"status": "FINISHED"`.

```
{
   "progress": 100,
   "status": "FINISHED",
   "name": "Project Zones Assignment Task",
```

```
    "id": "a6241aeb-909e-4689-af5a-940b52d216ff",
    "selfLink": "/iaas/api/request-tracker/a6241aeb-909e-4689-af5a-940b52d216ff"
}
```

If you list the zones provisioned in your project again, a snippet of the response shows the newly added cloud zone and updated provisioning priorities.

```
...
  }
     "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
     "priority": 0
     "maxNumberInstances": 0,
     "allocatedInstancesCount":0,
     ...
  },
  {
     "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
     "priority": 2
     "maxNumberInstances": 0,
     "allocatedInstancesCount":0,
     ...
  }
  {
     "zoneId": "66067958-7e43-47f8-9bc9-0d32594c47e9",
     "priority": 1
     "maxNumberInstances": 0,
     "allocatedInstancesCount":0,
     ...
  }
}
...
```

Deploying and Managing Resources

As a VCF Automation administrator, you can use the IaaS APIs to deploy and manage cloud resources.

Create and Deploy a Machine Resource

To create a new resource such as a VM, you can use the resources API to make a POST request with a project ID. The deployment creates a new resource without using a blueprint.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Assign an API version variable for the Deployment API.

  ```
  api_version_deployment='2020-08-25'
  ```

  Note:
  The VCF Automation Infrastructure as a Service (IaaS) service and the VCF Automation Deployment service have different API version values. You set the API version value for the VCF Automation Infrastructure as a Service (IaaS) service when you satisfied the general prerequisites.
- Verify that you have the ID of the cloud account where you want to deploy the VM. See Adding Cloud Accounts.
- Verify that you know the cloud zone in your cloud account where you want the new VM to be deployed. See Create a Cloud Zone .
- Verify that you have the ID for a project that includes the cloud zone in your cloud account. See Add a Cloud Zone to Your Project.
- Verify that the flavor and image for the new VM exist in your cloud account.

- Verify that you know the resource type for the AWS machine, GCP machine, vSphere machine, or Azure machine that you plan to create. For a list of all resource types and request schema, see the link to the schema that's available from VCF Automation Resource Schema Documentation.

This procedure shows how to provision a VM with a project and a cloud zone assigned to the project. The flavor and image for the VM must exist in your cloud account.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

3. List all cloud zones.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   zones?apiVersion=$api_version"  | jq "."
   ```

4. Examine the response for the zone where you want to place your VM.

   You use the `externalRegionId` to filter for fabric flavor and fabric image. You use the cloud zone ID for VM placement.

5. Assign the external region ID variable.

   ```
   external_region_id='<your_external_region_id>'
   ```

6. Assign the cloud zone placement variable.

   ```
   placement_name='/iaas/api/zone/<your_cloud_zone_id>'
   ```

7. To list the fabric images in your cloud account and zone, specify the cloud account ID and external region ID in the request.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
       "$url/iaas/api/fabric-images/?apiVersion=$api_version&"\
       '$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
       and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id')))" | jq "."
   ```

8. Examine the response to select a fabric image.

9. Assign the fabric image variable.

   ```
   image_name='<your_image_name>'
   ```

10. To list the fabric flavors in your cloud account and zone, specify the cloud account ID and external region ID in the request.

    ```
    curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
        "$url/iaas/api/fabric-flavors/?apiVersion=$api_version&"\
        '$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
        and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id')))" | jq "."
    ```

11. Examine the response to select a fabric flavor.

12. Assign the fabric flavor variable.

    ```
    flavor_name='<your_flavor_name>'
    ```

13. Assign the resource type variable.

    ```
    resource_type='<your_resource_type>'
    ```

14. Create and deploy the VM.

    ```
    curl -X POST \
      $url/deployment/api/resources?apiVersion=$api_version_deployment \
      -H "Authorization: Bearer $access_token" \
      -H "Content-Type: application/json" \
      -d '{
    ```

```
    "name":"<your_resource_name>",
    "projectId": "'$project_id'",
    "type":"'$resource_type'",
    "properties":{
        "imageRef": "'$image_name'",
        "flavor": "'$flavor_name'",
        "placement": "'$placement_name'"
  },
}' | jq "."
```

15. Examine the response for the deployment ID.
16. Assign the deployment ID.
17. Get the status of the deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version_deployment \
  -H "Authorization: Bearer $access_token" | jq "."
```

When the status shows `CREATE_SUCCESSFUL` the VM is deployed.

Create and deploy a VM

With a cloud account ID and cloud zone in the cloud account, create and deploy an AWS machine named `cloud_machine_1` using the `Cloud.AWS.EC2.Instance` resource type.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_deployment=' '
$ cloud_account_id='14e6b70c-0e76-4c5e-bb61-e6d70a5b43ef'
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

List all cloud zones.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/zones?apiVersion=$api_version"  | jq "."
```

A snippet of the response shows the cloud account ID with the external region ID and cloud zone ID.

```
...
      "externalRegionId": "eu-west-1",
      "cloudAccountId": "14e6b70c-0e76-4c5e-bb61-e6d70a5b43ef",
      "name": "AWS / eu-west-1-changed",
      "description": "test description",
      "id": "f32a30fd-67ac-43e3-9512-60cf6ef7bee8"
...
```

Assign the external region ID variable.

```
$ external_region_id='eu-west-1'
```

Assign the cloud zone placement variable.

```
$ placement_name='/iaas/api/zone/f32a30fd-67ac-43e3-9512-60cf6ef7bee8'
```

To list the fabric images in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
    "$url/iaas/api/fabric-images/?apiVersion=$api_version&"\
    '$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
    and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id')))" | jq "."
```

Examine the response to find the image ID that you want.

```
...
      "externalRegionId": "eu-west-1",
      "isPrivate": false,
      "externalId": "ami-9a9012e9",
...
```

To list the fabric flavors in your cloud account and zone, specify the cloud account ID and external region ID in the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" \
    "$url/iaas/api/fabric-flavors/?apiVersion=$api_version&"\
    '$filter'="(((externalRegionId eq '') or (externalRegionId eq '$external_region_id')) \
    and ((cloudAccountId ne '*') or cloudAccountId eq '$cloud_account_id')))" | jq "."
```

Examine the response to find the flavor ID that you want.

```
...          {
      "id": "x1e.xlarge",
      "name": "x1e.xlarge",
      "cpuCount": 4,
      "memoryInMB": 124928,
      "storageType": "ssd",
      "dataDiskSizeInMB": 122880,
      "dataDiskMaxCount": 1,
      "networkType": "Up to 10 Gigabit"
    },
...
```

Assign the resource type for the VM.

```
$ resource_type = 'Cloud.AWS.EC2.Instance'
```

To deploy the VM, assign variables for image and flavor.

```
$ image_name='ami-9a9012e9'
$ flavor_name='x1e.xlarge'
```

Create and deploy the VM.

```
curl -X POST \
  $url/deployment/api/resources?apiVersion=$api_version_deployment \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "name":"cloud_machine_1",
    "projectId": "'$project_id'",
    "type":"'$resource_type'",
    "properties":{
        "imageRef": "'$image_name",
        "flavor": "'$flavor_name",
        "placement": "'$placement_name'"
  },
}' | jq "."
```

The response shows the deployment ID.

```
{
    "deploymentId": "fccd2081-fd44-44c8-878c-f962ef71969a",
    "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
    "requestId": "1f8f2e4f-0b2e-448d-8439-f1a05b1e90c1",
    "resourceId": "9a510ccb-0543-47e8-a2f2-f1f65fcd0b0a"
}
```

Assign the deployment ID variable.

```
$ deployment_id='fccd2081-fd44-44c8-878c-f962ef71969a'
```

Get the status of the deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version"
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the deployment status.

```
...
  ],
  "status": "CREATE_SUCCESSFUL"
}
```

Managing IP Addresses

As an VCF Automation administrator, you can reserve IP addresses so that VCF Automation does not use them for deployment. If you no longer need them, you can release them.

Before reserving IP addresses, you must verify that available IP addresses exist within a network IP range.

Query for IP Addresses

When you use VCF Automation to create a deployment, VCF Automation allocates IP addresses to manage resources. To identify the IP addresses that have been allocated, you query the network IP range. You can only reserve IP addresses that have not been allocated.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

You create network IP ranges with a range of IP addresses. The following procedure shows how to determine the IP addresses in a range that are available.

1. To get the ID of the network range that you want to use, list the internal network IP ranges in your deployment.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   network-ip-ranges?apiVersion=$api_version"  | jq "."
   ```

   Examine the response to find the ID of the network IP range that you want to use.

2. Assign a variable for the network IP range ID.

   ```
   ip_range_id='<your_network_ip_range_id>'
   ```

3. To query IP ranges associated with a particular network and get information such as IP availability, you can use one of the following commands:

   - If you have the network IP range ID, use:

     ```
     curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
     api/network-ip-ranges/$ip_range_id?apiVersion=$api_version"  | jq "."
     ```

   - If you have the network ID, use:

     ```
     curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
     api/networks/<your_network_id>/network-ip-ranges?apiVersion=$api_version"  | jq "."
     ```

   - If you have the networking fabric ID, use:

     ```
     curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
     api/fabric-networks/<your_fabric_network_id>/network-ip-ranges?apiVersion=$api_version"  | jq "."
     ```

   - If you have the networking fabric for vSphere ID, use:

     ```
     curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
     api/fabric-networks-vsphere/<your_fabric_network_vsphere_id>/network-ip-ranges?apiVersion=$api_version"
       | jq "."
     ```

   Examine the response to see the number of allocated and available IP addresses. If the number of available IP addresses is too low, check another network IP range until you find one with enough IP addresses to fit your needs.

4. Use the network IP range ID to query the status of IP addresses within the range.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
network-ip-ranges/$ip_range_id/ip-addresses?apiVersion=$api_version"  | jq "."
```

Examine the result. All IP addresses appear with `"ipAddressStatus": "ALLOCATED"` or `"ipAddressStatus": "RELEASED"` and are not available.

Query for IP addresses within an IP range

The example query requests and responses show how to find allocated IP addresses.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=''
```

List the internal network IP ranges in your deployment.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges?apiVersion=$api_version"  | jq "."
```

A snippet of the response shows the network IP ranges with their IDs.

```
...
   "startIPAddress": "fe45::2",
   "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
   "endIPAddress": "fe45::fffe",
   "createdAt": "2023-02-28",
   "ipVersion": "IPv6",
   "name": "rangeIPv6",
   "id": "703a02df-3b6d-4ad7-b146-582b398055f2",
...
```

Assign the variable for the network IP range ID.

```
$ ip_range_id='703a02df-3b6d-4ad7-b146-582b398055f2'
```

Query the IP range associated with the network IP range ID.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges/$ip_range_id?apiVersion=$api_version"  | jq "."
```

A snippet of the response shows the number of available and allocated IPv6 addresses. It also shows the IP version and the start and end of the IP address sequence.

```
{
 "numberOfAllocatedIPs": 2,
 "numberOfAvailableIPs": 65531,
 "numberOfReleasedIPs": 0,
 "totalNumberOfIPs": 65533,
 "numberOfUserAllocatedIPs": 0,
 "numberOfSystemAllocatedIPs": 2,
 "startIPAddress": "fe45::2",
 "endIPAddress": "fe45::fffe",
 "ipVersion": "IPv6",
 "name": "rangeIPv6",
 "id": "703a02df-3b6d-4ad7-b146-582b398055f2",
 "createdAt": "2023-02-28",
 "updatedAt": "2023-02-28",
 "organizationId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
 "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
 "_links": {
     "self": {
         "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2"
```

```
    }
  }
}
```

The following example shows how to query of IP ranges associated with fabric network ID `33c2bbb5-5b26-4a5a-87c6-9e96db72451d`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d/network-ip-ranges?apiVersion=
$api_version"  | jq "."
```

The response shows two IP ranges with details about associated IPv4 addresses.


```
"content": [
    {
        "numberOfAllocatedIPs": 0,
        "numberOfAvailableIPs": 10,
        "numberOfReleasedIPs": 0,
        "totalNumberOfIPs": 10,
        "numberOfUserAllocatedIPs": 0,
        "numberOfSystemAllocatedIPs": 0,
        "startIPAddress": "10.10.10.1",
        "endIPAddress": "10.10.10.10",
        "ipVersion": "IPv4",
        "name": "range-1",
        "id": "578ec19a-2b52-497e-8777-029b57d685ab",
        "createdAt": "2023-03-15",
        "updatedAt": "2023-03-29",
        "orgId": "89db05f7-1b93-4bd8-b395-1772d50813a4",
        "_links": {
            "fabric-networks": {
                "hrefs": [
                    "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d",
                    "/iaas/api/fabric-networks/51f1ed6c-3db3-4877-857d-2bcc84f81897"
                ]
            },
            "self": {
                "href": "/iaas/api/network-ip-ranges/578ec19a-2b52-497e-8777-029b57d685ab"
            }
        }
    },
    {
        "numberOfAllocatedIPs": 0,
        "numberOfAvailableIPs": 10,
        "numberOfReleasedIPs": 0,
        "totalNumberOfIPs": 10,
        "numberOfUserAllocatedIPs": 0,
        "numberOfSystemAllocatedIPs": 0,
        "startIPAddress": "10.10.10.11",
        "endIPAddress": "10.10.10.20",
        "ipVersion": "IPv4",
        "name": "range-2",
        "id": "95a0053c-b57f-4500-a59b-970042f4ce8c",
        "createdAt": "2023-03-29",
        "updatedAt": "2023-03-29",
        "orgId": "89db05f7-1b93-4bd8-b395-1772d50813a4",
        "_links": {
            "fabric-networks": {
                "hrefs": [
```

```
                    "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d"
                ]
            },
            "self": {
                "href": "/iaas/api/network-ip-ranges/95a0053c-b57f-4500-a59b-970042f4ce8c"
            },
            "fabric-network": {
                "href": "/iaas/api/fabric-networks/33c2bbb5-5b26-4a5a-87c6-9e96db72451d"
            }
        }
    }
 ],
 "totalElements": 2,
 "numberOfElements": 2
}
```

If the IP range has a sufficient number of available IP addresses, get more information about the IP addresses in that range.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges/$ip_range_id/ip-addresses?apiVersion=$api_version"  | jq "."
```

The response provides details about the unavailable IP addresses including their IP version and how they were allocated.

- `"ipAllocationType": "SYSTEM"` indicates that an IP address was automatically allocated for use in a deployment.
- `"ipAllocationType": "USER"` indicates that an IP address was manually allocated for use by a user.

```
{
 "content": [
     {
         "ipAddress": "fe4500:10:118:136:fcd8:d68d:9701:4440",
         "ipAddressDecimalValue": 168561938,
         "ipVersion": "IPv6",
         "ipAddressStatus": "ALLOCATED",
         "ipAllocationType": "SYSTEM",
         "id": "e83d9a43-ea2d-428c-ae75-da396c1bb205",
         "createdAt": "2023-02-21",
         "updatedAt": "2023-02-21",
         "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
         "_links": {
             "network-ip-range": {
                 "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a"
             },
             "self": {
                 "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-
addresses/e83d9a43-ea2d-428c-ae75-da396c1bb205"
             },
             "connected-resource": {
                 "href": "/iaas/api/machines/83ede26c-656a-4a08-8716-bea29c21d3f4/network-
interfaces/051d92e6-5729-495d-ad66-9e443e5747c8"
             }
         }
     },
     {
         "ipAddress": "fe45:10:118:136:fcd8:d68d:9701:4450",
         "ipAddressDecimalValue": 168561941,
```

```
            "ipVersion": "IPv6",
            "ipAddressStatus": "ALLOCATED",
            "ipAllocationType": "SYSTEM",
            "id": "439f9c9b-2b2f-484d-8867-2d7b541ddeec",
            "createdAt": "2023-02-22",
            "updatedAt": "2023-02-22",
            "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
            "_links": {
                "network-ip-range": {
                    "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a"
                },
                "self": {
                    "href": "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-
addresses/439f9c9b-2b2f-484d-8867-2d7b541ddeec"
                },
                "connected-resource": {
                    "href": "/iaas/api/machines/81633fba-e86b-4bfa-a06f-3c4f7a754568/network-
interfaces/1c26b194-2862-4d7e-be9c-9881d6cdb871"
                }
            }
        }
    ],
    "totalElements": 2,
    "numberOfElements": 2
}
```


You can allocate any of the IP addresses in the network range except `fe4500:10:118:136:fcd8:d68d:9701:4440` and `fe45:10:118:136:fcd8:d68d:9701:4450`.

IP addresses with the status `"ipAddressStatus": "ALLOCATED"` or `"ipAddressStatus": "RELEASED"` are not available for allocation. You can allocate any of the remaining the IP addresses in the range. See Allocate IP Addresses.

Allocate IP Addresses

As a cloud administrator, you can use the network IP range API to allocate the IP addresses that you want to reserve and make them unavailable for deployment by VCF Automation.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the network IP range that includes the IP addresses that you want to reserve. See Query for IP Addresses.
- If you plan to specify the IP addresses from the network IP range, verify that you know the IPv4 or IPv6 addresses. See Query for IP Addresses.
- If you are a cloud administrator and you plan to specify the number of IP addresses from the network IP range, verify that the number is less than or equal to the number of available IP addresses in the range.

To allocate IP addresses, you submit a request with either:

- A list of specific IP addresses.
- The number of IP addresses.

You can also provide an optional description.

Note:
The following restrictions apply to the request:

- No more than 100 IP addresses can be allocated in a single call.
- If listing specific IP addresses, the addresses must all be of the same type. Mixing IPv4 and IPv6 addresses is not supported.

1. Assign the variable for the network IP range ID.

```
ip_range_id='<your_network_ip_range_id>'
```

2. Allocate the IP addresses by providing a list of addresses.

   The following request example allocates two IP addresses.

   - Specify either IPv4 or IPv6 addresses.
   - To release additional IP addresses, provide additional addresses in the payload.

   ```
   curl --location --request POST \
    $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
       "description":"<your_optional_description>",
       "ipAddresses":["<ipv_address_1>", "<ipv_address_2>"]
   }'
   ```

   Examine the response for the self link to track the request.

3. Optional: If you do not list IP addresses, provide the number of IP addresses to allocate.

   ```
   curl --location --request POST \
    $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
       "description":"<your_optional_description>",
       "numberOfIps": "<number_of_IPs>"
   }'
   ```

   Examine the response for the self link to track the request.

4. Assign the selfLink variable.

   ```
   selfLink_id='example-selfLink-alphanumeric-string'
   ```

5. Use the selfLink variable to track the request.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
   ```

   The IPs are allocated when the response includes `"status": "FINISHED"` and provides a list of allocated resources.

   Note:
   If you specified the number of IP addresses to allocate, you query for the details of each resource in the response to get the allocated IP address associated with the resource.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/network-ip-ranges/$ip_range_id/ip-addresses/<ip_address_id>?apiVersion=$api_version"  | jq "."
   ```

Allocate IP addresses within an IP range

Using the network IP range with the IP addresses that you want to reserve, specify the IP addresses to allocate.

- Two IPv4 IP addresses used in this example are: `["196.192.2.19", "196.192.2.20"]`
- Two IPv6 IP addresses used in this example are: `["fe45:10:118:136:fcd8:d68d:9701:8975","fe45:10:118:136:fcd8:d68d:9701:8985"]`

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ ip_range_id='703a02df-3b6d-4ad7-b146-582b398055f2'
```

Allocate IPv4 addresses by providing a list.

```
curl --location --request POST \
 $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
```

```
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
    "description":"Allocate ipv4 addresses for QA test machines",
    "ipAddresses":["196.192.2.19", "196.192.2.20"]
}'
```

Or allocate IPv6 addresses by providing a list.

```
curl --location --request POST \
 $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
    "description":"Allocate ipv6 addresses for QA test machines",
    "ipAddresses":["fe45:10:118:136:fcd8:d68d:9701:8975","fe45:10:118:136:fcd8:d68d:9701:8985"]
}'
```

Or specify the number of IP addresses to allocate, such as four.

```
curl --location --request POST \
 $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/allocate?apiVersion=$api_version \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
    "description":"Allocate four IP addresses for QA test machines",
    "numberOfIps": "4"
}'
```

The response provides the selfLink variable for tracking.

```
{
   "progress": 1,
   "status": "INPROGRESS",
   "name": "IP Address Allocation Task",
   "id": "ab726e31-ed8bfdfb-83d9-4964-b719-8b395c87d2c9",
   "selfLink": "/iaas/api/request-tracker/ab726e31-ed8bfdfb-83d9-4964-b719-8b395c87d2c9"
}
```

Assign the selfLink variable.

```
selfLink_id= 'ab726e31-e909-4bcc-87ef-5139294eb26a'
```

Track the request

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

When the request is complete, the response includes `"status": "FINISHED"`.

- This is an example of the response when two IPv4 or IPv6 addresses are specified and allocated. You know the IP addresses because you provided them in the request.

```
{
   "progress": 100,
   "status": "FINISHED",
   "resources": [
       "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-addresses/6212ac8b-
a885-4416-bf0d-ebb88628bef1",
       "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-addresses/9b65104d-
debd-4830-8296-3cb87509d293"
   ],
   "name": "IP Address Allocation Task",
```

```
    "id": "ab726e31-e909-4bcc-87ef-5139294eb26a",
    "selfLink": "/iaas/api/request-tracker/ab726e31-e909-4bcc-87ef-5139294eb26a"
}
```

- This is an example of the response when four IP addresses are requested and allocated.

```
{
    "progress": 100,
    "status": "FINISHED",
    "resources": [
        "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-addresses/
a4cceea7-3a94-439b-99fa-420774e61eba",
        "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-
addresses/4c492669-5ec2-4465-992b-c9c252edf052",
        "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-
addresses/0ce9ee20-1a40-460d-80df-fb1791fbd42a",
        "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-addresses/
d7d876b6-92b1-4530-a305-d586c8474f11"
    ],
    "name": "IP Address Allocation Task",
    "id": "ab726e31-e909-4bcc-87ef-5139294eb26a",
    "selfLink": "/iaas/api/request-tracker/ab726e31-e909-4bcc-87ef-5139294eb26a"
}
```

To get an IP address, query for the details of an allocated resource. The following example queries for the details of the resource with IP address ID `a4cceea7-3a94-439b-99fa-420774e61eba`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/a4cceea7-3a94-439b-99fa-420774e61eba?
apiVersion=$api_version"  | jq "."
```

The response provides details about the resource including the allocated IP address and IP version. `"ipAllocationType": "USER"` indicates that an VCF Automation administrator manually reserved the IP address.

```
{
            "ipAddress": "196.192.2.20",
            "ipAddressDecimalValue": 168561941,
            "ipVersion": "IPv4",
            "ipAddressStatus": "ALLOCATED",
            "ipAllocationType": "USER",
            "id": "439f9c9b-2b2f-484d-8867-2d7b541ddeec",
            "createdAt": "2023-02-22",
            "updatedAt": "2023-02-22",
            "orgId": "8040fbde-5ff0-41f3-a8a1-9e25a3311be2",
            "_links": {
                "network-ip-range": {
                    "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2"
                },
                "self": {
                    "href": "/iaas/api/network-ip-ranges/703a02df-3b6d-4ad7-b146-582b398055f2/ip-
addresses/439f9c9b-2b2f-484d-8867-2d7b541ddeec"
                },
                "connected-resource": {
                    "href": "/iaas/api/machines/81633fba-e86b-4bfa-a06f-3c4f7a754568/network-
interfaces/1c26b194-2862-4d7e-be9c-9881d6cdb871"
                }
            }
        }
    ]
}
```

If you no longer need allocated IP addresses, you can release them. See Release IP Addresses.

Release IP Addresses

If you no longer need reserved IP addresses, you can release them so that VCF Automation can use them for deployment.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the network IP range that includes the IP addresses that you want to release. See Query for IP Addresses.
- Verify that you know the IPv4 or IPv6 addresses that you want to release. The IP addresses must all be the same type. Releasing a mix of IPv4 and IPv6 addresses in the same request is not supported. See Allocate IP Addresses.

This procedure shows how to request specific IP addresses to release.

1. Assign the variable for the network IP range ID.

   ```
   ip_range_id='<your_network_ip_range_id>'
   ```

2. Release the IP addresses.

   The following request releases two IP addresses.

   - Specify either IPv4 or IPv6 addresses.
   - To release more IP addresses, provide the IP addresses in the payload.

     Note:
     No more than 100 IP addresses can be released in a single API call.

   ```
   curl --location --request POST \
    $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/release?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
       "ipAddresses":["<ipv_address_1>", "<ipv_address_2>"]
   }'
   ```

   Examine the response for the self link to track the request.

3. Assign the self link variable.

   ```
   selfLink_id='example-selfLink-alphanumeric-string'
   ```

4. Use the selfLink variable to track the request.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
   ```

   The IP addresses are released, when the response includes `"status": "FINISHED"`.

   Release IP addresses within an IP range

   To release IP addresses, make a request with the network IP range and specify the IP addresses to release.

   Assign variables.

   ```
   $ url='https://appliance.domain.com'
   $ api_version=' 2021-07-15'
   $ ip_range_id='4e6b700c-5d07-4247-b0c7-ae13d1d7188a'
   ```

   Release two IPv4 addresses.

   ```
   curl --location --request POST \
    $url/iaas/api/network-ip-ranges/$ip_range_id/ip-addresses/release?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
       "ipAddresses":["196.192.2.19", "196.192.2.20"]
   }'
   ```

The response provides a selfLink for tracking.

```
{
   "progress": 1,
   "status": "INPROGRESS",
   "name": "IP Address Release Task",
   "id": "452bb595-ca0f-4685-8fcc-8bd6321bf7c6",
   "selfLink": "/iaas/api/request-tracker/452bb595-ca0f-4685-8fcc-8bd6321bf7c6"
}
```

Assign the selfLink variable.

```
selfLink_id= '452bb595-ca0f-4685-8fcc-8bd6321bf7c6'
```

Track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The IP addresses are released, when the response includes `"status": "FINISHED"`.

```
{
   "progress": 100,
   "status": "FINISHED",
   "resources": [
       "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/3f08cabe-
fedf-4e9b-96f6-d3bf69016cc4",
       "/iaas/api/network-ip-ranges/4e6b700c-5d07-4247-b0c7-ae13d1d7188a/ip-addresses/ca287794-
c06b-401c-9626-0b8f5f75d1c2"
   ],
   "name": "IP Address Release Task",
   "id": "452bb595-ca0f-4685-8fcc-8bd6321bf7c6",
   "selfLink": "/iaas/api/request-tracker/452bb595-ca0f-4685-8fcc-8bd6321bf7c6"
}
```

Creating and Using a First Class Disk

Using a vSphere Storage Profile that supports First Class Disk (FCD) storage, you can create block device-based storage that is independent of a VM.

First Class Disks offer many advantages. For example, you can attach an FCD to a VM and take multiple snapshots of the disk over time. If you find that you do not need certain snapshots, you can delete them. You can also revert an FCD to an earlier snapshot.

> **Note:**
> If you attach an FCD to a VM and then delete the VM, both the FCD and its snapshots are deleted. And if the FCD is detached from a VM, you cannot delete the FCD without deleting its snapshots first.

Create a First Class Disk

To create a First Class Disk (FCD), you make a POST request using the block device specification. The request body includes a project ID, disk capacity, persistence setting, and constraints from the vSphere Storage Profile for an FCD creation.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have added a project and you have the project ID. See Create a Project to use in VCF Automation.
- Know the capacity of the disk and the persistence of the disk that you are creating.
- Verify that you have created a storage profile for an FCD and that you have the `defaultItem` and the tags from the response. See Create a vSphere Storage Profile for a First Class Disk.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Set the capacity and persistence for the disk.

```
capacity_in_gb=<integer>
persistence=<true|false>
```

3. Deploy the FCD.

   - With `mandatory` set to true, all tags in the `expression` must match a storage profile, otherwise provisioning fails.
   - The `expression` is the key:value tag pair used to create the storage profile. See Create a vSphere Storage Profile for a First Class Disk.

```
curl -X POST \
 $url/iaas/api/block-devices?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "projectId": "'$project_id'",
   "name": "FCD-example",
   "capacityInGB": "'$capacity_in_gb'",
   "persistent" : "'$persistent'",
   "constraints": [
       {
           "mandatory": "true",
           "expression": "type:fcd"
       }
   ]
}' | jq "."
```

   The response includes a selfLink value.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink variable to track the progress of the FCD creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

   In the list of resources, the response includes block devices with the block device ID in the path.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/block-devices/example-blockdevice-alphanumeric-string"
 ],
 ...
}
```

6. Assign the block device ID variable.

```
block_device_id='example-blockdevice-alphanumeric-string'
```

7. Optional: Retrieve the created block device object.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id?apiVersion=$api_version | jq "."
```

8. Optional: Retrieve all the FCD block device types.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
block-devices?%24filter=customProperties.diskType%20eq%20firstClass&apiVersion=$api_version" | jq "."
```

9. Optional: Delete the FCD block device.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id?apiVersion=$api_version | jq "."
```

Create a First Class Disk

With constraints from a vSphere Storage Profile for FCD storage, use the block device specification to deploy a First Class Disk for a project ID with a two GB capacity and persistence set to false.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ project_id='f5357a28-df59-47e0-b983-8a562910d0be'
$ capacity_in_gb=2
$ persistent=false
```

Deploy the FCD.

```
$ curl -X POST \
 $url/iaas/api/block-devices?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "projectId": "'$project_id'",
   "name": "FCD-example",
   "capacityInGB": "'$capacity_in_gb'",
   "persistent" : "'$persistent'",
   "constraints": [
       {
           "mandatory": "true",
           "expression": "type:fcd"
       }
   ]
}' | jq "."
```

The response provides a selfLink to the request.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "86707da6-d5d6-4ebc-94a2-0a22f3fcb794",
 "selfLink": "/iaas/api/request-tracker/86707da6-d5d6-4ebc-94a2-0a22f3fcb794"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='86707da6-d5d6-4ebc-94a2-0a22f3fcb794'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

After the request completes successfully, the response provides the block device ID.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2"
 ],
 "name": "Provisioning",
 "id": "86707da6-d5d6-4ebc-94a2-0a22f3fcb794",
 "selfLink": "/iaas/api/request-tracker/86707da6-d5d6-4ebc-94a2-0a22f3fcb794"
}
```

Assign the block device ID variable.

```
$ block_device_id='e1cbc8e1-76bb-4bef-8e51-a582437266c2'
```

Use the block device ID to attach your FCD to a VM and manage your FCD snapshots. See Attach a First Class Disk and Manage First Class Disk Snapshots.

Attach a First Class Disk

To attach a First Class Disk (FCD) to a VM, you make a POST request with the machine ID of the VM. The request body includes the block device ID that you obtained from creating the FCD.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have created an FCD and you have a block device ID. See Create a First Class Disk.
- Verify that the hardware version of the machine to which you plan to attach the FCD is vmx-13 or later.

1. Assign the block device ID variable.

   ```
   block_device_id='<your_block_device_id>'
   ```

2. Get a list of machines.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
   machines?apiVersion=$api_version | jq "."
   ```

3. Examine the response to find the machine that you want to attach the FCD to.
4. Assign a machine ID.

   ```
   machine_id='<your_machine_id>'
   ```

5. Attach the FCD to the machine.

   ```
   curl -X POST \
    $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
    -H 'Content-Type: application/json' \
    -H "Authorization: Bearer $access_token" \
    -d '{
      "blockDeviceId": "'$block_device_id'"
   }' | jq "."
   ```

   The response includes a selfLink value.

   ```
   {
    "progress": 0,
    "status": "INPROGRESS",
    "name": "Provisioning",
    "id": "example-selfLink-alphanumeric-string",
   ```

```
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

6. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

7. Use the selfLink to track the progress of the FCD attachment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

Once complete, the response includes a list of resources with a machine that has your machine ID in the path.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/machines/your-machine-id"
 ],
 ...
}
```

8. Optional: Detach the FCD.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
machines/$machine_id/disks/$block_device_id?apiVersion=$api_version | jq "."
```

Attach a First Class Disk

With the block device ID from the FCD, attach the FCD to a VM.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='e1cbc8e1-76bb-4bef-8e51-a582437266c2'
```

Get a list of machines.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/machines?apiVersion=$api_version | jq "."
```

Examine the response. Identify the machine that you want to attach the FCD to.

```
...
   {
     "powerState": "ON",
     "externalRegionId": "Datacenter:datacenter-3",
     "cloudAccountIds": [
       "683c647b-413d-4673-a236-08b3694cd652"
     ],
     "provisioningStatus": "READY",
     "customProperties": {
       "osType": "LINUX",
       "vcUuid": "8d6dabbb-46b4-41b2-b76e-7745330f8f7d",
       "memoryGB": "0",
       "datacenter": "Datacenter:datacenter-3",
       "instanceUUID": "502a55ea-580c-9ad0-4275-82f96d3a4683",
       "softwareName": "Red Hat Enterprise Linux 7 (64-bit)",
       "cpuCount": "1",
       "memoryInMB": "256"
     },
     "externalId": "502a55ea-580c-9ad0-4275-82f96d3a4683",
```

```
      "name": "Cloud_vSphere_Machine_1-mcm100156-139639218287",
      "id": "fcaad107-48c3-320f-989f-31b0c8d4a6a0",
      "createdAt": " 2022-04-02",
      "updatedAt": " 2022-04-02",
...
```

Assign the machine ID variable.

```
$ machine_id='fcaad107-48c3-320f-989f-31b0c8d4a6a0'
```

Attach the FCD to the machine.

```
$ curl -X POST \
 $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "blockDeviceId": "'$block_device_id'"
}' | jq "."
```

The response provides a selfLink to the request.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "18050d7d-e3b2-4dd0-b0a0-5883ec766999",
 "selfLink": "/iaas/api/request-tracker/18050d7d-e3b2-4dd0-b0a0-5883ec766999"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='18050d7d-e3b2-4dd0-b0a0-5883ec766999'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

After the request completes successfully, the response includes your machine ID.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/machines/fcaad107-48c3-320f-989f-31b0c8d4a6a0"
 ],
 ...
}
```

Manage First Class Disk Snapshots

To create a snapshot of a First Class Disk (FCD), you make a POST request with the block device ID of the FCD. Using the snapshot ID created, you can revert an FCD to a snapshot or delete a snapshot of an FCD.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have created an FCD and you have a block device ID. See Create a First Class Disk.

1. Assign the block device ID variable.

```
block_device_id='<your_block_device_id>'
```

2. Create a snapshot of the FCD.

```
curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "description": "example description"
 }' | jq "."
```

   The response includes a selfLink value.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

3. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink to track the progress of the FCD snapshot creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

   The response indicates if the snapshot is successful.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/block-devices/your-block-device-id"
 ],
 ...
}
```

5. Optional: To create additional FCD snapshots, repeat Step 2 to Step 4.
6. To get a snapshot ID, list all FCD snapshots.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

   If you created multiple snapshots, the response lists multiple snapshot IDs.

7. Examine the response and select a snapshot ID to assign as a variable.

```
snapshot_id=<your_snapshot_id_1>
```

8. Optional: You can list an individual snapshot.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

9. Optional: You can revert an FCD to a snapshot.

```
curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/revert?id=$snapshot_id\&apiVersion=$api_version \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '' | jq "."
```

To track the progress of the reversion, perform Step 3 and Step 4.

10. Optional: You can delete a snapshot.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

Create snapshots of a First Class Disk and revert an FCD to a snapshot

With the block device ID from the created FCD, create multiple snapshots of an FCD.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='e1cbc8e1-76bb-4bef-8e51-a582437266c2'
```

Create a snapshot of the FCD.

```
$ curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "description": "Example description 1"
 }' | jq "."
```

The response provides a selfLink to the request.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "d08bb46c-cf7e-40b6-bdf8-893390ba4d51",
 "selfLink": "/iaas/api/request-tracker/d08bb46c-cf7e-40b6-bdf8-893390ba4d51"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='d08bb46c-cf7e-40b6-bdf8-893390ba4d51'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

Create a second snapshot of the FCD.

```
$ curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "description": "Example description 2"
 }' | jq "."
```

List all the snapshots of the FCD.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/
api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

Examine the response to see all snapshot IDs.

```
[
 {
   "name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
   "desc": "Example description 1",
   "isCurrent": false,
   "id": "16cfdbb8-559c-49ff-8162-0a4c57079c81",
   "createdAt": " 2022-04-02",
   "updatedAt": " 2022-04-02",
   "owner": "user@mycompany.com",
   "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "_links": {
     "self": {
       "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/
snapshots/16cfdbb8-559c-49ff-8162-0a4c57079c81"
     }
   }
 },
 {
   "name": "b04f7513-c695-4662-b5e8-a023a7b1bfe7",
   "desc": "Example description 2",
   "isCurrent": true,
   "id": "ed1b09ff-1175-4cdd-b07e-7bb906a9ddc4",
   "createdAt": " 2022-04-22",
   "updatedAt": " 2022-04-22",
   "owner": "user@mycompany.com",
   "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "_links": {
     "self": {
       "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/snapshots/
ed1b09ff-1175-4cdd-b07e-7bb906a9ddc4"
     }
   }
 }
]
```


Assign a snapshot ID variable.

```
snapshot_id='16cfdbb8-559c-49ff-8162-0a4c57079c81'
```

List information about the snapshot.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

The response shows information about the single snapshot.


```
{
   "name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
   "desc": "Example description 1",
   "isCurrent": false,
   "id": "16cfdbb8-559c-49ff-8162-0a4c57079c81",
   "createdAt": " 2022-04-02",
   "updatedAt": " 2022-04-02",
   "owner": "user@mycompany.com",
   "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "_links": {
     "self": {
```

```
        "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/
snapshots/16cfdbb8-559c-49ff-8162-0a4c57079c81"
      }
    }
```

Revert the FCD to the snapshot.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

To validate the reversion, list information about the snapshot again.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

In the response, `"isCurrent":true` shows that the FCD has reverted to the snapshot.

```
{
   "name": "357ed3e5-8b2e-4533-b6fe-3ea6e15b8de5",
   "desc": "Example description 1",
   "isCurrent": true,
   "id": "16cfdbb8-559c-49ff-8162-0a4c57079c81",
   "createdAt": " 2022-04-02",
   "updatedAt": " 2022-04-02",
   "owner": "user@mycompany.com",
   "organizationId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "orgId": "b373cda4-ae0f-4d5a-9eca-f307bd30c9cd",
   "_links": {
     "self": {
       "href": "/iaas/api/block-devices/e1cbc8e1-76bb-4bef-8e51-a582437266c2/
snapshots/16cfdbb8-559c-49ff-8162-0a4c57079c81"
     }
   }
```

Working with Azure Disk Snapshots

You can use the VCF Automation IaaS API to create or delete snapshots of Azure managed disks. The snapshot provides a backup of your block device.

> **Note:**
> If you create a snapshot of an independent disk and then delete the disk, the snapshot is left behind. So before deleting a disk from the Azure cloud account, check for snapshots. If snapshots exist, delete them first before deleting the disk.

Create a Block Device

If you do not already have an Azure managed disk, create a block device to use for your snapshot. To create a block device, you make a POST request using the block device specification. The request body includes a project ID, disk capacity, persistence setting, and constraints from the Azure Storage Profile for a managed disk.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have added a project and you have the project ID. See Create a Project to use in VCF Automation.
- Know the capacity of the disk and the persistence of the disk that you are creating.
- Verify that you have created a storage profile for a managed disk and that you have the `defaultItem` and the tags from the response. See Create a Microsoft Azure Storage Profile for a Managed Disk.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

2. Set the capacity and persistence for the disk.

   ```
   capacity_in_gb=<integer>
   ```

```
persistence=<true|false>
```

3. Deploy the block device.
   - With `mandatory` set to false, tags in the `expression` are not required to match tags in an existing storage profile for the deployment to succeed. However if tags are provided, VCF Automation will try to match them when deploying the block device.
   - The `expression` is the key:value tag pair used to create the storage profile. See Create a vSphere Storage Profile for a First Class Disk.

```
curl -X POST \
 $url/iaas/api/block-devices?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "projectId": "'$project_id'",
   "name": "block-device-example",
   "capacityInGB": "'$capacity_in_gb'",
   "persistent" : "'$persistent'",
   "constraints": [
       {
           "mandatory": "false",
           "expression": "type:managed"
       }
   ]
}' | jq "."
```

   The response includes a selfLink value.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink variable to track the progress of the block device creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

   In the list of resources, the response includes block devices with the block device ID in the path.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/block-devices/example-blockdevice-alphanumeric-string"
 ],
 ...
}
```

6. Optional: If you want to retrieve the ID of an existing block device, use an `OData` filter with the block device name in the request.

```
block_device_name='<your_block_device_name>'
```

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
block-devices?apiVersion=$api_version&"'$filter'"=name%20eq%20$block_device_name" | jq "."
```

7. Assign the block device ID variable.

```
block_device_id='example-blockdevice-alphanumeric-string'
```

Create a Block Device

With constraints from an Azure Storage Profile for a managed disk, use the block device specification to deploy a managed disk for a project ID with a two GB capacity and persistence set to false.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ project_id='f5357a28-df59-47e0-b983-8a562910d0be'
$ capacity_in_gb=2
$ persistent=false
```

Deploy the block device.

```
$ curl -X POST \
 $url/iaas/api/block-devices?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "projectId": "'$project_id'",
   "name": "block-device-example",
   "capacityInGB": "'$capacity_in_gb'",
   "persistent" : "'$persistent'",
   "constraints": [
       {
           "mandatory": "false",
           "expression": "type:managed"
       }
   ]
}' | jq "."
```

The response provides a selfLink to the request.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
 "selfLink": "/iaas/api/request-tracker/22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
 "deploymentId":"cf33c90e-6f6d-48ed-82dd-a6a9f0e6f700"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='22bdaf20-ce48-4a9f-8c1f-f4e74263645f'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

After the request completes successfully, the response provides the block device ID.

```
{
 "progress": 100,
```

```
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101"
  ],
  "name": "Provisioning",
  "id": "22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "selfLink": "/iaas/api/request-tracker/22bdaf20-ce48-4a9f-8c1f-f4e74263645f",
  "deploymentId":"cf33c90e-6f6d-48ed-82dd-a6a9f0e6f700"
}
```

Assign the block device ID variable.

```
$ block_device_id='41d600c2-429e-4c90-98d4-638e77724101'
```

Use the block device ID to create a snapshot. See Create and Manage Azure Disk Snapshots.

Create and Manage Azure Disk Snapshots

To create a snapshot of a Azure managed disk, you make a POST request with the block device ID of the managed disk. Using the snapshot ID created, you can list a snapshot or delete a snapshot of a managed disk.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have an Azure managed disk and a block device ID. See Create a Block Device.

1. Assign the block device ID variable.

   ```
   block_device_id='<your_block_device_id>'
   ```

2. Create a snapshot of the managed disk.

   ```
   curl -X POST \
    $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
    -H 'Content-Type: application/json' \
    -H "Authorization: Bearer $access_token" \
    -d '{
      "name": "example_name"
    }' | jq "."
   ```

   In addition to the required snapshot name, you can include optional snapshot properties that can be used independently or combined as in the following example.

   ```
   curl -X POST \
    $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
    -H 'Content-Type: application/json' \
    -H "Authorization: Bearer $access_token" \
    -d '{
      "name": "example_name"
      "snapshotProperties": {
         "incremental": "true",
         "resourceGroupName": "newRG",
         "encryptionSetId" : "'"$encryption_key"'"
   },
    }' | jq "."
   ```

   - To create a snapshot that only consists of changes since the last snapshot, set `incremental` to true.
   - To specify a target resource group for the snapshot, include the `resourceGroupName` property. You can look up existing resource group names in the Azure portal, or specify a name for a new resource group, If not specified, VCF Automation creates the snapshot in the same resource group as the block device by default.
   - To encrypt the snapshot with a customer-managed key, include the `encryptionSetId` property.
     To get the `encryptionSetId` property for a particular region ID, use the following API request:

     ```
     curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
     fabric-azure-disk-encryption-sets?regionId=$region_id&apiVersion=$api_version | jq "."
     ```

     Examine the response for the `id` as in the following example.

```
    {
      "name": "test-encryption-1.1",
      "id": "/subscriptions/b8ef63a7-a5e3-44fa-8745-1ead33fa1f25/resourceGroups/DISKRG67970/providers/
  Microsoft.Compute/diskEncryptionSets/test-encryption-1.1",
      "regionId": "eastus",
      "key": "key1234",
      "vault": "KeyVault1234"
    },
```

Assign the encryption key variable.

```
$encryption_key='/subscriptions/b8ef63a7-a5e3-44fa-8745-1ead33fa1f25/resourceGroups/DISKRG67970/
providers/Microsoft.Compute/diskEncryptionSets/test-encryption-1.1'
```

3. Examine the response for the selfLink value.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

4. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

5. Use the selfLink to track the progress of the snapshot creation.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

The response indicates if the snapshot is successful.

```
{
 "progress": 100,
 "message": "success",
 "status": "FINISHED",
 "resources": [
   "/iaas/api/block-devices/your-block-device-id"
 ],
 ...
}
```

6. To get a snapshot ID, list all snapshots.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

If you created multiple snapshots, the response lists multiple snapshot IDs.

7. Examine the response and select a snapshot ID to assign as a variable.

```
snapshot_id=<your_snapshot_id_1>
```

8. Optional: You can list an individual snapshot.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

9. Optional: You can delete a snapshot.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

Create snapshots of a managed disk

With the block device ID, create multiple snapshots of a managed disk.

```
$ url='https://appliance.domain.com'
$ api_version=' 2021-07-15'
$ block_device_id='41d600c2-429e-4c90-98d4-638e77724101'
```

Create a snapshot of the managed disk.

```
$ curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "name": "demo-snapshot-1"
 }' | jq "."
```

The response provides a selfLink to the request.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Provisioning",
 "id": "66123d15-8e5a-42b0-a0b4-e9ed8e21180a",
 "selfLink": "/iaas/api/request-tracker/66123d15-8e5a-42b0-a0b4-e9ed8e21180a"
}
```

Assign the selfLink ID variable.

```
$ selfLink_id='66123d15-8e5a-42b0-a0b4-e9ed8e21180a'
```

Track the progress of the request.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

Create a second snapshot of the managed disk.

```
$ curl -X POST \
 $url/iaas/api/block-devices/$block_device_id/operations/snapshots?apiVersion=$api_version \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "name": "demo-snapshot-2"
 }' | jq "."
```

List all the snapshots of the managed disk.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots?apiVersion=$api_version | jq "."
```

Examine the response to see all snapshot IDs.

```
[
 {
   "name": "demo-snapshot-1",
   "snapshotProperties": {
     "incremental": "false"
   },
```

```
    "id": "38348991-00a1-48be-80d6-00d62afcd280",
    "createdAt": " 2022-04-02",
    "updatedAt": " 2022-04-02",
    "owner": "user@mycompany.com",
    "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/
snapshots/38348991-00a1-48be-80d6-00d62afcd280"
      }
    }
  },
  {
    "name": "demo-snapshot-2",
    "snapshotProperties": {
      "incremental": "false"
    },
    "id": "80407f78-7f90-4d9f-83f4-10d3a1e982ac",
    "createdAt": " 2022-04-02",
    "updatedAt": " 2022-04-02",
    "owner": "user@mycompany.com",
    "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
    "_links": {
      "self": {
        "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/
snapshots/80407f78-7f90-4d9f-83f4-10d3a1e982ac"
      }
    }
  }
]
```


Assign a snapshot ID variable.

```
$ snapshot_id='80407f78-7f90-4d9f-83f4-10d3a1e982ac'
```

List information about the snapshot.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/
iaas/api/block-devices/$block_device_id/snapshots/$snapshot_id?apiVersion=$api_version | jq "."
```

The response shows information about the single snapshot.


```
{
  "name": "demo-snapshot-2",
  "snapshotProperties": {
    "incremental": "false"
  },
  "id": "80407f78-7f90-4d9f-83f4-10d3a1e982ac",
  "createdAt": " 2022-04-02",
  "updatedAt": " 2022-04-02",
  "owner": "user@mycompany.com",
  "organizationId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
  "orgId": "1b6fd77b-f5d9-466b-88d3-97c0d9eb70c9",
  "_links": {
    "self": {
      "href": "/iaas/api/block-devices/41d600c2-429e-4c90-98d4-638e77724101/
snapshots/80407f78-7f90-4d9f-83f4-10d3a1e982ac"
    }
```

```
  }
 }
```

Update the Custom Properties of a Machine

After deploying a machine, you can use the IaaS APIs to update the machine with custom properties. Custom properties provide you with the flexibility to add any information about the machine that you want.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the virtual machine in your deployment. See Get Deployment Resource IDs.

For example, machine IDs are typically autogenerated. By updating the custom properties, you can identify the machine owner and include their contact email or phone information.

1. Assign your virtual machine ID variable.

   Assigning this variable is useful if you plan to update the machine again.

   ```
   virtual_machine_id='<your_virtual_machine_id>'
   ```

2. Update the machine with custom property names and values that you choose.

   ```
   curl -X PATCH \
    $url/iaas/api/machines/$virtual_machine_id?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
    "customProperties": {
      "additionalPropName1": "<custom_prop_value_1>",
      "additionalPropName2": "<custom_prop_value_2>",
      "additionalPropName3": "<custom_prop_value_3>"
    },
    "description": "string",
    "tags": "[ { \"key\" : \"ownedBy\", \"value\": \"Rainpole\" } ]"
   }' | jq "."
   ```

3. A snippet of the response lists the added custom properties.

   #### Add a Custom Properties to Your Virtual Machine

   Update the virtual machine with resource ID `42f49781-1490-4a08-ae21-8baf383a72ac` by adding custom properties.

   Assign variables.

   ```
   $ url='https://appliance.domain.com'
   $ api_version=' 2021-07-15'
   ```

   Assign the virtual machine ID.

   ```
   $ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
   ```

   Update the machine with custom properties.

   ```
   $ curl -X PATCH \
    $url/iaas/api/machines/$virtual_machine_id?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
    "customProperties": {
      "ownerName": "VMuser_Example",
      "ownerEmail": "VMuser_Example@mycompany.com",
      "ownerCell": "123.456.7890"
    },
    "description": "string",
    "tags": "[ { \"key\" : \"my.enumeration.type\", \"value\": \"ec2_instance\" } ]"
   ```

```
}' | jq "."
```

A snippet of the response shows that the request was successful.

```
...
 "customProperties": {
      "ownerName": "VMuser_Example",
      "ownerEmail": "VMuser_Example@mycompany.com",
      "ownerCell": "123.456.7890"
      "image": "ubuntu",
      "OStype": "LINUX",
      "imageId": "ami-b1234cc5",
 ...
 },
...
```

Provision a VLAN Private Network

After creating a network profile with a VLAN transport zone, you can provision a VLAN private network.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the project ID. See Create a Project to use in VCF Automation.
- Verify that you know the VLAN IDs that you want to use. Comma separated values can be 0 - 4094.

With VLAN IDs in your private network definition, VCF Automation allocates the blueprint to a network profile with a VLAN transport zone specified. If no VLAN ID is specified, the blueprint is allocated to a network profile with an overlay transport zone.

1. Assign your project ID variable.

   ```
   project_id='<your_project_id>'
   ```

2. Provision the network with VLAN IDs that you choose.

   ```
   curl -X POST \
     $url/iaas/api/networks?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
     -d '{
       "name": "<your_network_name>",
       "description": "<your_description>",
       "projectId": "'$project_id'",
       "customProperties": {
         "networkType": "PRIVATE",
         "vlanIds": "<integer_values_0_to_4094>"
     }
   }' | jq "."
   ```

   The response includes a selfLink value.

   ```
   {
     "progress": 0,
     "status": "INPROGRESS",
     "name": "Provisioning",
     "id": "example-selfLink-alphanumeric-string",
     "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
   }
   ```

3. Assign the selfLink variable.

   ```
   selfLink_id='example-selfLink-alphanumeric-string'
   ```

4. Use the selfLink variable to track the request.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   request-tracker/$selfLink_id | jq "."
   ```

Provision a VLAN Private Network

Provision a VLAN private network for a project ID

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
```

Assign the project ID.

```
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'
```

Provision the VLAN private network.

```
curl -X POST \
  $url/iaas/api/networks?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "example-vlan-network",
    "description": "Example VLAN Network created using API",
    "projectId": "'$project_id'",
    "customProperties": {
      "networkType": "PRIVATE",
      "vlanIds": "1004, 1005"
  }
}' | jq "."
```

The response includes a selfLink.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "dab1fe2f-4104-4fce-b534-e7ab5c172788",
  "selfLink": "/iaas/api/request-tracker/dab1fe2f-4104-4fce-b534-e7ab5c172788",
  "deploymentId": "8964a3f6-e829-40ad-b07c-0177abb7b4f4"
}
```

Assign the selfLink variable.

```
selfLink_id='dab1fe2f-4104-4fce-b534-e7ab5c172788'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id | jq "."
```

When the request completes successfully, the response shows the network has been created.

```
{
    "progress": 100,
    "message": "success",
    "status": "FINISHED",
    "resources": [
        "/iaas/api/networks/3f3b4f87-3e0d-48e8-a484-a8f2890977df"
    ],
    "name": "example-vlan-network Allocation",
    "id": "1cca9d9c-1e93-40d6-8bab-fa92dd89566a",
    "selfLink": "/iaas/api/request-tracker/1cca9d9c-1e93-40d6-8bab-fa92dd89566a",
    "deploymentId": "8b8eef00-e159-4ba9-9e78-09089a3f5786"
}
```

How do I use a placement policy to spread VMs by memory

If you want to balance your deployed resources across multiple cloud zones, you can use the IaaS API to define the placement policy in a project and its cloud zones. When you deploy a blueprint that uses the project, VCF Automation allocates new VMs to zones and clusters with the most free memory, effectively spreading them for better memory usage.

Prerequisites for defining a placement policy

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

How to specify spread by memory in your project

This example shows how to find the project with cloud zones where you want to place your VMs. Then using the project ID, you update the project with `"placementPolicy": "SPREAD_MEMORY"` to spread VM memory use across the cloud zones.

Get all projects.

```
curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/projects?
apiVersion=$api_version" | jq "."
```

A snippet of the response shows a project with its assigned cloud zones. The project ID `6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9` has two zones with zone IDs:

- `3c2bbe36-bf8e-4484-9c31-ce552422aaf1`
- `8992bdf0-136f-401c-822a-e22dae67259b`


```
{
    "administrators": [],
    "members": [],
    "viewers": [],
    "zones": [
        {
            "zoneId": "3c2bbe36-bf8e-4484-9c31-ce552422aaf1",
            "priority": 0,
            "maxNumberInstances": 0,
            "allocatedInstancesCount": 0,
            "memoryLimitMB": 0,
            "allocatedMemoryMB": 0,
            "cpuLimit": 0,
            "allocatedCpu": 0,
            "storageLimitGB": 0,
            "allocatedStorageGB": 0.0
        },
        {
            "zoneId": "8992bdf0-136f-401c-822a-e22dae67259b",
            "priority": 0,
            "maxNumberInstances": 0,
            "allocatedInstancesCount": 0,
            "memoryLimitMB": 0,
            "allocatedMemoryMB": 0,
            "cpuLimit": 0,
            "allocatedCpu": 0,
            "storageLimitGB": 0,
            "allocatedStorageGB": 0.0
        }
    ],
    "constraints": {},
```

```
    "operationTimeout": 0,
    "sharedResources": true,
    "placementPolicy": "DEFAULT",
    "customProperties": {},
    "name": "project1",
    "description": "",
    "id": "6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9",
    "orgId": "f098d692-e980-41a5-b349-83084fce1ea0",
    "_links": {
        "self": {
            "href": "/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9"
        }
    }
}
```

Use the project ID to update the placement policy and spread memory over the two cloud zones.

```
curl -X PATCH \
  "$url/iaas/api/projects/6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9?apiVersion=$api_version"
  -H 'Content-Type: application/json'
  -H "Authorization: Bearer $access_token"
  -d '{
    "placementPolicy": "SPREAD_MEMORY"
  }' | jq "."
```

How to specify spread by memory in cloud zones

This example shows how to use the cloud zone IDs in the project ID `6c2f2d0d-ecee-42e3-90be-7bb66d6da2f9` to check free memory in compute resources. Provided that free memory is available, you update the cloud zones with `"placementPolicy": "SPREAD_MEMORY"`.

To check available memory in zone ID `3c2bbe36-bf8e-4484-9c31-ce552422aaf1`, list compute resources.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
zones/3c2bbe36-bf8e-4484-9c31-ce552422aaf1/computes?apiVersion=$api_version"  | jq "."
```

The response shows two clusters with available memory:

- `ESO_PKS_VC01_Cluster03` has 747 Gbytes available memory.
- `ESO_PKS_VC01_Cluster04` has 397 Gbytes available memory.

```
{
  "content": [
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "2",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "4",
        "cpuCoreCount": "56",
        "vsanConfigId": "52a60c7a-ef2e-af08-7cb2-36b06f686ebb",
        "isVsanEnabled": "true",
        "MaxVCPUperInstance": "56",
        "MemoryAvailableBytes": "747407147008"
      },
```

```
      "externalId": "domain-c21",
      "name": "ESO_PKS_VC01_Cluster03",
      "id": "e03f62e1-9a48-4d2c-8aa7-7bdb97293749",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fce1ea0"
    },
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "1",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "2",
        "cpuCoreCount": "28",
        "vsanConfigId": "",
        "isVsanEnabled": "false",
        "MaxVCPUperInstance": "56",
        "MemoryAvailableBytes": "397091536896"
      },
      "externalId": "domain-c24",
      "name": "ESO_PKS_VC01_Cluster04",
      "id": "d2d42957-b6df-4e45-879a-93dfbec9a528",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fce1ea0"
    }
  ],
  "totalElements": 2,
  "numberOfElements": 2
}
```


Use the zone ID `3c2bbe36-bf8e-4484-9c31-ce552422aaf1` to spread memory over the clusters.

```
curl -X PATCH \
  "$url/iaas/api/zones/3c2bbe36-bf8e-4484-9c31-ce552422aaf1?apiVersion=$api_version"
  -H 'Content-Type: application/json'
  -H "Authorization: Bearer $access_token"
  -d '{
    "placementPolicy": "SPREAD_MEMORY"
  }' | jq "."
```

To check available memory in zone ID `8992bdf0-136f-401c-822a-e22dae67259b`, list compute resources.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
zones/8992bdf0-136f-401c-822a-e22dae67259b/computes?apiVersion=$api_version"  | jq "."
```

The response shows two clusters with available memory:

- `ESO_PKS_VC01_Cluster01` has 830 Gbytes available memory.
- `ESO_PKS_VC01_Cluster05` has 397 Gbytes available memory.


```
{
  "content": [
    {
      "externalRegionId": "Datacenter:datacenter-3",
```

```
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "5",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "10",
        "cpuCoreCount": "140",
        "vsanConfigId": "52de077e-2f21-b24c-536b-79ef9a412968",
        "isVsanEnabled": "true",
        "MaxVCPUperInstance": "56",
        "MemoryAvailableBytes": "830677712896"
      },
      "externalId": "domain-c18",
      "name": "ESO_PKS_VC01_Cluster01",
      "id": "98691ba2-2f84-4d31-a9f1-690d254c5305",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fce1ea0"
    },
    {
      "externalRegionId": "Datacenter:datacenter-3",
      "tags": [],
      "type": "Cluster",
      "lifecycleState": "READY",
      "powerState": "ON",
      "customProperties": {
        "vcUuid": "74620317-856c-4f55-a862-dd2b43f07373",
        "hostCount": "1",
        "datacenter": "Datacenter:datacenter-3",
        "cpuPkgCount": "2",
        "cpuCoreCount": "28",
        "vsanConfigId": "",
        "isVsanEnabled": "false",
        "MaxVCPUperInstance": "56",
        "MemoryAvailableBytes": "397045399552"
      },
      "externalId": "domain-c26",
      "name": "ESO_PKS_VC01_Cluster05",
      "id": "f576f346-ed3b-4180-acdc-3c217e9fa0fd",
      "createdAt": "2022-07-25",
      "updatedAt": "2022-07-26",
      "orgId": "f098d692-e980-41a5-b349-83084fce1ea0"
    }
  ],
  "totalElements": 2,
  "numberOfElements": 2
}
```


Use the zone ID `8992bdf0-136f-401c-822a-e22dae67259b` to spread memory over the clusters.

```
curl -X PATCH \
  "$url/iaas/api/zones/8992bdf0-136f-401c-822a-e22dae67259b?apiVersion=$api_version"
  -H 'Content-Type: application/json'
  -H "Authorization: Bearer $access_token"
  -d '{
```

```
  "placementPolicy": "SPREAD_MEMORY"
}' | jq "."
```

Protecting Sensitive Data

By using the VCF Automation IaaS API to mark certain data as sensitive in a request body, you can store the data in encrypted form, and ensure that only the encrypted form of data is visible in the response. VCF Automation decrypts the data only when the actual value is needed, for example before sending a request to the cloud.

Data encryption works for certain types of data and is limited to the following use cases:

- When provisioning resources such as machines, load balancers, disks, or networks, the following types of data support encryption:
  - Custom property values for all types of resources.
  - Remote access passwords for machines.
  - Sensitive parts of the cloud config for machines.
- When creating or updating projects, custom properties support encryption.
- When updating a deployed machine, custom properties support encryption.

  Note:
  Data encryption is only supported for deployed machines. It is not supported for discovered machines.
- When creating or updating image profiles, cloud config supports encryption. This means that you can mark parts of the cloud config script as sensitive. For example if the script includes passwords, you can mark the passwords as sensitive.

How to provision a machine with sensitive data

To mark data as sensitive, you add sensitive values with a prefix and suffix. The following example shows how to provision a new machine with sensitive values such as custom properties and a remote access password. This machine is also provisioned with a project that includes an encrypted custom property, so that the custom property is added to the machine.

1. In VCF Automation, create a cloud account. Add a cloud zone to the cloud account and add a flavor mapping and image mapping to the cloud zone.
2. In your browser or HTTP client application, verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
3. Create a project with the cloud zone that you created using the VCF Automation UI. Include a sensitive custom property for the Active Directory (AD) password. In this way, when users related to the project provision resources with the project, they have the same AD password.

   The following example shows the AD password enclosed with the `((sensitive:` prefix and the `))` suffix to mark it as sensitive.

   ```
   curl -X POST \
     "$url/iaas/api/projects?apiVersion=$api_version"
     -H 'Content-Type: application/json'
     -H "Authorization: Bearer $access_token"
     -d '{
         "name" : "example-project",
         "customProperties": {
              "activeDirectoryPassword":((sensitive:My-password123!))"
         }
     }' | jq "."
   ```

   A snippet of the response lists the project ID.

   ```
   ...
     "name": "example-project",
     "description": "This is an example project",
     "id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
   ```

```
    "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
  ...
```

4. Provision a virtual machine with sensitive data.
   The following example includes the custom property `costCenterPassword` and a password for remote access, with values that are both marked as sensitive using the `((sensitive:` prefix and the `))` suffix. The request body also includes the ID of the project with the encrypted AD password.

```
curl -X POST \
  "$url/iaas/api/machines?apiVersion=$api_version"
  -H 'Content-Type: application/json'
  -H "Authorization: Bearer $access_token"
  -d '{
      "name" : "example-vm",
      "image" : "ubuntu",
      "flavor" : "small",
      "projectId" : "5944aacb-91de-4541-bb9e-ef2a5403f81b",
      "customProperties": {
           "costCenterPassword":"((sensitive:Pass4costCtr$$$))"
      "remoteAccess": {
           "authentication":"usernamePassword",
           "username":"example-user",
           "password":"((sensitive:example-sensitive-pass!123))"
    }
  }' | jq "."
```

   Note:
   The password for remote access is marked sensitive as an example. If left unmarked, the remote access password is encrypted because it is sensitive by default.

5. After successfully provisioning the machine, issue a `GET /iaas/api/machines` request to obtain information about the machine.
   In a snippet of the response, values for the custom property `costCenterPassword` and remote access password are encrypted and appear in their encrypted form with the `((secret:v1:` prefix as in the following example.

```
..."customProperties": {
    ...
    "costCenterPassword": "((secret:v1:AAHeSZhRynh8+NSdswAdsfdsgSDffhbfh))",
    ...
},
...
"bootConfig": {
   "content": "#cloud-config\nusers:\n- default\n- name: example-user\n ...\n passwd:
 ((secret:v1:AAFPdqFQBiJbGKdklseiHSN28ckjSghjngj))\n..."
}
...
```

   VCF Automation converts the remote access information in the request into a cloud config script in the response. The encrypted password appears as a content value in the `bootConfig` .

Verify that the remote access password works

Even though the password is encrypted in the VCF Automation database, you can use the user name and plain text password from the request to log in to the machine because the password is decrypted before it is sent to the cloud.

Note:
You can choose to verify that your remote access password works only if the cloud provider allows remote access. For example, Azure might allow remote access while GCP or AWS might not.

To test your password, use the IP address of the newly provisioned machine such as `192.168.12.1234` and the user name such as `example-user`. Log in to the remote machine with:

```
$ ssh example-user@192.168.12.1234
```

When prompted for the password, copy and paste the plain text password from the request or `example-sensitive-pass!123`. A successful login verifies that the machine was provisioned with the remote access password provided in the request.

Properties that Support Encryption

Image profiles, projects and all types of provisioned entities can include sensitive information. The table below lists all endpoints that support encryption and the parts of the request body that can contain sensitive data.

Endpoint	You can apply encryption to:	Example Input
Create/update/list machines **Note:** Data encryption is only supported for deployed machines. It is not supported for discovered machines.	• customProperties: value • nics: customProperties: value • Sensitive parts of bootConfig: content • remoteAccess: password	`{ "name": "machine1", "customProperties": { "username": "guest", "password": "((sensitive:mypass))" } }`
Create/update/list image profiles	Sensitive parts of imageMapping: value: cloudConfig	`{ "imageMapping" : { "ubuntu": { "id": "{{awsUbuntuId}}", "cloudConfig": "#cloud-config\nchpasswd:\n list:|\nuser1: ((sensitive:password1))" } }, "name" : "aws-image-profile", "regionId": "{{awsRegionId}}" }`
Create/update/list projects	customProperties: value	`{ "name": "project1", "customProperties": { "vidm-password": "((sensitive:mypass))" } }`
Create/list load balancers	• customProperties: value • nics: customProperties: value	`{ "name": "load-balancer1", "nics": { "customProperties": { "dhcp-server-password": "((sensitive:mypass))" } }`
Create/list networks	customProperties: value	`{ "name": "network1", "customProperties": { "dhcp-server-password": "((sensitive:mypass))" } }`

Endpoint	You can apply encryption to:	Example Input

Create/list security groups	customProperties: value	<pre>{
		  "name": "security-group1",
		  "customProperties": {
		  "some-password":
		"((sensitive:mypass))"
		  }
		}</pre>

Create/list block device	customProperties: value	<pre>{
		  "name": "device1",
		  "customProperties": {
		  "some-password":
		"((sensitive:mypass))"
		  }
		}</pre>

Create/list compute gateway	customProperties: value	<pre>{
		  "name": "gateway1",
		  "customProperties": {
		  "some-password":
		"((sensitive:mypass))"
		  }
		}</pre>

Querying with the VCF Automation APIs

By adding query options to an API request, you control the amount of output returned by the server and make the response easier to interpret. The API service uses the options specified to transform the data by filtering or paginating before returning the results.

You can use the following query options in your API requests. The options do not apply to all endpoints.

`$top`	Number of records to get. For more information, see Using Pagination and Count.
`$skip`	Number of records to skip. For more information, see Using Pagination and Count.
`$count`	If set to `true`, shows the total number of records. If used with a filter, shows the number of records matching the filter. For more information, see Using Pagination and Count.
`$select`	Names the subset of properties to list in the response.
`$filter`	Filters results by a predicate expression with operators such as, `eq`, `ne`, `and`, and `or`. For specialized filtering examples, see: • Filtering Resources by Region ID • Filtering Operations for Projects • Filtering for Machine Status

Endpoints that support all query options

To query for any of the following endpoints, you can use all options. Examples show how to construct a request using the `$filter` option with a logical `or` operation.

> **Note:**
> VCF Automation APIs do not support filtering for nested properties with a "." in the property name.
>
> For example, you can filter for a property with the name `createdByEmail` as in the following example:

```
$filter=customProperties.createdByEmail%20eq%20'user@mycompany.com'
```

However, API filtering does not support a property with the name `my.createdByEmail` as in the following example:

```
$filter=customProperties.my.createdByEmail%20eq%20'user@mycompany.com'
```

Endpoint	Example
Machine	`$url/iaas/api/machines?$filter=name%20ne%20'example-name'%20or%20customProperties.osType%20eq%20'example-os'`
Cloud Account	`$url/iaas/api/cloud-accounts?$filter=name%20ne%20'example-cloud-account'%20or%20customProperties.isExternal%20eq%20'false'`
Fabric Azure Storage Account	`$url/iaas/api/fabric-azure-storage-accounts/?$filter=name%20ne%20'example-name'%20or%20type%20eq%20'example-type`
Fabric Compute	`$url/iaas/api/fabric-computes?$filter=name%20ne%20'example-name'%20or%20customProperties.isExternal%20eq%20'false'`
Fabric Image	`$url/iaas/api/fabric-images?$filter=name%20ne%20'example-name'%20or%20osFamily%20eq%20'example-os'`
Fabric Network	`$url/iaas/api/fabric-networks?$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'`
Fabric Network (vSphere)	`$url/iaas/api/fabric-networks-vsphere?$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'`
Fabric vSphere Datastores	`$url/iaas/api/fabric-vsphere-datastores?$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'`
Fabric vSphere Storage Policies	`$url/iaas/api/fabric-vsphere-storage-policies?$filter=name%20ne%20'example-name'%20or%20externalId%20eq%20'example-id'`

Querying for endpoints with a specified ID

To query for an endpoint with specified ID, you can only use the `$select` option. Examples show how to construct a request.

Endpoint	Example
Cloud Account by ID	`$url/iaas/api/cloud-account/{id}?$select=name`
Machine by ID	`$url/iaas/api/machines/{id}?$select=name`
Fabric Azure Storage Account by ID	`$url/iaas/api/fabric-azure-storage-accounts/{id}?$select=name`
Fabric Image by ID	`$url/iaas/api/fabric-images/{id}?$select=name`
Fabric Network by ID	`$url/iaas/api/fabric-networks/{id}?$select=name`
Fabric Network (vSphere) by ID	`$url/iaas/api/fabric-networks-vsphere/{id}?$select=name`

Endpoint	Example
Fabric vSphere Datastores by ID	`$url/iaas/api/fabric-vsphere-datastores/{id}?$select=name`
Fabric vSphere Storage Policies by ID	`$url/iaas/api/fabric-vsphere-storage-policies/{id}?$select=name`

Querying for a partial match

To query for the partial match of a name that starts with, ends with, or is contained within another name, the `$filter` options are the same for most IaaS endpoints but are different for `iaas/api/projects` and `iaas/api/deployments` endpoints. Examples show how to construct the partial match filters for the different endpoint types.

Filter Operation	Query with `iaas/api/projects` or `iaas/api/deployments`	Query with most IaaS endpoints
Name starts with `foo`	`?$filterstartswith(name, 'foo')`	`$filter=name%20eq%20'foo*'`
Name ends with `foo`	`?$filter=endswith(name, 'foo')`	`$filter=name%20eq%20'*foo'`
`foo` contained within the name	`?$filter=substringof('foo', name)`	`$filter=name%20eq%20'*foo*'`

Querying for deployments

To query for deployments, you can use all options except `$select`. The following example shows how to use the `$filter` option to list deployments that are not named `example-name` or have `projectId='example-id'`.

```
GET $url/iaas/api/deployments?$filter=name%20ne%20'example-name'%20or%20projectId%20eq%20'example-id'
```

Using Pagination and Count

Pagination controls the number of elements or the range of elements returned in an API response. When the count flag is specified, the response shows the total number of records. Use parameters in combination to traverse all elements in a result.

To paginate a response, you use the following parameters.

`$top=N`	Selects only the first N elements of the set. N must be a positive integer. Specifying N limits the maximum number of elements that the server returns in the response. The default IaaS API page size is 100. If `$top` is left unspecified, pagination selects the first 100 elements. If you want the output to include elements beyond the first 100 elements, specify a top value greater than 100.
`$skip=N`	Skips N elements and selects only the remaining elements starting with element N+1.

Pagination examples

The following examples show how to combine parameter values to control the elements returned in a vSphere deployment with 45 cloud accounts.

If you want to...	Use this request
List the first 20 cloud accounts, or 1–20	`curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts?apiVersion=$api_version&$filter=cloudAccountType%20eq%20%27vsphere%27&$top=20&$skip=0" | jq "."`
List the second set of 20 cloud accounts, or 21–40	`curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts?apiVersion=$api_version&$filter=cloudAccountType%20eq%20%27vsphere%27&$top=20&$skip=20" | jq "."`
List the third set of 20 cloud accounts, or 41–45	`curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts?apiVersion=$api_version&$filter=cloudAccountType%20eq%20%27vsphere%27&$top=20&$skip=40" | jq "."`

Count example

The following example shows how to count the AWS cloud accounts.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/cloud-accounts?$filter=cloudAccountType%20eq%20%27aws%27&$count=true" | jq "."
```

Filtering Resources by Region ID

You can use a filter in the IaaS APIs to identify resources provisioned in a cloud account of a particular region. By specifying the resources for which you want information such as network and security, compute, storage, and tags content, you limit the API response to provide only the information that you want.

You can also use filtering to identify and automatically update resources. For example, if you have automated builds on an vSphere private cloud account, you can use filtering to identify the image profiles associated with a region of the cloud account and automatically update those profiles in VCF Automation.

Obtaining the externalRegionID and cloudAccountId

To get information about a resource that is in a region of a cloud account, you filter by its externalRegionId and its cloudAccountId. The following example shows how to obtain the IDs for a vSphere cloud account and use them to construct a generic query. It assumes that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

1. List vSphere cloud accounts.

   ```
   curl -X GET "$url/iaas/api/cloud-accounts?apiVersion=$api_version&"'$filter='"cloudAccountType%20eq%20'vsphere'" -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"  | jq "."
   ```

2. Examine the response to obtain the cloud account ID and region ID for the vSphere account that you want.

   ```
   ...
          },
          "name": "vc60",
          "description": "Created by User",
          "id":
    "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341cba1f3-c39814fe67b8247557cab2652647d",
          "updatedAt": " ",
   ```

```
          "organizationId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
          "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
          "_links": {
            "regions": {
              "hrefs": [
                "/iaas/api/regions/277e3cd9fe87527557cab268cae5a"
              ]
            },
    ...
```

3. Assign the cloud account ID and region ID variables.

```
cloud_account_id='e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341cba1f3
c39814fe67b8247557cab2652647d'
region_id='277e3cd9fe87527557cab268cae5a'
```

4. List regions with the region ID and associated with the cloud account ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/regions/?apiVersion=$api_version&"'$filter'"=id%20eq%20'"$region_id"'%20and%20cloudAccountId%20eq
%20'"$cloud_account_id"'"  | jq "."
```

5. Examine the response to obtain the external region ID.

```
    ...
          {
            "externalRegionId": "Datacenter:datacenter-2",
            "name": "VC60-Datacenter",
            "cloudAccountId":
 "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341cba1f3-
c39814fe67b8247557cab2652647d",
            "id": "277e3cd9fe87527557cab268cae5a",
            "updatedAt": " 2022-04-02",
    ...
```

6. Assign the external region ID variable.

```
external_region_id='Datacenter:datacenter-2'
```

Using the externalRegionId and cloudAccountId, you can construct a generic filter to use in any IaaS API that queries for resources.

```
$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"
```

Constructing a query for a VMC cloud account

The VMC cloud account consists of an AWS cloud account facade and associated internal vSphere and NSX cloud accounts. To construct the query for a resource in the VMC cloud account, you use the cloudAccountId of the associated vSphere cloud account and not the cloudAccountId of the VMC cloud account.

The following example shows how to obtain the cloudAccountId and externalRegionId and use them to construct a generic query. It assumes that you know the name of the VMC cloud account and that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

1. Assign the cloud account name variable.

```
cloud_account_name='<my_vmc_cloud_account>'
```

2. List VMC cloud accounts with your VMC cloud account name.

```
curl -X GET "$url/iaas/api/cloud-accounts?apiVersion=$api_version&"'$filter='"cloudAccountType
%20eq%20'vmc'%20and%20name%20eq%20'$cloud_account_name'"  -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token"  | jq "."
```

3. Examine the response to obtain the cloud account ID for the associated vSphere cloud account.

   Under `_links`, two associated cloud accounts are listed. The first cloud account is the vSphere cloud account. The second is the NSX cloud account.

```
    ...
      "_links": {
```

```
                "regions": {
                    "hrefs": [
                        "/iaas/api/regions/5a8da7fe-63d9-4f82-84e8-f47bfdfef43c"
                    ]
                },
                "associated-cloud-accounts": {
                    "hrefs": [
                        "/iaas/api/cloud-accounts/df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05",
                        "/iaas/api/cloud-accounts/6993943b-82bc-4ec7-bad6-65f2f595da7e"
                    ]
                },
                "self": {
                    "href": "/iaas/api/cloud-accounts/95be1cc2-c70a-4311-9f18-6218786ac51b"
                }
            }

    ...
```

4. To assign the cloud account ID variable, use the associated vSphere cloud account.

```
assoc_cloud_account_id='df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05'
```

5. List regions in the associated vSphere cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
regions/?apiVersion=$api_version&"'$filter'"=cloudAccountId%20eq%20'"$assoc_cloud_account_id"'"  | jq "."
```

6. Examine the response to obtain the external region ID.

```
    ...
        {
           "externalRegionId": "Datacenter:datacenter-3",
           "cloudAccountId": "df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05",
           "id": "e572c96b-9084-4c37-b74d-bf61f0b08e79",
           "updatedAt": " 2022-04-02",
           "organizationId": "33a92056-18f0-469c-a088-9c8eae4e4888",
           "orgId": "33a92056-18f0-469c-a088-9c8eae4e4888",
           "_links": {
               "self": {
                   "href": "/iaas/api/regions/e572c96b-9084-4c37-b74d-bf61f0b08e79"
               },
               "cloud-account": {
                   "href": "/iaas/api/cloud-accounts/df59d7cd-d3ee-4bc3-bf4a-8a4027cefb05"
               }
           }
       }

    ...
```

7. Assign the external region ID variable.

```
external_region_id='Datacenter:datacenter-3'
```

Using the externalRegionId and cloudAccountId for the associated vSphere cloud account, you can construct a generic filter to use in any IaaS API that queries for resources in the VMC cloud account.

```
$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq
%20'$assoc_cloud_account_id'"
```

Resource Query Examples

The following examples show how to incorporate the filter into resource queries.

Resource	Query
Security Group	`$url/iaas/api/security-groups?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Security Group (NSX only)	`$url/iaas/api/security-groups?$filter="externalRegionId%20eq%20'global'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Region	`$url/iaas/api/regions?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Network	`$url/iaas/api/networks?$filter="externalRegionId%20eq%20'$region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Network Domains	`$url/iaas/api/network-domains?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Machine	`$url/iaas/api/machines?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Load Balancer	`$url/iaas/api/load-balancers?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Fabric vSphere Datastore	`$url/iaas/api/fabric-vsphere-datastores?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Fabric Network vSphere	`$url/iaas/api/fabric-networks-vsphere?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Fabric Network	`$url/iaas/api/fabric-networks?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Fabric Compute	`$url/iaas/api/fabric-computes?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Block Device	`$url/iaas/api/block-devices?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Network Profile	`$url/iaas/api/network-profiles?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Flavor Profile	`$url/iaas/api/flavor-profiles?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`

Resource	Query
Image Profiles	`$url/iaas/api/image-profiles?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Storage Profiles	`$url/iaas/api/storage-profiles?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Storage Profiles Azure	`$url/iaas/api/storage-profiles-azure?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Storage Profiles AWS	`$url/iaas/api/storage-profiles-aws?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`
Storage Profiles vSphere	`$url/iaas/api/storage-profiles-vsphere?$filter="externalRegionId%20eq%20'$external_region_id'%20and%20cloudAccountId%20eq%20'$cloud_account_id'"`

Filtering for Machine Status

To filter for machines with deployed or discovered status, you can use the IaaS APIs. A machine with the discovered status is in the cloud and has not yet been onboarded. A machine with the deployed status has been onboarded or provisioned from VCF Automation.

After onboarding, the machine has a deployed status and is associated with the deployment. Then you can manage it in the same way as any other provisioned machine.

To query for deployed or discovered status, you use the following parameters.

`$filter=deploymentId eq '*'`	Returns all machines that are deployed and under VCF Automation management. Deployed machines have a deployment ID.
`$filter=deploymentId ne '*'`	Returns all machines that are discovered in the cloud. Machines without the deployment ID property are not deployed.

Machine filter examples

The following examples show how to list deployed or discovered machines.

If you want to...	Use this request
List all deployed machines	`curl -X GET "$url/iaas/api/machines?apiVersion=$api_version&"'$filter='"deploymentId%20eq%20'*' -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`
List all discovered machines	`curl -X GET "$url/iaas/api/cloud-accounts?apiVersion=$api_version&"'$filter='"deploymentId%20ne%20'*' -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`

Filtering Operations for Projects

To restrict the output from a project endpoint, you include a filter in the `GET /project-service/api/projects` request. The API service uses the operation to filter for a collection of projects to paginate, order, or count your results.

Filter operation examples

The following examples show how to incorporate filter operations into project queries.

Operation	Query
equal	`$url/project-service/api/projects?$filter=name%20eq%20'00-TestProject'`
not equal	`$url/project-service/api/projects?$filter=name%20ne%20'00-TestProject'`
logical and	`$url/project-service/api/projects?$filter=name%20eq%20'00-TestProject'%20and%20sharedResources%20eq%20true`
logical or	`$url/project-service/api/projects?$filter=name%20eq%20'00-TestProject'%20or%20sharedResources%20eq%20false`
logical negation	`$url/project-service/api/projects?$filter=not%20startswith(name,'00')`
starts with	`$url/project-service/api/projects?$filter=startswith(name, '00')`
substring of	`$url/project-service/api/projects?$filter=substringof('00', name)`
ends with	`$url/project-service/api/projects?$filter=endswith(name, '00')`
length	`$url/project-service/api/projects?$filter=length(name)%20eq%205`
index of	`$url/project-service/api/projects?$filter=indexof(name,'00')%20eq%200`
substring from	`$url/project-service/api/projects?$filter=substring(name, 1)%20eq%20'0-Te'`
substring from to	`$url/project-service/api/projects?$filter=substring(name, 1, 2)%20eq%20'0-'`
to lower	`$url/project-service/api/projects?$filter=tolower(name)%20eq%20'00-testproject'`
to upper	`$url/project-service/api/projects?$filter=toupper(name)%20eq%20'00-TESTPROJECT'`
concatenate	`$url/project-service/api/projects?$filter=concat(concat(name,','),description)%20eq%20'test project,test project description'`
trim	`$url/project-service/api/projects?$filter=trim(name)%20eq%20'00-TestProject'`

Related query examples

The following examples show how to use related query options to paginate, order, or count your results.

If you want to...	Use this request
List the first twenty items	`curl -X GET $url/project-service/api/projects?$top=20 -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`
Skip the first ten items in the collection	`curl -X GET $url/project-service/api/projects?$skip=10 -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`

List the items in ascending order with `asc` or descending order with `desc`. If the type of order is not specified, the list defaults to ascending order.	For ascending order:
	`curl -X GET $url/project-service/api/projects?$skip=0$orderBy=name%20asc -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`
	For descending order:
	`curl -X GET $url/project-service/api/projects?$skip=0$orderBy=name%20desc -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`

List the total number of items in the collection.	`curl -X GET $url/project-service/api/projects?$count=true -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" | jq "."`

Setting up VCF Automation using APIs

As a VCF Automation administrator, you can use the Infrastructure as a Service (IaaS) APIs to set up connections with your cloud account vendor and integration applications in VCF Automation.

Adding Cloud Accounts

You use the IaaS APIs to create cloud accounts for various cloud platforms. VCF Automation uses permissions configured in the cloud accounts to collect data from regions or data centers, and to deploy blueprints to those regions.

After adding a cloud account, you use the cloud account ID to create a cloud zone, flavor mappings, image mappings, network, profiles, or storage profiles.

Add anAmazon Web Services Cloud Account

To create anAmazon Web Services cloud account with or without cloud zones, you make a POST request. The request body includes the parameters specific toAmazon Web Services that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have theAmazon Web Services access key ID and theAmazon Web Services secret access key for the new cloud account.

1. Assign theAmazon Web Services account variables.

```
aws_access_key_id='<your_aws_access_key_id>'
aws_secret_access_key='<your_aws_secret_access_key>
```

2. Submit a request to create anAmazon Web Services cloud account without default cloud zones.

   When the parameter `createDefaultZones` is left unspecified, the cloud account is created without default cloud zones by default.

```
curl -X POST \
```

```
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"privateKeyId": "'$aws_access_key_id'",
"cloudAccountType": "aws",
"name": "<your_aws_cloud_account>",
"description": "This is a demo AWS cloud account without cloud zones",
"regionIds": [ "<your_regions_id1>", "<your_region_id2>" ],
"privateKey": "'$aws_secret_access_key'"
}' | jq "."
```

3. Optional: Submit a request to create anAmazon Web Services cloud account with default cloud zones.

```
curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"privateKeyId": "'$aws_access_key_id'",
"cloudAccountType": "aws",
"name": "<your_aws_cloud_account>",
"description": "This is a demo AWS cloud account with default cloud zones",
"regionIds": [ "<your_region_id1>", "<your_region_id2>" ],
"createDefaultZones": true,
"privateKey": "'$aws_secret_access_key'"
}' | jq "."
```

4. To obtain your cloud account ID, examine the response.
5. Assign the cloud account ID variable to your cloud account ID.

```
cloud_account_id='<your_cloud_account_id>'
```

6. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token"  | jq "."
```

7. Optional: To look up the cloud account you created, list the cloud account with your cloud account ID.

```
curl -X GET $url/iaas/api/cloud-accounts/$cloud_account_id?apiVersion=$api_version -H 'Content-Type:
 application/json' -H "Authorization: Bearer $access_token" | jq "."
```

The response shows the cloud account name and ID for theAmazon Web Services cloud account you created.

Create anAmazon Web Services Cloud Account

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ aws_access_key_id='FEDCBA5JJ2F43TUVW7XA'
$ aws_secret_access_key='XYZfGo0/Vb5XPezeC58QRSvLMNOwsHhuB2IbEJxL'
```

Create a cloud account named `demo-aws-account` without default cloud zones.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"privateKeyId": "'$aws_access_key_id'",
"cloudAccountType": "aws",
```

```
"name": "demo-aws-account",
"description": "This is a demo AWS cloud account without cloud zones",
"regionIds": [ "us-east-1", "us-west-1" ],
"privateKey": "'$aws_secret_access_key'"
}' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
"tags": [],
"name": "demo-aws-account",
"id": "c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add avSphere Cloud Account

To create avSphere cloud account, you make a POST request. The request body includes the parameters specific tovSphere that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the following parameters for the new cloud account:
  - vSphere host name
  - vSphere user name
  - vSphere password
- Verify that you have an existingvSphere,NSX,NSX-V, or VMC cloud account that you want to associate with the new cloud account and obtain the cloud account ID.
  a. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token"  | jq "."
```

  b. Examine the response to obtain the cloud account ID such as the `id` value in this example.

```
...
    "name": "vsphere-account-example",
    "id": "b9fa1b42c767de7558ceff3b78004",
    "updatedAt": "2022-04-02",
    "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
...
```

The following procedure shows how to create avSphere cloud account that supports a trusted certificate. To obtain a trusted certificate, you submit a request to validate asynchronously with thevSphere cloud account specification. When the validation request completes successfully, you use the certificate ID from the response to obtain the trusted certificate that you submit when you create thevSphere cloud account.

1. List all cloud proxies.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
data-collectors?apiVersion=$api_version" | jq "."
```

2. To obtain the data collector ID, examine the response.
3. Assign the data collector ID variable.

```
data_collector_id='<your_datacollector_id>'
```

4. Assign thevSphere account variables.

```
vsphere_host_name='<your_vsphere_host_name>'
vsphere_user='<your_vsphere_user_name>'
vsphere_password='<your_vsphere_password>'
```

5. List external region IDs from avSphere cloud account.

```
curl -X POST \
"$url/iaas/api/cloud-accounts-vsphere/region-enumeration?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"cloudAccountType": "vsphere",
"username": "'$vsphere_user'",
"password": "'$vsphere_password'",
"hostName": "'$vsphere_host_name'",
"dcid": "'$data_collector_id'",
"acceptSelfSignedCertificate": "false"
}' | jq "."
```

6. To obtain the external region ID, examine the response and assign the region ID variable.

```
vsphere_region_id='<your_vsphere_region_id>'
```

7. Submit a request to validate asynchronously with thevSphere cloud account specification.

```
curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
cloud-accounts-vsphere?apiVersion=$api_version&validateOnly" | jq "."
```

   The response includes a selfLink.

```
{
"progress": 0,
"status": "INPROGRESS",
"name": "Cloud account specification validation",
"id": "example-selfLink-alphanumeric-string",
"selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

8. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

9. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

   When the validation request completes successfully, the response includes a resource with the certificate ID.

```
{
"progress": 0,
"message": "valid certificate found",
"status": "SUCCEEDED",
"resources": [
    "/iaas/api/certificates/example-certificate-id-string"
],
"name": "Cloud account specification validation",
"id": "example-selfLink-alphanumeric-string",
"selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

10. Assign the certificate ID variable.

```
certificate_id='example-certificate-id-string'
```

11. Submit a request to get the certificate information.

```
PEM_for_X509Certificate='curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer
 $access_token" "$url/iaas/api/certificates/$certificate_id?apiVersion=$api_version" | jq "."'
```

12. Assign the ID of the existing cloud account to associate with the new cloud account.

```
existing_cloud_account_ID='<your_existing_cloud_account_ID>'
```

13. Include the certificate in the request to create avSphere cloud account with default cloud zones.

    To create avSphere cloud account without default cloud zones, use `"createDefaultZones":false`.

```
curl -X POST \
"$url/iaas/api/cloud-accounts-vpshere?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "demo-vsphere-account",
  "description": "This is a demo vSphere account with default cloud zones",
  "username": "'$vsphere_user'",
  "password": "'$vsphere_password'",
  "hostName": "'$vsphere_host_name'",
  "acceptSelfSignedCertificate":false,
  "associatedCloudAccountIds": "'$existing_cloud_account_ID'",
  "createDefaultZones":true,
  "dcId": "'$data_collector_id'",
  "regions":[
    {
       "name": "'$vsphere_region_id'",
       "ExternalRegionId":"'$vsphere_region_id'"
    }
  ],
  "tags": [
    {
      "key": "env",
      "value": "dev"
    }
  ],
  "certificateInfo":{
     "certificate": "'$PEM_for_X509Certificate'"
     }
}' | jq "."
```

    Note:
    The following example shows how to create avSphere cloud account with multiple cloud account IDs. However, the payload can only include a single NSX-P-Cloud endpoint and a single VMC endpoint.

```
  "associatedCloudAccountIds": "[\"'$existing_NSXT_cloud_account_ID'\",
   \"'$existing_VMC_cloud_account_ID'\", \"'$existing_vSphere_cloud_account_ID'\"]",
```

14. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token" | jq "."
```

15. Examine the response and verify that the name and ID of thevSphere cloud account you created is listed.

    Create avSphere Cloud Account

    This example creates a cloud account with default cloud zones.

    Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
```

    List all cloud proxies.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/data-collectors?apiVersion=$api_version" | jq "."
```

A snippet of the response from your request shows the data collector IDs.

```
...
    {
    "dcId": "60740040-f3cd-4694-96da-15e547242bf7",
    "ipAddress": "10.108.78.154",
    "name": "example-prod-corp-rdc",
    "hostName": "corp-v783-dhcp-79-85.eng.mycompany.com",
    "status": "ACTIVE"
  },
...
```

Assign the data collector ID variable.

```
$ data_collector_id='60740040-f3cd-4694-96da-15e547242bf7'
```

Assign thevSphere account variables.

```
$ vsphere_host_name='corp-v783-dhcp-79-85.eng.mycompany.com'
$ vsphere_user='admin@mycompany.com'
$ vsphere_password='my_vsphere_password'
```

List external region IDs from yourvSphere cloud account.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts-vsphere/region-enumeration?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
"cloudAccountType": "vsphere",
"username": "'$vsphere_user'",
"password": "'$vsphere_password'",
"hostName": "'$vsphere_host_name'",
"dcid": "'$data_collector_id'",
"acceptSelfSignedCertificate": "false"
}' | jq "."
```

A snippet of the response shows the region ID to use.

```
...
{
"externalRegionIds": [
  "Datacenter:datacenter-2"
]
}
...
```

Assign the region ID variable.

```
$ vsphere_region_id='Datacenter:datacenter-2'
```

Submit request to validate asynchronously with thevSphere cloud account specification.

```
$ curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/cloud-accounts-vsphere?apiVersion=$api_version&validateOnly" | jq "."
```

A snippet of the response shows the selfLink.

```
{
"progress": 0,
"status": "INPROGRESS",
```

```
"name": "Cloud account specification validation",
"id": "bcdee18-a77d-46f8-b068-4013e80e2b55",
"selfLink": "/.../request-tracker/bbcdee18-a77d-46f8-b068-4013e80e2b55"
}
```

Assign the selfLink variable.

```
$ selfLink_id='bbcdee18-a77d-46f8-b068-4013e80e2b55'
```

Submit a request to track the request with the selfLink.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

When the request succeeds, the response shows the resource with the certificate ID.

```
{
"progress": 0,
"message": "valid certification path to requested target found",
"status": "SUCCEEDED",
"resources": [
    "/iaas/api/certificates/7fe4c108-64ff-4347-92de-b0790bda1a3c?apiversion=2021-07-15"
],
"name": "Cloud account specification validation",
"id": "bbcdee18-a77d-46f8-b068-4013e80e2b55",
"selfLink": "/iaas/api/request-tracker/bbcdee18-a77d-46f8-b068-4013e80e2b55"
}
```

Assign the certificate ID variable.

```
$ certificate_id='7fe4c108-64ff-4347-92de-b0790bda1a3c'
```

To get certficate information, submit a request with the certificate ID.

```
$ PEM_for_X509Certificate='curl -X GET -H 'Content-Type: application/json' -H "Authorization:
 Bearer $access_token" "$url/iaas/api/certificates/$certificate_id?apiVersion=$api_version" | jq
 "."'
```

Assign the ID of the existing cloud account to associate with the new cloud account.

```
$ existing_cloud_account_id ='b9fa1b42c767de7558ceff3b78004'
```

Create a cloud account named `demo-vsphere-account` with default cloud zones.

```
$ curl -X POST \
"$url/iaas/api/cloud-accounts-vpshere?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name": "demo-vsphere-account",
  "description": "This is a demo vSphere account with default cloud zones",
  "username": "'$vsphere_user'",
  "password": "'$vsphere_password'",
  "hostName": "'$vsphere_host_name'",
  "acceptSelfSignedCertificate":false,
  "associatedCloudAccountIds": "'$existing_cloud_account_id'",
  "createDefaultZones":true,
  "dcId": "'$data_collector_id'",
  "regions":[
    {
       "name": "'$vsphere_region_id'",
       "ExternalRegionId":"'$vsphere_region_id'"
    }
```

```
  ],
  "tags": [
    {
      "key": "env",
      "value": "dev"
    }
  ],
  "certificateInfo":{
     "certificate": "'$PEM_for_X509Certificate'"
     }
}' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
    "tags": [],
    "name": "demo-vsphere-account",
    "id": "515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a8da0",
    "updatedAt": "2022-04-02",
    "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add an NSX Cloud Account

To create an NSX Manager cloud account, you make a POST request. The request body includes the NSX-specific parameters required to create the cloud account. NSX Manager cloud accounts use policy APIs to communicate with NSX.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the following parameters for the new cloud account:
  - NSX host name
  - NSX user name
  - NSX password

As an alternative to using the `cloud-accounts` API call, you can use a `cloud-accounts-nsx-t` API call to list NSX Manager cloud accounts respectively.

1. Assign the NSX account variables.

   ```
   nsx_host_name='<your_nsx_host_name>'
   nsx_username='<your_nsx_user_name>'
   nsx_password='<your_nsx_password>'
   ```

2. Optional: If you have a vSphere cloud account that you want to associate with the NSX cloud account, assign the ID of the vSphere cloud account to the `vsphere_cloud_account_id` variable. See Add avSphere Cloud Account .

   ```
   vpshere_cloud_accout_id='<your_vsphere_cloud_account_id>'
   ```

3. Submit a request to create an NSX Manager cloud account, use `"cloudAccountType": "nsxt"`. This example includes the `vsphere_cloud_account_id` variable.

   ```
   curl -X POST \
     "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
     "cloudAccountType": "nsxt",
     "privateKeyId": "'$nsx_username'",
     "privateKey": "'$nsx_password'",
     "associatedCloudAccountIds": [
       "'$vsphere_cloud_account_id'"
   ```

```
  ],
  "cloudAccountProperties": {
    "hostName": "'$nsx_host_name'",
    "acceptSelfSignedCertificate": "true",
    "privateKeyId": "'$nsx_username'",
    "privateKey": "'$nsx_password'"
  },

  "tags": [
    {
      "key": "env",
      "value": "prod"
    }
  ],
  "name": "<your_nsx_cloud_account>",
  "description": "Example NSX cloud account description"
}' | jq "."
```

4. To obtain the NSX cloud account ID, examine the response.
5. Assign the NSX cloud account ID variable.

   ```
   nsx_cloud_id='<your_nsx_cloud_id>'
   ```

6. List all cloud accounts.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   cloud-accounts?apiVersion=$api_version"  | jq "."
   ```

7. Optional: List all NSX Manager cloud accounts using the `cloud-accounts-nsx-t` API call.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   cloud-accounts-nsx-t?apiVersion=$api_version"  | jq "."
   ```

8. Examine the response and verify that the name and ID of the NSX cloud account you created is listed.

   Note:
   The ID of the associated vSphere cloud account appears under `_links` and `hrefs` as in the following code snippet example.

```
...
      "name": "nsx manager",
      "id": "515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-d5a5e16bdc3eec75572358fd24ab6",
      "updatedAt": " ",
      "organizationId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "_links": {
        "associated-cloud-accounts": {
          "hrefs": [
            "/iaas/api/cloud-accounts/515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a8da0"
          ]
        },
        "self": {
          "href": "/iaas/api/cloud-accounts/515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-d5a5e16bdc3eec75572358fd24ab6"
...
```

Create an NSX Manager Cloud Account

This example creates an NSX Manager cloud account that includes an existing vSphere cloud account.

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
```

Assign the NSX account variables.

```
$ nsx_host_name='nsx-manager.mycompany.local'
$ nsx_username='admin'
$ nsx_password='my_nsx_password'
```

Assign the account variables for your existing vSphere cloud account.

```
$ vsphere_cloud_account_id='515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a8da0'
```

Create an NSX Manager cloud account named `demo-nsx-account`.

```
$ curl -X POST \
  "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
  "cloudAccountType": "nsxt",
  "privateKeyId": "'$nsx_username'",
  "privateKey": "'$nsx_password'",
  "associatedCloudAccountIds": [
    "'$vsphere_cloud_account_id'"
  ],
  "cloudAccountProperties": {
    "hostName": "'$nsx_host_name'",
    "acceptSelfSignedCertificate": "true",
    "privateKeyId": "'$nsx_username'",
    "privateKey": "'$nsx_password'"
  },

  "tags": [
    {
      "key": "env",
      "value": "prod"
    }
  ],
  "name": "demo-nsx-account",
  "description": "Example NSX cloud account description"
}' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
      "tags": [],
      "name": "demo-nsx-account",
      "id": "7b2c48362c94567559080d8f575a2",
      "updatedAt": " ",
      "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
      "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add aVMware Cloud on AWS Cloud Account with a Proxy

Adding aVMware Cloud on AWS cloud account with a proxy requires manual deployment of a cloud proxy VM. Then you make a POST request with a request body that includes the data collector ID along with parameters specific toVMware Cloud on AWS.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

- Verify that a cloud proxy VM has been manually deployed.
- Verify that you have the following parameters for the new cloud account:
  - VMC API token.
  - SDDC name.
  - vCenter private IP.
  - NSX Manager IP.
  - vCenter user name.
  - vCenter password.
  - vCenter data center ID.

1. List all cloud proxies.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
data-collectors?api_version=$api_version" | jq "."
```

2. Examine the response and assign the data collector variable.

```
data_collector_id='<your_datacollector>'
```

3. Assign the account variables for the VMC cloud account.

```
vmc_api_token='<your_vmc_api_token>'
vmc_sddc_name='<your_vmc_sddc_name>'
vmc_vcenter_private_ip='<your_vcenter_private_ip>'
vmc_nsx_manager_ip='<your_nsx_manager_ip>'
vmc_vcenter_username='<your_vcenter_username>'
vmc_vcenter_password='<your_vcenter_password>'
vmc_vcenter_datacenter_id='<your_datacenter_id>'
```

4. Submit a request to create a VMC cloud account.

```
curl -X POST \
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
   "name": "vmc-endpoint",
   "description": "VMC cloud account",
   "cloudAccountType": "vmc",
   "privateKeyId": "'"$vmc_vcenter_username"'",
   "privateKey": "'"$vmc_vcenter_password"'",
   "cloudAccountProperties": {
       "sddcId": "'"$vmc_sddc_name"'",
       "apiKey": "'"$vmc_api_token"'",
       "hostName": "'"$vmc_vcenter_private_ip"'",
       "nsxHostName": "'"$vmc_nsx_manager_ip"'",
       "dcId": "'"$vmc_data_collector_id"'",
       "acceptSelfSignedCertificate": "false"
   },
   "regionIds": [
       "'"$vmc_vcenter_datacenter_id"'"
   ]
}' | jq "."
```

The response includes a selfLink value.

5. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

6. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
request-tracker/$selfLink_id | jq "."
```

TheVMware Cloud on AWS cloud account is created when the response shows `"status": "FINISHED"`.

Create a VMC Cloud Account with a Proxy

This example creates aVMware Cloud on AWS cloud account with a cloud proxy VM that has been manually deployed.

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
```

List all cloud proxies.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/data-collectors?api_version=$api_version" | jq "."
```

A snippet of the response from your request shows the data collector IDs.

```
...
   {
     "dcId": "cd7d1eb4-573f-4150-8206-de3d536490ca",
     "ipAddress": "10.139.116.60",
     "name": "localhost.localdom",
     "hostName": "localhost.localdom",
     "status": "ACTIVE"
   },,
...
```

Assign the data collector ID variable.

```
$ data_collector_id='cd7d1eb4-573f-4150-8206-de3d536490ca'
```

Assign the variables for the VMC cloud account.

```
$ vmc_data_collector_id=a1235a7f-d49f-4365-8ed9-2d7d0805e4bc
$ vmc_api_token=ab392fba-32a8-49a5-a084-d422fa32c5b8
$ vmc_sddc_name=MYCOM-PRD-NSXT-M7GA-052019
$ vmc_vcenter_private_ip=10.70.57.196
$ vmc_nsx_manager_ip=10.70.57.131
$ vmc_vcenter_username=cloudadmin@vmc.local
$ vmc_vcenter_password=aBcqCW+m4+XEQg7
$ vmc_vcenter_datacenter_id=Datacenter:datacenter-1
```

Create a VMC cloud account named `demo-vmc-account`.

```
$ curl -X POST \
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
   "name": "demo-vmc-account",
   "description": "VMC cloud account",
   "cloudAccountType": "vmc",
   "privateKeyId": "'"$vmc_vcenter_username"'",
   "privateKey": "'"$vmc_vcenter_password"'",
   "cloudAccountProperties": {
       "sddcId": "'"$vmc_sddc_name"'",
       "apiKey": "'"$vmc_api_token"'",
       "hostName": "'"$vmc_vcenter_private_ip"'",
       "nsxHostName": "'"$vmc_nsx_manager_ip"'",
       "dcId": "'"$vmc_data_collector_id"'",
       "acceptSelfSignedCertificate": "false"
```

```
    },
    "regionIds": [
        "'"$vmc_vcenter_datacenter_id"'"
    ]
}' | jq "."
```

The response includes a selfLink variable.

```
{
   "progress": 0,
   "status": "INPROGRESS",
   "name": "Cloud account creation/update",
   "id": "0dc374ba-08ec-4422-8615-24f4f94ef5aa",
   "selfLink": "/iaas/api/request-tracker/0dc374ba-08ec-4422-8615-24f4f94ef5aa"
}
```

Assign the selfLink variable.

```
selfLink_id='0dc374ba-08ec-4422-8615-24f4f94ef5aa'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id | jq "."
```

When the request completes successfully, the response shows the cloud account ID at the end of the `resources` path.

```
{
   "progress": 100,
   "status": "FINISHED",
   "resources": [
       "/iaas/api/cloud-accounts/e6af1aa6-dc7a-4847-8adc-c4c73727e5b3"
   ],
   "name": "Cloud account creation/update",
   "id": "a85d1476-1b11-45b0-8d14-91951385c95d",
   "selfLink": "/iaas/api/request-tracker/a85d1476-1b11-45b0-8d14-91951385c95d"
}
```

Add aMicrosoft Azure Cloud Account

To create aMicrosoft Azure cloud account, you make a POST request. The request body includes parameters specific toMicrosoft Azure that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the following parameters for the new cloud account:
  - Microsoft Azure subscription ID
  - Microsoft Azure tenant ID
  - Microsoft Azure client application ID
  - Microsoft Azure client application secret key

As an alternative to using the `cloud-accounts` API call, you can use a `cloud-accounts-azure` API call that creates aMicrosoft Azure cloud account with fewer input parameters.

1. Assign theMicrosoft Azure account variables.

   ```
   azure_subscription_id='<your_azure_subscription_id>'
   azure_tenant_id='<your_azure_tenant_id>'
   azure_client_application_id='<your_azure_client_application_id>'
   azure_client_application_secret_key='<your_azure_client_application_secret_key>'
   ```

2. Submit a request to create aMicrosoft Azure cloud account with default cloud zones.

   ```
   curl -X POST \
   ```

```
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "cloudAccountType":"azure",
   "privateKeyId":"'$azure_client_application_id'",
   "privateKey":"'$azure_client_application_secret_key'",
   "cloudAccountProperties": {
     "userLink":"'$azure_subscription_id'",
     "azureTenantId":"'$azure_tenant_id'"
   },
   "regionIds":["<your_region_id>"],
   "createDefaultZones":true,
   "name":"<your_azure_cloud_account>",
   "description": "This is a demo Azure cloud account",
 }' | jq "."
```

3. Optional: Submit a request to create aMicrosoft Azure cloud account with the `cloud-accounts-azure` API call.

```
curl -X POST \
 "$url/iaas/api/cloud-accounts-azure?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "name":"<your_azure_cloud_account>",
   "description": "This is a demo Azure cloud account",
   "subscriptionId": "'$azure_subscription_id'",
   "tenantId": "'$azure_tenant_id'",
   "clientApplicationId": "'$azure_client_application_id'",
   "clientApplicationSecretKey": "'$azure_client_application_secret_key'",
   "regionIds": [ "<your_region_id1>", "<your_region_id2>" ],
   "createDefaultZones": true,
   "tags": [ { "key": "env", "value": "dev" } ]
 }' | jq "."
```

4. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token" | jq "."
```

5. Examine the response and verify that the name and ID of theMicrosoft Azure cloud account you created is listed.

Create aMicrosoft Azure Cloud Account

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ azure_subscription_id='r1e31415-4a08-4072-be4a-19de37d12345'
$ azure_tenant_id='s39138ca-3abc-4b4a-a4d6-cd92d9dd62f0'
$ azure_client_application_id='te21wxyz-b183-42ac-cd84-3c4a2459b9a9'
$ azure_client_application_secret_key='udv6lY8MwpP5ABCDfSztP3ABCDEaLMNOPQRmDEUeiI0='
```

Create a cloud account named `demo-azure-account`.

```
$ curl -X POST \
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "cloudAccountType":"azure",
   "privateKeyId":"'$azure_client_application_id'",
```

```
    "privateKey":"'$azure_client_application_secret_key'",
    "cloudAccountProperties": {
      "userLink":"'$azure_subscription_id'",
      "azureTenantId":"'$azure_tenant_id'"
    },
    "regionIds":["eastus"],
    "createDefaultZones":true,
    "name": "demo-azure-account",
    "description": "This is a demo Azure cloud account",
  }' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
"tags": [],
"name": "demo-azure-account",
"id": "c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-02",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Add a Google Cloud Platform Cloud Account

To create aGoogle Cloud Platform (GCP) cloud account, you make a POST request. The request body includes parameters specific toGoogle Cloud Platform that are required to create the cloud account.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the following parameters for the new cloud account:
  - GCP project ID
  - GCP private key ID
  - GCP private key
  - GCP client email

1. Assign the GCP account variables.

```
gcp_project_id='<your_gcp_project_id>'
gcp_private_key_id='<your_gcp_private_key_id>'
gcp_private_key='<your_gcp_private_key>'
gcp_client_email='<your_gcp_client_email>'
```

2. Submit a request to create a GCP cloud account with default cloud zones.

```
curl -X POST \
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "cloudAccountType":"gcp",
   "privateKeyId":"'$gcp_private_key_id'",
   "privateKey":"'$gcp_private_key'",
   "cloudAccountProperties":{
     "projectId":"'$gcp_project_id'",
     "clientEmail":"'$gcp_client_email'"
   },
   "regionIds":["<your_region_id>"],
   "createDefaultZones":true,
   "name":"<your_gcp_cloud_account>",
   "description": "This is a demo GCP cloud account",
 }' | jq "."
```

3. List all cloud accounts.

```
curl -X GET $url/iaas/api/cloud-accounts?apiVersion=$api_version -H 'Content-Type: application/json' -H
 "Authorization: Bearer $access_token" | jq "."
```

4. Examine the response and verify that the name and ID of the GCP cloud account you created is listed.

Create aGoogle Cloud Platform Cloud Account

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ gcp_project_id='Example-e2e'
$ gcp_private_key_id='defg3c20c85abcde6a95b44222c4c1d68554b87e'
$ gcp_private_key='-----BEGIN PRIVATE KEY-----\\nMIIEvQIBADANBgkqhkiG9w0BAQEFAA
...
3izE4KDeebLh7SkWFbUt7lFW25UL20\\nKAY7FRTKpvbO+6Z/BnVePVI=\\n-----END PRIVATE KEY-----\\n'
$ gcp_client_email='123456789123-example@developer.gserviceaccount.com'
```

Create the cloud account.

```
$ curl -X POST \
 "$url/iaas/api/cloud-accounts?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "cloudAccountType":"gcp",
   "privateKeyId":"'$gcp_private_key_id'",
   "privateKey":"'$gcp_private_key'",
   "cloudAccountProperties":{
     "projectId":"'$gcp_project_id'",
     "clientEmail":"'$gcp_client_email'"
   },
   "regionIds":["us-west2"],
   "createDefaultZones":true,
   "name":"demo-gcp-account",
   "description": "This is a demo GCP cloud account",
 }' | jq "."
```

A snippet of the response from your request shows the account ID.

```
...
"tags": [],
"name": "demo-GCP-account",
"id": "c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"updatedAt": "2022-04-02",
"organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
...
```

Integrating with other applications

To fully automate your organization setup, you can use the IaaS APIs to list, create, update, and delete VCF Automation integrations with other applications programmatically.

Using different input variables, you can create integrations with external systems such as GitHub, Ansible, and external IPAM providers. The following table lists all supported integrations.

Integration	Type	What do I need to create an integration?
Active Directory	`activedirectory`	• LDAP connection to Active Directory server. • Project configured with appropriate cloud zones, and image and flavor mappings to use with the Active Directory integration.
Ansible	`ansible`	• Ansible control machine running Ansible version 2.6.0 or later. • Read/write access to the directory where the Ansible inventory file is located. • Deactivated host key. • Vault password set.
Ansible Tower	`ansible.tower`	Credentials and templates in Ansible Tower configured to use with deployments.
Bitbucket	`org.bitbucket`	Personal access API token for Bitbucket.
GitHub	`com.github.saas`	Personal access API token for GitHub. See https://docs.gitlab.com/ee/user/profile/personal_access_tokens.html
GitLab	`com.gitlab.saas`	Personal access API token for GitLab.
IPAM	`ipam`	• IPAM provider package. • Administrator credentials for the account with the external IPAM provider.
Puppet	`puppet`	Puppet master name and hostname or IP address of the master.
Red Hat OpenShift	`cmx.openshift-endpoint`	Red Hat OpenShift server configured. **Note:** If needed, you can create an OpenShift cluster with a blueprint that VMware provides. See https://flings.vmware.com/red-hat-openshift-container-platform-as-a-service-on-vrealize-automation-cloud.
SaltStack	`saltstack`	• Salt master used in the VCF Automation integration that contains the Master Plugin. • VCF Automation service administrator role in VCF Automation. • VCF Automation service administrator role in VCF Automation.
SDDC Manager	`sddc`	SDDC manager 4.1 or later installed.
Terraform Runtime	`terraform.runtime`	Terraform runtime environment.
VMware Enterprise PKS	`cmx.pks-endpoint`	• Pivotal Container Service (PKS) server configured with UAA authentication. • Cloud administrator credentials.
VCF Operations orchestrator	`vro`	• VCF Operations orchestrator configured. • VCF Operations orchestrator URL • Cloud Extensibility proxy deployed.

Integration	Type	What do I need to create an integration?
VCF Operations	`vrops`	A local or non-local VCF Operations login account with read-only privileges to the vCenter adapter instance of the vSphere endpoint. **Note:** For non-local account login, username format is username@domain@authenticated-source, for example `jdoe@company.com@workspaceone`

Create an Integration with Github

To create an integration with GitHub, you make a POST request. The request body includes properties specific to GitHub.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have a personal access token for authentication to GitHub. See https://docs.gitlab.com/ee/user/profile/personal_access_tokens.html.

1. Assign variables for GitHub.

```
private_key='<your_GitHub_personal_access_token>'
```

2. Submit a request to create a GitHub integration.

```
curl -X POST \
 "$url/iaas/api/integrations?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "integrationType":"com.github.saas",
   "name":"<your_github_integration>",
   "privateKey":"'$private_key'",
   "integrationProperties":{
     "url":"https://api.github.com"
   },
 }' | jq "."
```

   The response includes a selfLink.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Integration creation/update",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

3. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

4. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

   When the request completes successfully, the response includes a resource with the integration ID.

```
{
 "progress": 100,
 "status": "FINISHED",
```

```
 "resources": [
     "/iaas/api/integrations/example-integration-id-string"
 ],
 "name": "Integration creation/update",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

5. Assign the integration ID variable.

```
integration_id='example-integration-alphanumeric-string'
```

6. Use the integration ID variable to list the integration.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
integrations/$integration_id?apiVersion=$api_version" | jq "."
```

7. Examine the response and verify that the name and ID of the integration that you created is listed.

Create a GitHub integration

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ private_key='8bc9401b5d28f4ec126929af0dc4e99dd0792b0f'
```

Create the integration.

```
$ curl -X POST \
 "$url/iaas/api/integrations?apiVersion=$api_version" \
 -H 'Content-Type: application/json' \
 -H "Authorization: Bearer $access_token" \
 -d '{
   "integrationType":"com.github.saas",
   "name":"Git integration example",
   "privateKey":"'$private_key'",
   "integrationProperties":{
     "url":"https://api.github.com"
   },
 }' | jq "."
```

The response includes a selfLink.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Integration creation/update",
 "id": "a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597",
 "selfLink": "/iaas/api/request-tracker/a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597"
}
```

Assign the selfLink variable

```
selfLink_id='a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

When the request completes successfully, the response includes a resource with the integration ID.

```
{
```

```
 "progress": 100,
 "status": "FINISHED",
 "resources": [
     "/iaas/api/integrations/e5dda941-bb17-4f19bd15-7db0b8eab88c"
 ],
 "name": "Integration creation/update",
 "id": "a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597",
 "selfLink": "/iaas/api/request-tracker/a0c5eb3a-9ffa-4bfb-b63b-c77510bcc597"
}
```

Assign the integration ID variable

```
integration_id='e5dda941-bb17-4f19bd15-7db0b8eab88c'
```

Use the integration ID variable to list the integration.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "."
```

When the request completes successfully, a snippet of the response shows integration details.

```
...
   {
      "integrationType": "com.github.saas",
      "integrationProperties": (
         "url": "https://api.github.com"
      },
      "name":"Git integration example",
      "id: "e5dda941-bb17-4f19bd15-7db0b8eab88c"
      "createdAt":"2022-04-02"
      "updatedAt": "2022-04-02"
      "orgId": "ce811934-ea1a-4f53-6bec-4656ca7d126",
      "_links": {
         "self": {
            "href": "/iaas/api/integrations/e5dda941-bb17-4f19bd15-7db0b8eab88c"
         }
      }
   }
...
```

Delete an Integration

To delete an integration, you make a DELETE request with the ID of the integration.

- Verify that all general prerequisites and prerequisites for theVCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the integration that you want to delete. See Create an Integration with Github.

The following procedure shows how to delete an integration including an optional step to list the integration details before deleting the integration. It is a good practice to check the details of the integration so that you delete the correct integration.

1. Assign your integration ID variable.

   ```
   integration_id='<your_integration_id>'
   ```

2. Optional: List the integration before deleting.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   integrations/$integration_id?apiVersion=$api_version" | jq "."
   ```

3. Optional: Examine the response to verify the integration details, such as integration name and integration type.
4. Submit a request to delete the integration.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/integrations/$integration_id?apiVersion=$api_version" | jq "."
```

The response includes a selfLink.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Integration deletion",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

5. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

6. Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
request-tracker/$selfLink_id | jq "."
```

When the request completes successfully, the response shows the status as `FINISHED`.

```
{
 "progress": 100,
 "message": "Deleted",
 "status": "FINISHED",
 "name": "Integration deletion",
 "id": "example-selfLink-alphanumeric-string",
 "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

Delete an integration

Assign the required variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
$ integration_id='e5dda941-bb17-4f19bd15-7db0b8eab88c'
```

Delete the integration.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/integrations/$integration_id?apiVersion=$api_version" | jq "."
```

The response includes a selfLink.

```
{
 "progress": 0,
 "status": "INPROGRESS",
 "name": "Integration deletion",
 "id": "fe472e75-f346-4de7-bbc4-5edddefd9dfa",
 "selfLink": "/iaas/api/request-tracker/fe472e75-f346-4de7-bbc4-5edddefd9dfa"
}
```

Assign the selfLink variable

```
selfLink_id='fe472e75-f346-4de7-bbc4-5edddefd9dfa'
```

Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id | jq "."
```

When the request completes successfully, the response shows the status as `FINISHED`.

```
{
 "progress": 100,
 "message": "Deleted",
 "status": "FINISHED",
 "name": "Integration deletion",
 "id": "fe472e75-f346-4de7-bbc4-5edddefd9dfa",
 "selfLink": "/iaas/api/request-tracker/fe472e75-f346-4de7-bbc4-5edddefd9dfa"
}
```

How do I import an IPAM package

To import an IPAM package using the IaaS API, you reserve enough space for the package then you make multiple PATCH requests to import smaller pieces of the package. After importing all the pieces, you use a POST request to upload the file onto the server.

- Verify that all general prerequisites and prerequisites for theVCF Automation IaaS API have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you know the size of the package that you want to upload. If the package is large, divide it into smaller packages. The following example divides the large package into smaller packages that are each 9000000 bytes in size.

  ```
  split -b 9000000 <your_package>.zip <your_package>.zip_split
  ```
- If using the API to complete the IPAM integration with a cloud account, verify that you have the following parameters for your cloud account:
  - *`privateKeyId`* used to create your cloud account.
  - *`privateKey`* used to create your cloud account.
  - *`hostName`* used to create your cloud account.
  - *`faasProviderEndpointID`* is your cloud account ID.

The IaaS API implements the TUS RFC protocol which provides a mechanism for resumable file uploads. By dividing a large package into smaller ZIP packages and importing the packages individually, you ensure that you will not exceed the server's file size limit with your upload.
For more information about the TUS RFC protocol, see https://github.com/tus/tus-resumable-upload-protocol/blob/main/protocol.md.

This procedure shows how to import your IPAM package using the integrations IPAM APIs. Then you complete the integration using theVCF Automation UI or the integrations API.

1. Reserve space for the package with `Upload-Length` equal to the size of the package in bytes.

   ```
   curl -X POST \
    "$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
    -H "Authorization: Bearer $access_token" \
    -H "Upload-Length: <package_size_in_bytes>" \
    -H "Tus-Resumable: 1.0.0" | jq "."
   ```

   Examine the response header to get the location for the package.

   Note:
   You must complete the package import process before the location reservation expires in approximately two hours.

2. Assign a variable for the location ID.

   ```
   location_id = '<package_location>'
   ```

3. Import the first small package with `Upload-Offset` equal to zero.

   ```
   curl -X PATCH \
    "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
    -H "Authorization: Bearer $access_token" \
    -H "Tus-Resumable: 1.0.0" \
    -H "Content-Type: application/offset+octet-stream" \
    -H "Upload-Offset: 0" | jq "."
   ```

A successful response returns `204 No Content` and includes an upload offset number. Use that number in the PATCH request for the next package.

4. Import the next small package.
   - You must import the small packages in a sequential order.
   - `Upload-Offset` is the offset value from the response header of the previous PATCH request.

```
curl -X PATCH \
 "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Tus-Resumable: 1.0.0" \
 -H "Content-Type: application/offset+octet-stream" \
 -H "Upload-Offset: <upload_offset_from_previous_response>" | jq "."
```

   Verify that the successful response returns `204 No Content` and a larger upload offset number. Continue making PATCH requests with the upload offset numbers from the previous response until all the smaller packages have been imported.

   Note:
   If you do not know the current upload offset number, make a HEAD request with the location ID.

```
curl -X HEAD \
 "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Tus-Resumable: 1.0.0" | jq "."
```

   Use the upload offset number in the response for the next PATCH request.

5. After all the small packages have been imported, upload the file.

   The input body includes:

   - `bundleId` for the location ID.
   - `"option": "OVERWRITE"` to overwrite any existing package.

```
curl -X POST \
 "$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Type: application/json" \
 -H "Tus-Resumable: 1.0.0" \
 -d '{
   "bundleId":"$location_id",
   "option" : "OVERWRITE"
   }
| jq "."
```

   A successful response includes the Provider ID, Provider name, and Provider version. To complete an IPAM integration, you can use theVCF Automation UI or the integrations endpoint in the IaaS API.

6. To complete an integration using theVCF Automation UI, select **Infrastructure** > **Integrations** > **Add integration** > **IPAM**. On the **New Integration** page that appears, select **Manage IPAM Providers** and choose the package with the Provider name and Provider version from the API response.

7. Optional: To complete an integration using the API:

   a) Assign a variable for the provider ID.

```
provider_id = '<provider_id_from_package_upload>'
```

   b) Create a new IPAM integration.

```
curl -X POST \
 "$url/iaas/api/integrations?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Type: application/json" \
 -d '{
   "integrationProperties": {
```

```
      "providerId" : "'$provider_id'",
      "faasProviderEndpointId": "<your_provider_endpoint_ID>",
      "privateKeyId": "<your_privateKeyId>",
      "privateKey": "<your_privateKey>",
      "hostName": "<your_hostName>",
      "dcId": "onprem"
    },
    "customProperties": {"isExternal": "true"},
    "integrationType": "ipam",
    "associatedCloudAccountIds": [],
    "associatedMobilityCloudAccountIds": {},
    "privateKey": "<your_privateKey>",
    "privateKeyId": "<your_privateKeyId>"
  } | jq "."
```

The response includes a self link to track the request.

c) Assign the selfLink variable.

```
selfLink_id='<example_selfLink_alphanumeric_string>'
```

d) Use the selfLink variable to track the request.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The integration is created with the response includes `"status": "FINISHED"`.

Import an IPAM Package

This example shows how to import an Infoblox IPAM package with a total size of 73342782 bytes. To import incrementally using smaller packages, you divide the larger package into smaller packages. This example assumes that you have divded the package into multiple smaller packages each 9000000 bytes in size.

After importing the IPAM package, you create an integration with a cloud account using the theVCF Automation UI or the integrations endpoint in the IaaS API. This example shows how to integrate the uploaded IPAM package with a cloud account that has the following parameters:

- faasProviderEndpointId: "f46337e9-e0c2-4ada-9244-766a8e54da48"
- hostName: "infoblox.sof-mbu.eng.mycompany.com"
- privateKey: "Password!23"
- privateKeyId: "administrator@mycompany.local"

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2021-07-15'
```

Reserve space for the package.

```
$ curl -X POST \
 "$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Upload-Length: 73342782" \
 -H "Tus-Resumable: 1.0.0" | jq "."
```

Examine the response to get the location ID.

```
status: 201
body: empty
headers: {location:/iaas/api/integrations-ipam/package-import/8a81db06-f27f-4ccb-
a7c5-960c35107ed6, ...}
```

Assign the location ID variable.

```
$ location_id='8a81db06-f27f-4ccb-a7c5-960c35107ed6'
```

Import the first package with a content length of 9000000 bytes and a zero upload offset value.

```
$ curl -X PATCH \
 "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Length: 9000000" \
 -H "Tus-Resumable: 1.0.0" \
 -H "Content-Type: application/offset+octet-stream" \
 -H "Upload-Offset: 0" | jq "."
```

Examine the header in the response to get the upload offset value.

```
status: 204
body: empty
headers: {upload-offset:9000000, ...}
```

Import the next package.

```
$ curl -X PATCH \
 "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Length: 9000000" \
 -H "Tus-Resumable: 1.0.0" \
 -H "Content-Type: "application/offset+octet-stream" \
 -H "Upload-Offset: 9000000" | jq "."
```

Examine the header in the response again to get the next upload offset value.

```
status: 204
body: empty
headers: {upload-offset:18000000, ...}
```

Import the next package.

```
$ curl -X PATCH \
 "$url/iaas/api/integrations-ipam/package-import/$location_id/?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Length: 9000000" \
 -H "Tus-Resumable: 1.0.0" \
 -H "Content-Type: "application/offset+octet-stream" \
 -H "Upload-Offset: 18000000" | jq "."
```

Repeat the process to get the upload offset value from the header response and use that value to import the next package.

After importing the final small package, upload the complete package.

```
$ curl -X POST \
 "$url/iaas/api/integrations-ipam/package-import?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Tus-Resumable: 1.0.0" \
 -d '{
   "bundleId":"$location_id",
   "option" : "OVERWRITE"
   }
| jq "."
```

A successful response includes the provider ID, provider name, and the provider version

```
{
 "providerId": "86801580-9042-49b6-879d-5a7361d33519",
 "providerName": "Infoblox",
 "providerVersion": "1.6"
```

```
 "logoIcon":"iVBORw...",
...
```

To create an IPAM integration using theVCF Automation UI, select **Infrastructure** > **Integrations** > **Add integration** > **IPAM**. On the **New Integration** page that appears, select **Manage IPAM Providers** and choose the package with the Provider name `Infoblox` and Provider version `1.6`.

To create an IPAM integration using the API, assign the provider ID variable.

```
$ provider_id = "86801580-9042-49b6-879d-5a7361d33519"
```

Create the integration.

```
$ curl -X POST \
 "$url/iaas/api/integrations?apiVersion=$api_version" \
 -H "Authorization: Bearer $access_token" \
 -H "Content-Type: application/json" \
 -d '{
   "integrationProperties": {
     "providerId" : "'$provider_id'",
     "faasProviderEndpointId": "f46337e9-e0c2-4ada-9244-766a8e54da48",
     "privateKeyId": "administrator@mycompany.local",
     "privateKey": "Password!23",
     "hostName": "infoblox.sof-mbu.eng.mycompany.com",
     "dcId": "onprem"
   },
   "customProperties": {"isExternal": "true"},
   "integrationType": "ipam",
   "associatedCloudAccountIds": [],
   "associatedMobilityCloudAccountIds": {},
   "privateKey": "Password!23",
   "privateKeyId": "administrator@mycompany.local"
 } | jq "."
```

Examine the response.

```
{
"progress": 0,
"status": "INPROGRESS",
"name": "Integration creation/update",
"id": "117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc",
"selfLink": "/iaas/api/request-tracker/117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc"
}
```

Assign the selfLink variable.

```
$ selfLink_id='117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc'
```

Use the selfLink for tracking.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/request-tracker/$selfLink_id?apiVersion=$api_version" | jq "."
```

The integration is complete when the response includes `"status": "FINISHED"`.

```
{
"progress": 100,
"status": "FINISHED",
"resources": [
    "/iaas/api/integrations/9df2b0a8-2ce6-4465-bf39-04290965da9e"     ],
    "name": "Integration creation/update",
    "id": "117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc",
    "selfLink": "/iaas/api/request-tracker/117ff057-9e26-4f0c-ae7b-eb9fcc1c15cc"
```

```
}
```

Using VCF Automation APIs to Build your Resource Infrastructure

You use the VCF Automation APIs to add a project and set up an infrastructure for the team members in VCF Automation so that new workloads can be provisioned.

Create a Cloud Zone

To create a cloud zone, you first make GET request to obtain a region ID with a cloud account ID as input. Then you make a POST request with the region ID.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the cloud account you added. See Add anAmazon Web Services Cloud Account .

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up the IDs for the region named `us-east-1` that is associated with the cloud account.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId
   %20eq%20'"$cloud_account_id"'"  | jq "."
   ```

3. To obtain a region ID, examine the response.
4. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

5. Create a cloud zone.

   ```
   curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" -d '{ "name":
    "Demo-zone-1", "description": "This zone is for Demo", "regionId": "'$region_id'", "placementPolicy":
    "DEFAULT" }' "$url/iaas/api/zones?apiVersion=$api_version"  | jq "."
   ```

6. To obtain a cloud zone ID, examine the response.
7. Assign the zone ID variable.

   ```
   zone_id='<your_cloud_zone_id>'
   ```

8. List all cloud zones.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   zones?apiVersion=$api_version"  | jq "."
   ```

9. Optional: List the cloud zone with your cloud zone ID.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   zones/$zone_id?apiVersion=$api_version"  | jq "."
   ```

10. Examine the output to verify that the cloud zone you created is listed.

    Create a Cloud Zone

    Assign variables, specify a cloud account and look up IDs for the region named `us-east-1` associated with the cloud account.

    ```
    $ url='https://appliance.domain.com'
    $ api_version=' '
    $ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-
    f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0'
    ```

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
```

A snippet of the response from your request shows your cloud account ID with a region ID.

```
...
"externalRegionId": "us-east-1",
"cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-f4226a20b65c4675574bc5fbff6c0",
"id": "4965d34c3bfe0275574bc5fd858e8",
"updatedAt": " ",
...
```

Specify a region ID and create a cloud zone.

```
$ region_id='4965d34c3bfe0275574bc5fd858e8'
$ curl -X POST -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" -d
 '{ "name": "Demo-zone-1", "description": "This zone is for Demo", "regionId": "'$region_id'",
 "placementPolicy": "DEFAULT" }' "$url/iaas/api/zones?apiVersion=$api_version"  | jq "."
```

A snippet of the response from your request shows the zone ID.

```
...
"name": "Aws / us-east-1",
"id": "4965d34c3bfe0275574bc5fd8782a",
"updatedAt": " ",
...
```

Create a Cloud Zone with a Folder

When creating a cloud zone, you can specify a folder with a path that relates to a datacenter, so that when you deploy a machine you deploy it to that folder.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

  Note:
  Users with the following RBAC permissions can also use the folder API to create a cloud zone:

  - provisioning_cloud-zones_read
  - provisioning_cloud-zones_manage
- Verify that you have the cloud account ID for the cloud account with the folder that you want to use. See Add avSphere Cloud Account .

To create a cloud zone with a folder, you first obtain an external region ID with a cloud account ID as input. Only the vSphere, VMC, and VCF cloud accounts support folders. Then you use the Folders API to get the external ID for the folder to specify.
The following procedure shows how to find a folder that you can use when creating a cloud zone.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. List the info for the cloud account.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   cloud-accounts/$cloud_account_id/?apiVersion=$api_version"  | jq "."
   ```

3. To find the region where you want to create a cloud zone, examine the enabledRegions section of the response and note the external region ID and its ID.

4. Assign the external region ID variable.

   ```
   external_region_id='<your_external_region_id>'
   region_id='<your_region_id>'
   ```

5. List all folders associated with the external region and the cloud account.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
folders?externalRegionid=$external_region_id&cloudaccountid=$cloud_account_id/?apiVersion=$api_version"  |
 jq "."
```

   Note:
   Query parameters are optional:

   - By providing the cloud account ID, you filter the folders by cloud account.
   - By providing the external region ID, you filter the folders by datacenter.

6. Examine the response and note the external ID of the folder that you want to use.
   Any of the folders listed in the response can be used in the cloud specification.
7. Assign the external ID variable.

```
external_id='<your_external_id>'
```

8. To create a cloud zone with a folder in the datacenter, specify the external ID as the value for the folder.

```
curl -X POST \
  "$url/iaas/api/zones?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "regionId": "'$region_id'",
    "name": "<cloud_zone_with_folder>",
    "folder": "'$external_id'"
  } | jq "."
```

   The response shows the folder and cloud zone ID.

Any resources that are deployed in the cloud zone are deployed to the folder.

Create a Cloud Zone with a Folder

Assign variables and specify an ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='2636b316-b514-47f3-93b8-78f18df51a29'
```

List the information for the vSphere cloud account.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/cloud-accounts/$cloud_account_id/?apiVersion=$api_version"  | jq "."
```

Examine the response to find the region where you want to create a cloud zone. A snippet of the response shows the enabledRegions with an external region ID and your cloud account ID.

```
...
"enabledRegions": [
    {
        "externalRegionId": "Datacenter:datacenter-3",
        "name": "SDDC-Datacenter",
        "cloudAccountId": "2636b316-b514-47f3-93b8-78f18df51a29",
        "id": "8d633554-90ca-4f8a-8a0f-1cd3cd90a522",
        "updatedAt": " ",
...
```

Assign the external region ID variable.

```
$ external_region_id= 'Datacenter:datacenter-3'
$ region_id= '8d633554-90ca-4f8a-8a0f-1cd3cd90a522'
```

List folders associated with the external region and cloud account.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/folders?externalRegionid=$external_region_id&cloudaccountid=$cloud_account_id/?apiVersion=
$api_version"  | jq "."
```

A snippet of the response shows the folders associated with the external region and the cloud account. The external ID is the value that you assign to the folder when creating the cloud zone.

```
...
        "cloudAccountIds": [
            "2636b316-b514-47f3-93b8-78f18df51a29"
        ]
        "externalId": "dbuyukliiska",
        "name": "dbuyukliiska",
        "id": "3863f2cc-0340-483f-b24e-d351b7c53c8c",
        "createdAt": " ",
        "updatedAt": " ",
...
```

Assign the external ID.

```
$ external_id='dbuyukliiska'
```

Create a cloud zone with a specified folder.

```
curl -X POST \
  "$url/iaas/api/zones?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "regionId": "'$region_id'",
    "name": "Demo-zone-with-folder-1",
    "folder": "'$external_id'"
  } | jq "."
```

A snippet of the response from your request shows the folder and cloud zone ID.

```
...
    "folder": "dbuyukliiska",
    "externalRegionId": "Datacenter:datacenter-3",
    "cloudAccountId": "2636b316-b514-47f3-93b8-78f18df51a29",
    "name": "Demo-zone-with-folder-1",
    "id": "258bccb1-a337-4b63-9a8a-44a414761d93",
    "updatedAt": "2021-11-22",
...
```

Create a Project to use in VCF Automation

As a VCF Automation administrator, you make a POST request with a project name to create a project. Then you add members and cloud zones to the project so that project members can deploy blueprints to the associated zones.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

  Note:

  A user with the project administrator or project member role can perform a limited number of project-related tasks. For a complete list of tasks and roles required, see What are the VCF Automation for VM Apps user roles.
- Prepare parameters including the project name, description, and email addresses for administrators, members, or viewers.

1. Assign the project name variable.

   ```
   project_name='<your_project_name>'
   ```

   your_project_name is a name that you choose.

2. Create a project.

   ```
   curl -X POST \
     "$url/iaas/api/projects?apiVersion=$api_version"
     -H 'Content-Type: application/json'
     -H "Authorization: Bearer $access_token"
     -d '{
       "name" : "'$project_name'",
       "description" : "your-description",
       "administrators" : [{ "email" : "<admin_email>", ["type" : <"user" | "group">]}],
       "members" : [{ "email" : "<member_email>", ["type" : <"user" | "group">]}],
       "viewers" : [{ "email" : "<viewer_email>", ["type" : <"user" | "group">]}],
     }' | jq "."
   ```

   - *admin_email*, *member_email*, and *viewer_email* are email addresses of an administrator, member, and viewer user or name of the group in the project.
   - The type parameter is optional. It assigns the administrator, member, or viewer to a user or group type. If unspecified, the value defaults to `user`.

3. Get a list of projects and filter for the project with *your_project_name*.

   ```
   curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   projects?apiVersion=$api_version&"'$filter'"=name%20eq%20'$project_name'" | jq "."
   ```

4. Examine the response and record the ID of your newly created project.

   Create a Project to use in VCF Automation

   Create a project named `Example-Assembler-project` with administrators, members, and viewers at `mycompany.com`. This example assumes that `Example-project` does not exist.

   ```
   $ url='https://appliance.domain.com'
   $ api_version=' '
   $ project_name='Example-Assembler-project'
   ```

   Create a project for VCF Automation.

   ```
   $ curl -X POST \
     "$url/iaas/api/projects?apiVersion=$api_version" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name" : "'$project_name'",
       "description" : "This is an example project for VCF Automation",
       "administrators" : [{"email" : "admin1@mycompany.com", "type" : "user"}],
       "members" : [{"email" : "member1@mycompany.com", "type" : "user"}],
       "viewers" : [{"email" : "viewer1@mycompany.com", "type" : "user"} ]
     }' | jq "."
   ```

   The response shows the administrators, members, and viewers related to the project and the project ID.

   ```
   {
     "administrators": [
       {
         "email": "admin1@mycompany.com",
         "type": "user"
       }
     ],
   ```

```
    "members": [
      {
        "email": "member1@mycompany.com",
        "type": "user"
      }
    ],
    "viewers": [
      {
        "email": "viewer1@mycompany.com",
        "type": "user"
      }
    ],
    "sharedResources": true,
    "name": "Example-Assembler-project",
    "description": "This is an example project for VCF Automation",
    "id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
}
```

Add a cloud zone to your project. See Add a Cloud Zone to Your Project. If you want to add an administrator or user, see Add Users to Your Project.

Add Users to Your Project

As a VCF Automation user with the project administrator role, you can use PATCH requests to add users and assign roles in your project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the project administrator role in your project and you have the project ID. See Create a Project to use in VCF Automation.
- Prepare parameters including additional email addresses for administrators, members, or viewers that you want to add to the project.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

   your_project_id is the ID of the new project you created.

2. List the details of your project.

   ```
   curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   projects/$project_id?apiVersion=$api_version" | jq "."
   ```

3. Examine the response to see the administrators and users who are already in your project.

4. Submit a request to add a new administrator that includes the existing administrator for the project.

   Note:
   If the call does not include existing administrators for the project, the PATCH request removes those administrators from the project. Specifying the administrator type is optional.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "administrators" : [
      {"email" : "<your_new_administrator_email>", "type" : "user"},
      {"email" : "<existing_administrator>", "type" : "user"}
    ]
  }' | jq "."
```

5. Submit a request to add a new member that includes the existing users for the project.

   Note:
   If the call does not include existing members for the project, the PATCH request removes those members from the project. Specifying the member type is optional.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "members" : [
      {"email" : "<your_new_member_email>", "type" : "user"},
      {"email" : "<existing_member>", "type" : "user"}
    ]
  }' | jq "."
```

Add Users to Your VCF Automation Project

For the project `Example-Assembler-project`, add another administrator and member at `mycompany.com`.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'
```

List the details of your project.

```
$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/projects/$project_id?apiVersion=$api_version" | jq "."
```

A snippet of the response shows existing administrators, members, and viewers.

```
...
"administrators": [
    {
      "email": "admin1@mycompany.com",
      "type": "user"
    }
  ],
  "members": [
    {
      "email": "member1@mycompany.com",
      "type": "user"
    }
  ],
  "viewers": [
    {
      "email": "viewer1@mycompany.com",
      "type": "user"
    }
  ],
...
```

Add the administrator. Include the existing administrator `admin1@mycompany.com` in the PATCH request.

```
$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "administrators" : [
```

```
      {"email" : "newadministrator@mycompany.com", "type" : "user"},
      {"email" : "admin1@mycompany.com", "type" : "user"}
    ]
  }' | jq "."
```

Add the member. Include the existing member `member1@mycompany.com` in the PATCH request.

```
$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "members" : [
      {"email" : "newmember@mycompany.com", "type" : "user"},
      {"email" : "member1@mycompany.com", "type" : "user"}
    ]
  }' | jq "."
```

The response shows the project with administrators, members, and viewers, including the new administrator and new member.

```
{
  "administrators": [
    {
      "email": "newadministrator@mycompany.com",
      "type": "user"
    },
    {
      "email": "admin1@mycompany.com",
      "type": "user"
    }
  ],
  "members": [
    {
      "email": "newmember@mycompany.com",
      "type": "user"
    },
    {
      "email": "member1@mycompany.com",
      "type": "user"
    }
  ],
  "viewers": [
    {
      "email": "viewer1@mycompany.com",
      "type": "user"
    }
  ],
  "sharedResources": true,
  "name": "Example-Assembler-project",
  "description": "This is an example project for VCF Automation",
  "id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
  "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
  }
}
```

Add a Cloud Zone to Your Project

As a VCF Automation administrator, you use the `PATCH iaas/api/projects` request to attach a cloud zone to a project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have added a project and you have the project ID. See Create a Project to use in VCF Automation.

If the project already has cloud zones attached, review them to ensure that all zones needed for the project are included in the PATCH request to add a new cloud zone.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

   your_project_id is the ID of the new project you created.

2. List all cloud zones.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   zones?apiVersion=$api_version"  | jq "."
   ```

3. To obtain a cloud zone ID, examine the response and find the ID of the zone that you want to attach to your project. For a snippet of the response, see Create a Cloud Zone.

4. Assign the cloud zone variable.

   ```
   zone_id='<your_zone_id>'
   ```

5. List the details of your project.

   ```
   curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   projects/$project_id?apiVersion=$api_version" | jq "."
   ```

6. Examine the response to see the zones already attached to your project.

7. Submit a request to attach and configure a new cloud zone that includes the ID of existing cloud zones for the project.

   Note:
   If the call does not include existing cloud zones that are already attached to the project, the PATCH request removes those cloud zones from the project.

```
curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "zoneAssignmentConfigurations" : [
      {
        "zoneId" : "'$zone_id'",
        "priority": 1,
        "maxNumberInstances": 50
      },
      {
        "zoneId" : "<existing_cloud_zone_id>",
        "priority": 2,
        "maxNumberInstances": 100
      }
    ]
  }' | jq "."
```

Attach a Cloud Zone to Your Project

For the new project `Example-project`, add a cloud zone.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='5944aacb-91de-4541-bb9e-ef2a5403f81b'
```

List all cloud zones.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/zones?apiVersion=$api_version"  | jq "."
```

Examine the response to find the cloud zone you want and assign the zone ID variable.

```
$ zone_id='4965d34c3bfe0275574bc5fd8782a'
```

List the details of your project.

```
$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
api/projects/$project_id?apiVersion=$api_version" | jq "."
```

A snippet of the response shows an existing cloud zone.

```
...
"zones": [
    {
      "zoneId": "3cc2ecb989eee87557b0d532d4bb0",
      "priority": 0,
      "maxNumberInstances": 0
    }
...
```

Add the new cloud zone. Include the existing cloud zone `3cc2ecb989eee87557b0d532d4bb0` in the PATCH request.

```
$ curl -X PATCH \
  "$url/iaas/api/projects/$project_id?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "zoneAssignmentConfigurations" : [
      {
        "zoneId" : "'$zone_id'",
        "priority": 1,
        "maxNumberInstances": 50
      },
      {
        "zoneId" : "3cc2ecb989eee87557b0d532d4bb0",
        "priority": 2,
        "maxNumberInstances": 100
      }
    ]
  }' | jq "."
```

The response after adding a cloud zone lists the project with its administrators, members, and zone.

```
{
  "administrators": [
    {
      "email": "admin1@mycompany.com"
    }
  ],
  "members": [
    {
      "email": "user1@mycompany.com"
    }
  ],
  "zones": [
    {
      "zoneId": "4965d34c3bfe0275574bc5fd8782a",
```

```
        "priority": 1,
        "maxNumberInstances": 50
      },
      {
        "zoneId": "3cc2ecb989eee87557b0d532d4bb0",
        "priority": 2,
        "maxNumberInstances": 100
      }
    ],
    "sharedResources": true,
    "name": "Example-project",
    "description": "This is an example project",
    "id": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
    "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
    "_links": {
      "self": {
        "href": "/iaas/api/projects/edfd6f26-5d82-428f-96b0-b10ac5e4aca9"
      }
    }
  }
```

Create Flavor Mappings

To create a flavor mapping, you make a POST request with a region ID associated with a cloud account. The cloud account can be an AWS, vSphere, Azure, or GCP cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

Cloud vendors use flavors, or instance types, to express standard deployment sizings such as small (1 CPU, 2 GB RAM) or large (2 CPU, 8 GB RAM) for compute resources. When you build a blueprint, you pick a flavor that fits your needs and map a flavor name to a value for each account or region.

The same API calls create a flavor profile for an AWS, vSphere, Azure, or GCP cloud account. However, the flavor mapping used to create the flavor profile varies for each type of cloud account. This procedure provides the steps to create a flavor profile for an AWS cloud account. Additional examples show how to create flavor profiles for vSphere and Azure cloud accounts.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId
   %20eq%20'"$cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

5. List all fabric flavors.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-flavors/?apiVersion=$api_version" | jq "."
   ```

6. To select fabric flavor names with resources that fit your needs, examine the response.
7. Create a flavor profile for an AWS account that uses the fabric flavor names in your flavor mapping.

   ```
   curl -X POST \
   ```

```
$url/iaas/api/flavor-profiles?apiVersion=$api_version \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token" \
-d '{
  "name":"<your_flavor_profile>",
  "description":"Example AWS Compute flavors",
  "flavorMapping": {
    "small": {
      "name":"<flavor_name1_from_response>"
    },
    "medium": {
      "name":"<flavor_name2_from_response>"
    },
    "large":{
      "name":"<flavor_name3_from_response>"
    }
  },
  "regionId":"'$region_id'"
}' | jq "."
```

8. To obtain the flavor profile ID, examine the response.
9. Assign the flavor profile ID variable.

   ```
   flavor_profile_id='<your_flavor_profile_id>'
   ```

10. Optional: Look up the flavor profile you created with your flavor profile ID.

    ```
    curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
    flavor-profiles/$flavor_profile_id?apiVersion=$api_version | jq "."
    ```

    The response shows the name and ID for the flavor profile you created.

    Note:
    Using the external region ID and the cloud account ID, you can also filter for the flavor profile with a query that does not require the flavor profile ID. See Filtering Resources by Region ID.

    Create flavor mappings for different cloud accounts

    Create a flavor mapping for an AWS cloud account.

    1. Assign the required variables including the cloud account ID for an AWS cloud account.

       ```
       $ url='https://appliance.domain.com'
       $ api_version=' '
       $ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
       3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
       ```

    2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

       ```
       $ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
        "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-
       east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
       ```

       A snippet of the response shows the region ID.

       ```
       ...
             "externalRegionId": "us-east-1",
             "cloudAccountId":
        "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
       ce5aad01092b47558644f6b6615d",
             "id": "37d6c1acf4a8275586468873c739",
             "updatedAt": " ",
       ...
       ```

    3. Assign the AWS region ID.

       ```
       $ aws_region_id='37d6c1acf4a8275586468873c739'
       ```

    4. List all fabric flavors.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/fabric-flavors/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows a fabric flavor name with its resource size.

```
...
    {
      "id": "t2.micro",
      "name": "t2.micro",
      "cpuCount": 1,
      "memoryInMB": 1024,
      "storageType": "EBS",
      "networkType": "Low to Moderate"
    },
...
```

5. Select fabric flavor names with resources that fit your needs and create an AWS flavor profile named `aws-flavor-profile`.

```
$ curl -X POST \
  $url/iaas/api/flavor-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"aws-flavor-profile",
    "description":"Example AWS Compute flavors",
    "flavorMapping": {
      "small": {
        "name":"t2.micro"
      },
      "medium": {
        "name":"t2.medium"
      },
      "large":{
        "name":"t2.large"
      }
    },
    "regionId":"'$aws_region_id'"
  }' | jq "."
```

A snippet of the response shows the flavor profile ID.

```
...
  "externalRegionId": "us-east-1",
  "name": "aws-flavor-profile",
  "description": "Example AWS Compute flavors",
  "id": "835249077934b47558eca5963e068",
  "updatedAt": " ",
...
```

Create a flavor mapping for a vSphere cloud account.

1. Assign the required variables including the cloud account ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

2. Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-2`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq
%20'Datacenter:datacenter-2'%20and%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "Datacenter:datacenter-2",
      "cloudAccountId": "c8c3c9bfdb449475-7f703c5265a63d87-
d06bf79904ce5096492a2a2fc557fb0457d7d3c5b5e7ae20b29957788812bb3d-
d5a5e16bdc3eec7557245925e1b08",
      "id": "2aaf79b789eee8755724592b06d39",
      "updatedAt": " ",
...
```

3. Assign the vSphere region ID.

```
$ vsphere_region_id='2aaf79b789eee8755724592b06d39'
```

4. Create a vSphere flavor profile named `vcenter-flavor-profile`.

```
$ curl -X POST \
  $url/iaas/api/flavor-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"vcenter-flavor-profile",
    "description":"Example vSphere Compute flavors",
    "flavorMapping": {
      "small": {
        "cpuCount":1,
        "memoryInMB": 1024
      },
      "medium": {
        "cpuCount":2,
        "memoryInMB": 2048
      },
      "large":{
        "cpuCount":4,
        "memoryInMB": 4096
      }
    },
    "regionId":"'$vsphere_region_id'"
  }' | jq "."
```

   A snippet of the response shows the flavor profile ID.

```
...
  "externalRegionId": "Datacenter:datacenter-2",
  "name": "vcenter-flavor-profile",
  "description": "Example vSphere Compute flavors",
  "id": "cfb7246505319275572e9e68372d0",
  "updatedAt": " ",
...
```

Create a flavor mapping with an Azure cloud account ID.

1. Assign the required variables including the cloud account ID for an Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
```

   A snippet of the response shows the region ID.

```
...
      "externalRegionId": "us-east-1",
```

```
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d",
      "id": "37d6c1acf4a8275586468873c739",
      "updatedAt": " ",
...
```

3. Assign the Azure region ID.

```
$ azure_region_id='37d6c1acf4a8275586468873c739'
```

4. List all fabric flavors.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/fabric-flavors/?apiVersion=$api_version" | jq "."
```

   A snippet of the response shows a fabric flavor name with its resource size.

```
...
    {
      "id": "Standard_A0",
      "name": "Standard_A0",
      "cpuCount": 1,
      "memoryInMB": 768,
      "bootDiskSizeInMB": 1047552,
      "dataDiskSizeInMB": 20480,
      "dataDiskMaxCount": 1
    },
...
```

5. Select fabric flavor names with resources that fit your needs and create an Azure flavor profile named `azure-flavor-profile`.

```
$ curl -X POST \
  $url/iaas/api/flavor-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"azure-flavor-profile",
    "description":"Example Azure Compute flavors",
    "flavorMapping": {
      "small": {
        "name":"Standard_A0"
      },
      "medium": {
        "name":"Standard_A1"
      },
      "large":{
        "name":"Standard_A2"
      }
    },
    "regionId":"'$azure_region_id'"
  }' | jq "."
```

   A snippet of the response shows the flavor profile ID.

```
...
  "externalRegionId": "us-east-1",
  "name": "azure-flavor-profile",
  "description": "Example Azure Compute flavors",
  "id": "4965d34c3bfe0275574bc6e505b78",
  "updatedAt": " ",
...
```

Create Image Mappings

To create an image mapping, you make a POST request with a region ID associated with a cloud account. The cloud account can be an AWS, vSphere, Azure, or GCP cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

Cloud vendors use images to configure a VM based on OS settings, such as an ubuntu-16 configuration. When you build a cloud template, you pick an image that fits your needs and map an image name to a value for each account or region. You can also add constraints and configuration scripts to further control resource placement.

The same API calls create an image profile for an AWS, vSphere, Azure, or GCP cloud account. The following example shows how to create an image profile for a vSphere cloud account with the external region ID `Datacenter:datacenter-3`.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacenter-3'%20and
   %20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

5. Create an image profile with an image mapping that specifies the OVA/OVF links.

   ```
   curl -X POST \
     $url/iaas/api/image-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name":"<your_image_profile>",
       "description":"<your_image_profile_description>",
       "imageMapping": {
         "ubuntu": {
            "externalId": "https://cloud-images.ubuntu.com/releases/16.04/release-20220305/ubuntu-16.04-
   server-cloudimg-amd64.ova"
         }
     },
     "regionId":"'$region_id'"
   }' | jq "."
   ```

6. To obtain the image profile ID, examine the response.

   Create image mapping

   Assign the required variables including a cloud account ID.

   ```
   $ url='https://appliance.domain.com''
   $ api_version=' '
   $ cloud_account_id='ff4e7585-4197-41fb-89cb-179ef4d24779'
   ```

   Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-3`.

   ```
   $ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
    "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq
   %20'Datacenter:datacenter-3'%20and%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
   ```

A snippet of the response shows the region ID.

```
...
          "externalRegionId": "Datacenter:datacenter-3",
          "name": "w01-vc08-DC",
          "cloudAccountId": "ff4e7585-4197-41fb-89cb-179ef4d24779",
          "id": "9b148b38-fc7c-4560-b413-5f47b30e57d8",
...
```

Assign the region ID.

```
$ region_id='9b148b38-fc7c-4560-b413-5f47b30e57d8'
```

Create an image profile named `example-image-profile`.

```
$ curl -X POST \
  $url/iaas/api/image-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"example-image-profile",
    "description":"Example image profile",
    "imageMapping": {
      "ubuntu": {
         "externalId": "https://cloud-images.ubuntu.com/releases/16.04/release-20220305/
ubuntu-16.04-server-cloudimg-amd64.ova"
      }
    },
    "regionId":"'$region_id'"
  }' | jq "."
```

A snippet of the response shows the image profile ID.

```
...
      "externalRegionId": "Datacenter:datacenter-3",
      "cloudAccountId": "ff4e7585-4197-41fb-89cb-179ef4d24779",
      "name": "example-image-profile",
      "id": "f670fdfc-66d6-4689-9793-d524e7066d1e-9b148b38-fc7c-4560-b413-5f47b30e57d8",
...
```

Working with Networks

As a VCF Automation administrator, you can use the IaaS APIs to create network profiles and get information about data collected and networks.

Create Network Profiles

To create a network profile, you make a POST request with a region ID associated with a cloud account.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

A VCF Automation network profile describes the behavior of the network to be deployed. For example, a network might need to be Internet facing versus internal only. Networks and their profiles are cloud-specific.

The networks in this example are used for provisioning to existing or public networks. If you are working with on-demand or deployment networks, see Using the Network APIs.

If you are provisioning to a private network, or outbound networks with one-way access to upstream networks, you create a network profile with isolation enabled by either subnet or security group. See Create a Network Profile with Isolation.

If you want to add firewall rules to all machines provisioned with a network profile, you create a network profile with security groups. See Create a Network Profile with Security Groups.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up regions associated with the cloud account ID and with the region name `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/regions/?apiVersion=$api_version&"'$filter'"=name%20eq%20'us-east-1'%20and%20cloudAccountId%20eq
   %20'"$cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

5. Filter for fabric networks associated with the cloud account ID and in the external region ID `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/
   api/fabric-networks?apiVersion=$api_version&"'$filter='"externalRegionId%20eq%20'us-east-1'%20and
   %20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
   ```

   For details on how to construct a filter, see Filtering Resources by Region ID.

6. Examine the response to find the IDs for the public networks that you want to include in your network profile.
7. Create a network profile.

   ```
   curl -X POST \
     $url/iaas/api/network-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name":"<your-network-profile>",
       "description":"Example Network Profile",
       "regionId":"'$region_id'",
       "fabricNetworkIds": [
         "<network_id1_from_response>",
         "<network_id2_from_response>"
       ],
       "tags": [ { "key": "env", "value": "prod" } ]
     }' | jq "."
   ```

8. To obtain the network profile ID, examine the response.
9. Assign the network profile ID variable.

   ```
   network_profile_id='<your_network_profile_id>'
   ```

10. Optional: Look up the network profile you created with your network profile ID.

    ```
    curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
    network-profiles/$network_profile_id?apiVersion=$api_version | jq "."
    ```

    The response shows the name and ID for the network profile you created.

Create a network profile

Assign the required variables including a cloud account ID.

```
url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'"$cloud_account_id"'"  | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "us-east-1",
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d",
      "id": "37d6c1acf4a8275586468873c739",
      "updatedAt": " ",
...
```

Assign the region ID.

```
$ region_id='37d6c1acf4a8275586468873c739'
```

Filter for fabric networks associated with the cloud account ID and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/
api/fabric-networks?apiVersion=$api_version | jq "."
```

A snippet of the response shows the fabric network ID for a public network that you can include in your network profile.

```
...
      "isPublic": true,
      "isDefault": true,
      "cidr": "172.31.16.0/20",
      "externalRegionId": "us-east-1",
      "tags": [
        {
          "key": "vmware.enumeration.type",
          "value": "ec2_subnet"
        }
      ],
      "cloudAccountIds": [
        "c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-5a45a4b9d5c72475575931611aa28",

 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d"
      ],
      "name": "subnet-0130834a",
      "id": "d43efed364ef18755759316540e3f",
...
```

Select the IDs of fabric networks that you want to include in your profile and create a network profile named `example-network-profile`.

```
$ curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"example-network-profile",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "fabricNetworkIds": [
      "d43efed364ef18755759316540e3d",
      "d43efed364ef18755759316540e3f"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
```

```
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile",
  "description": "Example Network Profile",
  "id": "9cb2d111c768927558f043ec13d70",
  "updatedAt": " ",
...
```

Create a Network Profile with Isolation

To create either private networks without access to outside networks or outbound networks with one-way access to upstream networks, you create a network profile with isolation and specify the isolation type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.
- Verify that you have the region ID for the regions you want to include in the profile. See the procedure in Create Network Profiles.
- For simplicity, examples use `us-east-1` as the external region ID.
- Verify that you have the IDs for the non-public fabric networks you want to include in the profile. See the procedure in Create Network Profiles.

This procedure provides the steps to create a network that supports isolation using a subnet, and includes optional steps that show how to create the network using an external subnet, security groups, or a VLAN transport zone. The network profile that uses a VLAN transport zone only supports private networks.

1. Assign the cloud account ID variable.

   ```
   cloud_account_id='<your_cloud_account_id>'
   ```

2. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

3. Filter for network domains associated with the cloud account ID and in the external region ID `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/network-domains?apiVersion=$api_version&"'$filter='"externalRegionId%20eq%20'us-east-1'%20and
   %20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
   ```

4. Examine the response to find the IDs for the network domain that you want to include in your network profile. If you are creating a network profile with a VLAN transport zone, select a network domain that has the custom property `"__transportZoneTrafficType":"VLAN_BACKED"`.

5. Create a network profile that supports isolation using a subnet and IDs for a non-public network.

   ```
   curl -X POST \
     $url/iaas/api/network-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name":"<your-network-profile-with-isolation-by-subnet>",
       "description":"Example Network Profile",
       "regionId":"'$region_id'",
       "isolationType" : "SUBNET",
       "isolationNetworkDomainId" : "<network_domain_id_from_response>",
       "isolatedNetworkCIDRPrefix" : "27",
       "fabricNetworkIds": [
         "<non_public_network_id1>",
         "<non_public_network_id2>"
       ],
       "tags": [ { "key": "env", "value": "prod" } ]
   ```

```
  }' | jq "."
```

The response shows the name and ID for the network profile you created.

6. Optional: Create a network profile that supports isolation using an external subnet.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"<your-network-profile-with-isolation-by-external-subnet>",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationType" : "SUBNET",
    "isolationNetworkDomainId" : "<network_domain_id_from_response>",
    "isolatedNetworkCIDRPrefix" : "27",
    "isolationExternalFabricNetworkId": "<non_public_network_id1>",
    "fabricNetworkIds": [
      "<non_public_network_id1>",
      "<non_public_network_id2>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "."
```

The response shows the name and ID for the network profile you created.

7. Optional: Create a network profile that supports isolation using security groups.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name":"<your-network-profile-with-isolation-by-security-group>",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationType" : "SECURITY_GROUP",
    "fabricNetworkIds": [
      "<non_public_network_id1>",
      "<non_public_network_id2>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "."
```

The response shows the name and ID for the network profile you created.

Create various types of network profiles with isolation

The following examples include the requests used to create a network profiles that support isolation using:

- A subnet.
- An external subnet.
- Security groups.
- A VLAN transport zone.

Assign the required variables including a cloud account ID and a region ID.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
$ region_id='37d6c1acf4a8275586468873c739'
```

Filter for network domains associated with the cloud account ID and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/network-domains?apiVersion=$api_version&"'$filter='"externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
```

A snippet of the response shows the ID for a network domain that you can include in your network profile.

```
...
      "externalId": "vpc-4511a53d",
      "name": "rainpole-dev",
      "id": "233df662ec3b4875575931653ef00",
      "createdAt": " ",
      "updatedAt": " ",
      "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
      "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
      "_links": {
        "cloud-accounts": {
          "hrefs": [
            "/iaas/api/cloud-accounts/c8c3c9bfdb449475-7f703c5265a63d87-
f8e705d89b2569e1aac66c6d00bf4fc7ef4b1c44100f0e944af31eb8ba3d2a5a-5a45a4b9d5c72475575931611aa28",
            "/iaas/api/cloud-accounts/
c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d"
          ]
        },
        "self": {
          "href": "/iaas/api/network-domains/233df662ec3b4875575931653ef00"
...
```

To create a network profile with a VLAN transport zone, ensure that the network domain you choose includes a custom property for the transport zone.

```
...
      "customProperties": {
        "__path": "/infra/sites/default/enforcement-points/default/transport-
zones/9a358e99-5734-4926-b718-37cf4862f4bf",
        "__host_identifier": "[\"host-16\",\"host-23\",\"host-21\"]",
        "__cluster_identifier": "[\"domain-c8\"]",
        "__transportZoneTrafficType": "VLAN_BACKED",
        "path": "/infra/sites/default/enforcement-points/default/transport-
zones/9a358e99-5734-4926-b718-37cf4862f4bf"
          }
...
```

With the IDs of fabric networks that you want to include in your profile and the network domain ID you want to include, create a network profile named `example-network-profile-with-isolation-by-subnet`.

```
$ curl -X POST  \
  $url/iaas/api/network-profiles?apiVersion=$api_version  \
  -H 'Content-Type: application/json'  \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name":"example-network-profile-with-isolation-by-subnet",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationType" : "SUBNET",
    "isolationNetworkDomainId" : "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix" : "27",
    "fabricNetworkIds": [
      "c19bd2921af95075575931654066a",
      "8fe650cc09d0627558d55c9ba1793"
```

```
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile-with-isolation-by-subnet",
  "description": "Example Network Profile",
  "id": "2065036880e1c47558f1693558870",
  "updatedAt": " ",
...
```

Provide the `isolationExternalFabricNetworkId` to create a network profile with isolation using an external subnet.

```
$ curl -X POST  \
  $url/iaas/api/network-profiles?apiVersion=$api_version  \
  -H 'Content-Type: application/json'  \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name":"example-network-profile-with-isolation-by-external-subnet",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationType" : "SUBNET",
    "isolationNetworkDomainId" : "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix" : "27",
    "isolationExternalFabricNetworkId": "c19bd2921af95075575931654066a",
    "fabricNetworkIds": [
      "c19bd2921af95075575931654066a",
      "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile-with-isolation-by-external-subnet",
  "description": "Example Network Profile",
  "id": "2065036880e1c47558f16bd085288",
  "updatedAt": " ",
...
```

Use the `"isolationType": "SECURITY_GROUP"` to create a network profile with isolation using a security group. Because this isolation does not use a subnet, this request does not use a network domain ID.

```
$ curl -X POST  \
  $url/iaas/api/network-profiles?apiVersion=$api_version  \
  -H 'Content-Type: application/json'  \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name":"example-network-profile-with-isolation-by-security-group",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationType" : "SECURITY_GROUP",
    "fabricNetworkIds": [
     "c19bd2921af95075575931654066a",
     "8fe650cc09d0627558d55c9ba1793"
  ],
  "tags": [ { "key": "env", "value": "prod" } ]
```

```
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile-with-isolation-by-security-group",
  "description": "Example Network Profile",
  "id": "bdab0d4c28af6e7558f16c78f5468",
  "updatedAt": " ",
...
```

Create a network profile with isolation using a VLAN transport zone.

```
$ curl -X POST  \
  $url/iaas/api/network-profiles?apiVersion=$api_version  \
  -H 'Content-Type: application/json'  \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name":"example-network-profile-with-VLAN-transport-zone",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "isolationNetworkDomainId" : "233df662ec3b4875575931653ef00",
    "isolatedNetworkCIDRPrefix" : "27",
    "fabricNetworkIds": [
      "c19bd2921af95075575931654066a",
      "8fe650cc09d0627558d55c9ba1793"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
  "name": "example-network-profile-with-VLAN-transport-zone",
  "description": "Example Network Profile",
  "id": "9cb2d111c768927558f043ec13d70",
  "updatedAt": " ",
...
```

Create a Network Profile with Security Groups

To create a network profile with security groups, you make a POST request and provide security group IDs.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.
- Verify that you have the region ID for the regions you want to include in the profile. See the procedure in Create Network Profiles.
- Verify that you have the IDs for the networks you want to include in the profile. See the procedure in Create Network Profiles.

You create a network profile with security groups so that you can add firewall rules to all machines provisioned with that network profile.

1. Assign the region ID variable.

   ```
   region_id='<your_region_id>'
   ```

2. Filter for security groups associated with the cloud account ID and in the external region ID `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/security-groups?apiVersion=$api_version&"'$filter='"externalRegionId%20eq%20'us-east-1'%20and
   %20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
   ```

3. Examine the response to find the IDs for the security groups that you want to include in your network profile.
4. Create a network profile with security groups using network IDs for a non-public network.

```
curl -X POST \
  $url/iaas/api/network-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
   -d '{
    "name":"example-network-profile-with-security-groups",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "fabricNetworkIds": [
      "<network_id1>",
      "<network_id1>"
    ],
    "securityGroupIds": [
      "<security_group_id1_from_response>",
      "<security_group_id2_from_response>"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
  }' | jq "."
```

The response shows the name and ID for the network profile you created.

Create a network profile with security groups

Assign the required variables including a cloud account ID and a region ID.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
$ region_id='37d6c1acf4a8275586468873c739'
```

Filter for security groups associated with the cloud account ID and in the external region ID `us-east-1`.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/security-groups?apiVersion=$api_version&"'$filter='"externalRegionId%20eq%20'us-
east-1'%20and%20cloudAccountId%20eq%20'$cloud_account_id'" | jq "."
```

A snippet of the response shows the ID for a security group that you can include in your network profile.

```
...
     "externalId": "sg-0305bc072a9f2727b",
      "name": "OC-LB-mcm681186-113024780265_SG",
      "id": "bdab0d4c28af6e7558f061f772518",
      "createdAt": " ",
      "updatedAt": " ",
      "organizationId": "8327d53f-91ea-420a-8613-ba8f3149db95",
      "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95",
      "_links": {
        "cloud-accounts": {
          "hrefs": [
            "/iaas/api/cloud-accounts/
c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d"
          ]
        },
        "self": {
          "href": "/iaas/api/security-groups/bdab0d4c28af6e7558f061f772518"
...
```

With the IDs of fabric networks that you want to include in your profile and and the security group IDs you want to include, create a network profile named `example-network-profile-with-security-groups`.

```
$ curl -X POST  \
  $url/iaas/api/network-profiles?apiVersion=$api_version  \
  -H 'Content-Type: application/json'  \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name":"example-network-profile-with-security-groups",
    "description":"Example Network Profile",
    "regionId":"'$region_id'",
    "fabricNetworkIds": [
      "d43efed364ef18755759316540e3d",
      "d43efed364ef18755759316540e3f"
    ],
    "securityGroupIds": [
      "bdab0d4c28af6e7558f061f772518",
      "ebdab0d4c28af6e7558efe6edd71c9"
    ],
    "tags": [ { "key": "env", "value": "prod" } ]
}' | jq "."
```

A snippet of the response shows the network profile ID.

```
...
 "name": "example-network-profile-with-security-groups",
  "description": "Example Network Profile",
  "id": "9cb2d111c768927558f1799bf9e48",
  "updatedAt": " ",
...
```

Using the Network APIs

As a VCF Automation administrator, you must understand the difference between using the fabric network APIs and the network APIs.

You use the fabric network APIs such as `/iaas/api/fabric-networks/` to get information about existing networks that you use to create a network profile.

You use the network APIs such as `/iaas/api/networks/` to get information about on-demand networks that are provisioned with a deployment. If provisioned, you can check these networks in the VCF Automation UI by going to **Resources** > **Deployments**.

To see a list of both public and on-demand networks in the VCF Automation UI, go to **Infrastructure** > **Resources** > **Networks**.

Creating Storage Profiles

Using different input variables, you can use the VCF Automation IaaS API to create storage profiles for AWS, vSphere, or Azure cloud accounts.

A VCF Automation storage profile describes the kind of storage to be deployed. Storage is usually profiled according to characteristics such as service level or cost, performance, or purpose, such as backup.

As a cloud administrator, you organize your storage profiles under cloud-specific regions. One cloud account can have multiple regions, with multiple storage profiles under each. Using a storage profile, you define your storage for the region.

Storage profiles include disk customizations, and a means of identifying the type of storage by capability tags. Tags are matched against provisioning service request constraints to create the desired storage at deployment time.

Create an Amazon Web Services Storage Profile

To create a storage profile, you make a POST request with a region ID associated with a cloud account. The request body includes an Amazon Web Services fabric volume type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

As an alternative to using the `storage-profiles` API call to create an Amazon Web Services storage profile, you can also use the `storage-profiles-aws` API call. Optional procedure steps show how to use the `storage-profiles-aws` API call. The example only includes the steps required to create an Amazon Web Services storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

   ```
   aws_cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId
   %20eq%20'"$aws_cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   aws_region_id='<your_region_id>'
   ```

5. List all Amazon Web Services fabric volume types.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-aws-volume-types/?apiVersion=$api_version" | jq "."
   ```

6. To select a volume type, examine the response.
7. Create a storage profile for the selected region.

   ```
   curl -X POST \
     $url/iaas/api/storage-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "defaultItem": false,
       "supportsEncryption": false,
       "tags": [ { "key": "env", "value": "dev" } ],
       "diskProperties": {
         "deviceType": "ebs",
         "volumeType": "<volume_type_from_response>",
         "iops": "400"
       },
       "regionId": "'$region_id'",
       "name": "<your-aws-storage-profile>",
       "description": "Example AWS storage profile"
   }' | jq "."
   ```

8. Optional: Create a storage profile for the selected region using the `storage-profiles-aws` API call.

   ```
   curl -X POST \
     $url/iaas/api/storage-profiles-aws?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "defaultItem": false,
       "supportsEncryption": false,
       "tags": [ { "key": "env", "value": "dev" } ]
   ```

```
    "deviceType": "ebs",
    "volumeType": "<volume_type_from_response>",
    "iops": "1000",
    "regionId": "'$region_id'",
    "name": "<your-aws-storage-profile>",
    "description": "Example AWS storage profile"
}' | jq "."
```

9. To obtain the storage profile ID, examine the response.
10. Assign the storage profile ID variable.

    ```
    aws_storage_profile_id='<your_storage_profile_id>'
    ```

11. Optional: Look up the storage profile you created with your storage profile ID.

    ```
    curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
    storage-profiles/$aws_storage_profile_id?apiVersion=$api_version | jq "."
    ```

    The response shows the name and ID for the storage profile you created.

    Note:
    Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See Filtering Resources by Region ID.

12. Optional: List all storage profiles using the `storage-profiles-aws` API call.

    ```
    curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
    storage-profiles-aws?apiVersion=$api_version | jq "."
    ```

13. Optional: Delete an Amazon Web Services storage profile. Alternatively, you can use the `storage-profiles-aws` API call.

    ```
    curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
    storage-profiles/$aws_storage_profile_id?apiVersion=$api_version | jq "."
    ```

Create an Amazon Web Services storage profile

Assign the required variables including the cloud account ID for an Amazon Web Services cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ aws_cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c403301
3bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID `us-east-1`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'"$aws_cloud_account_id"'"  | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "us-east-1",
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d",
      "id": "37d6c1acf4a8275586468873c739",
      "updatedAt": " ",
...
```

Assign the Amazon Web Services region ID.

```
$ aws_region_id='37d6c1acf4a8275586468873c739'
```

List all Amazon Web Services fabric volume types.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/fabric-aws-volume-types/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the volume types.

```
...
{
  "volumeTypes": [
    "standard",
    "io1",
    "gp2",
    "sc1",
    "st1"
  ]
},
...
```

Select volume type and create an AWS storage profile named `aws-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
      "deviceType": "ebs",
      "volumeType": "io1",
      "iops": "400"
    },
    "regionId": "'$aws_region_id'",
    "name": "aws-storage-profile",
    "description": "Example AWS storage profile"
    "regionId":"'$aws_region_id'"
  }' | jq "."
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "us-east-1",
  "name": "aws-storage-profile",
  "description": "Example AWS storage profile",
  "id": "3e3dc378-a090-4b7e-af41-57b1735d9526",
  "createdAt": " ",
  "updatedAt": " ",
...
```

Create a vSphere Storage Profile

To create a vSphere storage profile, you make a POST request with a region ID. Optional request body input includes a vSphere storage policy and a vSphere datastore.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

As an alternative to using the `storage-profiles` API call to create a vSphere storage profile, you can also use the `storage-profiles-vsphere` API call. Optional procedure steps show how to use the `storage-profiles-vsphere` API call. The example only includes the steps required to create a vSphere storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

   ```
   vsphere_cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-10`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacenter-10'%20and
   %20cloudAccountId%20eq%20'"$vsphere_cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   vsphere_region_id='<your_region_id>'
   ```

5. Optional: If using a vSphere storage policy, list all vSphere storage policies. If using a default storage policy, skip this step.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
   ```

   Examine the response and assign the vSphere storage policy ID.

   ```
   vsphere_storage_policy_id='<your_vsphere_storage_policy_id>'
   ```

6. Optional: If using a vSphere datastore, list all vSphere datastores. If provisioning any datastore or cluster, skip this step.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
   ```

   Examine the response and assign the vSphere datastore ID.

   ```
   vsphere_datastore_id='<your_vsphere_datastore_id>'
   ```

7. Create a vSphere storage profile.

   ```
   curl -X POST \
     $url/iaas/api/storage-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name": "<your-vsphere-storage-profile>",
       "description": "Example vSphere storage profile",
       "defaultItem": true,
       "supportsEncryption": true,
       "tags": [ { "key" : "env", "value": "dev" } ],
       "diskProperties": {
         "provisioningType": "thin",
         "independent":"true",
         "persistent":"true",
         "sharesLevel": "low",
         "shares": "500",
         "limitIops": "500"
       },
       "diskTargetProperties": {
         "storagePolicyId": "'$vsphere_storage_policy_id'",
         "datastoreId": "'$vsphere_datastore_id'" },
         "regionId": "'$vsphere_region_id'"
       }' | jq "."
   ```

8. To obtain the storage profile ID, examine the response.
9. Assign the storage profile ID variable.

   ```
   vsphere_storage_profile_id='<your_storage_profile_id>'
   ```

10. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$vsphere_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note:
Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See Filtering Resources by Region ID.

11. Optional: List all storage profiles using the `storage-profiles-vsphere` API call. API.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles-vsphere?apiVersion=$api_version | jq "."
```

12. Optional: Delete a vSphere storage profile. Alternatively, you can use the `storage-profiles-vsphere` API call..

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$vsphere_storage_profile_id?apiVersion=$api_version | jq "."
```

Create vSphere storage profile

Assign the required variables including the cloud account ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ vsphere_cloud_account_id='515684ccebafde75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-23b5c527d7083675572f5099a8da0'
```

Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-10`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq
%20'Datacenter:datacenter-10'%20and%20cloudAccountId%20eq%20'"$vsphere_cloud_account_id"'" | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "Datacenter:datacenter-10",
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-809fe6fef311fdd63aa6dac546574aa898213265e988e34cc851db19b8c05b96-
f405bb370210c875572d26445252e",
      "id": "cfb7246505319275572d26466a749",
...
```

Assign the vSphere region ID.

```
$ vsphere_region_id='cfb7246505319275572d26466a749'
```

If using a vSphere storage policy, perform the following steps.

1. List all vSphere storage policies.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the storage policy.

```
...
      "externalId": "f31f2442-8247-4517-87c2-8d69d7a6c696",
      "name": "Management Storage Policy - Stretched",
      "description": "Management Storage policy used for VMC stretched cluster",
      "id": "4aad51f0b02b5275572d264d28490",
...
```

2. Examine the response to assign the vSphere storage policy ID.

```
$ vsphere_storage_policy_id='4aad51f0b02b5275572d264d28490'
```

If provisioning a specific datastore or cluster, perform the following steps.

1. List all vSphere datastores.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
```

   A snippet of the response shows the datastore.

```
...
      "externalId": "WorkloadDatastore",
      "name": "WorkloadDatastore",
      "id": "c4f1dd4741d05e75572d264dcc590",
      "createdAt": " ",
      "updatedAt": " ",
...
```

2. Examine the response to assign the vSphere datastore ID.

```
$ vsphere_datastore_id='c4f1dd4741d05e75572d264dcc590'
```

Create a vSphere storage profile named `vsphere-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "vsphere-storage-profile",
    "description": "Example vSphere storage profile",
    "defaultItem": true,
    "supportsEncryption": true,
    "tags": [ { "key" : "env", "value": "dev" } ],
    "diskProperties": {
      "provisioningType": "thin",
      "independent":"true",
      "persistent":"true",
      "sharesLevel": "low",
      "shares": "500",
      "limitIops": "500"
    },
    "diskTargetProperties": {
      "storagePolicyId": "'$vsphere_storage_policy_id'",
      "datastoreId": "'$vsphere_datastore_id'"
    },
    "regionId": "'$vsphere_region_id'"
}' | jq "."
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "Datacenter:datacenter-10",
  "name": "vsphere-storage-profile",
  "description": "Example vSphere storage profile",
  "id": "b4fbd25e-a2dd-4fde-9186-0f7bd34a1df2",
  "createdAt": " ",
  "updatedAt": " ",
...
```

Create a vSphere Storage Profile for a First Class Disk

To create a vSphere Storage Profile that supports First Class Disk (FCD) storage, you make a POST request with a region ID and you include first class as the disk type property. Optional request body input includes a vSphere storage policy and a vSphere datastore.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the vSphere cloud account that you added. See Add avSphere Cloud Account .

1. Assign the cloud account ID variable.

   ```
   vsphere_cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-3`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/
   api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'Datacenter:datacenter-3'%20and
   %20cloudAccountId%20eq%20'"$vsphere_cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   vsphere_region_id='<your_region_id>'
   ```

5. Optional: If using a vSphere storage policy, list all vSphere storage policies. If using a default storage policy, skip this step.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
   ```

   Examine the response and assign the vSphere storage policy ID.

   ```
   vsphere_storage_policy_id='<your_vsphere_storage_policy_id>'
   ```

6. Optional: If using a vSphere datastore, list all vSphere datastores. If provisioning any datastore or cluster, skip this step.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
   ```

   Examine the response and assign the vSphere datastore ID.

   ```
   vsphere_datastore_id='<your_vsphere_datastore_id>'
   ```

7. Create a vSphere Storage Profile with the FCD property.

   ```
   curl -X POST \
     $url/iaas/api/storage-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name": "<your_vsphere_storage_profile_with_FCD>",
       "description": "Example First Class Disk vSphere Storage Profile",
       "defaultItem": true,
       "provisioningType": "thin",
       "diskType": "firstClass",
       "regionId": "'$vsphere_region_id'",
       "tags": [ { "key": "type", "value": "fcd" } ]
       }' | jq "."
   ```

8. Examine the response.
   - `"defaultItem": true` indicates that this storage profile is the default for the region.
   - Tags help you to locate, manage, and work with the infrastructure resources.

Create vSphere Storage Profile with FCD storage

Assign the required variables including the cloud account ID for a vSphere cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ vsphere_cloud_account_id='683c647b-413d-4673-a236-08b3694cd652'
```

Look up region IDs associated with the cloud account and in the external region ID `Datacenter:datacenter-3`.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
 "$url/iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq
%20'Datacenter:datacenter-10'%20and%20cloudAccountId%20eq%20'"$vsphere_cloud_account_id"'" | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "Datacenter:datacenter-3",
      "name": "Example external region name",
      "cloudAccountId": "683c647b-413d-4673-a236-08b3694cd652",
      "id": "0f182edc-1155-4df1-a53a-2c46be7bc373",
...
```

Assign the vSphere region ID.

```
$ vsphere_region_id='0f182edc-1155-4df1-a53a-2c46be7bc373'
```

If using a vSphere storage policy, perform the following steps.

1. List all vSphere storage policies.

   ```
   $ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
    "$url/iaas/api/fabric-vsphere-storage-policies/?apiVersion=$api_version" | jq "."
   ```

   A snippet of the response shows the storage policy.

   ```
   ...
         "externalId": "f31f2442-8247-4517-87c2-8d69d7a6c696",
         "name": "Management Storage Policy - Stretched",
         "description": "Management Storage policy used for VMC stretched cluster",
         "id": "4aad51f0b02b5275572d264d28490",
   ...
   ```

2. Examine the response to assign the vSphere storage policy ID.

   ```
   $ vsphere_storage_policy_id='4aad51f0b02b5275572d264d28490'
   ```

If provisioning a specific datastore or cluster, perform the following steps.

1. List all vSphere datastores.

   ```
   $ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token"
    "$url/iaas/api/fabric-vsphere-datastores/?apiVersion=$api_version" | jq "."
   ```

   A snippet of the response shows the datastore.

   ```
   ...
         "externalId": "WorkloadDatastore",
         "name": "WorkloadDatastore",
         "id": "c4f1dd4741d05e75572d264dcc590",
         "createdAt": " ",
         "updatedAt": " ",
   ...
   ```

2. Examine the response to assign the vSphere datastore ID.

   ```
   $ vsphere_datastore_id='c4f1dd4741d05e75572d264dcc590'
   ```

Create a vSphere Storage Profile named `vsphere-storage-profile-with-FCD`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name": "vsphere-storage-profile-with-FCD",
    "description": "Example First Class Disk vSphere Storage Profile",
    "defaultItem": true,
```

```
    "provisioningType": "thin",
    "diskType": "firstClass",
    "regionId": "'$vsphere_region_id'",
    "tags": [ { "key": "type", "value": "fcd" } ]
}' | jq "."
```

Included with storage profile ID, a snippet of the response shows the tags that you defined for the storage profile.

```
{
  "defaultItem": true,
  "tags": [
    {
      "key": "type",
      "value": "fcd"
    }
  ],
  "provisioningType": "thin",
  "externalRegionId": "Datacenter:datacenter-3",
  "cloudAccountId": "683c647b-413d-4673-a236-08b3694cd652",
  "diskType": "firstClass",
  "name": "vsphere-storage-profile-with-FCD",
  "description": "Example First Class Disk vSphere Storage Profile",
  "id": "6037ac02-83e0-41bb-ba6e-ed5784ae1101",
  "createdAt": " ",
  "updatedAt": " ",
...
```

Use the tags to create a First Class Disk. See Create a First Class Disk.

Create a Microsoft Azure Storage Profile

To create a Microsoft Azure storage profile, you make a POST request with a region ID. The request body includes a Microsoft Azure fabric storage account ID.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

As an alternative to using the `storage-profiles` API call to create a Microsoft Azure storage profile, you can also use the `storage-profiles-azure` API call. Optional procedure steps show how to use the `storage-profiles-azure` API call. The example only includes the steps required to create a Microsoft Azure storage profile using the `storage-profiles` API call.

1. Assign the cloud account ID variable.

   ```
   azure_cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and%20cloudAccountId
   %20eq%20'"$zaure_cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   azure_region_id='<your_region_id>'
   ```

5. List all Azure fabric storage accounts.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   fabric-azure-storage-accounts/?apiVersion=$api_version" | jq "."
   ```

6. To select a storage account ID, examine the response.
7. Create a storage profile for the selected region.

```
curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
    "azureOsDiskCaching": "None",
    "azureDataDiskCaching": "None"
  },
  "diskTargetProperties": { "storageAccountId": "<storage_account_id_from_response>" },
  "regionId": "'$azure_region_id'",
  "name": "<your-azure-storage-profile>",
  "description": "Example Azure storage profile"
}' | jq "."
```

8. Optional: Create a storage profile for the selected region using the `storage-profiles-azure` API call.

```
curl -X POST \
  $url/iaas/api/storage-profiles-azure?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "osDiskCaching": "None",
    "dataDiskCaching": "None",
    "storageAccountId": "<storage_account_id_from_response>",
    "regionId": "'$azure_region_id'",
    "name": "<your-azure-storage-profile>",
    "description": "Example Azure storage profile"
    "tags": [ { "key": "env", "value": "dev" } ]
}' | jq "."
```

9. To obtain the storage profile ID, examine the response.
10. Assign the storage profile ID variable.

```
azure_storage_profile_id='<your_storage_profile_id>'
```

11. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

    The response shows the name and ID for the storage profile you created.

    Note:
    Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See Filtering Resources by Region ID.

12. Optional: List all Azure storage profiles using the `storage-profiles-azure` API call.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles-azure?apiVersion=$api_version | jq "."
```

13. Optional: Delete a Microsoft Azure storage profile. Alternatively, you can use the `storage-profiles-azure` API call.

```
curl -X DELETE -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

Create a Microsoft Azure storage profile

Assign the required variables including the cloud account ID for a Microsoft Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ azure_cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254
c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID `us-east-1`

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'"$azure_cloud_account_id"'"  | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "us-east-1",
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d",
      "id": "37d6c1acf4a8275586468873c739",
      "updatedAt": " ",
...
```

Assign the Azure region ID.

```
$ azure_region_id='37d6c1acf4a8275586468873c739'
```

List all fabric storage accounts.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/fabric-azure-storage-accounts/?apiVersion=$api_version" | jq "."
```

A snippet of the response shows the storage accounts.

```
...
      "externalId": "/subscriptions/b8ef63a7-a5e3-44fa-8745-1ead33fa1f25/resourceGroups/default-rg/
providers/Microsoft.Storage/storageAccounts/azbasicsa80370",
      "name": "azbasicsa80370",
      "id": "f81c26bf-51b1-49cc-865c-de2ab3821c1d",
...
```

Select storage account ID and create an Azure storage profile named `azure-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "env", "value": "dev" } ],
    "diskProperties": {
    "azureOsDiskCaching": "None",
    "azureDataDiskCaching": "None"
  },
  "diskTargetProperties": { "storageAccountId": "f81c26bf-51b1-49cc-865c-de2ab3821c1d" },
```

```
  "regionId": "'$azure_region_id'",
  "name": "azure-storage-profile",
  "description": "Example Azure storage profile"
}' | jq "."
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "us-east-1",
  "name": "azure-storage-profile",
  "description": "Example Azure storage profile",
  "id": "f83d0fd4-45de-4ca7-a699-c98bc141ecaa",
  "createdAt": " ",
  "updatedAt": " ",
...
```

Create a Microsoft Azure Storage Profile for a Managed Disk

To create a Microsoft Azure storage profile for a managed disk, you make a POST request with a region ID and include disk properties to specify the managed disk type.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud account ID for the new cloud account that you added. See Adding Cloud Accounts.

1. Assign the cloud account ID variable.

   ```
   azure_cloud_account_id='<your_cloud_account_id>'
   ```

2. Look up region IDs associated with the cloud account and in the external region ID `eastus`.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/iaas/api/
   regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'eastus'%20and%20cloudAccountId%20eq
   %20'"$zaure_cloud_account_id"'"  | jq "."
   ```

3. Examine the response to find the ID for the region that you want.
4. Assign the region ID variable.

   ```
   azure_region_id='<your_region_id>'
   ```

5. Create a storage profile for the selected region.

   ```
   curl -X POST \
     $url/iaas/api/storage-profiles?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "defaultItem": false,
       "supportsEncryption": false,
       "tags": [ { "key": "type", "value": "managed" } ],
       "diskProperties": {
          "azureManagedDiskType": "Standard_LRS",
          "azureOsDiskCaching": "ReadWrite",
          "azureDataDiskCaching": "ReadWrite"
     },
     "regionId": "'$azure_region_id'",
     "name": "<your-azure-managed-disk-storage-profile>",
     "description": "Example Azure managed disk"
   }' | jq "."
   ```

6. To obtain the storage profile ID, examine the response.
7. Assign the storage profile ID variable.

   ```
   azure_storage_profile_id='<your_storage_profile_id>'
   ```

8. Optional: Look up the storage profile you created with your storage profile ID.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
storage-profiles/$azure_storage_profile_id?apiVersion=$api_version | jq "."
```

The response shows the name and ID for the storage profile you created.

Note:
Using the external region ID and the cloud account ID, you can also filter for the storage profile with a query that does not require the storage profile ID. See Filtering Resources by Region ID.

Create a Microsoft Azure storage profile

Assign the required variables including the cloud account ID for a Microsoft Azure cloud account.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ azure_cloud_account_id='c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254
c4033013bf3283908e4661cd1c6fb2f8b9ae-ce5aad01092b47558644f6b6615d'
```

Look up region IDs associated with the cloud account and in the external region ID `eastus`

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
iaas/api/regions/?apiVersion=$api_version&"'$filter'"=externalRegionId%20eq%20'us-east-1'%20and
%20cloudAccountId%20eq%20'"$azure_cloud_account_id"'"  | jq "."
```

A snippet of the response shows the region ID.

```
...
      "externalRegionId": "eastus",
      "cloudAccountId":
 "c8c3c9bfdb449475-7f703c5265a63d87-5fa34c478df36b060e1ca3551254c4033013bf3283908e4661cd1c6fb2f8b9ae-
ce5aad01092b47558644f6b6615d",
      "id": "20d6c1abc4a8275586468873c721",
      "updatedAt": " ",
...
```

Assign the Azure region ID.

```
$ azure_region_id='20d6c1abc4a8275586468873c721'
```

Create a Microsoft Azure storage profile named `azure-with-managed-disks-storage-profile`.

```
$ curl -X POST \
  $url/iaas/api/storage-profiles?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "defaultItem": false,
    "supportsEncryption": false,
    "tags": [ { "key": "type", "value": "managed" } ],
    "diskProperties": {
       "azureManagedDiskType": "Standard_LRS",
       "azureOsDiskCaching": "ReadWrite",
       "azureDataDiskCaching": "ReadWrite"
  },
  "regionId": "'$azure_region_id'",
  "name": "azure-with-managed-disks-storage-profile",
  "description": "Example Azure with managed disks storage profile"
}' | jq "."
```

A snippet of the response shows the storage profile ID.

```
...
  "externalRegionId": "eastus",
```

```
    "name": "azure-with-managed-disks-storage-profile",
    "description": "Example Azure with managed disks storage profile",
    "id": "f83d0fd4-45de-4ca7-a699-c20bc121abcd",
    "createdAt": " ",
    "updatedAt": " 2021-08-02",
  ...
```

Managing Your Projects Using the Project APIs

You use the Project APIs to manage projects that can be used in any of the VCF Automation component services. In addition to basic operations such as creating, updating, or deleting projects, you can use the Project APIs to modify principals assigned to a project or modify a project cost.

Create a Project with the Project Service API

Using the Project Service API, you can create a project. You can also modify or delete a project and list all projects in an organization.

- Verify that all general prerequisites and prerequisites for the Project service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

  Note:

  A user with the project administrator or project member role can perform a limited number of project-related tasks.
- Prepare parameters including the project name, description, and email addresses for administrators, members, or viewers.

Before creating a project, it is a good practice to get a list of projects so that you can verify that the project you plan to create does not exist. Then you create the project with users assigned to project roles.

1. Get a list of projects.

   ```
   curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-service/
   api/projects?apiVersion=$api_version" | jq "."
   ```

2. To verify that the project you plan to create is not already listed, examine the response.
3. Assign the project name variable.

   ```
   project_name='<your_project_name>'
   ```

   your_project_name is a name that you choose.
4. Create a project.

   ```
   curl -X POST \
     "$url/project-service/api/projects?apiVersion=$api_version" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "name" : "'$project_name'",
       "description" : "your-description",
       "administrators" : [{ "email" : "<admin_email>", ["type" : <"user" | "group">]}],
       "members" : [{ "email" : "<member_email>", ["type" : <"user" | "group">]}],
       "viewers" : [{ "email" : "<viewer_email>", ["type" : <"user" | "group">]}],
     }' | jq "."
   ```

   - *admin_email*, *member_email*, and *viewer_email* are email addresses of an administrator, member, and viewer user or name of the group in the project.
   - The type parameter is optional. It assigns the administrator, member, or viewer to a user or group type. If unspecified, the value defaults to `user`.

Create a Project

Create a project named `Example-project` with administrators, members, and viewers at `mycompany.com`. This example assumes that `Example-project` does not exist.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_name='Example-project'
```

Create a project with an administrator, member, and viewer assigned to user type roles.

```
$ curl -X POST \
  "$url/project-service/api/projects?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "name" : "'$project_name'",
    "description" : "This is an example project",
    "administrators" : [{"email" : "adminX@mycompany.com", "type" : "user"}],
    "members" : [{"email" : "memberX@mycompany.com", "type" : "user"}],
    "viewers" : [{"email" : "viewerX@mycompany.com", "type" : "user"}]
  }' | jq "."
```

The response shows the administrators, members, and viewers related to the project and the project ID.

```
{
      "id": "094a2fab-7715-4844-94f9-71b45452da27",
      "name": "Example-project",
      "description": "This is an example project",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "administrators": [
        {
          "email": "adminX@mycompany.com",
          "type": "user"
        }
      "members": [
        {
          "email": "memberX@mycompany.com",
          "type": "user"
        },
      ],
      "viewers": [
        {
          "email": "viewerX@mycompany.com",
          "type": "user"
        }
      ]
      "supervisors": [],
      "constraints": {
        "network": {
          "conditions": []
        }
      },
      "properties": {},
      "cost": {
        "cost": 0,
        "costSyncTime": "2019-05-13T12:47:10.624Z",
        "costUnit": "USD"
      },
      "operationTimeout": 0,
```

```
        "sharedResources": true
      },
  ...
```

Add Users to Your Project Using the Project Service API

As a service administrator, you can use a PATCH request to add, modify, or remove a project user.

- Verify that all general prerequisites and prerequisites for the Project service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the project administrator role in your project and you have the project ID. See Create a Project with the Project Service API.
- Verify that the project roles that you plan to assign have sufficient permissions to perform project-related tasks.

  Note:

  A user with the project administrator or project member role can perform a limited number of project-related tasks.
- Prepare parameters including additional email addresses for administrators, members, or viewers that you want to add to the project.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

   your_project_id is the ID of the new project you created.
2. List the details of your project.

   ```
   curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-service/
   api/projects/$project_id?apiVersion=$api_version" | jq "."
   ```
3. Examine the response to see the administrators and users who are already in your project.
4. Submit a request to add a new project administrator.

   ```
   curl -X PATCH \
     "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "modify" : [
         {
           "email" : "<your_new_administrator_email>",
           "role" : "administrator",
           "type" : "user"
         }
       ]
     }' | jq "."
   ```
5. Submit a request to add a new project member.

   ```
   curl -X PATCH \
     "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token" \
     -d '{
       "modify" : [
         {
           "email" : "<your_new_member_email>",
           "role" : "member",
           "type" : "user"
         }
       ]
     }' | jq "."
   ```

Add Users to Your Project

For the project `Example-project`, add a user with the administrator role and a user with the member role at `mycompany.com`.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='094a2fab-7715-4844-94f9-71b45452da27'
```

List the details of your project.

```
$ curl -X GET -H 'Accept: application/json' -H "Authorization: Bearer $access_token" "$url/project-
service/api/projects/$project_id?apiVersion=$api_version" | jq "."
```

The response shows existing administrators, members, and viewers.

```
{
      "id": "094a2fab-7715-4844-94f9-71b45452da27",
      "name": "Example-project",
      "description": "This is an example project",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "administrators": [
        {
          "email": "adminX@mycompany.com",
          "type": "user"
        }
      "members": [
        {
          "email": "memberX@mycompany.com",
          "type": "user"
        },
      ],
      "viewers": [
        {
          "email": "viewerX@mycompany.com",
          "type": "user"
        }
      ]
      "supervisors": [],
      "constraints": {
        "network": {
          "conditions": []
        }
      },
      "properties": {},
      "cost": {
        "cost": 0,
        "costSyncTime": "2019-05-13T12:47:10.624Z",
        "costUnit": "USD"
      },
      "operationTimeout": 0,
      "sharedResources": true
    },
...
```

Add the administrator.

```
curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
```

```
  -d '{
    "modify" : [
      {
        "email" : "newadminX@mycompany.com",
        "role" : "administrator",
        "type" : "user"
      }
    ]
  }' | jq "."
```

Add the member.

```
curl -X PATCH \
  "$url/project-service/api/projects/$project_id/principals?apiVersion=$api_version" \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "modify" : [
      {
        "email" : "newmemberX@mycompany.com",
        "role" : "member",
        "type" : "user"
      }
    ]
  }' | jq "."
```

The response lists the project including the added administrator and added member.

```
{
      "id": "094a2fab-7715-4844-94f9-71b45452da27",
      "name": "Example-project",
      "description": "This is an example project",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "administrators": [
        {
          "email": "newadminX@mycompany.com",
          "type": "user"
        }
        {
          "email": "adminX@mycompany.com",
          "type": "user"
        }
      "members": [
        {
          "email": "newmemberX@mycompany.com",
          "type": "user"
        }
        {
          "email": "memberX@mycompany.com",
          "type": "user"
        },
      ],
      "viewers": [
        {
          "email": "viewerX@mycompany.com",
          "type": "user"
        }
      ]
      "supervisors": [],
```

```
    "constraints": {
      "network": {
        "conditions": []
      }
    },
    "properties": {},
    "cost": {
      "cost": 0,
      "costSyncTime": "2019-05-13T12:47:10.624Z",
      "costUnit": "USD"
    },
    "operationTimeout": 0,
    "sharedResources": true
  }
```

Working with Blueprints/Cloud Templates

To create and update cloud templates, version cloud templates, and deploy cloud templates, you use the VCF Automation Blueprint APIs. Blueprint is the term used in the API specifications. In the product, blueprints are renamed to VCF Automation Templates.

Create and Update a Cloud Template

To create a cloud template, you make a POST request. The request body includes the name of the new cloud template and the project ID of an existing project. To update a cloud template, you make a PUT request that changes one of the properties of the cloud template.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have a project ID for the project that includes cloud zones configured to support the resource requirements of your cloud template. See Create a Project to use in VCF Automation.

Before creating a cloud template, you get a list of cloud templates to verify that the cloud template you plan to create does not already exist. After a cloud template is created, you can update it to change the cloud template definition.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

2. Assign your cloud template name variable.

   ```
   cloud_template_name='<your_cloud_template_name>'
   ```

   your_cloud_template_name is a name that you choose.

3. Get a list of cloud templates.

   ```
   curl -X GET $url/blueprint/api/blueprints -H "Authorization: Bearer $access_token"  | jq "."
   ```

4. To verify that the cloud template you plan to create is not already listed, examine the response.

5. Validate the cloud template before creating it.

   ```
   curl -X POST \
     $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
     -H 'Content-Type: application/json' \
     -H "Authorization: Bearer $access_token"  \
     -d '{
       "name" : "'"$cloud_template_name"'",
       "description" : "Basic Cloud Machine cloud template",
       "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
    description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
    Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
   ```

```
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
 properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n      image:
 '\''${input.image}'\''\n      count: '\''${input.count}'\''",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

6. Examine the response to confirm that you see `"valid":true`.
7. Create a new cloud template.

```
curl -X POST \
  $url/blueprint/api/blueprints?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
 properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n      image:
 '\''${input.image}'\''\n      count: '\''${input.count}'\''",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

8. Examine the response and record the ID of your newly created cloud template.
9. Assign the cloud template ID variable.

```
cloud_template_id='<your_cloud_template_id>'
```

   your_cloud_template_id is the ID of the new cloud template that you created.

10. To verify that the cloud template has been created, get a list of cloud templates and filter for *your_cloud_template_name*.

```
curl -X GET $url/blueprint/api/blueprints?name=$cloud_template_name -H "Authorization: Bearer
 $access_token"  | jq "."
```

11. Optional: To update the cloud template, use a PUT request and specify *your_cloud_template_id*.

```
curl -X PUT \
  $url/blueprint/api/blueprints/$cloud_template_id?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
 properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n      image:
 '\''${input.image}'\''\n      count: '\''${input.count}'\''\n      tags: [\n        {\n          \"key\":
 \"env\",\n          \"value\": \"prod\"\n        }\n      ]\n",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

Create a Cloud Template and Update it

Create a cloud template named `MyExampleCloudTemplate`. This example assumes that `MyExampleCloudTemplate` does not already exist.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
$ cloud_template_name='MyExampleCloudTemplate'
```

Validate the cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic cloud machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
   properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n
 image: '\''${input.image}'\''\n      count: '\''${input.count}'\''",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

Examine the response to confirm that `"valid":true`.

```
{
  "valid": true,
  "validationMessages": []
}
```

Create the cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprints?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic cloud machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
   properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n
 image: '\''${input.image}'\''\n      count: '\''${input.count}'\''",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

The response from your request to create the cloud template shows the cloud template ID.

```
{
  "id": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "createdAt": "2019-10-10T23:43:27.001Z",
```

```
  ...
  "selfLink": "/blueprint/api/blueprints/1f170637-81a3-4257-b1cd-b2219ee8034c"
  ...
}
```

Assign the cloud template ID variable.

```
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
```

Update the cloud template to set a tag on the machine properties in the cloud template.

```
$ curl -X PUT \
  $url/blueprint/api/blueprints/$cloud_template_id?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_name"'",
    "description" : "Basic cloud machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
   properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n
 image: '\''${input.image}'\''\n      count: '\''${input.count}'\''\n      tags: [\n        {\n
      \"key\": \"env\",\n          \"value\": \"prod\"\n        }\n      ]\n",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

Use the cloud template ID to version and release the cloud template to a catalog.

Setting up Policies

As a cloud administrator, you can use the VCF Automation APIs to create policies with rules or parameters that you apply to deployments in VCF Automation.

Create an Approval Policy

To add a layer of governance to deployment requests before they are run, you can create an approval policy using the Policies API. The policy controls who must agree to a deployment or Day 2 action before a request is provisioned.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See Prerequisites for API Use Case Examples.
- Assign an API version variable for the Approvals API.

  ```
  api_version_approval=' '
  ```

  Note:
  The Approvals APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the general prerequisites.
- Verify that you know the name of the Cloud Template to which you want to apply the approval policy.

You create an approval policy based on certain deployment criteria, such as deployments created from a specific cloud template. For example, if you specify a cloud template ID, you can create a policy that requires a specified level of approval for deployments created from that cloud template.
The following procedure shows how to use the Approval API to get the cloud template ID and list approval actions for a deployment before creating the approval policy using the Policy API.

1. List the cloud templates.

   ```
   curl -X GET \
     $url/approval/api/policy/data/blueprints?apiVersion=$api_version_approval \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

2. Examine the response to find the ID of the cloud template for the approval policy.
3. Assign the cloud template variable.

```
cloudtemplateId = "<your_cloud_template_ID>"
```

4. Optional: If you do not know the actions to specify in the policy, list the IDs of deployment actions.

```
curl -X GET \
  $url/approval/api/policy/data/actions?apiVersion=$api_version_approval&search=Deployment \
  -H "Authorization: Bearer $access_token" | jq "."
```

   Note the action IDs.

5. Create an approval policy with hard enforcement that is applied to deployments created from the cloud template with `cloudtemplateId`.
   - For an approval policy, you specify `"typeId": "com.vmware.policy.approval"`
   - `autoApprovalExpiry` specifies the number of days that the approvers have to act before triggering the `autoApprovalDecision`
   - `level` specifies the order in which the policy is applied with values 1-99. For example, level 1 approvals are applied first, followed by level 2 approvals and so forth.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_approval_policy_name>",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.approval"
    "definition": {
      "level": <policy_level>,
      "approverType": "USER",
      "approvalMode": "ALL_OF",
      "autoApprovalDecision": "APPROVE",
      "approvers": [
          "USER:<approver1_ID>",
          "USER:<approver2_ID>"
      ],
      "autoApprovalExpiry": <number_of_days>,
      "actions": [
          "<actionID_1>",
          "<actionID_2",
          "<actionID_3>"
      ]
    },
    "criteria": {
      "matchExpression": [
        {
           "key": "blueprintId",
           "operator": "eq",
           "value": "'$cloudtemplateId'"
        }
      ]
    }
  }' | jq "."
```

Create an approval policy

Create a policy named `Sample Approval Policy` to apply to deployments created from a cloud template named `template-1`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_approval=' '
```

List the cloud templates.

```
curl -X GET \
  $url/approval/api/policy/data/blueprints?apiVersion=$api_version_approval \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the cloud template named `template-1`.

```
...
   {
      "id": "77265efc-6d06-428e-9fad-3ad8f31441f3",
      "name": "template-1",
      "description": ""
    }
...
```

Assign the cloud template ID variable.

```
$ cloudtemplateId = "77265efc-6d06-428e-9fad-3ad8f31441f3"
```

List the deployment actions.

```
curl -X GET \
  $url/approval/api/policy/data/actions?apiVersion=$api_version_approval&search=Deployment \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the IDs of the deployment actions that you want to specify in your approval policy.

```
...
   {
      "id": "Deployment.Create",
      "name": "Create",
      "description": "Create a deployment",
      "resourceType": "Deployment"
    },
...
   {
      "id": "Cloud.Azure.Machine.PowerOn",
      "name": "Power On",
      "description": "Power on a machine",
      "resourceType": "Cloud.Azure.Machine"
    },
...
   {
      "id": "Cloud.Azure.Machine.PowerOff",
      "name": "Power Off",
      "description": "Power off a machine",
      "resourceType": "Cloud.Azure.Machine"
    },
...
```

Use the cloud template ID to create the approval policy of level 2 with hard enforcement named `Sample Approval Policy`. When a deployment is requested, users listed will act as approvers for the actions: provision, power on, and power off an Azure machine. If approvers do not act within three days, then the deployment actions are automatically approved.

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Approval Policy",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.approval"
    "definition": {
      "level": 2,
      "approverType": "USER",
      "approvalMode": "ALL_OF",
      "autoApprovalDecision": "APPROVE",
      "approvers": [
          "USER:mary@mycompany.com",
          "USER:susan@mycompany.com"
       ],
      "autoApprovalExpiry": 3,
      "actions": [
          "Deployment.Create",
          "Cloud.Azure.Machine.PowerOn",
          "Cloud.Azure.Machine.PowerOff"
       ]
    },
    "criteria": {
      "matchExpression": [
        {
          "key": "blueprintId",
          "operator": "eq",
          "value": "'$cloudtemplateId'"
        }
      ]
    }
  }' | jq "."
```

The response shows the approval policy.

```
{
  "id": "62ad2f02-0b2a-4ed8-a739-a6c40d761e49",
  "name": "Sample Approval Policy",
  "typeId": "com.vmware.policy.approval",
  "enforcementType": "HARD",
  "orgId": "d2994f92-bd52-45b1-9220-686b20944c2c",
  "definition": {
    "level": 2,
    "approverType": "USER",
    "approvalMode": "ALL_OF",
    "autoApprovalDecision": "APPROVE",
    "approvers": [
        "USER:mary@mycompany.com",
        "USER:susan@mycompany.com"
     ],
    "autoApprovalExpiry": 3,
    "actions": [
        "Deployment.Create",
        "Cloud.Azure.Machine.PowerOn",
        "Cloud.Azure.Machine.PowerOff"
    ]
```

```
    },
    "criteria": {
      "matchExpression": [
        {
          "key": "blueprintId",
          "operator": "eq",
          "value": "77265efc-6d06-428e-9fad-3ad8f31441f3"
        }
      ]
    },
    "createdAt": " T09:45:38.108885Z",
    "createdBy": "admin@mycompany.com",
    "lastUpdatedAt": " T09:45:38.108885Z",
    "lastUpdatedBy": "admin@mycompany.com"
  }
```

How to Create Resource Quota Policies

To limit the resource consumed by each user, project, or organization, you can create a resource quota policy using the Policies API. Applying the policy ensures that users do not consume all available resources.

With the policy defined, any new deployment requests are evaluated against the resource quota. If a resource request exceeds the quota, the deployment request fails with a message that indicates the reason for the failure. The resource quota does not apply to deployment requests that are already in-progress when the policy is defined.

In cases where multiple policies are defined for a resource at the same scope level, the request is evaluated against the quota with the lowest limit value.

For information about policy scope, see How do I configure policy scope in for VM Apps.

The following examples show how to define a resource quota policy with:

- Organization level limits.
- Project level limits.

For both definitions, you use `POST /policy/api/policies` to create the policy with the input parameter `"typeId": "com.vmware.policy.resource.quota"`.

Prerequisites for creating a resource quota policy

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you know the resource quotas that you want to apply.

Resource quota policy with organizational level limits

This example shows how to define a resource quota policy with hard enforcement and limits on resource consumption by the organization and each user within the organization.

```
curl -k -X POST \
  "$url/policy/api/policies?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Resource Quota Policy 1",
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.resource.quota",
    "definition": {
      "orgLevel": {
          "limits": {
              "cpu": {
                  "value": 50
              },
```

```
                "storage": {
                    "value": 150,
                    "unit": "GB"
                },
                "memory": {
                    "value": 150,
                    "unit": "GB"
                },
                "instances": {
                    "value": 50
                }
            },
            "userLevel": {
                "limits": {
                    "cpu": {
                        "value": 4
                    },
                    "storage": {
                        "value": 10,
                        "unit": "GB"
                    },
                    "memory": {
                        "value": 10,
                        "unit": "GB"
                    },
                    "instances": {
                        "value": 2
                    }
                }
            }
        }
    }
}' | jq "."
```

The response shows the resource quota policy with organization and organization user level limits.

```
{
    "id": "52f67297-3e15-4e0a-9336-35894d2be0bc",
    "name": "Sample Resource Quota Policy 1",
    "typeId": "com.vmware.policy.resource.quota",
    "enforcementType": "HARD",
    "orgId": "74a191fc-27e0-4f09-af0d-6acd04d60832",
    "definition": {
      "orgLevel": {
          "limits": {
              "cpu": {
                  "value": 50
              },
              "storage": {
                  "value": 150,
                  "unit": "GB"
              },
              "memory": {
                  "value": 150,
                  "unit": "GB"
              },
              "instances": {
                  "value": 50
```

```
                }
            },
            "userLevel": {
                "limits": {
                    "cpu": {
                        "value": 4
                    },
                    "storage": {
                        "value": 10,
                        "unit": "GB"
                    },
                    "memory": {
                        "value": 10,
                        "unit": "GB"
                    },
                    "instances": {
                        "value": 2
                    }
                }
            }
        }
    },
    "createdAt": " T11:51:47.742311Z",
    "createdBy": "admin@mycompany.com",
    "lastUpdatedAt": " T11:51:47.742311Z",
    "lastUpdatedBy": "admin@mycompany.com"
}
```

Resource quota policy with project level limits

This example shows how to define a resource quota policy with hard enforcement and limits on resource consumption by a project and each project user.

```
curl -k -X POST \
  "$url/policy/api/policies?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Resource Quota Policy 2",
    "enforcementType": "HARD",
    "projectId": "6df55bb1-7444-4c13-9997-4004ba0a321d",
    "scopeCriteria": null,
    "typeId": "com.vmware.policy.resource.quota",
    "definition": {
      "projectLevel": {
          "limits": {
              "cpu": {
                  "value": 50
              },
              "storage": {
                  "value": 80,
                  "unit": "GB"
              },
              "memory": {
                  "value": 80,
                  "unit": "GB"
              },
              "instances": {
```

```
                    "value": 40
                }
            },
            "userLevel": {
                "limits": {
                    "cpu": {
                        "value": 5
                    },
                    "storage": {
                        "value": 8,
                        "unit": "GB"
                    },
                    "memory": {
                        "value": 8,
                        "unit": "GB"
                    },
                    "instances": {
                        "value": 4
                    }
                }
            }
        }
    }
}' | jq "."
```

The response shows the resource quota policy with project and project user level limits.

```
{
    "id": "5809ec88-16fb-4553-98c3-5b588ebff322",
    "name": "Sample Resource Quota Policy 2",
    "typeId": "com.vmware.policy.resource.quota",
    "enforcementType": "HARD",
    "orgId": "74a191fc-27e0-4f09-af0d-6acd04d60832",
    "projectId": "6df55bb1-7444-4c13-9997-4004ba0a321d",
    "definition": {
      "projectLevel": {
          "limits": {
              "cpu": {
                  "value": 50
              },
              "storage": {
                  "value": 80,
                  "unit": "GB"
              },
              "memory": {
                  "value": 80,
                  "unit": "GB"
              },
              "instances": {
                  "value": 40
              }
          },
          "userLevel": {
              "limits": {
                  "cpu": {
                      "value": 5
                  },
                  "storage": {
```

```
                    "value": 8,
                    "unit": "GB"
                },
                "memory": {
                    "value": 8,
                    "unit": "GB"
                },
                "instances": {
                    "value": 4
                }
            }
        }
    }
  },
  "createdAt": " T12:09:34.160569Z",
  "createdBy": "admin@mycompany.com",
  "lastUpdatedAt": " T12:09:34.160569Z",
  "lastUpdatedBy": "admin@mycompany.com"
}
```

Create a Deployment Limit Policy

To limit resource consumption when users deploy cloud templates in VCF Automation and request catalog items in VCF Automation, you can create a limit policy using the Policies API. The policy applies limits to all deployments in an organization by default.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See Prerequisites for API Use Case Examples.
- Assign an API version variable for the Deployment Limit API.

  ```
  api_version_deploymentlimit=' '
  ```

  Note:
  The Deployment Limit APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the general prerequisites. The Deployment Limit APIs are grouped with the Approvals services. See https://techdocs.broadcom.com/us/en/vmware-cis/aria/aria-automation/8-16/what-is-the-api-and-how-do-i-use-it.html.
- Verify you know the resource criteria that you want to use to restrict the policy.

You create a deployment limit policy based on certain resource criteria, such as account names, account types, cloud templates, cloud zones, flavors, and many more. For example, if you specify a cloud template ID as the resource criteria, you can restrict the policy so that it only applies limits to deployments created from a specific cloud template.

Note:
If approval policy or resource quota policy definitions affect deployments within the policy scope, deployment limits are enforced before the other policy types.

The following procedure shows how to use the Deployment limit API to get the cloud template ID before creating the deployment limit policy using the Policy API.

1. List the cloud templates.

   ```
   curl -X GET \
    $url/deploymentlimit/api/policy/data/blueprints?apiVersion=$api_version_deploymentlimit \
    -H "Authorization: Bearer $access_token" | jq "."
   ```

2. Examine the response to find the ID of the cloud template used to create the deployments where you want to limit resource usage.
3. Assign the cloud template variable.

   ```
   cloudtemplateId = "<your_cloud_template_ID>"
   ```

4. Create a deployment limit policy with hard enforcement that is applied to deployments created from the cloud template with `cloudtemplateId`. For the deployment limit policy, you specify `"typeId": "com.vmware.policy.deployment.limit"`.

```
curl -X POST \
 $url/policy/api/policies?apiVersion=$api_version \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
   "name": "<your_limit_policy_name>",
   "enforcementType": "HARD",
   "typeId": "com.vmware.policy.deployment.limit"
   "definition": {
     "deploymentLimits": {
       "cpu": {
         "value": 6
       },
       "memory": {
         "unit": "GB",
         "value": 5
       },
       "instances": {
         "value": 3
       },
       "storage": {
         "unit": "GB",
         "value": "20
       }
     },
     "deploymentResourceLimits": {
       "resources": [
         {
           "name": "vSphere-Machine-Limits",
           "limits": {
             "cpu": {
               "value": 2
             },
             "memory": {
               "unit": "GB",
               "value": 2
             },
             "storage": {
               "unit": "GB",
               "value": "20
             }
           },
           "criteria": {
             "matchExpression": [
               {
                 "key": "type",
                 "value": "Cloud.vSphere.Machine",
                 "operator": "eq"
               }
             ]
           }
         }
       ]
     },
```

```
      "criteria": {
        "matchExpression": [
          {
            "key": "blueprintId",
            "operator": "eq",
            "value": "'$cloudtemplateId'"
          }
        ]
      }
    }
  }
| jq "."
```

Create a deployment limit policy

Create a deployment limit policy named `Sample Limit Policy` that is applied to limit resource usage in deployments created from a cloud template named `template2`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_deploymentlimit='2020-11-01'
$ orgId='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

List the cloud templates.

```
$ curl -X GET \
 $url/deploymentlimit/api/policy/data/blueprints?apiVersion=$api_version_deploymentlimit \
 -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the cloud template named `template2`.

```
...
{
     "id": "3d3c714f-0aeb-423d-a494-97e85e4a8566",
     "name": "template2",
     "description": "Cloud template for example deployment"
   },
...
```

Assign the cloud template ID variable.

```
$ cloudtemplateId = "3d3c714f-0aeb-423d-a494-97e85e4a8566"
```

Use the cloud template ID to create the deployment limit policy with hard enforcement named `Sample Limit Policy`. The value for the type ID is fixed as `com.vmware.policy.deployment.limit`.

```
$ curl -X POST \
 $url/policy/api/policies?apiVersion=$api_version \
 -H "Authorization: Bearer $access_token" \
 -H 'Content-Type: application/json' \
 -d '{
   "name": "Sample Limit Policy",
   "enforcementType": "HARD",
   "typeId": "com.vmware.policy.deployment.limit"
   "definition": {
     "deploymentLimits": {
       "cpu": {
         "value": 6
       },
       "memory": {
```

```
          "unit": "GB",
          "value": 5
        },
        "instances": {
          "value": 3
        },
        "storage": {
          "unit": "GB",
          "value": "20
        },
      },
      "deploymentResourceLimits": {
        "resources": [
          {
            "name": "vSphere-Machine-Limits",
            "limits": {
              "cpu": {
                "value": 2
              },
              "memory": {
                "unit": "GB",
                "value": 2
              },
              "storage": {
                "unit": "GB",
                "value": "20
              }
            },
            "criteria": {
              "matchExpression": [
                {
                  "key": "type",
                  "value": "Cloud.vSphere.Machine",
                  "operator": "eq"
                }
              ]
            }
          }
        ]
      },
      "criteria": {
        "matchExpression": [
          {
            "key": "blueprintId",
            "operator": "eq",
            "value": "'$cloudtemplateId'"
          }
        ]
      }
    }
  }
| jq "."
```

The response shows the deployment limit policy.

```
{
 "id": "62ad2f02-0b2a-4ed8-a739-a6c40d761e49",
 "name": "Sample Limit Policy",
```

```
"typeId": "com.vmware.policy.deployment.limit",
"enforcementType": "HARD",
"orgId": "d2994f92-bd52-45b1-9220-686b20944c2c",
"definition": {
  "deploymentLimits": {
    "cpu": {
      "value": 6
    },
    "memory": {
      "unit": "GB",
      "value": 5
    },
    "instances": {
      "value": 3
    },
    "storage": {
      "unit": "GB",
      "value": 20
    }
  },
  "deploymentResourceLimits": {
    "resources": [
      {
        "name": "vSphere-Machine-Limits",
        "limits": {
          "cpu": {
            "value": 2
          },
          "memory": {
            "unit": "GB",
            "value": 2
          },
          "storage": {
            "unit": "GB",
            "value": "20
          }
        },
        "criteria": {
          "matchExpression": [
            {
              "key": "type",
              "value": "Cloud.vSphere.Machine",
              "operator": "eq"
            }
          ]
        }
      }
    ]
  }
},
"criteria": {
  "matchExpression": [
    {
      "key": "blueprintId",
      "operator": "eq",
      "value": "7950795a-4f66-451c-a79f-be9ef6bd723c"
    }
```

```
      ]
    },
    "createdAt": " 8T09:45:38.108885Z",
    "createdBy": "user@mycompany.com",
    "lastUpdatedAt": " T09:45:38.108885Z",
    "lastUpdatedBy": "user@mycompany.com"
  }
```

Create a Content Sharing Policy

As a VCF Automation administrator, you can create a content sharing policy that entitles all VCF Automation users in a project to shared content defined in the policy.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See Prerequisites for API Use Case Examples.

  Note:
  The Policies API and the Catalog API have the same API version. You set the API version value for both services when you satisfy the prerequisites for the Policies service.
- Assign an API version variable for the Projects API.

  ```
  api_version_projects=' '
  ```

  Note:
  The Projects APIs and Policies APIs have different API version values. You set the API version value for the Policies APIs when you satisfied the prerequisites for the Policies service.

The following procedure shows how to use the Policies API to create a content sharing policy that defines shared content for users or groups that are part of the same project. To get the input required for the project, project users, and project groups, you use the Projects API. To get the catalog sources or catalog items to share in the policy, you use the Catalog API.
The steps to get catalog sources or items to share are optional, because a content sharing policy may not share both sources and items. However, the policy must share at least one catalog source or item.

1. Get a list of all projects in your organization.

   ```
   curl -X GET "$url/project-service/api/projects?apiVersion=$api_version_projects" -H "Authorization: Bearer
    $access_token"  | jq "."
   ```

   Examine the response to get the ID of the project for the shared policy. The project must include the users or groups for which you want to entitle content. See Create a Project with the Project Service API.

2. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

3. Get a list of users in the project.

   ```
   curl -X GET \
   $url/project-service/api/projects/$project_id/principals?apiVersion=
   $api_version_projects&expandGroups=true& \
   '$filter='"(substringof('{}',tolower(acct)))" \
   -H "Authorization: Bearer $access_token"  | jq "."
   ```

   Examine the response to get the user email addresses for the project users. Users selected in the policy are entitled to items shared with the project.

4. Get a list of groups in the project.

   ```
   curl -X GET \
   $url/project-service/api/projects/$project_id/groups?apiVersion=$api_version_projects&page=0 \
   '$filter='"(substringof('{}',tolower(displayName)))" \
   -H "Authorization: Bearer $access_token"  | jq "."
   ```

   Examine the response to get the group display name for the project users that are part of the group. Groups selected in the policy are entitled to items shared with the project.

5. Optional: Get a list of catalog sources for your project.

```
curl -X GET $url/catalog/api/admin/sources?projectId=$project_id&apiVersion=$api_version -H
 "Authorization: Bearer $access_token"  | jq "."
```

   Examine the response to find catalog sources that you want to share.

6. Optional: If you are sharing a catalog source, assign the catalog source ID variable.

```
catalog_source_id='<shared_catalog_source_id>'
```

7. Optional: Get a list of catalog items for your project.

```
curl -X GET $url/catalog/api/items?projectId=$project_id&apiVersion=$api_version -H "Authorization: Bearer
 $access_token"  | jq "."
```

   Examine the response to find catalog items that you want to share.

8. Optional: If you are sharing a catalog item, the catalog item ID variable.

```
catalog_item_id='<shared_catalog_item_id>'
```

9. Create a content sharing policy with hard enforcement.

   - The value for user type is always `USER`.
   - The group's display name is always in the format `groupDisplayName@groupDomain`.
   - The typeId for the content sharing policy is always `com.vmware.policy.catalog.entitlement`.

```
curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "<your_content_sharing_policy_name>",
    "projectId": "'$project_id'",
    "definition": {
        "entitledUsers": [
            {
                "userType": "USER",
                "principals": [
                    {
                        "type": "USER",
                        "referenceId": "<user1_email_address>"
                    },
                    {
                        "type": "USER",
                        "referenceId": "<user2_email_address>"
                    },
                    {
                        "type": "PROJECT",
                        "referenceId": "$project_id'"
                    },
                    {
                        "type": "GROUP",
                        "referenceId": "<groupDisplayName1@groupDomain>"
                    }
                ],
                "items": [
                    {
                        "id": "'$catalog_source_id'",
                        "type": "CATALOG_SOURCE_IDENTIFIER"
                    },
                    {
                        "id": "'$catalog_item_id'",
```

```
                        "type": "CATALOG_ITEM_IDENTIFIER"
                    }
                ]
            }
        ]
    },
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.catalog.entitlement"
  }
}' | jq "."
```

Create a content sharing policy

Create a policy named `Sample Sharing Policy`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_projects=' '
```

List the projects in your organization.

```
$ curl -X GET "$url/project-service/api/projects?apiVersion=$api_version_projects" -H
 "Authorization: Bearer $access_token"  | jq "."
```

Examine the response to find the project that you want to use for the shared content policy.

```
{
      "id": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e",
      "name": "Example-project",
      "description": "This is an example project",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
...
```

Assign the project ID.

```
$ project_id='1d0bcd42-4d8f-4a8f-8b31-f34a4707533e'
```

List the users in the project.

```
$ curl -X GET \
 $url/project-service/api/projects/$project_id/principals?apiVersion=
$api_version_projects&expandGroups=true& \
 '$filter='"(substringof('{}',tolower(acct)))" \
 -H "Authorization: Bearer $access_token"  | jq "."
```

A snippet of the response shows the email addresses for the users in the project.

```
...
    "content": [
        {
            "id": "ab373898-d29b-4e3b-8703-58023cadd140",
            "acct": "user1@mycompany.com",
            "domain": "mycompany.com"
        }
    ],
...
```

Get a list of groups in the project.

```
$ curl -X GET \
$url/project-service/api/projects/$project_id/groups?apiVersion=$api_version_projects&page=0 \
'$filter='"(substringof('{}',tolower(displayName)))" \
-H "Authorization: Bearer $access_token"  | jq "."
```

A snippet of the response shows the group display name for the users that are part of the group.

```
...
  "content": [
        {
            "id": "a9da96e7-ba19-47d4-9f38-dd1983e29424",
            "displayName": "test@mycompany.com",
            "groupType": "USER_GROUP",
            "usersCount": 2
        },
...
```

Get a list of catalog sources for your project.

```
$ curl -X GET $url/catalog/api/admin/sources?projectId=$project_id&apiVersion=$api_version -H
 "Authorization: Bearer $access_token"  | jq "."
```

Examine the response to find the catalog sources that you want to share.

```
...
    "content": [
        {
            "id": "600026c6-3155-4395-a990-580ff1159e82",
            "name": "BpContent-Quality Engineering",
            "description": "For Project-Quality Engineering",
            "typeId": "com.vmw.blueprint",
            "createdAt": "2022-10-12T10:37:01.751799Z",
            "createdBy": "admin@mycompany.com",
            "lastUpdatedAt": "2022-10-17T05:06:33.976796Z",
            "lastUpdatedBy": "system-user",
            "config": {
                "sourceProjectId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e"
            },
      ...
```

Assign the ID of the catalog source to share.

```
$ catalog_source_id='600026c6-3155-4395-a990-580ff1159e82'
```

Get a list of catalog items for your project.

```
$ curl -X GET $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version -H
 "Authorization: Bearer $access_token"  | jq "."
```

Examine the response to find the catalog item.

```
...
     "content": [
        {
            "id": "b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b",
            "name": "CF-SQAVC67-Centos-MultiMachine",
            "description": "CF-SQAVC67-Centos-MultiMachine vsphere components, disk, networks",
            "sourceId": "600026c6-3155-4395-a990-580ff1159e82",
            "sourceName": "BpContent-Quality Engineering",
...
```

Assign the ID of the catalog item to share.

```
$ catalog_item_id='b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b'
```

Create a content sharing policy with hard enforcement.

The following example shows the group's display name which is of the format `groupDisplayName@groupDomain` where:

- `groupDisplayName` is `test@mycompany.com`
- `groupDomain` is `mycompany.com`

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sample Sharing Policy",
    "projectId": "'$project_id'",
    "definition": {
        "entitledUsers": [
            {
                "userType": "USER",
                "principals": [
                    {
                        "type": "USER",
                        "referenceId": "user1@mycompany.com"
                    },
                    {
                        "type": "PROJECT",
                        "referenceId": "'$project_id'"
                    },
                    {
                        "type": "GROUP",
                        "referenceId": "test@mycompany.com@mycompany.com"
                    }
                ],
                "items": [
                    {
                        "id": "'$catalog_source_id'",
                        "type": "CATALOG_SOURCE_IDENTIFIER"
                    },
                    {
                        "id": "'$catalog_item_id'",
                        "type": "CATALOG_ITEM_IDENTIFIER"
                    }
                ]
            }
        ]
    },
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.catalog.entitlement"
}
```

The response shows the content sharing policy.

```
{
    "id": "0cecca9a-d778-47b5-acdf-c08248406052",
    "name": "Sample Sharing Policy",
    "projectId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e",
    "definition": {
        "entitledUsers": [
            {
                "userType": "USER",
                "principals": [
                    {
                        "type": "USER",
```

```
                        "referenceId": "user1@mycompany.com"
                    },
                    {
                        "type": "PROJECT",
                        "referenceId": "1d0bcd42-4d8f-4a8f-8b31-f34a4707533e"
                    },
                    {
                        "type": "GROUP",
                        "referenceId": "test@mycompany.com@mycompany.com"
                    }
                ],
                "items": [
                    {
                        "id": "600026c6-3155-4395-a990-580ff1159e82",
                        "type": "CATALOG_SOURCE_IDENTIFIER"
                    },
                    {
                        "id": "b2d0fba7-5f62-3c79-b1b8-a2aa7d38063b",
                        "type": "CATALOG_ITEM_IDENTIFIER"
                    }
                ]
            }
        ]
    },
    "enforcementType": "HARD",
    "typeId": "com.vmware.policy.catalog.entitlement",
    "orgId": "10ea6be1-7723-4bf0-a221-8b4f3c7a26f7",
    "createdAt": "2022-10-24T07:52:22.731448Z",
    "createdBy": "admin@mycompany.com",
    "lastUpdatedAt": "2022-10-24T07:52:22.731448Z",
    "lastUpdatedBy": "admin@mycompany.com"
}
```

Version and Release a Cloud Template to a VCF Automation Catalog

After creating a cloud template, you version and release your cloud template using a POST request. The request body includes the ID of an existing cloud template and the number of the version to release.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud template ID for the cloud template you want to version and release. See Create and Update a Cloud Template.
- Verify that you know the version of the cloud template that you want to create and release to the catalog.

By versioning and releasing the cloud template, you mark a cloud template version as ready to be consumed by VCF Automation. To show the released cloud template in VCF Automation, you must have a catalog source.

1. Assign the cloud template ID variable.

   ```
   cloud_template_id='<your_cloud_template_id>'
   ```

2. Assign a cloud template version variable.

   ```
   cloud_template_version='<your_cloud_template_version>'
   ```

   your_cloud_template_version is the version that you want to create.

3. To version the cloud template without releasing it, submit the request with `"release": false`.

   ```
   curl -X POST \
     $url/blueprint/api/blueprints/$cloud_template_id/versions?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
   ```

```
  -d '{
  "changeLog": "Creating a version '"$cloud_template_version"'",
  "description": "Creating a version from the current draft",
  "release": false,
  "version": "'"$cloud_template_version"'"
}' | jq "."
```

4. Release the cloud template.

```
curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/actions/release?
apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' | jq "."
```

Version and Release a Cloud Template

Release version 5 of your cloud template with ID `1f170637-81a3-4257-b1cd-b2219ee8034c`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ cloud_template_version='v5'
```

Version the cloud template without releasing it.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
  "changeLog": "Creating a version '"$cloud_template_version"'",
  "description": "Creating a version from the current draft",
  "release": false,
  "version": "'"$cloud_template_version"'"
}' | jq "."
```

Release the cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/actions/
release?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' | jq "."
```

A snippet of the response shows the new cloud template version with a `RELEASED` status.

```
...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "name": "MyExampleCloudTemplate",
  "description": "Basic Cloud Machine cloud template",
  "version": "v5",
  "tags": [],
  "content": "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
  properties:\n      name: BasicCloudMachine\n      flavor: '${input.flavor}'      \n      image:
 '${input.image}'\n      count: '${input.count}'\n      tags: [\n        {\n          \"key\":
 \"env\",\n          \"value\": \"prod\"\n        }\n      ]\n",
  "status": "RELEASED",
```

```
    "versionDescription": "Creating a version from the current draft",
    "versionChangeLog": "Creating a version v5",
    "valid": true
  }
```

Edit and Version a Custom Form in Your Cloud Template

After creating a cloud template in VCF Automation, versioning it, and importing it into your VCF Automation content, you can add a custom request form. Then you can use the form designer to edit and version the form, so that you can choose the version and use it later.

- Verify that you have satisfied all general prerequisites and prerequisites for the VCF Automation Catalog service. See Prerequisites for API Use Case Examples.
- Verify that you have versioned and released your cloud template in VCF Automation. See Version and Release a Cloud Template to a VCF Automation Catalog.
- Verify that you have imported your cloud template into VCF Automation. See Add VCF Automation templates to the VCF Automation catalog.

The following procedure assumes that your VCF Automation content includes a versioned VCF Automation cloud template. Using the Catalog service API, you find the ID of the cloud template and get the cloud template schema for one of the template versions. To create a custom form from the schema, you use the Form Service API. Additional steps show how to customize and version the custom form, and how to list and restore a versioned form.

For more information about Custom Forms, see Learn more about custom forms in VCF Automation for VM Apps.

1. List the items in your catalog.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/catalog/
   api/admin/items?apiVersion=$api_version"  | jq "."
   ```

   Examine the response to find the catalog ID of the cloud template that you want to use.

2. Assign a variable for the cloud template catalog ID.

   ```
   item_id='<cloud_template_catalog_ID>'
   ```

3. Get information about the catalog item.

   ```
   curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/catalog/
   api/admin/items/$item_id?apiVersion=$api_version"  | jq "."
   ```

   Examine the result to choose the version of the template that you want to use for the custom form.

4. Assign a variable for the cloud template version ID.

   The form that you create and customize a form will be associated with this cloud template version ID.

   ```
   version_id='<your_cloud_template_version_ID>'
   ```

5. Using the catalog ID and version ID, get the schema for the cloud template.

   ```
   $ schema='curl -k -X GET \
   "$url/catalog/api/admin/items/$item_id/versions/$version_id \
   -H "Authorization: Bearer $access_token" \
   -H 'Content-Type: application/json' | jq -r .schema'
   ```

6. Use the schema to create a custom form.

   ```
   curl -k -X POST \
   "$url/form-service/api/forms/designer/request?sourceType=com.vmw.blueprint.version&sourceId=$item_id/
   $version_id&formType=requestForm" \
   -H "Authorization: Bearer $access_token" \
   -H 'Content-Type: application/json' \
   -d '$schema' | jq "."
   ```

   You have a current version of your custom form.

7. Customize the form.
   - *your_modified_form_json_as_string* Used to update the form field properties. For information about form field properties, see Custom form designer field properties in VCF Automation for VM Apps.
   - *your_modified_styles_string* Used to update a cascading style sheet written in CSS Syntax Module Level 3. For information about the CSS file format, see Learn more about custom forms in VCF Automation for VM Apps.

   ```
   curl -k -X POST \
   -H "Authorization: Bearer $access_token" \
   -H 'Content-Type: application/json' \
   "$url/form-service/api/forms" \
   -d '{
   "name":"Demo Blueprint / 2.0.0",
   "form":"<your_modified_form_json_as_string>",
   "styles":"<your_modified_styles_string>",
   "status":"ON", // set ON to enable the custom form, OFF to disable
   "type":"requestForm",
   "sourceId":"$item_id/$version_id",
   "sourceType":"com.vmw.blueprint.version"
   }' | jq "."
   ```

   A response with the status `201 'Created'` indicates a successful form update.

8. To save the current form with a new version name, you version the form.

   ```
   curl -k -X POST \
   "$url/form-service/api/forms/versions" \
   -H "Authorization: Bearer $token" \
   -H 'Content-Type: application/json' \
   -d '{
   "name":"<your_form_version_name>",
   "sourceId":"$item_id/$version_id",
   "sourceType":"com.vmw.blueprint.version",
   "formType":"requestForm"
   }' | jq "."
   ```

   The form is saved with the version name.

   To create more versions, you repeat the steps to customize and version the form. If you customize without versioning, the changes you make are only applied to the current form.

9. Optional: If you have at least one form version in addition to the current form, you can list and restore the version that you want to use.
   - To list form versions:

     ```
     curl -k -X GET \
     "$url/form-service/api/forms/versions?sourceType=com.vmw.blueprint.version&sourceId=$item_id/
     $version_id&formType=requestForm" \
     -H "Authorization: Bearer $token" \
     -H 'Content-Type: application/json' | jq "."
     ```

     The response lists the names of stored form versions and their IDs or `formVersionId`.
   - To restore a previously versioned form, use the `formVersionId`:

     ```
     curl -k -X PATCH \
     "$url/form-service/api/forms/versions/$formVersionId/restore" \
     -H "Authorization: Bearer $token" \
     -H 'Content-Type: application/json' | jq "."
     ```

Edit and Version a Custom Form

Edit the custom form for a cloud template with the name `Demo Blueprint`. Then version the custom form it so that you can save it to use later.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2020-08-25'
```

List the items in your catalog.

```
$ curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" "$url/
catalog/api/admin/items?apiVersion=$api_version" | jq "."
```

Examine the response to find the cloud template named `Demo Blueprint` and get the catalog ID.

```
[
{
"id": "890275e3-cbc6-3778-af0d-d9eddf4664a0",
"name": "Demo Blueprint",
"description": "",
"sourceId": "d4c24bba-68cc-4923-9155-338c4bf3e663",
"sourceName": "template-content-source",
"type": {
"id": "com.vmw.blueprint",
"link": "/catalog/api/types/com.vmw.blueprint",
"name": "VMware Aria Automation Templates"
},
"createdAt": "2023-05-03T15:15:02.805617Z",
"createdBy": "user@mycompany.com",
"lastUpdatedAt": "2023-05-03T15:15:02.805617Z",
"iconId": "1495b8d9-9428-30d6-9626-10ff9281645e",
"bulkRequestLimit": 1
}
]
```

Assign the item ID variable with the catalog ID of the cloud template.

```
$ item_id='890275e3-cbc6-3778-af0d-d9eddf4664a0'
```

Get catalog information about the cloud template.

```
$ curl -X GET \
"$url/catalog/api/admin/items/$item_id?apiVersion=$api_version" \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $access_token"  | jq .content
```

A snippet of the response lists the imported versions.

```
[
{
"id": "2.0.0",
"description": "Added numberInput field",
"createdAt": "2023-03-29T06:19:48.748349Z",
"externalId": "/blueprint/api/blueprints/bbbb30b8-a73a-44bb-92db-27a504e0ec85"
},
{
"id": "1.0.0",
"description": "Added stringInput field",
"createdAt": "2023-03-29T06:19:30.845516Z",
"externalId": "/blueprint/api/blueprints/bbbb30b8-a73a-44bb-92db-27a504e0ec85"
}
]
```

The cloud template in this example has two versions in the catalog. To get the schema and create a custom form for version 2.0.0, assign a variable.

```
$ version_id='2.0.0'
```

Use the catalog ID and the version ID to get the schema for the cloud template.

```
$ schema='curl -k -X GET \
"$url/catalog/api/admin/items/$item_id/versions/$version_id \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' | jq -r .schema'
```

Use the schema to create a custom form.

```
$ curl -k -X POST \
"$url/form-service/api/forms/designer/request?sourceType=com.vmw.blueprint.version&sourceId=
$item_id/$version_id&formType=requestForm" \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
-d '$schema' | jq "."
```

The response shows the schema for `Demo Blueprint` version 2.0.0. It also shows the form ID or `4e383140-51a4-44a7-afcd-26296079596d`.


```
{
"tenant": "733178c1-0620-478c-96b4-2b04ece1478d",
"id": "4e383140-51a4-44a7-afcd-26296079596d",
"name": "Demo Blueprint / 2.0.0",
"form": "{\"layout\":{\"pages\":[{\"id\":\"page_general\",\"title\":\"General\",\"sections\":[{\"id
\":\"section_project\",\"fields\":[{\"id\":\"project\",\"display\":\"dropDown\",\"signpostPosition
\":\"right-middle\"}]},{\"id\":\"section_deploymentName\",\"fields\":[{\"id\":\"deploymentName\",
\"display\":\"textField\",\"signpostPosition\":\"right-middle\"}]},{\"id\":\"section_stringInput\",
\"fields\":[{\"id\":\"stringInput\",\"display\":\"textField\",\"state\":{\"visible\":true,\"read-
only\":false},\"signpostPosition\":\"right-middle\"}]},{\"id\":\"section_e0bcefb5\",\"fields\":
[{\"id\":\"numberInput\",\"display\":\"dropDown\",\"state\":{\"visible\":true,\"read-only\":false},
\"signpostPosition\":\"right-middle\"}]}]}]},\"schema\":{\"project\":{\"label\":\"Project\",\"type
\":{\"dataType\":\"string\",\"isMultiple\":false},\"valueList\":{\"id\":\"projects\",\"type\":
\"scriptAction\"},\"constraints\":{\"required\":true}},\"numberInput\":{\"label\":\"numberInput
\",\"type\":{\"dataType\":\"integer\",\"isMultiple\":false},\"valueList\":{\"id\":\"bdimov/
actionThatReturnsLong\",\"type\":\"scriptAction\",\"parameters\":[]},\"constraints\":{\"required
\":true}},\"stringInput\":{\"label\":\"stringInput\",\"type\":{\"dataType\":\"string\",\"isMultiple
\":false},\"constraints\":{\"required\":true}},\"deploymentName\":{\"label\":\"Deployment Name\",
\"type\":{\"dataType\":\"string\",\"isMultiple\":false},\"constraints\":{\"required\":true,\"max-
value\":900}}},\"options\":{\"externalValidations\":[]}}",
"sourceType": "com.vmw.blueprint.version",
"sourceId": "890275e3-cbc6-3778-af0d-d9eddf4664a0/2.0.0",
"type": "requestForm",
"status": "ON",
"formFormat": "JSON",
"styles": ""
}
```


Update the form schema to change the visibility of the Deployment Name from true to false.

```
$ curl -k -X POST \
-H "Authorization: Bearer $access_token" \
-H 'Content-Type: application/json' \
"$url/form-service/api/forms" \
-d '{
"name":"Demo Blueprint / 2.0.0",
"form":'{"layout":{"pages":[{"id":"page_general","sections":[{"id":"section_project","fields":
[{"id":"project","display":"dropDown","signpostPosition":"right-middle","state":
{"visible":true,"read-only":false}}]},{"id":"section_deploymentName","fields":
[{"id":"deploymentName","display":"textField","state":{"visible":"false","read-
only":false},"signpostPosition":"right-middle"}]}]}]},"schema":{"project":
```

```
{"label":"Project","type":{"dataType":"string","isMultiple":false},"valueList":
{"id":"projects","type":"scriptAction"},"constraints":{"required":true}},"deploymentName":
{"label":"Deployment Name","type":{"dataType":"string","isMultiple":false},"constraints":
{"required":true,"max-value":900}},"options":{"externalValidations":[]}}}'
"status":"ON", // set ON to enable the custom form, OFF to disable
"type":"requestForm",
"sourceId":"$item_id/$version_id",
"sourceType":"com.vmw.blueprint.version"
}'
```

If you want to see the updated form, use the form ID from the schema.

```
curl -k -X GET "$url/form-service/api/forms/4e383140-51a4-44a7-afcd-26296079596d" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' | jq "."
```

Version the form and give it a name that will help you identify it in a list later.

```
$ curl -k -X POST \
"$url/form-service/api/forms/versions" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json' \
-d '{
"name":"Version1_Deployment_Visibility_FALSE",
"sourceId":"$item_id/$version_id",
"sourceType":"com.vmw.blueprint.version",
"formType":"requestForm"
}' | jq "."
```

You can repeatedly update and version the form.

- Each time you update the form, you are updating the current version.
- Each time you version the form, you save the updated form with a new name.

If you want to restore a version of the form, list your form versions to find the version of the form that you want.

```
$ curl -k -X GET \
"$url/form-service/api/forms/versions?sourceType=com.vmw.blueprint.version&sourceId=$item_id/
$version_id&formType=requestForm" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json'
```

The response includes the IDs and names of the custom forms.

```
[
{
"id": "4bfdd822-8975-4883-a8b2-22396ddf9395",
"createdDate": "2023-05-17T14:54:07.187+0000",
"createdBy": "user@mycompany.com",
"name": "Version1_Deployment_Visibility_FALSE"
},
{
"id": "bed58877-e408-41ce-ad20-3bc48c569d84",
"createdDate": "2023-05-17T14:39:16.267+0000",
"createdBy": "user@mycompany.com",
"name": "Version2_TextField_Added"
}
]
```

Assign the ID of the form that you want to restore to the form version ID variable.

```
$ formVersionID='4bfdd822-8975-4883-a8b2-22396ddf9395'
```

Restore the form.

```
$ curl -k -X PATCH \
"$url/form-service/api/forms/versions/$formVersionId/restore" \
-H "Authorization: Bearer $token" \
-H 'Content-Type: application/json'
```

You have learned how to create, update, version, and restore a custom form.

Remove a Cloud Template Version from a VCF Automation Catalog

If you want to remove a cloud template version, you make a POST request. The body of the input indicates the version to remove.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the cloud template ID for the cloud template you want to remove. See Create and Update a Cloud Template.
- Verify that you have the cloud template version that you want to remove from the catalog. See Version and Release a Cloud Template to a VCF Automation Catalog.

If you have a versioned and released cloud template in VCF Automation, it appears in the catalog source. If you do not want that cloud template version to be available for deployment, you must remove it from the catalog.

1. Assign the cloud template ID variable.

   ```
   cloud_template_id='<your_cloud_template_id>'
   ```

2. Assign the cloud template version variable.

   ```
   cloud_template_version='<your_cloud_template_version>'
   ```

   your_cloud_template_version is the version that you want to remove.

3. Remove a version of your cloud template from the catalog.

   ```
   curl -X POST \
     $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/action/unrelease?
   apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' | jq "."
   ```

4. To see the change in the VCF Automation UI, select the **Content & Policies** tab.
   a) Select **Content Sources**.
   b) Select the name of the content source with your cloud template.
   c) On the Content Source Details screen that appears, click **Save and Import**.

The cloud template version you specified is removed from the VCF Automation catalog. Any other released cloud template versions remain listed.

Remove a Cloud Template Version

Remove version 5 of your cloud template with ID `fa6b42d5-ac46-451d-8917-b2f7e527b785`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='fa6b42d5-ac46-451d-8917-b2f7e527b785'
$ cloud_template_version='v5'
```

Remove the cloud template version.

```
$ curl -X POST \
  $url/blueprint/api/blueprints/$cloud_template_id/versions/$cloud_template_version/action/
unrelease?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
```

```
  -H 'Content-Type: application/json' | jq "."
```

A snippet of the response shows the cloud template version with a `VERSIONED` status.

```
...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "name": "MyExampleCloudTemplate",
  "description": "Basic Cloud Machine cloud template",
  "version": "v5",
  "tags": [],
  "content": "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
  properties:\n      name: BasicCloudMachine\n      flavor: '${input.flavor}'      \n      image:
 '${input.image}'\n      count: '${input.count}'\n      tags: [\n        {\n          \"key\":
 \"env\",\n          \"value\": \"prod\"\n        }\n      ]\n",
  "status": "VERSIONED",
  "versionDescription": "Creating a version from the current draft",
  "versionChangeLog": "Creating a version v5",
  "valid": true
}
```

Test Your Cloud Template Deployment

To test the deployment of a cloud template, you use the Blueprint APIs to make a POST request with the cloud template ID as input.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that the flavor mapping and image mapping specified in the released VCF Automation cloud template version exist in your cloud account. See Create Flavor Mappings and Create Image Mappings.
- Verify that you have the ID for the cloud template you want to deploy. See Create and Update a Cloud Template.

Before deploying a cloud template, you can test the syntax and placement of your cloud template to ensure deployment viability. If errors are reported in the test, you must fix the errors and test again before deploying the cloud template.

1. Assign the cloud template ID variable.

   ```
   cloud_template_id='<your_cloud_template_id>'
   ```

2. Assign image mapping and flavor mapping variables for the cloud template you intend to deploy.

   ```
   image_mapping='<your_image_mapping_name>'
   flavor_mapping='<your_flavor_mapping_name>'
   ```

   The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

3. Test the cloud template deployment.

   ```
   curl -X POST \
     $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
     -d '{
       "simulate":true,
       "blueprintId": "'"$cloud_template_id"'",
       "inputs": {
           "count": 2,
           "image":"'"$image_mapping"'",
           "flavor":"'"$flavor_mapping"'"
       }
   }' | jq "."
   ```

4. Examine the response and assign the cloud template request ID.

```
cloud_template_request_id='<your_cloud_template_request_id>'
```

5. Get the status of the test request.

```
curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Test a Deployment

For a cloud template with ID `1f170637-81a3-4257-b1cd-b2219ee8034c`, test the deployment with image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Test the cloud template deployment.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "simulate":true,
    "blueprintId": "'"$cloud_template_id"'",
    "inputs": {
        "count": 2,
        "image":"'"$image_mapping"'",
        "flavor":"'"$flavor_mapping"'"
    }
}' | jq "."
```

A snippet of the response shows the cloud template request ID.

```
{
  "id": "5c33355e-fc52-4a30-97c3-3752cf9b644e",
  "createdAt": "2019-10-11T00:11:55.544Z",
...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
...
```

Assign the cloud template request ID variable.

```
$ cloud_template_request_id='5c33355e-fc52-4a30-97c3-3752cf9b644e'
```

Request the status of the deployment.

```
$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the status of the deployment test request.

```
...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "inputs": {
    "count": 2,
```

```
        "image": "ubuntu",
        "flavor": "small"
      },
      "status": "FINISHED",
    ...
```

If your test deployment is successful, you are ready to deploy your cloud template.

Deploy Your Cloud Template

To request the deployment of a cloud template, you use the Blueprint APIs to make a POST request with the cloud template ID as input.

- Verify that all general prerequisites and prerequisites for the VCF Automation Blueprint service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have a project ID for the project that includes cloud zones configured to support the resource requirements of your cloud template. See Create a Project to use in VCF Automation.
- Verify that the flavor mapping and image mapping specified in the released VCF Automation cloud template version exist in your cloud account. See Create Flavor Mappings and Create Image Mappings.
- Verify that you have the ID for the cloud template you want to deploy. See Create and Update a Cloud Template.
- Verify that you have tested your cloud template deployment to ensure deployment viability. See Test Your Cloud Template Deployment.

The prerequisites of this task call for verifying that you have the cloud template ID of the cloud template you want to deploy. In addition, this procedure includes an optional step to deploy a cloud template without a cloud template ID by providing contents inline instead.

1. Assign the cloud template ID variable.

   ```
   cloud_template_id='<your_cloud_template_id>'
   ```

2. To deploy a cloud template, assign variables for image mapping and flavor mapping.

   ```
   image_mapping='<your_image_mapping_name>'
   flavor_mapping='<your_flavor_mapping_name>'
   ```

   The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

3. Request a deployment of a cloud template.

   ```
   curl -X POST \
     $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
     -d '{
       "description": "requesting deployment from cloud template",
       "blueprintId": "'"$cloud_template_id"'",
       "inputs": {
             "count": 2,
           "image":"'"$image_mapping"'",
           "flavor":"'"$flavor_mapping"'"
       }
   }' | jq "."
   ```

4. Examine the response to get the cloud template request ID and the deployment ID.

5. Assign the cloud template request ID and the deployment ID.

   ```
   cloud_template_request_id='<your_cloud_template_request_id>'
   deployment_id='<your_deployment_id>'
   ```

6. Optional: If you do not have a cloud template ID, you can also request a cloud template deployment with contents inline.
   a) Validate the cloud template before creating it.

```
curl -X POST \
  $url/blueprint/api/blueprint-validation?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token"  \
  -d '{
    "name" : "'"$cloud_template_id"'",
    "description" : "Basic Cloud Machine cloud template",
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
 properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n      image:
 '\''${input.image}'\''\n      count: '\''${input.count}'\''",
    "projectId" : "'"$project_id"'",
    "requestScopeOrg": false
}' | jq "."
```

   b) Examine the response to confirm that you see `"valid":true`.
   c) Request the cloud template deployment.

```
curl -X POST \
  $url/blueprint/api/blueprint-requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "requesting deployment from inline cloud template",
    "projectId": "'"$project_id"'",
    "inputs": {
          "count": "2",
        "image":"'"$image_mapping"'",
        "flavor":"'"$flavor_mapping"'"
    },
    "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\n  count:\n    type: integer\n    minimum: 1\n    default: 1\n    maximum:
 2\n    title: Number of Instances\nresources:\n  BasicCloudMachine:\n    type: Cloud.Machine\n
 properties:\n      name: BasicCloudMachine\n      flavor: '\''${input.flavor}'\''      \n      image:
 '\''${input.image}'\''\n      count: '\''${input.count}'\''"
}' | jq "."
```

7. Look up the status of the cloud template deployment.

```
curl -X GET \
  $url/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Deploy a Cloud Template

For a cloud template with ID `1f170637-81a3-4257-b1cd-b2219ee8034c`, request the deployment with image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version='2019-09-12'
$ cloud_template_id='1f170637-81a3-4257-b1cd-b2219ee8034c'
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request the deployment of a cloud template.

```
$ curl -X POST \
  $url/blueprint/api/blueprint-requests?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "requesting deployment from cloud template",
    "blueprintId": "'"$cloud_template_id"'",
    "inputs": {
          "count": 2,
        "image":"'"$image_mapping"'",
        "flavor":"'"$flavor_mapping"'"
    }
}' | jq "."
```

A snippet of the response shows the cloud template request ID, the deployment ID, and the cloud template ID.

```
{
  "id": "889f95a8-79a3-4b2f-b19e-32d1536dd69a",
  "createdAt": "2019-10-11T00:11:55.544Z",
...
  "projectName": "Example-project",
  "deploymentId": "15454178-63fc-42ea-b4ad-7ed8a5cdb128",
  "requestTrackerId": "889f95a8-79a3-4b2f-b19e-32d1536dd69a",
...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
...
```

Assign the cloud template request ID variable.

```
$ cloud_template_request_id='889f95a8-79a3-4b2f-b19e-32d1536dd69a'
```

Request the status of the deployment.

```
$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the cloud template request ID and the cloud template ID with the status of the deployment request. If the deployment fails, the failure message indicates the reason for the failure.

```
{
  "id": "889f95a8-79a3-4b2f-b19e-32d1536dd69a"
  ...
  "blueprintId": "1f170637-81a3-4257-b1cd-b2219ee8034c",
  "inputs": {
    "count": 2,
    "image": "ubuntu",
    "flavor": "small"
  },
  "status": "FINISHED",
...
```

Use the deployment ID to look up resource information for your deployment.

Specify SCSI disk placement using the VCF Automation API

When attaching a disk object to a vSphere VM, you can specify the SCSI controller and the logical unit number (LUN) disk properties so that you can identify the disk when taking day 2 actions.

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.

To attach a disk to a VM, you need the block device ID of the disk and the ID of the VM. The following procedure includes steps to:

- List all machines in your environment to find the ID of the vSphere VM that you want to attach a disk to.
- List all block devices in your environment to find the disk you want to attach.
- Attach the disk specifying the SCSI controller and LUN.

1. List all the machines in your environment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
machines?apiVersion=$api_version | jq "."
```

To identify the vSphere VM where you want to attach your disk, look for the name and the region ID in the response. The `self:href` path includes the machine ID.


```
{
      "powerState": "OFF",
      "externalRegionId": "Datacenter:datacenter-2",
      "cloudAccountIds": [

 "e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341cba1f3-
b37e594a2e813475574a7c3c42b5d"
      ],
      "provisioningStatus": "READY",
      "customProperties": {
        "osType": "LINUX",
        "vcUuid": "1f9678f0-90d1-4347-82ee-f1ac2fac4216",
        "memoryGB": "1",
        "datacenter": "Datacenter:datacenter-2",
        "instanceUUID": "503a00ea-5ce5-3ae8-db3d-ebbd537eed5f",
        "softwareName": "Ubuntu Linux (64-bit)",
        "cpuCount": "2",
        "memoryInMB": "1024"
      },
      "externalId": "503a00ea-5ce5-3ae8-db3d-ebbd537eed5f",
      "name": "wordpress-mcm827-142063808276-ovf-backing",
      "id": "0355cee4-5d88-36b6-9a87-5f37b5baa6e2",
      "createdAt": "2022-04-02",
      "updatedAt": "2022-04-02",
      "organizationId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "_links": {
        "network-interfaces": {
          "hrefs": [
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/network-interfaces/1c31ee02-
c83c-3229-84ec-929f3592494a"
          ]
        },
        "cloud-accounts": {
          "hrefs": [
            "/iaas/api/cloud-accounts/
e0f23c91d5ecca75-7f703c5265a63d87-7e3d8d60a55d1306cc791422547ead9153c3bdf1c802400819ad45a341cba1f3-
b37e594a2e813475574a7c3c42b5d"
          ]
        },
        "operations": {
          "hrefs": [
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/power-on",
```

```
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/snapshots",
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/operations/resize",
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks",
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/{id}"
          ]
        },
        "disks": {
          "hrefs": [
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/effb94a6-41ad-3553-
bb9a-22896785a283",
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/0815ed4c-0a33-3058-
a5f5-3032a426ded4",
            "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2/disks/3804a2b2-99fc-3fca-bd2f-
f0cfc3845b54"
          ]
        },
        "self": {
          "href": "/iaas/api/machines/0355cee4-5d88-36b6-9a87-5f37b5baa6e2"
        }
      }
    },
```

In this response example, the machine ID is `0355cee4-5d88-36b6-9a87-5f37b5baa6e2`.

2. Assign the machine ID.

```
machine_id='example-machineID-alphanumeric-string'
```

3. List the block devices in your environment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
block-devices?apiVersion=$api_version | jq "."
```

Examine the response to find the block device that you want to attach. The block device must be available and in the same external region as the vSphere VM. The `self:href` path includes the block device ID.

```
...
    {
      "capacityInGB": 30,
      "status": "AVAILABLE",
      "type": "HDD",
      "persistent": false,
      "externalRegionId": "Datacenter:datacenter-2",
      "cloudAccountIds": [
        "e0f23c91d5ecca75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-6b4f9990d36ee87558f04e6e8e0ca"
      ],
      "provisioningStatus":"READY",
      "customProperties": {
        "diskKind": "Unmanaged"
      },
      "externalId": "74213f6d-ee11-4549-996a-772c0621f7d1",
      "name": "Hard disk 1",
      "id": "02c39b60-32f5-4a7e-9317-88bf8a3fe20c",
      "createdAt": "2022-04-02",
      "updatedAt": "2022-04-02",
      "organizationId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "orgId": "f670fdfc-66d6-4689-9793-d524e7066d1e",
      "_links": {
        "cloud-accounts": {
          "hrefs": [
```

```
            "/iaas/api/cloud-accounts/e0f23c91d5ecca75-7f703c5265a63d87-
e78aab87e9c8d5cd4cd1da1a285403f0f4e77a5240720d093e147b830b172542-6b4f9990d36ee87558f04e6e8e0ca"
          ]
        },
        "operations": {
          "hrefs": [
            "/iaas/api/block-devices/02c39b60-32f5-4a7e-9317-88bf8a3fe20c?capacityInGB={capacityInGB}"
          ]
        },
        "self": {
          "href": "/iaas/api/block-devices/02c39b60-32f5-4a7e-9317-88bf8a3fe20c"
        }
      }
    },
...
```

In this response example, the block device ID is `02c39b60-32f5-4a7e-9317-88bf8a3fe20c`.

4. Assign the block device ID variable.

```
block_device_id='example-blockdeviceID-alphanumeric-string'
```

5. Attach the disk to the VM, specifying both the SCSI controller number and the LUN disk properties.
   - For the SCSI controller number, you can specify any of four values: `SCSI_Controller_0`, `SCSI_Controller_1`, `SCSI_Controller_2`, `SCSI_Controller_3`.
   - For the LUN, you can specify any integer value from 0 through 15. Unit 0 is the boot disk. Disks with higher LUN values attach after disks with lower LUN values.

     Note:
     If you do not specify a LUN, the disk attaches to the first available unit number. If you specify neither the SCSI controller nor the LUN, the disk attaches to the first available SCSI controller and first available unit number.

   This request example uses `"scsiController": "SCSI_Controller_0"` and `"unitNumber": "0"`.

```
curl -X POST \
  $url/iaas/api/machines/$machine_id/disks?apiVersion=$api_version \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $access_token" \
  -d '{
    "blockDeviceId": "'$block_device_id'",
    "scsiController": "SCSI_Controller_0",
    "name": "BootDisk ",
    "description": "Unit 0 is the boot disk",
    "unitNumber": "0"
}' | jq "."
```

   The response includes a selfLink value.

```
{
  "progress": 0,
  "status": "INPROGRESS",
  "name": "Provisioning",
  "id": "example-selfLink-alphanumeric-string",
  "selfLink": "/iaas/api/request-tracker/example-selfLink-alphanumeric-string"
}
```

6. Assign the selfLink variable.

```
selfLink_id='example-selfLink-alphanumeric-string'
```

7. Use the selfLink variable to track the progress of the disk attachment.

```
curl -X GET -H 'Content-Type: application/json' -H "Authorization: Bearer $access_token" $url/iaas/api/
request-tracker/$selfLink_id?apiVersion=$api_version | jq "."
```

After the request completes successfully, the response includes a list of resources with a machine that has your machine ID in the path.

```
{
  "progress": 100,
  "message": "success",
  "status": "FINISHED",
  "resources": [
    "/iaas/api/machines/example-machineID-alphanumeric-string"
  ],
  ...
}
```

8. Optional: To specify SCSI controller and LUN when attaching additional disks, repeat steps 5 to 7.

If you deploy a blueprint that includes a VM with multiple vSphere disk objects, the disks are assigned to the SCSI controller and LUN that you specified. In this way, you can identify the disks when taking day 2 actions such as formatting or resizing a disk.

How to Create Custom Naming Templates

If you enroll in custom naming in the VCF Automation UI, you can create custom naming templates for your organization or project using the IaaS API. Custom naming provides a way for you to name deployed resources using conventions that you define.

If you create a custom name using `POST /iaas/api/naming`, any project that does not already have a template applied can use the naming template. An organization-level naming template applies to any project that is added to an organization.

A maximum of two resource naming templates can be applied to a project: one project-level naming template and one organization-level naming template. Each can have multiple resource types. If naming templates exist at both the project and organization level, the project uses the naming convention used in the project-level naming template.

> **Note:**
> If you created a project and specified a custom naming template as part of a previous project definition, that template is deprecated when you enroll in custom naming. To replace that template, create a custom name at the project level and add a new naming template.

The following examples show how to define custom naming templates for an organization and a project.

Prerequisites for creating a custom name

- Verify that all general prerequisites and prerequisites for the VCF Automation Infrastructure as a Service (IaaS) service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have enrolled in Custom Naming.
  For information about enrolling in custom naming, see Custom naming deployed resources in for VM Apps.
- Verify that you know your organization ID. If you do not know your organization ID, perform the following steps:
  a. Log in to the organization console of your VCF Automation appliance, for example `https://appliance.domain.com/csp/gateway/portal/`
  b. Click the drop-down arrow by your name to display the organization ID below the organization name.
  c. Right click the icon next to the organization ID to copy the long string.
- If you are adding a project-level template, verify that you know the name and ID of the project that you want to assign to the template. For information on how to get a list of projects, see Create a Project to use in VCF Automation.

How to create a custom name with organization scope

This example shows how to define an organization-level custom name with a naming template for a machine resource type in `"orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"`.

```
curl -X POST \
  "$url/iaas/api/naming?apiVersion=$api_version" \
```

```
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "org-level custom name",
    "description": "Example organization-level custom name",
    "projects": [
      {
        "defaultOrg": true,
        "active": true,
        "projectName": "*",
        "projectId": "*",
        "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
      }
    ],
    "templates": [
      {
        "uniqueName": true,
        "staticPattern": "",
        "counters": [],
        "incrementStep": 1,
        "pattern": "myvm-${##}",
        "startCounter": 1,
        "resourceTypeName": "Machine",
        "resourceType": "COMPUTE",
        "resourceDefault": true
      }
    ]
  }' | jq "."
```

The response shows the organization-level custom name with the custom name ID.

```
{
    "name": "org-level custom name",
    "description": "Example organization-level custom name",
    "id": "6ca7be62-627b-41f0-9505-fc29c1349a85",
    "_links": {
        "self": {
            "href": "/iaas/api/naming/6ca7be62-627b-41f0-9505-fc29c1349a85"
        }
    }
}
```

How to create a custom name with project scope

This example shows how to define a project-level custom name with naming templates for network and machine resource types, and assigned to a project that has not already been added to a custom name with project scope.

```
curl -X POST \
  "$url/iaas/api/naming?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "proj-level custom name ",
    "description": "Example project-level custom name",
    "projects": [
      {
        "defaultOrg": false,
        "active": true,
        "projectName": "Example-CA-project",
```

```
        "projectId": "5944aacb-91de-4541-bb9e-ef2a5403f81b",
        "orgId": "8327d53f-91ea-420a-8613-ba8f3149db95"
      }
    ],
    "templates": [
      {
        "uniqueName": true,
        "staticPattern": "",
        "counters": [],
        "incrementStep": 1,
        "pattern": "myCAproject-${##}",
        "startCounter": 1,
        "resourceTypeName": "Network",
        "resourceType": "NETWORK",
        "resourceDefault": true
      },
      {
        "uniqueName": true,
        "staticPattern": "",
        "counters": [],
        "incrementStep": 1,
        "pattern": "myCAproject-${##}",
        "startCounter": 1,
        "resourceTypeName": "Machine",
        "resourceType": "COMPUTE",
        "resourceDefault": true
      }
    ]
}' | jq "."
```

The response shows the project-level custom name with the custom name ID.

```
{
    "name": "proj-level custom name",
    "description": "Example project-level custom name",
    "id": "9afa9636-9536-4867-a325-fc70eb073a86",
    "_links": {
        "self": {
            "href": "/iaas/api/naming/9afa9636-9536-4867-a325-fc70eb073a86"
        }
    }
}
```

Requesting a Deployment from a Catalog Item Using VCF Automation APIs

To create catalog items and request deployments, you use the VCF Automation Catalog and Deployment APIs.

In this use case, you create a catalog source with a project that you previously used to create a blueprint, so that you can request a deployment from the catalog item. Optionally, if you want to expire your deployment, you can create a lease policy.

Create a Catalog Source and List Discovered Items

To create a catalog source, you make a POST request with a project ID that has a blueprint version released to the project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Catalog service have been satisfied. See Prerequisites for API Use Case Examples.

- Verify that you have the ID for a project that has the blueprint versioned and released to it.

Because you are requesting the deployment of a blueprint from the catalog, this example lists the steps required to create a VCF Automation Templates source type.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. List all your catalog sources.

```
curl -X GET \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

3. Examine the response to confirm that the name of the catalog source that you plan to create is not listed.
4. List all catalog source types.

```
curl -X GET \
  $url/catalog/api/types?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

5. Examine the response to find the catalog source type that you want to create.
6. Assign the catalog item type ID variable for the VCF Automation Templates source type.

```
catalog_type_id='com.vmw.blueprint'
```

7. Create a catalog source for your blueprint.

```
curl -X POST \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "config":{
    "sourceProjectId":"'$project_id'"
  },
    "typeId":"'$catalog_type_id'",
    "name":"<your_catalog_source_name>"
}' | jq "."
```

8. To obtain the catalog source ID, examine the response.
9. List all items discovered in the project.

```
curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

10. To obtain the catalog item ID, examine the response.
11. Optional: Assign the blueprint name variable.

```
cloud_template_name='<your_cloud_template_name_that_was_released_to_catalog>'
```

12. Optional: To get the catalog item ID, you can also list discovered items by name.

```
curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&search=$cloud_template_name&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Create a catalog source and list discovered items

Create a catalog source for a blueprint named `BasicCloudMachine`.

Assign variables.

```
$ url='https://appliance.domain.com'
```

```
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

List all available sources in your catalog.

```
$ curl -X GET \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to confirm that the name of the catalog source that you plan to create is not listed. The following snippet shows that you cannot create a catalog source with the name `Catalog Source from Blueprintecho s`.

```
...
      "id": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
      "name": "Catalog Source from Blueprintecho s",
...
```

List all available catalog source types.

```
$ curl -X GET \
  $url/catalog/api/types?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to find the ID for a VCF Automation Templates source type.

```
...
      "id": "com.vmw.blueprint",
      "name": " VCF Automation Template",
      "baseUri": "http://catalog-service:8000/catalog/api/provider/blueprint",
      "createdBy": "deploymentservice",
...
```

Assign the source type to the catalog item type variable.

```
$ catalog_type_id='com.vmw.blueprint'
```

Create a catalog source of the VCF Automation Templates source type.

```
$ curl -X POST \
  $url/catalog/api/admin/sources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" \
  -H "Content-Type: application/json" \
  -d '{
    "config":{
    "sourceProjectId":"'$project_id'"
  },
    "typeId":"'$catalog_type_id'",
    "name":"Catalog Source from VCF Automation Templates"
}' | jq "."
```

A snippet of the response shows the catalog source ID.

```
{
  "id": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
  "name": "Catalog Source from VCF Automation Templates",
  "typeId": "com.vmw.blueprint",
  "createdAt": " T22:02:33.553Z",
...
```

Assign the catalog source ID.

```
$ catalog_source_id='753d24a3-e2b0-4d5e-bba6-9e32e5964c69'
```

List items discovered in your project.

```
$ curl -X GET \
  $url/catalog/api/admin/items?projectId=$project_id&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the catalog item ID with your blueprint name.

```
{
      "id": "718917c0-1e02-3141-8142-11da5acaed8f",
      "name": "BasicCloudMachine",
      "description": "Basic Cloud Machine blueprint",
      "sourceId": "753d24a3-e2b0-4d5e-bba6-9e32e5964c69",
...
```

Assign the catalog item ID.

```
$ catalog_item_id='718917c0-1e02-3141-8142-11da5acaed8f'
```

Before requesting a deployment, you use the `catalog_source_id` to create entitlement for the catalog source. Or you can use the `catalog_item_id` to create entitlement for the blueprint item.

Request Deployment

To request a deployment from a catalog item, you make a POST request with a project ID that has a blueprint version released to the project. The request body includes the ID of the catalog item from which you are requesting the deployment, and the version of the released blueprint.

- Verify that all general prerequisites and prerequisites for both the VCF Automation Catalog service and the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that the flavor mapping and image mapping specified in the released blueprint version exist in your cloud account. See Create Flavor Mappings and Create Image Mappings.
- Verify that you have the ID for a project that has the blueprint versioned and released to it.
- Verify that you have the ID of the catalog item from which you plan to request a deployment. See Create a Catalog Source and List Discovered Items.
- Verify that you have the version of a blueprint released to the project that you want to request for deployment.
- Verify that you have created an entitlement for your catalog item or create one.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

2. Assign the catalog item ID variable.

   ```
   catalog_item_id='<your_catalog_item_id>'
   ```

3. List the available versions of the catalog item that can be requested.

   ```
   curl -X GET \
     $url/catalog/api/items/$catalog_item_id/versions \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

4. Examine the response to verify the version of the item that you want has been published to the catalog.
5. Assign the catalog item version.

   ```
   catalog_item_version='<your_catalog_item_version>'
   ```

6. Assign your deployment name variable.

   ```
   deployment_name='<your_deployment_name>'
   ```

   If your deployment name includes spaces, use double quotes as in the following example.

   ```
   deployment_name="This deployment name includes spaces"
   ```

   a) To ensure that the deployment name you plan to use does not already exist, list all deployments.

      ```
      curl -X GET \
      ```

```
    -G --data-urlencode "name=$deployment_name" \
    $url/deployment/api/deployments?apiVersion=$api_version \
    -H "Authorization: Bearer $access_token" | jq "."
```

b) Examine the response. If your deployment name appears, create a new name and reassign your deployment name variable.

7. To deploy a blueprint, assign variables for image mapping and flavor mapping.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

8. Request the deployment from a catalog item.

```
curl -X POST \
  $url/catalog/api/items/$catalog_item_id/request?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token"  \
  -H 'Content-Type: application/json' \
  -d '{
    "deploymentName":"'"$deployment_name"'",
    "projectId":"'"$project_id"'",
    "inputs":{
        "count":1,
        "image":"'"$image_mapping"'",
        "flavor":"'"$flavor_mapping"'"
    },
    "version":"'"$catalog_item_version"'"
}' | jq "."
```

The `inputs` field includes values for request time variables such as size, image, or password.

9. To obtain the deployment ID, examine the response.

10. Assign the deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

11. Get the status of the deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

Request Deployment of a Blueprint from a Catalog Item

Request the deployment of a blueprint with catalog item ID `718917c0-1e02-3141-8142-11da5acaed8f`. When requesting the deployment, set image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
$ catalog_item_id='718917c0-1e02-3141-8142-11da5acaed8f'
```

List available versions.

```
$ curl -X GET \
  $url/catalog/api/items/$catalog_item_id/versions \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows version numbers.

```
...
{
```

```
      "id": "v2",
      "description": "Creating a version from the current draft",
      "createdAt": " 7T19:33:04.445Z"
    },
    {
      "id": "v1",
      "description": "Creating a version from the current draft",
      "createdAt": " T19:25:43.327Z"
    }
...
```

Assign the catalog item version number.

```
$ catalog_item_version='v2'
```

Assign a deployment name and check to ensure that it does not already exist.

```
$ deployment_name="Example Deployment of blueprint"
$ curl -X GET \
  -G --data-urlencode "name=$deployment_name" \
  $url/deployment/api/deployments?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows existing deployments. `Example Deployment of blueprint` is not listed.

```
{
      "id": "c14e787f-60ee-4cce-a5b5-c9440bf181ab",
      "name": "Not Example Deployment",
      "orgId": "c9258a19-fef0-4431-a999-d711e1741c60",
      "catalogItemId": "947b9db2-cf89-3b83-8035-bbcf83bd4c34",
...
```

To deploy a blueprint, you must assign image mapping and flavor mapping variables.

```
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request a deployment from the catalog item.

```
$ curl -X POST \
  $url/catalog/api/items/$catalog_item_id/request?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token"  \
  -H 'Content-Type: application/json' \
  -d '{
    "deploymentName":"'"$deployment_name"'",
    "projectId":"'"$project_id"'",
    "inputs":{
        "count":1,
        "image":"'"$image_mapping"'",
        "flavor":"'"$flavor_mapping"'"
    },
    "version":"'"$catalog_item_version"'"
}' | jq "."
```

The response provides the deployment ID.

```
{
  "deploymentId": "3721d9e2-fce3-48eb-96e5-d8f381354610",
  "deploymentName": "Example Deployment of blueprint"
}
```

Assign the deployment ID.

```
$ deployment_id='3721d9e2-fce3-48eb-96e5-d8f381354610'
```

Get the deployment status.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?apiVersion=$api_version"
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the deployment status.

```
},
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
  "status": "CREATE_SUCCESSFUL"
}
```

Create a Lease Policy

To create a lease policy for your deployment, you make a POST request with a project ID that has a blueprint version released to the project.

- Verify that all general prerequisites and prerequisites for the VCF Automation Policies service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID for a project that has the blueprint versioned and released to it.

Creating a lease policy is optional. For example, you can create a lease policy to specify when you want a deployment to expire. You specify the policy with either a soft or hard lease enforcement type.

- If specified with soft enforcement, the policy can be overridden and will have lower priority than policies with hard enforcement.
- If specified with hard enforcement, the policy must be enforced. If strict enforcement is not possible, for example in cases of conflicting policies, the policy can be overridden but VCF Automation will report an error.

1. Assign the project ID variable.

   ```
   project_id='<your_project_id>'
   ```

2. Assign a lease policy with soft enforcement to your project.

   ```
   curl -X POST \
     $url/policy/api/policies?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
     -d '{
       "name":"<your_lease_policy_name>",
       "projectId":"'$project_id'",
       "definition":{
           "leaseGrace":1,
           "leaseTermMax":10,
           "leaseTotalTermMax":100
       },
       "enforcementType":"SOFT",
       "typeId":"com.mycompany.policy.deployment.lease"
   }' | jq "."
   ```

Create a lease policy with soft enforcement

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
```

Create the soft lease policy named `Sample Lease`.

```
$ curl -X POST \
  $url/policy/api/policies?apiVersion=$api_version \
```

```
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "name":"Sample Lease",
    "projectId":"'$project_id'",
    "definition":{
        "leaseGrace":1,
        "leaseTermMax":10,
        "leaseTotalTermMax":100
    },
    "enforcementType":"SOFT",
    "typeId":"com.vmware.policy.deployment.lease"
}' | jq "."
```

The response shows the lease policy.

```
{
  "id": "49893797-208c-4322-8ed5-061467674d54",
  "name": "Sample Lease",
  "typeId": "com.mycompany.policy.deployment.lease",
  "enforcementType": "SOFT",
  "orgId": "c9258a19-fef0-4431-a999-d711e1741c60",
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
  "definition": {
    "leaseGrace": 1,
    "leaseTermMax": 10,
    "leaseTotalTermMax": 100
  },
  "createdAt": " T02:29:07.936Z",
  "createdBy": "admin@mycompany.com",
  "lastUpdatedAt": " T02:29:07.936Z",
  "lastUpdatedBy": "admin@mycompany.com"
}
```

Working with Deployments and Resources

To work with deployments, you use the VCF Automation and VCF Automation Deployment APIs. You also use the VCF Automation Deployment APIs to manage or get information about resources in your deployment.

These use cases include examples of procedures you can follow to reconfigure your deployment after initial blueprint deployment. The first example deploys a new blueprint with a deployment ID. In subsequent examples, you use the deployment ID to update the deployment or reconfigure the blueprint components.

If you want to onboard a machine that was deployed outside of VCF Automation, you use the Relocation Service APIs to create an onboarding plan so that you can onboard the machine. Onboarding use cases show how to create an onboarding plan and onboard a machine with or without a blueprint.

Deploy a Cloud Template with Contents Inline

To request a deployment of a cloud template with contents inline, you use the Blueprint APIs to make a POST request with a project ID. Before requesting the deployment, you use the Deployment API to ensure that the deployment name does not already exist.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID for a project that is associated with the deployment that you are managing. See Request Deployment.
- Verify that the flavor mapping and image mapping specified in the cloud template to be deployed exist in your cloud account. See Create Flavor Mappings and Create Image Mappings.

- Verify that the cloud zone that you are deploying into is associated with your project. See Add a Cloud Zone to Your Project.
- Verify that a network profile is configured for the cloud account associated with the project. See Create Network Profiles.
- Assign an API version variable for the Blueprint API.

```
api_version_blueprint=' '
```

Note:
The VCF Automation Deployment service and the VCF Automation Blueprint service have different API version values. You set the API version value for the VCF Automation Deployment service when you satisfied the general prerequisites.

In this example, you deploy a cloud template by providing contents inline instead of providing a cloud template ID. The new cloud template has different content from previously deployed cloud templates including a load balancer component, a virtual machine component, and a network component.

1. Assign the project ID variable.

```
project_id='<your_project_id>'
```

2. Assign your deployment name variable.

```
deployment_name='<your_deployment_name>'
```

If your deployment name includes spaces, use double quotes as in the following example.

```
deployment_name="This deployment name includes spaces"
```

a) To ensure that the deployment name you plan to use does not already exist, list all deployments.

```
curl -X GET \
  -G --data-urlencode "name=$deployment_name" \
  $url/deployment/api/deployments?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

b) To verify that the deployment name does not already exist, examine the response. If your deployment name appears, create a new name and reassign your deployment name variable.

3. To deploy a cloud template, assign variables for image mapping and flavor mapping.

```
image_mapping='<your_image_mapping_name>'
flavor_mapping='<your_flavor_mapping_name>'
```

The image mapping specifies the OS image for a VM. The flavor mapping specifies the CPU count and RAM of a VM.

4. Request deployment of a cloud template with contents inline.

```
curl -X POST \
    $url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
      "deploymentName": "'"$deployment_name"'",
      "description": "requesting deployment with contents inline",
      "projectId": "'"$project_id"'",
      "inputs": {
            "flavor": "'"$flavor_mapping"'",
            "image" : "'"$image_mapping"'"
      },
      "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\nresources:\n  cloud-vm:\n    type: Cloud.AWS.EC2.Instance\n    properties:\n
     name: cloudvm\n      flavor: '\''${input.flavor}'\''\n      image: '\''${input.image}'\''\n
     networks:\n        - name: '\''${resource.Cloud_Network_1.name}'\''\n          network:
'\''${resource.Cloud_Network_1.id}'\''\n  Provider_LoadBalancer_1:\n    type: Cloud.LoadBalancer\n
 properties:\n      name: OC-LB\n      routes:\n        - protocol: HTTP\n          port: '\''80'\''\n
```

```
          instanceProtocol: HTTP\n          instancePort: '\''80'\''\n          healthCheckConfiguration:
\n            protocol: HTTP\n            port: '\''80'\''\n            urlPath: /index.html\n
     intervalSeconds: 60\n            timeoutSeconds: 5\n            unhealthyThreshold: 5\n
  healthyThreshold: 2\n      network: '\''${resource.Cloud_Network_1.name}'\''\n      instances:\n
    - '\''${resource[\"cloud-vm\"].id}'\''\n      internetFacing: false\n  Cloud_Network_1:\n    type:
 Cloud.Network\n    properties:\n      name: provider\n      networkType: public\n"
}' | jq "."
```

5. To obtain the cloud template request ID and the deployment ID, examine the response.
6. Assign the cloud template request ID variable.

```
cloud_template_request_id='<your_cloud_template_request_id>'
```

7. Get the status of the request.

```
curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=$api_version_blueprint \
  -H "Authorization: Bearer $access_token" | jq "."
```

Deploy a Cloud Template with Contents Inline

Request a deployment named `Deployment with Cloud Template Contents Inline`. When requesting the deployment, set image mapping set to `ubuntu` and flavor mapping set to `small`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_blueprint=' '
$ project_id='394a4ccb-22c6-4ef0-8c75-8b77efbefb51'
$ deployment_name="Deployment with Cloud Template Contents Inline"
```

To request deployment, you must assign image mapping and flavor mapping variables.

```
$ image_mapping='ubuntu'
$ flavor_mapping='small'
```

Request deployment of a cloud template with contents inline.

```
$ curl -X POST \
    $url/blueprint/api/blueprint-requests?apiVersion=$api_version_blueprint \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
      "deploymentName": "'"$deployment_name"'",
      "description": "requesting deployment with contents inline",
      "projectId": "'"$project_id"'",
      "inputs": {
            "flavor": "'"$flavor_mapping"'",
            "image" : "'"$image_mapping"'"
      },
      "content" : "formatVersion: 1\ninputs:\n  flavor:\n    type: string\n    title: Flavor\n
 description: Flavor Mapping Name\n  image:\n    type: string\n    title: Image\n    description:
 Image Mapping Name\nresources:\n  cloud-vm:\n    type: Cloud.AWS.EC2.Instance\n    properties:\n
     name: cloudvm\n      flavor: '\''${input.flavor}'\''\n      image: '\''${input.image}'\''\n
     networks:\n        - name: '\''${resource.Cloud_Network_1.name}'\''\n          network:
 '\''${resource.Cloud_Network_1.id}'\''\n  Provider_LoadBalancer_1:\n    type: Cloud.LoadBalancer
\n    properties:\n      name: OC-LB\n      routes:\n        - protocol: HTTP\n          port:
 '\''80'\''\n          instanceProtocol: HTTP\n          instancePort: '\''80'\''\n
   healthCheckConfiguration:\n            protocol: HTTP\n            port: '\''80'\''\n
         urlPath: /index.html\n            intervalSeconds: 60\n            timeoutSeconds:
 5\n            unhealthyThreshold: 5\n            healthyThreshold: 2\n      network:
```

```
 '\''${resource.Cloud_Network_1.name}'\''\n      instances:\n        - '\''${resource[\"cloud-
vm\"].id}'\''\n      internetFacing: false\n  Cloud_Network_1:\n    type: Cloud.Network\n
 properties:\n      name: provider\n      networkType: public\n"
}' | jq "."
```

A snippet of the response provides the cloud template request ID and the deployment ID.

```
...
  "type": "blueprint-request",
  "id": "bec37f54-3de5-451d-b484-a110c0ed8772",
...
  "projectId": "394a4ccb-22c6-4ef0-8c75-8b77efbefb51",
  "projectName": "example-project",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
  "requestTrackerId": "bec37f54-3de5-451d-b484-a110c0ed8772",
  "deploymentName": "Deployment with Cloud Template Contents inline",
...
```

Assign the cloud template request ID.

```
$ cloud_template_request_id='bec37f54-3de5-451d-b484-a110c0ed8772'
```

Get the deployment status.

```
$ curl -X GET \
  $url/blueprint/api/blueprint-requests/$cloud_template_request_id?apiVersion=
$api_version_blueprint \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the deployment status.

```
...
  "inputs": {
    "image": "ubuntu",
    "flavor": "small"
  },
  "status": "FINISHED",
...
```

You can use the deployment ID to perform other operations on your deployment. Other operations include, changing the lease on your deployment, fetching deployment resources, reconfiguring a load balancer, or adding a disk to a machine and power it off.

Look up Deployment Details

To look up deployment details such as the resources provisioned for each deployment, you use the Deployment APIs to make a GET request that displays all available resources. Then you use the resource ID in the output to make a GET request that returns the details of a particular resource.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have a deployment ID for the deployment that you requested.

1. Assign the deployment ID variable.

   ```
   deployment_id='<your_deployment_id>'
   ```

2. Display all the available resources that are provisioned in your deployment.

   ```
   curl -X GET \
     $url/deployment/api/deployments/$deployment_id?expand=resources&apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

3. Examine the response to find the ID of the resource for which you want details.
4. Assign the deployment resource ID.

```
deployment_resource_id='<your_deployment_resource_id>'
```

5. Display the details of that resource.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$deployment_resource_id?apiVersion=$api_version
 \
  -H "Authorization: Bearer $access_token" | jq "."
```

6. List the deployment events.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/events?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

In case of deployment failures, listing deployment events can help with debugging.

Look up the details of a provisioned resource in your deployment

Display the resources provisioned in your deployment.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='15454178-63fc-42ea-b4ad-7ed8a5cdb128'
```

Look up deployment details.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id?expand=resources&apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the details for each resource details including a deployment resource ID.

```
...
    "resources": [
    {
      "id": "3994a33e-bd93-4eea-87f1-f99ff17717ce",
      "name": "BasicCloudMachine[0]",
...
```

Assign the deployment resource ID variable for the `BasicCloudMachine` resource.

```
$ deployment_resource_id='3994a33e-bd93-4eea-87f1-f99ff17717ce'
```

Display the details of that resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$deployment_resource_id?apiVersion=
$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the details of the single resource.

```
{
  "id": "3994a33e-bd93-4eea-87f1-f99ff17717ce",
  "name": "BasicCloudMachine[0]",
  "type": "Cloud.Machine",
  "dependsOn": [],
  "createdAt": " T17:56:09.463Z",
  "properties": {
    "id": "/resources/compute/3114189206b1763d",
```

```
    "name": "BasicCloudMachine",
...
    "service": "EC2",
    "storage": {
      "disks": [
        {
          "name": "boot-disk",
          "type": "HDD",
...
     "networks": [
      {
        "name": "BasicCloudMachine_nic",
        "address": "172.16.1.98",
        "assignment": "dynamic"
      }
        ],
...
    "__ext:ComputeReservationTaskState:STARTED:SELECTED": "true",
    "__ext:ComputeAllocationTaskState:START_COMPUTE_ALLOCATION": "true"
  },
  "state": "OK"
}
```

List events from the deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/events?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows successful events.

```
...
      "totalTasks": 3,
      "status": "SUCCESSFUL",
      "inputs": {
        "count": 2,
        "image": "ubuntu",
        "flavor": "small"
      }
...
```

Get Deployment Resource IDs

To get IDs of the resources in your deployment, you use the Deployment APIs to make a GET request.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the deployment you want to reconfigure. See Deploy a Cloud Template with Contents Inline.

To perform operations on the load balancer or virtual machine in your deployment, you need the IDs of those resources.

1. Assign your deployment ID variable.

   ```
   deployment_id='<your_deployment_id>'
   ```

2. List the resources in your deployment.

   ```
   curl -X GET \
     $url/deployment/api/deployments/$deployment_id/resources?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

3. Examine the response.
   - Find the resource named `Provider_LoadBalancer_1` and copy the value to assign to the load balancer ID.
   - Find the resource named `cloud-vm` and copy the value to assign to the virtual machine ID.
4. Assign variables for the resources.

```
load_balancer_id='<your_load_balancer_id>'
virtual_machine_id='<your_virtual_machine_id>'
```

Get Deployment Resource IDs

Get the resource IDs for your deployment with ID `5551a299-8b67-45e3-909e-a638d11b0d9f`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

List the resources in your deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the ID for the resource `Provider_LoadBalancer_1` and the ID for the resource `cloud-vm`.

```
...
{
      "id": "d5b4569d-2234-4fc4-a594-45e6b0251588",
      "name": "Provider_LoadBalancer_1",
      "type": "Cloud.LoadBalancer",
...
{
      "id": "42f49781-1490-4a08-ae21-8baf383a72ac",
      "name": "cloud-vm",
      "type": "Cloud.AWS.EC2.Instance",
...
```

Assign the load balancer ID and virtual machine ID variables.

```
$ load_balancer_id='d5b4569d-2234-4fc4-a594-45e6b0251588'
$ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
```

Use the load balancer ID to reconfigure your load balancer. Use the virtual machine ID to add a disk to the VM and power off the VM.

Change the Lease on Your Deployment

To change the lease on your deployment, you use the Deployment APIs to make a POST request with a new lease expiration date.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the deployment you want to reconfigure. See Deploy a Cloud Template with Contents Inline.

The lease on your deployment is set to never expire by default. The following procedure shows how to add a lease expiration date. It also includes an optional step that shows how to reset the lease on your deployment back to never expire.

1. Assign your deployment ID variable.

```
deployment_id='<your_deployment_id>'
```

2. Get a list of actions available for your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

3. Examine the response.
   - Confirm that you see the action `"name": "ChangeLease"`.
   - `"valid":true` indicates that the action is valid for the deployment.
4. Assign the action ID variable for the action `"name": "ChangeLease"`.

```
action_id='Deployment.ChangeLease'
```

5. List the deployment actions for the action ID .

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

6. Examine the response. The schema field shows the format of the input for an action on the deployment.
7. To change the lease on the deployment, assign the lease expiry date using the format specified in the schema as in the following example.

```
lease_expiry_date=2021-05-15T23:11:00Z
```

8. Change the lease expiration date.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Deployment.ChangeLease",
    "inputs": {
        "Lease Expiration Date": "'"$lease_expiry_date"'"
    }
}' | jq "."
```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

    If the request is successful, the response shows `"status":"SUCCESSFUL"`.

11. Optional: To change the lease on the deployment back to never expire, use the same `action_id='Deployment.ChangeLease'` but leave `inputs` empty.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Deployment.ChangeLease",
    "inputs": {}
}' | jq "."
```

    Use the request ID from the response to check the status of the request. The lease is changed when the response shows `"status":"SUCCESSFUL"`.

Change the Lease on Your Deployment

Change the lease on your deployment with ID `5551a299-8b67-45e3-909e-a638d11b0d9f`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

List the actions available for your deployment.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows `Deployment.ChangeLease`.

```
...
{
    "id": "Deployment.ChangeLease",
    "name": "ChangeLease",
    "displayName": "Change Lease",
    "description": "Set a deployment's expiration date",
    "valid": true,
    "actionType": "RESOURCE_ACTION"
  }
...
```

Assign the action ID variable.

```
$ action_id='Deployment.ChangeLease'
```

To get the schema for your action, list the deployment actions by ID.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response provides the schema for the lease expiration date.

```
...
      "properties": {
      "Deployment expires in": {
        "type": "string",
        "readOnly": true,
        "default": "9d 21h 27m"
      },
      "Lease Expiration Date": {
        "type": "string",
        "title": "Lease Expiration Date",
        "description": "The lease can be extended by up to 90d 0h 0m",
        "format": "date-time",
        "formatMinimum": "2021-02-12T21:47:00Z",
        "formatMaximum": "2021-05-20T19:15:00Z",
        "default": "2021-02-22T19:15:00Z"
...
```

Assign the lease expiry date variable in the propert format.

```
$ lease_expiry_date=2021-05-15T23:11:00Z
```

Submit a request to change the lease expiration.

```
$ curl -X POST \
```

```
  $url/deployment/api/deployments/$deployment_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Deployment.ChangeLease",
    "inputs": {
        "Lease Expiration Date": "'"$lease_expiry_date"'"
    }
}' | jq "."
```

A snippet of the response shows request ID.

```
...
  "id": "6b9b5534-0d84-4f07-9941-5c3cc26f7e3b",
  "name": "Change Lease",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
...
```

Assign the request ID variable.

```
$ request_id='6b9b5534-0d84-4f07-9941-5c3cc26f7e3b'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Deployment.ChangeLease",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL"
}
```

Delete Your Deployment

To delete a deployment, you use the VCF Automation Deployment API and if needed, the VCF Automation IaaS API. You make a DELETE request to remove the deployment and clean up the associated resources from the cloud provider.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the VCF Automation administrator service role.
- Verify that you know the deployment that you want to delete.
- Assign an API version variable for the IaaS API.

```
api_version_iaas='2021-07-15'
```

> **Note:**
> The VCF Automation Deployment service and the VCF Automation Infrastructure as a Service (IaaS) service have different API version values. You set the API version value for the VCF Automation Deployment service when you satisfied the general prerequisites.

If using the VCF Automation Deployment service fails to delete the deployment, you can use the VCF Automation IaaS service to force the deletion. The following procedure shows how to delete a deployment when both API services are used.

1. List all deployments.

```
curl -X GET "$url/deployment/api/deployments?apiVersion=$api_version" -H "Authorization: Bearer
 $access_token" | jq "."
```

2. Examine the response to find the name and ID of the deployment that you want to delete.
3. Assign your deployment ID variable.

   ```
   deployment_id='<your_deployment_id>'
   ```

4. Get a list of actions for the deployment.

   ```
   curl -X GET \
   "$url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version" \
   -H "Authorization: Bearer $access_token" | jq "."
   ```

5. Examine the response.
   - Confirm that you see the action `"id": "Deployment.Delete"`.
   - `"valid":true` indicates that the action is valid for the deployment.
6. Delete the deployment.

   ```
   curl -X DELETE \
   "$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version" \
   -H "Authorization: Bearer $access_token" | jq "."
   ```

7. Examine the response for the ID of the deletion request.
8. Assign the ID to the request ID variable.

   ```
   request_id='<delete_request_id>'
   ```

9. Get the status of the deployment request.

   ```
   curl -X GET \
   "$url/deployment/api/requests/$request_id?apiVersion=$api_version" \
   -H "Authorization: Bearer $access_token" | jq "."
   ```

10. Examine the response.
    - If the response shows `"status": "SUCCESSFUL"`, then the delete request succeeded.
    - If the response shows `"status": "FAILED"` then the delete request may have failed due to a dependency such as an unreleased IP in an AD account. If you are certain that you want to remove the deployment and all related resources, you can choose to ignore the deletion failures and submit a delete request that uses the VCF Automation IaaS service.
11. Optional: If you want to force deletion, use `forceDelete=true` in the VCF Automation IaaS service request to delete the deployment.

    ```
    curl -X DELETE \
    "$url/iaas/api/deployments/$deployment_id?forceDelete=true&apiVersion=$api_version_iaas" \
    -H "Authorization: Bearer $access_token" | jq "."
    ```

12. Optional: Repeat steps 7, 8, and 9.
13. Optional: Examine the response to verify that `"status": "SUCCESSFUL"` appears.

Delete Your Deployment

This example shows how to delete a deployment with the VCF Automation Deployment service and when that deletion fails, force the deletion with the VCF Automation IaaS service.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ api_version_iaas=' '
```

List all deployments.

```
curl -X GET "$url/deployment/api/deployments?apiVersion=$api_version" -H "Authorization: Bearer
 $access_token" | jq "."
```

Examine the response to find the deployment that you want to delete.

```
{
"id": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"name": "example_deployment",
"description": "",
...
},
```

Assign your deployment variable.

```
deployment_id='164f2d4e-1755-491e-b0a0-583f0ed4ae3e'
```

List the actions available for your deployment.

```
$ curl -X GET \
"$url/deployment/api/deployments/$deployment_id/actions?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the action `"id": "Deployment.Delete"` with `"valid": true` so you can delete the deployment.

```
...
{
"id": "Deployment.Delete",
"name": "Delete",
"displayName": "Delete",
"description": "Delete a deployment",
"valid": true,
"actionType": "RESOURCE_ACTION"
}
...
```

Delete the deployment.

```
curl -X DELETE \
"$url/deployment/api/deployments/$deployment_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the ID which is the request ID.

```
...
"id": "d9541db3-2806-42aa-bde0-fb870d114833",
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "PENDING",
...
```

Assign the request ID variable.

```
$ request_id='d9541db3-2806-42aa-bde0-fb870d114833'
```

Check the status of the request.

```
$ curl -X GET \
"$url/deployment/api/requests/$request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows that the request failed.

```
{
"id": "d9541db3-2806-42aa-bde0-fb870d114833",
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "FAILED",
...
```

If you are certain that you want to delete the deployment, force the deletion using the VCF Automation IaaS service.

```
curl -X DELETE \
"$url/iaas/api/deployments/$deployment_id?forceDelete=true&apiVersion=$api_version_iaas" \
-H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to get the new request ID.

```
{
"progress": 0,
"status": "INPROGRESS",
"name": "Delete",
"id": "a1b674a2-09aa-4b14-9459-35dddddb9bbc1",
"selfLink": "/iaas/api/request-tracker/a1b674a2-09aa-4b14-9459-35dddddb9bbc1"
...
```

Assign the new request ID.

```
$ new_request_id='a1b674a2-09aa-4b14-9459-35dddddb9bbc1'
```

Use the VCF Automation Deployment service to check the status of the request.

```
$ curl -X GET \
"$url/deployment/api/requests/$new_request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

Examine the response to verify that the action is in progress.

```
{
"id": "a1b674a2-09aa-4b14-9459-35dddddb9bbc1",
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "INPROGRESS",
...
```

Continue to check the status of the request.

```
$ curl -X GET \
"$url/deployment/api/requests/$new_request_id?apiVersion=$api_version" \
-H "Authorization: Bearer $access_token" | jq "."
```

The response shows when the deployment deletion request is successful.

```
{
"id": "a1b674a2-09aa-4b14-9459-35dddddb9bbc1",
```

```
"name": "Delete",
"requestedBy": "user@mycompany.com",
"actionId": "Deployment.Delete",
"deploymentId": "164f2d4e-1755-491e-b0a0-583f0ed4ae3e",
"resourceIds": [
...
],
"status": "SUCCESSFUL",
...
```

Reconfigure Load Balancer

To reconfigure the load balancer in your deployment, you use the Deployment APIs to make a POST request with the ID of the load balancer to update.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the deployment you want to reconfigure. See Deploy a Cloud Template with Contents Inline.
- Verify that you have the ID of the load balancer in your deployment. See Get Deployment Resource IDs.

1. Assign your deployment ID variable.

   ```
   deployment_id='<your_deployment_id>'
   ```

2. Assign your load balancer ID variable.

   ```
   load_balancer_id='<your_load_balancer_id>'
   ```

3. Get a list of actions available for the load balancer in your deployment.

   ```
   curl -X GET \
     $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions?apiVersion=
   $api_version \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

4. Examine the response.
   - Confirm that you see the action `"name": "LoadBalancer.Reconfigure"`.
   - `"valid":true` indicates that the action is valid for the deployment resource.

5. Assign the action ID variable for the reconfigure action `"name": "LoadBalancer.Reconfigure"`.

   ```
   reconfigure_action_id='Cloud.LoadBalancer.LoadBalancer.Reconfigure'
   ```

6. List the resource actions for the action ID .

   ```
   curl -X GET \
     $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions/
   $reconfigure_action_id?apiVersion=$api_version \
     -H "Authorization: Bearer $access_token" | jq "."
   ```

7. Examine the response. The schema field shows the format of the input for an action on the load balancer resource.
8. Reconfigure the load balancer with input inline.

   ```
   curl -X POST \
     $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/requests \
     -H "Authorization: Bearer $access_token" \
     -H 'Content-Type: application/json' \
     -d '{
     "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
     "inputs": {
           "routes": [
             {
               "port": "81",
   ```

```
              "protocol": "TCP",
              "instancePort": "81",
              "instanceProtocol": "TCP",
              "healthCheckConfiguration": {
                "port": "81",
                "urlPath": "/index.html",
                "protocol": "HTTP",
                "timeoutSeconds": 5,
                "intervalSeconds": 60,
                "healthyThreshold": 2,
                "unhealthyThreshold": 5
              }
            }
          ]
    }
}' | jq "."
```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

If the request is successful, the response shows `"status":"SUCCESSFUL"`.

Reconfigure the Load Balancer in Your Deployment

For your deployment with ID `5551a299-8b67-45e3-909e-a638d11b0d9f`, reconfigure the load balancer with resource ID `d5b4569d-2234-4fc4-a594-45e6b0251588`.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

Assign the load balancer ID.

```
$ load_balancer_id='d5b4569d-2234-4fc4-a594-45e6b0251588'
```

List the actions available for the load balancer resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions?apiVersion=
$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows `LoadBalancer.Reconfigure` action.

```
...
{
    "id": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
    "name": "LoadBalancer.Reconfigure",
    "displayName": "Reconfigure",
    "description": "Reconfigure Load Balancer",
    "valid": true,
    "actionType": "RESOURCE_ACTION"
  }
...
```

Assign the action ID variable.

```
$ reconfigure_action_id='Cloud.LoadBalancer.LoadBalancer.Reconfigure'
```

To get the schema for your action, list the resource actions by ID.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/actions/
$reconfigure_action_id?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response provides the schema for the load balancing action.

```
...
  "schema": {
    "type": "object",
    "title": "Reconfigure Load Balancer",
    "description": "Request schema for updating routes of load balancer resource",
    "properties": {
      "routes": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "protocol": {
              "type": "string",
              "title": "Protocol",
              "description": "The communication protocol for an incoming request to the load
 balancer. HTTP, HTTPS, or TCP.",
              "enum": [
                "HTTP",
                "HTTPS",
                "TCP"
              ]
            },
            "port": {
              "type": "string",
              "title": "Port",
              "description": "The listening port for an incoming request to the load balancer.",
              "pattern": "^\\d+$"
            },
            "instanceProtocol": {
              "type": "string",
              "title": "Instance protocol",
              "description": "The communication protocol used between the load balancer and the
 machines in the pool. HTTP, HTTPS, or TCP.",
              "enum": [
                "HTTP",
                "HTTPS",
                "TCP"
              ]
...
```

Submit a request to reconfigure the load balancer with new route properties.

```
$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$load_balancer_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
  "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
  "inputs": {
```

```
          "routes": [
            {
              "port": "81",
              "protocol": "TCP",
              "instancePort": "81",
              "instanceProtocol": "TCP",
              "healthCheckConfiguration": {
                "port": "81",
                "urlPath": "/index.html",
                "protocol": "HTTP",
                "timeoutSeconds": 5,
                "intervalSeconds": 60,
                "healthyThreshold": 2,
                "unhealthyThreshold": 5
              }
            }
          ]
    }
}' | jq "."
```

A snippet of the response shows request ID.

```
...
  "id": "7342a348-65e0-4376-9472-94be56b928a9",
  "name": "Reconfigure",
  "deploymentId": "13c04d0a-fd81-4bcc-99b1-ac499fb1821d",
...
```

Assign the request ID variable.

```
$ request_id='342a348-65e0-4376-9472-94be56b928a9'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version" \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Cloud.LoadBalancer.LoadBalancer.Reconfigure",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL",
}
```

Add a Disk to a Machine and Power It Off

To add a disk to a machine in your deployment, you use the Deployment APIs to make a POST request with the ID of the virtual machine to update. To power off the machine, you make a POST request and specify the action to perform.

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID of the deployment you want to reconfigure. See Deploy a Cloud Template with Contents Inline.
- Verify that you have the ID of the virtual machine in your deployment. See Get Deployment Resource IDs.

1. Assign your deployment ID variable.

   ```
   deployment_id='<your_deployment_id>'
   ```

2. Assign your virtual machine ID variable.

```
virtual_machine_id='<your_virtual_machine_id>'
```

3. Get a list of actions available for the virtual machine in your deployment.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions?apiVersion=
$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

4. Examine the response.
   - Confirm that you see the action `Add.Disk` with `"valid":true`. Copy the value to assign to the add disk action ID.
   - Confirm that you see the action `PowerOff` with `"valid":true`. Copy the value to assign to the power off action ID.

   `"valid":true` indicates that each action is valid for the deployment resource.

5. Assign variables for the resources.

```
add_disk_action_id='<your_add_disk_id>'
poweroff_machine_action_id='<your_poweroff_action_id>'
```

6. List the resource actions for the add disk action ID.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/actions/$reconfigure_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

7. Examine the response. The schema field shows the format of the input for an action on the virtual machine resource.
8. Attach a disk of size 1 GB to the machine.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Cloud.AWS.EC2.Instance.Add.Disk",
    "inputs":{
        "name":"disk1",
        "capacityGb":1,
        "type":"Cloud.Volume"
    }
}' | jq "."
```

9. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

10. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

    If the request is successful, the response shows `"status":"SUCCESSFUL"`.

11. List the resource actions for the power off action ID.

```
curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/
$poweroff_machine_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

12. Examine the response. No schema field indicates that no `inputs` field is required for this action on the virtual machine resource.
13. Power off the machine.

```
curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
  "actionId":"Cloud.AWS.EC2.Instance.PowerOff"
}' | jq "."
```

14. Examine the response and assign the request ID.

```
request_id='<your_request_id>'
```

15. Check the status of the request.

```
curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

If the request is successful, the response shows `"status":"SUCCESSFUL"`.

Add a Disk and Power Off Your Virtual Machine

For your deployment with ID `5551a299-8b67-45e3-909e-a638d11b0d9f`, reconfigure the virtual machine with resource ID `42f49781-1490-4a08-ae21-8baf383a72ac` by adding a disk and powering it off.

Assign variables.

```
$ url='https://appliance.domain.com'
$ api_version=' '
$ deployment_id='5551a299-8b67-45e3-909e-a638d11b0d9f'
```

Assign the virtual machine ID.

```
$ virtual_machine_id='42f49781-1490-4a08-ae21-8baf383a72ac'
```

List the actions available for the virtual machine resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions?apiVersion=
$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows the actions to `Add.Disk` and `PowerOff`.

```
...
  {
    "id": "Cloud.AWS.EC2.Instance.Add.Disk",
    "name": "Add.Disk",
    "displayName": "Add Disk",
    "description": "Add a disk to the machine",
    "valid": true,
    "actionType": "RESOURCE_ACTION"
  },
...
  {
    "id": "Cloud.AWS.EC2.Instance.PowerOff",
    "name": "PowerOff",
    "displayName": "Power Off",
    "description": "Power off a machine",
    "valid": true,
    "actionType": "RESOURCE_ACTION"
```

```
  },
...
```

Assign the action ID variables to add a disk and power off the virtual machine.

```
$ add_disk_action_id='Cloud.AWS.EC2.Instance.Add.Disk'
$ power_off_action_id='Cloud.AWS.EC2.Instance.PowerOff'
```

Get the add disk action for the virtual machine resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/
$add_disk_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response provides the schema to add a disk.

```
...
    "properties": {
      "name": {
        "type": "string",
        "title": "Name",
        "description": "Disk Name",
        "minLength": 1
      },
      "capacityGb": {
        "type": "integer",
        "title": "Size(GB)",
        "description": "Disk Capacity in GB",
        "minimum": 1
      },
      "type": {
        "type": "string",
        "title": "Type",
        "description": "Disk Resource Type.",
        "readOnly": true,
        "default": "Cloud.Volume"
      },
...
```

Follow the schema and submit a request to add a 1 GB disk to the virtual machine.

```
$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
    "actionId":"Cloud.AWS.EC2.Instance.Add.Disk",
    "inputs":{
        "name":"disk1",
        "capacityGb":1,
        "type":"Cloud.Volume"
    }
}' | jq "."
```

A snippet of the response shows request ID.

```
...
  "id": "17dec8d9-2e2a-4c29-9067-ce41c37be7a3",
  "name": "Add Disk",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
```

```
...
```

Assign the request ID variable.

```
$ request_id='17dec8d9-2e2a-4c29-9067-ce41c37be7a3'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Cloud.AWS.EC2.Instance.Add.Disk",
  "completedTasks": 3,
  "totalTasks": 3,
  "status": "SUCCESSFUL",
}
```

Get the power off action for the virtual machine resource.

```
$ curl -X GET \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/actions/
$poweroff_machine_action_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

The complete response shows that there is no schema for the power off action.

```
{
  "id": "Cloud.AWS.EC2.Instance.PowerOff",
  "name": "PowerOff",
  "displayName": "Power Off",
  "description": "Power off a machine",
  "dependents": [
    "Provider_LoadBalancer_1"
  ],
  "valid": true,
  "actionType": "RESOURCE_ACTION"
}
```

Power off the virtual machine.

```
$ curl -X POST \
  $url/deployment/api/deployments/$deployment_id/resources/$virtual_machine_id/requests \
  -H "Authorization: Bearer $access_token" \
  -H 'Content-Type: application/json' \
  -d '{
  "actionId":"Cloud.AWS.EC2.Instance.PowerOff"
}' | jq "."
```

A snippet of the response shows request ID.

```
...
  "id": "ab7d3aec-f850-4b0e-9c1c-47378c182a00",
  "name": "Power Off",
  "deploymentId": "5551a299-8b67-45e3-909e-a638d11b0d9f",
...
```

Assign the request ID variable.

```
$ request_id='ab7d3aec-f850-4b0e-9c1c-47378c182a00'
```

Check the status of the request.

```
$ curl -X GET \
  $url/deployment/api/requests/$request_id?apiVersion=$api_version \
  -H "Authorization: Bearer $access_token" | jq "."
```

A snippet of the response shows that the request was successful.

```
...
  "actionId": "Cloud.AWS.EC2.Instance.PowerOff",
  "completedTasks": 1,
  "totalTasks": 1,
  "status": "SUCCESSFUL",
  "inputs": {}
}
```

Viewing Billable Objects

As an VCF Automation or VCF Automation administrator, you can use the VCF Automation Deployment APIs to view billable objects that are used in your organization.

Prerequisites for viewing billable objects

- Verify that all general prerequisites and prerequisites for the VCF Automation Deployment service have been satisfied. See Prerequisites for API Use Case Examples.

How to get a summary of billable objects

The following request retrieves a categorized summary or total count of VCF Automation billable objects in your organization.

```
curl -X GET \
$url/deployment/api/billing-metrics?apiVersion=$api_version \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token"  | jq "."
```

The following sample response shows all billable objects.

```
{
  "billingMetrics": [
    {
      "id": "vSphereManagedVMCount",
      "displayName": "vSphere Managed VM Count",
      "value": 19
    },
    {
      "id": "vSphereCpuCount",
      "displayName": "vSphere CPU Count",
      "value": 20
    },
    {
      "id": "vSphereCpuCoreCount",
      "displayName": "vSphere CPU Core Count",
      "value": 52
    },
    {
      "id": "vmcManagedVMCount",
      "displayName": "VMC Managed VM Count",
      "value": 0
    },
    {
```

```
      "id": "vmcCpuCount",
      "displayName": "VMC CPU Count",
      "value": 0
    },
    {
      "id": "vmcCpuCoreCount",
      "displayName": "VMC CPU Core Count",
      "value": 0
    },
    {
      "id": "publicCloudManagedVMCount",
      "displayName": "Public Cloud Managed VM Count",
      "value": 58
    }
  ]
}
```

How to get information about billable objects

By applying the filter `billable=true`, the following request limits the amount of output from the call to include only the billable objects that are in your organization.

```
curl -X GET \
"$url/deployment/api/resources?apiVersion=$api_version&billable=true" \
-H 'Accept: application/json' \
-H "Authorization: Bearer $access_token" \
 | jq "."
```

The following sample response snippet provides the details about one of the billable objects.

```
{
  "content": [
    {
      "id": "9bc16a26-bd81-4660-9aa6-6eadfe72456d",
      "name": "Cloud_Machine_1-mcm1157065-230404373562",
      "type": "Cloud.AWS.EC2.Instance",
      "properties": {
        "hostName": "",
        "resourceId": "9bc16a26-bd81-4660-9aa6-6eadfe72456d",
        "externalLink": "https://us-east-1.console.aws.amazon.com/ec2/v2/home?region=us-
east-1#Instances:instanceId=i-0c7ec71b0aa26e080;sort=instanceState",
        "project": "7a90a6fb-f78d-4b91-9b3c-d88d3dd897fd",
        "storage": {
          "disks": [
            {
              "iops": "100",
              "name": "Cloud_Machine_1-mcm1157065-230404373562-boot-disk",
              "type": "SSD",
              "service": "ebs",
              "regionId": "us-east-1",
              "bootOrder": 1,
              "encrypted": false,
              "capacityGb": 30,
              "deviceName": "/dev/sda1",
              "deviceType": "ebs",
              "persistent": false,
              "snapshotId": "snap-00bbf435c29dcdf5c",
              "volumeType": "gp2",
              "endpointType": "aws",
```

```
        "resourceLink": "/resources/disks/3525c26f-30e9-4070-bb1d-a53430594f84",
        "SourceTaskLink": "/provisioning/resource-enumeration-tasks",
        "existingResource": "false"
      }
    ]
  },
  "networks": [
    {
      "id": "/resources/network-interfaces/25ff21a4-8f79-4d77-861f-03148e532823",
      "name": "public-subnet-us-east-1c",
      "tags": [],
      "address": "172.31.84.7",
      "assignment": "dynamic",
      "deviceIndex": 0,
      "mac_address": "12:f0:45:3a:bd:1f",
      "resourceName": "public-subnet-us-east-1c",
      "securityGroupNames": [
        "photon-model-sg"
      ]
    }
  ],
  "powerState": "OFF",
  "zone": "us-east-1c",
  "environmentName": "Amazon Web Services",
  "providerId": "i-0c7ec71b0aa26e080",
  "osType": "WINDOWS",
  "id": "/resources/compute/9bc16a26-bd81-4660-9aa6-6eadfe72456d",
  "cpuCount": 1,
  "isSimulate": "false",
  "image": "small",
  "totalMemoryMB": 1024,
  "componentType": "Cloud.AWS.EC2.Instance",
  "awsVpcId": "vpc-adf7f2d5",
  "imageId": "ami-0bad85cc4b10fde5e",
  "endpointType": "aws",
  "address": "",
  "endpointId": "f2a31ae2-6256-4d2e-b72e-4687d57ce246",
  "externalId": "i-0c7ec71b0aa26e080",
  "resourceName": "Cloud_Machine_1-mcm1157065-230404373562",
  "tags": [
    {
      "key": "ll"
    },
    {
      "key": "sdf"
    }
  ],
  "rootDeviceType": "ebs",
  "primaryMAC": "12:f0:45:3a:bd:1f",
  "flavor": "aws-flavor",
  "service": "ec2",
  "name": "Cloud_Machine_1",
  "flavorRef": "t2.micro",
  "accounts": [
    "aws-vaidehi",
    "aws-skarwa"
  ],
```

```
        "region": "us-east-1",
        "flavorMappingName": "aws-flavor",
        "account": "aws-vaidehi"
      },
      "createdAt": "2023-04-20T17:12:54.348099Z",
      "syncStatus": "SUCCESS",
      "origin": "DEPLOYED",
      "deploymentId": "f588a723-65f4-402f-b548-a13cdd1b74d0",
      "projectId": "7a90a6fb-f78d-4b91-9b3c-d88d3dd897fd",
      "orgId": "79395484-98e2-450e-b1ab-095d6dae71ef",
      "billable": true
    },
...
```

Onboarding virtual machines

To onboard machines that are not yet managed by an VCF Automation project, you use the Onboarding APIs.

The onboarding process begins with creating an onboarding plan that locates the machines to be onboarded and creates a deployment for those machines when the plan runs. When you add deployments to the plan, you have the option of adding the deployment with a blueprint that allows all day 2 actions on the deployment.

Onboard machines as a single deployment

As an administrator, you can use the Onboarding APIs to onboard unmanaged machines as a single VCF Automation deployment so that you can manage the machines. To onboard machines, you:

- Verify that all general prerequisites and prerequisites for the Onboarding service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the ID for the cloud account with cloud zones where the machines to be onboarded are located. See Adding Cloud Accounts and Create a Cloud Zone .
- Verify that you have the ID for a project with at least one user and with access to the cloud zones. See Create a Project to use in VCF Automation
- Create an onboarding plan.
- Query for unmanaged machines.
- Create a bulk deployment that adds the unmanaged machines.
- Run the plan.

When you run the plan you can check the plan progress to see if the plan completes successfully and onboards the machines.

The following procedure shows how to onboard machines and create a deployment in VCF Automation without using a blueprint. This is the fastest way to onboard machines. However, to run the **Update** day 2 action on the deployment after onboarding, you must create the deployment with a blueprint. See How do I add a cloud template to my onboarding plan.

1. Create your onboarding plan without placements.

   In this example,

   - project ID: `4c4f8a47-d746-43f4-b88c-ea94ac8bd573`
   - cloud account ID: `0cbc8179-d202-4a30-9460-c25d2653a677`

   ```
   curl -X POST \
       $url/relocation/onboarding/plan \
       -H "Authorization: Bearer $access_token" \
       -H 'Content-Type: application/json' \
       -d '{
         "name": "'<your_plan_name>'",
         "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
         "endpointIds": [
   ```

```
            "0cbc8179-d202-4a30-9460-c25d2653a677"
        ],
        "usePlacements": false
    }' | jq "."
```

Note:
To create an onboarding plan with placements, set `"usePlacements": true`. With placement turned on, the onboarded workload considers resource limits defined in your project cloud zones or resource quota plicy and takes longer to run.

Examine the response to verify the status, the project ID, the cloud ID, and to get the documentSelfLink, for example:

```
{
    "status": "OK",
    "nextRefreshTimeMicros": 0,
    "refreshIntervalMicros": 0,
    "name": "Plan-782",
    "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
    "endpointIds": [
        "0cbc8179-d202-4a30-9460-c25d2653a677"
    ]
    ...
    "documentSelfLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
    ...
}
```

The value for the `documentSelfLink` is used for the `planLink` in subsequent steps.

2. Use the `planLink` to discover the compute resources representing unmanaged machines.

```
curl -X POST \
    $url/relocation/api/wo/query-unmanaged-machine \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
      "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
      "expandFields": [
          "id",
          "documentSelfLink",
          "name",
          "powerState",
          "address",
          "creationTimeMicros",
          "expandedTags",
          "tagLinks",
          "endpointLinks"
      ],
      "optionExcludePlanMachines": true
  }' | jq "."
```

Examine the response to get the snippet for unmanaged machines with `"powerState": "ON"`, for example:

```
...
"/resources/compute/8bff887d-56fd-3752-bb99-d3f32a35abde": {
    "address": "10.10.10.17",
    "creationTimeMicros": 1717409843457000,
    "documentSelfLink": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
    "endpointLinks": "/resources/endpoints/aa6a5445-e63a-4659-a277-00073a2645c8",
    "expandedTags": [],
    "id": "5017d984-aa61-e00e-dfa6-a986403c01b8",
    "name": "Machine-76",
    "powerState": "ON",
    "tagLinks": []
```

```
    }
"/resources/compute/9bc16a26-bd81-4660-9aa6-6eadfe72456d": {
    "address": "10.10.2.243",
    "creationTimeMicros": 1717409843457000,
    "documentSelfLink": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
    "endpointLinks": "/resources/endpoints/2d57f1c4-fbe0-46ce-a061-3a5f1396f37b",
    "expandedTags": [],
    "id": "a6f324e4-101c-33f6-ac51-d9aaaa020123",
    "name": "Machine-25",
    "powerState": "ON",
    "tagLinks": []
    }...
```

3. Use the `planLink`, `documentSelfLink`, and `name` to create a single deployment for the unmanaged machines.

```
curl -X POST \
    $url/relocation/onboarding/task/create-deployment-bulk \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
      "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
      "deployments": [
          {
              "name": "'<your_deployment_name>'",
              "resources": [
                  {
                      "link": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
                      "name": "Machine-76"
                  },
                  {
                      "link": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
                      "name": "Machine-25"
                  }
              ]
          }
      ]
  }' | jq "."
```

The response shows that the discovered machines have been assigned to the deployment.

```
{
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
    "deployments": [
        {
            "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
            "name": "Deployment-562",
            "resources": [
               {
                   "link": "/resources/compute/dd646fdc-e6db-3b16-a72e-d1c905f7a9c8",
                   "name": "Machine-76"
               },
               {
                   "link": "/resources/compute/8615bbfe-ea90-420a-a1cf-8048ba4271d4",
                   "name": "Machine-25"
               }
            ],
            "tenantLink": "/core/tenants/dc96003b28356c75",
            "documentVersion": 0,
            "documentUpdateTimeMicros": 0,
            "documentExpirationTimeMicros": 0
```

```
            }
        ],
        "tenantLink": "/core/tenants/dc96003b28356c75"
    }
```

4. Use the `planLink` to run the plan and onboard the machines.

```
curl -X POST \
    $url/relocation/api/wo/execute-plan \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
      "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
  }' | jq "."
```

5. Use the `planLink` to the check the run.

```
curl -X GET \
    $url/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json'
 | jq "."
```

   The status in the response changes as the plan moves through `executing` or `updating`. If the run fails, the status shows `"status": "error"`.

```
{
    "status": "OK",
    "nextRefreshTimeMicros": 0,
    "refreshIntervalMicros": 0,
    "name": "Plan-782",
    "projectId": "4c4f8a47-d746-43f4-b88c-ea94ac8bd573",
    "endpointIds": [
        "0cbc8179-d202-4a30-9460-c25d2653a677"
    ],
    "createdBy": "example_admin@example_company.com",
    "enableExtensibilityEvents": true,
    "organizationId": "174adb59-8132-46f0-9cd8-2b2361e9cb2c",
    "customProperties": {},
    "usePlacements": false,
    "isQuick": false,
    "tenantLink": "/core/tenants/dc96003b28356c75",
    "documentVersion": 0,
    "documentKind":
 "com:vmware:relocation:services:onboarding:plan:OnboardingPlanService:OnboardingPlanState",
    "documentSelfLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a",
    "documentUpdateTimeMicros": 1718190629415000,
    "documentUpdateAction": "POST",
    "documentExpirationTimeMicros": 0,
    "documentAuthPrincipalLink": "/core/authz/users/example_admin@example_company.com"
}
```

How do I add a cloud template to my onboarding plan

When creating an onboarding plan, you can attach an existing VMware cloud template (VCT) to the deployment. The cloud template allows you to iteratively update the deployment and avoid the need to delete onboarded resources and create new ones while gradually scaling out your deployment.

With a cloud template attached to your deployment, you can run all day 2 actions including the **Update** day 2 action provided that the YAML of the selected cloud template accepts user input.

> **Note:**
> You can choose any cloud template but depending on the cloud template details, an **Update** day 2 action might delete and re-create original resources.

By attaching a cloud template with resource mapping, you can create an onboarding deployment that maps VMs to template resources in the same way as the deployment that was originally provisioned in VCF Automation using the same cloud template. The only restriction is that the onboarding deployment must have the same number of machines as the selected cloud template and the admin must specify the VM to template resource mapping.

A deployment that uses a cloud template with resource mapping includes the **Update** day 2 action if the selected template accepts user input and the update only applies changes to existing resources. It will not re-create the original resources unless there are major changes between the current version of cloud template and original deployment. Onboarded deployments can also be iteratively updated without deletion and re-creation of the onboarded machines.

Prerequisites for adding deployments with a cloud template

- Verify that all general prerequisites and prerequisites for the Onboarding service have been satisfied. See Prerequisites for API Use Case Examples.
- Verify that you have the `planLink` for an existing onboarding plan. For example, to list onboarding plans use `GET /relocation/api/onboarding/plan`. See Onboard machines as a single deployment.
- Verify that you have the IDs and names of the resources that you are planning to onboard. For example, to list unmanaged machines, use `POST /relocation/api/wo/query-unmanaged-machine`. See Onboard machines as a single deployment.
- Verify that the VMware cloud template that you plan to attach exists. For example, to list cloud templates use `GET /blueprint/api/blueprints`.
- If adding a cloud template with mapping, verify that you have the correct template component name to link to each machine. For example:
  - To get template component names, use `GET /blueprint/api/blueprints/<blueprint_id>` where *`blueprint_id`* is the ID for your cloud template. In the response, the blueprint `content` field lists the machine component names.
  - To list unmanaged machines, use `POST /relocation/api/wo/query-unmanaged-machine`.

Example payloads include the following input:

- planLink for existing onboarding plan

```
"planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
```

- Template to attach if onboarding with template but without mapping

```
"name": "singleMachines"
"link": "/blueprint/api/blueprints/b79612a9-6e2f-4b48-a775-88eb7676dc2c"
```

- Template to attach if onboarding with template and with mapping

```
"template": {
   "name": "vc01",
   "link": "/blueprint/api/blueprints/74bbcdae-b945-4a95-ac86-8bed0a75e2c2",
   "components":
      {
         "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a": "Cloud_vSphere_Machine_1",
         "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689": "Cloud_vSphere_Machine_2"
      }
}
```

How do I add a deployment with a cloud template

The following example adds a deployment with the template `singleMachine` to bring the VMs `East-centos-small-000011` and `est` under management.

```
curl -X POST \
    $url/relocation/onboarding/task/create-deployment-bulk \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
```

```
        "deployments": [
            {
               "resources": [
                  {
                     "link": "/resources/compute/b82870f1-575b-308b-ac2e-874fad85703b",
                     "name": "East-centos-small-000011",
                     "tagLinks": []
                  },
                  {
                     "link": "/resources/compute/e12fefb6-2d0a-3ee1-bb7f-1ceceb3d1ca6",
                     "name": "est",
                     "tagLinks": []
                  }
                ],
                "template": {
                    "name": "singleMachine",
                    "link": "/blueprint/api/blueprints/b79612a9-6e2f-4b48-a775-88eb7676dc2c"
                }
            }
        ],
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
}' | jq "."
```

How do I add a deployment with a cloud template and mapping

The following example adds a deployment with the template `vc01` to bring:

- VMs `VM-20` and `VM-21` under management with:
  - `VM-20` mapped to `Cloud_vSphere_Machine_1` in the template.
  - `VM-21` mapped to `Cloud_vSphere_Machine_2` in the template.
- VMs `VM-0014` and `VM-0015` under management with:
  - `VM-0014` mapped to `Cloud_vSphere_Machine_1` in the template.
  - `VM-0015` mapped to `Cloud_vSphere_Machine_2` in the template.

```
curl -X POST \
    $url/relocation/onboarding/task/create-deployment-bulk \
    -H "Authorization: Bearer $access_token" \
    -H 'Content-Type: application/json' \
    -d '{
        "deployments": [
            {
               "template": {
                     "name": "vc01",
                     "link": "/blueprint/api/blueprints/74bbcdae-b945-4a95-ac86-8bed0a75e2c2",
                     "components":
                         {
                            "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a":
 "Cloud_vSphere_Machine_1",
                            "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689": "Cloud_vSphere_Machine_2"
                         }
                  },
               "resources": [
                     {
                       "link": "/resources/compute/bf07d26a-38cd-392e-b3d6-0004cf36168a",
                       "name": "VM-20",
                       "tagLinks": []
                     },
                     {
```

```
                        "link": "/resources/compute/cc628c21-5531-414d-a09e-5f35916ce689",
                        "name": "VM-21",
                        "tagLinks": []
                     }
                ]
            },
            {
               "template": {
                     "name": "vc01",
                     "link": "/blueprint/api/blueprints/74bbcdae-b945-4a95-ac86-8bed0a75e2c2",
                     "components":
                        {
                            "/resources/compute/2ce64191-867e-3bb2-9017-6f5aa5d3a3c2":
  "Cloud_vSphere_Machine_1",
                            "/resources/compute/c4f46940-e4e1-45fb-bd30-58794398a50c":
  "Cloud_vSphere_Machine_2"
                        }
                },
                "resources": [
                     {
                        "link": "/resources/compute/2ce64191-867e-3bb2-9017-6f5aa5d3a3c2",
                        "name": "VM-0014",
                        "tagLinks": []
                     },
                     {
                        "link": "/resources/compute/c4f46940-e4e1-45fb-bd30-58794398a50c",
                        "name": "VM-0015",
                        "tagLinks": []
                     }
                ]
             }
         ],
    "planLink": "/relocation/onboarding/plan/d2010bcd-34f6-4fac-9d67-f7f4c6ec686a"
}' | jq "."
```

Help and Support for VCF SDKs, APIs, and VCF PowerCLI

This guide outlines the available resources and procedures for troubleshooting and obtaining technical assistance for VCF SDKs, APIs, and VCF PowerCLI cmdlets.

Self-Help Resources

For Self-Help use the available documentation and community resources to search for a solution to your issue. The following resources are recommended:

- Broadcom Developer Portal **:** Access official technical documentation, VCF SDKs and API references, VCF PowerCLI cmdlets and integration guides.
- VCF Code Community: Engage with peer experts and engineers to discuss use cases and community-driven solutions.

Official Technical Support

If an issue cannot be resolved through self-help, customers with a valid Broadcom VCF support contract can open a formal support case.

Our specialized VCF API/SDK Support Team provides comprehensive assistance across the following areas:

- Installation and Setup **-** Installing and troubleshooting issues for VCF SDKs, APIs, or VCF PowerCLI modules.

- Functional Issues - Investigating and resolving instances where VCF SDKs, APIs, or VCF PowerCLI cmdlets do not perform according to documentation.
- Architectural Guidance - Offering design advice on VCF SDKs/APIs to ensure scalable integrations, best practices, and alignment with the lifecycle of the components.
- Performance Optimization - Analyzing execution logic and VCF API call patterns to identify bottlenecks, reduce latency, and improve resource efficiency in scripts/workflows.
- Use Case Clarification - Providing architectural guidance and clarifying the supported scope for specific automation tasks.
- Scripting Support - Troubleshooting and assistance for custom or sample scripts is limited to those using VCF SDKs, APIs, or VCF cmdlets.
- Bug Reporting - Processing and escalating verified defects in VCF SDKs/APIs/VCF cmdlets to engineering teams.

Filing a Support Case

Note:

The VCF API, SDK, and VCF PowerCLI are not listed as standalone product selections in the Broadcom Support Portal.

When filing a case, the component should be set to the primary product in use, and then the sub-component should be set to `API/SDK Software Development Kit (API, Automation).`

You can find more information on how to engage with the VCF API/SDK Support Team in the Broadcom knowledge base article: 436447 .

Program Information

For more information regarding the VCF API/SDK support program and scope of service visit:

- VMware® SDKs and APIs Solution Brief
- Supported Product Matrix for VCF SDK/API
- Broadcom Developer Portal